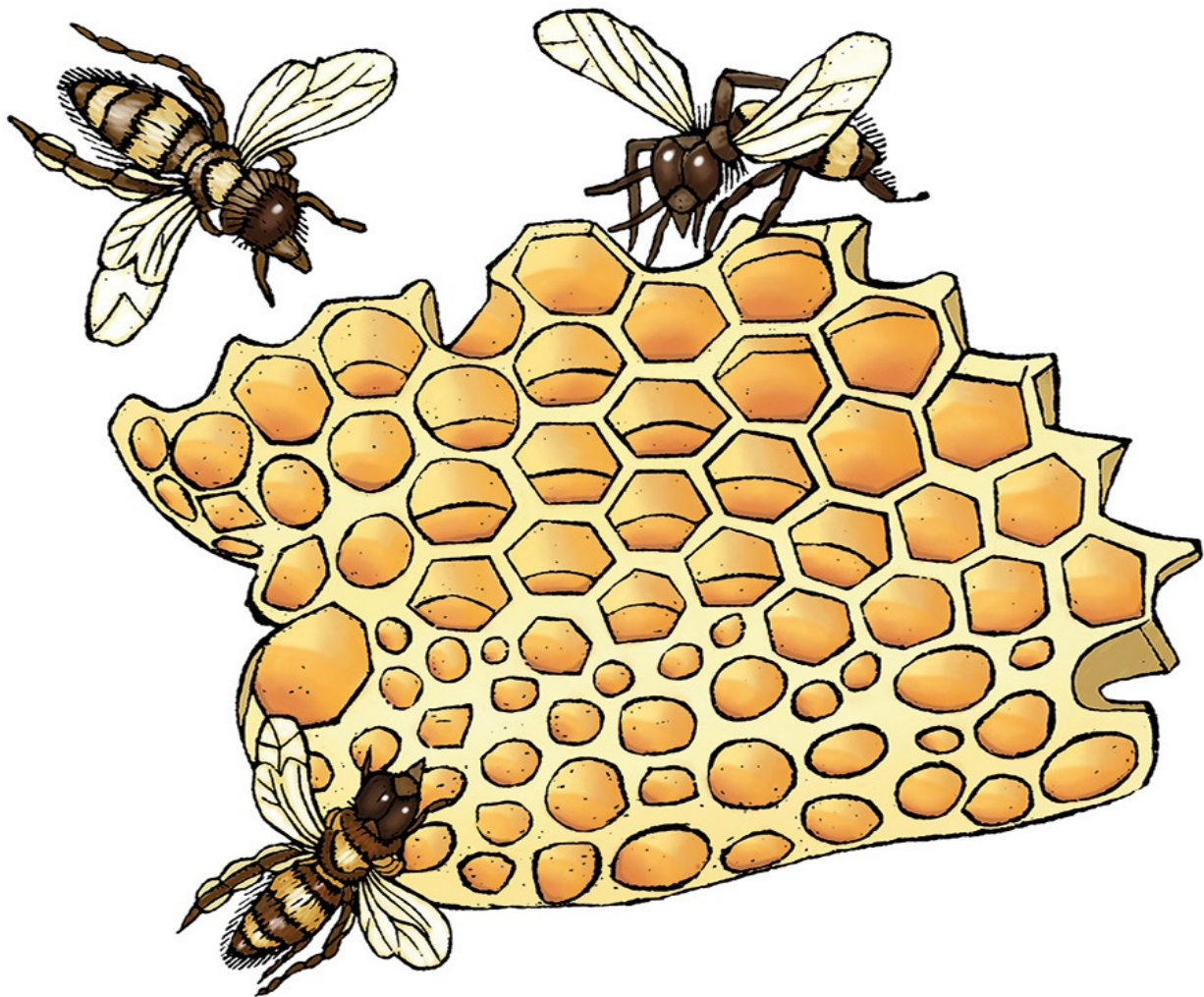


O'REILLY®

Segunda
Edição

Criando Microserviços

Projetando sistemas com componentes
menores e mais especializados



novatec

Sam Newman

Criando Microserviços

Projetando sistemas com componentes menores e mais especializados

Sam Newman

O'REILLY*
Novatec

Authorized Portuguese translation of the English edition of *Building Microservices, 2nd Edition*, ISBN 9781492034025 © 2021 Sam Newman. This translation is published and sold by permission of O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

Tradução em português autorizada da edição em inglês da obra *Building Microservices, 2nd Edition*, ISBN 9781492034025 © 2021 Sam Newman. Esta tradução é publicada e vendida com a permissão da O'Reilly Media, Inc., detentora de todos os direitos para publicação e venda desta obra.

© Novatec Editora Ltda. [2022].

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates

Tradução: Lúcia A. Kinoshita

Revisão gramatical: Tássia Carvalho

ISBN do impresso: 978-65-86057-88-1

ISBN do ebook: 978-65-86057-89-8

Histórico de impressões:

Março/2022 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil

Tel.: +55 11 2959-6529

Email: novatec@novatec.com.br

Site: <https://novatec.com.br>

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/in/novatec

GRP20220308

Sumário

Prefácio

parte I Conceitos básicos

capítulo 1 O que são microserviços?

Apresentação rápida dos microserviços

Conceitos essenciais dos microserviços

Implantações independentes

Modelagem com base em um domínio de negócios

Responsáveis pelo próprio estado

Tamanho

Flexibilidade

Alinhamento entre arquitetura e organização

Sistema monolítico

Sistema monolítico com um único processo

Sistema monolítico modular

Sistema monolítico distribuído

Sistemas monolíticos e desavença nas entregas

Vantagens dos sistemas monolíticos

Tecnologias que viabilizam os microserviços

Agregação de logs e tracing distribuído

Contêineres e Kubernetes

Streaming

Nuvem pública e produtos serverless

Vantagens dos microserviços

Heterogeneidade de tecnologias

Robustez

Escalabilidade

Facilidade de implantação

Alinhamento organizacional

Composição

Complexidades dos microsserviços

Experiência dos desenvolvedores

Sobrecarga tecnológica

Custos

Relatórios

Monitoração e resolução de problemas

Segurança

Testes

Latência

Consistência dos dados

Devo usar microsserviços?

A quem os microsserviços talvez não sejam apropriados

Quando os microsserviços funcionam bem

Resumo

capítulo 2 Como modelar microsserviços

Apresentando a MusicCorp

O que é uma boa fronteira para um microsserviço?

Ocultação de informações

Coesão

Acoplamento

Interação entre acoplamento e coesão

Tipos de acoplamento

Acoplamento de domínio

Acoplamento de passagem

Acoplamento de dados em comum

Acoplamento de conteúdo

Apenas o suficiente sobre design orientado a domínios

Linguagem ubíqua

Agregado

Contexto delimitado

Mapeando agregados e contextos delimitados aos microsserviços

Event storming

Design Orientado a Domínios no caso dos microsserviços

Alternativas às fronteiras baseadas em domínios de negócios

Volatilidade

[Dados](#)
[Tecnologia](#)
[Estrutura organizacional](#)
[Combinando modelos e exceções](#)
[Resumo](#)

capítulo 3 Dividindo o sistema monolítico

[Tenha um objetivo](#)
[Migração gradual](#)
[O sistema monolítico raramente é o inimigo](#)
[Perigos de uma decomposição prematura](#)
[O que separar antes?](#)
[Decomposição por camadas](#)
[Código antes](#)
[Dados antes](#)
[Padrões convenientes para decomposição](#)
[Padrão strangler fig](#)
[Execução paralela](#)
[Flag de funcionalidade](#)
[Preocupações na decomposição de dados](#)
[Desempenho](#)
[Integridade dos dados](#)
[Transações](#)
[Ferramentas](#)
[Banco de dados para relatórios](#)
[Resumo](#)

capítulo 4 Estilos de comunicação entre microserviços

[Da comunicação interna ao processo à comunicação entre processos](#)
[Desempenho](#)
[Alterando as interfaces](#)
[Tratamento de erros](#)
[Tecnologias para comunicação entre processos: muitas opções](#)
[Estilos de comunicação entre microserviços](#)
[Estilos mistos](#)
[Padrão: síncrono bloqueante](#)

[Vantagens](#)
[Desvantagens](#)
[Quando usar](#)
[Padrão: assíncrono não bloqueante](#)
[Vantagens](#)
[Desvantagens](#)
[Quando usar](#)
[Padrão: comunicação por meio de dados em comum](#)
[Implementação](#)
[Vantagens](#)
[Desvantagens](#)
[Quando usar](#)
[Padrão: comunicação por meio de requisição-resposta](#)
[Implementação: síncrono versus assíncrono](#)
[Quando usar](#)
[Padrão: comunicação orientada a eventos](#)
[Implementação](#)
[O que há em um evento?](#)
[Quando usar](#)
[Prossiga com cautela](#)
[Resumo](#)

parte II Implementação

capítulo 5 Implementando a comunicação entre microsserviços

[Em busca da tecnologia ideal](#)
[Facilite a compatibilidade com versões anteriores](#)
[Deixe sua interface explícita](#)
[Mantenha suas APIs independentes de tecnologia](#)
[Deixe seu serviço simples para os consumidores](#)
[Oculte detalhes de implementação internos](#)
[Opções de tecnologia](#)
[Remote procedure calls](#)
[REST](#)
[GraphQL](#)
[Brokers de mensagens](#)

[Formatos de serialização](#)

[Formatos textuais](#)

[Formatos binários](#)

[Esquemas](#)

[Incompatibilidades de contrato estruturais versus semânticas](#)

[Devemos usar esquemas?](#)

[Lidando com mudanças entre microserviços](#)

[Evitando mudanças que causam incompatibilidade](#)

[Mudanças com expansão](#)

[Leitor tolerante](#)

[Tecnologia correta](#)

[Interface explícita](#)

[Identifique rapidamente as mudanças que causam incompatibilidades
acidentais](#)

[Gerenciando mudanças que causam incompatibilidade](#)

[Implantação sincronizada](#)

[Permitir a coexistência de versões incompatíveis de microserviços](#)

[Emular a interface antiga](#)

[Qual é a minha abordagem preferida?](#)

[Contrato social](#)

[Monitorando o uso](#)

[Medidas extremas](#)

[DRY e os perigos da reutilização de código em um mundo de
microserviços](#)

[Compartilhando código por meio de bibliotecas](#)

[Descoberta de serviços](#)

[Domain Name System \(DNS\)](#)

[Registros dinâmicos de serviços](#)

[Não se esqueça das pessoas!](#)

[Service meshes e gateways de API](#)

[Gateways de API](#)

[Service meshes](#)

[E quanto aos outros protocolos?](#)

[Documentação dos serviços](#)

[Esquemas explícitos](#)

[Sistema que se autodescreve](#)

Resumo

capítulo 6 Fluxo de trabalho

Transações em bancos de dados

Transações ACID

Ainda ACID, mas sem atomicidade?

Transações distribuídas – commit de duas fases

Transações distribuídas – simplesmente diga não

Sagas

Modos de falha das sagas

Implementando sagas

Sagas versus transações distribuídas

Resumo

capítulo 7 Construção

Uma rápida introdução à integração contínua

Você realmente coloca a CI em prática?

Modelos de branching

Pipelines de construção e entrega contínua

Ferramentas

Relação de custo-benefício e ambientes

Criação de artefatos

Maapeando códigos-fontes e builds aos microserviços

Um repositório gigante, uma build gigante

Padrão: um repositório por microserviço (também conhecido como multirepo)

Padrão: monorepo

Qual abordagem devo utilizar?

Resumo

capítulo 8 Implantação

Do lógico ao físico

Várias instâncias

Banco de dados

Ambientes

Princípios da implantação de microserviços

- [Execução isolada](#)
- [Foco na automação](#)
- [Infraestrutura como código](#)
- [Implantação sem downtime](#)
- [Gerenciamento de estado desejado](#)
- [Opções para implantação](#)
 - [Máquinas físicas](#)
 - [Máquinas virtuais](#)
 - [Contêineres](#)
 - [Contêineres de aplicações](#)
 - [Plataforma como Serviço \(PaaS\)](#)
 - [Função como Serviço \(FaaS\)](#)
- [Qual é a opção de implantação correta para você?](#)
- [Kubernetes e a orquestração de contêineres](#)
 - [Razões para a orquestração de contêineres](#)
 - [Uma visão simplificada dos conceitos do Kubernetes](#)
 - [Multitenancy e federação](#)
 - [Cloud Native Computing Foundation](#)
 - [Plataformas e portabilidade](#)
 - [Helm, Operator e CRDs, minha nossa!](#)
 - [E o Knative](#)
 - [Futuro](#)
 - [Devo usá-lo?](#)
- [Entrega progressiva](#)
 - [Diferenciando implantação de lançamento](#)
 - [Em direção a uma entrega progressiva](#)
 - [Flags de funcionalidade](#)
 - [Lançamento canário](#)
 - [Execução paralela](#)
- [Resumo](#)

[capítulo 9 Testes](#)

- [Tipos de testes](#)
- [Escopo dos testes](#)
 - [Testes de unidade](#)
 - [Testes de serviço](#)

[Testes fim a fim](#)
[Custo-benefício](#)
[Implementando testes de serviço](#)
[Mocks ou stubs](#)
[Um serviço de stubs mais inteligente](#)
[Implementando \(aqueles complicados\) testes fim a fim](#)
[Testes frágeis e instáveis](#)
[Quem escreve esses testes fim a fim?](#)
[Por quanto tempo os testes fim a fim devem ser executados?](#)
[Acúmulo de alterações](#)
[Metaversão](#)
[Incapacidade de testar de forma independente](#)
[Devemos evitar testes fim a fim?](#)
[Testes de contrato e CDCs](#)
[Palavra final](#)
[Experiência dos desenvolvedores](#)
[Dos testes em pré-produção aos testes em produção](#)
[Tipos de testes no ambiente de produção](#)
[Deixando os testes em produção seguros](#)
[MTTR em vez de MTBF?](#)
[Testes multifuncionais](#)
[Testes de desempenho](#)
[Testes de robustez](#)
[Resumo](#)

[capítulo 10 Da monitoração à observabilidade](#)

[Instabilidade, pânico e confusão](#)
[Um único microsserviço, um único servidor](#)
[Um único microsserviço, vários servidores](#)
[Vários serviços, vários servidores](#)
[Observabilidade versus monitoração](#)
[Os pilares da observabilidade? Espere um pouco](#)
[Blocos de construção para a observabilidade](#)
[Agregação de logs](#)
[Agregação de métricas](#)
[Tracing distribuído](#)

[Estamos indo bem?](#)
[Alertas](#)
[Monitoração semântica](#)
[Testes no ambiente de produção](#)
[Padronização](#)
[Seleção de ferramentas](#)
[Democráticos](#)
[Fáceis de integrar](#)
[Fornecem contexto](#)
[Tempo real](#)
[Apropriados para a sua escala](#)
[Inteligência na máquina](#)
[Por onde começar](#)
[Resumo](#)

[capítulo 11 Segurança](#)

[Princípios básicos](#)
[Princípio do privilégio mínimo](#)
[Defesa em profundidade](#)
[Automação](#)
[Incluindo a segurança no processo de entregas](#)
[As cinco funções da cibersegurança](#)
[Identificar](#)
[Proteger](#)
[Detectar](#)
[Responder](#)
[Recuperar-se](#)
[Fundamentos da segurança de aplicações](#)
[Credenciais](#)
[Patching](#)
[Backups](#)
[Reconstrução](#)
[Confiança implícita versus confiança zero](#)
[Confiança implícita](#)
[Confiança zero](#)
[É um espectro](#)

[Protegendo os dados](#)

[Dados em trânsito](#)

[Dados em repouso](#)

[Autenticação e autorização](#)

[Autenticação de serviço a serviço](#)

[Autenticação de pessoas](#)

[Implementações comuns de SSO](#)

[Gateway de SSO](#)

[Autorizações mais específicas](#)

[Problema do confused deputy](#)

[Autorização upstream centralizada](#)

[Descentralizando a autorização](#)

[JSON Web Tokens](#)

[Resumo](#)

[capítulo 12 Resiliência](#)

[O que é resiliência?](#)

[Robustez](#)

[Recuperação](#)

[Extensibilidade com elegância](#)

[Adaptabilidade sustentável](#)

[E a arquitetura de microsserviços](#)

[As falhas estão em todos os lugares](#)

[Quando é demais?](#)

[Degradação das funcionalidades](#)

[Padrões de estabilidade](#)

[Timeouts](#)

[Novas tentativas](#)

[Bulkheads](#)

[Circuit breakers](#)

[Isolamento](#)

[Redundância](#)

[Middleware](#)

[Idempotência](#)

[Diluindo o seu risco](#)

[Teorema CAP](#)

[Sacrificando a consistência](#)
[Sacrificando a disponibilidade](#)
[Sacrificando a tolerância à partição?](#)
[AP ou CP?](#)
[Não é uma questão de tudo ou nada](#)
[E o mundo real](#)
[Engenharia de caos](#)
[Game Days](#)
[Experimentos no ambiente de produção](#)
[Partindo da robustez e indo além](#)
[Acusações](#)
[Resumo](#)

[capítulo 13 Escalabilidade](#)

[Quatro eixos da escalabilidade](#)
[Escalabilidade vertical](#)
[Duplicação horizontal](#)
[Partição de dados](#)
[Decomposição funcional](#)
[Combinando modelos](#)
[Comece em pequena escala](#)
[Caching](#)
[Para desempenho](#)
[Para escalabilidade](#)
[Para robustez](#)
[Onde usar um cache](#)
[Invalidação](#)
[Regra de ouro do caching](#)
[Dados atualizados versus otimização](#)
[Envenenamento de cache: uma lição](#)
[Escalabilidade automática](#)
[Começando de novo](#)
[Resumo](#)

[parte III Pessoas](#)

capítulo 14 Interfaces de usuário

Em direção ao digital

Modelos de responsabilidade

Razões para haver equipes dedicadas de frontend

Em direção a equipes alinhadas a fluxos

Compartilhando especialistas

Garantindo a consistência

Resolvendo os desafios técnicos

Padrão: frontend monolítico

Quando usar

Padrão: microfrontends

Implementação

Quando usar

Padrão: decomposição baseada em páginas

Quando usar

Padrão: decomposição baseada em widgets

Implementação

Quando usar

Limitações

Padrão: gateway de agregação central

Responsabilidade

Diferentes tipos de interfaces de usuário

Preocupações variadas

Quando usar

Padrão: backend para frontend (BFF)

Quanto BFFs?

Reutilização de código e BFFs

BFFs para web desktop e além

Quando usar

GraphQL

Uma abordagem híbrida

Resumo

capítulo 15 Estruturas organizacionais

Empresas com baixo acoplamento

[Lei de Conway](#)

[Evidências](#)

[Tamanho das equipes](#)

[Entendendo a lei de Conway](#)

[Equipes pequenas, empresa grande](#)

[Sobre a autonomia](#)

[Responsabilidade forte versus coletiva](#)

[Responsabilidade forte](#)

[Responsabilidade coletiva](#)

[No nível de uma equipe versus no nível organizacional](#)

[Achando um equilíbrio entre os modelos](#)

[Equipes de capacitação](#)

[Comunidades de prática](#)

[A plataforma](#)

[Microserviços compartilhados](#)

[Difícil demais para separar](#)

[Alterações abrangentes](#)

[Gargalos na entrega](#)

[Open source interno](#)

[Papel dos core committers](#)

[Maturidade](#)

[Ferramentas](#)

[Microserviços modulares e plugáveis](#)

[Revisões de alterações](#)

[Serviço órfão](#)

[Estudo de caso: realestate.com.au](#)

[Distribuição geográfica](#)

[Lei de Conway ao inverso](#)

[Pessoas](#)

[Resumo](#)

[capítulo 16 O arquiteto em evolução](#)

[O que há em um nome?](#)

[O que é arquitetura de software?](#)

[Tornando as mudanças possíveis](#)

[Uma visão evolucionária para o arquiteto](#)

[Definindo as fronteiras do sistema](#)

[Uma construção social](#)

[Habitabilidade](#)

[Uma abordagem com princípios](#)

[Objetivos estratégicos](#)

[Princípios](#)

[Práticas](#)

[Combinado princípios e práticas](#)

[Um exemplo do mundo real](#)

[Direcionando uma arquitetura em evolução](#)

[Arquitetura em uma empresa alinhada a fluxos](#)

[Construindo uma equipe](#)

[Padrão necessário](#)

[Monitoração](#)

[Interfaces](#)

[Segurança na arquitetura](#)

[Governança e a estrada pavimentada](#)

[Exemplares](#)

[Template personalizado para os microserviços](#)

[Estrada pavimentada em escala](#)

[Débito técnico](#)

[Tratamento de exceções](#)

[Resumo](#)

[Posfácio: reunindo tudo](#)

[Bibliografia](#)

[Glossário](#)

Prefácio

Os microsserviços são uma forma de abordagem aos sistemas distribuídos, promovendo o uso de serviços altamente especializados, possíveis de serem alterados, implantados e lançados independentemente. Para empresas que estejam caminhando em direção a sistemas menos acoplados, com equipes autônomas entregando funcionalidades voltadas aos usuários, os microsserviços são extremamente apropriados. Além disso, os microsserviços oferecem um número enorme de opções para a construção de sistemas, proporcionando bastante flexibilidade para garantir que possam ser alterados a fim de atender às necessidades dos usuários.

Contudo, os microsserviços não deixam de ter pontos negativos relevantes. Por serem um sistema distribuído, carregam consigo uma série de complexidades, muitas das quais podem ser uma novidade até mesmo para os desenvolvedores mais experientes.

As experiências das pessoas ao redor do mundo, junto com o surgimento de novas tecnologias, exercem um impacto profundo no modo como os microsserviços estão sendo usados. Este livro reúne essas ideias, além de mostrar exemplos concretos, do mundo real, que o ajudarão a entender se os microsserviços são uma opção certa para você.

Quem deve ler este livro

O escopo de *Criando Microsserviços* é amplo, assim como são as implicações das arquiteturas de microsserviços. Desse modo, este livro deve interessar às pessoas envolvidas com aspectos de design, desenvolvimento, implantação, testes e manutenção de sistemas. Aqueles entre vocês que já embarcaram na jornada em direção a arquiteturas com componentes altamente especializados, seja para uma aplicação totalmente nova ou como parte da decomposição de um sistema existente, mais monolítico, encontrarão vários conselhos práticos para ajudá-los. Este livro

também ajudará as pessoas que querem saber a razão de toda essa agitação em torno dos microsserviços, de modo a poder determinar se são uma opção correta para elas.

Por que escrevi este livro

Em certo aspecto, escrevi este livro porque queria me certificar de que as informações contidas na primeira edição permanecessem atualizadas, precisas e úteis. Escrevi a primeira edição porque havia ideias realmente interessantes que eu gostaria de compartilhar. Tive a sorte de estar em um momento em que tinha tempo e apoio para escrever a primeira edição, e podia fazer isso de um ponto de vista razoavelmente neutro, pois não trabalhava para nenhum grande fornecedor de tecnologia. Eu não estava vendendo uma solução, tampouco esperava que estivesse *vendendo* a ideia de microsserviços – apenas achava essa ideia fascinante, adorava explicar o conceito de microsserviços e encontrar maneiras de compartilhá-la de modo mais amplo.

Quando consegui realmente esmiuçá-la, escrevi uma segunda edição por dois motivos principais. Em primeiro lugar, achei que poderia fazer um trabalho melhor desta vez. Aprendi mais, e espero ter melhorado um pouco como escritor. No entanto, também escrevi esta segunda edição porque desempenhei um pequeno papel em contribuir para que essas ideias se disseminassem no mercado e, desse modo, sinto-me na obrigação de garantir que sejam apresentadas de forma sensata e equilibrada. Para muitas pessoas, os microsserviços passaram a ser a opção default de arquitetura. Isso é algo que acho difícil de justificar, e gostaria de ter uma oportunidade para explicar o porquê.

Este livro não é a *favor* dos microsserviços, mas também não é *contra*. Gostaria apenas de garantir que o contexto no qual essas ideias funcionam bem fosse devidamente explorado, além de explicar os problemas que elas podem causar.

O que mudou desde a primeira edição?

Escrevi a primeira edição de *Criando Microsserviços* em aproximadamente um ano, começando no início de 2014; o livro foi lançado em fevereiro de 2015. Estávamos no início da história dos microsserviços, ao menos no que diz respeito ao conhecimento mais amplo do termo pelo mercado. Desde então, os microsserviços se tornaram muito populares, de um modo que eu não poderia ter imaginado. Com esse crescimento, houve um conjunto muito maior de experiências no qual se basear, e surgiram mais tecnologias para serem exploradas.

À medida que trabalhava com mais equipes logo após a primeira edição, comecei a aprimorar meu raciocínio em torno das ideias associadas aos microsserviços. Em alguns casos, isso significou que ideias que estavam apenas na periferia do meu pensamento, como a ocultação de informações, começaram a ficar mais claras como conceitos fundamentais, que precisavam ser mais bem esclarecidos. Em outras áreas, novas tecnologias apresentavam tanto soluções novas como problemas novos para nossos sistemas. Ver tantas pessoas se voltando para o Kubernetes na esperança de que ele resolveria todos os problemas com as arquiteturas de microsserviços certamente me fez parar para pensar.

Além do mais, eu havia escrito a primeira edição de *Criando Microsserviços* para oferecer não só uma explicação sobre os microsserviços, mas também para apresentar uma visão geral de como essa abordagem de arquitetura modifica alguns aspectos do desenvolvimento de software. Assim, enquanto analisava aspectos relacionados à segurança e resiliência com mais detalhes, eu me vi querendo voltar e expandir os assuntos que são cada vez mais importantes no desenvolvimento moderno de software.

Desse modo, nesta segunda edição, dediquei mais tempo apresentando exemplos explícitos para explicar melhor as ideias. Cada capítulo foi revisto, e cada frase foi revisada. Não restou muito da primeira edição no que concerne a uma prosa concreta, mas todas as ideias continuam presentes. Tentei ser mais claro em minhas opiniões, mas ainda reconhecendo que, frequentemente, há várias maneiras de resolver um problema. Isso significou expandir a discussão sobre comunicação entre processos (inter-process communication), que agora foi separada em três

capítulos. Também dediquei mais tempo analisando as implicações de tecnologias como contêineres, Kubernetes e serverless (sem servidor); como resultado, agora há capítulos separados para build (construção) e implantação.

Minha esperança era escrever um livro aproximadamente do mesmo tamanho que a primeira edição, ao mesmo tempo que encontrasse uma maneira de incluir mais ideias. Como podemos ver, fracassei em atingir meu objetivo – esta edição é maior! Contudo, acho que fui bem-sucedido em ser mais claro no modo como articulei as ideias.

Navegando pelo livro

Este livro está essencialmente organizado em um formato que se baseia em tópicos. Estructurei e escrevi o livro como você o leria, do início ao fim, mas é claro que é possível ir direto aos tópicos específicos que mais interessem a você. Se decidir explorar um capítulo específico diretamente, talvez você ache conveniente consultar o Glossário no final do livro, o qual contém explicações sobre termos novos e desconhecidos. Falando de terminologia, utilizei os termos *microserviço* e *serviço* indiscriminadamente no livro; você pode pressupor que ambos se referem ao mesmo conceito, a menos que eu diga explicitamente o contrário. Também resumi algumas das principais referências contidas no livro na seção de Bibliografia caso você queira *realmente* ir direto para o final – lembre-se, porém, de que você perderá *muitos* detalhes se fizer isso!

O corpo principal do livro está dividido em três partes: Conceitos básicos, Implementação e Pessoas. Vamos ver o que cada parte inclui.

Parte I, Conceitos básicos

Nessa parte, detalharei algumas das ideias fundamentais acerca dos microserviços.

Capítulo 1, O que são microserviços?

É uma introdução geral aos microserviços, na qual discuto rapidamente uma série de tópicos que serão expandidos posteriormente no livro.

Capítulo 2, Como modelar microsserviços

Esse capítulo analisa a importância de conceitos como ocultação de informações, acoplamento, coesão e design orientado a domínios para ajudar a definir fronteiras corretas para seus microsserviços.

Capítulo 3, Dividindo o sistema monolítico

Esse capítulo oferece algumas orientações sobre como partir de uma aplicação monolítica existente e separá-la em microsserviços.

Capítulo 4, Estilos de comunicação entre microsserviços

O último capítulo dessa parte apresenta uma discussão sobre os diferentes tipos de comunicação entre os microsserviços, incluindo chamadas assíncronas *versus* síncronas e estilos de colaboração do tipo requisição-resposta (request-response) e orientado a eventos (event-driven).

Parte II, Implementação

Passando dos conceitos mais genéricos para os detalhes de implementação, nessa parte, veremos técnicas e tecnologias que podem ajudar você a tirar o máximo de proveito dos microsserviços.

Capítulo 5, Implementando a comunicação entre microsserviços

Nesse capítulo, exploraremos melhor as tecnologias específicas usadas para implementar a comunicação entre os microsserviços.

Capítulo 6, Fluxo de trabalho

Esse capítulo apresenta uma comparação entre sagas e transações distribuídas, além de discutir sua utilidade na modelagem de processos de negócios envolvendo vários microsserviços.

Capítulo 7, Construção

Esse capítulo explica como fazer um mapeamento de um microsserviço para repositórios e builds.

Capítulo 8, Implantação

Nesse capítulo, veremos uma discussão sobre a variedade de opções associadas à implantação de um microsserviço, incluindo discussões

sobre contêineres, Kubernetes e FaaS.

Capítulo 9, Testes

Nesse capítulo, serão discutidos os desafios de testar os microsserviços, incluindo os problemas causados por testes fim a fim (end-to-end tests), e como testes com contratos orientados ao consumidor (consumer-driven contracts) e de produção (in-production) podem ajudar.

Capítulo 10, Da monitoração à observabilidade

Esse capítulo discute a mudança de foco, partindo de atividades de monitoração estáticas para um pensamento mais amplo sobre como melhorar a observabilidade das arquiteturas de microsserviços, além de apresentar algumas recomendações específicas quanto a ferramentas.

Capítulo 11, Segurança

As arquiteturas de microsserviços podem aumentar a área da superfície de ataque, mas também nos dão mais oportunidades para melhorar a defesa. Nesse capítulo, exploraremos esse equilíbrio.

Capítulo 12, Resiliência

Esse capítulo oferece uma visão mais ampla sobre o que é resiliência e o papel que os microsserviços podem desempenhar para melhorar a resiliência de suas aplicações.

Capítulo 13, Escalabilidade

Nesse capítulo, apresentarei os quatro eixos da escalabilidade e mostrarei como podem ser usados em conjunto para escalar uma arquitetura de microsserviços.

Parte III, Pessoas

Ideias e tecnologias não significam nada sem as pessoas e uma organização para dar suporte a elas.

Capítulo 14, Interfaces de usuário

Ao se afastar de equipes de frontend dedicadas e se voltar para o uso de BFFs e GraphQL, esse capítulo explora o modo como os microsserviços e as interfaces de usuário podem funcionar juntos.

Capítulo 15, Estruturas organizacionais

O foco desse penúltimo capítulo está em como as equipes alinhadas a fluxo (stream-aligned teams) e as equipes de capacitação (enabling teams) podem trabalhar no contexto das arquiteturas de microsserviços.

Capítulo 16, O arquiteto em evolução

Arquiteturas de microsserviços não são estáticas, portanto é possível que sua visão da arquitetura de sistemas precise mudar – um assunto que esse capítulo explorará em detalhes.

Convenções usadas no livro

As convenções tipográficas a seguir são usadas neste livro:

Itálico

Indica novos termos, URLs, endereços de email, nomes de arquivo e extensões de arquivo.

Largura fixa

Usada em listagens de programas, assim como dentro de parágrafos para se referir a elementos do programa como nomes de variáveis e funções, bancos de dados, tipos de dados, variáveis de ambiente, instruções e palavras-chave.



Este elemento significa uma dica ou sugestão.



Este elemento significa uma observação geral.



Este elemento significa um aviso ou uma precaução.

Como entrar em contato conosco

Envie comentários e dúvidas sobre este livro para: novatec@novatec.com.br.

Temos uma página da web para este livro, na qual incluimos a lista de erratas, exemplos e qualquer outra informação adicional.

- Página da edição em português

<https://novatec.com.br/livros/criando-microservicos-2ed>

- Página da edição original, em inglês

https://oreil.ly/Building_Microservices2

Para obter mais informações sobre livros da Novatec, acesse nosso site em:

<https://novatec.com.br>

Agradecimentos

Não me canso de ficar impressionado com o apoio que tenho de minha família, particularmente de minha esposa Lindy Stephens, que tolera tanta coisa com uma elegância incrível. Dizer que este livro não existiria sem ela nem seria necessário, ainda que ela nem sempre acredite em mim quando digo isso. Este livro é dedicado a ela. Também é dedicado ao meu pai, a Jack, Josie, Kane e ao clã Gilmanco Staynes mais amplo.

Boa parte deste livro foi escrita durante uma pandemia global; enquanto escrevo estes agradecimentos, ela continua em andamento. Pode não significar muito, mas gostaria de estender meus agradecimentos ao Serviço Nacional de Saúde do Reino Unido e a todas as pessoas do mundo que estão nos protegendo, seja trabalhando com vacinas, tratando os doentes, entregando nossa comida ou ajudando de milhares de outras maneiras que eu nem saberia dizer. Meus agradecimentos também a todos vocês.

Esta segunda edição não existiria sem a primeira, portanto gostaria de reiterar meus agradecimentos a todos que me ajudaram durante o processo desafiador de escrever meu primeiro livro, incluindo os revisores técnicos Ben Christensen, Martin Fowler e Venkat Subramaniam, a James Lewis pelas nossas diversas conversas esclarecedoras, à equipe da O'Reilly composta de Brian MacDonald, Rachel Monaghan, Kristen Brown e Betsy Waliszewski; agradeço também pelos excelentes feedbacks de revisão dos leitores Anand Krishnaswamy, Kent McNeil, Charles Haynes, Chris Ford, Aidy Lewis, Will Thames, Jon Eaves, Rolf Russell, Badrinath Janakiraman, Daniel Bryant, Ian Robinson, Jim Webber, Stewart Gleadow, Evan Bottcher, Eric Sword e Olivia Leonard. E obrigado a Mike Loukides, acho, por me colocar nessa confusão, antes de tudo!

Na segunda edição, mais uma vez, Martin Fowler foi o revisor técnico, junto com Daniel Bryant e Sarah Wells, que foram generosos com seu tempo e feedback. Gostaria também de agradecer a Nicky Wrightson e a Alexander von Zitzerwitz por ajudarem a levar a revisão técnica até a linha de chegada. Do lado da O'Reilly, todo o processo foi supervisionado pela minha incrível editora Nicole Taché – sem ela, eu certamente teria enlouquecido; agradeço a Melissa Duffield, que parece fazer um trabalho melhor do que o meu para administrar minha carga de trabalho, a Deb Baker, Arthur Johnson e aos demais da equipe de produção (sinto muito por não saber o nome de todos vocês, mas obrigado!), e a Mary Treseler, por comandar o navio durante alguns momentos de dificuldades.

Além disso, a segunda edição se beneficiou enormemente com a ajuda e os insights de várias pessoas, incluindo (em nenhuma ordem em particular) Dave Coombes e a equipe da Tyro, Dave Halsey e a equipe da Money Supermarket, Tom Kerkhove, Erik Doernenburg, Graham Tackley, Kent Beck, Kevlin Henney, Laura Bell, Adrian Mouat, Sarah Taraporewalla, Uwe Friedrichse, Liz Fong-Jones, Kane Stephens, Gilmanco Staynes, Adam Tornhill, Venkat Subramaniam, Susanne Kaiser, Jan Schaumann, Grady Booch, Pini Reznik, Nicole Forsgren, Jez Humble, Gene Kim, Manuel Pais, Matthew Skelton e os South Sydney Rabbitohs. Por fim, gostaria de agradecer aos incríveis leitores da versão preliminar acessível do livro, que deram feedbacks de valor inestimável; estão aí incluídos Felipe de Moraes, Mark Gardner, David Lauzon, Assam Zafar, Michael Bleterman, Nicola Musatti, Eleonora Lester, Nathan DiMauro, Daniel Lemke, Soner Eker, Ripple Shah, Joel Lim e Himanshu Pant. Por fim, um alô para Jason Isaacs.

Escrevi a maior parte deste livro durante todo o ano de 2020 e na primeira metade de 2021. Utilizei o Visual Studio Code no macOS na maior parte do processo de escrita, embora, em raras ocasiões, também tenha usado o Working Copy no iOS. O OmniGraffle foi utilizado para criar todos os diagramas que estão no livro. O AsciiDoc foi usado para formatar o livro e, de modo geral, foi excelente, enquanto o conjunto de ferramentas Atlas da O'Reilly fez a magia de fazer um livro aparecer do outro lado.

PARTE I

Conceitos básicos

0 que são microsserviços?

Os microsserviços (microservices) se tornaram uma opção cada vez mais popular de arquitetura nos últimos cinco anos ou mais, desde que escrevi a primeira edição deste livro. Não posso alegar que eu tenha sido o responsável pela subsequente explosão da popularidade dos microsserviços, mas a pressa em fazer uso das arquiteturas de microsserviços implica que muitas das ideias que apresentei antes estão hoje em dia testadas e comprovadas, enquanto novas ideias foram acrescentadas à mistura, ao mesmo tempo que práticas anteriores caíram em desuso. Portanto, mais uma vez, é hora de destilar a essência da arquitetura de microsserviços, enfatizando os conceitos fundamentais que fazem com que os microsserviços funcionem.

Este livro como um todo foi concebido para apresentar uma visão geral do impacto causado pelos microsserviços em vários aspectos da entrega de softwares. Para começar, este capítulo apresentará as principais ideias nas quais os microsserviços se baseiam, as técnicas anteriores que nos trouxeram até os dias atuais e algumas das razões para essas arquiteturas serem tão amplamente empregadas.

Apresentação rápida dos microsserviços

Microsserviços (microservices) são serviços que podem ser lançados de forma independente e são modelados com base em um domínio de negócios. Um serviço encapsula uma funcionalidade e a torna acessível a outros serviços através de redes – podemos criar um sistema mais complexo a partir desses blocos de construção. Um microsserviço pode representar um estoque, outro pode representar o gerenciamento de pedidos, enquanto outro ainda representa a expedição de mercadorias;

juntos, porém, eles podem compor um sistema completo de comércio eletrônico. Os microsserviços são uma opção de arquitetura com foco em oferecer diversas possibilidades para solucionar problemas com os quais você pode deparar.

A arquitetura de microsserviços é um *tipo* de arquitetura orientada a serviços, com uma definição bem clara sobre como as fronteiras dos serviços devem ser traçadas, e para a qual a possibilidade de implantações independentes é fundamental. É uma arquitetura independente de tecnologia, e essa é uma das vantagens que ela oferece.

Do ponto de vista externo, um microsserviço individual é tratado como uma caixa-preta. Ele hospeda uma funcionalidade de negócios em um ou mais endpoints da rede (por exemplo, uma fila ou uma API REST, conforme vemos na Figura 1.1), utilizando quaisquer protocolos que sejam mais apropriados. Os consumidores, não importa se sejam outros microsserviços ou outros tipos de programas, acessam essas funcionalidades por meio desses endpoints na rede. Os detalhes internos de implementação (como a tecnologia com a qual o serviço foi implementado ou o modo como os dados são armazenados) permanecem totalmente ocultos para o mundo externo. Isso significa que as arquiteturas de microsserviços evitam o uso de bancos de dados compartilhados na maioria das circunstâncias; em vez disso, cada microsserviço encapsula o seu próprio banco de dados quando for necessário.

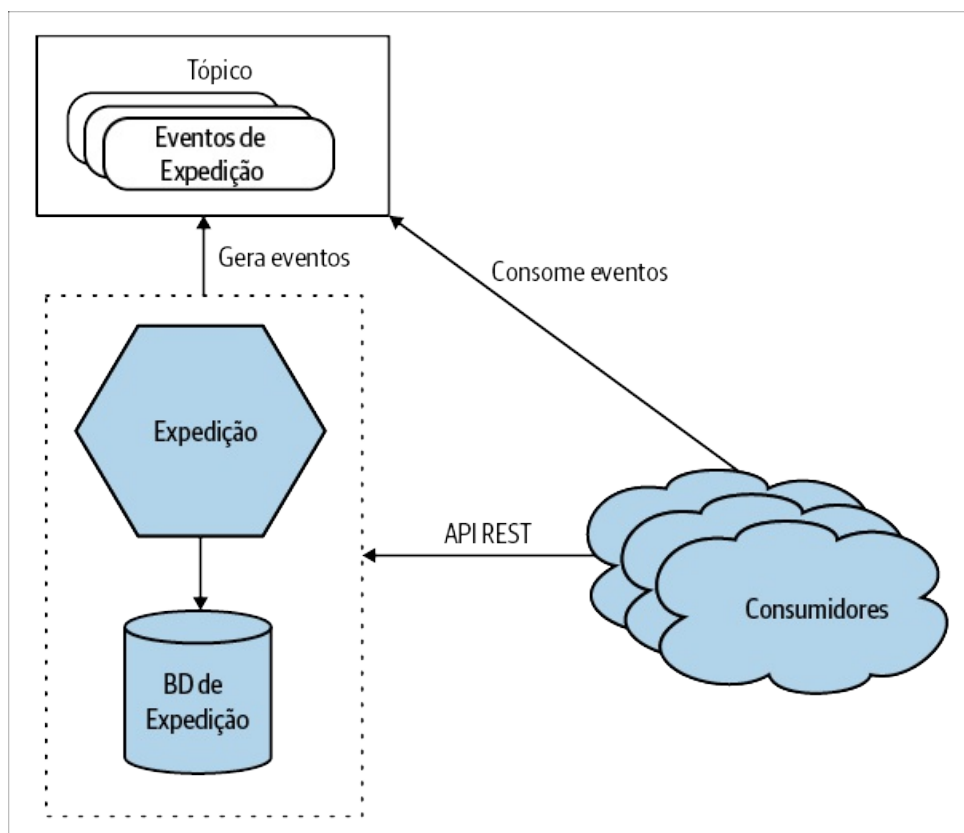


Figura 1.1 – Um microserviço expondo sua funcionalidade por meio de uma API REST e um tópico.

Os microserviços adotam o conceito de ocultação de informações.¹ *Ocultação de informações* significa ocultar o máximo possível de informações dentro de um componente e expor o mínimo possível por meio de interfaces externas. Isso permite ter uma clara separação entre o que pode ser mudado facilmente e o que é mais difícil de ser alterado. Implementações ocultas aos componentes externos podem ser modificadas livremente, desde que as interfaces de rede expostas pelos microserviços não mudem de modo a se tornarem incompatíveis com versões anteriores. Alterações dentro das fronteiras de um microserviço (conforme mostra a Figura 1.1) não devem afetar um consumidor upstream, o que permite que a funcionalidade seja lançada de forma independente. Esse aspecto é essencial para que possamos trabalhar em nossos microserviços de forma isolada e eles possam ser lançados por demanda. Ter fronteiras de serviço claras e estáveis, que não mudem quando a implementação interna mudar, resulta em sistemas com menos acoplamento e mais coesão.

Por falar em ocultar detalhes internos de implementação, eu seria negligente se não mencionasse o padrão de *Arquitetura Hexagonal*, inicialmente detalhado por Alistair Cockburn². Esse padrão descreve a importância de manter a implementação interna separada de suas interfaces externas, com base na ideia de que podemos querer interagir com a mesma funcionalidade usando diferentes tipos de interfaces. Eu desenho meus microsserviços como hexágonos parcialmente para diferenciá-los dos serviços “normais”, mas também como uma homenagem a essa forma de arte do passado.

Arquitetura orientada a serviços e microsserviços são diferentes?

A SOA (Service-Oriented Architecture, ou Arquitetura Orientada a Serviços) é uma abordagem de design na qual vários serviços atuam em colaboração para oferecer um determinado conjunto final de recursos. (Um *serviço*, nesse caso, geralmente significa um processo totalmente separado no sistema operacional.) A comunicação entre esses serviços se dá por meio de chamadas em rede, em vez de serem chamadas de métodos dentro das fronteiras de um processo.

A SOA surgiu como uma abordagem para combater os problemas apresentados por aplicações grandes e monolíticas. Essa abordagem tem como objetivo promover a reusabilidade do software; duas ou mais aplicações de usuários finais, por exemplo, poderiam utilizar os mesmos serviços. A SOA tem como propósito facilitar a manutenção ou a reescrita de softwares, pois, teoricamente, podemos substituir um serviço por outro sem que ninguém saiba, desde que a semântica do serviço não mude muito.

Em sua essência, a SOA é uma ideia sensata. No entanto, apesar de muitos esforços, há falta de um bom consenso acerca de como empregar *bem* a SOA. Em minha opinião, boa parte do mercado fracassou em encarar o problema de forma suficientemente holística e apresentar uma alternativa atraente à narrativa criada pelos diversos fornecedores nessa área.

Muitos dos problemas atribuídos à SOA na verdade são problemas com itens como protocolos de comunicação (por exemplo, SOAP), middleware de fornecedores, falta de diretrizes sobre o nível de

especialização dos serviços ou orientações incorretas sobre a escolha dos pontos para separar o seu sistema. Um cínico poderia sugerir que os fornecedores assumiram o controle do movimento SOA (e, em alguns casos, o direcionaram) como uma forma de vender mais produtos e, no final, esses mesmos produtos comprometeram o objetivo da SOA.

Já vi muitos exemplos de SOA nos quais as equipes estavam se esforçando para diminuir o tamanho dos serviços, mas continuavam com tudo acoplado a um banco de dados e tinham de implantar todos os serviços juntos. Orientado a serviços? Sim. Mas não são microsserviços.

A abordagem de microsserviços surgiu a partir do uso no mundo real, com base em nossa melhor compreensão sobre sistemas e arquitetura para um bom emprego da SOA. Devemos pensar nos microsserviços como uma abordagem específica para a SOA, da mesma forma que a XP (Extreme Programming, ou Programação Extrema) ou o Scrum são uma abordagem específica para um desenvolvimento de software Agile (Ágil).

Conceitos essenciais dos microsserviços

Algumas ideias essenciais devem ser compreendidas quando exploramos os microsserviços. Considerando que alguns aspectos são frequentemente ignorados, é essencial explorar melhor esses conceitos a fim de garantir que você entenda exatamente o que faz com que os microsserviços funcionem.

Implantações independentes

A possibilidade de *implantações independentes* é a ideia de que podemos fazer uma alteração em um microsserviço, implantá-lo e disponibilizar essa alteração aos nossos usuários, sem ter de implantar outros microsserviços. Acima de tudo, não é apenas o fato de que seja possível fazer isso, mas de que esse é *realmente* o modo de gerenciar as implantações em seu sistema. É uma disciplina adotada como abordagem padrão para o lançamento de versões. É uma ideia que, apesar de simples, é complexa quanto à execução.



Se você vai reter somente uma ideia deste livro e do conceito de microsserviços em geral, deve ser a seguinte: garanta que vai adotar o conceito de implantações independentes para seus microsserviços. Adquirir o hábito de implantar e lançar alterações em um único microsserviço no ambiente de produção, sem ter de fazer outras implantações. A partir daí, muitas coisas boas virão.

Para garantir implantações independentes, precisamos garantir que nossos microsserviços tenham *baixo acoplamento*: devemos ser capazes de modificar um serviço sem ter de alterar outras partes. Isso significa que precisamos ter contratos explícitos, bem definidos e estáveis entre os serviços. Algumas opções de implementação dificultam essa tarefa – o compartilhamento de bancos de dados, por exemplo, é particularmente problemático.

A possibilidade de implantações independentes por si só é, claramente, muito importante. Contudo, para que isso seja possível, há várias outras tarefas que devem ser feitas corretamente, as quais, por sua vez, trazem seus próprios benefícios. Desse modo, podemos encarar o foco nas implantações independentes como uma força motriz – ao manter o foco nela como resultado, conquistaremos uma série de benefícios complementares.

O desejo de ter serviços com baixo acoplamento, com interfaces estáveis, deve direcionar o nosso raciocínio acerca de como determinar as fronteiras de nossos microsserviços, antes de tudo.

Modelagem com base em um domínio de negócios

Técnicas como o DDD (Domain-Driven Design, ou Design Orientado a Domínios) permitem estruturar seu código de modo a representar melhor o domínio em que o software atua no mundo real.³ Nas arquiteturas de microsserviços, empregamos essa mesma ideia para definir as fronteiras de nossos serviços. Ao modelar os serviços com base nos domínios de negócios, podemos facilitar o lançamento de novas funcionalidades e recombinar os microsserviços de maneiras distintas a fim de disponibilizar novas funcionalidades aos nossos usuários.

Lançar uma funcionalidade que exija alterações em mais de um microserviço é uma tarefa custosa. É preciso coordenar o trabalho entre os serviços (e, possivelmente, entre equipes distintas) e gerenciar cuidadosamente a ordem na qual as novas versões desses serviços serão implantadas. Isso exige muito mais trabalho do que fazer a mesma alteração em um único serviço (ou num sistema monolítico, se for o caso). Consequentemente, queremos encontrar meios de garantir que alterações que envolvam mais de um serviço tenham a menor frequência possível.

Muitas vezes, vejo arquiteturas em camadas, conforme exemplificada pela arquitetura de três camadas da Figura 1.2. Nesse caso, cada camada da arquitetura representa uma fronteira de serviço diferente, e cada fronteira de serviço é definida com base em funcionalidades técnicas associadas. Se eu precisasse fazer uma alteração somente na camada de apresentação nesse exemplo, essa arquitetura seria razoavelmente eficaz. No entanto, a experiência mostra que mudanças em funcionalidade geralmente englobam várias camadas nesses tipos de arquitetura – exigindo alterações nas camadas de apresentação, de aplicação e de dados. Esse problema se agravará mais ainda se a arquitetura tiver mais camadas além das que vemos no exemplo simples da Figura 1.2; com frequência, cada camada se divide em outras camadas.

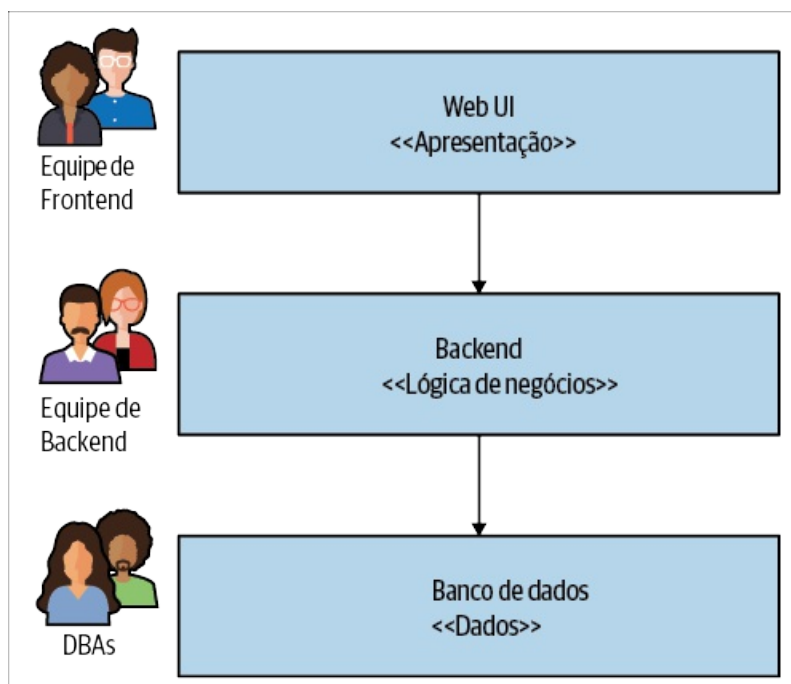


Figura 1.2 – Uma arquitetura tradicional de três camadas.

Ao fazer com que nossos serviços sejam fatias completas das funcionalidades de negócios, garantimos que nossa arquitetura esteja organizada de modo a tornar as alterações nas funcionalidades de negócios o mais eficiente possível. Sem dúvida, com os microsserviços, tomamos a decisão de dar prioridade a uma alta coesão nas funcionalidades de negócios, em vez de uma alta coesão nas funcionalidades técnicas.

Retomaremos a interação entre o design orientado a domínios e o design organizacional mais adiante neste capítulo.

Responsáveis pelo próprio estado

Um dos pontos com os quais vejo as pessoas terem mais dificuldade é a ideia de que os microsserviços devem evitar o uso de bancos de dados compartilhados. Se um microsserviço quiser acessar dados mantidos por outro microsserviço, ele deve solicitar esses dados a esse segundo microsserviço. Isso dá aos microsserviços a capacidade de decidir o que será compartilhado e o que estará oculto, permitindo fazer uma distinção clara entre uma funcionalidade que pode ser alterada livremente (nossa implementação interna) e uma funcionalidade que queremos modificar raramente (o contrato externo utilizado pelos consumidores).

Se quisermos que as implantações independentes se tornem uma realidade, devemos garantir que alterações em nossos microsserviços que causem incompatibilidade com versões anteriores sejam limitadas. Se quebrarmos a compatibilidade com os consumidores upstream, forçaremos que eles mudem também. Ter uma separação clara entre detalhes internos de implementação e o contrato externo de um microsserviço pode ajudar a reduzir a necessidade de mudanças que causem incompatibilidade com versões anteriores.

Ocultar o estado interno em um microsserviço é análogo à prática de encapsulamento na programação orientada a objetos (OO). O encapsulamento de dados em sistemas OO é um exemplo da técnica de ocultação de informações em ação.



Não compartilhe bancos de dados, a menos que seja realmente necessário. Mesmo nesse caso, faça tudo que for possível para evitar isso. Em minha opinião, compartilhar bancos de dados é uma das piores escolhas que você pode fazer se estiver tentando conseguir implantações independentes.

Conforme discutido na seção anterior, queremos pensar em nossos serviços como fatias completas de funcionalidades de negócios, as quais, quando apropriado, encapsulam a UI (User Interface, ou Interface de Usuário), a lógica de negócios e os dados. Fazemos isso porque queremos reduzir os esforços necessários para alterar as funcionalidades relacionadas aos negócios. O encapsulamento dos dados e do comportamento dessa forma possibilita ter um alto nível de coesão nas funcionalidades de negócios. Ao ocultar o banco de dados que dá suporte ao nosso serviço, também garantimos uma redução no acoplamento. A discussão sobre acoplamento e coesão será retomada no Capítulo 2.

Tamanho

“Qual deve ser o tamanho de um microserviço?” é uma das perguntas mais comuns que costumo ouvir. Considerando que o prefixo “micro” está presente no próprio nome, isso não é nenhuma surpresa. No entanto, quando chegamos aos motivos pelos quais os microserviços funcionam como um tipo de arquitetura, o conceito de tamanho, na verdade, é um dos aspectos menos interessantes.

Como avaliamos o tamanho? Contando as linhas de código? Isso não faz muito sentido para mim. Algo que poderia exigir 25 linhas de código em Java poderia ser escrito com 10 linhas de Clojure. Isso não quer dizer que Clojure seja melhor ou pior do que Java; algumas linguagens são apenas mais expressivas que outras.

James Lewis, diretor técnico da Thoughtworks, é conhecido por ter dito que “um microserviço deve ser tão grande quanto a minha cabeça.” À primeira vista, isso não parece ajudar muito. Afinal de contas, quão grande é a cabeça de James, exatamente? O raciocínio por trás dessa afirmação é que um microserviço deve ter um tamanho tal que possa ser facilmente

compreendido. O problema, é claro, é que a capacidade de diferentes pessoas entenderem algo nem sempre é igual e, desse modo, será necessário fazer seu próprio julgamento no que concerne ao tamanho que será apropriado para você. Uma equipe experiente poderia ser capaz de gerenciar melhor uma base de código maior em comparação com outra equipe. Portanto, talvez seja melhor ler a citação de James como “um microsserviço deve ser tão grande quanto a *sua* cabeça”.

Acho que o mais próximo que consigo pensar quanto ao “tamanho” ter algum sentido no que diz respeito aos microsserviços é em algo que Chris Richardson, autor de *Microservice Patterns* (Manning Publications), disse uma vez: que o objetivo dos microsserviços é ter “a menor interface possível”. Essa ideia, mais uma vez, está alinhada ao conceito de ocultar informações, mas também representa uma tentativa de dar um sentido que não estava presente inicialmente no termo “microsserviços”. Quando o termo foi usado pela primeira vez para definir essas arquiteturas, o foco, ao menos no começo, não estava particularmente no tamanho das interfaces.

Em última análise, o conceito de tamanho é extremamente contextual. Fale com uma pessoa que tenha trabalhado em um sistema por quinze anos, e ela achará que seu sistema com cem mil linhas de código é realmente fácil de entender. Peça a opinião de alguém que seja novo no projeto, e essa pessoa dirá que o sistema é grande demais. Do mesmo modo, pergunte a uma empresa que tenha acabado de iniciar sua transição para os microsserviços e possua, talvez, dez ou menos microsserviços, e você terá uma resposta diferente daquela que teria de uma empresa de porte semelhante, na qual os microsserviços sejam a norma há vários anos e que atualmente tem centenas deles.

Peço às pessoas que não se preocupem com o tamanho. No início, será muito mais importante que você se concentre em dois pontos essenciais. Em primeiro lugar, com quantos microsserviços você é capaz de lidar? À medida que tiver mais serviços, a complexidade de seu sistema aumentará e você terá de adquirir novas habilidades (e talvez adotar novas tecnologias) para lidar com eles. Uma transição para microsserviços introduzirá novas fontes de complexidade, com todos os desafios que isso

possa acarretar. É por esse motivo que sou um forte defensor de uma migração gradual para uma arquitetura de microsserviços. Em segundo lugar, como definir fronteiras para os microsserviços de modo a tirar o máximo de proveito deles, sem transformar tudo em uma enorme confusão, com alto grau de acoplamento? Ao iniciar sua jornada, será muito mais importante se concentrar nesses tópicos.

Flexibilidade

Outra citação de James Lewis é que os “microsserviços comprem opções para você”. James escolheu deliberadamente suas palavras – eles *compram* opções para você. Essas opções têm um custo, e você deverá decidir se vale a pena pagar pelas opções que quer ter. A flexibilidade resultante, presente em vários eixos – organizacional, técnico, de escala, de robustez –, pode ser extremamente atraente.

Não sabemos o que o futuro nos reserva, portanto gostaríamos de ter uma arquitetura que teoricamente possa nos ajudar a resolver quaisquer problemas que enfrentemos no futuro. Encontrar um equilíbrio entre manter as opções abertas e arcar com o custo de arquiteturas como essa pode ser uma verdadeira arte.

Pense na adoção dos microsserviços nem tanto como um interruptor a ser ligado, porém mais como um botão a ser girado. À medida que girar o botão e houver mais microsserviços, maior será a flexibilidade. Contudo, é provável que as dificuldades também aumentem. Esse é outro motivo pelo qual defendo insistentemente uma adoção gradual dos microsserviços. Ao girar o botão aos poucos, você poderá avaliar melhor o impacto à medida que o fizer, e poderá parar se for necessário.

Alinhamento entre arquitetura e organização

A MusicCorp, uma empresa de comércio eletrônico que vende CDs online, utiliza a arquitetura simples de três camadas exibida na Figura 1.2. Decidimos levar a MusicCorp com força total ao século 21 e, como parte desse processo, vamos avaliar a arquitetura do sistema existente. Temos uma web UI, uma camada de lógica de negócios na forma de um backend

monolítico, e a armazenagem de dados é feita com um banco de dados tradicional. Essas camadas, como é usual, estão sob a responsabilidade de diferentes equipes. Retornaremos às provocações e agruras da MusicCorp no decorrer do livro.

Queremos fazer uma alteração simples em nossa funcionalidade: queremos permitir que nossos clientes especifiquem seu gênero musical favorito. Essa atualização exige que modifiquemos a UI para exibir o controle para escolha do gênero, o serviço de backend para permitir que o gênero seja apresentado na UI e o valor possa ser alterado, e o banco de dados para aceitar essa mudança. Essas alterações terão de ser gerenciadas por cada equipe e implantadas na ordem correta, conforme representado na Figura 1.3.

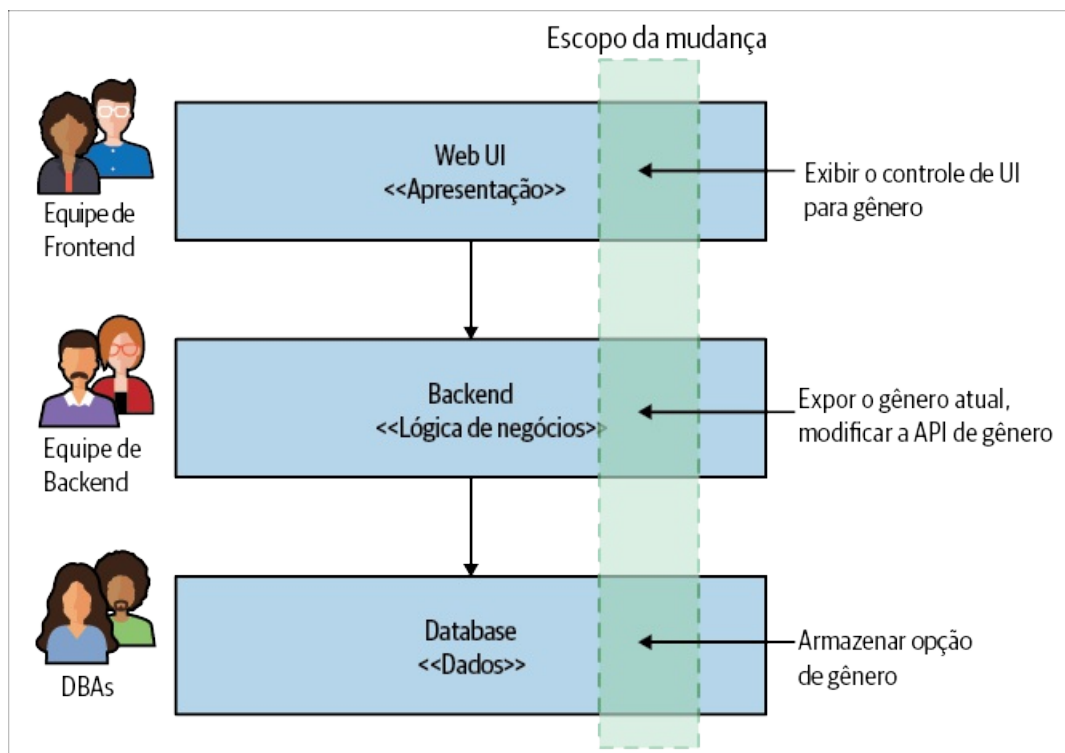


Figura 1.3 – Fazer uma alteração envolvendo todas as três camadas é mais complicado.

Essa arquitetura não é ruim. Qualquer arquitetura acaba sendo otimizada de acordo com um conjunto de objetivos. A arquitetura de três camadas é muito comum, em parte por ser universal – todo mundo já ouviu falar dela. Assim, a tendência em escolher uma arquitetura comum, que você talvez já

tenha visto em outros lugares, muitas vezes é um dos motivos para continuarmos vendo esse padrão. Contudo, acho que o principal motivo pelo qual vemos essa arquitetura repetidamente é porque ela se baseia no modo como organizamos nossas equipes.

A atualmente famosa lei de Conway estabelece o seguinte:

Organizações que projetam sistemas...estão limitadas a produzir designs que são cópias das estruturas de comunicação dessas organizações.⁴

– Melvin Conway, “How Do Committees Invent?” (<https://oreil.ly/NhE86>,
Como os comitês inventam?)

A arquitetura de três camadas é um bom exemplo dessa lei em ação. No passado, o principal modo de as empresas de TI agruparem as pessoas era com base em sua competência básica: administradores de bancos de dados compunham uma equipe com outros administradores de bancos de dados, desenvolvedores Java compunham uma equipe com outros desenvolvedores Java, enquanto desenvolvedores de frontend (que, atualmente, têm conhecimento de tópicos exóticos como JavaScript e desenvolvimento de aplicativos móveis nativos) estavam ainda em outra equipe. Agrupamos as pessoas com base em sua principal competência e, assim, criamos componentes de TI que estão alinhados com essas equipes.

Isso explica por que essa arquitetura é tão comum. Ela não é ruim; está apenas otimizada em torno de um conjunto de forças – no modo como, tradicionalmente, agrupávamos as pessoas, com base na familiaridade. No entanto, as forças mudaram. Nossas aspirações para nossos softwares mudaram. Hoje em dia, agrupamos pessoas em equipes com múltiplas habilidades a fim de reduzir a transferência de responsabilidades (handoffs) e os silos. Queremos lançar softwares com uma rapidez muito maior do que jamais fizemos. Isso nos tem levado a fazer diferentes escolhas sobre o modo de organizar nossas equipes, estruturando-as com base no modo como dividimos nossos sistemas.

A maior parte das alterações que somos solicitados a fazer em nosso sistema se relaciona com mudanças nas funcionalidades de negócios. Contudo, na Figura 1.3, nossa funcionalidade de negócios está distribuída

em todas as três camadas, aumentando as chances de uma mudança em uma funcionalidade se estender por diferentes camadas. Essa é uma arquitetura com alto nível de coesão entre tecnologias relacionadas, porém baixa coesão nas funcionalidades de negócios. Se quisermos facilitar as alterações, devemos mudar o modo como agrupamos o código, optando pela coesão nas funcionalidades de negócios, em vez de coesão nas tecnologias. Cada serviço poderá ou não acabar com uma mistura dessas três camadas, mas essa será uma preocupação local da implementação do serviço.

Vamos comparar essa arquitetura com uma possível arquitetura alternativa, exibida na Figura 1.4. Em vez de ter uma arquitetura e uma organização com camadas horizontais, dividimos nossa organização e a arquitetura em linhas de negócios verticais. Nesse caso, vemos uma equipe dedicada, com total responsabilidade sobre as alterações em aspectos do perfil do usuário, do início ao fim, garantindo que o escopo das mudanças nesse exemplo esteja limitado a uma única equipe.

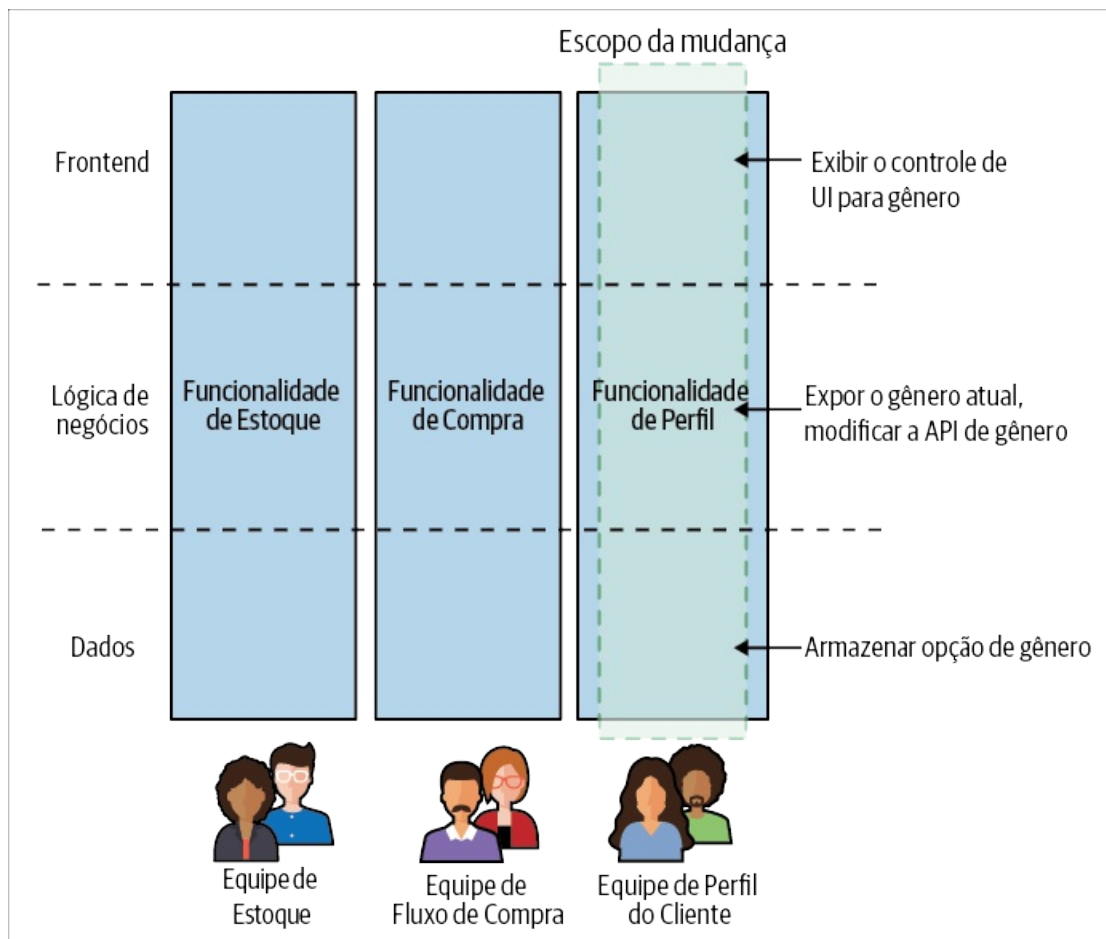


Figura 1.4 – A UI é dividida e fica sob a responsabilidade de uma equipe que também gerencia a funcionalidade que dá suporte à UI do lado do servidor.

No que concerne à implementação, ela poderia ser feita com um único microserviço sob responsabilidade da equipe de perfil; o microserviço poderia expor uma UI para permitir que os clientes atualizassem suas informações, e o estado do cliente seria armazenado também nesse microserviço. A escolha de um gênero musical favorito está associada a um determinado cliente, portanto essa alteração será muito mais localizada. Na Figura 1.5, também mostramos a lista dos gêneros musicais disponíveis sendo acessada em um microserviço *catálogo*, algo que provavelmente já deve existir. Também vemos um novo serviço *Recomendações* acessando informações sobre o nosso gênero musical favorito, algo que poderia ser facilmente incluído em uma versão subsequente.

Em uma situação como essa, nosso microserviço *cliente* encapsulará uma pequena parte de cada uma das três camadas – conterá um pouco da UI,

um pouco da lógica da aplicação e um pouco da armazenagem de dados. Nosso domínio de negócios passa a ser a principal força determinante da arquitetura de nosso sistema, e esperamos que isso facilite fazer alterações, além de simplificar o alinhamento de nossas equipes com as linhas de negócios da empresa.

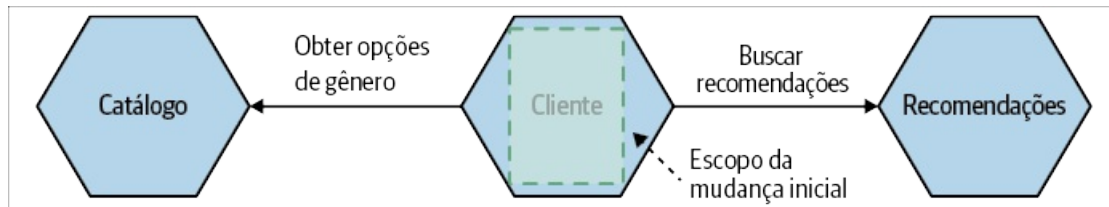


Figura 1.5 – Um serviço dedicado cliente pode facilitar bastante o armazenamento do gênero musical favorito de um cliente.

Muitas vezes, a UI não é disponibilizada diretamente pelo microserviço, mas, ainda que fosse esse o caso, esperaríamos que a parte da UI relativa a essa funcionalidade ainda estivesse sob a responsabilidade da Equipe de Perfil do Cliente, como mostra a Figura 1.4. Esse conceito de uma equipe responsável por uma fatia completa da funcionalidade voltada ao usuário tem ganhado impulso. O livro *Team Topologies*⁵ apresenta a ideia de uma equipe alinhada a fluxo (stream-aligned team), a qual inclui o seguinte conceito:

Uma equipe alinhada a fluxo (stream-aligned team) é uma equipe alinhada a um único fluxo de trabalho importante...A equipe é capacitada a desenvolver e agregar valor ao cliente ou usuário do modo mais rápido, seguro e independente possível, sem que haja transferência de responsabilidades para outras equipes fazerem partes do trabalho.⁶

As equipes mostradas na Figura 1.4 seriam equipes alinhadas a fluxo, um conceito que exploraremos com mais detalhes nos capítulos 14 e 15, incluindo como esses tipos de estruturas organizacionais funcionam na prática e como se alinham aos microserviços.

Uma nota sobre empresas “fake”

Ao longo do livro, em diferentes etapas, conheceremos a MusicCorp, a FinanceCo, a FoodCo, a AdvertCo e a PaymentCo.

FoodCo, AdvertCo e PaymentCo são empresas reais, cujos nomes

foram trocados por questões de confidencialidade. Além disso, ao compartilhar informações sobre essas empresas, de modo geral, omiti certos detalhes a fim de proporcionar mais clareza. O mundo real muitas vezes é confuso. Entretanto, sempre me esforcei para eliminar apenas os detalhes irrelevantes, que não tivessem nenhuma contribuição, porém garantindo que a realidade subjacente fosse preservada.

A MusicCorp, por outro lado, é uma empresa fake, composta de várias empresas com as quais trabalhei. As histórias que compartilho sobre a MusicCorp são reflexos de situações reais que vi, mas nem todas aconteceram com a mesma empresa!

Sistema monolítico

Já falamos dos microsserviços, mas eles são mais frequentemente discutidos como uma abordagem de arquitetura alternativa à arquitetura monolítica. Para distinguir mais claramente a arquitetura de microsserviços, além de ajudar você a entender melhor se vale a pena considerá-la, preciso discutir também o que eu quero dizer exatamente com *sistemas monolíticos*.

Quando falo de sistemas monolíticos neste livro, estou me referindo principalmente a uma unidade de implantação. Quando todas as funcionalidades de um sistema tiverem de ser implantadas em conjunto, considero que esse é um sistema monolítico. Sem dúvida, várias arquiteturas se enquadram nessa definição, mas discutirei aquelas que vejo com mais frequência: o sistema monolítico com um único processo, o sistema monolítico modular e o sistema monolítico distribuído.

Sistema monolítico com um único processo

O exemplo mais comum, que vêm à mente quando discutimos sistemas monolíticos, é um sistema no qual todo o código é implantado como um *único processo*, como mostra a Figura 1.6. Podemos ter várias instâncias desse processo por questões de robustez ou escalabilidade, mas, basicamente, todo o código está contido em um único processo. Na

verdade, esses sistemas com um único processo podem ser considerados sistemas distribuídos simples, pois quase sempre acabam lendo ou armazenando dados em um banco de dados, ou apresentando informações para aplicações web ou móveis.

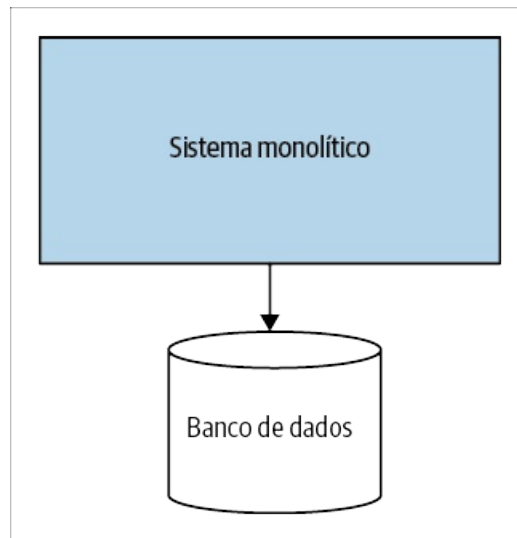


Figura 1.6 – Em um sistema monolítico com um único processo, todo o código está contido em um só processo.

Embora isso se enquadre no que a maioria das pessoas entende como um sistema monolítico clássico, a maioria dos sistemas que vejo são, de certo modo, mais complexos que esse. Podemos ter dois ou mais sistemas monolíticos altamente acoplados, possivelmente com algum software de terceiros no meio.

A implantação de um sistema monolítico clássico com um único processo pode fazer sentido para muitas empresas. David Heinemeier Hansson, criador do Ruby on Rails, argumentou de forma convincente que uma arquitetura como essa faz sentido para empresas menores.⁷ À medida que a empresa crescer, porém, o sistema monolítico poderá crescer com ela, o que nos levará a um sistema monolítico modular.

Sistema monolítico modular

Como um subconjunto dos sistemas monolíticos com um único processo, o *sistema monolítico modular* é uma variação na qual o processo único é composto de módulos separados. É possível trabalhar em cada módulo de

forma independente, porém tudo precisa ser combinado para a implantação, conforme mostra a Figura 1.7. O conceito de separar o software em módulos não é nenhuma novidade; um software modular tem suas raízes em trabalhos feitos com base na programação estruturada nos anos 1970, e mesmo anteriores a essa época. Apesar disso, essa é uma abordagem que ainda não vejo muitas empresas adotarem de forma apropriada.

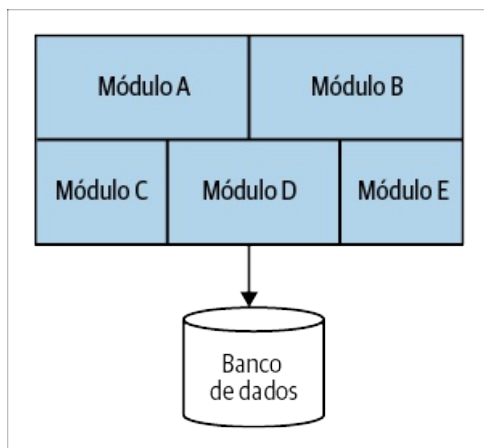


Figura 1.7 – Em um sistema monolítico modular, o código do processo é dividido em módulos.

Para muitas empresas, o sistema monolítico modular pode ser uma excelente opção. Se as fronteiras dos módulos forem bem definidas, é possível ter um grau elevado de paralelismo nos trabalhos, ao mesmo tempo que os problemas da arquitetura de microsserviços, mais distribuída, são evitados, pois há uma topologia muito mais simples para implantação. A Shopify é um ótimo exemplo de uma empresa que empregou essa técnica como uma alternativa à decomposição em microsserviços, e parece funcionar muito bem para essa empresa.⁸

Um dos desafios de um sistema monolítico modular é que o banco de dados tende a não passar por uma decomposição semelhante àquela que vemos no nível do código, resultando em problemas significativos caso você queira separar o sistema monolítico no futuro. Já vi algumas equipes tentarem levar adiante a ideia de um sistema monolítico modular, fazendo o banco de dados ser decomposto de forma similar aos módulos, como mostra a Figura 1.8.

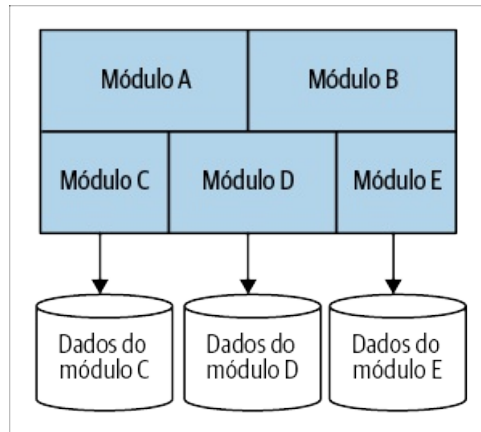


Figura 1.8 – Um sistema monolítico modular com um banco de dados decomposto.

Sistema monolítico distribuído

Um sistema distribuído é um sistema no qual a falha de um computador que você nem sabia que existia pode deixar o seu próprio computador inutilizável.²

– *Leslie Lamport*

Um *sistema monolítico distribuído* é um sistema composto de vários serviços, mas, por algum motivo, o sistema como um todo deve ser implantado em conjunto. Um sistema monolítico distribuído pode muito bem corresponder à definição de uma SOA, porém, com muita frequência, não cumpre as promessas dessa arquitetura. De acordo com minha experiência, um sistema monolítico distribuído tem todas as desvantagens de um sistema distribuído e as desvantagens de um sistema monolítico com um único processo, sem vantagens suficientes de nenhum desses sistemas. Ter visto vários sistemas monolíticos distribuídos em meu trabalho exerceu profunda influência no meu próprio interesse pela arquitetura de microsserviços.

Sistemas monolíticos distribuídos geralmente surgem em um ambiente no qual não houve muito foco em conceitos como ocultação de informações e coesão nas funcionalidades de negócios. Arquiteturas altamente acopladas fazem com que as alterações tenham reflexos para além das fronteiras dos serviços, e modificações aparentemente inocentes, que pareciam ser locais

quanto ao escopo, causam falhas em outras partes do sistema.

Sistemas monolíticos e desavença nas entregas

À medida que mais pessoas trabalham no mesmo local, elas acabam ficando no caminho umas das outras – por exemplo, diferentes desenvolvedores querendo alterar a mesma parte do código, diferentes equipes querendo lançar funcionalidades em momentos distintos (ou postergar as implantações) e confusão acerca de quem é o responsável por qual parte e quem toma decisões. Diversos estudos mostram os desafios das linhas de responsabilidade confusas.¹⁰ Chamo esse problema de *desavença nas entregas* (delivery contention).

Ter um sistema monolítico não significa que você vai certamente deparar com os desafios da desavença nas entregas, assim como ter uma arquitetura de microsserviços não significa que você jamais terá de enfrentar esse problema. No entanto, uma arquitetura de microsserviços nos dá fronteiras mais concretas quanto às linhas de responsabilidade que podem ser traçadas em um sistema, proporcionando muito mais flexibilidade para atenuar esse problema.

Vantagens dos sistemas monolíticos

Alguns sistemas monolíticos, por exemplo, os sistemas monolíticos com um único processo ou os sistemas monolíticos modulares, também têm uma série de vantagens. Sua topologia de implantação, muito mais simples, é capaz de evitar várias das armadilhas associadas aos sistemas distribuídos. Esses sistemas monolíticos podem resultar em fluxos de trabalho muito mais simples para os desenvolvedores, e atividades como monitoração, resolução de problemas e testes fim a fim podem ser bastante simplificadas também.

Os sistemas monolíticos também podem simplificar a reutilização de código dentro do próprio sistema monolítico. Se quisermos reutilizar código em um sistema distribuído, devemos decidir se queremos copiar o código, separar em bibliotecas ou passar a funcionalidade compartilhada para um serviço. Em um sistema monolítico, nossas opções serão mais

simples, e muitas pessoas gostam dessa simplicidade – todo o código está ali; basta usá-lo!

Infelizmente, as pessoas passaram a ver os sistemas monolíticos como algo a ser evitado, isto é, algo que é inerentemente problemático. Já conheci várias pessoas a quem o termo *sistema monolítico* é sinônimo de *legado*. Isso é um problema. Uma arquitetura monolítica é uma opção – e é uma opção válida. Eu poderia ir além e dizer que, em minha opinião, é a opção default sensata como estilo de arquitetura. Em outras palavras, estou procurando um motivo para ser convencido a utilizar microsserviços, em vez de procurar um motivo para não usar.

Se cairmos na armadilha de desqualificar sistematicamente o sistema monolítico como uma opção viável para entregar nossos softwares, correremos o risco de não agir corretamente, tanto conosco como para os usuários de nosso software.

Tecnologias que viabilizam os microsserviços

Conforme mencionei antes, não acho necessário adotar muitas tecnologias novas ao começar a trabalhar com microsserviços. Com efeito, isso pode ser contraproducente. Em vez disso, à medida que ampliar sua arquitetura de microsserviços, você deve observar constantemente os problemas causados pelo seu sistema cada vez mais distribuído, e então procurar a tecnologia que possa ajudar.

Apesar disso, a tecnologia tem desempenhado um papel importante na adoção dos microsserviços como um conceito. Conhecer as ferramentas disponíveis para ajudar você a tirar o máximo de proveito dessa arquitetura será uma parte essencial para fazer com que qualquer implementação de microsserviços seja um sucesso. Na verdade, eu diria até que os microsserviços exigem bastante conhecimento da tecnologia de suporte, até um nível em que distinções anteriores entre arquitetura lógica e física poderiam ser problemáticas – se você está envolvido no trabalho de ajudar a modelar uma arquitetura de microsserviços, será preciso entender muito bem esses dois mundos.

Exploraremos várias dessas tecnologias detalhadamente em capítulos subsequentes, mas, antes disso, vamos apresentar rapidamente algumas das tecnologias que tornam os microsserviços viáveis e que poderão ajudar caso você decida usá-los.

Agregação de logs e tracing distribuído

Com um número cada vez maior de processos para gerenciar, talvez seja difícil entender como seu sistema se comporta em um ambiente de produção. Por sua vez, isso pode dificultar muito mais a resolução de problemas. Exploraremos essas ideias com mais detalhes no Capítulo 10, mas, no mínimo, defendo insistentemente a implementação de um sistema de agregação de logs como um pré-requisito para a adoção de uma arquitetura de microsserviços.



Seja cauteloso na adoção de muitas tecnologias novas quando começar a trabalhar com microsserviços. Apesar disso, uma ferramenta de agregação de logs é tão importante que você deve considerá-la como um pré-requisito para a adoção dos microsserviços.

Esses sistemas permitem coletar e agregar logs de todos os seus serviços, oferecendo um local centralizado no qual os logs possam ser analisados, podendo, inclusive, fazer parte de um sistema ativo de geração de alertas. Várias opções nessa área atendem a diversas situações. Sou um grande fã do Humio (<https://www.humio.com>) por uma série de motivos, mas os serviços simples de logging oferecidos pelos principais provedores de nuvem pública podem ser suficientes para começar.

Você pode tornar essas ferramentas de agregação de logs mais úteis ainda se implementar IDs de correlação, em que um único ID é usado para um conjunto relacionado de chamadas de serviço – por exemplo, a cadeia de chamadas que pode ser acionada pela interação de um usuário. Ao registrar esse ID como parte de cada entrada de log, isolar os logs associados a um dado fluxo de chamadas será muito mais fácil, o que, por sua vez, facilitará bastante a resolução de problemas.

À medida que a complexidade de seu sistema aumentar, será essencial considerar ferramentas que permitam explorar melhor o que ele faz,

oferecendo a possibilidade de analisar traces entre vários serviços, detectar gargalos e fazer perguntas sobre seu sistema, as quais você nem sabia que gostaria de fazer, antes de tudo. Ferramentas open source podem oferecer alguns desses recursos. Um exemplo é o Jaeger (<https://www.jaegertracing.io>), cujo foco está no lado do tracing distribuído da equação.

Contudo, produtos como o Lightstep (<https://lightstep.com>) e o Honeycomb (<https://www.honeycomb.io>) – exibidos na Figura 1.9 – dão um passo além no que concerne a esses ideais. Elas representam uma nova geração de ferramentas que vão além das abordagens de monitoração tradicionais, facilitando bastante explorar o estado de seu sistema em execução. Talvez você já tenha ferramentas mais convencionais à disposição, mas deveria realmente verificar os recursos oferecidos por esses produtos. Eles foram criados desde o princípio para resolver os tipos de problemas com os quais os administradores de arquiteturas de microsserviços precisam lidar.

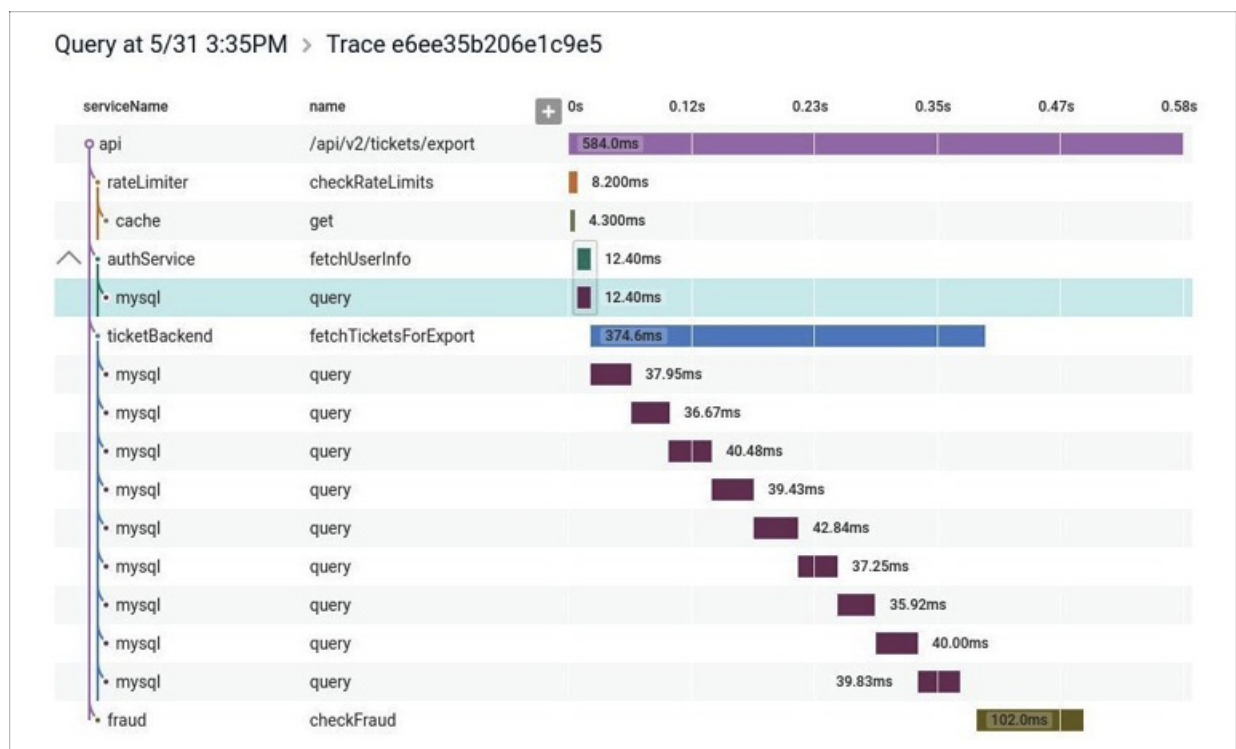


Figura 1.9 – Um trace distribuído exibido no Honeycomb, permitindo identificar os pontos em que o tempo está sendo consumido em operações que

podem abranger vários microsserviços.

Contêineres e Kubernetes

O ideal é que você execute cada instância de microsserviço isoladamente. Isso garante que problemas em um microsserviço não afetem outro microsserviço – por exemplo, consumindo toda a CPU. A virtualização é uma forma de criar ambientes de execução isolados em um hardware existente, mas técnicas usuais de virtualização podem ser muito pesadas se considerarmos o tamanho de nossos microsserviços. Os *contêineres*, por outro lado, oferecem um modo muito mais leve de possibilitar uma execução isolada para as instâncias de serviços, com mais rapidez na inicialização de novas instâncias de contêineres, além de serem mais economicamente vantajosas para diversas arquiteturas.

Depois que começar a trabalhar com contêineres, você também perceberá que precisará de algo para administrar esses contêineres distribuídos em várias máquinas subjacentes. Plataformas de orquestração de contêineres como o Kubernetes fazem exatamente isso, permitindo distribuir instâncias de contêineres de modo a ter a robustez e o throughput exigidos pelo seu serviço, ao mesmo tempo que possibilitam um uso eficiente das máquinas subjacentes. No Capítulo 8, exploraremos os conceitos de isolamento operacional, contêineres e Kubernetes.

Não se sinta pressionado a adotar rapidamente o Kubernetes, ou nem mesmo os contêineres, se for o caso. Sem dúvida, eles oferecem vantagens significativas em relação às técnicas de implantação mais tradicionais, mas sua adoção é difícil de ser justificada caso você tenha poucos microsserviços. Assim que o overhead de administrar as implantações começar a se tornar uma dor de cabeça significativa, comece a considerar a containerização de seu serviço e o uso do Kubernetes. No entanto, se acabar fazendo essa opção, esforce-se ao máximo para garantir que outra pessoa administre o cluster Kubernetes para você, talvez utilizando um serviço gerenciado em um provedor de nuvem pública. Administrar o seu próprio cluster Kubernetes pode representar um volume significativo de trabalho!

Streaming

Com os microsserviços, embora nos distanciemos dos bancos de dados monolíticos, ainda será necessário encontrar formas de compartilhar dados entre os serviços. Isso vem acontecendo ao mesmo tempo que as empresas querem se afastar das operações de relatórios em batch (lote) e se encaminham para ter um feedback mais próximo do tempo real, permitindo que elas reajam com mais rapidez. Assim, produtos que facilitam o streaming e o processamento de dados que em geral podem ser volumosos têm se tornado populares entre as pessoas que utilizam arquiteturas de microsserviços.

Para muitos, o Apache Kafka (<https://kafka.apache.org>) passou a ser a opção padrão do mercado para streaming de dados em um ambiente de microsserviços, e há um bom motivo para isso. Recursos como persistência de mensagens, compactação e a capacidade de escalar para lidar com grandes volumes de mensagens podem ser extremamente convenientes. O Kafka começou a acrescentar recursos de processamento de streams na forma do KSQLDB, mas ele também pode ser usado com soluções dedicadas de processamento de streams, como o Apache Flink (<https://flink.apache.org>). O Debezium (<https://debezium.io>) é uma ferramenta open source desenvolvida para auxiliar no streaming de dados já existentes usando o Kafka, ajudando a garantir que fontes tradicionais de dados possam fazer parte de uma arquitetura baseada em streams. No Capítulo 4, veremos como a tecnologia de streaming pode desempenhar um papel relevante na integração dos microsserviços.

Nuvem pública e produtos serverless

Os provedores de nuvem pública ou, mais especificamente, os três principais provedores – Google Cloud, Microsoft Azure e Amazon Web Services (AWS) – oferecem um conjunto enorme de serviços gerenciados e opções de implantação para administrar sua aplicação. À medida que sua arquitetura de microsserviços crescer, cada vez mais trabalho surgirá na área operacional. Os provedores de nuvem pública oferecem uma série de serviços gerenciados, variando de instâncias de banco de dados gerenciadas

a clusters Kubernetes, brokers de mensagens ou sistemas de arquivo distribuídos. Ao fazer uso desses serviços gerenciados, você transferirá boa parte desse trabalho para um terceiro que, sem dúvida, estará mais capacitado para lidar com essas tarefas.

Entre os produtos oferecidos na nuvem pública e que despertam interesse em particular estão aqueles que se apresentam sob o rótulo de *serverless* (sem servidor). Esses produtos ocultam as máquinas subjacentes, permitindo trabalhar com um nível mais elevado de abstração. Exemplos de produtos *serverless* incluem brokers de mensagens, soluções de armazenamento e bancos de dados. As plataformas FaaS (Function as a Service, ou Função como Serviço) são particularmente interessantes, pois oferecem uma boa abstração para a implantação de códigos. Em vez de se preocupar com a quantidade de servidores necessária para executar seu serviço, basta implantar o seu código e deixar que a plataforma subjacente lide com a inicialização das instâncias de seu código por demanda. Veremos os produtos *serverless* com mais detalhes no Capítulo 8.

Vantagens dos microsserviços

As vantagens proporcionadas pelos microsserviços são muitas e variadas. Muitas dessas vantagens podem ser atribuídas a qualquer sistema distribuído. Os microsserviços, porém, tendem a conseguir essas vantagens em um grau mais elevado, principalmente porque assumem uma postura mais rigorosa quanto ao modo como as fronteiras dos serviços são definidas. Ao combinar os conceitos de ocultação de informações e um design orientado a domínios com a eficácia dos sistemas distribuídos, os microsserviços podem ajudar a proporcionar ganhos significativos em comparação com outras formas de arquiteturas distribuídas.

Heterogeneidade de tecnologias

Com um sistema composto de vários microsserviços que colaboram entre si, podemos optar por empregar diferentes tecnologias em cada microsserviço. Isso permite escolher a ferramenta correta para cada tarefa, em vez de selecionarmos uma única abordagem padronizada, que sirva

para qualquer propósito, a qual muitas vezes acaba sendo o menor denominador comum.

Se uma parte de nosso sistema precisa ter o desempenho melhorado, podemos optar pelo uso de um conjunto de tecnologias diferente, mais apropriado para alcançar os níveis de desempenho exigidos. Poderíamos também decidir que o modo como armazenamos nossos dados deve mudar para diferentes partes de nosso sistema. Por exemplo, em uma rede social, poderíamos armazenar as interações de nossos usuários em um banco de dados orientado a grafos para refletir a natureza altamente conectada de um grafo social, mas talvez as postagens feitas pelos usuários pudessem ser armazenadas em um banco de dados orientado a documentos, fazendo surgir uma arquitetura heterogênea, como aquela mostrada na Figura 1.10.

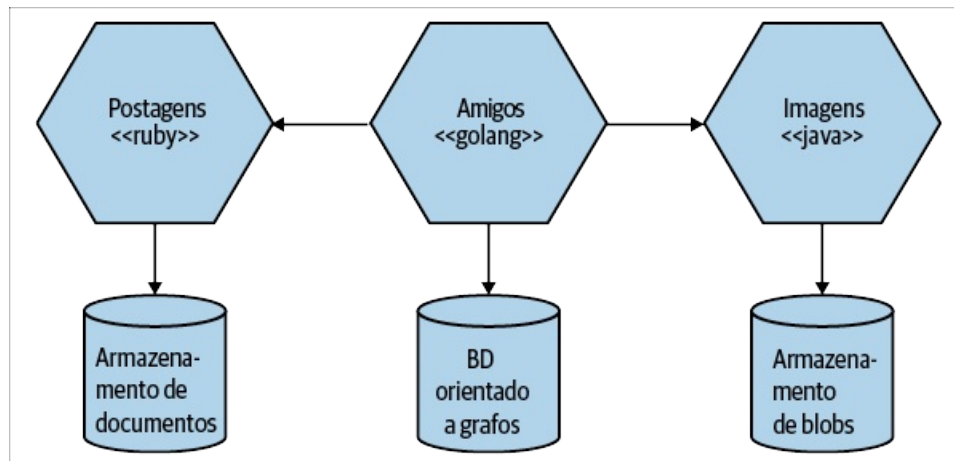


Figura 1.10 – Os microsserviços podem facilitar a adoção de diferentes tecnologias.

Com os microsserviços, também podemos adotar tecnologias com mais rapidez e entender como os novos avanços poderiam nos ajudar. Um dos maiores obstáculos para experimentar e adotar uma nova tecnologia são os riscos associados a ela. Em uma aplicação monolítica, se eu quisesse testar uma nova linguagem de programação, um banco de dados ou um framework, qualquer mudança afetaria boa parte do meu sistema. Em um sistema composto de vários serviços, há muitos lugares novos para experimentar uma nova tecnologia. Eu poderia escolher um microsserviço talvez com o risco mais baixo e empregar aí a tecnologia, sabendo que poderia limitar qualquer possível impacto negativo. Muitas empresas

consideram que essa capacidade de absorver novas tecnologias com mais rapidez é uma verdadeira vantagem.

Adotar várias tecnologias não deixa de ter um custo adicional, é claro. Algumas empresas optam por impor certas restrições nas escolhas das linguagens. A Netflix e o Twitter, por exemplo, utilizam principalmente a JVM (Java Virtual Machine, ou Máquina Virtual Java) como plataforma porque essas empresas têm um bom conhecimento da confiabilidade e do desempenho desse sistema. Elas também desenvolvem bibliotecas e ferramentas para a JVM, as quais facilitam a operação em escala; no entanto, depender de bibliotecas específicas para JVM dificulta a situação para os serviços ou clientes que não têm Java como base. No entanto, nem o Twitter nem a Netflix utilizam um único conjunto de tecnologias para todas as tarefas.

O fato de a implementação interna de tecnologias estar oculta aos consumidores também pode facilitar o upgrading das tecnologias. Toda a sua arquitetura de microsserviços poderia ser baseada no Spring Boot, por exemplo, mas você poderia mudar a versão da JVM ou do framework somente para um microsserviço, facilitando gerenciar os riscos dos upgrades.

Robustez

Um conceito essencial para melhorar a robustez de sua aplicação é o bulkhead (antepara). Um componente de um sistema pode falhar, mas, desde que essa falha não seja em cascata, o problema poderá ser isolado e o resto do sistema continuará funcionando. As fronteiras de serviços serão suas anteparas óbvias. Em um serviço monolítico, se o serviço falhar, tudo para de funcionar. Em um sistema monolítico, podemos executar esse serviço em várias máquinas para reduzir as chances de falha, mas, com os microsserviços, podemos criar sistemas que saibam lidar com a falha total de alguns de seus serviços, fazendo com que as funcionalidades se degradem de maneira proporcional.

Entretanto, precisamos ser cuidadosos. Para garantir que nossos sistemas de microsserviços possam tirar o devido proveito desse aumento na

robustez, devemos conhecer os novos motivos de falhas com os quais os sistemas distribuídos têm de lidar. As redes podem e vão falhar, assim como as máquinas. Precisamos saber como lidar com essas falhas e conhecer o impacto causado por elas (se houver) nos usuários finais de nosso software. Posso afirmar com certeza que já trabalhei com equipes que acabaram com um sistema menos robusto após terem migrado para os microsserviços por não terem considerado esses problemas com a devida seriedade.

Escalabilidade

Em um serviço grande e monolítico, é necessário escalar tudo junto. Talvez uma pequena parte do nosso sistema como um todo apresente limitações quanto ao desempenho, mas se esse comportamento estiver contido em uma aplicação monolítica gigantesca, teremos de escalar tudo junto. Com serviços menores, podemos escalar apenas os serviços que precisam ser escalados, permitindo que outras partes do sistema sejam executadas em um hardware menor, menos potente, conforme mostra a Figura 1.11.

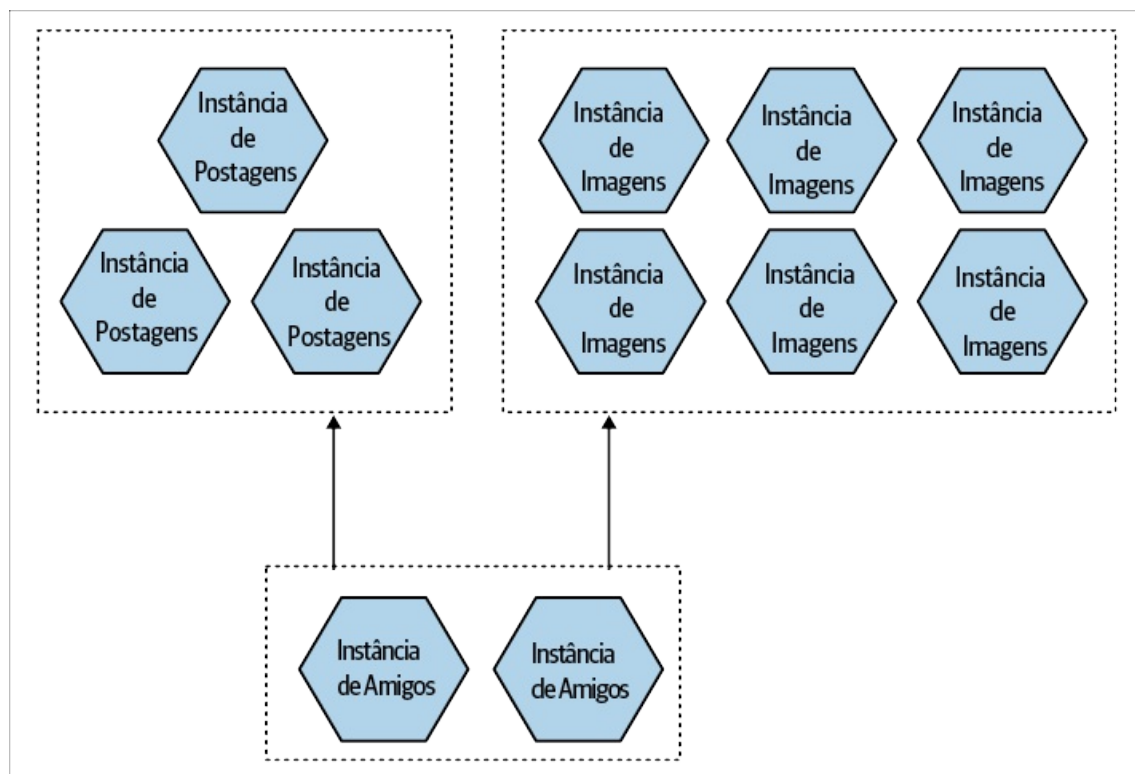


Figura 1.11 – Podemos escalar visando somente aos microsserviços que

precisam ser escalados.

A Gilt, uma loja de vestuários online, adotou os microsserviços exatamente por esse motivo. Tendo começado em 2007 com uma aplicação Rails monolítica, por volta de 2009, o sistema da Gilt era incapaz de suportar a carga imposta à aplicação. Ao separar as partes principais de seu sistema, a Gilt conseguiu lidar melhor com os picos de tráfego e, atualmente, possui mais de 450 microsserviços, cada um executando em várias máquinas distintas.

Ao adotar os sistemas de provisionamento por demanda, como aqueles oferecidos pela AWS, podemos inclusive aplicar essa escalabilidade por demanda nas partes em que ela for necessária. Isso permite controlar nossos custos com mais eficácia. Poder correlacionar tão de perto uma abordagem de arquitetura com uma economia quase imediata não é algo que ocorra com muita frequência.

Em última análise, podemos escalar nossas aplicações de várias maneiras, e os microsserviços podem ser uma parte eficaz nesse processo. Discutiremos a escalabilidade dos microsserviços com mais detalhes no Capítulo 13.

Facilidade de implantação

Uma mudança de uma linha em uma aplicação monolítica de um milhão de linhas exige que toda a aplicação seja implantada para que a alteração seja disponibilizada. Essa implantação poderia ter alto impacto e risco. Na prática, implantações como essas acabam ocorrendo raramente em virtude de um temor compreensível. Infelizmente, isso significa que nossas alterações continuarão se acumulando entre as versões, até que a nova versão de nossa aplicação, a ser colocada no ambiente de produção, tenha um grande volume de alterações. E, quanto maior a diferença entre as versões, maior será o risco de haver algo errado!

Com os microsserviços, é possível fazer uma alteração em um único serviço e implantá-lo independentemente do resto do sistema. Isso permite que nosso código seja implantado com mais rapidez. Se houver algum problema, ele poderá ser prontamente isolado em um serviço específico, facilitando efetuar um rollback rapidamente. Também significa que

podemos entregar a nova funcionalidade aos nossos clientes com mais rapidez. Esse é um dos principais motivos pelos quais empresas como a Amazon e a Netflix utilizam essas arquiteturas – para garantir que o máximo possível dos obstáculos para o lançamento de um software seja eliminado.

Alinhamento organizacional

Muitos de nós já vivenciaram os problemas associados a equipes e bases de código grandes. Esses problemas podem ser exacerbados quando a equipe está distribuída. Também sabemos que equipes menores trabalhando com bases de código menores tendem a ser mais produtivas.

Os microsserviços permitem alinhar melhor a nossa arquitetura com a nossa organização, ajudando a reduzir o número de pessoas que trabalham em qualquer base de código específica, de modo a alcançar a relação ideal entre tamanho de equipe e produtividade. Os microsserviços também possibilitam que os responsáveis pelos microsserviços mudem à medida que a organização mudar – isso permite manter o alinhamento entre a arquitetura e a organização da empresa no futuro.

Composição

Uma das principais promessas dos sistemas distribuídos e das arquiteturas orientadas a serviços é criar oportunidades para a reutilização de funcionalidades. Com os microsserviços, permitimos que nossas funcionalidades sejam consumidas de diferentes maneiras, para diferentes propósitos. Esse aspecto pode ser particularmente importante quando pensamos no modo como nossos consumidores utilizam nosso software.

Foi-se o tempo em que podíamos ter um pensamento limitado sobre o nosso site desktop ou nosso aplicativo móvel. Hoje em dia, precisamos pensar na infinidade de maneiras de podermos interconectar os recursos para web, aplicações nativas, web móvel, aplicações para tablet ou dispositivos wearable (vestíveis). À medida que as empresas deixam de pensar com base em canais limitados e passam a adotar conceitos mais holísticos para o envolvimento dos clientes, precisamos de arquiteturas que

possam acompanharem esse processo.

Com os microsserviços, pense que criaremos aberturas em nosso sistema que poderão ser utilizadas por terceiros. Quando as circunstâncias mudarem, será possível criar aplicações de diferentes maneiras. Com uma aplicação monolítica, com frequência, temos uma grande abertura genérica que pode ser usada por um terceiro. Se precisar dividi-la para ter algo que seja mais conveniente, eu teria de usar um martelo!

Complexidades dos microsserviços

As arquiteturas de microsserviços proporcionam uma série de vantagens, conforme já vimos. Contudo, elas também trazem consigo diversas complexidades. Se você está considerando adotar uma arquitetura de microsserviços, é importante que possa comparar os pontos bons e os pontos ruins. Na verdade, a maioria das complexidades associadas aos microsserviços pode ser atribuída aos sistemas distribuídos e, desse modo, estariam evidentes em um sistema monolítico distribuído, assim como em uma arquitetura de microsserviços.

Discutiremos muitas dessas questões em detalhes na parte restante do livro – com efeito, eu diria que a maior parte deste livro diz respeito a lidar com a dor, o sofrimento e o horror de possuir uma arquitetura de microsserviços.

Experiência dos desenvolvedores

À medida que houver mais e mais serviços, a experiência dos desenvolvedores poderá começar a sofrer as consequências. Runtimes que exijam mais recursos, como a JVM, podem limitar o número de microsserviços possíveis de serem executados na máquina de um único desenvolvedor. É provável que eu possa executar quatro ou cinco microsserviços baseados em JVM como processos separados em meu notebook, mas seria possível executar dez ou vinte? Provavelmente não. Mesmo com runtimes menos exigentes, há um limite para a quantidade de componentes que podem ser executados localmente, o que, inevitavelmente, dará início a discussões sobre o que fazer quando não for

mais possível executar todo o sistema em uma só máquina. A situação pode se complicar mais ainda se você estiver usando serviços de nuvem que não podem ser executados localmente.

Soluções radicais podem envolver um “desenvolvimento na nuvem”, quando os desenvolvedores não poderão mais desenvolver localmente. Não sou fã dessa solução, pois os ciclos de feedback podem sofrer um impacto significativo. Em vez disso, acho que limitar o escopo das partes de um sistema em que um desenvolvedor precisa trabalhar provavelmente seria uma abordagem muito mais simples. No entanto, poderia ser problemático caso você quisesse adotar um modelo mais próximo de uma “responsabilidade coletiva”, na qual se espera que qualquer desenvolvedor possa trabalhar em qualquer parte do sistema.

Sobrecarga tecnológica

O próprio peso das novas tecnologias que surgiram para permitir a adoção das arquiteturas de microsserviços pode ser opressivo. Vou ser franco e dizer que muitas dessas tecnologias foram simplesmente rebatizadas como “amigáveis aos microsserviços”, mas, de fato, alguns avanços ajudaram a lidar com a complexidade desses tipos de arquiteturas. Contudo, existe o perigo de que essa variedade de novos brinquedos possa levar a uma espécie de fetichismo tecnológico. Já vi várias empresas adotando arquiteturas de microsserviços e que decidiram que também era a hora certa para introduzir uma série de tecnologias novas e muitas vezes desconhecidas.

Os microsserviços podem muito bem dar a você a *opção* de escrever cada microsserviço em uma linguagem de programação diferente, ser executado em um runtime diferente ou utilizar um banco de dados diferente – mas são opções, e não requisitos. É necessário balancear cuidadosamente a abrangência e a complexidade da tecnologia usada em relação aos custos que um conjunto diversificado de tecnologias pode acarretar.

Ao começar a adotar microsserviços, alguns desafios fundamentais serão inevitáveis: você terá de dedicar bastante tempo para entender problemas relacionados a consistência de dados, latência, modelagem de serviços, e

tópicos semelhantes. Se estiver tentando entender como essas ideias mudam o modo como você pensa no desenvolvimento de software, ao mesmo tempo em que adota uma variedade enorme de novas tecnologias, você terá muitas dificuldades. Também vale a pena enfatizar que os recursos consumidos para tentar entender todas essas novas tecnologias reduzirão o tempo que você tem disponível para realmente lançar novas funcionalidades aos seus usuários.

À medida que você aumentar (gradualmente) a complexidade de sua arquitetura de microsserviços, procure introduzir novas tecnologias conforme forem necessárias. Não será necessário ter um cluster Kubernetes se houver apenas três serviços! Além de garantir que você não fique sobrecarregado com a complexidade dessas novas ferramentas, esse crescimento gradual tem a vantagem adicional de permitir que você descubra maneiras novas e melhores de fazer outras tarefas que, sem dúvida, surgirão com o tempo.

Custos

É muito provável que, no curto prazo pelo menos, você verá um aumento nos custos por uma série de fatores. Em primeiro lugar, você provavelmente terá mais itens para executar – mais processos, mais computadores, mais rede, mais armazenamento e mais softwares auxiliares (que implicarão taxas adicionais de licença).

Em segundo lugar, qualquer mudança feita em uma equipe ou organização causará mais lentidão no curto prazo. Conhecer novas ideias e descobrir como usá-las com eficácia exige tempo. Enquanto esse processo estiver em andamento, haverá impacto em outras atividades. O resultado será uma maior lentidão na entrega de novas funcionalidades, ou a necessidade de acrescentar mais pessoas para compensar esse custo.

Com base em minha experiência, posso dizer que os microsserviços são uma opção ruim para uma empresa cuja preocupação principal seja reduzir custos, pois uma mentalidade voltada à redução de custos – na qual o TI é visto como um centro de custos, e não de lucros – será constantemente um obstáculo para tirar o máximo de proveito dessa arquitetura. Por outro

lado, os microsserviços podem ajudar você a ganhar mais dinheiro se essas arquiteturas forem empregadas para alcançar mais clientes ou desenvolver mais funcionalidades em paralelo. Então, os microsserviços são uma forma de garantir mais lucros? Talvez. Os microsserviços são uma maneira de reduzir custos? Nem tanto.

Relatórios

Em um sistema monolítico, em geral haverá um banco de dados monolítico. Isso significa que os stakeholders (pessoas-chaves) que querem analisar todos os dados em conjunto, muitas vezes com o envolvimento de grandes operações de junção entre os dados, terão um esquema pronto com base no qual gerarão seus relatórios. Eles podem gerá-los diretamente a partir do banco de dados monolítico, talvez em uma réplica para leitura, conforme mostra a Figura 1.12.

Em uma arquitetura de microsserviços, esse esquema monolítico estará separado. Isso não significa que a necessidade de ter relatórios envolvendo todos os nossos dados tenha deixado de existir; nós apenas dificultamos o processo, pois agora nossos dados estarão espalhados em vários esquemas isolados do ponto de vista lógico.

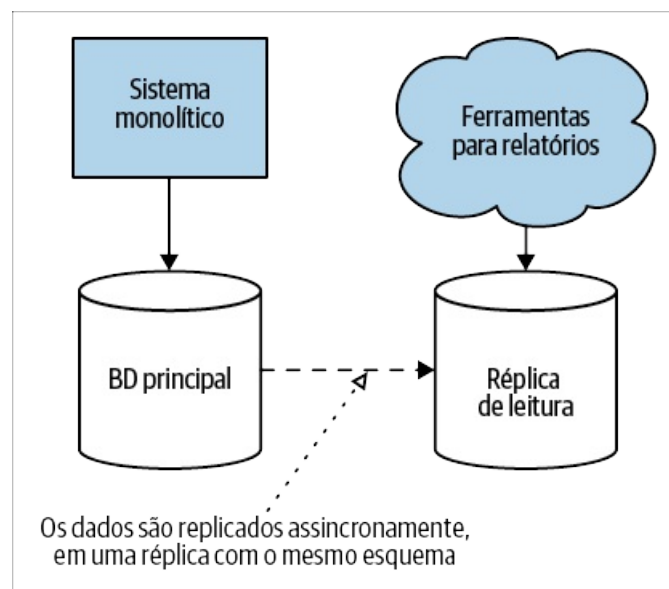


Figura 1.12 – Relatórios gerados diretamente no banco de dados de um sistema monolítico.

Abordagens mais modernas aos relatórios – por exemplo, com o uso de streaming para permitir relatórios com grandes volumes de dados em tempo real – podem funcionar bem em uma arquitetura de microsserviços, porém, em geral, exigem a adoção de novas ideias e tecnologias associadas. Como alternativa, você poderia simplesmente publicar os dados de seus microsserviços em um banco de dados centralizado para relatórios (ou, talvez, em data lakes menos estruturados) para possibilitar os casos de uso de relatórios.

Monitoração e resolução de problemas

Em uma aplicação monolítica padrão, podemos adotar uma abordagem razoavelmente simplista para a monitoração. Há poucas máquinas com que se preocupar, e o modo de falha da aplicação é, de certo modo, binário: em geral, a aplicação estará totalmente ativa ou totalmente inativa. Em uma arquitetura de microsserviços, saberemos qual é o impacto se apenas uma única instância de um serviço ficar inativa?

Em um sistema monolítico, se nossa CPU estiver com 100% de uso constante por um longo período, sabemos que esse é um grande problema. Em uma arquitetura de microsserviços com dezenas ou centenas de processos, podemos dizer o mesmo? Será necessário acordar alguém às três horas da manhã se houver apenas um processo com 100% de consumo constante de CPU?

Felizmente há muitas ideias nessa área que podem ajudar. Se quiser explorar esse conceito com mais detalhes, recomendo o livro *Distributed Systems Observability* de Cindy Sridharan (O'Reilly) como um excelente ponto de partida, embora tenhamos nossa própria discussão sobre monitoração e observabilidade no Capítulo 10.

Segurança

Em um sistema monolítico com um único processo, boa parte de nossas informações fluía dentro desse processo. Agora, há mais informações fluindo entre os nossos serviços, através de redes. Esse fato pode deixar nossos dados mais vulneráveis a serem observados em trânsito, além de

poderem ser manipulados como parte de ataques do tipo man-in-the-middle. Isso significa que você talvez precise dedicar mais atenção a proteger os dados em trânsito e garantir que os endpoints de seus microsserviços estejam protegidos, de modo que somente partes autorizadas possam usá-los. O Capítulo 11 é totalmente dedicado a discutir os desafios nessa área.

Testes

Com qualquer tipo de testes funcionais automatizados, há sempre uma decisão delicada a ser tomada. Quanto mais funcionalidades um teste executar – isto é, quanto maior o escopo do teste –, mais confiança você terá em sua aplicação. Por outro lado, quanto maior o escopo do teste, mais difícil será configurar os dados de teste e os fixtures auxiliares, mais tempo o teste poderá demorar para executar e mais difícil poderá ser descobrir o problema quando houver uma falha. No Capítulo 9, apresentarei diversas técnicas para fazer com que os testes funcionem nesse ambiente mais desafiador.

Testes fim a fim (end-to-end tests) para qualquer tipo de sistema estão na extremidade do espectro no que concerne às funcionalidades cobertas, e estamos acostumados com o fato de serem mais problemáticos para escrever e manter, em comparação com os testes de unidade, de escopo mais reduzido. Em geral, porém, vale a pena tê-los, pois queremos ter a confiança proporcionada pela existência de um teste fim a fim que utilize nossos sistemas do mesmo modo que um usuário o faria.

No entanto, em uma arquitetura de microsserviços, o escopo de nossos testes fim a fim se torna *muito* amplo. Teríamos agora de executar testes envolvendo vários serviços, e todos teriam de estar implantados e configurados de forma apropriada para os cenários de teste. Também teremos de estar preparados para os falso-negativos que ocorrerem quando problemas de ambiente, por exemplo, instâncias de serviço que morrem ou timeouts de rede por causa de implantações com falha, fizerem nossos testes falharem.

Esses fatores implicam que, à medida que sua arquitetura de

microserviços crescer, você terá um retorno cada vez menor do investimento no que diz respeito aos testes fim a fim. Os testes terão um custo maior, mas não conseguirão proporcionar a você o mesmo nível de confiança que conferiam no passado. Isso levará você a outras formas de testes, por exemplo, testes orientados a contratos (contract-driven testing) ou testes em produção, assim como à exploração de técnicas de entregas progressivas como execuções paralelas (parallel run) ou lançamentos canário (canary releases), que serão discutidas no Capítulo 8.

Latência

Em uma arquitetura de microserviços, processamentos que antes poderiam ser feitos localmente em um único processador agora poderão ficar separados em vários microserviços distintos. Informações que antes fluíam dentro de um único processo agora precisam ser serializadas, transmitidas e desserializadas através de redes, e isso terá de ser colocado em prática com muito mais frequência. Tudo isso pode resultar em uma latência pior em seu sistema.

Embora possa ser difícil avaliar o impacto exato da latência das operações na fase de design ou de programação, esse é outro motivo pelo qual é importante conduzir qualquer migração para microserviços de forma gradual. Faça uma pequena alteração, e então avalie o impacto. Estamos pressupondo que você dispõe de alguma maneira de avaliar a latência fim a fim das operações que interessem a você – ferramentas de tracing distribuído como o Jaeger podem ajudar nesse caso. No entanto, você terá de saber também qual é a latência aceitável para essas operações. Às vezes, deixar uma operação mais lenta será perfeitamente aceitável, desde que ela ainda seja rápida o suficiente!

Consistência dos dados

Passar de um sistema monolítico, no qual os dados são armazenados e gerenciados em um único banco de dados, para um sistema muito mais distribuído, em que vários processos gerenciam estados em diferentes bancos de dados, implica possíveis desafios com respeito à consistência dos dados. Enquanto, no passado, talvez você contasse com as transações no

banco de dados para gerenciar as mudanças de estado, é preciso entender que um nível similar de segurança não será facilmente oferecido em um sistema distribuído. O uso de transações distribuídas na maioria dos casos prova ser extremamente problemático para coordenar mudanças de estado. Como alternativa, talvez você precise começar a utilizar conceitos como sagas (algo que vou apresentar detalhadamente no Capítulo 6) e consistência eventual (eventual consistency) para pensar nos estados de seu sistema e gerenciá-los. Essas ideias podem exigir mudanças fundamentais no modo como você pensa nos dados de seus sistemas, algo que pode ser bastante assustador ao fazer a migração de sistemas existentes. Mais uma vez, esse é outro bom motivo para ser cauteloso com a pressa com que você vai decompor a sua aplicação. Adotar uma abordagem gradual para a decomposição, de modo que seja possível avaliar o impacto das mudanças de sua arquitetura em produção, é realmente importante.

Devo usar microsserviços?

Apesar de algumas tendências de fazer das arquiteturas de microsserviços uma abordagem padrão para softwares, acho que, em virtude dos vários desafios que apresentei, adotá-las ainda exige uma avaliação cuidadosa. É necessário avaliar seus próprios problemas, as habilidades e o panorama de tecnologias, e saber o que você está tentando alcançar, antes de decidir se os microsserviços são a opção certa para você. Elas são *uma* abordagem de arquitetura, e não *a* abordagem de arquitetura. Seu próprio contexto deve desempenhar um papel muito importante na decisão de escolher esse caminho.

Gostaria então de apresentar algumas situações que, em geral, servem de pista para eu me afastar – ou me aproximar – dos microsserviços.

A quem os microsserviços talvez não sejam apropriados

Considerando a importância de definir fronteiras de serviço estáveis, acho que as arquiteturas de microsserviços muitas vezes são uma opção ruim para produtos ou startups extremamente novos. Em qualquer um dos casos, o domínio com o qual você trabalhará geralmente estará sujeito a

mudanças significativas, à medida que você iterar pelas bases daquilo que está tentando criar. Essa mudança nos modelos de domínio, por sua vez, resultará em mais mudanças nas fronteiras dos serviços, e coordenar mudanças em fronteiras de serviços é um empreendimento custoso. Em geral, acho que é mais apropriado esperar até que uma parte suficiente do modelo do domínio tenha se estabilizado, antes de tentar definir fronteiras para os serviços.

Vejo que as startups ficam tentadas a utilizar microsserviços desde o princípio, com base no seguinte raciocínio: “Se realmente tivermos sucesso, precisaremos escalar!”. O problema é que você nem mesmo sabe exatamente se alguém vai querer usar o seu novo produto. E mesmo que tenha sucesso a ponto de precisar de uma arquitetura altamente escalável, o que você acabará entregando para os seus usuários talvez seja muito diferente daquilo que você começou a criar, antes de tudo. O foco inicial do Uber eram as limusines, e o Flickr surgiu de tentativas de criar um jogo online para vários jogadores. O processo de encontrar produtos adequados ao mercado implica que você pode acabar com um produto muito diferente no final, em comparação com o produto que achou que criaria quando começou.

As startups em geral também têm menos pessoas disponíveis para desenvolver o sistema, o que resulta em mais desafios com respeito aos microsserviços. Os microsserviços trazem consigo novos trabalhos e mais complexidade, e isso pode comprometer recursos valiosos. Quanto menor a equipe, mais elevado será esse custo. Ao trabalhar com equipes menores, com apenas um punhado de desenvolvedores, hesito muito em sugerir o uso de microsserviços por esse motivo.

Para as startups, o desafio representado pelos microsserviços será agravado pelo fato de, geralmente, a maior limitação delas serem as pessoas. Para uma equipe pequena, uma arquitetura de microsserviços pode ser difícil de ser justificada porque há trabalhos necessários somente para lidar com a implantação e o gerenciamento dos próprios microsserviços. Algumas pessoas descrevem isso como a “taxa dos microsserviços”. Se esse investimento beneficia muitas pessoas, será mais fácil justificá-lo. Contudo,

se uma pessoa em uma equipe de cinco estiver gastando seu tempo com essas questões, haverá muito tempo valioso que não será dedicado à criação de seu produto. Será mais fácil migrar para microsserviços mais tarde, depois que você souber onde estão as limitações e as complicações em sua arquitetura – assim, será possível concentrar suas energias em utilizar os microsserviços nos pontos mais sensíveis.

Por fim, as empresas que criam softwares que serão implantados e gerenciados pelos seus clientes poderão ter dificuldades com os microsserviços. Conforme já foi discutido, uma arquitetura de microsserviços pode agregar muitas complexidades na implantação e no domínio operacional. Se você mesmo administra o software, será possível superar essas novas complexidades adotando novas tecnologias, desenvolvendo novas habilidades e mudando as práticas de trabalho. Isso não é algo que você possa esperar que seus clientes façam. Se estiverem acostumados a receber o seu software na forma de um instalador Windows, seus clientes ficarão extremamente chocados se você lhes enviar a próxima versão de seu software e disser: “Basta colocar esses 20 pods em seu cluster Kubernetes!”. É bem provável que eles não tenham a mínima ideia do que seja um pod, o Kubernetes ou nem mesmo um cluster.

Quando os microsserviços funcionam bem

Com base em minha experiência, posso dizer que, provavelmente, a principal razão para as empresas adotarem os microsserviços é permitir que mais desenvolvedores trabalhem no mesmo sistema, sem ficarem no caminho uns dos outros. Defina sua arquitetura e as fronteiras organizacionais corretamente, e você permitirá que mais pessoas trabalhem de forma independente, reduzindo a desavença nas entregas. Uma startup com cinco pessoas provavelmente achará que uma arquitetura de microsserviços é um peso. Uma empresa em rápido crescimento, com uma centena de pessoas, provavelmente achará que seu crescimento será muito mais fácil se ela tiver uma arquitetura de microsserviços devidamente alinhada com seus esforços de desenvolvimento de produtos.

Aplicações SaaS (Software as a Service, ou Software como Serviço), de

modo geral, também são muito apropriadas para uma arquitetura de microsserviços. Geralmente, espera-se que esses produtos funcionem por 24 horas, 7 dias por semana, o que representa um desafio quando se trata do rollout de alterações. A possibilidade de implantações independentes nas arquiteturas de microsserviços é uma grande vantagem nessa área. Além do mais, os microsserviços podem ser redimensionados para cima ou para baixo, conforme necessário. Isso significa que, ao definir uma linha de base razoável para as características de carga de seu sistema, você terá mais controle para garantir que poderá escalá-lo com o melhor custo possível.

A natureza independente de tecnologia dos microsserviços garante que seja possível tirar o máximo de proveito possível das plataformas de nuvem. Provedores de nuvem pública oferecem uma ampla variedade de serviços e sistemas de implantação para o seu código. Você poderá atender com mais facilidade aos requisitos de serviços específicos com os serviços de nuvem que mais ajudem a implementá-los. Por exemplo, você pode decidir implantar um serviço como um conjunto de funções, outro como uma VM (Virtual Machine, ou Máquina Virtual) gerenciada e outro em uma plataforma PaaS (Platform as a Service, ou Plataforma como Serviço) gerenciada.

Embora valha a pena observar que adotar diversas tecnologias muitas vezes pode ser um problema, ser capaz de experimentar novas tecnologias com facilidade é uma ótima maneira de, rapidamente, identificar novas abordagens que possam trazer benefícios. A crescente popularidade das plataformas FaaS é um exemplo desse tipo. Para as cargas de trabalho apropriadas, uma plataforma FaaS pode reduzir drasticamente o volume de overhead operacional, mas, atualmente, não é um método de implantação apropriado para todos os casos.

Os microsserviços também proporcionam vantagens claras para as empresas que buscam oferecer serviços aos seus clientes por meio de diversos canais novos. Muitos esforços de transformação digital parecem envolver a tentativa de trazer à tona funcionalidades ocultas em sistemas existentes. O desejo das empresas é criar outras experiências para os clientes com o intuito de atender às necessidades dos usuários por meio de

qualquer método de interação que faça mais sentido.

Acima de tudo, uma arquitetura de microsserviços é uma arquitetura que provê muita flexibilidade à medida que seu sistema continua evoluindo. Essa flexibilidade tem um custo, é claro, mas se quiser manter suas opções abertas no que concerne a mudanças que você possa querer fazer no futuro, esse seria um preço que valeria a pena pagar.

Resumo

As arquiteturas de microsserviços podem proporcionar muita flexibilidade para escolher tecnologias, lidar com robustez e escalabilidade, organizar equipes, e outras tarefas. Essa flexibilidade é, parcialmente, o motivo pelo qual muitas pessoas estão adotando as arquiteturas de microsserviços. No entanto, os microsserviços carregam consigo um grau significativo de complexidade, e você precisa garantir que essa complexidade se justifique. Para muitas pessoas, essas arquiteturas se tornaram a opção padrão, a ser usada praticamente em qualquer situação. Entretanto, ainda acho que elas são uma opção de arquitetura, cujo uso deve ser justificado pelos problemas que você estiver tentando resolver; muitas vezes, abordagens mais simples podem funcionar mais facilmente.

Apesar disso, muitas empresas, particularmente as maiores, têm demonstrado a eficácia dos microsserviços. Quando os conceitos fundamentais dos microsserviços são compreendidos e implementados de forma adequada, eles podem ajudar a criar arquiteturas eficazes e produtivas, contribuindo para que os sistemas se tornem mais do que a soma de suas partes.

Espero que este capítulo tenha servido como uma boa introdução a esses tópicos. A seguir, veremos como definir as fronteiras dos microsserviços, explorando assuntos como programação estruturada e design orientado a domínios durante o processo.

¹ Esse conceito foi apresentado inicialmente por David Parnas em "Information Distribution Aspects of Design Methodology" (<https://oreil.ly/rDPWA>, Aspectos da distribuição de informações na metodologia de design), *Information Processing: Proceedings of the IFIP Congress 1971* (Processamento de informações: anais do congresso IFIP de 1971) (Amsterdã: Holanda do Norte,

1972), 1:339–44.

- 2 Alistair Cockburn, “Hexagonal Architecture” (Arquitetura Hexagonal), 4 de janeiro de 2005, <https://oreil.ly/NfvTP>.
- 3 Para uma introdução mais detalhada sobre o design orientado a domínios, consulte o livro *Domain-Driven Design* de Eric Evans (Addison-Wesley) [N.T.: Edição publicada no Brasil: *Domain-Driven Design* (Alta Books, 2016)] – ou, se quiser uma versão mais resumida, o livro *Domain-Driven Design Distilled* de Vaughn Vernon (Addison-Wesley).
- 4 N.T.: Tradução livre com base na citação original em inglês.
- 5 Matthew Skelton e Manuel Pais, *Team Topologies* (Portland, OR: IT Revolution, 2019).
- 6 N.T.: Tradução livre com base na citação original em inglês.
- 7 David Heinemeier Hansson, “The Majestic Monolith” (O majestoso sistema monolítico), Signal v. Noise, 29 de fevereiro de 2016, <https://oreil.ly/WwG1C>.
- 8 Para alguns insights interessantes sobre a ideia da Shopify de utilizar um sistema monolítico modular em vez de microsserviços, assista ao vídeo “Deconstructing the Monolith” (<https://oreil.ly/Rpi1e>, Desconstruindo o sistema monolítico) de Kirsten Westeinde.
- 9 Leslie Lamport, mensagem de email (<https://oreil.ly/2nHF1>) enviada para um bulletin board DEC SRC às 12:23:29 PDT em 28 de maio de 1987. [N.T.: Tradução livre da citação, com base no original em inglês.]
- 10 A Microsoft Research conduziu estudos nessa área, e recomendo todos, mas, como ponto de partida, sugiro “Don’t Touch My Code! Examining the Effects of Ownership on Software Quality” (<https://oreil.ly/0ahXX>, Não mexa no meu código! Analisando os efeitos da responsabilidade na qualidade do software) de Christian Bird et al.

Como modelar microsserviços

O raciocínio do meu oponente me faz recordar do pagão que, ao ser questionado sobre onde está o mundo, respondeu: “Sobre uma tartaruga”. Mas onde está a tartaruga? “Sobre outra tartaruga.”

– *Rev. Joseph Frederick Berg (1854)*

Então você já sabe o que são microsserviços e, espero, tem uma noção de suas principais vantagens. Você provavelmente deve estar ansioso para sair e começar a criá-los, certo? Mas por onde começar? Neste capítulo, veremos alguns conceitos fundamentais, como ocultação de informações, acoplamento e coesão, e entenderemos como eles mudarão nosso pensamento sobre definição de fronteiras em torno de nossos microsserviços. Também veremos diferentes formas de decomposição que podem ser usadas, além de manter um foco mais específico no design orientado a domínios como uma técnica extremamente útil nessa área.

Veremos como pensar nas fronteiras dos nossos microsserviços de modo a maximizar as vantagens e evitar algumas das possíveis desvantagens. Antes, porém, precisamos ter algo com o qual trabalhar.

Apresentando a MusicCorp

Livros sobre ideias funcionam melhor com exemplos. Quando possível, compartilharei histórias de situações reais, mas achei que também seria conveniente ter um cenário fictício com o qual trabalhar. Ao longo do livro, retornaremos a esse cenário, observando como o conceito de microsserviços funciona nesse mundo.

Vamos então voltar nossa atenção para a MusicCorp, uma loja de varejo online de vanguarda. A MusicCorp, até recentemente, era apenas uma loja física, mas após a acentuada queda nos negócios de discos de vinil, ela

passou a concentrar seus esforços cada vez mais no mercado online. A empresa tem um site, mas acha que agora é o momento de investir com tudo no mundo online. Afinal de contas, aqueles smartphones para música são apenas uma moda passageira (os Zunes são muito melhores, obviamente), e os fãs de música estão muito satisfeitos em esperar que os CDs cheguem até suas portas. Qualidade acima da conveniência, certo? E, embora a empresa tenha acabado de descobrir que o Spotify é, na verdade, um serviço digital de músicas, e não algum tipo de tratamento de pele para adolescentes, a MusicCorp está muito satisfeita com seu próprio foco nos negócios e está certa de que todo esse negócio de streaming acabará logo.

Apesar de estar um pouco atrasada, a MusicCorp tem grandes ambições. Felizmente ela decidiu que sua melhor chance de conquistar o mundo é garantir que será capaz de fazer mudanças do modo mais fácil possível. Microsserviços para vencer!

O que é uma boa fronteira para um microsserviço?

Antes que a equipe da MusicCorp saia correndo por aí criando serviço após serviço, em uma tentativa de entregar fitas de oito faixas musicais para todos os lados, vamos pôr o pé no freio e falar um pouco da ideia básica mais importante que devemos ter em mente. Queremos que nossos microsserviços possam ser alterados e implantados, e suas funcionalidades sejam lançadas aos nossos usuários de modo independente. A capacidade de alterar um microsserviço isoladamente é essencial. Então, o que devemos ter em mente quando pensamos em definir fronteiras em torno dos microsserviços?

Em sua essência, os microsserviços são apenas outra forma de decomposição modular, embora seja uma com interações baseadas em rede entre os modelos, com todos os desafios associados que essa solução traz. Felizmente, isso significa que podemos contar com muitas técnicas anteriores da área de software modular e de programação estruturada para ajudar a nos guiar e descobrir como nossas fronteiras serão definidas. Com isso em mente, vamos analisar três conceitos essenciais com mais detalhes, os quais mencionamos rapidamente no Capítulo 1 e cujo entendimento é

vital quando se trata de descobrir o que é uma boa fronteira para um microsserviço: ocultação de informações, coesão e acoplamento.

Ocultação de informações

Ocultação de informações é um conceito desenvolvido por David Parnas para abordar o modo mais eficaz de definir fronteiras para módulos.¹ A ocultação de informações descreve o desejo de ocultar o máximo possível de detalhes atrás da fronteira de um módulo (ou, em nosso caso, de um microsserviço). Parnas analisou as vantagens que os módulos, teoricamente, nos proporcionariam; são elas:

Prazo melhor para o desenvolvimento

Ao permitir que os módulos sejam desenvolvidos de forma independente, podemos permitir que mais trabalho seja feito em paralelo e reduzir o impacto do acréscimo de mais desenvolvedores em um projeto.

Capacidade de compreensão

Cada módulo pode ser visto e compreendido isoladamente. Por sua vez, isso facilita entender o que o sistema como um todo faz.

Flexibilidade

Módulos podem ser alterados de forma independente uns dos outros, permitindo que mudanças sejam feitas em funcionalidades do sistema sem exigir que outros módulos sejam alterados. Além disso, os módulos podem ser combinados de diferentes maneiras para disponibilizar novas funcionalidades.

A lista de características desejáveis complementa bem o que estamos tentando fazer com as arquiteturas de microsserviços – e, de fato, vejo atualmente os microsserviços como apenas outra forma de arquitetura modular. Adrian Colyer consultou uma série de artigos de David Parnas dessa época e os analisou com respeito aos microsserviços, e vale muito a pena ler seus resumos.²

A verdade é que, conforme explorado por Parnas em boa parte de seu trabalho, apenas ter módulos não resulta em atingir verdadeiramente esses resultados. Muitos aspectos dependerão de *como* as fronteiras dos módulos

são definidas. Sua própria pesquisa mostra que a ocultação de informações foi uma técnica essencial para ajudar a tirar o máximo de proveito das arquiteturas modulares e, com um olhar moderno, podemos dizer que o mesmo se aplica também aos microsserviços.

De outro artigo de Parnas³, temos esta joia:

As conexões entre os módulos são as suposições que os módulos fazem uns sobre os outros.

Ao reduzir o número de suposições que um módulo (ou microsserviço) faz sobre outro, causamos impactos diretos nas conexões entre eles. Ao manter o número de suposições baixo, será mais fácil garantir que poderemos alterar um módulo sem causar impacto em outros. Se um desenvolvedor que estiver alterando um módulo tiver um bom entendimento acerca de como o módulo é usado por outros, será mais fácil para o desenvolvedor fazer alterações de forma segura, de modo que quem faz chamadas upstream não precisem alterar também.

Isso se aplica aos microsserviços também, exceto que também temos a oportunidade de fazer a implantação desse microsserviço alterado sem ter de implantar outros serviços; sem dúvida, isso amplia as três características desejáveis que Parnas descreve, isto é, prazo melhor para o desenvolvimento, capacidade de compreensão e flexibilidade.

As implicações da ocultação de informações aparecem em vários aspectos, e esse tema será retomado ao longo do livro.

Coesão

Eis uma das definições mais sucintas que já ouvi para descrever a coesão: “códigos que mudam juntos devem permanecer juntos”.⁴ Para nosso contexto, essa é uma definição muito boa. Conforme já discutimos, estamos otimizando nossa arquitetura de microsserviços com vistas à facilidade de alterações nas funcionalidades de negócios – portanto queremos que as funcionalidades sejam agrupadas de modo que possamos fazer alterações no menor número possível de lugares.

Queremos que comportamentos relacionados estejam juntos, e

comportamentos não relacionados estejam em outro lugar. Por quê? Bem, se quisermos mudar o comportamento, queremos ser capazes de alterá-lo em um só lugar e lançar essa alteração o mais rápido possível. Se tivermos de modificar esse comportamento em vários lugares diferentes, teremos de lançar vários serviços diferentes (talvez ao mesmo tempo) para disponibilizar essa alteração. Fazer mudanças em vários lugares diferentes é um processo mais lento, e implantar vários serviços de uma só vez é arriscado – portanto, queremos evitar as duas situações.

Desse modo, queremos encontrar fronteiras no domínio de nosso problema que ajudem a garantir que comportamentos relacionados estejam em um só lugar e se comuniquem com outras fronteiras o mínimo possível. Se as funcionalidades relacionadas estiverem espalhadas pelo sistema, dizemos que a coesão é fraca – em nossas arquiteturas de microsserviços, nosso objetivo é ter uma forte coesão.

Acoplamento

Quando os serviços têm baixo acoplamento, uma mudança em um serviço não deve exigir que outro serviço seja alterado. O aspecto mais importante de um microsserviço é que seja possível alterar e implantar um serviço sem que seja necessário alterar outras partes do sistema. Isso é realmente muito importante.

Que tipo de fatores causam um alto acoplamento? Um erro clássico é escolher um estilo de integração que vincule fortemente um serviço a outro, fazendo com que mudanças internas ao serviço exijam uma mudança nos consumidores.

Um serviço com baixo acoplamento sabe o mínimo necessário sobre os serviços com os quais atua em colaboração. Isso também significa que provavelmente vamos querer limitar o número de diferentes tipos de chamadas de um serviço para outro porque, além do possível problema de desempenho, uma comunicação excessiva pode resultar em alto acoplamento.

O acoplamento, porém, assume diversas formas, e já vi uma série de equívocos acerca da natureza do acoplamento ser pertinente a uma

arquitetura baseada em serviços. Com isso em mente, acho que é importante explorarmos esse assunto com mais detalhes – algo que faremos em breve.

Interação entre acoplamento e coesão

Conforme já mencionamos, os conceitos de acoplamento e coesão estão evidentemente relacionados. Do ponto de vista lógico, se funcionalidades relacionadas estiverem espalhadas pelo nosso sistema, mudanças em uma funcionalidade se estenderão para além de suas fronteiras, implicando um acoplamento mais alto. A lei de Constantine, batizada em homenagem ao pioneiro do design estruturado Larry Constantine, sintetiza bem essa ideia:

Uma estrutura será estável se a coesão for alta e o acoplamento for baixo.⁵

Esse conceito de estabilidade é importante para nós. Para que as fronteiras de nossos microsserviços cumpram o prometido acerca da possibilidade de implantações independentes, permitindo que trabalhem nos microsserviços em paralelo e reduzindo o trabalho de coordenação entre equipes que trabalham nesses serviços, precisamos de certo grau de estabilidade nas próprias fronteiras. Se o contrato exposto por um microsserviço mudar constantemente de modo a se tornar incompatível com versões anteriores, isso fará com que os consumidores upstream também tenham de mudar constantemente.

O acoplamento e a coesão estão fortemente relacionados e, em certo nível pelo menos, sem dúvida são iguais, pois ambos os conceitos descrevem o relacionamento entre itens. A coesão se aplica ao relacionamento entre itens *dentro* de uma fronteira (um microsserviço, em nosso contexto), enquanto o acoplamento descreve o relacionamento entre itens que *cruzam* uma fronteira. Não há uma *melhor* maneira única de organizar o código; o acoplamento e a coesão são apenas uma forma de articular os vários compromissos que assumimos em relação a como agrupamos o código, e por quê. Tudo que podemos fazer é nos esforçar para encontrar o equilíbrio correto entre essas duas ideias, um que faça mais sentido para o seu contexto específico e os problemas que você estiver enfrentando no

momento.

Lembre-se de que o mundo não é estático – é possível que, à medida que os requisitos de seu sistema mudarem, você encontre motivos para rever suas decisões. Às vezes, partes de seu sistema podem estar sujeitas a muitas mudanças, a ponto de não ser possível ter uma estabilidade. Veremos um exemplo desse caso no Capítulo 3, quando compartilharei as experiências da equipe de desenvolvimento de produto responsável pelo Snap CI.

Tipos de acoplamento

Com base na visão geral anterior, você poderia inferir que qualquer acoplamento é ruim. Isso não é estritamente verdadeiro. Em última análise, um pouco de acoplamento em nosso sistema será inevitável. O que queremos fazer é reduzir o volume de acoplamento que teremos.

Muito trabalho foi feito para analisar as diferentes formas de acoplamento no contexto da programação estruturada, que, de modo geral, levava em consideração um software modular (não distribuído, monolítico). Muitos desses diferentes modelos para avaliar o acoplamento apresentam pontos em comum ou entram em conflito, mas, em todos os casos, discutem aspectos principalmente no nível de código, em vez de considerar as interações entre serviços. Como os microsserviços são um estilo de arquitetura modular (ainda que tenha a complexidade adicional dos sistemas distribuídos), podemos usar muitos desses conceitos originais e aplicá-los no contexto de nossos sistemas baseados em microsserviços.

Técnicas anteriores em programação estruturada

Boa parte de nosso trabalho em computação envolve criações com base em trabalhos que vieram antes. Às vezes, é impossível reconhecer tudo que já foi feito, mas, nesta segunda edição, meu objetivo foi dar destaque às criações anteriores sempre que foi possível – em parte para dar o crédito quando devido, em parte como uma forma de garantir que deixaria algumas pistas aos leitores que quisessem explorar determinados tópicos com mais detalhes, mas também para mostrar que muitas dessas ideias já foram testadas e comprovadas.

Quando se trata de criar algo com base em um trabalho que já tenha

sido feito antes, há poucos assuntos neste livro que tiveram tantas criações anteriores quanto a programação estruturada. Já mencionei Larry Constantine; seu livro com Edward Yourdon, *Structured Design*⁶, é considerado um dos textos mais importantes nessa área. *The Practical Guide to Structured Systems Design*⁷ de Meilir Page-Jones também é útil. Infelizmente, um aspecto em comum entre esses livros é a dificuldade em adquiri-los, pois sua publicação está esgotada e não estão disponíveis em formato eletrônico. Eis um outro motivo para você contribuir com a sua biblioteca local!

Nem todas as ideias podem ser claramente mapeadas, portanto eu me esforcei ao máximo para sintetizar um modelo funcional para os diferentes tipos de acoplamento com microsserviços. Quando as ideias podiam ser claramente mapeadas às definições anteriores, mantive os termos. Em outros pontos, tive de criar outros termos ou combinar ideias provenientes de outros lugares. Portanto, considere o que será apresentado a seguir como algo construído com base em várias criações anteriores nessa área, às quais tento dar mais sentido no contexto dos microsserviços.

Na Figura 2.1, temos uma visão geral resumida dos diferentes tipos de acoplamentos, organizados do baixo (desejável) para o alto (indesejável) acoplamento.

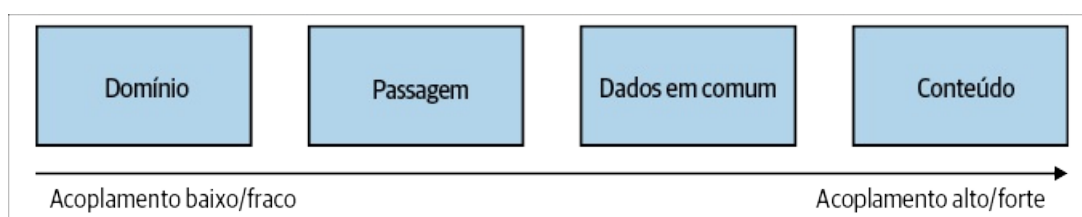


Figura 2.1 – Diferentes tipos de acoplamentos, do baixo para o alto acoplamento.

Em seguida, analisaremos cada forma de acoplamento individualmente, e veremos exemplos que mostram como essas formas podem se manifestar em nossa arquitetura de microsserviços.

Acoplamento de domínio

Um acoplamento de domínio descreve uma situação na qual um microsserviço deve interagir com outro microsserviço porque o primeiro

microserviço precisa utilizar a funcionalidade que o outro microserviço oferece.⁸

Na Figura 2.2, vemos parte de como os pedidos de CDs são gerenciados na MusicCorp. Nesse exemplo, *Processador de Pedido* chama o microserviço *Depósito* para reservar itens do estoque, e o microserviço *Pagamento* para receber o pagamento. O *Processador de Pedido*, portanto, depende dos microserviços *Depósito* e *Pagamento*, e está acoplado a eles, nessa operação. Contudo, não vemos esse tipo de acoplamento entre *Depósito* e *Pagamento*, pois eles não interagem.

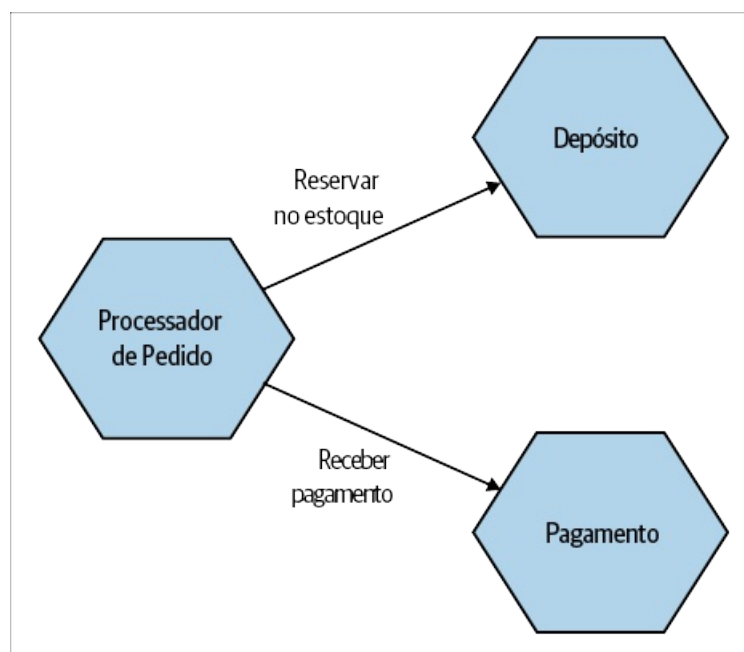


Figura 2.2 – Um exemplo de acoplamento de domínio, no qual Processador de Pedido precisa utilizar a funcionalidade disponibilizada por outros microserviços.

Em uma arquitetura de microserviços, esse tipo de interação é, em sua maior parte, inevitável. Um sistema baseado em microserviços depende de vários microserviços que colaboram entre si para que seu trabalho seja feito. Ainda queremos, porém, manter essa interação em um nível mínimo; sempre que você vir um único microserviço depender de vários serviços downstream dessa forma, pode ser um motivo para preocupação – poderia implicar que um microserviço está fazendo tarefas demais.

Como regra geral, um acoplamento de domínio é considerado uma forma

de acoplamento baixo, embora, mesmo nesse caso, seja possível que enfrentemos problemas. Um microserviço que precise conversar com vários microserviços downstream pode sinalizar uma situação em que haja muita lógica centralizada. Um acoplamento de domínio também pode se tornar problemático à medida que conjuntos mais complexos de dados forem enviados entre os serviços – muitas vezes, essa situação pode apontar para formas mais complicadas de acoplamento que exploraremos em breve.

Lembre-se da importância de ocultar informações. Compartilhe somente o que for absolutamente necessário e envie apenas a quantidade mínima de dados que você precisar.

Uma observação rápida sobre acoplamento temporário

Outra forma de acoplamento da qual você talvez já tenha ouvido falar é o *acoplamento temporário*. De um ponto de vista centrado em código para o acoplamento, um acoplamento temporário refere-se a uma situação em que alguns conceitos são reunidos somente porque ocorrem ao mesmo tempo. Um acoplamento temporário tem um significado sutilmente diferente no contexto de um sistema distribuído, e se refere a uma situação em que um microserviço precisa que outro microserviço faça algo no mesmo instante para que a operação seja executada.

Os dois microserviços devem estar ativos e disponíveis para se comunicarem um com o outro no mesmo instante para que a operação seja executada. Assim, na Figura 2.3, na qual o **Processador de Pedido** da MusicCorp faz uma chamada HTTP síncrona para o serviço **Depósito**, esse serviço deve estar ativo e disponível no instante em que a chamada for feita.

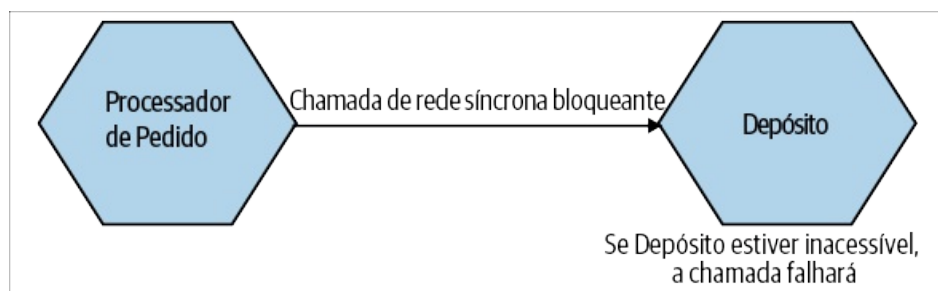


Figura 2.3 – Um exemplo de acoplamento temporário, em que Processador de

Pedido faz uma chamada HTTP síncrona para o microserviço Depósito.

Se, por algum motivo, o serviço Depósito não puder ser acessado por Processador de Pedido, a operação falhará, pois não será possível reservar os CDs a serem despachados. Processador de Pedido também terá de ficar bloqueado esperando uma resposta de Depósito, e isso pode causar problemas no que diz respeito à contenção de recursos.

Um acoplamento temporário nem sempre é ruim; é apenas algo do qual devemos estar cientes. À medida que tivermos mais microserviços, com interações mais complexas entre si, os desafios do acoplamento temporário poderão aumentar, até um ponto em que se tornará mais difícil escalar seu sistema e mantê-lo funcionando. Um dos modos de evitar um acoplamento temporário é utilizar algum tipo de comunicação assíncrona, por exemplo, um broker de mensagens.

Acoplamento de passagem

Um “acoplamento de passagem”² (pass-through coupling) descreve uma situação na qual um microserviço passa dados para outro microserviço somente porque esses dados são necessários a um outro microserviço mais adiante. Em vários aspectos, é uma das formas mais problemáticas de acoplamento na implementação, pois implica que quem faz a chamada não só sabe que o microserviço que ele chama faz uma chamada a outro microserviço, mas também que ele possivelmente precisa saber como esse microserviço a um passo além funciona.

Como exemplo de um acoplamento de passagem, vamos observar mais de perto o funcionamento de uma parte do processamento de pedidos na MusicCorp. Na Figura 2.4, temos um Processador de Pedido enviando uma requisição para Depósito a fim de preparar um pedido para ser despachado. Como parte do payload da requisição, enviamos junto um Manifesto de Expedição. Esse Manifesto de Expedição contém não só o endereço do cliente, como também o tipo de frete. O Depósito apenas passa esse manifesto para o microserviço Expedição downstream.

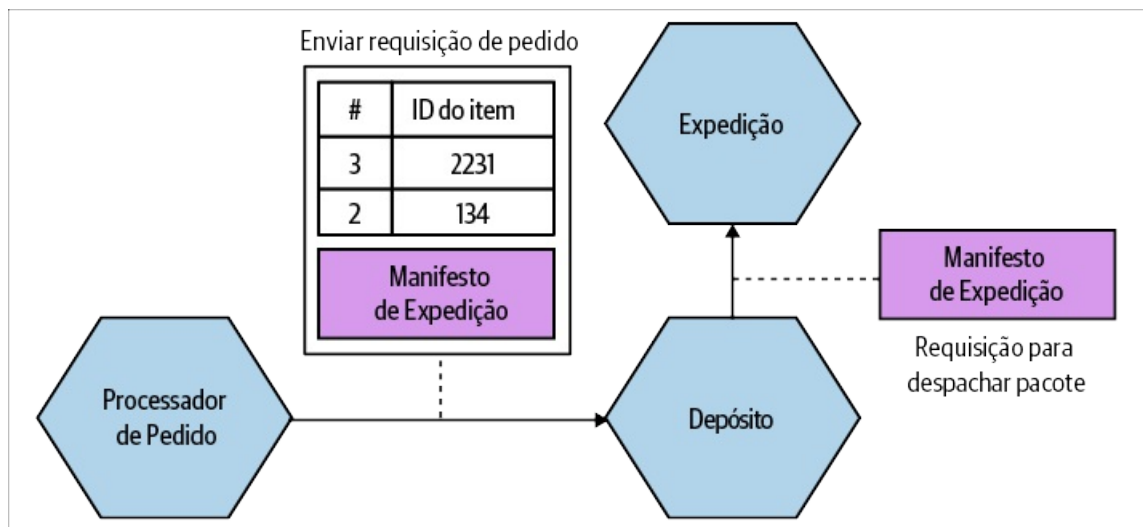


Figura 2.4 – Acoplamento de passagem, no qual os dados são passados para um microsserviço somente porque outro serviço downstream precisará deles.

O principal problema com o acoplamento de passagem é que uma mudança nos dados necessários mais adiante pode causar uma mudança upstream mais significativa. Em nosso exemplo, se *Expedição* precisar agora que o formato ou o conteúdo dos dados seja alterado, tanto *Depósito* como *Processador de Pedido* provavelmente terão de mudar.

Há algumas maneiras de corrigir isso. A primeira é considerar se faz sentido que o microsserviço que faz a chamada simplesmente ignore o intermediário. Em nosso exemplo, significaria que *Processador de Pedido* conversaria diretamente com *Expedição*, como mostra a Figura 2.5. No entanto, essa solução causaria outras dores de cabeça. Nosso *Processador de Pedido* está aumentando seu acoplamento de domínio, pois *Expedição* seria outro microsserviço que ele precisaria conhecer – se essa fosse a única complicação, ainda poderia não ser um problema, pois um acoplamento de domínio, é claro, é uma forma mais fraca de acoplamento. No entanto, essa solução se torna mais complexa nesse caso, pois um item deve ser reservado por *Depósito* antes que o pacote seja despachado usando *Expedição*; depois que a expedição for feita, é preciso atualizar devidamente o estoque. Com isso, mais complexidade e lógica, que estavam antes ocultas em *Depósito*, terão de ser levadas para *Processador de Pedido*.

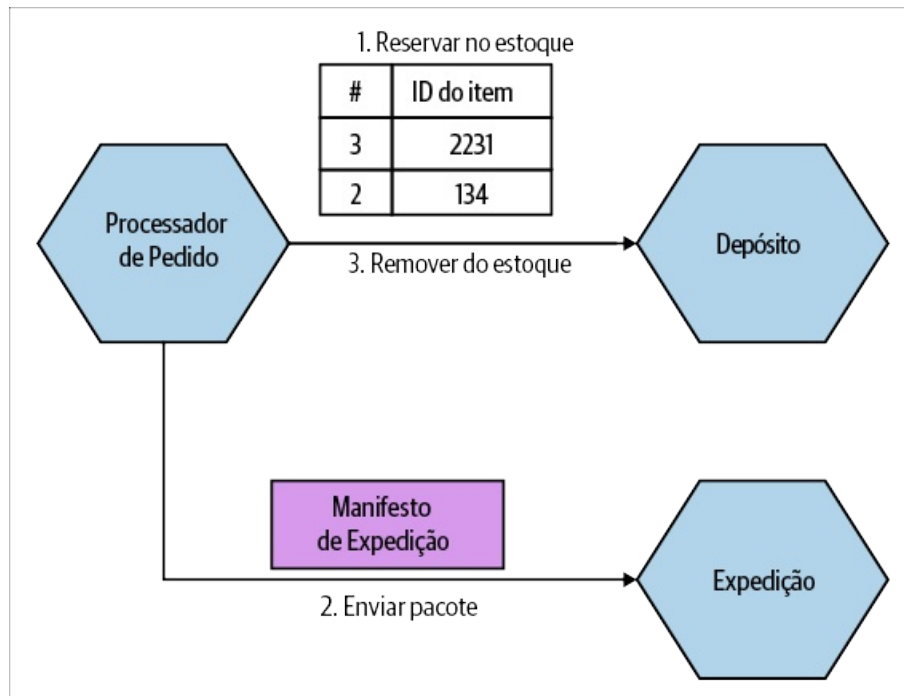


Figura 2.5 – Uma maneira de contornar o acoplamento de passagem envolve comunicar-se diretamente com o serviço downstream.

Nesse exemplo específico, poderíamos considerar uma mudança mais simples (embora com mais nuances), ou seja, ocultar totalmente a exigência de um Manifesto de Expedição por Processador de Pedido. A ideia de delegar o trabalho tanto do gerenciamento do estoque como os preparativos para despachar o pacote para o nosso serviço Depósito faz sentido, mas não gostamos do fato de termos vazado alguns detalhes de implementação de mais baixo nível, isto é, o fato de o serviço Expedição querer um Manifesto de Expedição. Uma forma de ocultar esse detalhe seria fazer com que Depósito recebesse as informações necessárias como parte de seu contrato, e então fazê-lo construir o Manifesto de Expedição localmente, conforme vemos na Figura 2.6. Isso significa que, se o serviço Expedição alterar seu contrato de serviço, essa mudança será invisível do ponto de vista de Processador de Pedido, desde que Depósito reúna os dados necessários.

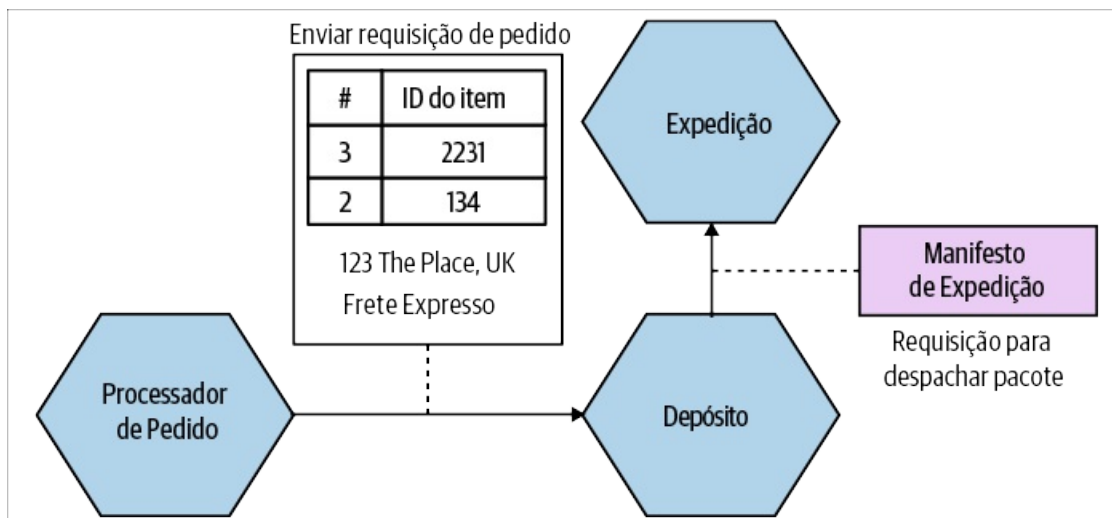


Figura 2.6 – Ocultando a necessidade de um Manifesto de Expedição de Processador de Pedido.

Embora isso vá ajudar a proteger o microserviço `Depósito` de algumas mudanças em `Expedição`, ainda há outras situações que exigirão que todas as partes sejam alteradas. Vamos considerar a ideia de que queremos começar a enviar produtos internacionalmente. Como parte disso, o serviço `Expedição` precisará que uma `Declaração de Alfândega` seja incluída no `Manifesto de Expedição`. Se esse for um parâmetro opcional, poderíamos implantar uma nova versão do microserviço `Expedição`, sem problemas. Todavia, se for um parâmetro obrigatório, `Depósito` teria de criá-lo. Ele poderia fazer isso com informações existentes, que já possui (ou recebe), ou poderia exigir que informações adicionais lhe fossem passadas por `Processador de Pedido`.

Embora, nesse caso, não tenhamos eliminado a necessidade de fazer alterações em todos os três microserviços, temos muito mais controle sobre onde e como essas alterações poderiam ser feitas. Se tivéssemos o alto acoplamento (de passagem) do exemplo inicial, acrescentar essa nova `Declaração de Alfândega` poderia exigir um rollout sincronizado dos três microserviços. Ao ocultar esse detalhe, poderíamos ao menos separar mais facilmente a implantação em fases.

Uma última abordagem que poderia ajudar a reduzir o acoplamento de passagem seria fazer o `Processador de Pedido` continuar enviando o `Manifesto de Expedição` para o microserviço `Expedição` por meio de `Depósito`, porém fazer com que `Depósito` não tivesse nenhuma ciência da estrutura do `Manifesto de`

Expedição propriamente dito. O Processador de Pedido enviaria o manifesto como parte da requisição do pedido, porém Depósito não faria nenhuma tentativa de olhar ou processar o campo – ele apenas o trataria como uma massa de dados, sem se importar com o seu conteúdo. Ele apenas passaria esses dados adiante. Uma mudança no formato do Manifesto de Expedição ainda exigiria uma alteração tanto no microserviço Processador de Pedido como no microserviço Expedição, mas, como Depósito não se importa nem um pouco com o conteúdo do manifesto, ele não precisaria ser modificado.

Acoplamento de dados em comum

Um acoplamento de dados em comum ocorre quando dois ou mais microserviços fazem uso de um conjunto de dados em comum. Um exemplo simples e usual dessa forma de acoplamento seria ter vários microserviços utilizando o mesmo banco de dados compartilhado, mas esse tipo de acoplamento também poderia se manifestar com o uso de uma memória compartilhada ou um sistema de arquivos compartilhado.

O principal problema do acoplamento de dados em comum é que mudanças na estrutura dos dados podem causar impacto em vários microserviços ao mesmo tempo. Considere o exemplo de alguns dos serviços da MusicCorp na Figura 2.7. Como discutimos antes, a MusicCorp atua no mundo todo, portanto ela precisa de várias informações sobre os países em que tem operações. Nesse caso, vários serviços leem dados de referência estáticos de um banco de dados compartilhado. Se o esquema desse banco de dados mudar de uma forma que não seja compatível com versões anteriores, isso exigiria mudanças em cada consumidor do banco de dados. Na prática, dados compartilhados como esses, consequentemente, tendem a ser muito difíceis de serem alterados.

O exemplo da Figura 2.7 é relativamente benigno. Isso porque, em virtude de sua própria natureza, dados de referência estáticos tendem a não mudar com frequência; além disso, esses dados são somente para leitura – sendo assim, eu não me preocuparia muito em compartilhar dados de referência estáticos dessa forma. Um acoplamento de dados em comum se tornará

problemático, porém, se a estrutura dos dados em comum mudar com mais frequência, ou se vários microsserviços lerem e escreverem os mesmos dados.

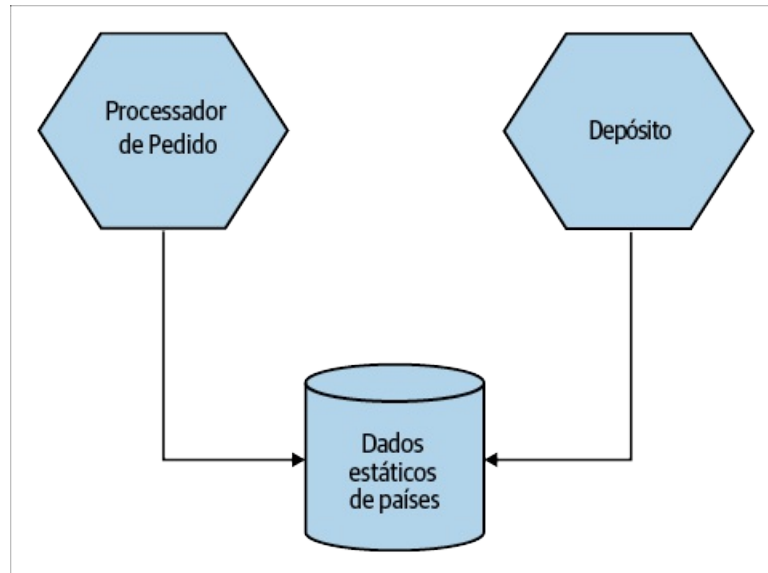


Figura 2.7 – Vários serviços acessando dados de referência estáticos e compartilhados, relacionados a países, no mesmo banco de dados.

A Figura 2.8 mostra uma situação em que tanto o serviço `Processador de Pedido` como o serviço `Depósito` leem e escrevem em uma tabela compartilhada de `Pedidos` para gerenciar o processo de enviar CDs aos clientes da MusicCorp. Os dois microsserviços atualizam a coluna `status`. O `Processador de Pedido` pode definir os status `FEITO`, `PAGO` e `CONCLUÍDO`, enquanto `Depósito` aplicará os status `SELECIONANDO` OU `ENVIADO`.

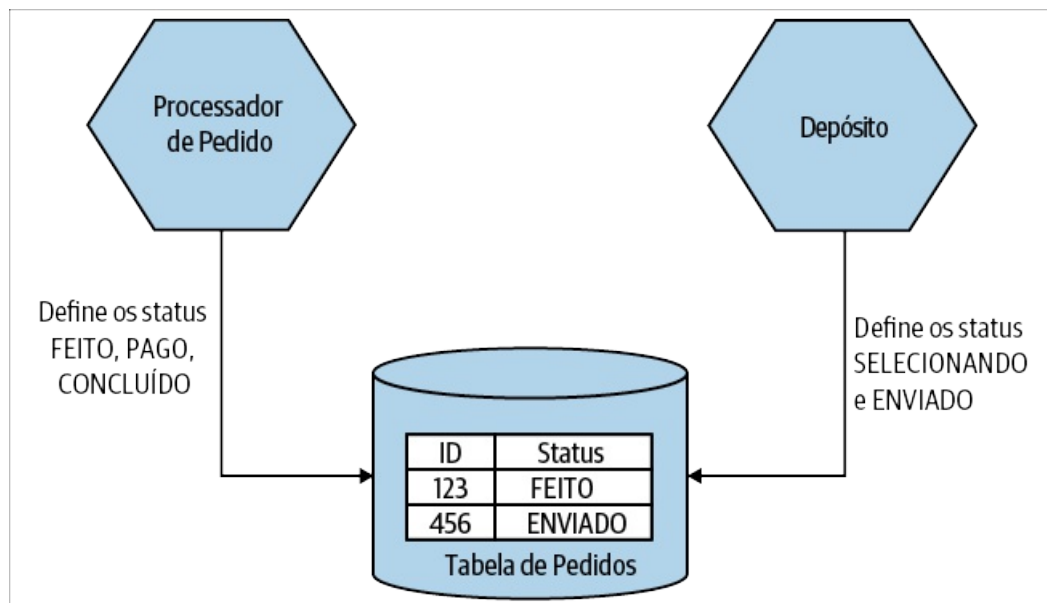


Figura 2.8 – Um exemplo de acoplamento de dados em comum, em que tanto Processador de Pedido como Depósito atualizam o mesmo registro de pedido.

Embora possamos considerar a Figura 2.8 como, de certo modo, artificial, esse exemplo simples de acoplamento de dados em comum ajuda a ilustrar um problema essencial. Conceitualmente, temos dois microserviços, *Processador de Pedido* e *Depósito*, gerenciando diferentes aspectos do ciclo de vida de um pedido. Ao fazer mudanças em *Processador de Pedido*, eu poderia ter certeza de que não estaria alterando os dados de pedido de um modo que corrompesse a visão de mundo de *Depósito*, ou vice-versa?

Uma maneira de garantir que o estado de algo seja alterado de forma correta seria criar uma máquina de estados finita. Uma máquina de estados pode ser usada para gerenciar a transição de alguma entidade de um estado para outro, garantindo que transições de estado inválidas sejam proibidas. Na Figura 2.9, podemos ver as transições de estado permitidas para um pedido na MusicCorp. Um pedido pode ir diretamente de *FEITO* para *PAGO*, mas não de *FEITO* para *SELECIONANDO* (essa máquina de estados provavelmente não seria suficiente para processos de negócio reais, envolvidos em um ciclo completo, do início ao fim, de compra e envio de mercadorias, mas eu queria apresentar um exemplo simples, que ilustrasse a ideia).

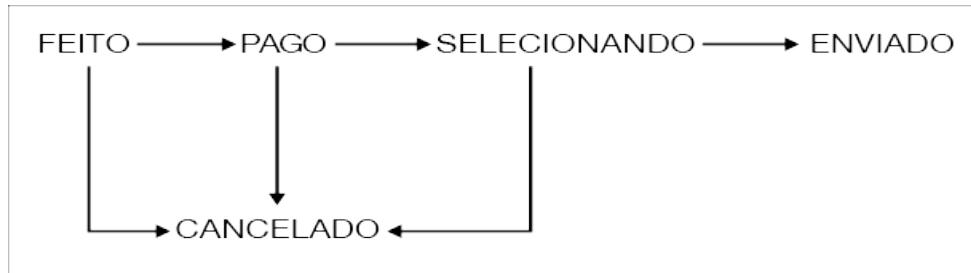


Figura 2.9 – Visão geral das transições de estado permitidas para um pedido na MusicCorp.

O problema, nesse exemplo específico, é que tanto `Depósito` como `Processador de Pedido` compartilham responsabilidades para gerenciar essa máquina de estados. Como garantir que eles concordam no que diz respeito às transições permitidas? Há maneiras de gerenciar processos como esse, que ultrapassam fronteiras de microsserviços; retomaremos esse assunto quando discutirmos as sagas no Capítulo 6.

Uma possível solução nesse caso seria garantir que um único microsserviço gerencie o estado do pedido. Na Figura 2.10, tanto `Depósito` como `Processador de Pedido` podem enviar requisições de atualização de status ao serviço `Pedido`. O microsserviço `Pedido` seria a fonte da verdade para qualquer pedido em particular. Nessa situação, é realmente importante que vejamos as requisições de `Depósito` e de `Processador de Pedido` apenas como isso, ou seja, como *requisições*. Nesse cenário, gerenciar as transições de estado aceitáveis associadas a um agregado de dados pedido é função do serviço `Pedido`. Assim, se o serviço `Pedido` receber uma requisição de `Processador de Pedido` para modificar um estado diretamente de `FEITO` para `CONCLUÍDO`, ele terá a liberdade de rejeitar essa requisição se essa for uma alteração inválida.

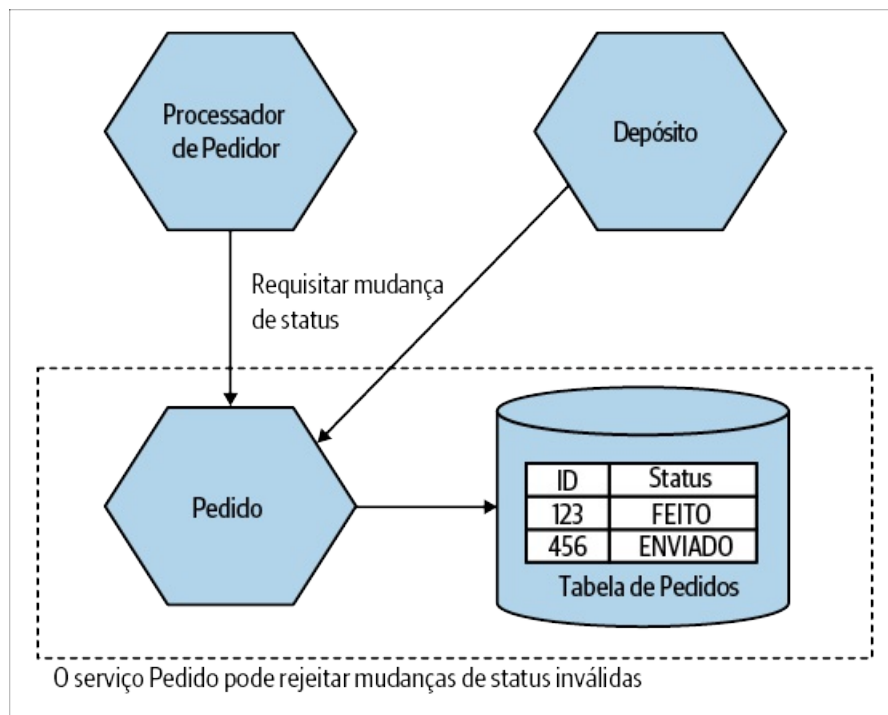


Figura 2.10 – Tanto Processador de Pedido como Depósito podem pedir que mudanças sejam feitas em um pedido, porém o microserviço Pedido decidirá quais requisições são aceitáveis.



Não se esqueça de encarar uma requisição enviada para um microserviço downstream como algo que esse microserviço poderá rejeitar caso a requisição seja inválida.

Uma abordagem alternativa que costumo ver em casos como esse é implementar o serviço **Pedido** como pouco mais que um wrapper em torno de operações CRUD do banco de dados; as requisições são simplesmente mapeadas de forma direta às atualizações no banco de dados. É semelhante a um objeto com campos privados, mas com getters e setters públicos – o comportamento deixou de estar restrito ao microserviço e se espalhou por consumidores upstream (reduzindo a coesão), e estamos de volta ao mundo em que o gerenciamento de transições de estado aceitáveis é feito por vários serviços distintos.



Se você vir um microserviço que pareça ser apenas um wrapper fino em torno de operações CRUD no banco de dados, é um sinal de que pode haver baixa coesão e alto acoplamento, pois a lógica para

gerenciar os dados, que deveria estar nesse serviço, está espalhada em outros pontos do sistema.

As razões para um acoplamento de dados em comum também podem ser razões para uma contenção de recursos (resource contention). Vários microsserviços fazendo uso do mesmo sistema de arquivos ou banco de dados poderiam sobrecarregar esse recurso compartilhado, podendo causar problemas significativos caso o recurso compartilhado se torne lento ou até mesmo totalmente indisponível. Um banco de dados compartilhado é particularmente suscetível a esse problema, pois vários consumidores podem executar queries arbitrárias no próprio banco de dados, as quais, por sua vez, podem ter características muito distintas de desempenho. Já vi mais de um banco de dados ser levado ao seu limite por causa de uma query SQL custosa – posso até ter sido o culpado uma ou duas vezes¹⁰.

Então, um acoplamento de dados em comum às vezes pode não ser um problema, mas, com frequência, será. Ainda que o acoplamento seja benigno, isso significa que estaremos limitados quanto às mudanças que poderão ser feitas nos dados compartilhados; em geral, porém, indicará uma falta de coesão em nosso código. Também poderemos ter problemas relacionados a uma contenção operacional. É por esses motivos que consideramos o acoplamento de dados em comum como uma das formas menos desejáveis de acoplamento – no entanto, a situação pode piorar.

Acoplamento de conteúdo

O acoplamento de conteúdo descreve uma situação na qual um serviço upstream acessa um dado interno de um serviço downstream e modifica seu estado interno. A manifestação mais comum desse caso é um serviço externo que acessa o banco de dados de outro microsserviço, alterando-o diretamente. As diferenças entre acoplamento de conteúdo e acoplamento de dados em comum são sutis. Em ambos os casos, dois ou mais microsserviços leem e escrevem no mesmo conjunto de dados. No acoplamento de dados em comum, você sabe que está fazendo uso de uma dependência externa compartilhada e sabe que ela não está sob seu controle. No acoplamento de conteúdo, as linhas de responsabilidade se tornam menos claras, e é mais difícil para os desenvolvedores alterarem um

sistema.

Vamos rever nosso exemplo anterior da MusicCorp. Na Figura 2.11, temos um serviço `Pedido` que é esperado que vá gerenciar as mudanças de estado permitidas nos pedidos de nosso sistema. O `Processador de Pedido` envia requisições ao serviço `Pedido`, delegando não só a mudança exata no estado a ser feita, mas também a responsabilidade por decidir quais transições de estado são permitidas. Por outro lado, o serviço `Depósito` atualiza diretamente a tabela na qual os dados do pedido são armazenados, ignorando qualquer funcionalidade do serviço `Pedido` que possa verificar as mudanças que são permitidas. Temos de esperar que o serviço `Depósito` tenha um conjunto consistente de lógica que garanta que apenas mudanças válidas sejam feitas. No melhor caso, isso representa uma duplicação de lógica. No pior caso, a verificação quanto às mudanças permitidas presente em `Depósito` seria diferente daquela que está no serviço `Pedido` e, como resultado, poderíamos acabar com pedidos em estados confusos e inusitados.

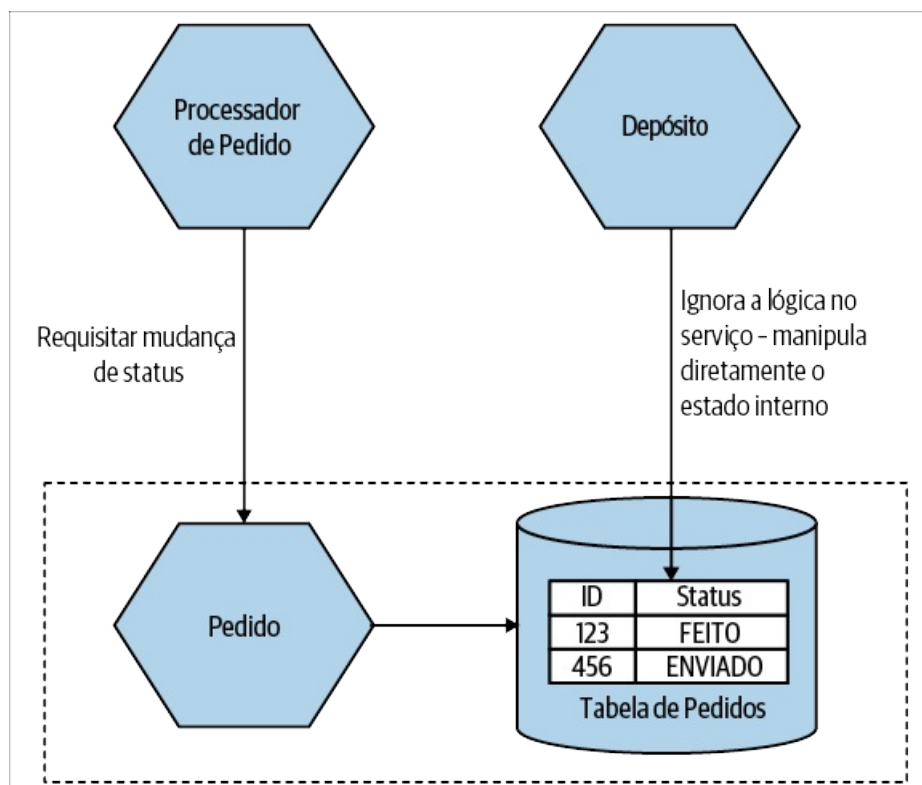


Figura 2.11 – Um exemplo de acoplamento de conteúdo, no qual `Depósito`

acessa diretamente os dados internos do serviço Pedido.

Nessa situação, também temos o problema de a estrutura de dados interna de nossa tabela de pedidos ser exposta a um componente externo. Ao modificar o serviço `Pedido`, teremos agora de ser extremamente cuidadosos ao fazer alterações nessa tabela em particular – isso, supondo inclusive que seja óbvio para nós que essa tabela é acessada diretamente por um componente externo. A correção fácil, nesse caso, é fazer com que `Depósito` envie requisições ao serviço `Pedido`, que poderá analisá-las; com isso, também ocultaremos os detalhes internos, fazendo com que mudanças subsequentes no serviço `Pedido` sejam muito mais fáceis.

Se você estiver trabalhando com um microsserviço, é essencial ter uma separação clara entre o que pode ser livremente alterado e o que não pode ser. Para ser explícito, como desenvolvedor, você deve saber quando está alterando uma funcionalidade que faz parte do contrato que seu serviço expõe para o mundo externo. É necessário garantir que, se você fizer alterações, elas não causarão falhas nos consumidores upstream. Funcionalidades que não causem impacto no contrato que seu microsserviço expõe podem ser alteradas sem maiores preocupações.

Em dúvida, os problemas que ocorrem com um acoplamento de dados em comum também se aplicam ao acoplamento de conteúdo, mas o acoplamento de conteúdo tem alguns problemas adicionais que o tornam mais complicado, a ponto de algumas pessoas o chamarem de *acoplamento patológico*.

Ao permitir que uma entidade externa acesse diretamente o *seu* banco de dados, o banco de dados, com efeito, se tornará parte desse contrato externo, além de ser um contrato sobre o qual você não saberá facilmente o que pode ou não pode ser alterado. Você perderia a capacidade de definir o que é compartilhado (e, portanto, não pode ser facilmente alterado) e o que está oculto. A ocultação de informações foi jogada pela janela.

Em suma, evite um acoplamento de conteúdo.

Apenas o suficiente sobre design orientado a domínios

Conforme apresentei no Capítulo 1, o principal método que usamos para

identificar fronteiras de microsserviços se baseia no próprio domínio, e faz uso do DDD (Domain-Driven Design, ou Design Orientado a Domínios) para ajudar na criação de um modelo para o nosso domínio. Vamos ampliar agora o nosso conhecimento acerca de como o DDD funciona no contexto dos microsserviços.

O desejo de ver nossos programas representarem melhor o mundo real no qual atuarão não é nenhuma novidade. As linguagens de programação orientadas a objetos como o Simula foram desenvolvidas para nos permitir modelar domínios reais. No entanto, são necessários mais do que os recursos de uma linguagem de programação para que essa ideia realmente tome forma.

O livro *Domain-Driven Design* de Eric Evans¹¹ apresenta uma série de ideias importantes que nos ajudam a representar melhor o domínio do problema em nossos programas. Uma exploração completa dessas ideias está além do escopo deste livro, mas há alguns conceitos básicos do DDD que valem a pena destacar, incluindo:

Linguagem ubíqua

Definir e adotar uma linguagem comum, a ser usada no código e na descrição do domínio, para facilitar a comunicação.

Agregado

Um conjunto de objetos gerenciado como uma única entidade, geralmente se referindo a conceitos do mundo real.

Contexto delimitado

Uma fronteira explícita em um domínio de negócios que provê funcionalidades a um sistema mais amplo, mas que também oculta as complexidades.

Linguagem ubíqua

Uma linguagem ubíqua se refere à ideia de que devemos nos esforçar para empregar os mesmos termos utilizados pelos usuários em nosso código. A ideia é que ter uma linguagem comum entre a equipe de desenvolvimento e as pessoas facilitará modelar o domínio do mundo real, além de melhorar a

comunicação.

Como contraexemplo, eu me recordo de uma situação da época em que trabalhava em um grande banco global. Estávamos trabalhando na área de liquidez corporativa, um termo chique que, basicamente, se refere à capacidade de transferir dinheiro entre diferentes contas da mesma entidade corporativa. A dona do produto era uma pessoa ótima para trabalhar, e ela tinha uma compreensão extremamente profunda dos diversos produtos que queria levar ao mercado. Ao trabalhar com ela, tínhamos discussões sobre assuntos como *haircuts* e *end-of-day sweeps*, tudo que fazia muito sentido em seu mundo e para seus clientes.

O código, por outro lado, não continha nenhuma dessas terminologias. Em algum momento no passado, uma decisão de utilizar um modelo de dados padrão para o banco de dados havia sido tomada. Ele era amplamente referenciado como “o modelo bancário da IBM”, mas eu tinha dúvidas se esse era um produto padrão da IBM ou apenas a criação de um consultor dessa empresa. Ao definir o conceito genérico de “negociação” (arrangement), segundo a teoria, qualquer operação bancária poderia ser modelada. Fazer um empréstimo? Era uma negociação. Comprar uma ação? Era uma negociação! Solicitar um cartão de crédito? Adivinhe só – era uma negociação também!

O modelo de dados havia poluído o código a tal ponto que qualquer entendimento real do sistema que estávamos construindo havia sido removido da base de código. Não estávamos criando uma aplicação bancária genérica. Estávamos construindo um sistema especificamente para gerenciar uma liquidez corporativa. O problema é que tínhamos de mapear a rica linguagem do domínio usada pelo dono do produto aos conceitos genéricos do código – ou seja, havia muito trabalho para auxiliar na tradução. Como consequência, nossos analistas de negócios muitas vezes gastavam seu tempo explicando os mesmos conceitos repetidamente.

Ao inserir a linguagem do mundo real no código, tudo se tornou muito mais fácil. Era bem mais provável que um desenvolvedor que visse uma história escrita com termos usados diretamente pelo dono do produto entendesse seu significado e descobrisse o que precisaria ser feito.

Agregado

No DDD, um *agregado* é um conceito de certa forma confuso, com muitas definições diferentes por aí. É apenas um conjunto arbitrário de objetos? É a menor unidade que deve ser obtida de um banco de dados? O modelo que sempre funcionou para mim é primeiro considerar um agregado como uma representação de um conceito real do domínio – pense em algo como um Pedido, uma Fatura, um Item do Estoque, e assim por diante. Em geral, os agregados têm um ciclo de vida associado, o que dá abertura para que sejam implementados como uma máquina de estados.

Como exemplo no domínio da MusicCorp, um Pedido é um agregado que pode conter várias linhas representando os itens do pedido. Essas linhas de itens têm sentido somente como parte do agregado Pedido como um todo.

Queremos tratar os agregados como unidades autocontidas; queremos garantir que o código que trata as transições de estado de um agregado esteja agrupado, junto com o próprio estado. Assim, um agregado deve ser gerenciado por um único microserviço, embora um único microserviço possa ser responsável pelo gerenciamento de vários agregados.

Em geral, porém, devemos pensar em um agregado como algo que tenha estado, identidade e um ciclo de vida que será gerenciado como parte do sistema. Em geral, os agregados se referem a conceitos do mundo real.

Um único microserviço cuidará do ciclo de vida e da armazenagem de dados de um ou mais tipos diferentes de agregados. Se uma funcionalidade em outro serviço quiser modificar um desses agregados, ele deverá requisitar diretamente uma mudança nesse agregado ou fazer com que o próprio agregado reaja a outros eventos do sistema para iniciar suas próprias transições de estado, talvez se registrando para receber eventos gerados por outros microserviços.

O ponto principal a ser compreendido nesse caso é que, se uma entidade externa requisitar uma transição de estado em um agregado, o agregado poderá dizer não. O ideal é que você implemente seus agregados de modo que transições de estado ilegais não sejam possíveis.

Os agregados podem ter relacionamentos com outros agregados. Na Figura

2.12, temos um agregado Cliente associado a um ou mais Pedidos e uma ou mais Listas de Desejos. Esses agregados poderiam ser gerenciados pelo mesmo microserviço ou por microserviços diferentes.

Se esses relacionamentos entre agregados estiverem presentes no escopo de um único microserviço, poderiam facilmente ser armazenados com algum método como um relacionamento de chave estrangeira, se um banco de dados relacional estiver em uso. Entretanto, se os relacionamentos entre esses agregados ultrapassarem as fronteiras do microserviço, precisaremos de algum método para modelar esses relacionamentos.

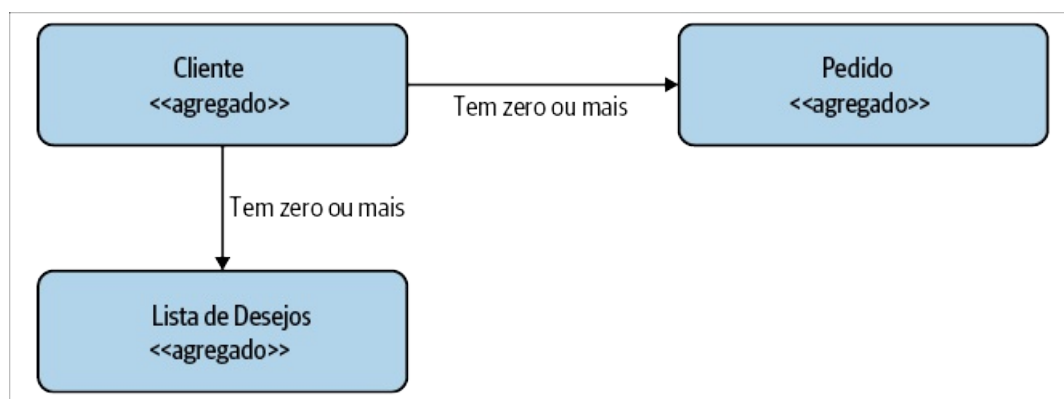


Figura 2.12 – Um agregado Cliente pode estar associado a um ou mais agregados Pedido ou Lista de Desejos.

Poderíamos simplesmente armazenar o ID do agregado diretamente em nosso banco de dados local. Por exemplo, considere um microserviço *Finanças* que gerencie um registro contábil, o qual armazena as transações de um cliente. Localmente, no banco de dados do microserviço *Finanças*, poderíamos ter uma coluna `custID` que contém o ID do cliente em questão. Se quiséssemos obter mais informações sobre esse cliente, teríamos de fazer uma consulta ao microserviço *cliente* usando esse ID.

O problema com esse conceito é que ele não é muito explícito – com efeito, o relacionamento entre a coluna `custID` e o cliente remoto é totalmente implícito. Para saber como esse ID é usado, teríamos de analisar o código do microserviço *Finanças*. Seria bom se pudéssemos armazenar uma referência a um agregado estrangeiro de um modo que fosse mais evidente.

Na Figura 2.13, fizemos alterações para deixar o relacionamento mais explícito. Em vez de um ID comum como referência para o cliente,

armazenamos um URI, que poderíamos usar se estivéssemos construindo um sistema baseado em REST¹².

As vantagens dessa abordagem se duplicam. A natureza do relacionamento é explícita e, em um sistema REST, poderíamos desreferenciar (dereference) diretamente esse URI a fim de consultar o recurso associado. Mas, e se não estivéssemos criando um sistema REST? Phil Calçado descreve uma variação dessa abordagem, em uso na SoundCloud¹³, na qual desenvolveram um esquema de pseudo-URI para referências entre serviços. Por exemplo, `soundcloud:tracks:123` seria uma referência para uma track cujo ID é 123. Isso é muito mais explícito para uma pessoa que estivesse olhando para esse identificador, mas é também um esquema conveniente o bastante, a ponto de ser fácil imaginar a criação de um código que facilitasse consultas de agregados entre microserviços, se fosse necessário.

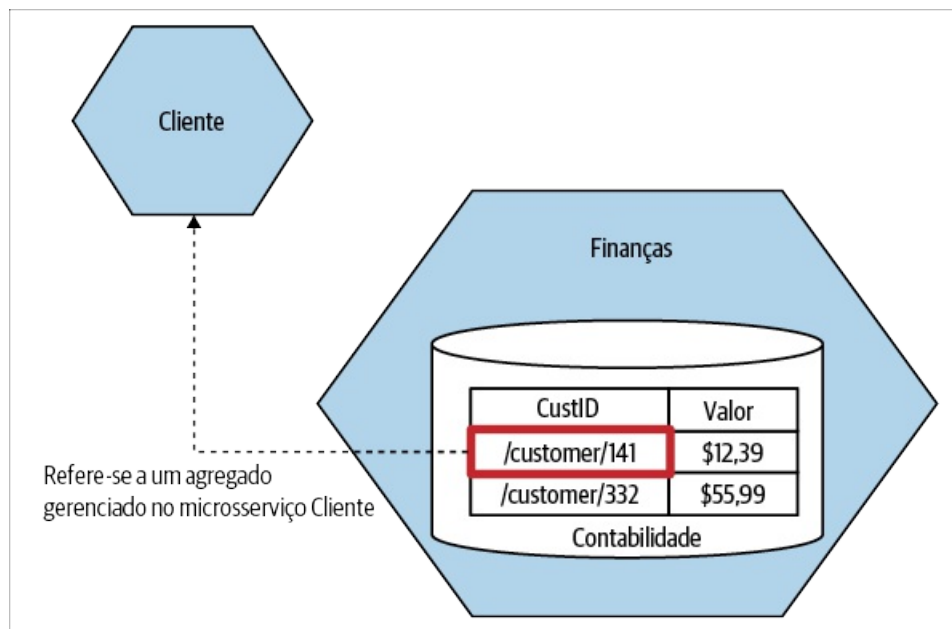


Figura 2.13 – Um exemplo de como um relacionamento entre dois agregados em diferentes microserviços pode ser implementado.

Há muitas maneiras de dividir um sistema em agregados, e algumas opções são extremamente subjetivas. Com o tempo, você poderia decidir remodelar os agregados, por questões de desempenho ou de facilidade de implementação. Considero, porém, que as preocupações com a implementação são secundárias; eu começo deixando que o modelo mental

dos usuários do sistema me oriente no design inicial, até que outros fatores entrem em cena.

Contexto delimitado

Um *contexto delimitado* (bounded context) em geral representa uma fronteira organizacional maior. No escopo dessa fronteira, responsabilidades explícitas devem ser colocadas em prática. Tudo isso é muito vago, portanto vamos analisar outro exemplo específico.

Na MusicCorp, nosso depósito está repleto de atividades – gerenciar pedidos a serem enviados (e as devoluções inesperadas), receber novos itens no estoque, fazer corridas de empilhadeiras, e assim por diante. Em outro lugar, o departamento financeiro talvez seja menos amante de diversões, mas ainda cumpre uma função importante em nossa organização, cuidando da folha de pagamentos, pagando por remessas, e tarefas desse tipo.

Os contextos delimitados ocultam os detalhes de implementação. Há algumas preocupações internas – por exemplo, os tipos de empilhadeira usados são de pouco interesse para qualquer pessoa que não seja funcionário do depósito. Essas preocupações internas devem permanecer ocultas ao mundo externo – eles não precisam saber, nem deveriam se importar.

Do ponto de vista da implementação, os contextos delimitados contêm um ou mais agregados. Alguns agregados podem ser expostos para fora do contexto delimitado; outros poderão estar ocultos internamente. Como no caso dos agregados, os contextos delimitados podem ter relacionamentos com outros contextos delimitados – quando são mapeados aos serviços, essas dependências se tornam dependências entre serviços.

Vamos voltar para os negócios da MusicCorp por um instante. Nosso domínio são todos os negócios nos quais atuamos. Incluem de tudo, do depósito à recepção, do setor financeiro aos pedidos. Podemos ou não modelar tudo isso em nosso software, mas, ainda assim, esse é o domínio no qual atuamos. Vamos pensar nas partes desse domínio que pareçam com os contextos delimitados aos quais Eric Evans se refere.

Modelos ocultos

Na MusicCorp, podemos considerar o departamento financeiro e o depósito como dois contextos delimitados distintos. Ambos têm uma interface explícita com o mundo externo (no que concerne a relatórios de estoque, holerites etc.), e têm detalhes que somente eles precisam conhecer (empilhadeiras, calculadoras).

O departamento financeiro não precisa saber do funcionamento interno detalhado do depósito. Ele precisa, no entanto, ter *algumas* informações; por exemplo, precisa saber dos níveis de estoque para manter as contas em dia. A Figura 2.14 mostra um exemplo de diagrama de contexto. Vemos conceitos que são internos ao depósito, por exemplo, um atendente de pedidos (alguém que recebe os pedidos), prateleiras que representam o lugar em que estão os itens do estoque, e assim por diante. Do mesmo modo, entradas no registro contábil geral são uma parte integrante do setor financeiro, mas não são compartilhados externamente nesse caso.

Para ser capaz de fazer uma avaliação da empresa, porém, os funcionários do departamento financeiro precisam de informações sobre o estoque existente. O item de estoque, então, passa a ser um modelo compartilhado entre os dois contextos. No entanto, observe que não precisamos expor cegamente todos os dados de um item do estoque que está no contexto do depósito. Na Figura 2.15, vemos como *Item do Estoque* no contexto delimitado do depósito contém referências às localizações das prateleiras, mas a representação compartilhada inclui apenas um contador. Então, temos uma representação somente interna, e a representação externa que é exposta.

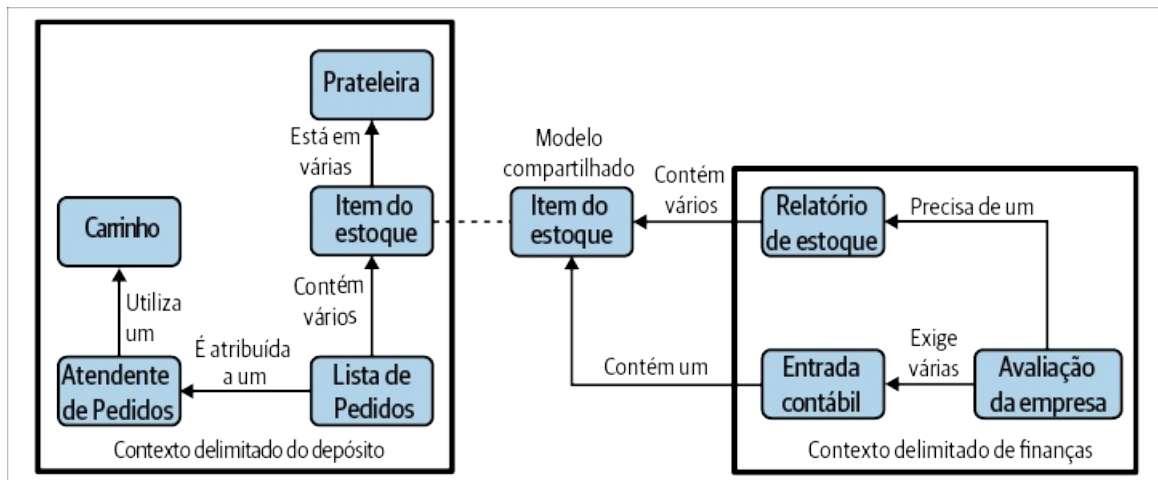


Figura 2.14 – Um modelo compartilhado entre o departamento financeiro e o depósito.

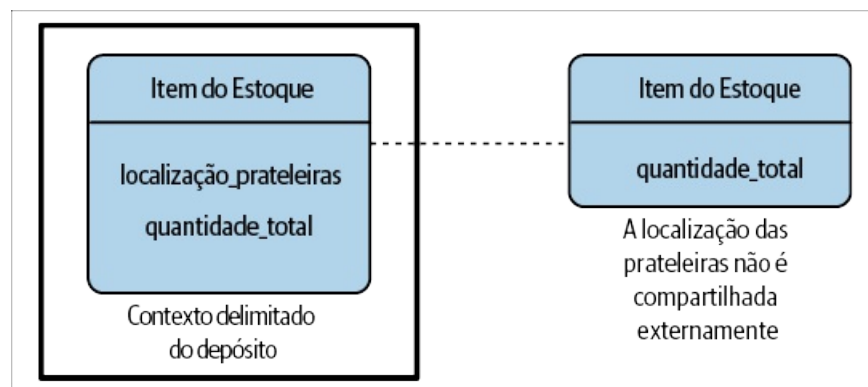


Figura 2.15 – Um modelo que é compartilhado pode decidir ocultar informações que não devam ser compartilhadas externamente.

Com frequência, quando temos representações internas e externas distintas, pode ser vantajoso nomeá-las de modo diferente para evitar confusão – nessa situação, uma abordagem poderia ser chamar o Item do Estoque que é compartilhado de Quantidade de Item do Estoque.

Modelos compartilhados

Também podemos ter conceitos que aparecem em mais de um contexto delimitado. Na Figura 2.14, vimos que há um cliente em dois lugares. O que isso significa? O cliente é copiado? Conceitualmente, devemos pensar que tanto finanças como o depósito precisam saber algo do cliente. Finanças precisa saber dos pagamentos feitos por um cliente, enquanto o depósito precisa ter informações suficientes do cliente para saber quais

pacotes foram enviados, de modo que as entregas possam ser rastreadas.

Quando você estiver em uma situação como essa, um modelo compartilhado, como do cliente, pode ter diferentes significados nos diferentes contextos delimitados e, desse modo, poderia ter nomes distintos. Poderíamos ficar satisfeitos em manter o nome “cliente” para finanças, porém, no depósito, poderíamos chamá-los de “destinatário”, pois esse é o papel que os clientes desempenham nesse contexto. Armazenamos informações sobre o cliente nos dois lugares, mas são informações diferentes. Finanças armazena informações sobre os pagamentos do cliente (ou reembolsos); o depósito armazena informações relacionadas aos produtos enviados. Talvez seja necessário fazer uma ligação entre os dois conceitos locais com um cliente global, e poderíamos querer consultar informações comuns, compartilhadas, sobre esse cliente, como nome ou endereço de email – para isso, uma técnica como aquela mostrada na Figura 2.13 poderia ser usada.

Mapeando agregados e contextos delimitados aos microsserviços

Tanto o agregado como o contexto delimitado nos fornecem unidades de coesão com interfaces bem definidas, dentro do sistema mais amplo. O agregado é uma máquina de estados autocontida com foco no conceito de um único domínio em nosso sistema, enquanto o contexto delimitado representa um conjunto de agregados associados, novamente, com uma interface explícita para o mundo mais amplo.

Desse modo, ambos podem funcionar bem como fronteiras de serviços. Ao iniciar os trabalhos, conforme já mencionei, você deve reduzir o número de serviços com os quais vai trabalhar. Como resultado, você provavelmente deverá visar aos serviços que englobem contextos delimitados inteiros. À medida que se sentir mais seguro e decidir separar esses serviços em serviços menores, lembre-se de que os agregados não devem ser separados – um microsserviço pode gerenciar um ou mais agregados, mas não queremos que um agregado seja gerenciado por mais de um microsserviço.

Tartarugas até o fim

No início, você provavelmente identificará uma série de contextos

delimitados mais gerais. Entretanto, esses contextos delimitados, por sua vez, podem conter outros contextos delimitados. Por exemplo, poderíamos decompor o depósito em funcionalidades associadas a atendimento de pedidos, gerenciamento de estoque e recepção de mercadorias. Ao considerar as fronteiras de seus microsserviços, pense inicialmente em contextos maiores, mais gerais, e então os subdivida nesses contextos aninhados quando quiser desfrutar as vantagens de separá-los nessas linhas.

Um truque, nesse caso, é que, mesmo que você decida separar um serviço que modele um contexto delimitado completo em serviços menores mais tarde, será possível ocultar essa decisão do mundo externo – talvez apresentando uma API mais genérica aos consumidores. A decisão de decompor um serviço em partes menores é, sem dúvida, uma decisão de implementação, portanto podemos ocultá-la se quisermos. Na Figura 2.16, vemos um exemplo desse caso. Separamos Depósito em Estoque e Expedição. No que concerne ao mundo externo, continua havendo somente um microsserviço Depósito. Internamente, porém, decompomos o serviço para permitir que Estoque gerencie *Itens do Estoque*, e temos Expedição gerenciando *Envio de Produtos*. Lembre-se de que queremos que um agregado individual continue pertencendo a um único microsserviço.

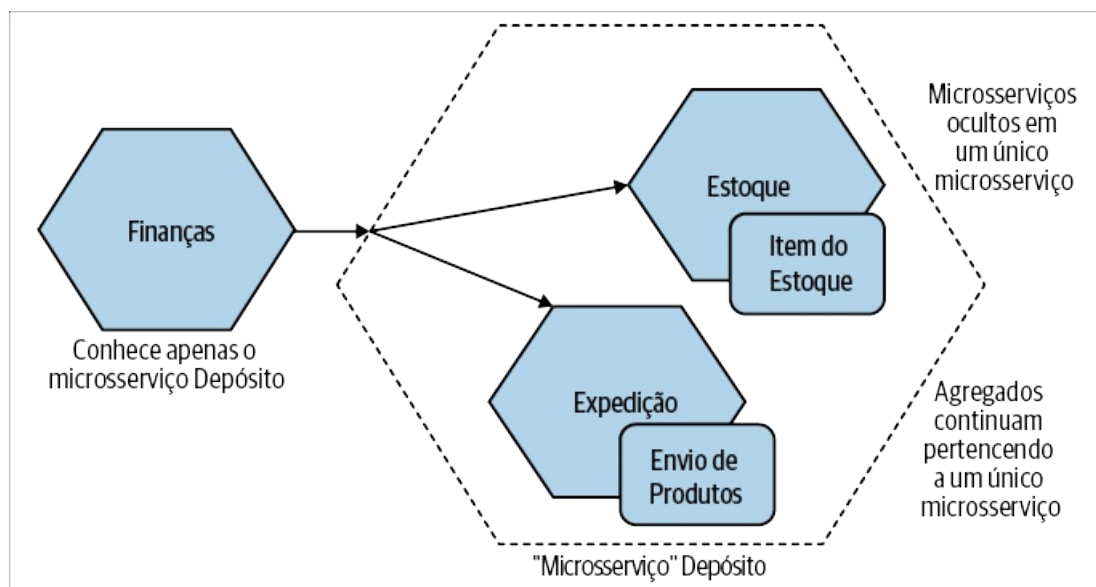


Figura 2.16 – O serviço Depósito foi internamente dividido nos microsserviços

Essa é outra forma de ocultar informações – ocultamos uma decisão acerca da implementação interna de modo que, se esse detalhe de implementação mudar novamente no futuro, nossos consumidores não saberão.

Outro motivo para preferir a abordagem aninhada poderia ser a possibilidade de dividir sua arquitetura em partes para simplificar os testes. Por exemplo, ao testar os serviços que consomem o serviço `Depósito`, não será necessário criar stubs para cada serviço no contexto do depósito – apenas para as APIs de nível mais alto. Isso também pode proporcionar a você uma unidade de isolamento ao considerar testes em escopos maiores. Eu poderia, por exemplo, tomar a decisão de fazer testes fim a fim nos quais iniciaria todos os serviços no contexto do depósito, mas, para todos os demais colaboradores, poderia criar stubs. Exploraremos melhor os testes e o isolamento no Capítulo 9.

Event storming

Event storming (tempestade de eventos), uma técnica desenvolvida por Alberto Brandolini, é um exercício colaborativo de brainstorming criado para ajudar na descoberta de um modelo de domínio. Em vez de termos um arquiteto sentado em um canto criando a sua própria representação do que seria o modelo de domínio¹⁴, o event storming reúne stakeholders que são tanto da área técnica como aqueles que não são, em um exercício conjunto. A ideia é que, ao fazer do desenvolvimento do modelo de domínio uma atividade conjunta, você acabará com uma visão de mundo conjunta e compartilhada.

A essa altura, vale a pena mencionar que, embora os modelos de domínio definidos durante um event storming possam ser usados para implementar sistemas orientados a eventos – e, com efeito, o mapeamento é muito simples – um modelo de domínio como esse também pode ser empregado para criar um sistema mais orientado a requisições/respostas.

Logística

Alberto tem visões muito específicas acerca de como um event storming

deve ser conduzido, e eu absolutamente concordo com alguns desses pontos. Em primeiro lugar, reúna todos em uma sala. Com frequência, esse é o passo mais difícil – conseguir que as agendas das pessoas se alinhem pode ser um problema, assim como encontrar uma sala espaçosa o suficiente. Esses problemas realmente ocorriam em um mundo pré-COVID, mas, como estou escrevendo este livro durante o lockdown relacionado ao vírus na Inglaterra, sei que esse passo poderá ser mais problemático ainda no futuro. O segredo, porém, é fazer com que todos os stakeholders estejam presentes ao mesmo tempo. Deve haver representantes de todas as partes do domínio que você planeja modelar: usuários, especialistas no assunto, donos de produto (product owners) – quem quer que esteja mais bem posicionado para ajudar a representar cada parte do domínio.

Assim que todos estiverem reunidos em uma sala, Alberto sugere que todas as cadeiras sejam removidas para garantir que todos fiquem em pé e se envolvam. Por ser uma pessoa com problema nas costas, embora essa estratégia seja algo que compreendo, reconheço que talvez ela não funcione para todos. Um aspecto com o qual concordo com Alberto é a necessidade de haver bastante espaço para que a modelagem possa ser feita. Uma solução comum é fixar folhas grandes de papel pardo nas paredes da sala, permitindo que todas as paredes sejam usadas para capturar informações.

A principal ferramenta de modelagem são notinhas adesivas para capturar os diversos conceitos, com notinhas de cores diferentes para representar diferentes conceitos.

Processo

O exercício começa com os participantes identificando os *eventos do domínio*. Esses eventos representam o que acontece no sistema – são os fatos que interessam a você. “Pedido Feito” seria um evento no qual estaríamos interessados no contexto da MusicCorp, assim como “Pagamento Recebido”. Esses eventos são capturados com notinhas adesivas laranja. É nesse ponto que discordo mais uma vez da estrutura de Alberto, pois os eventos são, de longe, os itens mais numerosos a serem capturados, e notinhas adesivas de cor laranja são surpreendentemente

difíceis de conseguir.¹⁵

Em seguida, os participantes identificam os comandos que fazem esses eventos acontecerem. Um comando é uma decisão tomada por um ser humano (um usuário de um software) para fazer algo. Nesse caso, estamos tentando entender a fronteira do sistema e identificar os principais atores humanos. Os comandos são identificados com notinhas adesivas azuis.

Os profissionais técnicos na sessão de event storming devem ouvir o que seus colegas que não são da área técnica têm a dizer nesse caso. Uma parte essencial desse exercício é não deixar que nenhuma implementação atual turve a percepção acerca do que é o domínio (a implementação virá depois). Nessa etapa, queremos criar um espaço no qual possamos tirar os conceitos que estão na mente dos principais stakeholders, trazendo-os à tona.

Com os eventos e os comandos identificados, a seguir, temos os agregados. Os eventos que temos nessa etapa são úteis não só para compartilhar o que acontece no sistema, mas também para começar a esclarecer quais são os possíveis agregados. Pense no evento de domínio que mencionamos antes, “Pedido Feito”. O nome, nesse caso – “Pedido” –, poderia muito bem ser um agregado em potencial. E “Feito” descreve algo que pode acontecer com um pedido, portanto poderia ser parte do ciclo de vida do agregado. Os agregados são representados por notinhas adesivas amarelas, e os comandos e eventos associados a esse agregado são movidos e agrupados em torno do agregado. Isso também ajudará você a entender como os agregados se relacionam entre si – os eventos de um agregado podem disparar um comportamento em outro.

Com os agregados identificados, eles são agrupados em contextos delimitados. Os contextos delimitados geralmente obedecem à estrutura organizacional de uma empresa, e os participantes do exercício estão em uma boa posição para entender quais agregados são usados por quais partes da organização.

Há mais detalhes sobre um event storming além dos que acabei de descrever – o objetivo foi apenas apresentar uma visão geral rápida. Para ver mais detalhes sobre o event storming, sugiro que você leia o livro

(atualmente em progresso) *EventStorming* de Alberto Brandolini (Leanpub).¹⁶

Design Orientado a Domínios no caso dos microsserviços

Exploramos como o DDD pode funcionar no contexto dos microsserviços, então vamos sintetizar como essa abordagem seria conveniente para nós.

Em primeiro lugar, grande parte do que faz o DDD ser tão eficaz é que os contextos delimitados, muito importantes no DDD, dizem respeito explicitamente a ocultar informações – proporcionam uma fronteira clara em relação ao sistema mais amplo, ao mesmo tempo que ocultam a complexidade interna, possibilitando que haja alterações sem causar impactos em outras partes do sistema. Isso significa que, quando seguimos uma abordagem com o DDD, independentemente de percebermos ou não, adotamos a estratégia de ocultar informações – e, como já vimos, isso é essencial para ajudar a identificar fronteiras estáveis para os microsserviços.

Em segundo lugar, o foco na definição de uma linguagem comum e ubíqua ajuda bastante na definição de endpoints para microsserviços. De forma conveniente, isso nos fornece um vocabulário compartilhado ao qual poderemos recorrer ao criar APIs, formatos para eventos, e itens relacionados. Também ajuda a resolver o problema de definir até que ponto deve ir a padronização das APIs no que concerne a permitir uma mudança de linguagem dentro dos contextos delimitados – uma mudança dentro de uma fronteira que cause impactos nessa própria fronteira.

As mudanças que implementamos em nosso sistema muitas vezes estão relacionadas a mudanças que a empresa deseja fazer no modo como o sistema se comporta. Estamos modificando funcionalidades – recursos – expostos aos nossos clientes. Se nossos sistemas forem decompostos de acordo com os contextos delimitados que representam nosso domínio, qualquer mudança que quisermos fazer provavelmente estará isolada dentro das fronteiras de um único microsserviço. Isso reduzirá a quantidade de lugares em que uma mudança terá de ser feita e permitirá que essa mudança seja implantada com rapidez.

Basicamente o DDD coloca o domínio de negócios no coração do software que desenvolvemos. O incentivo que ele nos dá em inserir a linguagem dos negócios em nosso código e no design dos serviços ajuda a aumentar a expertise sobre o domínio entre as pessoas que desenvolvem o software. Por sua vez, isso ajuda a melhorar a compreensão e aumentar a empatia pelos usuários de nosso software, além de aprimorar a comunicação entre as equipes técnicas de entrega, de desenvolvimento de produtos e os usuários finais. Se você estiver interessado em seguir em direção a equipes alinhadas a fluxos (stream-aligned teams), o DDD se encaixa perfeitamente como um método que ajudará a alinhar a arquitetura técnica com a estrutura organizacional mais ampla. Em um mundo no qual estamos cada vez mais tentando eliminar os silos entre TI e “os negócios”, essa não é uma escolha ruim.

Alternativas às fronteiras baseadas em domínios de negócios

Conforme apresentado, o DDD pode ser extremamente conveniente ao criar arquiteturas de microsserviços, porém seria um erro pensar que essa é a única técnica que você deveria considerar para determinar as fronteiras dos microsserviços. De fato, muitas vezes utilizo vários métodos em conjunto com o DDD para ajudar a identificar como (e se) um sistema deve ser dividido. Vamos analisar alguns dos outros fatores que poderíamos considerar ao identificar as fronteiras.

Volatilidade

Tenho ouvido cada vez mais a respeito de uma resistência à decomposição orientada a domínios, muitas vezes por defensores da ideia de que a volatilidade deveria ser o principal fator determinante da decomposição. Uma decomposição baseada em volatilidade exige que você identifique as partes de seu sistema com alterações mais frequentes, e essas funcionalidades devem então ser extraídas em seus próprios serviços, facilitando trabalhar com elas. Conceitualmente, não tenho nenhum problema com esse método, porém promovê-lo como a única forma de trabalhar não ajuda, particularmente quando consideramos os diferentes

fatores que poderiam nos levar em direção aos microsserviços. Se meu principal problema estiver relacionado à necessidade de escalar minha aplicação, por exemplo, é improvável que uma decomposição baseada em volatilidade possa trazer muitas vantagens.

A mentalidade que justifica a decomposição baseada em volatilidade também se torna evidente em abordagens como de TI bimodal. Um conceito definido pela Gartner, o TI bimodal separa organizadamente o mundo nas categorias chamadas de “Modo 1” (também conhecido como Sistemas de Registro) e “Modo 2” (também conhecido como Sistemas de Inovação), com base na rapidez (ou lentidão) com que diferentes sistemas devem progredir. Sistemas de Modo 1 não mudam muito e não precisam de muito envolvimento dos negócios – ao menos, é o que dizem. O Modo 2 é onde se encontra a ação, com sistemas que precisam mudar rapidamente e exigem um envolvimento mais próximo dos negócios. Deixando de lado por um instante a extrema simplificação inerente a um esquema de classificação como esse, a abordagem implica também uma visão bastante imutável do mundo, e vai contra os tipos de transformações evidentes em todo o mercado à medida que as empresas procuram “ser digitais”. Partes dos sistemas das empresas que não precisavam mudar muito no passado de repente passaram a exigir mudanças, a fim de criar oportunidades no mercado e oferecer serviços aos clientes, de maneiras que essas empresas não haviam imaginado antes.

Vamos voltar para a MusicCorp. Sua primeira incursão naquilo que atualmente chamamos de digital foi simplesmente ter uma página web; tudo que ela oferecia no passado, em meados dos anos 1990, era uma lista do que estava à venda, mas era necessário telefonar para a MusicCorp para fazer um pedido. A página web era pouco mais que um anúncio de jornal. Então, as compras online passaram a ganhar importância, e todo o depósito, que até então era exclusivamente gerenciado em papel, precisou ser digitalizado. Quem sabe a MusicCorp, em algum momento, terá de considerar disponibilizar as músicas digitalmente! Embora você possa considerar que a MusicCorp esteja ultrapassada, ainda será possível apreciar o nível da revolução pela qual as empresas têm passado à medida que compreendem como as mudanças de tecnologia e o comportamento

dos clientes podem exigir alterações significativas em partes de um negócio, as quais não poderiam ter sido previstas facilmente.

Não gosto do TI bimodal como conceito, pois passa a ser uma maneira de as pessoas jogarem partes que são difíceis de alterar em uma bela caixa e dizer que “não precisamos lidar com os problemas que estão aí – esse é o Modo 1”. É outro modelo que uma empresa pode adotar para garantir que nada precise realmente mudar. Além disso, faz com que as empresas não encarem o fato de que, com muita frequência, mudanças em funcionalidades exigem mudanças nos “Sistemas de Registro” (Modo 1) para permitir que alterações sejam feitas nos “Sistemas de Inovação” (Modo 2). Com base em minha experiência, posso dizer que as empresas que adotam o TI bimodal acabam tendo duas velocidades – lenta e mais lenta.

Para ser justo com os proponentes da decomposição baseada em volatilidade, muitos não recomendam necessariamente esses modelos simplistas como o TI bimodal. Com efeito, acho essa técnica extremamente útil para ajudar a determinar fronteiras quando o fator determinante diz respeito a alcançar mais rapidamente o mercado – extrair as funcionalidades que estão mudando ou precisam mudar com frequência faz todo o sentido em uma situação como essa. Mas, novamente, é o objetivo que determinará o método mais apropriado.

Dados

A natureza dos dados que você armazena e gerencia pode direcioná-lo para diferentes formas de decomposição. Por exemplo, você pode querer limitar os serviços que lidam com PII (Personally Identifiable Information, ou Informações de Identificação Pessoal), tanto para reduzir o risco de violações nos dados como para simplificar o gerenciamento e a implementação de algo como o GDPR.

Em um de meus clientes recentes – uma empresa de pagamento que chamaremos de PaymentCo – o uso de determinados tipos de dados influenciou diretamente nas decisões que tomamos acerca da decomposição do sistema. A PaymentCo lida com dados de cartões de

crédito, o que significa que seu sistema precisa atender a vários requisitos definidos pelos padrões do PCI (Payment Card Industry, ou Mercado de Cartões para Pagamento) sobre como esses dados devem ser gerenciados. Como parte dessa conformidade, o sistema e os processos da empresa precisavam ser auditados. A PaymentCo tinha necessidade de lidar com dados completos de cartões de crédito, e com um volume que implicava que seu sistema devia estar em conformidade com o PCI Nível 1; esse é o nível mais rigoroso, exigindo avaliações trimestrais externas dos sistemas e práticas relacionados ao modo como os dados são administrados.

Muitos dos requisitos do PCI fazem parte do senso comum, porém garantir que todo o sistema está em conformidade com esses requisitos, sem falar na necessidade de o sistema precisar ser auditado por uma entidade externa, estava se provando ser muito custoso. Consequentemente, a empresa queria separar a parte do sistema que lidava com os dados completos de cartões de crédito – isso significava que apenas um subconjunto do sistema exigiria esse nível adicional de auditoria. Na Figura 2.17, vemos uma forma simplificada do design que criamos. Os serviços atuando na área verde (cercados por uma linha verde pontilhada) jamais veem nenhuma informação sobre cartões de crédito – esses dados estão limitados aos processos (e redes) na área vermelha (cercada por uma linha vermelha tracejada). O gateway desvia chamadas aos serviços apropriados (e à zona apropriada); quando informações de cartões de crédito passarem por esse gateway, eles estarão efetivamente na zona vermelha.

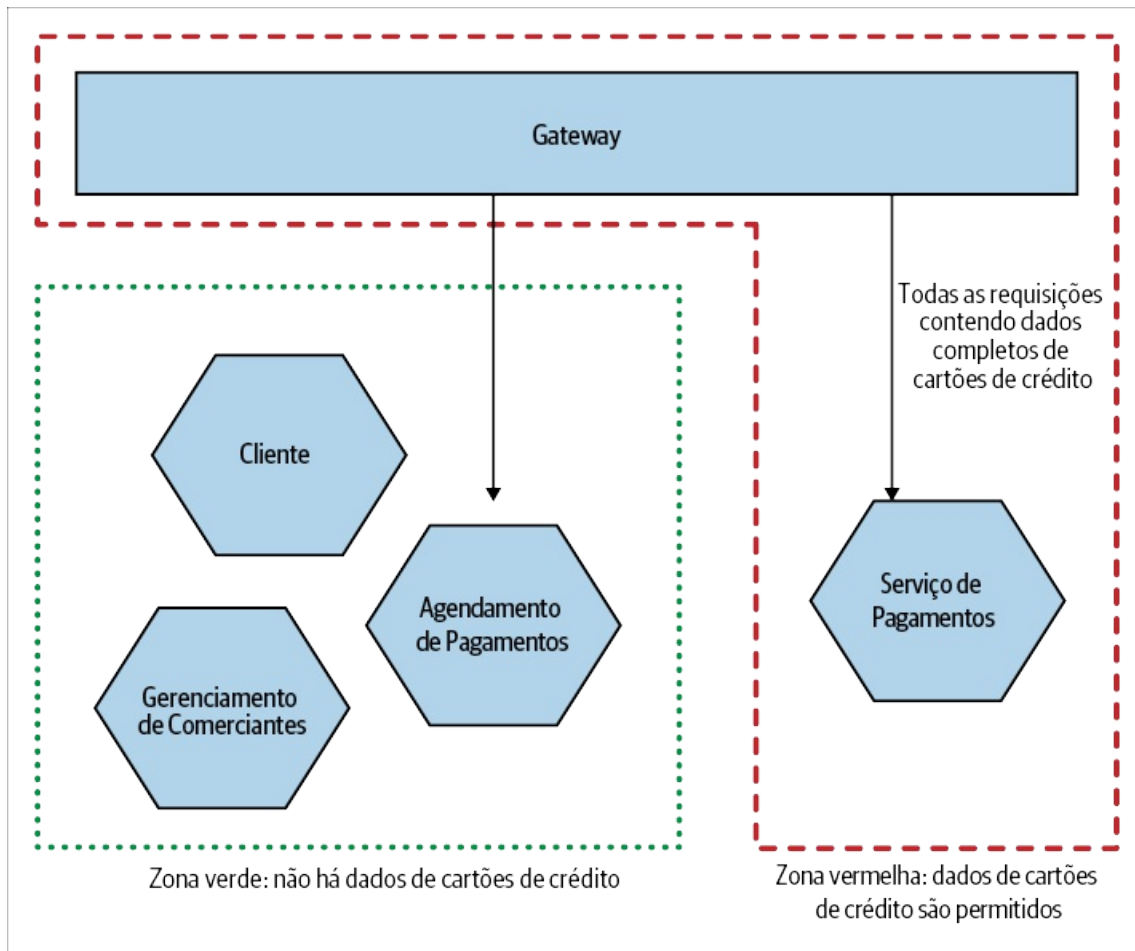


Figura 2.17 – PaymentCo, que segrega processos com base no uso de informações de cartões de crédito a fim de limitar o escopo dos requisitos de PCI.

Como as informações de cartões de crédito nunca fluem para a zona verde, todos os serviços nessa área estarão isentos de uma auditoria de PCI completa. Os serviços na área vermelha estão no escopo de uma auditoria desse tipo. Ao trabalhar no design, fizemos tudo que podíamos para limitar o que deveria estar na zona vermelha. É essencial observar que tivemos de garantir que informações de cartões de crédito jamais deveriam fluir para a zona verde – se um microserviço na zona verde pudesse requisitar essas informações, ou elas pudessem ser enviadas para a zona verde por um microserviço da zona vermelha, as linhas claras de separação seriam rompidas.

A segregação de dados é frequentemente determinada por uma série de

preocupações com privacidade e segurança; retomaremos esse assunto e o exemplo com a PaymentCo no Capítulo 11.

Tecnologia

A necessidade de fazer uso de diferentes tecnologias também pode ser um fator no que diz respeito à determinação de uma fronteira. Podemos acomodar diferentes bancos de dados em um único microsserviço em execução, mas, se quisermos misturar diferentes modelos de runtime, poderemos enfrentar um problema. Se você determinar que parte de suas funcionalidades precisa ser implementada em um runtime como o Rust, o qual possibilita um desempenho adicional, esse passará a ser um fator determinante significativo.

É claro que temos de estar cientes da direção em que podemos ser levados caso isso seja adotado como um método geral de decomposição. A arquitetura clássica de três camadas que discutimos no capítulo inicial, e que será exibida novamente na Figura 2.18, é um exemplo de tecnologias relacionadas que estão agrupadas. Conforme já exploramos, essa, geralmente, não é uma arquitetura ideal.

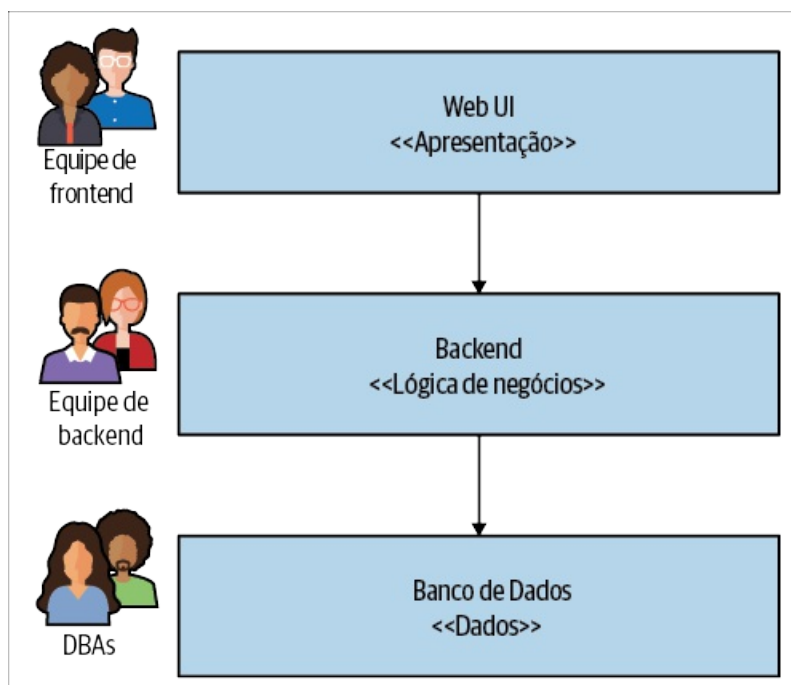


Figura 2.18 – Uma arquitetura tradicional de três camadas muitas vezes é

determinada por fronteiras tecnológicas.

Estrutura organizacional

Conforme já foi dito quando apresentei a lei de Conway no Capítulo 1, há uma interação inerente entre a estrutura organizacional e a arquitetura de sistema que você acabará criando. Além dos estudos que mostram essa ligação, com base em minha própria experiência, já vi isso em ação repetidamente. O modo como você se organiza acaba direcionando a arquitetura de seus sistemas, para o bem ou para o mal. Quando se trata de nos ajudar a definir as fronteiras de nossos serviços, devemos considerar esse aspecto como uma parte importante de nossa tomada de decisões.

É improvável que definir uma fronteira de serviço cuja responsabilidade perpassasse várias equipes diferentes gere o resultado que desejamos – como iremos explorar melhor no Capítulo 15, uma responsabilidade compartilhada nos microsserviços é uma questão complicada. Segue-se daí que devemos levar em consideração a estrutura organizacional existente ao considerar onde e quando definir fronteiras e, em algumas situações, talvez devêssemos considerar até mesmo modificar a estrutura organizacional a fim de dar suporte à arquitetura que queremos.

É claro que devemos considerar também o que acontecerá caso a nossa estrutura organizacional mude. Isso significa que teremos agora de redefinir a arquitetura de nosso software? Bem, no pior caso, essa situação poderia nos obrigar a analisar um microsserviço existente, que agora precisará ser dividido, pois contém uma funcionalidade cuja responsabilidade será de duas equipes distintas, quando, antes, uma única equipe era responsável por ambas as partes. Por outro lado, com frequência, mudanças organizacionais exigiriam que somente o responsável por um microsserviço existente mudasse. Considere uma situação na qual a equipe responsável pelas operações do depósito lidasse antes também com funcionalidades de descobrir quantos itens devem ser pedidos aos fornecedores. Vamos supor que tenhamos decidido passar essa responsabilidade para uma equipe dedicada de previsão de reposição, que deseja extrair informações das vendas atuais e das promoções planejadas para definir o que precisa ser pedido. Se a equipe de depósito tivesse um

microserviço Pedido para Fornecedores dedicado, este poderia ser simplesmente transferido para a nova equipe de previsões. Por outro lado, se essa funcionalidade estivesse anteriormente integrada em um sistema de escopo mais amplo, cuja responsabilidade fosse do depósito, talvez ela precisasse ser separada.

Mesmo quando trabalhamos em uma estrutura organizacional existente, há o perigo de não traçarmos nossas fronteiras nos lugares corretos. Muitos anos atrás, alguns colegas e eu estávamos trabalhando com um cliente na Califórnia, ajudando a empresa a adotar algumas práticas mais limpas de programação e caminhar em direção a testes mais automatizados. Havíamos começado com algumas tarefas mais acessíveis, como a decomposição de serviços, quando percebemos algo muito mais preocupante. Não posso dar muitos detalhes sobre o que a aplicação fazia, mas era uma aplicação voltada para o público, com uma base de clientes grande e global.

A equipe – e o sistema – havia crescido. O sistema originalmente refletia a visão de uma única pessoa, porém mais e mais funcionalidades haviam sido agregadas, e havia um número cada vez maior de usuários. Em algum momento, a empresa decidiu aumentar a capacidade da equipe, fazendo com que um novo grupo de desenvolvedores no Brasil assumisse parte do trabalho. O sistema foi dividido, com a metade frontal da aplicação sendo essencialmente stateless (sem estados), implementando o site voltado ao público, como vemos na Figura 2.19. A outra metade do sistema era simplesmente uma interface RPC (Remote Procedure Call, ou Chamada de Procedimento Remoto) para um sistema de armazenamento de dados. Imagine, basicamente, que você tenha tomado a camada de repositório de sua base de código e tenha feito dela um serviço separado.

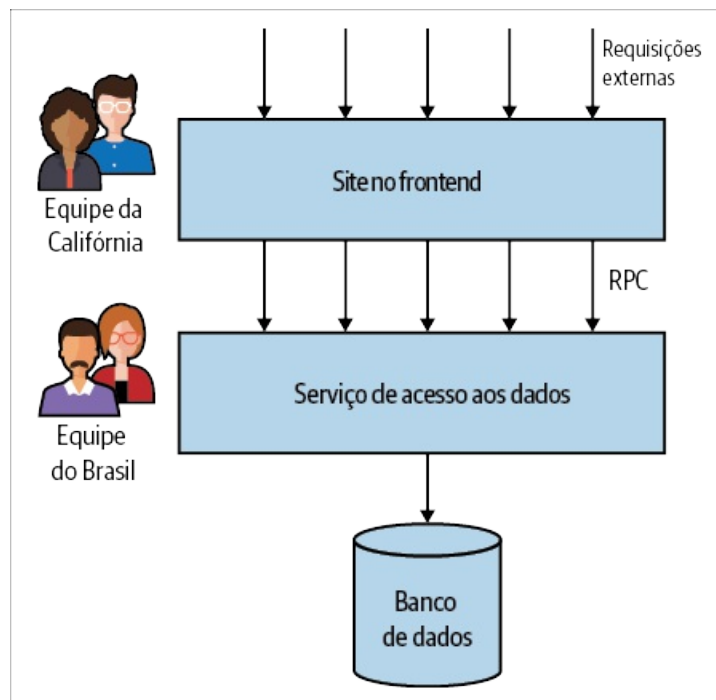


Figura 2.19 – Separação da fronteira de um serviço com base em fronteiras técnicas.

Mudanças tinham de ser feitas frequentemente nos dois serviços. Ambos os serviços falavam por meio de chamadas de métodos do tipo RPC, de baixo nível, que eram excessivamente frágeis (discutiremos melhor esse assunto no Capítulo 4). Havia muita comunicação na interface dos serviços também, resultando em problemas de desempenho. Isso levou à necessidade de criar mecanismos mais sofisticados de batching de RPC. Chamei isso de “arquitetura cebola”, pois ela tinha muitas camadas e me faziam chorar quando tínhamos de atravessá-las.

Diante disso, a ideia de separar o sistema anteriormente monolítico em linhas geográficas/organizacionais faz todo sentido, conforme detalharemos melhor no Capítulo 15. Nesse caso, porém, em vez de definir uma fatia vertical na pilha, com foco nos negócios, a equipe tomou o que era antes uma API interna ao processo e criou uma fatia horizontal. Um modelo mais conveniente teria sido a equipe da Califórnia ser responsável por uma fatia vertical completa, composta de partes relacionadas do frontend e da funcionalidade de acesso aos dados, com a equipe no Brasil assumindo a responsabilidade por outra fatia.

Camadas internas versus camadas externas

Como espero que você possa perceber a essa altura, não sou fã de arquiteturas com camadas horizontais. No entanto, a divisão em camadas pode ter sua relevância. Dentro das fronteiras de um microsserviço, pode ser totalmente razoável delinear diferentes camadas para deixar o código mais fácil de ser gerenciado. O problema ocorre quando essa divisão em camadas passa a ser o método pelo qual as fronteiras de seu microsserviço e as responsabilidades são determinadas.

Combinando modelos e exceções

Espero que, a essa altura, esteja claro que não sou dogmático acerca do modo como você determina essas fronteiras. Se seguir as diretrizes relacionadas à ocultação de informações e apreciar a interação entre acoplamento e coesão, é provável que você evitará algumas das piores armadilhas de qualquer que seja o método que escolher. Apenas acho que, ao manter o foco nessas ideias, é *mais* provável que você acabará com uma arquitetura orientada a domínios, mas isso nem sempre ocorre. O fato, porém, é que, muitas vezes, pode haver motivos para combinar modelos, mesmo que “orientado a domínios” seja a opção que você decidiu escolher como seu principal método para definição das fronteiras dos microsserviços.

Os diferentes métodos que apresentamos até agora também têm muito potencial para interações. Ser restrito demais em suas opções fará você seguir um dogma, em vez de fazer o que é certo. Uma decomposição baseada em volatilidade pode fazer muito sentido se seu foco estiver em aumentar a velocidade de entrega; porém, se isso fizer com que você extraia um serviço que cruze fronteiras organizacionais, espere que seu ritmo de mudanças vá sofrer as consequências em virtude da desavença nas entregas (delivery contention).

Eu poderia definir um belo serviço `Depósito` com base em minha compreensão do domínio de negócios, mas se uma parte desse sistema tiver de ser implementada em C++ e outra parte em Kotlin, será necessário

decompô-lo com base nessas fronteiras técnicas.

As fronteiras de serviço organizacionais e orientadas a domínio são o meu ponto de partida. Contudo, essa é apenas a minha abordagem padrão. Em geral, uma série de fatores que apresentei neste capítulo entram em cena, e aqueles que influenciarão suas decisões dependerão dos problemas que você estiver tentando resolver. Você precisa observar as próprias circunstâncias para determinar o que funcionará melhor – espero que eu tenha conseguido apresentar algumas opções diferentes para você considerar. Lembre-se apenas de que, se alguém disser “A única maneira de fazer isso é X!”, é provável que essa pessoa esteja apenas vendendo mais um dogma. Você consegue fazer melhor do que isso.

Depois de dizer tudo isso, vamos explorar melhor o tópico da modelagem de domínios, explorando o design orientado a domínios com um pouco mais de detalhes.

Resumo

Neste capítulo, conhecemos um pouco mais a respeito do que é uma boa fronteira para microsserviços, e como descobrir as linhas de separação no domínio de nosso problema, que nos proporcionem o duplo benefício de baixo acoplamento e forte coesão. Ter uma compreensão minuciosa de nosso domínio pode ser uma ferramenta essencial para nos ajudar a determinar essas linhas; ao alinhar nossos microsserviços com essas fronteiras, garantimos que o sistema resultante tenha todas as chances de manter essas virtudes intactas. Também tivemos uma pista de como podemos subdividir os nossos microsserviços.

As ideias apresentadas no livro *Domain-Driven Design* de Eric Evans¹⁷ são muito úteis para nós ao determinar fronteiras que sejam razoáveis para os nossos serviços, e eu abordei o assunto apenas superficialmente – o livro de Eric dá muito mais detalhes. Se quiser mais informações, posso recomendar o livro *Implementing Domain-Driven Design*¹⁸ de Vaughn Vernon para ajudar a entender os aspectos práticos dessa abordagem, enquanto *Domain-Driven Design Distilled*¹⁹ de Vernon apresenta uma ótima visão geral condensada, caso você esteja à procura de algo mais

rápido.

Boa parte deste capítulo descreveu como podemos determinar as fronteiras de nossos microsserviços. No entanto, o que acontecerá se já tivermos uma aplicação monolítica e quisermos fazer uma migração para uma arquitetura de microsserviços? Esse é um assunto que exploraremos com mais detalhes no próximo capítulo.

-
- 1 David, Parnas, “On the Criteria to Be Used in Decomposing Systems into Modules” (Sobre os critérios a serem usados para decompor sistemas em módulos), (contribuição para uma revista acadêmica, Universidade Carnegie Mellon, 1971), <https://oreil.ly/BnVVg>.
 - 2 O ponto de partida óbvio é o resumo de Adrian de “On the Criteria...” (Sobre os critérios..., <https://oreil.ly/cCtSV>), mas a abordagem de Adrian sobre os trabalhos anteriores de Parnas, “Information Distribution Aspects of Design Methodology” (Aspectos da distribuição de informações na metodologia de design, <https://oreil.ly/6JyKv>), contém alguns insights muito bons, além de comentários do próprio Parnas.
 - 3 Parnas, “Information Distribution Aspects” (Aspectos da distribuição de informações).
 - 4 Infelizmente não consegui descobrir a fonte original dessa definição.
 - 5 Em meu livro *Monolith to Microservices* (O’Reilly) [N.T.: Edição publicada no Brasil: *Migrando sistemas monolíticos para microsserviços* (Novatec, 2020)], atribuí essa citação ao próprio Larry Constantine. Embora a afirmação sintetize bem boa parte de seu trabalho nessa área, a citação, na verdade, deveria ser atribuída a Albert Endres e Dieter Rombach, de seu livro *A Handbook of Software and Systems Engineering* (Addison-Wesley) de 2003.
 - 6 Edward Yourdon e Larry L. Constantine, *Structured Design* (Nova York: Yourdon Press, 1976) [N.T.: Edição publicada no Brasil: *Projeto Estruturado de Sistemas* (Editora Campus, 1992)].
 - 7 Meilir Page-Jones, *The Practical Guide to Structured Systems Design* (Nova York: Yourdon Press Computing, 1980) [N.T.: Edição publicada no Brasil: *Projeto Estruturado de Sistemas* (McGraw-Hill, 1988)].
 - 8 Esse conceito é similar ao protocolo de domínio da aplicação, que define as regras segundo as quais os componentes interagem em um sistema baseado em REST.
 - 9 Acoplamento de passagem é o nome que dei para o que originalmente foi descrito como “acoplamento tramp” por Meilir Page-Jones em *The Practical Guide to Structured Systems Design* [N.T.: Edição publicada no Brasil: *Projeto Estruturado de Sistemas* (McGraw-Hill, 1988)]. Optei por usar um termo diferente nesse caso porque achei o termo original de certa forma problemático, não sendo extremamente significativo para um público-alvo mais amplo.
 - 10 Está bem, mais que uma ou duas vezes. *Muito* mais do que uma ou duas vezes...
 - 11 Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software* (Boston: Addison-Wesley, 2004) [N.T.: Edição publicada no Brasil: *Domain-Driven Design: atacando as complexidades no coração do software* (Alta Books, 2016)].
 - 12 Sei que algumas pessoas fazem objeção ao uso de templates de URI em sistemas REST, e entendo o motivo – eu apenas quis manter a simplicidade nesse exemplo.

- 13 Phil Calçado, “Pattern: Using Pseudo-URIs with Microservices” (Padrão: usando pseudo-URIs com microsserviços), <https://oreil.ly/xOYMr>.
- 14 Não quero ofender caso esse seja o seu caso – eu mesmo já fiz isso mais de uma vez.
- 15 Quero dizer, por que não amarelo? É a cor mais comum!
- 16 Alberto Brandolini, *EventStorming* (Victoria, BC: Leanpub, em breve).
- 17 N.T.: Edição publicada no Brasil: *Domain-Driven Design* (Alta Books, 2016).
- 18 Vaughn Vernon, *Implementing Domain-Driven Design* (Upper Saddle River, NJ: Addison-Wesley, 2013) [N.T.: Edição publicada no Brasil: *Implementando Domain-Driven Design* (Alta Books, 2016)].
- 19 Vaughn Vernon, *Domain-Driven Design Distilled* (Boston: Addison-Wesley, 2016).

Dividindo o sistema monolítico

Muitos de vocês que estão lendo este livro provavelmente não terão um quadro branco para fazer o design de seu sistema; mesmo que tivessem, começar com microsserviços poderia não ser uma boa ideia, pelos motivos que exploramos no Capítulo 1. Muitos já terão um sistema existente, talvez alguma forma de arquitetura monolítica, que queiram migrar para uma arquitetura de microsserviços.

Neste capítulo, apresentarei alguns passos iniciais, padrões e dicas em geral que ajudarão você no processo de transição para uma arquitetura de microsserviços.

Tenha um objetivo

Os microsserviços não são o objetivo. Você não “vence” por ter microsserviços. Adotar uma arquitetura de microsserviços deve ser uma decisão consciente, tomada de forma racional. Você deve pensar na migração para uma arquitetura de microsserviços somente se não puder encontrar uma maneira mais fácil de avançar em direção ao seu objetivo com sua arquitetura atual.

Sem uma clara compreensão do que está tentando alcançar, você poderá cair na armadilha de confundir atividade com resultado. Já vi equipes obcecadas com a criação de microsserviços, sem nem mesmo se perguntarem por quê. Isso é extremamente problemático, considerando as novas fontes de complexidade que os microsserviços podem introduzir.

Fixar-se nos microsserviços em vez de se concentrar no objetivo final também significa que você provavelmente vai parar de pensar em outras formas com as quais poderia concretizar as mudanças que está tentando fazer. Por exemplo, os microsserviços podem ajudar a escalar seu sistema;

porém, com frequência, há diversas técnicas alternativas para escalar e que deveriam ser consideradas antes. Disparar algumas cópias adicionais de seu sistema monolítico atual atrás de um balanceador de carga (load balancer) poderia muito bem ajudar a escalar seu sistema com muito mais eficácia, em vez de se sujeitar a um processo complexo e demorado de decomposição em microsserviços.



Trabalhar com microsserviços não é fácil. Tente primeiro as alternativas mais simples.

Por fim, sem um objetivo claro, será difícil saber por onde começar. Qual microsserviço deve ser criado antes? Sem uma compreensão geral do que você está tentando alcançar, seu voo será às cegas.

Portanto, saiba claramente qual é a mudança que você está tentando fazer e avalie maneiras mais fáceis de atingir esse objetivo antes de considerar os microsserviços. Se os microsserviços realmente forem a melhor maneira de seguir adiante, monitore seu progresso em relação ao seu objetivo e altere o curso conforme for necessário.

Migração gradual

Se você fizer uma reescrita Big Bang, a única coisa que poderá garantir é que terá um Big Bang.

– *Martin Fowler*

Se chegar ao ponto de decidir que dividir seu sistema monolítico atual é a atitude correta a ser tomada, recomendo fortemente que você o separe extraindo pequenas porções de cada vez. Uma abordagem gradual ajudará a conhecer os microsserviços à medida que você prosseguir, além de limitar o impacto de fazer algo errado (e você fará!). Pense em seu sistema monolítico como se fosse um bloco de mármore. Poderíamos explodir o bloco todo, mas isso raramente acabaria bem. Faz muito mais sentido ir apenas talhando, de maneira gradual.

Divida a grande jornada em vários passos pequenos. Podemos dar um passo de cada vez e aprender com eles. Se tivermos de dar um passo para

trás, será apenas um passo pequeno. De qualquer maneira, aprenderemos com ele, e o próximo passo a ser dado terá uma base mais sólida por causa dos passos que foram dados antes.

Uma divisão em partes menores também permite identificar vitórias rápidas e aprender com elas. Isso pode contribuir para que o próximo passo seja mais fácil e ajudará a ganhar força. Ao separar os microsserviços um de cada vez, também poderemos desfrutar gradualmente suas vantagens, em vez de ter de esperar por alguma implantação Big Bang.

Tudo isso leva àquilo que passou a ser meu conselho rotineiro para as pessoas que estão considerando os microsserviços: se você acha que os microsserviços são uma boa ideia, comece com algo pequeno. Escolha uma ou duas áreas de funcionalidades, implemente-as como microsserviços, faça com que sejam implantadas no ambiente de produção e reflita se a criação de seus novos microsserviços ajudou você a se aproximar de seu objetivo.



Você não apreciará o verdadeiro horror, a dor e o sofrimento que uma arquitetura de microsserviços pode proporcionar até executá-la no ambiente de produção.

O sistema monolítico raramente é o inimigo

Embora já tenha argumentado no início do livro que alguma forma de arquitetura monolítica possa ser uma opção totalmente válida, vale a pena repetir que uma arquitetura monolítica não é inerentemente *ruim* e, desse modo, não deve ser vista como o inimigo. Não mantenha o foco em “não ter um sistema monolítico”; em vez disso, concentre-se nas vantagens que você espera desfrutar com sua mudança de arquitetura.

É comum que a arquitetura monolítica atual continue existindo após uma mudança em direção aos microsserviços, embora, geralmente, com uma capacidade reduzida. Por exemplo, uma mudança para melhorar a capacidade da aplicação de lidar com mais carga poderia ser satisfeita com a remoção de 10% das funcionalidades que atualmente são um gargalo no sistema, deixando os 90% restantes no sistema monolítico.

Muitas pessoas acham que um sistema monolítico coexistindo com microsserviços pode ser uma realidade “confusa” – contudo, a arquitetura de um sistema que executa no mundo real nunca é totalmente limpa ou pura. Se quiser uma arquitetura “limpa”, coloque uma capa protetora em volta de uma versão impressa e idealizada da arquitetura de sistema que você poderia ter caso tivesse uma visão perfeita do futuro, além de investimentos ilimitados. Uma arquitetura de sistemas de verdade está em constante evolução, e ela deve se adaptar à medida que as necessidades e o conhecimento mudarem. A habilidade consiste em se acostumar com essa ideia, assunto que retomarei no Capítulo 16.

Ao fazer de sua migração para microsserviços uma jornada gradual, você será capaz de ir dividindo aos poucos a arquitetura monolítica atual, entregando melhorias durante o processo e, acima de tudo, saberá quando deve parar.

Em circunstâncias surpreendentemente raras, desativar o sistema monolítico poderá ser um requisito imutável. Com base em minha experiência, posso dizer que, com frequência, esse caso estará limitado a situações em que o sistema monolítico existente tem como base uma tecnologia morta ou que está morrendo, está associado a uma infraestrutura que precisa ser desativada ou talvez seja um sistema de terceiros caro, do qual você quer se livrar. Mesmo nessas situações, uma abordagem gradual para a decomposição se justifica pelos motivos que apresentei.

Perigos de uma decomposição prematura

Há perigos em criar microsserviços se não houver um entendimento claro do domínio. Um exemplo dos problemas que isso pode causar vem da empresa em que trabalhei antes, a Thoughtworks. Um de seus produtos era o Snap CI, uma ferramenta hospedada de integração contínua (continuous integration) e entrega contínua (continuous delivery) – esses conceitos serão discutidos no Capítulo 7. A equipe havia trabalhado antes em outra ferramenta semelhante, o GoCD – atualmente é uma ferramenta open source de entrega contínua que pode ser implantada localmente, em

vez de estar hospedada na nuvem.

Embora tivesse havido um pouco de reutilização de código bem no início entre os projetos Snap CI e GoCD, no final, o Snap CI acabou com uma base de código totalmente nova. Apesar disso, a experiência anterior da equipe no domínio de ferramentas de CD os encorajou a agir mais rapidamente na identificação das fronteiras e a construir seu sistema como um conjunto de microsserviços.

Depois de alguns meses, porém, ficou claro que os casos de uso do Snap CI eram sutilmente diferentes, de modo que a primeira definição das fronteiras dos serviços não estava muito correta. Isso resultou em muitas mudanças que tiveram de ser feitas envolvendo vários serviços, com um custo alto associado às mudanças. Posteriormente, a equipe reuniu os serviços novamente em um sistema monolítico, dando tempo aos membros da equipe para entenderem melhor os locais em que as fronteiras deveriam estar. Um ano depois, a equipe pôde então separar o sistema monolítico em microsserviços, cujas fronteiras se mostraram ser muito mais estáveis. Este não é, nem de longe, o único exemplo que já vi dessa situação. Decompor um sistema em microsserviços de modo prematuro pode ser custoso, particularmente se o domínio for uma novidade para você. Em muitos aspectos, ter uma base de código existente que você queira decompor em microsserviços é muito mais fácil do que tentar adotar os microsserviços desde o princípio, exatamente por esse motivo.

O que separar antes?

Assim que tiver um bom domínio do *porquê* de achar que os microsserviços são uma boa ideia, você pode usar esse conhecimento para ajudar a priorizar os microsserviços que devem ser criados antes. Quer escalar a aplicação? Uma funcionalidade que atualmente limite a capacidade do sistema de lidar com a carga estará no topo da lista. Quer melhorar o tempo para alcançar o mercado? Observe a volatilidade do sistema a fim de identificar as funcionalidades que mudam com mais frequência e veja se elas funcionariam como microsserviços. Você pode usar ferramentas de análise estática como o CodeScene

(<https://www.codescene.com>) para identificar rapidamente as partes voláteis de sua base de código. Podemos ver um exemplo de uma visualização no CodeScene na Figura 3.1, na qual vemos hotspots do projeto open source Apache Zookeeper.

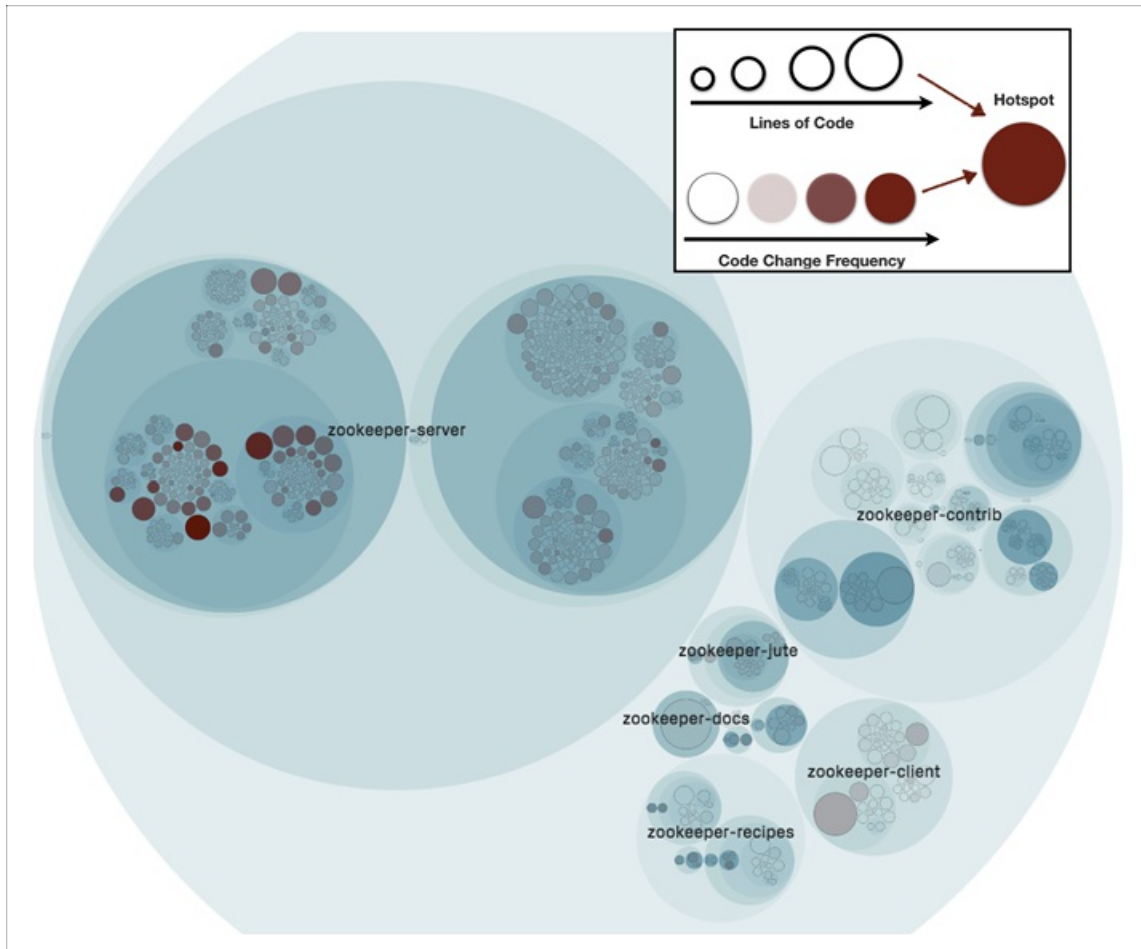


Figura 3.1 – Visualização de hotspots no CodeScene, ajudando a identificar partes da base de código que mudam com frequência.

No entanto, é necessário considerar também quais decomposições são viáveis. Algumas funcionalidades podem estar tão entranhadas na aplicação monolítica existente a ponto de ser impossível ver como ela poderia ser desembaraçada. Ou, quem sabe, a funcionalidade em questão é tão crítica à aplicação que qualquer mudança seria considerada de alto risco. Por outro lado, a funcionalidade que você quer migrar talvez já esteja, de certo modo, autocontida, e a extração pode parecer muito simples.

Basicamente, a decisão sobre qual funcionalidade deve ser separada em um microsserviço acabará sendo o resultado do equilíbrio entre essas duas forças – a facilidade da extração *versus* a vantagem de extrair o microsserviço, antes de tudo.

Meu conselho para os primeiros microsserviços seria escolher partes que estejam um pouco mais inclinadas à extremidade “fácil” do espectro – um microsserviço que achamos que causará algum impacto para atingir o nosso objetivo global, certamente, porém algo que considerariamos ser uma conquista fácil. Em uma transição como essa, sobretudo em uma que poderia demorar meses ou anos, é importante ganhar impulso rapidamente. Portanto você precisará de algumas conquistas rápidas na manga.

Por outro lado, se tentar extrair o que você considera ser o microsserviço mais fácil e não conseguir fazê-lo funcionar, talvez valha a pena reconsiderar se os microsserviços são realmente uma opção correta para você e a sua empresa.

Com alguns sucessos e algumas lições aprendidas, você estará em uma posição muito melhor para enfrentar extrações mais complexas, que também poderão envolver funcionalidades em áreas mais críticas.

Decomposição por camadas

Então, você identificou o primeiro microsserviço a ser extraído; e agora? Bem, podemos dividir mais ainda essa decomposição em passos menores.

Se considerarmos as três camadas tradicionais de uma pilha de serviços web, podemos olhar para a funcionalidade que queremos extrair com base em sua interface de usuário, o código da aplicação no backend e os dados.

O mapeamento de um microsserviço para uma interface de usuário em geral não é de 1:1 (esse é um assunto que exploraremos com muito mais detalhes no Capítulo 14). Assim, extrair uma funcionalidade de interface de usuário relacionada ao microsserviço poderia ser considerado um passo separado. Farei uma advertência nesse caso, a respeito de ignorar a parte referente à interface de usuário da equação. Cansei de ver empresas

olharem apenas para as vantagens de decompor a funcionalidade no backend, o que, com frequência, resulta em uma abordagem com muitos silos em qualquer reestruturação de arquitetura. Às vezes, as maiores vantagens podem vir da decomposição da UI; portanto, ignore-a por sua conta e risco. Muitas vezes, a decomposição da UI tende a ficar atrás da decomposição do backend em microsserviços, pois até os microsserviços estarem disponíveis, é difícil ver as possibilidades para a decomposição da UI; apenas se certifique de que a decomposição da UI não fique muito atrás.

Se olharmos para o código do backend e para o sistema de armazenamento relacionado, é essencial que ambos estejam no escopo ao extrair um microsserviço. Vamos considerar a Figura 3.2, na qual queremos extrair uma funcionalidade relacionada ao gerenciamento da lista de desejos (wishlist) de um cliente. Há um pouco de código da aplicação no sistema monolítico, e algum armazenamento de dados associados a essa funcionalidade no banco de dados. Então, qual parte devemos extrair antes?

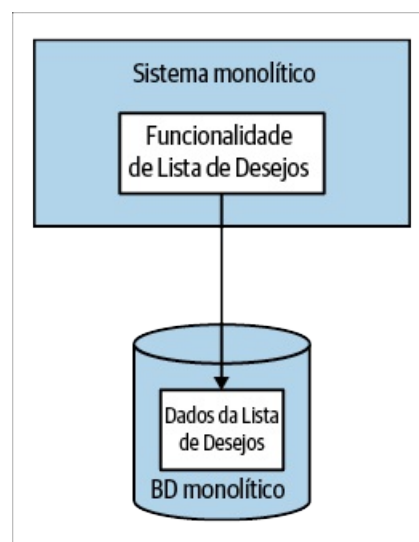


Figura 3.2 – Código e dados da lista de desejos na aplicação monolítica existente.

Código antes

Na Figura 3.3, extraímos o código associado à funcionalidade de lista de

desejos em um novo microsserviço. Os dados da lista de desejos permanecem no banco de dados monolítico nessa etapa – a decomposição não estará concluída até termos removido também os dados relacionados ao novo microsserviço *Lista de Desejos*.

Com base em minha experiência, posso dizer que esse tende a ser o passo inicial mais comum. O principal motivo é que ele tende a proporcionar mais vantagens no curto prazo. Se deixarmos os dados no banco de dados monolítico, teremos muitos problemas no futuro, portanto esses dados precisam ser tratados também, mas teríamos muitos ganhos com nosso novo microsserviço.

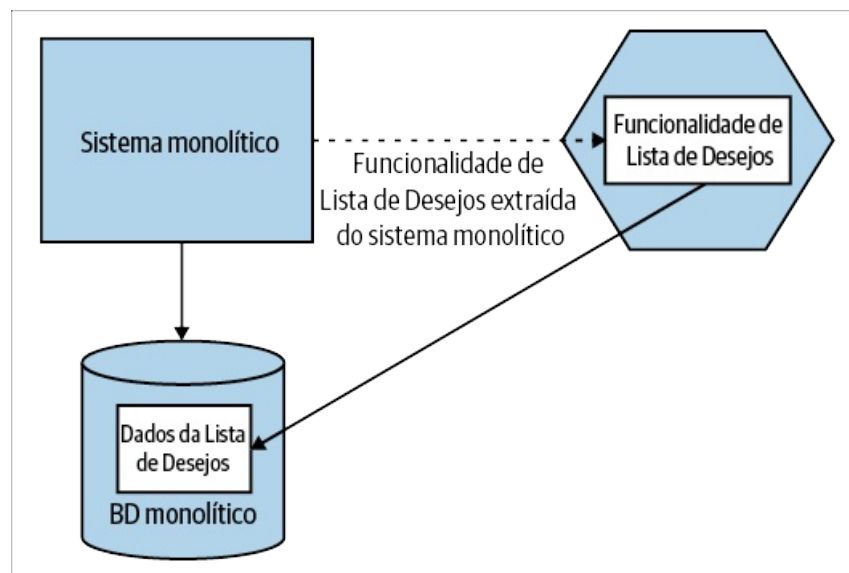


Figura 3.3 – Movendo o código da lista de desejos para um novo microsserviço antes, deixando os dados no banco de dados monolítico.

Extrair o código da aplicação tende a ser mais simples do que extrair itens do banco de dados. Se acharmos que seria impossível extrair o código da aplicação de forma limpa, poderíamos interromper qualquer trabalho subsequente, evitando a necessidade de desembaraçar os dados do banco de dados. Entretanto, se o código da aplicação for extraído de forma limpa, mas a extração dos dados se provar ser impossível, poderíamos ter um problema – assim, é essencial que, mesmo que você tenha decidido extrair o código da aplicação antes dos dados, você tenha analisado os dados associados e tenha alguma ideia acerca da viabilidade da extração e de

como ela será conduzida. Desse modo, você deve fazer sua lição de casa e vislumbrar como o código da aplicação, bem como os dados, serão extraídos, antes de começar.

Dados antes

Na Figura 3.4, vemos os dados serem extraídos antes do código da aplicação. Vejo essa abordagem com menos frequência, mas ela pode ser conveniente em situações nas quais você não tenha certeza se os dados poderão ser separados facilmente. Nesse caso, você provaria que isso pode ser feito antes de passar para o código da aplicação, cuja extração espera-se que seja mais fácil.

A principal vantagem dessa abordagem no curto prazo é reduzir o risco da extração completa do microserviço. Ela forçará você a lidar previamente com problemas como perda da integridade de dados garantida pelo seu banco de dados ou a falta de operações de transação entre os dois conjuntos de dados. Falaremos rapidamente das implicações desses dois problemas mais adiante neste capítulo.

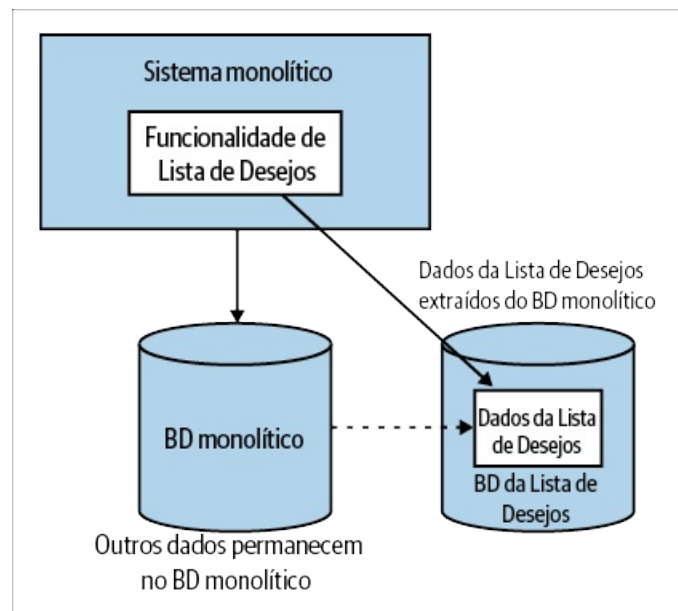


Figura 3.4 – As tabelas associadas à funcionalidade de lista de desejos são extraídas antes.

Padrões convenientes para decomposição

Alguns padrões podem ser úteis para ajudar na separação de um sistema existente. Muitos desses padrões são explorados com detalhes em meu livro *Monolith to Microservices*¹; em vez de repetir todos eles neste livro, apresentarei uma visão geral de alguns para que você tenha uma ideia sobre o que é possível fazer.

Padrão strangler fig

Uma técnica de uso frequente durante a reescrita de sistemas é o *padrão strangler fig*, um termo criado por Martin Fowler (<https://oreil.ly/u33bI>). Inspirado em um tipo de planta (figueira estranguladora), o padrão descreve o processo de encapsular um sistema antigo com o novo sistema com o passar do tempo, permitindo que o novo sistema assuma o controle de cada vez mais funcionalidades do sistema antigo, gradualmente.

A abordagem, conforme vemos na Figura 3.5, é simples. Interceptamos chamadas ao sistema existente – em nosso caso, a aplicação monolítica atual. Se a chamada para a funcionalidade em questão estiver implementada em nossa nova arquitetura de microsserviços, ela será redirecionada para o microsserviço. Se a funcionalidade ainda é tratada pelo sistema monolítico, a chamada poderá prosseguir para o sistema monolítico propriamente dito.

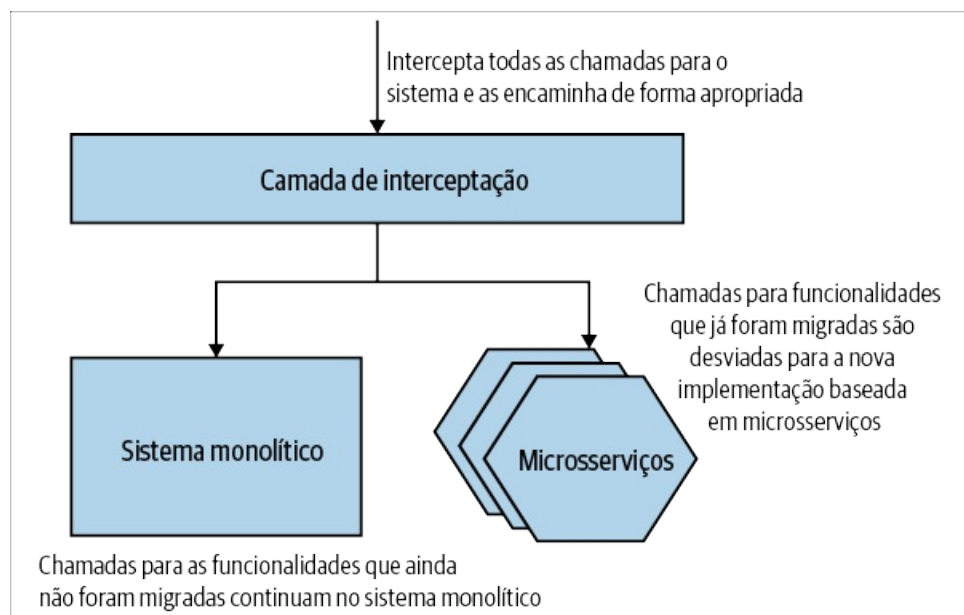


Figura 3.5 – Visão geral do padrão strangler fig.

A beleza desse padrão está no fato de que, em geral, ele pode ser usado sem nenhuma modificação na aplicação monolítica subjacente. A aplicação monolítica nem mesmo tem ciência de que foi “encapsulada” por um sistema mais novo.

Execução paralela

Passar de funcionalidades tratadas por uma arquitetura de aplicação testada e comprovada para uma arquitetura de microserviços totalmente nova pode causar certo nervosismo, sobretudo se a funcionalidade sendo migrada for crucial para a sua empresa.

Uma maneira de garantir que a nova funcionalidade esteja funcionando bem, sem riscos ao comportamento atual do sistema, é utilizar o padrão de execução paralela (parallel run): executar tanto a sua implementação monolítica da funcionalidade como a nova implementação com microserviço, lado a lado, atendendo às mesmas requisições e comparando os resultados. Exploraremos esse padrão com mais detalhes na seção “Execução paralela”, na página [325](#).

Flag de funcionalidade

Uma flag de funcionalidade (feature toggle ou feature flag) é um método

que permite que uma funcionalidade seja ativada ou desativada, ou permite alternar entre duas implementações diferentes da mesma funcionalidade. A flag de funcionalidade é um padrão com boa aplicação geral, mas pode ser particularmente conveniente como parte de uma migração para microsserviços.

Conforme descrito na aplicação do padrão strangler fig, muitas vezes, durante a transição, deixamos a funcionalidade existente como está no sistema monolítico, mas queremos ser capazes de alternar entre suas diferentes versões: a funcionalidade no sistema monolítico e a que está no novo microsserviço. Com o exemplo do padrão strangler fig no qual um proxy HTTP foi usado, poderíamos implementar a flag de funcionalidade na camada de proxy para permitir que haja um controle simples de alternância entre as implementações.

Para uma introdução mais completa às flags de funcionalidade, recomendo o artigo “Feature Toggles (aka Feature Flags)” de Pete Hodgson.²

Preocupações na decomposição de dados

Ao começar a separar os bancos de dados, poderemos causar uma série de problemas. Eis alguns dos desafios que você poderá enfrentar, e algumas dicas para ajudar.

Desempenho

Bancos de dados, particularmente os bancos de dados relacionais, são bons para junções de dados entre diferentes tabelas. Muito bons. Tão bons, na verdade, que tomamos isso como certo. Muitas vezes, porém, quando separamos bancos de dados por causa dos microsserviços, acabamos tendo de mover as operações de junção da camada de dados para dentro dos próprios microsserviços. E, por mais que tentemos, é improvável que elas sejam igualmente rápidas.

Considere a Figura 3.6, que mostra uma situação na qual nos encontramos na MusicCorp. Decidimos extrair nossa funcionalidade de catálogo – algo que possa gerenciar e expor informações sobre artistas, faixas musicais e álbuns. Atualmente, nosso código relacionado a catálogos no sistema

monolítico utiliza a tabela `Álbuns` para armazenar informações sobre os CDs que podem estar disponíveis para venda. Esses álbuns acabam sendo referenciados em nossa tabela de `contabilidade`, que é a tabela na qual registramos todas as vendas. As linhas da tabela de `contabilidade` registram a data em que um item foi vendido, além de um identificador que referencia esse item; o identificador em nosso exemplo se chama SKU (Stock Keeping Unit, ou Unidade de Manutenção de Estoque) – uma prática comum em sistemas de varejo.³

No final de cada mês, devemos gerar um relatório apresentando os CDs com as maiores vendas. A tabela de `contabilidade` nos ajuda a saber qual SKU vendeu o maior número de cópias, mas as informações sobre esse SKU estão na tabela `Álbuns`. Queremos deixar o relatório bonito e fácil de ler; portanto, em vez de dizer que “Vendemos 400 cópias do SKU 123 e ganhamos 1.596 dólares”, gostaríamos de acrescentar mais informações sobre o que foi vendido, dizendo: “Vendemos 400 cópias de *Now That’s What I Call Death Polka* e ganhamos 1.596 dólares”. Para isso, a query no banco de dados disparada pelo nosso código de finanças deve fazer uma junção das informações da tabela de `contabilidade` com a tabela `Álbuns`, como mostra a Figura 3.6.

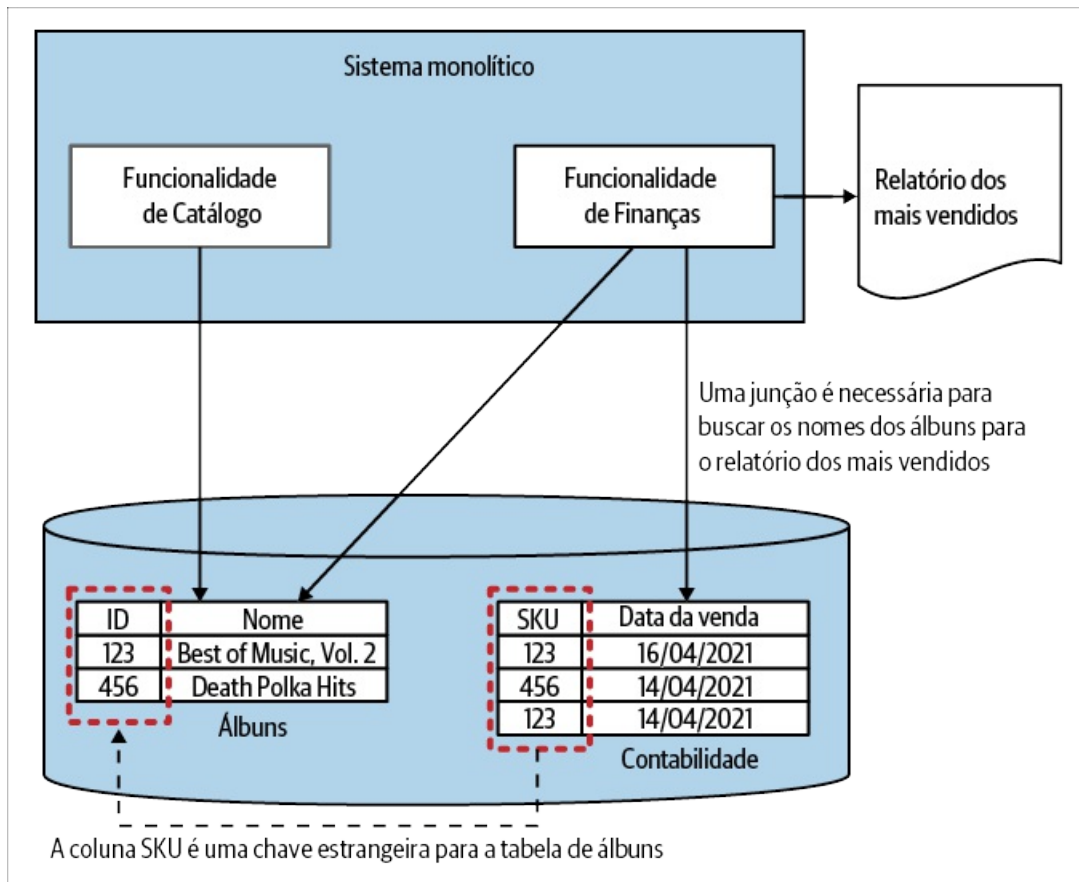


Figura 3.6 – Uma operação de junção no banco de dados monolítico.

Em nosso novo mundo baseado em microsserviços, o novo microsserviço **Finanças** é responsável por gerar o relatório dos itens mais vendidos, mas não tem os dados dos álbuns localmente. Desse modo, ele terá de buscar esses dados junto ao novo microsserviço **catálogo**, conforme vemos na Figura 3.7. Ao gerar o relatório, o microsserviço **Finanças** inicialmente consulta a tabela de **contabilidade**, extraindo a lista de SKUs dos mais vendidos no último mês. A essa altura, a única informação que temos localmente é uma lista de SKUs e o número de cópias vendidas para cada um.

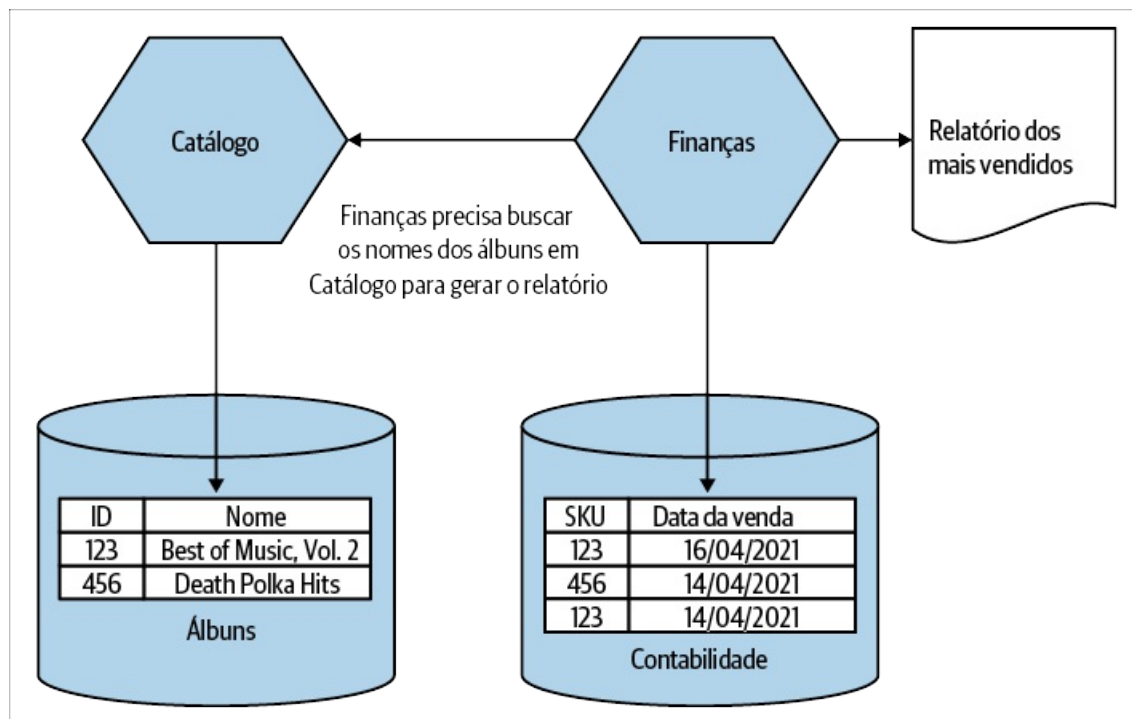


Figura 3.7 – Substituindo uma operação de junção no banco de dados por chamadas de serviço.

Em seguida, temos de chamar o microserviço *catálogo*, solicitando informações sobre cada um desses SKUs. Essa requisição, por sua vez, faria o microserviço *catálogo* executar seu próprio `SELECT` local em seu banco de dados.

Do ponto de vista lógico, a operação de junção continua ocorrendo, mas agora acontece dentro do microserviço *Finanças*, e não no banco de dados. A junção passou da camada de dados para a camada de código da aplicação. Infelizmente essa operação não estará nem perto de ser tão eficiente quanto seria se a junção permanecesse no banco de dados. Passamos de um mundo no qual tínhamos uma única instrução `SELECT` para um novo mundo em que temos uma query `SELECT` na tabela de *Contabilidade*, seguida de uma chamada ao microserviço *catálogo*, o qual, por sua vez, dispara uma instrução `SELECT` na tabela *Álbuns*, como mostra a Figura 3.7.

Nessa situação, eu ficaria *muito* surpreso se a latência total dessa operação não aumentasse. Talvez não seja um problema significativo nesse caso em particular, pois esse relatório é gerado mensalmente e, desse modo, um cache agressivo poderia ser usado (exploraremos esse assunto com mais

detalhes na seção “Caching”, na página [511](#)). No entanto, se essa fosse uma operação frequente, poderia ser mais problemático. É possível atenuar o provável impacto desse aumento de latência se permitirmos que os SKUs sejam consultados no microsserviço `catálogo` em grandes volumes ou, quem sabe, até mesmo fazendo caching das informações necessárias sobre os álbuns localmente.

Integridade dos dados

Bancos de dados podem ser convenientes para garantir a integridade de nossos dados. Voltando à Figura 3.6, com as duas tabelas, `Álbuns` e `Contabilidade`, no mesmo banco de dados, poderíamos definir um relacionamento de chave estrangeira (e provavelmente o faríamos) entre as linhas da tabela de `Contabilidade` e da tabela `Álbuns`. Isso garantiria que sempre poderíamos navegar de um registro na tabela de `Contabilidade` para as informações sobre o álbum vendido, pois não seria possível apagar registros da tabela `Álbuns` se eles fossem referenciados em `Contabilidade`.

Com essas tabelas estando agora em bancos de dados diferentes, não temos mais a garantia da integridade de nosso modelo de dados. Não há nada que nos impeça de apagar uma linha na tabela de `Álbuns`, causando um problema quando tentássemos descobrir exatamente qual foi o item vendido.

Até certo ponto, você simplesmente terá de se acostumar com o fato de que não poderá mais contar com seu banco de dados para garantir a integridade de relacionamentos entre entidades. Obviamente, para os dados que permanecerem no banco de dados único, esse não será um problema.

Há uma série de soluções de contorno, embora “padrões de enfrentamento” seria um termo mais apropriado para as maneiras com as quais poderíamos lidar com esse problema. Poderíamos utilizar uma remoção soft na tabela de `Álbuns`, de modo que não removeríamos realmente um registro, mas apenas o marcaríamos como apagado. Outra opção seria copiar o nome do álbum para a tabela de `Contabilidade` quando uma venda fosse feita, mas teríamos de resolver como lidar com a

sincronização das mudanças no nome do álbum.

Transações

Muitos de nós passaram a contar com as garantias que temos em virtude de gerenciarmos os dados com transações. Com base nessa certeza, criamos aplicações de uma determinada maneira, sabendo que podemos contar com o banco de dados para lidar com uma série de tarefas para nós. Assim que começarmos a separar os dados em vários bancos de dados, porém, não teremos mais a proteção das transações ACID com as quais estávamos acostumados. (Explicarei o acrônimo ACID e discutirei as transações ACID com mais detalhes no Capítulo 6.)

Para as pessoas que estiverem partindo de um sistema em que todas as mudanças de estado podiam ser gerenciadas dentro de uma única fronteira de transação, a mudança para sistemas distribuídos pode ser um choque; com frequência, a reação é tentar implementar transações distribuídas para reaver as garantias que as transações ACID nos davam em arquiteturas mais simples. Infelizmente, conforme discutiremos minuciosamente na seção “Transações em bancos de dados”, na página [215](#), as transações distribuídas não só são complexas de implementar, mesmo quando bem-feitas, como também não nos dão as mesmas garantias que viemos a esperar das transações de bancos de dados com um escopo mais restrito.

Conforme exploraremos na seção “Sagas”, na página [222](#), há métodos alternativos (e preferíveis) às transações distribuídas para administrar mudanças de estado entre vários microsserviços, mas eles carregam consigo novas fontes de complexidade. Assim como no caso da integridade de dados, temos de aceitar o fato de que, ao separar nossos bancos de dados por motivos que podem ser muito bons, vamos deparar com um novo conjunto de problemas.

Ferramentas

Mudar os bancos de dados é difícil por vários motivos, e um deles é o fato de haver ferramentas limitadas à disposição para que possamos fazer as mudanças com facilidade. Com o código, temos ferramentas de refatoração

incluídas em nossos IDEs, além da vantagem adicional de os sistemas que estamos modificando serem basicamente stateless (sem estados). Com um banco de dados, estamos mudando algo que possui estados, porém não há ferramentas tão boas quanto aquelas usadas na refatoração.

Há muitas ferramentas por aí que podem ajudar você a gerenciar o processo de alterar o esquema de um banco de dados relacional, mas a maioria segue o mesmo padrão. Cada mudança de esquema é definida em um script de delta (diferença) com controle de versões. Esses scripts são então executados em uma sequência rigorosa, de forma idempotente. As migrações do Rails funcionam dessa forma, assim como fazia o DBDeploy, uma ferramenta que ajudei a criar vários anos atrás.

Atualmente, indico às pessoas o Flyway (<https://flywaydb.org>) ou o Liquibase (<https://www.liquibase.org>) para terem o mesmo resultado, caso elas ainda não tenham uma ferramenta que funcione dessa forma.

Banco de dados para relatórios

Como parte da extração de microsserviços de nossa aplicação monolítica, também separamos nossos bancos de dados, pois queremos ocultar o acesso à armazenagem de dados interna. Ao ocultar um acesso direto aos nossos bancos de dados, podemos criar interfaces mais estáveis, que tornem as implantações independentes possíveis. Infelizmente, teremos problemas quando tivermos casos de uso legítimos que acessem dados de mais de um microsserviço, ou quando esses dados forem disponibilizados de modo mais apropriado em um banco de dados, em vez de estarem acessíveis com algo como uma API REST.

Com um banco de dados para relatórios, criamos um banco de dados dedicado, projetado para ser acessado externamente, e fazemos com que seja responsabilidade do microsserviço enviar dados do sistema de armazenamento interno para o banco de dados de relatórios, conforme vemos na Figura 3.8.

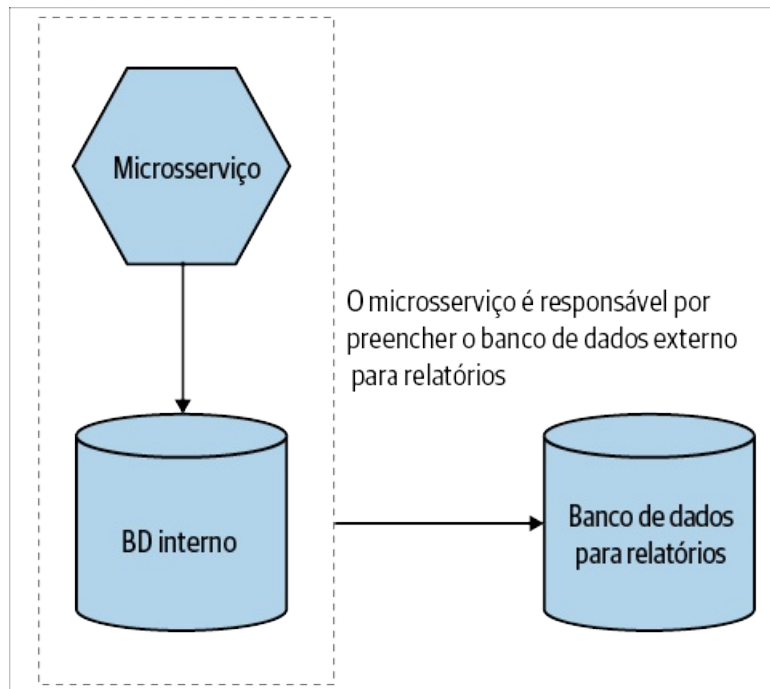


Figura 3.8 – Visão geral do padrão de banco de dados para relatórios.

O banco de dados para relatórios nos permite ocultar o gerenciamento de estados internos, ao mesmo tempo que apresenta os dados em um banco de dados – algo que pode ser muito conveniente. Por exemplo, talvez você queira permitir que as pessoas executem queries SQL *ad hoc*, executem junções em larga escala ou façam uso de conjuntos de ferramentas existentes, as quais esperam ter acesso a um endpoint SQL. O banco de dados para relatórios é uma ótima solução para esse problema.

Há dois pontos essenciais a serem enfatizados nesse caso. Em primeiro lugar, ainda queremos colocar em prática a ocultação de informações. Assim, devemos expor somente o mínimo necessário de dados no banco de dados para relatórios. Isso significa que o que estiver no banco de dados para relatórios pode ser apenas um subconjunto dos dados armazenados pelo microserviço. No entanto, como esse não é um mapeamento direto, teremos a oportunidade de definir o design do esquema do banco de dados para relatórios de modo que seja personalizado exatamente conforme os requisitos dos consumidores – essa solução poderia envolver o uso de um esquema radicalmente diferente ou, quem sabe, até mesmo de um tipo totalmente diferente de tecnologia para banco de dados.

O segundo ponto essencial é que o banco de dados para relatórios deve ser tratado como qualquer outro endpoint de microsserviço, e é função do mantenedor do microsserviço garantir que a compatibilidade desse endpoint seja mantida, mesmo que o microsserviço altere seus detalhes de implementação internos. A responsabilidade pelo mapeamento do estado interno para o banco de dados de relatórios é das pessoas que desenvolvem o microsserviço em questão.

Resumo

Para resumir tudo, ao embarcar na jornada de migração de funcionalidades de uma arquitetura monolítica para uma arquitetura de microsserviços, você deve ter uma compreensão clara sobre *o que* espera alcançar. Esse objetivo vai definir o modo como você abordará o trabalho, além de ajudar a entender se você está caminhando na direção correta.

A migração deve ser gradual. Faça uma alteração, faça o seu rollout, avalie e repita. Até mesmo o ato de separar um microsserviço pode ser, ele próprio, separado em uma série de passos menores.

Se quiser explorar qualquer um dos conceitos que estão neste capítulo com mais detalhes, outro livro que escrevi – *Monolith to Microservices* (O'Reilly)⁴ – explora minuciosamente cada um deles.

Boa parte deste capítulo, de certa forma, apresentou uma visão geral dos assuntos. No próximo capítulo, porém, começaremos a nos aprofundar tecnicamente, analisando como os microsserviços se comunicam entre si.

¹ Sam Newman, *Monolith to Microservices* (Sebastopol: O'Reilly, 2019). [N.T.: Edição publicada no Brasil: *Migrando sistemas monolíticos para microsserviços* (Novatec, 2020).]

² Pete Hodgson, “Feature Toggles (aka Feature Flags)” (Toggles de funcionalidade (também conhecidos como flags de funcionalidade), martinfowler.com, 9 de outubro de 2017, <https://oreil.ly/XiU2t>).

³ Obviamente, essa é uma simplificação de como seria um sistema real. Parece razoável, por exemplo, que registrássemos o valor pelo qual um item foi vendido em um registro contábil!

⁴ N.T.: Edição publicada no Brasil: *Migrando sistemas monolíticos para microsserviços* (Novatec, 2020).

Estilos de comunicação entre microsserviços

Implementar corretamente a comunicação entre microsserviços é um problema para muitos; acho que, em boa medida, isso se deve ao fato de as pessoas se sentirem atraídas por uma determinada abordagem tecnológica escolhida, sem antes considerar os diferentes tipos de comunicação que elas possam querer. Neste capítulo, tentarei diferenciar os vários estilos de comunicação para ajudar você a entender os prós e contras de cada um, assim como para saber qual seria a abordagem mais apropriada no domínio de seu problema.

Veremos métodos de comunicação síncrona bloqueante (synchronous blocking communication) e de comunicação assíncrona não bloqueante (asynchronous nonblocking communication), além de comparar os métodos de colaboração do tipo requisição-resposta (request-response collaboration) e orientados a eventos (event-driven collaboration).

No final deste capítulo, você estará muito mais preparado para entender as diferentes opções à sua disposição, e terá um conhecimento básico que ajudará quando começarmos a analisar as questões relativas à implementação com mais detalhes nos capítulos subsequentes.

Da comunicação interna ao processo à comunicação entre processos

Tudo bem, vamos resolver os problemas mais fáceis primeiro – ou, pelo menos, o que eu *espero* que sejam os problemas mais fáceis. Chamadas *entre* diferentes processos por meio de uma rede (inter-process) são *muito* diferentes de chamadas *dentro* de um único processo (in-process). Em certo nível, podemos ignorar essa distinção. É fácil, por exemplo, pensar em um

objeto fazendo uma chamada de método em outro objeto, e então simplesmente mapear essa interação para dois microsserviços que se comunicam por meio de uma rede. Deixando de lado o fato de que os microsserviços não são apenas objetos, esse raciocínio pode nos levar a muitos problemas.

Vamos analisar algumas dessas diferenças e como elas podem mudar o modo como pensamos nas interações entre os microsserviços.

Desempenho

O desempenho em uma chamada interna ao processo (in-process) é basicamente diferente de uma chamada feita entre processos (inter-process). Quando fazemos uma chamada interna ao processo, o compilador e o runtime subjacentes podem fazer uma série de otimizações a fim de reduzir o impacto da chamada, incluindo deixá-la inline, ou seja, seria, antes de tudo, como se jamais houvesse uma chamada. Otimizações como essas não são possíveis para chamadas entre processos. Pacotes precisam ser enviados. Você pode esperar que o overhead de uma chamada entre processos seja significativo quando comparado com o overhead de uma chamada interna ao processo. O primeiro é claramente mensurável – um único pacote indo e voltando de um data center é medido em milissegundos – enquanto o overhead de fazer uma chamada de método é algo com o qual não precisamos nos preocupar.

Muitas vezes, isso pode levar você a querer repensar nas APIs. Uma API que faça sentido dentro de um processo talvez não faça sentido em situações envolvendo comunicação entre processos. Posso fazer mil chamadas que cruzem a fronteira de uma API no interior de um processo sem me preocupar. Eu gostaria de fazer mil chamadas de rede entre dois microsserviços? Talvez não.

Quando passo um parâmetro para um método, a estrutura de dados que é passada geralmente não sai do lugar – o mais provável é que eu passe um ponteiro para uma posição de memória. Passar um objeto ou uma estrutura de dados para outro método não exige que mais memória seja alocada para copiar os dados.

Quando fazemos chamadas entre microsserviços por meio de uma rede, por outro lado, os dados precisam ser serializados para algum formato que possa ser transmitido por uma rede. Os dados devem então ser enviados e desserializados do outro lado. Desse modo, talvez seja necessário prestar mais atenção no tamanho dos payloads enviados a outros processos. Quando foi a última vez que você se preocupou com o tamanho de uma estrutura de dados que estava sendo passada internamente a um processo? A verdade é que você provavelmente não precisava saber, mas agora precisa. Isso poderia levar você a reduzir o volume de dados enviados ou recebidos (talvez não seja uma má ideia, se pensarmos na ocultação de informações), escolher métodos mais eficientes para serialização, ou até mesmo descarregar dados em um sistema de arquivos e passar uma referência à localização do arquivo em questão, como alternativa.

Essas diferenças talvez não causem problemas de imediato, mas, sem dúvida, você deverá estar ciente delas. Já vi muitas tentativas de ocultar do desenvolvedor o fato de uma chamada de rede estar inclusive sendo feita. Nosso desejo de criar abstrações para ocultar detalhes é uma parte importante para permitir que façamos mais tarefas com mais eficiência, porém, às vezes, criamos abstrações que ocultam demais. Um desenvolvedor deverá estar ciente caso faça algo que resulte em uma chamada de rede; do contrário, você não deveria ficar surpreso se acabasse com alguns gargalos (bottlenecks) de desempenho terríveis no futuro, causados por interações inesperadas entre serviços, que não estavam visíveis ao desenvolvedor que escreveu o código.

Alterando as interfaces

Quando consideramos modificações em uma interface interna a um processo, o ato de fazer um rollout da mudança é simples. O código que implementa a interface e o código que a chama estão todos no mesmo processo. Com efeito, se a assinatura de um método fosse alterada em um IDE com recursos de refatoração, em geral o próprio IDE faria automaticamente a refatoração das chamadas a esse método alterado. Fazer o rollout de uma mudança como essa pode ser uma operação atômica – os dois lados da interface estão juntos em um único processo.

Na comunicação entre microsserviços, porém, o microsserviço que expõe uma interface e os microsserviços que a consomem são microsserviços que podem ser implantados separadamente. Ao fazer uma alteração que cause incompatibilidade com versões anteriores na interface de um microsserviço, precisamos fazer uma implantação sincronizada com os consumidores, garantindo que eles estejam atualizados de modo a poderem utilizar a nova interface, ou teremos de encontrar alguma maneira de fazer o rollout do novo contrato do microsserviço em fases. Exploraremos esse conceito com mais detalhes posteriormente neste capítulo.

Tratamento de erros

No interior de um processo, se chamarmos um método, a natureza dos erros tende a ser muito direta. De modo simplista, os erros são esperados e fáceis de serem tratados, ou são catastróficos a ponto de serem simplesmente propagados para cima na pilha de chamadas (call stack). Os erros, de modo geral, são determinísticos.

Em um sistema distribuído, a natureza dos erros pode ser diferente. Estaremos vulneráveis a uma série de erros que estão além do nosso controle. Há timeouts na rede. Microsserviços downstream podem estar temporariamente indisponíveis. Redes são desconectadas, contêineres são encerrados por consumirem muita memória e, em situações extremas, partes de seu data center poderão pegar fogo.¹

Em seu livro *Distributed Systems*,² Andrew Tanenbaum e Maarten Steen separam os cinco tipos de modos de falha que podemos ver quando observamos uma comunicação entre processos. Eis uma versão simplificada:

Falha de crash

Tudo ia bem até ocorrer um crash no servidor. Reinicialize!

Falha por omissão

Você enviou algo, mas não obteve resposta. Inclui também situações em que esperamos que um microsserviço downstream envie mensagens (talvez incluindo eventos), e ele simplesmente para.

Falha de timing

Algo aconteceu tarde demais (você não recebeu algo no instante em que deveria), ou aconteceu cedo demais!

Falha de resposta

Você obteve uma resposta, mas ela simplesmente parece estar incorreta. Por exemplo, você pediu um resumo dos pedidos, mas partes necessárias da informação estão ausentes na resposta.

Falha arbitrária

Também conhecida como falha bizantina, ocorre quando algo deu errado, mas os participantes são incapazes de chegar a um consenso sobre a falha ter ocorrido (ou por que ocorreu). Como podemos ver, isso é sempre ruim.

Vários desses erros muitas vezes são transientes por natureza – são problemas de curta duração, que podem desaparecer. Considere a situação em que enviamos uma requisição para um microserviço, mas não recebemos uma resposta (um tipo de falha por omissão). Isso poderia significar, antes de tudo, que o microserviço downstream não chegou a receber a requisição, portanto precisamos enviá-la novamente. Outros problemas talvez não possam ser tratados facilmente e exigiriam a intervenção de um operador humano. Como resultado, pode ser importante ter um conjunto com uma semântica mais variada para os erros devolvidos, de modo que os clientes possam tomar a atitude apropriada.

O HTTP é um exemplo de um protocolo que sabe da importância desse fato. Toda resposta HTTP tem um código, com os códigos das séries 400 e 500 reservados para erros. Os códigos de erro da série 400 são erros de requisição – basicamente, um serviço downstream está dizendo ao cliente que há algo errado com a requisição original. Desse modo, é provavelmente um caso em que você deverá desistir – faz algum sentido fazer uma nova tentativa para `404 Not Found` (Não encontrado), por exemplo? Os códigos de resposta da série 500 relacionam-se com problemas downstream, um subconjunto dos quais sinaliza ao cliente que o problema pode ser temporário. Um `503 Service Unavailable` (Serviço indisponível), por exemplo, informa que o servidor downstream é incapaz de tratar a

requisição, mas esse pode ser um estado temporário, caso em que um cliente upstream poderia decidir enviar novamente a requisição. Por outro lado, se um cliente receber uma resposta `501 Not Implemented` (Não implementado), é improvável que uma nova tentativa ajude muito.

Independentemente de optar ou não por um protocolo baseado em HTTP para a comunicação entre microsserviços, se você tiver um conjunto semântico variado acerca da natureza do erro, será mais fácil para os clientes executarem ações compensatórias, o que, por sua vez, deve ajudar você a criar sistemas mais robustos.

Tecnologias para comunicação entre processos: muitas opções

Em um mundo em que temos opções demais e tempo de menos, o óbvio a fazer é simplesmente ignorar algumas coisas.

– *Seth Godin*

A variedade de tecnologias disponíveis para comunicação entre processos é enorme. Como resultado, muitas vezes podemos ficar sobrecarregados com a escolha. Em geral, percebo que as pessoas são atraídas por uma tecnologia que lhes é familiar, ou talvez apenas para a última tecnologia da moda, da qual tiveram conhecimento em um congresso. O problema disso é que, ao adotar uma opção específica de tecnologia, você geralmente adotará um conjunto de ideias e terá um conjunto de restrições que acompanham essa opção. Essas restrições podem não ser apropriadas para você – e a mentalidade por trás da tecnologia talvez não esteja realmente alinhada aos problemas que você está tentando resolver.

Se você está tentando criar um site, uma tecnologia de aplicação single page (página única) como o Angular ou o React seria uma péssima escolha. Do mesmo modo, tentar usar o Kafka para requisições-respostas não é realmente uma boa ideia, pois ele foi projetado para interações mais voltadas a eventos (tópicos que discutiremos em breve). Apesar disso, vejo tecnologias sendo usadas nos lugares errados o tempo todo. As pessoas escolhem a última tecnologia da moda (por exemplo, os microsserviços!), sem considerar se ela é realmente apropriada para os seus problemas.

Assim, quando se trata da incrível variedade de tecnologias à nossa disposição para comunicação entre microsserviços, acho importante discutir antes a questão do estilo de comunicação que você deseja, e somente então procurar a tecnologia correta que implemente esse estilo. Com isso em mente, vamos analisar um modelo que utilizo há vários anos para ajudar a fazer a distinção entre as diferentes abordagens para a comunicação de microsserviço a microsserviço, o qual, por sua vez, poderá ajudar a filtrar as opções de tecnologia que você queira analisar.

Estilos de comunicação entre microsserviços

Na Figura 4.1, vemos uma representação do modelo que utilizo para pensar nos diferentes estilos de comunicação. Esse modelo não tem a pretensão de ser exaustivo (não estou tentando apresentar uma teoria majestosa e unificada de comunicação entre processos), mas oferece uma boa visão geral para considerar os diferentes estilos de comunicação mais amplamente empregados em arquiteturas de microsserviços.

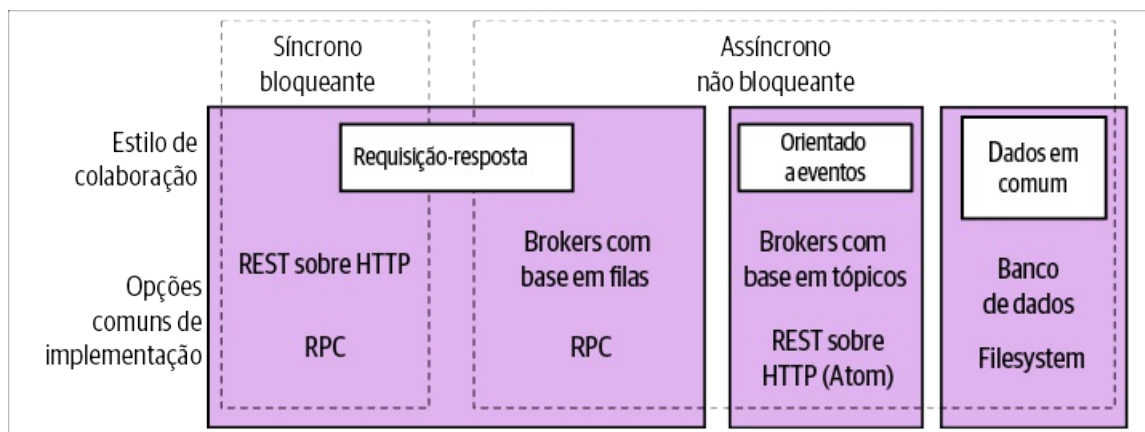


Figura 4.1 – Diferentes estilos de comunicação entre microsserviços, com exemplos de tecnologias que implementam esses estilos.

Veremos os diferentes elementos desse modelo com mais detalhes em breve, mas, antes disso, gostaria de apresentá-los rapidamente:

Síncrono bloqueante

Um microsserviço faz uma chamada para outro microsserviço e a operação permanece bloqueada enquanto ele espera uma resposta.

Assíncrono não bloqueante

O microsserviço que faz uma chamada é capaz de prosseguir com o processamento, independentemente de a chamada ter sido recebida.

Requisição-resposta

Um microsserviço envia uma requisição para outro microsserviço pedindo que algo seja feito. Ele espera receber uma resposta informando o resultado.

Orientado a eventos

Os microsserviços geram eventos, que outros microsserviços consomem e aos quais reagem devidamente. O microsserviço que gera o evento não sabe quais microsserviços, se houver algum, consumirão o evento que ele gerou.

Dados em comum

Não é visto com frequência como um estilo de comunicação; a colaboração entre os microsserviços se dá por meio de alguma fonte de dados compartilhada.

Ao empregar esse modelo para ajudar as equipes a decidirem qual é a abordagem correta, dedico bastante tempo para entender o contexto em que elas atuam. Suas necessidades no que diz respeito a uma comunicação confiável, uma latência aceitável e o volume de comunicação desempenharão um papel importante na escolha da tecnologia. Em geral, porém, minha tendência é começar decidindo se um estilo de colaboração do tipo requisição-resposta ou um estilo orientado a eventos seria mais apropriado para a situação em questão. Se estiver considerando o estilo de requisição-resposta, tanto as implementações síncronas como as assíncronas continuarão disponíveis para mim, portanto terei uma segunda escolha a fazer. Se optar por um estilo de colaboração orientado a eventos, porém, minhas escolhas para a implementação estarão limitadas às opções assíncronas não bloqueantes.

Uma série de outras considerações entram em cena na escolha da tecnologia correta, as quais vão além do estilo de comunicação – por exemplo, a necessidade de uma comunicação com menos latência,

aspectos relacionados à segurança ou a capacidade de escalar. É improvável que você consiga fazer uma escolha sensata de tecnologia sem levar em consideração os requisitos (e as limitações) do domínio específico de seu problema. Quando analisarmos as opções de tecnologia no Capítulo 5, discutiremos algumas dessas questões.

Estilos mistos

É importante observar que uma arquitetura de microsserviços como um todo pode conter uma mistura de estilos de colaboração, e essa geralmente será a norma. Algumas interações simplesmente fazem sentido como requisição-resposta, enquanto outras serão mais apropriadas se forem orientadas a eventos. Com efeito, é comum que um único microsserviço implemente mais de uma forma de colaboração. Considere um microsserviço `Pedido` que exponha uma API para requisição-resposta, a qual permite que pedidos sejam feitos ou alterados, e então gere eventos quando essas alterações são feitas.

Vamos então analisar esses diferentes estilos de comunicação com mais detalhes.

Padrão: síncrono bloqueante

Em uma chamada síncrona bloqueante, um microsserviço envia algum tipo de chamada para um processo downstream (provavelmente, para outro microsserviço) e fica bloqueado até a chamada ser completada e, possivelmente, uma resposta ser recebida. Na Figura 4.2, `Processador de Pedido` envia uma chamada para o microsserviço `Fidelidade` para informá-lo de que alguns pontos devem ser adicionados à conta de um cliente.

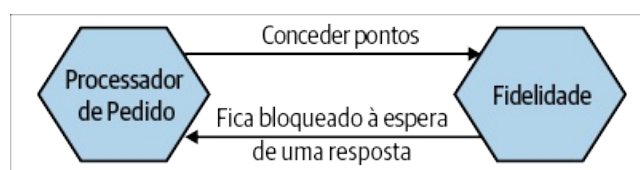


Figura 4.2 – Processador de Pedido envia uma chamada síncrona para o microsserviço Fidelidade, fica bloqueado e espera uma resposta.

Em geral, uma chamada síncrona bloqueante é uma chamada que fica à

espera de uma resposta do processo downstream. Isso pode ocorrer porque o resultado da chamada é necessário para alguma operação subsequente, ou apenas porque o serviço quer garantir que a chamada funcionou e, do contrário, poder efetuar algum tipo de nova tentativa. Como resultado, praticamente qualquer chamada síncrona bloqueante que eu vejo também acaba sendo uma chamada do tipo requisição-resposta, algo que veremos em breve.

Vantagens

Há algo simples e familiar em uma chamada síncrona bloqueante. Muitos de nós aprendemos a programar em um estilo basicamente síncrono – lemos um código como se fosse um script, com cada linha executando na sequência, e a próxima linha de código esperando a sua vez para fazer algo. Na maioria das situações em que você teria usado chamadas entre processos, deve tê-lo feito com um estilo síncrono bloqueante – executando uma query SQL em um banco de dados, por exemplo, ou fazendo uma requisição HTTP para uma API downstream.

Ao se afastar de uma arquitetura menos distribuída, como no caso de um sistema monolítico com um único processo, ater-se às ideias que são conhecidas, quando há tantas outras novidades ao redor, pode fazer sentido.

Desvantagens

O principal desafio com as chamadas síncronas é o seu acoplamento temporário inerente – um assunto que exploramos rapidamente no Capítulo 2. Quando `Processador de Pedido` faz uma chamada para `Fidelidade` no exemplo anterior, o microsserviço `Fidelidade` deve estar acessível para que a chamada funcione. Se o microsserviço `Fidelidade` estiver indisponível, a chamada falhará, e `Processador de Pedido` precisa decidir qual tipo de ação compensatória deverá executar – isso pode envolver uma nova tentativa imediata, guardar a chamada para tentar novamente mais tarde ou, quem sabe, desistir totalmente da chamada.

Esse acoplamento é bidirecional. Com esse estilo de integração, a resposta

em geral é enviada por meio da mesma conexão de rede para o microsserviço upstream. Desse modo, se o microsserviço `Fidelidade` quiser enviar uma resposta de volta para `Processador de Pedido`, mas a instância upstream deixou de executar logo depois, a resposta será perdida. O acoplamento temporário, nesse caso, não ocorre somente entre dois microsserviços, mas ocorre entre duas instâncias específicas desses microsserviços.

Como o originador da chamada permanece bloqueado à espera do microsserviço downstream responder, segue-se daí que, se o microsserviço downstream demorar para responder, ou se houver algum problema com a latência da rede, quem originou a chamada ficará bloqueado por um longo período, à espera de uma resposta. Se o microsserviço `Fidelidade` estiver sujeito a uma carga significativa e demorar para responder às requisições, isso, por sua vez, fará com que `Processador de Pedido` demore para responder.

Assim, o uso de chamadas síncronas pode deixar o sistema vulnerável a problemas em cascata causados por falhas de serviços downstream, mais facilmente do que ocorreria se usássemos chamadas assíncronas.

Quando usar

Para arquiteturas simples de microsserviços, não tenho muito problema com o uso de chamadas síncronas bloqueantes. O fato de muitas pessoas terem familiaridade com esse tipo de chamadas é uma vantagem no processo de tentar compreender os sistemas distribuídos.

Para mim, esses tipos de chamadas começam a se tornar problemáticos quando começamos a ter mais cadeias de chamadas – na Figura 4.3, por exemplo, temos uma amostra de um fluxo na MusicCorp no qual verificamos um pagamento para saber se há alguma possível atividade fraudulenta. `Processador de Pedido` chama o serviço `Pagamento` para receber o pagamento. O serviço `Pagamento`, por sua vez, deve verificar junto ao microsserviço `Deteção de Fraudes` se esse pagamento deve ou não ser permitido. O microsserviço `Deteção de Fraudes`, por sua vez, precisa obter informações do microsserviço `Cliente`.

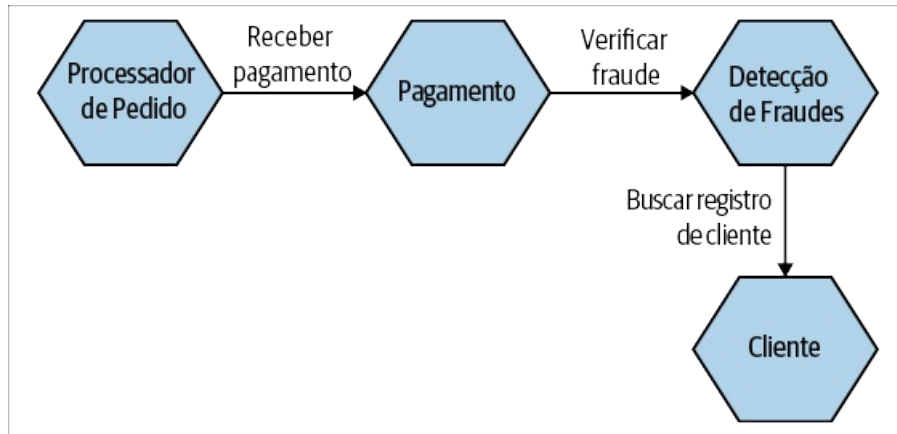


Figura 4.3 – Verificando um possível comportamento fraudulento como parte do fluxo de processamento de pedidos.

Se todas essas chamadas forem síncronas e bloqueantes, há uma série de problemas que poderíamos enfrentar. Um problema em qualquer um dos quatro microsserviços envolvidos, ou nas chamadas de rede entre eles, poderia fazer com que toda a operação falhasse. Tudo isso, sem nem mesmo considerar o fato de que esses tipos de cadeias longas podem causar uma significativa *contenção de recursos* (resource contention). Internamente, é provável que **Processador de Pedido** tenha uma conexão de rede aberta esperando uma resposta de **Pagamento**. **Pagamento**, por sua vez, tem uma conexão de rede aberta à espera de uma resposta de **Detecção de Fraudes**, e assim por diante. Ter muitas conexões que precisam permanecer abertas pode causar impacto no sistema em execução – é muito mais provável que você vá enfrentar problemas com a falta de conexões disponíveis ou sofrerá de um aumento de congestionamento na rede, como resultado.

Para melhorar essa situação, poderíamos, antes de tudo, rever as interações entre os microsserviços. Por exemplo, talvez possamos remover o uso de **Detecção de Fraudes** do fluxo principal de compra, conforme mostra a Figura 4.4, fazendo com que ele seja executado em segundo plano. Caso ele encontre algum problema com um cliente específico, seus registros serão devidamente atualizados, e isso é algo que poderia ser verificado antes no processo de pagamento. Na prática, significa que faremos parte desse trabalho em paralelo. Ao reduzir o tamanho da cadeia de chamadas, veremos uma melhora na latência geral da operação e removeremos um de nossos microsserviços (**Detecção de Fraudes**) do caminho crítico no fluxo de

compras, e assim teremos uma dependência a menos com a qual nos preocupar em uma operação que é de extrema importância.

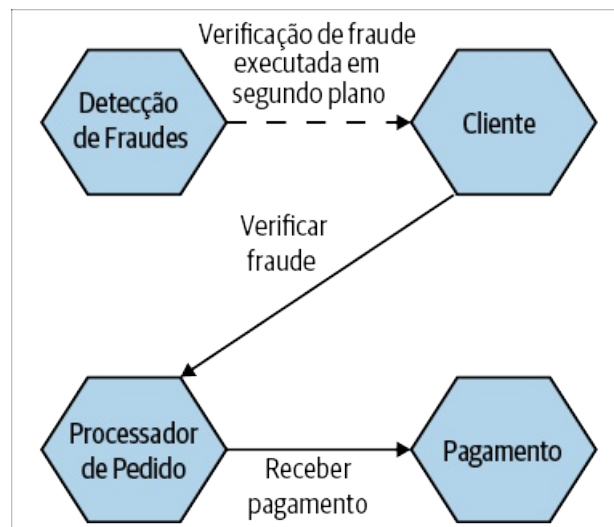


Figura 4.4 – Passar Detecção de Fraudes para um processo em segundo plano pode reduzir as preocupações associadas ao tamanho da cadeia de chamadas.

É claro que também poderíamos substituir o uso de chamadas bloqueantes por algum estilo de interação não bloqueante sem alterar o fluxo de trabalho nesse caso – uma abordagem que exploraremos a seguir.

Padrão: assíncrono não bloqueante

Na comunicação assíncrona, o ato de enviar uma chamada por meio da rede não bloqueia o microserviço que faz essa chamada. Ele é capaz de prosseguir com qualquer outro processamento, sem precisar esperar por uma resposta. A comunicação assíncrona não bloqueante se apresenta de várias formas, mas veremos com mais detalhes os três estilos mais comuns, que costumo ver em arquitetura de microserviços. São eles:

Comunicação por meio de dados em comum

O microserviço upstream altera alguns dados em comum, que um ou mais microserviços usarão mais tarde.

Requisição-resposta

Um microserviço envia uma requisição para outro microserviço, pedindo a ele que faça algo. Quando a operação solicitada é concluída,

independentemente de ter tido sucesso ou não, o microserviço upstream receberá uma resposta. Particularmente, *qualquer* instância do microserviço upstream deve ser capaz de lidar com a resposta.

Interação orientada a eventos

Um microserviço faz o broadcast de um evento, que pode ser pensado como uma declaração factual sobre algo que aconteceu. Outros microserviços podem ficar à escuta dos eventos que sejam de seu interesse e reagir de acordo com esses eventos.

Vantagens

Em uma comunicação assíncrona não bloqueante, o microserviço que faz a chamada inicial e o microserviço (ou microserviços) que recebe a chamada estarão temporariamente desacoplados. Os microserviços que recebem a chamada não precisam estar acessíveis no mesmo instante em que a chamada for feita. Isso significa que evitamos as preocupações com o acoplamento temporário, discutido no Capítulo 2 (veja a seção “Uma observação rápida sobre acoplamento temporário”, na página 68).

Esse estilo de comunicação também será vantajoso se a funcionalidade acionada por uma chamada demorar muito para ser processada. Vamos voltar ao nosso exemplo da MusicCorp, especificamente no processo de enviar um pacote. Na Figura 4.5, `Processador de Pedido` recebeu o pagamento e decidiu que é hora de despachar o pacote, portanto ele envia uma chamada para o microserviço `Depósito`. O processo de encontrar os CDs, retirá-los da prateleira, embalá-los e fazer com que sejam coletados poderia demorar muitas horas e, possivelmente, até mesmo dias, dependendo de como o processo de expedição propriamente dito funciona. Desse modo, faz sentido que `Processador de Pedido` faça uma chamada assíncrona não bloqueante para `Depósito`, fazendo com que `Depósito` efetue posteriormente uma chamada de resposta para informar `Processador de Pedido` sobre o seu progresso. Essa é uma forma de comunicação assíncrona com requisição-resposta.

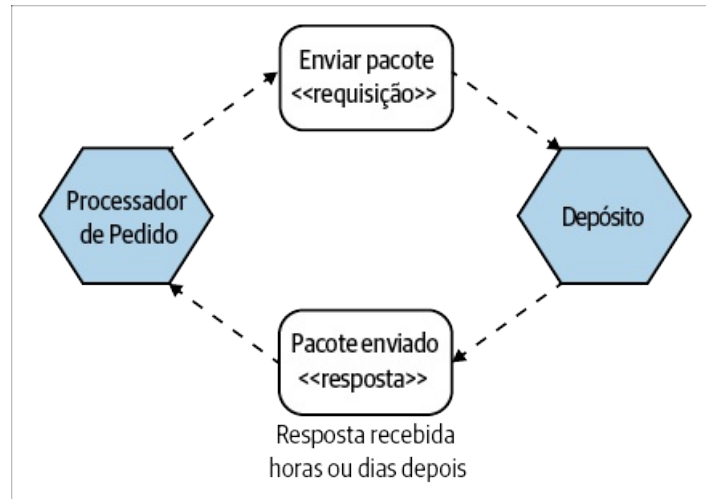


Figura 4.5 – Processador de Pedido inicia o processo de embalar e enviar um pedido, que é feito de modo assíncrono.

Se tentássemos fazer algo parecido com chamadas síncronas bloqueantes, teríamos de reestruturar as interações entre **Processador de Pedido** e **Depósito** — não seria viável que **Processador de Pedido** iniciasse uma conexão, enviasse uma requisição, bloqueasse quaisquer operações subsequentes ao disparar a cadeia de chamadas e esperasse uma resposta que poderia demorar horas ou dias.

Desvantagens

As principais desvantagens da comunicação assíncrona não bloqueante em comparação com a comunicação síncrona bloqueante são o nível de complexidade e a variedade de opções. Conforme já dissemos, há diferentes estilos de comunicação assíncrona entre os quais escolher — qual é a opção correta para você? Quando começarmos a explorar como esses diferentes estilos de comunicação são implementados, poderemos ver uma lista impressionante de tecnologias a serem consideradas.

Se a comunicação assíncrona não se encaixa em seus modelos mentais de computação, adotar um estilo assíncrono de comunicação será desafiador à primeira vista. À medida que explorarmos melhor os detalhes dos diversos estilos de comunicação assíncrona, veremos que há várias maneiras diferentes e interessantes de você acabar se envolvendo em *muitos* problemas.

Async/await e quando a comunicação assíncrona continua sendo bloqueante

Como acontece em muitas áreas da computação, podemos empregar o mesmo termo em diferentes contextos, com significados muito distintos. Um estilo de programação que parece ser particularmente popular consiste no uso de construções como `async/await` para trabalhar com uma fonte de dados possivelmente assíncrona, porém com um estilo síncrono bloqueante.

No Exemplo 4.1, vemos um exemplo muito simples desse caso em JavaScript. As taxas de câmbio flutuam frequentemente ao longo do dia, e recebemos essas informações por meio de um broker de mensagens. Definimos uma `Promise`. De modo geral, uma promise é algo que será resolvido como um estado em algum instante no futuro. Em nosso caso, nosso `eurToGbp` em algum momento será resolvido como a próxima taxa de câmbio de euro para GBP (libra).

Exemplo 4.1 – Um exemplo de como trabalhar com uma chamada possivelmente assíncrona de um modo síncrono bloqueante

```
async function f() {  
  let eurToGbp = new Promise((resolve, reject) => {  
    // código para buscar a última taxa de câmbio entre EUR e GBP  
    ...  
  });  
  
  var latestRate = await eurToGbp; ❶  
  process(latestRate); ❷  
}
```

❶ Espera até que a última taxa de câmbio de EUR para GBP seja buscada.

❷ Não executará até que a promise seja cumprida.

Quando referenciamos `eurToGbp` usando `await`, ficamos bloqueados até que o estado de `latestRate` seja preenchido – `process` não será alcançado até que o estado de `eurToGbp` seja resolvido.³

Ainda que nossas taxas de câmbio sejam recebidas de modo assíncrono, o uso de `await` nesse contexto significa que estaremos *bloqueados* até que o estado de `latestRate` seja resolvido. Portanto, mesmo que a tecnologia subjacente que usamos para obter a taxa pudesse ser considerada assíncrona por natureza (por exemplo, ficando à espera da taxa), do

ponto de vista do nosso código, essa é uma interação inerentemente síncrona e bloqueante.

Quando usar

Em última instância, quando estiver avaliando se a comunicação assíncrona é a opção correta, você deverá considerar também o *tipo* de comunicação assíncrona que deseja escolher, pois cada tipo tem suas próprias vantagens e desvantagens. Em geral, porém, há alguns casos de uso específicos que me fariam recorrer a alguma forma de comunicação assíncrona. Processos de longa duração são um óbvio candidato, conforme exploramos na Figura 4.5. Além disso, situações em que houver cadeias longas de chamadas que não possam ser facilmente reestruturadas poderiam ser outro bom candidato. Exploraremos melhor esse assunto quando analisarmos três das formas mais comuns de comunicação assíncrona – chamadas do tipo requisição-resposta, comunicação orientada a eventos e comunicação por meio de dados em comum.

Padrão: comunicação por meio de dados em comum

Um estilo de comunicação que dá origem a diversas implementações é a comunicação por meio de dados em comum. Esse padrão é usado quando um microsserviço insere dados em um local definido, e então outro microsserviço (ou, possivelmente, vários microsserviços) faz uso desses dados. Pode ser uma solução tão simples quanto um microsserviço deixando um arquivo em algum lugar e, algum tempo depois, outro microsserviço acessa esse arquivo e faz algo com ele. Esse estilo de integração é basicamente assíncrono por natureza.

Um exemplo desse estilo pode ser visto na Figura 4.6, na qual `Importador de Novos Produtos` cria um arquivo que será então lido pelos microsserviços `downstream` `Estoque` e `Catálogo`.

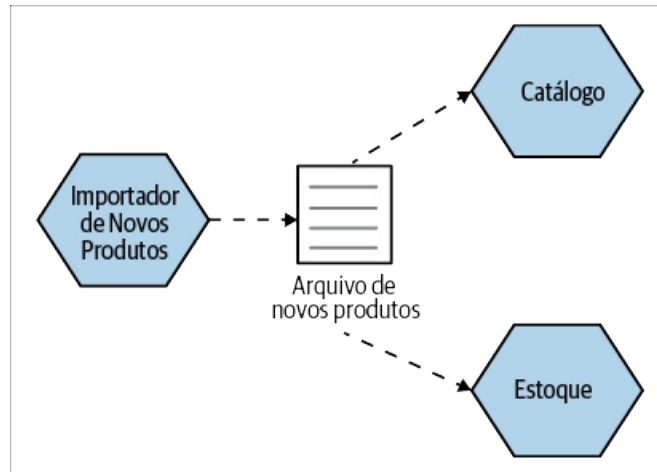


Figura 4.6 – Um microserviço escreve um arquivo que será usado por outros microserviços.

Em certos aspectos, esse é o padrão geral mais comum de comunicação entre processos que você verá; apesar disso, às vezes não o encaramos como um padrão de comunicação – acho que, em boa medida, porque a comunicação entre processos ocorre de forma muito indireta, a ponto de ser difícil de ser reconhecida.

Implementação

Para implementar esse padrão, você precisará de alguma espécie de armazenagem persistente para os dados. Em muitos casos, um sistema de arquivos servirá. Já construí vários sistemas que apenas verificam periodicamente um sistema de arquivos, observam a presença de um arquivo novo e reagem de acordo com isso. Você poderia também, é claro, utilizar algum tipo de armazenagem em memória que seja robusta e distribuída. Vale a pena observar que qualquer microserviço downstream que vá interagir com esses dados precisará de sua própria solução para identificar se há novos dados disponíveis – o polling é uma solução comum para esse problema.

Dois exemplos comuns desse padrão são o data lake (lago de dados) e o armazém de dados (data warehouse). Nos dois casos, essas soluções são geralmente projetadas para ajudar no processamento de grandes volumes de dados, mas, sem dúvida, estão em extremidades opostas do espectro no que concerne ao acoplamento. Com um data lake, os produtores de dados

fazem o upload dos dados brutos em qualquer formato que considerem apropriado, e a expectativa é que os consumidores downstream desses dados brutos saibam processar essas informações. Em um armazém de dados, o próprio armazém é um sistema de armazenagem de dados estruturados. Os microsserviços que enviam dados para o armazém de dados precisam conhecer a sua estrutura – se a estrutura do armazém de dados mudar de modo a ser incompatível com versões anteriores, esses produtores de dados terão de ser atualizados.

Tanto no caso do armazém de dados como no caso do data lake, a suposição é que o fluxo de informações ocorre em uma única direção. Um microsserviço publica dados na área comum de armazenagem de dados, enquanto os consumidores downstream leem esses dados e executam as ações apropriadas. Esse fluxo unidirecional pode facilitar a compreensão do fluxo de informações. Uma implementação mais problemática seria com o uso de um banco de dados compartilhado, na qual vários microsserviços tanto leem como escrevem no mesmo banco de dados; um exemplo disso foi discutido no Capítulo 2, quando exploramos o acoplamento de dados em comum – a Figura 4.7 mostra tanto `Processador de Pedido` como `Depósito` atualizando o mesmo registro.

Vantagens

Esse padrão pode ser implementado de modo muito simples, utilizando uma tecnologia amplamente conhecida. Se você puder ler ou escrever em um arquivo ou ler e escrever em um banco de dados, será possível utilizar esse padrão. O uso de uma tecnologia disseminada e bem conhecida também permite que haja interoperabilidade entre diferentes tipos de sistemas, incluindo aplicações mainframe mais antigas ou produtos de software COTS (Customizable Off-The-Shelf, ou Produto Comercial Customizável). Os volumes de dados também causam menos preocupações nesse caso – se você pretende enviar grandes volumes de dados de uma só vez, esse padrão pode funcionar bem.

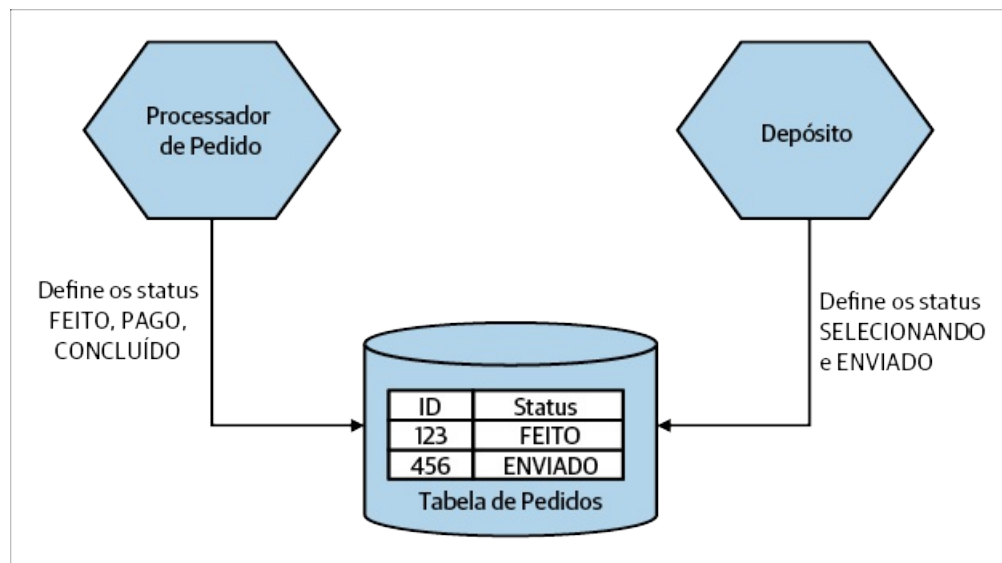


Figura 4.7 – Um exemplo de acoplamento de dados em comum, no qual tanto Processador de Pedido como Depósito atualizam o mesmo registro de pedido.

Desvantagens

Microserviços downstream que são consumidores em geral saberão que há novos dados a serem processados por meio de algum tipo de método de polling ou, talvez, por intermédio de uma tarefa disparada periodicamente. Isso significa que é improvável que esse método seja conveniente em situações que exijam baixa latência. É claro que podemos combinar esse padrão com algum outro tipo de chamada que informe a um microserviço downstream que há novos dados disponíveis. Por exemplo, eu poderia escrever um arquivo em um sistema de arquivos compartilhado, e então enviar uma chamada ao microserviço interessado informando-o de que há novos dados que ele possa querer. Isso poderia cobrir a lacuna entre os dados serem publicados e esses dados serem processados. Em geral, porém, se você estiver usando esse padrão para volumes muito grandes de dados, é menos provável que uma baixa latência esteja no topo de sua lista de requisitos. Se estiver interessado em enviar volumes maiores de dados e tê-los processados mais próximo do “tempo real”, então empregar algum tipo de tecnologia de streaming como o Kafka seria uma opção mais apropriada.

Outra grande desvantagem – e algo que deveria ser razoavelmente óbvio se

você se recorda de nossa exploração do acoplamento de dados em comum na Figura 4.7 – é que os repositórios de dados em comum se tornam uma possível causa de acoplamento. Se a estrutura desse repositório de dados mudar de alguma forma, isso poderá causar falhas na comunicação entre os microsserviços.

A robustez da comunicação também se reduzirá à robustez do repositório de dados subjacente. Estritamente falando, essa não é uma desvantagem, mas é algo do qual devemos estar cientes. Se você vai deixar um arquivo em um sistema de arquivos, certifique-se de que o próprio sistema de arquivos não vá falhar de maneiras interessantes.

Quando usar

Esse padrão realmente se destaca para permitir uma interoperabilidade entre processos que possam ter restrições quanto à tecnologia que podem usar. Fazer um sistema existente conversar com a interface GRPC de seu microsserviço ou se registrar para seu tópico no Kafka pode ser muito mais conveniente do ponto de vista do microsserviço, mas não do ponto de vista de um consumidor. Sistemas mais antigos podem ter limitações quanto às tecnologias que aceitam, e os custos para mudança podem ser altos. Por outro lado, até mesmo sistemas mainframe antigos serão capazes de ler dados de um arquivo. Tudo isso, é claro, depende do uso de uma tecnologia de armazenagem de dados que seja amplamente aceita – esse padrão também poderia ser implementado com algo como um cache Redis. No entanto, seu antigo sistema mainframe é capaz de conversar com o Redis?

Outra boa vantagem desse padrão está no compartilhamento de grandes volumes de dados. Se você precisa enviar um arquivo com vários gigabytes para um sistema de arquivos ou carregar alguns milhões de linhas em um banco de dados, esse é o padrão que você deverá escolher.

Padrão: comunicação por meio de requisição-resposta

Com o estilo de requisição-resposta, um microsserviço envia uma requisição para um serviço downstream pedindo a ele que faça algo, e

espera receber uma resposta com o resultado da requisição. Essa interação pode ocorrer por meio de uma chamada síncrona bloqueante, ou poderia ser implementada de um modo assíncrono não bloqueante. A Figura 4.8 mostra um exemplo simples dessa interação, na qual o microserviço `Tabela`, que reúne os CDs mais vendidos de diferentes gêneros musicais, envia uma requisição ao serviço `Estoque` solicitando os níveis de estoque atuais de alguns CDs.

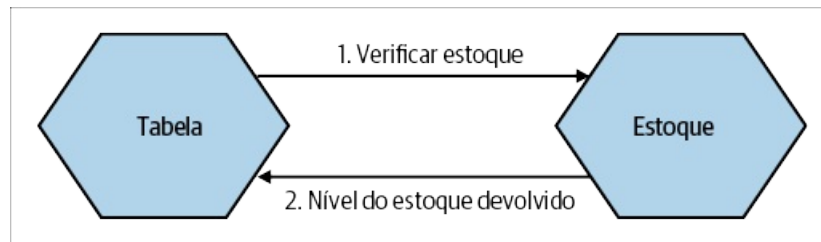


Figura 4.8 – O microserviço `Tabela` envia uma requisição para `Estoque` solicitando os níveis de estoque.

Obter dados de outros microserviços dessa forma é um caso de uso comum para uma chamada do tipo requisição-resposta. Às vezes, porém, você só precisa garantir que algo seja feito. Na Figura 4.9, o microserviço `Depósito` recebe uma requisição de `Processador de Pedido` solicitando que uma reserva seja feita no estoque. `Processador de Pedido` só precisa saber que uma reserva foi feita com sucesso no estoque antes de poder prosseguir para receber o pagamento. Se não foi possível fazer uma reserva no estoque – talvez porque um item não está mais disponível –, o pagamento poderá ser cancelado. Utilizar chamadas do tipo requisição-resposta em situações como essa, em que as chamadas precisam ser concluídas em determinada sequência, é comum.

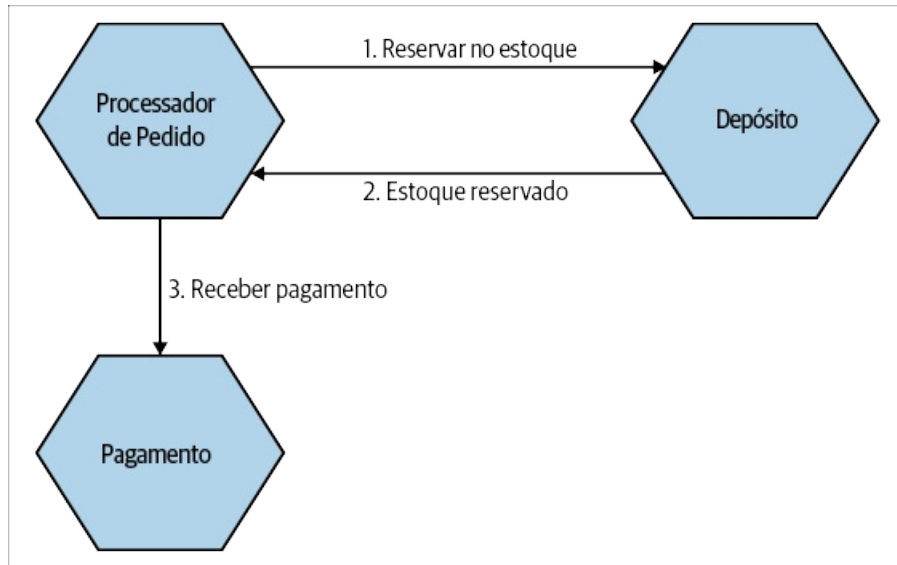


Figura 4.9 – Processador de Pedido deve garantir que uma reserva seja feita no estoque antes que o pagamento possa ser recebido.

Comandos versus requisições

Já ouvi algumas pessoas falarem de enviar comandos em vez de requisições, especificamente no contexto da comunicação assíncrona do tipo requisição-resposta. A intenção ao usar o termo *comando* sem dúvida é a mesma de quando o termo *requisição* é empregado, isto é, um microserviço upstream está pedindo a um microserviço downstream que faça algo.

Pessoalmente, porém, prefiro o termo *requisição*. Um comando implica uma ordem que deve ser obedecida, e pode levar a uma situação em que as pessoas achem que uma atitude deve ser necessariamente tomada a respeito do comando. Uma requisição implica algo que pode ser rejeitado. É apropriado que um microserviço analise o mérito de cada requisição e, com base em sua própria lógica interna, decida se ela deve resultar em alguma ação. Se a requisição enviada ao microserviço violar sua lógica interna, ele deve rejeitá-la. Embora a diferença seja sutil, não acho que o termo *comando* transmita o mesmo significado.

Vou me ater ao uso de *requisição* em vez de *comando*, mas, independentemente do termo que você decida empregar, lembre-se de que um microserviço pode rejeitar a requisição/comando se achar apropriado.

Implementação: síncrono versus assíncrono

Chamadas do tipo requisição-resposta como essas podem ser implementadas com um estilo síncrono bloqueante ou assíncrono não bloqueante. Em uma chamada síncrona, o que geralmente veremos é uma conexão de rede ser iniciada com o microserviço downstream, e a requisição será então enviada por meio dessa conexão. A conexão permanece aberta enquanto o microserviço upstream espera o microserviço downstream responder. Nesse caso, o microserviço que envia a resposta não precisa saber realmente nada a respeito do microserviço que enviou a requisição – ele apenas envia uma resposta para uma conexão de entrada. Se essa conexão for encerrada, talvez porque a instância do microserviço upstream ou do microserviço downstream foi encerrada, poderíamos ter um problema.

Em um estilo assíncrono de requisição-resposta, a situação é um pouco mais complicada. Vamos rever o processo associado à reserva de itens no estoque. Na Figura 4.10, a requisição para reserva de estoque é enviada na forma de uma mensagem, por meio de algum tipo de broker de mensagens (exploraremos os brokers de mensagens mais adiante neste capítulo). Em vez de ir diretamente do microserviço `Processador de Pedido` para `Depósito`, a mensagem fica em uma fila. `Depósito` consome mensagens dessa fila quando puder. Ele lê a requisição, faz o trabalho associado à reserva no estoque, e então precisa enviar a resposta de volta para uma fila que será lida por `Processador de Pedido`. O microserviço `Depósito` precisa saber para onde deve encaminhar a resposta. Em nosso exemplo, ele envia essa resposta por meio de outra fila que, por sua vez, é consumida por `Processador de Pedido`.

Assim, em uma interação assíncrona não bloqueante, o microserviço que recebe a requisição precisa saber implicitamente para onde deve encaminhar a resposta, ou deverá ser informado disso. Ao usar uma fila, temos a vantagem adicional de várias requisições poderem ser armazenadas nessa fila, à espera de serem tratadas. Isso pode ajudar em situações em que as requisições não podem ser tratadas com suficiente rapidez. O microserviço poderá consumir a próxima requisição quando estiver pronto, em vez de ficar sobrecarregado com um excesso de chamadas. É

claro que muito, então, dependerá da capacidade da fila de absorver essas requisições.

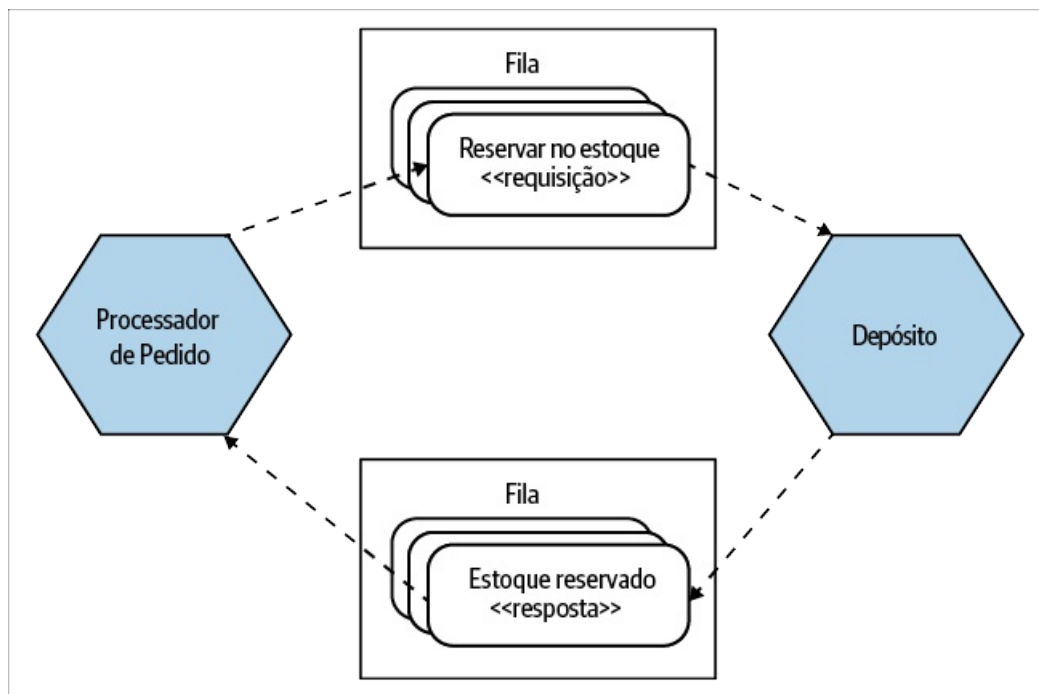


Figura 4.10 – Usando filas para enviar requisições de reserva no estoque.

Quando um microserviço recebe uma resposta dessa maneira, ele talvez precise relacioná-la com a requisição original. Isso pode ser desafiador, pois muito tempo pode ter passado e, dependendo da natureza do protocolo em uso, a resposta talvez não seja devolvida para a mesma instância do microserviço que enviou a requisição. Em nosso exemplo de reserva no estoque como parte de fazer um pedido, teríamos de saber como associar a resposta “estoque reservado” a um dado pedido para podermos continuar com o processamento desse pedido em particular. Uma maneira fácil de lidar com essa situação seria armazenar qualquer estado associado à requisição original em um banco de dados, de modo que, quando a resposta chegar, a instância receptora possa recarregar qualquer estado associado e agir de acordo com essa informação.

Uma última observação: é provável que qualquer forma de interação com requisição-resposta exija lidar com algum tipo de timeout para evitar problemas em que o sistema fique bloqueado à espera de algo que poderia jamais vir a acontecer. O modo de implementar essa funcionalidade de

timeout pode variar de acordo com a tecnologia de implementação, porém ela será necessária. Veremos os timeouts com mais detalhes no Capítulo 12.

Chamadas em paralelo versus sequenciais

Ao trabalhar com interações do tipo requisição-resposta, com frequência você vai deparar com uma situação em que terá de fazer várias chamadas antes de poder continuar com algum processamento.

Considere uma situação em que a MusicCorp precise verificar o preço de um dado item junto a três atacadistas diferentes, o que é feito por meio de chamadas de API. Queremos obter os preços de todos os três atacadistas antes de decidir para qual deles faremos o pedido de novos itens para o estoque. Poderíamos decidir que as três chamadas serão feitas em sequência – esperamos que cada uma seja concluída antes de prosseguir para a próxima. Em uma situação como essa, teríamos de esperar pela soma das latências de cada uma das chamadas. Se a chamada de API para cada fornecedor demorasse um segundo para ser respondida, teríamos de esperar três segundos para decidir para quem faríamos um pedido.

Uma melhor opção seria executar essas três requisições em paralelo; então, a latência total da operação dependeria da chamada de API mais lenta, e não da soma das latências de cada chamada de API.

Extensões do Reactive e métodos como `async/await` podem ser muito úteis para ajudar na execução de chamadas em paralelo, e isso pode resultar em melhorias significativas na latência de algumas operações.

Quando usar

Chamadas do tipo requisição-resposta fazem todo sentido em qualquer situação em que o resultado de uma requisição seja necessário para que outros processamentos possam ocorrer. Também são muito apropriadas em situações nas quais um microserviço deseja saber se uma chamada não funcionou para que ele possa executar algum tipo de ação compensatória, por exemplo, fazer uma nova tentativa. Se algum desses cenários se assemelha à sua situação, o uso de requisição-resposta será uma abordagem sensata; a única questão pendente será decidir se a

implementação será síncrona ou assíncrona, com as mesmas vantagens e desvantagens que discutimos antes.

Padrão: comunicação orientada a eventos

A comunicação orientada a eventos parece muito diferente quando comparada às chamadas com requisição-resposta. Em vez de termos um microserviço pedindo a outro microserviço que faça algo, um microserviço gerará eventos que poderão ou não ser recebidos por outros microserviços. É uma interação inerentemente assíncrona, pois os listeners dos eventos executarão em suas próprias threads de execução.

Um evento é uma declaração a respeito de algo que aconteceu, quase sempre algo que ocorreu no mundo do microserviço que gerou o evento. O microserviço que gera o evento não tem nenhum conhecimento da intenção dos outros microserviços que utilizarão esse evento e, com efeito, talvez nem saiba que há outros microserviços. Ele gera o evento quando for necessário, e suas responsabilidades terminam por aí.

Na Figura 4.11, vemos `Depósito` gerando eventos relacionados ao processo de embalagem de um pedido. Esses eventos são recebidos por dois microserviços, `Notificações` e `Estoque`, e eles reagem a esses eventos de forma apropriada. O microserviço `Notificações` envia um email para atualizar nosso cliente acerca das mudanças no status do pedido, enquanto o microserviço `Estoque` pode atualizar os níveis de estoque quando os itens do pedido do cliente forem embalados.

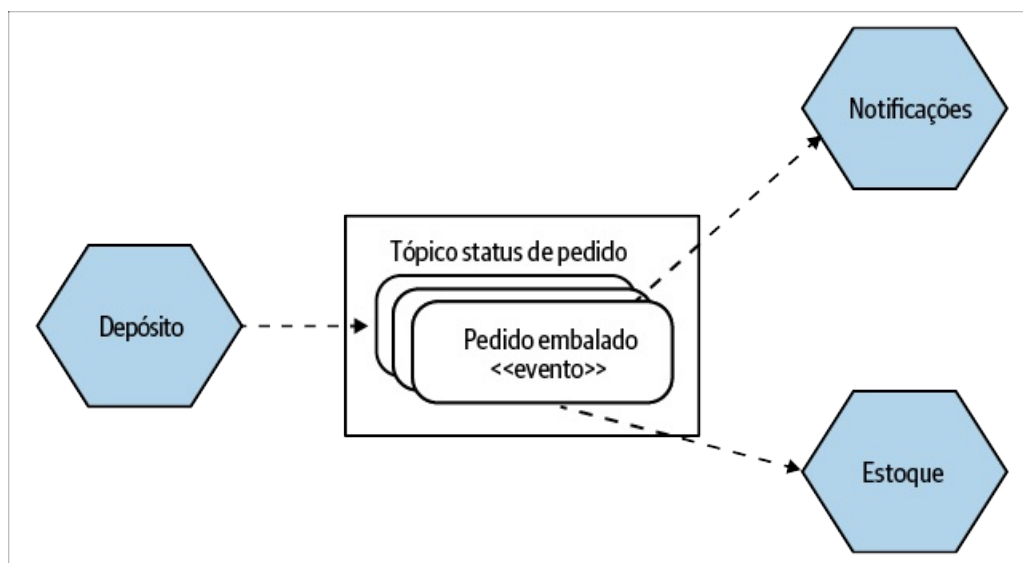


Figura 4.11 – Depósito gera eventos aos quais alguns microserviços downstream se registram para receber.

Depósito simplesmente faz um broadcast dos eventos, supondo que as partes interessadas reagirão de acordo com eles. Esse microserviço não sabe quem são os receptores desses eventos, fazendo com que as interações orientadas a eventos, de modo geral, tenham muito menos acoplamento. Ao comparar esse estilo com uma chamada do tipo requisição-resposta, talvez custe um pouco para você se acostumar com a ideia da inversão de responsabilidades. Na interação com requisição-resposta, esperaríamos que Depósito dissesse ao microserviço Notificações para enviar emails quando fosse apropriado. Em um modelo como esse, Depósito teria de saber quais eventos exigem uma notificação para o cliente. Em uma interação orientada a eventos, passamos essa responsabilidade para o microserviço Notificações.

A intenção de um evento poderia ser considerada oposta à de uma requisição. Quem gera o evento deixa a cargo dos receptores decidirem o que fazer. Na requisição-resposta, o microserviço que envia a requisição sabe o que deve ser feito e diz ao outro microserviço o que ele acha que deve acontecer na sequência. Isso, é claro, significa que, em uma interação do tipo requisição-resposta, quem faz a requisição precisa saber o que o receptor downstream pode fazer, implicando um maior acoplamento de domínio. Na colaboração orientada a eventos, quem gera os eventos não

precisa saber o que os microsserviços downstream podem fazer e, com efeito, podem nem saber que esses microsserviços existem – consequentemente, o acoplamento será bastante reduzido.

A distribuição de responsabilidades que vemos em nossas interações orientadas a eventos pode espelhar a distribuição de responsabilidades que vemos nas empresas que tentam criar equipes mais autônomas. Em vez de manter todas as responsabilidades centralizadas, queremos passá-las para as próprias equipes, permitindo que elas atuem com mais autonomia – um conceito que retomaremos no Capítulo 15. Em nosso caso, estamos passando a responsabilidade de `Depósito` para `Notificações e Estoque` – isso pode nos ajudar a reduzir a complexidade de microsserviços como `Depósito`, resultando em uma distribuição mais equilibrada da “inteligência” em nosso sistema. Exploraremos essa ideia com mais detalhes quando compararmos coreografia e orquestração no Capítulo 6.

Eventos e mensagens

Já vi ocasionalmente os termos *mensagens* e *eventos* serem confundidos. Um evento é um fato – uma declaração de que algo aconteceu, junto com algumas informações sobre o que exatamente aconteceu. Uma mensagem é algo que enviamos por meio de um método de comunicação assíncrona, por exemplo, um broker de mensagens.

Em uma colaboração orientada a eventos, queremos divulgar esse evento, e um modo típico de implementar esse método de divulgação seria colocar o evento em uma mensagem. A mensagem é o meio; o evento é o payload.

De modo semelhante, podemos enviar uma requisição no payload de uma mensagem – nesse caso, estaríamos implementando uma forma de interação assíncrona com requisição-resposta.

Implementação

Há dois aspectos principais que devemos considerar nesse caso: um modo de nossos microsserviços gerarem eventos e um modo para nossos consumidores descobrirem que esses eventos ocorreram.

Tradicionalmente, brokers de mensagens como o RabbitMQ tentam lidar com os dois problemas. Os produtores utilizam uma API para publicar um

evento para o broker. O broker cuida das inscrições, permitindo que os consumidores sejam informados quando um evento chegar. Esses brokers podem inclusive lidar com o estado dos consumidores – por exemplo, ajudando a monitorar quais mensagens eles já viram. Esses sistemas são geralmente projetados para serem escaláveis e resilientes, mas isso não é gratuito. Pode haver uma complexidade adicional no processo de desenvolvimento, pois é outro sistema que terá de ser executado para desenvolver e testar seus serviços. Máquinas e conhecimentos adicionais também podem ser necessários para manter essa infraestrutura em execução. Contudo, assim que estiver pronta, ela poderá ser uma maneira extremamente eficaz para implementar arquiteturas orientadas a eventos com baixo acoplamento. Em geral, sou fã dessa solução.

Tome cuidado, porém, com o mundo dos middlewares, do qual o broker de mensagens é apenas uma pequena parte. As filas, por si só, são soluções perfeitamente sensatas e convenientes. No entanto, os fornecedores tendem a querer incluir muitos softwares com elas, o que pode resultar em um volume cada vez maior de inteligência sendo transferida para o middleware, conforme evidenciado por soluções como o enterprise service bus (barramento de serviços corporativos). Certifique-se de saber o que você está adquirindo: mantenha seu middleware burro, e deixe a inteligência nos endpoints.

Outra abordagem é tentar usar o HTTP como uma forma de propagar eventos. O Atom é uma especificação compatível com REST, que define a semântica (entre outros aspectos) para publicar feeds de recursos. Há muitas bibliotecas de cliente que nos permitem criar e consumir esses feeds. Então, nosso serviço cliente poderia simplesmente publicar um evento em um feed como esse sempre que mudasse. Bastaria que nossos consumidores fizessem um polling no feed em busca das mudanças. Por um lado, o fato de podermos reutilizar a especificação Atom existente, e quaisquer bibliotecas associadas, é conveniente, e sabemos que o HTTP lida muito bem com escala. No entanto, esse uso do HTTP não é apropriado para baixa latência (área em que alguns brokers de mensagens se destacam), e ainda teremos de lidar com o fato de que os consumidores precisam manter o controle das mensagens que já viram e gerenciar sua

própria agenda de polling.

Já vi pessoas gastarem muito tempo implementando várias funcionalidades que já estão à disposição em um broker de mensagens apropriado para fazerem o Atom funcionar em alguns casos de uso. Por exemplo, o *padrão de consumidor concorrente* descreve um método em que fazemos várias instâncias workers competirem por mensagens, o que funciona bem para escalar o número de workers a fim de lidar com uma lista de tarefas independentes (retomaremos esse assunto no próximo capítulo). Entretanto, queremos evitar a situação em que dois ou mais workers vejam a mesma mensagem, pois acabaremos executando a mesma tarefa mais vezes do que o necessário. Com um broker de mensagens, uma fila padrão cuidará disso. Com o Atom, precisaríamos agora gerenciar nosso próprio estado compartilhado entre todos os workers para tentar reduzir as chances de trabalhos duplicados.

Se você já tem um bom broker de mensagens resiliente à disposição, considere utilizá-lo para lidar com a publicação de eventos e a inscrição para recebê-los. Se você ainda não tiver um, dê uma olhada no Atom, mas esteja ciente da falácia do custo irreversível (sunk cost fallacy)⁴. Se você se vir querendo ter cada vez mais do suporte oferecido por um broker de mensagens, em determinado momento talvez você deva mudar sua abordagem.

No que diz respeito àquilo que realmente enviamos por meio desses protocolos assíncronos, as mesmas considerações sobre a comunicação síncrona se aplicam. Se você estiver atualmente satisfeito em codificar as requisições e respostas usando JSON, continue com ele.

O que há em um evento?

Na Figura 4.12, vemos um microserviço *cliente* fazendo broadcast de um evento, informando às partes interessadas que um novo cliente se cadastrou no sistema. Dois microserviços downstream, *Fidelidade* e *Notificações*, se interessam por esse evento. O microserviço *Fidelidade* reage à recepção do evento criando uma conta para o novo cliente para que ele possa começar a ganhar pontos, enquanto o microserviço *Notificações*

envia um email para o cliente recém-cadastrado dando-lhe as boas-vindas ao mundo maravilhoso da MusicCorp.

Em uma requisição, pedimos a um microserviço que faça algo e fornecemos as informações necessárias para que a operação solicitada seja executada. Com um evento, divulgamos um fato no qual outras partes *poderiam* estar interessadas; no entanto, como o microserviço que gera o evento não pode nem deve saber quem o receberá, como saberemos quais são as informações que as outras partes poderiam precisar a respeito do evento? O que, exatamente, deve estar incluído no evento?

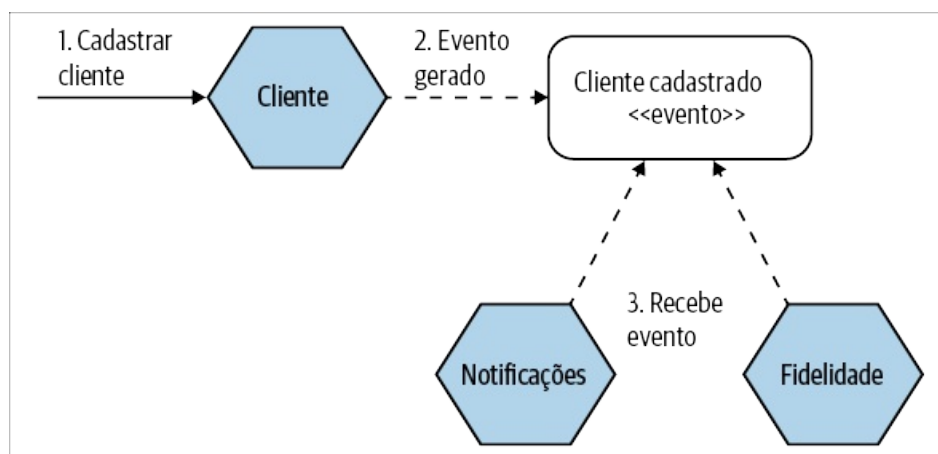


Figura 4.12 – Os microserviços Notificações e Fidelidade recebem um evento quando um novo cliente se cadastra.

Somente um ID

Uma opção é que o evento simplesmente contenha um identificador para o cliente recém-cadastrado, como vemos na Figura 4.13. O microserviço *Fidelidade* precisa somente desse identificador para criar a conta de fidelidade associada, portanto ele tem todas as informações necessárias. No entanto, embora o microserviço *Notificações* saiba que precisa enviar um email de boas-vindas quando esse tipo de evento for recebido, ele precisará de informações adicionais para fazer o seu trabalho – no mínimo, precisará de um endereço de email e, provavelmente, do nome do cliente também, para dar aquele toque pessoal ao email. Como essas informações não estão no evento recebido pelo microserviço *Notificações*, ele não terá outra opção a não ser buscá-las junto ao microserviço *cliente* – algo que vemos na

Figura 4.13.

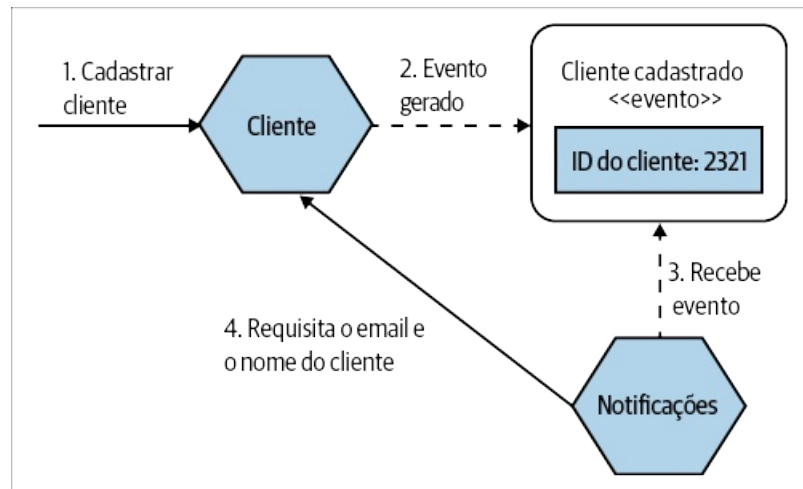


Figura 4.13 – O microserviço Notificações precisa requisitar ao microserviço cliente outros detalhes que não estão incluídos no evento.

Há algumas desvantagens nessa abordagem. Em primeiro lugar, o microserviço Notificações agora precisa saber da existência do microserviço cliente, aumentando o acoplamento de domínio. Ainda que o acoplamento de domínio, conforme discutimos no Capítulo 2, esteja na extremidade mais fraca do espectro de acoplamentos, devemos evitá-lo sempre que for possível. Se o evento recebido pelo microserviço Notificações contivesse todas as informações necessárias, essa callback não seria necessária. A callback do microserviço que recebe o evento também pode resultar em outra desvantagem relevante, isto é, em uma situação na qual há muitos microserviços que recebem o evento, e o microserviço que o gerou, como resultado, receberia uma enxurrada de requisições. Suponha que cinco microserviços diferentes tenham recebido o mesmo evento de criação de cliente, e todos precisassem requisitar informações adicionais – todos teriam de enviar imediatamente uma requisição para o microserviço cliente a fim de obterem os dados necessários. À medida que o número de microserviços interessados em um evento específico aumentar, o impacto dessas chamadas poderá se tornar significativo.

Eventos com detalhes completos

A alternativa, que é a que prefiro, consiste em colocar no evento todos os

dados que você ficaria satisfeito em compartilhar por meio de uma API. Se você vai deixar que o microserviço `Notificações` peça o endereço de email e o nome de um dado cliente, por que já não inserir todas essas informações no evento desde o princípio? Na Figura 4.14, vemos essa abordagem – `Notificações` agora é mais autossuficiente e capaz de fazer seu trabalho sem precisar se comunicar com o microserviço `cliente`. De fato, ele poderia nem saber que o microserviço `cliente` existe.

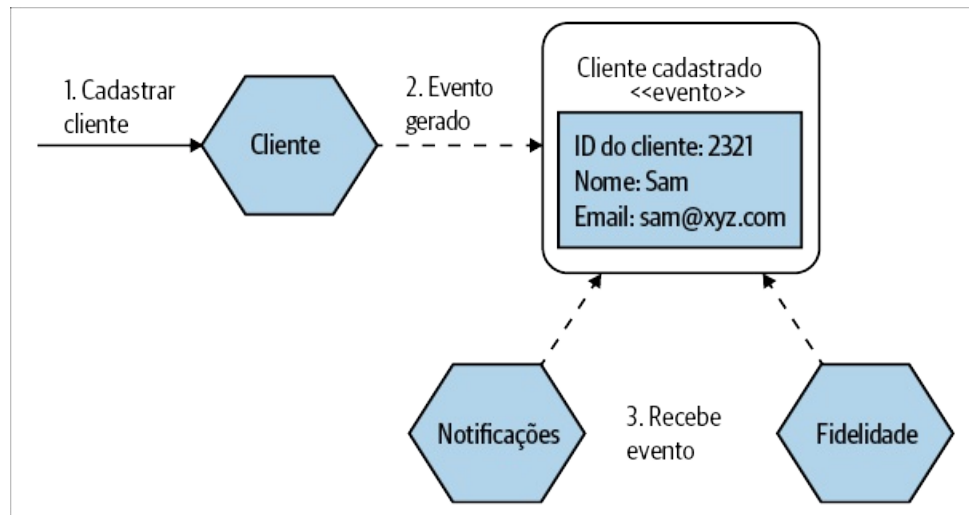


Figura 4.14 – Um evento contendo mais informações permite que os microsserviços que receberem esse evento possam agir sem exigir que façam chamadas subsequentes a quem originou o evento.

Além do fato de eventos com mais informações possibilitarem ter menos acoplamento, eventos com mais informações podem servir também como um registro histórico do que aconteceu com uma dada entidade. Isso poderia ajudar você como parte da implementação de um sistema de auditoria ou, quem sabe, até mesmo para proporcionar a você a capacidade de reconstituir uma entidade em determinados pontos do tempo; isso significa que esses eventos poderiam ser empregados como parte de um *event sourcing* (histórico de eventos) – um conceito que exploraremos em breve.

Embora, definitivamente, essa seja a minha abordagem predileta, ela não deixa de ter suas desvantagens. Em primeiro lugar, se houver muitos dados associados a um evento, poderemos ter problemas com o seu tamanho. Brokers de mensagem modernos (supondo que você esteja utilizando um

para implementar seu método de broadcast de eventos) têm limites razoavelmente generosos para o tamanho das mensagens; o tamanho default máximo para uma mensagem no Kafka é de 1 MB, e a versão mais recente do RabbitMQ tem um limite superior teórico de 512 MB para uma única mensagem (menor que o limite anterior de 2 GB!), embora possamos esperar que houvesse alguns problemas interessante de desempenho com mensagens tão grandes como essas. Contudo, mesmo o 1 MB de tamanho máximo permitido para uma mensagem no Kafka nos dá espaço para o envio de muitos dados. Em última análise, se você está se aventurando em uma área na qual está começando a se preocupar com o tamanho de seus eventos, eu recomendaria uma abordagem híbrida, em que parte das informações esteja no evento, porém outros dados (maiores) possam ser consultados se forem necessários.

Na Figura 4.14, *Fidelidade* não precisa saber o endereço de email nem o nome do cliente, mas, apesar disso, ele os recebe com o evento. Isso poderia resultar em problemas se estivéssemos tentando limitar o escopo dos dados que cada microserviço possa ver – por exemplo, eu poderia querer limitar os microserviços que poderiam ver dados PII (Personally Identifiable Information, ou Informações de Identificação Pessoal), detalhes de cartões para pagamento ou dados confidenciais semelhantes. Uma maneira de resolver isso seria enviando dois tipos diferentes de eventos – um que contenha dados de PII e que possa ser visto por alguns microserviços, e outro que os exclua e possa ser divulgado de modo mais amplo. Haverá um aumento de complexidade para gerenciar a visibilidade de diferentes eventos e garantir que os dois eventos sejam realmente gerados. O que acontecerá se um microserviço enviar o primeiro tipo de evento, mas se tornar inativo antes que o segundo evento tenha sido enviado?

Outro fato a ser considerado é que, assim que colocarmos dados em um evento, eles farão parte de nosso contrato com o mundo externo. Temos de estar cientes de que, se removermos um campo de um evento, poderemos causar falhas em componentes externos. A ocultação de informações continua sendo um conceito importante na colaboração orientada a eventos – quanto mais dados colocarmos em um evento, mais

pressuposições os componentes externos farão a respeito do evento. Eis a minha regra geral: não tenho problemas em colocar informações em um evento se eu estiver satisfeito em compartilhar esses mesmos dados com uma API para requisição-resposta.

Quando usar

Uma colaboração orientada a eventos se destaca em situações em que queremos fazer um broadcast de informações, e naquelas em que estaremos satisfeitos em inverter as responsabilidades. Distanciar-se de um modelo em que devemos dizer aos outros o que eles devem fazer e, em vez disso, deixar que os próprios microsserviços downstream determinem o que farão é muito atraente.

Em uma situação na qual seu foco esteja em um baixo acoplamento acima de outros fatores, a colaboração orientada a eventos será obviamente mais atraente.

A advertência é que, muitas vezes, há novas causas de complexidade que se destacarão com esse estilo de colaboração, sobretudo se você teve pouca exposição a ele. Se você não tem muita certeza a respeito dessa forma de comunicação, lembre-se de que nossa arquitetura de microsserviços pode (e, provavelmente, vai) conter uma mistura de diferentes estilos de interação. Você não precisa adotar a colaboração orientada a eventos em todos os casos; pode talvez começar com apenas um evento e partir daí.

Pessoalmente, eu me vejo atraído em direção a uma colaboração orientada a eventos quase como padrão. Meu cérebro parece ter se reprogramado de tal modo que esses tipos de comunicação simplesmente parecem *óbvios* para mim. Isso não ajuda muito, pois pode ser complicado tentar explicar o *porquê* disso, além de dizer que *parece* certo. Contudo, essa é apenas a tendência que eu incorporei – sou naturalmente atraído por aquilo que *eu* conheço, com base em minhas próprias experiências. Há uma grande possibilidade de que minha atração por essa forma de interação tenha sido determinada quase totalmente pelas minhas experiências anteriores ruins com sistemas excessivamente acoplados. Talvez eu seja apenas um general que esteja lutando sua última batalha repetidamente, sem considerar que

dessa vez, quem sabe, seria realmente diferente.

Deixando de lado minhas próprias tendências, posso dizer que vejo muito mais equipes substituindo interações do tipo requisição-resposta por interações orientadas a eventos do que o contrário.

Prossiga com cautela

Parte dessas interações assíncronas parecem interessantes, não é mesmo? As arquiteturas orientadas a eventos parecem resultar em sistemas escaláveis, significativamente mais desacoplados. E podem. Entretanto, esses estilos de comunicação também levam a um aumento de complexidade. Não é somente a complexidade necessária para gerenciar a publicação de mensagens e o processo de inscrição para recebê-las, conforme acabamos de discutir, mas também uma complexidade em outros problemas que poderemos enfrentar. Por exemplo, ao considerar uma interação assíncrona do tipo requisição-resposta de longa duração, devemos pensar no que fazer quando a resposta retornar. Ela chegará no mesmo nó que iniciou a requisição? Em caso afirmativo, o que acontecerá se esse nó estiver inativo? Se não chegar, terei de armazenar informações em algum lugar para poder reagir de modo apropriado? Interações assíncronas de curta duração podem ser mais fáceis de administrar se tivermos as APIs corretas, mas, mesmo nesse caso, será um modo diferente de pensar para os programadores que estiverem acostumados a chamadas com mensagens síncronas dentro do mesmo processo.

É hora de narrar uma história que serve como advertência. Lá pelos idos de 2006, eu estava trabalhando na criação de um sistema de atribuição de preços para um banco. Observávamos eventos no mercado e determinávamos quais itens de um portfólio precisavam ter os preços recalculados. Assim que determinávamos a lista de itens com os quais trabalhar, nós os colocávamos em uma fila de mensagens. Utilizávamos uma tabela para criar um pool de workers para atribuição de preços, permitindo que redimensionássemos o farm de workers por demanda. Esses workers utilizavam o padrão de consumidores concorrentes, cada um consumindo mensagens o mais rápido possível, até que não restasse nada

para processar.

O sistema estava ativo e em execução, e estávamos nos sentindo bastante orgulhosos. Um dia, porém, logo depois de termos lançado uma nova versão, nós deparamos com um sério problema: nossos workers ficavam morrendo. E morrendo. E morrendo.

Algum tempo depois, conseguimos identificar o problema. Um bug havia sido introduzido e ocorria quando um determinado tipo de requisição de atribuição de preços fazia um worker falhar. Estávamos utilizando uma fila com transações: quando um worker morria, ocorria um timeout no lock da requisição, e a requisição de atribuição de preço era colocada de volta na fila – somente para que outro worker a selecionasse e morresse. Esse era um exemplo clássico daquilo que Martin Fowler chamou de failover catastrófico (<https://oreil.ly/8HwcP>).

Além do bug propriamente dito, não havíamos especificado um limite máximo de novas tentativas para a tarefa na fila. Assim, corrigimos o bug e configuramos um número máximo de novas tentativas. Contudo, percebemos também que precisávamos de uma forma de visualizar e, possivelmente, reproduzir essas mensagens ruins. Acabamos tendo de implementar um hospital de mensagens (ou fila de mensagens não entregues), para onde as mensagens eram enviadas caso falhassem. Também criamos uma UI para visualizar essas mensagens e enviá-las novamente, se fosse necessário. Esses tipos de problema não são imediatamente óbvios caso você tenha familiaridade somente com uma comunicação síncrona ponto a ponto.

A complexidade associada às arquiteturas orientadas a eventos e à programação assíncrona em geral me levam a acreditar que você deve ser cauteloso, e não deve começar a adotar essas ideias afoitamente. Certifique-se de ter um bom sistema de monitoração implantado e considere fortemente o uso de IDs de correlação, que permitem rastrear requisições que cruzem as fronteiras dos processos, conforme será discutido com detalhes no Capítulo 10.

Também recomendo consultar o livro *Enterprise Integration Patterns* de Gregor Hohpe e Bobby Woolf², que contém muito mais detalhes sobre os

diferentes padrões de troca de mensagens que você possa querer levar em consideração nessa área.

Contudo, também precisamos ser honestos quanto aos estilos de integração que poderíamos considerar como “mais simples” – os problemas associados a saber se algo funcionou ou não corretamente não estão limitados às formas assíncronas de integração. Em uma chamada síncrona bloqueante, se houver um timeout, isso ocorreu porque a requisição se perdeu e o serviço downstream não a recebeu? Ou a requisição chegou, mas a resposta se perdeu? O que você fará nessa situação? E se você tentar novamente, mas a requisição original já ter chegado? (Bem, é nesse cenário que a idempotência entra em cena – um tópico que discutiremos no Capítulo 12.)

Sem dúvida, no que concerne ao tratamento de falhas, as chamadas síncronas bloqueantes podem igualmente nos causar muitas dores de cabeça quando se trata de descobrir se algo aconteceu (ou não). Essas dores de cabeça talvez sejam apenas mais conhecidas por nós!

Resumo

Neste capítulo, detalhei alguns dos principais estilos de comunicação entre microsserviços e discutimos suas diversas vantagens e desvantagens. Nem sempre haverá uma única opção *certa*, mas espero que eu tenha detalhado informações suficientes a respeito das chamadas síncronas e assíncronas e dos estilos de comunicação orientados a eventos e com requisição-resposta a fim de ajudar você a tomar a decisão correta para o seu contexto em particular. Minha própria tendência a uma colaboração assíncrona, orientada a eventos, é função não só de minha experiência, mas também de minha aversão ao acoplamento, de modo geral. Contudo, esse estilo de comunicação carrega consigo um volume significativo de complexidade que não pode ser ignorada, e cada situação é única.

Neste capítulo, mencionei rapidamente algumas tecnologias específicas que podem ser usadas para implementar esses estilos de interação. Agora estamos prontos para iniciar a segunda parte deste livro – a implementação. No próximo capítulo, exploraremos a implementação da

comunicação entre microserviço com mais detalhes.

1 História verídica.

2 Maarten van Steen e Andrew S. Tanenbaum, *Distributed Systems*, 3ª edição (Scotts Valley, CA: CreateSpace Independent Publishing Platform, 2017).

3 Observe que esse é um exemplo bastante simplificado – eu omiti totalmente o código para tratamento de erros, por exemplo. Se quiser saber mais sobre `async/await`, particularmente em JavaScript, o *Modern JavaScript Tutorial* (Tutorial para JavaScript moderno, <https://javascript.info>) é um ótimo ponto de partida.

4 N.T.: A falácia do custo irrecuperável (sunk cost fallacy) refere-se à tendência de prosseguir com uma tarefa na qual já se investiu muito tempo ou recursos, sem parar para avaliar se continua sendo vantajoso prosseguir com a empreitada, uma vez que um alto investimento já foi feito.

5 Gregor Hohpe e Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

PARTE II

Implementação

Implementando a comunicação entre microserviços

Conforme discutimos no capítulo anterior, sua escolha da tecnologia deve ser orientada principalmente pelo estilo da comunicação que você quer ter. Decidir entre chamadas síncronas bloqueantes ou assíncronas não bloqueantes, e colaboração por meio de requisição-resposta ou orientada a eventos, ajudará você a reduzir o que poderia ser uma longa lista de tecnologias. Neste capítulo, veremos algumas das tecnologias comumente empregadas na comunicação entre microserviços.

Em busca da tecnologia ideal

Há um conjunto impressionante de opções para um microserviço poder conversar com outro. Mas qual é a opção correta? SOAP? XML-RPC? REST? gRPC? E novas opções estão sempre surgindo. Então, antes de discutirmos uma tecnologia específica, vamos pensar naquilo que queremos da tecnologia a ser escolhida.

Facilite a compatibilidade com versões anteriores

Ao fazer mudanças em nossos microserviços, precisamos garantir que não causaremos incompatibilidades com nenhum microserviço consumidor. Desse modo, queremos garantir que, qualquer que seja a tecnologia escolhida, ela facilitará fazer mudanças que sejam compatíveis com versões anteriores. Operações simples, como o acréscimo de novos campos, não devem causar falhas nos clientes. O ideal seria que fôssemos capazes de validar se as mudanças que fizemos são compatíveis com versões anteriores – e termos um modo de obter esse feedback antes de implantar nosso

microserviço no ambiente de produção.

Deixe sua interface explícita

É importante que a interface exposta por um microserviço ao mundo externo seja explícita. Isso significa que, para um consumidor de um microserviço, é preciso que esteja claro quais são as funcionalidades expostas por esse microserviço. Contudo, significa também que deve estar claro para um desenvolvedor que trabalhe com o microserviço quais são as funcionalidades que devem permanecer intactas para os componentes externos – devemos evitar a situação em que uma alteração em um microserviço provoque uma incompatibilidade acidental.

Esquemas explícitos podem contribuir muito para garantir que a interface exposta por um microserviço seja explícita. Algumas das tecnologias disponíveis exigem o uso de um esquema; para outras, a utilização de um esquema é opcional. Qualquer que seja o caso, incentivo bastante o uso de um esquema explícito, assim como a existência de uma documentação auxiliar que seja suficiente para deixar claro quais são as funcionalidades que um consumidor pode esperar de um microserviço.

Mantenha suas APIs independentes de tecnologia

Se você está no mercado de TI há mais de 15 minutos, não é preciso que eu diga a você que trabalhamos em uma área que muda rapidamente. A única certeza é a mudança. Novas ferramentas, frameworks e linguagens surgem o tempo todo, implementando novas ideias que podem nos ajudar a trabalhar com mais rapidez e eficácia. Neste momento, você talvez trabalhe com .NET. Mas e daqui a um ano, ou cinco? E se você quiser fazer experimentos com um conjunto alternativo de tecnologias que poderiam aumentar sua produtividade?

Sou um grande adepto de manter minhas opções em aberto, e é por isso que sou tão fã dos microserviços. É por isso também que acho muito importante garantir que você mantenha as APIs usadas para comunicação entre os microserviços independentes de tecnologia. Isso significa evitar uma tecnologia de integração que determine os conjuntos de tecnologias

que poderão ser usados para implementar nossos microsserviços.

Deixe seu serviço simples para os consumidores

Queremos facilitar o uso de nosso microsserviço pelos consumidores. Ter um microsserviço incrivelmente fatorado não será de grande valor se o custo para um consumidor utilizar esse serviço for extremamente alto! Vamos então pensar no que faz com que o nosso novo serviço incrível seja usado com mais facilidade pelos consumidores. O ideal é que dêsemos total liberdade aos nossos clientes quanto à sua escolha da tecnologia; por outro lado, disponibilizar uma biblioteca de cliente pode facilitar a adoção do serviço. Muitas vezes, porém, bibliotecas como essas são incompatíveis com outros objetivos que queremos alcançar. Por exemplo, podemos usar bibliotecas de cliente para facilitar a vida dos consumidores, porém à custa de um aumento no acoplamento.

Oculte detalhes de implementação internos

Não queremos que nossos consumidores dependam de nossa implementação interna, pois isso resultará no aumento do acoplamento; por sua vez, esse acoplamento significa que, se quisermos alterar algo dentro do nosso microsserviço, poderemos causar problemas aos nossos consumidores, exigindo que eles mudem também. O custo da alteração será mais elevado – é exatamente isso que estamos tentando evitar. Também significa que é menos provável que faremos uma mudança por medo de precisarmos fazer um upgrade em nossos consumidores, o que pode levar a um aumento no débito técnico do serviço. Assim, qualquer tecnologia que nos leve a expor detalhes da representação interna deve ser evitada.

Opções de tecnologia

Há uma infinidade de tecnologias que poderíamos considerar, mas, em vez de analisar uma lista enorme de opções, destacarei algumas que são mais conhecidas e interessantes. Eis as opções que veremos:

Remote procedure calls (chamadas de procedimento remoto)

Frameworks que permitem que chamadas de métodos locais sejam feitas em um processo remoto. Opções comuns incluem SOAP e gRPC.

REST

Um estilo de arquitetura em que expomos recursos (Cliente, Pedido etc.) que podem ser acessados utilizando um conjunto comum de verbos (GET, POST). O REST é um pouco mais do que isso, e veremos mais detalhes em breve.

GraphQL

Um protocolo relativamente novo que permite que os consumidores definam queries personalizadas para buscar informações de vários microsserviços downstream, filtrando os resultados de modo a devolver somente o que for necessário.

Brokers de mensagens

Middlewares que permitem uma comunicação assíncrona por meio de filas ou tópicos.

Remote procedure calls

Uma *RPC* (Remote Procedure Call, ou Chamada de Procedimento Remoto) refere-se à técnica de fazer uma chamada local e tê-la executada em um serviço remoto em algum lugar. Há várias implementações diferentes de *RPC* em uso. A maioria das tecnologias nessa área exige um esquema explícito, por exemplo, o SOAP ou o gRPC. No contexto de *RPC*, muitas vezes o esquema é referenciado como IDL (Interface Definition Language, ou Linguagem de Definição de Interface), com o SOAP referindo-se ao seu formato de esquema como WSDL (Web Service Definition Language, ou Linguagem de Definição de Serviço Web). O uso de um esquema separado facilita a geração de stubs de clientes e servidores para diferentes conjuntos de tecnologias – por exemplo, eu poderia ter um servidor Java expondo uma interface SOAP e um cliente .NET gerado a partir da mesma definição WSDL da interface. Outras tecnologias, como o Java RMI, implicam um maior acoplamento entre o cliente e o servidor, exigindo que ambos utilizem a mesma tecnologia subjacente, porém evitam a necessidade de uma definição explícita do serviço, pois essa

definição é fornecida implicitamente pelas definições de tipos em Java. Todas essas tecnologias, no entanto, têm a mesma característica fundamental: fazem uma chamada remota se parecerem com uma chamada local.

Em geral, o uso de uma tecnologia de RPC implica a adoção de um protocolo de serialização. O framework RPC define como os dados são serializados e desserializados. Por exemplo, o gRPC utiliza o formato de serialização de buffer de protocolo (protocol buffer) para isso. Algumas implementações estão vinculadas a um protocolo de rede específico (como o SOAP, que faz uso nominal do HTTP), enquanto outras podem permitir o uso de diferentes tipos de protocolos de rede, os quais podem oferecer recursos adicionais. Por exemplo, o TCP oferece garantias de entrega, ao contrário do UDP, porém o UDP tem muito menos overhead. Isso permite que utilizemos diferentes tecnologias de rede para diferentes casos de uso.

Os frameworks RPC que têm um esquema explícito facilitam bastante a geração de um código de cliente. Isso pode evitar a necessidade de bibliotecas de cliente, pois qualquer cliente pode simplesmente gerar o seu próprio código com base na especificação do serviço em questão. No entanto, para que a geração de código do lado cliente funcione, o cliente precisará ter alguma maneira de obter o esquema previamente – em outras palavras, o consumidor precisará ter acesso ao esquema antes do momento em que planeja fazer as chamadas. O Avro RPC é uma exceção interessante, pois tem a opção de enviar o esquema completo junto com o payload, permitindo que os clientes interpretem dinamicamente esse esquema.

A facilidade de geração de código do lado cliente é um dos principais atrativos do RPC. O fato de eu poder fazer uma chamada de método comum e, teoricamente, ignorar o resto é uma grande vantagem.

Desafios

Conforme vimos, o RPC oferece algumas vantagens ótimas, mas não deixa de ter suas desvantagens – e algumas implementações de RPC podem ser mais problemáticas que outras. Podemos resolver vários desses problemas,

mas eles merecem uma exploração mais minuciosa.

Acoplamento de tecnologias. Algumas soluções de RPC, como o Java RMI, estão extremamente vinculadas a uma plataforma específica, podendo limitar a tecnologia que poderá ser usada no cliente e no servidor. O Thrift e o gRPC têm um suporte impressionante para linguagens alternativas, o que pode, até certo ponto, reduzir essa desvantagem, mas saiba que a tecnologia de RPC às vezes apresenta restrições quanto à interoperabilidade.

De certo modo, esse acoplamento de tecnologia pode ser uma forma de expor detalhes técnicos de implementação internos. Por exemplo, o uso de RMI vincula não só o cliente à JVM, mas também o servidor.

Para ser justo, há uma série de implementações de RPC que não têm essa restrição – gRPC, SOAP e Thrift são exemplos que permitem interoperabilidade entre diferentes conjuntos de tecnologias.

Chamadas locais não são como chamadas remotas. A ideia básica do RPC está em ocultar a complexidade de uma chamada remota. No entanto, isso pode levar a ocultar demais. Em algumas formas de RPC, a motivação para fazer com que chamadas de método remotas se pareçam com chamadas de método locais esconde o fato de que essas duas chamadas são diferentes. Posso fazer várias chamadas locais, internas a um processo, sem me preocupar muito com o desempenho. Com o RPC, porém, o custo de serializar e desserializar os payloads pode ser significativo, sem mencionar o tempo consumido para enviar informações pela rede. Isso significa que é preciso pensar de modo diferente no design de APIs para interfaces remotas em comparação com as interfaces locais. Apenas partir de uma API local e tentar fazer dela uma fronteira de serviço, sem pensar melhor a respeito do assunto, provavelmente fará com que você tenha problemas. Em alguns dos piores exemplos, os desenvolvedores poderiam utilizar chamadas remotas sem sabê-lo, caso a abstração seja excessivamente opaca.

É preciso pensar na rede propriamente dita. A primeira famosa falácia da computação distribuída é “A rede é confiável” (<https://oreil.ly/8J4Vh>). As redes *não são* confiáveis. Elas podem e vão falhar, mesmo que seu cliente e

o servidor com o qual você estiver conversando não tenham nenhum problema. As redes podem falhar rapidamente, podem falhar lentamente, e podem até mesmo deixar seus pacotes com um formato indevido. Você deve partir do pressuposto de que suas redes estão infestadas de entidades malévolas, prontas para lançar sua ira em um momento de fúria. Portanto, espere encontrar modos de falha com os quais jamais teve de lidar com um software monolítico mais simples. Uma falha poderia ser causada pelo servidor remoto devolvendo um erro, ou por uma chamada ruim que você tenha feito. Você saberia diferenciá-las e, em caso afirmativo, poderia fazer algo a respeito? E o que você fará se o servidor remoto simplesmente começar a demorar para responder? Discutiremos esse tópico quando falarmos de resiliência no Capítulo 12.

Fragilidade. Algumas das implementações mais populares de RPC podem levar a certas formas ruins de fragilidade, com o Java RMI sendo um bom exemplo disso. Vamos considerar uma interface Java bem simples para a qual decidimos criar uma API remota em nosso serviço cliente (`Customer`). O Exemplo 5.1 declara os métodos que vamos expor remotamente. O Java RMI então gera os stubs de cliente e de servidor para o nosso método.

Exemplo 5.1 – Definindo um endpoint de serviço usando Java RMI

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface CustomerRemote extends Remote {  
    public Customer findCustomer(String id) throws RemoteException;  
  
    public Customer createCustomer(  
        String firstname, String surname, String emailAddress)  
        throws RemoteException;  
}
```

Nessa interface, `createCustomer` recebe o primeiro nome, o sobrenome e o endereço de email. O que acontecerá se decidirmos que o objeto `customer` também poderá ser criado somente com um endereço de email? Poderíamos facilmente acrescentar um novo método nesse caso, assim:

```
...  
public Customer createCustomer(String emailAddress) throws RemoteException;
```

...

O problema é que agora precisamos gerar novamente os stubs de cliente também. Os clientes que queiram consumir o novo método precisam dos novos stubs e, dependendo da natureza das mudanças na especificação, os consumidores que não precisarem do novo método talvez precisem também que seus stubs sejam atualizados. Isso é administrável, é claro, mas somente até certo ponto. O fato é que alterações como essas são razoavelmente comuns. Os endpoints RPC geralmente acabam tendo vários métodos para diferentes maneiras de criar ou interagir com objetos. Em parte, isso se deve ao fato de ainda pensarmos nessas chamadas remotas como se fossem chamadas locais.

Entretanto, há outro tipo de fragilidade. Vamos analisar o aspecto de nosso objeto `Customer`:

```
public class Customer implements Serializable {  
    private String firstName;  
    private String surname;  
    private String emailAddress;  
    private String age;  
}
```

E se, embora expuséssemos o campo `age` em nossos objetos `Customer`, nenhum de nossos consumidores chegasse a usá-lo? Decidimos que queremos remover esse campo. No entanto, se `age` for removido da definição desse tipo na implementação do servidor, mas o mesmo não for feito em todos os consumidores, ainda que eles jamais tenham utilizado esse campo, o código associado à desserialização do objeto `customer` no lado do consumidor terá problemas. Para fazer o rollout dessa alteração, teríamos de fazer mudanças no código do cliente para que a nova definição seja aceita e implantar esses clientes atualizados no mesmo instante em que fizéssemos o rollout da nova versão do servidor. Esse é um problema sério em qualquer solução de RPC que promova o uso de geração de stubs binários: não é possível separar as implantações dos clientes e dos servidores. Se essa tecnologia for empregada, a necessidade de lançamentos sincronizados poderá estar presente em seu futuro.

Problemas semelhantes ocorrerão se quisermos reestruturar o objeto

`customer`, mesmo que não removêssemos nenhum campo – por exemplo, se quiséssemos encapsular `firstName` e `surname` em um novo tipo `naming` para facilitar o gerenciamento. Poderíamos, é claro, corrigir isso passando dicionários como parâmetros de nossas chamadas, mas, com isso, perderíamos muitas das vantagens dos stubs gerados, pois teríamos de associar e extrair manualmente os campos desejados.

Na prática, objetos usados como parte de uma serialização binária transmitida pela rede podem ser pensados como tipos “somente expansíveis”. Essa fragilidade resulta na transformação dos tipos expostos na transmissão em uma massa de campos, com alguns desses campos não mais sendo usados, porém não sendo possível removê-los com segurança.

Quando usar

Apesar de suas deficiências, na verdade, eu gosto muito do RPC, e as implementações mais modernas, como o gRPC, são excelentes, enquanto outras implementações têm problemas sérios, que me fariam passar bem longe delas. O Java RMI, por exemplo, tem uma série de problemas de fragilidade e opções limitadas de tecnologia, enquanto o SOAP é bastante pesado do ponto de vista de um desenvolvedor, sobretudo quando comparado às opções mais modernas.

Esteja ciente das possíveis armadilhas associadas ao RPC caso escolha esse modelo. Não faça uma abstração de suas chamadas remotas a ponto de ocultar totalmente a rede, e garanta que a interface do servidor possa evoluir sem precisar insistir em upgrades sincronizados com os clientes. Encontrar o equilíbrio certo para o código de seu cliente, por exemplo, é importante. Certifique-se de que seus clientes não desconheçam o fato de que uma chamada de rede será feita. Bibliotecas de cliente são muitas vezes usadas no contexto do RPC; se não estiverem devidamente estruturadas, elas poderão ser um problema. Falaremos mais dessas bibliotecas em breve.

Se eu estivesse procurando opções nessa área, o gRPC estaria no topo da minha lista. Criado para tirar proveito do HTTP/2, ele tem algumas características impressionantes de desempenho e, de modo geral, é fácil de

usar. Também gosto do ecossistema que há em torno do gRPC, incluindo ferramentas como o Protolock (<https://protolock.dev>), que será discutido mais adiante neste capítulo, quando falarmos de esquemas.

O gRPC se enquadra bem em um modelo síncrono de requisição-resposta, mas também pode funcionar em conjunto com extensões Reactive. Estará no topo da minha lista sempre que eu estiver em situações nas quais tenho uma boa dose de controle do lado tanto do cliente como do servidor no espectro. Se você precisa oferecer suporte a uma ampla variedade de outras aplicações que terão de conversar com seus microsserviços, a necessidade de compilar um código do lado cliente com base em um esquema do lado do servidor pode ser um problema. Nesse caso, alguma forma de API REST sobre HTTP provavelmente seria mais apropriada.

REST

O REST (Representational State Transfer, ou Transferência de Representação de Estado) é um estilo de arquitetura inspirado na web. Há vários princípios e limitações associados ao estilo REST, mas vamos manter o foco naqueles que realmente nos ajudarão quando estivermos diante dos desafios de integração em um mundo de microsserviços, e quando estivermos procurando uma alternativa ao RPC para nossas interfaces de serviços.

O mais importante quando pensamos no REST é o conceito de recursos. Podemos pensar em um recurso como algo que o serviço propriamente dito conhece, por exemplo, um `cliente`. O servidor cria diferentes representações desse `cliente` por demanda. O modo como um recurso é mostrado ao mundo externo é totalmente desacoplado do modo como ele é armazenado internamente. Um serviço cliente poderia pedir uma representação JSON de um `cliente`, por exemplo, mesmo que ele estivesse armazenado em um formato totalmente diferente. Assim que um serviço cliente tiver uma representação desse `cliente`, ele poderá fazer requisições para modificá-lo, e o servidor poderá ou não concordar com elas.

Há muitos estilos diferentes de REST, e falarei deles apenas superficialmente. Recomendo bastante que você consulte o Modelo de

Maturidade de Richardson (<https://oreil.ly/AlDzu>), no qual os diferentes estilos de REST são comparados.

O REST, por si só, não fala de protocolos subjacentes, embora seja comumente utilizado com o HTTP. Já vi implementações de REST que utilizavam protocolos muito diferentes, porém isso pode exigir bastante trabalho. Alguns dos recursos que o HTTP nos oferece como parte de sua especificação, por exemplo, os verbos, facilitam a implementação do REST sobre HTTP; por outro lado, com outros protocolos, você teria de lidar com esses recursos por conta própria.

REST e HTTP

O próprio HTTP define alguns recursos convenientes que funcionam muito bem com o estilo REST. Por exemplo, os verbos HTTP (como GET, POST e PUT) já têm significados muito conhecidos na especificação do HTTP quanto ao modo como devem funcionar com os recursos. O estilo de arquitetura REST na verdade nos diz que esses verbos devem se comportar do mesmo modo em todos os recursos, e a especificação do HTTP por acaso define uma série de verbos que podemos usar. GET obtém um recurso de modo idempotente, por exemplo, enquanto POST cria um recurso. Isso significa que podemos evitar vários métodos `createCustomer` ou `editCustomer` diferentes. Em vez disso, podemos simplesmente fazer um POST de uma representação do cliente para pedir ao servidor que crie um recurso, e então poderemos iniciar uma requisição GET para obter uma representação do recurso. Conceitualmente, há um único *endpoint* na forma de um recurso `customer` (cliente) nesses casos, e as operações que podemos executar aí fazem parte do protocolo HTTP.

O HTTP também traz consigo um grande ecossistema de ferramentas e tecnologias de apoio. Podemos usar proxies de caching HTTP como o Varnish e balanceadores de carga como o `mod_proxy`, e várias ferramentas de monitoração já têm bastante suporte disponível para HTTP. Esses blocos de construção nos permitem lidar com grandes volumes de tráfego HTTP e encaminhá-los de forma inteligente, de um modo razoavelmente transparente. Também podemos utilizar todos os controles de segurança disponíveis no HTTP para proteger nossas comunicações. Da autenticação

básica aos certificados de clientes, o ecossistema do HTTP oferece muitas ferramentas para facilitar o processo de segurança; exploraremos melhor esse assunto no Capítulo 11. Apesar do que foi dito, para desfrutar essas vantagens, é preciso empregar o HTTP de forma apropriada. Utilize-o de modo indevido, e ele estará tão desprotegido e será tão difícil de escalar como qualquer outra tecnologia existente por aí. Utilize-o corretamente, porém, e você terá muita ajuda.

Observe que o HTTP pode ser usado para implementar o RPC também. O SOAP, por exemplo, é encaminhado por meio do HTTP, mas, infelizmente, utiliza muito pouco de sua especificação. Os verbos são ignorados, assim como itens simples como os códigos de erro do HTTP. Por outro lado, o gRPC foi projetado para tirar proveito dos recursos do HTTP/2, por exemplo, a capacidade de enviar vários streams de requisição-resposta por meio de uma única conexão. Mas é claro que, ao usar o gRPC, você não terá o REST somente porque está usando HTTP!

HATEOAS

Outro princípio introduzido no REST que pode nos ajudar a evitar o acoplamento entre cliente e servidor é o conceito de *hipermídia como a engine do estado da aplicação* (geralmente abreviado como HATEOAS [Hypermedia As The Engine Of Application State] – e nossa, isso realmente merecia uma abreviatura). É uma expressão bastante densa e um conceito muito interessante, portanto vamos esclarecê-lo um pouco melhor.

Hipermídia é um conceito em que um conteúdo contém links para vários outros conteúdos em diversos formatos (por exemplo, texto, imagens, áudios). Isso deve parecer muito familiar a você, pois é o que acontece em uma página web típica: seguimos links, que são uma forma de controles de hipermídia, para ver conteúdos relacionados. A ideia por trás do HATEOAS é que os clientes devem interagir com o servidor (podendo resultar em transições de estado) por meio desses links para outros recursos. Eles não precisam saber exatamente onde estão os clientes da loja no servidor porque conhecem o URI a ser acessado; em vez disso, os clientes procuram links e navegam por eles para encontrar o que precisam.

Esse é um conceito meio inusitado, portanto vamos dar um passo para trás e considerar como as pessoas interagem com uma página web, que já concordamos que tem controles abundantes de hipermídia.

Pense no site de compras Amazon.com. A localização do carrinho de compras mudou com o tempo. A imagem mudou. O link mudou. Mas, como seres humanos, somos suficientemente inteligentes para continuar vendo um carrinho de compras, sabemos o que ele é e interagimos com ele. Temos uma compreensão do que significa um carrinho de compras, ainda que a forma exata e o controle subjacente usados para representá-lo tenham mudado. Sabemos que, se quisermos visualizar o carrinho, esse é o controle com o qual devemos interagir. É assim que as páginas web podem mudar gradualmente com o tempo. Desde que esses contratos implícitos entre os clientes da loja e o site continuem sendo obedecidos, as mudanças não devem causar problemas.

Com os controles de hipermídia, tentamos alcançar o mesmo nível de “inteligência” para nossos consumidores eletrônicos. Vamos observar um controle de hipermídia que poderíamos ter na MusicCorp. No Exemplo 5.1, acessamos um recurso que representa uma entrada no catálogo, referente a um dado álbum. Junto com as informações sobre o álbum, vemos alguns controles de hipermídia.

Exemplo 5.2 – Controles de hipermídia usados em um álbum listado

```
<album>
  <name>Give Blood</name>
  <link rel="/artist" href="/artist/theBrakes" /> ❶
  <description>
    Awesome, short, brutish, funny and loud. Must buy!
  </description>
  <link rel="/instantpurchase"href="/instantPurchase/1234" /> ❷
</album>
```

❶ Esse controle de hipermídia nos mostra onde encontrar informações sobre o artista.

❷ Se quisermos comprar o álbum, agora sabemos para onde ir.

Nesse documento, temos dois controles de hipermídia. O cliente que lê um documento desse tipo precisa saber que um controle com um

relacionamento `artist` é aquele que ele deve acessar para obter informações sobre o artista, e que `instantpurchase` faz parte do protocolo usado na compra do álbum. O cliente precisa conhecer a semântica da API, do mesmo modo que uma pessoa precisa saber que, em um site de compras, o carrinho é o local em que os itens a serem comprados estarão.

Como cliente, não preciso saber qual é o esquema de URI a ser acessado para *comprar* o álbum; preciso apenas acessar o recurso, encontrar o controle para compra e navegar até ele. O controle para compra poderia mudar de lugar, o URI poderia mudar ou o site poderia até mesmo me enviar para outro serviço totalmente diferente e, como cliente, eu não me importaria. Isso nos proporciona uma boa dose de desacoplamento entre o cliente e o servidor.

Nesse caso, temos um alto nível de abstração dos detalhes subjacente. Poderíamos alterar totalmente a implementação para a apresentação do controle, desde que o cliente continue sendo capaz de encontrar o controle que corresponda ao seu entendimento do protocolo, do mesmo modo que um controle para carrinho de compras pode mudar de um simples link para um controle JavaScript mais complexo. Também temos a liberdade de acrescentar novos controles ao documento, talvez representando novas transições de estado que possamos fazer no recurso em questão. Poderíamos acabar causando problemas aos nossos consumidores somente se mudássemos a semântica básica de um dos controles, de maneira que ele se comportasse de modo muito diferente, ou se o removêssemos totalmente.

Ao usar esses controles para desacoplar o cliente e o servidor, teoricamente, teremos vantagens significativas com o tempo, as quais esperamos que compensem o tempo adicional necessário para fazer com que esses protocolos funcionem de modo apropriado. Infelizmente, embora todas essas ideias pareçam sensatas na teoria, tenho constatado que essa forma de REST raramente é colocada em prática, por motivos que não entendo completamente. Isso faz com que eu tenha mais dificuldade em promover o conceito de HATEOAS para aqueles que já se comprometeram com o uso do REST. Basicamente, muitas das ideias do

REST estão voltadas à criação de sistemas de hipermídia distribuídos, e isso não é o que a maioria das pessoas acaba criando.

Desafios

No que diz respeito à facilidade de consumo, historicamente, não éramos capazes de gerar um código do lado cliente para o seu protocolo de aplicação REST sobre HTTP, como podemos fazer com implementações RPC. Muitas vezes, isso leva as pessoas a criarem APIs REST que disponibilizem bibliotecas de cliente para os consumidores poderem usar. Essas bibliotecas de cliente oferecem um binding (vinculação) para a API a fim de facilitar a integração do cliente. O problema é que as bibliotecas de cliente podem causar algumas complicações quanto ao acoplamento entre o cliente e o servidor – algo que discutiremos na seção “DRY e os perigos da reutilização de código em um mundo de microsserviços”, na página [190](#).

Nos últimos anos, esse problema, de certo modo, foi atenuado. A especificação OpenAPI (<https://oreil.ly/ldr1p>), que surgiu a partir do projeto Swagger, atualmente dá a você a capacidade de definir informações suficientes em um endpoint REST para permitir a geração de código do lado cliente em diversas linguagens. Com base em minha experiência, posso dizer que não vi muitas equipes fazendo uso efetivamente dessa funcionalidade, ainda que já estivessem usando o Swagger para documentação. Suspeito que isso se deva às dificuldades de fazer uma readequação de seu uso para as APIs atuais. Também tenho preocupações sobre uma especificação que anteriormente era utilizada somente para documentação, e agora é empregada para definir um contrato mais explícito. Isso pode levar a uma especificação muito mais complexa – comparar um esquema OpenAPI com um esquema de buffer de protocolo, por exemplo, revela um contraste gritante. Apesar de minhas reservas, porém, é bom que temos agora essa opção.

O desempenho também pode ser um problema. Payloads REST sobre HTTP na verdade podem ser mais compactos que o SOAP porque o REST aceita formatos alternativos como JSON ou até mesmo binário; apesar disso, ele não está nem perto de ser um protocolo binário tão enxuto quanto o do Thrift. O overhead do HTTP para cada requisição também

pode ser uma preocupação para requisitos de baixa latência. Todos os protocolos HTTP convencionais em uso atualmente exigem a utilização do TCP (Transmission Control Protocol, ou Protocolo de Controle de Transmissão) internamente, o qual tem suas ineficiências quando comparado com outros protocolos de rede; algumas implementações de RPC permitem usar protocolos de rede alternativos ao TCP, por exemplo, o UDP (User Datagram Protocol, ou Protocolo de Datagrama de Usuário).

As limitações impostas ao HTTP em virtude do requisito de utilizar o TCP estão sendo resolvidas. O HTTP/3, que atualmente está no processo de ser finalizado, procura fazer uma mudança em direção ao uso do protocolo QUIC mais recente. O QUIC oferece os mesmos tipos de recursos que o TCP (por exemplo, melhores garantias em relação ao UDP), porém com melhorias que se mostraram significativas na redução de latência e no uso de banda. É provável que vão ser necessários vários anos até que o HTTP/3 tenha um impacto generalizado na internet pública, mas parece razoável supor que as empresas poderão desfrutar mais cedo essas melhorias em suas próprias redes.

Com respeito especificamente ao HATEOAS, poderemos deparar com outros problemas de desempenho. Como os clientes precisam navegar por vários controles para encontrar os endpoints corretos para uma dada operação, isso poderia resultar em protocolos com muita comunicação – muitas idas e vindas poderiam ser necessárias para cada operação. Em última análise, é uma relação de custo-benefício. Se você decidir adotar um estilo de REST com HATEOAS, eu sugiro que você comece fazendo com que seus clientes acessem esses controles antes, e então otimizem depois se for necessário. Lembre-se de que o uso do HTTP significa que teremos muita ajuda à disposição, algo que já discutimos antes. Os perigos de uma otimização prematura já foram bem documentados, portanto não precisarei detalhá-los neste livro. Observe também que muitas dessas abordagens foram desenvolvidas para criar sistemas de hipertexto distribuídos, e nem todas serão apropriadas! Às vezes, você se verá simplesmente querendo um bom e velho RPC.

Apesar dessas desvantagens, o REST sobre HTTP é uma opção padrão

sensata para interações entre serviços. Se quiser saber mais, recomendo o livro *REST in Practice: Hypermedia and Systems Architecture* (O'Reilly) de Jim Webber, Savas Parastatidis e Ian Robinson, que aborda o tópico REST sobre HTTP com detalhes.

Quando usar

Em virtude de seu uso disseminado no mercado, uma API baseada em REST sobre HTTP é uma escolha óbvia para uma interface síncrona com requisição-resposta se você está querendo permitir um acesso pela maior variedade possível de clientes. Seria um erro pensar em uma API REST apenas como uma opção “suficientemente boa para a maioria das tarefas”, mas há certa verdade nisso. É um estilo de interface amplamente conhecido, com o qual a maioria das pessoas tem familiaridade, e ele garante interoperabilidade com diversas tecnologias.

Em virtude principalmente dos recursos do HTTP e do fato de o REST ter esses recursos como base (em vez de ocultá-los), as APIs baseadas em REST se destacam nas situações em que queremos um caching de requisições eficiente e em larga escala. É por esse motivo que elas são a opção óbvia para expor APIs a componentes externos ou interfaces de clientes. Contudo, elas podem estar em desvantagem quando comparadas com protocolos de comunicação mais eficientes, e embora possamos criar protocolos de interação assíncrona com base em APIs REST, essa não seria realmente uma ótima opção se comparada às alternativas para comunicação entre microsserviços em geral.

Apesar de apreciar intelectualmente os objetivos por trás do HATEOAS, ainda não vi muitas evidências de que o trabalho adicional para implementar esse estilo de REST proporcione vantagens que compensem no longo prazo, nem me lembro de ter conversado nos últimos anos com nenhuma equipe que estivesse implementando uma arquitetura de microsserviços que pudesse defender a importância de usar o HATEOAS. Minhas experiências representam, obviamente, apenas um conjunto de dados, e eu não duvido de que, para algumas pessoas, o HATEOAS possa ter funcionado bem. Entretanto, esse conceito não parece ter feito tanto sucesso quanto eu achei que faria. Pode ser que os conceitos na base do

HATEOAS sejam muito diferentes para que pudéssemos entendê-los, ou talvez seja a falta de ferramentas ou padrões nessa área ou, quem sabe, o modelo simplesmente não funcione para os tipos de sistemas que acabamos criando. Também é possível, é claro, que os conceitos nos quais o HATEOAS se baseia não combinem muito bem com o modo como criamos os microsserviços.

Assim, para uso no perímetro, ele funciona incrivelmente bem, e para uma comunicação síncrona baseada em requisições-respostas entre microsserviços, ele é ótimo.

GraphQL

Nos últimos anos, o GraphQL (<https://graphql.org>) ganhou mais popularidade, principalmente pelo fato de ele se destacar bem em uma área específica. O GraphQL permite que um dispositivo do lado cliente defina queries que possam evitar a necessidade de fazer várias requisições a fim de obter as mesmas informações. Isso pode proporcionar melhorias significativas no que concerne ao desempenho de dispositivos limitados do lado cliente, além de evitar a necessidade de implementar uma agregação personalizada do lado do servidor.

Para um exemplo simples, pense em um dispositivo móvel que queira exibir uma página apresentando uma visão geral dos últimos pedidos feitos por um cliente. A página precisa conter algumas informações sobre o cliente, além de informações sobre os seus cinco últimos pedidos. A tela precisa somente de alguns campos do registro do cliente, e apenas a data, o valor e o status de entrega de cada pedido. O dispositivo móvel poderia fazer chamadas para dois microsserviços downstream a fim de obter as informações necessárias, mas isso envolveria fazer várias chamadas, incluindo a obtenção de informações que não seriam realmente necessárias. Particularmente com os dispositivos móveis, isso seria um desperdício – há um consumo maior do plano de dados de um dispositivo móvel, além do necessário, e pode haver mais demora.

O GraphQL permite que o dispositivo móvel execute uma única query para obter todas as informações necessárias. Para que isso funcione, você

precisará de um microserviço que exponha um endpoint GraphQL ao dispositivo cliente. Esse endpoint GraphQL é a entrada para todas as queries de clientes e expõe um esquema para que os dispositivos clientes possam usar. Esse esquema expõe os tipos disponíveis ao cliente, e um bom query builder (construtor de consultas) gráfico também é oferecido para facilitar a criação dessas queries. Ao reduzir o número de chamadas e a quantidade de dados acessada pelo dispositivo cliente, você poderá lidar muito bem com alguns dos desafios que surgem ao criar interfaces de usuário com arquiteturas de microserviços.

Desafios

No início, um dos desafios era a falta de suporte a linguagens para a especificação do GraphQL, e o JavaScript era a única opção disponível. Isso melhorou bastante, com todas as principais tecnologias atualmente oferecendo suporte para a especificação. Com efeito, houve melhorias significativas no GraphQL e em suas diversas implementações no mercado, fazendo dele uma possibilidade muito menos arriscada em comparação com o que tínhamos alguns anos atrás. Apesar disso, você deve estar ciente de alguns desafios que ainda existem com essa tecnologia.

Para começar, o dispositivo cliente pode executar queries que mudem dinamicamente, e já ouvi falar de equipes que tiveram problemas com consultas do GraphQL, provocando uma carga significativa do lado do servidor, como resultado dessa funcionalidade. Quando comparamos o GraphQL com algo como o SQL, vemos um problema semelhante. Uma instrução SQL custosa pode provocar problemas significativos em um banco de dados, podendo causar um grande impacto no sistema mais amplo. O mesmo problema se aplica ao GraphQL. A diferença é que, com o SQL, ao menos temos ferramentas como query planners (planejadores de consultas) para nossos bancos de dados, que podem nos ajudar a diagnosticar queries problemáticas, enquanto um problema semelhante com o GraphQL poderia ser mais difícil de ser rastreado. Um throttling (limitação) de requisições do lado do servidor seria uma possível solução, mas, como a execução da chamada poderia estar espalhada entre vários microserviços, isso estaria longe de ser uma tarefa simples.

Em comparação com APIs REST comuns, baseadas em HTTP, o caching também seria mais complicado. Com APIs baseadas em REST, posso definir um dos muitos cabeçalhos da resposta para ajudar os dispositivos do lado cliente – ou caches intermediários como os CDNs (Content Delivery Networks, ou Redes de Fornecimento de Conteúdo) – a fazerem o cache das respostas, de modo que não precisassem ser solicitadas novamente. Não é possível fazer o mesmo com o GraphQL. Os conselhos que já vi relacionados a esses problemas aparentemente dizem respeito a associar um ID a cada recurso devolvido (e lembre-se de que uma query GraphQL poderia conter vários recursos), e então fazer o dispositivo cliente efetuar um cache da requisição com base nesse ID. Até onde posso dizer, isso dificultará extremamente o uso de CDNs ou de caching em proxies reversos se um trabalho adicional não for feito.

Embora eu já tenha visto algumas soluções com implementações específicas para esse problema (por exemplo, aquelas encontradas na implementação do JavaScript Apollo), tenho a impressão de que o caching foi ignorado, de forma consciente ou não, como parte do desenvolvimento inicial do GraphQL. Se as queries que você estiver executando forem de natureza extremamente específica para um usuário em particular, essa falta de caching no nível das requisições não seria um motivo para deixar de usá-lo, é claro, pois sua taxa de acertos no cache provavelmente seria baixa. Fico pensando, porém, se essa limitação significa que você acabará com uma solução híbrida para os dispositivos clientes, com algumas requisições (mais genéricas) usando APIs REST comuns, baseadas em HTTP, enquanto outras utilizariam o GraphQL.

Outro problema é que, embora o GraphQL, teoricamente, seja capaz de lidar com escritas, ele não parece ser tão apropriado para isso, do mesmo modo que é para leituras. Isso leva a situações em que as equipes utilizariam o GraphQL para leitura, mas o REST para escritas.

O último problema é algo que pode ser totalmente subjetivo, mas, apesar disso, acho que vale a pena mencioná-lo. O GraphQL dá a impressão de que estamos trabalhando apenas com dados, o que pode reforçar a ideia de que os microsserviços com os quais estivermos conversando seriam apenas

wrappers em torno de bancos de dados. Com efeito, já vi muitas pessoas compararem o GraphQL com o OData, uma tecnologia criada para ser uma API genérica para acesso a bancos de dados. Como já discutimos bastante, a ideia de tratar os microsserviços somente como wrappers em torno de bancos de dados pode ser bem problemática. Os microsserviços expõem funcionalidades por meio de interfaces na rede. Algumas dessas funcionalidades podem exigir ou resultar na exposição de dados, mas elas ainda devem ter sua própria lógica e comportamento internos. Só porque você está usando o GraphQL, não comece a pensar em seus microsserviços somente como algo um pouco além de uma API para um banco de dados – é essencial que sua API GraphQL não esteja acoplada aos repositórios de dados subjacentes aos seus microsserviços.

Quando usar

O lugar ideal para usar o GraphQL é no perímetro do sistema, expondo funcionalidades aos clientes externos. Esses clientes em geral são GUIs, e o GraphQL é muito apropriado para dispositivos móveis, considerando suas limitações no que concerne à capacidade de apresentar dados ao usuário final e a natureza das redes móveis. Contudo, o GraphQL também pode ser usado para APIs externas, e o GitHub foi um dos primeiros a adotá-lo. Se você tem uma API externa à qual seus clientes externos fazem várias chamadas com frequência a fim de obter as informações de que precisam, o GraphQL poderá ajudar a deixar a API muito mais eficiente e conveniente.

Basicamente, o GraphQL é um método de agregação e filtragem de chamadas, portanto, no contexto de uma arquitetura de microsserviços, ele seria utilizado para agregar chamadas para vários microsserviços downstream. Desse modo, ele não seria algo que substituiria uma comunicação genérica de microsserviço para microsserviço.

Uma alternativa ao uso do GraphQL seria considerar um padrão diferente como o padrão BFF (Backend For Frontend) – veremos esse padrão e o compararemos com o GraphQL e com outras técnicas de agregação no Capítulo 14.

Brokers de mensagens

Os brokers de mensagens são intermediários, muitas vezes chamados de middleware, que ficam entre os processos para gerenciar a comunicação entre eles. São uma opção popular para ajudar a implementar uma comunicação assíncrona entre os microsserviços, pois oferecem vários recursos eficazes.

Como já discutimos antes, uma mensagem é um conceito genérico que define aquilo que um broker de mensagens envia. Uma mensagem pode conter uma requisição, uma resposta ou um evento. Em vez de um microsserviço se comunicar diretamente com outro microsserviço, o microsserviço envia uma mensagem para um broker, com informações sobre como ela deve ser enviada.

Tópicos e filas

Os brokers tendem a disponibilizar filas ou tópicos, ou ambos. As filas em geral são ponto a ponto. Um remetente envia uma mensagem a uma fila, e um consumidor a lê dessa fila. Com um sistema baseado em tópicos, vários consumidores podem se inscrever para um tópico, e cada consumidor inscrito receberá uma cópia dessa mensagem.

Um consumidor poderia representar um ou mais microsserviços – em geral, modelados como um grupo consumidor. Isso será conveniente se você tiver várias instâncias de um microsserviço, e quiser que qualquer um deles seja capaz de receber uma mensagem. Na Figura 5.1, vemos um exemplo no qual `Processador de Pedido` tem três instâncias implantadas, todas como parte do mesmo grupo consumidor. Quando uma mensagem é inserida na fila, somente um membro do grupo consumidor a receberá; isso significa que a fila funciona como um método de distribuição de carga. Esse é um exemplo do padrão de consumidores concorrentes (competing consumers pattern) que mencionamos rapidamente no Capítulo 4.

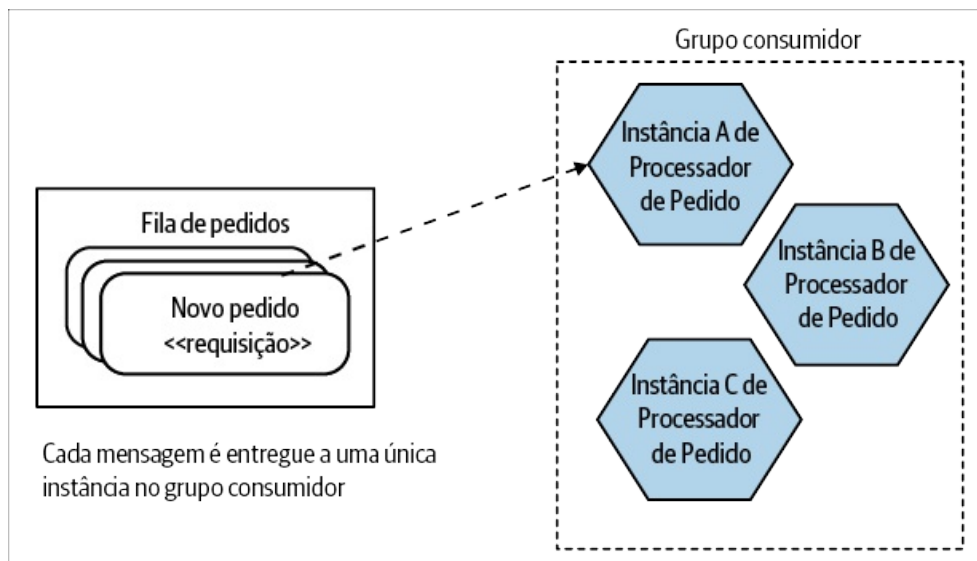


Figura 5.1 – Uma fila permite que haja um grupo consumidor.

Com os tópicos, podemos ter vários grupos consumidores. Na Figura 5.2, um evento representando um pedido pago é inserido no tópico *Status do Pedido*. Uma cópia desse evento é recebida tanto pelo microserviço *Depósito* como pelo microserviço *Notificações*, que estão em grupos consumidores distintos. Apenas uma instância em cada grupo consumidor verá esse evento.

À primeira vista, uma fila se parece com um tópico com um único grupo consumidor. Boa parte da distinção entre ambos está no fato de que, quando uma mensagem é enviada por meio de uma fila, sabe-se para quem a mensagem será enviada. No caso do tópico, essa informação não é conhecida pelo originador da mensagem – ele não sabe quem (se houver alguém) acabará recebendo a mensagem.

Os tópicos são muito apropriados para uma colaboração baseada em eventos, enquanto as filas seriam mais adequadas para uma comunicação com requisição/resposta. Isso, porém, deve ser considerado como uma orientação geral, e não como uma regra rígida.

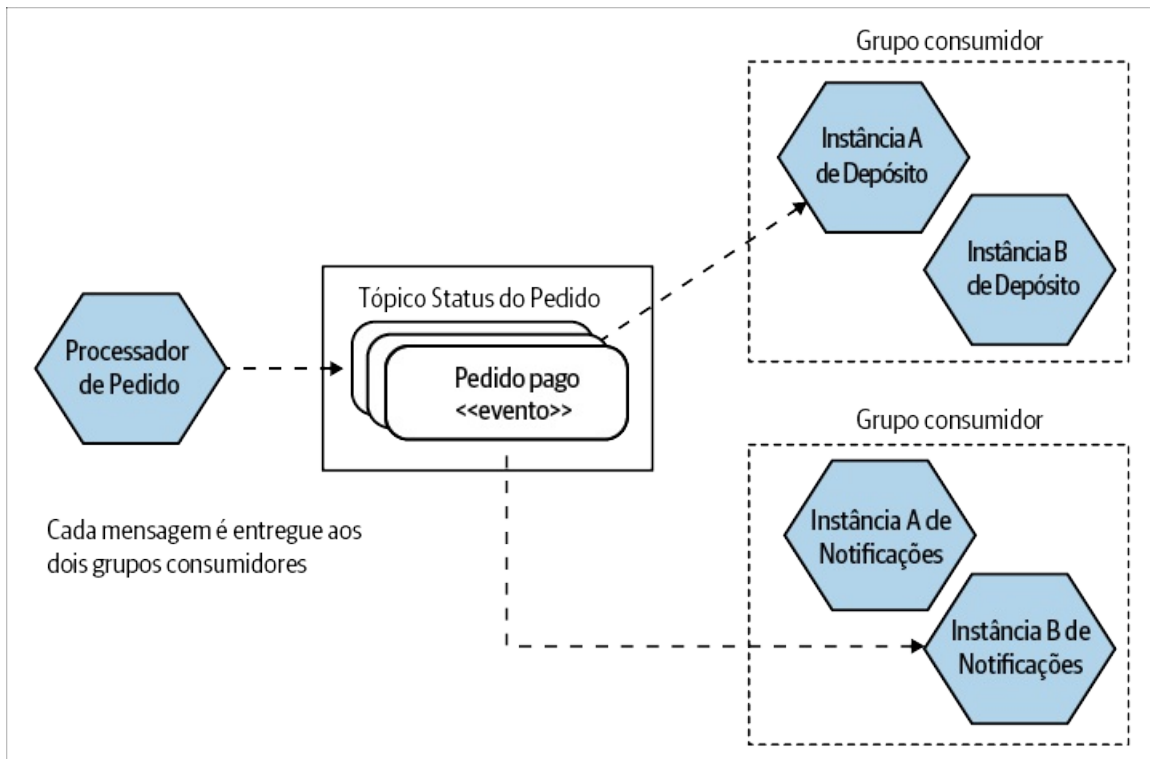


Figura 5.2 – Os tópicos permitem que vários inscritos recebam as mesmas mensagens, o que é conveniente para um broadcast de evento.

Garantia de entrega

Então, por que usar um broker? Basicamente, os brokers oferecem alguns recursos que podem ser muito convenientes em uma comunicação assíncrona. As propriedades que eles oferecem variam, mas o recurso mais interessante é a garantia de entrega, algo que todos os brokers amplamente utilizados proporcionam de alguma forma. Uma garantia de entrega descreve um compromisso do broker em garantir que a mensagem seja entregue.

Do ponto de vista do microserviço que envia a mensagem, isso pode ser muito útil. O fato de o destinatário downstream estar indisponível não será um problema – o broker armazenará a mensagem até que ela possa ser entregue. Com isso, podemos reduzir o número de itens com os quais um microserviço upstream precisa se preocupar. Compare esse caso com uma chamada síncrona direta – por exemplo, uma requisição HTTP: se o destino downstream não estiver acessível, o microserviço upstream terá de decidir o que fazer com a requisição; deverá tentar fazer a chamada

novamente ou desistir?

Para que haja garantia na entrega, um broker precisa garantir que qualquer mensagem que ainda não tenha sido entregue seja armazenada de forma duradoura até que ela possa ser entregue. Para cumprir essa promessa, um broker geralmente executa na forma de algum tipo de sistema baseado em cluster, garantindo que a perda de uma única máquina não cause a perda de uma mensagem. Em geral, há vários aspectos envolvidos na execução correta de um broker, parcialmente por causa dos desafios em gerenciar um software baseado em cluster. Muitas vezes, a promessa de uma garantia de entrega pode ser prejudicada se o broker não estiver configurado corretamente. Como exemplo, o RabbitMQ exige que as instâncias em um cluster se comuniquem por meio de redes com uma latência relativamente baixa; caso contrário, as instâncias poderão ficar confusas acerca do estado atual das mensagens sendo tratadas, resultando na perda de dados. Não estou de forma alguma enfatizando essa limitação em particular como uma forma de dizer que o RabbitMQ seja ruim – todos os brokers têm restrições sobre como devem ser executados para cumprirem a promessa de garantia de entrega. Se você planeja executar o seu próprio broker, não se esqueça de ler com atenção a documentação.

Também vale a pena observar que aquilo que cada broker em particular quer dizer com garantia de entrega pode variar. Novamente, ler a documentação é um ótimo ponto de partida.

Confiança

Um dos principais atrativos de um broker é a propriedade de garantia de entrega. Contudo, para que isso funcione, você precisa confiar não só nas pessoas que criaram o broker, como também no modo como o broker opera. Se você criou seu sistema com base na suposição de uma entrega garantida, mas isso acabar não acontecendo por causa de um problema no broker subjacente, você poderá ter problemas significativos. A expectativa, é claro, é que você passe esse trabalho para um software criado por pessoas que possam fazê-lo melhor do que você. Em última instância, você precisa decidir até que ponto vai confiar no broker que estiver usando.

Outras características

Além da garantia de entrega, os brokers podem ter outras características que você talvez ache convenientes.

A maioria dos brokers é capaz de garantir a ordem na qual as mensagens são entregues, mas isso não é universal, e até mesmo o escopo dessa garantia pode ser limitado. Com o Kafka, por exemplo, a ordem é garantida somente dentro de uma única partição. Se você não puder ter certeza de que as mensagens serão recebidas na ordem, seu consumidor talvez precise efetuar alguma espécie de compensação, talvez postergando o processamento das mensagens recebidas fora de ordem até que qualquer mensagem que esteja faltando seja recebida.

Alguns brokers oferecem transações na escrita – por exemplo, o Kafka permite que você escreva em vários tópicos em uma única transação. Outros brokers também podem oferecer transações para leitura, algo do qual já tirei proveito quando utilizei vários brokers com as APIs do JMS (Java Message Service, ou Serviço de Mensagens do Java). Isso pode ser útil se você quiser garantir que a mensagem seja processada pelo consumidor antes que ela seja removida do broker.

Outro recurso, de certa forma controverso, prometido por alguns brokers é o da entrega única. Uma das maneiras mais fáceis de oferecer uma garantia de entrega é permitir que a mensagem seja reenviada. Isso pode resultar em um consumidor vendo a mesma mensagem mais de uma vez (ainda que essa seja uma situação rara). A maioria dos brokers fará o que for possível para reduzir as chances de isso acontecer, ou ocultará esse fato do consumidor, porém alguns brokers vão além, garantindo que uma entrega seja efetuada somente uma vez. Esse é um assunto complexo, pois já falei com alguns especialistas que afirmam que garantir uma entrega exatamente apenas uma vez em todos os casos é impossível, enquanto outros dizem que isso pode ser feito basicamente com algumas soluções de contorno simples. Qualquer que seja o caso, se seu broker preferido alega implementar esse recurso, preste *realmente* muita atenção no modo como isso está implementado. Melhor ainda, crie seus consumidores de modo que estejam preparados para o fato de poderem receber uma mensagem

mais de uma vez e saibam lidar com essa situação. Um exemplo muito simples seria fazer com que cada mensagem tenha um ID, que poderá ser verificado pelo consumidor sempre que uma mensagem for recebida. Se uma mensagem com o ID em questão já tiver sido processada, a nova mensagem poderá ser ignorada.

Opções

Há diversas opções de brokers de mensagens. Exemplos conhecidos incluem o RabbitMQ, o ActiveMQ e o Kafka (que exploraremos com mais detalhes a seguir). Os principais provedores de nuvem pública também oferecem diversos produtos que desempenham essa função, desde versões gerenciadas desses brokers, que você pode instalar em sua própria infraestrutura, até implementações personalizadas, específicas para uma determinada plataforma. A AWS, por exemplo, tem o SQS (Simple Queue Service, ou Serviço de Fila Simples), o SNS (Simple Notification Service, ou Serviço de Notificação Simples) e o Kinesis, cada um oferecendo diferentes variações de um broker totalmente gerenciado. O SQS, com efeito, foi o segundo produto lançado pela AWS, lá pelos idos de 2006.

Kafka

Vale a pena destacar o Kafka como um broker específico, principalmente por causa de sua recente popularidade. Parte dessa popularidade se deve ao uso do Kafka para auxiliar na transferência de grandes volumes de dados, como parte da implementação de pipelines de processamento de streams. Isso pode ajudar na transição de processamentos baseados em batch (lote) para um processamento mais próximo do tempo real.

Há algumas características do Kafka que valem a pena ressaltar. Em primeiro lugar, ele foi projetado para uma escala bem grande – foi criado no LinkedIn para substituir vários clusters de mensagens existentes por uma única plataforma. O Kafka foi criado para permitir vários consumidores e produtores – falei com um especialista em uma grande empresa de tecnologia que tinha mais de cinquenta mil produtores e consumidores funcionando no mesmo cluster. Para ser franco, pouquíssimas empresas têm problemas nesse nível de escala, mas, para

algumas, a capacidade de escalar facilmente o Kafka (falando de modo relativo) pode ser muito conveniente.

Outro recurso quase exclusivo do Kafka é a persistência de mensagens. Em um broker de mensagens comum, assim que o último consumidor tiver recebido uma mensagem, o broker não precisaria mais armazenar essa mensagem. No Kafka, as mensagens podem ser armazenadas por um período configurável. Isso significa que as mensagens podem ser armazenadas indefinidamente. Esse recurso permite que os consumidores possam reprocessar mensagens que já haviam consumido, ou permitir que consumidores recém-implantados processem mensagens que haviam sido enviadas antes.

Por fim, o Kafka tem incluído suporte para processamento de streams. Em vez de usar o Kafka para enviar mensagens para uma ferramenta dedicada de processamento de streams, como o Apache Flink, algumas tarefas podem ser feitas no próprio Kafka. Ao usar o KSQL, podemos definir instruções semelhantes ao SQL, capazes de processar um ou mais tópicos durante a execução. Esse recurso pode proporcionar a você algo semelhante a uma visão de banco de dados materializada, atualizada dinamicamente, cuja fonte de dados seriam os tópicos do Kafka em vez de um banco de dados. Essas funcionalidades dão abertura para algumas possibilidades muito interessantes acerca de como os dados são gerenciados em sistemas distribuídos. Se quiser explorar essas ideias com mais detalhes, posso recomendar o livro *Designing Event-Driven Systems* (O'Reilly) de Ben Stopford. (Preciso recomendar o livro de Ben, pois fui eu que escrevi seu prefácio!) Para uma exploração mais detalhada do Kafka em geral, sugiro o livro *Kafka: The Definitive Guide* (O'Reilly) de Neha Narkhede, Gwen Shapira e Todd Palino.

Formatos de serialização

Algumas das opções de tecnologia que vimos – em particular, algumas das implementações de RPC – fazem escolhas para você no que diz respeito ao modo como os dados são serializados e desserializados. Com o gRPC, por exemplo, qualquer dado enviado será convertido no formato de buffer de

protocolo. Muitas das opções de tecnologia, porém, nos dão bastante liberdade para converter os dados nas chamadas de rede. Escolha o Kafka como sua opção de broker, e você poderá enviar mensagens em diversos formatos. Então, qual formato você deve escolher?

Formatos textuais

O uso de formatos textuais padrões dão aos clientes muita flexibilidade para consumir recursos. As APIs REST, na maioria das vezes, utilizam um formato textual para os corpos das requisições e respostas, ainda que, teoricamente, você possa muito bem enviar dados binários por meio do HTTP. Com efeito, é assim que o gRPC funciona – ele utiliza o HTTP na camada inferior, porém envia buffers de protocolo binários.

O JSON superou o XML como opção para o formato de serialização de texto. Podemos encontrar uma série de motivos para explicar por que isso aconteceu, mas a principal razão é que um dos principais consumidores de APIs geralmente é um navegador, para o qual o JSON é muito apropriado. O JSON se tornou popular parcialmente como resultado da reação contrária ao XML, e os proponentes citam sua relativa compactação e a simplicidade quando comparadas ao XML como outro fator a favor do JSON. A verdade é que raramente há uma diferença enorme entre o tamanho de um payload JSON e um payload XML, sobretudo porque esses payloads em geral são compactados. Também vale a pena destacar que parte da simplicidade do JSON tem seu custo – em nossa pressa de adotar protocolos mais simples, os esquemas foram jogados pela janela (mais sobre esse assunto depois).

O Avro é um formato de serialização interessante. Aceita JSON como estrutura subjacente e o utiliza para definir um formato baseado em esquema. O Avro ganhou bastante popularidade como um formato para payload de mensagens, parcialmente por causa de sua capacidade de enviar o esquema como parte do payload, o que facilita bastante o suporte a vários formatos diferentes de mensagens.

Pessoalmente, porém, continuo sendo fã do XML. O suporte de algumas ferramentas ao XML é melhor. Por exemplo, se eu quisesse extrair somente

determinadas partes do payload (uma técnica que discutiremos com mais detalhes na seção “Lidando com mudanças entre microserviços”, na página [177](#)), poderia utilizar o XPATH, que é um padrão bem conhecido, com muitas ferramentas para suporte, ou até mesmo seletores CSS, que muitos acham ainda mais fácil. Com o JSON, temos o JSONPath, mas ele não tem um suporte tão amplo. Acho estranho as pessoas escolherem o JSON por ser bom e leve, mas depois tentarem incluir aí conceitos como controles de hipermídia, que já existem no XML. Aceito, porém, o fato de que, provavelmente, sou a minoria nesse caso, e que o JSON é o formato preferido por muitas pessoas!

Formatos binários

Embora os formatos textuais tenham vantagens como facilitar a leitura para as pessoas, além de possibilitarem um grau elevado de interoperabilidade com diferentes ferramentas e tecnologias, o mundo dos protocolos de serialização binários é onde você vai querer estar se estiver começando a se preocupar com o tamanho dos payloads ou com a eficiência na escrita e leitura dos payloads. Os buffers de protocolo existem há um tempo e, com frequência, são usados fora do escopo do gRPC – provavelmente representam o formato mais conhecido de serialização de dados binários para uma comunicação entre microserviços.

Contudo, essa área é ampla, e vários outros formatos foram desenvolvidos, com diferentes requisitos em mente. Simple Binary Encoding (<https://oreil.ly/p8UbH>), Cap’n Proto (<https://capnproto.org>) e FlatBuffers (<https://oreil.ly/VdqVB>) são exemplos que vêm à mente. Embora benchmarks para cada um desses formatos sejam abundantes, destacando suas principais vantagens em comparação com buffers de protocolo, JSON ou outros formatos, os benchmarks sofrem de um problema fundamental, que é o fato de não necessariamente refletirem o modo como você irá utilizar esses protocolos. Se pretende tirar proveito de seu formato de serialização até os últimos bytes, ou quer eliminar microssegundos do tempo necessário para ler ou escrever esses payloads, recomendo que faça suas próprias comparações entre esses diversos formatos. Com base em minha experiência, posso dizer que a grande maioria dos sistemas

raramente precisará se preocupar com otimizações desse tipo, pois geralmente poderão conseguir as melhorias que procuram enviando menos dados ou deixando totalmente de fazer a chamada. No entanto, se você estiver criando um sistema distribuído com uma latência extremamente baixa, certifique-se de que está preparado para mergulhar de cabeça no mundo dos formatos de serialização binários.

Esquemas

Uma discussão que vem à tona ocasionalmente é se devemos usar esquemas (schemas) para definir o que nossos endpoints expõem e aceitam. Os esquemas podem ser de vários tipos diferentes, e escolher um formato de serialização em geral definirá a tecnologia de esquema que você poderá usar. Se estiver trabalhando com XML puro, você utilizará o XSD (XML Schema Definition, ou Definição de Esquema XML); se estiver trabalhando com JSON puro, usará o JSON Schema. Algumas das opções de tecnologia que mencionamos (particularmente, um subconjunto considerável das opções de RPC) exigem o uso de esquemas explícitos; portanto, se você escolher essas tecnologias, terá de usar esquemas. O SOAP funciona com o uso de uma WSDL, enquanto o gRPC exige a utilização de uma especificação de buffer de protocolo. Outras opções de tecnologia que exploramos fazem uso opcional de esquemas, e é aí que a situação começa a ficar interessante.

Como já discutimos, sou a favor de ter esquemas explícitos para endpoints de microsserviços por dois motivos principais. Em primeiro lugar, eles ajudam muito como uma representação explícita daquilo que um endpoint de microsserviço expõe e pode aceitar. Isso facilita a vida tanto dos desenvolvedores que trabalham com o microsserviço como de seus consumidores. Os esquemas não substituem a necessidade de uma boa documentação, mas, sem dúvida, podem ajudar a reduzir o volume de documentação necessário.

O outro motivo pelo qual gosto de esquemas explícitos, porém, é que eles ajudam a identificar incompatibilidades acidentais nos endpoints de microsserviços. Exploraremos como lidar com mudanças entre

microserviços em breve, mas, antes disso, vale a pena explorar os diferentes tipos de incompatibilidades e o papel que os esquemas podem desempenhar.

Incompatibilidades de contrato estruturais versus semânticas

Falando de modo geral, podemos separar as incompatibilidades de contrato em duas categorias – incompatibilidades *estruturais* e incompatibilidades *semânticas*. Uma incompatibilidade estrutural é a situação em que a estrutura do endpoint muda de tal modo que um consumidor agora deixaria de ser compatível – poderia ocorrer por causa da remoção de campos ou métodos, ou do acréscimo de novos campos necessários. Uma incompatibilidade semântica refere-se a uma situação em que a estrutura do endpoint do microserviço permanece a mesma, porém há uma mudança no comportamento, de modo que as expectativas dos consumidores não serão mais atendidas.

Vamos considerar um exemplo simples. Suponha que você tenha um microserviço extremamente complexo chamado `Hard Calculations` que expõe um método `calculate` em seu endpoint. Esse método `calculate` aceita dois inteiros, e os dois campos são obrigatórios. Se você mudasse `Hard Calculations` de modo que o método `calculate` agora aceitasse somente um inteiro, isso deixaria os consumidores incompatíveis – eles enviariam requisições com dois inteiros, as quais seriam rejeitadas pelo microserviço `Hard Calculations`. Esse é um exemplo de uma mudança estrutural e, em geral, esse tipo de mudança pode ser mais fácil de ser identificado.

Uma mudança semântica é mais problemática. Ela ocorre quando a estrutura do endpoint não muda, porém o comportamento sofre alterações. Voltando para o nosso método `calculate`, suponha que, na primeira versão, os dois inteiros fornecidos fossem somados e o resultado fosse devolvido. Até agora, não há problemas. Modificamos então `Hard Calculations` de modo que o método `calculate` passe a multiplicar os inteiros e devolva o resultado. A semântica do método `calculate` mudou de tal modo que as expectativas dos consumidores não serão mais atendidas.

Devemos usar esquemas?

Ao usar esquemas e comparar suas diferentes versões, podemos identificar as incompatibilidades estruturais. Identificar as incompatibilidades semânticas exige o uso de testes. Se você não tiver esquemas, ou se os tiver, porém decide não comparar as mudanças nos esquemas para verificar se há compatibilidade, a responsabilidade de identificar as incompatibilidades estruturais antes que elas alcancem o ambiente de produção também será dos testes. Podemos dizer que a situação é, de certa forma, análoga à tipagem estática em relação à tipagem dinâmica nas linguagens de programação. Em linguagens com tipagem estática, os tipos são determinados em tempo de compilação – se seu código fizer algo que não seja permitido com uma instância de um tipo (por exemplo, chamar um método que não existe), o compilador poderá identificar esse erro. Isso permite que você concentre seus esforços de teste em outros tipos de problemas. Em uma linguagem com tipagem dinâmica, porém, parte de seus testes terá de identificar erros que um compilador identificaria em linguagem com tipagem estática.

Não tenho nenhum problema com linguagens com tipagem estática ou com tipagem dinâmica, e sei que sou muito produtivo (relativamente falando) com ambas. Certamente as linguagens com tipagem dinâmica oferecem algumas vantagens significativas que, para muitas pessoas, justificam renunciar à segurança oferecida pela compilação. Pessoalmente, porém, se levarmos a discussão de volta às interações entre microsserviços, não acho que haja uma relação similar de custo-benefício quando se trata de uma comunicação com esquema *versus* uma comunicação sem esquema. Falando de modo simples, acho que ter um esquema explícito mais do que compensa qualquer vantagem perceptível que uma comunicação sem esquema possa oferecer.

Na verdade, a questão realmente não é se você tem ou não um esquema, mas se esse esquema é ou não *explícito*. Ainda que você consuma dados de uma API sem esquema, você terá algumas expectativas quanto aos dados que devem estar presentes e como devem estar estruturados. O código que lidará com esses dados será escrito com um conjunto de pressupostos em mente acerca de como esses dados estarão estruturados. Em um caso como esse, eu argumentaria que você tem um esquema, mas ele está totalmente

implícito, ao invés de estar explícito.¹ Boa parte do meu desejo de ter um esquema explícito é determinado pelo fato de eu achar importante ser o mais explícito possível quanto àquilo que um microsserviço expõe (ou não).

O principal argumento a favor de endpoints sem esquema parece ser o fato de que os esquemas exigem mais trabalho e não agregam valor suficiente. Isso, em minha humilde opinião, em parte se deve a uma falta de imaginação e, em parte, a uma falha em ter boas ferramentas que ajudem os esquemas a agregarem mais valor quando se trata de utilizá-los para identificar incompatibilidades estruturais.

Em última análise, boa parte daquilo que os esquemas oferecem é uma representação explícita de parte do contrato da estrutura entre um cliente e um servidor. Eles ajudam a deixar tudo explícito e podem ajudar bastante na comunicação entre as equipes, além de funcionarem como uma rede de proteção. Nas situações em que o custo das mudanças é menor – por exemplo, quando tanto o cliente como o servidor estão sob responsabilidade da mesma equipe – fico mais à vontade com o fato de não haver esquemas.

Lidando com mudanças entre microsserviços

Provavelmente a pergunta mais comum que me fazem sobre os microsserviços, depois de “Qual é o tamanho que devem ter?”, é esta: “Como você lida com as versões?”. Quando essa pergunta é feita, raramente se trata do tipo de esquema de numeração que você deve empregar, porém mais sobre como lidar com mudanças de contrato entre os microsserviços.

O modo de lidar com as mudanças realmente pode ser separado em dois tópicos. Em breve, veremos o que acontece se você precisar fazer uma mudança que cause incompatibilidade. Antes, porém, vamos ver o que podemos fazer para evitar que uma mudança que causa incompatibilidade seja feita.

Evitando mudanças que causam incompatibilidade

Se quiser evitar mudanças que causem incompatibilidade, há algumas ideias básicas que valem a pena explorar, muitas das quais já foram mencionadas no início do capítulo. Se puder colocar essas ideias em prática, você verá que é muito mais fácil permitir que os microsserviços sejam alterados de modo independente uns dos outros.

Mudanças com expansão

Acrescente novos itens na interface de um microsserviço; não remova os itens antigos.

Leitor tolerante

Ao consumir a interface de um microsserviço, seja flexível quanto àquilo que espera.

Tecnologia correta

Escolha uma tecnologia que facilite fazer alterações que sejam compatíveis com versões anteriores na interface.

Interface explícita

Seja explícito quanto àquilo que um microsserviço expõe. Isso facilita a vida do cliente e permite que os mantenedores do microsserviço saibam com mais facilidade o que é que pode ser alterado livremente.

Identifique rapidamente as mudanças que causam incompatibilidades acidentais

Tenha mecanismos à disposição para identificar mudanças na interface que possam causar falhas nos consumidores no ambiente de produção, antes que essas mudanças sejam implantadas.

Essas ideias se reforçam entre si, e muitas se baseiam no conceito básico de ocultação de informações discutido com frequência. Vamos analisar cada ideia individualmente.

Mudanças com expansão

Provavelmente o ponto de partida mais fácil é apenas acrescentar itens novos no contrato de um microsserviço, sem remover outros itens.

Considere o exemplo de acrescentar um novo campo em um payload – supondo que o cliente seja, de certo modo, tolerante a mudanças desse tipo, isso não deverá causar um impacto significativo. Por exemplo, acrescentar um novo campo `dateOfBirth` no registro de um cliente não deverá ser um problema.

Leitor tolerante

O modo como o consumidor de um microserviço é implementado pode contribuir muito para facilitar fazer alterações que sejam compatíveis com versões anteriores. Especificamente, queremos evitar que o código do cliente esteja excessivamente vinculado à interface de um microserviço. Vamos considerar um microserviço `Email` cuja tarefa seja enviar emails aos nossos clientes de tempos em tempos. Ele é solicitado a enviar um email de “pedido enviado” para um cliente cujo ID é 1234; o microserviço acessa o cliente com esse ID e recebe algo como a resposta exibida no Exemplo 5.3.

Exemplo 5.3 – Exemplo de resposta do serviço Cliente (Customer)

```
<customer>
  <firstname>Sam</firstname>
  <lastname>Newman</lastname>
  <email>sam@magpiebrain.com</email>
  <telephoneNumber>555-1234-5678</telephoneNumber>
</customer>
```

Para enviar o email, o microserviço `Email` precisa somente dos campos `firstname`, `lastname` e `email`. Não precisamos conhecer o campo `telephoneNumber`. Queremos simplesmente extrair os campos que nos interessam e ignorar os demais. Algumas tecnologias de vinculação (binding), particularmente aquelas usadas por linguagens fortemente tipadas, podem tentar fazer um bind de *todos* os campos, independentemente de o consumidor os querer ou não. O que acontecerá se percebermos que ninguém está usando o campo `telephoneNumber` e decidir que iremos removê-lo? Isso poderia causar falhas desnecessariamente nos consumidores.

De modo semelhante, e se quiséssemos reestruturar nosso objeto `customer` (Cliente) para aceitar outros detalhes, talvez acrescentando mais estrutura, como no Exemplo 5.4? Os dados que nosso serviço `Email` quer continuam

presentes, com o mesmo nome, mas se nosso código fizer suposições muito explícitas acerca do local em que os campos `firstname` e `lastname` estarão armazenados, ele poderia falhar novamente. Nesse exemplo, poderíamos usar o XPath para extrair os campos que nos interessam, permitindo que sejamos ambivalentes em relação ao local em que estão os campos, desde que possamos encontrá-los. Esse padrão – de implementar um leitor capaz de ignorar mudanças nas quais não estamos interessados – é o que Martin Fowler chama de leitor tolerante (<https://oreil.ly/G65yf>).

Exemplo 5.4 – Um recurso `Customer` reestruturado: todos os dados continuam presentes, mas nossos consumidores poderão encontrá-los?

```
<customer>
  <naming>
    <firstname>Sam</firstname>
    <lastname>Newman</lastname>
    <nickname>Magpiebrain</nickname>
    <fullname>Sam "Magpiebrain" Newman</fullname>
  </naming>
  <email>sam@magpiebrain.com</email>
</customer>
```

O exemplo de um cliente que tenta ser o mais flexível possível para consumir um serviço é uma demonstração da lei de Postel (<https://oreil.ly/GVqeI>) – também conhecida como *princípio da robustez* –, que afirma o seguinte: “Seja conservador no que faz, mas liberal no que aceita dos outros”. O contexto original dessa sábia citação era a interação de dispositivos por meio de redes, em que devemos esperar todo tipo de ocorrências inusitadas. No contexto das interações entre microsserviços, ela nos leva a tentar estruturar o código de nossos clientes de modo que sejam tolerantes a mudanças nos payloads.

Tecnologia correta

Conforme já exploramos, algumas tecnologias podem ser mais frágeis quando se trata de permitir que façamos alterações nas interfaces – já ressaltai minhas frustrações pessoais com o Java RMI. Por outro lado, algumas implementações de integração fazem de tudo para facilitar ao máximo que mudanças possam ser feitas sem causar falhas nos clientes.

No lado mais simples do espectro, os buffers de protocolo – o formato de serialização usado como parte do gRPC – têm o conceito de número de campo. Cada entrada em um buffer de protocolo precisa definir um número de campo, que o código do cliente espera encontrar. Se novos campos forem adicionados, o cliente não se importará. O Avro permite que o esquema seja enviado junto com o payload, permitindo que os clientes possam interpretar um payload de modo muito parecido com um tipo dinâmico.

No lado mais extremo do espectro, o conceito de HATEOAS do REST diz respeito principalmente a permitir que os clientes façam uso de endpoints REST mesmo que eles mudem, utilizando links de hipermídia, assunto que já discutimos antes. Isso, é claro, exige que você adote uma mentalidade totalmente voltada para o HATEOAS.

Interface explícita

Sou *grande* fã de um microserviço expor um esquema explícito que represente o que seus endpoints fazem. Ter um esquema explícito deixa claro o que os consumidores podem esperar, mas também deixa muito mais claro aos desenvolvedores que trabalham com um microserviço quais são as partes que devem permanecer inalteradas a fim de garantir que não provoquem falhas nos consumidores. Falando de outra maneira, um esquema explícito contribui bastante para deixar as fronteiras da ocultação de informações mais explícitas – o que está exposto no esquema, por definição, não está oculto.

Ter um esquema explícito para RPC é uma prática que existe há muito tempo e, na verdade, é um requisito para muitas implementações de RPC. O REST, por outro lado, em geral considera o conceito de esquema como opcional, a ponto de eu achar que encontrar esquemas explícitos para endpoints REST é cada vez mais raro. Isso está mudando, com opções como a especificação OpenAPI mencionada antes ganhando impulso, e com a especificação JSON Schema também ganhando maturidade.

Protocolos assíncronos de mensagens têm tido mais dificuldades nessa área. Podemos ter facilmente um esquema para o payload de uma

mensagem e, com efeito, essa é uma área em que o Avro é utilizado com frequência. No entanto, para ter uma interface explícita, é preciso ir além. Se considerarmos um microsserviço que gere eventos, quais são os eventos que ele expõe? Algumas tentativas de tornar os esquemas explícitos para endpoints baseados em eventos estão atualmente em andamento. Uma delas é o AsyncAPI (<https://www.asyncapi.com>), que conquistou alguns usuários renomados, mas a opção que está ganhando mais impulso parece ser o CloudEvents (<https://cloudevents.io>), uma especificação que tem o suporte da CNCF (Cloud Native Computing Foundation, ou Fundação para Computação Nativa de Nuvem). O produto Event Grid do Azure aceita os formatos do CloudEvents, um sinal de que diferentes fornecedores estão aceitando esse formato, o que deve ajudar com a interoperabilidade. Essa é uma área ainda relativamente nova, portanto será interessante ver como a situação vai se desenrolar nos próximos anos.

Versões semânticas

Não seria ótimo se, como cliente, você pudesse simplesmente olhar para o número da versão de um serviço e saber se poderá se integrar com ele? A especificação de versões semânticas (semantic versioning, <https://semver.org>) permite fazer exatamente isso. Com as versões semânticas, cada número de versão tem o formato MAJOR.MINOR.PATCH. Quando o número MAJOR (principal) é incrementado, significa que mudanças incompatíveis com versões anteriores foram feitas. Quando MINOR (secundário) é incrementado, novas funcionalidades foram acrescentadas, as quais devem ser compatíveis com versões anteriores. Por fim, uma mudança em PATCH (correção) informa que correções de bug foram feitas em funcionalidades existentes.

Para ver quão útil podem ser as versões semânticas, vamos analisar um caso de uso simples. Nossa aplicação de helpdesk foi criada para trabalhar com a versão 1.2.0 do serviço cliente. Se uma nova funcionalidade for acrescentada, fazendo com que o serviço cliente passe para 1.3.0, nossa aplicação de helpdesk não deverá ter nenhuma mudança de comportamento, e a expectativa é que ela não precisará fazer nenhuma mudança. Contudo, não poderíamos garantir que seria possível trabalhar com a versão 1.1.0 do serviço cliente, pois podemos

depende de uma funcionalidade acrescentada na versão 1.2.0. Também devemos esperar que teríamos de fazer alterações em nossa aplicação se uma nova versão 2.0.0 do serviço cliente fosse lançada.

Você pode decidir adotar versões semânticas para o serviço, ou até mesmo para um endpoint individual de um serviço, caso esteja coexistindo com eles, conforme será detalhado na próxima seção.

Esse esquema de numeração de versões permite incluir muitas informações e expectativas em apenas três campos. A especificação completa descreve, de modo muito simples, as expectativas que os clientes podem ter quanto às mudanças nesses números, e simplifica o processo de comunicação para informar se as mudanças causarão algum impacto nos consumidores. Infelizmente não tenho visto essa abordagem ser suficientemente utilizada em sistemas distribuídos a ponto de entender sua eficácia nesse contexto – algo que realmente não mudou desde a primeira edição deste livro.

Identifique rapidamente as mudanças que causam incompatibilidades acidentais

É essencial que identifiquemos as alterações que causem falhas nos consumidores o mais rápido possível, pois, ainda que escolhêssemos a melhor tecnologia possível, uma mudança inocente em um microsserviço poderia causar falhas nos consumidores. Conforme já mencionei, utilizar esquemas pode nos ajudar a identificar mudanças estruturais, supondo que utilizemos algum tipo de ferramenta que ajude a comparar as versões dos esquemas. Há diversas ferramentas por aí que fazem isso para diferentes tipos de esquemas. Temos o Protolock (<https://oreil.ly/wwxBx>) para buffers de protocolo, o json-schema-diff-validator (<https://oreil.ly/COSIr>) para JSON Schema e o openapi-diff para a especificação OpenAPI.² Outras ferramentas parecem estar surgindo o tempo todo nessa área. No entanto, o que buscamos é algo que não só informe as diferenças entre dois esquemas, mas que seja bem-sucedida ou falhe com base na compatibilidade; isso permitiria que um build de CI falhe caso esquemas incompatíveis sejam identificados, garantindo que seu microsserviço não vá ser implantado.

O open source Confluent Schema Registry (<https://oreil.ly/qcggd>) aceita

JSON Schema, Avro e buffers de protocolo, além de ser capaz de comparar versões cujo upload acabou de ser feito para ver se são compatíveis com versões anteriores. Embora tenha sido criado para ajudar como parte de um ecossistema em que o Kafka esteja executando, não há nada que impeça você de usá-lo para armazenar e validar esquemas usados em uma comunicação que não seja baseada no Kafka.

Ferramentas de comparação de esquemas podem nos ajudar a identificar incompatibilidades estruturais, mas e quanto às incompatibilidades semânticas? E se, antes de tudo, você não estiver fazendo uso de esquemas? Então devemos voltar nossa atenção para os testes. Esse é um assunto que exploraremos com mais detalhes na seção “Testes de contrato e CDCs”, na página [348](#), mas eu gostaria de destacar os testes de CDC (Consumer-Driven Contracts, ou Contratos Orientados a Consumidores), que explicitamente ajudam nessa área – o Pact é um excelente exemplo de uma ferramenta voltada especificamente para esse problema. Lembre-se apenas de que, se você não tem esquemas, a expectativa é que seus testes tenham mais trabalho para identificar mudanças que causem incompatibilidades.

Se você oferece suporte para várias bibliotecas diferentes de cliente, executar testes utilizando cada biblioteca disponibilizada, em conjunto com o serviço mais recente, é outra técnica que pode ajudar. Assim que perceber que causará falhas em um consumidor, você terá a opção de tentar evitar totalmente a incompatibilidade, ou poderá aceitá-la e dar início às discussões apropriadas com as pessoas responsáveis pelos serviços consumidores.

Gerenciando mudanças que causam incompatibilidade

Então, você tem feito o máximo para garantir que as alterações que faz na interface de um microsserviço sejam compatíveis com versões anteriores, mas percebeu que terá simplesmente de fazer uma mudança que causará uma incompatibilidade. O que fazer em uma situação como essa? Há três opções principais:

Implantação sincronizada

Exija que o microsserviço que expõe a interface e todos os consumidores

dessa interface sejam alterados ao mesmo tempo.

Permitir a coexistência de versões incompatíveis de microsserviços

Execute as versões antiga e nova do microsserviço, lado a lado.

Emular a interface antiga

Faça o seu microsserviço expor a nova interface, mas também emule a interface antiga.

Implantação sincronizada

É claro que uma implantação sincronizada vai totalmente contra a ideia de implantações independentes. Se quisermos ser capazes de implantar uma nova versão de nosso microsserviço com uma alteração na interface que cause incompatibilidade, porém continuar fazendo isso de modo independente, teremos de dar tempo aos nossos consumidores para que façam um upgrade para a nova interface. Isso nos leva às duas próximas opções que poderíamos considerar.

Permitir a coexistência de versões incompatíveis de microsserviços

Outra solução com base em versões, citada com frequência, consiste em ter diferentes versões do serviço ativas ao mesmo tempo, com os consumidores mais antigos encaminhando seu tráfego para a versão mais antiga, enquanto os consumidores mais recentes veem a nova versão, conforme mostra a Figura 5.3. Essa é a abordagem empregada esporadicamente pela Netflix, em situações nas quais o custo de alterar consumidores mais antigos seja alto demais, sobretudo nos raros casos em que dispositivos legados continuam dependentes das versões mais antigas da API. Pessoalmente, não sou fã dessa ideia, e entendo por que a Netflix a utiliza raramente. Em primeiro lugar, se fosse necessário fazer uma correção de um bug interno ao meu serviço, eu teria agora de fazer a correção e implantar dois conjuntos diferentes do serviço. Isso provavelmente implicaria a necessidade de criar um branch na base de código do meu serviço, o que é sempre problemático. Em segundo lugar, significa que eu precisaria implementar uma inteligência para lidar com o direcionamento dos consumidores para o microsserviço correto. Esse

comportamento acabaria ficando inevitavelmente em algum middleware, ou em um punhado de scripts `nginx`, dificultando compreender o comportamento do sistema. Por fim, considere qualquer estado persistente que nosso serviço possa gerenciar. Os clientes da loja, criados por qualquer que seja a versão do serviço, devem ser armazenados e ser visíveis a todos os serviços, não importa a versão utilizada para criar os dados. Essa pode ser uma fonte adicional de complexidade.

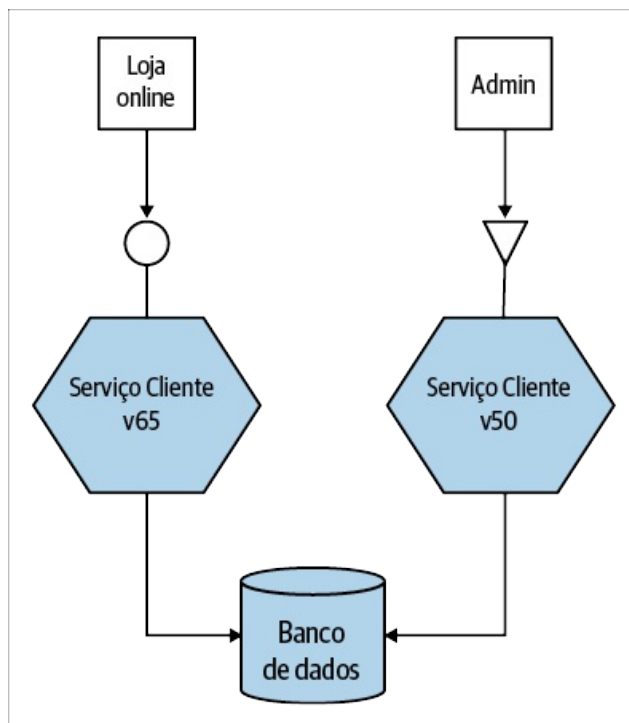


Figura 5.3 – Executando várias versões do mesmo serviço para dar suporte a endpoints antigos.

A coexistência de versões concorrentes de um serviço por um curto período pode fazer muito sentido, sobretudo quando fazemos algo como um lançamento canário (canary release) – esse padrão será discutido com mais detalhes na seção “Em direção a uma entrega progressiva”, na página [323](#). Nessas situações, poderíamos ter uma coexistência de versões somente por alguns minutos, ou talvez algumas horas, e em geral, teríamos apenas duas versões diferentes do serviço presentes ao mesmo tempo. Quanto mais tempo demorar para que haja um upgrade dos consumidores para a versão mais recente e esses consumidores sejam lançados, mais você deverá recorrer à coexistência de diferentes endpoints no mesmo microserviço,

em vez de permitir a coexistência de versões totalmente distintas. Continuo duvidando de que esse trabalho compense em um projeto mais comum.

Emular a interface antiga

Se você fez tudo que era possível para evitar a introdução de uma alteração na interface que causasse uma incompatibilidade, sua próxima tarefa será limitar o impacto. Queremos evitar forçar um upgrade sincronizado dos consumidores conosco, pois devemos sempre preservar a capacidade de lançar microsserviços independentemente uns dos outros. Uma abordagem que tenho usado com sucesso para lidar com essa situação é a coexistência das interfaces antiga e nova no mesmo serviço em execução. Assim, se quisermos lançar uma alteração que cause incompatibilidade, implantamos uma nova versão do serviço que exponha tanto a versão antiga quanto a versão nova do endpoint.

Isso nos permite lançar o novo microsserviço o mais rápido possível, com a nova interface, ao mesmo tempo que concede tempo aos consumidores para que façam suas alterações. Assim que todos os consumidores deixarem de utilizar o endpoint antigo, será possível removê-lo, junto com o código associado a ele, conforme vemos na Figura 5.4.

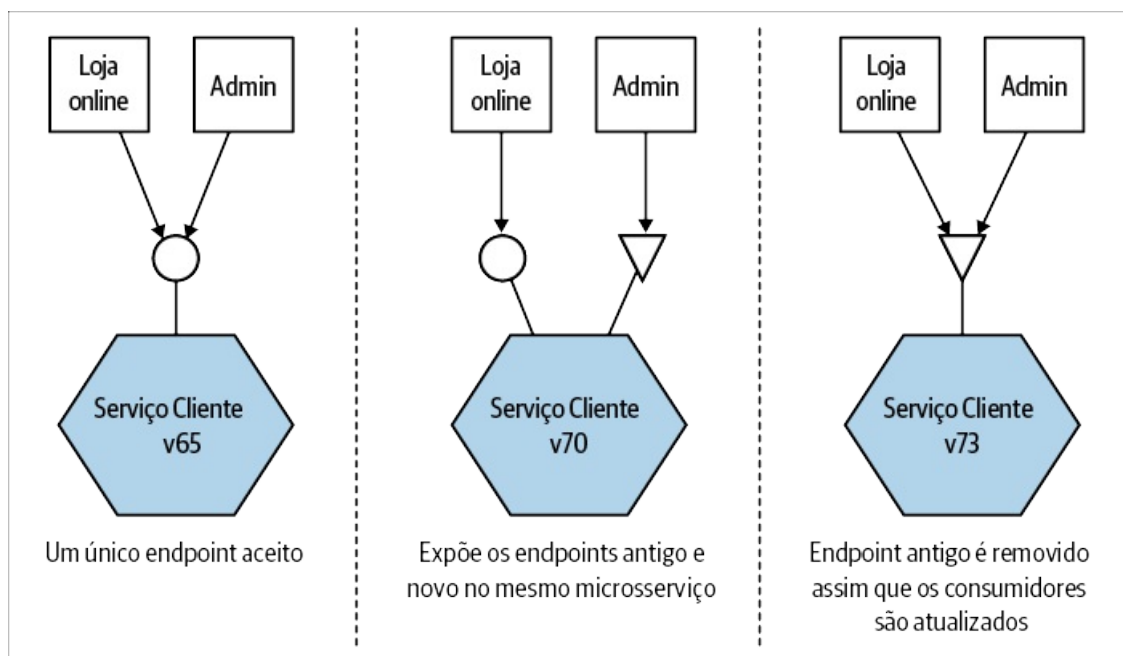


Figura 5.4 – Um microsserviço emulando o endpoint antigo e expondo o novo

endpoint incompatível com versões anteriores.

Na última vez que utilizei essa abordagem, acabamos entrando em uma situação um tanto quanto confusa com o número de consumidores que tínhamos e o número de alterações incompatíveis que havíamos feito. Na verdade, acabamos ficando com três versões diferentes do endpoint coexistindo. Isso não é algo que eu recomendaria! Fazer a manutenção de todos os códigos e os testes associados necessários para garantir que todos funcionassem foi, sem dúvida, um fardo extra. Para deixar a situação mais administrável, transformamos internamente todas as requisições para o endpoint V1 em uma requisição V2, e então as requisições V2 para o endpoint V3. Isso significava que podíamos delinear claramente o código que seria removido quando o(s) antigo(s) endpoint(s) fosse(m) desativado(s).

Com efeito, esse é um exemplo do padrão de expansão e contração, que nos permite introduzir alterações que causem incompatibilidade. *Expandimos* as funcionalidades oferecidas, aceitando tanto a maneira antiga como a nova de fazer algo. Assim que os consumidores antigos passarem a fazer a tarefa do modo novo, *contraímos* nossa API, removendo a funcionalidade antiga.

Se você pretende ter endpoints coexistindo, quem faz as chamadas deverá ter uma forma de encaminhar corretamente as suas requisições. Para sistemas que fazem uso de HTTP, já vi isso ser feito com números de versões tanto nos cabeçalhos das requisições como no próprio URI – por exemplo, */v1/customer/* ou */v2/customer/*. Não tenho certeza da abordagem que faz mais sentido. Por um lado, gosto do fato de os URIs serem opacos para não incentivar os clientes a deixarem os templates de URI fixos no código, mas, por outro lado, essa abordagem deixa tudo muito claro e pode simplificar o encaminhamento das requisições.

No caso do RPC, a situação pode se tornar um pouco mais complicada. Já lidei com esse problema usando buffers de protocolo, colocando meus métodos em namespaces distintos – por exemplo, `v1.createCustomer` e `v2.createCustomer` – mas, se você estiver tentando oferecer suporte para diferentes versões dos mesmos tipos enviados pela rede, essa abordagem

pode se tornar realmente problemática.

Qual é a minha abordagem preferida?

Nas situações em que a mesma equipe administra tanto o microserviço como todos os consumidores, não me preocupo tanto com um lançamento sincronizado em situações limitadas. Supondo que seja realmente uma situação única, fazer isso quando o impacto estiver limitado a uma única equipe pode ser justificável. Contudo, sou muito cauteloso quanto a esse assunto, pois há o perigo de que uma atividade que deveria ocorrer apenas uma vez se torne algo habitual, e lá se vai a capacidade de fazer implantações independentes. Utilize implantações sincronizadas com muita frequência, e logo você acabará com um sistema monolítico distribuído.

A coexistência de diferentes versões do mesmo microserviço pode ser problemática, conforme já discutimos. Eu consideraria fazer isso somente em situações em que planejamos executar as versões do microserviço lado a lado somente por pouco tempo. A verdade é que, quando precisamos dar tempo aos consumidores para que façam um upgrade, poderíamos esperar semanas ou mais. Em outras situações em que possa haver coexistência de versões diferentes do microserviço, talvez como parte de uma implantação azul-verde (blue-green deployment) ou de um lançamento canário (canary release), as durações envolvidas são muito menores, compensando as desvantagens dessa abordagem.

Minha preferência em geral é utilizar uma emulação de endpoints antigos sempre que for possível. Os desafios de implementar a emulação são, em minha opinião, muito mais simples de lidar, em comparação com aqueles resultantes da coexistência de diferentes versões do microserviço.

Contrato social

A abordagem que você escolher se deverá, em grande parte, às expectativas que os consumidores tiverem acerca de como essas alterações serão feitas. Manter a interface antiga presente pode ter um custo, e o ideal é que ela seja desativada, e o código e a infraestrutura associados sejam removidos o

mais rápido possível. Por outro lado, você quer dar aos consumidores o máximo de tempo possível para que façam uma alteração. E lembre-se de que, em muitos casos, as alterações incompatíveis que você está fazendo muitas vezes são mudanças que foram solicitados pelos consumidores e/ou que acabarão por beneficiá-los. É necessário haver um equilíbrio, é claro, entre as necessidades dos mantenedores do microserviço e as necessidades dos consumidores, e isso precisa ser discutido.

Tenho percebido que, em muitas situações, o modo de lidar com essas alterações jamais é discutido, resultando em todo tipo de problemas. Como no caso dos esquemas, ser explícito em certa medida quanto ao modo como alterações que causem incompatibilidade serão feitas pode simplificar bastante a questão.

Não é preciso necessariamente ter pilhas de papéis nem fazer reuniões enormes para chegar a um consenso sobre como lidar com as alterações. Contudo, supondo que você não vá optar pelo caminho dos lançamentos sincronizados, eu sugeriria que tanto o responsável pelo microserviço como o responsável pelo consumidor deixem bem claro os seguintes itens:

- Como será levantada a questão sobre a necessidade de mudança na interface?
- Como as equipes responsáveis pelos consumidores e pelo microserviço vão colaborar para chegar a um acordo sobre como será a mudança?
- Quem se espera que fará o trabalho de upgrade dos consumidores?
- Quando houver um consenso sobre a mudança, quanto tempo os consumidores terão para passar para a nova interface antes que a interface antiga seja removida?

Lembre-se de que um dos segredos de uma arquitetura de microserviços eficaz é adotar uma abordagem do consumidor em primeiro lugar. Seus microserviços existem para serem chamados por outros consumidores. As necessidades dos consumidores são o que há de mais importante, e se você está fazendo alterações em um microserviço que causarão problemas aos consumidores upstream, isso deve ser levado em consideração.

Em algumas situações, é claro, talvez não seja possível modificar os

consumidores. Já ouvi da Netflix que eles tiveram problemas (ao menos, historicamente) com set-top boxes antigos que utilizavam versões mais antigas das APIs da Netflix. Esses set-top boxes não podem ser facilmente atualizados, portanto os endpoints antigos têm de permanecer disponíveis a menos que, ou até que, o número de set-top boxes mais antigos se reduza a um nível que possam ter o suporte desativado. Decisões para impedir que consumidores antigos sejam capazes de acessar seus endpoints às vezes acabam sendo de caráter financeiro – quanto custará dar suporte à interface antiga, em contraposição a quanto você lucrará com esses consumidores.

Monitorando o uso

Mesmo que se chegue a um acordo sobre um prazo para que os consumidores parem de utilizar a interface antiga, você teria como saber se eles realmente pararam de usá-la? Garantir que você tenha um logging ativo para cada endpoint exposto pelo seu microserviço pode ajudar, assim como garantir que haja algum tipo de identificador de cliente para que você possa conversar com a equipe em questão caso precise trabalhar com eles para convencê-los a migrar da interface antiga para a nova. Poderia ser algo simples como pedir aos consumidores que coloquem seu identificador no cabeçalho `user-agent` ao fazerem requisições HTTP, ou você poderia exigir que todas as chamadas fossem feitas por alguma espécie de gateway de API no qual os clientes precisassem de chaves para se identificar.

Medidas extremas

Então, supondo que você saiba que um consumidor continua usando uma interface antiga que você quer remover, e eles estejam procrastinando para passar para a nova versão, o que você poderia fazer a esse respeito? Bem, a primeira atitude seria conversar com eles. Talvez você possa oferecer ajuda para que as alterações sejam feitas. Se tudo mais falhar, e eles ainda não fizerem o upgrade, mesmo depois de terem concordado em fazê-lo, há algumas técnicas radicais que já vi serem empregadas.

Em certa empresa grande de tecnologia, discutimos como ela lidou com

esse problema. Internamente, a empresa concedia um período muito generoso de um ano até que as interfaces antigas fossem desativadas. Perguntei como eles sabiam se os consumidores continuavam utilizando as interfaces antigas, e a empresa respondeu que eles realmente não se importavam em monitorar essa informação; depois de um ano, a interface antiga era simplesmente desativada. Era de conhecimento interno que, se isso causasse uma falha em um consumidor, a culpa seria da equipe responsável pelo microsserviço consumidor – eles tiveram um ano para fazer a alteração e não fizeram. É claro que essa abordagem não funcionaria para muitos (eu disse que era radical!). Ela também resulta em muita ineficiência. Por não saber se a interface antiga estava sendo usada, a empresa negava a si mesma a oportunidade de removê-la até que um ano tivesse se passado. Pessoalmente, ainda que eu sugerisse simplesmente desativar o endpoint após determinado período, certamente eu ainda gostaria de monitorar quem seria impactado.

Outra medida extrema que já vi foi no contexto de bibliotecas que ficariam obsoletas, mas, teoricamente, poderia ser aplicada também a endpoints de microsserviços. O exemplo mencionado era de uma antiga biblioteca que as pessoas estavam tentando desativar dentro da empresa, em favor de uma biblioteca melhor e mais nova. Apesar do muito trabalho para alterar o código a fim de utilizar a nova biblioteca, algumas equipes ainda estavam procrastinando. A solução foi inserir um sleep na biblioteca antiga, de modo que ela respondesse mais lentamente às chamadas (com um logging para mostrar o que estava acontecendo). Com o tempo, a equipe à frente do processo de obsolescência simplesmente continuava a aumentar a duração do sleep até que, algum tempo depois, as outras equipes começaram a compreender a mensagem. Obviamente você precisa estar totalmente certo de já ter esgotado as demais alternativas razoáveis para fazer com que os consumidores fizessem um upgrade para considerar uma medida como essa!

DRY e os perigos da reutilização de código em um mundo de microsserviços

Um dos acrônimos que nós desenvolvedores ouvimos muito é o DRY: Don't Repeat Yourself (Não se Repita). Embora sua definição às vezes seja simplificada como uma tentativa de evitar duplicação de código, de modo mais exato, DRY significa que queremos evitar a duplicação de *comportamento e conhecimento* em nosso sistema. De modo geral, esse é um conselho muito sensato. Ter muitas linhas de código que fazem o mesmo deixa sua base de código maior do que o necessário e, desse modo, será mais difícil de entendê-la. Quando quiser alterar algum comportamento, e esse comportamento estiver duplicado em várias partes de seu sistema, será fácil se esquecer de algum lugar em que a alteração terá de ser feita, o que pode resultar em bugs. Assim, utilizar o DRY como um mantra em geral faz sentido.

O princípio DRY é o que nos leva a criar um código que possa ser reutilizado. Extraímos o código duplicado em abstrações que poderão ser então chamadas de vários lugares. Talvez possamos ir longe, a ponto de criar uma biblioteca compartilhada que passe a ser utilizada em qualquer lugar! O fato, porém, é que compartilhar código em um ambiente de microserviços é um pouco mais complicado que isso. Como sempre, temos mais de uma opção a considerar.

Compartilhando código por meio de bibliotecas

Um problema que queremos evitar a todo custo é um excessivo acoplamento entre um microserviço e os consumidores, de modo que qualquer pequena alteração no microserviço cause alterações desnecessárias no consumidor. Às vezes, porém, o uso de código compartilhado pode causar exatamente esse tipo de acoplamento. Por exemplo, em certo cliente, tínhamos uma biblioteca de objetos comuns do domínio, que representavam as principais entidades em uso no nosso sistema. Essa biblioteca era utilizada por todos os serviços que tínhamos. No entanto, quando uma mudança era feita em uma delas, todos os serviços precisavam ser atualizados. Nosso sistema se comunicava por meio de filas de mensagens, as quais também precisavam ser esvaziadas de seus conteúdos agora *inválidos* – e aí de você se esquecesse de fazer isso.

Se seu uso de código compartilhado ultrapassar as fronteiras de seu serviço, você introduzirá uma possível forma de acoplamento. O uso de um código comum, por exemplo, de bibliotecas de logging, não é um problema, pois elas representam conceitos internos que são invisíveis ao mundo externo. O site realestate.com.au utiliza um template de serviço personalizado para ajudar na criação de serviços. Em vez de fazer com que esse código seja compartilhado, a empresa o copia para cada novo serviço a fim de garantir que não vai introduzir um acoplamento.

A questão realmente importante no que diz respeito ao compartilhamento de código por meio de bibliotecas é que não é possível atualizar todos os usos da biblioteca de uma só vez. Embora vários microsserviços possam utilizar a mesma biblioteca, em geral eles fazem isso incluindo essa biblioteca na implantação do microsserviço. Para fazer um upgrade da versão da biblioteca em uso, você teria, portanto, de reimplantar o microsserviço. Se quiser atualizar a mesma biblioteca em todos os lugares, exatamente no mesmo instante, o resultado poderia ser uma implantação em larga escala de vários microsserviços diferentes, todos ao mesmo tempo, com todas as dores de cabeça associadas a esse cenário.

Desse modo, se estiver usando bibliotecas para reutilização de código, que cruzem fronteiras entre microsserviços, você precisa aceitar que várias versões diferentes da mesma biblioteca poderão existir simultaneamente. É claro que você pode tentar atualizar todas as bibliotecas para a versão mais recente com o tempo, mas, se esse fato não for um problema para você, então reutilize código por meio de bibliotecas. Se realmente for necessário atualizar esse código para todos os usuários exatamente no mesmo instante, procure então reutilizar códigos por meio de um microsserviço dedicado.

Contudo, há um caso de uso específico associado à reutilização de código por meio de bibliotecas, o qual vale a pena explorar com mais detalhes.

Bibliotecas de cliente

Já falei com mais de uma equipe que insistia que criar bibliotecas de cliente para seus serviços era, antes de tudo, uma parte essencial da criação de

serviços. A argumentação era que isso facilitaria a utilização do seu serviço e evitaria a duplicação de código necessária para consumir o serviço em questão.

O problema, é claro, é que, se as mesmas pessoas criarem tanto a API do servidor como a API do cliente, há o perigo de que uma lógica que deveria estar no servidor comece a extravasar para o cliente. Sei disso porque eu mesmo já o fiz. Quanto mais lógica extravasar para a biblioteca de cliente, maior será a quebra de coesão, e você se verá tendo de alterar vários clientes para implantar correções em seu servidor. Você também limitará as opções de tecnologia, particularmente se obrigar que a biblioteca de cliente tenha de ser usada.

Um modelo que eu gosto para bibliotecas de cliente é o modelo da Amazon Web Services (AWS). As chamadas para web services SOAP ou REST subjacentes podem ser feitas diretamente, porém todos acabam utilizando um dos diversos SDKs (Software Development Kits, ou Kits de Desenvolvimento de Software) existentes, que oferecem abstrações para a API subjacente. No entanto, esses SDKs foram escritos pela comunidade mais ampla, ou então por pessoas dentro da AWS que não são as mesmas que trabalham com a API propriamente dita. Esse grau de separação parece funcionar e evita algumas das armadilhas das bibliotecas de cliente. Parte do motivo de essa solução funcionar tão bem é que os clientes podem decidir quando farão o upgrade. Se você vai optar pelo uso de bibliotecas de cliente, garanta que esse será o seu caso.

A Netflix em particular coloca uma ênfase especial na biblioteca de cliente, mas minha preocupação é que as pessoas vejam isso exclusivamente da perspectiva de evitar uma duplicação de código. Com efeito, as bibliotecas de cliente utilizadas pela Netflix dizem respeito igualmente – ou até mais – a garantir a confiabilidade e a escalabilidade de seus sistemas. As bibliotecas de cliente da Netflix lidam com descoberta de serviços (service discovery), modos de falha, logging e outros aspectos que não são realmente da natureza do próprio serviço. Sem essas bibliotecas de cliente compartilhadas, seria difícil garantir que cada comunicação entre cliente/servidor se comportasse bem diante da escala massiva com a qual a

Netflix atua. O uso dessas bibliotecas na Netflix certamente facilitou colocar o serviço rapidamente em operação e aumentou sua produtividade, além de garantir que o sistema se comporte bem. No entanto, de acordo com pelo menos uma pessoa na Netflix, com o tempo, isso levou a certo grau de acoplamento entre cliente e servidor, o qual tem sido problemático.

Se a abordagem com bibliotecas de cliente é algo no qual você está pensando, talvez seja importante separar o código do cliente que lida com o protocolo de transporte subjacente, o qual é capaz de lidar com aspectos como descoberta de serviços e falhas, de aspectos relacionados ao serviço de destino. Decida se você vai ou não insistir no uso de bibliotecas de cliente, ou se vai permitir que pessoas que utilizem diferentes conjuntos de tecnologias façam chamadas à API subjacente. Por fim, certifique-se de que os clientes sejam responsáveis por decidir quando farão o upgrade de suas bibliotecas de cliente: devemos garantir que manteremos a capacidade de lançar nossos serviços de modo independente!

Descoberta de serviços

Assim que você tiver mais do que alguns microsserviços implantados, sua atenção inevitavelmente se voltará em saber onde está cada um deles. Talvez você queira saber o que está executando em um dado ambiente para saber o que deve monitorar. Poderia ser uma informação simples como saber onde está o seu microsserviço *contas* para que seus consumidores saibam onde encontrá-lo. Ou, quem sabe, você somente queira que os desenvolvedores em sua empresa saibam com mais facilidade quais são as APIs disponíveis para que eles não reinventem a roda. Falando de modo geral, todos esses casos de uso se enquadram no rótulo de *descoberta de serviços* (service discovery). E, como sempre ocorre com os microsserviços, temos várias opções diferentes à nossa disposição para lidar com isso.

Todas as soluções que veremos lidam com o problema em duas partes. Em primeiro lugar, elas oferecem algum método para que uma instância se registre e diga “Estou aqui!”. Em segundo lugar, oferecem alguma forma de encontrar o serviço assim que a instância estiver registrada. A descoberta

de serviços se torna mais complicada, porém, quando consideramos um ambiente no qual destruímos e implantamos novas instâncias de serviços constantemente. O ideal é que qualquer solução que escolhêssemos pudesse lidar com isso.

Vamos analisar algumas das soluções mais comuns para descoberta de serviços e considerar nossas opções.

Domain Name System (DNS)

É bom começar com algo simples. O DNS (Domain Name System, ou Sistema de Nomes de Domínio) permite associar um nome ao endereço IP de uma ou mais máquinas. Poderíamos decidir, por exemplo, que nosso microserviço *contas* estará sempre em *contas.musiccorp.net*. Teríamos, então, esse ponto de entrada no endereço IP do host que executa esse microserviço, ou talvez fizéssemos com que ele fosse resolvido para um balanceador de carga (load balancer) que distribua a carga entre várias instâncias. Isso significa que teríamos de lidar com a atualização dessas entradas como parte da implantação do nosso serviço.

Ao lidar com instâncias de um serviço em diferentes ambientes, tenho visto que um template de domínio com base em convenções funciona bem. Por exemplo, poderíamos ter um template definido como *<nomedoserviço>-<ambiente>.musiccorp.net*, o que resultaria em entradas como *contas-uat.musiccorp.net* ou *contas-dev.musiccorp.net*.

Um modo mais sofisticado de lidar com diferentes ambientes é ter diferentes servidores de nomes de domínio para esses ambientes. Assim, eu poderia supor que *contas.musiccorp.net* é o local em que sempre encontrarei o microserviço *contas*, mas ele poderia ser resolvido para hosts distintos, dependendo de onde eu fizer a consulta. Se você já tem seus ambientes em diferentes segmentos de rede e se sente à vontade em gerenciar seus próprios servidores e entradas DNS, essa poderia ser uma ótima solução, mas será um bocado de trabalho se você não vai desfrutar outros benefícios dessa configuração.

O DNS tem uma série de vantagens, a principal sendo o fato de ser um padrão muito usado e conhecido, aceito por praticamente todos os

conjuntos de tecnologias. Infelizmente, embora haja diversos serviços para gerenciar o DNS dentro de uma empresa, poucos parecem ter sido projetados para um ambiente no qual lidamos com hosts extremamente descartáveis, fazendo com que a atualização das entradas de DNS seja, de certo modo, complicada. O serviço Route 53 da Amazon faz um bom trabalho nesse caso, mas ainda não vi nenhuma opção hospedada localmente que seja tão boa quanto ele, embora (conforme discutiremos em breve) algumas ferramentas dedicadas de descoberta de serviços como o Consul possam ajudar. Além dos problemas de atualizar as entradas de DNS, a própria especificação de DNS pode nos causar alguns problemas.

As entradas de DNS para nomes de domínio têm um *TTL* (Time To Live, ou Tempo de Vida). É o tempo que um cliente pode considerar que a entrada está atualizada. Se quisermos mudar o host ao qual o nome de domínio se refere, atualizaremos essa entrada, porém teremos de supor que os clientes continuarão contando com o IP antigo *pelo menos* durante o período definido pelo TTL. As entradas de DNS podem ficar em cache em vários lugares (até mesmo a JVM armazenará entradas de DNS em cache, a menos que você lhe diga para não fazer isso), e quanto maior o número de lugares em que elas estiverem armazenadas em cache, mais desatualizada uma entrada poderá estar.

Uma maneira de contornar esse problema é fazer com que a entrada do nome de domínio de seu serviço aponte para um balanceador de carga, o qual, por sua vez, apontará para as instâncias de seu serviço, conforme mostra a Figura 5.5. Ao implantar uma nova instância, você poderá remover a instância antiga da entrada do balanceador de carga e acrescentar a nova instância. Algumas pessoas utilizam roud-robin no DNS, caso em que as próprias entradas de DNS se referem a um grupo de máquinas. Essa técnica é extremamente problemática, pois o cliente não saberá qual é o host subjacente e, desse modo, não poderá parar facilmente de encaminhar o tráfego para um dos hosts caso ele venha a ter problemas.

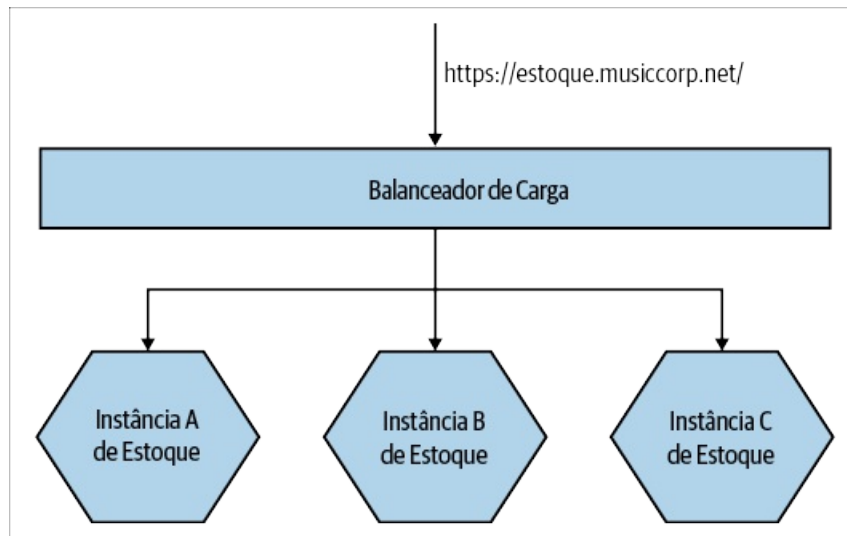


Figura 5.5 – Usando DNS com resolução para um balanceador de carga a fim de evitar entradas de DNS desatualizadas.

Conforme mencionado, o DNS é bem conhecido e amplamente aceito. Entretanto, ele apresenta uma ou duas desvantagens. Eu sugeriria que você investigue se ele seria uma solução apropriada para você antes de escolher algo mais complexo. Para uma situação em que você tenha apenas nós individuais, fazer com que o DNS se refira diretamente aos hosts provavelmente não seria um problema. Contudo, nas situações em que for necessário ter mais de uma instância de um host, faça com que as entradas de DNS sejam resolvidas para balanceadores de carga, capazes de lidar com hosts individuais que sejam ativados e desativados conforme for apropriado.

Registros dinâmicos de serviços

As desvantagens do DNS como uma forma de encontrar nós em um ambiente altamente dinâmico levaram a uma série de sistemas alternativos, a maioria dos quais envolvendo o próprio serviço se inscrevendo junto a algum registro centralizado que, por sua vez, oferece a capacidade de consultar esses serviços posteriormente. Com frequência, esses sistemas fazem mais do que simplesmente oferecer os recursos de registro e descoberta de serviços, o que pode ou não ser bom. Essa é uma área repleta de opções, portanto veremos apenas algumas para que você tenha uma ideia do que existe disponível.

ZooKeeper

O ZooKeeper (<http://zookeeper.apache.org>) foi originalmente desenvolvido como parte do projeto Hadoop. É usado em uma variedade incrível de casos de uso, incluindo gerenciamento de configuração, sincronização de dados entre serviços, eleição de líder, filas de mensagens e (convenientemente para nós) como um serviço de nomes.

Como em muitos tipos semelhantes de sistemas, o ZooKeeper depende da execução de vários nós em um cluster para oferecer diversas garantias. Isso significa que você deve esperar a execução de pelo menos três nós com o Zookeeper. A maior parte da inteligência do ZooKeeper está relacionada a garantir que os dados sejam replicados de forma segura entre esses nós, e que a consistência seja mantida caso haja uma falha em algum dos nós.

Em sua essência, o ZooKeeper disponibiliza um namespace hierárquico para armazenar informações. Os clientes podem inserir novos nós nessa hierarquia, alterá-los ou consultá-los. Além disso, eles podem acrescentar observadores (watches) nos nós para que sejam informados quando esses nós mudarem. Isso significa que poderíamos armazenar informações sobre onde estão nossos serviços nessa estrutura e, como clientes, seríamos informados quando esses dados mudarem. O ZooKeeper é usado com frequência como um repositório de configuração genérico, portanto podemos armazenar aí as configurações específicas de um serviço, permitindo que você execute tarefas como mudar os níveis de log dinamicamente ou desativar funcionalidades de um sistema em execução.

Na verdade, há soluções melhores para um registro dinâmico de serviços, a ponto de dizer que, hoje em dia, eu evitaria o ZooKeeper para esse caso de uso.

Consul

Assim como o ZooKeeper, o Consul (<https://www.consul.io>) oferece suporte tanto para gerenciamento de configurações como para descoberta de serviços. Contudo, ele vai além do ZooKeeper, oferecendo um suporte adicional para esses casos de uso principais. Por exemplo, o Consul expõe uma interface HTTP para descoberta de serviços, e um de seus melhores

recursos é disponibilizar um servidor DNS pronto; de modo mais específico, o Consul disponibiliza registros SRV, que fornecem tanto um IP como uma porta para um dado nome. Isso significa que, se parte de seu sistema já utiliza DNS e aceita registros SRV, você poderá simplesmente instalar o Consul e começar a usá-lo sem nenhuma alteração no sistema existente.

O Consul também inclui outras funcionalidades que você talvez ache convenientes, por exemplo, a capacidade de fazer verificações de sanidade nos nós. Desse modo, o Consul poderia muito bem substituir as funcionalidades oferecidas por outras ferramentas dedicadas de monitoração, embora seja mais provável que você vá utilizá-lo como uma fonte dessas informações, e então enviar esses dados para um sistema de monitoração mais completo.

O Consul utiliza uma interface HTTP RESTful para tudo, desde o registro de um serviço até a consulta de chaves/valores armazenados ou a inserção de verificações de sanidade. Isso faz com que a integração com diferentes conjuntos de tecnologias seja muito simples. Há também um conjunto de ferramentas que funciona muito bem com o Consul, aumentando ainda mais a sua conveniência. Um exemplo é o `consul-template` (<https://oreil.ly/llwVQ>), que oferece uma forma de atualizar arquivos-texto com base em entradas no Consul. À primeira vista, isso não parece ser tão interessante, até considerarmos o fato de que, com o `consul-template`, agora podemos alterar um valor no Consul – talvez a localização de um microsserviço, ou um parâmetro de configuração – e teremos arquivos de configuração atualizados dinamicamente em todo o sistema. Repentinamente, qualquer programa que leia sua configuração de um arquivo-texto poderá ter seus arquivos-texto atualizados dinamicamente, sem precisar saber nada a respeito do Consul. Um ótimo caso de uso para isso seria adicionar ou remover nós dinamicamente do pool de um balanceador de carga de software como o HAProxy.

Outra ferramenta que se integra bem com o Consul é o Vault, uma ferramenta de gerenciamento de dados sigilosos que vamos rever na seção “Secrets”, na página [419](#). O gerenciamento de dados sigilosos pode ser

complicado, mas a combinação de Consul e Vault certamente poderá facilitar sua vida.

etcd e Kubernetes

Se você está executando seu sistema em uma plataforma que gerencie cargas de trabalho em contêineres, há uma boa chance de que você já tenha à sua disposição um método para descoberta de serviços. Com o Kubernetes não é diferente, e, em parte, isso se deve ao etcd (<https://etcd.io>), um repositório para gerenciamento de configuração incluído no Kubernetes. O etcd tem recursos semelhantes aos do Consul, e o Kubernetes o utiliza para gerenciar um conjunto variado de informações de configuração.

Exploraremos o Kubernetes com mais detalhes na seção “Kubernetes e a orquestração de contêineres”, na página 309, mas, em poucas palavras, este é o modo como a descoberta de serviços funciona no Kubernetes: implantamos um contêiner em um pod, e então um serviço identifica dinamicamente quais pods devem fazer parte de um serviço por meio de uma correspondência de padrões em metadados associados ao pod. É um método muito elegante e pode ser muito eficaz. Requisições a um serviço serão então encaminhadas para um dos pods que compõem esse serviço.

Os recursos que já estão incluídos no Kubernetes podem fazer com que você se contente apenas com o que já existe na plataforma básica, evitando o uso de ferramentas dedicadas como o Consul; para muitas pessoas, isso faz bastante sentido, sobretudo se o ecossistema mais amplo de ferramentas relacionadas ao Consul não for do seu interesse. No entanto, se você executa seu sistema em um ambiente diversificado, no qual há cargas de trabalho executando no Kubernetes e em outro lugar, ter uma ferramenta dedicada de descoberta de serviços que possa ser utilizada nas duas plataformas talvez seja a opção mais apropriada.

Utilizando o seu próprio sistema

Uma abordagem que eu mesmo já usei e já vi ser usada em outros lugares é ter o seu próprio sistema. Em certo projeto, estávamos fazendo uso intenso da AWS, que disponibiliza o recurso de adicionar tags às instâncias. Ao

iniciar instâncias de serviços, eu aplicava tags para ajudar a definir o que era a instância e a finalidade para a qual era utilizada. Isso permitia que alguns metadados interessantes fossem associados a um dado host – por exemplo:

- serviço = contas
- ambiente = produção
- versão = 154

Então eu utilizava as APIs da AWS para consultar todas as instâncias associadas a uma dada conta da AWS para que eu pudesse encontrar as máquinas que fossem do meu interesse. Nesse caso, a própria AWS lidava com o armazenamento dos metadados associados a cada instância, oferecendo a possibilidade de consultá-los. Em seguida, criei ferramentas de linha de comando para interagir com essas instâncias e disponibilizei interfaces gráficas para visualizar rapidamente o status das instâncias. Tudo isso acaba sendo uma tarefa muito simples se você puder coletar informações sobre as interfaces dos serviços por meio de programação.

Na última vez em que empreguei essa solução, não fomos tão longe a ponto de ter serviços utilizando as APIs da AWS para encontrar suas dependências, mas não haveria motivos para não fazer isso. Obviamente, se quiser que os serviços upstream sejam alertados quando a localização de um serviço downstream mudar, você terá de resolver esse problema por conta própria.

Hoje em dia, essa não seria a opção que eu escolheria. A variedade de ferramentas nessa área está suficientemente madura, portanto esse não seria somente o caso de reinventar a roda, mas de recriar uma roda muito pior.

Não se esqueça das pessoas!

Os sistemas que analisamos até agora facilitam que uma instância de serviço se registre e consulte outros serviços com os quais precise conversar. Contudo, como seres humanos, às vezes podemos querer essas informações também. Deixar as informações disponíveis de maneiras que permitam que as pessoas as consumam, talvez com a utilização de APIs

para extrair esses detalhes em um formato humanamente legível (um assunto que retomaremos em breve), pode ser essencial.

Service meshes e gateways de API

Poucas áreas de tecnologia associadas aos microsserviços tiveram tanta atenção, estiveram tão em evidência e causaram tanta confusão quanto os service meshes e os gateways de API. Ambos têm seus lugares, porém, de modo confuso, também podem se sobrepor no que diz respeito às responsabilidades. O gateway de API em particular é suscetível a um uso indevido (e a uma propaganda enganosa), portanto é importante compreender como esses tipos de tecnologia podem se enquadrar em nossa arquitetura de microsserviços. Em vez de tentar apresentar uma visão detalhada sobre o que você pode fazer com esses produtos, gostaria de apresentar uma visão geral das situações em que eles serão apropriados, como poderão ajudar e algumas das armadilhas a serem evitadas.

No linguajar típico dos data centers, falaríamos de tráfego “leste-oeste” para se referir ao tráfego interno ao data center, enquanto o tráfego “norte-sul” está relacionado às interações de entrada e saída do data center de/para o mundo externo. Do ponto de vista de rede, o conceito de data center se tornou de certa forma nebuloso; assim, em nosso caso, falaremos de modo mais genérico de um perímetro de rede. Isso poderia estar relacionado a um data center completo, a um cluster Kubernetes, ou talvez apenas a um conceito de rede virtual, por exemplo, um grupo de máquinas executando na mesma LAN virtual.

Falando de modo geral, um gateway de API está localizado no perímetro de seu sistema e lida com um tráfego norte-sul. Suas principais preocupações são gerenciar o acesso do mundo externo aos seus microsserviços internos. Um service mesh, por outro lado, lida bem de perto com a comunicação entre os microsserviços dentro do perímetro – o tráfego leste-oeste – conforme mostra a Figura 5.6.

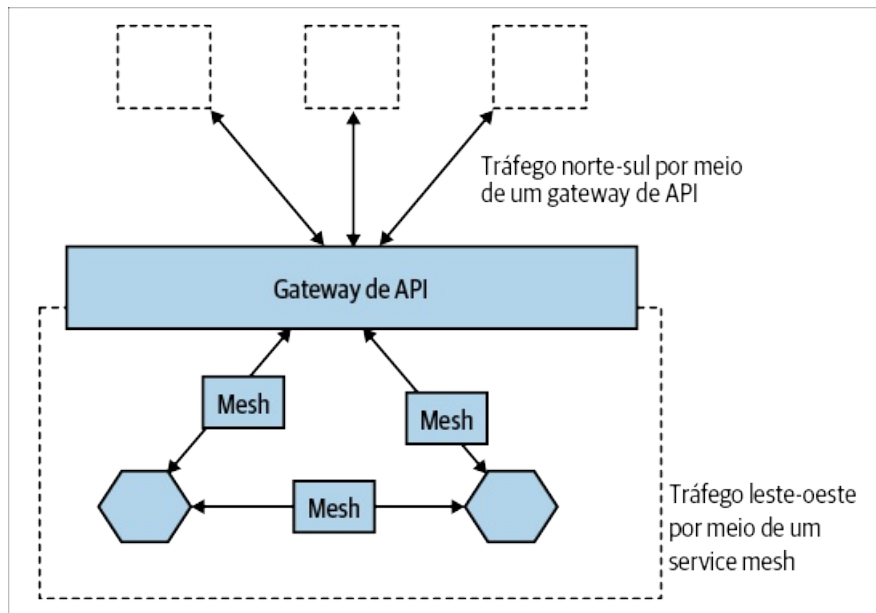


Figura 5.6 – Visão geral dos pontos em que os gateways de API e os service meshes são usados.

Os service meshes e os gateways de API podem permitir que os microsserviços compartilhem código sem exigir a criação de novas bibliotecas de cliente ou novos microsserviços. Falando de modo (muito) simples, os service meshes e os gateways de API podem funcionar como proxies entre os microsserviços. Isso pode implicar a possibilidade de eles serem usados para implementar alguns comportamentos que independam dos microsserviços, os quais, de outro modo, teriam de estar implementados no código, por exemplo, a descoberta de serviços ou logging.

Se você estiver usando um gateway de API ou um service mesh para implementar um comportamento compartilhado e comum aos seus microsserviços, é essencial que esse comportamento seja totalmente genérico – em outras palavras, o comportamento no proxy não deve ter nenhuma relação com um comportamento específico de um microsserviço em particular.

Entretanto, depois dessa explicação, terei de explicar também que o mundo nem sempre é assim tão bem delimitado. Vários gateways de API tentam oferecer recursos para um tráfego leste-oeste também, mas esse é um assunto que discutiremos em breve. Inicialmente, vamos analisar os

gateways de API e os tipos de tarefas que podem fazer.

Gateways de API

Com mais foco no tráfego norte-sul, a principal preocupação de um gateway de API em um ambiente de microsserviços é mapear as requisições de componentes externos para os microsserviços internos. Essa responsabilidade é semelhante àquilo que poderíamos ter com um simples proxy HTTP e, com efeito, os gateways de API em geral disponibilizam outras funcionalidades com base em proxies HTTP existentes, e geralmente funcionam como proxies reversos. Além disso, os gateways de API podem ser usados para implementar soluções como chaves de API para componentes externos, logging, limitação de taxas e tarefas desse tipo. Alguns produtos que são gateways de API também disponibilizam portais para desenvolvedores, geralmente voltados para consumidores externos.

Parte da confusão em torno dos gateways de API tem razões históricas. Um tempo atrás, houve um enorme interesse no que foi chamado de “mercado de APIs”. O mercado havia começado a entender a eficácia de oferecer APIs para soluções gerenciadas, variando de produtos SaaS como o Salesforce a plataformas como a AWS, pois estava claro que uma API proporcionava aos consumidores muito mais flexibilidade no modo como seus softwares eram usados. Isso fez com que muitas pessoas comessem a olhar para o software que já possuíam e considerassem as vantagens de expor essas funcionalidades aos seus clientes não só por meio de uma GUI, mas também com uma API. A expectativa era que isso abriria novas oportunidades no mercado e, bem, proporcionaria mais lucros. Em meio a esse interesse, vários gateways de API surgiram como produtos para ajudar a tornar esses objetivos alcançáveis. Seus conjuntos de recursos estavam fortemente inclinados para o gerenciamento de chaves de API para terceiros, garantia de limites e monitoração de uso com vistas a chargeback (estornos). A verdade é que, embora as APIs tenham, sem dúvida, se mostrado ser uma excelente forma de oferecer serviços para alguns clientes, o tamanho do mercado de APIs não era tão grande quanto alguns esperavam, e muitas empresas começaram a perceber que haviam comprado gateways de API repletos de funcionalidades de que elas, na

verdade, jamais precisariam.

Na maior parte do tempo, tudo que um gateway de API realmente faz é gerenciar o acesso aos microsserviços de uma empresa a partir de seus próprios clientes GUI (páginas web, aplicativos móveis nativos) através da internet pública. Não há nenhum “terceiro” no cenário, nesse caso. A necessidade de ter alguma forma de gateway de API para o Kubernetes é essencial, pois o Kubernetes lida de modo nativo com a rede somente dentro do cluster, e não faz nada para lidar com a comunicação de e para o próprio cluster. Contudo, em um caso de uso como esse, um gateway de API projetado para acesso externo de terceiros é um exagero enorme.

Assim, se você quiser um gateway de API, saiba claramente o que espera dele. Com efeito, eu iria um pouco além e diria que você provavelmente deveria evitar ter um gateway de API que faça tarefas demais. Veremos isso a seguir.

Quando usá-los

Assim que começar a entender os tipos de casos de uso que você tem, será um pouco mais fácil saber o tipo de gateway de que você precisará. Se foi apenas o caso de expor microsserviços executando no Kubernetes, você poderia executar seus próprios proxies reversos – ou, melhor ainda, poderia procurar um produto específico para isso, como o Ambassador, criado desde o princípio com esse caso de uso em mente. Se você realmente se vir precisando gerenciar muitos usuários que sejam terceiros acessando a sua API, é provável que haja outros produtos a serem considerados. É possível, de fato, que você possa acabar com mais de um gateway no cenário para lidar melhor com a separação de responsabilidades, e posso ver que essa é uma atitude sensata em várias situações, embora as ressalvas usuais acerca do aumento da complexidade geral do sistema e do aumento de hops (saltos) na rede continuem se aplicando.

Em algumas ocasiões, já me vi envolvido em trabalhar diretamente com os fornecedores para auxiliar na seleção de ferramentas. Posso dizer, sem hesitações, que já vi mais propagandas enganosas ou comportamentos inadequados na área de gateway de API do que em qualquer outra área – e,

como consequência, você não encontrará referências para fornecedores de produtos neste capítulo. Atribuo muito disso às empresas financiadas por VCs, que haviam criado um produto no auge da era do mercado de APIs, somente para descobrir que esse mercado não existe; desse modo, elas lutam em duas frentes: lutam pelo pequeno número de usuários que realmente precisa daquilo que os gateways mais complexos oferecem, ao mesmo tempo que perdem negócios para produtos com mais enfoque em gateways de API, criados para a maioria das necessidades mais simples.

O que evitar

Parcialmente por causa do aparente desespero de alguns fornecedores de gateways de API, muitas alegações foram feitas para o que esses produtos podem fazer. Isso levou a vários usos indevidos desses produtos, o que, por sua vez, resultou em uma infeliz desconfiança daquilo que, basicamente, é um conceito muito simples. Dois exemplos significativos que já vi quanto a um uso indevido dos gateways de API foram para agregação de chamadas e reescrita de protocolo, mas também já vi uma insistência maior no uso de gateways de API para chamadas internas ao perímetro (leste-oeste).

Neste capítulo, já vimos rapidamente a utilidade de um protocolo como o GraphQL para nos ajudar em uma situação na qual precisamos fazer uma série de chamadas, e então agregar e filtrar os resultados, mas as pessoas muitas vezes ficam tentadas a solucionar esse problema nas camadas de gateway de API também. Tudo começa de modo muito inocente: você combina duas chamadas e devolve um único payload. Em seguida, passa a fazer outra chamada downstream como parte do mesmo fluxo agregado. Então começa a querer adicionar uma lógica condicional e, não muito tempo depois, percebe que incluiu processos essenciais aos negócios em uma ferramenta de terceiros que é bastante inapropriada para a tarefa.

Se você se vir precisando fazer agregação de chamadas e filtragem, analise o potencial do GraphQL ou do padrão BFF, que discutiremos no Capítulo 14. Se a agregação de chamadas que você estiver fazendo for basicamente um processo de negócios, ela será feita de modo mais apropriado com uma saga explicitamente modelada, assunto que abordaremos no Capítulo 6.

Além da perspectiva da agregação, a reescrita de protocolo também é uma tarefa para a qual muitas vezes insistem que os gateways de API deveriam ser usados. Lembro-me de um fornecedor cujo nome não vou citar, promovendo agressivamente a ideia de que seu produto poderia “transformar qualquer API SOAP em uma API REST”. Em primeiro lugar, o REST é uma mentalidade totalmente associada à arquitetura, que não pode ser simplesmente implementada em uma camada de proxy. Em segundo, a reescrita de protocolo, que é basicamente o que está se tentando fazer nesse caso, não deveria ser feita em camadas intermediárias, pois isso forçaria muitos comportamentos a estarem no lugar errado.

O principal problema tanto com a funcionalidade de reescrita de protocolo como com a implementação de agregação de chamadas dentro de gateways de API é que violaremos a regra de manter os pipes (intermediários) burros e os endpoints inteligentes. A “inteligência” em nosso sistema deve estar em nosso código, sobre o qual poderemos ter um controle total. O gateway de API nesse exemplo é um pipe – queremos que ele seja o mais simples possível. Com os microsserviços, insistimos na adoção de um modelo no qual as mudanças possam ser feitas e lançadas com mais facilidade por meio de implantações independentes. Manter a inteligência em nossos microsserviços ajuda nesse sentido. Se tivermos agora de fazer alterações em camadas intermediárias, a situação começará a ficar mais problemática. Considerando a criticidade dos gateways de API, mudanças nesses gateways em geral são rigorosamente controladas. Parece improvável que equipes individuais terão a liberdade de fazer mudanças por conta própria nesses serviços que são geralmente gerenciados de forma centralizada. O que isso significa? Tickets. Para fazer o rollout de uma alteração em seu software, você acabará tendo de pedir à equipe do gateway de API que faça alterações para você. Quanto mais comportamentos se insinuarem para dentro dos gateways de API (ou para os enterprise service buses [barramentos de serviços corporativos]), maior será o risco de haver transferência de responsabilidades (handoffs), maior será a necessidade de coordenação e as entregas serão mais demoradas.

O último problema está no uso de um gateway de API como um intermediário para todas as chamadas entre microsserviços. Isso pode ser

extremamente problemático. Se inserirmos um gateway de API ou um proxy de rede comum entre dois microsserviços, em geral adicionaremos ao menos um hop de rede. Uma chamada do microsserviço A para o microsserviço B será feita inicialmente de A para o gateway de API, e então do gateway de API para B. Devemos considerar o impacto da latência com a chamada de rede adicional, e o overhead de qualquer que seja o trabalho que o proxy fizer. Os service meshes, que exploraremos a seguir, são muito mais apropriados para resolver esse problema.

Service meshes

Com um service mesh, uma funcionalidade comum associada à comunicação entre microsserviços é passada para o mesh. Isso reduz as funcionalidades que um microsserviço precisa implementar internamente, ao mesmo tempo que oferece consistência para o modo como determinadas tarefas são feitas.

Recursos comuns implementados por service meshes incluem TLS mútuo, correlação de IDs, descoberta de serviços e balanceamento de carga, além de outros. Com frequência, esses tipos de funcionalidade são razoavelmente genéricos de um microsserviço para outro, portanto acabaríamos fazendo uso de uma biblioteca compartilhada para lidar com eles. No entanto, teríamos então de lidar com o que aconteceria se diferentes microsserviços tiverem versões distintas das bibliotecas em execução, ou se tivéssemos microsserviços escritos em runtimes diferentes.

Historicamente, pelo menos, a Netflix obrigava que qualquer comunicação de rede não local tinha de ser feita de JVM para JVM. Isso servia para garantir que as bibliotecas comuns, testadas e comprovadas, que eram uma parte essencial do gerenciamento eficaz da comunicação entre microsserviços, pudessem ser reutilizadas. Com o uso de um service mesh, porém, temos a possibilidade de reutilizar funcionalidades comuns aos microsserviços em diferentes microsserviços escritos em linguagens de programação distintas. Os service meshes também podem ser extremamente convenientes para implementar comportamentos padrões entre microsserviços criados por diferentes equipes – e o uso de um service

mesh, particularmente no Kubernetes, tem se tornado cada vez mais uma parte esperada de qualquer plataforma específica que você possa criar para implantação e gerenciamento próprio de seus microsserviços.

Facilitar a implementação de comportamentos comuns entre microsserviços é uma das grandes vantagens de um service mesh. Se essas funcionalidades comuns forem implementadas somente por meio de bibliotecas compartilhadas, alterar esses comportamentos exigiria que todos os microsserviços utilizassem uma nova versão da biblioteca em questão e fossem implantados para que a mudança se tornasse ativa. Com um service mesh, teremos muito mais flexibilidade para fazer o rollout de mudanças na comunicação entre microsserviços, sem exigir sua reconstrução e reimplantação.

Como funcionam

Em geral, esperaríamos ter menos tráfego norte-sul do que tráfego leste-oeste em uma arquitetura de microsserviços. Uma única chamada norte-sul – fazer um pedido, por exemplo – poderia resultar em várias chamadas leste-oeste. Isso significa que, ao considerar qualquer tipo de proxy para chamadas internas ao perímetro, devemos estar cientes do overhead que essas chamadas adicionais poderão causar, e essa é uma consideração essencial para o modo como os service meshes são criados.

Os service meshes se apresentam em diferentes formatos e tamanhos, porém o que os une é o fato de sua arquitetura se basear na tentativa de limitar o impacto causado pelas chamadas de e para o proxy. Isso é feito basicamente por meio da distribuição dos processos proxy de modo a executarem nas mesmas máquinas físicas que as instâncias dos microsserviços, garantindo que o número de chamadas de rede remotas seja limitado. Na Figura 5.7, vemos isso em ação – `Processador de Pedido` envia uma requisição para o microsserviço `Pagamento`. Essa chamada é inicialmente encaminhada de forma local, para uma instância do proxy executando na mesma máquina que `Processador de Pedido`, antes de prosseguir para o microsserviço `Pagamento` por meio de sua instância de proxy local. `Processador de Pedido` acha que está fazendo uma chamada de rede usual, sem saber que a chamada será encaminhada localmente na

máquina, o que é significativamente mais rápido (além de ser menos suscetível a partições).

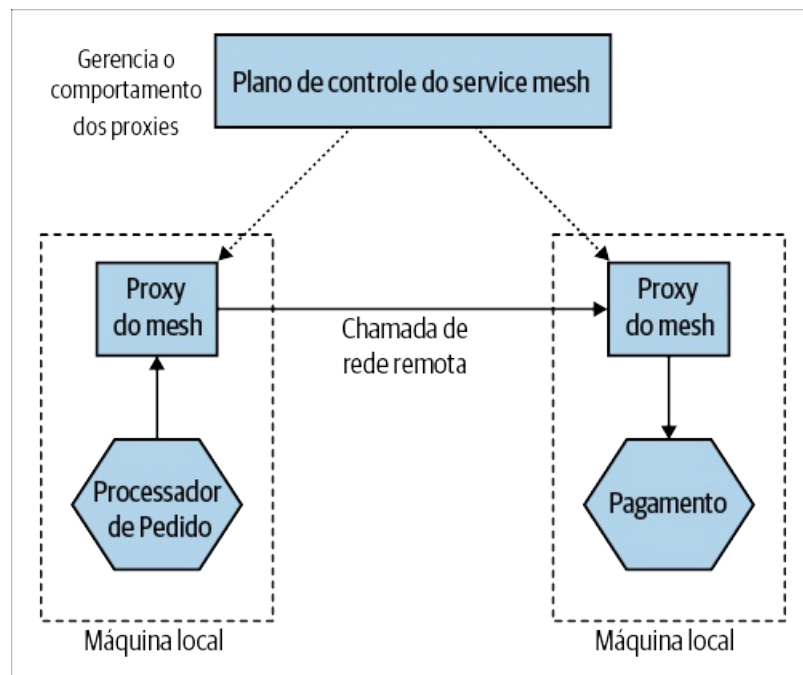


Figura 5.7 – Um service mesh é implantado para lidar com qualquer comunicação direta entre microsserviços.

Um plano de controle estaria acima dos proxies locais do mesh, funcionando como um lugar em que o comportamento desses proxies pode ser alterado e no qual podemos coletar informações sobre as atividades dos proxies.

Ao fazer a implantação no Kubernetes, implantaríamos cada instância do microsserviço em um pod com seu próprio proxy local. Um único pod é sempre implantado como uma única unidade, portanto sempre saberemos que haverá um proxy disponível. Além disso, a falha em um único proxy causaria impacto somente no pod específico em questão. Essa configuração também permite configurar cada proxy de modo diferente, com diferentes propósitos. Veremos esses conceitos com mais detalhes na seção “Kubernetes e a orquestração de contêineres”, na página 309.

Muitas implementações de service mesh utilizam o proxy Envoy (<https://www.envoyproxy.io>) como base para esses processos que executam localmente. O Envoy é um proxy C++ leve, geralmente usado como bloco

de construção dos service meshes e outros tipos de softwares baseados em proxy – é um bloco de construção importante para o Istio e o Ambassador, por exemplo.

Esses proxies, por sua vez, são gerenciados por um plano de controle. Esse plano é composto de um conjunto de softwares que ajudarão você a verificar o que acontece e a controlar o que é feito. Ao utilizar um service mesh para a implementação de TLS mútuo, por exemplo, o plano de controle seria usado para distribuir certificados de clientes e servidores.

Os service meshes não são pipes inteligentes?

Então, toda essa conversa de passar comportamentos comuns para dentro de um service mesh talvez tenha feito soar alguns alarmes para alguns de vocês. Essa abordagem não seria propícia aos mesmos tipos de problemas dos enterprise service buses (barramentos de serviços corporativos) ou dos gateways de API com excesso de tarefas? Não corremos o risco de passar “inteligência” demais para o nosso service mesh?

O principal ponto a ser lembrado nesse caso é que os comportamentos comuns que estamos passando para o mesh não são específicos de nenhum microsserviço em particular. Não houve vazamento externo de nenhuma funcionalidade de negócios. Estamos configurando aspectos genéricos, por exemplo, o modo de lidar com timeouts de requisições. Quanto a um comportamento comum que possa precisar de ajustes individuais de acordo com cada microsserviço, em geral, isso será devidamente resolvido, sem a necessidade de qualquer trabalho a ser feito em uma plataforma centralizada. Por exemplo, com o Istio, posso definir meus requisitos de timeout por conta própria, apenas modificando a definição do meu serviço.

Você precisa de um service mesh?

Quando o uso dos service meshes começou a se tornar popular, logo após o lançamento da primeira edição deste livro, vi muito mérito na ideia, mas percebi também que havia muita rotatividade nessa área. Diferentes modelos de implantação foram sugeridos, criados e então descartados, e o número de empresas oferecendo soluções nessa área aumentou drasticamente; porém, mesmo para as ferramentas que já existiam há

muito tempo, havia uma aparente falta de estabilidade. A Linkerd (<https://linkerd.io>), que, sem dúvida, fez mais do que qualquer outra empresa como pioneira nessa área, recriou totalmente o seu produto do zero na passagem da v1 para a v2. O Istio (<https://istio.io>), que era o service mesh abençoado pelo Google, levou anos para chegar a uma versão 1.0 inicial e, mesmo assim, fez mudanças subsequentes significativas em sua arquitetura (passando para um modelo de implantação mais monolítico de seu plano de controle, o que foi de certo modo irônico, porém sensato).

Em boa parte dos últimos cinco anos, quando me perguntavam “Devemos adquirir um service mesh?”, meu conselho havia sido: “Se puder esperar seis meses antes de tomar uma decisão, então espere seis meses”. Eu havia me rendido à ideia, porém estava preocupado com a estabilidade. E algo como um service mesh não é um assunto com o qual eu, pessoalmente, gostaria de correr muitos riscos – ele é extremamente importante e essencial para que tudo funcione bem. Você o colocará em seu caminho crítico. Ele estará no mesmo nível que selecionar um broker de mensagens ou um provedor de nuvem no que diz respeito à seriedade com a qual eu o encararia.

Desde então, fico feliz em dizer que essa área amadureceu. A rotatividade, até certo ponto, diminuiu, mas ainda temos uma diversidade (saudável) de fornecedores. Apesar do que dissemos, os service meshes não são apropriados para todos. Em primeiro lugar, se você não estiver no Kubernetes, suas opções serão limitadas. Em segundo, eles adicionam complexidade. Se você tiver cinco microsserviços, não acho que seria fácil justificar um service mesh. (É discutível se você poderá justificar o Kubernetes caso tenha somente cinco microsserviços!) Para empresas que tenham mais microsserviços, sobretudo se quiserem a opção de ter esses microsserviços escritos em diferentes linguagens de programação, considerar os service meshes valerá muito a pena. Contudo, faça a sua lição de casa – trocar de service meshes é complicado!

A Monzo é uma empresa que falou abertamente sobre como o seu uso de um service mesh foi essencial para permitir que ela executasse sua arquitetura na escala que o faz. Seu uso da versão 1 do Linkerd para ajudar

a gerenciar as chamadas RPC entre microsserviços se provou ser extremamente vantajoso. O interessante é que a Monzo precisou lidar (<https://oreil.ly/5dLGC>) com as dificuldades de uma migração de service mesh para ajudá-la a atingir a escala de que precisava quando a arquitetura mais antiga do Linkerd v1 deixou de atender seus requisitos. No final, ela passou efetivamente para um service mesh desenvolvido internamente, fazendo uso do proxy Envoy.

E quanto aos outros protocolos?

Os gateways de API e os service meshes são usados principalmente para lidar com chamadas relacionadas ao HTTP. Assim, REST, SOAP, gRPC e protocolos semelhantes podem ser gerenciados por meio desses produtos. Contudo, a situação se complica um pouco mais quando começamos a considerar a comunicação por meio de outros protocolos, por exemplo, com o uso de brokers de mensagens como o Kafka. Em geral, nesse ponto, o service mesh será ignorado – a comunicação será feita diretamente com o próprio broker. Isso significa que você não pode supor que seu service mesh será capaz de atuar como um intermediário para qualquer chamada entre microsserviços.

Documentação dos serviços

Ao decompor nossos sistemas em microsserviços menores, espera-se que muitas fronteiras serão expostas na forma de APIs que as pessoas poderão utilizar para a execução de inúmeras tarefas incríveis. Se nossas descobertas forem feitas corretamente, saberemos onde cada serviço está. Mas como saberemos o que cada serviço faz ou como usá-los? Uma opção, obviamente, é ter uma documentação das APIs. É claro que a documentação muitas vezes pode estar desatualizada. O ideal é que devemos garantir que nossa documentação esteja sempre atualizada em relação à API do microsserviço, facilitando consultá-la se soubermos onde está o endpoint de um serviço.

Esquemas explícitos

Ter esquemas explícitos contribui muito para facilitar a compreensão do que qualquer endpoint em particular expõe, mas, por si só, isso geralmente não bastará. Conforme já discutimos, os esquemas ajudam a mostrar a estrutura, porém não contribuem muito para descrever o comportamento de um endpoint, portanto uma boa documentação ainda pode ser necessária para ajudar os consumidores a saberem como utilizar um endpoint. Evidentemente, vale a pena observar que, se você decidiu não utilizar um esquema explícito, sua documentação acabará tendo *mais* trabalho. Será preciso explicar o que o endpoint faz, além de documentar a estrutura e os detalhes da interface. Além do mais, sem um esquema explícito, saber se sua documentação está atualizada em relação aos endpoints reais será mais difícil. Uma documentação desatualizada é sempre um problema; um esquema explícito, porém, terá mais chances de estar atualizado.

Já apresentei o OpenAPI como um formato de esquemas, mas ele também é muito eficaz para fornecer uma documentação e, hoje em dia, há muitas ferramentas open source e comerciais que oferecem suporte para consumir descritores OpenAPI com o intuito de ajudar na criação de portais convenientes, permitindo que os desenvolvedores leiam a documentação. Vale a pena observar que os portais open source para visualização do OpenAPI parecem ser, de certo modo, básicos – tive dificuldade de encontrar um que oferecesse a funcionalidade de pesquisa, por exemplo. Para aqueles que utilizam o Kubernetes, o portal do desenvolvedor (<https://oreil.ly/8pg12>) do Ambassador é particularmente interessante. O Ambassador já é uma opção popular como gateway de API para o Kubernetes, e seu Developer Portal (Portal do Desenvolvedor) é capaz de descobrir automaticamente os endpoints OpenAPI disponíveis. A ideia de implantar um novo microsserviço e ter sua documentação automaticamente disponível é extremamente atraente para mim.

No passado, não contávamos com um bom suporte para documentar interfaces baseadas em eventos. Hoje em dia, ao menos temos opções. O formato AsyncAPI teve início como uma adaptação do OpenAPI, e atualmente temos também o CloudEvents, que é um projeto da CNCF. Ainda não utilizei nenhum deles intensamente (isto é, em um ambiente

real), porém me sinto mais atraído pelo CloudEvents somente porque ele parece ter recursos amplos para integração e suporte, principalmente por causa de sua associação com a CNCF. Historicamente, pelo menos, o CloudEvents pareceu ser mais restritivo no que se refere ao formato dos eventos quando comparado ao AsyncAPI, com suporte apropriado somente para JSON, até que o suporte para buffers de protocolo foi recentemente reintroduzido, depois de ter sido anteriormente removido; desse modo, esse é um aspecto que deve ser considerado.

Sistema que se autodescreve

Durante o início da evolução do SOA, padrões como UDDI (Universal Description, Discovery, and Integration, ou Descrição, Descoberta e Integração Universais) surgiram para nos ajudar a entender o que os serviços executavam. Essas abordagens eram razoavelmente pesadas, o que levou à criação de técnicas alternativas para tentar compreender os nossos sistemas. Martin Fowler discutiu o conceito de humane registry (registro para pessoas, <https://oreil.ly/UI0YJ>), uma abordagem muito mais leve, na qual as pessoas podem registrar informações sobre os serviços da empresa em algo tão básico quanto uma wiki.

Ter uma imagem de nosso sistema e de como ele se comporta é importante, sobretudo quando atuamos em grande escala. Já discutimos uma série de técnicas diferentes que nos ajudarão a adquirir informações diretamente de nosso sistema. Ao monitorar a saúde dos nossos serviços downstream, junto com IDs de correlação para nos ajudar a ver as cadeias de chamadas, poderemos obter dados reais para o modo como nossos serviços se inter-relacionam. Ao utilizar sistemas de descoberta de serviços como o Consul, podemos ver onde nossos microsserviços estão executando. Soluções como o OpenAPI e o CloudEvents podem nos ajudar a ver quais recursos estão hospedados em qualquer dado endpoint, enquanto nossas páginas de monitoração de sanidade e os sistemas de monitoração nos permitem saber como está a saúde tanto do sistema como um todo como dos serviços individuais.

Todas essas informações estão disponíveis por meio de programação.

Todos esses dados nos permitem deixar nosso “registro para pessoas” mais eficaz em comparação com uma página wiki simples que, sem dúvida, ficará desatualizada. Devemos usá-lo para capturar e exibir todas as informações que nosso sistema gerar. Ao criar painéis de controle personalizados, podemos reunir o vasto conjunto de informações disponíveis para nos ajudar a compreender o nosso ecossistema.

Comece com algo simples, como uma página web estática ou uma wiki que talvez consiga extrair um pouco de dados do sistema em execução. Contudo, procure incluir cada vez mais informações com o tempo. Deixar essas informações prontamente disponíveis é uma ferramenta essencial para gerenciar a complexidade que surgirá da execução desses sistemas em maior escala.

Já falei com algumas empresas que tiveram esses problemas e que acabaram criando registros internos simples para ajudar a reunir metadados sobre os serviços. Alguns desses registros simplesmente percorrem os repositórios de códigos-fontes, procurando arquivos de metadados para criar uma lista dos serviços existentes. Essas informações poderão ser combinadas com dados reais provenientes de sistemas de descoberta de serviços, como o Consul ou o etcd, a fim de compor um quadro mais completo sobre o que está sendo executado e com quem você poderia conversar a respeito desses serviços.

O *Financial Times* criou o Biz Ops para ajudar a resolver esse problema. A empresa tem várias centenas de serviços desenvolvidos por equipes no mundo todo. A ferramenta Biz Ops (Figura 5.8) oferece à empresa um local único no qual é possível encontrar muitas informações úteis sobre seus microsserviços, além de informações sobre outros serviços da infraestrutura de TI, como redes e servidores de arquivos. Construído com base em um banco de dados de grafos, o Biz Ops oferece muita flexibilidade quanto aos dados que coleta e como as informações podem ser modeladas.

A ferramenta Biz Ops, contudo, vai além da maioria das ferramentas semelhantes que já vi. Ela calcula o que chama de System Operability Score (Pontuação para a Operacionalidade do Sistema), como mostra a

Figura 5.9. A ideia é que há certas tarefas que os serviços e suas equipes devem fazer para garantir que os serviços possam ser facilmente operados. Essas tarefas podem variar, desde garantir que as equipes tenham fornecido as informações corretas no registro, até certificar-se de que os serviços tenham verificações de sanidade apropriadas. O System Operability Score, assim que é calculado, permite que as equipes vejam rapidamente se há algo que precise ser corrigido.

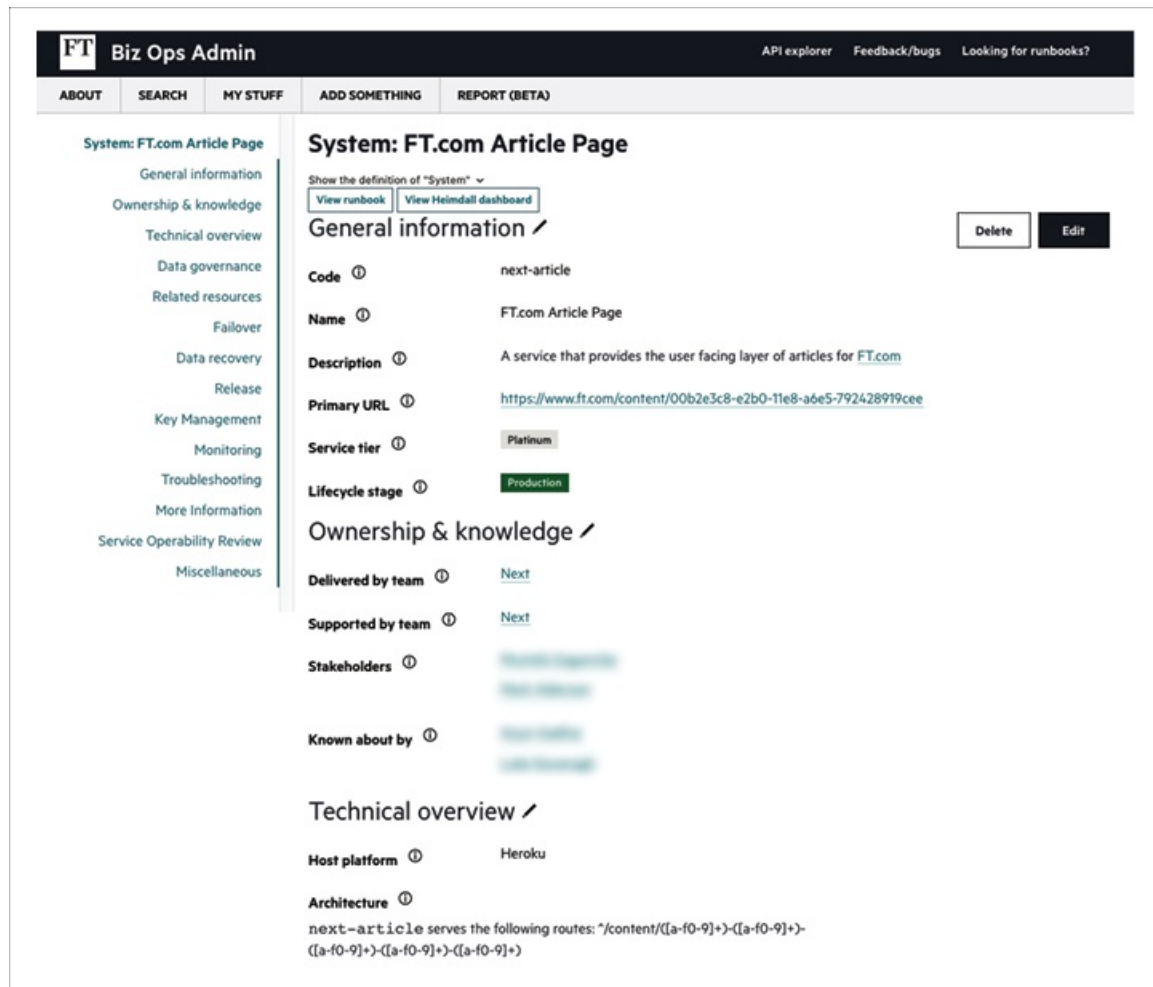


Figura 5.8 – Ferramenta Biz Ops do Financial Times, que reúne informações sobre seus microserviços.

Essa é uma área em expansão. No mundo do open source, a ferramenta Backstage (<https://backstage.io>) do Spotify disponibiliza um mecanismo para criar um catálogo de serviços como o Biz Ops, com um modelo plugin que possibilita acréscimos sofisticados, por exemplo, a capacidade de

disparar a criação de um novo microserviço ou extrair informações de um cluster Kubernetes durante a execução. O próprio Service Catalog (<https://oreil.ly/7o649>) do Ambassador está mais especificamente centrado na visibilidade dos serviços no Kubernetes, o que significa que ele talvez não seja tão atraente ao público em geral como o Biz Ops do Financial Times; apesar disso, é bom ver que há algumas novidades relacionadas a essa ideia, que estão disponíveis de modo geral.

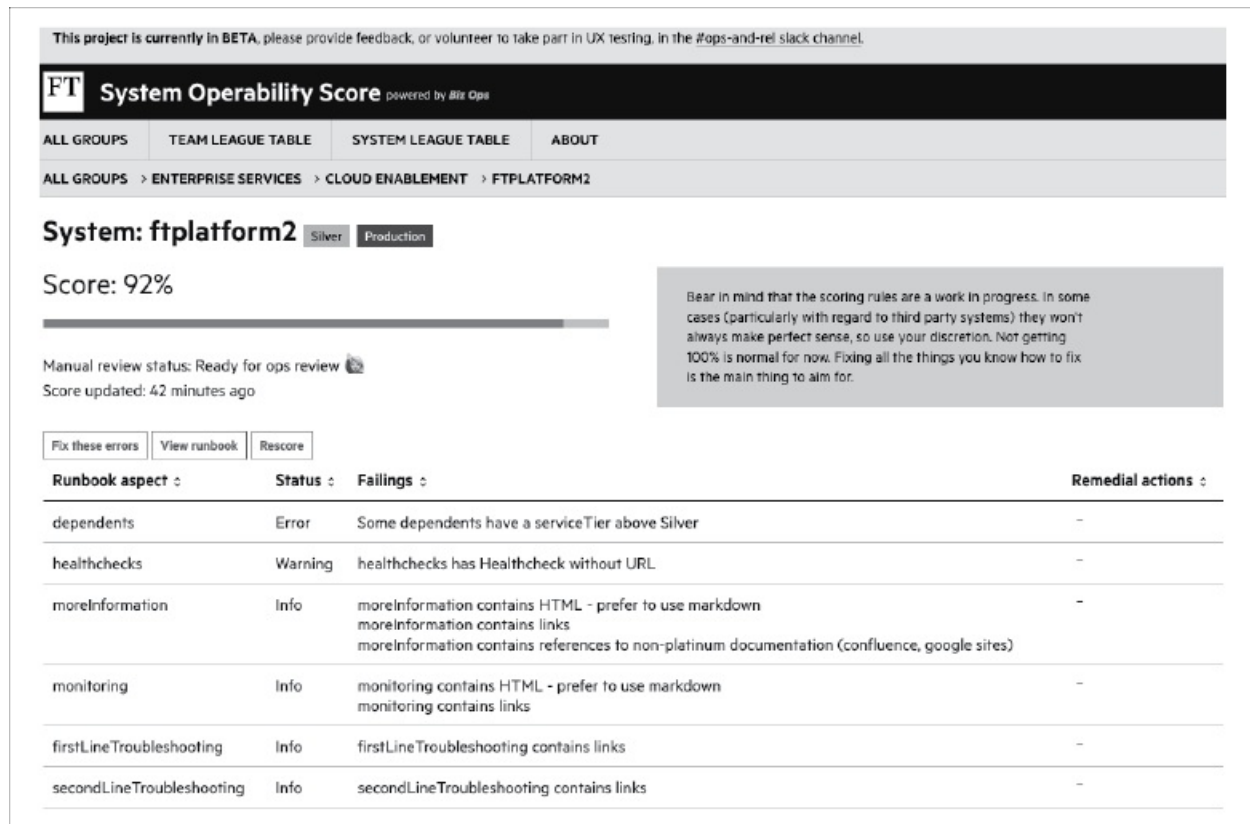


Figura 5.9 – Um exemplo da Service Operability Score (Pontuação para a Operacionalidade de um Serviço) para um microserviço do Financial Times.

Resumo

Abordamos vários assuntos neste capítulo – vamos destacar alguns:

- Para começar, garanta que o problema que você está tentando resolver oriente a sua escolha de tecnologia. Com base em seu contexto e em seu estilo preferido de comunicação, selecione a tecnologia que seja mais apropriada para você – não caia na armadilha de escolher a tecnologia

antes. O resumo dos estilos de comunicação entre microserviços, apresentado inicialmente no Capítulo 4 e exibido novamente na Figura 5.10, pode ajudar a orientar a sua tomada de decisões, porém apenas seguir esse modelo não é um substituto para parar e pensar em sua própria situação.

- Qualquer que seja a escolha que você fizer, considere o uso de esquemas, em parte para ajudar a deixar seus contratos mais explícitos, mas também para ajudar a identificar incompatibilidades acidentais.

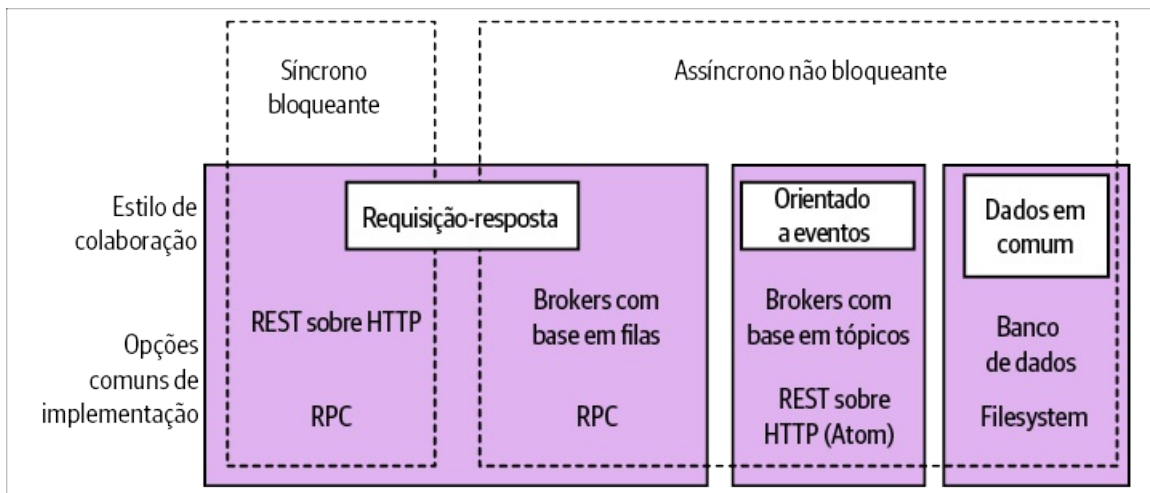


Figura 5.10 – Diferentes estilos de comunicação entre microserviços, com exemplos de tecnologias que implementam esses estilos.

- Sempre que for possível, esforce-se para fazer alterações que sejam compatíveis com versões anteriores a fim de garantir que as implantações independentes continuem sendo uma possibilidade.
- Se você tiver de fazer alterações que sejam incompatíveis com versões anteriores, encontre uma forma de permitir que os consumidores tenham tempo para fazer um upgrade, evitando implantações sincronizadas.
- Pense no que você pode fazer para ajudar a trazer à tona informações sobre seus endpoints para as pessoas – considere o uso de registros para pessoas (humane registries) e ferramentas desse tipo para ajudar a entender o caos.

Vimos como podemos implementar uma chamada entre dois

microserviços, mas o que acontecerá se precisarmos coordenar as operações entre vários microserviços? Esse será o foco do nosso próximo capítulo.

- 1 Martin Fowler explora esse assunto com mais detalhes no contexto da armazenagem de dados sem esquema (schemaless data storage, <https://oreil.ly/Ew8Jq>).
- 2 Observe que há, na verdade, três ferramentas distintas com o mesmo nome nessa área! A ferramenta openapi-diff em <https://github.com/Azure/openapi-diff> parece estar mais próxima de uma ferramenta que seja bem-sucedida ou falhe com base na compatibilidade.

Fluxo de trabalho

Nos dois capítulos anteriores, analisamos aspectos dos microsserviços relacionados ao modo como um microsserviço conversa com outro. Mas o que acontecerá se quisermos que vários microsserviços colaborem entre si, talvez para implementar um processo de negócios? Modelar e implementar esses tipos de fluxo de trabalho em sistemas distribuídos pode ser uma tarefa complicada para ser feita corretamente.

Neste capítulo, veremos as armadilhas associadas ao uso de transações distribuídas para solucionar esse problema, e veremos também as sagas – um conceito que pode nos ajudar a modelar os fluxos de trabalho de nossos microsserviços de uma maneira muito mais satisfatória.

Transações em bancos de dados

Falando de modo geral, quando pensamos em uma transação no contexto da computação, pensamos em uma ou mais ações que irão ocorrer, as quais queremos tratar como uma unidade. Ao fazer várias alterações como parte da mesma operação geral, queremos confirmar se todas as alterações foram efetuadas. Também queremos ter uma forma de fazer uma limpeza caso haja algum erro durante essas alterações. Em geral, esse cenário nos faz utilizar uma solução como uma transação de banco de dados.

Em um banco de dados, utilizamos uma transação para garantir que uma ou mais mudanças de estado sejam feitas com sucesso. Essas mudanças poderiam incluir remoção, inserção ou modificação de dados. Em bancos de dados relacionais, poderiam envolver a atualização de várias tabelas em uma única transação.

Transações ACID

Em geral, quando falamos de transações de banco de dados, estamos falando de transações ACID. ACID é um acrônimo que representa as principais propriedades das transações de banco de dados, as quais resultam em um sistema com o qual podemos contar para garantir a durabilidade e a consistência de nosso repositório de dados. *ACID* quer dizer *Atomicity, Consistency, Isolation, and Durability* (Atomicidade, Consistência, Isolamento e Durabilidade), e essas propriedades nos oferecem o seguinte:

Atomicidade

Garante que todas as operações que tentarmos executar na transação serão concluídas, ou que todas falharão. Se alguma das alterações que estivermos tentando fazer falhar por algum motivo, a operação como um todo será cancelada, e será como se nenhuma mudança jamais tivesse sido feita.

Consistência

Quando alterações são feitas em nosso banco de dados, garantimos que ele permanecerá em um estado válido e consistente.

Isolamento

Permite que várias transações sejam feitas ao mesmo tempo, sem interferência entre elas. Isso é feito garantindo que qualquer mudança de estado intermediária feita durante uma transação seja invisível às demais transações.

Durabilidade

Garante que, assim que uma transação for concluída, podemos ter a certeza de que os dados não serão perdidos caso haja alguma falha no sistema.

Vale a pena observar que nem todos os bancos de dados oferecem transações ACID. Todos os sistemas de bancos de dados relacionais que já usei oferecem, assim como muitos dos bancos de dados NoSQL mais recentes, como o Neo4j. O MongoDB, por muitos anos, oferecia suporte para transações ACID somente para mudanças feitas em um único documento, o que poderia causar problemas se quiséssemos fazer uma

atualização atômica em mais de um documento.¹

Este não é um livro para uma exploração detalhada desses conceitos; sem dúvida, eu simplifiquei algumas dessas descrições para ser mais conciso. Para aqueles que quiserem explorar melhor esses conceitos, recomendo o livro *Designing Data-Intensive Applications*.² Nossa principal preocupação nas próximas seções será com a atomicidade. Isso não quer dizer que as outras propriedades não sejam igualmente importantes, mas descobrir como lidar com a atomicidade nas operações de bancos de dados tende a ser o primeiro problema que enfrentamos quando começamos a separar as funcionalidades em microsserviços.

Ainda ACID, mas sem atomicidade?

Gostaria de deixar claro que podemos continuar usando transações em estilo ACID quando utilizamos microsserviços. Um microsserviço é livre para usar uma transação ACID para operações em seu próprio banco de dados, por exemplo. Apenas o escopo dessas transações estará reduzido às mudanças de estado que ocorrerem localmente, no interior desse único microsserviço. Considere a Figura 6.1. Nesse caso, estamos controlando o processo envolvido no cadastramento de um novo cliente da MusicCorp. Alcançamos o final do processo, que envolve modificar o `status` do cliente 2346 de `PENDENTE` para `VERIFICADO`. Como o cadastramento agora está completo, também queremos remover a linha correspondente da tabela `CadastramentosPendentes`. Se houvesse um único banco de dados, isso seria feito no escopo de uma única transação ACID no banco de dados – as duas mudanças de estado ocorreriam, ou nenhuma delas ocorreria.

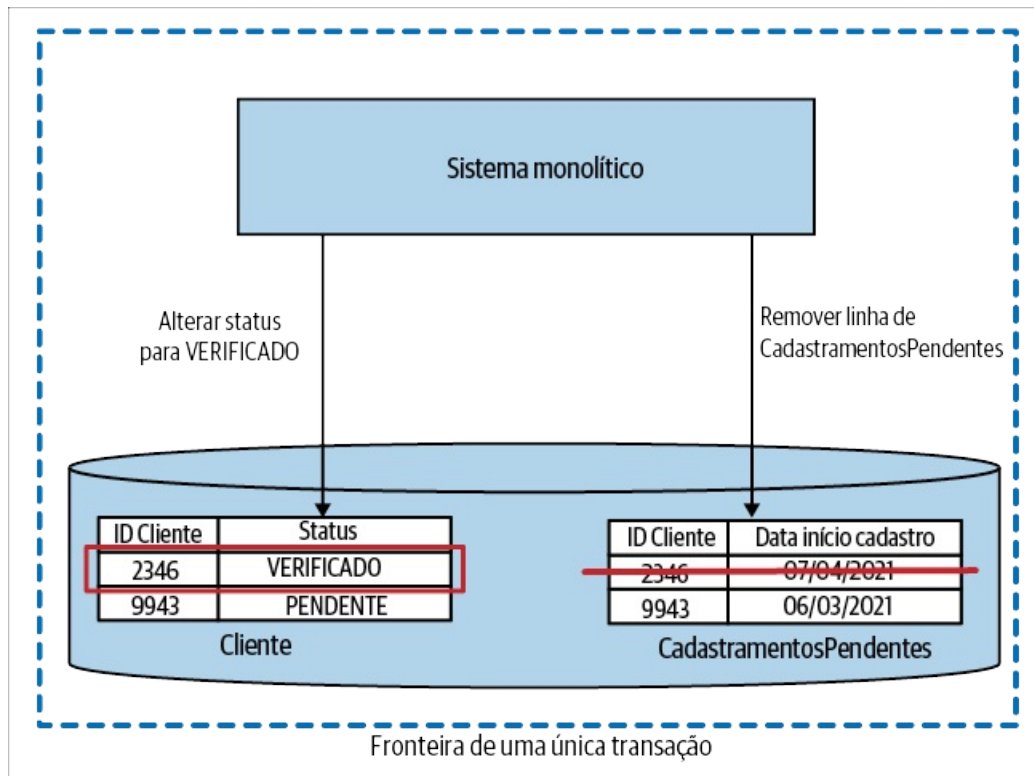


Figura 6.1 – Atualizando duas tabelas no escopo de uma única transação ACID.

Compare isso com a Figura 6.2, na qual fazemos exatamente a mesma mudança, mas cada alteração é feita em um banco de dados diferente. Isso significa que há duas transações a serem consideradas, cada uma com a possibilidade de ser bem-sucedida ou de falhar de forma independente.

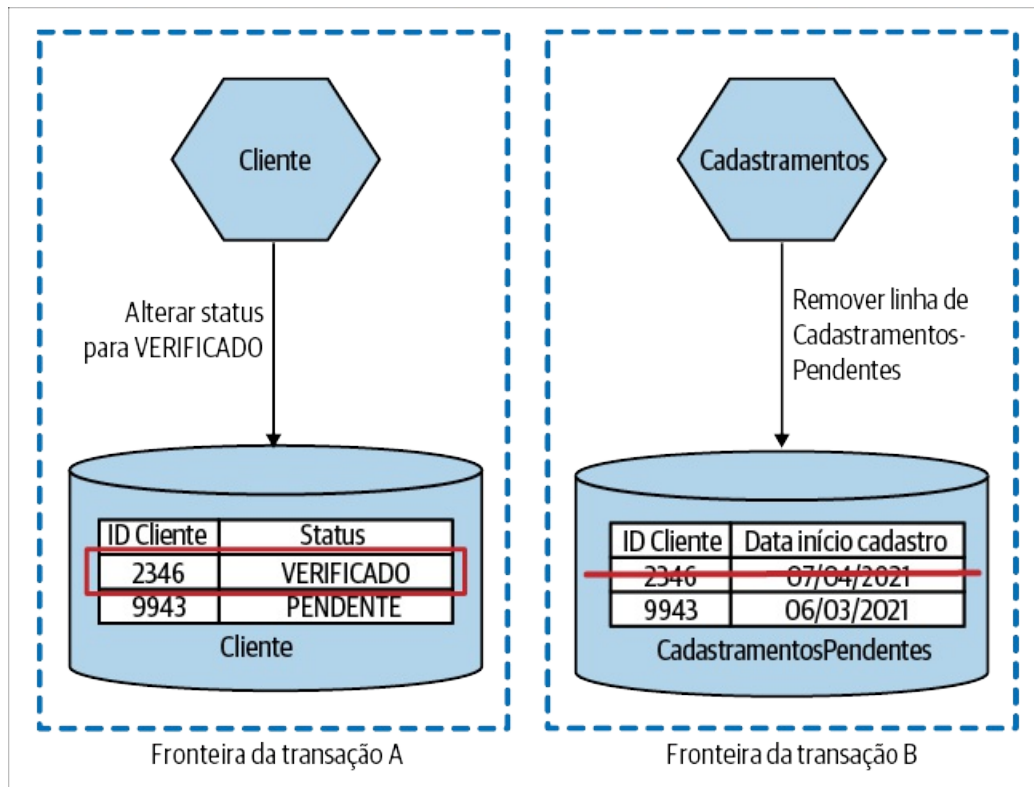


Figura 6.2 – Alterações feitas pelos microsserviços Cliente e Cadastramentos estão agora no escopo de duas transações distintas.

Poderíamos ter decidido executar essas duas transações em sequência, é claro, removendo uma linha da tabela `CadastramentosPendentes` somente se conseguíssemos modificar a linha na tabela `cliente`. Contudo, ainda teríamos de pensar no que fazer caso a remoção na tabela `CadastramentosPendentes` falhasse, ou seja, em toda a lógica que teríamos de implementar por conta própria. Entretanto, ser capaz de reordenar os passos para lidar com esses casos de uso de modo mais apropriado poderia ser realmente uma boa ideia (vamos voltar a ela quando explorarmos as sagas). Basicamente, porém, temos de aceitar que, ao decompor essa operação em duas transações de banco de dados separadas, perdemos a garantia da atomicidade da operação como um todo.

Essa falta de atomicidade pode causar problemas significativos, sobretudo se estivermos fazendo a migração de sistemas que antes contavam com essa propriedade. Geralmente, a primeira opção que as pessoas consideram é continuar utilizando uma única transação, porém uma transação que envolva vários processos, isto é, uma transação distribuída. Infelizmente,

conforme veremos, as transações distribuídas talvez não sejam a opção correta. Vamos ver um dos algoritmos mais comuns para implementar as transações distribuídas – o commit de duas fases (two-phase commit) – como uma forma de explorar as dificuldades associados às transações distribuídas de modo geral.

Transações distribuídas – commit de duas fases

O algoritmo de *commit de duas fases* (two-phase commit, às vezes abreviado como *2PC*) geralmente é usado na tentativa de permitir que possamos fazer mudanças transacionais em um sistema distribuído, nas quais vários processos distintos precisariam ser atualizados como parte da operação como um todo. As transações distribuídas, e mais especificamente os commits de duas fases, muitas vezes são levados em consideração por equipes que estejam migrando para arquiteturas de microsserviços, como uma maneira de solucionar os desafios que enfrentam. Contudo, como veremos, esse tipo de transação talvez não resolva seus problemas, podendo inclusive causar mais confusão no sistema.

O algoritmo 2PC é dividido em duas fases (daí o nome *commit de duas fases*): uma fase de votação e uma fase de commit. Durante a *fase de votação*, um coordenador centralizado entra em contato com todos os workers que farão parte da transação e solicita uma confirmação para saber se uma mudança de estado pode ou não ser feita. Na Figura 6.3, vemos duas requisições: uma para modificar o status de um cliente para VERIFICADO e outra para remover uma linha de nossa tabela `CadastramentosPendentes`. Se todos os workers concordarem que a mudança de estado sobre a qual estão sendo inquiridos pode ocorrer, o algoritmo passará para a próxima fase. Se algum worker informar que a mudança não pode ser feita, talvez porque a mudança de estado solicitada viole alguma condição local, a operação como um todo será cancelada.

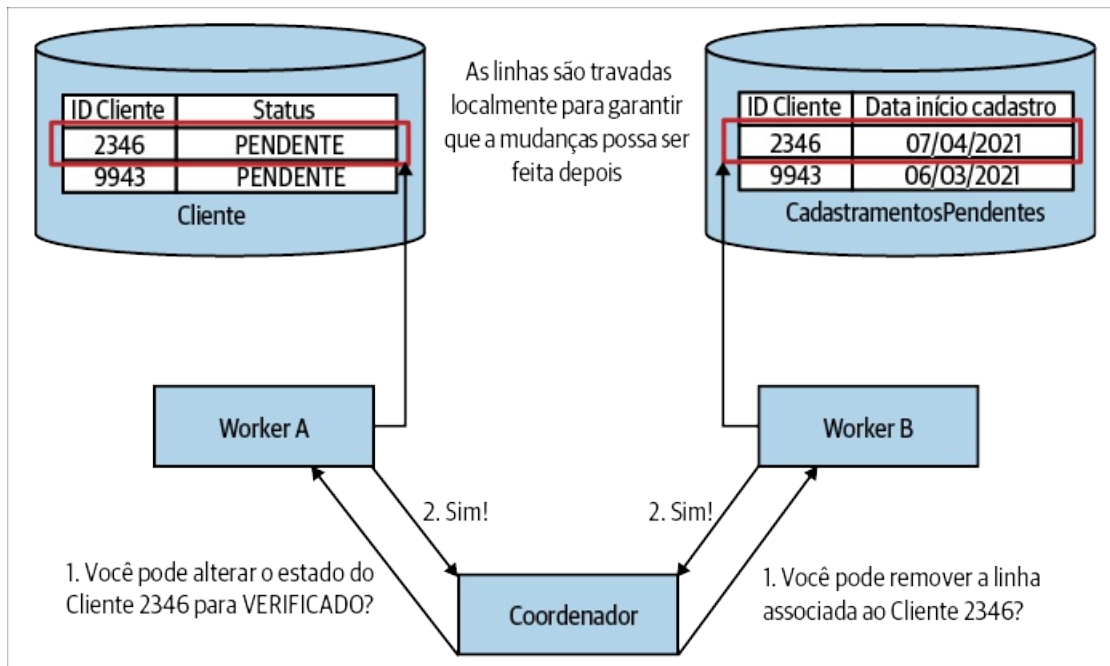


Figura 6.3 – Na primeira fase de um commit de duas fases, os workers votam para decidir se podem fazer uma mudança de estado local.

É importante enfatizar que a mudança não será efetivada logo após um worker informar que pode fazê-la. O worker está apenas garantindo que será capaz de fazer essa mudança em algum instante no futuro. Como o worker pode oferecer essa garantia? Na Figura 6.3, por exemplo, o Worker A informa que pode mudar o estado da linha na tabela cliente de modo a atualizar o status desse cliente específico para VERIFICADO. O que acontecerá se uma operação diferente no futuro apagar essa linha, ou fizer outra mudança menor que, apesar disso, implique que uma mudança para VERIFICADO mais tarde seja inválida? Para garantir que a mudança para VERIFICADO possa ser feita depois, o Worker A provavelmente terá de travar o registro a fim de garantir que outras mudanças não ocorram.

Se algum worker votar dizendo que não é favorável ao commit, uma mensagem de rollback terá de ser enviada para todas as partes a fim de garantir que elas possam fazer uma limpeza local, o que permitirá que os workers liberem qualquer lock (trava) que pudessem estar mantendo. Se todos os workers concordarem em fazer a alteração, passamos para a fase de commit, conforme vemos na Figura 6.4. Nessa etapa, as mudanças são realmente efetuadas e os locks associados são liberados.

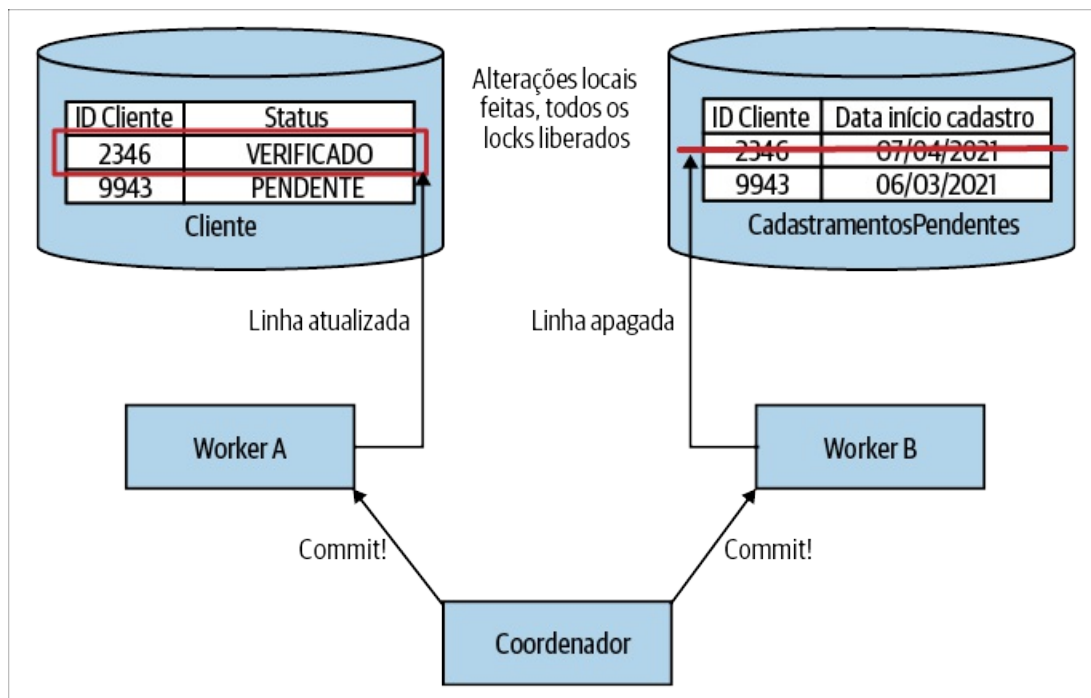


Figura 6.4 – Na fase de commit de um commit de duas fases, as mudanças são realmente aplicadas.

É importante observar que, em um sistema como esse, não podemos de modo algum garantir que esses commits ocorrerão exatamente no mesmo instante. O coordenador precisa enviar a requisição de commit a todos os participantes, e essa mensagem poderia chegar e ser processada em momentos diferentes. Isso significa que seria possível que víssemos a alteração feita no Worker A, mas ainda não no Worker B, caso pudéssemos observar diretamente o estado de qualquer um dos workers. Quanto maior a latência entre o Coordenador e os participantes do commit de duas fases, e quanto mais lentamente os workers processarem a resposta, maior será essa janela de inconsistência. Voltando à nossa definição de ACID, o *isolamento* garante que não veremos estados intermediários durante uma transação. Contudo, em um commit de duas fases, não teremos essa garantia.

Quando um commit de duas fases é utilizado, em sua essência, ele apenas irá coordenar locks distribuídos, de modo geral. Os workers precisam travar os recursos locais a fim de garantir que o commit possa ocorrer na segunda fase. Gerenciar locks e evitar deadlocks em um sistema com um

único processo não é nada divertido. Imagine agora os desafios que surgem ao coordenar locks entre vários participantes. Essa não é uma perspectiva agradável.

Há inúmeros modos de falha associados aos commits de duas fases, os quais não teremos tempo para explorar. Considere o problema de um worker votando para prosseguir com a transação, mas, em seguida, não respondendo quando solicitado a fazer o commit. O que deveríamos fazer nesse caso? Alguns desses modos de falha podem ser tratados automaticamente, mas outros podem deixar o sistema em tal estado que seria necessária uma intervenção manual de um operador.

Quanto mais participantes houver, e quanto maior a latência do sistema, mais problemas haverá em um commit de duas fases. O 2PC pode ser um modo rápido de introduzir bastante latência em seu sistema, particularmente se o escopo do locking for amplo ou a duração da transação for longa. É por esse motivo que os commits de duas fases em geral são usados somente em operações de curta duração. Quanto mais demorada a operação, maior será o período em que os recursos ficarão travados!

Transações distribuídas – simplesmente diga não

Por todos os motivos apresentados até agora, sugiro fortemente que você evite o uso de transações distribuídas, por exemplo, com o commit de duas fases, para coordenar mudanças de estado entre seus microsserviços. Então, o que mais podemos fazer?

Bem, a primeira opção poderia ser simplesmente não separar os dados, para começar. Se você tiver estados que queira administrar de forma realmente atômica e consistente, e não consegue encontrar uma solução que ofereça essas características de modo razoável sem uma transação em estilo ACID, deixe esses estados em um único banco de dados, e deixe a funcionalidade que administra esses estados em um único serviço (ou em seu sistema monolítico). Se estiver no processo de definir os pontos de separação de seu sistema monolítico, e identificar as decomposições que poderiam ser mais fáceis (ou mais difíceis), você poderia muito bem decidir

que separar os dados que atualmente são gerenciados com uma transação é uma tarefa difícil demais para ser feita nesse momento. Trabalhe em alguma outra área do sistema e volte para essa parte mais tarde.

Mas o que acontecerá se você realmente precisar separar esses dados, mas não quiser ter todas as dificuldades relacionadas ao gerenciamento de transações distribuídas? Como é possível executar operações envolvendo vários serviços, porém evitando um locking? E se a operação demorar minutos, dias ou, quem sabe, até mesmo meses? Em casos como esses, podemos considerar uma abordagem alternativa: as sagas.

Transações distribuídas em bancos de dados

Estou argumentando contra o uso geral das transações distribuídas para coordenar mudanças de estados entre microsserviços. Em situações como essas, cada microsserviço gerencia seu próprio estado local persistente (por exemplo, em seu banco de dados). Algoritmos de transações distribuídas estão sendo utilizados com sucesso em alguns bancos de dados de larga escala, e o Spanner do Google é um desses sistemas. Nessa situação, do ponto de vista de uma aplicação, a transação distribuída é aplicada de modo transparente pelo banco de dados subjacente, e é usada para coordenar as mudanças de estado somente em um único banco de dados lógico (embora seja um banco de dados que possa estar distribuído em várias máquinas e, possivelmente, em vários data centers).

O que a Google conseguiu fazer com o Spanner é impressionante, mas também vale a pena observar que o que eles tiveram de fazer para que o Spanner funcionasse dá uma ideia dos desafios envolvidos. Vamos apenas dizer que a solução envolve data centers de custo bastante elevado e relógios atômicos baseados em satélites (verdade). Para uma boa visão geral de como o Spanner faz esse trabalho, recomendo a apresentação “Google Cloud Spanner: Global Consistency at Scale” (Google Cloud Spanner: consistência global em escala).³

Sagas

De modo diferente de um commit de duas fases, uma *saga* é, por design, um algoritmo capaz de coordenar várias mudanças de estado, mas que

evita a necessidade de travar recursos por longos períodos. Uma saga faz isso modelando os passos envolvidos na forma de atividades discretas que podem ser executadas de modo independente. O uso de sagas tem como benefício adicional o fato de sermos forçados a modelar explicitamente nossos processos de negócio, o que pode proporcionar vantagens significativas.

A ideia básica, inicialmente apresentada em “Sagas”, de Hector Garcia-Molina e Kenneth Salem,⁴ aborda a melhor forma de lidar com operações conhecidas como *LLTs* (Long Lived Transactions, ou Transações de Longa Duração). Essas transações podem demorar muito (minutos, horas, ou talvez até mesmo dias), e como parte desse processo, exigem que mudanças sejam feitas em um banco de dados.

Se mapearmos diretamente uma *LLT* a uma transação usual de banco de dados, uma única transação se estenderia por todo o ciclo de vida da *LLT*. Isso poderia resultar em várias linhas ou até mesmo em tabelas inteiras sendo travadas por longos períodos enquanto a *LLT* fosse executada, causando problemas significativos caso outros processos tentassem ler ou modificar esses recursos travados.

Em vez de fazer isso, os autores do artigo sugerem que devemos dividir essas *LLTs* em uma sequência de transações, em que cada uma possa ser tratada de modo independente. A ideia é que a duração de cada uma dessas “subtransações” seja menor, e elas modifiquem somente uma parte dos dados afetados por toda a *LLT*. Como resultado, haverá muito menos contenção no banco de dados subjacente, pois o escopo e a duração dos locks serão bastante reduzidos.

Embora, originalmente, as sagas tenham sido concebidas como um método para ajudar nas *LLTs* atuando em um único banco de dados, o modelo também funciona bem para coordenar alterações envolvendo vários serviços. Podemos separar um único processo de negócios em um conjunto de chamadas que serão feitas para serviços que colaboram entre si – é isso que constitui uma saga.



Antes de prosseguir, você deve compreender que uma saga *não* nos dá

a atomicidade fornecida pelo ACID, do modo como estamos acostumados em uma transação usual de banco de dados. Como dividimos a LLT em transações individuais, não temos a atomicidade no nível da saga em questão. Porém, teremos atomicidade para cada transação individual dentro da saga como um todo, pois cada uma delas pode ser relacionada a uma mudança transacional ACID se for necessário. O que uma saga nos dá são informações suficientes para entender em que estado ela se encontra; cabe a nós lidarmos com as implicações disso.

Vamos analisar um fluxo simples de atendimento de pedido na MusicCorp, representado na Figura 6.5, que usaremos para explorar melhor as sagas no contexto de uma arquitetura de microsserviços.

Nesse caso, o processo de atendimento do pedido é representado por uma única saga, na qual cada passo do fluxo representa uma operação que pode ser executada por um serviço diferente. Em cada serviço, qualquer mudança de estado pode ser tratada com uma transação ACID local. Por exemplo, quando verificamos e fazemos uma reserva no estoque utilizando o serviço `Depósito`, internamente, esse serviço poderia criar uma linha em sua tabela local de `Reservas` registrando a reserva; essa mudança seria tratada com uma transação comum no banco de dados.

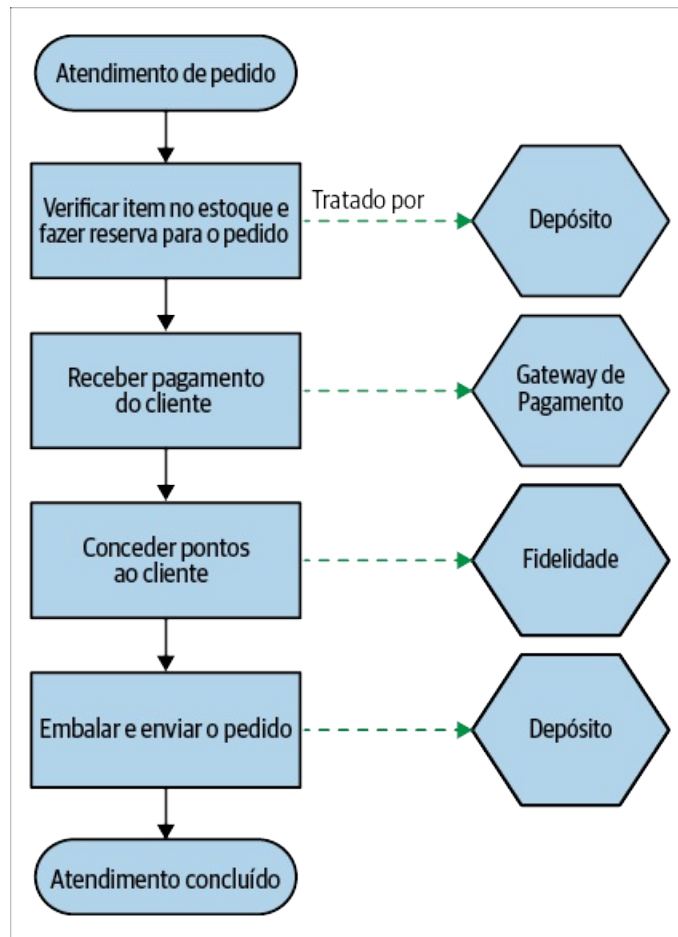


Figura 6.5 – Um exemplo de fluxo de atendimento de pedido, junto com os serviços responsáveis por executar a operação.

Modos de falha das sagas

Com uma saga dividida em transações individuais, temos de considerar o tratamento das falhas – ou, mais especificamente, da recuperação em caso de falha. O artigo original sobre sagas descreve dois tipos de recuperação: recuperação com retrocesso (backward recovery) e recuperação com avanço (forward recovery).

Uma *recuperação com retrocesso* envolve reverter a falha e fazer uma limpeza depois – é um rollback. Para que ela funcione, é necessário definir ações compensatórias que nos permitam desfazer transações cujo commit já tenha sido feito. Uma *recuperação com avanço* nos permite prosseguir do ponto em que a falha ocorreu e continuar o processamento. Para que ela funcione, deverá ser possível fazer novas tentativas para as transações, o

que, por sua vez, implica que nosso sistema deve efetuar a persistência de informações suficientes para permitir que essas novas tentativas ocorram.

Dependendo da natureza do processo de negócios sendo modelado, você pode esperar que qualquer modo de falha vá disparar uma recuperação com retrocesso, uma recuperação com avanço, ou talvez uma combinação de ambas.

É realmente importante observar que uma saga nos permite recuperar de falhas de *negócios*, e não de falhas *técnicas*. Por exemplo, se tentássemos receber um pagamento do cliente, mas esse cliente não tivesse saldo suficiente, essa seria uma falha de negócios que podemos esperar que seja tratada pela saga. Por outro lado, se houver um timeout do Gateway de Pagamento ou um 500 Internal Service Error (Erro Interno do Serviço) ocorrer, essa seria uma falha técnica, com a qual teríamos de lidar separadamente. A saga pressupõe que os componentes subjacentes estão funcionando corretamente, isto é, que o sistema subjacente é confiável, e que, portanto, estamos coordenando o trabalho de componentes confiáveis. Exploraremos algumas das maneiras de deixar nossos componentes técnicos mais confiáveis no Capítulo 12, mas, para saber mais sobre essa limitação das sagas, recomendo o artigo “The Limits of the Saga Pattern” (Os limites do padrão saga, <https://oreil.ly/II0an>), de Uwe Friedrichsen.

Rollbacks de sagas

Com uma transação ACID, se depararmos com um problema, acionamos um rollback antes que um commit ocorra. Após o rollback, é como se nada tivesse acontecido: a mudança que estávamos tentando fazer não ocorreu. Com a saga, porém, temos várias transações envolvidas, e o commit de algumas delas já pode ter ocorrido antes de termos decidido fazer o rollback de toda a operação. Então, como podemos fazer o rollback de transações depois que seu commit já tiver sido efetuado?

Vamos retomar o nosso exemplo do processamento de um pedido, conforme representado na Figura 6.5. Considere um modo de falha possível. Chegamos ao ponto de tentar embalar o item, somente para descobrir que não é possível encontrá-lo no depósito, como mostra a

Figura 6.6. Nosso sistema acha que o item existe, mas ele simplesmente não está na prateleira!

Vamos agora supor que decidimos fazer o rollback de todo o pedido, em vez de dar ao consumidor a opção de aguardar pela disponibilidade do produto. O problema é que já recebemos o pagamento e concedemos os pontos de fidelidade referentes ao pedido.

Se todos esses passos tivessem sido executados em uma única transação do banco de dados, um rollback simples faria a limpeza de tudo. No entanto, cada passo do processo de atendimento do pedido foi tratado por uma chamada de serviço distinta, cada uma atuando em um escopo de transação diferente. Não há um “rollback” simples para toda a operação.

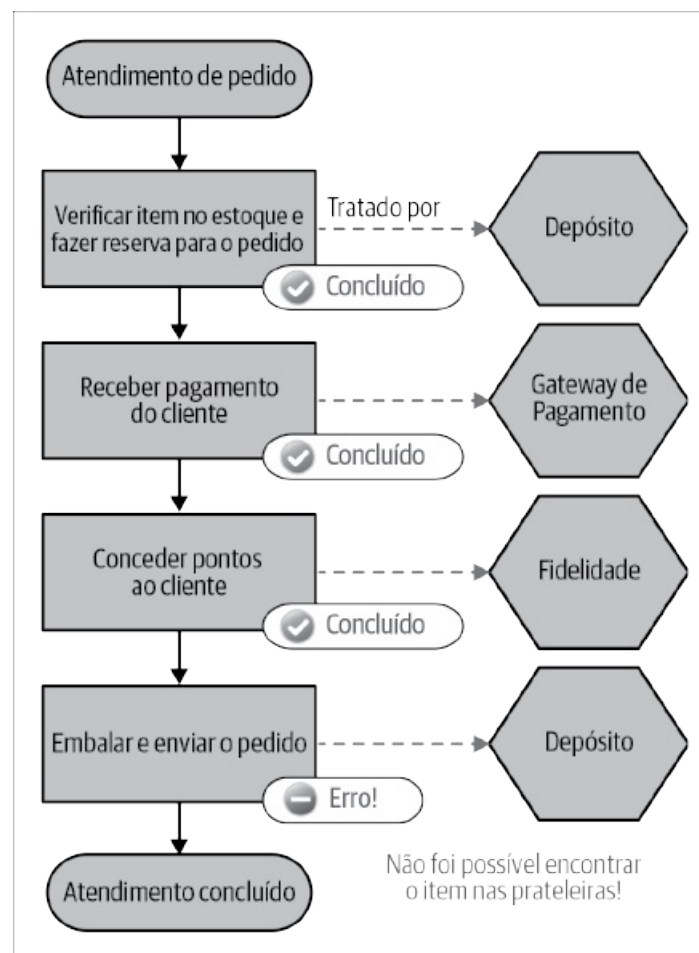


Figura 6.6 – Tentamos embalar o nosso item, mas não foi possível encontrá-lo no depósito.

Se quisermos implementar um rollback, teremos de implementar uma transação compensatória. Uma *transação compensatória* é uma operação que desfaz uma transação cujo commit já foi efetuado. Para fazer um rollback de nosso processo de atendimento de pedido, dispararíamos uma transação compensatória para cada passo de nossa saga cujo commit já tivesse sido feito, como mostra a Figura 6.7.

Vale a pena chamar a atenção para o fato de que essas transações compensatórias podem não ter exatamente o mesmo comportamento de um rollback usual no banco de dados. Um rollback no banco de dados ocorre antes do commit; após o rollback, é como se a transação jamais tivesse ocorrido. Em nosso cenário, essas transações *realmente* ocorreram, obviamente. Criamos outra transação para reverter as mudanças feitas pela transação original, mas não podemos voltar no tempo e fazer com que a transação original jamais tivesse ocorrido.

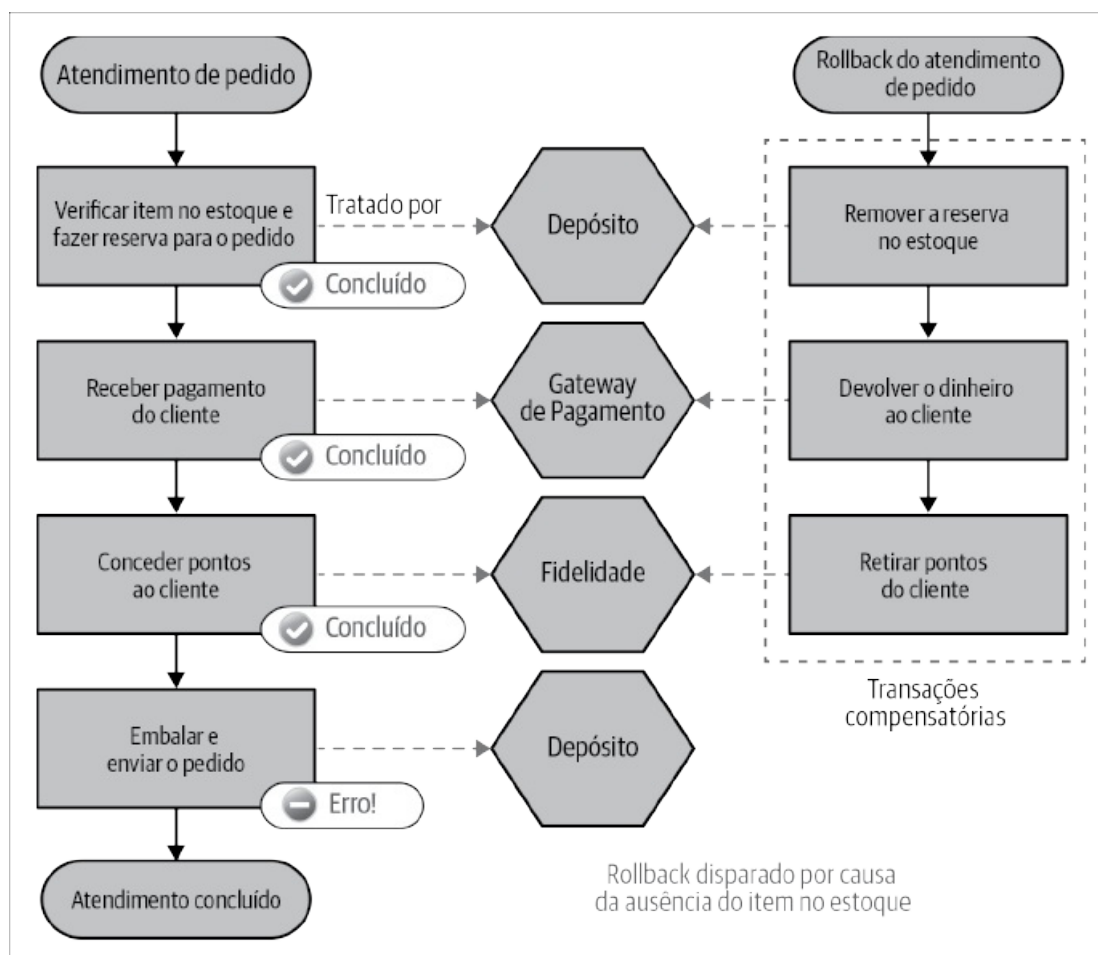


Figura 6.7 – Disparando um rollback para toda a saga.

Como nem sempre conseguiremos reverter uma transação de forma totalmente limpa, dizemos que essas transações compensatórias são *rollbacks semânticos*. Nem sempre será possível limpar tudo, mas fazemos o suficiente no contexto de nossa saga. Como exemplo, um de nossos passos pode ter envolvido o envio de um email para um cliente a fim de lhe informar que seu pedido estava a caminho. Se decidirmos fazer um rollback dessa operação, não será possível “desenviar” um email!⁵ Em vez disso, nossa transação compensatória poderia fazer com que um segundo email fosse enviado ao cliente, informando que houve um problema com o pedido, e que esse foi cancelado.

É totalmente apropriado que haja persistência de informações relacionadas ao rollback no sistema. Com efeito, essas informações podem ser muito importantes. Talvez você queira manter um registro no serviço `Pedido` para esse pedido cancelado, junto com informações sobre o que aconteceu, por uma série de motivos.

Reordenando os passos do fluxo de trabalho para reduzir os rollbacks

Na Figura 6.7, poderíamos ter deixado nossos possíveis cenários de rollback um pouco mais simples, reorganizando os passos de nosso fluxo de trabalho original. Uma mudança simples seria conceder os pontos somente quando o pedido tiver sido realmente despachado, como vemos na Figura 6.8.

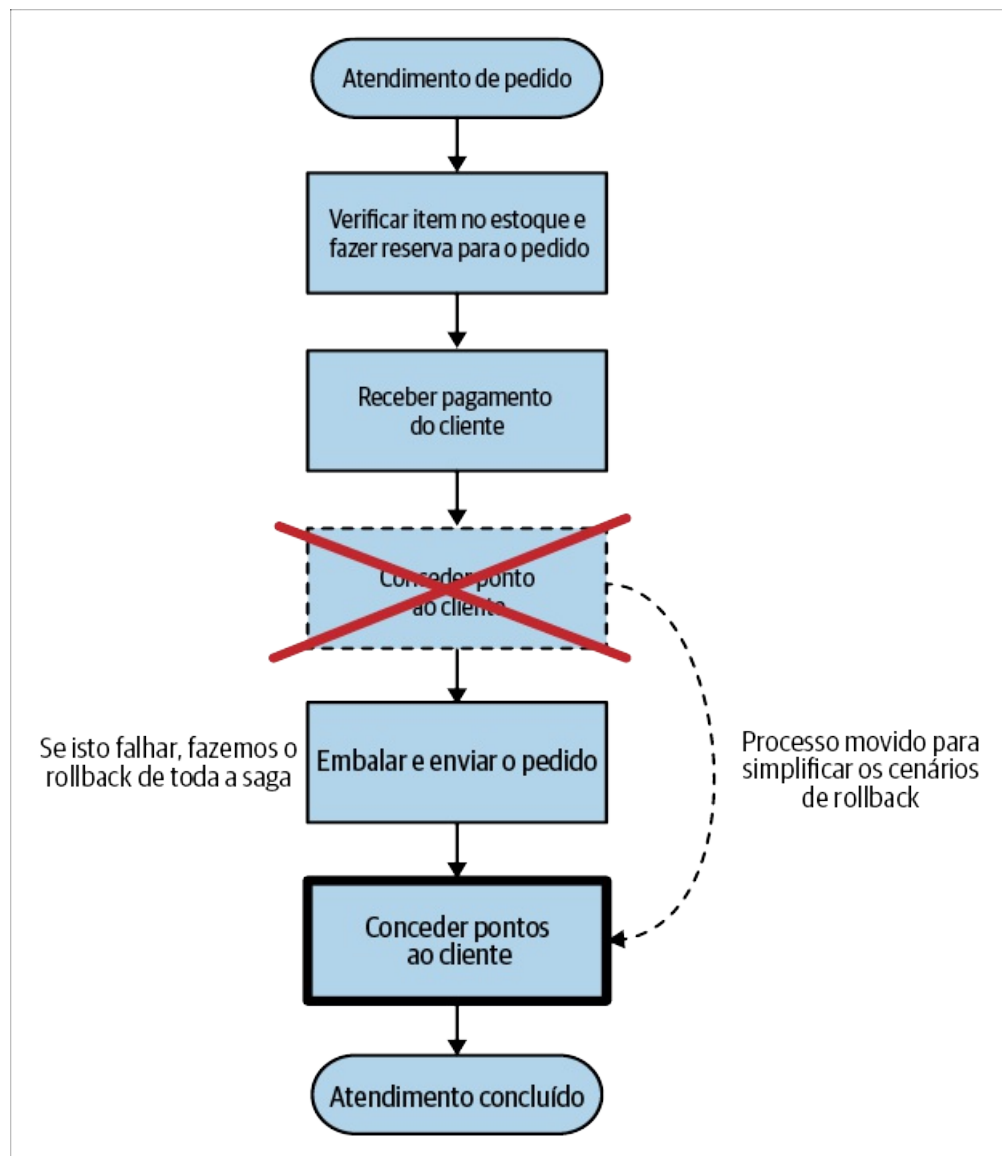


Figura 6.8 – Mover passos para uma fase posterior na saga pode reduzir as transações que precisam de rollback em caso de falha.

Dessa maneira, não precisaríamos nos preocupar com o rollback dessa etapa caso tivéssemos algum problema quando tentássemos embalar e enviar a encomenda. Às vezes, podemos simplificar as operações de rollback simplesmente ajustando o modo como o seu fluxo de trabalho se desenrola. Ao postergar os passos que tenham mais chances de falhar e provocar uma falha mais cedo no processo, você evitará ter de disparar transações compensatórias depois, pois esses passos nem teriam sido executados, para começar.

Essas mudanças, se puderem ser acomodadas, podem deixar sua vida muito mais fácil, evitando a necessidade até mesmo de criar transações compensatórias para alguns passos. Isso pode ser particularmente importante se a implementação de uma transação compensatória for difícil. Você poderá mover um passo para uma fase posterior do processo, para uma etapa na qual ele jamais vá precisar de um rollback.

Combinando situações de falhas com retrocesso e situações de falhas com avanço

É totalmente apropriado ter uma combinação de modos de recuperação de falhas. Algumas falhas podem exigir um rollback (falha com retrocesso); em outras, pode haver falhas com avanço. No processamento de pedidos, por exemplo, assim que tivermos recebido o pagamento do cliente e o item estiver embalado, o único passo que resta será despachar o pacote. Se, por algum motivo, não for possível despachar o pacote (talvez a empresa de entrega que utilizamos não tenha espaço em seus furgões para retirar o pedido hoje), seria muito estranho fazer um rollback de todo o pedido. Em vez disso, é provável que simplesmente tentássemos fazer o envio outra vez (talvez colocando-o na fila para o dia seguinte) e, se isso falhasse, exigiríamos uma intervenção humana para resolver a situação.

Implementando sagas

Até agora, vimos o modelo lógico para o funcionamento das sagas, mas precisamos nos aprofundar um pouco mais para analisar as maneiras de implementá-las. Podemos discutir dois estilos de implementação de sagas. As *sagas orquestradas* seguem mais de perto a área da solução original e dependem principalmente de uma coordenação e de uma monitoração centralizadas. Elas podem ser comparadas com as *sagas coreografadas*, que evitam a necessidade de uma coordenação centralizada em favor de um modelo com menos acoplamento, mas que pode deixar a monitoração do progresso de uma saga mais complicada.

Sagas orquestradas

As sagas orquestradas utilizam um coordenador centralizado (que chamaremos de *orquestrador* a partir de agora) para definir a ordem de

execução e disparar qualquer ação compensatória necessária. Podemos pensar nas sagas orquestradas como uma abordagem do tipo comando e controle (command-and-control): o orquestrador central controla o que acontece e quando acontece; com isso, há um bom grau de visibilidade para o que acontece com qualquer saga em particular.

Considerando o processo de atendimento de pedido exibido na Figura 6.5, vamos ver como esse processo de coordenação centralizada funcionaria como um conjunto de serviços que colaboram entre si, como mostra a Figura 6.9.

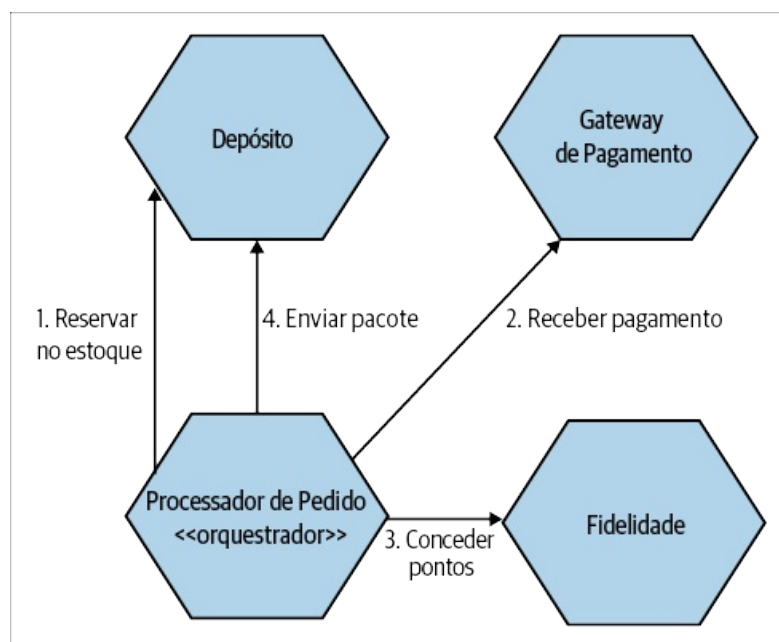


Figura 6.9 – Um exemplo de como uma saga orquestrada pode ser usada para implementar nosso processo de atendimento de pedido.

Nesse exemplo, nosso **Processador de Pedido** central, desempenhando a função de orquestrador, coordena o processo de atendimento de pedido. Ele sabe quais serviços são necessários para que a operação seja executada e decide quando deve fazer as chamadas para esses serviços. Se as chamadas falharem, ele pode decidir o que será feito como resultado. Em geral, as sagas orquestradas tendem a fazer uso intenso de interações com requisição-resposta entre os serviços: **Processador de Pedido** envia uma requisição aos serviços (por exemplo, para um **Gateway de Pagamento**), e espera uma resposta que lhe permita saber se a requisição foi bem-sucedida e que

forneça o resultado da requisição.

Ter um processo de negócios explicitamente modelado no `Processador de Pedido` é extremamente vantajoso. Isso nos permite olhar para um só local em nosso sistema e compreender como esse processo deve funcionar. Desse modo, a inclusão de novas pessoas no projeto se torna mais fácil, e mais pessoas poderão compreender melhor as partes essenciais do sistema.

Contudo, há algumas desvantagens que devem ser consideradas. Em primeiro lugar, em virtude de sua natureza, essa é uma abordagem, de certo modo, acoplada. Nosso `Processador de Pedido` precisa conhecer todos os serviços associados, resultando em mais acoplamento de domínio. Embora um acoplamento de domínio não seja inerentemente ruim, ainda gostaríamos de mantê-lo a um nível mínimo, se for possível. Nesse exemplo, nosso `Processador de Pedido` deve ter conhecimento e controlar várias operações, a ponto de essa forma de acoplamento ser difícil de ser evitada.

Outro problema, mais sutil, é que a lógica que deveria estar nos serviços pode começar a ser absorvida pelo orquestrador. Se isso começar a acontecer, seus serviços poderão se tornar anêmicos, com pouco comportamento próprio, somente recebendo ordens de orquestradores como o `Processador de Pedido`. É importante que você continue considerando os serviços que compõem esses fluxos orquestrados como entidades que tenham seu próprio estado local e seu próprio comportamento. Eles são responsáveis por suas próprias máquinas de estado locais.



Se houver um local em que uma lógica possa ser centralizada, ela se tornará centralizada!

Uma das maneiras de evitar que haja muita centralização nos fluxos orquestrados é garantir que haja diferentes serviços desempenhando o papel de orquestrador para diferentes fluxos. Você poderia ter um microsserviço `Processador de Pedido` que cuide de aceitar um pedido, um microsserviço `Devoluções` para lidar com o processo de devolução e reembolso, um microsserviço `Recepção de Mercadorias` que cuide da chegada de novos itens no estoque e de sua colocação nas prateleiras, e assim por

diante. Algo como o nosso microserviço `Depósito` poderia ser usado por todos esses orquestradores; um modelo como esse facilitaria manter a funcionalidade dentro do próprio microserviço `Depósito`, permitindo a reutilização da funcionalidade em todos esses fluxos.

Ferramentas BPM

Ferramentas BPM (Business Process Modeling, ou Modelagem de Processos de Negócio) estão disponíveis há muitos anos. De modo geral, elas foram criadas para permitir que profissionais que não são desenvolvedores definam fluxos de processos de negócios, muitas vezes usando ferramentas visuais com recursos de arrastar e soltar. A ideia é que os desenvolvedores criariam os blocos de construção desses processos e, em seguida, aqueles que não são desenvolvedores ligariam esses blocos de construção, criando os fluxos de processo maiores. O uso de ferramentas como essa parece estar muito bem alinhado com a forma de implementar sagas orquestradas e, com efeito, a orquestração de processos é, de modo geral, o principal caso de uso das ferramentas BPM (ou, do contrário, o uso de ferramentas BPM resulta na necessidade de adotar uma orquestração).

Em minha experiência, passei a desgostar bastante das ferramentas BPM. O principal motivo é que o conceito básico – o fato de que os profissionais que não são desenvolvedores definirão o processo de negócios – de acordo com a minha experiência, quase nunca é verdadeiro. As ferramentas voltadas para quem não é desenvolvedor acabam sendo usadas pelos desenvolvedores e, infelizmente, essas ferramentas muitas vezes funcionam de maneiras que são estranhas ao modo como os desenvolvedores gostam de trabalhar. Com frequência, elas exigem o uso de GUIs para alterar os fluxos, os fluxos criados podem ser difíceis (ou impossíveis) de gerenciar com um sistema de controle de versões, os próprios fluxos podem não ser criados com testes em mente, entre outros problemas.

Se são os desenvolvedores que irão implementar seus processos de negócio, deixe que eles utilizem ferramentas que conheçam e compreendam, e que sejam apropriadas aos seus fluxos de trabalho. Em geral, isso significa simplesmente deixar que utilizem código para implementá-los! Se for necessário ter visibilidade sobre o modo como

um processo de negócios foi implementado, ou de que modo ele funciona, será muito mais fácil criar uma representação visual de um fluxo de trabalho a partir do código, em vez de utilizar uma representação visual de seu fluxo de trabalho para descrever como o seu código deve funcionar.

Há esforços para criar ferramentas BPM mais apropriadas aos desenvolvedores. Feedbacks dos desenvolvedores a respeito dessas ferramentas parecem ser variados, mas elas têm funcionado bem para alguns, e é bom ver que há pessoas tentando aperfeiçoar esses frameworks. Se achar que precisa explorar melhor essas ferramentas, consulte o Camunda (<https://camunda.com>) e o Zeebe (<https://github.com/camunda-cloud/zeebe>) – ambos são frameworks de orquestração open source, voltados aos desenvolvedores de microsserviços, e estariam no topo da minha lista se eu decidisse que uma ferramenta BPM seria apropriada para mim.

Sagas coreografadas

Uma saga coreografada tem como objetivo distribuir a responsabilidade pela operação da saga entre vários serviços que colaboram entre si. Se a orquestração é uma abordagem do tipo comando e controle, as sagas coreografadas representam uma arquitetura do tipo confiar, mas verificar (trust-but-verify). Conforme veremos em nosso exemplo na Figura 6.10, as sagas coreografadas em geral fazem uso intensivo de eventos para colaboração entre os serviços.

Há muitas tarefas acontecendo nesse caso, portanto vale a pena analisá-las com mais detalhes. Em primeiro lugar, esses microsserviços reagem a eventos recebidos. Conceitualmente, um broadcast dos eventos é feito no sistema, e as partes interessadas podem recebê-los. Lembre-se de que, conforme discutimos no Capítulo 4, não enviamos eventos *para* um microsserviço; apenas os geramos, e os microsserviços interessados nesses eventos poderão recebê-los e agir de acordo com eles. Em nosso exemplo, quando o serviço `Depósito` recebe o primeiro evento de `Pedido Feito`, ele sabe que seu trabalho é fazer uma reserva apropriada no estoque e gerar um evento assim que isso for feito. Se o item não puder ser reservado no

estoque, Depósito deverá gerar um evento apropriado (um evento de Estoque Insuficiente, talvez), que poderia resultar no cancelamento do pedido.

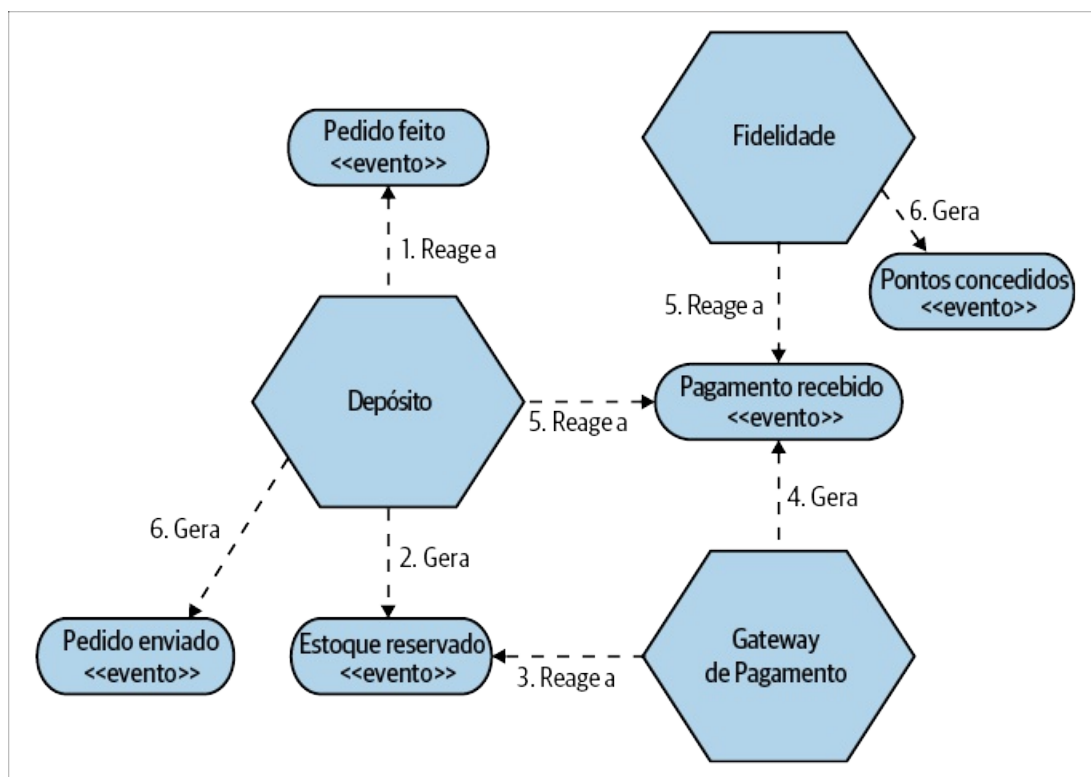


Figura 6.10 – Um exemplo de uma saga coreografada para implementar o atendimento de um pedido.

Nesse exemplo, também podemos ver como os eventos podem facilitar um processamento em paralelo. Quando o evento **Pagamento Recebido** é gerado por **Gateway de Pagamento**, ele faz com que tanto o microsserviço **Fidelidade** como o microsserviço **Depósito** reajam. **Depósito** reage despachando o pacote, enquanto o microsserviço **Fidelidade** reage concedendo pontos ao cliente.

Em geral, usaríamos algum tipo de broker de mensagens para gerenciar o broadcast e a entrega de eventos de forma confiável. É possível que vários microsserviços possam reagir ao mesmo evento, e nesse caso poderíamos usar um tópico. As partes interessadas em um certo tipo de evento poderiam se inscrever para um tópico específico, sem ter de se preocupar com a origem desses eventos, e o broker garantirá a durabilidade do tópico e se certificará de que os eventos relacionados a esse tópico sejam entregues com sucesso aos inscritos. Como exemplo, podemos ter um serviço de

Recomendações que também observe os eventos de `Pedido Feito`, utilizando-os para construir um banco de dados de opções musicais que possam ser do gosto do cliente.

Na arquitetura anterior, nenhum serviço sabe da existência dos demais microsserviços. Eles precisam apenas saber o que devem fazer quando um determinado evento é recebido – assim, reduzimos drasticamente o volume de acoplamento de domínio. Isso faz com que haja, inerentemente, uma arquitetura muito menos acoplada. À medida que a implementação do processo é decomposta e distribuída entre os três microsserviços nesse exemplo, também evitamos as preocupações com a centralização da lógica (se não houver um local em que a lógica possa estar centralizada, ela não será centralizada!).

O outro lado da moeda é que agora poderá ser mais difícil descobrir o que está acontecendo. Com a orquestração, nosso processo estava explicitamente modelado em nosso orquestrador. Agora, com essa arquitetura conforme foi apresentada, como você construiria um modelo mental para aquilo que o processo deveria fazer? Seria necessário observar o comportamento de cada serviço isoladamente e reconstituir essa imagem em sua própria mente – está longe de ser uma tarefa fácil, mesmo com um processo de negócios simples como esse.

A falta de uma representação explícita de nosso processo de negócios já é bastante ruim e, além disso, não temos um modo de saber em que estado está uma saga; com isso, não teremos a oportunidade de associar ações compensatórias quando forem necessárias. Podemos passar parte da responsabilidade de executar as ações compensatórias aos serviços individuais, mas, basicamente, precisamos de uma maneira de saber em que estado está uma saga para alguns tipos de recuperação. A falta de um local centralizado para interrogar o status de uma saga é um grande problema. Temos isso com a orquestração; então, como resolveremos o problema nesse caso?

Um dos modos mais fáceis de resolver a questão é criar uma projeção do estado de uma saga, consumindo os eventos que são gerados. Se gerarmos um ID único para a saga, que é conhecido como *ID de correlação*, será

possível inseri-lo em todos os eventos gerados como parte dessa saga. Quando um de nossos serviços reagir a um evento, o ID de correlação será extraído e utilizado por qualquer processo local de logging, além de ser passado para a frente, em qualquer chamada feita posteriormente ou qualquer evento disparado. Poderíamos, então, ter um serviço cuja tarefa seria simplesmente consumir todos esses eventos e apresentar uma visão do estado em que está cada pedido e, talvez, por meio de programação, executar ações para resolver os problemas como parte do processo de atendimento de um pedido, caso os outros serviços não consigam fazê-lo por conta própria. Eu considero que alguma forma de ID de correlação seja essencial em sagas coreografadas como essa, mas os IDs de correlação também são muito importantes de modo geral – esse assunto será explorado com mais detalhes no Capítulo 10.

Combinando estilos

Embora possa parecer que as sagas orquestradas e as sagas coreografadas sejam visões diametralmente opostas de como as sagas poderiam ser implementadas, podemos facilmente considerar uma mistura e uma combinação dos modelos. Talvez possamos ter alguns processos de negócios em nosso sistema que sejam mais naturalmente adequados a um ou a outro modelo. Podemos ter também uma única saga que tenha uma combinação de estilos. No caso de uso de atendimento de pedidos, por exemplo, dentro das fronteiras do serviço `Depósito`, quando gerenciamos a embalagem e o envio de um pedido, podemos empregar um fluxo orquestrado, ainda que a requisição original tenha sido feita como parte de uma saga coreografada maior.⁶

Se optar pela combinação de estilos, é importante que continue havendo um modo claro de saber o estado em que está uma saga, e quais atividades já ocorreram como parte dessa saga. Sem isso, entender os modos de falha será complicado, e será difícil se recuperar dessas falhas.

Rastreando chamadas

Não importa se você opte pela coreografia ou pela coordenação, ao implementar um processo de negócios utilizando vários microsserviços, é comum querer rastrear todas as chamadas relacionadas ao processo.

Às vezes, isso poderia servir apenas para ajudar a entender se o processo de negócios está funcionando corretamente, ou para ajudar a diagnosticar um problema. No Capítulo 10, veremos conceitos como IDs de correlação e agregação de logs, e como eles podem ajudar nesse cenário.

Devo usar coreografia ou orquestração (ou uma combinação de ambos)?

A implementação de sagas coreografadas pode vir acompanhada de ideias com as quais você e sua equipe talvez não tenham familiaridade. Em geral, elas pressupõem um uso intensivo de colaboração orientada a eventos, que não é um assunto compreendido por qualquer pessoa. No entanto, com base em minha experiência, posso dizer que a complexidade adicional relacionada à monitoração do progresso de uma saga quase sempre será superada pelas vantagens associadas em ter uma arquitetura com menos acoplamento.

Deixando de lado minhas preferências pessoais, porém, o conselho geral que dou em relação à orquestração *versus* a coreografia é que eu me sinto muito confortável com o uso de sagas orquestradas quando uma equipe é responsável pela implementação de toda a saga. Em uma situação como essa, a arquitetura inerentemente mais acoplada será muito mais fácil de ser administrada dentro das fronteiras da equipe. Se houver várias equipes envolvidas, dou preferência à saga coreografada, mais decomposta, pois será melhor para distribuir as responsabilidades pela implementação da saga às equipes, com a arquitetura menos acoplada permitindo que essas equipes trabalhem mais isoladamente.

Vale a pena observar que, como regra geral, é mais provável que você seja atraído em direção às chamadas do tipo requisição-resposta no caso da orquestração, enquanto a coreografia tende a fazer um uso mais intensivo de eventos. Essa não é uma regra imutável, mas somente uma observação genérica. Minha própria inclinação geral à coreografia provavelmente é função do fato de eu ter uma tendência a ser atraído para os modelos de interação orientados a eventos – se você acha difícil adotar uma mentalidade apropriada ao uso de uma colaboração orientada a eventos, talvez a coreografia não seja uma solução adequada para você.

Sagas versus transações distribuídas

Como espero ter esclarecido a essa altura, as transações distribuídas são acompanhadas de alguns desafios significativos, e afora algumas situações muito específicas, minha tendência é evitá-las. Pat Helland, um pioneiro em sistemas distribuídos, sintetiza os principais desafios associados à implementação de transações distribuídas para os tipos de aplicações criadas atualmente:⁷

Na maioria dos sistemas de transação distribuída, a falha em um único nó faz com que o commit da transação seja interrompido. Por sua vez, isso faz com que a aplicação trave. Em sistemas como esses, quanto mais isso acontecer, maior será a probabilidade de um sistema ficar inativo. Ao pilotar um avião que exija que todos os seus equipamentos funcionem, o acréscimo de um equipamento reduzirá a disponibilidade do avião.

Com base em minha experiência, posso dizer que modelar explicitamente os processos de negócio como uma saga evita muitas das dificuldades relacionadas às transações distribuídas, ao mesmo tempo que tem a vantagem adicional de deixar os processos que, de outro modo, seriam implicitamente modelados, muito mais explícitos e evidentes para os desenvolvedores. Fazer com que os processos de negócio essenciais de seu sistema sejam um conceito de primeira classe proporcionará uma série de vantagens.

Resumo

Então, como podemos ver, o caminho para implementar fluxos de trabalho em nossa arquitetura de microsserviços se reduz a modelar explicitamente os processos de negócios que estivermos tentando implementar. Isso nos leva de volta à ideia de modelar aspectos de nosso domínio de negócios em nossa arquitetura de microsserviços – modelar explicitamente os processos de negócios faz sentido se as fronteiras de nossos microsserviços também forem definidas essencialmente com base no nosso domínio de negócios.

Independentemente de decidir caminhar mais em direção à orquestração

ou em direção à coreografia, espero que você esteja em uma posição muito melhor para saber qual modelo será mais adequado ao domínio de seu problema.

Se quiser explorar esse assunto com mais detalhes, embora as sagas não sejam *explicitamente* abordadas, o livro *Enterprise Integration Patterns*, de Gregor Hohpe e Bobby Woolf, apresenta uma série de padrões que podem ser extremamente úteis ao implementar diferentes tipos de fluxos de trabalho.⁸ Também posso recomendar de coração o livro *Practical Process Automation*, de Bernd Ruecker.⁹ O livro de Bernd tem um foco muito maior no lado da orquestração das sagas, mas está repleto de informações úteis que fazem dele um complemento natural para esse assunto.

Agora temos uma ideia de como nossos microsserviços podem se comunicar e como podem estar coordenados entre si, mas como podemos criá-los, antes de tudo? Em nosso próximo capítulo, veremos como aplicar conceitos como controle de versões de códigos-fontes (source control), integração contínua (continuous integration) e entrega contínua (continuous delivery) no contexto de uma arquitetura de microsserviços.

¹ Isso mudou, com o lançamento do suporte para transações ACID em vários documentos como parte do Mongo 4.0. Pessoalmente, ainda não utilizei esse recurso do Mongo; só sei que ele existe!

² Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol: O'Reilly, 2017).

³ Robert Kubis, “Google Cloud Spanner: Global Consistency at Scale” (Google Cloud Spanner: consistência global em escala), Devvxx, 7 de novembro de 2017, vídeo do YouTube, 33:22, <https://oreil.ly/XHvY5>.

⁴ Hector Garcia-Molina e Kenneth Salem, “Sagas”, *ACM Sigmod Record* 16, nº 3 (1987): 249–59.

⁵ Realmente, não é possível. Eu tentei!

⁶ Está fora do escopo deste livro, mas Hector Garcia-Molina e Kenneth Salem exploraram como várias sagas poderiam ser “aninhadas” para implementar processos mais complexos. Para ler mais sobre esse assunto, veja Hector Garcia-Molina et al, “Modeling Long-Running Activities as Nested Sagas” (Modelando atividades de longa duração como sagas aninhadas), *Data Engineering* 14, nº 1 (março de 1991: 14–18).

⁷ Veja Pat Helland, “Life Beyond Distributed Transactions: An Apostate’s Opinion” (Vida além das transações distribuídas: opinião de um apóstata), *acmqueue* 14, nº 5 (12 de dezembro de 2016).

⁸ Gregor Hohpe e Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

⁹ Bernd Ruecker, *Practical Process Automation* (Sebastopol: O'Reilly, 2021).

Construção

Dedicamos bastante tempo discutindo os aspectos relativos ao design dos microsserviços, mas precisamos começar a nos aprofundar mais no modo como o seu processo de desenvolvimento talvez precise mudar para acomodar esse novo estilo de arquitetura. Nos próximos capítulos, veremos como implantar e testar nossos microsserviços; antes disso, porém, precisamos discutir o que vem antes – o que acontece quando um desenvolvedor está pronto para fazer o check-in de uma mudança?

Começaremos essa exploração analisando alguns conceitos básicos: integração contínua (continuous integration) e entrega contínua (continuous delivery). São conceitos importantes, não importa o tipo de arquitetura de sistema que você possa utilizar, mas os microsserviços dão margem a uma série de questões que lhes são exclusivas. Em seguida, veremos os pipelines e as diferentes maneiras de gerenciar os códigos-fontes de seus serviços.

Uma rápida introdução à integração contínua

A *integração contínua* (CI) existe há anos. No entanto, vale a pena dedicar um pouco de tempo para rever o básico, pois há algumas opções diferentes a serem consideradas, sobretudo quando pensamos no mapeamento entre microsserviços, construções (builds) e repositórios para controle de versões.

Com a CI, o principal objetivo é manter todos em sincronia, o que é feito garantindo frequentemente que um código cujo check-in tenha sido feito recentemente se integre de modo apropriado ao código existente. Para isso, um servidor de CI detecta que houve commit de um código, faz seu check-out e executa alguma espécie de verificação, por exemplo, garante que o

código compila e que os testes sejam bem-sucedidos. No mínimo, esperamos que essa integração seja feita diariamente, embora, na prática, já trabalhei com muitas equipes em que um desenvolvedor tinha suas alterações integradas várias vezes ao dia.

Como parte desse processo, muitas vezes criamos artefatos que são usados para uma validação posterior, por exemplo, implantamos um serviço para que possamos testá-lo (exploraremos os testes com detalhes no Capítulo 9). O ideal é que esses artefatos sejam criados uma e somente uma vez, e sejam utilizados em todas as implantações dessa versão de código. Isso serve para evitar a execução da mesma tarefa repetidamente, e para poder confirmar que os artefatos que implantamos são iguais aos que testamos. Para permitir que esses artefatos sejam reutilizados, nós os colocamos em algum tipo de repositório, que pode ser fornecido pela própria ferramenta de CI ou por outro sistema.

Veremos a função dos artefatos com mais detalhes em breve, e discutiremos os testes minuciosamente no Capítulo 9.

Há várias vantagens na CI. Temos um feedback rápido quanto à qualidade do nosso código, por meio de análises estáticas e testes. A CI também nos permite automatizar a criação de nossos artefatos binários. Todo o código necessário para construir o artefato está sujeito a um controle de versões, portanto será possível recriar o artefato caso seja necessário. Também podemos fazer um rastreamento de um artefato implantado de volta para o código e, dependendo dos recursos da própria ferramenta de CI, podemos ver quais testes foram executados no código e igualmente no artefato. Se adotarmos uma infraestrutura como código (*infrastructure as code*), também podemos submeter todo o código necessário para configurar a infraestrutura do nosso microsserviço, junto com o código do próprio microsserviço, a um sistema de controle de versões, deixando as alterações mais transparentes e facilitando ainda mais a reprodução dos processos de construção (*builds*). É por esses motivos que a CI faz tanto sucesso.

Você realmente coloca a CI em prática?

A CI é uma prática essencial que nos permite fazer mudanças de forma

rápida e fácil, sem a qual a jornada para os microsserviços seria complicada. Suspeito que você provavelmente utilize uma ferramenta de CI em sua empresa, mas isso pode não ser exatamente o mesmo que colocar a CI em prática. Já vi muitas pessoas confundirem a adoção de uma ferramenta de CI com realmente adotar a CI. Uma ferramenta de CI devidamente utilizada ajudará você a colocar a CI em prática – no entanto, usar uma ferramenta como Jenkins, CircleCI, Travis ou uma das várias outras opções que existem por aí não garante que você esteja realmente colocando a CI em prática da forma correta.

Então, como saber se você está realmente colocando a CI em prática? Gosto muito das três perguntas que Jez Humble faz às pessoas para testar se elas realmente entendem do que se trata a CI – talvez seja interessante você fazer essas mesmas perguntas a si mesmo:

Você faz check-in para a linha principal (mainline) uma vez ao dia?

Você precisa garantir que seu código será integrado. Se não verificar seu código em conjunto com as alterações feitas por todas as outras pessoas com frequência, você acabará dificultando futuras integrações. Ainda que você utilize branches de curta duração para gerenciar as mudanças, faça integrações com um único branch principal com a maior frequência possível – pelo menos, uma vez ao dia.

Você tem um conjunto de testes para validar suas alterações?

Sem testes, saberemos apenas que, do ponto de vista sintático, nossa integração foi bem-sucedida, mas não saberemos se causamos falhas no comportamento do sistema. Uma CI sem algum tipo de verificação para saber se nosso código se comporta conforme esperado não é CI.

Quando há problemas com a build, corrigi-la é a prioridade número um da equipe?

Uma build com sinal verde significa que nossas mudanças foram integradas com segurança. Uma build com sinal vermelho significa que nossa última alteração provavelmente não foi integrada com sucesso. É preciso interromper todos os check-ins subsequentes que não estejam envolvidos na correção da build para que ela possa ter um sinal verde novamente. Se você permitir que outras alterações sejam adicionadas, o

tempo necessário para corrigir a build aumentará drasticamente. Já trabalhei com equipes em que a build havia ficado com problemas por dias, resultando em esforços significativos para ter finalmente uma build bem-sucedida.

Modelos de branching

Poucos tópicos relacionados à construção e implantação parecem causar tanta controvérsia quanto a questão de utilizar branching de código-fonte para o desenvolvimento de funcionalidades. Um branching no código-fonte permite que o desenvolvimento seja feito isoladamente, sem atrapalhar o trabalho de outras pessoas. Analisando superficialmente, criar um branch de código-fonte para trabalhar com cada funcionalidade – também conhecido como branching por funcionalidade (feature branching) – parece ser um conceito útil.

O problema é que, quando trabalhamos em um branch de funcionalidade, não integramos nossas alterações com o código de outras pessoas com regularidade. Basicamente, *postergamos* a integração. Ao decidir finalmente integrar suas alterações com todos os demais, você terá um merge muito mais complexo.

A abordagem alternativa é fazer com que todos façam check-in no mesmo “tronco” (trunk) de código-fonte. Para evitar que as mudanças causem impacto em outras pessoas, técnicas como flags de funcionalidades (feature flags) são empregadas para “ocultar” trabalhos incompletos. Essa técnica de todos trabalharem com base no mesmo tronco é chamada de *desenvolvimento baseado em tronco*.

A discussão em torno desse tópico apresenta várias nuances, mas minha opinião é que as vantagens de uma integração frequente – e a validação dessa integração – são suficientemente significativas, a ponto de um desenvolvimento baseado em tronco ser o meu estilo favorito de desenvolvimento. Além do mais, o trabalho para implementar flags de funcionalidades geralmente é vantajoso para uma entrega progressiva (progressive delivery) – um conceito que exploraremos no Capítulo 8.



Tome cuidado com os branches

Integre logo, e integre com frequência. Evite utilizar branches de longa duração para o desenvolvimento de funcionalidades, e considere um desenvolvimento baseado em tronco em seu lugar. Se realmente tiver de usar branches, faça com que tenham uma curta duração!

Muito além de minha própria experiência pessoal, há um volume crescente de pesquisas que mostra a eficácia de reduzir o número de branches e adotar um desenvolvimento baseado em tronco. O relatório State of DevOps (Estado do DevOps) de 2016, de DORA e Puppet¹, conduziu uma pesquisa rigorosa sobre as práticas de entrega das empresas pelo mundo e analisou as práticas comumente utilizadas por equipes com alto desempenho:

Constatamos que ter branches ou forks com tempo de vida muito curto (menos de um dia) antes de fazer um merge com o tronco, e menos de três branches ativos no total, são aspectos importantes da entrega contínua, e tudo isso contribui para um melhor desempenho. O mesmo vale para merging do código no tronco ou na linha principal efetuado diariamente.

O relatório State of DevOps prosseguiu explorando esse assunto com mais detalhes nos anos seguintes, e continuou encontrando evidências da eficácia dessa abordagem.

Uma abordagem com uso intensivo de branches ainda é comum no desenvolvimento de código aberto (open source), geralmente com a adoção do modelo de desenvolvimento “GitFlow”. Vale a pena observar que um desenvolvimento de código aberto não é igual a um desenvolvimento habitual, feito no dia a dia. Um desenvolvimento de código aberto é caracterizado por um número elevado de colaborações *ad hoc*, cujo commit é feito por pessoas “não confiáveis”, que dispõem de pouco tempo, e essas alterações exigem uma avaliação de um número pequeno de colaboradores “confiáveis”. Um desenvolvimento de código fechado (closed source) típico, conduzido no dia a dia, geralmente é feito por uma equipe bastante integrada, na qual todos os membros têm direitos de commit, ainda que decidam adotar alguma forma de processo de revisão de código. Assim, o

que poderia funcionar para um desenvolvimento de código aberto talvez não funcione em seu trabalho cotidiano. Apesar disso, o relatório State of DevOps de 2019,² explorando melhor esse assunto, revelou alguns insights interessantes sobre o desenvolvimento de código aberto e o impacto dos branches de “longa duração”:

As descobertas de nossa pesquisa se estendem ao desenvolvimento de código aberto em algumas áreas:

- fazer commit do código mais cedo é melhor: em projetos de código aberto, muitas pessoas observaram que fazer merge de patches mais rapidamente para evitar rebases ajuda os desenvolvedores a serem mais ágeis.
- trabalhar com conjuntos menores é melhor: fazer merge de “bombas de patches” enormes em um projeto é mais difícil e mais demorado do que fazer o merge de conjuntos menores e mais legíveis de patches, pois os mantenedores precisarão de mais tempo para revisar as alterações.

Independentemente de estar trabalhando com uma base de código fechada ou com um projeto de código aberto, branches de curta duração, patches pequenos e legíveis e testes automáticos das alterações deixam todos mais produtivos.

Pipelines de construção e entrega contínua

Logo que começamos a colocar a CI em prática, meus colegas na Thoughtworks e eu percebemos a importância de ocasionalmente ter várias etapas em um processo de construção (build). Os testes são um caso muito comum no qual essa abordagem entra em cena. Eu poderia ter vários testes rápidos com um escopo menor, e alguns testes demorados com escopo maior. Se todos os testes fossem executados em conjunto, e tivéssemos de esperar pelo término de nossos testes demorados, com um escopo maior, não teríamos um feedback rápido quando nossos testes falhassem. Além disso, se os testes rápidos falharem, provavelmente não faria muito sentido executar os testes mais demorados, de qualquer maneira! Uma solução para esse problema está em ter diferentes etapas em nosso processo de

construção, criando o que é conhecido como *pipeline de construção* (build pipeline). Assim, podemos ter uma etapa dedicada incluindo todos os testes rápidos, a qual seria executada antes; se todos os testes passarem, executamos então uma etapa diferente que inclua os testes mais demorados.

O conceito de pipeline de construção oferece uma boa forma de monitorar o progresso do nosso software à medida que ele passar por cada etapa com sucesso, ajudando-nos a ter insights sobre a qualidade desse software. Criamos um artefato que pode ser implantado, isto é, aquilo que, em última instância, será implantado no ambiente de produção, e utilizamos esse artefato ao longo do pipeline. Em nosso contexto, esse artefato está relacionado a um microsserviço que queremos implantar. Na Figura 7.1, vemos esse processo acontecer – o mesmo artefato é utilizado em cada etapa do pipeline, aumentando nosso nível de confiança de que o software irá funcionar no ambiente de produção.

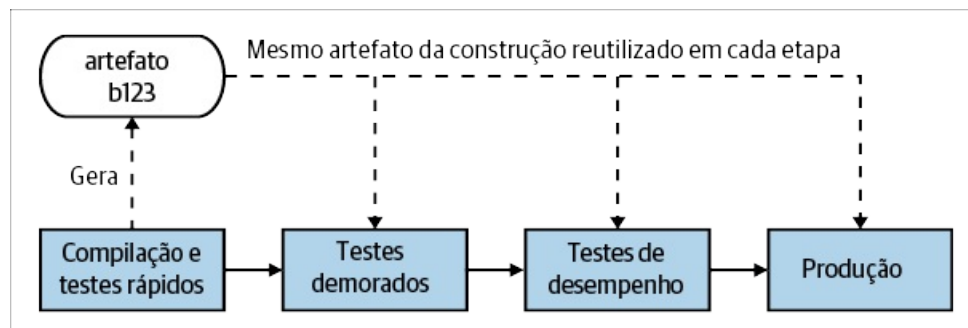


Figura 7.1 – Um processo simples para o lançamento de uma versão do nosso serviço Catálogo, modelado como um pipeline de construção.

A *entrega contínua* (continuous delivery, ou CD) se baseia nesse e em outros conceitos. Conforme descrito no livro de Jez Humble e Dave Farley,³ a CD é a abordagem por meio da qual temos um feedback contínuo para saber se todo e qualquer check-in está pronto para o ambiente de produção; desse modo, todo e qualquer check-in é tratado como um candidato para uma versão a ser lançada.

Para adotar esse conceito de forma completa, é necessário modelar todos os processos envolvidos para levar nosso software do check-in até o ambiente de produção, e precisamos saber a posição em que se encontra

qualquer dada versão do software no que concerne à sua aprovação para um lançamento. Na CD, fazemos isso modelando cada uma das etapas pelas quais o nosso software deve passar, tanto as etapas manuais como as automatizadas; a Figura 7.1 mostra um exemplo para o nosso serviço catálogo. A maioria das ferramentas de CI atualmente oferece algum suporte para definir e visualizar o estado dos pipelines de construção como esse.

Se o novo serviço catálogo for aprovado em quaisquer verificações feitas em uma etapa do pipeline, ele poderá então continuar para o próximo passo. Se não for aprovado em uma etapa, nossa ferramenta de CI poderá nos informar as etapas em que a construção foi bem-sucedida, dando-nos visibilidade acerca do que falhou. Se for necessário tomar alguma atitude para correção, faríamos uma alteração e efetuaríamos seu check-in, permitindo que a nova versão do nosso microsserviço tentasse passar por todas as etapas, até estar disponível para uma implantação. Na Figura 7.2, vemos um exemplo disso: a `build-120` foi reprovada na etapa de testes rápidos, a `build-121` foi reprovada nos testes de desempenho, mas a `build-122` conseguiu chegar até o ambiente de produção.

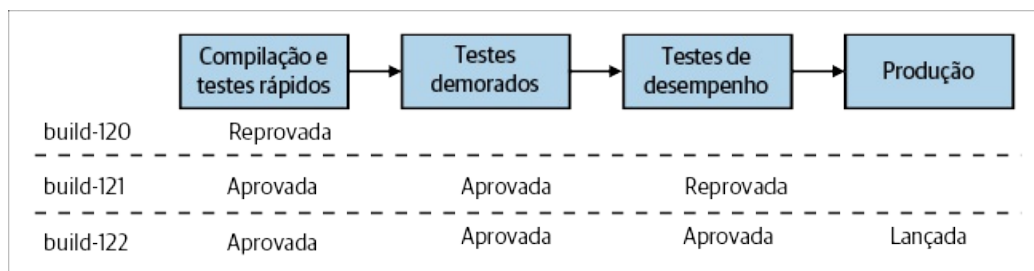


Figura 7.2 – Nosso microsserviço catálogo poderá ser implantado somente se passar por todas as etapas de nosso pipeline.

Entrega contínua versus implantação contínua

Já vi ocasionalmente uma certa confusão relacionada aos termos *entrega contínua* (continuous delivery) e *implantação contínua* (continuous deployment). Conforme já discutimos, a entrega contínua é o conceito segundo o qual cada check-in é tratado como um candidato para uma versão a ser lançada, e com o qual podemos avaliar a qualidade de cada candidato ao lançamento para decidir se essa versão está pronta para ser implantada. Na implantação contínua, por outro lado, todos os check-ins precisam ser validados com métodos automatizados (testes,

por exemplo), e qualquer software que tenha sido aprovado nessas verificações será automaticamente implantado, sem intervenção humana. A implantação contínua, portanto, pode ser considerada uma extensão da entrega contínua. Sem a entrega contínua, não podemos ter uma implantação contínua. Contudo, *podemos* ter entrega contínua *sem* colocar a implantação contínua em prática.

A implantação contínua não é apropriada para todos – muitas pessoas querem alguma interação humana para decidir se um software deve ser implantado, algo que é totalmente compatível com a entrega contínua. No entanto, adotar uma entrega contínua implica um foco constante na otimização de seu trajeto até o ambiente de produção, com uma maior visibilidade para facilitar ver os pontos em que as otimizações devem ser feitas. Muitas vezes, o envolvimento humano no processo pós-check-in é um gargalo que precisa ser considerado – veja, por exemplo, a mudança dos testes de regressão manuais para os testes funcionais automatizados. Como resultado, à medida que automatizar cada vez mais o seu processo de construção, implantação e lançamento, você poderá se ver cada vez mais próximo de uma implantação contínua.

Ferramentas

O ideal é que você tenha uma ferramenta que inclua a entrega contínua como um conceito de primeira classe. Já vi muitas pessoas tentarem fazer um hack e ampliar as ferramentas de CI para que possam ser usadas para CD, geralmente resultando em sistemas complexos, que não chegam nem perto de serem tão fáceis de usar quanto as ferramentas que já têm um suporte para CD incluído desde o princípio. Ferramentas com suporte completo para CD permitem que você defina e visualize esses pipelines, modelando todo o caminho de seu software até o ambiente de produção. À medida que uma versão de nosso código passar pelo pipeline, se ela for aprovada em uma dessas etapas de verificação automática, ela passará para a próxima etapa.

Algumas etapas podem ser manuais. Por exemplo, se tivermos um processo de UAT (User Acceptance Testing, ou Testes de Aceitação de Usuários) manual, devo ser capaz de utilizar uma ferramenta de CD para modelá-lo.

Eu poderia ver a próxima build disponível pronta para ser implantada no ambiente de UAT, e então a implantar; em seguida, caso ela passe pelas verificações manuais, poderia marcar essa etapa como aprovada para que a construção do software pudesse prosseguir para a próxima fase. Se a etapa subsequente for automatizada, ela será disparada automaticamente.

Relação de custo-benefício e ambientes

À medida que nosso artefato – isto é, o nosso microsserviço – se move por esse pipeline, o microsserviço será implantado em diferentes ambientes. Diferentes ambientes servem a propósitos distintos, e podem ter características diferentes.

Estruturar um pipeline e, desse modo, definir quais são os ambientes necessários, por si só, é uma decisão que exige uma avaliação de custo-benefício. No início do pipeline, queremos um feedback rápido para saber se nosso software estará pronto para o ambiente de produção. Queremos permitir que os desenvolvedores saibam o mais rápido possível se há algum problema – quanto mais cedo obtivermos um feedback sobre um problema que ocorrer, mais rápido será para corrigi-lo. À medida que nosso software se aproximar do ambiente de produção, queremos ter mais certeza de que ele funcionará e, desse modo, faremos a sua implantação em ambientes cada vez mais parecidos com o ambiente de produção – podemos ver essa relação de custo-benefício na Figura 7.3.

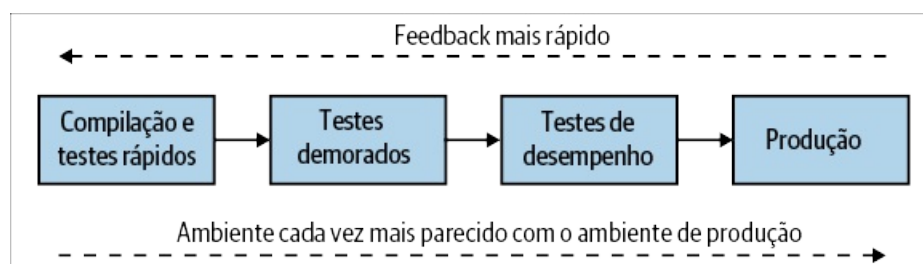


Figura 7.3 – Encontrando um equilíbrio em um pipeline de construção, de modo a ter feedbacks rápidos e ambientes de execução semelhantes ao ambiente de produção.

Você terá um feedback mais rápido em seu notebook de desenvolvimento – mas ele está longe de ser parecido com um ambiente de produção. Poderia fazer o rollout de qualquer commit em um ambiente que seja uma réplica

fiel de seu ambiente de produção real, mas isso provavelmente vai exigir mais tempo e dinheiro. Portanto, é essencial encontrar um equilíbrio, e continuar revendo a relação de custo-benefício entre um feedback rápido e a necessidade de ter ambientes parecidos com o ambiente de produção pode ser uma atividade contínua extremamente importante.

Os desafios de criar um ambiente semelhante ao ambiente de produção também fazem parte dos motivos pelos quais mais pessoas estão executando certas formas de testes em produção, incluindo técnicas como testes de fumaça (smoke testing) e execuções paralelas (parallel runs). Retomaremos esse assunto no Capítulo 8.

Criação de artefatos

À medida que nosso microserviço passar por diferentes ambientes, devemos ter algo para implantar. O fato é que há uma série de opções diferentes para o tipo de artefato de implantação que podemos usar. Em geral, o artefato que criaremos dependerá bastante da tecnologia que decidirmos adotar para a implantação. Veremos esse assunto com detalhes no próximo capítulo, mas gostaria de apresentar algumas dicas muito importantes sobre como a criação de artefatos deve se enquadrar em seu processo de construção quando usamos CI/CD.

Para manter a simplicidade, deixaremos de lado o tipo exato de artefato que criaremos – considere apenas que ele é, por enquanto, um único artefato genérico, possível de ser implantado. Há duas regras importantes que precisamos considerar. Em primeiro lugar, conforme já mencionei, devemos criar um artefato uma e somente uma vez. Construir o mesmo artefato repetidamente é um desperdício de tempo e é ruim para o planeta; além disso, teoricamente, pode haver problemas caso a configuração de construção não seja exatamente a mesma todas as vezes. Em algumas linguagens de programação, uma flag de construção diferente pode fazer o software se comportar de modo bem diverso. Em segundo lugar, o artefato que você verificar deverá ser o artefato a ser implantado! Se você constrói e testa um microserviço, diz “sim, está funcionando”, e então constrói esse microserviço novamente para implantá-lo no ambiente de produção,

como saberá se o software que você validou é o mesmo que foi implantado?

Juntando essas duas ideias, temos uma abordagem bem simples. Construa seu artefato para implantação uma e somente uma vez; o ideal é que isso seja feito bem no início do pipeline. Em geral, eu faria isso após a compilação do código (se for necessária) e a execução de meus testes rápidos. Assim que o artefato for criado, ele será armazenado em um repositório apropriado – poderia ser em algo como o Artifactory ou o Nexus, ou talvez em um registro de contêineres (container registry). Sua opção de artefato para implantação provavelmente determinará a natureza do repositório para o artefato. Esse mesmo artefato poderá então ser usado em todas as etapas subsequentes do pipeline, inclusive na implantação no ambiente de produção. Assim, voltando ao nosso pipeline anterior, na Figura 7.4, podemos ver que criamos um artefato para o nosso serviço catálogo durante a primeira etapa do pipeline, e então implantamos o mesmo artefato `build-123` como parte das etapas de testes demorados, testes de desempenho e produção.

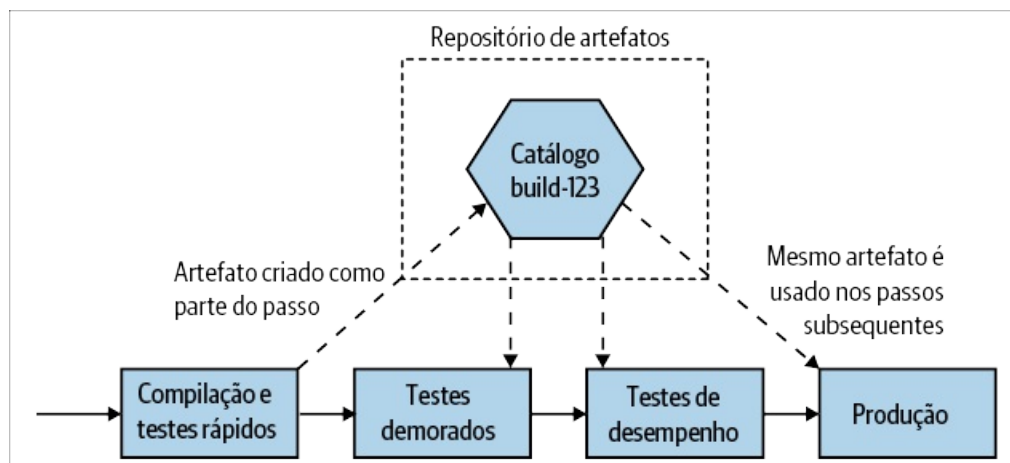


Figura 7.4 – O mesmo artefato é implantado em cada ambiente.

Se o mesmo artefato vai ser usado em vários ambientes, quaisquer aspectos da configuração que variem de ambiente para ambiente deverão ser mantidos fora do artefato propriamente dito. Como um exemplo simples, eu poderia configurar os logs da aplicação de modo que tudo que estivesse no nível `DEBUG` e acima dele fosse logado na execução da etapa de Testes

Demorados, fornecendo mais informações para diagnosticar o motivo de um teste falhar. Contudo, eu poderia decidir mudar isso para INFO a fim de reduzir o volume de logs nos Testes de Desempenho e na implantação em Produção.



Dicas para criação de artefatos

Construa um artefato para a implantação de seu microserviço somente uma vez. Reutilize esse artefato em todos os locais que você quiser implantar essa versão de seu microserviço. Mantenha o seu artefato de implantação independente do ambiente – armazene configurações específicas para um ambiente em outro lugar.

Mapeando códigos-fontes e builds aos microserviços

Já falamos de um assunto que pode deixar facções em guerra – branching por funcionalidades *versus* desenvolvimento com base em tronco – porém, o fato é que a controvérsia ainda não acabou neste capítulo. Outro assunto que provavelmente deve elicitar opiniões muito distintas é a organização do código de nossos microserviços. Tenho minhas próprias preferências, mas, antes de falar delas, vamos explorar as principais opções para organizar o código de nossos microserviços.

Um repositório gigante, uma build gigante

Se começarmos com a opção mais simples, poderíamos deixar tudo em um só lugar. Teremos um único repositório gigante armazenando todo o nosso código, e uma única build, conforme vemos na Figura 7.5. Qualquer check-in nesse repositório de códigos-fontes vai disparar nosso processo de construção, e executaremos todos os passos de verificação associados a todos os nossos microserviços, gerando diversos artefatos, todos associados à mesma build.

Em comparação com outras abordagens, essa solução parece muito mais simples à primeira vista: menos repositórios com os quais se preocupar, e uma build conceitualmente mais simples. Do ponto de vista de um desenvolvedor, as tarefas são muito simples também. Basta fazer o check-in

do código. Se for necessário trabalhar em vários serviços ao mesmo tempo, terei de me preocupar somente com um único commit.

Esse modelo pode funcionar perfeitamente se você aceitar a ideia de lançamentos sincronizados, em que você não se importará em implantar vários serviços de uma só vez. Em geral, esse é um padrão a ser definitivamente evitado, mas, bem no início de um projeto, sobretudo se houver apenas uma equipe trabalhando com tudo, esse modelo poderia fazer sentido por um curto período.

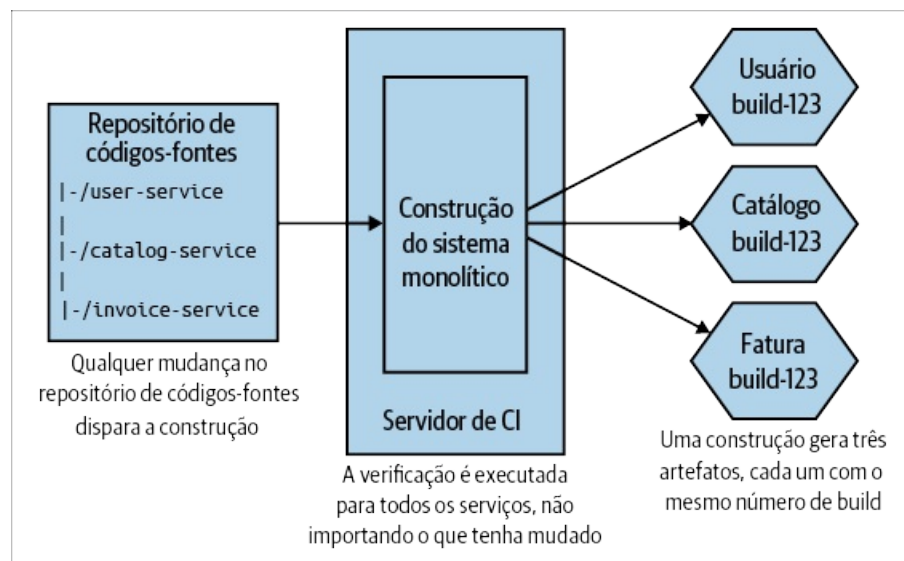


Figura 7.5 – Utilizando um único repositório de códigos-fontes e uma única build de CI para todos os microsserviços.

Deixe-me agora explicar algumas desvantagens significativas dessa abordagem. Se eu fizer uma alteração de uma só linha em um único serviço – por exemplo, alterar o comportamento do serviço `Usuário` que vemos na Figura 7.5 – todos os demais serviços serão verificados e construídos. Isso poderia demandar mais tempo do que o necessário – terei de esperar por algo que provavelmente nem precisaria ser testado. Haverá um impacto no tempo de execução de um ciclo, isto é, na velocidade com a qual uma única alteração poderia passar da fase de desenvolvimento para a fase em que estará ativa em produção. Mais problemático, porém, será saber quais artefatos devem ou não ser implantados. Precisarei agora implantar todos os serviços construídos para que minha pequena alteração seja colocada no ambiente de produção? Pode ser difícil dizer; tentar adivinhar quais

serviços *realmente* mudaram apenas lendo as mensagens de commit é complicado. As empresas que utilizam essa abordagem em geral acabam optando por implantar tudo, o que é algo que realmente queremos evitar.

Além disso, se minha alteração de uma só linha no serviço `Usuário` causar falha na build, nenhuma outra alteração poderá ser feita nos demais serviços até que a falha seja corrigida. Pense também em um cenário em que você tenha várias equipes, todas compartilhando essa build gigantesca. Quem é o responsável?

Sem dúvida, essa abordagem é uma forma de monorepo (repositório único). Na prática, porém, a maioria das implementações de monorepo que já vi mapeiam várias builds a diferentes partes do repositório, algo que exploraremos com mais detalhes em breve. Assim, poderíamos encarar esse padrão de um único repositório mapeado para uma única build como a *pior* forma de monorepo para as pessoas que queiram construir vários microsserviços possíveis de serem implantados de modo independente.

Na prática, raramente vejo essa abordagem ser usada, exceto nas etapas iniciais de um projeto. Para ser franco, prefiro muito mais as duas abordagens a seguir, portanto manteremos o foco nessas opções.

Padrão: um repositório por microsserviço (também conhecido como multirepo)

Com o padrão de um repositório por microsserviço (mais comumente chamado de padrão *multirepo* quando comparado ao padrão monorepo), o código de cada microsserviço é armazenado em seu próprio repositório de código-fonte, conforme vemos na Figura 7.6. Essa abordagem implica um mapeamento direto entre as alterações no código-fonte e as builds de CI.

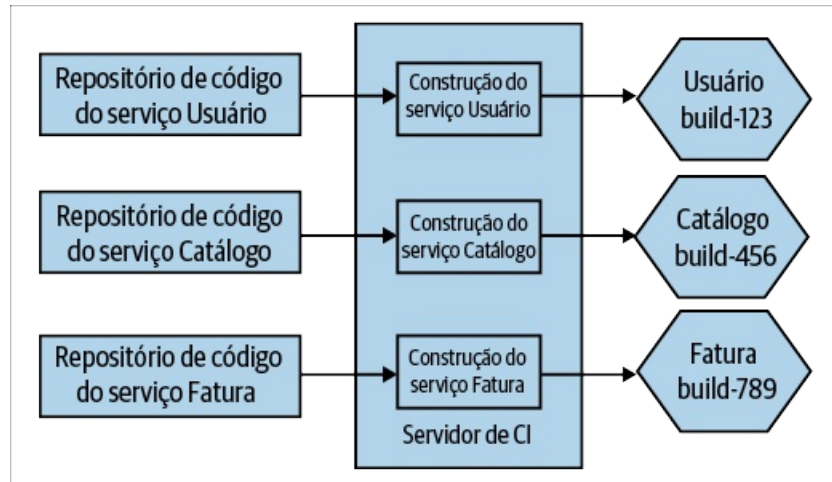


Figura 7.6 – O código-fonte de cada microserviço é armazenado em um repositório de código-fonte separado.

Qualquer mudança no repositório de código-fonte de `Usuário` dispara a construção correspondente do microserviço; se a build for aprovada, teremos uma nova versão do microserviço `Usuário` disponível para implantação. Ter um repositório separado para cada microserviço também permite que a responsabilidade (ownership) pelo repositório mude individualmente, algo que fará sentido se considerarmos um modelo de responsabilidade forte (strong ownership) para seus microserviços (mais sobre esse assunto em breve).

A natureza direta desse padrão, no entanto, dá margem a alguns desafios. Especificamente, os desenvolvedores poderão se ver trabalhando com vários repositórios ao mesmo tempo, o que é particularmente complicado se estiverem tentando fazer mudanças em vários repositórios de uma só vez. Além do mais, as alterações não poderão ser feitas atomicamente nos diferentes repositórios, pelo menos não com o Git.

Reutilizando código em diferentes repositórios

Ao utilizar esse padrão, não há nada que impeça um microserviço de depender de outro código gerenciado em outros repositórios. Um método simples para implementar isso é fazer com que o código que você queira reutilizar esteja em uma biblioteca, a qual então se tornará uma dependência explícita dos microserviços downstream. Podemos ver um exemplo desse caso na Figura 7.7, em que tanto o serviço `Fatura` como o

serviço Folha de Pagamento fazem uso da biblioteca Conexão.

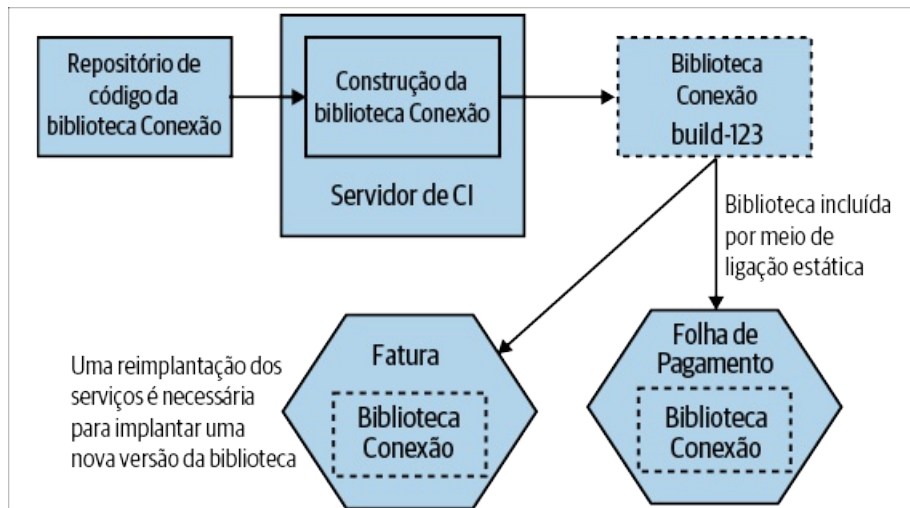


Figura 7.7 – Reutilizando código entre diferentes repositórios.

Se quiser fazer o rollout de uma alteração na biblioteca *conexão*, você teria de fazer essa mudança no repositório de código-fonte correspondente e esperar que a construção da biblioteca fosse concluída, resultando em um novo artefato com uma nova versão. Para implantar novas versões dos serviços *Fatura* e *Folha de Pagamento* que utilizem essa nova versão da biblioteca, será necessário mudar a versão da biblioteca *conexão* que esses serviços utilizam. Isso poderia exigir uma alteração manual (caso você dependa de uma versão específica), ou poderia ser configurado para ocorrer dinamicamente, conforme a natureza das ferramentas de CI que você estiver utilizando. Os conceitos associados a esse tópico estão descritos com mais detalhes no livro *Continuous Delivery*, de Jez Humble e Dave Farley.⁴

O aspecto importante a ser lembrado, é claro, é que, se quiser fazer o rollout da nova versão da biblioteca *conexão*, será necessário também implantar os serviços *Fatura* e *Folha de Pagamento* recém-construídos. Lembre-se de que todas as ressalvas exploradas na seção “DRY e os perigos da reutilização de código em um mundo de microsserviços”, na página 190, quanto à reutilização de código em microsserviços continuam se aplicando – se você optar por reutilizar código por meio de bibliotecas, deverá aceitar o fato de que o rollout dessas alterações não poderá ser feito atômicamente, pois, do contrário, nosso objetivo de ter implantações

independentes será prejudicado. Você também deve estar ciente de que poderá ser mais complicado saber se algum microsserviço utiliza uma versão específica de uma biblioteca, o que poderia ser problemático caso você esteja tentando obsoletar uma versão antiga da biblioteca.

Trabalhando com vários repositórios

Além de reutilizar código por meio de bibliotecas, de que outra maneira poderemos fazer alterações em mais de um repositório? Vamos analisar outro exemplo. Na Figura 7.8, eu gostaria de modificar a API exposta pelo serviço *Estoque*, e preciso atualizar também o serviço *Expedição* para que ele possa fazer uso dessa alteração. Se os códigos tanto de *Estoque* como de *Expedição* estivessem no mesmo repositório, eu poderia fazer o commit do código de uma só vez. Em vez disso, terei de separar as alterações em dois commits – um para *Estoque* e outro para *Expedição*.

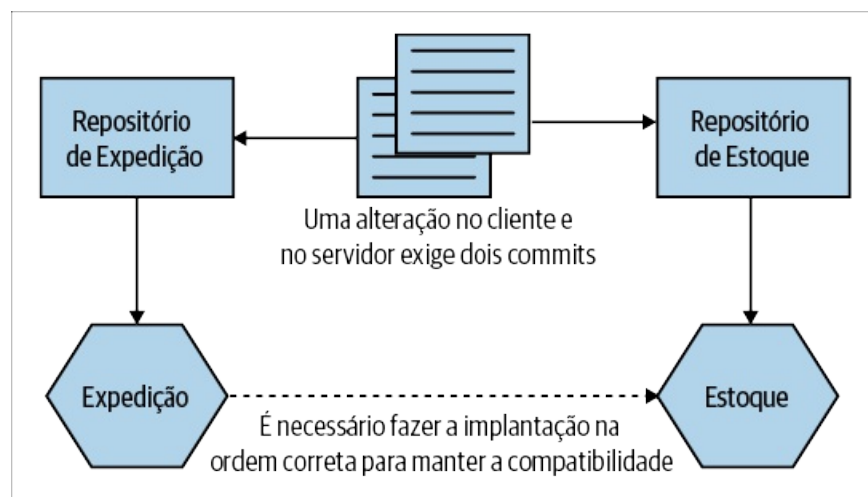


Figura 7.8 – Alterações que cruzam fronteiras entre repositórios exigem vários commits.

Ter essas alterações separadas poderia causar problemas se um commit falhar, porém o outro funcionar – talvez eu tivesse de fazer duas alterações para efetuar o rollback da mudança, por exemplo, e isso seria complicado se outras pessoas tiverem feito check-ins nesse íterim. A verdade é que, nessa situação específica, é provável que eu queira, de certo modo, fazer os commits em fases, de qualquer maneira. Eu iria querer garantir que o commit para alterar o serviço *Estoque* funcionasse antes de modificar

qualquer código de cliente no serviço *Expedição* — se a nova funcionalidade não estiver presente na API, não haverá motivos para ter um código de cliente que faça uso dela.

Já conversei com várias pessoas que acham que a falta de implantações atômicas nesse caso é um problema relevante. Sem dúvida, posso compreender a complexidade resultante dessa situação, mas acho que, na maioria dos casos, o cenário aponta para um problema subjacente mais importante. Se você faz continuamente alterações envolvendo vários microsserviços, talvez as fronteiras de seus serviços não estejam no lugar certo, e isso poderia implicar um excesso de acoplamento entre os seus serviços. Conforme já discutimos, estamos tentando otimizar a nossa arquitetura — e as fronteiras de nossos microsserviços — de modo que seja mais provável que as alterações sejam aplicadas dentro das fronteiras de um microsserviço. Alterações que cruzem fronteiras deveriam ser a exceção, e não a norma.

Com efeito, eu argumentaria que a dificuldade de trabalhar com vários repositórios poderia servir para ajudar a definir melhor as fronteiras dos microsserviços, pois obrigará você a pensar com mais cuidado no local em que essas fronteiras estão, e na natureza das interações entre elas.



Se você faz constantemente alterações que envolvam vários microsserviços, é provável que as fronteiras de seus microsserviços estejam no lugar errado. Talvez valha a pena considerar a fusão de alguns microsserviços caso você perceba que isso esteja acontecendo.

Há também o incômodo de ter de extrair códigos de vários repositórios e enviá-los para vários repositórios como parte de seu fluxo de trabalho usual. Com base em minha experiência, posso dizer que isso pode ser simplificado seja pela utilização de um IDE que aceite vários repositórios (é uma tarefa com a qual todos os IDEs que utilizei nos últimos cinco anos é capaz de lidar), seja escrevendo scripts wrapper simples para deixar o trabalho na linha de comando mais fácil.

Quando usar esse padrão

Utilizar a abordagem de um repositório por microsserviço funciona bem

tanto para equipes pequenas como para equipes grandes, mas, se você se vir fazendo várias alterações que cruzem as fronteiras dos microsserviços, talvez não seja a solução adequada para você, e o padrão monorepo que discutiremos a seguir talvez seja mais apropriado – embora fazer várias alterações que cruzem as fronteiras dos microsserviços possa ser considerada uma advertência de que algo não está certo, conforme discutimos antes. Esse padrão também pode deixar a reutilização de código mais complexa quando comparado com a abordagem com monorepo, pois você dependerá de códigos contidos em artefatos com versões específicas.

Padrão: monorepo

Com uma abordagem monorepo, o código de vários microsserviços (ou outros tipos de projetos) é armazenado no mesmo repositório de códigos-fontes. Já vi situações em que um monorepo era utilizado somente por uma equipe para gerenciar o controle de versões de todos os seus serviços, embora o conceito tenha se popularizado com algumas empresas de tecnologia muito grandes, nas quais várias equipes e centenas, se não milhares, de desenvolvedores podem trabalhar no mesmo repositório de códigos-fontes.

Ao ter todo o código-fonte no mesmo repositório, permitimos que alterações no código-fonte possam ser feitas entre diferentes projetos de forma atômica, além de possibilitar uma reutilização de códigos mais específicos de um projeto para outro. A Google provavelmente é o exemplo mais conhecido de uma empresa que utiliza a abordagem de monorepo, embora esteja longe de ser a única. Embora haja outras vantagens nessa abordagem, por exemplo, mais visibilidade no código de outras pessoas, a capacidade de reutilizar código facilmente e de fazer alterações que causem impacto em vários projetos diferentes frequentemente é citada como o principal motivo para a adoção desse padrão.

Se considerarmos o exemplo que acabamos de discutir, no qual queremos fazer uma alteração no microsserviço `Estoque` para que ele exponha um novo comportamento, e atualizar o serviço `Expedição` para que utilize essa nova funcionalidade exposta, essas alterações poderiam ser feitas com um único

commit, conforme vemos na Figura 7.9.

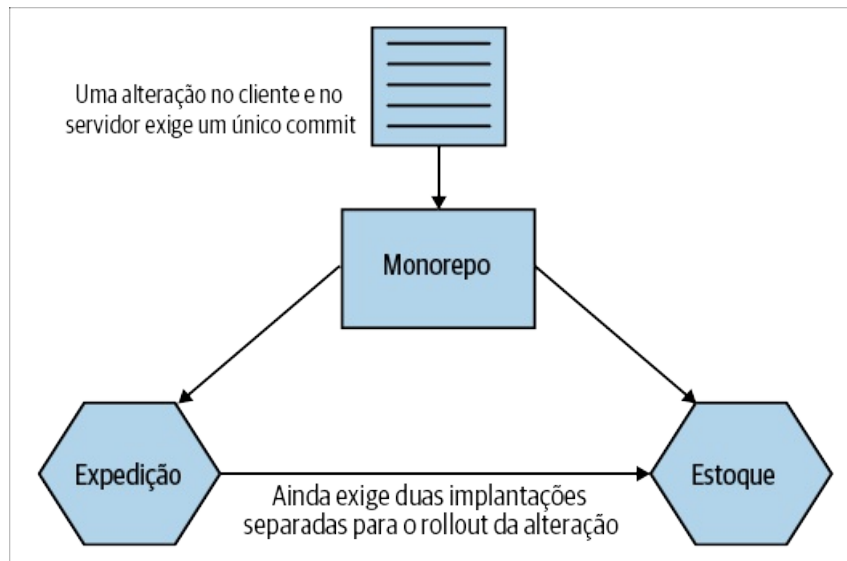


Figura 7.9 – Utilizando um único commit para fazer alterações em dois microsserviços usando um monorepo.

É claro que, como ocorre no padrão multirepo discutido antes, ainda precisaremos lidar com o lado da implantação nesse caso. Provavelmente teríamos de considerar cuidadosamente a ordem das implantações se quisermos evitar uma implantação sincronizada.

Commits atômicos versus implantação atômica

Ser capaz de fazer um commit atômico envolvendo vários serviços não implica um rollout atômico. Se você se vir querendo alterar um código que envolva vários serviços simultaneamente e fazer o rollout de tudo ao mesmo tempo no ambiente de produção, o princípio fundamental de implantações independentes será violado. Para outras informações sobre esse assunto, consulte a seção “DRY e os perigos da reutilização de código em um mundo de microsserviços”, na página [190](#).

Mapeamento para processos de construção

Com um único repositório de código-fonte por microsserviço, o mapeamento do código-fonte para um processo de construção é simples. Qualquer alteração no repositório de código-fonte em questão pode disparar um processo de construção de CI correspondente. Em um

monorepo, a situação é um pouco mais complexa.

Um ponto de partida simples no monorepo seria mapear pastas a um processo de construção, como mostra a Figura 7.10. Uma mudança feita na pasta `user-service` dispararia a construção do serviço `Usuário`, por exemplo. Se você fizer check-in de um código que tenha alterado arquivos tanto na pasta `user-service` como na pasta `catalog-service`, a construção tanto do microserviço `Usuário` como do microserviço `catálogo` será disparada.

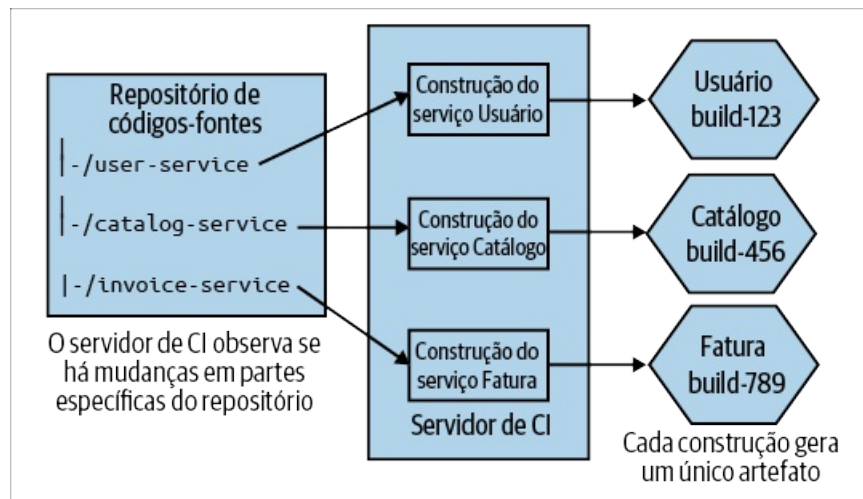


Figura 7.10 – Um único repositório de códigos-fontes, com subdiretórios mapeados para processos de construções independentes.

A situação se torna mais complicada à medida que houver estruturas de pasta mais complexas. Em projetos maiores, você poderá acabar com várias pastas diferentes querendo disparar o mesmo processo de construção, com algumas pastas disparando mais de um processo. Na extremidade mais simples do espectro, você poderia ter uma pasta “comum” utilizada por todos os microserviços, e uma mudança nessa pasta faria com que todos os microserviços fossem reconstruídos. Na extremidade mais complexa, as equipes acabariam tendo de adotar ferramentas de construção mais voltadas para grafos, como a ferramenta open source Bazel (<https://bazel.build>), para gerenciar essas dependências com mais eficácia (o Bazel é uma versão open source da própria ferramenta interna de construção da Google). Implementar um novo sistema de construção pode ser um empreendimento significativo, portanto não é algo que deva ser feito de forma leviana – no entanto, o próprio monorepo da

Google seria impossível se não houvesse ferramentas como essa.

Uma das vantagens de uma abordagem com monorepo é que podemos ter uma reutilização de código altamente especializada entre os projetos. Em um modelo multirepo, se eu quisesse reutilizar o código de outra pessoa, esse código provavelmente teria de ser empacotado como um artefato com uma versão atribuída, o qual eu poderia então incluir como parte da minha build (por exemplo, um pacote Nuget, um arquivo JAR ou um NPM). Com nossa unidade de reutilização sendo uma biblioteca, possivelmente incluiremos mais código do que realmente iríamos querer. Teoricamente, com um monorepo, eu poderia simplesmente depender de um único arquivo-fonte de outro projeto – embora isso, é claro, me forçaria a ter um mapeamento mais complexo no processo de construção.

Definição de responsabilidades

Com equipes menores e bases de código menores, os monorepos provavelmente funcionarão bem com as ferramentas tradicionais de construção e de gerenciamento de códigos-fontes com as quais você está acostumado. No entanto, à medida que seu monorepo crescer, é provável que você precise começar a considerar diferentes tipos de ferramentas. Exploraremos os modelos de responsabilidade (ownership) com mais detalhes no Capítulo 15, mas, por enquanto, vale a pena explorar rapidamente o papel que eles desempenham quando pensamos no controle de versões.

Martin Fowler já escreveu (<https://oreil.ly/nNNWd>) sobre diferentes modelos de responsabilidade, descrevendo uma escala móvel de responsabilidade que varia da *responsabilidade forte* (strong ownership) à *responsabilidade fraca* (weak ownership), além da *responsabilidade coletiva* (collective ownership). Desde que Martin definiu esses termos, as práticas de desenvolvimento sofreram mudanças, portanto vale a pena revê-los e redefini-los.

Com uma responsabilidade forte, um código pertence a um grupo específico de pessoas. Se alguém de fora desse grupo quiser fazer uma alteração, essa pessoa terá de pedir que os responsáveis façam a

modificação para ela. Uma responsabilidade fraca ainda mantém o conceito de responsáveis, mas as pessoas que estão fora do grupo de responsáveis têm permissão para fazer alterações, embora qualquer uma dessas alterações precise ser revisada e deva ser aceita por alguém do grupo de responsáveis. Isso incluiria o caso de um pull request enviado à equipe principal responsável pelo código, para que esse fosse revisado antes que o merge seja efetuado. Com uma abordagem de responsabilidade coletiva, qualquer desenvolvedor pode alterar qualquer parte do código.

Com menos desenvolvedores (vinte ou menos, como regra geral), você pode se dar ao luxo de adotar um modelo de responsabilidade coletiva – no qual qualquer desenvolvedor pode alterar qualquer outro microsserviço. No entanto, à medida que houver mais pessoas, é provável que você queira passar para um modelo de responsabilidade forte ou fraca a fim de definir melhor as fronteiras das responsabilidades. Isso pode representar um desafio para equipes que utilizem monorepos caso a sua ferramenta de controle de versões não ofereça suporte para controles mais minuciosos de responsabilidade.

Algumas ferramentas de controle de códigos-fontes permitem especificar os responsáveis por diretórios específicos, ou até mesmo por paths de arquivo específicos dentro de um único repositório. A Google implementou inicialmente esse sistema com base no Perforce para o seu próprio monorepo, até desenvolver o seu próprio sistema de controle de versões; esse é um recurso também aceito pelo GitHub (<https://oreil.ly/zxmXn>) desde 2016. No GitHub, criamos um arquivo CODEOWNERS, que permite mapear responsáveis (owners) a diretórios ou paths de arquivos. Podemos ver alguns exemplos a seguir (Exemplo 7.1), extraídos da própria documentação do GitHub, que mostram os tipos de flexibilidade proporcionados por esses sistemas.

Exemplo 7.1 – Exemplos de como especificar responsáveis por diretórios específicos em um arquivo CODEOWNERS do GitHub.

```
# Neste exemplo, @doctocat é responsável por qualquer arquivo do diretório
  build/logs
# na raiz do repositório e por qualquer um de seus
# subdiretórios.
```

```
/build/logs/ @doctocat
```

```
# Neste exemplo, @doctocat é responsável por qualquer arquivo em um diretório apps  
# em qualquer lugar no repositório.  
apps/ @doctocat
```

```
# Neste exemplo, @doctocat é responsável por qualquer arquivo do diretório  
`/docs`  
# na raiz de seu repositório.  
/docs/ @doctocat
```

O próprio conceito de responsabilidade pelo código (code ownership) no GitHub garante que os responsáveis pelos arquivos-fontes sejam requisitados a fazer uma revisão sempre que um pull request for feito nos arquivos relevantes. Isso poderia ser um problema com pull requests maiores, pois você poderia acabar precisando da autorização de vários revisores, mas, de qualquer maneira, há uma série de bons motivos para ter como meta o uso de pull requests menores.

Ferramentas

O próprio monorepo da Google é gigantesco, e exige muito trabalho de engenharia para que funcione em larga escala. Considere aspectos como um sistema de construção baseado em grafos que tenha passado por várias gerações, um linker de objetos distribuído para melhorar os tempos de construção, plug-ins para IDEs e editores de texto que possam manter dinamicamente atualizados os arquivos de dependência – é um volume enorme de trabalho. À medida que a Google cresceu, ela foi atingindo limitações em seu uso do Perforce e acabou precisando criar sua própria ferramenta proprietária de controle de versões chamada Piper. Quando trabalhei com essa parte na Google em 2007 e 2008, havia mais de uma centena de pessoas trabalhando com a manutenção de diversas ferramentas para desenvolvedores, com uma parte significativa desses esforços dedicada a lidar com as implicações da abordagem com monorepo. Isso é algo que você poderia justificar se tivesse dezenas de milhares de engenheiros, é claro.

Para uma visão geral mais detalhada do raciocínio por trás do uso de um monorepo pela Google, recomendo o artigo “Why Google Stores Billions

of Lines of Code in a Single Repository” (Por que a Google armazena bilhões de linhas de código em um único repositório, <https://oreil.ly/wMyH3>), de Rachel Potvin e Josh Levenberg.⁵ Com efeito, eu sugeriria que é uma leitura obrigatória para qualquer um que pense: “Devemos usar um monorepo, pois a Google utiliza um!”. Sua empresa provavelmente não é a Google, e é provável que não tenha os tipos de problemas, as restrições ou os recursos que a Google tem. Falando de outra maneira, qualquer que seja o monorepo que você acabe tendo, provavelmente ele não será igual ao da Google.

A Microsoft passou por problemas semelhantes com a escala. Ela adotou o Git para ajudar no gerenciamento do repositório principal de códigos-fontes do Windows. Um diretório de trabalho completo para essa base de código tem aproximadamente 270 GB de arquivos-fontes.⁶ Fazer o download de tudo isso exigiria muito tempo, além de não ser necessário – os desenvolvedores acabariam trabalhando somente em uma pequena parte do sistema como um todo. Assim, a Microsoft precisou criar um sistema de arquivos virtual dedicado, o VFS for Git (anteriormente conhecido como GVFS), que garante que somente os arquivos-fontes necessários a um desenvolvedor sejam realmente baixados.

O VFS for Git é uma realização impressionante, assim como o conjunto de ferramentas próprio da Google, embora justificar esses investimentos em tecnologias desse tipo seja muito mais fácil em empresas como essas. Também vale a pena ressaltar que, embora o VSF for Git seja open source, ainda não vi nenhuma equipe além da Microsoft que a utilizasse – e a maior parte do conjunto de ferramentas próprio da Google que oferece suporte para o seu monorepo tem código fechado (o Bazel se destaca como uma exceção, porém não está claro até que ponto o Bazel open source espelha o que é utilizado na própria Google).

O artigo “Monorepos in the Wild” (Monorepos por aí, <https://oreil.ly/1SR0A0>), de Markus Oberlehner, me apresentou ao Lerna (<https://lerna.js.org>), uma ferramenta criada pela equipe responsável pelo Babel, um compilador de JavaScript. O Lerna foi projetado para facilitar a geração de artefatos com diferentes versões a partir do mesmo repositório

de códigos-fontes. Não posso falar diretamente da eficácia do Lerna nessa tarefa (além de ter várias outras deficiências notáveis, não sou um desenvolvedor experiente em JavaScript), mas a partir de uma análise superficial, de certa forma, ele parece simplificar essa abordagem.

Até que ponto “mono” é mono?

A Google não armazena *todo* o seu código em um monorepo. Há alguns projetos, sobretudo aqueles desenvolvidos abertamente, que são mantidos em outros lugares. Apesar disso, ao menos com base no artigo da ACM mencionado antes, 95% do código da Google estava armazenado em um monorepo em 2016. Em outras empresas, um monorepo pode ter como escopo um único sistema, ou um pequeno número de sistemas. Isso significa que uma empresa poderia ter um pequeno número de monorepos para diferentes partes da empresa.

Também já conversei com equipes que utilizam monorepos por equipe. Embora, tecnicamente falando, isso provavelmente não corresponda à definição original desse padrão (o qual geralmente diz respeito a várias equipes compartilhando o mesmo repositório), ainda acho que essa abordagem está mais para “monorepo” do que qualquer outra opção. Nessa situação, cada equipe tem o seu próprio monorepo sob seu total controle. Todos os microsserviços sob a responsabilidade dessa equipe têm seu código armazenado no monorepo dessa equipe, como mostra a Figura 7.11.

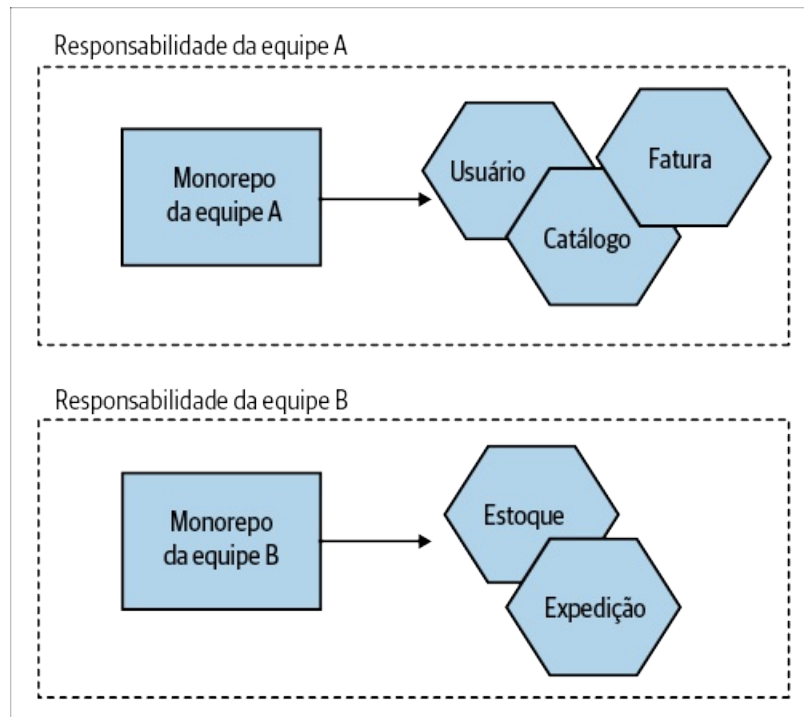


Figura 7.11 – Uma variação do padrão, na qual cada equipe tem o seu próprio monorepo.

Para as equipes que adotam a abordagem de responsabilidade coletiva, esse modelo tem muitas vantagens, sem dúvida proporcionando muitos dos benefícios de uma abordagem monorepo, ao mesmo tempo que evita alguns dos problemas que surgem em escalas maiores. Essa solução híbrida pode fazer bastante sentido para trabalhar com as fronteiras de responsabilidade que já existam em uma empresa, e, de certa forma, pode reduzir as preocupações quanto ao uso desse padrão em uma escala maior.

Quando usar esse padrão

Algumas empresas que trabalham em uma escala muito maior perceberam que a abordagem com monorepo funciona muito bem para elas. Já mencionamos a Google e a Microsoft, e podemos acrescentar o Facebook, o Twitter e o Uber na lista. Todas essas empresas têm um ponto em comum – são empresas grandes, com foco em tecnologia, capazes de dedicar recursos significativos para tirar o melhor proveito desse padrão. Também vejo os monorepos funcionarem bem na outra extremidade do espectro, quando há menos desenvolvedores e equipes. Se houver de dez a

vinde desenvolvedores, será mais fácil gerenciar as fronteiras de responsabilidades e manter o processo de construção simples usando uma abordagem com monorepo. As dificuldades parecem surgir com empresas que estão no meio – aquelas que atingiram uma escala na qual começaram a deparar com problemas que exigem novas ferramentas ou outras formas de trabalhar, mas que não têm recursos sobressalentes para investir nessas ideias.

Qual abordagem devo utilizar?

Com base em minha experiência, posso dizer que as principais vantagens de uma abordagem com monorepo – reutilização de código mais especializada e commits atômicos – não parecem compensar os problemas que surgem com a escala. Para equipes menores, qualquer uma das abordagens seria apropriada, mas, à medida que escalar, acho que a abordagem de um repositório por microserviço (multirepos) é mais simples. Basicamente, eu me preocupo com o incentivo a mudanças que envolvam mais de um serviço, com as linhas de responsabilidade mais confusas e com a necessidade de novas ferramentas que os monorepos podem exigir.

Um problema que já vi repetidamente é o caso de empresas que começaram pequenas, nas quais um modelo de responsabilidade coletiva (e, portanto, de monorepos) funcionava bem inicialmente, mas que tiveram muita dificuldade para passar para outros modelos depois, em virtude de o conceito de monorepo estar extremamente arraigado. À medida que as entregas de software aumentam, as dificuldades com um monorepo aumentam, mas o mesmo acontece com o custo da migração para uma abordagem alternativa. Isso é mais complicado ainda para as empresas que cresceram rapidamente, pois, com frequência, é somente depois desse rápido crescimento que os problemas se tornam evidentes; a essa altura, o custo da migração para uma abordagem multirepo parece ser alto demais. Isso pode levar à falácia do custo irrecuperável (sunk cost fallacy): você investiu tanto para fazer o monorepo funcionar até agora – só um pouquinho mais de investimento fará ele funcionar tão bem quanto costumava funcionar, não é mesmo? Talvez não – mas é necessária uma

alma corajosa para reconhecer que a empresa está jogando um bom dinheiro fora e tomar uma decisão para mudar o curso de ação.

As preocupações com responsabilidades e monorepos podem ser atenuadas com o uso de controles mais específicos de responsabilidade, porém isso tende a exigir ferramentas e/ou um nível maior de disciplina. Minha opinião sobre essa questão pode mudar à medida que as ferramentas associadas aos monorepos amadurecerem mais, porém, apesar do muito trabalho sendo feito no desenvolvimento open source de ferramentas de construção baseadas em grafos, ainda vejo pouca adoção desses conjuntos de ferramentas. Então, para mim, vou de multirepos.

Resumo

Discutimos algumas ideias importantes neste capítulo, as quais devem deixar você em uma boa posição, independentemente de acabar ou não usando microsserviços. Há vários outros aspectos relacionados a essas ideias que podem ser explorados, variando da entrega contínua ao desenvolvimento baseado em tronco, passando por monorepos e multirepos. Apresentei uma série de recursos e indiquei leituras complementares, mas é hora de passarmos para um assunto que é importante para ser explorado com detalhes: a implantação.

-
- ¹ Alanna Brown, Nicole Forsgren, Jez Humble, Nigel Kersten e Gene Kim, *2016 State of DevOps Report*, <https://oreil.ly/YqEEh>.
 - ² Nicole Forsgren, Dustin Smith, Jez Humble e Jessie Frazelle, *Accelerate: State of DevOps 2019*, <https://oreil.ly/mfkIJ>.
 - ³ Para mais detalhes, veja Jez Humble e David Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation* (Upper Saddle River, NJ: Addison Wesley, 2010).
 - ⁴ Consulte “Managing Dependency Graphs” (Gerenciando grafos de dependência) em *Continuous Delivery*, pp. 363–73.
 - ⁵ Rachel Potvin e Josh Levenberg, “Why Google Stores Billions of Lines of Code in a Single Repository” (Por que a Google armazena bilhões de linhas de código em um único repositório), *Communications of the ACM* 59, nº 7 (julho de 2016): 78–87.
 - ⁶ Veja Git Virtual File System Design History (História do projeto Git Virtual File System, <https://oreil.ly/SM7d4>).

Implantação

Implantar uma aplicação monolítica com um único processo é uma tarefa razoavelmente simples. Os microsserviços, com sua interdependência e a variedade de opções de tecnologia, são um caso totalmente diferente. Quando escrevi a primeira edição deste livro, este capítulo já tinha muito a dizer sobre a variedade enorme de opções disponíveis a você. Desde então, o Kubernetes ganhou relevância, e as plataformas FaaS (Function as a Service, ou Função como Serviço) nos oferecem outras formas de pensar em como disponibilizamos nosso software.

Embora as tecnologias possam ter mudado na última década, acho que muitos dos princípios fundamentais associados à construção de softwares não mudaram. Com efeito, acho mais importante ainda que tenhamos uma compreensão profunda dessas ideias fundamentais, pois elas podem nos ajudar a entender como navegar por esse panorama caótico de novas tecnologias. Com isso em mente, este capítulo enfatizará alguns dos princípios básicos relacionados à implantação que são importantes para conhecer, ao mesmo tempo que mostrará como as diferentes ferramentas disponíveis a você poderão ajudar a (ou dificultar) pôr esses princípios em prática.

Para começar, porém, vamos espiar rapidamente por trás das cortinas e observar o que acontece quando passamos de uma visão lógica da arquitetura de nossos sistemas para uma verdadeira topologia de implantação física.

Do lógico ao físico

Até agora, quando discutimos os microsserviços, falamos deles de um ponto de vista lógico, e não físico. Poderíamos falar de como o nosso

microserviço `Fatura` se comunica com o microserviço `Pedido`, conforme vemos na Figura 8.1, sem realmente olhar para a topologia física na qual esses serviços estão implantados. Uma visão lógica de uma arquitetura geralmente abstrai qualquer preocupação com a implantação física subjacente – será preciso rever essa noção no escopo deste capítulo.



Figura 8.1 – Uma visão lógica simples de dois microserviços.

Essa visão lógica de nossos microserviços pode ocultar uma série de complexidades quando se trata de executá-los em uma infraestrutura real. Vamos analisar os tipos de detalhes que podem estar ocultos em um diagrama como esse.

Várias instâncias

Quando pensamos na topologia de implantação dos dois microserviços (na Figura 8.2), não será tão simples quanto mostrar um componente conversando com outro. Para começar, parece muito provável que teremos mais de uma instância de cada serviço. Ter várias instâncias de um serviço permite lidar com mais carga, além de aumentar a robustez de seu sistema, pois a falha de uma única instância será mais facilmente tolerada. Desse modo, teremos possivelmente uma ou mais instâncias de `Fatura` conversando com uma ou mais instâncias de `Pedido`. Exatamente como a comunicação entre essas instâncias será tratada dependerá da natureza do método de comunicação, mas, se partirmos do pressuposto de que, nessa situação, utilizaremos alguma forma de API baseada em HTTP, um balanceador de carga (load balancer) seria suficiente para cuidar do encaminhamento das requisições para diferentes instâncias, conforme vemos na Figura 8.2.

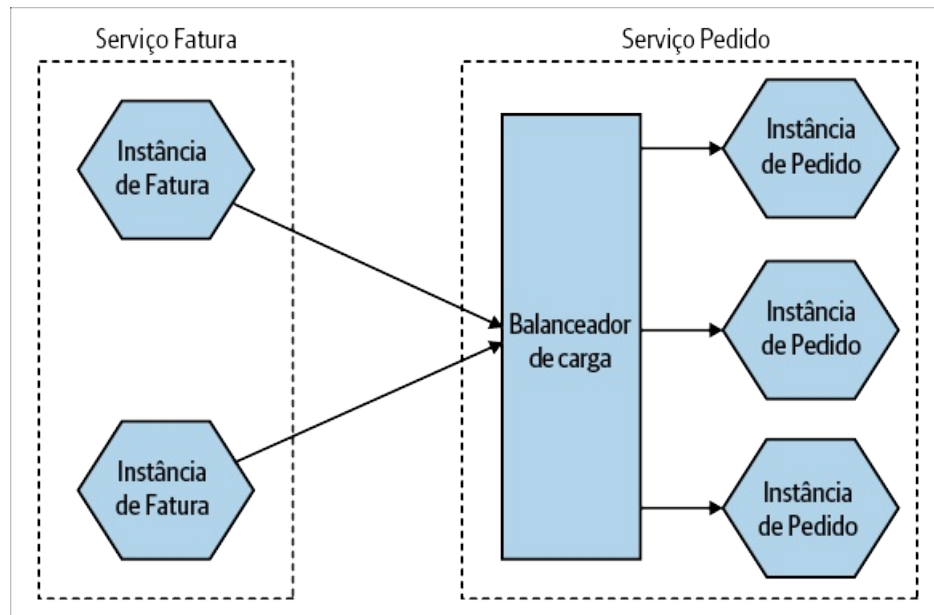


Figura 8.2 – Utilizando um balanceador de carga para mapear requisições a instâncias específicas do microsserviço Pedido.

O número de instâncias que você vai querer dependerá da natureza de sua aplicação – é preciso avaliar a redundância necessária, os níveis de carga esperados e outros aspectos como esses para determinar um número viável com o qual trabalhar. Talvez seja preciso também levar em consideração o local em que essas instâncias serão executadas. Se houver várias instâncias de um serviço por motivos de robustez, é provável que você queira garantir que essas instâncias não estejam todas no mesmo hardware subjacente. Indo além, isso poderia exigir que houvesse diferentes instâncias distribuídas não só entre várias máquinas, mas também em data centers distintos a fim de oferecer proteção contra a indisponibilidade de um data center inteiro. Esse cenário pode levar a uma topologia de implantação como a que vemos na Figura 8.3.

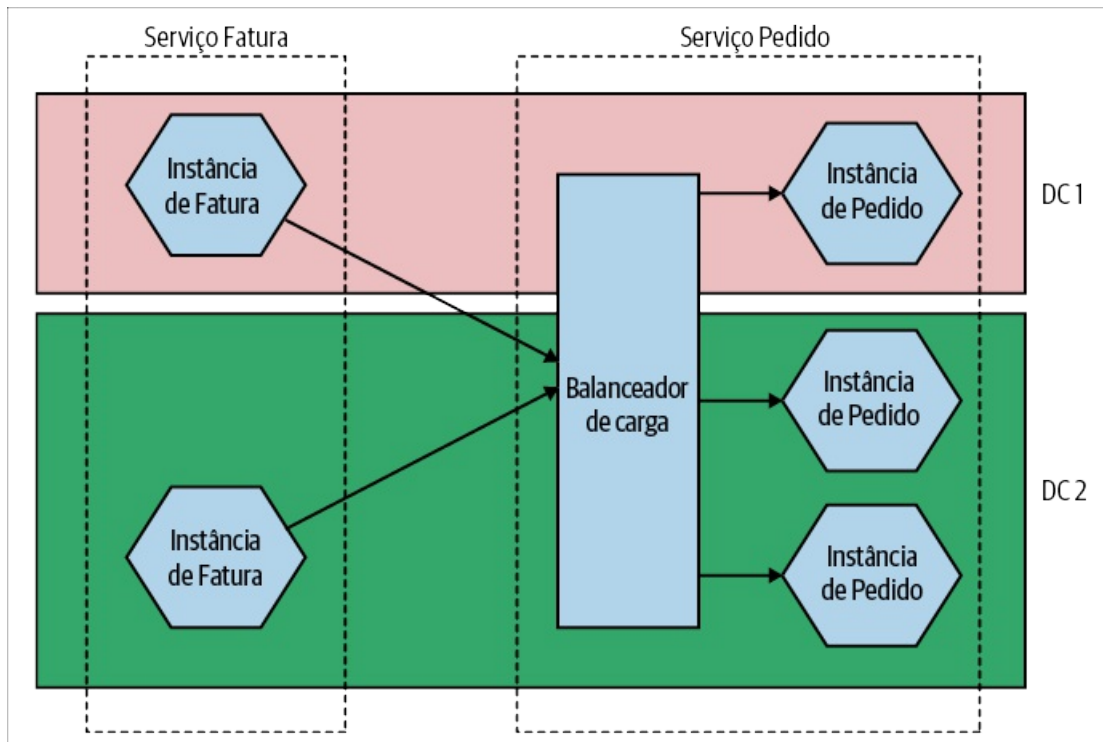


Figura 8.3 – Distribuindo instâncias em vários data centers distintos.

Parece haver aí um excesso de precaução – quais seriam as chances de um data center inteiro ficar indisponível? Bem, não posso responder a essa pergunta para todas as situações, mas, ao menos quando trabalhar com os principais provedores de nuvem, esse é um fato que você deve absolutamente levar em consideração. Quando se trata de algo como uma máquina virtual gerenciada, nem a AWS nem o Azure ou o Google fornecerá um SLA para uma única máquina, ou um SLA para uma única zona de disponibilidade (que é o equivalente mais próximo a um data center para esses provedores). Na prática, isso significa que qualquer solução que você implantar deverá estar distribuída em várias zonas de disponibilidade.

Banco de dados

Dando um passo além, há outro componente importante que ignoramos até agora – o banco de dados. Conforme já discutimos, queremos que um microserviço oculte seu gerenciamento de estado interno, portanto qualquer banco de dados utilizado por um microserviço para gerenciar seu estado será considerado como oculto dentro do microserviço. Isso nos

leva ao mantra constantemente repetido “não compartilhe bancos de dados”, que espero já ter sido suficientemente defendido a essa altura.

Entretanto, como isso funciona quando consideramos o fato de termos várias instâncias de microserviços? Cada *instância* do microserviço deverá ter o próprio banco de dados? Em uma só palavra, a resposta é não. Na maioria dos casos, se eu acessar qualquer instância do meu serviço Pedido, quero ser capaz de obter informações sobre o mesmo pedido. Assim, será necessário ter um certo nível de compartilhamento de estados entre as diferentes instâncias do mesmo serviço lógico. A Figura 8.4 mostra isso.

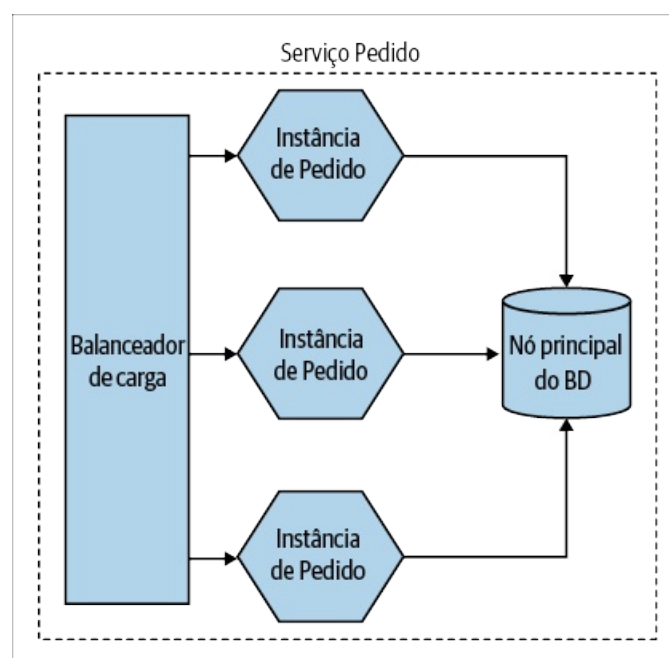


Figura 8.4 – Várias instâncias do mesmo microserviço podem compartilhar um banco de dados.

Mas isso não violaria nossa regra de “não compartilhar bancos de dados”? Na verdade, não. Uma de nossas principais preocupação é que, ao compartilhar um banco de dados entre vários microserviços diferentes, a lógica associada ao acesso e à manipulação desse estado agora estaria espalhada em diferentes microserviços. Nesse caso, porém, os dados são compartilhados por diferentes instâncias do *mesmo* microserviço. A lógica para acessar e manipular estados continua em um único microserviço lógico.

Implantação e escalabilidade dos bancos de dados

Assim como no caso de nossos microsserviços, até agora falamos do banco de dados principalmente de um ponto de vista lógico. Na Figura 8.3, ignoramos quaisquer preocupações com as necessidades de redundância ou de escalabilidade do banco de dados subjacente.

Falando de modo geral, a implantação física de um banco de dados pode ser feita por meio de várias máquinas, por uma série de motivos. Um exemplo comum seria para dividir a carga de leituras e escritas entre um nó principal e um ou mais nós criados somente para leitura (esses nós geralmente são chamados de réplicas para leitura). Se implantássemos essa ideia para o nosso serviço *Pedido*, poderíamos acabar ficando com uma situação igual àquela exibida na Figura 8.5.

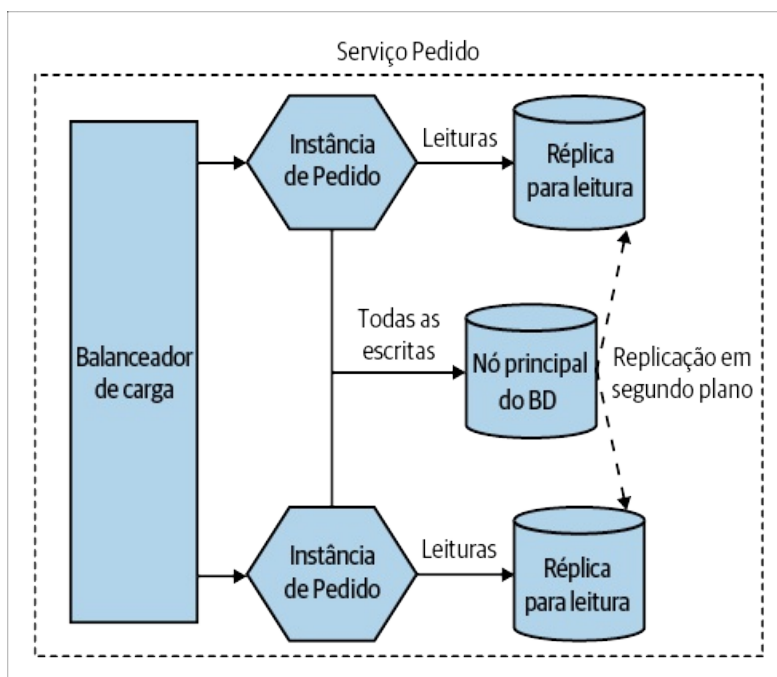


Figura 8.5 – Utilizando réplicas de leitura para distribuição de carga.

Todo o tráfego somente de leitura será direcionado para um dos nós com uma réplica para leitura, e o tráfego de leitura poderia ser escalado no futuro, com o acréscimo de outros nós para leitura. Por causa do modo como os bancos de dados relacionais funcionam, é mais difícil escalar as escritas com o acréscimo de outras máquinas (em geral, modelos de sharding são necessários, o que aumenta a complexidade), portanto

encaminhar o tráfego somente de leitura para essas réplicas de leitura geralmente permitirá que o nó de escrita tenha mais capacidade disponível, possibilitando que o sistema seja escalado.

A esse quadro complexo, soma-se o fato de que a mesma infraestrutura de banco de dados pode oferecer suporte para vários bancos de dados isolados do ponto de vista lógico. Desse modo, os bancos de dados de *Fatura* e de *Pedido* poderiam ambos ser atendidos pela mesma engine de banco de dados e o mesmo hardware subjacentes, como vemos na Figura 8.6. Isso pode proporcionar vantagens significativas – permite que o mesmo hardware possa servir alternadamente a vários microserviços, diminuir os custos com licenças, além de ajudar a reduzir o trabalho relacionado ao gerenciamento dos próprios bancos de dados.

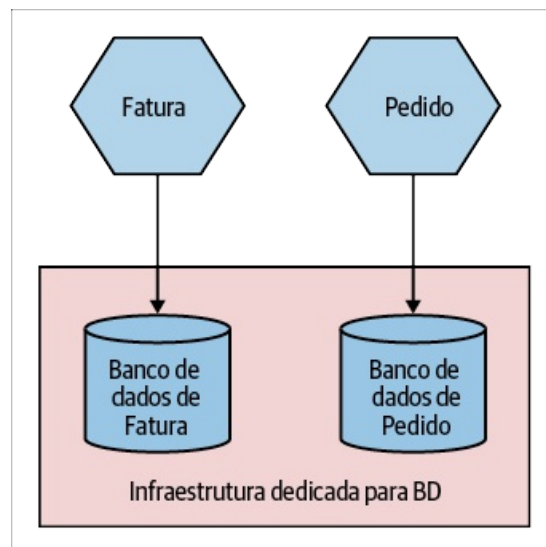


Figura 8.6 – A mesma infraestrutura física de banco de dados hospedando dois bancos de dados isolados do ponto de vista lógico.

O aspecto importante a ser percebido nesse caso é que, embora esses dois bancos de dados possam executar no mesmo hardware e na mesma engine de banco de dados, eles continuam sendo bancos de dados isolados do ponto de vista lógico. Eles não podem interferir um no outro (a menos que você permita). Outro aspecto importante a ser considerado é que, se essa infraestrutura compartilhada de bancos de dados falhar, poderá haver impacto em vários microserviços, e isso poderia ser catastrófico.

Com base em minha experiência, posso dizer que empresas que

administram a própria infraestrutura e executam “on-prem” (localmente) tendem a ter muito mais chances de possuir vários bancos de dados diferentes hospedados em uma infraestrutura compartilhada em virtude dos custos mencionados antes. Fazer o provisionamento e o gerenciamento de um hardware é complicado (e, ao menos historicamente, é menos provável que os bancos de dados sejam executados em infraestruturas virtualizadas), portanto você vai querer ter menos tarefas desse tipo.

Por outro lado, é muito *mais* provável que equipes que trabalhem com provedores de nuvem pública façam o provisionamento de uma infraestrutura dedicada para banco de dados para cada microsserviço, como vemos na Figura 8.7. Os custos de provisionamento e gerenciamento dessa infraestrutura são muito menores. O RDS (Relational Database Service, ou Serviço de Banco de Dados Relacional) da AWS, por exemplo, é capaz de lidar automaticamente com tarefas como backups, upgrades e failover com várias zonas de disponibilidade, e produtos semelhantes estão disponíveis em outros provedores de nuvem pública. Com isso, o custo de ter uma infraestrutura mais isolada para o seu microsserviço será muito menor, possibilitando que os responsáveis por cada microsserviço tenham mais controle, em vez de terem de depender de um serviço compartilhado.

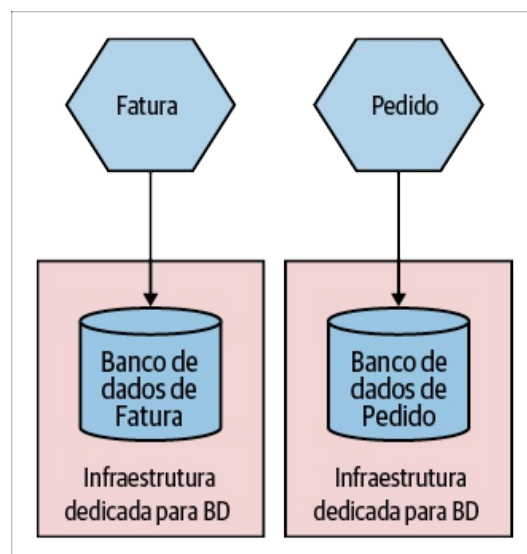


Figura 8.7 – Cada microsserviço fazendo uso de sua própria infraestrutura dedicada para BD.

Ambientes

Ao implantar seu software, ele executará em um ambiente. Em geral, cada ambiente terá propósitos distintos, e o número exato de ambientes que você poderá ter variará bastante de acordo com o modo como seu software é desenvolvido e de como será implantado para o usuário final. Alguns ambientes terão dados de produção, enquanto outros não. Alguns ambientes podem conter todos os serviços; outros poderão ter apenas alguns, com qualquer serviço ausente sendo substituído por serviços simulados para testes.

Em geral, pensamos em nosso software como algo que passa por uma série de ambientes anteriores à produção, cada um servindo a determinados propósitos, a fim de permitir que o software seja desenvolvido e testado de modo que saibamos se ele está pronto para o ambiente de produção – esse assunto foi explorado na seção “Relação de custo-benefício e ambientes”, na página [245](#). Do notebook de um desenvolvedor a um servidor de integração contínua, ou um ambiente de testes integrados e outros – a natureza e a quantidade exata de ambientes dependerão de uma série de fatores, mas serão determinados principalmente pela forma como você decide desenvolver seu software. Na Figura 8.8, vemos um pipeline para o microsserviço `catálogo` da MusicCorp. O microsserviço passa por diferentes ambientes, até finalmente chegar em um ambiente de produção, no qual nossos usuários conseguirão utilizar o novo software.

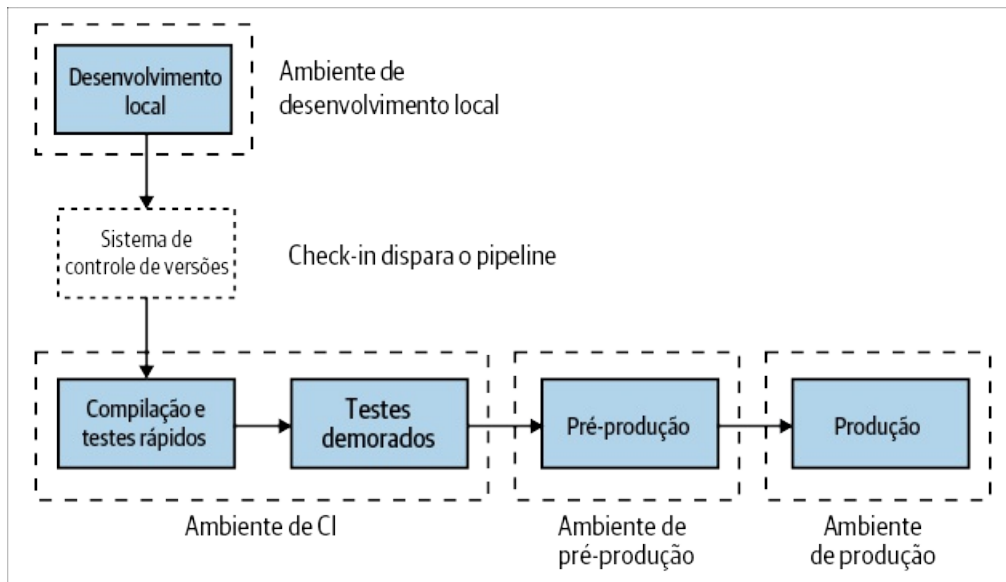


Figura 8.8 – Diferentes ambientes usados em diferentes partes do pipeline.

O primeiro ambiente no qual nosso microserviço executará será em qualquer lugar em que o desenvolvedor trabalhe com o código antes de fazer seu check-in – provavelmente, será em seu notebook local. Após o commit do código, o processo de CI será disparado com os testes rápidos. Tanto na etapa de testes rápidos como na de testes demorados, há implantações em nosso ambiente de CI. Se o microserviço passar pelos testes demorados, ele será implantado no ambiente de pré-produção a fim de possibilitar uma verificação manual (a qual é totalmente opcional, mas continua sendo importante para muitas pessoas). Se passar na verificação manual, o microserviço será então implantado no ambiente de produção.

O ideal seria que cada ambiente nesse processo fosse uma cópia exata do ambiente de produção. Isso nos deixaria mais confiantes de que nosso software funcionará quando chegar ao ambiente de produção. No entanto, em geral não podemos realmente nos dar ao luxo de executar várias cópias do nosso ambiente de produção completo por causa do custo elevado que isso implica.

Também queremos configurar os ambientes nesse processo de modo a permitir feedbacks rápidos. É essencial que saibamos o mais cedo possível se o nosso software funciona ou não, para que possamos fazer correções rápidas caso seja necessário. Quanto antes soubermos de um problema em

nosso software, mais rápido será corrigi-lo, e menor será o impacto da falha. É muito melhor encontrar um problema em nosso notebook local do que o detectar nos testes de pré-produção, mas, de modo semelhante, encontrar um problema nos testes de pré-produção poderia ser muito melhor para nós do que detectar uma falha no ambiente de produção (entretanto, exploraremos alguns prós e contras importantes relacionados a essa questão no Capítulo 9).

Isso significa que ambientes mais próximos do desenvolvedor serão configurados para fornecer feedbacks rápidos, o que pode comprometer o seu “nível de semelhança” com o ambiente de produção. Contudo, à medida que os ambientes se aproximam do ambiente de produção, queremos que eles sejam cada vez mais parecidos com esse ambiente a fim de garantir que iremos detectar qualquer problema.

Como um exemplo simples desse cenário na prática, vamos rever nosso exemplo anterior com o serviço `catálogo` e observar os diferentes ambientes. Na Figura 8.9, nosso serviço está implantado no notebook local do desenvolvedor na forma de uma única instância executando localmente. A construção do software é rápida, porém ele é implantado como uma única instância executando em um hardware muito diferente daquele que esperamos ver no ambiente de produção. No ambiente de CI, implantamos duas cópias do nosso serviço para testes, garantindo que a lógica de nosso balanceador de carga esteja funcionando corretamente. Implantamos as duas instâncias na mesma máquina – isso reduz o custo e deixa o sistema mais rápido, além de nos proporcionar um feedback suficiente nessa etapa do processo.

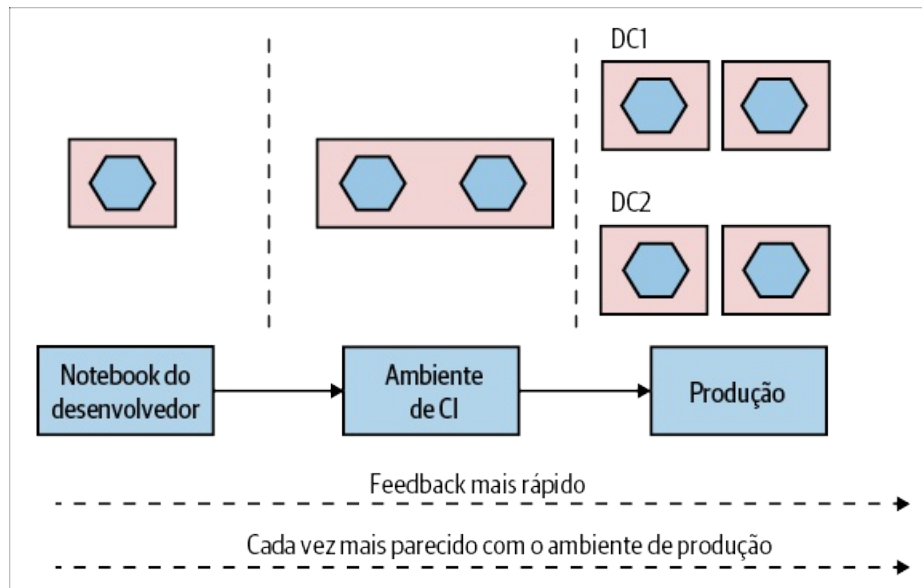


Figura 8.9 – Um microserviço pode variar quanto ao modo de implantação, de um ambiente para outro.

Por fim, no ambiente de produção, nosso microserviço é implantado com quatro instâncias, distribuídas em quatro máquinas, as quais, por sua vez, estão distribuídas em dois data centers distintos.

Esse é somente um exemplo de como poderíamos utilizar os ambientes; a configuração exata que você precisará ter variará bastante, dependendo do que você estiver construindo e de como fará a implantação. Você poderia, por exemplo, ter vários ambientes de produção se precisasse implantar uma cópia de seu software para cada cliente.

O ponto principal, porém, é que a topologia exata de seu microserviço mudará de um ambiente para outro. Desse modo, será necessário encontrar maneiras de alterar a quantidade de instâncias de um ambiente para outro, e poder modificar qualquer configuração que seja específica de um ambiente. As instâncias de seu serviço também devem ser construídas uma e somente uma vez; segue-se daí que qualquer informação que seja específica de um ambiente terá de estar separada do artefato de serviço implantado.

O procedimento para variar a topologia de seu microserviço de um ambiente para outro dependerá bastante do método utilizado para fazer a implantação e do nível de variação das topologias. Se a única diferença de

um ambiente para outro for o número de instâncias do microsserviço, o procedimento poderia ser tão simples quanto parametrizar esse valor a fim de permitir que números diferentes possam ser passados como parte da atividade de implantação.

Assim, para resumir, um único microsserviço lógico pode ser implantado em vários ambientes. De um ambiente para outro, o número de instâncias de cada microsserviço pode variar de acordo com os requisitos de cada ambiente.

Princípios da implantação de microsserviços

Com tantas opções à sua disposição para implantar seus microsserviços, acho importante apresentar alguns princípios fundamentais dessa área. Uma sólida compreensão desses princípios deixará você em uma boa posição, independentemente das escolhas que acabar fazendo. Veremos cada princípio em detalhes logo mais, porém, somente para começar, estas são as ideias básicas que discutiremos:

Execução isolada

Execute as instâncias dos microsserviços de forma isolada, de modo que elas tenham os próprios recursos computacionais, e sua execução não cause impacto nas instâncias de outros microsserviços executando nas proximidades.

Foco na automação

À medida que o número de microsserviços aumentar, a automação será cada vez mais importante. Mantenha o foco na escolha de uma tecnologia que permita um grau elevado de automação e adote a automação como uma parte fundamental de sua cultura.

Infraestrutura como código

Crie uma representação da configuração de sua infraestrutura para facilitar a automação e possibilitar o compartilhamento de informações. Armazene esse código no sistema de controle de versões para permitir que os ambientes sejam recriados.

Implantação sem downtime

Vá um passo além nas implantações independentes e garanta que a implantação de uma nova versão de um microsserviço possa ser feita sem nenhum downtime para os usuários de seu serviço (sejam eles pessoas ou outros microsserviços).

Gerenciamento de estado desejado

Utilize uma plataforma que mantenha o seu microsserviço em um estado definido, iniciando novas instâncias se for necessário, na eventualidade de haver alguma falha ou um aumento de tráfego.

Execução isolada

Você poderia ficar tentado, sobretudo no início de sua jornada para os microsserviços, a colocar todas as instâncias de seu microsserviço em uma única máquina (a qual poderia ser uma única máquina física ou uma única VM), como mostra a Figura 8.10. Do ponto de vista exclusivo do gerenciamento de hosts, esse modelo é mais simples. Em um mundo no qual uma equipe gerencia a infraestrutura e outra equipe gerencia o software, a carga de trabalho da equipe de infraestrutura geralmente será uma função da quantidade de hosts que ela tem para gerenciar. Se houver mais serviços em um único host, a carga de trabalho para o gerenciamento desse host não aumentará com o aumento no número de serviços.

Há, porém, alguns desafios com o uso desse modelo. Em primeiro lugar, a monitoração poderia ser mais complicada. Por exemplo, ao monitorar a CPU, terei de monitorar a CPU de um serviço independentemente dos demais? Ou devo me importar com a CPU do host como um todo? Os efeitos colaterais também podem ser difíceis de evitar. Se um serviço estiver sujeito a uma carga significativa, isso poderia acabar reduzindo os recursos disponíveis às outras partes do sistema. Esse foi um problema com o qual a Gilt (<https://oreil.ly/yRrWG>), uma loja online de moda, deparou. Tendo começado com um sistema monolítico usando Ruby on Rails, a Gilt decidiu passar para microsserviços para que fosse mais fácil escalar a aplicação e acomodar melhor o número crescente de desenvolvedores. Inicialmente, muitos microsserviços coexistiam em uma única máquina na Gilt, porém uma carga desigual em um dos microsserviços causava um

impacto negativo em tudo mais que estivesse executando nesse host. Isso fazia com que uma análise de impacto de falhas em hosts também fosse mais complexa – tirar um único host de serviço poderia provocar um grande efeito em cascata.

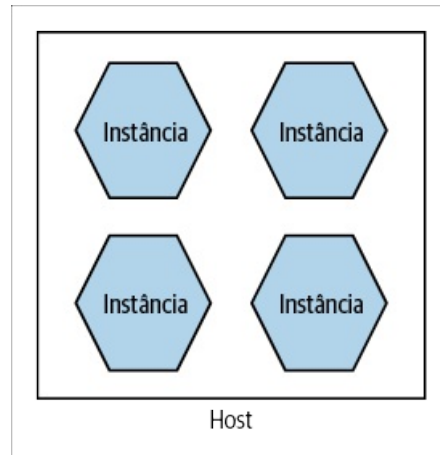


Figura 8.10 – Vários microserviços por host.

A implantação dos serviços também pode se tornar, de certo modo, mais complexa, pois garantir que uma implantação não afete outros serviços levará a outras preocupações. Por exemplo, se cada microserviço tiver dependências distintas (e possivelmente contraditórias), que precisem ser instaladas no host compartilhado, como fazer para que isso funcione?

Esse modelo também pode inibir a autonomia das equipes. Se serviços de diferentes equipes forem instalados no mesmo host, a quem caberá configurar o host para seus serviços? É muito provável que isso acabe sendo feito por uma equipe centralizada, o que significa que exigirá um trabalho adicional de coordenação para que os serviços sejam implantados.

Basicamente, executar várias instâncias de microserviços na mesma máquina (virtual ou física) acabará prejudicando bastante um dos princípios básicos dos microserviços como um todo – a possibilidade de implantações independentes. Segue-se daí, portanto, que devemos realmente executar as instâncias dos microserviços de forma isolada, conforme vemos na Figura 8.11.

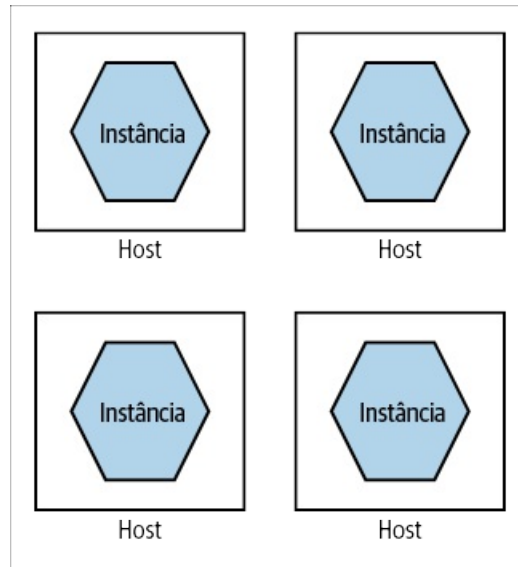


Figura 8.11 – Um único microserviço por host.

Cada instância de microserviço terá o próprio ambiente de execução isolado, poderá instalar as próprias dependências e ter o próprio conjunto de recursos isolados.

Como diz meu colega de longa data Neal Ford, muitas de nossas práticas de trabalho associadas à implantação e ao gerenciamento de hosts são uma tentativa de fazer otimizações em virtude da escassez de recursos. No passado, se quiséssemos que uma máquina estivesse isolada, nossa única opção seria comprar ou alugar outra máquina física. Com frequência, isso exigia um planejamento com bastante antecedência e resultava em um compromisso financeiro de longo prazo. Com base em minha experiência, posso dizer que não é incomum que os clientes provisionem novos servidores somente a cada dois ou três anos, e tentar adquirir novas máquinas fora desse cronograma é difícil. Contudo, as plataformas de computação por demanda reduziram drasticamente os custos dos recursos computacionais, e melhorias na tecnologia de virtualização oferecem mais flexibilidade, mesmo para infraestruturas hospedadas na própria empresa.

Com a containerização se juntando ao cenário, temos mais opções do que nunca para o provisionamento de um ambiente de execução isolado. Como mostra a Figura 8.12, falando de modo geral, vamos de um extremo no qual temos máquinas físicas dedicadas para os nossos serviços, o que nos proporciona o melhor isolamento possível mas provavelmente terá o

custo mais elevado, até a outra extremidade, com os contêineres, que nos dão um isolamento mais fraco, porém tendem a ter um custo menor e ser muito mais rápidos de serem provisionados. Retomaremos algumas das especificidades relacionadas às tecnologias, por exemplo, à containerização, mais adiante neste capítulo.

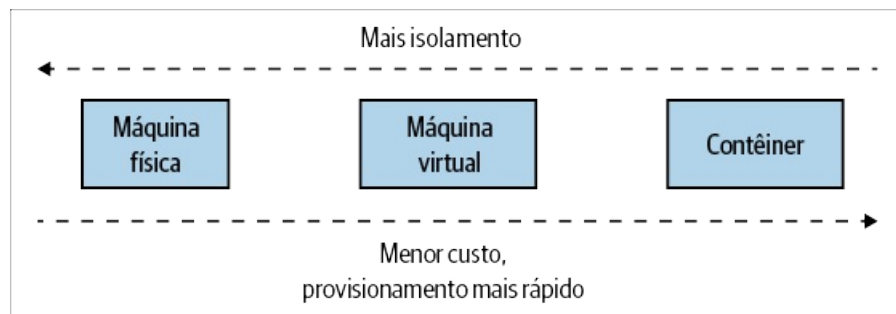


Figura 8.12 – Diferentes relações de custo-benefício relacionadas aos modelos usados para isolamento.

Se você implantasse seus microsserviços em plataformas mais abstratas, por exemplo, no AWS Lambda ou no Heroku, esse isolamento estaria à sua disposição. Dependendo da natureza da plataforma em questão, você poderia provavelmente esperar que, nos bastidores, a instância de seu microsserviço acabasse sendo executada em um contêiner ou em uma VM dedicada.

Em geral, o isolamento proporcionado pelos contêineres melhorou bastante, a ponto de fazer com que sejam uma escolha mais natural para cargas de trabalho com microsserviços. A diferença entre contêineres e VMs no que concerne ao isolamento diminuiu tanto que, para a maioria das cargas de trabalho, os contêineres são “bons o bastante”, o que, em boa medida, é o motivo de serem uma opção tão popular e tenderem a ser a minha opção por norma, na maioria das situações.

Foco na automação

À medida que adicionar mais microsserviços, você terá mais partes com as quais terá de lidar – mais processos, mais itens para configurar, mais instâncias para monitorar. Migrar para microsserviços agrega muita complexidade à área operacional, e se você gerencia seus processos

operacionais de forma manual em sua maior parte, isso significa que, com mais serviços, será necessário ter cada vez mais pessoas para fazer o trabalho.

Desse modo, você precisará ter um foco incansável na automação. Selecione ferramentas e tecnologias que permitam fazer tudo automaticamente; o ideal é fazer isso com vistas a trabalhar com uma infraestrutura como código (que será discutido em breve).

À medida que o número de instâncias de microsserviços aumentar, a automação será cada vez mais importante. Avalie cuidadosamente as tecnologias que permitam ter um grau elevado de automação e adote a automação como uma parte essencial de sua cultura.

A automação é também a forma de garantir que seus desenvolvedores continuem sendo produtivos. Permitir que os próprios desenvolvedores sejam capazes de provisionar serviços individuais ou grupos de serviços é essencial para facilitar suas vidas.

A escolha de uma tecnologia que viabilize a automação começa com as ferramentas utilizadas para gerenciar hosts. Você é capaz de escrever uma linha de código para iniciar ou fazer o shutdown de uma máquina virtual? É capaz de implantar automaticamente o software que você escreveu? Consegue implantar alterações no banco de dados sem que haja uma intervenção manual? Abraçar uma cultura de automação é essencial caso você queira manter as complexidades das arquiteturas de microsserviços sob controle.

Dois casos de estudo sobre a eficácia da automação

Provavelmente será útil apresentar alguns exemplos concretos que expliquem a eficácia de uma boa automação. A empresa australiana realestate.com.au (REA) disponibiliza listas de imóveis para clientes no varejo e para clientes comerciais na Austrália e em outros lugares na região da Ásia-Pacífico. Ao longo de alguns anos, ela veio migrando a sua plataforma para um design com microsserviços distribuídos. Quando iniciou essa jornada, a empresa teve de investir bastante tempo para obter as ferramentas apropriadas aos serviços – facilitando a vida dos

desenvolvedores para que pudessem provisionar as máquinas, implantar seu código e monitorar seus serviços. Muito trabalho prévio, portanto, foi necessário para que pudessem dar início às mudanças.

Nos três primeiros meses desse trabalho, a REA havia conseguido migrar somente dois novos microsserviços para o ambiente de produção, com a equipe de desenvolvimento assumindo total responsabilidade pela construção, implantação e suporte aos serviços. Nos três meses seguintes, entre 10 a 15 serviços foram lançados de modo semelhante. No final de um período de 18 meses, a REA possuía mais de 70 serviços no ambiente de produção.

Esse tipo de padrão também se mostrou apropriado de acordo com as experiências da Gilt, a qual havíamos mencionado antes. Novamente, a automação, sobretudo com ferramentas para ajudar os desenvolvedores, direcionou a explosão no uso de microsserviços pela Gilt. Um ano depois de terem iniciado sua migração para microsserviços, a Gilt tinha aproximadamente 10 microsserviços lançados; por volta de 2012, possuía mais de 100, e em 2014, mais de 450 microsserviços estavam ativos – ou cerca de três microsserviços para cada desenvolvedor na Gilt. Esse tipo de razão entre microsserviços e desenvolvedores não é incomum entre empresas experientes no uso de microsserviços, e o *Financial Times* é uma empresa que apresenta uma taxa semelhante.

Infraestrutura como código

Levando adiante o conceito de automação, a IAC (Infrastructure As Code, ou Infraestrutura como Código) é um conceito segundo o qual sua infraestrutura é configurada com o uso de um código legível por máquinas. Você poderia definir a configuração de seu serviço em um arquivo Chef ou Puppet, ou talvez escrever alguns scripts bash para fazer a configuração – mas, independentemente da ferramenta que acabar usando, seu sistema poderá ser levado a um estado conhecido com o uso de um código-fonte. Sem dúvida, o conceito de IAC poderia ser considerado uma forma de implementar a automação. Acho, porém, que vale a pena considerá-lo como algo à parte, pois está relacionado ao *modo* como a automação deve

ser feita. A infraestrutura como código levou conceitos da área de desenvolvimento de software para a área de operações. Quando definimos nossa infraestrutura utilizando um código, essa configuração pode estar em um sistema de controle de versões, pode ser testada e reproduzida conforme desejarmos. Para saber mais sobre esse assunto, recomendo o livro *Infrastructure as Code*, 2ª edição, de Kief Morris.¹

Teoricamente, poderíamos usar qualquer linguagem de programação para aplicar as ideias de infraestrutura como código, porém há ferramentas especializadas nessa área, por exemplo, o Puppet, o Chef, o Ansible e outras – todas tiveram como base a CFEngine, mais antiga. Essas ferramentas são declarativas: permitem definir, em um formato textual, como você espera que será uma máquina (ou outro conjunto de recursos); quando esses scripts são aplicados, a infraestrutura é levada para esse estado. Ferramentas mais recentes foram além de olhar para a configuração de uma máquina e passaram a visar à configuração de conjuntos inteiros de recursos na nuvem – o Terraform tem feito bastante sucesso nessa área, e estou animado com o potencial do Pulumi, que tem como propósito fazer algo semelhante, embora permitindo que as pessoas utilizem linguagens de programação comuns, em vez das linguagens específicas de domínio, geralmente utilizadas por essas ferramentas. O AWS CloudFormation e o AWS Cloud Development Kit (CDK) são exemplos de ferramentas específicas de plataforma – nesse caso, aceitam somente a AWS, embora valha a pena observar que, mesmo que trabalhasse exclusivamente com a AWS, eu ainda preferiria ter a flexibilidade de uma ferramenta multiplataforma como o Terraform.

Ter o código de sua infraestrutura sujeito ao sistema de controle de versões nos dá transparência para saber quem fez qualquer alteração – algo que os auditores adoram. Também facilita reproduzir um ambiente em qualquer dado instante. Isso pode ser particularmente conveniente quando tentamos rastrear problemas. Em um exemplo memorável, um de meus clientes, como parte de um caso que foi levado à justiça, teve de recriar um sistema completo em execução, conforme ele se apresentava em um instante específico alguns anos atrás, incluindo até mesmo os níveis de patch dos sistemas operacionais e os conteúdos dos brokers de mensagens. Se a

configuração do ambiente tivesse sido armazenada em um sistema de controle de versões, a tarefa teria sido muito mais simples – da forma como ocorreu, eles acabaram gastando mais de três meses tentando reconstruir meticulosamente uma réplica de um ambiente de produção anterior, vasculhando emails e notas de lançamento de versões (release notes) para tentar descobrir o que havia sido feito, e por quem. O caso na justiça, que já estava em andamento há muito tempo, ainda não havia sido resolvido na época em que concluí meu trabalho com esse cliente.

Implantação sem downtime

Como você provavelmente está cansado de me ouvir falar, a possibilidade de ter implantações independentes é muito importante. No entanto, essa também não é uma qualidade absoluta. Até que ponto, exatamente, algo é independente? Antes deste capítulo, havíamos olhado para a capacidade de implantações independentes sob a perspectiva de evitar acoplamentos na implementação. Mais cedo neste capítulo, falamos da importância de oferecer um ambiente de execução isolado para uma instância de microsserviço a fim de garantir que ela tenha certo grau de independência no nível da implantação física. Podemos, contudo, ir além.

Implementar a capacidade de implantação sem downtime pode ser um passo enorme para permitir que os microsserviços sejam desenvolvidos e implantados. Sem uma implantação com downtime zero, um trabalho de coordenação com os consumidores upstream poderia ser necessário quando meu software fosse lançado, a fim de alertá-los sobre uma possível interrupção no serviço.

Sarah Wells do *Financial Times* menciona a capacidade de implantação sem downtime como sendo a principal vantagem no que diz respeito a melhorar a velocidade de entrega. Com a confiança de que os lançamentos de versões não interromperão seus usuários, o *Financial Times* conseguiu aumentar drasticamente a frequência dos lançamentos. Além disso, um lançamento de versão sem downtime pode ser muito mais fácil de ser feito em horário comercial. Afora o fato de que fazer isso melhora a qualidade de vida das pessoas envolvidas no lançamento da versão (em comparação

com trabalhar à noite e nos fins de semana), uma equipe bem descansada trabalhando durante o dia tem menos chances de cometer erros e contará com o apoio de vários de seus colegas se houver alguma necessidade de corrigir problemas.

O objetivo nesse caso é que os consumidores upstream não percebam, de forma alguma, que você fez o lançamento de uma nova versão. Tornar isso possível pode depender bastante da natureza de seu microsserviço. Se você já faz uso de uma comunicação assíncrona com auxílio de um middleware entre seu microsserviço e seus consumidores, essa tarefa pode ser trivial para implementar – as mensagens enviadas a você serão entregues quando seu serviço estiver no ar novamente. Se a comunicação usada for síncrona, porém, a situação poderia ser mais complicada.

Conceitos como rolling upgrades (upgrades sem interrupção) poderiam ser convenientes nesse caso, e essa é uma área em que o uso de uma plataforma como o Kubernetes poderia facilitar muito a sua vida. Com um rolling upgrade, seu microsserviço não será totalmente desativado antes de a nova versão ser implantada; em vez disso, o número de instâncias de seu microsserviço será gradualmente reduzido, enquanto o número de instâncias com versões mais recentes de seu software irá aumentar. Contudo, vale a pena observar que, se tudo que você está buscando é uma solução para facilitar as implantações sem downtime, implementar o Kubernetes provavelmente seria um exagero enorme. Algo simples como um método de implantação azul-verde (blue-green deployment) – que exploraremos melhor na seção “Diferenciando implantação de lançamento”, na página [322](#) – pode funcionar de modo igualmente eficaz.

Outros desafios podem surgir, relacionados a lidar com problemas como conexões de longa duração e situações desse tipo. Certamente é verdade que, se um microsserviço já for criado com uma implementação sem downtime em mente, isso provavelmente facilitará muito mais a sua vida, em comparação com partir de uma arquitetura de sistemas existente e tentar introduzir esse conceito posteriormente. Não importa se você conseguirá ou não fazer uma implantação de seus serviços sem downtime desde o princípio, mas, se conseguir chegar lá, com certeza você apreciará

esse nível maior de independência.

Gerenciamento de estado desejado

Gerenciamento de estado desejado é a capacidade de especificar os requisitos de infraestrutura para a sua aplicação, e fazer com que esses requisitos sejam mantidos sem que haja uma intervenção manual. Se o sistema em execução mudar de modo que o estado desejado deixou de existir, a plataforma subjacente executará os passos necessários para trazer o sistema de volta ao estado desejado.

Como um exemplo simples de como o gerenciamento de estado desejado pode funcionar, poderíamos especificar o número de instâncias exigido pelo seu microsserviço, talvez especificando também as quantidades de memória e CPU necessárias a essas instâncias. Uma plataforma subjacente toma essa configuração e a aplica, levando o sistema ao estado desejado. Cabe à plataforma, entre outras tarefas, identificar quais máquinas possuem recursos sobressalentes, que possam ser alocados para executar o número requisitado de instâncias. Como mostra a Figura 8.13, se uma dessas instâncias se tornar inativa, a plataforma reconhecerá que o estado atual não corresponde ao estado desejado e tomará a medida apropriada, iniciando uma instância substituta.

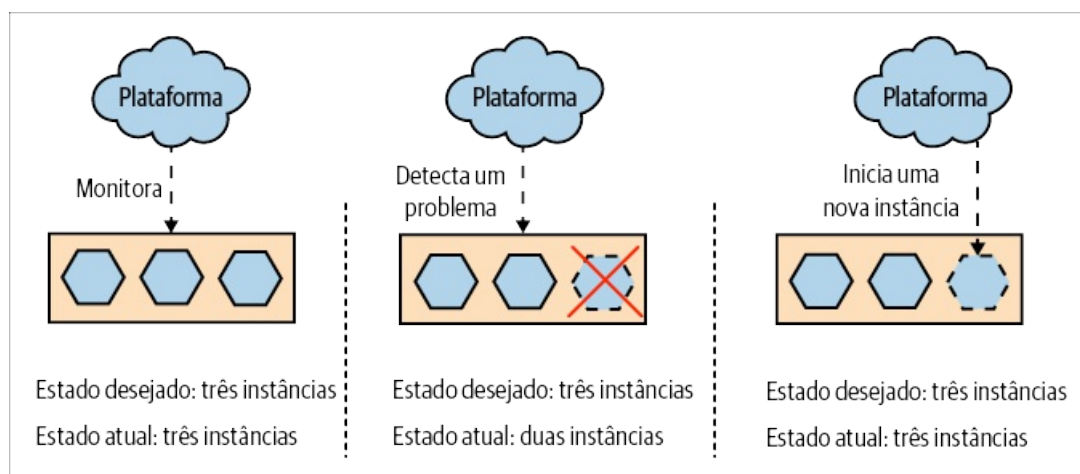


Figura 8.13 – Uma plataforma que oferece um gerenciamento de estado desejado, disparando uma nova instância caso uma instância se torne inativa. A beleza do gerenciamento de estado desejado está no fato de a própria

plataforma gerenciar a maneira como o estado desejado deve ser mantido. Com isso, o pessoal de desenvolvimento e de operações não precisará se preocupar com o modo exato como as tarefas são feitas – eles deverão se concentrar, antes de tudo, somente na criação de uma definição correta para o estado desejado. Significa também que, caso haja algum problema – por exemplo, uma instância se tornar inativa, o hardware subjacente falhar ou houver shutdown de um data center – a plataforma lidará com o problema para você, sem que seja necessária uma intervenção humana.

Embora seja possível criar o próprio conjunto de ferramentas para gerenciamento de estado desejado, em geral utilizamos uma plataforma que já ofereça suporte para esse recurso. O Kubernetes é uma ferramenta desse tipo, que abraça essa ideia; também podemos ter algo semelhante utilizando conceitos como grupos de escalabilidade automática (autoscaling groups) em um provedor de nuvem pública como o Azure ou a AWS. Outra plataforma capaz de oferecer esse recurso é o Nomad (<https://www.nomadproject.io>). De modo diferente do Kubernetes, cujo foco está na implantação e gerenciamento de cargas de trabalho baseadas em contêineres, o Nomad adota um modelo bastante flexível para executar também outros tipos de cargas de trabalho de aplicações, por exemplo, aplicações Java, VMs, jobs Hadoop e outros. Talvez valha a pena considerá-lo caso você queira uma plataforma para gerenciar cargas de trabalho mistas, mas que também faça uso de conceitos como o de gerenciamento de estado desejado.

Essas plataformas sabem da disponibilidade dos recursos subjacentes e são capazes de fazer uma correspondência entre as requisições para um estado desejado e os recursos disponíveis (ou ainda informar que isso não é possível). Na função de operador, você se distanciará da configuração de baixo nível – poderá dizer algo simples como “Quero quatro instâncias distribuídas nos dois data centers”, e contar com sua plataforma para garantir que isso seja feito para você. Diferentes plataformas oferecem diferentes níveis de controle – é possível criar uma definição de estado desejado muito mais complexa, se você quiser.

O gerenciamento de estado desejado ocasionalmente poderá causar

problemas caso você se esqueça de que está fazendo uso desse recurso. Eu me lembro de uma situação na qual estava fazendo shutdown de um cluster de desenvolvimento na AWS antes de ir para casa. Eu estava fazendo shutdown das instâncias de máquinas virtuais gerenciadas (disponibilizadas pelo produto EC2 da AWS) com vistas a economizar – elas não seriam utilizadas durante a noite. No entanto, percebi que, assim que matava uma das instâncias, outra instância surgia. Demorei um tempo para perceber que eu havia configurado um grupo de escalabilidade automática (autoscaling group) para garantir que houvesse um número mínimo de máquinas. A AWS via uma instância morrer e iniciava uma substituta. Demorei quinze minutos nessa brincadeira até perceber o que estava acontecendo. O problema é que pagávamos o EC2 por hora. Ainda que uma instância executasse apenas por um minuto, éramos cobrados por uma hora completa. Assim, minhas idas e vindas no final do dia acabavam tendo um custo bem alto. De certa forma, esse era um sinal de sucesso (pelo menos, é isso que eu dizia a mim mesmo) – havíamos configurado o grupo com escalabilidade automática fazia um tempo, e ele havia funcionado tão bem a ponto de termos esquecido de sua existência. Foi somente uma questão de escrever um script para desativar o grupo de escalabilidade automática como parte do shutdown do cluster para que o problema fosse corrigido a partir de então.

Pré-requisitos

Para tirar proveito do gerenciamento de estado desejado, a plataforma precisa ter alguma forma de iniciar instâncias de seu microserviço automaticamente. Assim, ter uma implantação totalmente automatizada para as instâncias de microserviços é claramente um pré-requisito para o gerenciamento de estado desejado. Além disso, você talvez tenha de considerar com atenção o tempo que suas instâncias demoram para iniciar. Se você utiliza o gerenciamento de estado desejado para garantir que haja recursos computacionais suficientes para lidar com a carga de usuários, então, se uma instância morrer, você vai querer que uma instância substituta preencha a lacuna o mais rápido possível. Se o provisionamento de uma nova instância demorar muito, talvez seja preciso ter uma reserva

de capacidade para lidar com a carga caso uma instância se torne inativa, de modo que você tenha uma folga suficiente para poder iniciar uma nova cópia.

Embora você mesmo possa criar uma solução para fazer o gerenciamento de estado desejado, não estou convencido de que esse seja um bom uso de seu tempo. Se quiser abraçar esse conceito, acho que seria melhor adotar uma plataforma que já o inclua como um conceito de primeira classe. Como isso implica um esforço para entender o que seria uma nova plataforma de implantação, com todas as ideias e ferramentas associadas a ela, talvez você queira postergar a adoção de um gerenciamento de estado desejado até ter alguns microsserviços plenamente em execução. Essa estratégia permitirá que você tenha familiaridade com o básico sobre os microsserviços, antes de ficar sobrecarregado com uma nova tecnologia. Plataformas como o Kubernetes realmente ajudam quando há muitos itens para gerenciar – se você tem apenas alguns processos com que se preocupar, poderá esperar um pouco mais para adotar essas ferramentas.

GitOps

O GitOps, um conceito razoavelmente recente, trazido inicialmente pela Weaveworks, reúne os conceitos de gerenciamento de estado desejado e de infraestrutura como código. O GitOps foi originalmente concebido no contexto de trabalho com o Kubernetes, e esse é o foco das ferramentas associadas, embora, sem dúvida, ele descreva um fluxo de trabalho que outras pessoas já usavam antes.

Com o GitOps, o estado desejado para a sua infraestrutura é definido com um código e armazenado em um sistema de controle de versões. Quando mudanças são feitas nesse estado desejado, algumas ferramentas garantem que esse estado atualizado seja aplicado no sistema em execução. A ideia é oferecer um fluxo de trabalho simplificado aos desenvolvedores para trabalhar com suas aplicações.

Se você já utilizou ferramentas de configuração de infraestrutura como o Chef ou o Puppet, já deve ter familiaridade com esse modelo para gerenciar uma infraestrutura. Ao usar o Chef Server ou o Puppet Master, tínhamos

um sistema centralizado, capaz de implantar as mudanças dinamicamente quando elas eram feitas. A diferença com o GitOps é que essa ferramenta utiliza os recursos do Kubernetes para ajudar a gerenciar as aplicações, e não somente a infraestrutura.

Ferramentas como o Flux (<https://oreil.ly/YWS1T>) estão facilitando muito a adoção dessas ideias. Vale a pena observar, é claro, que, embora as ferramentas possam facilitar as mudanças no seu modo de trabalhar, elas não podem forçar você a adotar novas abordagens. Falando de outra maneira, somente o fato de você ter o Flux (ou outra ferramenta GitOps) não implica que você vai adotar as ideias de gerenciamento de estado desejado ou de infraestrutura como código.

Se você está no mundo do Kubernetes, adotar uma ferramenta como o Flux e os fluxos de trabalho que ele promove poderão muito bem agilizar a introdução de conceitos como gerenciamento de estado desejado e infraestrutura como código. No entanto, certifique-se de que não perderá de vista os objetivos dos conceitos subjacentes, nem se deixe cegar por todas as novas tecnologias existentes nessa área!

Opções para implantação

Quando se trata de abordagens e ferramentas que podemos usar com as cargas de trabalho de nossos microsserviços, há uma *infinitude* de opções. No entanto, devemos considerar essas opções com base nos princípios que acabei de apresentar. Queremos que nossos microsserviços executem isoladamente, e o ideal é que sejam implantados de modo a evitar que haja downtime. Queremos que as ferramentas que escolhermos permitam adotar uma cultura de automação, definir nossa infraestrutura e a configuração das aplicações por meio de código, e o ideal é que gerenciem também o estado desejado para nós.

Vamos resumir rapidamente as diversas opções de implantação antes de analisar até que ponto elas abarcam essas ideias:

Máquina física

Uma instância do microsserviço é implantada diretamente em uma

máquina física, sem virtualização.

Máquina virtual

Uma instância do microsserviço é implantada em uma máquina virtual.

Contêiner

Uma instância do microsserviço executa em um contêiner separado, em uma máquina virtual ou física. O runtime desse contêiner pode ser gerenciado por uma ferramenta de orquestração de contêineres como o Kubernetes.

Contêiner de aplicações

Uma instância do microsserviço é executada em um contêiner de aplicações que gerencia outras instâncias de aplicações, geralmente no mesmo runtime.

PaaS (Platform as a Service, ou Plataforma como Serviço)

Uma plataforma com um nível mais alto de abstração é usada para implantar instâncias de microsserviços, em geral abstraindo todos os conceitos dos servidores subjacentes usados para executar seus microsserviços. Exemplos incluem o Heroku, o Google App Engine e o AWS Beanstalk.

FaaS (Function as a Service, ou Função como Serviço)

Uma instância do microsserviço é implantada como uma ou mais funções, executadas e gerenciadas por uma plataforma subjacente como o AWS Lambda ou o Azure Functions. Sem dúvida, o FaaS é um tipo específico de PaaS, porém merece ser explorado individualmente, considerando a recente popularidade da ideia e as questões que suscita quanto ao mapeamento de um microsserviço para um artefato implantado.

Máquinas físicas

É uma opção cada vez mais rara, mas você poderá se ver implantando microsserviços *diretamente* em máquinas físicas. Por “diretamente”, quero dizer que não há nenhuma camada de virtualização nem de containerização entre você e o hardware subjacente. Isso tem se tornado

cada vez menos comum por alguns motivos diferentes. Em primeiro lugar, implantar diretamente em um hardware físico pode resultar em uma baixa utilização de seus equipamentos. Se eu tiver uma única instância de um microsserviço executando em uma máquina física e utilizar somente metade da CPU, da memória e dos recursos de E/S disponibilizados pelo hardware, os recursos restantes serão desperdiçados. Esse problema levou à virtualização da maior parte da infraestrutura de computação, permitindo a coexistência de várias máquinas virtuais na mesma máquina física. A virtualização possibilita uma maior utilização de sua infraestrutura, o que resulta em vantagens evidentes no que concerne a um custo menor.

Se você tem acesso direto a um hardware físico sem a opção de virtualização, a tentativa seria colocar vários microsserviços na mesma máquina – é claro que isso violaria o princípio do qual falamos, que diz respeito a ter um *ambiente de execução isolado* para seus serviços. Poderíamos utilizar ferramentas como o Puppet ou o Chef para configurar a máquina – ajudando a implementar uma infraestrutura como código. O problema é que, se estivermos trabalhando somente no nível de uma única máquina física, implementar conceitos como gerenciamento de estado desejado, implantação sem downtime e outros exige que trabalhem em um nível mais alto de abstração, utilizando algum tipo de camada superior de gerenciamento. Esses tipos de sistemas são comumente utilizados em conjunto com máquinas virtuais, algo que exploraremos melhor em breve.

Em geral, implantar microsserviços diretamente em máquinas físicas é algo que raramente vemos hoje em dia, e é provável que haja alguns requisitos (ou restrições) muito específicos no seu caso para justificar essa abordagem, diante da flexibilidade cada vez maior que tanto a virtualização ou a containerização podem proporcionar.

Máquinas virtuais

A virtualização transformou os data centers, permitindo que dividíssemos máquinas físicas já existentes em máquinas virtuais menores. A virtualização tradicional, como aquela oferecida pelo VMware ou utilizada pelos principais provedores de nuvem, gerenciava uma infraestrutura de

máquinas virtuais (como o serviço EC2 da AWS), trazendo enormes benefícios no aumento da utilização da infraestrutura computacional, ao mesmo tempo que reduziu o overhead do gerenciamento de hosts.

Basicamente, a virtualização permite dividir uma máquina subjacente em várias máquinas “virtuais” menores que atuam como servidores comuns para os softwares que executam nas máquinas virtuais. Podemos atribuir partes da CPU, da memória, dos recursos de E/S e da capacidade de armazenamento subjacentes para cada máquina virtual, o que, em nosso contexto, permite ter muito mais ambientes de execução isolados para as instâncias de seus microsserviços em uma única máquina física.

Cada máquina virtual contém um sistema operacional completo e um conjunto de recursos que podem ser utilizados pelo software executado na VM. Isso garante que haverá um grau muito bom de isolamento entre as instâncias se cada uma for implantada em uma VM diferente. Cada instância dos microsserviços é capaz de configurar totalmente o sistema operacional da VM de acordo com as próprias necessidades locais. Contudo, ainda teremos um problema: se o hardware subjacente executando essas máquinas virtuais falhar, poderemos perder várias instâncias de microsserviços. Há maneiras de ajudar a solucionar esse problema em particular, incluindo o uso de gerenciamento de estado desejado, que foi discutido antes.

Custo da virtualização

À medida que colocarmos mais máquinas virtuais no mesmo hardware subjacente, perceberemos que haverá retornos cada vez menores no que diz respeito aos recursos computacionais disponíveis às próprias VMs. Por que isso acontece?

Pense em nossa máquina física como se fosse uma gaveta de meias. Se colocarmos muitas divisórias de madeira em nossa gaveta, poderemos armazenar mais ou menos meias? A resposta é menos: as próprias divisórias também ocupam espaço! Nossa gaveta pode ficar mais fácil de ser manuseada e estará mais organizada; talvez pudéssemos decidir guardar camisetas em um dos espaços agora, em vez de apenas meias, mas ter mais

divisórias implica ter menos espaço no total.

No mundo da virtualização, temos um overhead parecido com o das divisórias de nossa gaveta de meias. Para entender o que causa esse overhead, vamos analisar como a maioria das virtualizações é feita. A Figura 8.14 mostra uma comparação entre dois tipos de virtualização. À esquerda, vemos as diversas camadas envolvidas naquilo que chamamos de *virtualização do tipo 2*; à direita, vemos a *virtualização baseada em contêineres*, que exploraremos em breve.

A virtualização do tipo 2 é o tipo de virtualização implementado pela AWS, VMware, vSphere, Xen e KVM. (A virtualização do tipo 1 refere-se à tecnologia segundo a qual as VMs executam diretamente no hardware, e não com base em outro sistema operacional.) Em nossa infraestrutura física, temos um sistema operacional host. Nesse SO, executamos algo que se chama *hipervisor*, o qual tem duas funções principais. Em primeiro lugar, ele mapeia recursos como CPU e memória do host virtual para o host físico. Em segundo, atua como uma camada de controle, permitindo que manipulemos as máquinas virtuais em questão.

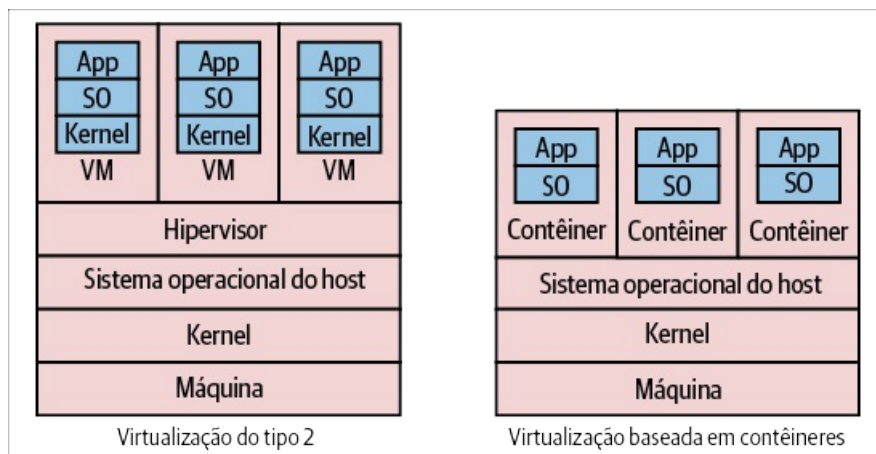


Figura 8.14 – Comparação entre uma virtualização padrão do tipo 2 e contêineres leves.

Dentro das VMs, temos o que parecem ser hosts totalmente distintos. Podem executar seus próprios sistemas operacionais, com seus próprios kernels, e podem ser considerados máquinas quase hermeticamente fechadas, sendo mantidos isolados do host físico subjacente e das demais

máquinas virtuais pelo hipervisor.

O problema com a virtualização do tipo 2 é que o hipervisor, nesse caso, precisa reservar recursos para fazer o seu trabalho. Ele consome CPU, E/S e memória que poderiam ser usados em outros lugares. Quanto mais hosts o hipervisor gerenciar, mais recursos serão necessários. Em determinado momento, esse overhead passará a ser uma limitação para continuar dividindo a sua infraestrutura física. Na prática, significa que, em geral, haverá retornos cada vez menores para o fatiamento de uma máquina física em partes cada vez menores, pois, proporcionalmente, cada vez mais recursos serão consumidos por causa do overhead do hipervisor.

São boas para os microsserviços?

Voltando aos nossos princípios, as máquinas virtuais se saem muito bem no que concerne ao isolamento, porém têm um custo. A facilidade para automação pode variar de acordo com a tecnologia específica em uso – VMs gerenciadas no Google Cloud, no Azure ou na AWS, por exemplo, são fáceis de automatizar por meio de APIs que contam com um bom suporte, e por um ecossistema de ferramentas criadas com base nessas APIs. Além disso, essas plataformas oferecem conceitos como grupos de escalabilidade automática, ajudando a implementar um gerenciamento de estado desejado. Colocar em prática uma implantação sem downtime exigirá mais trabalho, mas, se a plataforma de VM utilizada por você oferecer uma boa API, os blocos de construção estarão à sua disposição. O problema é que muitas pessoas utilizam VMs gerenciadas oferecidas por plataformas tradicionais de virtualização, como aquelas disponibilizadas pela VMware, as quais, embora teoricamente permitam o uso de automação, de modo geral, não são empregadas nesse contexto. Essas plataformas tendem a estar sob o controle centralizado de uma equipe dedicada de operações e, conseqüentemente, haverá restrições quanto à capacidade de automatizá-las diretamente.

Embora os contêineres estejam se provando ser mais populares para cargas de trabalho de microsserviços em geral, muitas empresas têm utilizado máquinas virtuais para executar sistemas de microsserviços em larga escala, com muita eficácia. A Netflix – uma das vitrines para microsserviços –

criou a maior parte de seus microsserviços com base em máquinas virtuais gerenciadas da AWS, usando o EC2. Se você precisar de níveis mais rigorosos de isolamento além do que essas soluções podem oferecer, ou não é capaz de containerizar a sua aplicação, as VMs podem ser uma ótima opção.

Contêineres

Desde a primeira edição deste livro, os contêineres passaram a ser um conceito dominante na implantação de softwares do lado do servidor e, para muitos, são o padrão consolidado pelo mercado para empacotamento e execução de arquiteturas de microsserviços. O conceito de contêiner, popularizado pelo Docker, aliado ao suporte de uma plataforma de orquestração de contêineres como o Kubernetes, passou a ser a escolha preferida de muitas pessoas para executar arquiteturas de microsserviços em escala.

Antes de falarmos do motivo pelo qual isso aconteceu e do relacionamento entre contêineres, Kubernetes e Docker, precisamos explorar antes o que é exatamente um contêiner, além de analisar especificamente como ele difere das máquinas virtuais.

Isolados, porém de modo diferente

Os contêineres surgiram inicialmente em sistemas operacionais do tipo UNIX, e durante muitos anos, eram um prospecto viável somente nesses tipos de sistemas operacionais, por exemplo, no Linux. Embora contêineres Windows sejam uma realidade, foi nos sistemas operacionais Linux que os contêineres causaram o maior impacto até agora.

No Linux, processos são executados por um dado usuário e têm determinados recursos com base no modo como as permissões são definidas. Os processos podem dar origem a outros processos. Por exemplo, se eu iniciar um processo em um terminal, esse processo geralmente será considerado como filho do processo do terminal. A função do kernel do Linux é manter essa árvore de processos, garantindo que somente usuários com permissão possam acessar os processos. Além disso, o kernel do Linux é capaz de atribuir recursos a esses diferentes processos –

tudo isso faz parte da criação de um sistema operacional multiusuário viável, no qual não queremos que as atividades de um usuário causem falhas em outras partes do sistema.

Contêineres que executam na mesma máquina fazem uso do mesmo kernel subjacente (embora haja exceções a essa regra, as quais serão exploradas em breve). Em vez de gerenciar diretamente os processos, podemos pensar em um contêiner como uma abstração de uma subárvore da árvore geral de processos do sistema, com o kernel fazendo todo o trabalho pesado. Esses contêineres podem ter recursos físicos alocados a eles, algo que será tratado pelo kernel para nós. Essa abordagem geral está presente sob diversas formas, por exemplo, no Solaris Zones e no OpenVZ, mas foi com o LXC que essa ideia se tornou predominante nos sistemas operacionais Linux. O conceito de contêineres Linux evoluiu mais ainda quando o Docker passou a oferecer um nível mais alto de abstração sobre os contêineres, no início utilizando o LXC internamente, e então substituindo-o em sua totalidade.

Se observarmos o diagrama de camadas de um host executando um contêiner na Figura 8.14, vemos algumas diferenças quando o comparamos com a virtualização do tipo 2. Em primeiro lugar, não precisamos de um hipervisor. Em segundo, o contêiner não parece ter um kernel – isso porque ele faz uso do kernel da máquina subjacente. Na Figura 8.15, vemos esse detalhe com mais clareza. Um contêiner pode executar o próprio sistema operacional, mas esse sistema faz uso de uma parte do kernel compartilhado – é nesse kernel que está a árvore de processos de cada contêiner. Isso significa que o sistema operacional do nosso host poderia executar o Ubuntu, enquanto nossos contêineres executariam o CentOS, desde que ambos pudessem executar como parte do mesmo kernel subjacente.

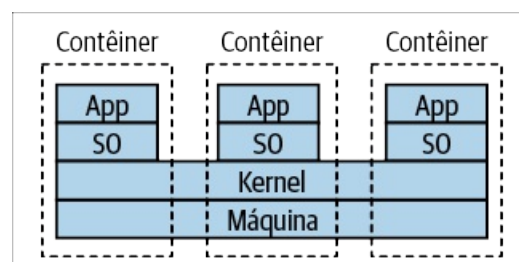


Figura 8.15 – Geralmente os contêineres na mesma máquina compartilham o mesmo kernel.

Com os contêineres, não apenas nos beneficiamos dos recursos economizados por não haver necessidade de um hipervisor, como também ganhamos no que diz respeito ao feedback. Os contêineres Linux são *muito* mais rápidos de provisionar do que as máquinas virtuais completas. Não é incomum que uma VM demore vários minutos para iniciar – com contêineres Linux, porém, a inicialização poderia demorar apenas alguns segundos. Também temos um controle mais minucioso dos próprios contêineres no que concerne à atribuição de recursos, o que facilita bastante ajustar as configurações para tirar o máximo de proveito do hardware subjacente.

Em virtude da natureza mais leve dos contêineres, podemos ter muito mais deles executando no mesmo hardware, em comparação com o que seria possível com as VMs. Ao implantar um serviço por contêiner, como vemos na Figura 8.16, teremos um grau de isolamento de outros contêineres (embora não seja perfeito), e fazê-lo com um custo muito menor do que seria possível se quiséssemos executar cada serviço em sua própria VM. Retomando a nossa analogia anterior com a gaveta de meias, com os contêineres, as divisórias da gaveta de meias são muito mais finas que no caso das VMs, o que significa que uma proporção maior da gaveta será usada para as meias.

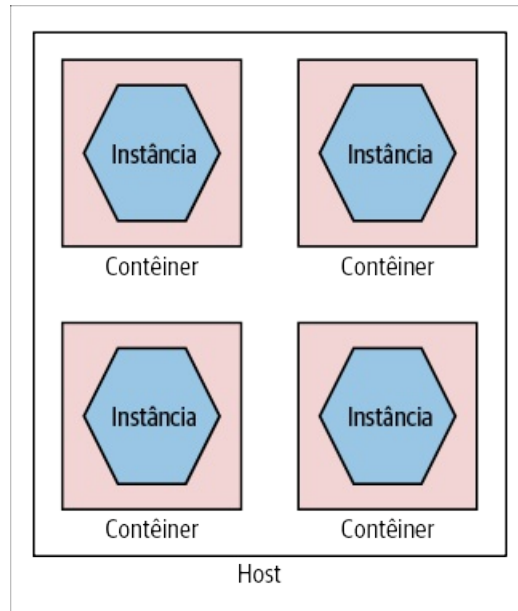


Figura 8.16 – Executando serviços em contêineres separados.

Os contêineres também podem ser usados com uma virtualização completa; na verdade, isso é comum. Já vi mais de um projeto fazer o provisionamento de uma instância grande no AWS EC2, executando aí vários contêineres para tirar o melhor proveito dos dois mundos: uma plataforma de computação efêmera por demanda na forma do EC2, associada a contêineres rápidos, altamente flexíveis, executando nessa plataforma.

Não são perfeitos

Os contêineres Linux, porém, não deixam de ter seus problemas. Suponha que tivéssemos vários microsserviços executando em seus próprios contêineres em um host. Como o mundo externo os veria? Seria necessário ter alguma forma de fazer o roteamento do mundo externo para os contêineres subjacentes, algo que muitos hipervisores fariam para você em uma virtualização usual. Com tecnologias mais antigas como o LXC, essa era uma tarefa com a qual você precisava lidar por conta própria – nessa área, a abordagem do Docker para os contêineres ajudou bastante.

Outra questão para se ter em mente é que esses contêineres podem ser considerados isolados do ponto de vista de recursos – posso alocar conjuntos isolados de CPU, memória e outros recursos para cada contêiner

– mas isso não corresponde necessariamente ao mesmo grau de isolamento obtido com as máquinas virtuais, ou se for o caso, com máquinas físicas separadas. No início, havia uma série de maneiras documentadas e conhecidas pelas quais um processo de um contêiner podia sair por aí interagindo com outros contêineres ou com o host subjacente.

Um volume enorme de trabalho foi investido para solucionar esses problemas, e os sistemas de orquestração de contêineres e os runtimes subjacentes nos contêineres fizeram um bom trabalho para analisar a melhor maneira de executar cargas de trabalho em contêineres, de modo que esse isolamento melhorou, mas você terá de pensar cuidadosamente nos tipos de cargas de trabalho que quer executar. Meu conselho nesse caso é que, em geral, você deve encarar os contêineres como uma ótima maneira de isolar a execução de softwares confiáveis. Se você pretende executar um código escrito por outras pessoas e está preocupado com o fato de um terceiro mal-intencionado tentar burlar o isolamento no nível dos contêineres, será necessário fazer uma análise mais profunda do atual estado da arte para lidar com situações desse tipo – discutiremos parte desse assunto em breve.

Contêineres Windows

Historicamente, os usuários do Windows viam com inveja seus usuários contemporâneos do Linux, pois os contêineres eram algo indisponível ao sistema operacional Windows. Nos últimos anos, porém, isso mudou, e os contêineres agora são um conceito totalmente aceito nesse sistema. O atraso na verdade se devia à capacidade do sistema operacional Windows e do kernel subjacente aceitarem os mesmos tipos de funcionalidades que existiam no território do Linux para que os contêineres funcionassem. Foi com o lançamento do Windows Server 2016 que boa parte desse cenário mudou, e desde então os contêineres Windows têm evoluído.

Um dos primeiros obstáculos na adoção de contêineres Windows era o tamanho do próprio sistema operacional Windows. Lembre-se de que é necessário executar um sistema operacional em cada contêiner; portanto, ao fazer o download da imagem de um contêiner, você fará também o download de um sistema operacional. O Windows, porém, é *grande* – tão

grande a ponto de deixar os contêineres muito pesados, não somente no que concerne ao tamanho das imagens, mas também quanto aos recursos necessários para executá-los.

A Microsoft reagiu a esse problema criando um sistema operacional mais compacto, chamado Windows Nano Server. A ideia é que o Nano Server seria um SO com pouco uso de recursos, capaz de executar algo como instâncias de microsserviços. Além disso, a Microsoft oferece suporte também para um Windows Server Core maior, para auxiliar na execução de aplicações Windows legadas em contêineres. O problema é que esses tipos de sistemas ainda são bem grandes quando comparados aos seus equivalentes Linux – versões iniciais do Nano Server ainda tinham muito mais que 1 GB de tamanho, quando comparadas aos sistemas operacionais Linux com pouco uso de recursos, como o Alpine, que exigem apenas alguns megabytes.

Embora a Microsoft continue tentando reduzir o tamanho do Nano Server, essa disparidade ainda existe. Na prática, porém, como é possível que camadas comuns entre imagens de contêineres sejam armazenadas em cache, esse problema talvez não seja tão grave.

Um aspecto que desperta interesse especial no mundo dos contêineres Windows é o fato de aceitarem diferentes níveis de isolamento. Um contêiner Windows padrão oferece um isolamento de processos, de modo muito parecido com suas contrapartidas no Linux. Com o isolamento de processos, cada contêiner executa em uma parte do mesmo kernel subjacente, o qual gerencia o isolamento entre os contêineres. Com os contêineres Windows, também temos a opção de oferecer mais isolamento, executando os contêineres dentro de sua própria VM Hyper-V. Isso nos proporciona algo mais parecido com o nível de isolamento da virtualização completa, mas o interessante é que você pode escolher entre o Hyper-V ou o isolamento de processos ao iniciar o contêiner – a imagem não precisa ser alterada.

Ter flexibilidade para executar imagens com diferentes tipos de isolamento pode ter suas vantagens. Em algumas situações, seu modelo para ameaças poderia determinar a necessidade de um maior isolamento entre seus

processos em execução, além do que um isolamento simples no nível de processos poderia oferecer. Por exemplo, talvez você execute um código de terceiros “não confiável” junto com seus próprios processos. Em uma situação como essa, ser capaz de executar essas cargas de trabalho como contêineres Hyper-V seria muito conveniente. Observe que, evidentemente, é provável que o isolamento com o Hyper-V cause algum impacto quanto ao tempo de inicialização e tenha um custo de runtime mais próximo do custo de uma virtualização usual.

Linhas indistintas

Há uma tendência cada vez maior de pessoas procurando soluções que ofereçam o melhor isolamento fornecido pelas VMs, e que, ao mesmo tempo, tenham a natureza leve dos contêineres. Exemplos incluem os contêineres Hyper-V da Microsoft, que permitem ter kernels separados, e o Firecracker (<https://oreil.ly/o9rBz>), que, de modo confuso, é chamado de VM baseado em kernel. O Firecracker tem se provado ser popular como um detalhe de implementação de ofertas de serviços como o AWS Lambda, no qual há necessidade de isolar totalmente as cargas de trabalho de diferentes clientes, ao mesmo tempo que há uma tentativa de manter o tempo de inicialização baixo e de reduzir a exigência operacional das cargas de trabalho.

Docker

Os contêineres tinham um uso limitado até o surgimento do Docker ter disseminado o conceito no mercado em geral. O conjunto de ferramentas Docker cuida da maior parte do trabalho relacionado aos contêineres. O Docker gerencia o provisionamento dos contêineres, lida com alguns dos problemas de rede para você, e até mesmo oferece o próprio conceito de registro (registry), que permite armazenar aplicações Docker. Antes do Docker, não tínhamos o conceito de “imagem” para contêineres – esse aspecto, somado a um conjunto muito mais interessante de ferramentas para trabalhar com contêineres, ajudaram a tornar os contêineres muito mais fáceis de usar.

A abstração de imagem do Docker é conveniente para nós, pois os detalhes de como nosso microsserviço é implementado permanecem ocultos.

Fazemos com que os processos de construção (builds) de nosso microsserviço criem uma imagem Docker como um artefato da construção, armazenamos essa imagem no registro do Docker, e pronto. Ao iniciar uma instância de uma imagem Docker, temos um conjunto genérico de ferramentas para gerenciar essa instância, não importa a tecnologia subjacente utilizada – microsserviços escritos em Go, Python, NodeJS ou qualquer que seja a linguagem poderão ser todos tratados do mesmo modo.

O Docker também é capaz de diminuir algumas das desvantagens de executar vários serviços localmente para desenvolvimento e testes. No passado, eu poderia ter empregado uma ferramenta como o Vagrant que permite hospedar várias VMs independentes em minha própria máquina de desenvolvimento. Isso me permitia ter uma VM semelhante à do ambiente de produção executando minhas instâncias de serviço localmente. Contudo, essa era uma abordagem bastante pesada, e havia um limite para a quantidade de VMs que eu poderia executar. Com o Docker, é fácil executá-lo diretamente na minha máquina de desenvolvimento, provavelmente utilizando o Docker Desktop (<https://oreil.ly/g19BE>). Agora posso simplesmente criar uma imagem Docker para a instância de meu microsserviço, ou extrair uma imagem previamente construída e executá-la localmente. Essas imagens Docker podem (e devem) ser idênticas à imagem do contêiner que, posteriormente, será executada no ambiente de produção.

Quando o Docker surgiu inicialmente, seu escopo estava limitado a gerenciar contêineres em uma só máquina. Esse uso era limitado – e se quiséssemos gerenciar contêineres em várias máquinas distintas? Isso é essencial caso você queira manter seu sistema saudável, uma máquina venha a falhar ou se quiser apenas executar uma quantidade suficiente de contêineres para lidar com a carga do sistema. O Docker lançou dois produtos totalmente diferentes para resolver esse problema; de modo confuso, eles se chamam “Docker Swarm” e “Docker Swarm Mode” – quem foi que disse mesmo que era difícil dar nomes às coisas? Na verdade, porém, quando se trata de gerenciar vários contêineres em diversas máquinas, o Kubernetes é insuperável, ainda que você possa utilizar o

conjunto de ferramentas Docker para criar e gerenciar contêineres individuais.

Adequação aos microsserviços

Os contêineres, como conceito, funcionam muito bem para os microsserviços, e o Docker deixou os contêineres muito mais viáveis. Temos nosso isolamento, porém com um custo administrável. Também ocultamos a tecnologia subjacente, permitindo combinar diferentes conjuntos de tecnologias. No entanto, quando se trata de implementar conceitos como gerenciamento de estado desejado, precisaremos de algo como o Kubernetes para cuidar disso para nós.

O Kubernetes é suficientemente importante a ponto de justificar uma discussão mais detalhada, portanto vamos retomá-lo mais adiante neste capítulo. Por enquanto, porém, pense nele apenas como uma forma de gerenciar contêineres distribuídos em várias máquinas, e isso bastará por ora.

Contêineres de aplicações

Se você tem familiaridade com a implantação de aplicações .NET com IIS, ou com aplicações Java em algo como o Weblogic ou o Tomcat, deve conhecer bem o modelo em que várias aplicações ou serviços distintos estão em um único contêiner de aplicações, o qual, por sua vez, está em um único host, conforme vemos na Figura 8.17. A ideia é que o contêiner de aplicações no qual estão seus serviços proporcione vantagens no que diz respeito a uma melhor capacidade de gerenciamento, por exemplo, suporte a clusters para lidar com o agrupamento de várias instâncias, ferramentas de monitoração e aspectos desse tipo.

Essa configuração também pode trazer vantagens na redução do overhead em runtimes para determinadas linguagens. Considere a execução de cinco serviços Java em um único contêiner de servlets Java. Teremos o overhead de uma única JVM. Compare isso com a execução de cinco JVMs independentes no mesmo host ao usar contêineres. Apesar disso, ainda acho que esses contêineres de aplicações têm desvantagens suficientes, e você deve avaliar se eles são realmente necessários.

A primeira das desvantagens é que eles restringem inevitavelmente as opções de tecnologia. Você terá de se comprometer com um conjunto de tecnologias. Isso poderá limitar não só as opções de tecnologia para a implementação do próprio serviço, mas também as opções no que concerne à automação e ao gerenciamento de seus sistemas. Conforme discutiremos em breve, uma das maneiras de lidar com o overhead de gerenciar vários hosts é por meio da automação, portanto restringir nossas opções para solucionar esse problema poderia ser bastante prejudicial.

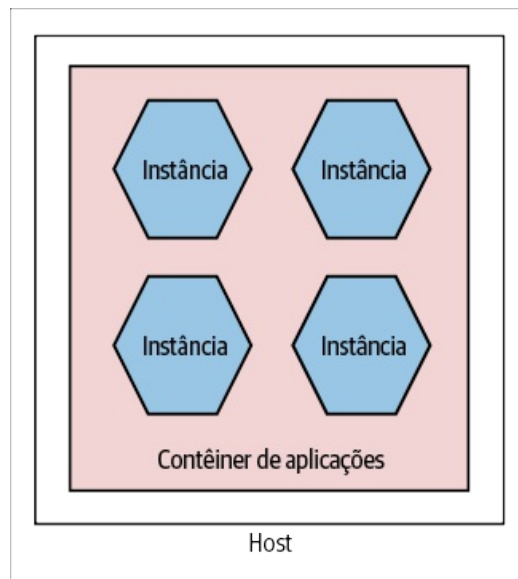


Figura 8.17 – Vários microsserviços por contêiner de aplicações.

Eu também questionaria a importância de alguns recursos oferecidos por esses contêineres de aplicações. Muitos deles anunciam que são capazes de gerenciar clusters com suporte ao compartilhamento de estados de sessão em memória, algo que queremos absolutamente evitar de qualquer modo, por causa dos problemas que isso pode causar na hora de escalar nossos serviços. Além do mais, os recursos de monitoração que eles oferecem não são suficientes se considerarmos os tipos de monitoração conjunta que queremos fazer em um mundo de microsserviços, conforme veremos no Capítulo 10. Muitos também têm tempos de inicialização muito demorados, causando impacto nos ciclos de feedback para os desenvolvedores.

Há também outros conjuntos de problemas. Tentar fazer um

gerenciamento apropriado do ciclo de vida das aplicações em plataformas como a JVM pode ser problemático e mais complexo do que simplesmente reiniciar uma JVM. Analisar o uso de recursos e threads também é muito mais complicado, pois haverá muitas aplicações compartilhando o mesmo processo. Lembre-se também de que, ainda que você desfrute vantagens com contêineres que utilizem tecnologias específicas, elas não serão gratuitas. Além do fato de muitos serem produtos comerciais e, desse modo, terem uma implicação quanto ao custo, essas soluções, por si só, acrescentam um overhead aos recursos.

Em última análise, essa abordagem é, mais uma vez, uma tentativa de otimização em virtude da escassez de recursos, e essa é uma justificativa que talvez não se sustente mais. Independentemente de você decidir ou não ter vários serviços por host como um modelo de implantação, sugiro fortemente que você considere microsserviços que possam ser implantados de modo autocontido como artefatos, com cada instância de microsserviço executando em seu próprio processo isolado.

Basicamente, a falta de isolamento desse modelo é um dos principais motivos pelos quais ele está se tornando cada vez mais raro entre as pessoas que estão adotando arquiteturas de microsserviços.

Plataforma como Serviço (PaaS)

Ao utilizar uma PaaS (Platform as a Service, ou Plataforma como Serviço), trabalhamos com um nível mais alto de abstração em comparação a um único host. Algumas dessas plataformas dependem da utilização de um artefato com uma tecnologia específica, por exemplo, um arquivo WAR Java ou uma gema Ruby, provisionando e executando esse artefato automaticamente para você. Algumas dessas plataformas tentarão lidar com o redimensionamento do sistema, para cima ou para baixo, de modo transparente; outras permitirão que você tenha certo controle sobre a quantidade de nós em que seu serviço poderá executar, mas cuidarão do resto.

Assim como na época em que escrevi a primeira edição deste livro, a maior parte das melhores e mais sofisticadas soluções de PaaS são hospedadas. O

Heroku definiu o padrão de qualidade para oferecer uma interface conveniente ao desenvolvedor e, sem dúvida, tem mantido o padrão de ouro para o PaaS, apesar de seu crescimento limitado em seu conjunto de recursos nos últimos anos. Plataformas como o Heroku não apenas executam a sua instância de aplicação como também oferecem recursos como a execução de instâncias de banco de dados para você – algo que poderá ser muito complicado de implementar por conta própria.

Quando as soluções de PaaS funcionam bem, elas funcionam realmente muito bem. No entanto, se não funcionarem muito bem, em geral você não terá muito controle para ajustá-las internamente e fazer correções. Isso faz parte da relação de custo-benefício que você terá. Com base em minha experiência, eu diria que, quanto mais inteligentes tentam ser as soluções de PaaS, mais elas erram. Já utilizei mais de uma solução PaaS que tentava escalar automaticamente com base no uso das aplicações, porém fazia isso de forma inadequada. Invariavelmente, a heurística que orienta essa inteligência tende a ser personalizada para uma aplicação média, e não para o seu caso de uso específico. Quanto mais diferente do padrão for a sua aplicação, maiores serão as chances de que ela não funcionará bem com uma PaaS.

Como as boas soluções de PaaS cuidam de várias tarefas para você, elas podem ser uma forma excelente de lidar com o maior overhead que teremos como consequência de haver muito mais partes no sistema. Apesar disso, ainda não tenho certeza de que já termos definido os modelos corretos nessa área, e as opções limitadas de auto-hospedagem sinalizam que essa abordagem poderia não ser apropriada para você. Quando escrevi a primeira edição deste livro, eu tinha esperanças de que veríamos um maior crescimento nessa área, porém isso não aconteceu do modo como eu esperava. Por outro lado, acho que o aumento de produtos serverless (sem servidor) oferecidos principalmente pelos provedores de nuvem pública começou a atender a essa necessidade. Em vez de oferecer plataformas caixa-preta para hospedar uma aplicação, eles oferecem soluções turnkey gerenciadas para itens como brokers de mensagens, bancos de dados, armazenagem e outros recursos, permitindo misturar e combinar as partes que quisermos a fim de construir o que precisamos. É com esse pano de

fundo que a Função como Serviço (Function as a Service) – um tipo específico de produto serverless – vem ganhando bastante tração.

Avaliar até que ponto as ofertas de produtos PaaS para microsserviços são apropriadas é difícil, pois esses produtos se apresentam em vários formatos e tamanhos. O Heroku parece ser muito diferente do Netlify, por exemplo, mas ambos poderiam ser apropriados para você como uma plataforma de implantação para seus microsserviços, dependendo da natureza de sua aplicação.

Função como Serviço (FaaS)

Nos últimos anos, a única tecnologia a chegar um pouco perto do Kubernetes a ponto de gerar alguma comoção (ao menos no contexto dos microsserviços) é a tecnologia serverless (sem servidor). Serverless, na verdade, é um termo abrangente para uma série de diferentes tecnologias nas quais, do ponto de vista da pessoa que as utiliza, os computadores subjacentes não importam. Os detalhes relacionados ao gerenciamento e à configuração das máquinas ficarão ocultos a você. Nas palavras de Ken Fromm (<https://oreil.ly/hM2uq>) – que, até onde posso dizer, foi quem cunhou o termo *serverless*:

O termo “serverless” não significa que não há mais servidores envolvidos. Significa apenas que os desenvolvedores não precisam mais pensar tanto assim neles. Os recursos computacionais passam a ser utilizados como serviços, sem a necessidade de gerenciamento de capacidades ou limitações físicas. Os provedores de serviços estão assumindo cada vez mais a responsabilidade pelo gerenciamento de servidores, pela armazenagem de dados e por outros recursos da infraestrutura. Os desenvolvedores poderiam criar as próprias soluções open source, mas isso significa que teriam de gerenciar os servidores, as filas, e tudo mais.

– Ken Fromm, “*Why the Future of Software and Apps Is Serverless*” (Por que o futuro do software e das apps é serverless)

A FaaS (Function as a Service, ou Função como Serviço) se tornou uma parte tão importante do serverless que, para muitas pessoas, os dois termos

são intercambiáveis. Isso é lamentável, pois ofusca a importância de outros produtos serverless, como bancos de dados, filas, soluções para armazenamento e itens semelhantes. Apesar disso, esse uso indistinto dos termos reflete a empolgação gerada pela FaaS, a ponto de ter dominado a discussão.

Foi o produto Lambda da AWS, lançado em 2014, que gerou a empolgação em torno da FaaS. Em certo nível, o conceito é maravilhosamente simples. Fazemos a implantação de um código (uma “função”). Esse código fica dormiente até que algo aconteça para dispará-lo. Você é responsável por decidir a razão do disparo – poderia ser um arquivo que chegue em um determinado local, um item que apareça em uma fila de mensagens, uma chamada que chegue por meio de HTTP, ou outro motivo.

Quando ocorre o disparo da função, ela é executada e, quando terminar, ela será desativada. A plataforma subjacente cuida da inicialização e do encerramento dessas funções por demanda, e lidará com as execuções concorrentes das funções, de modo que seja possível ter várias cópias executando ao mesmo tempo quando for apropriado.

As vantagens, nesse caso, são numerosas. Um código que não estiver executando não terá nenhum custo financeiro – você pagará somente por aquilo que usar. Isso pode fazer da FaaS uma ótima opção para situações em que haja uma carga baixa ou imprevisível. A plataforma subjacente cuida do disparo e do encerramento das funções para você, oferecendo um certo grau elevado de disponibilidade e de robustez implícitos, sem que você tenha de fazer qualquer trabalho. Basicamente, o uso de uma plataforma FaaS, assim como de vários outros produtos serverless, permite reduzir drasticamente o volume de overhead operacional com o qual você teria de se preocupar.

Limitações

Internamente, todas as implementações de FaaS que conheço fazem uso de algum tipo de tecnologia de contêiner. Isso permanece oculto a você – em geral, você não terá de se preocupar em criar um contêiner a ser executado;

bastará disponibilizar seu código com alguma forma de empacotamento. Entretanto, isso implica a ausência de certo grau de controle sobre o que, exatamente, pode ser executado; como resultado, você precisará que o provedor de FaaS ofereça suporte para a linguagem de sua escolha. O Azure Functions fez o melhor trabalho nesse caso, entre os principais provedores de nuvem, aceitando uma variedade enorme de diferentes runtimes, enquanto os produtos Cloud Functions do Google Cloud aceitam poucas linguagens em contrapartida (atualmente – quando este livro foi escrito – o Google aceita somente Go, algumas versões de Node e Python). Vale a pena observar que a AWS atualmente permite que você defina o seu próprio runtime personalizado para suas funções, possibilitando, teoricamente, que você implemente suporte para linguagens que não estejam incluídas no produto, embora isso vá gerar um outro volume de overhead operacional cuja manutenção será necessária.

Essa falta de controle sobre o runtime subjacente também se estende à falta de controle sobre os recursos atribuídos a cada chamada de função. Entre o Google Cloud, o Azure e a AWS, você pode controlar somente a memória concedida a cada função. Por sua vez, isso parece implicar que uma certa quantidade de CPU e de recursos de E/S são atribuídos ao runtime de sua função, mas você não poderá controlar esses aspectos diretamente. Pode significar que você acabará tendo de conceder mais memória para uma função, ainda que ela não precise, somente para que ela tenha a CPU necessária. Em última análise, se você achar que precisa fazer muitos ajustes finos relacionados aos recursos disponíveis às suas funções, acho que, a essa altura pelo menos, a FaaS provavelmente não seria uma ótima opção para você.

Outra limitação da qual você deve estar ciente é que as chamadas de funções podem ter limites quanto ao tempo que podem executar. As funções no Google Cloud, por exemplo, estão atualmente limitadas a nove minutos de execução, enquanto, no AWS Lambda, elas podem executar por até quinze minutos. As funções no Azure podem executar indefinidamente se você quiser (dependendo do tipo de plano adquirido). Pessoalmente, acho que, se você tem funções que executam por muito tempo, provavelmente isso aponta para um tipo de problema para o qual

as funções não são muito apropriadas.

Por fim, a maioria das chamadas de funções é considerada stateless (sem estado). Conceitualmente, isso significa que uma função não pode acessar um estado deixado por uma chamada anterior da função, a menos que esse estado esteja armazenado em outro lugar (por exemplo, em um banco de dados). Isso dificulta ter várias funções encadeadas – considere uma função coordenando uma série de chamadas para outras funções downstream. Uma exceção digna de nota é o Azure Durable Functions (<https://oreil.ly/I6ZSc>), que resolve esse problema de uma maneira realmente interessante. O Durable Functions é capaz de suspender o estado de uma dada função e permitir que ela reinicie do ponto em que a chamada foi deixada – tudo isso é tratado de modo transparente, com o uso de extensões Reactive. Essa é uma solução que eu acho significativamente mais conveniente para o desenvolvedor, em comparação com o Step Functions da AWS, que faz a associação de várias funções utilizando uma configuração baseada em JSON.

WebAssembly (Wasm)

O Wasm é um padrão oficial, originalmente definido para oferecer aos desenvolvedores uma forma de executar programas escritos em diversas linguagens de programação em uma sandbox, em navegadores clientes. Definindo tanto um formato de empacotamento quanto um ambiente de runtime, o objetivo do Wasm é permitir que um código arbitrário seja executado de uma maneira segura e eficaz em dispositivos clientes. Isso permite que aplicações muito mais sofisticadas do lado cliente sejam criadas, utilizando navegadores web comuns. Como um exemplo concreto, a eBay utilizou o Wasm para disponibilizar um software de scanner de código de barras na web, cujo núcleo foi escrito em C++ e que estava anteriormente disponível somente para aplicações Android ou iOS nativas.²

O WASI (WebAssembly System Interface, ou Interface de Sistema WebAssembly) foi definido como uma forma de permitir que o Wasm passasse do navegador para qualquer lugar em que uma implementação compatível com WASI pudesse ser encontrada. Um exemplo disso é a capacidade de executar o Wasm em redes de fornecimento de conteúdo

como o Fastly ou o Cloudflare.

Por causa de sua natureza leve e dos conceitos fortes de sandbox incluídos em sua especificação básica, o Wasm tem o potencial de desafiar o uso dos contêineres como o formato preferido para implantação de aplicações do lado do servidor. No curto prazo, o que provavelmente deve estar detendo o Wasm são as plataformas do lado do servidor disponíveis para executá-lo. Embora possamos, teoricamente, executar o Wasm no Kubernetes, por exemplo, você acabará deixando o Wasm em contêineres, o que, sem dúvida, não fará sentido, pois você terá uma implantação mais leve dentro de um contêiner (comparativamente) mais pesado.

Uma plataforma de implantação do lado do servidor com suporte nativo para WASI provavelmente será necessária para tirar o máximo de proveito do potencial do Wasm. Na teoria, ao menos, um escalonador como o Nomad estaria em uma melhor posição para dar suporte ao Wasm, pois ele aceita um modelo de drivers plugáveis. O tempo dirá!

Desafios

Além das limitações que acabamos de ver, há outros desafios que você poderá enfrentar quando usar a FaaS.

Em primeiro lugar, é importante considerar uma preocupação que muitas vezes surge com a FaaS, que é a noção de tempo de inicialização. Conceitualmente, as funções não ficam executando, a menos que sejam necessárias. Isso significa que elas precisam ser iniciadas para atender a uma requisição de entrada. Para alguns runtimes, iniciar uma nova versão é um processo demorado – esse tempo geralmente é chamado de “inicialização a frio” (cold start time). Os runtimes JVM e .NET sofrem muito com esse problema, portanto um tempo de inicialização a frio para funções que utilizem esses runtimes geralmente poderá ser significativo.

Na verdade, porém, esses runtimes raramente iniciam a frio. Na AWS pelo menos, os runtimes são mantidos “aquecidos” (warm), de modo que as requisições que chegam são atendidas por instâncias que já foram iniciadas e estão em execução. Isso acontece a ponto de ser difícil avaliar o impacto

de uma “inicialização a frio” nos dias de hoje em virtude das otimizações feitas internamente pelos provedores de FaaS. Apesar disso, se essa for uma preocupação sua, ater-se a linguagens cujos runtimes tenham uma inicialização rápida (Go, Python, Node e Ruby vêm à mente) pode anular o efeito desse problema com eficácia.

Por fim, o aspecto de escalabilidade dinâmica das funções pode, na verdade, acabar sendo um problema. As funções são iniciadas quando há um trigger. Todas as plataformas que utilizei têm um limite fixo para o número máximo de chamadas de função concorrentes, que é algo que você talvez tenha de considerar com atenção. Já conversei com mais de uma equipe que teve problema com funções que foram escaladas, sobrecarregando outras partes de sua infraestrutura que não tinham as mesmas propriedades para escalar. Steve Faulkner, da Bustle, apresentou (<https://oreil.ly/tFdCk>) um exemplo desse tipo, em que funções foram escaladas, sobrecarregando a infraestrutura de Redis da Bustle, o que causou problemas no ambiente de produção. Se uma parte de seu sistema puder escalar dinamicamente, mas as demais partes não, talvez você venha a descobrir que essa divergência pode provocar dores de cabeça significativas.

Mapeamento para microsserviços

Até agora em nossas discussões sobre as diversas opções para implantação, o mapeamento de uma instância de microsserviço para um método de implantação era bastante simples. Uma única instância de microsserviço poderia ser implantada em uma máquina virtual, empacotada como um único contêiner, ou até mesmo deixada em um contêiner de aplicações como o Tomcat ou o IIS. Com a FaaS, a situação se torna um pouco mais confusa.

Função por microsserviço. Agora, obviamente, uma única instância de microsserviço pode ser implantada como uma única função, conforme vemos na Figura 8.18. Esse, provavelmente, é um ponto de partida razoável. Com isso, o conceito de uma instância de microsserviço como uma unidade de implantação é mantido, e esse é o modelo que mais exploramos até agora.

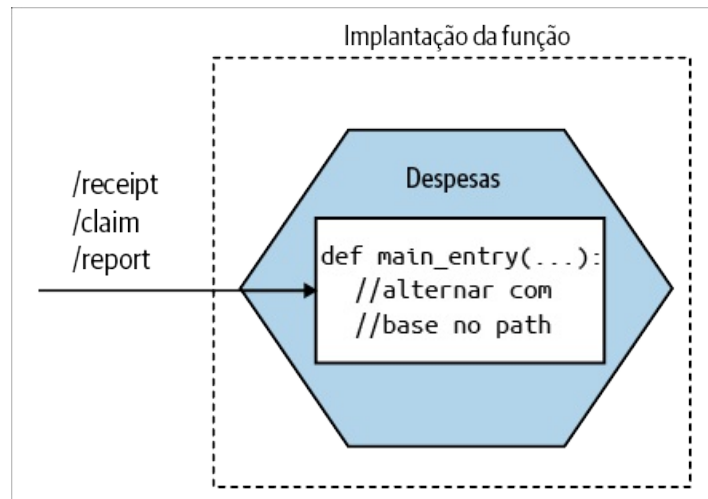


Figura 8.18 – Nosso serviço Despesas está implementado como uma única função.

Quando chamada, a plataforma FaaS disparará um único ponto de entrada em sua função implantada. Isso significa que, se você vai ter a implantação de uma única função para todo o serviço, será necessário ter alguma forma de dispatching a partir desse ponto de entrada, para diferentes funcionalidades em seu microserviço. Se você implementasse o serviço **Despesas** como um microserviço baseado em REST, poderia ter diversos recursos expostos, por exemplo, `/receipt`, `/claim` ou `/report`. Com esse modelo, uma requisição para qualquer um desses recursos chegaria nesse mesmo ponto de entrada, portanto você teria de redirecionar a chamada de entrada para a funcionalidade apropriada com base no path da requisição de entrada.

Função por agregado. Então, como dividiríamos uma instância de microserviço em funções menores? Se você faz uso de um design orientado a domínio, talvez já tenha modelado explicitamente os seus agregados (uma coleção de objetos gerenciados como uma única entidade e que, em geral, refere-se a conceitos do mundo real). Se sua instância de microserviço lida com vários agregados, um modelo que faz sentido para mim é criar uma função para cada agregado, como mostra a Figura 8.19. Isso garante que toda a lógica para um único agregado esteja autocontida na função, facilitando que haja uma implementação consistente para o gerenciamento do ciclo de vida do agregado.

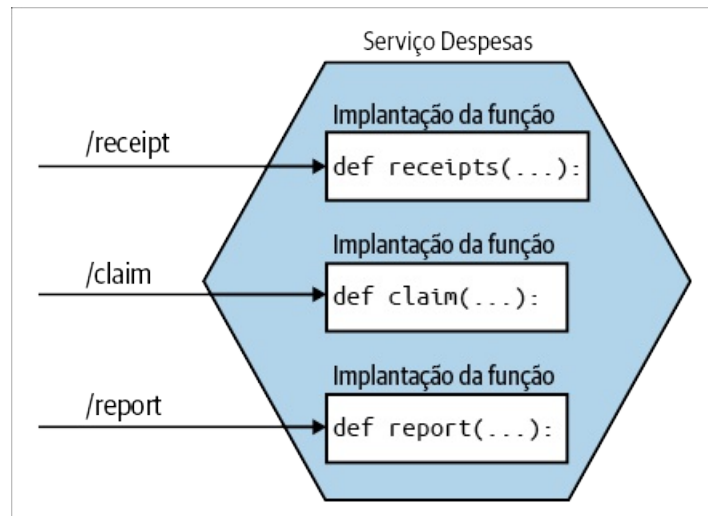


Figura 8.19 – Um serviço Despesas implantado na forma de várias funções, cada uma lidando com um agregado diferente.

Com esse modelo, nossa instância de microserviço não será mais mapeada para uma única unidade de implantação. Em vez disso, nosso microserviço agora passará a ser mais um conceito lógico, composto de várias funções diferentes que, teoricamente, podem ser implantadas de modo independente.

Há, porém, algumas ressalvas. Em primeiro lugar, insistiria fortemente que você mantivesse uma interface externa mais genérica. Para os consumidores upstream, eles ainda conversarão com o serviço Despesas – eles não estarão cientes de que as requisições serão mapeadas para agregados com um escopo menor. Isso garante que, caso você mude de ideia e queira recombina as partes, ou até mesmo reestruturar o modelo de agregados, não haverá impacto para os consumidores upstream.

A segunda questão se relaciona aos dados. Esses agregados devem continuar usando um banco de dados compartilhado? Quanto a isso, eu não me preocupo tanto. Supondo que a mesma equipe gerencie todas essas funções, e que, conceitualmente, continuamos tendo um único “serviço”, eu não veria problemas se elas continuassem usando o mesmo banco de dados, como mostra a Figura 8.20.

Com o tempo, porém, se as necessidades da função de cada agregado divergirem, eu estaria mais inclinado a tentar separar o uso de dados, conforme vemos na Figura 8.21, sobretudo se você começar a perceber que

há um acoplamento na camada de dados que está prejudicando a sua capacidade de alterá-los facilmente. A essa altura, você poderia argumentar que essas funções seriam, por si só, microsserviços independentes – embora, como acabei de explicar, poderia ser vantajoso continuar representando-as como um único microsserviço para os consumidores upstream.

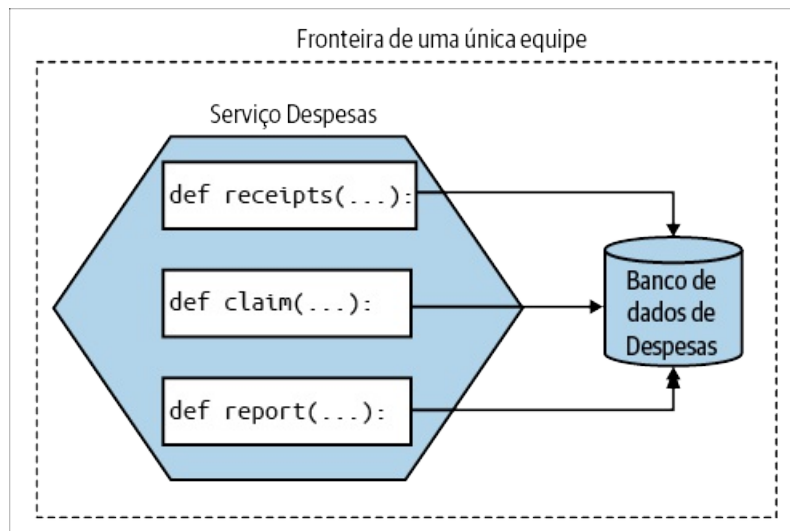


Figura 8.20 – Diferentes funções utilizando o mesmo banco de dados, pois todas fazem parte, do ponto de vista lógico, do mesmo microsserviço, e são gerenciadas pela mesma equipe.

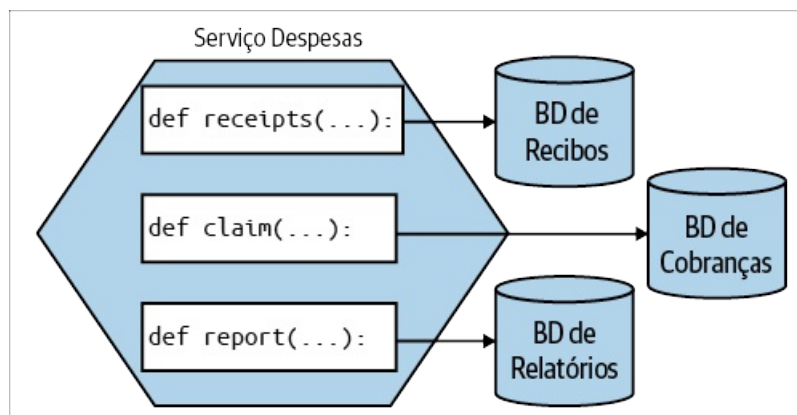


Figura 8.21 – Cada função utilizando o próprio banco de dados.

Esse mapeamento de um único microsserviço para várias unidades menores de implantação de certo modo distorce nossa definição anterior de microsserviço. Em geral, consideramos um microsserviço como uma

unidade possível de ser implantada de forma independente – agora, um microsserviço é composto de *várias unidades diferentes*, que podem ser implantadas de modo independente. Nesse exemplo, o microsserviço passa a ser um conceito mais lógico do que físico.

Divisão em partes menores ainda. Se quiser ter partes menores ainda, você poderá ficar tentado a dividir suas funções, atualmente separadas por agregado, em unidades menores ainda. Sou muito mais cauteloso nesse caso. Além da explosão no número de funções que essa solução provavelmente causará, ela também viola um dos princípios fundamentais de um agregado – o fato de querermos tratá-lo como uma unidade a fim de garantir que poderemos administrar melhor a integridade do próprio agregado.

Já considerei a ideia de fazer com que cada transição de estado de um agregado tivesse a própria função, mas desisti dela em virtude dos problemas associados à inconsistência. Quando temos diferentes unidades que podem ser implantadas de modo independente, cada uma gerenciando uma parte diferente das transições de estado como um todo, garantir que tudo seja feito da forma correta será realmente muito difícil. Isso nos coloca na área das sagas, que foram discutidas no Capítulo 6. Ao implementar processos de negócio complexos, conceitos como o de sagas são importantes, e o trabalho será justificável. Contudo, tenho dificuldade de ver a vantagem de acrescentar essa complexidade no nível do gerenciamento de um único agregado, o qual poderia ser facilmente tratado por uma única função.

Direção a ser seguida

Continuo convencido de que o futuro da maioria dos desenvolvedores está em utilizar uma plataforma que oculte boa parte dos detalhes subjacentes. Durante muitos anos, o Heroku era o mais próximo que eu podia indicar como um produto que havia encontrado o equilíbrio correto, mas, atualmente temos a FaaS e o ecossistema mais amplo de produtos serverless turnkey que mostram um caminho diferente.

Ainda há problemas a serem resolvidos na FaaS, mas acho que, embora o

conjunto atual de produtos ofertados ainda precise ser modificado para resolver seus problemas, esse é o tipo de plataforma que a maioria das pessoas acabará usando. Nem todas as aplicações se enquadrarão muito bem em um ecossistema FaaS considerando suas restrições, mas para aquelas que se enquadram, as pessoas já têm percebido vantagens significativas. Com mais trabalho dedicado aos produtos FaaS baseados em Kubernetes, as pessoas que não conseguem fazer uso direto das soluções de FaaS oferecidas pelos principais provedores de nuvem poderão cada vez mais tirar proveito dessa nova forma de trabalhar.

Então, embora a FaaS possa não ser apropriada para todos os casos, sem dúvida é uma opção que peço às pessoas que explorem. Para os meus clientes que pretendem passar para soluções Kubernetes baseadas em nuvem, venho pedindo a vários deles que explorem antes a FaaS, pois ela pode lhes fornecer tudo de que precisam, ao mesmo tempo que oculta um volume significativo de complexidades e dispensa bastante trabalho.

Vejo cada vez mais empresas utilizando a FaaS como parte de uma solução mais ampla, escolhendo-a para casos de uso específicos, nos quais ela se enquadre bem. Um bom exemplo seria a BBC, que utiliza funções Lambda como parte de seu conjunto básico de tecnologias usadas no site da BBC News. O sistema como um todo utiliza uma combinação de Lambda e instâncias EC2 – com as instâncias EC2 frequentemente usadas em situações nas quais as chamadas de funções Lambda seriam muito caras.³

Qual é a opção de implantação correta para você?

Puxa, temos muitas opções, não é mesmo? E, provavelmente, não ajudei muito me esforçando para apresentar tantos prós e contras de cada abordagem. Se você chegou até este ponto, talvez esteja um pouco confuso acerca do que deve fazer.



Bem, antes de prosseguir, espero realmente que nem é preciso dizer que, se o que você faz atualmente funciona para você, então *continue fazendo!* Não deixe que a moda determine suas decisões técnicas.

Se você acha que precisa mudar o modo como implanta seus

microserviços, deixe-me então tentar sintetizar boa parte do que já discutimos e apresentar algumas orientações úteis.

Ao rever nossos princípios de implantação de microserviços, um dos aspectos mais importantes no qual nos concentramos foi garantir o isolamento desses serviços. Contudo, usar apenas esse aspecto como um princípio norteador poderia nos levar em direção ao uso de máquinas físicas dedicadas para cada instância de microserviço! Essa solução, é claro, provavelmente teria um custo muito alto e, conforme já discutimos, há algumas ferramentas muito eficazes que não seriam possíveis de utilizar caso optássemos por esse caminho.

Há muitos prós e contras nesse caso. Atingir um equilíbrio entre custo e facilidade de uso, isolamento, familiaridade... tudo isso pode causar muita apreensão. Vamos, portanto, analisar um conjunto de regras que gosto de chamar de Regras Realmente Gerais e Básicas de Sam para Descobrir Onde Implantar as Coisas:

1. Se não está quebrado, não conserte.⁴
2. Abstenha-se do máximo de controle com o qual estiver satisfeito, e então se abstenha de um pouquinho mais. Se puder passar todo o seu trabalho para um bom PaaS como o Heroku (ou para uma plataforma FaaS), faça isso e seja feliz. Você realmente precisa fazer ajustes em cada configuração possível?
3. Colocar seus microserviços em contêineres não é uma tarefa isenta de dificuldades, mas é realmente um bom compromisso no que diz respeito ao custo do isolamento, além de proporcionar algumas vantagens incríveis para o desenvolvimento local, ao mesmo tempo que dará a você certo nível de controle sobre o que acontece. Espere ter o Kubernetes em seu futuro.

Muitas pessoas têm proclamado que é o “Kubernetes, ou nada!” – o que eu acho que não ajuda muito. Se você está na nuvem pública e a FaaS como modelo de implantação é apropriada para o seu problema, utilize-a e ignore o Kubernetes. Seus desenvolvedores provavelmente acabarão sendo muito mais produtivos. Conforme discutiremos melhor no Capítulo 16, não deixe que o medo de ficar preso a uma solução o coloque em uma

confusão que você mesmo criou.

Encontrou um PaaS incrível como o Heroku ou o Zeit, e tem uma aplicação que se enquadra às restrições da plataforma? Coloque todo o trabalho na plataforma e passe mais tempo trabalhando com o seu produto. Tanto o Heroku como o Zeit são plataformas fantásticas, com uma usabilidade incrível do ponto de vista de um desenvolvedor. Afinal de contas, seus desenvolvedores não merecem ser felizes?

Para os demais, a containerização é o caminho a ser seguido, o que significa que precisamos falar sobre o Kubernetes.

Qual é o papel do Puppet, do Chef e de outras ferramentas?

Este capítulo mudou significativamente desde a primeira edição. Em parte, isso se deve à evolução do mercado como um todo, mas também a novas tecnologias que se tornaram cada vez mais convenientes. O surgimento de novas tecnologias também resultou na diminuição da relevância de outras tecnologias – assim, vemos ferramentas como Puppet, Chef, Ansible e Salt desempenhando um papel muito menor na implantação de arquiteturas de microsserviços, em comparação com o que havia antes, por volta de 2014.

O principal motivo para isso é basicamente o surgimento do contêiner. A eficácia de ferramentas como Puppet e Chef está no fato de elas oferecerem uma forma de levar uma máquina a um estado desejado, com esse estado desejado definido na forma de código. Podemos definir os runtimes necessários, os locais em que os arquivos de configuração devem estar e outros aspectos, de um modo que essa configuração possa ser repetidamente executada na mesma máquina, de forma determinística, garantindo que a máquina possa ser sempre levada ao mesmo estado.

A maioria das pessoas cria um contêiner com um Dockerfile. Isso permite definir os mesmos requisitos que seriam definidos com o Puppet ou o Chef, com algumas diferenças. Um contêiner é destruído ao ser reimplantado, portanto a criação de cada contêiner é feita do zero (estou, até certo ponto, fazendo uma simplificação). Isso significa que boa parte da complexidade inerente ao Puppet e ao Chef para lidar com as ferramentas executadas repetidamente nas mesmas máquinas

não será necessária.

O Puppet, o Chef e ferramentas similares continuam sendo extremamente úteis, mas sua função foi atualmente afastada do contêiner e está nas camadas mais inferiores da pilha. As pessoas utilizam ferramentas como essas para gerenciar aplicações e infraestruturas legadas, ou para criar os clusters nos quais as cargas de trabalho em contêineres agora executam. No entanto, é ainda menos provável que os desenvolvedores entrem em contato com essas ferramentas, em comparação com o que ocorria no passado.

O conceito de infraestrutura como código continua sendo de muito importância. Apenas o tipo de ferramentas que os desenvolvedores provavelmente utilizam é que mudou. Para aqueles que trabalham na nuvem, por exemplo, algo como o Terraform (<http://terraform.io>) pode ser muito útil para o provisionamento de uma infraestrutura na nuvem. Recentemente, passei a ser um grande fã do Pulumi (<https://www.pulumi.com>), que dispensa o uso de DSLs (Domain-Specific Languages, ou Linguagens de Domínio Específico) em favor de utilizar linguagens de programação comuns para ajudar os desenvolvedores a gerenciarem sua infraestrutura de nuvem. Vejo um grande futuro para o Pulumi à medida que as equipes de entrega de software assumem cada vez mais a responsabilidade pelo mundo operacional, e suspeito que o Puppet, o Chef e as ferramentas desse tipo, embora continuem a desempenhar um papel importante nas operações, provavelmente se afastarão cada vez mais das atividades cotidianas de desenvolvimento.

Kubernetes e a orquestração de contêineres

Quando os contêineres começaram a ganhar impulso, muitas pessoas passaram a procurar soluções para gerenciá-los em várias máquinas. O Docker fez duas tentativas para isso (com o Docker Swarm e o Docker Swarm Mode, respectivamente); empresas como Rancher e CoreOS apresentaram suas próprias iniciativas, e plataformas de propósito mais genérico como o Mesos foram usadas para executar contêineres junto com outros tipos de cargas de trabalho. Em última análise, porém, apesar dos muitos esforços investidos nesses produtos, nos últimos anos, o

Kubernetes passou a dominar essa área.

Antes de falar do Kubernetes propriamente dito, precisamos discutir primeiro o motivo da necessidade de termos uma ferramenta desse tipo.

Razões para a orquestração de contêineres

Falando de modo geral, o Kubernetes pode ser descrito de forma variada como uma plataforma de orquestração de contêineres (container orchestration platform) ou, para empregar um termo que caiu em desuso, um escalonador de contêineres (container scheduler). Então, o que são essas plataformas, e por que iríamos querê-las?

Os contêineres são criados isolando-se um conjunto de recursos em uma máquina subjacente. Ferramentas como o Docker permitem definir como deve ser um contêiner e criar uma instância desse contêiner em uma máquina. Contudo, a maioria das soluções exige que nosso software esteja definido em várias máquinas, talvez para lidar com uma carga necessária, ou para garantir que o sistema tenha redundância de modo a tolerar a falha de um único nó. As plataformas de orquestração de contêineres lidam com o modo e o lugar em que as cargas de trabalho em contêineres serão executadas. O termo “escalonamento” começa a fazer mais sentido nesse contexto. O operador diz: “Gostaria que isto fosse executado”, e o orquestrador descobre como escalonar essa tarefa – encontrando recursos disponíveis, realocando-os se for necessário e cuidando dos detalhes para o operador.

As diversas plataformas de orquestração de contêineres também lidam com o gerenciamento de estado desejado para nós, garantindo que o estado esperado para um conjunto de contêineres (instâncias de microserviços, em nosso caso) seja mantido. Elas também nos permitem especificar o modo como queremos que essas cargas de trabalho sejam distribuídas, possibilitando otimizar a utilização de recursos e melhorar a latência entre os processos, ou com vistas à robustez.

Sem uma ferramenta como essa, você teria de gerenciar a distribuição de seus contêineres, algo que, com base em minha própria experiência, posso dizer que se torna desatualizada rapidamente. Escrever scripts para

gerenciar a inicialização e a colocação das instâncias de contêineres na rede não é uma tarefa divertida.

Falando de modo geral, todas as plataformas de orquestração de contêineres, incluindo o Kubernetes, oferecem essas funcionalidades de alguma forma. Se observarmos os escalonadores de propósito geral como o Mesos ou o Nomad, soluções gerenciadas como o ECS da AWS, o Docker Swarm Mode e outras, veremos um conjunto semelhante de recursos. Contudo, por motivos que serão explorados em breve, o Kubernetes conseguiu dominar essa área. Ele também inclui um ou dois conceitos interessantes que valem a pena ser explorados rapidamente.

Uma visão simplificada dos conceitos do Kubernetes

Há muitos outros conceitos no Kubernetes, portanto peço desculpas por não explicar todos (sem dúvida, isso certamente justificaria a existência de outro livro). O que tentarei fazer agora é apresentar as principais ideias com as quais você terá de lidar quando começar a trabalhar com a ferramenta. Vamos analisar antes o conceito de cluster, conforme mostra a Figura 8.22.

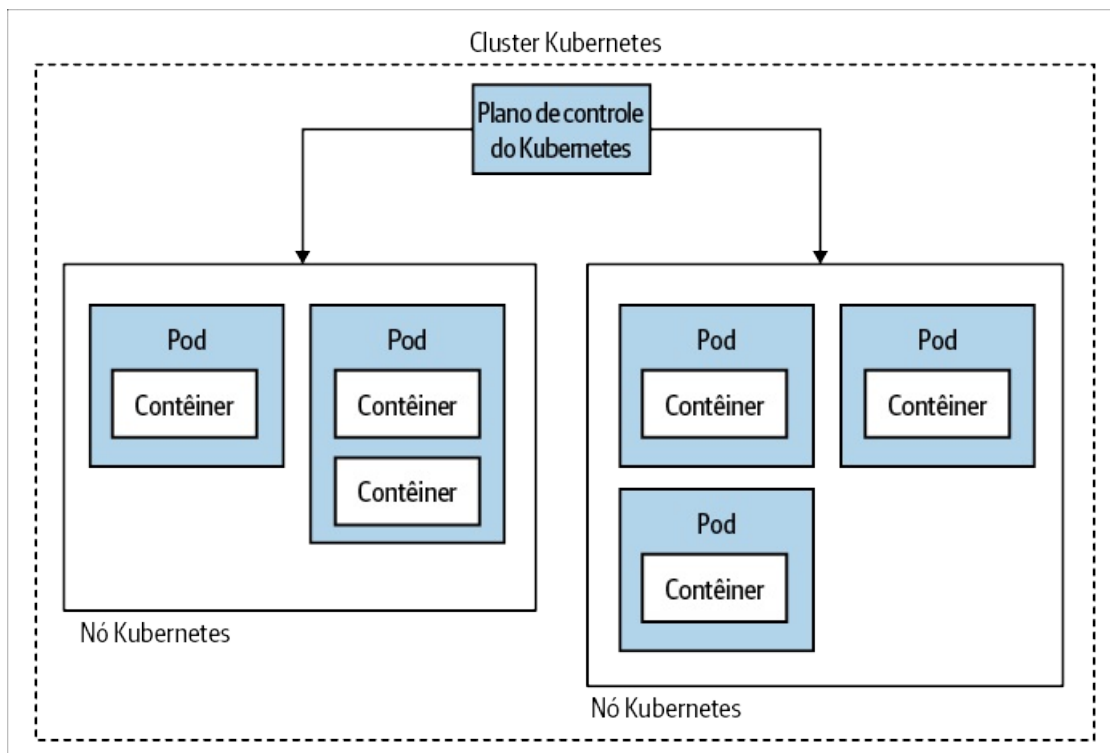


Figura 8.22 – Uma visão geral simples da topologia do Kubernetes.

Basicamente, um cluster Kubernetes é composto de dois itens. Em primeiro lugar, há um conjunto de máquinas nas quais as cargas de trabalho serão executadas, e que são chamados de nós. Em segundo lugar, temos um conjunto de softwares de controle que gerenciam esses nós, o qual é conhecido como plano de controle. Esses nós podem executar em máquinas físicas ou em máquinas virtuais subjacentes. Em vez de escalonar um contêiner, o Kubernetes escalona algo que ele chama de *pod*. Um pod é composto de um ou mais contêineres que serão implantados juntos.

Geralmente você terá apenas um contêiner em um pod – por exemplo, uma instância de seu microsserviço. Há, porém, algumas ocasiões (raras, com base em minha experiência) em que ter vários contêineres implantados em conjunto pode fazer sentido. Um bom exemplo disso está no uso de proxies sidecar como o Envoy, muitas vezes como parte de um service mesh – um assunto que discutimos na seção “Service meshes e gateways de API”, na página [199](#).

O próximo conceito que é conveniente conhecer se chama *serviço*. No contexto do Kubernetes, podemos pensar em um serviço como um endpoint estável para roteamento – basicamente, é uma forma de fazer um mapeamento dos pods que estão em execução para uma interface de rede estável, disponível no cluster. O Kubernetes cuida do roteamento dentro do cluster, como vemos na Figura 8.23.

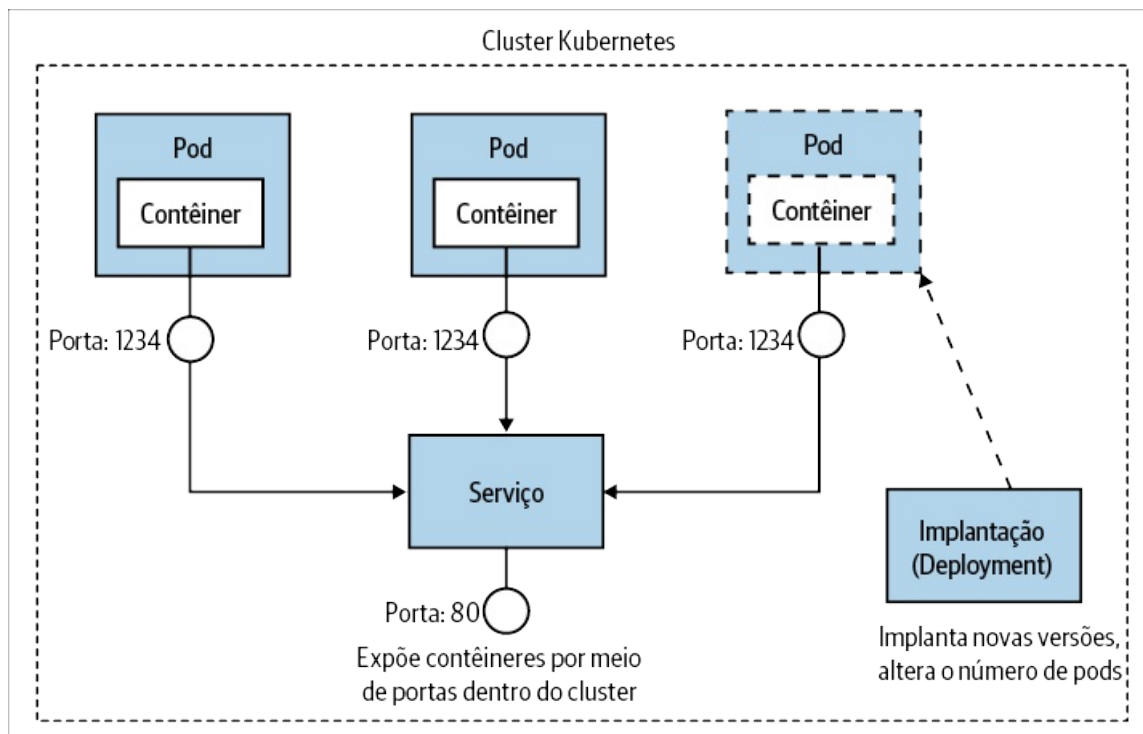


Figura 8.23 – Como um pod, um serviço e um deployment (implantação) funcionam em conjunto.

A ideia é que um dado pod pode ser considerado efêmero – pode ser desativado por uma série de motivos – enquanto um serviço como um todo continua ativo. O serviço existe para encaminhar chamadas de e para os pods, e é capaz de lidar com o shutdown ou com a inicialização de novos pods. Do ponto de vista exclusivamente da terminologia, isso pode ser confuso. Falamos de modo mais genérico sobre implantar um serviço, mas, no Kubernetes, não implantamos um serviço – implantamos pods que são mapeados para um serviço. Pode demorar um pouco para que você se acostume com essa ideia.

Em seguida, temos um *conjunto de réplicas* (replica set). Com um conjunto de réplicas, definimos o estado desejado para um conjunto de pods. É nesse momento que dizemos “Quero quatro desses pods”, e o Kubernetes cuidará do resto. Na prática, não é esperado que você lide diretamente com conjuntos de réplicas; em vez disso, eles serão tratados para você por meio de um *deployment* (implantação), que é o último conceito que veremos. Um deployment é o modo de aplicar mudanças em seus pods e conjuntos de réplicas. Com um deployment, podemos executar

tarefas como conduzir rolling upgrades (upgrades sem interrupção, para substituir pods por uma versão mais recente de forma gradual, evitando que haja downtime) ou rollbacks, aumentar o número de nós etc.

Assim, para implantar nosso microserviço, definimos um *pod*, que conterà a instância do nosso microserviço, definimos um *serviço*, que permitirá que o Kubernetes saiba como o microserviço será acessado, e aplicamos mudanças nos pods em execução utilizando um *deployment*. Parece fácil quando digo isso, não é mesmo? Vamos apenas dizer que deixei muitos detalhes de fora nesse caso, para ser mais conciso.

Multitenancy e federação

Do ponto de vista da eficácia, você iria querer reunir todos os recursos computacionais à sua disposição em um único cluster Kubernetes e fazer com que todas as cargas de trabalho da empresa como um todo fossem executadas aí. Isso provavelmente possibilitaria um maior uso dos recursos subjacentes, pois os recursos não utilizados poderiam ser livremente realocados a quem precisasse deles. Por sua vez, isso reduziria os custos na mesma proporção.

O desafio está no fato de que, embora o Kubernetes seja bem capaz de gerenciar diferentes microserviços para diferentes propósitos, ele tem limitações quanto ao nível de “multitenancy” (multi-inquilino) da plataforma. Diferentes departamentos de sua empresa podem querer ter diferentes níveis de controle sobre diversos recursos. Esses tipos de controle não foram incluídos no Kubernetes – uma decisão que parece razoável no que diz respeito a tentar manter o escopo do Kubernetes, de certo modo, limitado. Para contornar esse problema, as empresas parecem explorar alguns caminhos diferentes.

A primeira opção é adotar uma plataforma criada com base no Kubernetes, a qual ofereça esses recursos – o OpenShift da Red Hat, por exemplo, tem um conjunto rico de controles de acesso, além de outros recursos criados com empresas de grande porte em mente, e que podem facilitar, de certa forma, o conceito de multitenancy. Afora qualquer implicação financeira quanto ao uso desses tipos de plataformas, para que elas funcionem, às

vezes será necessário trabalhar com as abstrações disponibilizadas pelo fornecedor que você escolher – isso significa que seus desenvolvedores precisarão saber não só como utilizar o Kubernetes, mas também como usar a plataforma desse fornecedor específico.

Outra abordagem é considerar um modelo de federação, representado na Figura 8.24. Com a federação, temos vários clusters separados, com alguma camada de software acima permitindo que você faça alterações em todos os clusters, se for necessário. Em muitos casos, as pessoas poderiam trabalhar diretamente com um cluster, o que lhes proporcionaria uma experiência muito familiar com o Kubernetes; em outras situações, porém, você talvez queira distribuir uma aplicação em vários clusters, talvez se esses clusters estiverem em regiões geográficas distintas e você quisesse que sua aplicação fosse implantada com certa capacidade para lidar com a perda de um cluster inteiro.

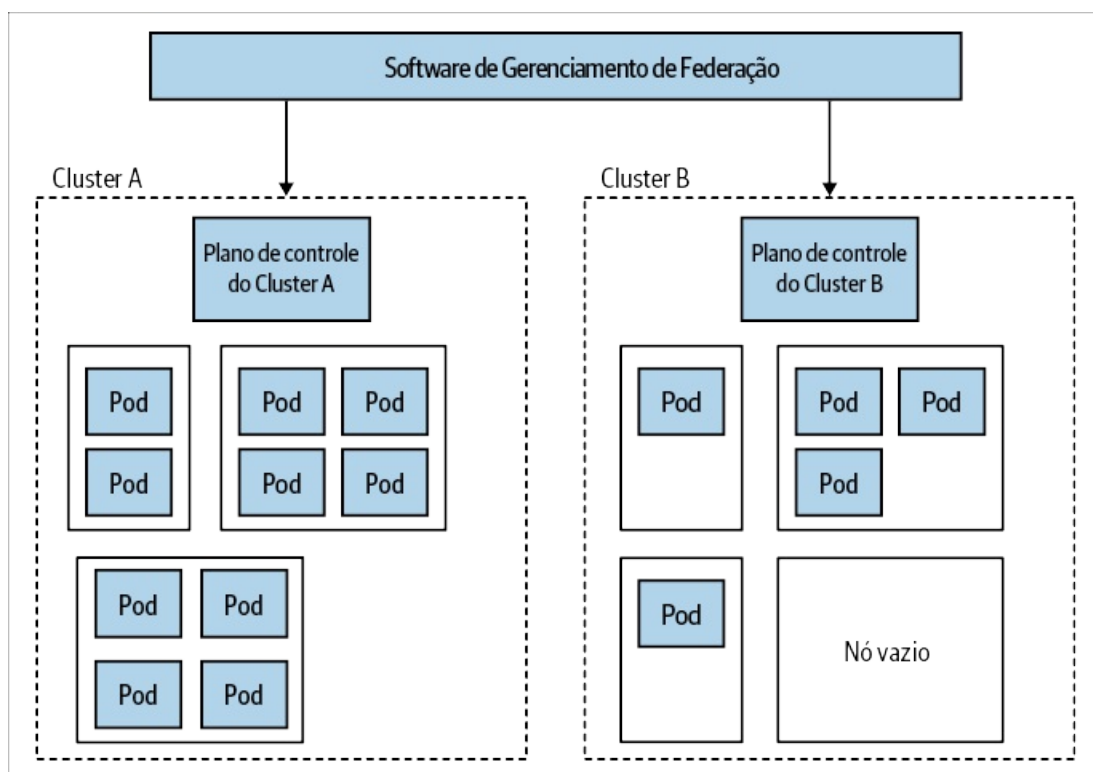


Figura 8.24 – Exemplo de federação no Kubernetes.

A natureza de federação deixa a distribuição de recursos mais complicada. Conforme vemos na Figura 8.24, o Cluster A está totalmente utilizado,

enquanto o Cluster B tem muita capacidade não utilizada. Se quiséssemos executar mais cargas de trabalho no Cluster A, fazer isso seria possível somente se pudéssemos lhe dar mais recursos, por exemplo, passando o nó vazio do Cluster B para o Cluster A. O nível de dificuldade para passar o nó de um cluster para outro dependerá da natureza do software de federação em uso, mas posso muito bem imaginar que essa não seria uma mudança trivial. Tenha em mente que um único nó pode fazer parte de um só cluster e, portanto, não poderá executar pods de ambos os Clusters, A e B.

Vale a pena observar que ter vários clusters pode ser vantajoso se considerarmos os desafios de fazer um upgrade no próprio cluster. Talvez seja mais fácil e seguro mover um microsserviço para um cluster recém-atualizado do que fazer o upgrade do cluster in-place (no próprio local).

Basicamente, esses desafios estão associados à escala. Em algumas empresas, você nunca terá problemas como esses, pois ficará satisfeito em compartilhar um único cluster. Para outras empresas que estão em busca de eficiência em larga escala, certamente essa é uma área que elas vão querer explorar com mais detalhes. Devemos observar que há várias visões diferentes sobre como deve ser uma federação no Kubernetes, e há muitos conjuntos diferentes de ferramentas por aí para gerenciá-la.

A história do Kubernetes

O Kubernetes começou como um projeto open source no Google, e se inspirou nos sistemas anteriores de gerenciamento de contêineres, Omega e Borg. Muitos dos conceitos fundamentais do Kubernetes têm como base os conceitos relacionados ao modo como as cargas de trabalho em contêineres são gerenciadas no Google, ainda que com um alvo um pouco diferente em mente. O Borg executa sistemas em uma escala global gigantesca, lidando com dezenas, se não centenas, de milhares de contêineres distribuídos em data centers pelo mundo todo. Se quiser mais detalhes sobre como as diferentes formas de pensar associadas a essas três plataformas do Google se comparam, embora sejam com um ponto de vista centrado nessa empresa, recomendo o artigo “Borg, Omega, and Kubernetes” (<https://oreil.ly/fVCSS>), de Brendan Burns et al., que apresenta uma boa visão geral do assunto.

Embora o Kubernetes compartilhe parte de seu DNA com o Borg e o Omega, trabalhar em uma escala gigantesca não foi o principal fator determinante do projeto. Tanto o Nomad quanto o Mesos (ambos foram inspirados no Borg) encontraram seu nicho em situações nas quais havia necessidade de ter clusters com milhares de máquinas, conforme demonstrado pelo uso do Mesos na Apple para o serviço Siri⁵, ou pelo uso do Nomad pela Roblox (<https://oreil.ly/tyWof>).

O Kubernetes resolveu partir das ideias do Google, porém desejava oferecer uma experiência mais conveniente aos desenvolvedores, em comparação com aquela proporcionada pelo Borg

ou o Omega. É possível encarar a decisão do Google de investir bastante esforço de engenharia na criação de uma ferramenta open source de um ponto de vista puramente altruísta; embora eu esteja certo de que essa era a intenção de algumas pessoas, a verdade é que a decisão se deveu mais ao risco percebido pela empresa no que diz respeito à concorrência na área de nuvem pública, sobretudo da AWS.

No mercado de nuvem pública, o Google Cloud ganhou terreno, mas está distante, em terceiro lugar, atrás do Azure e da AWS (que está bem à frente), e segundo algumas análises, está sendo empurrado para o quarto lugar pelo Alibaba Cloud. Apesar do aumento de participação no mercado, o Google Cloud ainda não está nem perto da posição desejada pelo Google.

Parece provável que uma preocupação importante estava no fato de que o evidente líder de mercado, a AWS, poderia, em algum momento, ter praticamente o monopólio da área de computação em nuvem. Além do mais, preocupações associadas ao custo da migração de um provedor para outro sinalizavam que uma posição como essa de liderança no mercado seria difícil de mudar. Então surgiu o Kubernetes, com sua promessa de disponibilizar uma plataforma padrão para executar cargas de trabalho em contêineres, possíveis de serem executadas por vários fornecedores. A expectativa era que isso permitiria migrar de um provedor para outro e evitar um futuro que fosse exclusivamente da AWS.

Assim, podemos ver o Kubernetes como uma generosa contribuição do Google para o mercado mais amplo de TI, ou como uma tentativa da empresa de continuar sendo relevante na área extremamente dinâmica da nuvem pública. Não tenho nenhum problema em ver as duas perspectivas como igualmente verdadeiras.

Cloud Native Computing Foundation

A CNCF (Cloud Native Computing Foundation, ou Fundação para Computação Nativa de Nuvem) é uma ramificação da organização sem fins lucrativos Linux Foundation. A CNCF tem como foco organizar o ecossistema de projetos para ajudar a promover o desenvolvimento nativo de nuvem, embora, na prática, isso signifique dar suporte ao Kubernetes e a projetos que trabalham com ele, ou que foram criados com base nele. Os projetos em questão não são criados nem diretamente desenvolvidos pela CNCF; em vez disso, podemos ver a CNCF como um local único em que esses projetos – os quais, do contrário, seriam desenvolvidos isoladamente – podem ser reunidos, possibilitando o desenvolvimento de padrões comuns e permitindo que haja interoperabilidade.

Dessa forma, o papel da CNCF lembra o papel da Apache Software Foundation – assim como na CNCF, um projeto que faça parte da Apache Software Foundation geralmente implica certo nível de qualidade e um suporte da comunidade mais ampla. Todos os projetos patrocinados pela CNCF são open source, embora o desenvolvimento desses projetos possa

ser direcionado por entidades comerciais.

Além de ajudar a orientar o desenvolvimento desses projetos associados, a CNCF também organiza eventos, disponibiliza documentação e conteúdos para treinamento, e define os vários programas de certificação associados ao Kubernetes. O grupo tem membros de várias áreas do mercado, e embora possa ser difícil que grupos menores ou independentes desempenhem um papel muito relevante na própria organização, o grau de suporte oferecido pelo mercado como um todo (incluindo várias empresas que são concorrentes entre si) é impressionante.

Para quem vê de fora, a CNCF parece que tem tido bastante sucesso em ajudar a divulgar a utilidade dos projetos que patrocina. Ela também atua como um local em que a evolução de projetos importantes pode ser discutida abertamente, garantindo que haja muitas contribuições. A CNCF tem desempenhado um papel muito relevante no sucesso do Kubernetes – é fácil imaginar que, sem ela, ainda teríamos um panorama fragmentado nessa área.

Plataformas e portabilidade

Você ouvirá com frequência o Kubernetes ser descrito como uma “plataforma”. Contudo, ele não é realmente uma plataforma no sentido em que um desenvolvedor entenderia o termo. À primeira vista, tudo o que ele realmente oferece a você é a capacidade de executar cargas de trabalho em contêineres. A maioria das pessoas que utiliza o Kubernetes acaba montando a própria plataforma, instalando softwares auxiliares como service meshes, brokers de mensagens, ferramentas para agregação de logs, e outros. Em empresas maiores, isso acaba ficando sob a responsabilidade de uma equipe de engenharia de plataforma; essa equipe configura e gerencia a plataforma, além de ajudar os desenvolvedores a usá-la com eficiência.

Isso pode ser tanto uma bênção quanto uma maldição. Essa abordagem de “pegar e combinar” se torna possível em virtude de um ecossistema de ferramentas razoavelmente compatíveis (em boa medida, graças ao trabalho feito pela CNCF). Significa que, se você quiser, poderá escolher

suas ferramentas favoritas para tarefas específicas. No entanto, também pode levar à tirania da escolha – podemos facilmente ficar sobrecarregados com tantas opções. Produtos como o OpenShift da Red Hat parcialmente eliminaram a necessidade de fazermos essas escolhas, pois disponibilizam uma plataforma pronta, com algumas decisões já tomadas por nós.

Isso significa que, embora no nível básico o Kubernetes ofereça uma abstração portátil para a execução de contêineres, na prática, não é tão simples quanto tomar uma aplicação que funciona em um cluster e esperar que ela vá funcionar em outro lugar. Sua aplicação, suas operações e o fluxo de trabalho dos desenvolvedores podem muito bem depender de sua plataforma personalizada. Passar de um cluster Kubernetes para outro também pode exigir que você reconstrua a plataforma em seu novo destino. Já conversei com muitas empresas que haviam adotado o Kubernetes principalmente porque estavam preocupadas com o fato de ficarem dependentes de um único fornecedor, porém essas empresas não haviam se dado conta dessa nuance – aplicações criadas com base no Kubernetes são portáteis entre clusters Kubernetes na teoria, mas nem sempre é o que acontece na prática.

Helm, Operator e CRDs, minha nossa!

Uma área em que há sempre confusão no domínio do Kubernetes é o modo de gerenciar a implantação e o ciclo de vida de aplicações e subsistemas de terceiros. Considere a necessidade de executar o Kafka em seu cluster Kubernetes. Você poderia criar o próprio pod, o serviço e as especificações de deployment, e executá-los por conta própria. Mas e quanto ao gerenciamento de um upgrade em sua instalação de Kafka? E o que dizer de outras tarefas comuns de manutenção com as quais você talvez tenha de lidar, por exemplo, com o upgrade de um software stateful (com estado) em execução?

Uma série de ferramentas surgiu com o propósito de permitir que você seja capaz de gerenciar esses tipos de aplicações com um nível mais razoável de abstração. A ideia é que alguém crie algo semelhante a um pacote para o Kafka, e você o execute em seu cluster Kubernetes de uma maneira mais

parecida com uma caixa-preta. Duas das soluções mais conhecidas nessa área são o Operator e o Helm. O Helm se apresenta como “o gerenciador de pacotes que faltava” para o Kubernetes, e embora o Operator seja capaz de gerenciar uma instalação inicial, seu foco parece estar mais no gerenciamento contínuo da aplicação. De forma confusa, apesar de podermos ver o Operator e o Helm como alternativas entre si, também é possível utilizar ambos em algumas situações (o Helm para uma instalação inicial e o Operator para operações do ciclo de vida).

Uma evolução mais recente nessa área está em algo chamado CRD (Custom Resource Definitions, ou Definições de Recursos Personalizados). Com os CRDs, podemos estender as APIs básicas do Kubernetes, permitindo associar novos comportamentos ao cluster. O interessante a respeito dos CRDs é que eles se integram de modo razoavelmente natural com a interface de linha de comando existente, os controles de acesso, e outros recursos – assim, sua extensão personalizada não se parecerá com um acréscimo estranho. Os CRDs permitem basicamente que você implemente as suas próprias abstrações no Kubernetes. Pense nas abstrações de pod, conjunto de réplicas, serviço e deployment que discutimos antes – com os CRDs, você pode acrescentar a própria abstração ao conjunto.

Podemos utilizar CRDs para tudo, desde gerenciar pequenas porções de configuração a controlar service meshes como o Istio, ou um software baseado em cluster como o Kafka. Com um conceito tão flexível e eficaz como esse, acho difícil saber quais são os pontos em que os CRDs teriam melhor uso, e não parece haver um consenso a esse respeito entre os especialistas com quem conversei. Essa área como um todo parece ainda não ter se estabilizado tão rapidamente quanto eu esperava, e não há tanto consenso quanto eu gostaria que houvesse – uma tendência no ecossistema do Kubernetes.

E o Knative

O Knative (<https://knative.dev>) é um projeto open source cujo propósito é oferecer fluxos de trabalho no estilo FaaS aos desenvolvedores, utilizando o

Kubernetes nos bastidores. Basicamente, o Kubernetes não é extremamente amigável a um desenvolvedor, sobretudo se o compararmos com a usabilidade de outras soluções como o Heroku ou plataformas semelhantes. O propósito do Knative é levar a experiência dos desenvolvedores na FaaS para o Kubernetes, ocultando a complexidade desse último. Por sua vez, isso provavelmente implica que as equipes de desenvolvimento serão capazes de gerenciar com mais facilidade o ciclo de vida completo de seus softwares.

Já discutimos os service meshes, e mencionamos especificamente o Istio, no Capítulo 5. Um service mesh é essencial para que o Knative execute. Embora, teoricamente, o Knative permita conectar diferentes service meshes, somente o Istio é considerado estável atualmente (com um suporte para outros meshes como o Ambassador e o Gloo ainda em versão alfa). Na prática, significa que, se você quiser adotar o Knative, também deverá ter comprado a ideia de usar o Istio.

Tanto com o Kubernetes como com o Istio, projetos conduzidos principalmente pelo Google, demorou bastante para que ambos chegassem em uma fase na qual pudessem ser considerados estáveis. O Kubernetes ainda sofreu mudanças significativas após o lançamento da versão 1.0, e apenas muito recentemente, o Istio, que oferecerá sustentação para o Knative, teve sua arquitetura totalmente remodelada. Esse histórico de entregar projetos estáveis, prontos para o ambiente de produção, me faz pensar que o Knative pode demorar muito mais até estar pronto para ser usado pela maioria de nós. Embora algumas empresas estejam utilizando essa ferramenta, e você provavelmente possa usá-la também, a experiência mostra que não demorará muito para que haja grandes mudanças, as quais exigirão uma migração complicada. É parcialmente por esse motivo que tenho sugerido que as empresas mais conservadoras, que estejam considerando um produto do tipo FaaS para seu cluster Kubernetes, procurem outras alternativas – projetos como o OpenFaaS já estão sendo usados em ambientes de produção por empresas em todo o mundo, e não exigem um service mesh subjacente. No entanto, se você vai subir no trem do Knative agora, não se surpreenda caso venha a sofrer um descarrilamento inesperado no futuro.

Outra observação: é lamentável que o Google tenha decidido que o Knative não faria parte da CNCF – podemos supor que isso aconteceu somente porque a empresa deseja, ela mesma, determinar a direção que terá a ferramenta. O Kubernetes era um prospecto confuso para muitos quando foi lançado, parcialmente porque refletia a mentalidade do Google sobre o modo como os contêineres deveriam ser gerenciados. Ele se beneficiou bastante com o envolvimento de uma parcela maior do mercado, e é muito ruim que, a essa altura pelo menos, o Google tenha decidido que não está interessado no envolvimento amplo desse mesmo mercado no caso do Knative.

Futuro

Olhando para a frente, não vejo nenhum sinal de que o rolo compressor implacável do Kubernetes vá parar tão cedo, e espero realmente ver mais empresas implementando seus próprios clusters Kubernetes em nuvens privadas, ou fazendo uso de clusters gerenciados em ambientes da nuvem pública. No entanto, acho que o que estamos vendo agora, com os desenvolvedores tendo de aprender a usar o Kubernetes diretamente, será um período relativamente curto. O Kubernetes é ótimo para gerenciar cargas de trabalho em contêineres e servir de plataforma para que outros sistemas possam ser criados aí. Contudo, ele não é o que poderia ser considerado uma experiência amigável ao desenvolvedor. O próprio Google tem nos mostrado isso, com o esforço investido no Knative, e acho que continuaremos a ver o Kubernetes sendo oculto por baixo de camadas de abstração cada vez mais altas. Então, no futuro, espero ver o Kubernetes em todos os lugares. Apenas não saberemos que ele está lá.

Isso não quer dizer que os desenvolvedores poderão se esquecer de que estão criando um sistema distribuído. Eles ainda precisarão compreender a diversidade de desafios que esse tipo de arquitetura implica. Apenas não terão de se preocupar tanto com os detalhes acerca de como seus softwares serão mapeados para os recursos computacionais subjacentes.

Devo usá-lo?

Então, para aqueles que ainda não são membros definitivos do clube do

Kubernetes, você deveria se associar? Bem, deixe-me apresentar algumas orientações. Em primeiro lugar, implementar e gerenciar o próprio cluster Kubernetes não é para os fracos – é um empreendimento significativo. Boa parte da qualidade da experiência que seus desenvolvedores terão ao usar a sua instalação de Kubernetes dependerá da eficiência da equipe que vai administrar o cluster. Por esse motivo, algumas empresas maiores com as quais conversei, que haviam optado pelo caminho de ter o Kubernetes administrado internamente, terceirizaram essa tarefa para empresas especializadas.

Melhor ainda, utilize um cluster totalmente gerenciado. Se puder utilizar a nuvem pública, utilize soluções totalmente gerenciadas, como aquelas oferecidas pelo Google, pelo Azure e pela AWS. Eu diria, porém, que, se você puder utilizar a nuvem pública, considere se o Kubernetes é realmente aquilo que você quer. Se você está procurando uma plataforma amigável aos desenvolvedores para cuidar da implantação e do ciclo de vida de seus microsserviços, as plataformas FaaS que já discutimos poderiam ser bem apropriadas. Você também poderia considerar outras ofertas do tipo PaaS, como o Azure Web Apps, o Google App Engine ou alguns dos provedores menores como o Zeit ou o Heroku.

Antes de decidir que vai começar a usar o Kubernetes, faça com que alguns de seus administradores e desenvolvedores o utilizem. Os desenvolvedores poderiam começar executando algo leve localmente, por exemplo, o minikube ou o MicroK8s, o que lhes proporcionaria algo muito próximo de uma experiência completa com o Kubernetes, porém em seus notebooks. As pessoas que farão o gerenciamento da plataforma talvez precisem se aprofundar mais. O Katacoda (<https://www.katacoda.com>) tem alguns tutoriais ótimos online para compreender os conceitos básicos, e a CNCF contribui com a divulgação de muito conteúdo para treinamento nessa área. Certifique-se de que as pessoas que realmente utilizarão esses produtos consigam testá-los antes de tomar uma decisão.

Não caia na armadilha de pensar que você precisa ter o Kubernetes só “porque todo mundo está usando”. Essa é uma justificativa tão perigosa para escolher o Kubernetes quanto para optar pelos microsserviços. Por

melhor que seja o Kubernetes, ele não serve para todos – faça a sua própria avaliação. Mas, vamos ser francos – se você tiver poucos desenvolvedores e somente alguns microsserviços, é provável que o Kubernetes seja um exagero enorme, mesmo com o uso de uma plataforma totalmente gerenciada.

Entrega progressiva

Durante a última década aproximadamente, passamos a implantar softwares para nossos usuários de modo mais inteligente. Novas técnicas surgiram em várias partes do mercado de TI, motivadas por uma série de casos de uso diferentes, mas todas tinham como foco principal o ato de implantar um novo software com muito menos riscos. E se o lançamento de um software se torna menos arriscado, podemos lançá-lo com mais frequência.

Há uma série de atividades que executamos antes de lançar o nosso software, as quais podem nos ajudar a identificar problemas antes que eles causem impacto nos usuários reais. Os testes em pré-produção desempenham um papel muito importante nesse caso, embora, conforme será discutido no Capítulo 9, eles possam ir somente até certo ponto.

Em seu livro *Accelerate*,⁶ Nicole Forsgren, Jez Humble e Gene Kim mostram evidências claras, obtidas de pesquisas abrangentes, de que empresas de alto desempenho fazem implantações com mais frequência que suas contrapartidas com menor desempenho, e, ao mesmo tempo, têm *taxas muito menores de falhas por mudanças*.

A ideia de “ser rápido e causar falhas” não parece realmente se aplicar quando se trata do lançamento de softwares – fazer lançamentos frequentes e ter taxas mais baixas de falha andam de mãos dadas, e as empresas que perceberam isso mudaram seu modo de pensar sobre os lançamentos de software.

Essas empresas utilizam técnicas como flags de funcionalidades (feature toggles), lançamentos canário (canary releases), execuções paralelas (parallel runs), além de outras, que serão detalhadas nesta seção. Essa

mudança de mentalidade a respeito do lançamento de funcionalidades se enquadra no que chamamos de *entrega progressiva* (progressive delivery). As funcionalidades são disponibilizadas aos usuários de forma controlada; em vez de fazer uma implantação enorme, usamos de inteligência para controlar quem pode ver quais funcionalidades – por exemplo, fazendo o rollout de uma nova versão de nosso software para um subconjunto de nossos usuários.

Basicamente, o que todas essas técnicas têm em sua essência é uma mudança simples no modo como pensamos no lançamento de software: a possibilidade de podermos separar o conceito de implantação do conceito de lançamento.

Diferenciando implantação de lançamento

Jez Humble, coautor de *Continuous Delivery*, defende a diferenciação dessas duas ideias e faz disso um princípio fundamental para lançamentos de software com baixo risco (<https://oreil.ly/VzpLc>):

A implantação é o que acontece quando instalamos uma versão de nosso software em um ambiente específico (geralmente implica o ambiente de produção). Lançamento é quando fazemos com que um sistema ou uma parte dele (por exemplo, uma funcionalidade) se torne disponível aos usuários.

Jez argumenta que, ao separar essas duas ideias, podemos garantir que nosso software funcionará no ambiente de produção sem que as falhas sejam vistas por nossos usuários. Uma implantação azul-verde (blue-green deployment) é um dos exemplos mais simples desse conceito em ação – temos uma versão de nosso software ativa (azul) e, em seguida, implantamos uma nova versão ao lado da versão antiga em produção (verde). Verificamos se a nova versão está funcionando conforme esperado e, se estiver, redirecionamos os clientes para que vejam a nova versão de nosso software. Se encontrarmos algum problema antes da virada, nenhum cliente será impactado.

Embora as implantações azul-verde estejam entre os exemplos mais simples desse princípio, há uma série de outras técnicas mais sofisticadas

que podemos empregar se adotarmos esse conceito.

Em direção a uma entrega progressiva

James Governor, cofundador da firma RedMonk, de análise de mercado com foco no desenvolvedor, criou inicialmente (<https://oreil.ly/1nFrg>) o termo *entrega progressiva* (progressive delivery) para se referir a uma série de técnicas diferentes empregadas nessa área. Ele descreve a entrega progressiva como uma “entrega contínua com um controle minucioso sobre seu raio de ação” (<https://oreil.ly/opHOq>) – portanto é uma extensão da entrega contínua, mas é também uma técnica que nos permite controlar o possível impacto de nosso software recém-lançado.

Dando prosseguimento a esse tema, Adam Zimman e LaunchDarkly descrevem (<https://oreil.ly/zeeNc>) como a entrega progressiva causa impactos “nos negócios”. Partindo desse ponto de vista, é necessária uma mudança de mentalidade quanto ao modo como as novas funcionalidades chegam até nossos clientes. Não haverá mais um único rollout – podemos ter agora uma atividade dividida em fases. Acima de tudo, porém, a entrega progressiva pode dar autonomia ao dono do produto (product owner), conforme diz Adam, “delegando o controle da funcionalidade ao dono do produto que tiver a maior responsabilidade pelo resultado”. Todavia, para que isso funcione, o dono do produto em questão precisa entender o funcionamento da técnica de entrega progressiva em uso, o que implica um dono de produto, de certo modo, experiente do ponto de vista técnico, ou o suporte de um grupo de pessoas devidamente experiente.

Já mencionamos as implantações azul-verde como uma técnica de entrega progressiva. Vamos discutir rapidamente mais algumas técnicas.

Flags de funcionalidade

Com as flags de funcionalidade (feature flags ou feature toggles), ocultamos as funcionalidades implantadas com uma flag (toggle) que pode ser utilizada para ativá-las ou desativá-las. Seu uso é mais comum como parte de um desenvolvimento baseado em tronco, em que é possível fazer o check-in de uma funcionalidade que ainda não foi concluída e implantá-la,

mas de forma oculta aos usuários finais; entretanto, há várias outras aplicações além dessa. A técnica poderia ser conveniente para ativar uma funcionalidade em um horário especificado, ou desativar uma funcionalidade que esteja causando problemas.

Podemos utilizar as flags de funcionalidade de forma mais especializada, talvez permitindo que uma flag tenha um estado diferente de acordo com a natureza do usuário que fizer uma requisição. Assim, poderíamos, por exemplo, ter um grupo de clientes que veria uma funcionalidade ativa (talvez um grupo de testes beta), enquanto a maioria das pessoas a veria desativada – isso poderia ajudar na implementação de um rollout canário, que é algo que discutiremos a seguir. Há soluções totalmente gerenciadas para administrar as flags de funcionalidades, incluindo o LaunchDarkly (<https://launchdarkly.com>) e o Split (<https://www.split.io>). Por mais impressionantes que sejam essas plataformas, acho que você pode começar com algo muito mais simples – somente um arquivo de configuração talvez seja suficiente no início; avalie então essas tecnologias, à medida que começar a pensar melhor em como você vai querer utilizar as flags.

Para uma exploração mais detalhada do mundo das flags de funcionalidades, posso recomendar o artigo “Feature Toggles (aka Feature Flags)” (Toggles de funcionalidades [ou flags de funcionalidades], <https://oreil.ly/5B9ie>), de Pete Hodgson, que dá muitos detalhes quanto ao modo de implementá-las, além de apresentar várias maneiras pelas quais elas podem ser utilizadas.

Lançamento canário

Errar é humano, mas para realmente estragar tudo, você precisa de um computador.⁷

Todos cometemos erros, e os computadores nos permitem cometer erros de modo mais rápido e em uma escala maior do que jamais foi visto antes. Considerando que erros são inevitáveis (e, acredite em mim, eles são), faz sentido tomar atitudes que nos permitam limitar o impacto desses erros. Os lançamentos canário são uma técnica desse tipo.

A técnica recebeu esse nome por causa dos canários que eram levados às

minas como um sistema de aviso prévio aos mineiros, com o intuito de avisá-los da presença de gases tóxicos; com um rollout canário, a ideia é que um subconjunto limitado de nossos clientes verá a nova funcionalidade. Se houver algum problema com o rollout, apenas uma parte de nossos clientes será impactada. Caso a funcionalidade seja bem-sucedida para esse grupo canário, o rollout poderá ser efetuado para outros clientes, até que todos possam ver a nova versão.

Em uma arquitetura de microsserviços, uma flag poderia ser configurada no nível de um microsserviço individual, ativando a funcionalidade (ou desativando-a) para as requisições que ela receber, provenientes do mundo externo ou de outros microsserviços. Outra técnica consiste em ter duas versões diferentes de um microsserviço executando lado a lado e utilizar a flag para encaminhar as requisições para a versão antiga ou para a versão nova. Nesse caso, a implementação do canário terá de estar em algum lugar no path de encaminhamento/rede, em vez de estar em um microsserviço.

Na primeira vez em que fiz um lançamento canário, controlamos o rollout manualmente. Podíamos configurar o percentual de nosso tráfego que podia ver a nova funcionalidade e, no período de uma semana, aumentamos gradativamente esse valor até que todos pudessem vê-la. Durante essa semana, prestamos bastante atenção em nossas taxas de erro, nos relatórios de bugs e em dados desse tipo. Atualmente, é mais comum que esse processo seja tratado automaticamente. Ferramentas como o Spinnaker (<https://www.spinnaker.io>), por exemplo, são capazes de elevar automaticamente o número de chamadas com base em métricas, por exemplo, aumentando a porcentagem das chamadas para uma nova versão do microsserviço caso as taxas de erros estejam em um nível aceitável.

Execução paralela

Em um lançamento canário, uma requisição para uma funcionalidade será atendida pela versão antiga ou pela versão nova. Isso significa que não será possível comparar como as duas versões da funcionalidade tratariam a mesma requisição – algo que pode ser importante se você quiser garantir que a nova funcionalidade se comporte exatamente do mesmo modo que a versão antiga dela.

Com uma execução paralela, fazemos exatamente isso: executamos duas implementações diferentes da mesma funcionalidade lado a lado e enviamos uma requisição à funcionalidade para as duas implementações. Em uma arquitetura de microsserviços, a abordagem mais óbvia seria despachar uma chamada de serviço para duas versões distintas do mesmo serviço e comparar os resultados. Uma alternativa é permitir a coexistência das duas implementações da funcionalidade no mesmo serviço, o que muitas vezes facilitará a comparação.

Ao executar as duas implementações, é importante observar que você provavelmente vai querer apenas o resultado de uma das chamadas. Uma implementação será considerada a fonte da verdade – é a implementação na qual você atualmente confia e, em geral, será a implementação existente. Dependendo da natureza da funcionalidade sendo comparada em uma execução paralela, será preciso pensar cuidadosamente nesse detalhe – você não iria querer, por exemplo, enviar duas atualizações de pedido idênticas para um cliente, nem pagar uma fatura duas vezes.

Eu faço uma exploração do padrão de execução paralela com muito mais detalhes no Capítulo 3 do meu livro *Monolith to Microservices*.⁸ Nessa obra, exploro o uso desse padrão para ajudar na migração de funcionalidades de um sistema monolítico para uma arquitetura de microsserviços; nesse caso, queremos garantir que nosso novo microsserviço se comporte do mesmo modo que a funcionalidade equivalente no sistema monolítico. Em outro contexto, o GitHub faz uso desse padrão quando efetua um retrabalho das partes essenciais de sua base de código, e lançou uma ferramenta open source Scientist (<https://oreil.ly/LXtNJ>) para ajudar no processo. Nesse caso, a execução paralela é feita dentro de um único processo, com o Scientist ajudando a comparar as chamadas.



Com implantações azul-verde, flags de funcionalidades, lançamentos canário e execuções paralelas, mal tocamos a superfície da área de entrega progressiva. Essas ideias podem funcionar bem juntas (já mencionamos como as flags de funcionalidade poderiam ser usadas para implementar um rollout canário, por exemplo), mas você

provavelmente vai querer avançar aos poucos. Para começar, lembre-se apenas de separar os dois conceitos: implantação e lançamento. Em seguida, comece procurando maneiras de ajudar a implantar seu software com mais frequência, porém de forma segura. Trabalhe com o dono de seu produto ou com outros stakeholders de negócios para entender como algumas dessas técnicas poderiam ajudar você a ser mais rápido, mas também contribuir para reduzir as falhas.

Resumo

Tudo bem, abordamos muitos assuntos neste capítulo. Vamos recapitular rapidamente antes de prosseguir. Inicialmente, vamos recordar os princípios de implantação que foram apresentados:

Execução isolada

Execute as instâncias dos microsserviços de forma isolada, de modo que elas tenham os próprios recursos computacionais e sua execução não cause impacto nas instâncias de outros microsserviços executando nas proximidades.

Foco na automação

Escolha as tecnologias que permitam um grau elevado de automação e adote a automação como uma parte essencial de sua cultura.

Infraestrutura como código

Crie uma representação da configuração de sua infraestrutura para facilitar a automação e possibilitar o compartilhamento de informações. Armazene esse código no sistema de controle de versões para permitir que os ambientes sejam recriados.

Vise a implantações sem downtime

Leve as implantações independentes um passo além e garanta que a implantação de uma nova versão de um microsserviço possa ser feita sem qualquer downtime para os usuários de seu serviço (sejam eles pessoas ou outros microsserviços).

Gerenciamento de estado desejado

Utilize uma plataforma que mantenha seu microsserviço em um estado definido, iniciando novas instâncias se for necessário, na eventualidade de haver alguma falha no serviço ou um aumento de tráfego.

Além disso, apresentei minhas próprias diretrizes para selecionar a plataforma correta de implantação:

1. Se não está quebrado, não conserte.²
2. Abstenha-se do máximo de controle com o qual estiver satisfeito, e então se abstenha de um pouquinho a mais. Se puder passar todo o seu trabalho para um bom PaaS como o Heroku (ou para uma plataforma FaaS), faça isso e seja feliz. Você realmente precisa fazer ajustes em cada configuração possível?
3. Colocar seus microsserviços em contêineres não é uma tarefa isenta de dificuldades, mas é realmente um bom compromisso no que diz respeito ao custo do isolamento, além de proporcionar algumas vantagens incríveis para o desenvolvimento local, ao mesmo tempo que dará a você certo nível de controle sobre o que acontece. Espere ter o Kubernetes em seu futuro.

Também é importante *conhecer* os seus requisitos. O Kubernetes poderia ser bastante apropriado para você, mas talvez algo mais simples também pudesse funcionar bem. Não se sinta envergonhado em escolher uma solução mais simples, e não se preocupe demais em passar o trabalho para outra pessoa – se eu puder passar o trabalho para a nuvem pública, vou passar, pois isso me permite ter mais foco em meu próprio trabalho.

Acima de tudo, essa área está passando por muitas mudanças. Espero que eu tenha conseguido apresentar alguns insights sobre as principais tecnologias nessa área, mas que tenha também compartilhado alguns princípios que provavelmente sobreviverão ao conjunto atual de tecnologias da moda. Não importa o que vier a seguir, espero que você esteja muito mais preparado para lidar com a situação.

No próximo capítulo, exploraremos com mais detalhes um tópico que mencionamos rapidamente neste capítulo: testar nossos microsserviços para garantir que eles realmente funcionam.

- 1 Kief Morris, *Infrastructure as Code*, 2ª edição (Sebastopol: O'Reilly, 2020).
- 2 Senthil Padmanabhan e Pranav Jha, “WebAssembly at eBay: A Real-World Use Case” (WebAssembly na eBay: um caso de uso real), eBay, 22 de maio de 2019, <https://oreil.ly/SfvHT>.
- 3 Johnathan Ishmael, “Optimising Serverless for BBC Online” (Otimização de serverless para BBC online), Technology and Creativity at the BBC (blog), BBC, 26 de janeiro de 2021, <https://oreil.ly/gkSdp>.
- 4 Talvez não tenha sido eu quem inventou essa regra.
- 5 Daniel Bryant, “Apple Rebuilds Siri Backend Services Using Apache Mesos” (Apple recria serviços de backend do Siri usando o Apache Mesos), InfoQ, 3 de maio de 2015, <https://oreil.ly/NLUMX>.
- 6 Nicole Forsgren, Jez Humble e Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).
- 7 Essa citação geralmente é atribuída ao biólogo Paul Ehrlich, mas sua verdadeira origem não está clara (<https://oreil.ly/3SOop>).
- 8 Sam Newman, *Monolith to Microservices* (Sebastopol: O'Reilly, 2019). [N.T.: Edição publicada no Brasil: *Migrando sistemas monolíticos para microsserviços* (Novatec, 2020).]
- 9 Talvez não tenha sido eu quem inventou essa regra.

Testes

O mundo dos testes automatizados avançou significativamente desde que comecei a escrever código, e todos os meses parece haver alguma ferramenta ou técnica nova para melhorar mais ainda. Contudo, os desafios relacionados a testar as funcionalidades de nosso código com eficácia e eficiência quando envolvem um sistema distribuído continuam presentes. Este capítulo descreve os problemas associados aos testes de sistemas compostos de partes menores e mais especializadas, além de apresentar algumas soluções para ajudar a garantir que você possa lançar sua nova funcionalidade com confiança.

Os testes incluem vários tópicos. Mesmo quando falamos *somente* de testes automatizados, há vários aspectos a serem considerados. Com os microsserviços, acrescentamos um nível adicional de complexidade. Entender quais são os diferentes tipos de testes que podemos executar é importante para nos ajudar a encontrar um equilíbrio entre as forças ocasionalmente opostas entre levar nosso software para o ambiente de produção o mais rápido possível e garantir que ele tenha qualidade suficiente. Considerando o escopo dos testes como um todo, não tentarei fazer uma exploração abrangente do assunto. Em vez disso, este capítulo tem como foco principal analisar como os testes em uma arquitetura de microsserviços diferem dos testes em sistemas menos distribuídos, por exemplo, em aplicações monolíticas com um único processo.

Os pontos em que os testes são feitos também mudaram desde a primeira edição deste livro. No passado, os testes eram predominantemente executados antes que o software chegasse ao ambiente de produção. Cada vez mais, porém, buscamos testar nossas aplicações assim que chegam nesse ambiente – deixando as linhas entre as atividades relacionadas ao

desenvolvimento e à produção mais indistintas; esse é um assunto que exploraremos neste capítulo, antes de analisar os testes no ambiente de produção de modo mais completo no Capítulo 10.

Tipos de testes

Como muitos consultores, sou culpado por ocasionalmente utilizar quadrantes como uma forma de classificar o mundo, e estava começando a me preocupar com o fato de que este livro não teria um. Felizmente, Brian Marick criou um sistema de classificação incrível e bastante apropriado para os testes. A Figura 9.1 mostra uma variação dos quadrantes de Marick, do livro *Agile Testing*¹, de Lisa Crispin e Janet Gregory, que ajudam a classificar os diferentes tipos de testes.

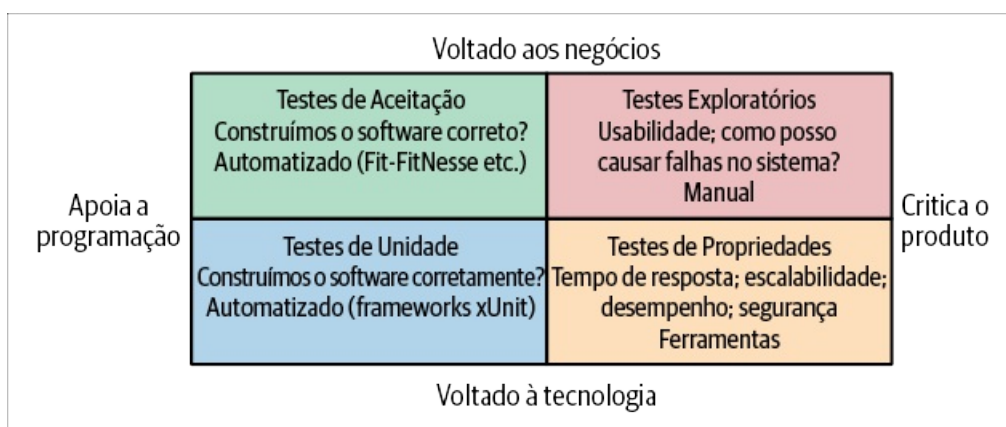


Figura 9.1 – Quadrantes de testes de Brian Marick. Lisa Crispin e Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*, © 2009.

Na parte inferior dos quadrantes, temos testes que são *voltados à tecnologia* – isto é, testes que ajudam os desenvolvedores a criarem o sistema, antes de tudo. Por exemplo, os testes de propriedades, os testes de desempenho e os testes de unidade com escopo pequeno se enquadram nessa categoria; em geral, todos são automatizados. A metade superior dos quadrantes inclui os testes que ajudam os stakeholders que não são da área técnica a entenderem como o seu sistema funciona, e são chamados de testes *voltados aos negócios*. Podem ser testes fim a fim, com escopo maior, conforme exibido no quadrante de Testes de Aceitação na parte superior à

esquerda, ou testes manuais (exemplificado por testes de usuários feitos em um sistema UAT), como mostra o quadrante de Testes Exploratórios.

A essa altura, vale a pena destacar o fato de que a grande maioria desses testes tem como foco uma *validação anterior à produção*. De modo mais específico, usamos esses testes para garantir que o software tenha uma qualidade suficiente antes que seja implantado em um ambiente de produção. Em geral, o fato de o software passar (ou falhar) nesses testes seria uma condição determinante para decidir se ele deve ser implantado.

Cada vez mais, percebemos a importância de testar nosso software assim que ele alcança o ambiente de produção. Falaremos mais sobre o equilíbrio entre essas duas ideias mais adiante neste capítulo, mas, por enquanto, vale a pena ressaltar uma limitação dos quadrantes de Marick quanto a essa questão.

Cada tipo de teste mostrado nesses quadrantes tem seu lugar. A quantidade exata de cada teste que você vai fazer dependerá da natureza de seu sistema, mas o ponto principal a ser compreendido é que há muitas opções para testar o seu sistema. Recentemente, a tendência tem sido se afastar de qualquer teste manual de larga escala em favor de automatizar o máximo possível os testes repetitivos, e eu certamente concordo com essa abordagem. Se você faz atualmente uma grande quantidade de testes manuais, eu sugeriria que resolvesse isso antes de avançar demais pelo caminho dos microsserviços, pois não será possível desfrutar muitas de suas vantagens se você não for capaz de validar seu software com rapidez e eficiência.

Testes exploratórios manuais

Em geral, a mudança de uma arquitetura monolítica para uma arquitetura de microsserviços causará um mínimo de impacto nos testes exploratórios, afora qualquer mudança organizacional mais ampla que possa ocorrer. Conforme será explorado na seção “Em direção a equipes alinhadas a fluxos”, na página [536](#), esperaríamos que a interface de usuário da aplicação também fosse dividida de acordo com as diferentes equipes. Em um contexto como esse, a responsabilidade pelos testes manuais pode mudar bastante.

A capacidade de automatizar algumas tarefas, por exemplo, verificar a aparência de um item, estava antes limitada exclusivamente aos testes exploratórios manuais. O amadurecimento de ferramentas que permitem asserções visuais nos permitiu começar a automatizar tarefas que antes eram feitas manualmente. Contudo, você não deve ver isso como um motivo para não ter nenhum teste manual; em vez disso, encare como uma oportunidade para que os responsáveis pelos testes tenham mais tempo para se concentrar em testes *exploratórios*, menos repetitivos.

Quando feitos de forma apropriada, os testes exploratórios manuais, em sua maior parte, dizem respeito a descobertas. Reservar tempo para explorar a aplicação no papel de um usuário final pode revelar problemas que, de outra maneira, não seriam evidentes. Os testes manuais também podem ser vitais em situações em que a implementação de um teste automatizado seria impraticável, talvez por causa do custo de escrever esse teste. A automação diz respeito a eliminar tarefas repetitivas a fim de dar liberdade às pessoas para que executem atividades *ad hoc* mais criativas. Desse modo, pense na automação como uma forma de podermos ter mais de nossa capacidade cerebral à disposição para as tarefas que fazemos melhor.

Neste capítulo, de modo geral, ignoraremos os testes exploratórios manuais. Isso não quer dizer que esse tipo de testes não seja importante, mas somente que o escopo deste capítulo tem como foco principal o modo como os testes de microsserviços diferem dos testes de aplicações monolíticas mais comuns. Contudo, quando se trata de testes automatizados, quantos testes de cada tipo vamos querer? Há outro modelo que nos ajudará a responder a essa pergunta e a entender quais poderiam ser os diferentes prós e contras.

Escopo dos testes

Em seu livro *Succeeding with Agile*,² Mike Cohn apresenta um modelo chamado de pirâmide de testes para ajudar a explicar quais são os tipos de testes automatizados necessários. A pirâmide nos ajuda a pensar não só no escopo dos testes, mas também nas proporções entre os diferentes tipos de

testes aos quais devemos visar. O modelo original de Cohn separa os testes automatizados em testes de unidade, testes de serviço e testes de UI, conforme mostrado na Figura 9.2.

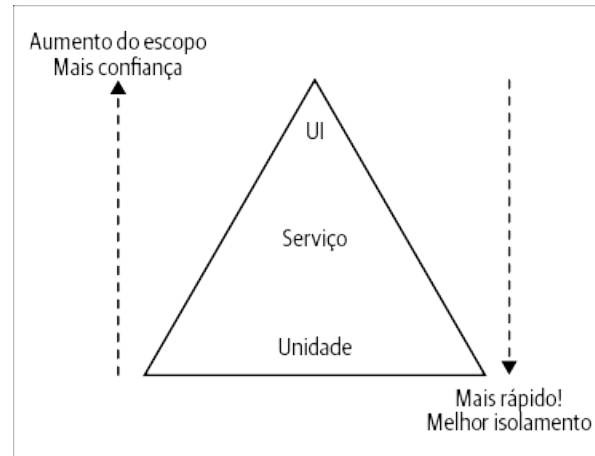


Figura 9.2 – Pirâmide de testes de Mike Cohn. Mike Cohn, Succeeding with Agile: Software Development Using Scrum, 1ª edição, © 2010.

O ponto principal a ser compreendido ao ler a pirâmide é que, à medida que subimos, o escopo dos testes aumenta, assim como a nossa confiança de que a funcionalidade em teste funciona. Por outro lado, a duração do ciclo de feedback aumenta à medida que os testes demoram mais para executar, e se um teste falhar, poderá ser mais difícil determinar qual é a funcionalidade que está com problema. À medida que descemos na pirâmide, em geral os testes serão muito mais rápidos, portanto teremos ciclos de feedback muito mais curtos. Identificamos mais rapidamente as funcionalidades com problema, nossos processos de construção (build) na integração contínua são mais rápidos, e é menos provável que passemos para uma nova tarefa antes de descobrir que causamos algum problema. Quando esses testes de escopo menor falham, também estaremos mais inclinados a saber o que causou o problema, com frequência identificando até mesmo a linha exata de código – cada teste estará mais bem *isolado*, facilitando entender e corrigir as falhas. Por outro lado, não teremos muita confiança de que o nosso sistema como um todo funcionará se tivermos testado somente uma linha de código!

O problema desse modelo é que todos esses termos têm um significado diferente para pessoas diferentes. O termo *serviço* em particular engloba

muitos sentidos, e há várias definições para teste de unidade por aí. Um teste será de unidade se eu testar somente uma linha de código? Eu diria que sim. Continuará sendo um teste de unidade se eu testar várias funções ou classes? Eu diria que não, mas muitas pessoas discordariam! Minha tendência é manter os nomes *unidade* e *serviço* apesar de sua ambiguidade, mas muitos preferem chamar os testes de *UI* de testes *fim a fim* (end-to-end tests), e é isso que farei a partir de agora.

Praticamente todas as equipes com as quais já trabalhei usavam nomes diferentes daqueles utilizados por Cohn na pirâmide. Independentemente do modo como você os chama, o principal ponto a ser compreendido é que você deve ter testes funcionais automatizados com diferentes escopos para diferentes propósitos.

Considerando a confusão, vale a pena discutir o que essas camadas distintas significam.

Vamos analisar um exemplo que criamos. Na Figura 9.3, temos nossa aplicação de helpdesk e nosso site principal, ambos interagindo com o nosso microsserviço *cliente* para obter, verificar e editar detalhes de clientes. Nosso microsserviço *cliente*, por sua vez, conversa com nosso microsserviço *Fidelidade*, que concede pontos aos nossos clientes por comprarem os CDs de Justin Bieber. Provavelmente. É claro que essa é uma porção minúscula do nosso sistema completo da MusicCorp, mas é uma parte boa o bastante para explorarmos alguns cenários diferentes que possamos querer testar.

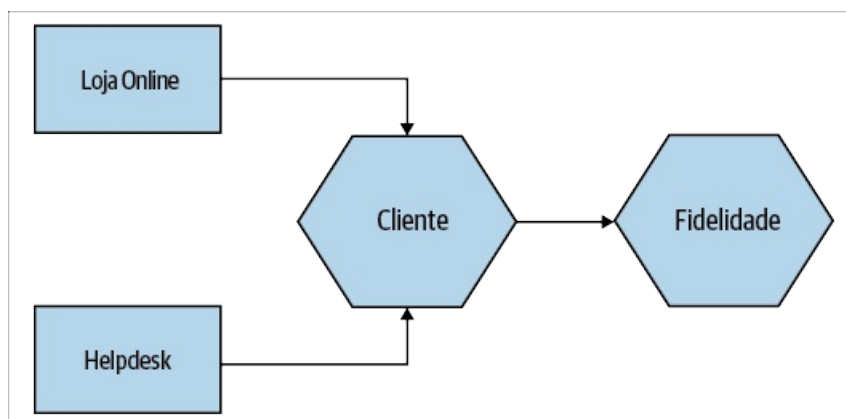


Figura 9.3 – Parte de nossa loja musical em teste.

Testes de unidade

Os testes de unidade geralmente testam uma única chamada de função ou de método. Os testes gerados como efeito colateral do *TDD* (Test-Driven Design, ou Design Orientado a Testes) se enquadrarão nessa categoria, assim como os tipos de testes gerados por técnicas como os testes baseados em propriedades. Não iniciamos os microsserviços nesse caso, e limitamos o uso de arquivos externos ou conexões de rede. Em geral, você vai querer ter uma grande quantidade desses tipos de testes. Se forem feitos corretamente, esses testes serão extremamente rápidos; em um hardware moderno, você poderia esperar executar milhares deles em menos de um minuto. Conheço muitas pessoas que fazem esses testes serem executados de modo automático quando há arquivos alterados localmente – com linguagens interpretadas em particular, podemos ter ciclos muito rápidos de feedback.

Os testes de unidade ajudam os desenvolvedores e, desse modo, seriam voltados à tecnologia, mas não aos negócios, de acordo com a terminologia de Marick. São também o modo como esperamos capturar a maioria de nossos bugs. Assim, em nosso exemplo, quando pensamos no microsserviço `cliente`, os testes de unidade iriam cobrir pequenas partes do código isoladamente, como mostra a Figura 9.4.

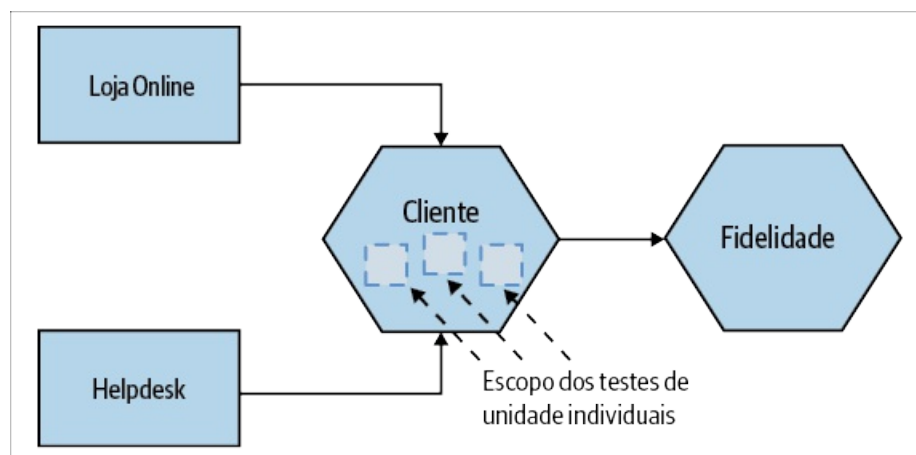


Figura 9.4 – Escopo dos testes de unidade em nosso sistema de exemplo.

O principal objetivo desses testes é nos dar feedbacks muito rápidos para saber se nossa funcionalidade está boa. Os testes de unidade também são

importantes para auxiliar na refatoração de código, permitindo que o nosso código seja reestruturado, pois nos sentiremos mais seguros sabendo que nossos testes de escopo menor nos protegerão caso venhamos a cometer algum erro.

Testes de serviço

Os testes de serviço são concebidos para ignorar a interface de usuário e testar diretamente os nossos microsserviços. Em uma aplicação monolítica, poderíamos simplesmente testar um conjunto de classes que oferece um *serviço* à UI. Em um sistema composto de vários microsserviços, um teste de serviço testaria as capacidades de um único microsserviço.

Ao executar testes em um único microsserviço dessa forma, teremos mais confiança de que o serviço se comportará conforme esperado, porém o escopo do teste continuará até certo ponto isolado. A causa de uma falha no teste deverá estar limitada somente ao microsserviço em teste. Para conseguir esse isolamento, precisamos criar stubs para todos os colaboradores externos, de modo que somente o microsserviço em questão esteja no escopo, como mostra a Figura 9.5.

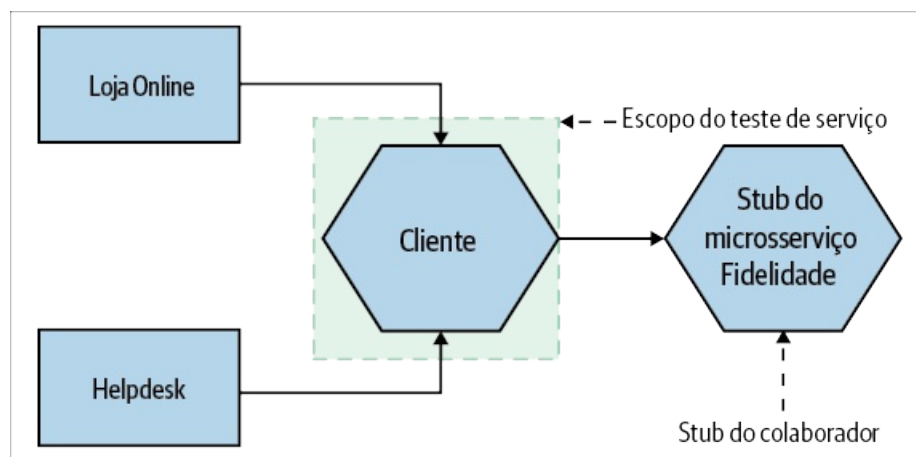


Figura 9.5 – Escopo dos testes de serviço em nosso sistema de exemplo.

Alguns desses testes poderiam ser tão rápidos quanto nossos testes de unidade de escopo menor, mas, se você decidir fazer o teste com um banco de dados de verdade ou se comunicar com stubs de colaboradores downstream por meio da rede, os testes poderão ser mais demorados. Esses testes também têm um escopo maior do que um teste de unidade

simples, de modo que, quando falharem, poderá ser mais difícil detectar a causa da falha, em comparação com o que ocorre com um teste de unidade. No entanto, esses testes têm consideravelmente menos partes e, desse modo, são menos frágeis que os testes com escopos maiores.

Testes fim a fim

Testes fim a fim são os testes executados em todo o sistema. Com frequência, direcionarão uma GUI em um navegador, mas poderiam facilmente simular outros tipos de interação com usuários, por exemplo, fazendo o upload de um arquivo.

Esses testes englobam bastante código de produção, conforme vemos na Figura 9.6. Assim, quando são bem-sucedidos, a sensação é boa: temos bastante confiança de que o código em teste funcionará no ambiente de produção. Contudo, esse escopo maior tem suas desvantagens, e conforme veremos em breve, os testes fim a fim podem ser muito complicados de executar em um contexto de microsserviços.

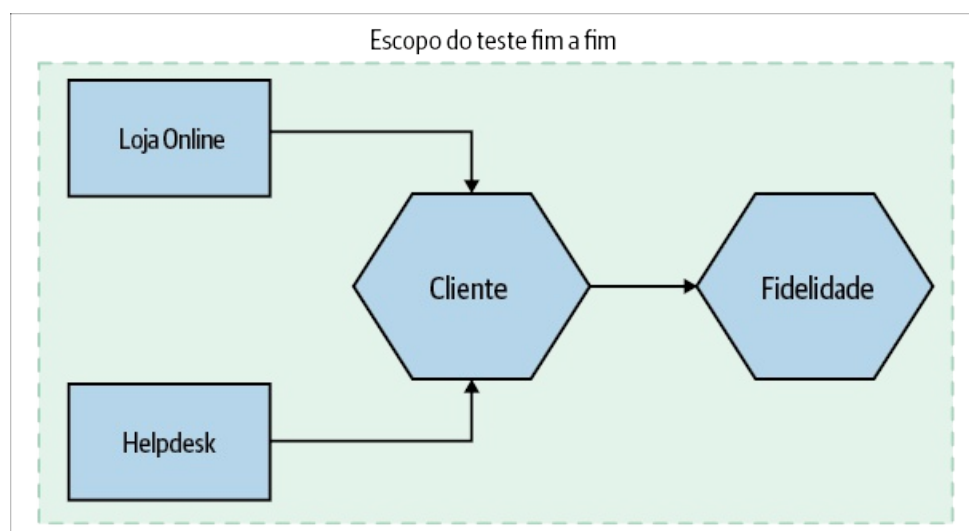


Figura 9.6 – Escopo dos testes fim a fim em nosso sistema de exemplo.

E quanto aos testes de integração?

Você talvez tenha notado que eu não descrevi explicitamente os testes de integração. Isso foi proposital. Tenho constatado que o termo é frequentemente usado por pessoas diferentes para descrever tipos diferentes de testes. Para alguns, um teste de integração poderia

envolver somente as interações entre dois serviços ou, quem sabe, a vinculação entre um código e um banco de dados. Para outros, os testes de integração acabam sendo iguais aos testes fim a fim completos. Tentei utilizar termos mais explícitos neste capítulo; espero que seja fácil para você fazer a correspondência entre o que quer que você chame de “teste de integração” e os termos que empreguei neste livro.

Custo-benefício

O que estamos nos esforçando para fazer com os diferentes tipos de testes incluídos na pirâmide é chegar a um equilíbrio razoável. Queremos feedbacks rápidos, e queremos ter confiança de que o nosso sistema funciona.

Os testes de unidade têm um escopo menor, portanto, quando falham, podemos encontrar rapidamente o problema. Também são rápidos de serem escritos, e são *realmente* rápidos para executar. À medida que o escopo de nossos testes aumenta, temos mais confiança em nosso sistema, porém nosso feedback começa a ser prejudicado, pois os testes demoram mais para executar. Eles também terão um custo maior para serem escritos e mantidos.

Você frequentemente tentará avaliar a quantidade de cada tipo de testes de que precisará para chegar a um número ideal. Você acha que sua suíte de testes está demorando muito para executar? Quando testes com um escopo maior, como os nossos testes de serviço ou os testes fim a fim, falharem, escreva um teste de unidade com um escopo menor para encontrar mais rapidamente o problema. Procure substituir alguns testes de escopo maior (e que são mais lentos) por testes de unidade mais rápidos, com um escopo menor. Por outro lado, se um bug consegue passar despercebido e chegar ao ambiente de produção, esse pode ser um sinal de que há algum teste faltando.

Assim, se todos esses testes têm seus prós e contras, quantos testes de cada tipo você vai querer? Uma boa regra geral é que você provavelmente vai querer uma quantidade de testes que seja uma ordem de grandeza maior à medida que descer pela pirâmide. É importante saber que há diferentes

tipos de testes automatizados, e é importante saber se a sua proporção atual de testes causa algum problema!

Já trabalhei com um sistema monolítico, por exemplo, no qual tínhamos 4.000 testes de unidade, 1.000 testes de serviço e 60 testes fim a fim. Decidimos que, do ponto de vista de feedback, tínhamos testes de serviço e testes fim a fim demais (esses últimos eram os piores ofensores a influenciarem os ciclos de feedback), e, desse modo, nos esforçamos bastante para que a cobertura dos testes fosse feita por testes com escopos menores.

Um antipadrão comum é aquele geralmente chamado de *snow cone*, ou pirâmide invertida de testes. Nesse caso, há poucos ou nenhum teste de escopo pequeno, com toda a cobertura de testes feita com testes de escopo maior. Nesses projetos, os testes em geral são extremamente lentos para executar e os ciclos de feedback são muito demorados. Se esses testes forem executados como parte de uma integração contínua, você não terá muitas builds, e a natureza dos tempos de construção de software implica que uma build poderá ficar com problemas por bastante tempo quando houver alguma falha.

Implementando testes de serviço

Implementar testes de unidade é uma tarefa razoavelmente simples se considerarmos o panorama geral, e há muita documentação por aí que explica como escrevê-los. Os testes de serviço e os testes fim a fim são os mais interessantes, sobretudo no contexto dos microsserviços, portanto é neles que manteremos o foco a seguir.

Nossos testes de serviço devem testar uma funcionalidade em todo o microsserviço, e somente nesse microsserviço. Assim, se quiséssemos escrever um teste de serviço para o microsserviço `cliente` da Figura 9.3, implantaríamos uma instância desse microsserviço – e, conforme já discutimos, criaríamos um stub para o microsserviço `Fidelidade` a fim de garantir que uma falha no teste possa ser mapeada para um problema no microsserviço `cliente` em questão.

Conforme exploramos no Capítulo 7, assim que fizemos o check-in de

nosso software, uma das primeiras tarefas que o nosso processo automático de construção de software fará será criar um artefato binário para o nosso microsserviço – por exemplo, criará uma imagem de contêiner para essa versão de software. Desse modo, implantá-la será uma tarefa muito simples. No entanto, como devemos lidar com a simulação dos colaboradores downstream?

A suíte de testes do nosso serviço deve utilizar stubs dos colaboradores downstream e configurar o microsserviço em teste para que se conecte com os stubs desses serviços. Em seguida, precisamos configurar os stubs de modo que enviem respostas para simular os microsserviços no mundo real.

Mocks ou stubs

Quando falo de usar stubs de colaboradores downstream, quero dizer que criaremos um stub do microsserviço, o qual enviará respostas prontas para requisições conhecidas feitas pelo microsserviço em teste. Por exemplo, posso dizer ao stub do meu microsserviço *Fidelidade* que, quando o saldo do cliente 123 for requisitado, ele deverá devolver 15.000. Para o teste, não importa se o stub será chamado 0, 1 ou 100 vezes. Uma variação disso seria utilizar um mock no lugar de um stub.

Ao usar um mock, vamos um pouco além e garantimos que a chamada foi feita. Se a chamada esperada não for feita, o teste falhará. Implementar essa abordagem exige mais inteligência nos colaboradores simulados que criarmos, e se usada em excesso, pode deixar os testes mais frágeis. Conforme observamos, porém, para um stub, não importa se ele será chamado 0, 1 ou várias vezes.

Às vezes, porém, os mocks podem ser muitos convenientes para garantir que os efeitos colaterais esperados ocorram. Por exemplo, posso querer verificar se, ao criar um cliente, uma nova conta para controle de pontos será criada para esse cliente. O equilíbrio entre o uso de stubs e de mocks é delicado, e é complicado tanto nos testes de serviço quanto nos testes de unidade. Em geral, porém, eu utilizo muito mais stubs do que mocks nos testes de serviço. Para uma discussão mais detalhada dessa relação de custo-benefício, consulte o livro *Growing Object-Oriented Software, Guided*

by Tests, de Steve Freeman e Nat Pryce.³

Em geral, raramente utilizo mocks para esse tipo de testes. No entanto, ter uma ferramenta capaz de implementar tanto mocks quanto stubs é conveniente.

Embora eu ache que os stubs e os mocks sejam muito diferentes, sei que a distinção pode ser confusa para algumas pessoas, sobretudo quando elas mencionam termos como *fakes*, *spies* e *dummies*. Gerard Meszaros chama tudo isso, incluindo os stubs e os mocks, de “Test Doubles” (Dublês de teste, <https://oreil.ly/8Pp2y>).

Um serviço de stubs mais inteligente

Em geral, quando se trata de serviços de stubs, eu mesmo os tenho implementado. Já usei de tudo, do servidor web Apache ao nginx, passando por contêineres com Jetty, ou até mesmo servidores web Python iniciados na linha de comando, a fim de iniciar servidores de stub para casos de teste como esses. É provável que eu tenha repetido o mesmo trabalho várias vezes criando esses stubs. Um antigo colega meu na Thoughtworks, Brandon Byars, provavelmente deve ter feito muitos de nós pouparmos bastante trabalho com o seu servidor de stub/mock chamado mountebank (<http://www.mbtest.org>).

Podemos pensar no mountebank como um pequeno dispositivo de software programável por meio de HTTP. O fato de que ele por acaso foi escrito em NodeJS é totalmente opaco para qualquer serviço que o chame. Quando o mountebank é iniciado, enviamos comandos para ele dizendo que crie um ou mais “impostores”, os quais responderão em uma dada porta, com um protocolo específico (atualmente TCP, HTTP, HTTPS e SMTP são aceitos), e as respostas que esses impostores devem enviar quando houver requisições enviadas. O mountebank também aceita definir expectativas se você quiser usá-lo como um mock. Como uma única instância do mountebank aceita a criação de vários impostores, podemos usá-lo para criar stubs de vários microsserviços downstream.

O mountebank também tem usos que vão além dos testes funcionais automatizados. A Capital One, por exemplo, utilizou o mountebank para

substituir uma infraestrutura existente de mocking para seus testes de desempenho em larga escala.⁴

Uma limitação do mountebank está no fato de não aceitar stubbing para protocolos de mensagens – por exemplo, se quiséssemos garantir que um evento foi devidamente enviado (e talvez recebido) por meio de um broker, seria necessário procurar outra solução para isso. Essa é uma área em que o Pact poderia ajudar – é um assunto que veremos melhor em breve.

Assim, se quiséssemos executar nossos testes de serviço somente para o nosso microserviço `cliente`, poderíamos iniciar esse microserviço e uma instância do mountebank que atuaria como o nosso microserviço `Fidelidade` na mesma máquina. Se esses testes passarem, poderemos implantar imediatamente o serviço `cliente`! Será? E os serviços que chamam o microserviço `cliente` – o `Helpdesk` e a `Loja Online`? Sabemos se fizemos alguma mudança que possa ter causado alguma falha neles? É claro que esquecemos os testes importantes que estão no topo da pirâmide: os testes fim a fim.

Implementando (aqueles complicados) testes fim a fim

Em um sistema de microserviços, as funcionalidades que expomos por meio de nossas interfaces de usuário são disponibilizadas por vários microserviços. O propósito dos testes fim a fim, conforme representado na pirâmide de Mike Cohn, é exercitar a funcionalidade por meio dessas interfaces de usuário com tudo que estiver abaixo dela, a fim de termos algum feedback sobre a qualidade do sistema como um todo.

Assim, para implementar um teste fim a fim, precisamos implantar vários microserviços juntos, e então executar um teste que envolva todos eles. Obviamente, esse teste tem um escopo muito maior, resultando em mais confiança de que o nosso sistema funciona! Por outro lado, eles estão sujeitos a serem mais lentos e podem dificultar o diagnóstico das falhas. Vamos explorá-los um pouco mais utilizando nosso exemplo anterior para ver como esses testes se encaixam no panorama geral.

Suponha que você queira lançar uma nova versão do microserviço `cliente`.

Queremos implantar nossas alterações no ambiente de produção o mais rápido possível, mas estamos preocupados com a possibilidade de introduzir uma mudança que possa causar problemas tanto no Helpdesk como na Loja Online. Tudo bem – vamos implantar todos os nossos serviços juntos e executar alguns testes envolvendo o Helpdesk e a Loja Online para ver se introduzimos algum bug. Uma abordagem ingênua seria simplesmente acrescentar esses testes no final do pipeline do nosso serviço cliente, como vemos na Figura 9.7.

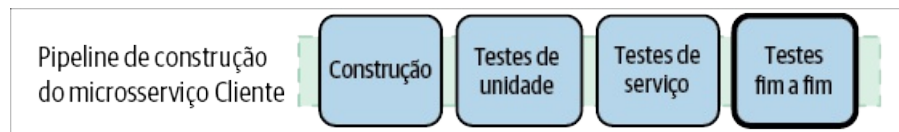


Figura 9.7 – Acrescentando nossa etapa de testes fim a fim: é a abordagem correta?

Até agora, tudo bem. No entanto, a primeira pergunta que devemos fazer a nós mesmos é: qual versão dos outros microsserviços devemos usar? Devemos executar nossos testes com as versões do Helpdesk e da Loja Online que estão no ambiente de produção? É uma suposição razoável – mas e se uma nova versão do Helpdesk ou da Loja Online estiver prestes a ser lançada? O que devemos fazer nesse caso?

Eis outro problema: se tivermos um conjunto de testes fim a fim para cliente, que impliquem a implantação de vários microsserviços com os quais os testes serão executados, como serão os testes fim a fim executados pelos outros microsserviços? Se eles fizerem os mesmos testes, poderemos nos ver cobrindo muitas áreas idênticas, duplicando boa parte do trabalho de implantação de todos esses microsserviços.

Podemos lidar com esses dois problemas de modo elegante, com vários pipelines efetuando um “fan-in” para uma única etapa de testes fim a fim. Nesse caso, quando um de uma série de processos diferentes de construção de software for disparado, ele poderá resultar no disparo de passos de construção compartilhados. Por exemplo, na Figura 9.8, um processo de construção bem-sucedido para qualquer um dos quatro microsserviços acabaria disparando a etapa compartilhada de testes fim a fim. Algumas ferramentas de CI com suporte melhor para pipelines de construção já

incluem a possibilidade de usar modelos de fan-in como esse.

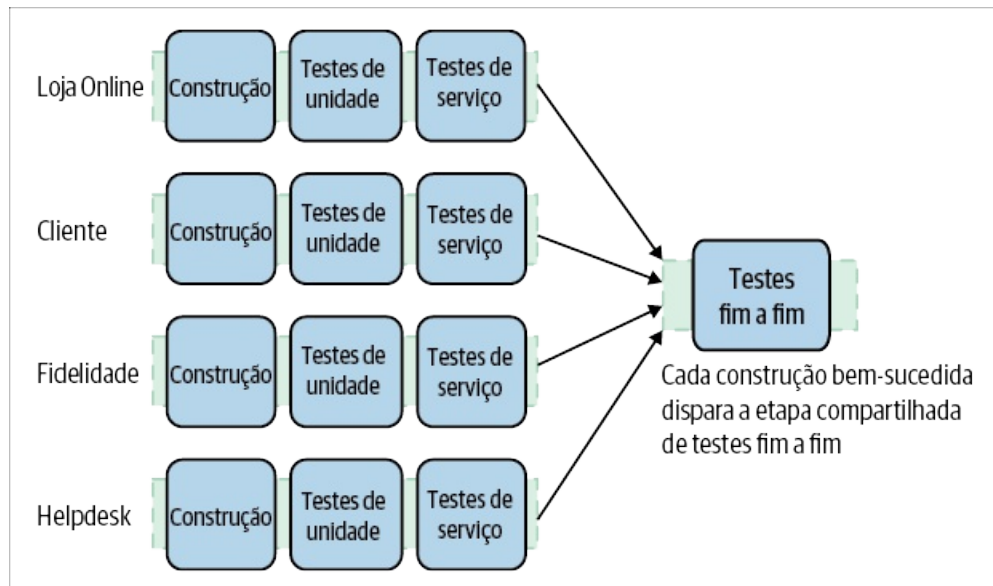


Figura 9.8 – Uma forma padrão de lidar com testes fim a fim envolvendo vários serviços.

Sempre que qualquer um de nossos serviços for alterado, executamos os testes locais a esse serviço. Se esses testes passarem, disparamos nossos testes de integração. Muito bom, não é mesmo? Bem, infelizmente, há muitas desvantagens nos testes fim a fim.

Testes frágeis e instáveis

À medida que o escopo dos testes aumenta, o mesmo ocorre com o número de partes envolvidas. Essas partes podem introduzir falhas nos testes que não mostram um erro na funcionalidade, mas sinalizam que algum outro problema ocorreu. Como exemplo, se tivermos um teste para verificar se podemos fazer o pedido de um único CD, e executarmos esse teste envolvendo quatro ou cinco microsserviços, se um deles estiver inativo, poderíamos ter uma falha que não tem nenhuma relação com a natureza do teste em questão. De modo semelhante, um glitch de rede temporário poderia fazer com que um teste falhasse, sem que nada pudesse ser concluído a respeito da funcionalidade em teste.

Quanto mais partes estiverem envolvidas, mais frágeis e menos determinísticos poderão ser os nossos testes. Se você tiver testes que

falham *de vez em quando*, mas todos simplesmente os executam novamente porque esses testes poderiam passar depois, você terá testes frágeis. E testes que incluem vários processos diferentes não são os únicos culpados. Testes que incluem uma funcionalidade exercitada em várias threads (e que envolvem vários processos) também são geralmente problemáticos; uma falha poderia implicar que houve uma condição de concorrência (race condition) ou um timeout, ou que a funcionalidade pode realmente estar com falha. Testes frágeis são um inimigo. Quando falham, esses testes não nos dizem muito. Executamos novamente nossas builds de CI na esperança de que esses testes passarão novamente mais tarde, somente para ver os check-ins se acumularem e, repentinamente, nos vemos com muitas funcionalidades com falha.

Ao detectar a presença de testes frágeis, é essencial nos esforçarmos ao máximo para eliminá-los. Caso contrário, começaremos a perder a confiança em uma suíte de testes que “sempre falha dessa maneira”. Uma suíte de testes com testes frágeis pode se tornar uma vítima daquilo que Diane Vaughan chama de *normalização do desvio* (normalization of deviance) – a ideia de que, com o tempo, podemos ficar tão acostumados com algo errado, a ponto de começarmos a aceitá-lo como sendo normal, e não como um problema.⁵ Essa tendência muito humana indica que precisamos encontrar e eliminar esses testes frágeis o mais rápido possível, antes de começarmos a aceitar que testes com falhas não são um problema.

Em “Eradicating Non-Determinism in Tests,”⁶ Martin Fowler defende a abordagem de que, se você tem testes frágeis, deve identificá-los e – se não puder corrigi-los imediatamente – deve removê-los da suíte de testes para poder lidar com eles. Veja se é possível reescrevê-los a fim de evitar testar códigos que executem em várias threads. Veja se você consegue deixar o ambiente subjacente mais estável. Melhor ainda, veja se consegue substituir o teste frágil por um teste de escopo menor, com menos chances de apresentar problemas. Em alguns casos, alterar o software para facilitar seu teste também poderia ser o caminho certo a ser seguido.

Quem escreve esses testes fim a fim?

Com os testes que executam como parte do pipeline de um microsserviço específico, o ponto de partida sensato é que a equipe responsável por esse serviço escreva esses testes (falaremos mais da responsabilidade pelo serviço no Capítulo 15). No entanto, se considerarmos que pode haver várias equipes envolvidas, e que o passo dos testes fim a fim é efetivamente compartilhado entre as equipes, quem escreverá e cuidará da manutenção desses testes?

Já vi uma série de problemas nesse caso. Esses testes se tornam acessíveis a qualquer um, com todas as equipes com acesso permitido para adicionar testes, sem saber se a suíte como um todo está saudável. Com frequência, isso pode resultar em uma explosão de casos de teste, culminando ocasionalmente no “snow cone” (pirâmide invertida) de testes sobre o qual falamos antes. Também já vi situações em que, por não haver nenhum óbvio responsável por esses testes, seus resultados eram ignorados. Quando falhavam, todos supunham que era problema de outra pessoa, portanto não se importavam se os testes passavam ou não.

Uma solução que já vi nesse caso é designar determinados testes fim a fim como sendo da responsabilidade de uma dada equipe, ainda que envolvam vários microsserviços com várias equipes diferentes trabalhando nesses serviços. Ouvi falar dessa abordagem pela primeira vez com Emily Bache.⁷ A ideia é que, mesmo que façamos uso de uma etapa de “fan in” em nosso pipeline, a suíte de testes fim a fim seria dividida em grupos de funcionalidades sob responsabilidade de diferentes equipes, conforme vemos na Figura 9.9.

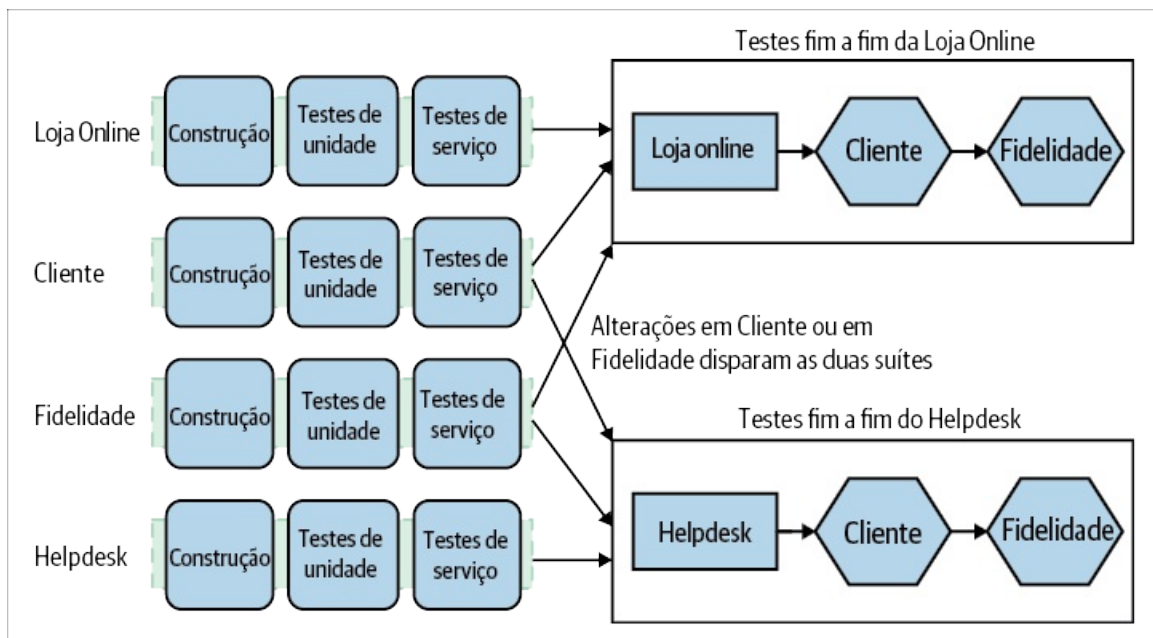


Figura 9.9 – Uma forma padrão de lidar com testes fim a fim envolvendo vários serviços.

Nesse exemplo em particular, uma alteração em `Loja Online` que tenha passado na etapa de testes de serviço dispararia os testes fim a fim associados, com a suíte estando sob responsabilidade da mesma equipe responsável pela `Loja Online`. De modo semelhante, qualquer alteração em `Helpdesk` dispararia somente os testes fim a fim associados. No entanto, alterações em `Cliente` ou em `Fidelidade` disparariam *ambos* os conjuntos de testes. Isso pode nos levar a uma situação em que uma mudança feita no microsserviço `Fidelidade` pudesse causar falha nos dois conjuntos de testes fim a fim, possivelmente exigindo que as equipes responsáveis por essas duas suítes de testes procurassem o responsável pelo microsserviço `Fidelidade` para que uma correção fosse feita. Embora esse modelo tenha ajudado no caso de Emily, como podemos ver, ele ainda tem seus problemas. Basicamente, é problemático ter uma equipe responsável por testes que podem falhar por causa do trabalho de pessoas de uma equipe diferente.

Às vezes, as empresas reagem criando uma equipe dedicada para escrever esses testes. Isso pode ser desastroso. A equipe que desenvolve o software passa a ficar cada vez mais distante dos testes de seu código. A duração dos ciclos aumenta, pois os responsáveis pelos serviços acabam tendo que

esperar que a equipe de testes escreva os testes fim a fim para a funcionalidade que acabaram de implementar. Como é outra equipe que escreve esses testes, a equipe que implementou o serviço terá menos envolvimento e, desse modo, será menos provável que saiba como executar e corrigir esses testes. Embora, infelizmente, esse continue sendo um padrão organizacional comum, vejo que há prejuízos significativos sempre que uma equipe se distancia da escrita dos testes para o código que ela mesma escreveu.

Fazer essa tarefa corretamente é muito difícil. Não queremos duplicar esforços, nem queremos que essa atividade fique totalmente centralizada a ponto de as equipes que criam os serviços fiquem distantes demais dos testes. Se puder encontrar uma forma simples de atribuir os testes fim a fim a uma equipe específica, então faça isso. Caso contrário, se não puder encontrar uma maneira de eliminar os testes fim a fim e substituí-los por outros tipos de testes, é provável que você vá ter de tratar a suíte de testes fim a fim como uma base de código compartilhada, porém com uma responsabilidade compartilhada. As equipes terão liberdade para fazer check-in nessa suíte, mas a responsabilidade por manter a suíte saudável deve ser compartilhada entre as equipes que desenvolvem os serviços em questão. Se quiser fazer uso intensivo de testes fim a fim com várias equipes, acho que essa abordagem é essencial; apesar disso, vejo-a sendo raramente utilizada, e nunca sem que haja problemas. Em última instância, estou convencido de que, por esse motivo, em um certo nível de escala organizacional, você terá de se afastar dos testes fim a fim que envolvam várias equipes.

Por quanto tempo os testes fim a fim devem ser executados?

Os testes fim a fim podem demorar um pouco. Já vi testes durarem um dia para executar, se não mais; em um projeto no qual trabalhei, uma suíte completa de regressão demorava seis semanas! Raramente vejo equipes realmente organizarem seus testes fim a fim de modo a reduzir as sobreposições na cobertura dos testes, nem dedicar tempo suficiente para torná-los mais rápidos.

Essa lentidão, somada ao fato de esses testes muitas vezes serem frágeis, pode ser um grande problema. Uma suíte de testes que demore um dia todo e que frequentemente apresente falhas que não tenham nenhuma relação com uma funcionalidade com problema é um desastre. Mesmo que sua funcionalidade *tenha* falhas, poderia demorar horas para que você descobrisse – a essa altura, é provável que você já estivesse envolvido com outras atividades, e a mudança de contexto para se concentrar de volta na correção do problema seria complicada.

Podemos melhorar um pouco executando testes em paralelo – por exemplo, fazendo uso de ferramentas como o Selenium Grid. No entanto, essa abordagem não é um substituto para realmente compreender o que deve ser testado e *eliminar* os testes que não sejam mais necessários.

Remover testes às vezes pode ser uma tarefa que causa apreensão, e suspeito que aqueles que já tentaram têm muito em comum com as pessoas que querem eliminar determinadas medidas de segurança nos aeroportos. Não importa quão ineficazes as medidas de segurança possam ser, qualquer discussão sobre eliminá-las geralmente é recebida com reações automáticas sobre não se importar com a segurança das pessoas ou querer que os terroristas vençam. É difícil ter uma discussão equilibrada sobre o valor que algo agrega *versus* o fardo que isso implica. Também pode haver uma relação de custo-benefício difícil entre risco e recompensa. Alguém agradecerá se você eliminar um teste? Talvez. Mas, com certeza, você será acusado caso um teste que você tenha removido permitir que um bug passe despercebido. Quando se trata das suítes de testes com escopos maiores, porém, é exatamente o que devemos ser capazes de fazer. Se a mesma funcionalidade é coberta por vinte testes diferentes, talvez possamos nos livrar da metade deles, pois esses vinte testes demoram dez minutos para executar! O que essa situação exige é uma melhor compreensão dos riscos – algo no qual os seres humanos são reconhecidamente ruins. Como resultado, essa organização e gerenciamento inteligentes dos testes com escopos maiores, que constituem um fardo pesado, são incrivelmente raros. Desejar que as pessoas façam isso com mais frequência não é o mesmo que fazer com que esse trabalho aconteça.

Acúmulo de alterações

Os ciclos demorados de feedback associados aos testes fim a fim não são um problema somente no que diz respeito à produtividade dos desenvolvedores. Com uma suíte de testes demorada, a correção de qualquer falha será mais lenta, o que reduz o prazo esperado para que os testes fim a fim passem. Se implantarmos somente os softwares que passaram por todos os nossos testes com sucesso (e é o que devíamos fazer), isso significa que menos de nossos serviços chegarão ao ponto de estarem prontos para serem implantados no ambiente de produção.

Essa situação pode levar a um acúmulo de alterações. Enquanto uma etapa de testes de integração com falha estiver sendo corrigida, outras alterações de equipes upstream poderão se acumular. Afora o fato de que isso pode dificultar a correção da build, significa que o escopo das alterações a serem implantadas aumentará. O modo ideal de lidar com essa situação é não deixar que as pessoas façam check-in se houver falhas nos testes fim a fim, mas considerando que uma suíte de testes demore para executar, essa solução geralmente seria impraticável. Experimente dizer: “Vocês, trinta desenvolvedores: não façam check-ins até corrigirmos essa build de sete horas de duração!”. No entanto, permitir check-ins com uma suíte de testes fim a fim com falhas seria realmente corrigir o problema errado. Se você permitir check-ins em uma build com problema, ela poderá permanecer mais tempo com problemas, prejudicando sua eficácia na hora de proporcionar feedbacks rápidos sobre a qualidade do código. A resposta correta é deixar a suíte de testes mais rápida.

Quanto maior o escopo de uma implantação e quanto maior o risco de um lançamento, maiores serão as chances de introduzirmos um problema. Assim, queremos garantir que seja possível lançar alterações pequenas e bem testadas com frequência. Se os testes fim a fim diminuem nossa capacidade de lançar alterações pequenas, eles poderão trazer mais danos do que vantagens.

Metaversão

Com o passo associado aos testes fim a fim, é fácil começar a pensar o

seguinte: *Sei que todos esses serviços nessas versões funcionam em conjunto, então por que não os implantamos juntos?* Isso rapidamente levará a uma conversa nesta linha: *Então por que não usamos um número de versão para todo o sistema?* Para citar Brandon Byars (<https://oreil.ly/r7Mzz>), “Agora você tem 2.1.0 problemas”.

Ao atribuir o mesmo número de versão para alterações feitas em vários serviços, abraçamos efetivamente a ideia de que alterar e implantar vários serviços de uma só vez é aceitável. Isso passa a ser a norma; passa a ser aceitável. Ao fazer isso, renunciamos a uma das principais vantagens de uma arquitetura de microsserviços: a capacidade de implantar um serviço por si só, independentemente de outros serviços.

Com muita frequência, a abordagem de aceitar a implantação de vários serviços em conjunto se transforma em uma situação na qual os serviços passam a ficar acoplados. Sem demora, serviços que estavam devidamente separados se tornam cada vez mais entrelaçados sem que você perceba, pois jamais havia tentado implantá-los de forma independente. Você acabará com um emaranhado confuso; terá de coordenar a implantação de vários microsserviços de uma só vez e, conforme já discutimos antes, esse tipo de acoplamento pode nos colocar em uma situação pior do que estaríamos se tivéssemos uma única aplicação monolítica.

Isso é ruim.

Incapacidade de testar de forma independente

Voltamos frequentemente ao tópico das implantações independentes serem uma propriedade importante para que seja mais fácil às equipes trabalharem com mais autonomia, permitindo que o software seja lançado com mais eficácia. Se sua equipe trabalha de modo independente, segue-se daí que ela deveria ser capaz de fazer testes de modo independente. Conforme vimos, os testes fim a fim podem reduzir a autonomia das equipes, podendo exigir que haja níveis cada vez maiores de coordenação, com os desafios associados resultantes dessa situação.

A mudança em direção à capacidade de testar de modo independente se estende ao uso da infraestrutura associada aos testes. Com frequência, vejo

pessoas tendo de fazer uso de ambientes de teste compartilhados, nos quais testes de várias equipes são executados. Um ambiente como esse geralmente tem muitas restrições, e qualquer problema pode causar outros problemas significativos. O ideal é que, se quiser que suas equipes possam desenvolver e testar seus códigos de modo independente, elas devam ter os próprios ambientes de teste também.

A pesquisa resumida no livro *Accelerate* constatou que equipes de alto desempenho tinham mais chances de “fazer a maior parte de seus testes por demanda, sem exigir um ambiente de testes integrado”.⁸

Devemos evitar testes fim a fim?

Apesar das desvantagens que acabamos de mencionar, para muitos usuários, os testes fim a fim ainda podem ser administráveis se houver poucos microsserviços; nessas situações, eles ainda fazem muito sentido. Mas o que acontecerá se houver 3, 4, 10 ou 20 serviços? Rapidamente, essas suítes de testes se tornarão extremamente infladas e, no pior caso, poderão resultar em uma explosão cartesiana dos cenários em teste.

Com efeito, mesmo com poucos microsserviços, será difícil se houver várias equipes compartilhando os testes fim a fim. Com uma suíte de testes fim a fim compartilhada, seu objetivo de ter implantações independentes será prejudicado. Para que sua equipe seja capaz de implantar um microsserviço, será necessário agora que os testes de uma suíte compartilhada por várias equipes passem.

Qual é um dos principais problemas que estamos tentando resolver quando utilizamos os testes fim a fim descritos anteriormente? Estamos tentando garantir que, quando implantamos um novo serviço no ambiente de produção, nossas mudanças não causarão falhas aos consumidores. Conforme discutimos em detalhes na seção “Incompatibilidades de contrato estruturais versus semânticas”, na página [175](#), ter esquemas explícitos para as interfaces de nossos microsserviços pode nos ajudar a identificar incompatibilidades estruturais, e isso, sem dúvida, pode reduzir a necessidade de termos testes fim a fim mais complexos.

No entanto, os esquemas não são capazes de capturar incompatibilidades semânticas, isto é, alterações no *comportamento* que causem falhas em virtude de uma incompatibilidade com versões anteriores. Os testes fim a fim definitivamente podem ajudar a identificar essas incompatibilidades semânticas, mas fazem isso com um custo alto. O ideal é que tivéssemos algum tipo de teste capaz de identificar alterações que causem incompatibilidades semânticas e que execute em um escopo reduzido, melhorando o isolamento dos testes (e, desse modo, permita um feedback mais rápido). É nesse cenário que os testes de contrato e os contratos orientados a consumidores entram em cena.

Testes de contrato e CDCs

Com os *testes de contrato*, uma equipe cujo microserviço consuma um serviço externo escreve testes que descrevem como ela espera que um serviço externo vá se comportar. Isso diz menos respeito a testar o seu próprio microserviço, e mais sobre especificar como você espera que um serviço externo irá se comportar. Um dos principais motivos para esses testes de contrato serem úteis é o fato de poderem ser executados com qualquer stub ou mock que você utilizar, e que representem os serviços externos – seus testes de contrato devem passar quando você executar seus próprios stubs, assim como deveriam passar com um serviço externo real.

Os testes de contrato passaram a ser muito úteis quando usados como parte dos CDCs (Consumer-Driven Contracts, ou Contratos Orientados a Consumidores). Os testes de contrato, na verdade, são uma representação explícita, por meio de código, de como o microserviço consumidor (upstream) espera que o microserviço produtor (downstream) se comporte. Com os CDCs, a equipe do consumidor garante que esses testes de contrato sejam compartilhados com a equipe do produtor para permitir que a equipe do produtor se certifique de que seu microserviço atenderá a essas expectativas. Em geral, isso é feito fazendo com que a equipe do produtor downstream execute os contratos do consumidor para cada microserviço consumidor como parte de sua suíte de testes, que seria executada a cada processo de construção. Muito importante do ponto de vista do feedback, esses testes devem ser executados somente com um

único produtor isolado, para que possam ser mais rápidos e mais confiáveis do que os testes fim a fim que eles poderiam estar substituindo.

Como exemplo, vamos retomar o nosso cenário anterior. O `microserviço cliente` tem dois consumidores diferentes: o `Helpdesk` e a `Loja Online`. As duas aplicações consumidoras têm expectativas quanto ao comportamento do `microserviço cliente`. Nesse exemplo, criamos um conjunto de testes para cada consumidor: um que representa as expectativas do `Helpdesk` para o `microserviço cliente`, e outro conjunto que representa as expectativas da `Loja Online`.

Como esses CDCs representam as expectativas acerca de como o `microserviço cliente` deve se comportar, precisamos executar somente o próprio `microserviço cliente`, o que significa que teremos o mesmo escopo efetivo de teste que nossos testes de serviço. Esses teriam características de desempenho semelhantes e exigiriam que executássemos somente o próprio `microserviço cliente`, com stubs para quaisquer dependências externas.

Uma boa prática nesse caso é ter uma pessoa da equipe do produtor e outra da equipe do consumidor trabalhando em conjunto para criar os testes – desse modo, pessoas das equipes da `Loja Online` e do `Helpdesk` talvez pudessem trabalhar junto com pessoas da equipe do `serviço cliente`. Sem dúvida, os contratos orientados a consumidores dizem muito mais respeito a promover linhas claras de comunicação e de colaboração entre os `microserviços` e as equipes que os consomem quando for necessário. Poderíamos argumentar que, de fato, a implementação de CDCs apenas deixa mais explícita uma comunicação que já deveria existir entre as equipes. Na colaboração entre diferentes equipes, os CDCs são um lembrete explícito da lei de Conway.

Os CDCs estão no mesmo nível que os testes de serviço na pirâmide de testes, embora tenham um foco muito diferente, como vemos na Figura 9.10.

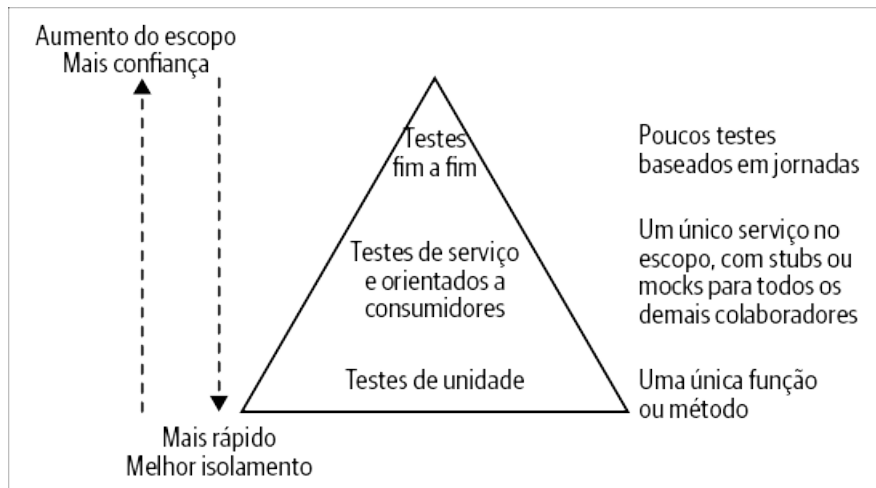


Figura 9.10 – Integrando os testes orientados a consumidores na pirâmide de testes.

Esses testes têm como foco o modo como um consumidor utilizará o serviço, e aquilo que esses testes acionam caso não passem será muito diferente se comparado aos testes de serviço. Se um desses CDCs falhar durante a construção do serviço Cliente, o consumidor que será impactado será evidente. Nesse ponto, você poderá corrigir o problema, ou então dar início a uma discussão sobre a introdução de uma alteração que causará uma incompatibilidade, nos moldes que discutimos na seção “Lidando com mudanças entre microsserviços”, na página [177](#). Desse modo, com os CDCs, podemos identificar uma mudança que vá causar incompatibilidade antes de nosso software alcançar o ambiente de produção, sem precisar de um teste fim a fim possivelmente custoso.

Pact

O Pact (<https://pact.io>) é uma ferramenta de testes orientada a consumidores, originalmente desenvolvida pela realestate.com.au para uso interno, mas que hoje em dia é open source. No início, o Pact foi escrito somente para Ruby e tinha como foco apenas os protocolos HTTP; atualmente ele aceita várias linguagens e plataformas, como JVM, JavaScript, Python e .NET, e pode ser usado também em interações com mensagens.

Com o Pact, começamos definindo as expectativas do produtor utilizando uma DSL em uma das linguagens aceitas. Em seguida, iniciamos um

servidor Pact local e executamos aí essa expectativa a fim de criar um arquivo de especificação Pact. O arquivo Pact é somente uma especificação JSON formal; é claro que poderíamos criá-lo manualmente, porém utilizar o SDK específico para a linguagem é muito mais fácil.

Uma propriedade realmente interessante desse modelo é que o servidor de mock executado localmente para gerar o arquivo Pact também funciona como um stub local para microsserviços downstream. Ao definir suas expectativas localmente, definimos como esse serviço de stubs local deve responder. Ele pode substituir a necessidade de ferramentas como o mountebank (ou as soluções de stubbing ou mocking que você mesmo tenha criado).

Do lado do produtor, verificamos então se a especificação desse consumidor é atendida utilizando a especificação Pact em JSON para direcionar chamadas para o seu microsserviço e verificar as respostas. Para que isso funcione, o produtor precisa acessar o arquivo Pact. Conforme discutimos antes na seção “Mapeando códigos-fontes e builds aos microsserviços”, na página 248, esperamos que o consumidor e o produtor estejam em builds distintas. Isso significa que precisamos ter alguma maneira para que esse arquivo JSON, que será gerado para a build do consumidor, seja disponibilizado pelo produtor.

Poderíamos armazenar o arquivo Pact no repositório de artefatos de nossa ferramenta de CI/CD, ou utilizar o Pact Broker (<https://oreil.ly/kHTkY>), que permite armazenar várias versões de suas especificações Pact. Isso possibilitaria a execução de seus testes de contrato orientados a consumidores com várias versões distintas dos consumidores, caso você quisesse testar, digamos, a versão do consumidor que está no ambiente de produção e a versão do consumidor do processo de construção mais recente.

O Pact Broker, na verdade, tem uma série de recursos úteis. Além de servir como um local onde os contratos podem ser armazenados, também podemos descobrir quando esses contratos foram validados. Além disso, como o Pact Broker conhece o relacionamento entre o consumidor e o produtor, ele é capaz de mostrar as dependências entre os microsserviços.

Outras opções

O Pact não é a única opção de ferramenta relacionada aos contratos orientados a consumidores. O Spring Cloud Contract (<https://oreil.ly/fjufx>) é um desses exemplos. No entanto, vale a pena observar que – de modo diferente do Pact, que foi criado desde o princípio para aceitar diferentes conjuntos de tecnologias – o Spring Cloud Contract é útil somente em um ecossistema puramente JVM.

É uma questão de discutir

No Agile, muitas vezes nos referimos às histórias como um local em que uma discussão é sintetizada. O mesmo acontece com os CDCs. Eles passam a ser uma codificação de um conjunto de discussões sobre como deve ser a API de um serviço, e, quando falham, se transformam em um ponto de partida para discussões sobre como essa API deve evoluir.

É importante entender que os CDCs exigem uma boa comunicação e confiança entre o serviço consumidor e o serviço produtor. Se as duas partes estiverem na mesma equipe (ou forem a mesma pessoa!), isso não deverá ser difícil. No entanto, se você consome um serviço disponibilizado por um terceiro, talvez não tenha a frequência de comunicação, ou a confiança, para fazer os CDCs funcionarem. Nessas situações, você talvez precise se contentar com testes de integração limitados, com escopos maiores, para os componentes *não confiáveis*. Como alternativa, se você está criando uma API para milhares de possíveis consumidores, por exemplo, uma API de web service disponível publicamente, pode ser que seja necessário desempenhar você mesmo o papel de consumidor (ou talvez trabalhar com um subconjunto de seus consumidores) ao definir esses testes. Causar falhas em um número enorme de consumidores externos é uma péssima ideia, e isso, no mínimo, aumenta mais ainda a importância dos CDCs!

Palavra final

Conforme já foi apresentado em detalhes neste capítulo, os testes fim a fim têm diversas desvantagens que aumentam significativamente à medida que

mais partes são acrescentadas aos testes. Com base em conversas com pessoas que vêm implementando microsserviços em escala há um tempo, constatei que a maioria delas, com o tempo, elimina totalmente a necessidade de ter testes fim a fim em favor de outros métodos para validar a qualidade de seus softwares – por exemplo, com o uso de esquemas explícitos e CDCs, com testes em produção ou, talvez, com algumas das técnicas de entrega progressiva que discutimos, por exemplo, os lançamentos canário.

Podemos encarar a execução de testes fim a fim anteriores à implantação no ambiente de produção como as rodinhas auxiliares de uma bicicleta. Enquanto você aprende como os CDCs funcionam e aperfeiçoa suas técnicas de monitoração do ambiente de produção e de implantação, esses testes fim a fim podem servir como uma rede de proteção conveniente; com eles, ponderamos entre o tempo de duração de um ciclo e a diminuição dos riscos. Contudo, à medida que promover melhorias nessas outras áreas, e à medida que o custo relativo de criar testes fim a fim aumentar, sua dependência em relação a esses testes poderá diminuir, até chegar a um ponto em que eles não sejam mais necessários. Contudo, eliminar os testes fim a fim sem compreender totalmente o que você perderá provavelmente não é uma boa ideia.

Obviamente você deve conhecer melhor do que eu o perfil de risco de sua própria empresa, mas eu o desafiaria a pensar muito bem na quantidade de testes fim a fim que você realmente precisa fazer.

Experiência dos desenvolvedores

Um dos desafios significativos que podem surgir quando os desenvolvedores descobrem que precisam trabalhar com uma quantidade cada vez maior de microsserviços é que sua experiência pode começar a sofrer, pelo simples motivo de tentarem executar cada vez mais microsserviços localmente. Com frequência, ocorre em situações em que um desenvolvedor precisa executar um teste com um escopo maior, envolvendo vários microsserviços sem stubs.

A rapidez com que isso se torna um problema dependerá de uma série de

fatores. A quantidade de microsserviços que um desenvolvedor precisa executar localmente, o conjunto de tecnologias com o qual esses microsserviços foram escritos e a potência da máquina local – tudo isso pode influenciar. Alguns conjuntos de tecnologias exigem mais recursos já de partida – os microsserviços baseados em JVM vêm à mente. Por outro lado, outros conjuntos de tecnologia podem resultar em microsserviços com menos uso de recursos ou com usos mais rápidos, talvez permitindo executar muito mais microsserviços localmente.

Uma abordagem para lidar com esse problema é fazer com que os desenvolvedores façam o seu trabalho de desenvolvimento e de testes em um ambiente de nuvem. A ideia é que você poderá ter muito mais recursos disponíveis para executar os microsserviços necessários. Afora o fato de esse modelo exigir que sempre haja conectividade com os recursos da nuvem, outro problema principal é que seus ciclos de feedback poderão ser prejudicados. Se você tiver de fazer uma alteração no código localmente e efetuar o upload da nova versão desse código (ou de um artefato construído localmente) para a nuvem, isso pode acrescentar um atraso significativo nos seus ciclos de desenvolvimento e testes, sobretudo se você atua em uma parte do mundo em que a conectividade com a internet seja mais limitada.

Um desenvolvimento totalmente na nuvem é uma das possibilidades para resolver o problema dos ciclos de feedback; IDEs para nuvem como o Cloud9, que pertence atualmente à AWS, têm mostrado que isso é possível. No entanto, embora algo desse tipo possa ser o futuro do desenvolvimento, certamente não é o presente para a grande maioria de nós.

Basicamente acho que o uso de ambientes de nuvem para permitir que um desenvolvedor execute mais microsserviços em seus ciclos de desenvolvimento e testes não resolve o problema principal, resultando em mais complexidade do que o necessário, além de elevar os custos. O ideal é que seu objetivo seja fazer com que um desenvolvedor precise executar somente os microsserviços nos quais ele realmente trabalha. Se um desenvolvedor faz parte de uma equipe responsável por cinco

microserviços, esse desenvolvedor deve ser capaz de executar esses microserviços do modo mais eficaz possível; para um feedback mais rápido, minha preferência sempre será que esses serviços sejam executados localmente.

Mas, e se os cinco microserviços sob responsabilidade de sua equipe quiserem chamar outros sistemas e microserviços que pertencem a outras equipes? Sem eles, o ambiente local de desenvolvimento e testes não funcionará, certo? Mais uma vez, o uso de stubs vem para nos salvar. Preciso ser capaz de criar stubs locais que simulem os microserviços que estejam fora do escopo da minha equipe. Os únicos microserviços reais que devem ser executados localmente são os microserviços nos quais você trabalha. Se você trabalha em uma empresa na qual é esperado que uma pessoa trabalhe com centenas de microserviços diferentes, bem, haverá problemas muito mais sérios com os quais será preciso lidar – esse é um assunto que exploraremos com mais detalhes na seção “Responsabilidade forte versus coletiva”, na página [581](#).

Dos testes em pré-produção aos testes em produção

Historicamente, o foco principal dos testes estava relacionado aos testes de nossos sistemas *antes* de chegarem ao ambiente de produção. Com nossos testes, definimos uma série de modelos com os quais esperamos provar que o nosso sistema funciona e se comporta como gostaríamos, tanto do ponto de vista funcional como não funcional. No entanto, se nossos modelos não forem perfeitos, enfrentaremos problemas quando nossos sistemas forem usados intensamente. Bugs passam despercebidos e chegam ao ambiente de produção, novos modos de falha são descobertos e nossos usuários utilizam o sistema de maneiras que jamais poderíamos prever.

Uma reação a essa situação muitas vezes está em definir cada vez mais testes e aperfeiçoar nossos modelos, com o intuito de identificar mais problemas com antecedência e reduzir o número de problemas encontrados em nosso sistema em execução no ambiente de produção. No entanto, em determinado momento, temos de aceitar que teremos cada vez menos retornos com essa abordagem. Com os testes anteriores à

implantação, não é possível reduzir as chances de falha a zero.

A complexidade de um sistema distribuído é tal que pode ser inviável capturar todos os possíveis problemas que pudessem ocorrer antes de chegarmos ao ambiente de produção.

Falando de modo geral, o propósito de um teste é nos dar feedback para saber se nosso software tem ou não um nível suficiente de qualidade. O ideal é que esse feedback seja obtido o mais rápido possível, e gostaríamos de saber se há algum problema com nosso software antes que um usuário final experimentasse esse problema. É por isso que muitos testes são feitos antes do lançamento do nosso software.

Limitar os testes somente a um ambiente de pré-produção, porém, não é eficaz. Reduzimos os pontos em que podemos identificar problemas, além de eliminar a possibilidade de testar a qualidade de nosso software no lugar mais importante, isto é, no lugar em que será usado.

Podemos e devemos também procurar aplicar os testes em um ambiente de produção. Isso pode ser feito de forma segura, e fornecerá um feedback de melhor qualidade do que os testes de pré-produção; além disso, como podemos notar, é provavelmente algo que você já faz, independentemente de perceber ou não.

Tipos de testes no ambiente de produção

Há uma longa lista de diferentes testes que podemos executar no ambiente de produção, variando dos mais simples aos mais complexos. Para começar, vamos pensar em algo simples como uma verificação de ping para garantir que um microsserviço está ativo. Simplesmente verificar se uma instância de microsserviço está executando é um tipo de teste – nós apenas não o encaramos dessa forma, pois é uma atividade conduzida tipicamente pelo pessoal de “operações”. Basicamente, porém, uma tarefa simples como determinar se um microsserviço está ativo ou não pode ser encarada como um teste – um teste que executamos com frequência em nosso software.

Testes de fumaça (smoke tests) são outro exemplo de teste em ambiente de produção. Um teste de fumaça geralmente é executado como parte das

atividades de implantação e garante que o software implantado funciona corretamente. Esses testes de fumaça em geral são feitos no software real em execução, antes de ser lançado aos usuários (mais sobre esse assunto em breve).

Os lançamentos canário, que discutimos no Capítulo 8, também são um método que, sem dúvida, diz respeito aos testes. Lançamos uma nova versão de nosso software para uma pequena parcela dos usuários com o intuito de “testar” se ele funciona corretamente. Se funcionar, podemos fazer o rollout do software para uma parcela maior de nossa base de usuários, talvez de uma forma totalmente automatizada.

Outro exemplo de testes no ambiente de produção consiste em introduzir o comportamento de um usuário simulado no sistema a fim de garantir que ele funciona conforme esperado – por exemplo, fazer um pedido para um cliente simulado ou cadastrar um novo usuário (simulado) no sistema de produção real. Esse tipo de teste às vezes pode sofrer rejeições porque as pessoas se preocupam com o impacto que ele possa causar no sistema em produção. Assim, se você pretende criar testes como esses, certifique-se de que sejam seguros.

Deixando os testes em produção seguros

Se você decidir fazer testes em produção (e deveria!), é importante que os testes não causem problemas nesse ambiente, seja introduzindo instabilidade no sistema, seja corrompendo os dados do ambiente de produção. Algo simples como fazer um ping na instância de um microserviço a fim de garantir que ela está ativa provavelmente será uma operação segura – se isso causar instabilidade no sistema, é provável que haja problemas muito mais graves a serem resolvidos, a menos que você tenha acidentalmente deixado que seu sistema de verificação de sanidade fizesse um ataque interno de negação de serviço (denial of service).

Em geral, os testes de fumaça são seguros, pois as operações que executam geralmente são feitas no software antes de seu lançamento. Conforme exploramos na seção “Diferenciando implantação de lançamento”, na página [322](#), diferenciar o conceito de implantação do conceito de

lançamento pode ser extremamente conveniente. Quanto se trata de testes no ambiente de produção, os testes efetuados no software implantado em produção antes que ele seja lançado devem ser seguros.

As pessoas tendem a se preocupar mais com a segurança de operações como introduzir o comportamento de um usuário simulado no sistema. Não queremos que o pedido seja realmente enviado, nem que o pagamento seja feito. Isso é algo que exige devido cuidado e atenção e, apesar dos desafios, esse tipo de teste pode ser extremamente vantajoso. Retomaremos esse assunto na seção “Monitoração semântica”, na página [394](#).

MTTR em vez de MTBF?

Ao recorrer a técnicas como uma implantação azul-verde ou um lançamento canário, teremos uma forma de testar que está mais próxima do ambiente de produção (ou que está nesse próprio ambiente), e criamos ferramentas que nos ajudam a lidar com uma falha caso ela ocorra. O uso dessas abordagens representa um reconhecimento tácito de que não é possível encontrar e identificar todos os problemas antes do lançamento de nosso software.

Às vezes, dedicar o mesmo esforço para agilizar a correção de problemas quando ocorrerem pode ser significativamente mais vantajoso do que acrescentar mais testes funcionais automatizados. No mundo das operações web, isso geralmente é conhecido como o custo-benefício entre otimizar para *MTBF* (Mean Time Between Failures, ou Tempo Médio Entre Falhas) e otimizar para *MTTR* (Mean Time To Repair, ou Tempo Médio para Correção).

Técnicas para reduzir o tempo de recuperação podem ser tão simples quanto fazer rollbacks muito rápidos, associadas a uma boa monitoração (assunto que será discutido no Capítulo 10). Se pudermos encontrar um problema no ambiente de produção e fazer um rollback rápido, reduziremos o impacto para os nossos clientes.

Em diferentes empresas, o custo-benefício entre *MTBF* e *MTTR* variará, e boa parte disso dependerá de entender o verdadeiro impacto das falhas em um ambiente de produção. No entanto, a maioria das empresas que vejo

investindo tempo na criação de suítes de testes funcionais muitas vezes dedicam pouco ou nenhum esforço para melhorar a monitoração ou agilizar a recuperação em caso de falha. Assim, embora o número de defeitos que ocorrem possa ser reduzido, essas empresas não são capazes de eliminar todos os problemas, e não estão preparadas para lidar com essas falhas caso elas se manifestem no ambiente de produção.

Há outros prós e contras além desses entre o MTBF e o MTTR. Por exemplo, se você está tentando descobrir se alguém vai realmente usar o seu software, talvez faça muito mais sentido lançar algo agora com o intuito de comprovar a ideia ou o modelo de negócios antes de construir um software robusto. Em um ambiente no qual isso aconteça, os testes podem ser um exagero, pois o impacto de não saber se sua ideia funcionará será muito maior do que ter um defeito no ambiente de produção. Nessas situações, talvez seja bastante sensato evitar totalmente os testes até chegar ao ambiente de produção.

Testes multifuncionais

A maior parte deste capítulo teve como foco testar funcionalidades específicas e como isso difere quando testamos um sistema baseado em microsserviços. No entanto, é importante discutir outra categoria de testes. *Requisitos não funcionais* é um termo abrangente usado para descrever as características exibidas pelo seu sistema, mas que não podem ser implementadas simplesmente como uma funcionalidade comum. Esses requisitos incluem aspectos como a latência aceitável para uma página web, o número de usuários com o qual um sistema deve ser capaz de lidar, o nível de acessibilidade que a sua interface de usuário deve ter para pessoas com deficiência ou o nível de proteção que os dados de seus clientes devem ter.

Nunca achei o termo *não funcional* apropriado. Alguns dos aspectos englobados por esse termo me parecem ser de natureza bastante funcional! Prefiro muito mais o termo *requisitos multifuncionais* (CFR, ou Cross-Functional Requirements), criado por uma ex-colega minha, Sarah Taraporewalla. O termo representa melhor o fato de que esses

comportamentos do sistema vêm à tona somente como resultado de várias tarefas que se entrelaçam.

Muitos CRFs, se não todos, serão atendidos somente no ambiente de produção. Apesar disso, podemos definir estratégias de teste que nos ajudem a ver se estamos ao menos na direção correta para atender a esses objetivos. Esses tipos de testes se encaixam no quadrante de Testes de Propriedades. Um ótimo exemplo desse caso é o teste de desempenho, que será discutido com mais detalhes em breve.

Talvez você queira manter o controle de alguns CFRs no nível de um microsserviço individual. Por exemplo, você poderia decidir que a durabilidade exigida para o seu serviço de pagamento é significativamente alta, porém, se houver um downtime maior em seu serviço de recomendações musicais, você ainda se dará por satisfeito, sabendo que a parte principal de seus negócios sobreviverá, ainda que o sistema seja incapaz de recomendar artistas semelhantes ao Metallica durante dez minutos ou algo parecido. Essas relações de custo-benefício acabarão causando um grande impacto no modo como você fará o design de seu sistema e a maneira como ele evoluirá; mais uma vez, em um sistema baseado em microsserviços, por ser composto de várias partes menores por natureza, haverá muito mais chances de ponderar esses prós e contras. Ao observar os CFRs pelos quais um dado microsserviço ou uma equipe possam assumir a responsabilidade, é comum que eles surjam como parte dos SLOs (Service-Level Objectives, ou Objetivos de Nível de Serviço) de uma equipe – um assunto que exploraremos melhor na seção “Estamos indo bem?”, na página [387](#).

Testes relacionados aos CFRs devem obedecer à pirâmide também. Alguns testes deverão ser fim a fim, como os testes de carga, mas outros não precisariam ser. Por exemplo, assim que você identificar um gargalo no desempenho em um teste de carga fim a fim, escreva um teste de escopo menor para ajudar a identificar o problema no futuro. Outros CFRs podem ser facilmente enquadrados como testes rápidos. Eu me lembro de ter trabalhado em um projeto no qual havíamos insistido em garantir que nossa marcação (markup) HTML estivesse utilizando recursos apropriados

de acessibilidade para ajudar pessoas com deficiência a utilizarem nosso site. Verificar a marcação gerada para garantir que os controles apropriados estavam presentes era uma tarefa que poderia ser feita com muita rapidez, sem a necessidade de nenhum acesso de ida e volta na rede.

Com muita frequência, as considerações sobre CFRs são feitas muito tardiamente. Sugiro fortemente que você analise seus CFRs o mais cedo possível e os revise com regularidade.

Testes de desempenho

Vale a pena destacar explicitamente os testes de desempenho como uma forma de garantir que alguns de nossos requisitos multifuncionais possam ser atendidos. Ao decompor sistemas em microsserviços menores, aumentamos o número de chamadas que cruzam as fronteiras de rede. Uma operação que antes poderia envolver uma única chamada ao banco de dados, agora poderia envolver três ou quatro chamadas para outros serviços, cruzando fronteiras de rede, com um número correspondente de chamadas ao banco de dados. Tudo isso pode diminuir a velocidade com que nossos sistemas operam. Identificar as causas de latência é particularmente importante. Se tivermos uma cadeia de chamadas com várias chamadas síncronas e uma parte dessa cadeia começar a ficar lenta, tudo será afetado, possivelmente causando um impacto significativo. Isso faz com que ter alguma forma de fazer um teste de desempenho em suas aplicações seja mais importante ainda do que seria em um sistema mais monolítico. Muitas vezes, o motivo pelo qual esse tipo de teste acaba sendo postergado é porque, no começo, não há partes suficientes do sistema para executá-lo. Eu compreendo esse problema, mas, com muita frequência, as pessoas deixam o assunto de lado, e os testes de desempenho muitas vezes acabam sendo feitos pouco antes de o sistema ser implantado pela primeira vez – se é que chegam a ser executados! Não caia nessa armadilha.

Assim como no caso dos testes funcionais, você pode adotar uma solução mista. Pode decidir que quer testes de desempenho em serviços individuais isolados, mas pode começar com testes que verifiquem as principais jornadas (journeys) em seu sistema. Talvez você possa simplesmente

executar testes fim a fim de jornadas em grandes volumes.

Para gerar resultados relevantes, em geral será necessário executar determinados cenários com números cada vez maiores de clientes simulados. Esse teste permite ver como a latência das chamadas varia conforme o aumento da carga. Isso significa que os testes de desempenho podem demorar um pouco para serem executados. Além do mais, você vai querer que o sistema seja o mais parecido possível com o sistema de produção a fim de garantir que os resultados que verá sejam um indicativo do desempenho esperado dos sistemas em produção. Isso pode implicar que será necessário adquirir um volume de dados mais parecido com o que está em produção, e talvez mais máquinas para ter uma infraestrutura correspondente – são tarefas que podem ser desafiadoras. Ainda que você tenha dificuldade em deixar o ambiente para testes de desempenho realmente parecido com o ambiente de produção, os testes serão úteis para identificar gargalos. Saiba, porém, que os resultados obtidos podem ser falso-negativos – ou, pior ainda, falso-positivos.

Em virtude do tempo necessário para executar os testes de desempenho, nem sempre será viável executá-los a cada check-in. Uma prática comum consiste em executar um subconjunto dos testes todos os dias, e um conjunto maior a cada semana. Independentemente da abordagem que você escolher, certifique-se de que os testes sejam executados com a máxima regularidade possível. Quanto mais tempo você passar sem executar os testes de desempenho, mais difícil poderá ser identificar o culpado. Problemas de desempenho são particularmente difíceis de resolver, portanto, se puder reduzir o número de commits que precisam ser verificados para encontrar um problema recém-introduzido, sua vida será muito mais fácil.

Além disso, não se esqueça de observar os resultados! Fico muito surpreso com a quantidade de equipes que vejo dedicarem muito trabalho para implementar e executar os testes, mas que jamais verificam realmente os números. Muitas vezes isso ocorre porque as pessoas não sabem como seria um “bom” resultado. É realmente necessário ter metas. Ao disponibilizar um microsserviço para ser usado como parte de uma

arquitetura mais ampla, é comum que haja determinadas expectativas às quais você se comprometa a atender – os SLOs que mencionei antes. Se, como parte disso, você se compromete a oferecer um determinado nível de desempenho, faz sentido que qualquer teste automatizado dê a você um feedback sobre a possibilidade de atingir (ou, melhor ainda, de exceder) ou não essa meta.

No que concerne às metas específicas de desempenho, os testes automatizados podem ser muito úteis para ajudar a ver como o desempenho de seu microsserviço varia à medida que você fizer alterações. Eles podem servir como uma rede de proteção caso você faça alguma alteração que provoque uma degradação significativa no desempenho. Assim, uma alternativa para uma meta específica poderia ser fazer o teste falhar caso a diferença de desempenho de uma build para a próxima varie muito.

Os testes de desempenho precisam ser feitos levando em consideração um conhecimento do desempenho real do sistema (assunto que será discutido melhor no Capítulo 10), e o ideal é que as mesmas ferramentas usadas no ambiente de produção para visualizar o comportamento do sistema sejam empregadas em seu ambiente de testes de desempenho. Essa abordagem pode facilitar muito fazer esse tipo de comparação.

Testes de robustez

Uma arquitetura de microsserviços geralmente será tão confiável quanto o seu elo mais fraco; como resultado, é comum que nossos microsserviços incluam métodos que permitam melhorar sua robustez a fim de aumentar a confiabilidade do sistema. Exploraremos melhor esse tópico na seção “Padrões de estabilidade”, na página [464](#), mas exemplos incluem executar várias instâncias de um microsserviço atrás de um balanceador de carga para tolerar a falha de uma instância, ou utilizar circuit breakers (interruptores de circuito) no código para lidar com situações em que os microsserviços downstream não possam ser contatados.

Em situações como essas, talvez seja conveniente ter testes que permitam recriar determinadas falhas a fim de garantir que seu microsserviço

continuará funcionando como um todo. Em virtude de sua natureza, esses testes podem ser um pouco mais complicados de implementar. Por exemplo, talvez você precise gerar um timeout de rede artificial entre um microsserviço em teste e um stub externo. Apesar disso, é uma tarefa que pode valer a pena, sobretudo se você estiver criando uma funcionalidade compartilhada, a ser utilizada por vários microsserviços – por exemplo, se estiver usando uma implementação default de service mesh para cuidar do circuit breaking.

Resumo

Concluindo, o que apresentei neste capítulo foi uma abordagem holística para os testes, a qual espero tenha fornecido algumas diretrizes gerais a você acerca de como proceder ao testar seus próprios sistemas. Para reiterar o básico:

- Otimize para ter um feedback rápido e separe os tipos de testes de acordo com isso.
- Evite a necessidade de testes fim a fim que envolvam mais de uma equipe – em vez disso, considere utilizar contratos orientados a consumidores.
- Utilize contratos orientados a consumidores para criar pontos focais para discussões entre as equipes.
- Procure entender os prós e contras entre investir mais esforços nos testes e detectar problemas com mais rapidez no ambiente de produção (otimização para MTBF *versus* otimização para MTTR).
- Experimente fazer testes no ambiente de produção!

Se estiver interessado em ler mais a respeito dos testes, recomendo o livro *Agile Testing*, de Lisa Crispin e Janet Gregory (Addison-Wesley), o qual, entre outros assuntos, discute o uso dos quadrantes de testes com mais detalhes. Para um aprofundamento sobre a pirâmide de testes, junto com alguns exemplos de código e outras referências para ferramentas, também recomendo o artigo “The Practical Test Pyramid”, de Ham Vocke.⁹

Este capítulo teve como foco principal garantir que nosso código funcione

antes de chegar ao ambiente de produção, mas também começamos a considerar os testes de nossa aplicação tão logo ela alcance esse ambiente. Esse é um assunto que precisamos explorar com muito mais detalhes. O fato é que os microsserviços dão origem a uma série de desafios para entender como o nosso software se comporta no ambiente de produção – um assunto que discutiremos melhor a seguir.

- 1 Lisa Crispin e Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams* (Upper Saddle River, NJ: Addison-Wesley, 2008).
- 2 Mike Cohn, *Succeeding with Agile* (Upper Saddle River, NJ: Addison-Wesley, 2009).
- 3 Steve Freeman e Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (Upper Saddle River, NJ: Addison-Wesley, 2009).
- 4 Jason D. Valentino, “Moving One of Capital One’s Largest Customer-Facing Apps to AWS” (Passando uma das maiores aplicações da Capital One, voltada aos clientes, para a AWS), Capital One Tech, 24 de maio de 2017, <https://oreil.ly/5UM5W>.
- 5 Diane Vaughan, *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA* (Chicago: University of Chicago Press, 1996).
- 6 Martin Fowler, “Eradicating Non-Determinism in Tests” (Erradicando a falta de determinismo nos testes), martinoflower.com, 14 de abril de 2011, <https://oreil.ly/7Ve7e>.
- 7 Emily Bache, “End-to-End Automated Testing in a Microservices Architecture – Emily Bache” (Testes fim a fim automatizados em uma arquitetura de microsserviços – Emily Bache), NDC Conferences, 5 de julho de 2017, vídeo no YouTube, 56:48, NDC Oslo 2017, <https://oreil.ly/QX3EK>.
- 8 Nicole Forsgren, Jez Humble e Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).
- 9 Ham Vocke, “The Practical Test Pyramid” (Pirâmide de testes na prática), martinoflower.com, 26 de fevereiro de 2018, <https://oreil.ly/J7lc6>.

Da monitoração à observabilidade

Conforme espero ter mostrado até agora, dividir nosso sistema em vários microsserviços menores e mais especializados resulta em diversas vantagens. Como discutimos com certa profundidade, isso também gera novas fontes significativas de complexidade. Não há nenhuma outra situação na qual essa complexidade adicional seja mais evidente do que aquela em que queremos entender o comportamento de nossos sistemas em um ambiente de produção. Você logo perceberá que as ferramentas e técnicas que funcionavam bem para aplicações monolíticas relativamente mais simples, com um só processo, não funcionam tão bem para sua arquitetura de microsserviços.

Neste capítulo, veremos os desafios associados à monitoração de nossa arquitetura de microsserviços, e mostrarei que, embora novas ferramentas possam ajudar, talvez seja basicamente necessário mudar por completo a sua mentalidade quando se trata de descobrir o que realmente acontece no ambiente de produção. Falaremos também do foco cada vez maior no conceito de observabilidade – aprender como é possível fazer perguntas sobre o nosso sistema para que possamos descobrir o que há de errado.



Sufrimento no ambiente de produção

Você não apreciará realmente a dor, o sofrimento e a angústia em potencial, causados por uma arquitetura de microsserviços, até que ela esteja executando no ambiente de produção, atendendo a um tráfego de verdade.

Instabilidade, pânico e confusão

Imagine a cena: é uma tarde de sexta-feira tranquila, e a equipe está ansiosa

para sair e ir direto para um happy hour, como uma forma de começar o fim de semana distante do trabalho. De repente, os emails começam a chegar. O site está se comportando mal! O Twitter está fervilhando com as falhas de sua empresa, seu chefe não para de fazer cobranças e as esperanças de um fim de semana tranquilo desaparecem no ar.

Poucas frases sintetizam tão bem o problema quanto o tweet a seguir:

Substituímos nosso sistema monolítico por microsserviços para que cada interrupção de serviço pudesse se parecer mais com um mistério de assassinato.¹

Localizar o que houve de errado – e o que causou o problema – é a nossa primeira tarefa. No entanto, isso será mais difícil se houver uma longa lista de suspeitos.

No mundo da aplicação monolítica com um só processo, ao menos há um local muito óbvio para iniciar nossas investigações. O site está lento? É o sistema monolítico. O site está gerando erros inesperados? É o sistema monolítico. A CPU está em 100%? Sistema monolítico. Cheiro de queimado? Bem, você entendeu a ideia.² Ter um único ponto de falha, de certo modo, facilita a investigação do problema!

Vamos agora pensar em nosso sistema baseado em microsserviços. As funcionalidades que oferecemos aos nossos usuários são disponibilizadas por vários microsserviços, alguns dos quais se comunicam com outros microsserviços para realizar suas tarefas. Há uma série de vantagens em uma abordagem como essa (o que é bom, pois, do contrário, este livro seria um desperdício de tempo), mas, no mundo da monitoração, temos um problema mais complexo em mãos.

Temos agora vários servidores para monitorar, vários arquivos de log para filtrar e vários lugares nos quais a latência da rede poderia causar problemas. Nossa área de superfície para falhas aumentou, e o mesmo ocorre com o que precisa ser investigado. Então, como abordar esse problema? Precisamos compreender o que, de outro modo, seria um sistema confuso caótico e emaranhado – é a última coisa com a qual alguém iria querer lidar em uma tarde de sexta-feira (ou, pensando bem, em qualquer momento!).

Em primeiro lugar, precisamos monitorar partes pequenas e disponibilizar um recurso de agregação para que possamos ver o quadro geral. Então precisamos garantir que há ferramentas disponíveis para analisar minuciosamente esses dados como parte de nossa investigação. Por fim, temos de ser mais inteligentes a respeito de como pensamos na saúde do sistema, adotando conceitos como testes em produção. Discutiremos cada uma dessas necessidades neste capítulo. Vamos começar.

Um único microserviço, um único servidor

A Figura 10.1 apresenta uma configuração muito simples: um host que executa uma instância de um microserviço. Precisamos monitorá-lo para saber quando há algo de errado, de modo que possamos fazer a correção. Então, que informações devemos procurar?

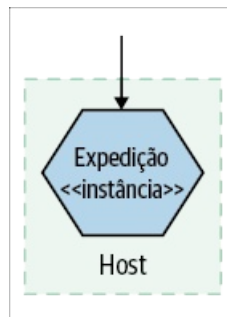


Figura 10.1 – Uma única instância de microserviço em um único host.

Inicialmente, vamos querer informações sobre o host em questão. CPU, memória – tudo isso pode ser útil. Em seguida, queremos ter acesso aos logs da instância do microserviço. Se um usuário relatar um erro, devemos ser capazes de ver esse erro nesses logs, com a expectativa de que essa informação nos proporcione uma maneira de descobrir o que houve de errado. A essa altura, com o nosso host único, provavelmente bastaria fazer somente um registro local de logs no host e utilizar ferramentas de linha de comando para consultar esse log.

Por fim, queremos monitorar a própria aplicação – observando-a de fora. No mínimo, monitorar os tempos de resposta do microserviço é uma boa ideia. Se você tivesse um servidor web na frente de sua instância de microserviço, talvez pudesse simplesmente consultar os logs do servidor

web. Ou, quem sabe, poderia acrescentar um pouco mais de sofisticação, utilizando algo como um endpoint para verificação de sanidade a fim de ver se o microserviço está ativo e “saudável” (exploraremos o que isso significa mais adiante).

O tempo passa, as cargas aumentam, e nos vemos em uma situação em que precisamos escalar...

Um único microserviço, vários servidores

Agora temos várias cópias do serviço executando em hosts diferentes, como mostra a Figura 10.2, com requisições para as diferentes instâncias distribuídas por meio de um balanceador de carga (load balancer). A situação começa a ficar um pouco mais complicada agora. Ainda queremos monitorar tudo que monitorávamos antes, mas precisamos fazer isso de um modo que seja possível isolar o problema. Se o uso da CPU está alto, esse é um problema que vemos em todos os hosts, o que poderia indicar um problema com o serviço em questão? Ou o problema está isolado em um único host, implicando que o problema está no próprio host – talvez um processo do SO com problemas?

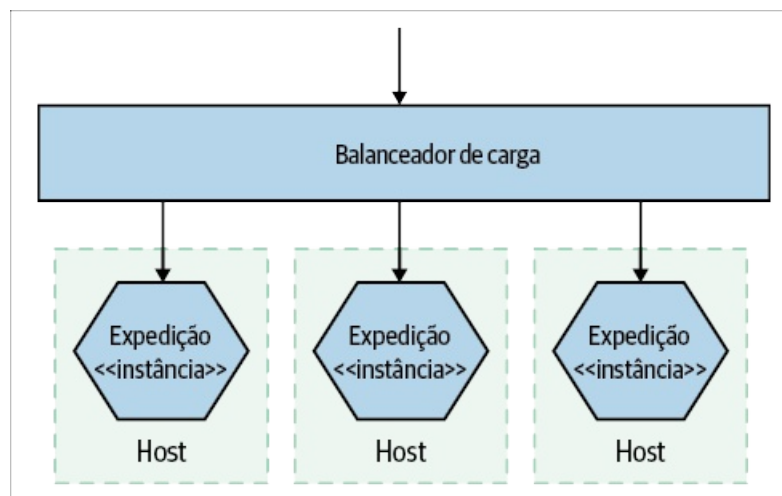


Figura 10.2 – Um único serviço distribuído em vários hosts.

A essa altura, ainda queremos monitorar as métricas no nível do host, e talvez até mesmo gerar alertas com base nelas caso ultrapassem algum tipo de limiar. No entanto, queremos ver agora como são essas métricas

considerando todos os hosts, assim como nos hosts individuais. Em outras palavras, queremos agregá-las, mas ainda com a possibilidade de analisá-las de forma isolada. Portanto, precisamos de algo que seja capaz de coletar todas essas métricas dos vários hosts e nos permita analisá-las de modo mais minucioso.

Além disso, temos os nossos logs. Com nosso serviço executando em mais de um servidor, provavelmente ficaremos cansados de fazer login em cada máquina para observar esses logs. Se houver poucos hosts, porém, podemos utilizar ferramentas como multiplexadores de SSH, que nos permitem executar os mesmos comandos em vários hosts. Com a ajuda de um monitor grande, e executando `grep "Error"` no log de nosso microserviço, será possível encontrar o culpado. Quero dizer, não é uma solução ideal, mas pode ser *boa o bastante* por um tempo. Contudo, ela logo deixará de ser satisfatória.

Para tarefas como monitoração de tempos de resposta, podemos capturar esses tempos no balanceador de carga para chamadas downstream aos microserviços. No entanto, também temos de considerar o que acontecerá caso o balanceador de carga por acaso se torne o gargalo de nosso sistema – talvez seja necessário capturar os tempos de resposta tanto no balanceador de carga como nos próprios microserviços. A essa altura, também nos interessa muito saber qual é a aparência de um serviço saudável, pois iremos configurar nosso balanceador de carga para que remova de nossa aplicação os nós que não estejam saudáveis. Espero que, quando chegarmos nesse ponto, tenhamos ao menos alguma ideia de como é um serviço saudável.

Vários serviços, vários servidores

Na Figura 10.3, a situação começa a ficar muito mais interessante. Vários serviços colaboram para oferecer funcionalidades aos nossos usuários, e esses serviços executam em vários hosts, sejam eles físicos ou virtuais. Como encontraremos o erro que estamos procurando em milhares de linhas de logs em vários hosts? Como determinamos se um servidor está se comportando de forma indevida, ou se o problema é sistêmico? E como

associamos um erro encontrado lá adiante, em uma cadeia de chamadas envolvendo vários hosts, de volta àquilo que o causou?

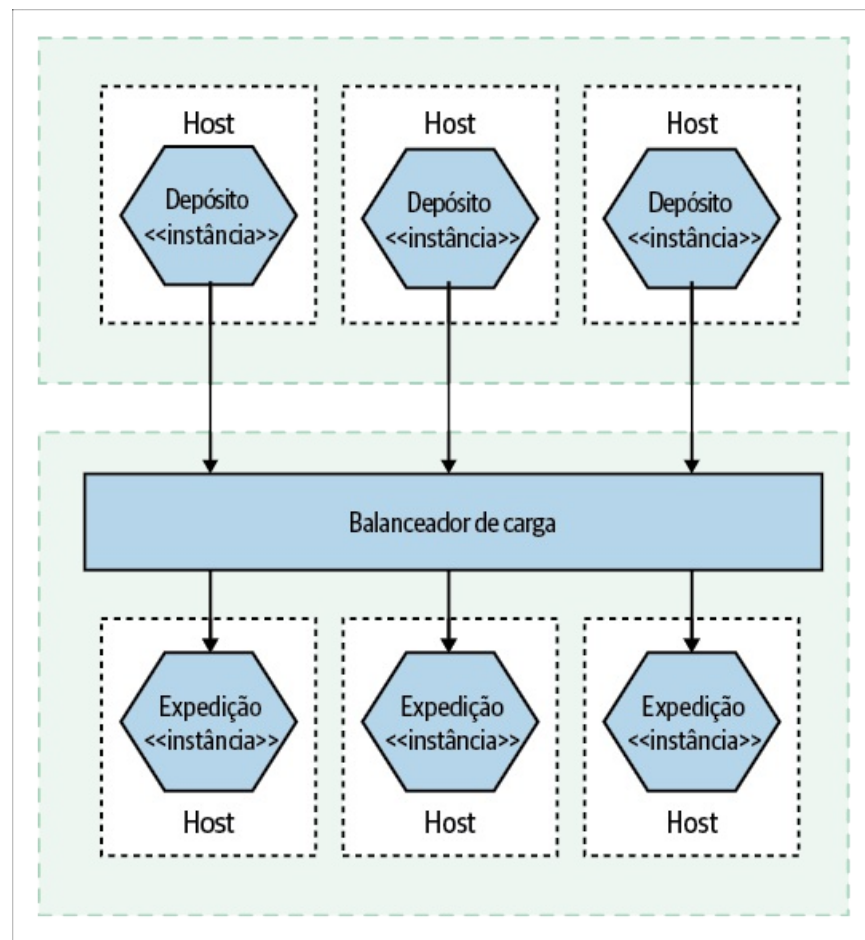


Figura 10.3 – Vários serviços em colaboração, distribuídos em vários hosts.

A agregação de informações – métricas e logs – desempenha um papel crucial para termos uma solução. Contudo, esse não é o único aspecto que devemos considerar. Precisamos descobrir como filtrar essa enorme afluência de dados e tentar entendê-la por completo. Acima de tudo, trata-se, em boa medida, de uma mudança de mentalidade, passando de um cenário razoavelmente estático de monitoração para o mundo mais ativo da observabilidade e dos testes em produção.

Observabilidade versus monitoração

Vamos iniciar explorando como começar a resolver alguns dos problemas que acabamos de mencionar; antes disso, porém, acho que é importante

analisar um termo que tem ganhado bastante popularidade desde que escrevi a primeira edição deste livro – *observabilidade*.

Como ocorre com frequência, o conceito de observabilidade existe há décadas, mas apenas recentemente chegou ao desenvolvimento de software. A observabilidade de um sistema é a extensão com que podemos compreender o estado interno desse sistema a partir de saídas externas. Em geral, isso exige uma compreensão mais holística de seu software – vê-lo mais como um *sistema* em vez de olhar para um conjunto de entidades díspares.

Na prática, quanto mais observável é um sistema, mais fácil será para nós entender qual é o problema se houver algo de errado. Nossa compreensão das saídas externas nos ajuda a localizar com mais rapidez o problema subjacente. O desafio está no fato de que, em geral, teremos de criar essas saídas externas e empregar diferentes tipos de ferramentas para entender essas saídas.

A monitoração, por outro lado, é algo que fazemos. Nós *monitoramos* o sistema. Nós o observamos. A situação começará a ficar ruim se você mantiver o foco somente na monitoração – a atividade – sem pensar no que quer alcançar com essa atividade.

Abordagens mais tradicionais para a monitoração fariam você pensar com antecedência naquilo que poderia haver de errado, e definir métodos de geração de alertas para informar quando essas situações ocorreram. Contudo, à medida que o sistema se torna cada vez mais distribuído, você vai deparar com problemas que jamais teria previsto. Com um sistema altamente observável, você terá um conjunto de saídas externas que poderá ser interrogado de diferentes maneiras – o resultado concreto de ter um sistema observável é que será possível fazer perguntas sobre o seu sistema em produção, as quais você nunca teria pensado em fazer antes.

Desse modo, podemos ver a monitoração como uma atividade – algo que fazemos – enquanto a observabilidade é uma propriedade do sistema.

Os pilares da observabilidade? Espere um pouco

Algumas pessoas tentaram sintetizar a ideia de observabilidade a alguns

conceitos básicos. Outras colocaram o foco nos “três pilares” da observabilidade na forma de métricas, logging e tracing distribuído. A New Relic até mesmo cunhou o termo MELT (Metrics, Events, Logs, and Traces, ou Métricas, Eventos, Logs e Traces), que não teve uma verdadeira adesão, mas, ao menos, a New Relic está tentando. Embora eu tenha sido muito atraído por esse modelo simples no início (e eu sou louco por um acrônimo!), com o tempo, me afastei dessa ideia por ser redutora demais, além de, possivelmente, não apreender os pontos mais importantes.

Em primeiro lugar, reduzir uma propriedade de um sistema a detalhes de implementação dessa maneira parece um processo invertido para mim. A observabilidade é uma propriedade, e há várias maneiras pelas quais eu poderia alcançar essa propriedade. Manter um foco excessivo em detalhes específicos de implementação nos faz correr o risco de nos concentrarmos na atividade, e não no resultado. É análogo ao mundo atual de TI, em que centenas, ou até mesmo milhares, de empresas se apaixonaram pela construção de sistemas baseados em microsserviços, sem realmente entender o que estão tentando alcançar.

Em segundo lugar, há sempre linhas concretas entre esses conceitos? Eu argumentaria que muitos deles se sobrepõem. Eu poderia colocar as métricas em um arquivo de log, se quiser. De modo semelhante, posso implementar um trace distribuído a partir de uma série de linhas de log, o que é feito com frequência.



A observabilidade é a extensão com que podemos entender o que o sistema está fazendo com base em entradas externas. Logs, eventos e métricas podem ajudar a deixar o sistema observável, mas não se esqueça de manter o foco em tornar o sistema compreensível, em vez de apenas fazer uso de diversas ferramentas.

Eu poderia cinicamente sugerir que a insistência nessa narrativa simplista é uma estratégia para vender ferramentas. Você precisa de uma ferramenta para métricas, uma ferramenta diferente para logs, e outra ainda para traces! E precisa enviar todas essas informações de modo distinto! Ao fazer o marketing de um produto, é muito mais fácil vender suas funcionalidades por meio de um exercício em que marcamos os itens com um X, em vez de

falar de resultados. Como disse, eu *poderia* sugerir isso com uma visão cínica, mas estamos em 2021, e estou tentando ser um pouco mais positivo em meu raciocínio.³

Sem dúvida, todos esses três (ou quatro!) conceitos são apenas exemplos específicos de um conceito mais genérico. Basicamente, podemos ver qualquer informação que pudermos obter de nosso sistema – qualquer uma dessas saídas externas – na forma genérica de um evento. Um dado evento poderia conter uma quantidade pequena ou grande de informações. Poderia conter uma taxa de CPU, informações sobre um pagamento que falhou, o fato de um cliente ter feito login, ou qualquer outra informação. A partir desse fluxo de eventos (supondo que seja possível correlacioná-los), podemos fazer a projeção de um trace, criar um índice que possa ser pesquisado ou fazer uma agregação numérica. Embora, atualmente, tenhamos optado por coletar essas informações de modos diferentes, utilizando ferramentas e protocolos distintos, nossos conjuntos atuais de ferramentas não deveriam limitar nosso raciocínio acerca da melhor maneira de obter as informações de que precisamos.

Quando se trata de deixar seu sistema observável, pense nas saídas que você precisa de seu sistema como eventos que possam ser coletados e interrogados. Talvez seja necessário usar ferramentas diferentes para expor diferentes tipos de eventos atualmente, mas isso talvez não seja verdade no futuro.

Blocos de construção para a observabilidade

Então, o que é que precisamos? Precisamos saber que os usuários de nosso software estão satisfeitos. Se houver um problema, queremos sabê-lo – o ideal é que seja antes que nossos usuários descubram sozinhos. Quando um problema ocorre, precisamos descobrir o que podemos fazer para que o sistema volte a funcionar novamente; assim que a poeira tiver baixado, queremos ter informações suficientes à disposição para descobrir o que, afinal de contas, houve de errado, e o que poderá ser feito para evitar que esse problema ocorra novamente.

Na parte restante deste capítulo, veremos como fazer tudo isso.

Discutiremos uma série de blocos de construção que podem ajudar a melhorar a observabilidade da arquitetura de seu sistema:

Agregação de logs

Coletar informações de vários microsserviços é um bloco de construção essencial para qualquer solução de monitoração ou de observabilidade.

Agregação de métricas

Capturar números brutos de nossos microsserviços e da infraestrutura para ajudar a detectar problemas, orientar o planejamento de capacidade, e talvez até mesmo escalar nossas aplicações.

Tracing distribuído

Rastrear um fluxo de chamadas que cruzem as fronteiras de vários microsserviços a fim de descobrir o que houve de errado e extrair informações precisas de latência.

Estamos indo bem?

Olhar para orçamentos de erros (error budgets), SLAs, SLOs e itens semelhantes para ver como podem ser usados como parte da tarefa de garantir que nosso microsserviço atende às necessidades de seus consumidores.

Alertas

Quais ocorrências devem gerar alertas? Como seria um bom alerta?

Monitoração semântica

Pensar de modo diferente na saúde de nossos sistemas, e o que seria um motivo para nos despertar às três da manhã.

Testes em produção

Um resumo de diversas técnicas de testes no ambiente de produção.

Vamos começar com o item que talvez seja o mais simples para ser colocado em prática, mas que trará muitas recompensas: a agregação de logs.

Agregação de logs

Com muitos servidores e instâncias de microsserviços em uma arquitetura

de microsserviços até mesmo modesta, fazer login em várias máquinas ou usar multiplexadores de SSH para obter os logs realmente não seria satisfatório. Em vez disso, procuramos utilizar subsistemas especializados para obter nossos logs e disponibilizá-los de forma centralizada.

Os logs rapidamente se tornarão um dos métodos mais importantes para ajudar a entender o que acontece em seu sistema de produção. Em arquiteturas de implantação mais simples, muitas vezes pensamos em nossos arquivos de log, nas informações que inserimos aí e como tratamos os logs somente *a posteriori*. Com um sistema cada vez mais distribuído, eles se tornarão uma ferramenta crucial, não só para ajudar a diagnosticar o que houve de errado quando constatarmos que ocorreu um problema, mas, antes de tudo, para informar que houve um problema que exige a nossa atenção.

Conforme discutiremos em breve, há diversas ferramentas nessa área, mas, de modo geral, elas funcionam do mesmo modo, conforme representado na Figura 10.4. Os processos (como as instâncias de nossos microsserviços) registram os logs em seu sistema de arquivos local. Um processo daemon local periodicamente coleta e encaminha esse log para uma espécie de repositório que pode ser consultado pelos operadores. Um dos aspectos interessantes desses sistemas é que a sua arquitetura de microsserviços, de modo geral, não precisa ter ciência deles. Não será necessário modificar seu código para que utilize algum tipo de API especial; basta registrar o log em um sistema de arquivos local. No entanto, é preciso conhecer os modos de falha relacionados a esse processo de envio de logs, sobretudo se quiser compreender as situações nas quais os logs poderiam ser perdidos.

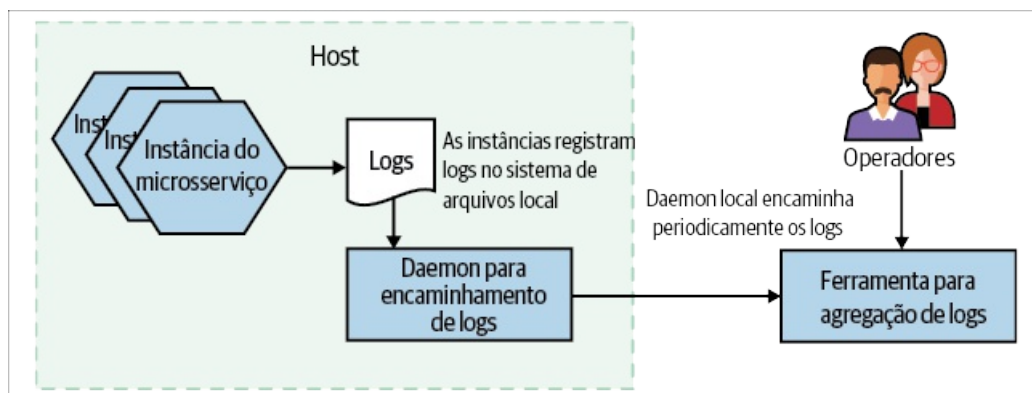


Figura 10.4 – Visão geral de como os logs são coletados como parte da agregação de logs.

Espero que você tenha percebido que eu tento evitar ser dogmático a respeito dos assuntos. Em vez de simplesmente dizer que você *deve* fazer X ou Y, procuro apresentar o contexto, fornecer orientações e explicar as nuances de determinadas decisões – isto é, tenho procurado apresentar as ferramentas para que você possa tomar a decisão correta de acordo com o seu contexto. Contudo, quando se trata da agregação de log, chegarei o mais próximo possível de dar um conselho que sirva para todos: você deve encarar a implementação de uma ferramenta para agregação de logs como um *pré-requisito* para implementar uma arquitetura de microsserviços.

Meus motivos para ter esse ponto de vista se desdobram em dois. Em primeiro lugar, a agregação de logs é extremamente útil. Para aqueles que tratam seus arquivos de log como um local para despejar desinformações, isso poderá ser uma surpresa. Mas, acredite em mim – quando feita de forma correta, a agregação de logs pode ser extremamente importante, sobretudo quando usada com outro conceito que discutiremos em breve: os IDs de correlação.

Em segundo lugar, a implementação de uma agregação de logs, quando comparada com as outras causas de dor e sofrimento em uma arquitetura de microsserviços, não é tão difícil assim. Se sua empresa é incapaz de implementar uma solução simples de agregação de logs com sucesso, é provável que verá que lidar com os demais aspectos de uma arquitetura de microsserviços será muito pior. Assim, considere utilizar a implementação de uma solução como essa como uma forma de testar se sua empresa está pronta para as outras dificuldades que estarão por vir.



Antes de tudo

Antes de qualquer outra tarefa na construção de sua arquitetura de microsserviços, deixe uma ferramenta de agregação de logs pronta para executar. Considere-a como um pré-requisito para a construção de uma arquitetura de microsserviços. Você me agradecerá mais tarde.

Contudo, também é verdade que a agregação de logs tem suas limitações e,

com o tempo, você provavelmente vai querer procurar ferramentas mais sofisticadas para expandir, ou até mesmo substituir parte daquilo que a agregação de logs tem para oferecer. Apesar de tudo isso, ela continua sendo um excelente ponto de partida.

Formato comum

Se você pretende ter uma agregação de logs, vai querer executar consultas nos logs para extrair informações úteis. Para que isso funcione, é importante escolher um formato padrão de logs que seja razoável – caso contrário, suas consultas acabarão sendo difíceis, ou até mesmo impossíveis de escrever. Você vai querer que a data, a hora, o nome do microserviço, o nível do log e outras informações estejam em posições consistentes em cada log.

Alguns agentes que fazem encaminhamento de logs permitem reformatar os logs antes de encaminhá-los ao seu repositório centralizado de logs. Pessoalmente, eu evitaria isso sempre que possível. O problema é que reformatar logs pode exigir muitos recursos computacionais, a ponto de eu já ter visto problemas reais no ambiente de produção causados pelo consumo de CPU em virtude dessa tarefa. Será muito melhor alterar os logs à medida que forem escritos pelo seu próprio microserviço. Eu restringiria o uso da reformatação de logs pelos agentes que fazem seu encaminhamento aos locais em que não seria possível alterar o formato original do log – por exemplo, no caso de um software legado ou de terceiros.

Eu achava que, da época em que escrevi a primeira edição deste livro até agora, um padrão comum para logging teria ganhado impulso no mercado, mas isso parece não ter acontecido. Muitas variações parecem existir; em geral, envolvem partir do formato de log de acesso padrão aceito por servidores web como o Apache e o nginx, e expandi-lo com o acréscimo de outras colunas de dados. O ponto principal é que, em sua própria arquitetura de microserviços, você deve escolher um formato padronizado internamente.

Se um formato de log razoavelmente simples for empregado, você irá gerar

linhas de texto simples com informações específicas em determinadas posições da linha de log. No Exemplo 10.1, vemos um formato de exemplo.

Exemplo 10.1 – Alguns logs de exemplo

```
15-02-2020 16:00:58 Order INFO [abc-123] Customer 2112 has placed order 988827
```

```
15-02-2020 16:01:01 Payment INFO [abc-123] Payment $20.99 for 988827 by cust 2112
```

A ferramenta de agregação de logs precisará saber como fazer parse dessa string a fim de extrair as informações que possamos querer consultar – o timestamp, o nome do microserviço ou o nível do log, por exemplo. Nesse exemplo, isso é viável, pois esses dados estão em posições estáticas em todos os logs – a data está na primeira coluna, a hora está na segunda coluna, e assim por diante. A situação será mais complicada, porém, se quisermos encontrar as linhas de log relacionadas a um dado cliente – o ID do cliente está presente nas duas linhas de log, porém é exibido em posições diferentes. É nessa situação que começaríamos a pensar em escrever linhas de log mais estruturadas, talvez utilizando um formato JSON, para que possamos encontrar informações como o ID de um cliente ou de um pedido em um local consistente. Repetindo, a ferramenta de agregação de logs deverá ser configurada para fazer o parse e extrair as informações necessárias do log. Outro aspecto a ser observado é que, se você fizer o log usando JSON, poderá ser mais difícil para as pessoas lerem esse log diretamente, sem que tenham ferramentas adicionais para fazer o parse dos valores necessários – simplesmente ler o log com um visualizador de texto simples talvez não seja muito conveniente.

Correlações entre linhas de log

Com um número maior de serviços interagindo para disponibilizar qualquer dada funcionalidade a um usuário final, uma única chamada inicial pode acabar gerando várias chamadas de serviços downstream. Por exemplo, vamos considerar um exemplo na MusicCorp, conforme mostrado na Figura 10.5. Estamos cadastrando um cliente em nosso novo serviço de streaming. O cliente seleciona seu pacote de streaming e clica no botão de submissão. Nos bastidores, quando o botão é clicado na UI, isso chega até o Gateway, que está no perímetro de nosso sistema. Por sua vez, ele

passará a chamada ao microserviço `streaming`. Esse microserviço se comunica com `Pagamento` para receber o primeiro pagamento, utiliza o microserviço `cliente` para atualizar o fato de que esse cliente agora tem o streaming ativado e envia um email ao cliente usando nosso microserviço `Email`, confirmando que ele agora está inscrito.

O que acontecerá se a chamada ao microserviço `Pagamento` acabar gerando um erro inesperado? Falaremos bastante sobre como lidar com falhas no Capítulo 12, mas considere a dificuldade de diagnosticar o que aconteceu.

O problema é que o único microserviço que registra um erro é o nosso microserviço `Pagamento`. Se estivermos com sorte, poderemos descobrir qual foi a requisição que causou o problema, e poderíamos até mesmo olhar os parâmetros da chamada. No entanto, não vemos esse erro no contexto mais amplo em que ocorreu. Nesse exemplo em particular, mesmo que partíssemos do pressuposto de que cada interação gera somente uma linha de log, teríamos cinco linhas de log contendo informações sobre esse fluxo de chamadas. Ser capaz de ver essas linhas de log agrupadas pode ser extremamente conveniente.

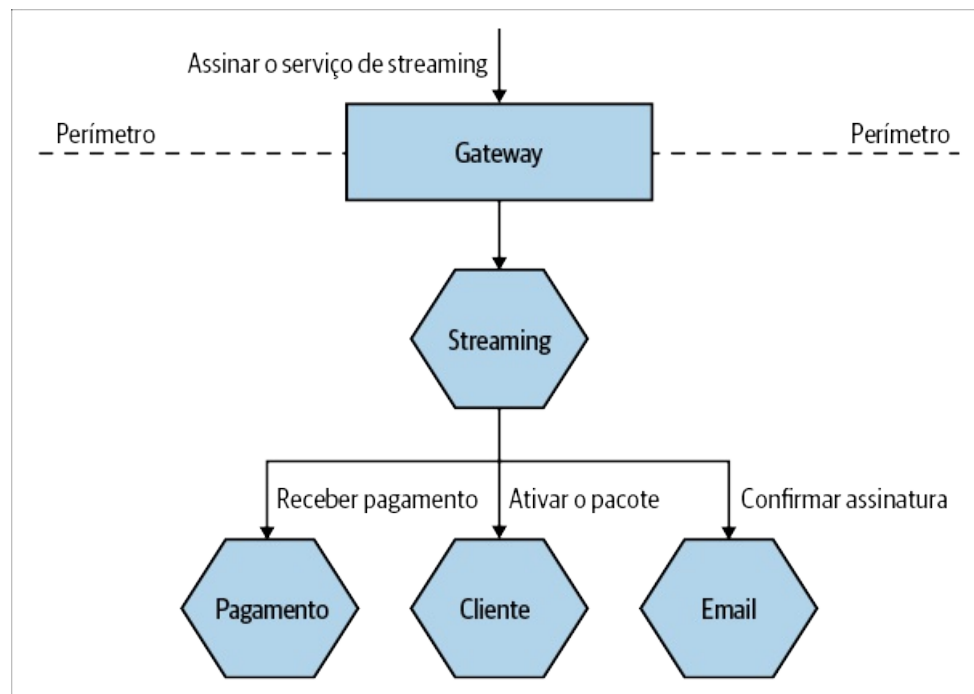


Figura 10.5 – Uma série de chamadas envolvendo vários microserviços relacionados à inscrição de um cliente.

Uma abordagem que pode ser útil nesse caso é utilizar IDs de correlação – algo que mencionamos pela primeira vez no Capítulo 6, quando discutimos as sagas. Quando a primeira chamada é feita, geramos um ID único que será usado para correlacionar todas as chamadas subsequentes relacionadas à requisição. Na Figura 10.6, geramos esse ID em Gateway, e ele é então passado como parâmetro para todas as chamadas subsequentes.

O logging de qualquer atividade de um microserviço, resultante dessa chamada de entrada, será registrado com o mesmo ID de correlação, que será colocado em um local consistente em cada linha de log, como mostra o Exemplo 10.2. Isso facilitará a extração de todos os logs associados a um dado ID de correlação mais tarde.

Exemplo 10.2 – Utilizando um ID de correlação em uma posição fixa em uma linha de log

```
15-02-2020 16:01:01 Gateway INFO [abc-123] Signup for streaming
15-02-2020 16:01:02 Streaming INFO [abc-123] Cust 773 signs up ...
15-02-2020 16:01:03 Customer INFO [abc-123] Streaming package added ...
15-02-2020 16:01:03 Email INFO [abc-123] Send streaming welcome ...
15-02-2020 16:01:03 Payment ERROR [abc-123] ValidatePayment ...
```

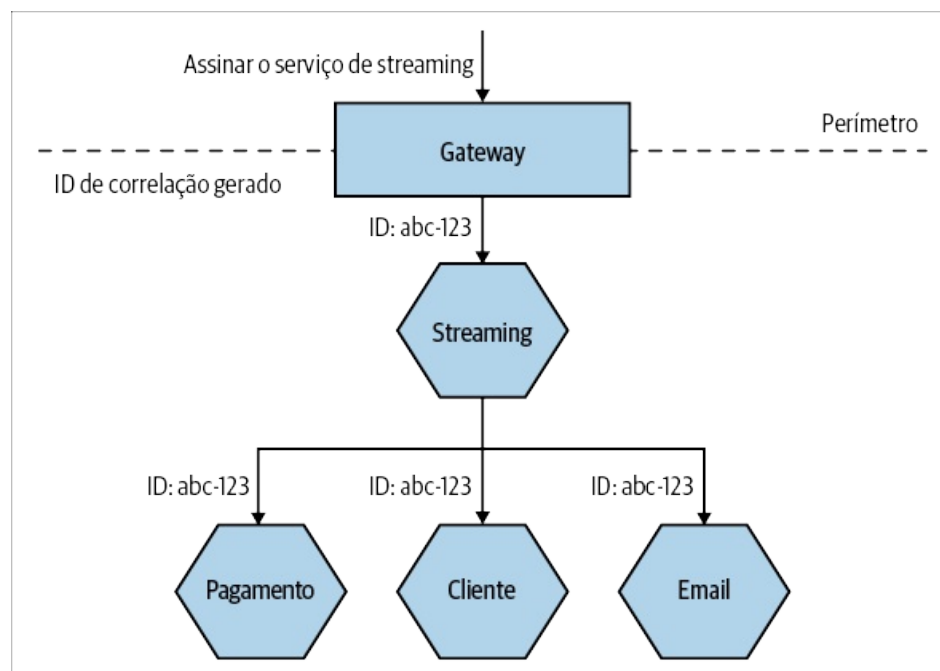


Figura 10.6 – Gerando um ID de correlação para um conjunto de chamadas.
É claro que será necessário garantir que cada serviço saiba como passar

adiante o ID de correlação. É por isso que devemos ter um padrão e insistir que ele seja usado em todo o sistema. Feito isso porém, podemos criar ferramentas para rastrear todo tipo de interações. Ferramentas como essas podem ser úteis para rastrear event storms (tempestade de eventos) ou casos excepcionais, ou até mesmo para identificar transações particularmente custosas, pois será possível visualizar toda a cascata de chamadas.

IDs de correlação em logs são o tipo de informação que não parece ser muito útil no começo, mas confie em mim – com o tempo, poderão ser *extremamente* úteis. Infelizmente talvez seja difícil inseri-los no sistema posteriormente. É por esse motivo que insisto que os IDs de correlação sejam implementados no logging o mais cedo possível. Os logs, é claro, podem ir somente até certo ponto nesse aspecto – alguns tipos de problemas serão resolvidos de modo mais apropriado por ferramentas de tracing distribuído, que exploraremos em breve. No entanto, um simples ID de correlação em um arquivo de log pode ser extremamente útil para começar, o que significa que você poderia adiar o uso de uma ferramenta dedicada de tracing até seu sistema alcançar um nível de complexidade que justifique o uso desse recurso.



Assim que tiver uma agregação de logs, implemente os IDs de correlação o mais cedo possível. Fáceis de implementar no começo, mas difíceis de serem introduzidos posteriormente, esses IDs deixarão seus logs muito mais relevantes.

Tempo

Quando observamos uma lista de linhas de log, podemos ser levados a pensar que vemos os fatos em uma ordem cronológica exata, o que nos ajudará a entender o que aconteceu, e em qual sequência os fatos ocorreram. Afinal de contas, cada linha de nossos logs inclui uma data e um horário – então por que não podemos usá-los como uma forma de determinar a ordem em que os fatos aconteceram? Na sequência de chamadas do Exemplo 10.2, vemos uma linha de log de Gateway, seguida de logs dos microsserviços Streaming, Cliente (Customer), Email, e então Pagamento

(Payment). Poderíamos concluir que essa é a ordem em que as chamadas realmente aconteceram. Infelizmente, nem sempre poderemos considerar isso como verdadeiro.

As linhas de log são geradas nas máquinas em que essas instâncias de microsserviços executam. Depois de terem sido escritas localmente, em algum momento esses logs são encaminhados adiante. Isso significa que os registros de data das linhas de log são gerados nas máquinas em que os microsserviços estão executando. Infelizmente não podemos garantir que os relógios nessas diferentes máquinas estejam em sincronia. Significa que o relógio da máquina em que o microsserviço `Email` está executando poderia estar alguns segundos adiantado em relação ao relógio da máquina em que `Pagamento` está executando – isso poderia dar a impressão de que algo aconteceu no microsserviço `Email` antes de ter ocorrido no microsserviço `Pagamento`, mas poderia ser somente por causa dessa divergência nos relógios.

A divergência nos relógios pode causar todo tipo de problemas em sistemas distribuídos. Há protocolos que tentam reduzir a divergência nos relógios dos sistemas – o NTP (Network Time Protocol, ou Protocolo de Tempo para Redes) é o exemplo mais amplamente usado. Contudo, não há garantias de que o NTP vai funcionar, e mesmo quando funciona, tudo que ele pode fazer é diminuir a diferença, em vez de eliminá-la. Se houver uma sequência de chamadas muito próximas, você poderá descobrir que, mesmo um segundo de diferença entre as máquinas será suficiente para alterar todo o seu entendimento acerca da sequência de chamadas.

Basicamente, isso significa que temos duas limitações quando se trata de horários nos logs. Não podemos ter informações totalmente precisas sobre tempo no fluxo geral das chamadas, nem será possível compreender a causalidade.

No que diz respeito a ajudar a solucionar esse problema para que possamos saber a verdadeira ordem dos acontecimentos, Leslie Lamport⁴ propôs um sistema de relógios lógico, no qual um contador é usado para manter o controle da ordem das chamadas. Poderíamos implementar um esquema parecido se quiséssemos, e há uma série de variações desse

esquema. Pessoalmente, porém, se quisesse informações mais exatas sobre a ordem das chamadas, e quisesse também horários mais exatos, eu estaria mais inclinado a fazer uso de uma ferramenta de tracing distribuído, que resolveria os dois problemas para mim. Veremos o tracing distribuído com mais detalhes mais adiante neste capítulo.

Implementações

Poucas áreas em nosso mercado são tão disputadas quanto a de agregação de logs, e elas oferecem inúmeras soluções.

Um conjunto popular de ferramentas open source para agregação de logs faz uso de um agente para encaminhamento de logs como o Fluentd (<https://www.fluentd.org>) para enviar logs para o Elasticsearch (<https://oreil.ly/m0Evo>), utilizando o Kibana (<https://oreil.ly/zw8ds>) como uma forma de organizar o fluxo resultante de logs. O principal desafio desse conjunto de ferramentas tende a ser o overhead de gerenciar o próprio Elasticsearch, mas o problema pode ser atenuado caso você precise executá-lo para outros propósitos, ou se fizer uso de um provedor gerenciado. Farei duas advertências adicionais a respeito do uso desse conjunto de ferramentas. Em primeiro lugar, muito esforço foi investido no Elasticsearch para vendê-lo como um banco de dados. Pessoalmente, nunca me senti confortável com isso. Tomar algo que sempre foi anunciado como um índice de pesquisa e rebatizá-lo como um banco de dados pode ser extremamente problemático. Fazemos suposições implícitas sobre a atuação e o comportamento dos bancos de dados, e os tratamos de acordo com essas suposições, considerando-os uma fonte da verdade para dados essenciais. Por design, porém, um índice para pesquisa não é uma fonte da verdade; é uma projeção da fonte da verdade. No passado, o Elasticsearch apresentava problemas que me fizeram parar para pensar.⁵ Embora eu tenha certeza de que muitos desses problemas já foram resolvidos, minha própria leitura sobre eles faz com que eu seja cauteloso quanto ao uso do Elasticsearch em determinadas situações e, certamente, para considerá-lo um banco de dados. Ter um índice de pesquisa que possa ocasionalmente perder dados não seria um problema se for possível reindexá-lo. No entanto, tratá-lo como um banco de dados é uma história

totalmente diferente. Se eu estivesse usando esse conjunto de ferramentas e não pudesse me dar ao luxo de perder informações de log, eu iria querer garantir que seria possível reindexar os logs originais caso houvesse algum problema.

O segundo conjunto de preocupações tem menos relação com os aspectos técnicos do Elasticsearch e do Kibana, e mais com o comportamento da Elastic, a empresa responsável por esses projetos. Recentemente, a Elastic tomou a decisão de alterar a licença do código-fonte tanto do banco de dados básico do Elasticsearch como do Kibana, passando de uma licença open source amplamente usada e aceita (Apache 2.0) para uma SSPL (Server Side Public License, ou Licença Pública do Lado do Servidor), que não é open source.⁶ O fator determinante dessa mudança na licença parece ter sido a frustração da Elastic com empresas como a AWS, que criaram produtos comerciais bem-sucedidos com base nessa tecnologia, os quais superaram as próprias ofertas comerciais da Elastic. Afora as preocupações de que a SSPL pode ser “viral” por natureza (de um modo semelhante à GNU General Public License), essa decisão deixou muitas pessoas enfurecidas. Muito mais de mil pessoas contribuíam com código para o Elasticsearch, na expectativa de que estavam fazendo uma doação para um produto open source. Há também outro aspecto irônico, que é o fato de o próprio Elasticsearch – e, desse modo, boa parte da empresa Elastic como um todo – ter sido criado com base na tecnologia do projeto open source Lucene (<https://lucene.apache.org>). Quando este livro foi escrito, a AWS, como era previsto, havia se comprometido a criar e manter um fork (ramificação) open source tanto para o Elasticsearch como para o Kibana sob a licença open source Apache 2.0 utilizada anteriormente.

Em vários aspectos, o Kibana foi uma tentativa louvável de criar uma alternativa open source para opções comerciais caras como o Splunk. Por melhor que pareça ser o Splunk, cada cliente com quem conversei também me disse que esse produto pode ser extremamente caro, tanto no que concerne às taxas de licença como aos custos de hardware. Muitos desses clientes, porém, percebem a sua importância. Apesar disso, há inúmeras opções comerciais no mercado. Pessoalmente, sou grande fã do Humio (<https://www.humio.com>), muitas pessoas gostam de usar o Datadog

(<https://www.datadoghq.com>) para agregação de logs, e há soluções prontas básicas, porém viáveis, para agregação de logs, oferecidas por alguns provedores de nuvem pública, como o CloudWatch para AWS ou o Application Insights para o Azure.

A verdade é que há uma diversidade enorme de opções nessa área, variando de produtos open source a comerciais, com opções com hospedagem própria ou terceirizada. Se quiser construir uma arquitetura de microsserviços, esse não é um problema que você achará difícil de resolver.

Deficiências

Os logs são um modo fácil e incrível de obter informações de seus sistemas em execução rapidamente. Continuo convencido de que, na etapa inicial de uma arquitetura de microsserviços, há poucos recursos que oferecerão um melhor retorno aos investimentos do que os logs, quando se trata de melhorar a visibilidade de sua aplicação no ambiente de produção. Eles são essenciais para a coleta de informações e diagnósticos. Apesar disso, você deve estar ciente de alguns dos possíveis desafios relevantes acerca dos logs.

Em primeiro lugar, conforme já mencionamos, em virtude da divergência nos relógios, nem sempre você poderá contar com os logs para ajudar a entender a ordem na qual as chamadas ocorreram. Essa divergência nos relógios das máquinas também implica que determinar a ordem cronológica exata de uma sequência de chamadas será complicado, limitando possivelmente a utilidade dos logs para rastrear gargalos de latência.

O principal problema com os logs, porém, é que, à medida que houver mais microsserviços e mais chamadas, você acabará gerando MUITOS dados. Um volume enorme. Gigantesco. Isso pode resultar em custos mais elevados por exigir mais hardware, além de poder aumentar o valor pago ao seu provedor de serviço (alguns provedores cobram pelo uso). E, dependendo de como o seu conjunto de ferramentas para agregação de logs for configurado, pode haver também problemas de escalabilidade.

Algumas soluções de agregação de logs tentam criar um índice quando recebem dados de log, com o intuito de agilizar as consultas. O problema é que manter um índice exige muitos recursos computacionais – e quanto mais logs você receber, e quanto maior for o índice, mais complicada poderá ser a situação. Isso resulta na necessidade de especificar melhor o que será registrado no log a fim de atenuar o problema, o que, por sua vez, pode implicar mais trabalho, correndo-se o risco de descartar informações de logging que, do contrário, poderiam ser importantes. Conversei com uma equipe que gerenciava o cluster de Elasticsearch para ferramentas de desenvolvedor baseadas em SaaS; ela constatou que o maior cluster de Elasticsearch que ela conseguia operar de modo satisfatório era capaz de lidar com apenas seis semanas de logging para um de seus produtos, exigindo que a equipe tivesse de constantemente transferir dados para outros lugares a fim de deixar a situação administrável. Parte dos motivos pelos quais gosto do Humio é que ele foi desenvolvido com esse problema em mente – em vez de manter um índice, seus desenvolvedores se concentraram em uma ingestão de dados eficiente e escalável, com algumas soluções inteligentes para tentar deixar os tempos de consulta baixos.

Ainda que você tenha uma solução para armazenar o volume de logs que quiser, esses logs poderão acabar contendo muitas informações importantes e sigilosas. Isso significa que seria necessário limitar o acesso aos logs (o que poderia complicar seus esforços de adotar uma abordagem de responsabilidade coletiva sobre os seus microsserviços em produção), e os logs poderiam ser alvo de agentes mal-intencionados. Desse modo, talvez seja preciso considerar que certos tipos de informação não sejam registrados em logs (conforme será mencionado na seção “Seja frugal”, na página [439](#), se você não armazenar dados, eles não poderão ser roubados) para reduzir o impacto de um acesso por indivíduos não autorizados.

Agregação de métricas

Assim como o desafio de observar logs de diferentes hosts, precisamos encontrar maneiras melhores de coletar e visualizar dados de nossos sistemas. Pode ser difícil saber o que é “bom” quando observamos métricas de um sistema mais complexo. Nosso site apresenta aproximadamente

50 códigos de erro HTTP 4XX por segundo. Isso é ruim? A carga da CPU no serviço de catálogo aumentou em 20% desde a hora do almoço; há algo errado? O segredo de saber quando entrar em pânico e quando relaxar é coletar métricas sobre o comportamento do sistema durante um período suficientemente longo, a ponto de surgirem padrões claros.

Em um ambiente mais complexo, provisionaremos novas instâncias de nossos microsserviços com muita frequência, portanto queremos que o sistema que escolhermos facilite bastante a coleta de métricas de novos hosts. Queremos ser capazes de observar uma métrica agregada para todo o sistema – por exemplo, a carga média de CPU –, mas queremos também agregar essa métrica para todas as instâncias de um dado serviço, ou até mesmo para uma única instância desse serviço. Isso significa que devemos ser capazes de associar metadados à métrica para que possamos inferir essa estrutura.

Outra vantagem essencial em entender suas tendências está relacionada ao planejamento de capacidade. Estamos atingindo o nosso limite? Quanto tempo temos até precisarmos de mais hosts? No passado, quando comprávamos hosts físicos, essa era geralmente uma atividade anual. Na nova era da computação por demanda oferecida por fornecedores de IaaS (Infrastructure as a Service, ou Infraestrutura como Serviço), é possível aumentar ou reduzir o número de hosts em minutos, ou até mesmo em segundos. Isso significa que, se conhecermos nossos padrões de uso, podemos garantir que teremos uma infraestrutura suficiente para atender às nossas necessidades. Quanto mais inteligentes formos para monitorar nossas tendências e saber o que fazer com elas, melhores serão os custos de nossos sistemas, e mais responsivos eles serão.

Em virtude da natureza desse tipo de dado, poderemos armazenar e informar essas métricas com resoluções distintas. Por exemplo, posso querer uma amostra das CPUs de meus servidores a uma resolução de uma amostra a cada dez segundos nos últimos 30 minutos, de modo que seja possível reagir melhor a uma situação que esteja se desdobrando no momento. Por outro lado, as amostras de CPU dos meus servidores referentes ao último mês provavelmente serão necessárias somente para

uma análise geral de tendência, portanto eu ficaria satisfeito em calcular uma média das amostras de CPU a cada hora. Geralmente isso é feito em plataformas padrões de métricas para ajudar a diminuir o tempo das consultas, além de reduzir o armazenamento de dados. Para algo tão simples quanto uma taxa de CPU, essa solução poderia ser apropriada, mas o processo de agregar dados do passado nos faz perder informações. O problema com a necessidade de agregar esses dados é que, com frequência, será preciso decidir o que será agregado com antecedência – temos de dar um palpite sobre quais informações poderão ser perdidas sem que haja problemas.

Ferramentas de métricas padrões podem ser absolutamente apropriadas para conhecer tendências ou modos de falha simples. Com efeito, elas podem ser essenciais. No entanto, em geral, elas não nos ajudam a deixar nossos sistemas mais observáveis, pois restringem os tipos de perguntas que queremos fazer. A situação começa a ficar interessante quando passamos de informações simples como tempos de resposta, CPU ou uso de espaço em disco para pensar mais amplamente nos tipos de informações que queremos capturar.

Cardinalidade baixa versus alta

Muitas ferramentas, sobretudo as mais recentes, foram criadas para acomodar o armazenamento e a recuperação de dados de alta cardinalidade. Há uma série de maneiras de descrever a cardinalidade, mas podemos pensar nela como o número de campos que podem ser facilmente consultados em um determinado dado. Quanto mais possíveis campos queremos consultar em nossos dados, maior será a cardinalidade que devemos aceitar. Basicamente, a situação se torna mais complicada com bancos de dados de séries temporais por razões que não detalharei neste texto, mas que estão relacionadas à forma como muitos desses sistemas foram projetados.

Por exemplo, posso querer capturar e consultar dados com base no nome do microsserviço, no ID do cliente, no ID da requisição, no número da build de software e no ID do produto, o tempo todo. Em seguida, decido capturar informações sobre a máquina em uma fase específica – o SO, a

arquitetura do sistema, o provedor de nuvem, e assim por diante. Eu talvez precisasse capturar todas essas informações para cada dado coletado. À medida que o número de informações com base nas quais posso querer consultar aumenta, maior será a cardinalidade, e mais problemas apresentarão os sistemas que não foram projetados com esse caso de uso em mente. Conforme explica Charity Majors,⁷ fundador da Honeycomb:

Tudo se reduz essencialmente à métrica. A métrica é um dado, um único número com um nome e algumas tags de identificação. Todo o contexto que você puder obter deve estar incluído nessas tags. Contudo, a explosão de escritas de todas essas tags é custosa por causa do modo como as métricas são armazenadas em disco. Armazenar uma métrica é extremamente barato, mas armazenar uma tag é custoso; e armazenar muitas tags por métrica fará sua engine de armazenamento travar rapidamente.

Falado de modo prático, sistemas criados com baixa cardinalidade em mente terão muitas dificuldades se você tentar inserir aí dados com cardinalidade alta. Sistemas como o Prometheus, por exemplo, foram criados para armazenar informações razoavelmente simples, como taxa de CPU para uma dada máquina. Em vários aspectos, podemos ver o Prometheus e ferramentas semelhantes como uma ótima implementação para armazenamento e consulta de métricas tradicionais. Contudo, a incapacidade de aceitar dados de cardinalidade mais alta pode ser um fator limitante. Os desenvolvedores do Prometheus são bastante abertos quanto a essa limitação (<https://oreil.ly/LCoVM>):

Lembrem-se de que cada combinação única de pares de labels (rótulos) chave-valor representa uma nova série temporal, o que pode aumentar drasticamente o volume de dados armazenados. Não utilize labels para armazenar dimensões com alta cardinalidade (muitos valores diferentes de labels), como ID de usuários, endereços de email, ou outros conjuntos de valores ilimitados.

Sistemas capazes de lidar com alta cardinalidade estão mais aptos a permitir que você faça uma série de perguntas diferentes sobre seus sistemas – muitas vezes, perguntas que você previamente nem sabia que

precisava fazer. Esse talvez seja um conceito difícil de entender, particularmente se você vinha gerenciando seu sistema monolítico com um único processo de forma bastante satisfatória com ferramentas mais “convencionais”. Mesmo as pessoas que possuíam sistemas maiores se contentavam com sistemas de mais baixa cardinalidade, em geral porque não tinham opção. No entanto, à medida que a complexidade de seu sistema aumentar, será necessário melhorar a qualidade das saídas fornecidas pelo seu sistema a fim de aumentar a observabilidade. Isso significa coletar mais informações e ter ferramentas que permitam analisar minuciosamente esses dados.

Implementações

Desde a primeira edição deste livro, o Prometheus se tornou uma ferramenta open source popular para coleta e agregação de métricas, e nos casos em que eu poderia ter recomendado anteriormente o uso do Graphite (que recomendei na primeira edição), o Prometheus pode ser um substituto sensato. A área comercial também expandiu bastante, com novos e antigos fornecedores igualmente criando ou remodelando soluções existentes com vistas aos usuários de microsserviços.

Tenha em mente, porém, as minhas preocupações quanto a dados de baixa *versus* alta cardinalidade. Será muito difícil adaptar sistemas criados para lidar com dados de baixa cardinalidade de modo a aceitarem a armazenagem e o processamento de dados de alta cardinalidade. Se você está à procura de sistemas capazes de armazenar e gerenciar dados de alta cardinalidade, permitindo uma observação (e um questionamento) muito mais sofisticada do comportamento de seu sistema, eu sugeriria fortemente que você considerasse o Honeycomb (<https://www.honeycomb.io/visualize>) ou o Lightstep (<https://lightstep.com>). Embora essas ferramentas muitas vezes sejam vistas como soluções para tracing distribuído (assunto que exploraremos melhor mais adiante), elas são altamente capacitadas para armazenar, filtrar e consultar dados de alta cardinalidade.

Sistemas de monitoração e observabilidade são sistemas para ambientes de produção

Com um conjunto cada vez maior de ferramentas para ajudar a

gerenciar nossa arquitetura de microsserviços, lembre-se de que essas ferramentas são sistemas para o ambiente de produção. Plataformas de agregação de logs, ferramentas para tracing distribuído, sistemas de alertas são, todas elas, aplicações de propósito essencial, tão vitais quanto o nosso próprio software, se não mais. O mesmo grau de zelo aplicado ao software que escrevemos e mantemos deve ser dedicado à manutenção de nossas ferramentas de monitoração do ambiente de produção.

Devemos também reconhecer que essas ferramentas podem se tornar possíveis vetores de ataque por parte de agentes externos. Atualmente – quando este livro foi escrito – o governo norte-americano e outras empresas ao redor do mundo estão lidando com a descoberta de uma falha no software de gerenciamento de rede da SolarWinds. Embora a natureza exata da falha ainda esteja sendo explorada, acredita-se que seja algo conhecido como ataque de cadeia de fornecedores (supply chain attack). Assim que foi instalado nos clientes (e o SolarWinds é utilizado por 425 empresas Fortune 500 dos Estados Unidos), esse software permitia que agentes mal-intencionados tivessem acesso externo às redes dos clientes, incluindo a rede do Tesouro dos Estados Unidos.

Tracing distribuído

Até agora, de modo geral, discutimos a coleta de informações de forma isolada. Sim, estamos agregando essas informações, mas entender o contexto mais amplo no qual essas informações foram capturadas pode ser essencial. Basicamente, uma arquitetura de microsserviços é um conjunto de processos que atuam em colaboração para executar algum tipo de tarefa – exploramos as diversas maneiras de coordenar essas atividades no Capítulo 6. Então, faz sentido, quando queremos entender como nosso sistema realmente se comporta em um ambiente de produção, que sejamos capazes de ver os relacionamentos entre os nossos microsserviços. Isso pode nos ajudar a entender melhor como o nosso sistema se comporta, avaliar o impacto de um problema, ou melhorar nossa capacidade de descobrir o que, exatamente, não está funcionando conforme esperado.

À medida que nossos sistemas se tornam mais complexos, portanto, será importante ter uma maneira de observar essas pistas em nossos sistemas. Precisamos ser capazes de extrair esses dados díspares para termos uma visão conjunta de uma série de chamadas correlacionadas. Conforme já vimos, fazer algo simples como inserir IDs de correlação em nossos arquivos de log é um bom ponto de partida, mas é uma solução razoavelmente simplista, sobretudo porque acabaremos tendo de criar nossas próprias ferramentas personalizadas para ajudar na visualização e na análise detalhada desses dados. É nesse caso que o tracing distribuído entra em cena.

Funcionamento

Embora as implementações exatas variem, falando de modo geral, todas as ferramentas de tracing distribuído funcionam de modo semelhante. As atividades locais em uma thread são capturadas em um *span*. Esses spans individuais são correlacionados com algum identificador único. Os spans são então enviados para um coletor central, capaz de transformar esses spans relacionados em um único *trace*. Na Figura 10.7, vemos uma imagem do Honeycomb que mostra um trace em uma arquitetura de microsserviços.

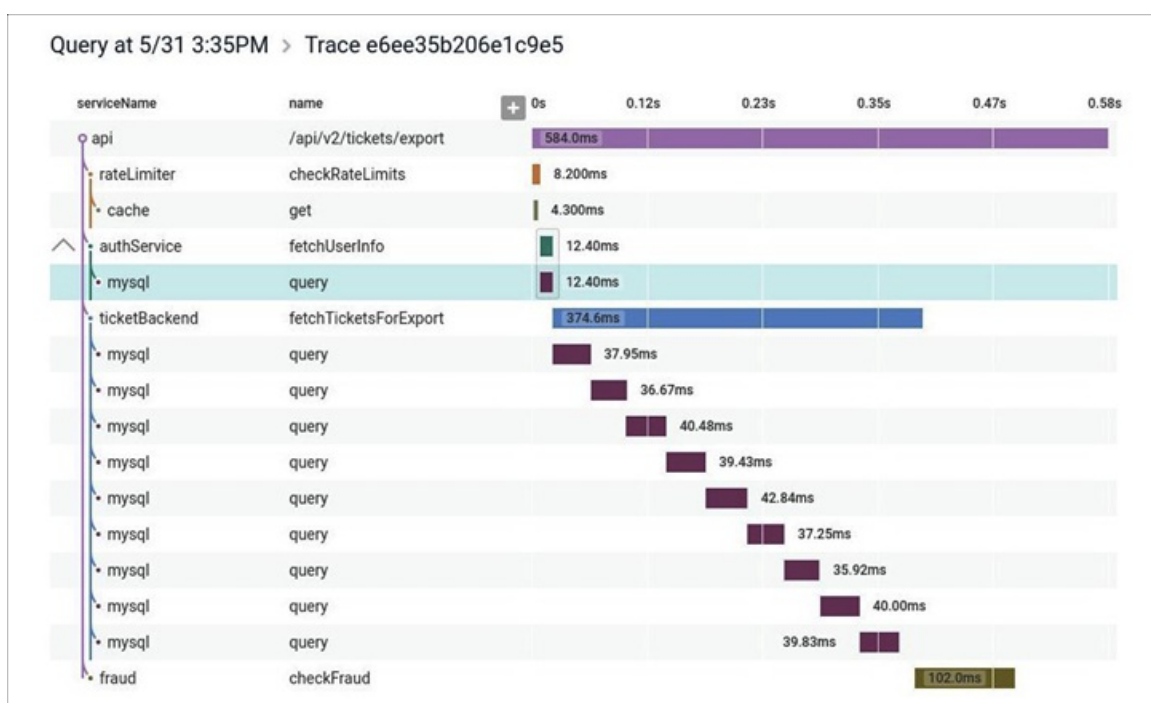


Figura 10.7 – Um trace distribuído exibido no Honeycomb, permitindo identificar em que pontos o tempo está sendo consumido em operações que podem envolver vários microsserviços.

Esses spans permitem coletar uma série de informações. Os dados exatos a serem coletados dependerá do protocolo usado, mas, no caso da OpenTracing API, cada span conterá um horário de início e de fim, um conjunto de logs associados ao span e um conjunto arbitrário de pares chave-valor para auxiliar em consultas posteriores (poderiam ser usados para enviar informações como ID de um cliente, ID de um pedido, um nome de host, um número de build, e dados semelhantes).

Coletar informações suficientes que nos permitam rastrear as chamadas em nosso sistema pode ter um impacto direto no próprio sistema. Haverá a necessidade de termos alguma forma de amostragem, na qual algumas informações serão explicitamente excluídas do nosso conjunto de traces a fim de garantir que o sistema continue funcionando. O desafio está em garantir que as informações corretas sejam descartadas, e que ainda coletaremos amostras suficientes, que nos permitam fazer uma extrapolação correta das observações.

As estratégias de amostragem podem ser muito básicas. O sistema Dapper do Google, que serviu de inspiração para muitas das ferramentas de tracing distribuído que vieram depois, efetuava uma amostragem aleatória extremamente agressiva. Uma certa porcentagem das chamadas era amostrada, e isso era tudo. O Jaeger, por exemplo, captura somente 1 a cada 1.000 chamadas em sua configuração default. A ideia, nesse caso, é capturar informações suficientes para entender o que o nosso sistema faz, porém não capturar tantas informações a ponto de o próprio sistema não conseguir lidar com isso. Ferramentas como o Honeycomb e o Lightstep podem oferecer amostragem dinâmicas, com mais nuances, em vez dessa amostragem aleatória simples. Um exemplo de amostragem dinâmica poderia ser aquele em que queremos mais amostras de determinados tipos de eventos – por exemplo, poderíamos amostrar tudo que gere um erro, mas ficaríamos satisfeitos em amostrar apenas 1 de 100 operações bem-sucedidas caso elas sejam todas muito parecidas.

Implementando um tracing distribuído

Deixar um tracing distribuído pronto para executar em seu sistema exige algumas ações. Em primeiro lugar, precisamos capturar informações de span em nossos microsserviços. Se você utiliza uma API padrão como o OpenTracing ou a API OpenTelemetry mais recente, perceberá que algumas das bibliotecas e frameworks de terceiros incluirão suporte pronto para essas APIs, e já enviarão informações úteis (por exemplo, capturarão informações sobre chamadas HTTP automaticamente). No entanto, ainda que o façam, há uma boa chance de você ainda querer instrumentar o seu próprio código, disponibilizando informações úteis sobre o que seu microsserviço faz em qualquer dado instante no tempo.

Em seguida, será necessário ter alguma forma de enviar essas informações de span para o seu coletor – você poderia enviar esses dados diretamente de sua instância de microsserviço para o coletor centralizado, porém é muito mais comum utilizar um agente local para encaminhamento desses dados. Assim como no caso da agregação de logs, executamos um agente local à nossa instância de microsserviço, o qual enviará periodicamente as informações de span para um coletor centralizado. O uso de um agente local geralmente permite ter recursos mais sofisticados, por exemplo, alterar a amostragem ou acrescentar tags extras, além de permitir o armazenamento das informações a serem enviadas em um buffer, de modo mais eficaz.

Por fim, é claro, precisamos de um coletor capaz de receber essas informações e compreendê-las.

Na área de open source, o Jaeger tem se destacado como uma opção popular para uma solução de tracing distribuído. Entre as ferramentas comerciais, eu começaria considerando o Lightstep e o Honeycomb, já mencionados. Eu pediria, porém, que você escolhesse algo que esteja comprometido em oferecer suporte para a OpenTelemetry API. A OpenTelemetry (<https://opentelemetry.io>) é uma especificação de API aberta que facilita bastante ter códigos como drivers de banco de dados ou frameworks web com suporte imediato para tracing, além de possibilitar uma portabilidade mais fácil entre diferentes fornecedores do lado da

coleta. Com base no trabalho feito pelas APIs OpenTracing e OpenConsensus anteriores, essa API tem ampla aceitação no mercado atualmente.

Estamos indo bem?

Falamos bastante daquilo que poderíamos fazer como operador de um sistema – a mentalidade necessária, as informações que precisaremos coletar. Mas como saber se não estamos fazendo demais – ou não estamos fazendo o suficiente? Como saber se nosso trabalho é suficientemente bom, ou se nosso sistema está funcionando da forma devida?

Conceitos binários de um sistema estando “ativo” (up) ou “inativo” (down) começam a ter cada vez menos significado à medida que o sistema se torna mais complexo. Em um sistema monolítico com um único processo, será mais fácil olhar para a saúde do sistema como uma qualidade do tipo “preto ou branco”. Mas e quanto a um sistema distribuído? Se uma instância de um microserviço estiver inacessível, será um problema? Um microserviço estará “saudável” se estiver acessível? E o que dizer de uma situação em que nosso microserviço *Devoluções* está disponível, porém metade das funcionalidades que oferece exigem o uso do microserviço downstream *Estoque*, o qual está passando por problemas no momento? Isso significa que vamos considerar o microserviço *Devoluções* saudável ou não saudável?

À medida que a complexidade aumenta, torna-se cada vez mais importante dar um passo para trás e pensar no sistema a partir de outra perspectiva. Pense em uma colmeia. Poderíamos olhar para uma abelha individual e constatar que ela não está feliz. Talvez ela tenha perdido uma de suas asas e, desse modo, não consiga mais voar. Isso, sem dúvida, é um problema para essa abelha individual, mas seria possível fazer qualquer observação sobre a saúde da colmeia em questão a partir daí? Não – será preciso olhar para a saúde da colmeia de um modo mais holístico. Uma abelha ferida não implica que a colmeia como um todo não está saudável.

Podemos tentar descobrir se um serviço está *saudável* decidindo, por exemplo, o que é um bom nível de CPU, ou o que podemos considerar um

tempo de resposta aceitável. Se nosso sistema de monitoração detectar que os valores reais estão fora desse nível de segurança, podemos disparar um alerta. Entretanto, em vários aspectos, esses valores estão um passo distante daquilo que queremos realmente monitorar – ou seja, *o sistema está funcionando?* Quanto mais complexas as interações entre os serviços, mais distante estaremos de responder realmente a essa pergunta simplesmente observando uma métrica de forma isolada.

Desse modo, podemos reunir muitas informações, mas, por si só, elas não nos ajudarão a responder à pergunta acerca do sistema estar funcionando de modo apropriado. Para isso, temos de começar a pensar um pouco mais na definição do que seria um comportamento aceitável. Muito trabalho tem sido feito na área de *SRE* (Site Reliability Engineering, ou Engenharia de Confiabilidade de Sites), cujo foco está em como garantir que nossos sistemas sejam confiáveis, ao mesmo tempo que permitimos que haja mudanças. Há alguns conceitos úteis a serem explorados, os quais são oriundos dessa área.

Apertem os cintos – estamos prestes a entrar no reino dos acrônimos.

Acordo de nível de serviço

Um *SLA* (Service-Level Agreement, ou Acordo de Nível de Serviço) é um acordo entre as pessoas que criam o sistema e as pessoas que o utilizam. Descreve não só o que os usuários podem esperar, mas também o que acontecerá se o sistema não atingir esse nível de comportamento aceitável. Os SLAs geralmente tendem a definir o nível “mínimo” esperado, muitas vezes de um modo que, se o sistema somente se restringir a alcançar essas metas, o usuário final ficaria insatisfeito. Como exemplo, a AWS tem um SLA para seu serviço de computação. Ela deixa claro que não há uma garantia efetiva de uptime para uma única instância EC2 (uma máquina virtual gerenciada⁸). A AWS declara que se esforça ao máximo para garantir 90% de uptime para uma dada instância, mas, se esse valor não puder ser atingido, ela simplesmente não cobrará você pelo tempo em que a instância estiver indisponível. No entanto, se suas instâncias EC2 falhassem constantemente em alcançar 90% de disponibilidade em uma dada hora, causando instabilidades significativas no sistema, você talvez

não vá ser cobrado por isso, mas também não ficaria muito satisfeito. Com base em minha experiência, posso dizer que, na prática, a AWS vai muito além do que o SLA define, como geralmente ocorre com as SLAs.

Objetivos de nível de serviço

Mapear um SLA a uma equipe é complicado, particularmente se o SLA for, de certo modo, genérico e abrangente. Em vez de SLA, no nível de uma equipe, falamos de *SLOs* (Service-Level Objectives, ou Objetivos de Nível de Serviço). Os *SLOs* definem o que a equipe se compromete a oferecer. Atingir os *SLOs* de todas as equipes da empresa satisfará (e, provavelmente, excederá em grande medida) os requisitos de SLAs da empresa. Exemplos de *SLOs* poderiam incluir algo como uptime esperado ou tempos de resposta aceitáveis para uma dada operação.

Seria simplista demais pensar nos *SLOs* como o que uma equipe precisa fazer para que a empresa atinja seus SLAs. Sim, se a empresa atingir todos os seus *SLOs*, poderíamos supor que todos os SLAs foram cumpridos, mas os *SLOs* podem fazer menção a outros objetivos não definidos no SLA – ou poderiam representar aspirações, ou serem metas de caráter interno (por exemplo, tentar fazer alguma alteração interna). Os *SLOs* muitas vezes podem refletir algo que a equipe em questão queira alcançar, sem nenhuma relação com um SLA.

Indicadores de nível de serviço

Para determinar se estamos cumprindo nossos *SLOs*, precisamos coletar dados reais. É isso que são os nossos *SLIs* (Service-Level Indicators, ou Indicadores de Nível de Serviço). Um *SLI* é uma medida de algo que nosso software faz. Poderia ser, por exemplo, um tempo de resposta de um processo, um cliente sendo cadastrado, um erro exibido ao cliente ou um pedido feito. Precisamos coletar e exibir esses *SLIs* a fim de garantir que estamos cumprindo nossos *SLOs*.

Orçamentos de erros

Quando tentamos introduzir novidades, introduzimos mais instabilidade em potencial em nossos sistemas. Desse modo, o desejo de manter (ou

melhorar) a estabilidade do sistema poderia resultar no desencorajamento de uma mudança. Os *orçamentos de erros* (error budgets) são uma tentativa de evitar esse problema, deixando clara a quantidade de erros que é aceitável em um sistema.

Se você já se decidiu a respeito de um SLO, então determinar seu orçamento de erros deve ser muito simples. Por exemplo, você poderia dizer que seu microsserviço precisa estar disponível 99,9% do tempo por trimestre, 24 horas por dia, 7 dias por semana. Isso significa que, na verdade, o sistema poderia ficar inativo por 2 horas e 11 minutos por trimestre. No que concerne a esse SLO, esse seria o seu orçamento de erros.

Os orçamentos de erros contribuem para que você tenha um entendimento mais claro sobre o bom cumprimento (ou não) do SLO, permitindo que você tome decisões melhores acerca dos riscos que pode correr. Se você estiver bem abaixo de seu orçamento de erros para o trimestre, talvez não haja problemas em fazer o rollout daquele microsserviço escrito em uma nova linguagem de programação. Se já tiver excedido o seu orçamento, talvez seja melhor adiar esse rollout e fazer a equipe se concentrar mais na melhoria da confiabilidade de seu sistema.

Mais do que tudo, os orçamentos de erros dizem respeito a dar espaço de manobra para que as equipes possam tentar algo novo.

Alertas

Ocasionalmente (espero que raramente, mas é provável que seja com mais frequência que o desejado), algo acontecerá com nossos sistemas, o qual exigirá que um operador humano seja notificado para que tome alguma atitude. Um microsserviço pode ter ficado indisponível de forma inesperada, talvez haja um número maior que o esperado de erros ou, quem sabe, o sistema como um todo tenha se tornado indisponível aos nossos usuários. Nesses casos, precisamos que certas pessoas sejam informadas a respeito do que está acontecendo para que possam tentar corrigir a situação.

O problema é que, em uma arquitetura de microsserviços, considerando o

número maior de chamadas e processos, e a infraestrutura subjacente mais complexa, haverá algo errado com mais frequência. O desafio em um ambiente de microsserviços é descobrir exatamente quais tipos de problemas devem fazer com que uma pessoa seja informada, e exatamente como ela deve ser informada.

Alguns problemas são piores que outros

Quando houver algo errado, queremos tomar conhecimento do problema. Ou não? Todos os problemas são iguais? À medida que as causas dos problemas aumentam, será mais importante ser capaz de priorizá-los a fim de decidir se – e como – um operador humano deve ser envolvido. Muitas vezes, a grande pergunta que me vejo fazendo quando se trata de gerar alertas é: “Esse problema deve fazer com que alguém seja despertado às três da manhã?”.

Vi um exemplo desse raciocínio quando passei um tempo no campus do Google muitos anos atrás. Na área de recepção de um dos prédios em Mountain View havia um antigo rack de máquinas, deixado ali como uma espécie de exposição. Percebi alguns detalhes. Em primeiro lugar, esses servidores não estavam em gabinetes para servidores; eram apenas placas-mães encaixadas no rack. O principal detalhe que percebi, porém, foi que os discos rígidos estavam presos com velcro. Perguntei o porquê disso a um dos funcionários do Google. “Ah” – ele disse – “os discos rígidos falham tanto que não queremos deixá-los parafusados. Simplesmente os puxamos para retirá-los, jogamos no lixo e prendemos outro com velcro.”

Os sistemas construídos pelo Google supunham que os discos rígidos iriam falhar. Eles otimizaram o design desses servidores a fim de garantir que a substituição dos discos rígidos fosse o mais fácil possível. Como o sistema havia sido construído para tolerar a falha de um disco rígido, embora uma troca ocasional de disco fosse importante, havia uma boa chance de que a falha de um único disco rígido não causasse nenhum problema significativo que fosse visível ao usuário. Com milhares de servidores em um data center do Google, a tarefa diária de uma pessoa poderia ser simplesmente caminhar por uma fileira de racks e ir substituindo os discos rígidos. Sem dúvida, a falha em um disco era um problema, mas com a

qual era possível lidar como parte da rotina. Uma falha em um disco rígido era considerada rotina – não justificaria chamar alguém fora do horário comercial, mas seria apenas uma ocorrência que deveria ser informada a alguém em seu horário de expediente regular.

À medida que as causas dos possíveis problemas aumentam, é necessário ter mais eficácia para priorizar os tipos de ocorrências que geram determinados tipos de alertas. Do contrário, você poderá se ver com dificuldade para separar o trivial do urgente.

Fadiga de alertas

Muitas vezes, um excesso de alertas pode causar problemas significativos. Em 1979, houve um derretimento parcial de um reator na usina de energia nuclear de Three Mile Island nos Estados Unidos. A investigação sobre o incidente revelou que os operadores da instalação ficaram tão sobrecarregados com os alertas que viram, a ponto de ter sido impossível determinar qual seria a atitude necessária a ser tomada. Houve um alerta informando o problema fundamental a ser resolvido, mas não foi muito claro para os operadores, pois vários outros alertas foram disparados ao mesmo tempo. Durante a audiência pública sobre o incidente, um dos operadores, Craig Faust, recordou que “Minha vontade era ter jogado fora o painel de alarmes. Ele não estava nos dando nenhuma informação útil”. O relatório sobre o incidente concluiu que a sala de controle “era extremamente inadequada para administrar um acidente”.²

Mais recentemente, vimos esse problema de alertas em excesso disparados no contexto de uma série de incidentes com a aeronave 737 Max, incluindo dois acidentes aéreos distintos que resultaram na morte de 346 pessoas no total. O relatório inicial¹⁰ sobre os problemas, feito pelo NTSB (National Transportation Safety Board, ou Comitê de Segurança dos Transportes Nacionais) dos Estados Unidos chamou a atenção para os alertas confusos disparados em condições reais, os quais foram considerados fatores que contribuíram para os acidentes. Segundo o relatório:

Uma pesquisa sobre fatores humanos constatou que, em condições anormais, como aquelas que envolvem uma falha de sistema com vários

alertas e várias ações podem ser exigidas da tripulação de voo, permitir que os pilotos saibam quais são as ações que devem ser prioritárias é uma necessidade essencial. Isso é particularmente verdadeiro no caso de funções implementadas de forma distribuída entre vários sistemas da aeronave, pois uma falha em um sistema com arquiteturas altamente integradas pode resultar em vários alertas e indicações à tripulação de voo, uma vez que cada sistema associado registrará a falha...Desse modo, é importante que as interações entre os sistemas e a interface da cabine de comando sejam projetadas para ajudar a orientar os pilotos a executarem a(s) ação(ões) de mais alta prioridade.

Então, cá estamos nós falando de operar um reator nuclear e pilotar uma aeronave. Suspeito que muitos de vocês estão se perguntando neste momento o que isso tem a ver com o sistema que estão construindo. É possível (ou provável) que você não esteja criando sistemas como esses em que a segurança é crítica, mas podemos aprender muito com esses exemplos. Ambos envolvem sistemas inter-relacionados, altamente complexos, em que um problema em uma área pode causar problema em outra área. Quando geramos alertas demais, ou não permitimos que os operadores priorizem os alertas nos quais devem manter o foco, um desastre pode ocorrer. Sobrecarregar um operador com alertas pode causar verdadeiros problemas. Segundo o relatório, novamente:

Além disso, a pesquisa sobre as respostas dos pilotos a situações anômalas diversas/simultâneas, junto com dados sobre os acidentes, mostra que vários alertas concorrentes podem esgotar os recursos mentais disponíveis e desviar o foco da atenção, resultando em respostas demoradas ou com prioridades inadequadas.

Então, pense duas vezes antes de jogar mais alertas em cima de um operador – você talvez não vá conseguir o que quer.

Alarme versus alertas

Ao analisar mais amplamente o tópico sobre alertas, encontrei um volume muito grande de pesquisas e práticas extremamente úteis em uma série de contextos, muitas das quais não diziam respeito especificamente aos alertas em sistemas de TI. O termo *alarme* é comum quando vemos esse tópico na engenharia e em outras áreas, enquanto

tendemos a empregar o termo *alerta* com mais frequência em TI. Conversei com algumas pessoas que veem uma distinção entre esses dois termos, mas, por estranho que pareça, as distinções que elas fizeram entre os termos não pareceram ser consistentes. Com base no fato de que a maioria das pessoas parece ver os termos *alerta* e *alarme* como praticamente iguais, e no fato de que, quando uma distinção foi feita entre esses termos, ela não foi consistente, decidi utilizar o termo *alerta* como padrão neste livro.

Em direção a alertas mais apropriados

Queremos então evitar ter alertas demais, assim como evitar que haja alertas inúteis. Quais são as diretrizes que devemos observar para nos ajudar a criar alertas mais apropriados?

Steven Shorrock desenvolve esse assunto em seu artigo “Alarm Design: From Nuclear Power to WebOps”¹¹, que é uma ótima leitura e um excelente ponto de partida para outras leituras nessa área. Segundo o artigo:

O propósito [dos alertas] é direcionar a atenção do usuário para os aspectos significativos da operação ou do equipamento que exijam uma atenção imediata.

Com base em trabalhos externos ao desenvolvimento de software, temos um conjunto conveniente de regras que, por incrível que pareça, são da EEMUA (Engineering Equipment and Materials Users Association, ou Associação dos Usuários de Materiais e Equipamentos de Engenharia), que elaborou a melhor descrição que já vi acerca do que seria um bom alerta:

Relevante

Certifique-se de que o alerta seja importante.

Único

Garanta que o alerta não seja uma duplicata de outro.

Na hora certa

Precisamos receber o alerta com rapidez suficiente para podermos usá-lo.

Priorizado

Dê informações suficientes ao operador para que ele possa decidir a ordem em que deve tratar os alertas.

Compreensível

As informações no alerta devem ser claras e legíveis.

Apresenta um diagnóstico

O alerta deve deixar claro o que há de errado.

Apresenta orientações

Ajude o operador a saber quais atitudes devem ser tomadas.

Com foco

Chame a atenção para os problemas mais importantes.

Olhando para minha carreira no passado, quando eu trabalhava com suporte à produção, é deprimente pensar que raramente os alertas com os quais tive de lidar seguiam essas regras.

Com muita frequência, infelizmente, as pessoas que fornecem informações aos nossos sistemas de alerta e as pessoas que estão realmente do lado da recepção dos alertas são pessoas diferentes. De acordo com Shorrock novamente:

Entender a natureza do tratamento dos alarmes e os problemas de design associados pode ajudar você – o especialista em seu trabalho – a ser um usuário mais bem informado, contribuindo para criar os melhores sistemas de alarmes para auxiliar em seu trabalho.

Uma técnica que pode ajudar a reduzir o número de alertas que competem por nossa atenção envolve, antes de tudo, mudar o modo como pensamos nos problemas que exigem chamar a atenção dos operadores. Vamos explorar esse assunto a seguir.

Monitoração semântica

Na monitoração semântica, definimos um modelo para o que é a semântica aceitável em nosso sistema. Quais são as propriedades que o sistema deve ter para que achemos que ele está funcionando com médias aceitáveis? Em boa medida, a monitoração semântica exige uma mudança em nosso

comportamento. Em vez de verificar a presença de erros, precisamos fazer constantemente uma pergunta: o sistema está se comportando do modo como esperamos? Se estiver se comportando corretamente, isso nos ajudará a entender melhor como lidar com a priorização dos erros detectados.

O próximo trabalho a ser feito é definir um modelo para um sistema que se comporta corretamente. Podemos utilizar uma abordagem extremamente formal (de um modo bastante literal, com algumas empresas utilizando métodos formais para isso), mas fazer algumas afirmações simples de valor pode ajudar bastante. Por exemplo, no caso da MusicCorp, o que deve ser verdadeiro para que acreditemos que o sistema está funcionando corretamente e fiquemos satisfeitos? Bem, talvez pudéssemos dizer que:

- Novos clientes podem se cadastrar.
- Estamos vendendo no mínimo 20 mil dólares em produtos por hora em nosso horário de pico.
- Estamos despachando pedidos a uma taxa considerada normal.

Se essas três afirmações puderem se provar serem corretas, então, falando de modo geral, acharemos que o sistema está funcionando de modo suficientemente apropriado. Retornando às nossas discussões anteriores sobre SLAs e SLOs, esperamos que nosso modelo de semântica apropriada exceda em muito as nossas obrigações em um SLA, e esperamos ter SLOs concretos que nos permitam fazer uma verificação em relação a esse modelo. Falando de outra maneira, fazer essas afirmações acerca de como esperamos que nosso software irá atuar será muito útil para identificarmos nossos SLOs.

Um dos maiores desafios é chegar a um consenso acerca de como será esse modelo. Como podemos ver, não estamos falando de algo no baixo nível como “o uso de disco não deve exceder 95%”; estamos fazendo afirmações de nível mais alto sobre o nosso sistema. Na função de operador do sistema, ou como a pessoa que escreveu e testou o microsserviço, você talvez não esteja em uma posição para decidir quais devem ser essas afirmações de valor. Em uma empresa orientada a produtos, é nessa hora que o dono do produto (product owner) deve entrar em cena – mas talvez

seja sua tarefa como operador garantir que a discussão com o dono do produto realmente ocorra.

Assim que tiver decidido qual será o seu modelo, a questão se reduzirá a descobrir se o comportamento atual do sistema está de acordo com esse modelo. Falando de modo geral, temos duas maneiras básicas de fazer isso – monitoração de usuários reais e transações sintéticas. Veremos as transações sintéticas em breve, pois elas se enquadram no tópico mais amplo de testes em produção, mas vamos analisar antes a monitoração de usuários reais.

Monitoração de usuários reais

Na monitoração de usuários reais, observamos o que realmente acontece em nosso sistema de produção e comparamos isso com nosso modelo semântico. Na MusicCorp, observaríamos quantos clientes se cadastram, quantos pedidos enviamos, e assim por diante.

O desafio na monitoração de usuários reais é que, com frequência, as informações de que precisamos não estarão disponíveis a nós rapidamente. Considere a expectativa de que a MusicCorp deve vender no mínimo 20 mil dólares em produtos por hora. Se essa informação estiver guardada em um banco de dados em algum lugar, talvez não seja possível coletá-la e atuar sobre ela. É por isso que você deve aprimorar sua capacidade de expor o acesso a informações que você consideraria anteriormente como métricas de “negócios” às suas ferramentas de produção. Se você pode enviar uma taxa de CPU ao seu repositório de métricas, e esse repositório de métricas pode ser usado para gerar alertas sobre essa condição, por que não seria possível também registrar uma venda e um valor monetário nesse mesmo repositório?

Uma das principais desvantagens da monitoração de usuários reais é que ela é basicamente repleta de ruídos. Você obterá muitas informações – filtrá-las para descobrir se há algum problema pode ser difícil. Também vale a pena notar que a monitoração de usuários reais fornece informações sobre algo que já aconteceu e, como consequência, um problema só será identificado depois de já ter ocorrido. Se um cliente não conseguiu se

cadastrar, esse será um cliente insatisfeito. Com as transações sintéticas – outra forma de fazer testes em produção, a qual veremos em breve – temos a oportunidade não só de reduzir o ruído, mas também de identificar problemas antes que nossos usuários os percebam.

Testes no ambiente de produção

Não testar no ambiente de produção é como não ensaiar com a orquestra completa porque sua apresentação solo parecia boa em casa.¹²

Charity Majors

Conforme já abordamos diversas vezes neste livro, desde nossa discussão sobre conceitos como implantações canário na seção “Lançamento canário”, na página 324, até quando falamos de ponderar sobre executar testes pré e pós-produção, conduzir alguma forma de testes no ambiente de produção pode ser uma atividade extremamente conveniente – e segura. Vimos uma série de diferentes tipos de testes no ambiente de produção neste livro, e há várias outras formas além desses, portanto achei que seria conveniente resumir alguns dos diferentes tipos de testes que já vimos, além de apresentar outros exemplos que são comuns. Fico surpreso com o número de pessoas que ficam assustadas com o conceito de testes no ambiente de produção, mas que já os executam sem realmente perceber.

Todas as formas de testes em produção são, sem dúvida, uma forma de atividade de “monitoração”. Executamos esses tipos de testes em produção para garantir que nosso sistema em produção esteja executando conforme esperado, e muitas formas de teste em produção podem ser extremamente eficazes para identificar problemas antes que nossos usuários cheguem a perceber.

Transações sintéticas

Com as transações sintéticas, inserimos o comportamento de um usuário simulado em nosso sistema de produção. O comportamento desse usuário simulado apresenta entradas conhecidas e saídas esperadas. Na MusicCorp, por exemplo, poderíamos criar artificialmente um cliente, e

então verificar se esse cliente foi criado com sucesso. Essas transações seriam disparadas regularmente, o que nos dá a chance de identificar problemas o mais rápido possível.

Fiz isso pela primeira vez nos idos de 2005. Eu fazia parte de uma pequena equipe na Thoughtworks, que estava construindo um sistema para um banco de investimentos. Durante o dia de negociações, ocorriam muitos eventos que representavam mudanças no mercado. Nosso trabalho consistia em reagir a essas mudanças e observar o impacto no portfólio do banco. Estávamos trabalhando com prazos bastante apertados, e um objetivo complicado era concluir todos os nossos cálculos em menos de dez segundos após a chegada do evento. O sistema em questão era composto de cerca de cinco serviços discretos, com pelo menos um executando em um sistema de computação em grade (grid), que, entre outras coisas, buscava ciclos de CPU não usados em aproximadamente 250 desktops hosts no centro de recuperação de desastres do banco.

O fato de o sistema ser composto de várias partes implicava que havia muito ruído nas diversas métricas de nível mais baixo que coletávamos. Também não tínhamos a vantagem de escalar gradualmente nem de poder executar o sistema por alguns meses para entender como seria uma “boa” execução no que dizia respeito às métricas de baixo nível, como taxa de CPU ou tempos de resposta. Nossa abordagem consistia em gerar eventos simulados para atribuir preços em parte do portfólio, os quais não eram enviados aos sistemas downstream. Aproximadamente a cada minuto, usávamos uma ferramenta chamada Nagios para executar um job de linha de comando que inseria um evento simulado em uma de nossas filas. Nosso sistema detectava esse evento e executava todos os diversos cálculos, como qualquer outro job, exceto que os resultados apareciam em um registro contábil “lixo”, que era usado somente para testes. Se uma nova atribuição de preços não fosse vista dentro de um determinado período, o Nagios informava que havia um problema.

Na prática, percebi que o uso de transações sintéticas para efetuar uma monitoração semântica como esse era um indicador muito melhor para problemas nos sistemas, em comparação com a geração de alertas com

base nas métricas de nível mais baixo. Elas, porém, não substituem a necessidade de ter os detalhes de nível mais baixo – vamos querer essas informações quando precisarmos descobrir *por que* uma transação sintética falhou.

Implementando transações sintéticas. No passado, implementar transações sintéticas era uma tarefa razoavelmente complicada. No entanto, o mundo evoluiu, e os meios para implementá-las estão ao nosso alcance imediato! Você está executando testes para seus sistemas, certo? Se não estiver, leia o Capítulo 9 e volte. Pronto? Ótimo!

Se observarmos os testes que temos para um dado serviço fim a fim, ou até mesmo para o nosso sistema fim a fim, veremos que temos boa parte do que precisamos para implementar uma monitoração semântica. Nosso sistema já expõe os hooks (ganchos) necessários para iniciar o teste e verificar o resultado. Então, por que não executar simplesmente um subconjunto desses testes, de forma contínua, como uma forma de monitorar nosso sistema?

Há algumas tarefas que precisamos fazer, é claro. Em primeiro lugar, devemos ter cuidado com os requisitos de dados para os nossos testes. Talvez precisemos encontrar uma maneira de nossos testes se adaptarem aos diferentes dados atuais caso mudem com o tempo, ou então definir uma fonte de dados diferente. Por exemplo, poderíamos ter um conjunto de usuários simulados que usamos no ambiente de produção, com um conjunto conhecido de dados.

Do mesmo modo, precisamos garantir que não iremos acidentalmente gerar efeitos colaterais imprevistos. Um amigo me contou uma história sobre uma empresa de comércio eletrônico que acidentalmente executou seus testes em seus sistemas de pedidos no ambiente de produção. Ela não havia percebido seu erro até várias máquinas de lavar terem chegado na sede da empresa.

Testes A/B

Com um teste A/B, implantamos duas versões diferentes da mesma funcionalidade, com os usuários vendo a funcionalidade “A” ou a

funcionalidade “B”. Você será então capaz de ver qual versão da funcionalidade tem o melhor desempenho. Isso é comum quando tentamos decidir entre duas abordagens diferentes para fazer algo – por exemplo, podemos testar dois formulários distintos para cadastro de clientes a fim de ver qual deles é mais eficaz para aumentar o número de inscrições.

Lançamento canário

Uma pequena porção de sua base de usuários passa a ver a nova versão de uma funcionalidade. Se essa nova funcionalidade se comportar bem, podemos aumentar a parcela da base de usuários que verá a nova funcionalidade, até o ponto em que a nova versão seja utilizada por todos os usuários. Por outro lado, se a nova funcionalidade não se comportar conforme esperado, o impacto estaria restrito somente a uma pequena parcela de nossa base de usuários e a mudança poderia ser revertida, ou tentaríamos corrigir qualquer que fosse o problema identificado.

Execução paralela

Em uma execução paralela, executamos duas implementações equivalentes e distintas da mesma funcionalidade, lado a lado. Qualquer requisição de usuário será encaminhada para as duas versões e o resultado poderá ser comparado. Assim, em vez de direcionar um usuário para a versão antiga ou para a versão nova, como ocorre em um lançamento canário, executamos ambas as versões – no entanto, o usuário verá apenas uma. Isso permite fazer uma comparação completa entre as duas versões distintas, o que é extremamente conveniente se quisermos entender melhor alguns aspectos como as características de carga de uma nova implementação de alguma funcionalidade essencial.

Testes de fumaça

Os testes de fumaça são usados após o software ser implantado no ambiente de produção, porém antes de ser lançado, e são executados para garantir que o software está funcionando de modo apropriado. Esses testes em geral são totalmente automatizados e podem variar, desde atividades

muito simples, como garantir que um dado microsserviço está ativo, até executar transações sintéticas completas.

Transações sintéticas

Uma interação completa de um usuário simulado é introduzida no sistema. Com frequência, essas interações são muito parecidas com o tipo de testes fim a fim que você poderia escrever.

Engenharia de caos

É um assunto que discutiremos melhor no Capítulo 12; a engenharia de caos (chaos engineering) pode envolver a inserção de falhas em um sistema de produção a fim de garantir que ele seja capaz de lidar com esses problemas previstos. O exemplo mais conhecido dessa técnica provavelmente é o Chaos Monkey da Netflix, capaz de desativar máquinas virtuais em produção, com a expectativa de que o sistema seja suficientemente robusto a ponto de esses shutdowns não interromperem as funcionalidades dos usuários finais.

Padronização

Conforme discutimos antes, um dos aspectos sobre os quais você terá de ponderar continuamente será quando permitir que as decisões sejam feitas especificamente para um único microsserviço *versus* quando será necessário padronizá-las para todo o sistema. Em minha opinião, monitoração e observabilidade são áreas em que uma padronização pode ser extremamente importante. Com microsserviços que colaboram entre si de várias maneiras diferentes para oferecer funcionalidades aos usuários por meio de várias interfaces, será preciso olhar para o sistema de forma holística.

Tente escrever seus logs em um formato padrão. Certamente, você vai querer ter todas as suas métricas em um só lugar, e talvez queira ter uma lista de nomes padrões para suas métricas também; seria muito irritante que um serviço tivesse uma métrica chamada `ResponseTime`, e outro serviço tivesse uma métrica chamada `RspTimeSecs`, quando ambas tivessem o mesmo

significado.

Como sempre ocorre na padronização, as ferramentas podem ajudar. Conforme já disse antes, o segredo é permitir que seja fácil fazer o que é certo – assim, ter uma plataforma com muitos dos blocos de construção prontos, por exemplo, uma agregação de logs, faz muito sentido. Cada vez mais, boa parte dessas tarefas será de responsabilidade da equipe de plataforma, cuja função exploraremos de modo mais completo no Capítulo 15.

Seleção de ferramentas

Conforme já discutimos, há diversas ferramentas diferentes que você poderá adquirir para ajudar a melhorar a observabilidade de seu sistema. No entanto, como já mencionamos, essa é uma área que está mudando rapidamente, e é muito provável que as ferramentas que usaremos no futuro serão muito diferentes daquelas que temos agora. Com plataformas como o Honeycomb e o Lightstep liderando no que diz respeito ao modo como são as ferramentas para observabilidade de microsserviços, e com o resto do mercado, até certo ponto, procurando alcançá-las, espero realmente que essa área passe por muitas mudanças no futuro.

Então, é bem possível que você vá precisar de ferramentas diferentes daquelas que temos atualmente caso esteja adotando os microsserviços agora, e é possível que vá querer também outras ferramentas no futuro, pois as soluções nessa área continuam passando por melhorias. Com isso em mente, gostaria de apresentar algumas ideias acerca dos critérios que acho serem importantes para quaisquer ferramentas nessa área.

Democráticos

Se você tem ferramentas que são tão difíceis de trabalhar, a ponto de somente operadores experientes poderem utilizá-las, o número de pessoas que poderiam participar das atividades no ambiente de produção será limitado. De modo semelhante, se escolher ferramentas que são tão caras a ponto de seu uso ser proibitivo em qualquer situação que não seja em ambientes críticos de produção, os desenvolvedores não terão exposição a

essas ferramentas antes que seja tarde demais.

Escolha ferramentas que considere as necessidades de todas as pessoas que você quer que as utilize. Se quiser realmente passar para um modelo de mais responsabilidade coletiva sobre seu software, o software terá de ser utilizável por todos da equipe. Garantir que qualquer ferramenta escolhida também seja usada nos ambientes de desenvolvimento e de testes contribuirá bastante para fazer com que esse objetivo se torne uma realidade.

Fáceis de integrar

Obter as informações corretas de sua arquitetura de aplicações e dos sistemas nos quais ela executa é essencial; conforme já discutimos, talvez seja necessário extrair mais informações do que antes, em diferentes formatos. Deixar esse processo o mais simples possível é fundamental. Iniciativas como o OpenTracing (<https://opentracing.io>) têm ajudado no que diz respeito a disponibilizar APIs padrões aceitos por bibliotecas de cliente e plataformas, facilitando a integração e a portabilidade entre os diferentes conjuntos de ferramentas. De interesse especial, conforme falamos, é a nova iniciativa OpenTelemetry, que vem sendo conduzida por vários participantes.

Escolher ferramentas que aceitem esses padrões abertos facilitará o trabalho de integração e de uma possível mudança de fornecedores mais tarde.

Fornecem contexto

Ao observar uma informação, preciso que a ferramenta forneça o máximo possível de contexto para me ajudar a entender o que deve ser feito na sequência. Eu realmente gosto do sistema de classificação a seguir para diferentes tipos de contexto, o qual encontrei em uma postagem de blog da Lightstep:¹³

Contexto temporal

Como as informações se comparam a um minuto, uma hora, um dia ou um mês atrás?

Contexto relativo

Como as informações mudaram em relação a outros aspectos do sistema?

Contexto relacional

Há algo que dependa dessas informações? Essas informações dependem de algo mais?

Contexto proporcional

Quão ruim são essas informações? O escopo é grande ou pequeno?
Quem será impactado?

Tempo real

Você não pode esperar anos por essas informações. Você precisa delas agora. Sua definição de “agora”, é claro, pode variar um pouco, mas, no contexto de seus sistemas, você precisará de informações com uma rapidez suficiente para ter a chance de identificar um problema antes que o usuário perceba ou, no mínimo, precisará ter as informações à sua disposição quando alguém reclamar. Na prática, estamos falando de segundos, e não de minutos ou horas.

Apropriados para a sua escala

Boa parte do trabalho na área de observabilidade de sistemas distribuídos foi inspirado em trabalhos feitos para sistemas distribuídos de larga escala. Infelizmente isso pode nos levar a tentar recriar soluções para sistemas de escala muito maior que a nossa, sem entender o custo-benefício.

Sistemas com escala gigantesca muitas vezes terão de fazer negociações específicas de custo-benefício para reduzir as funcionalidades de seus sistemas a fim de lidar com a escala com a qual operam. O Dapper, por exemplo, teve de fazer uso de uma amostragem aleatória de dados extremamente agressiva (efetivamente “descartando” muitas informações) que fosse capaz de lidar com a escala do Google. Conforme diz Ben Sigelman, fundador da LightStep e criador do Dapper:¹⁴

Os microsserviços do Google geram aproximadamente 5 bilhões de RPCs por segundo; criar ferramentas de observabilidade que escalam a

5B RPCs/seg, portanto, se reduz a criar ferramentas de observabilidade que são extremamente pobres de recursos. Se sua empresa faz algo mais próximo de 5 milhões de RPCs/seg, esse número ainda será bastante impressionante, mas é quase certo que você não deverá utilizar o que o Google utiliza: em uma escala de 1/1000, será possível ter recursos muito mais eficazes.

O ideal é que você também queira uma ferramenta que *possa* escalar à medida que seu sistema escalar. Novamente, o custo poderá entrar em cena nesse caso. Mesmo que sua ferramenta preferida possa tecnicamente escalar de modo a atender ao crescimento esperado para o seu sistema, será viável que você continue pagando por ela?

Inteligência na máquina

Falei bastante sobre ferramentas neste capítulo, talvez mais do que em qualquer um dos outros capítulos do livro. Isso se deve parcialmente à mudança fundamental de ver o mundo exclusivamente com base na monitoração, e passando a pensar em como deixar nossos sistemas mais observáveis; essa mudança de comportamento exige ferramentas de apoio. Seria um erro, porém, ver essa mudança como algo exclusivamente relacionado a novas ferramentas, conforme espero já ter explicado. Apesar disso, com vários fornecedores diferentes competindo por nossa atenção, também precisamos ser cautelosos.

Por mais de uma década agora, tenho visto muitos fornecedores argumentando que seus sistemas têm certo grau de inteligência, permitindo que detectem problemas como num passe de mágica e nos informem exatamente o que precisaremos fazer para corrigi-los. Isso parece surgir em ondas, mas, com o recente alvoroço em torno do ML (Machine Learning, ou Aprendizado de Máquina) e da IA (Inteligência Artificial) aumentando cada vez mais, vejo mais produtos que alegam fazerem uma detecção automática de anomalias. Tenho minhas dúvidas quanto ao nível de eficácia dessa tarefa quando é totalmente automatizada, e mesmo nesse caso, seria complicado supor que toda a inteligência necessária pudesse estar automatizada.

Boa parte das motivações relacionadas à IA sempre se relacionaram à tentativa de codificar um conhecimento especializado em um sistema automatizado. A ideia de que podemos automatizar a inteligência pode ser atraente para algumas pessoas, mas é também uma ideia possivelmente perigosa nessa área, pelo menos com o nosso conhecimento atual. Por que automatizar a inteligência? Para que você não precise investir em ter operadores especializados para operar o seu sistema. Não estou tentando levantar nenhuma questão a respeito de uma revolução no trabalho causada pelos avanços tecnológicos – estou querendo dizer que as pessoas estão vendendo essa ideia agora, e as empresas a estão comprando, na esperança de que esses problemas sejam totalmente solucionáveis (e passíveis de automação). A verdade é que, atualmente, não são.

Trabalhei recentemente com uma startup com foco em ciência de dados na Europa. A startup estava trabalhando com uma empresa que fornecia hardware para monitoração de leitos, capaz de coletar vários dados sobre um paciente. Os cientistas de dados ajudavam a ver padrões nos dados, mostrando a existência de conjuntos peculiares de pacientes que podiam ser determinados por meio de uma correlação entre diversas facetas dos dados. Os cientistas de dados poderiam dizer que “esses pacientes parecem estar relacionados”, mas não tinham nenhum conhecimento do significado desse relacionamento. Era necessário que um médico explicasse que alguns desses conjuntos se referiam a pacientes que, em geral, estavam mais doentes que outros. Era necessário ter certa expertise para identificar o conjunto, mas um tipo diferente de expertise para entender o que esse conjunto significava e colocar esse conhecimento em ação. Voltando às nossas ferramentas para monitoração e observabilidade, eu poderia ver uma ferramenta como essa alertando alguém sobre o fato de que “algo parece estranho”, mas saber o que fazer com essa informação ainda exige certo grau de expertise.

Embora eu tenha certeza de que recursos como “detecção automática de anomalias” possam muito bem continuar a ser aprimorados, temos de reconhecer que, no momento, o especialista no sistema é, e continuará sendo por um tempo, um ser humano. Podemos criar ferramentas capazes de informar melhor o operador acerca do que precisa ser feito, e prover

automação para ajudar o operador a colocar suas decisões em prática de modo mais eficiente. Contudo, o ambiente fundamentalmente variado e complexo de um sistema distribuído implica que precisamos do auxílio de operadores humanos e especializados. Queremos que nossos especialistas utilizem sua expertise para fazer as perguntas certas e tomar as melhores decisões que puderem. Não devemos pedir que utilizem sua expertise para lidar com as deficiências de ferramentas precárias. Nem devemos sucumbir à conveniente noção de que alguma nova ferramenta da moda resolverá todos os nossos problemas.

Por onde começar

Conforme apresentei, há muitos aspectos em que devemos pensar. No entanto, gostaria de apresentar um ponto de partida básico para uma arquitetura de microsserviços simples, no que diz respeito a quais informações você deve capturar, e como fazê-lo.

Para começar, você deve ser capaz de capturar informações básicas sobre os hosts nos quais seus microsserviços estão executando – taxa de CPU, E/S, e assim por diante – e garantir que possa fazer a correspondência entre uma instância de microsserviço e o host no qual esse serviço está executando. Para cada instância de microsserviço, capture os tempos de resposta das suas interfaces de serviço e registre todas as chamadas downstream em logs. Inclua IDs de correlação em seus logs desde o princípio. Registre outros passos importantes de seus processos de negócios no log. Isso exigirá que você tenha no mínimo uma métrica básica e um conjunto de ferramentas para agregação de logs instalado.

Eu hesitaria em dizer que você precisa começar com uma ferramenta dedicada de tracing distribuído. Se for necessário executar e hospedar a ferramenta por conta própria, uma complexidade significativa poderá ser acrescentada. Por outro lado, se puder utilizar facilmente um serviço totalmente gerenciado, instrumentar seus microsserviços desde o princípio pode fazer bastante sentido.

Para operações essenciais, considere seriamente a criação de transações sintéticas como uma forma de entender melhor se os aspectos vitais de seu

sistema estão funcionando de forma apropriada. Construa seu sistema com essa capacidade em mente.

Tudo isso faz parte apenas da coleta básica de informações. Acima de tudo, é preciso garantir que será possível filtrar essas informações para fazer perguntas sobre o seu sistema em execução. Você é capaz de dizer com confiança que o sistema está funcionando de modo adequado aos seus usuários? Com o tempo, será necessário coletar mais informações e aprimorar suas ferramentas (e o modo de usá-las) para melhorar a observabilidade de sua plataforma.

Resumo

Sistemas distribuídos podem ser complicados de entender, e quanto mais distribuídos forem, mais difícil será a tarefa de resolver problemas no ambiente de produção. Quando houver pressão, alertas estiverem disparando e os clientes estiverem gritando, é importante que você tenha as informações corretas à disposição para descobrir o que, afinal de contas, está acontecendo, e o que será precisa fazer para corrigir o problema.

À medida que sua arquitetura de microsserviços se torna mais complexa, será mais difícil saber com antecedência os problemas que poderão ocorrer. Com frequência, você será surpreendido com os tipos de problemas que enfrentará. Desse modo, será essencial mudar sua maneira de pensar, afastando-se das atividades que, em sua maior parte, são de monitoração (passivas), e avançando em direção a deixar o seu sistema mais observável. Isso envolve não só provavelmente mudar o seu conjunto de ferramentas, mas também se afastar de painéis de controle estáticos, passando para atividades mais dinâmicas de exploração dos dados.

Em um sistema simples, o básico já ajudará bastante. Implemente uma solução de agregação de logs desde o princípio, e tenha IDs de correlação em suas linhas de log também. O tracing distribuído pode vir depois, mas fique atento para quando chegar a hora de implantar essa ferramenta.

Mude o seu entendimento acerca da saúde de seu sistema ou dos seus microsserviços, afastando-se de um estado binário do tipo “satisfatório” ou “insatisfatório”; em vez disso, perceba que a verdade sempre tem outros

tipos de nuances. Não deixe que cada pequeno problema gere um alerta e pense de forma mais holística no que é aceitável. Considere fortemente adotar SLOs e alertas com bases nesses preceitos a fim de reduzir a fadiga de alertas (alert fatigue) e manter sua atenção com um foco apropriado.

Acima de tudo, trata-se de aceitar que nem tudo será conhecido antes de o sistema chegar ao ambiente de produção. Melhore sua capacidade de lidar com o desconhecido.

Discutimos vários assuntos, mas há outros aspectos a serem explorados. Se quiser conhecer os conceitos de observabilidade com mais detalhes, recomendo o livro *Observability Engineering*, de Charity Majors, Liz Fong-Jones e George Miranda.¹⁵ Também recomendo os livros *Site Reliability Engineering*¹⁶ e *The Site Reliability Workbook*¹⁷ como bons pontos de partida para uma discussão mais ampla sobre SLOs, SLIs e itens desse tipo. Vale a pena notar que esses dois últimos livros foram escritos de um ponto de vista bastante relacionado ao modo como o trabalho é (ou era) feito no Google, e isso significa que esses conceitos nem sempre serão traduzíveis. É provável que você não seja o Google, e provavelmente não tem problemas do mesmo porte que o Google. Apesar disso, há muitas informações que são recomendáveis nesses livros.

No próximo capítulo, apresentaremos uma visão diferente, ainda que continue sendo holística, de nossos sistemas, e consideraremos algumas das vantagens exclusivas – e desafios – que as arquiteturas compostas de várias partes menores e mais especializadas podem proporcionar na área de segurança.

¹ Honestly Black Lives Matter (@honest_update), 7 de outubro de 2015, 19h10min., <https://oreil.ly/Z28BA>.

² Um incêndio como causa de falha em um sistema não é uma ideia totalmente despropositada. Certa vez, ajudei no trabalho que se sucedeu a uma falha em produção provocada por um incêndio na SAN (Storage Area Network, ou Rede de Área de Armazenamento). O fato de terem se passado vários dias para que fôssemos informados que um incêndio havia ocorrido é outra história.

³ Eu não disse que estava sendo bem-sucedido.

⁴ Leslie Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System” (Tempo, relógios e a ordem dos eventos em um sistema distribuído), *Communications of the ACM* 21, n. 7 (julho de 1978): 558–65, <https://oreil.ly/qzYmh>.

- 5 Veja a análise que Kyle Kingsbury fez sobre o Elasticsearch 1.1.0 em “Jepsen: Elasticsearch”, <https://oreil.ly/uO9wU> e sobre o Elasticsearch 1.5.0 em “Jepsen: Elasticsearch 1.5.0”, <https://oreil.ly/8fBCt>.
- 6 Renato Losio, “Elastic Changes Licences for Elasticsearch and Kibana: AWS Forks Both” (Elastic muda licença do Elasticsearch e do Kibana: AWS cria forks para ambos), InfoQ, 25 de janeiro de 2021, <https://oreil.ly/VdWzD>.
- 7 Charity Majors, “Metrics: Not the Observability Droids You’re Looking For” (Métricas: não são os andróides de observabilidade que você está procurando), Honeycomb (blog), 24 de outubro de 2017, <https://oreil.ly/TEETp>.
- 8 Em boa medida. A AWS também disponibiliza instâncias bare metal (físicas), que confunde um pouco o meu cérebro.
- 9 Comissão do presidente dos Estados Unidos sobre o acidente em Three Mile Island, *The Need for Change, the Legacy of TMI: Report of the President’s Commission on the Accident at Three Mile Island* (A necessidade de mudança, o legado de TMI: relatório da comissão do presidente sobre o acidente em Three Mile Island) (Washington, DC: The Commission, 1979).
- 10 National Transportation Safety Board, *Safety Recommendation Report: Assumptions Used in the Safety Assessment Process and the Effects of Multiple Alerts and Indications on Pilot Performance* (Relatório sobre recomendações de segurança: pressupostos usados no processo de avaliação de segurança e os efeitos de vários alertas e indicações no desempenho dos pilotos) (Washington, DC: NTSB, 2019).
- 11 Steven Shorrock, “Alarm Design: From Nuclear Power to WebOps” (Design de alarmes: da energia nuclear ao WebOps), Humanistic Systems (blog), 16 de outubro de 2015, <https://oreil.ly/RCHDL>.
- 12 Charity Majors (@mipsytipsy), Twitter, 7 de julho de 2019, 9h48min, <https://oreil.ly/4VUAX>.
- 13 “Observability: A Complete Overview for 2021” (Observabilidade: uma visão geral completa em 2021), Lightstep, acessado em 16 de junho de 2021, <https://oreil.ly/a1ERu>.
- 14 Ben Sigelman, “Three Pillars with Zero Answers – Towards a New Scorecard for Observability” (Três pilares sem nenhuma resposta – em direção a uma nova tabela de desempenho para observabilidade), Lightstep (postagem de blog), 5 de dezembro de 2018, <https://oreil.ly/R3LwC>.
- 15 Charity Majors, Liz Fong-Jones e George Miranda, *Observability Engineering* (Sebastopol: O’Reilly, 2022). Estava em fase de lançamento preliminar quando escrevi este livro.
- 16 Betsy Beyer et al., eds., *Site Reliability Engineering: How Google Runs Production Systems* (Sebastopol: O’Reilly, 2016). [N.T.: Edição publicada no Brasil: *Engenharia de Confiabilidade do Google: como o Google administra seus sistemas de produção* (Novatec, 2016).]
- 17 Betsy Beyer et al., eds., *The Site Reliability Workbook* (Sebastopol: O’Reilly, 2018).

Segurança

Gostaria de prefaciar este capítulo dizendo que eu não me considero um expert na área de segurança de aplicações. Meu objetivo é apenas ser um *incompetente consciente* – em outras palavras, quero saber o que eu não sei e estar ciente dos meus limites. Mesmo que eu aprenda mais a respeito dessa área, percebo que ainda há mais para saber. Isso não quer dizer que se educar a respeito de assuntos como esse não faça sentido – acho que tudo que aprendi sobre essa área nos últimos dez anos me fez ser um desenvolvedor e um arquiteto mais eficaz.

Neste capítulo, destacarei os aspectos da segurança que considero que valem a pena ser conhecidos por um desenvolvedor, um arquiteto ou um operador em geral que trabalhe com uma arquitetura de microsserviços. Ainda haverá a necessidade do auxílio de experts na área de segurança de aplicações – mas, ainda que você tenha acesso a essas pessoas, é importante ter um pouco de base sobre esses tópicos. Do mesmo modo que os desenvolvedores aprenderam mais sobre testes ou gerenciamento de dados – assuntos que eram antes restritos a especialistas – ter um conhecimento geral sobre assuntos de segurança pode ser essencial para que nossos softwares tenham medidas de segurança incluídas desde o princípio.

Ao comparar os microsserviços com arquiteturas menos distribuídas, o fato é que estaremos diante de uma dicotomia interessante. Por um lado, temos agora mais dados que fluem pelas redes, os quais permaneciam antes em uma só máquina, e temos uma infraestrutura mais complexa para executar nossa arquitetura – a área de nossa superfície de ataque é muito maior. Por outro, os microsserviços nos dão mais oportunidades de ter uma defesa em profundidade e limitar o escopo de acesso, podendo aumentar a área de

projeção de nosso sistema, ao mesmo tempo que reduz o impacto de um ataque caso ele ocorra. Esse paradoxo aparente – o fato de os microsserviços poderem deixar nossos sistemas igualmente *menos e mais* seguros – é, na verdade, apenas um aspecto delicado a ser ponderado. Minha esperança é que, no final deste capítulo, você acabe do lado certo dessa equação.

Para ajudar a encontrar o equilíbrio correto quando se trata da segurança de sua arquitetura de microsserviços, discutiremos os seguintes assuntos:

Princípios básicos

Conceitos fundamentais, cuja adoção é conveniente quando procuramos construir softwares mais seguros.

As cinco funções da cibersegurança

Identificar, proteger, detectar, responder e se recuperar – uma visão geral das cinco áreas funcionais essenciais para a segurança de aplicações.

Fundamentos da segurança de aplicações

Alguns conceitos fundamentais específicos da segurança de aplicações e como se aplicam aos microsserviços, incluindo credenciais e secrets (segredos), patching, backups e reconstrução (rebuild).

Confiança implícita versus confiança zero

Diferentes abordagens para confiança em nosso ambiente de microsserviços e como isso causa impacto nas atividades relacionadas à segurança.

Proteção de dados

Como protegemos os dados à medida que trafegam pelas redes ou permanecem em disco.

Autenticação e autorização

Como o SSO (Single Sign-On, ou Login Único) funciona em uma arquitetura de microsserviços, modelos de autorização centralizada *versus* descentralizada e o papel dos tokens JWT como parte desse cenário.

Princípios básicos

Com frequência, quando o assunto sobre segurança de microsserviços vem à tona, as pessoas querem começar a falar de questões tecnológicas razoavelmente sofisticadas, como o uso de tokens JWT ou a necessidade de TLS mútuo (tópicos que exploraremos mais adiante neste capítulo). No entanto, o problema com a segurança é que você estará tão protegido quanto o seu aspecto menos seguro. Para utilizar uma analogia, se você pretende proteger sua casa, seria um erro concentrar todos os seus esforços em ter uma porta da frente que seja extremamente resistente, com luzes e câmeras para deter indivíduos mal-intencionados, se você vai deixar a porta dos fundos aberta.

Desse modo, há alguns aspectos fundamentais sobre a segurança de aplicações que devemos discutir, embora rapidamente, para ressaltar a variedade de problemas dos quais você deve estar ciente. Veremos como esses problemas essenciais se tornam mais (ou menos) complexos no contexto dos microsserviços, mas são aplicáveis de modo geral ao desenvolvimento de software como um todo. Para aqueles que querem se adiantar e passar para as “partes boas”, certifiquem-se de que não irão manter um foco excessivo em proteger a sua porta da frente, deixando a porta dos fundos aberta.

Princípio do privilégio mínimo

Ao conceder acesso às aplicações a indivíduos, sistemas externos ou internos, ou até mesmo para os nossos próprios microsserviços, devemos prestar muita atenção ao acesso concedido. O princípio do privilégio mínimo (principle of least privilege) descreve a ideia de que, ao conceder um acesso, devemos conceder o mínimo necessário a uma entidade para que ela execute a funcionalidade, e somente pelo tempo que precisar. A principal vantagem desse princípio é garantir que, se as credenciais forem comprometidas por um invasor, elas concederão o acesso mais limitado possível à parte mal-intencionada.

Se um microsserviço tiver acesso somente de leitura em um banco de dados, um invasor que tiver acesso às credenciais desse banco de dados terá somente acesso de leitura, e apenas para esse banco de dados. Se as

credenciais para o banco de dados expirarem antes de haver um comprometimento, essas credenciais serão inúteis. Esse conceito pode ser ampliado para limitar quais entidades específicas podem se comunicar com quais microsserviços.

Como veremos mais adiante neste capítulo, o princípio do privilégio mínimo pode ser estendido para garantir que controles de acesso também sejam concedidos somente por períodos limitados, restringindo mais ainda quaisquer consequências negativas no caso de haver algum comprometimento.

Defesa em profundidade

O Reino Unido, lugar em que moro, está cheio de castelos. Esses castelos são um lembrete parcial da história de nosso país (particularmente, um lembrete de uma época anterior ao Reino Unido ser, bem, unido, até certo ponto). Eles nos lembram de uma época em que as pessoas sentiam a necessidade de defender suas propriedades dos inimigos. Às vezes, os tipos de inimigos eram diferentes – muitos dos castelos próximos ao local em que moro em Kent foram projetados para se defender de invasões costeiras vindas da França.¹ Não importa o motivo, os castelos podem ser um ótimo exemplo do princípio da defesa em profundidade.

Ter um único método de proteção será um problema se um invasor encontrar uma maneira de burlar essa defesa, ou se o método de proteção oferece defesa apenas contra determinados tipos de invasores. Pense em um forte de defesa costeira, cuja única muralha esteja voltada para o mar, deixando-o totalmente indefeso contra uma invasão por terra. Se observarmos o Castelo de Dover, que é muito próximo do local em que moro, há vários tipos de meios de proteção fáceis de notar. Em primeiro lugar, o castelo está no alto de uma grande colina, fazendo com que um ataque ao castelo por terra seja difícil. O castelo não tem apenas uma muralha, mas duas – uma abertura na primeira ainda exigirá que um invasor lide com a segunda. E assim que você passar pela última muralha, há uma torre de proteção grande e imponente com a qual será necessário lidar.

O mesmo princípio deve se aplicar quando inserimos métodos de proteção para garantir a segurança de nossas aplicações. Ter vários métodos de proteção implantados para se defender dos invasores é essencial. Em arquiteturas de microsserviços, há muito mais lugares em que podemos proteger nossos sistemas. Ao dividir nossas funcionalidades em diferentes microsserviços e limitar o escopo daquilo que esses microsserviços podem fazer, já estamos aplicando uma defesa em profundidade. Também podemos executar os microsserviços em diferentes segmentos de rede, aplicar métodos de proteção baseados em rede em mais lugares, e até mesmo fazer uso de uma combinação de tecnologias para construir e executar esses microsserviços de modo que um único exploit zero-day (dia zero) não cause impactos em tudo que temos.

Os microsserviços dão mais oportunidades para uma defesa em profundidade, em comparação com as aplicações monolíticas equivalentes, com um único processo; como resultado, eles podem ajudar as empresas na criação de sistemas mais seguros.

Tipos de controles de segurança

Ao considerar os controles de segurança que podemos implantar para proteger nosso sistema, podemos classificá-los da seguinte maneira:²

Controles para prevenção

Impedem que um ataque ocorra. Incluem o armazenamento seguro de secrets (segredos), criptografia dos dados em repouso e em trânsito, e a implementação de métodos apropriados para autenticação e autorização.

Controles para detecção

Alertam para o fato de que um ataque está ocorrendo/ocorreu. Firewalls de aplicações e serviços de detecção de invasão são bons exemplos.

Controles para resposta

Ajudam a responder durante/após um ataque. Ter um sistema automatizado para reconstruir seu sistema, backups funcionais para recuperação de dados e planos de comunicação apropriados para serem colocados em prática logo após um incidente podem ser

essenciais.

Uma combinação de todos os três controles será necessária para proteger adequadamente um sistema, e você poderá ter vários controles de cada tipo. Retomando o nosso exemplo dos castelos, podemos ter várias muralhas, que representam diversos controles para prevenção. Poderíamos ter torres de vigilância e um sistema de iluminação para que pudéssemos perceber caso um ataque estivesse ocorrendo. Por fim, poderíamos ter alguns carpinteiros e pedreiros de prontidão caso precisássemos reforçar as portas ou as muralhas logo após um ataque. Obviamente, é improvável que você construa castelos como meio de vida, portanto veremos exemplos desses controles em uma arquitetura de microsserviços, mais adiante neste capítulo.

Automação

Um tema recorrente neste livro é a automação. Como temos muito mais partes constituintes nas arquiteturas de microsserviços, a automação passa a ser fundamental para nos ajudar a administrar a crescente complexidade de nosso sistema. Ao mesmo tempo, temos uma orientação em direção ao aumento na rapidez das entregas, e a automação é essencial nesse caso. Os computadores são muito melhores que os seres humanos para executar as mesmas tarefas repetidamente – eles fazem isso mais rápido e com mais eficiência do que poderíamos fazer (e com menos variabilidade também). Além disso, os computadores podem reduzir os erros humanos e facilitam implementar o princípio do privilégio mínimo – podemos atribuir privilégios específicos para scripts específicos, por exemplo.

Conforme veremos neste capítulo, a automação pode nos ajudar na recuperação logo após um incidente. Podemos usá-la para revogar e fazer uma rotação das chaves de segurança, além de utilizar ferramentas que ajudem a detectar possíveis problemas de segurança com mais facilidade. Assim como em outros aspectos da arquitetura de microsserviços, adotar uma cultura de automação ajudará bastante quando o assunto é a segurança.

Incluindo a segurança no processo de entregas

Como em muitos outros aspectos da entrega de software, com muita frequência, a segurança é considerada *a posteriori*. Historicamente pelo menos, abordar aspectos da segurança de um sistema é algo feito depois que o código foi escrito, possivelmente resultando em retrabalhos significativos mais tarde. Em geral, a segurança é vista como uma espécie de obstáculo que atrapalha o lançamento de um software.

Nos últimos vinte anos, temos visto problemas parecidos com testes, usabilidade e operações. Esses aspectos da entrega de software eram frequentemente tratados de forma isolada, muitas vezes depois que a maior parte do código havia sido concluída. Certa vez, um antigo colega meu, Jonny Schneider, fez uma comparação da abordagem empregada na usabilidade de software com uma mentalidade do tipo “Você quer batatas fritas para acompanhar?”. Em outras palavras, a usabilidade é pensada *a posteriori* – algo que é colocado sobre o “prato principal”.³

A verdade é que um software que não é usável, não é seguro, não funciona de modo apropriado no ambiente de produção e está repleto de bugs não é, de forma alguma, um “prato principal” – no melhor caso, é um produto com defeito. Temos melhorado para dar mais relevância aos testes no processo principal de entrega, como fizemos com os aspectos operacionais (alguém de DevOps?) e a usabilidade – com a segurança, não deveria ser diferente. Precisamos garantir que os desenvolvedores tenham mais conhecimento dos problemas relacionados à segurança em geral, os especialistas encontrem uma forma de se incluírem nas equipes de entrega quando forem necessários e as ferramentas sejam aperfeiçoadas de modo que nos permitam incluir uma mentalidade relacionada à segurança em nosso software.

Isso pode criar dificuldades para as empresas que adotam equipes alinhadas a fluxos (stream-aligned teams), com graus mais elevados de autonomia no que diz respeito à responsabilidade pelos seus microsserviços. Qual é o papel dos especialistas em segurança? Na seção “Equipes de capacitação”, na página 586, veremos como os especialistas, por exemplo, em segurança, podem trabalhar para auxiliar as equipes alinhadas a fluxo, e ajudar os responsáveis pelos microsserviços a incluírem

uma mentalidade mais voltada à segurança em seus softwares, além de garantir também que você tenha a expertise apropriada à sua disposição quando precisar.

Existem ferramentas automáticas capazes de fazer sondagens em nossos sistemas em busca de vulnerabilidades, por exemplo, procurando ataques de cross-site scripting. O Zed Attack Proxy (também conhecido como ZAP) é um bom exemplo. Com base no trabalho do OWASP, o ZAP tenta recriar ataques maliciosos ao seu site. Há outras ferramentas que utilizam análises estáticas para procurar erros comuns de programação, capazes de criar brechas de segurança, por exemplo, o Brakeman (<https://brakemanscanner.org>) para Ruby; há também ferramentas como o Snyk (<https://snyk.io>) que, entre outras tarefas, é capaz de identificar dependências de bibliotecas de terceiros que tenham vulnerabilidades conhecidas. Se essas ferramentas puderem ser facilmente integradas aos processos de construção (builds) usuais de CI, integrá-las aos seus check-ins padrões será um ótimo ponto de partida. É claro que vale a pena observar que muitos desses tipos de ferramentas atuam somente em problemas locais – por exemplo, uma vulnerabilidade em um código específico. Elas não substituem a necessidade de entender a segurança de seu sistema em um nível mais amplo e sistêmico.

As cinco funções da cibersegurança

Com esses princípios fundamentais em mente, vamos considerar agora as atividades abrangentes relacionadas à segurança, as quais devemos executar. Prosseguiremos então explicando como essas atividades variam no contexto de uma arquitetura de microsserviços. O modelo de minha preferência para descrever o universo da segurança de aplicações é o do NIST (National Institute of Standards and Technology, ou Instituto Nacional de Padrões e Tecnologia) dos Estados Unidos, que descreve um modelo conveniente de cinco partes (<https://oreil.ly/MSAuU>) para as diversas atividades envolvidas na cibersegurança:

- *Identificar* quem são seus possíveis invasores, quais são os alvos que tentam adquirir e quais são seus pontos de maior vulnerabilidade.

- *Proteger* seus bens mais importantes de hackers em potencial.
- *Detectar* se houve um ataque, apesar de seus melhores esforços.
- *Responder* quando você descobrir que algo ruim aconteceu.
- *Recuperar-se* logo após um incidente.

Acho esse modelo particularmente conveniente por causa de sua natureza holística. É muito fácil investir todos os seus esforços em proteger a sua aplicação sem antes considerar quais são as ameaças que você realmente enfrentará, ou nem mesmo determinar o que poderia ser feito caso um invasor inteligente consiga passar pelas suas defesas.

Vamos explorar cada uma dessas funções com um pouco mais de detalhes e ver como uma arquitetura de microsserviços pode mudar seu modo de abordar essas ideias, em comparação com uma arquitetura monolítica mais tradicional.

Identificar

Antes de determinar o que devemos proteger, precisamos descobrir quem poderia estar interessado em nossos bens, e o que, exatamente, eles estariam procurando. Em geral, é difícil assumirmos a mentalidade de um invasor, mas é exatamente isso que devemos fazer para garantir que nossos esforços estejam concentrados no lugar certo. A modelagem de ameaças (threat modeling) é o primeiro ponto a ser analisado ao considerar esse aspecto da segurança das aplicações.

Como seres humanos, somos péssimos em compreender riscos. Com frequência, fixamos nossa atenção nos pontos errados, ignorando os problemas mais importantes que possam estar fora do nosso campo de visão. Isso, é claro, se estende à área de segurança. Nossa compreensão quanto aos riscos de segurança aos quais podemos estar expostos muitas vezes é influenciada pela nossa visão limitada do sistema, pelas nossas habilidades e pelas nossas experiências.

Quando converso com desenvolvedores sobre riscos de segurança no contexto de uma arquitetura de microsserviços, eles imediatamente começam a falar de JWTs e TLS mútuo. Eles buscam soluções técnicas para problemas técnicos sobre os quais têm certa visibilidade. Não quero

apontar o dedo somente para os desenvolvedores – todos nós temos uma visão limitada do mundo. Retomando a analogia que utilizamos antes, é assim que poderíamos acabar com uma porta da frente extremamente segura e uma porta dos fundos totalmente aberta.

Em uma empresa na qual trabalhei, houve uma série de discussões sobre a necessidade de instalar câmeras de CCTV (Closed-Circuit TV, ou Circuito Fechado de TV) nas áreas de recepção dos escritórios da empresa em todo o mundo. Isso por causa de um incidente em que uma pessoa não autorizada havia conseguido acesso à área de recepção do escritório, e então à rede corporativa. Acreditava-se que um sistema de câmeras de CCTV não só impediria outras pessoas de tentarem o mesmo novamente, mas também ajudaria a identificar os indivíduos envolvidos após a ocorrência do fato.

O fantasma da vigilância corporativa desencadeou uma onda de angústia na empresa com preocupações do tipo “big brother”. Dependendo do lado que você estivesse na discussão, era uma questão de espiar os funcionários (se você fosse “pró” câmeras) ou de estar contente com o fato de os invasores poderem ter acesso aos prédios (se você fosse “anti” câmeras). Deixando de lado a natureza complicada de uma discussão polarizada como essa, ⁴ um funcionário se manifestou de uma maneira um tanto acanhada, sugerindo que talvez a discussão pudesse estar um pouco equivocada, pois alguns problemas mais sérios estavam sendo ignorados – as pessoas não pareciam preocupadas com o fato de a porta da frente de um dos escritórios principais estar com a fechadura quebrada e, durante anos, as pessoas chegavam pela manhã somente para deparar com a porta destrancada.

Essa história radical (porém verdadeira) é um ótimo exemplo de um problema comum que enfrentamos quando tentamos proteger nossos sistemas. Sem ter tempo para avaliar todos os fatores e entender quais são seus maiores riscos, você poderá acabar desconsiderando os lugares em que seu tempo será mais bem investido. O objetivo da modelagem de ameaças é ajudar a entender o que um invasor poderia querer de seu sistema. O que eles estão procurando? Há diferentes tipos de indivíduos

mal-intencionados que queiram ter acesso a diferentes bens? Em boa medida, a modelagem de ameaças, quando feita corretamente, diz respeito a assumir a mentalidade do invasor, pensando de fora para dentro. Essa visão de fora é importante, e é um dos motivos pelos quais ter alguém externo que auxilie na condução de um exercício de modelagem de ameaças pode ser muito conveniente.

A ideia principal da modelagem de ameaças não muda muito quando consideramos as arquiteturas de microsserviços, afora o fato de que qualquer arquitetura analisada agora poderá ser mais complexa. O que muda é como tomamos o resultado de um modelo de ameaças e o colocamos em ação. Um dos resultados de um exercício de modelagem de ameaças será uma lista de recomendações para os controles de segurança que devem ser implantados – esses controles podem incluir algo como uma mudança de processo, uma mudança de tecnologia, ou talvez uma modificação na arquitetura do sistema. Algumas dessas mudanças podem ser gerais e causar impacto em várias equipes e nos microsserviços associados. Outras podem resultar em um trabalho mais específico. Basicamente, porém, ao fazer uma modelagem de ameaças, é preciso ter uma visão holística – manter o foco dessa análise em um subconjunto pequeno demais de seu sistema – por exemplo, em um ou dois microsserviços – pode resultar em uma falsa sensação de segurança. Você poderia acabar investindo seu tempo na construção de uma porta da frente incrivelmente segura, somente para deixar a janela aberta.

Para mais detalhes sobre esse assunto, posso recomendar o livro *Threat Modeling: Designing for Security*⁵, de Adam Shostack.

Proteger

Assim que tivermos identificado nossos bens mais valiosos – e mais vulneráveis –, precisamos garantir que estejam devidamente protegidos. Conforme já mencionei, as arquiteturas de microsserviços sem dúvida apresentam uma superfície de ataque com uma área muito maior e, desse modo, há mais itens que podem precisar de proteção, mas também nos oferecem mais opções para uma defesa em profundidade. A maior parte

deste capítulo terá como foco os diversos aspectos relacionados à proteção, principalmente porque essa é a área em que as arquiteturas de microsserviços dão origem aos maiores desafios.

Detectar

Com uma arquitetura de microsserviços, detectar um incidente pode ser mais complexo. Temos mais redes para monitorar e mais máquinas para vigiar. As fontes de informações aumentam bastante, o que pode dificultar mais ainda a detecção dos problemas. Muitas das técnicas que exploramos no Capítulo 10, por exemplo, a agregação de logs, podem nos ajudar a coletar informações que ajudarão a detectar se algo ruim pode estar acontecendo. Além disso, há ferramentas especiais, como sistemas de detecção de invasão, que podem ser executadas para identificar um comportamento indevido. Os softwares para lidar com a complexidade cada vez maior de nossos sistemas estão melhorando, sobretudo na área de cargas de trabalho em contêineres, com ferramentas como o Aqua (<https://oreil.ly/OQn0O>).

Responder

Se o pior aconteceu e você descobriu, o que você deve fazer? Definir uma abordagem eficaz para resposta a incidentes é essencial para limitar os danos causados por uma violação. Ela geralmente tem início com o entendimento do escopo da violação e com o conhecimento dos dados que foram expostos. Se os dados expostos incluírem PII (Personally Identifiable Information, ou Informações de Identificação Pessoal), será necessário seguir os processos de resposta e notificação de incidentes tanto de segurança como de privacidade. Isso pode muito bem implicar que você terá de conversar com diferentes áreas de sua empresa e, em algumas situações, terá a obrigação legal de informar um agente de proteção de dados designado, caso determinados tipos de violação tenham ocorrido.

Várias empresas tiveram o impacto de uma violação acentuado em virtude de um tratamento indevido logo após o incidente, muitas vezes resultando em multas elevadas, além dos danos causados à marca e ao relacionamento com os clientes. Desse modo, é importante saber não só o que deve ser

feito por motivos legais e por questões de conformidade, mas também o que você deve fazer para cuidar dos usuários de seu software. O GDPR, por exemplo, exige que violações de dados pessoais sejam informadas às autoridades relevantes em um período de 72 horas – um prazo que não parece ser extremamente oneroso. Isso não significa que você não possa se esforçar para informar as pessoas com mais rapidez caso uma violação em seus dados tenha ocorrido.

Afora os aspectos relativos à comunicação externa da resposta, o modo de lidar internamente com a situação também é essencial. As empresas que têm uma cultura de acusações e medo provavelmente terão uma conduta ruim logo após a ocorrência de um incidente relevante. Lições não serão aprendidas, e os fatores que contribuíram para o incidente não serão revelados abertamente. Por outro lado, uma empresa com foco em abertura e segurança estará em uma posição melhor para aprender as lições que garantirão que incidentes similares tenham menos chances de acontecer. Retomaremos esse assunto na seção “Acusações”, na página [489](#).

Recuperar-se

A recuperação refere-se à nossa capacidade de deixar o sistema ativo novamente logo após um ataque, e à nossa capacidade de implementar aquilo que aprendemos a fim de garantir que haja menos chances de haver problemas novamente. Em uma arquitetura de microsserviços, há muito mais partes constituintes, o que pode tornar a recuperação mais complexa caso um problema cause um impacto amplo. Assim, mais adiante neste capítulo, veremos como algo simples como automação e backups pode ajudar você a reconstruir um sistema de microsserviços por demanda, deixando seu sistema pronto para executar o mais rápido possível.

Fundamentos da segurança de aplicações

Tudo bem, agora que já discutimos alguns dos princípios fundamentais e temos um pouco de noção a respeito do mundo amplo que as atividades de segurança podem englobar, vamos discutir alguns assuntos básicos de segurança no contexto de uma arquitetura de microsserviços se você deseja

criar um sistema mais seguro – credenciais, patching, backups e reconstrução (rebuild).

Credenciais

Falando de modo geral, as credenciais dão acesso a uma pessoa (ou computador) a alguma forma de recurso restrito. Poderia ser um banco de dados, um computador, uma conta de usuário ou outro recurso. Em uma arquitetura de microsserviços, no que diz respeito ao modo como ela se compara a uma arquitetura monolítica equivalente, provavelmente teremos o mesmo número de pessoas envolvidas, porém haverá *muito* mais credenciais no cenário, representando diferentes microsserviços, máquinas (virtuais), bancos de dados e itens desse tipo. Isso pode gerar certo grau de confusão acerca de como restringir (ou não) o acesso e, em muitos casos, pode levar a uma abordagem “preguiçosa”, em que poucas credenciais com amplos privilégios são usadas em uma tentativa de simplificar o problema. Por sua vez, essa solução pode resultar em mais problemas caso as credenciais sejam comprometidas.

Podemos dividir o assunto sobre credenciais em duas áreas essenciais. Em primeiro lugar, temos as credenciais dos usuários (e operadores) de nosso sistema. Em geral, são o ponto mais fraco de nosso sistema, e são comumente usadas como um vetor de ataque por indivíduos mal-intencionados, conforme veremos em breve. Em segundo, podemos considerar os secrets (segredos) – informações essenciais à execução de nossos microsserviços. Para os dois conjuntos de credenciais, devemos considerar as questões relacionadas à rotatividade, revogação e limitação de escopo.

Credenciais de usuários

Credenciais de usuários, por exemplo, combinações de email e senha, continuam sendo essenciais para o modo como muitos de nós trabalhamos com nosso software, mas também são um ponto fraco em potencial quando se trata de nossos sistemas serem acessados por agentes mal-intencionados. O 2020 Data Breach Investigations Report (Relatório de Investigações sobre Violações de Dados de 2020, <https://oreil.ly/hqXfM>)

constatou que alguma forma de roubo de credenciais havia sido utilizada em 80% dos casos de hacking. Incluíam situações em que as credenciais haviam sido roubadas por métodos como ataques de phishing ou em que a força bruta havia sido empregada nas senhas.

Há alguns conselhos excelentes por aí acerca de como lidar com dados como senhas de modo apropriado – conselhos que, apesar de serem simples e claros para seguir, continuam não sendo adotados de forma suficientemente ampla. Troy Hunt apresenta uma visão geral excelente sobre os conselhos mais recentes tanto do NIST como do National Cyber Security Centre (Centro Nacional de Cibersegurança) do Reino Unido.⁶ Esses conselhos incluem recomendações para usar gerenciadores de senhas e senhas longas, evitar o uso de regras de senha complexas e – surpreendentemente, até certo ponto – evitar mudanças obrigatórias de senha com regularidade. Vale a pena ler a postagem completa de Troy com atenção.

Na era atual dos sistemas orientados a APIs, nossas credenciais também se estendem a gerenciar itens como chaves de API para sistemas de terceiros, por exemplo, de contas para o seu provedor de nuvem pública. Se um agente mal-intencionado tiver acesso à sua conta raiz da AWS, por exemplo, ele poderia tomar a decisão de destruir tudo que estiver executando nessa conta. Em um exemplo radical, por exemplo, um ataque desse tipo resultou no fim dos negócios de uma empresa chamada Code Spaces⁷ – todos os seus recursos estavam executando com uma única conta, incluindo backups e tudo mais. O fato irônico de que a Code Spaces oferecia “Hosting de Svn, hosting de Git e gerenciamento de projetos extremamente sólidos e seguros a preços acessíveis” não passou despercebido para mim.

Ainda que alguém conseguisse suas chaves de API para seu provedor de nuvem e não decidisse destruir tudo que você construiu, esse invasor poderia disparar algumas máquinas virtuais caras a fim de executar mineração de bitcoins, na esperança de que você não fosse perceber. Um problema desse tipo aconteceu com um de meus clientes, que descobriu que alguém havia gastado mais de 10K dólares fazendo exatamente isso,

antes de a conta ter sido desativada. O fato é que os invasores também sabem fazer uso de automação – há bots por aí que simplesmente farão um scan em busca de credenciais e tentarão usá-las para iniciar máquinas para mineração de criptomoedas.

Secrets

Falando de modo geral, os secrets (segredos) são informações essenciais que um microserviço precisa para funcionar, e são também suficientemente sigilosos a ponto de exigir proteção contra agentes mal-intencionados. Exemplos de secrets que um microserviço pode precisar incluem:

- certificados para TLS;
- chaves SSH;
- pares de chaves públicas/privadas para APIs;
- credenciais para acesso a bancos de dados.

Se consideramos o ciclo de vida de um secret, podemos começar diferenciando os diversos aspectos do gerenciamento de secrets que podem exigir diferentes requisitos de segurança:

Criação

Como o secret deve ser criado, para começar?

Distribuição

Assim que um secret for criado, como garantir que ele estará no lugar certo (e somente no lugar certo)?

Armazenamento

O secret é armazenado de um modo que garanta que somente as partes autorizadas possam acessá-lo?

Monitoração

Sabemos como esse secret é usado?

Rotatividade

Somos capazes de alterar o secret sem causar problemas?

Se tivermos vários microserviços, cada um exigindo diferentes conjuntos

de secrets, precisaremos de ferramentas que ajudem a gerenciar tudo isso.

O Kubernetes oferece uma solução pronta para secrets. É, de certa forma, uma solução limitada no que diz respeito às funcionalidades, mas está incluída como parte da instalação básica do Kubernetes, portanto poderia ser suficiente para muitos casos de uso.⁸

Se você está à procura de uma ferramenta mais sofisticada nessa área, vale a pena considerar o Vault (<https://www.vaultproject.io>) da Hashicorp. É uma ferramenta open source com opções comerciais à disposição, e um verdadeiro canivete suíço para gerenciamento de secrets, lidando com tudo, desde os aspectos básicos da distribuição de secrets até a geração de credenciais com tempo limitado para bancos de dados e plataformas de nuvem. O Vault tem a vantagem adicional de a ferramenta auxiliar consul-template (<https://oreil.ly/qNmAZ>) ser capaz de atualizar dinamicamente os secrets em um arquivo de configuração comum. Isso significa que partes de seu sistema que quiserem ler os secrets de um sistema de arquivos local não precisarão mudar para aceitar a ferramenta de gerenciamento de secrets. Quando um secret é alterado no Vault, o consul-template é capaz de atualizar essa entrada no arquivo de configuração, permitindo que seus microsserviços mudem dinamicamente os secrets que estiverem usando. Isso é ótimo para gerenciar credenciais em larga escala.

Alguns provedores de nuvem pública também oferecem soluções nessa área; por exemplo, o AWS Secrets Manager (<https://oreil.ly/cuwRX>) ou o Key Vault do Azure (<https://oreil.ly/rV3Sb>) vêm à mente. No entanto, certas pessoas não gostam da ideia de armazenar informações cruciais de secrets em um serviço de nuvem pública como esses. Novamente, tudo se reduzirá ao seu modelo de ameaças. Se essa for uma preocupação séria, não há nada que impeça você de executar o Vault no provedor de nuvem pública de sua preferência e lidar com esse sistema por conta própria. Ainda que os dados em repouso sejam armazenados no provedor de nuvem, com o backend de armazenamento apropriado, é possível garantir que os dados sejam criptografados de modo que, mesmo que um agente externo consiga pôr as mãos nos dados, ele não poderá fazer nada com eles.

Rotatividade

O ideal é que façamos uma rotação frequente das credenciais a fim de limitar os danos que alguém possa causar caso consiga acesso às credenciais. Se um indivíduo mal-intencionado tiver acesso ao par de chaves pública/privada da API da AWS, porém essas credenciais forem alteradas uma vez por semana, ele terá somente uma semana para fazer uso delas. Os invasores ainda poderão causar muitos danos em uma semana, é claro, mas deu para você ter uma ideia. Alguns tipos de invasores gostam de ter acesso aos sistemas e prosseguir sem que sejam detectados, o que lhes permite coletar mais dados importantes com o tempo e encontrar maneiras de acessar outras partes de seu sistema. Se eles utilizarem credenciais roubadas para conseguir acesso, talvez seja possível detê-los caso as credenciais empregadas expirem antes que eles possam fazer muito uso delas.

Um ótimo exemplo de rotatividade de credenciais de operadores seria a geração de chaves de API com tempo limitado para utilizar a AWS. Muitas empresas atualmente geram chaves de API para seus funcionários de forma dinâmica, com pares de chaves públicas e privadas sendo válidos somente por pouco tempo – geralmente, menos de uma hora. Isso permite que você gere as chaves de API de que precisará para executar qualquer que seja a operação necessária com segurança, sabendo que, mesmo que um agente mal-intencionado consiga depois um acesso a essas chaves, ele será incapaz de usá-las. Ainda que você faça check-in acidentalmente desse par de chaves no GitHub público, elas não teriam utilidade para ninguém depois que expirassem.

O uso de credenciais com tempo limitado pode ser conveniente para os sistemas também. O Vault da Hashicorp é capaz de gerar credenciais com tempo limitado para bancos de dados. Em vez de sua instância de microserviço ler os detalhes de conexão com um banco de dados a partir de um repositório de configuração ou de um arquivo-texto, ela poderá gerá-los dinamicamente, para uma instância específica de seu microserviço.

Passar para um processo de rotação frequente de credenciais, por exemplo, de chaves, pode ser complicado. Já conversei com empresas que tiveram

incidentes como consequência da rotação de chaves, nos quais os sistemas paravam de funcionar quando as chaves eram alteradas. Com frequência, isso se deve ao fato de não estar muito claro quem está utilizando uma credencial em particular. Se o escopo das credenciais for limitado, o impacto em potencial da rotatividade será significativamente reduzido. No entanto, se as credenciais tiverem um uso amplo, determinar o impacto de uma mudança pode ser difícil. Não digo isso para desestimular a rotatividade, mas somente para que você esteja ciente dos possíveis riscos, e continuo convencido de que é a atitude certa a ser tomada. O modo mais sensato de prosseguir com essa tarefa provavelmente seria adotando ferramentas que ajudem a automatizar esse processo e, ao mesmo tempo, limitando o escopo de cada conjunto de credenciais.

Revogação

Ter uma política definida para garantir que haja uma rotatividade frequente de credenciais pode ser uma forma sensata de limitar o impacto do vazamento de credenciais; no entanto, o que acontecerá se você *souber* que uma dada credencial caiu nas mãos erradas? Será preciso esperar até que a rotação agendada seja disparada para que essa credencial deixe de ser válida? Isso não seria prático – nem razoável. Em vez disso, o ideal é que você seja capaz de revogar automaticamente as credenciais e, possivelmente, gerá-las novamente quando algo assim acontecer.

O uso de ferramentas que permitam um gerenciamento centralizado de secrets pode ajudar nesse caso, mas pode exigir que seus microsserviços sejam capazes de reler os valores gerados novamente. Se seu microsserviço lê diretamente os secrets de algo como o repositório de secrets do Kubernetes ou do Vault, ele poderá ser notificado quando esses valores forem alterados, permitindo que seu microsserviço utilize os valores modificados. Como alternativa, se seu microsserviço ler esses secrets somente na inicialização, talvez seja necessário fazer um rolling restart (reinicialização sem interrupção) de seu sistema para recarregar essas credenciais. Entretanto, se houver uma rotação regular das credenciais, há uma boa chance de que você já precisará ter resolvido o problema de seus microsserviços serem capazes de reler essas informações. Se você se sente à

vontade com uma rotação frequente de suas credenciais, há uma boa chance de já estar preparado também para lidar com uma revogação de emergência.

Scanning de chaves

Fazer check-in acidentalmente de chaves privadas nos repositórios de códigos-fontes é uma forma comum de causar vazamento de credenciais para partes não autorizadas – isso ocorre com uma frequência surpreendente. O GitHub faz um scan automático dos repositórios em busca de alguns tipos de secrets, mas você também pode executar o seu próprio scanning. Seria ótimo se você conseguisse identificar os secrets antes de fazer check-in, e o `git-secrets` (<https://oreil.ly/Ra9ii>) permite fazer exatamente isso. Ele faz o scan de commits existentes em busca de possíveis secrets, mas, ao configurá-lo como um hook (gancho) de commit, ele poderá até mesmo impedir que os commits sejam feitos. Há também uma ferramenta similar, o `gitleaks` (<https://oreil.ly/z8xrf>), que, além de aceitar hooks prévios a um commit e fazer um scanning geral de commits, tem algumas funcionalidades que o tornam possivelmente mais útil como uma ferramenta genérica para scanning de arquivos locais.

Limitando o escopo

Limitar o escopo das credenciais é fundamental para a ideia de adotar o princípio do privilégio mínimo. Isso pode ser aplicado em todas as formas de credenciais, porém limitar o escopo ao qual um dado conjunto de credenciais concede acesso pode ser extremamente conveniente. Por exemplo, na Figura 11.1, cada instância do microserviço `Estoque` recebe o mesmo nome de usuário e senha para o banco de dados auxiliar. Também concedemos acesso somente de leitura para o processo auxiliar `Debezium` (<https://debezium.io>), que será usado para ler dados e enviá-los por meio do Kafka, como parte do processo de ETL existente. Se o nome do usuário e a senha dos microserviços forem comprometidos, um agente externo poderia, teoricamente, ter acesso de leitura e escrita no banco de dados. Se um invasor tiver acesso às credenciais do `Debezium`, porém, o acesso seria somente de leitura.

A limitação de escopo pode ser aplicada tanto no que diz respeito àquilo que o conjunto de credenciais pode acessar como também a quem tem acesso a esse conjunto de credenciais. Na Figura 11.2, fizemos uma alteração de modo que cada instância de *Estoque* obtenha um conjunto diferente de credenciais. Isso significa que poderíamos fazer a rotação de cada credencial de modo independente, ou simplesmente revogá-la para uma das instâncias caso ela venha a ser comprometida. Além do mais, com credenciais mais específicas, seria mais fácil descobrir de onde e como as credenciais foram obtidas. Obviamente, há outras vantagens que resultam de ter um nome de usuário unicamente identificável para uma instância de microsserviço nesse caso; seria mais fácil rastrear qual instância executou uma query custosa, por exemplo.

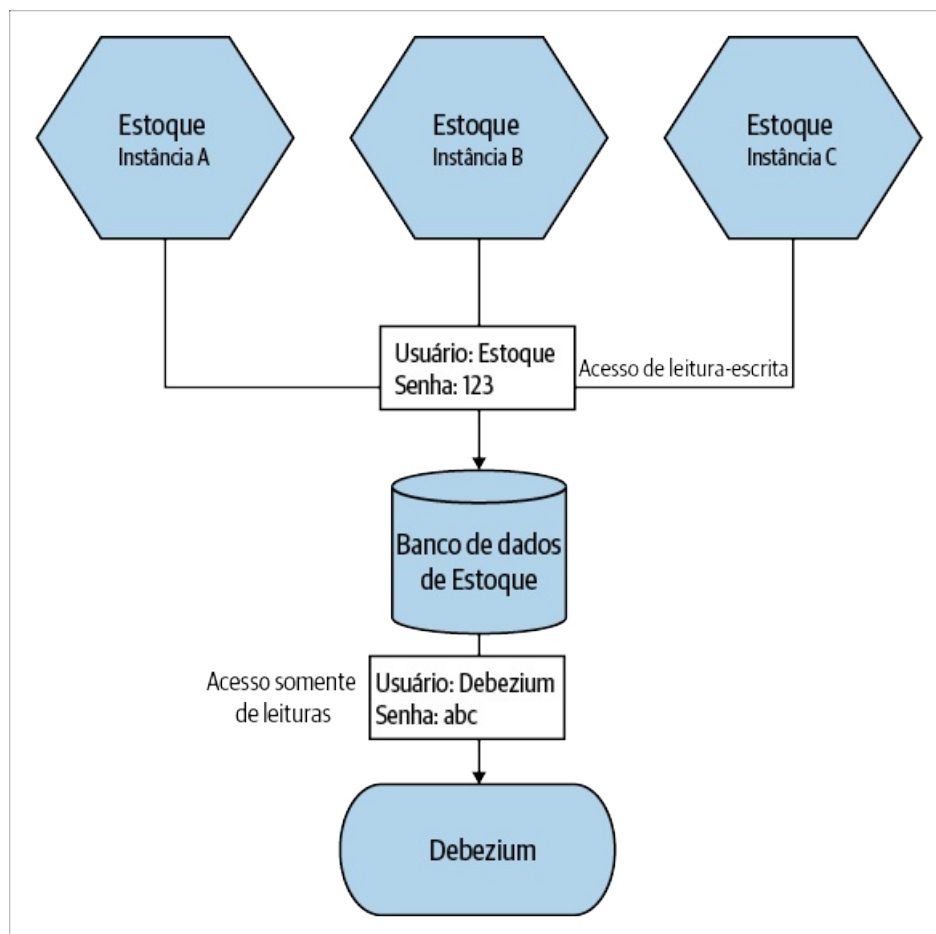


Figura 11.1 – Limitando o escopo das credenciais a fim de limitar o impacto de um uso indevido.

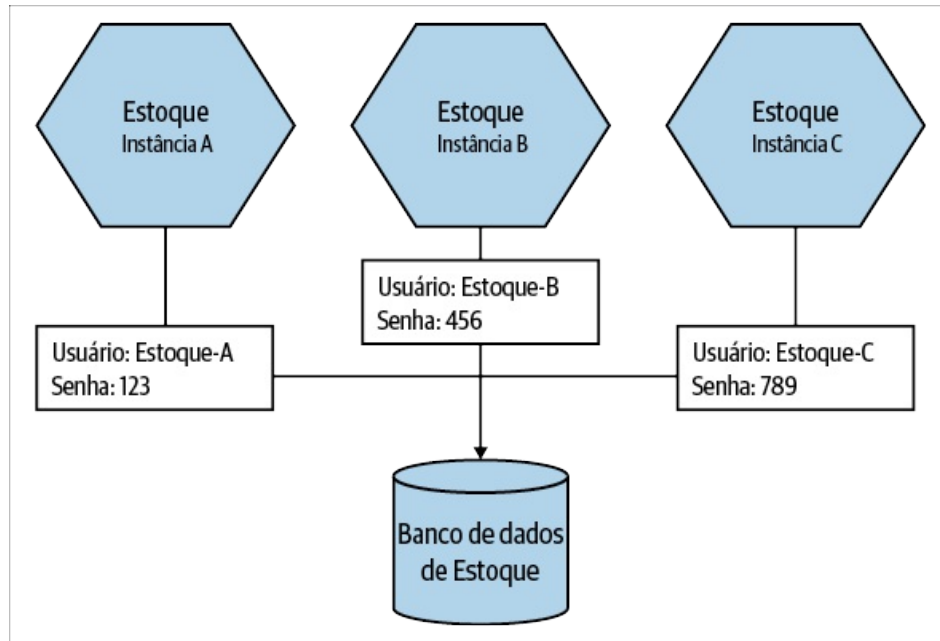


Figura 11.2 – Cada instância de Estoque tem suas próprias credenciais de acesso para o banco de dados, limitando melhor o acesso.

Como já discutimos, gerenciar credenciais mais específicas em larga escala pode ser complexo; se você realmente quer adotar uma abordagem desse tipo, alguma forma de automação será essencial – sistemas de armazenamento de secrets como o Vault vêm à mente como soluções perfeitas para implementar esquemas como esses.

Patching

A violação de dados da Equifax em 2017 é um ótimo exemplo da importância do patching. Uma vulnerabilidade conhecida no Apache Struts foi usada para conseguir um acesso não autorizado aos dados da Equifax. Como a Equifax é uma agência de crédito, essas informações eram particularmente sigilosas. No final, descobriu-se que os dados de mais de 160 milhões de pessoas haviam sido comprometidos na violação. A Equifax acabou tendo de fazer um acordo que lhe custou 700 milhões de dólares.

Meses antes da violação, a vulnerabilidade no Apache Struts havia sido identificada, e uma nova versão havia sido lançada pelos mantenedores, corrigindo o problema. Infelizmente, a Equifax não havia feito a

atualização para a nova versão do software, apesar de ela estar disponível meses antes do ataque. Se a Equifax tivesse feito a atualização desse software rapidamente, é provável que o ataque não tivesse sido possível.

O problema de estar atualizado quanto ao patching está se tornando cada vez mais complicado à medida que implantamos sistemas mais complexos. Precisamos nos aperfeiçoar mais no que concerne ao modo de lidar com esse conceito razoavelmente básico.

A Figura 11.3 mostra um exemplo das camadas da infraestrutura e de software subjacentes a um típico cluster Kubernetes. Se você executar toda essa infraestrutura por conta própria, você será o responsável por gerenciar e fazer o patching de todas essas camadas. Até que ponto você se sente confiante de que está atualizado quanto ao seu patching? Obviamente, se for possível passar parte desse trabalho para um provedor de nuvem pública, será possível se livrar de parte desse fardo também.

Se você pretende utilizar um cluster Kubernetes gerenciado em um dos principais provedores de nuvem pública, por exemplo, o escopo de sua responsabilidade seria drasticamente reduzido, conforme vemos na Figura 11.4.

Os contêineres lançam uma bola em curva interessante para nós nesse caso. Tratamos uma dada instância de contêiner como imutável. Entretanto, um contêiner inclui não só o nosso software, mas também um sistema operacional. E você sabe de onde veio esse contêiner? Os contêineres têm como base uma imagem, a qual, por sua vez, pode estender outras imagens – você tem certeza de que as imagens de base que está usando já não incluem backdoors (portas dos fundos)? Se você não alterou uma instância de contêiner por seis meses, são seis meses de patches do sistema operacional que não foram aplicados. Estar atento a isso é complicado, e é por isso que empresas como a Aqua (<https://www.aquasec.com>) oferecem ferramentas para ajudar a analisar seus contêineres em execução no ambiente de produção, para que você possa saber quais são os problemas que precisam ser resolvidos.

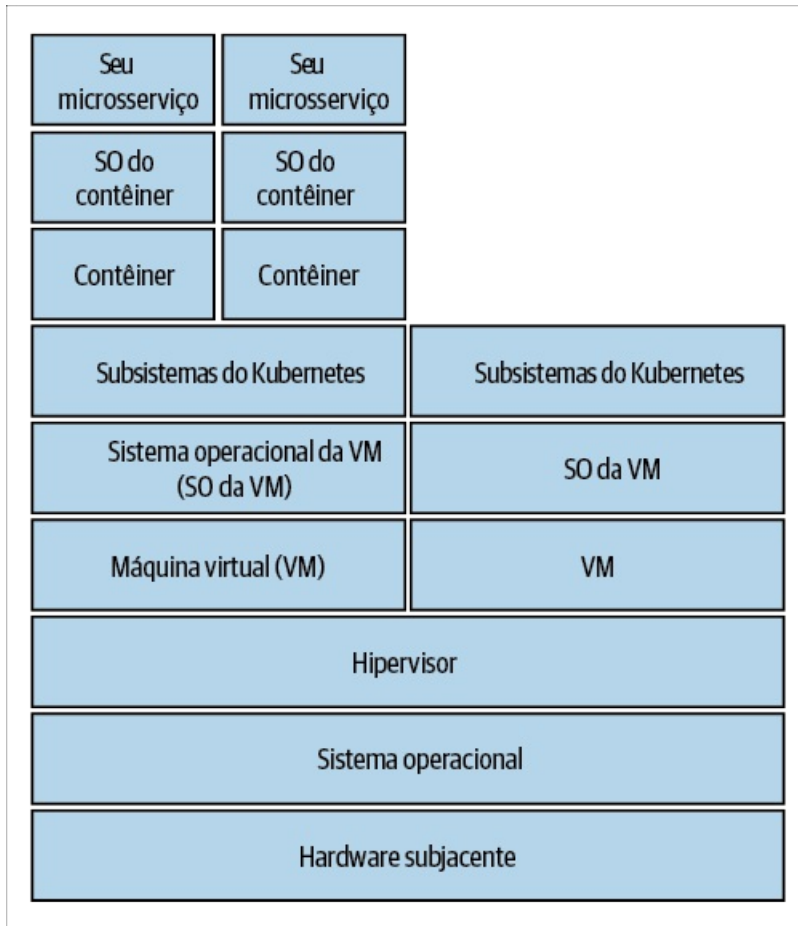


Figura 11.3 – Todas as diferentes camadas de uma infraestrutura moderna que exigem manutenção e patching.

No topo desse conjunto de camadas, é claro, está o código de nossa aplicação. Ele está atualizado? Não se trata apenas do código que *nós* escrevemos; e quanto aos códigos de terceiros que utilizamos? Um bug em uma biblioteca de terceiros pode deixar nossa aplicação vulnerável a ataques. No caso da violação na Equifax, a vulnerabilidade resultante da ausência de patch, na verdade, estava no Struts – um framework Java para web.

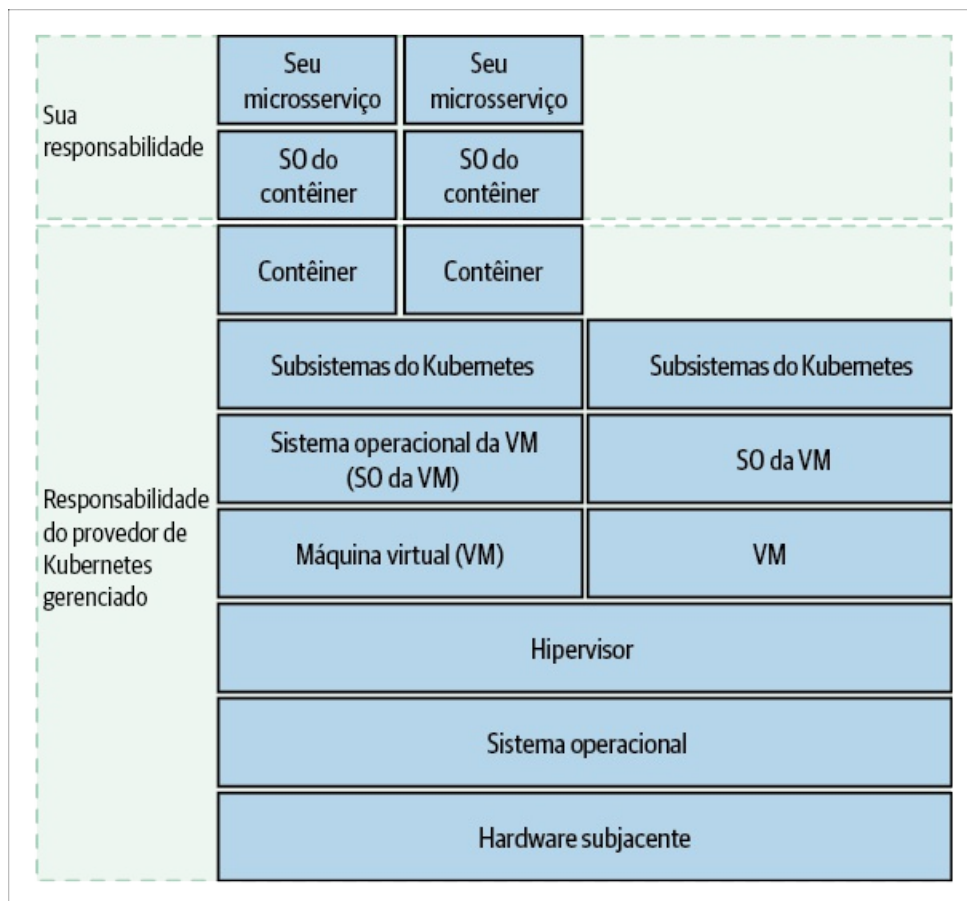


Figura 11.4 – Passar a responsabilidade por algumas camadas dessa pilha para outros permite uma redução na complexidade.

Em larga escala, descobrir quais microserviços estão ligados a bibliotecas com vulnerabilidades conhecidas pode ser extremamente difícil. Essa é uma área na qual recomendo fortemente o uso de ferramentas como o Snyk ou o GitHub code scanning, capaz de fazer um scan automaticamente de suas dependências de terceiros e gerar alertas caso haja alguma ligação com bibliotecas que tenham vulnerabilidades conhecidas. Se uma dessas ligações for encontrada, a ferramenta pode enviar um pull request para ajudar na atualização para as versões mais recentes, que incluam patches. Você pode até mesmo incluir essa verificação em seu processo de CI, fazendo com que a construção de um microserviço falhe caso haja ligações com bibliotecas que tenham problemas.

Backups

Às vezes, acho que fazer backups é como passar fio dental: há muito mais pessoas que dizem que fazem do que pessoas que realmente o fazem. Não acho que seja necessário insistir muito na importância dos backups, além de dizer que você deve fazer backups, pois os dados são valiosos e você não vai querer perdê-los.

Os dados nunca foram tão importantes quanto agora, mas, apesar disso, às vezes me pergunto se as melhorias nas tecnologias fizeram com que deixássemos de priorizar os backups. Os discos estão mais confiáveis do que costumavam ser. É mais provável que os bancos de dados tenham replicação incluída a fim de evitar uma perda de dados. Com sistemas como esses, podemos nos convencer de que não precisamos mais de backups. Mas, e se um erro catastrófico ocorrer e seu cluster do Cassandra for destruído por completo? Ou se um bug de programação fizer com que sua aplicação apague dados importantes? Os backups continuam sendo tão importantes quanto sempre foram. Então, por favor, faça backup dos seus dados cruciais.

Com a implantação de nossos microsserviços sendo automatizada, não precisamos fazer backups completos das máquinas, pois podemos reconstruir nossa infraestrutura a partir de um código-fonte. Desse modo, não tentamos copiar o estado de máquinas inteiras; nossos backups terão como alvo os estados que são mais importantes. Isso significa que nosso foco ao fazer backups estará limitado a itens como os dados em nossos bancos de dados, ou talvez os logs de nossas aplicações. Com a tecnologia correta para sistema de arquivos, é possível obter clones quase instantâneos, no nível de blocos, dos dados de um banco de dados, sem que haja uma interrupção de serviço perceptível.



Evitando o backup de Schrödinger

Ao criar backups, devemos evitar o que eu chamo de backups de Schrödinger.² É um backup que pode ou não ser realmente um backup. Até tentar restaurá-lo de verdade, você não saberá se o backup é realmente utilizável¹⁰, ou se é somente um punhado de 1s e 0s gravados em disco. A melhor maneira de evitar esse problema é garantir que o backup é real, fazendo a sua restauração. Encontre

maneiras de incluir restaurações regulares dos backups em seu processo de desenvolvimento de software – por exemplo, utilizando backups de produção para criar os dados para seus testes de desempenho.

A “antiga” diretriz sobre backups determinava que eles deveriam ser mantidos fora das instalações da empresa, com base na ideia de que um incidente em seus escritórios ou no data center não afetaria seus backups se eles estivessem em outro lugar. Mas o que significa “fora das instalações da empresa” se sua aplicação estiver implantada em uma nuvem pública? O importante é que seus backups estejam armazenados do modo mais isolado possível de seus sistemas básicos, de modo que, se um sistema básico for comprometido, isso não coloque seus backups em risco também. A Code Spaces, que mencionamos antes, tinha backups – no entanto, eles estavam armazenados na AWS, na mesma conta que havia sido comprometida. Se sua aplicação executa na AWS, você poderia armazenar seus backups aí também, mas deve fazê-lo em uma conta separada, com recursos de nuvem distintos – você poderia até mesmo considerar colocá-los em uma região diferente da nuvem a fim de diminuir o risco de um problema que atinja toda uma região, ou ainda armazená-los junto a outro provedor.

Portanto, certifique-se de que fará um backup dos dados cruciais, mantenha esses backups em um sistema separado de seu ambiente de produção principal e garanta que os backups realmente funcionem, restaurando-os regularmente.

Reconstrução

Podemos fazer o melhor para garantir que um agente mal-intencionado não tenha acesso aos nossos sistemas, mas o que acontecerá se conseguir? Bem, em geral, a tarefa mais importante logo após um incidente é deixar o sistema em operação novamente, mas de um modo que o acesso da parte não autorizada tenha sido removido. Isso, porém, nem sempre é simples. Eu me lembro de uma de nossas máquinas ter sido hackeada vários anos atrás por um rootkit. Um rootkit é um software projetado para ocultar as atividades de um agente não autorizado, e é uma técnica comum, usada

por invasores que querem permanecer ocultos, dando-lhes tempo para explorar o sistema. Em nosso caso, percebemos que o rootkit havia alterado comandos básicos do sistema como `ls` (listagem de arquivos) ou `ps` (para exibir listas de processos) a fim de ocultar as pistas sobre o invasor externo. Notamos isso somente quando pudemos verificar os hashes dos programas que estavam executando na máquina, comparando-os com os pacotes oficiais. No final, tivemos basicamente de reinstalar todo o servidor do zero.

A capacidade de simplesmente limpar um servidor e reconstruí-lo totalmente pode ser extremamente eficaz, não só logo após um ataque conhecido, mas também para reduzir o impacto de invasores persistentes. Talvez você não perceba a presença de um agente mal-intencionado em seu sistema, mas, se reconstruir seus servidores com frequência e fizer uma rotação nas credenciais, poderá limitar drasticamente o impacto daquilo que ele poderá fazer sem que você perceba.

Sua capacidade de reconstruir um dado microsserviço, ou até mesmo um sistema inteiro, se resume à qualidade de sua automação e dos backups. Se puder implantar e configurar cada microsserviço do zero com base em informações armazenadas em um sistema de controle de versões, você já terá um bom ponto de partida. É claro que será preciso combinar isso com um processo bastante sólido de restauração de backups para os dados. No que concerne aos backups, a melhor maneira de garantir que uma implantação e configuração automáticas de seus microsserviços funcionam é fazê-las com frequência – e o modo mais fácil de fazer isso é simplesmente utilizar o mesmo processo empregado em cada implantação na reconstrução de um microsserviço. Obviamente, esse é o modo como a maioria dos processos de implantação baseados em contêineres funciona. Implantamos um novo conjunto de contêineres executando a nova versão de seu microsserviço e desativamos o conjunto antigo. Executar esse procedimento de operação rotineiro faz com que uma reconstrução seja quase isenta de ocorrências inesperadas.

Há uma ressalva, nesse caso, sobretudo se você estiver fazendo implantações em uma plataforma de contêineres como o Kubernetes. Você

estaria frequentemente destruindo e reimplantando instâncias de contêineres, mas e a plataforma subjacente de contêineres propriamente dita? Você é capaz de reconstruí-la do zero? Se você estiver usando um provedor de Kubernetes totalmente gerenciado, iniciar um novo cluster talvez não seja tão difícil, mas, se você instalou e gerencia o seu próprio cluster, essa tarefa pode representar um volume de trabalho que não é trivial.



Ser capaz de reconstruir seu microserviço e recuperar seus dados de modo automático ajudará você a se recuperar logo após um ataque, além de ter a vantagem de facilitar suas implantações em todos os aspectos, proporcionando vantagens positivas nas atividades de desenvolvimento, testes e operação no ambiente de produção.

Confiança implícita versus confiança zero

Nossa arquitetura de microserviços é composta de um grande volume de comunicação entre as partes. Usuários humanos interagem com o nosso sistema por meio de interfaces de usuário. Essas interfaces de usuário, por sua vez, fazem chamadas aos microserviços, e os microserviços acabam chamando outros microserviços. Quando se trata da segurança de aplicações, devemos considerar a questão da confiança entre todos esses pontos de contato. Como estabelecemos um nível aceitável de confiança? Exploraremos esse assunto em breve no que concerne à autenticação e autorização, tanto para seres humanos quanto para microserviços, mas, antes disso, vamos considerar alguns modelos básicos relacionados à segurança.

Confiamos em tudo que está executando em nossa rede? Ou encaramos tudo com suspeita? Podemos considerar dois tipos de mentalidade nesse caso: a mentalidade de confiança implícita (implicit trust) e a mentalidade de confiança zero (zero trust).

Confiança implícita

Nossa primeira opção poderia ser simplesmente supor que qualquer

chamada a um serviço, feita de dentro de nosso perímetro, seja implicitamente confiável.

Dependendo da confidencialidade dos dados, pode ser uma boa solução. Algumas empresas tentam garantir a segurança no perímetro de suas redes e, desse modo, supõem que não será necessário fazer mais nada quando dois serviços se comunicarem. No entanto, se houver uma invasão em sua rede, tudo pode acontecer. Se o invasor decidir interceptar e ler os dados sendo enviados, alterá-los sem que você saiba ou até mesmo, em algumas circunstâncias, fingir que é o componente com o qual você está conversando, talvez você não fique sabendo.

Essa é, de longe, a forma mais comum de confiança interna ao perímetro que vejo nas empresas. Não estou dizendo que seja algo bom! Na maioria das empresas que vejo utilizarem esse modelo, me preocupo com o fato de o modelo de confiança implícita não ser uma decisão consciente; em vez disso, as pessoas, antes de tudo, não estão cientes dos riscos.

Confiança zero

“Jill, rastreamos a chamada – está vindo de dentro da casa!”

*When a Stranger Calls*¹¹

Quando operamos em um ambiente de confiança zero, devemos *supor* que operamos em um ambiente que já foi comprometido – os computadores com as quais você conversa poderiam ter sido comprometidos, as conexões de entrada poderiam ser de agentes hostis, os dados que você escreve poderiam ser lidos por pessoas mal-intencionadas. Paranoia? Sim! Bem-vindo à confiança zero.

A confiança zero, basicamente, é um tipo de mentalidade. Não é algo que você possa implementar como se fosse mágica utilizando um produto ou uma ferramenta; é uma ideia, e é a ideia de que, se você opera com o pressuposto de que está em um ambiente hostil, no qual agentes mal-intencionados poderiam já estar presentes, então você deve adotar medidas de precaução para garantir que ainda poderá operar de forma segura. Com efeito, o conceito de “perímetro” não tem sentido em uma abordagem de confiança zero (por esse motivo, a abordagem de confiança zero muitas

vezes é conhecida também como “computação sem perímetro”).

Como partimos do pressuposto de que nosso sistema foi comprometido, todas as chamadas de entrada provenientes de outros microsserviços devem ser devidamente avaliadas. *Esse é realmente* um cliente no qual devo *confiar*? De modo semelhante, todos os dados devem ser armazenados de forma segura, e todas as chaves de criptografia devem estar protegidas; como devemos supor que há alguém escutando nossos dados, todos os dados sigilosos em trânsito dentro de nosso sistema devem ser criptografados.

O interessante é que, se você implementar uma mentalidade de confiança zero de forma apropriada, poderá começar a tomar atitudes que parecerão bastante incomuns:

[Com a confiança zero,] você poderá tomar certas decisões de acesso que são contraintuitivas, e, por exemplo, permitir conexões provenientes da internet com seus serviços internos, pois sua rede “interna” será tratada com o mesmo nível de confiança com que a internet é tratada (ou seja, nenhuma).

— Jan Schaumann¹²

O argumento de Jan, nesse caso, é que, se você supõe que nada dentro de sua rede é confiável, e que essa confiança deve ser reafirmada, você poderá ser muito mais flexível quanto ao ambiente no qual se encontra o seu microsserviço – não há expectativas de que o ambiente mais amplo seja seguro. Lembre-se, porém, de que a confiança zero não é algo que se ativa com uma chave. É um princípio subjacente ao modo de trabalhar. Ela deve direcionar sua tomada de decisões acerca de como seu sistema será criado e como evoluirá – será algo no qual você deverá investir constantemente para ter suas recompensas.

É um espectro

Não quero dar a entender que você tenha uma escolha bem distinta entre a confiança implícita e a confiança zero. Até que ponto você pode confiar (ou não) em outras partes de seu sistema pode variar de acordo com a confidencialidade das informações acessadas. Você poderia decidir, por

exemplo, adotar um conceito de confiança zero para qualquer microsserviço que lide com PII, mas ser menos rígido em outras áreas. Novamente, o custo de implementação de qualquer medida de segurança deve ser justificado (e orientado) pelo seu modelo de ameaças. Deixe que seu entendimento acerca das ameaças e o impacto associado orientem sua tomada de decisões para determinar se vale a pena adotar a abordagem de confiança zero.

Como exemplo, vamos analisar a MedicalCo, uma empresa com a qual trabalhei e que gerenciava dados sigilosos de saúde pertencentes a vários indivíduos. Todas as informações mantidas pela empresa eram classificadas com base em uma abordagem razoavelmente sensata e direta:

Dados públicos

Dados que podiam ser livremente compartilhados com qualquer parte externa. Essas informações eram efetivamente de domínio público.

Dados privados

Informações que deviam estar disponíveis somente para usuários logados. O acesso a essas informações poderia estar mais limitado em virtude de restrições na autorização. Esses dados podiam incluir informações como o plano de saúde do cliente.

Dados sigilosos

Informações extremamente sigilosas sobre os indivíduos, que podiam ser acessadas por pessoas que não fossem o indivíduo em questão somente em situações muito específicas. Incluía informações sobre os dados de saúde de um indivíduo.

Os microsserviços foram então classificados com base nos dados mais sensíveis que utilizassem, e tinham de executar em um ambiente (zona) correspondente, com os controles correspondentes, conforme vemos na Figura 11.5. Um microsserviço tinha de executar na zona correspondente aos dados mais sensíveis que utilizasse. Por exemplo, um microsserviço que executasse na zona pública poderia utilizar somente dados públicos. Por outro lado, um microsserviço que utilizasse dados públicos e privados tinha de executar na zona privada, enquanto um microsserviço que

acesse informações sigilosas sempre tinha de executar na zona sigilosa.

Os microserviços em cada zona podiam se comunicar entre si, mas não podiam fazer acessos diretos a dados ou funcionalidades das zonas inferiores, mais seguras. Os microserviços nas zonas mais seguras, porém, podiam acessar funcionalidades que executassem nas zonas menos seguras.

Nesse caso, a MedicalCo havia se dado a chance de ser mais flexível, podendo variar sua abordagem em cada zona. A zona pública, menos segura, podia operar de um modo mais parecido com um ambiente de confiança implícita, enquanto a zona sigilosa adotava a confiança zero. Sem dúvida, se a MedicalCo adotasse uma abordagem de confiança zero em todo o seu sistema, ter microserviços implantados em zonas separadas não teria sido necessário, pois todas as chamadas entre serviços exigiriam autenticação e autorização adicionais. Apesar disso, pensando mais uma vez na defesa em profundidade, não posso deixar de imaginar que eu ainda teria considerado essa abordagem com zonas, dada a confidencialidade dos dados!

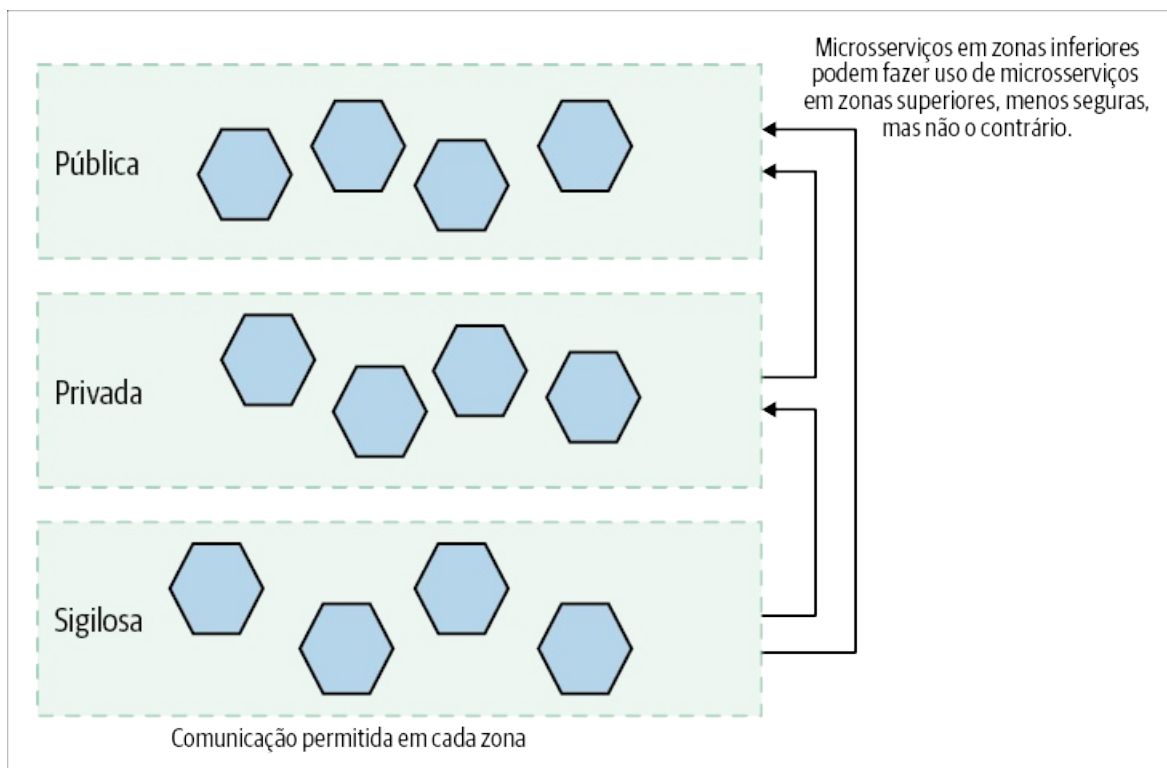


Figura 11.5 – Implantando microserviços em diferentes zonas de acordo com

a confidencialidade dos dados com os quais lidam.

Protegendo os dados

À medida que separamos nosso software monolítico em microsserviços, nossos dados passam a se mover pelos nossos sistemas com muito mais frequência. Eles não só fluem pelas redes como também permanecem em disco. Ter mais dados importantes espalhados em mais lugares pode ser um pesadelo quando se trata de proteger nossa aplicação, se não formos cuidadosos. Vamos analisar com mais detalhes como é possível proteger nossos dados à medida que se movem pelas redes, e enquanto permanecem em repouso.

Dados em trânsito

A natureza das proteções que tivermos dependerá, em boa medida, da natureza dos protocolos de comunicação que escolhermos. Se usarmos HTTP, por exemplo, seria natural considerar o uso de HTTP com TLS (Transport Layer Security, ou Segurança da Camada de Transporte) – um assunto que exploraremos melhor na próxima seção; contudo, se utilizarmos protocolos alternativos, como uma comunicação por meio de um broker de mensagens, talvez tenhamos de considerar o suporte oferecido por essa tecnologia em particular para proteger os dados em trânsito. Em vez de analisar os detalhes de diversas tecnologias nessa área, acho que seria mais importante considerar, de modo mais geral, as quatro principais áreas de interesse quando se trata de proteger dados em trânsito, e observar como esses problemas poderiam ser resolvidos usando o HTTP como exemplo. Espero que não seja muito difícil mapear essas ideias para qualquer que seja o protocolo de comunicação que você escolher.

Na Figura 11.6, podemos ver as quatro principais preocupações relacionadas aos dados em trânsito.

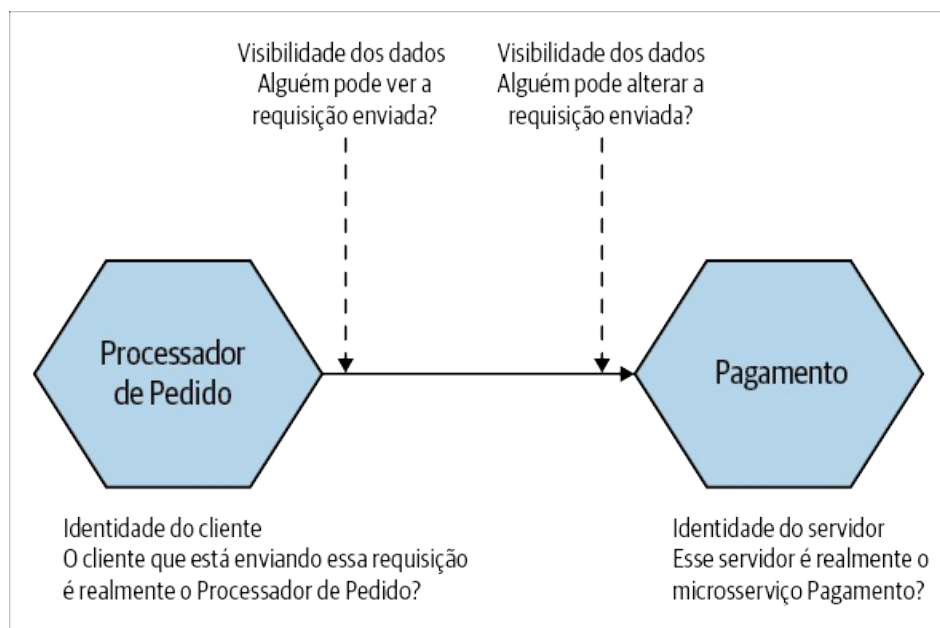


Figura 11.6 – Quatro principais preocupações quando se trata de dados em trânsito.

Vamos analisar cada uma dessas preocupações com um pouco mais de detalhes.

Identidade do servidor

Um dos pontos mais fáceis de verificar é se o servidor com o qual você está se comunicando é exatamente quem ele alega ser. Isso é importante porque um agente mal-intencionado teoricamente poderia personificar um endpoint – e extrair quaisquer dados úteis que você lhe enviar. Validar a identidade do servidor é uma preocupação antiga na internet pública, levando a uma pressão pelo uso mais amplo do HTTPS – e, até certo ponto, podemos nos beneficiar do trabalho feito para ajudar na proteção da internet pública quando se trata de gerenciar endpoints HTTPS internos.

Quando falam de “HTTPS”, em geral as pessoas se referem ao uso do HTTP com TLS.¹³ Na maior parte da comunicação na internet pública, em virtude dos diversos vetores de ataque possíveis por aí (Wi-Fi sem segurança, envenenamento de DNS [DNS poisoning] e vetores desse tipo), é essencial garantir que, quando acessamos um site, ele seja realmente o site que alega ser. Com o HTTPS, nosso navegador pode analisar o

certificado desse site e garantir que seja válido. É um método de segurança extremamente sensato – o “HTTPS Everywhere” se tornou um grito de guerra na internet pública, e por um bom motivo.

Vale a pena observar que alguns protocolos de comunicação que utilizam internamente o HTTP podem tirar proveito do HTTPS – assim, podemos facilmente executar SOAP ou gRPC com o HTTPS, sem que haja problemas. O HTTPS também nos fornece proteções adicionais muito além de simplesmente confirmar que estamos conversando com quem esperamos estar conversando. Voltaremos a esse assunto em breve.

Identidade do cliente

Quando nos referimos à identidade do cliente nesse contexto, estamos nos referindo ao microserviço que faz a chamada – portanto estamos tentando confirmar e autenticar a identidade do microserviço upstream. Veremos como autenticamos pessoas (usuários humanos!) mais adiante.

Podemos verificar a identidade de um cliente de diversas maneiras. Poderíamos pedir ao cliente que envie alguma informação na requisição informando quem ele é. Um exemplo seria usar algum tipo de dado secreto compartilhado ou um certificado do lado cliente para assinar a requisição. Quando o servidor tiver de verificar a identidade do cliente, queremos que essa tarefa seja o mais eficaz possível – já vi algumas soluções (incluindo aquelas que os fornecedores de gateways de API nos empurram) que envolvem o servidor precisando fazer chamadas a serviços centralizados para verificar a identidade do cliente, o que é uma loucura se consideramos as implicações de latência.

Tenho dificuldade de pensar em uma situação na qual eu verificasse a identidade do cliente sem também verificar a identidade do servidor – para verificar ambos, em geral acabaríamos implementando alguma forma de *autenticação mútua*. Na autenticação mútua, as duas partes se autenticam entre si. Assim, na Figura 11.6, o Processador de Pedido autentica o microserviço Pagamento, e o microserviço Pagamento autentica o Processador de Pedido.

Isso pode ser feito com o uso do *TLS mútuo*, caso em que tanto o cliente

como o servidor fazem uso de certificados. Na internet pública, verificar a identidade de um dispositivo cliente geralmente é menos importante que verificar a identidade da pessoa que utiliza esse dispositivo. Desse modo, o TLS mútuo é raramente usado. Em nossa arquitetura de microsserviços, porém, sobretudo se operamos em um ambiente de confiança zero, isso será mais comum.

Historicamente, o desafio para implementar esquemas como o TLS mútuo tem sido as ferramentas. Atualmente, esse problema é menos grave. Ferramentas como o Vault podem facilitar bastante a distribuição de certificados, e o desejo de simplificar o uso do TLS mútuo é um dos principais motivos pelos quais as pessoas implementam service meshes, assunto que foi explorado na seção “Service meshes e gateways de API”, na página [199](#).

Visibilidade dos dados

Ao enviar dados de um microsserviço para outro, alguém poderá vê-los? Para algumas informações, por exemplo, o preço dos álbuns de Peter Andre, talvez não nos importemos muito, pois os dados já estão no domínio público. Por outro lado, alguns dados poderiam incluir informações como PII, as quais devemos garantir que estejam protegidas.

Ao usar o bom e velho HTTPS ou o TLS mútuo, os dados não estarão visíveis às partes intermediárias – isso porque o TLS criptografa os dados enviados. Isso pode ser um problema se você quiser que os dados sejam explicitamente enviados às claras – por exemplo, proxies reversos como o Squid ou o Varnish são capazes de fazer cache de respostas HTTP, mas isso não será possível com o HTTPS.

Manipulação dos dados

Poderíamos imaginar uma série de situações em que manipular os dados enviados poderia ser ruim – alterar um valor monetário sendo enviado, por exemplo. Assim, na Figura 11.6, precisamos garantir que um invasor em potencial seja incapaz de alterar a requisição enviada de `Pagamento` para `Processador de Pedido`.

Em geral, os tipos de proteção que deixam os dados invisíveis também

garantirão que esses dados não possam ser manipulados (o HTTPS, por exemplo, faz isso). No entanto, poderíamos decidir enviar dados abertamente, mas ainda garantir que não possam ser manipulados. Para o HTTP, uma abordagem desse tipo consiste em usar um HMAC (Hash-based Message Authentication Code, ou Código de Autenticação de Mensagem Baseado em Hash) para assinar os dados enviados. Com o HMAC, um hash é gerado e enviado com os dados, e o receptor pode conferir o hash em relação aos dados a fim de confirmar que os dados não foram alterados.

Dados em repouso

Dados em repouso são uma responsabilidade, principalmente se forem sigilosos. Espero que tenhamos feito de tudo para garantir que os invasores não possam atacar nossa rede, não possam invadir nossas aplicações nem os sistemas operacionais para ter acesso aos dados subjacentes. No entanto, precisamos estar preparados caso consigam – a defesa em profundidade é o segredo.

Muitas das violações de segurança famosas das quais ouvimos falar envolvem dados em repouso que foram adquiridos por um invasor, com esses dados estando legíveis. Isso acontece porque os dados estavam armazenados em um formato não criptografado ou porque o método usado para protegê-los tinha alguma falha básica.

Os métodos para proteger os dados em repouso são abundantes e variados, mas há alguns aspectos genéricos que devemos ter em mente.

Opte pelo que é bem conhecido

Em alguns casos, é possível passar o trabalho de criptografia dos dados a um software existente – por exemplo, fazendo uso do suporte para criptografia incluído em seu banco de dados. No entanto, se houver necessidade de criptografar e descriptografar dados em seu próprio sistema, certifique-se de optar por implementações testadas e bem conhecidas. O modo mais fácil de cometer erros com a criptografia de dados é tentar implementar seus próprios algoritmos de criptografia, ou até mesmo tentar implementar o algoritmo de outra pessoa. Qualquer que seja

a linguagem de programação utilizada, você terá acesso a implementações revisadas de algoritmos de criptografia bem avaliados, com patching frequente. Utilize-os! E inscreva-se nas listas de distribuição/advertências da tecnologia que você escolher a fim de garantir que estará ciente das vulnerabilidades à medida que forem encontradas, de modo que seja possível manter seus produtos atualizados e com os patches em dia.

Para proteger senhas, não deixe de utilizar uma técnica chamada hashing de senhas com salt (<https://oreil.ly/kXUbY>). Ela garante que as senhas jamais sejam armazenadas em formato texto simples, e mesmo que um invasor utilize de força bruta em uma senha com hash, ele não poderá ler automaticamente outras senhas.¹⁴

Uma criptografia mal implementada poderia ser pior do que não ter criptografia nenhuma, pois a falsa sensação de segurança poderia fazer você desviar os olhos da bola.

Escolha seus alvos

Supor que tudo deve ser criptografado pode simplificar a situação, até certo ponto. Não será preciso dar palpites sobre o que precisa ou não ser protegido. No entanto, ainda será necessário pensar em quais dados poderiam ser colocados em arquivos de log para ajudar na identificação de problemas, e o overhead computacional de criptografar tudo pode se tornar bastante oneroso, exigindo um hardware mais potente como resultado. A situação será mais desafiadora se houver migrações de bancos de dados como parte de esquemas de refatoração. Dependendo das mudanças em andamento, os dados talvez precisem ser descriptografados, migrados e criptografados novamente.

Ao subdividir seu sistema em serviços mais numerosos e especializados, talvez você identifique um banco de dados completo que possa ser totalmente criptografado, mas isso é improvável. Limitar essa criptografia a um conjunto conhecido de tabelas é uma abordagem sensata.

Seja frugal

À medida que o espaço em disco se torna mais barato e as funcionalidades dos bancos de dados são aperfeiçoadas, a facilidade com que grandes

volumes de informações podem ser capturados e armazenados aumenta rapidamente. Esses dados são valiosos – não só para os próprios negócios, que cada vez mais veem os dados como um bem valioso, mas igualmente para os usuários, que valorizam sua privacidade. Os dados pertencentes a um indivíduo ou que poderiam ser usados para derivar informações sobre um indivíduo são aqueles que merecem mais o nosso cuidado.

Que tal, porém, se facilitássemos um pouco as nossas vidas? Por que não remover o máximo possível de informações que possam ser pessoalmente identificáveis, e fazer isso o mais cedo possível? Ao fazer o log de uma requisição de usuário, precisamos armazenar o endereço IP completo indeterminadamente, ou poderíamos substituir os últimos dígitos por um x? Precisamos armazenar o nome, a idade, o gênero e a data de nascimento de uma pessoa a fim de apresentar ofertas de produtos a ela, ou a faixa etária e o CEP seriam informações suficientes?

As vantagens de ser frugal na coleta de dados se desdobram. Em primeiro lugar, se você não armazenar os dados, ninguém poderá roubá-los. Em segundo, se esses dados não forem armazenados, ninguém (por exemplo, uma agência governamental) poderá pedi-los!

A palavra em alemão *Datensparsamkeit* representa esse conceito. Proveniente da legislação alemã sobre privacidade, ela inclui o conceito de armazenar somente as informações que sejam *absolutamente necessárias* para conduzir as operações de negócios ou satisfazer as leis locais.

Obviamente, há uma tensão direta entre isso e o movimento em direção a um armazenamento cada vez maior de informações, mas apenas perceber que essa tensão existe já é um bom ponto de partida!

Tem tudo a ver com chaves

A maioria das formas de criptografia envolve o uso de alguma chave, em conjunto com um algoritmo apropriado para criar dados criptografados. Para descriptografar os dados de modo que possam ser lidos, as partes autorizadas precisam ter acesso a uma chave – pode ser a mesma chave ou uma chave diferente (no caso da criptografia com chave pública). Então, onde você armazena suas chaves? Se estou criptografando meus dados

porque me preocupo com o fato de alguém poder roubar todo o meu banco de dados, e eu armazeno a chave no mesmo banco de dados, então não teria feito muita coisa! Desse modo, precisamos armazenar as chaves em outro lugar. Mas onde?

Uma solução é usar um dispositivo de segurança diferente para criptografar e descriptografar dados. Outra consiste em utilizar um vault (cofre) de chaves distinto, que seu serviço poderá acessar quando precisar de uma chave. O gerenciamento do ciclo de vida das chaves (e o acesso para alterá-las) pode ser uma operação essencial, e esses sistemas poderão lidar com essas tarefas para você. É nesse cenário que o Vault da HashiCorp também pode ser muito conveniente.

Alguns bancos de dados incluem até mesmo um suporte para criptografia, por exemplo, o Transparent Data Encryption do SQL Server, cujo objetivo é lidar com essa tarefa de modo transparente. Ainda que o banco de dados de sua preferência inclua um suporte desse tipo, pesquise para saber como as chaves são tratadas e entenda se a ameaça contra a qual você está se protegendo será realmente atenuada.

Novamente, esse assunto é complexo. Evite implementar sua própria criptografia, e faça uma boa pesquisa!



Criptografe os dados da primeira vez que os vir. Descriptografe somente por demanda, e garanta que os dados jamais sejam armazenados em outro lugar.

Criptografe os backups

Backups são bons. Queremos fazer backup de nossos dados importantes. Parece ser uma questão óbvia, mas, se os dados forem suficientemente sigilosos a ponto de querermos criptografá-los em nosso sistema em execução no ambiente de produção, é provável que vamos querer garantir também que qualquer backup dos mesmos dados também seja criptografado!

Autenticação e autorização

Autenticação e autorização são conceitos básicos quando se trata de pessoas e componentes que interagem com nosso sistema. No contexto da segurança, a *autenticação* é o processo pelo qual confirmamos que uma parte é quem diz ser. Em geral, autenticamos um usuário humano fazendo-o digitar seu nome de usuário e a senha. Partimos do pressuposto de que somente o usuário propriamente dito tem acesso a essas informações e, desse modo, a pessoa que as insere deve ser esse usuário. É claro que há sistemas mais complexos também. Nossos telefones atualmente permitem utilizar nossa impressão digital ou a face para confirmar que somos quem dizemos ser. De modo geral, quando falamos de modo abstrato sobre quem ou o que está sendo autenticado, nos referimos a essa parte como o *principal*¹⁵.

A *autorização* é o método pelo qual mapeamos um principal à ação que permitimos que ele faça. Com frequência, quando um principal é autenticado, recebemos informações sobre ele, as quais nos ajudarão a decidir o que permitiremos que ele faça. Poderíamos, por exemplo, ser informados sobre o departamento ou o escritório no qual o principal trabalha – uma informação que nosso sistema poderia usar para decidir o que o principal pode e não pode fazer.

A facilidade de uso é importante – queremos fazer com que seja fácil para nossos usuários acessarem o nosso sistema. Não queremos que cada um tenha de fazer login separadamente para acessar diferentes microsserviços, utilizando um nome de usuário e uma senha diferentes para cada um. Desse modo, também precisamos discutir como implementar o SSO (Single Sign-On, ou Login Único) em um ambiente de microsserviços.

Autenticação de serviço a serviço

Discutimos antes o TLS mútuo, o qual, além de proteger os dados em trânsito, também nos permite implementar uma forma de autenticação. Quando um cliente conversa com um servidor usando TLS mútuo, o servidor é capaz de autenticar o cliente, e o cliente é capaz de autenticar o servidor – essa é uma forma de autenticação de serviço a serviço. Outros esquemas de autenticação podem ser usados além do TLS mútuo. Um exemplo comum é empregar chaves de API, em que o cliente precisa

utilizar a chave para criar o hash de uma requisição de tal modo que o servidor seja capaz de conferir se o cliente usou uma chave válida.

Autenticação de pessoas

Estamos acostumados com as pessoas se autenticando por meio da combinação familiar de nome de usuário e senha. Cada vez mais, porém, isso é usado como parte de uma abordagem de autenticação multifator, na qual um usuário pode precisar de mais de uma informação (um *fator*) para se autenticar. O mais comum é que se use uma MFA (Multifactor Authentication, ou Autenticação Multifator)¹⁶, em que mais de um fator é necessário. A MFA geralmente envolve o uso de uma combinação comum de nome de usuário e senha, além de pelo menos um fator adicional.

Os diferentes tipos de fatores de autenticação aumentaram nos últimos anos – de códigos enviados por SMS a links mágicos enviados por email ou aplicativos móveis dedicados como o Authy (<https://authy.com>) e dispositivos de hardware USB e NFC como o YubiKey (<https://www.yubico.com>). Fatores biométricos são mais comuns hoje em dia também, pois os usuários têm mais acesso a hardwares que aceitam fatores como impressão digital ou reconhecimento facial. Embora a MFA tenha se mostrado ser muito mais segura como uma abordagem geral, e muitos serviços públicos a aceitem, ela não se disseminou como um esquema de autenticação no mercado de massa, embora eu espere que essa situação vá mudar. Para gerenciar a autenticação de serviços essenciais que sejam vitais à execução de seu software ou permitir acesso a informações particularmente sigilosas (por exemplo, acesso ao código-fonte), eu consideraria o uso da MFA como obrigatória.

Implementações comuns de SSO

Uma abordagem comum à autenticação consiste em usar algum tipo de solução de SSO (Single Sign-On, ou Login Único) para garantir que um usuário tenha de se autenticar apenas uma vez por sessão, mesmo que, durante essa sessão, ele possa acabar interagindo com vários serviços ou aplicações downstream. Por exemplo, ao fazer login com a sua conta Google, você estará logado no Google Calendar, no Gmail e no Google

Docs, ainda que eles sejam sistemas distintos.

Quando um principal tenta acessar um recurso (por exemplo, uma interface web), ele será direcionado para se autenticar com um *provedor de identidade*. O provedor de identidade pode solicitar que um nome de usuário e uma senha sejam fornecidos, ou pode exigir o uso de algo mais sofisticado, como a MFA. Assim que o provedor de identidade estiver satisfeito com o fato de o principal ter sido autenticado, ele dará informações para o *provedor de serviços*, permitindo que ele decida se deve conceder acesso ao recurso.

O provedor de identidade poderia ser um sistema hospedado externamente, ou algo interno à sua empresa. O Google, por exemplo, disponibiliza um provedor de identidade OpenID Connect. Para as empresas, porém, é comum ter um provedor de identidade próprio, que pode estar associado ao *serviço de diretórios* da empresa. Um serviço de diretórios poderia ser algo como o LDAP (Lightweight Directory Access Protocol, ou Protocolo Leve de Acesso a Diretório) ou o Active Directory. Esses sistemas permitem armazenar informações sobre os principais, por exemplo, as funções que desempenham na empresa. Com frequência, o serviço de diretórios e o provedor de identidade são os mesmos, embora, ocasionalmente, sejam distintos, porém ligados. O Okta, por exemplo, é um provedor de identidade SAML hospedado, que lida com tarefas como a autenticação de dois fatores, mas pode estar ligado aos serviços de diretório de sua empresa que servem como fonte da verdade.

Assim, o provedor de identidade dá ao sistema informações sobre quem é o principal, mas o sistema decidirá o que esse principal tem permissão para fazer.

O SAML é um padrão baseado em SOAP, e é conhecido por ser bastante complexo para trabalhar, apesar das bibliotecas e ferramentas disponíveis para suporte, e desde a primeira edição deste livro, tem caído rapidamente em desuso.¹⁷ O OpenID Connect é um padrão que surgiu como uma implementação específica do OAuth 2.0, com base no modo como o Google e outros lidam com o SSO. Utiliza chamadas REST mais simples, e em parte por causa de sua relativa simplicidade e do suporte amplo, é o

método predominante para SSO de usuários finais, tendo ganhado significativa aceitação por parte das empresas.

Gateway de SSO

Poderíamos decidir lidar com o redirecionamento para – e o handshaking com – o provedor de identidade em cada microserviço, de modo que qualquer requisição não autenticada de uma entidade externa seja tratada de forma apropriada. Obviamente, isso poderia implicar muitas funcionalidades duplicadas entre nossos microserviços. Uma biblioteca compartilhada poderia ajudar, mas teríamos de ser cuidadosos a fim de evitar o acoplamento que poderia resultar de um código compartilhado (veja a seção “DRY e os perigos da reutilização de códigos em um mundo de microserviços”, na página [190](#) para saber mais). Uma biblioteca compartilhada também não ajudaria caso tivéssemos microserviços escritos em diferentes conjuntos de tecnologias.

Em vez de ter cada serviço gerenciando o handshaking com nosso provedor de identidade, uma abordagem mais comum consiste em usar um gateway que atue como um proxy, o qual estaria entre seus serviços e o mundo externo (como vemos na Figura 11.7). A ideia é podermos centralizar o comportamento para redirecionar o usuário e efetuar o handshake em um só lugar.

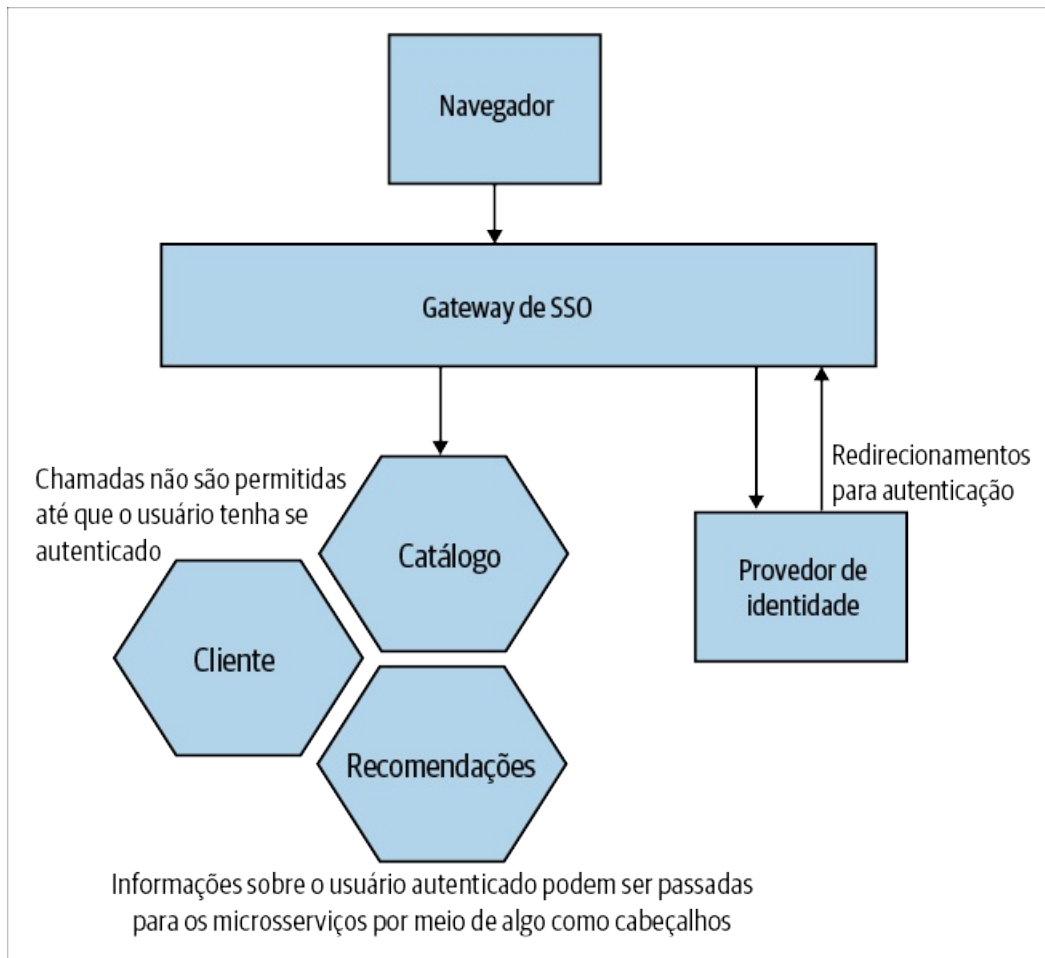


Figura 11.7 – Utilizando um gateway para lidar com o SSO.

No entanto, ainda precisamos resolver o problema de como o serviço downstream receberá informações sobre os principals, por exemplo, seu nome de usuário ou as funções (roles) que desempenham. Se estiver usando HTTP, você poderia configurar seu gateway de modo que os cabeçalhos sejam preenchidos com essas informações. O Shibboleth é uma ferramenta que pode fazer isso para você, e já o vi ser usado com o servidor web Apache para cuidar da integração com provedores de identidade baseados em SAML, com muita eficácia. Uma alternativa, que veremos com mais detalhes em breve, consiste em criar um JWT (JSON Web Token) contendo todas as informações sobre o principal; essa solução tem uma série de vantagens, incluindo ser algo que pode ser mais facilmente passado de um microsserviço para outro.

Uma consideração adicional a respeito do uso de um gateway de SSO é

que, se decidirmos passar a responsabilidade pela autenticação a um gateway, poderá ser mais difícil entender como um microsserviço se comporta se o analisarmos isoladamente. Você se lembra de que, no Capítulo 9, exploramos alguns dos desafios de reproduzir ambientes semelhantes ao de produção? Se você decidir usar um gateway, certifique-se de que seus desenvolvedores possam iniciar seus serviços atrás de um, sem que tenham muito trabalho.

Um último problema com essa abordagem é que ela pode ludibriar você, deixando-o com uma falsa sensação de segurança. Novamente, gosto de voltar à ideia da defesa em profundidade – incluindo perímetro de rede, sub-rede, firewall, máquina, sistema operacional e o hardware subjacente. Você é capaz de implementar medidas de segurança em todos esses pontos. Já vi algumas pessoas colocarem todos os ovos em uma só cesta, contando com o gateway para lidar com todos os passos para elas. E todos sabemos o que acontecerá se tivermos um único ponto de falha...

Obviamente, esse gateway poderia ser usado para outras tarefas. Por exemplo, poderíamos decidir também terminar o HTTPS nesse nível, executar uma detecção de invasão, e assim por diante. No entanto, seja cuidadoso. As camadas de gateway tendem a assumir cada vez mais funcionalidades, e ele próprio pode acabar se transformando em um ponto gigantesco de acoplamento. Quanto mais funcionalidades algo tiver, maior será a sua superfície de ataque.

Autorizações mais específicas

Um gateway talvez seja capaz de prover uma autenticação mais genérica com bastante eficácia. Por exemplo, poderia impedir que qualquer usuário não logado acesse a aplicação de helpdesk. Supondo que nosso gateway possa extrair atributos sobre o principal como resultado da autenticação, ele poderia tomar decisões mais específicas. Por exemplo, é comum colocar pessoas em grupos ou atribuir funções (roles) a elas. Podemos usar essas informações para saber o que elas podem fazer. Assim, no caso da aplicação de helpdesk, poderíamos permitir o acesso somente aos principais com uma função específica (por exemplo, `STAFF`). Além de

permitir (ou impedir) o acesso a recursos ou endpoints específicos, porém, precisamos deixar o resto a cargo do próprio microsserviço; ele terá de tomar outras decisões sobre as operações permitidas.

Voltando à nossa aplicação de helpdesk: permitiremos que qualquer membro do staff veja todo e qualquer detalhe? É mais provável que tenhamos diferentes funções no trabalho. Por exemplo, um principal no grupo `CALL_CENTER` poderia ter permissão para visualizar qualquer informação sobre um cliente, exceto os detalhes de pagamentos. Esse principal também poderia ter permissão para gerar reembolsos, mas esse valor poderia estar limitado. Alguém que tivesse a função de `CALL_CENTER_TEAM_LEADER`, no entanto, poderia gerar reembolsos com valores mais elevados.

Essas decisões devem ser locais ao microsserviço em questão. Já vi pessoas utilizarem os diversos atributos disponibilizados pelos provedores de identidade de maneiras horríveis, usando funções realmente bem específicas como `CALL_CENTER_50_DOLLAR_REFUND`, acabando por colocar informações específicas de uma funcionalidade de um microsserviço em seus serviços de diretório. É um pesadelo para a manutenção, e reduz bastante o escopo para que nossos serviços tenham o próprio ciclo de vida independente, pois, repentinamente, uma informação sobre como um serviço deve se comportar estará em outro lugar, talvez em um sistema gerenciado por uma parte diferente da empresa.

Garantir que o microsserviço tenha as informações de que precisa para avaliar requisições de autorização mais específicas é um assunto que vale a pena ser mais bem discutido – retomaremos esse tópico quando discutirmos os JWTs em breve.

Dê preferência a funções (roles) mais genéricas, modeladas com base no modo de funcionamento de sua empresa. Retornando aos primeiros capítulos, lembre-se de que estamos criando um software que corresponda ao modo como a nossa empresa funciona. Portanto utilize suas funções dessa maneira também.

Problema do confused deputy

Fazer um principal se autenticar junto ao sistema como um todo utilizando algo como um gateway de SSO é muito simples, e poderia bastar para controlar o acesso a um dado microsserviço. No entanto, o que acontecerá se esse microsserviço tiver de fazer chamadas adicionais para concluir uma operação? Isso pode nos deixar suscetíveis a um tipo de vulnerabilidade conhecida como o problema do confused deputy (agente enganado)). Ocorre quando um componente upstream engana um componente intermediário, levando-o a executar tarefas que não deveria. Vamos analisar um exemplo concreto na Figura 11.8, que mostra o site da loja online da MusicCorp. Nossa UI JavaScript para navegador conversa com o microsserviço `Loja online` do lado do servidor, que é um tipo de padrão backend para frontend. Exploraremos esse tópico com mais detalhes na seção “Padrão: backend para frontend (BFF)”, na página 559, mas, por ora, pense nele como um componente do lado do servidor que faz agregação de chamadas e filtragem para uma interface externa específica (em nosso caso, nossa UI JavaScript para navegador). Chamadas feitas entre o navegador e a `Loja online` podem ser autenticadas usando o OpenID Connect. Até agora, tudo bem.

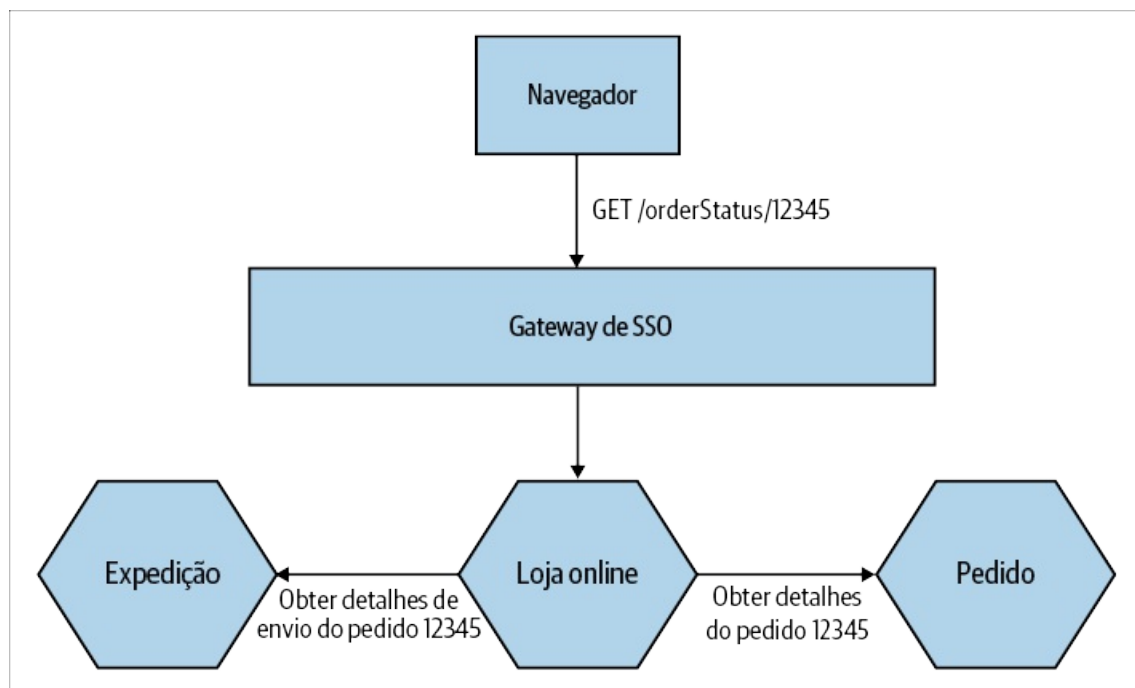


Figura 11.8 – Um exemplo no qual um confused deputy (agente enganado) poderia entrar em ação.

Quando um usuário faz login, ele pode clicar em um link para visualizar detalhes a respeito de um pedido. Para exibir as informações, precisamos recuperar o pedido original a partir do serviço `Pedido`, mas também queremos consultar as informações de envio desse pedido. Assim, quando um cliente logado clicar em um link para `/orderStatus/12345`, essa requisição será encaminhada à `Loja Online`, que então precisará fazer chamadas para os microsserviços downstream `Pedido` e `Expedição`, solicitando os detalhes do pedido 12345.

Entretanto os serviços downstream devem aceitar as chamadas da `Loja Online`? Poderíamos adotar uma postura de confiança implícita: como a chamada veio de nosso perímetro interno, tudo bem. Poderíamos até mesmo utilizar certificados ou chaves de API para confirmar que é realmente a `Loja Online` que está solicitando essas informações. Mas isso seria suficiente? Por exemplo, um cliente logado no sistema de compras online pode ver os detalhes de sua conta pessoal. E se o cliente pudesse enganar a UI da loja online, fazendo com que ela solicitasse os detalhes de *outra pessoa*, talvez fazendo uma chamada com suas próprias credenciais de login?

Nesse exemplo, o que impediria o cliente de solicitar pedidos que não fossem seus? Assim que fizer login, o cliente poderia começar a enviar requisições para outros pedidos que não fossem seus para ver se poderia obter alguma informação útil. Ele poderia começar a adivinhar IDs de pedidos para ver se poderia extrair informações sobre outras pessoas. Basicamente, o que aconteceu nesse caso é que, embora tenhamos *autenticado* o usuário em questão, não damos a *autorização* suficiente. O que queremos é que alguma parte de nosso sistema seja capaz de julgar se uma requisição para visualizar detalhes do Usuário A pode ser permitida somente se for o Usuário A que a esteja solicitando. Mas onde estará essa lógica?

Autorização upstream centralizada

Uma opção para evitar o problema do *confused deputy* é proporcionar toda a autorização necessária assim que a requisição for recebida em nosso

sistema. Na Figura 11.8, isso significa que procuraríamos autorizar a requisição no próprio gateway de SSO ou na Loja Online. A ideia é que, quando as chamadas forem enviadas aos microsserviços Pedido ou Expedição, partiremos do pressuposto de que as requisições foram permitidas.

Essa forma de autorização upstream implica efetivamente que aceitamos alguma forma de confiança implícita (em oposição à confiança zero) – os microsserviços Expedição e Pedido devem supor que recebem somente as requisições às quais têm permissão para atender. O outro problema é que uma entidade upstream – por exemplo, um gateway ou algo similar – precisa ter conhecimento das funcionalidades disponibilizadas pelos microsserviços downstream, e deve saber como limitar o acesso a essas funcionalidades.

O ideal, porém, é que nossos microsserviços sejam tão autocontidos quanto possível, facilitando ao máximo fazer alterações e rollouts de novas funcionalidades. Queremos que nossos lançamentos sejam o mais simples possível – queremos ter a capacidade de fazer implantações independentes. Se a ação de implantação agora envolve tanto a implantação de um novo microsserviço como a aplicação de alguma configuração relacionada à autorizações em um gateway upstream, isso não me parece ser extremamente “independente” para mim.

Portanto, iríamos querer passar a decisão sobre autorizar ou não a chamada para o mesmo microsserviço em que está a funcionalidade sendo requisitada. Isso deixa o microsserviço mais autocontido, além de nos dar a opção de implementar a abordagem de confiança zero se quisermos.

Descentralizando a autorização

Considerando os desafios da autorização centralizada em um ambiente de microsserviços, iríamos querer passar essa lógica para o microsserviço downstream. O microsserviço Pedido é o serviço que contém a funcionalidade para acessar detalhes de um pedido, portanto faria mais sentido que esse serviço decidisse se a chamada é válida. Nesse caso específico, porém, o microsserviço Pedido precisa de informações sobre a pessoa que está fazendo a requisição. Então, como disponibilizamos essas

informações para o microserviço `Pedido`?

No nível mais simples, poderíamos apenas exigir que o identificador da pessoa que fez a requisição seja enviado ao microserviço `Pedido`. Se o HTTP estiver em uso, por exemplo, poderíamos simplesmente inserir o nome do usuário em um cabeçalho. Contudo, em uma situação como essa, o que impediria uma parte mal-intencionada de inserir somente qualquer nome antigo na requisição a fim de obter as informações de que precisasse? O ideal é que tenhamos uma maneira de garantir que a requisição está *realmente* sendo feita em nome de um usuário autenticado, e que possamos passar adiante as informações adicionais sobre esse usuário – por exemplo, os grupos aos quais esse usuário poderia pertencer.

Historicamente, há diversas maneiras diferentes com as quais esse problema tem sido tratado (incluindo técnicas como asserções SAML aninhadas que, sim, são tão complicadas quanto parecem), mas, recentemente, a solução mais comum para esse problema em particular tem sido o uso de JSON Web Tokens.

JSON Web Tokens

Os JWTs permitem armazenar várias alegações (claims) sobre um indivíduo em uma string que pode ser passada adiante. Esse token pode ser assinado para garantir que sua estrutura não foi manipulada, além de poder ser opcionalmente criptografado a fim de oferecer garantias acerca de quem pode ler os dados. Embora os JWTs possam ser usados para trocas genéricas de informações, em que é importante garantir que os dados não tenham sido adulterados, eles são mais comuns para auxiliar na transmissão de informações com vistas à autorização.

Assim que forem assinados, os JWTs podem ser facilmente transmitidos por meio de diversos protocolos, e os tokens podem, opcionalmente, ser configurados para expirar após determinado período. São amplamente aceitos, com vários provedores de identidade que aceitam a geração de JWTs, e há muitas bibliotecas para que você possa usar os JWTs em seu próprio código.

Formato

O payload principal de um JWT é uma estrutura JSON que, falando de modo geral, pode conter qualquer dado que você quiser. Podemos ver um token no Exemplo 11.1. O padrão JWT descreve (<https://oreil.ly/2Yo5X>) alguns campos com nomes específicos (“public claims”, ou alegações públicas) que você deve usar caso sejam relevantes para o seu caso. Por exemplo, `exp` define a data de vencimento de um token. Se fizer uso desses campos de alegações públicas corretamente, há uma boa chance de as bibliotecas que você usar serem capazes de utilizá-los de modo apropriado – rejeitando um token, por exemplo, caso o campo `exp` determine que o token já expirou.¹⁸ Mesmo que você não use todas essas alegações públicas, vale a pena saber o que elas são a fim de garantir que você não acabará utilizando-os em casos de uso específicos de sua aplicação, pois isso causaria alguns comportamentos inesperados nas bibliotecas auxiliares.

Exemplo 11.1 – Exemplo de um payload JSON em um JWT

```
{
  "sub": "123",
  "name": "Sam Newman",
  "exp": 1606741736,
  "groups": "admin, beta"
}
```

No Exemplo 11.2, vemos o token do Exemplo 11.1 codificado. Esse token é composto somente de uma única string, mas está separada em três partes delimitadas por um “.” – o cabeçalho, o payload e a assinatura.

Exemplo 11.2 – Resultado da codificação de um payload JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. ❶  
eyJzdWIiOiIxMjMlIjoiU2FtIE5ld21hbiIsImV4cCI6MTYwNjc0MTczNiwiZ3J... ❷  
Z9HMH0DGs60I0P5bVVSFixeDxJjGovQEtLNui__iE_0 ❸
```

❶ O cabeçalho

❷ O payload (truncado)

❸ A assinatura

Para deixar o exemplo mais claro, separei cada parte em uma linha, mas, na verdade, temos uma única string, sem quebra de linhas. O cabeçalho contém informações sobre o algoritmo de assinatura em uso. Isso permite

que o programa que decodificará o token aceite diferentes esquemas de assinatura. O payload é o local em que armazenamos as informações sobre as alegações feitas pelo token – é somente o resultado da codificação da estrutura JSON do Exemplo 11.1. A assinatura é usada para garantir que o payload não foi manipulado, além de poder ser utilizado para garantir que o token foi gerado por quem você acha que o gerou (supondo que o token esteja assinado com uma chave privada).

Por ser uma string simples, esse token pode ser facilmente passado adiante por meio de diferentes protocolos de comunicação – como um cabeçalho no HTTP (no campo `Authorization`), por exemplo, ou talvez como um metadado em uma mensagem. Essa string codificada, é claro, pode ser enviada por um protocolo de transporte criptografado – por exemplo, TLS sobre HTTP – caso em que o token não estaria visível às pessoas que observassem a comunicação.

Usando tokens

Vamos analisar um modo comum de usar tokens JWT em uma arquitetura de microsserviços. Na Figura 11.9, nosso cliente faz login do modo usual, e assim que for autenticado, geramos algum tipo de token para representar sua sessão logada (provavelmente um token OAuth), que será armazenado no dispositivo do cliente. Requisições subsequentes desse dispositivo chegarão ao nosso gateway, o qual gerará um token JWT que será válido pela duração dessa requisição. É esse token JWT que será então passado para os microsserviços downstream. Eles são capazes de validar o token e extrair as alegações do payload para determinar os tipos de autorização apropriados.

Uma variação dessa abordagem consiste em gerar um token JWT quando o usuário se autenticar inicialmente junto ao sistema, e então fazer com que esse token seja armazenado no dispositivo do cliente. Vale a pena considerar, porém, que um token como esse terá de ser válido enquanto durar a sessão logada; conforme já foi discutido, gostaríamos de limitar o período de validade das credenciais geradas pelo sistema a fim de reduzir as chances de essas credenciais serem usadas indevidamente, e diminuir o impacto caso precisemos alterar as chaves usadas para gerar o token

codificado. Gerar um token JWT a cada requisição parece ser a solução mais comum para esse problema, como mostra a Figura 11.9. Ter algum tipo de troca de tokens feita no gateway também pode facilitar bastante a adoção dos tokens JWT sem que seja necessário alterar qualquer parte do fluxo de autenticação que envolva uma comunicação com o dispositivo do cliente – se você já tem uma solução de SSO funcional implantada, ocultar o fato de os tokens JWT serem usados do fluxo principal de autenticação de usuários fará com que uma mudança como essa cause menos perturbações.

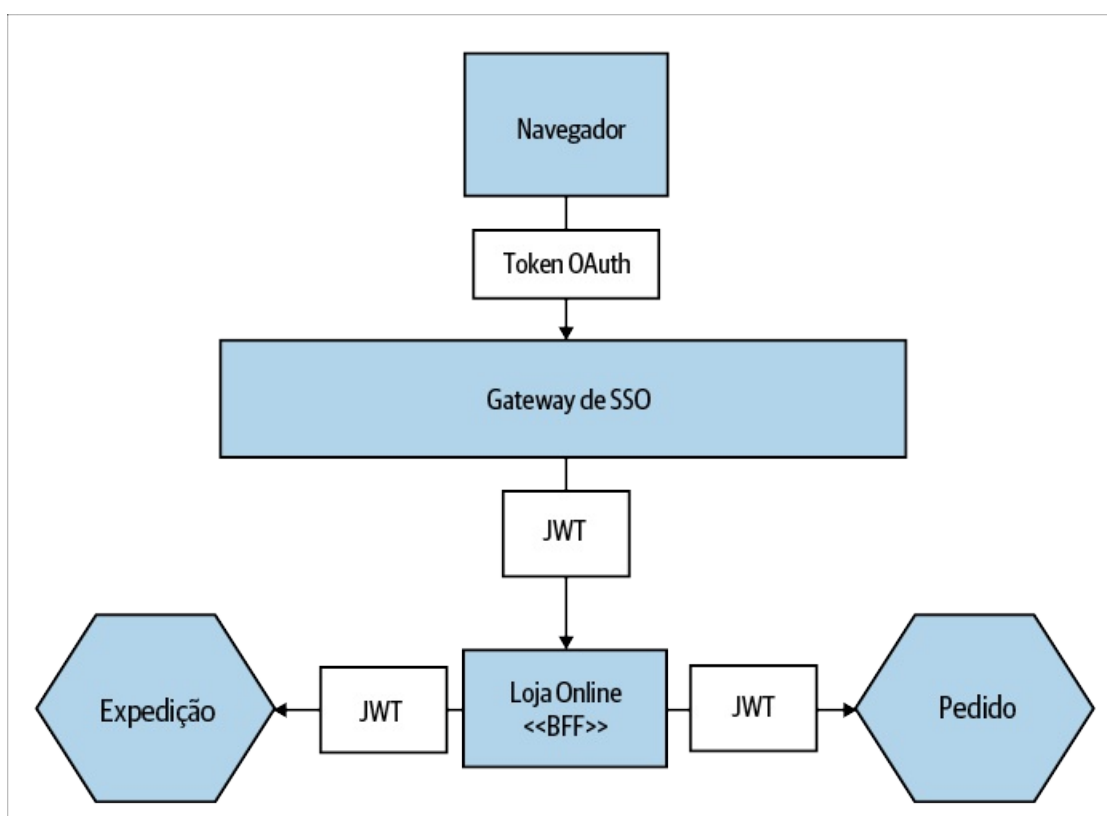


Figura 11.9 – Um token JWT é gerado para uma requisição específica e passado para os microsserviços downstream.

Assim, com uma geração apropriada de tokens JWT, nossos microsserviços downstream serão capazes de obter todas as informações de que precisam para confirmar a identidade do usuário que fez a requisição, bem como obter informações adicionais como os grupos ou funções (roles) associados a esse usuário. A validade desse token também pode ser verificada pelo microsserviço simplesmente conferindo a assinatura do

token JWT. Comparada com as soluções anteriores nessa área (por exemplo, as asserções SAML aninhadas), os tokens JWT deixaram o processo de descentralização da autorização muito mais simples em uma arquitetura de microsserviços.

Desafios

Há algumas questões relacionadas aos tokens JWT que valem a pena ter em mente. A primeira é a questão das chaves. No caso dos tokens JWT assinados, para verificar uma assinatura, o receptor de um token JWT precisará de algumas informações que deverão ser comunicadas por fora – geralmente, precisará de uma chave pública. Todos os aspectos relacionados ao gerenciamento de chaves se aplicam nesse caso. Como o microsserviço obtém a chave pública? O que acontecerá se a chave pública tiver de ser alterada? O Vault é um exemplo de uma ferramenta que poderia ser usada por um microsserviço para obter (e lidar com a rotatividade das) chaves públicas, e já foi projetado para funcionar em um ambiente altamente distribuído. É claro que poderíamos simplesmente deixar uma chave pública fixa em um arquivo de configuração para o microsserviço receptor, mas então teríamos o problema de lidar com a alteração das chaves públicas.

A segunda questão é definir o prazo de validade correto para um token, que pode ser uma tarefa complicada caso haja processamentos de longa duração envolvidos. Considere a situação em que um cliente tenha feito um pedido. Esse fato dispara um conjunto de processos assíncronos que poderia demorar horas, ou então dias, para ser concluído, sem nenhum envolvimento subsequente do cliente (receber pagamento, enviar email de notificação, fazer com que o item seja embalado para envio etc.). Desse modo, temos de gerar um token com um período de validade correspondente? A pergunta, nesse caso, é até que ponto ter um token de longa duração será mais problemático do que não ter nenhum token? Conversei com algumas equipes que já tiveram de lidar com esse problema. Algumas geravam um token especial de longa duração, que tinham como escopo trabalhar somente nesse contexto específico; outras simplesmente paravam de usar o token em determinado ponto do fluxo. Ainda não

analisei exemplos suficientes desse problema a fim de determinar qual seria a solução correta nesse caso, mas é um problema do qual você deve estar ciente.

Por fim, em algumas situações, você poderia acabar precisando de tantas informações no token JWT, a ponto de o próprio tamanho do token se tornar um problema. Embora seja rara, essa situação pode acontecer. Alguns anos atrás, eu estava conversando com uma equipe sobre o uso de um token para gerenciar a autorização de um aspecto em particular de seu sistema que lidava com o gerenciamento de direitos musicais. A lógica envolvida era incrivelmente complexa – meu cliente havia constatado que, para qualquer dada faixa musical, ela poderia precisar de até 10 mil entradas em um token para lidar com os diferentes cenários. No entanto, percebemos que, ao menos naquele domínio, era somente um único caso de uso específico que precisava dessa quantidade enorme de informações, enquanto a maior parte do sistema ficaria satisfeita com um token simples, com menos campos. Em uma situação como essa, fazia sentido lidar com o processo mais complexo de autorização no gerenciamento de direitos musicais de modo diferente – basicamente utilizando o token JWT para a autorização inicial “simples”, e então fazendo uma consulta subsequente em uma base de dados para buscar os campos adicionais à medida que fossem necessários. Isso implicava que a maior parte do sistema podia trabalhar somente com base nos tokens.

Resumo

Conforme espero ter explicado neste capítulo, construir um sistema seguro não diz respeito a uma única tarefa. Isso exige que você tenha uma visão holística de seu sistema, utilizando algum tipo de exercício de modelagem de ameaças para entender os tipos de controles de segurança que devem ser implantados.

Ao pensar nesses controles, uma combinação de soluções é essencial para construir um sistema seguro. A defesa em profundidade não significa apenas ter várias medidas de proteção implantadas; também significa que você terá uma abordagem multifacetada na construção de um sistema mais

seguro.

Também retomamos um tema fundamental do livro: ter um sistema decomposto em serviços menores e mais especializados nos dá muito mais opções para resolver problemas. Ter microsserviços não só pode reduzir o impacto de qualquer violação, mas também permite fazer ponderações entre o overhead de abordagens mais complexas e seguras quando os dados forem sigilosos, e uma abordagem mais leve quando os riscos forem mais baixos.

Para uma visão mais ampla sobre a segurança de aplicações em geral, recomendo o livro *Agile Application Security*, de Laura Bell et al.¹⁹

A seguir, veremos como podemos deixar nossos sistemas mais confiáveis, passando para o tópico sobre resiliência.

¹ Por favor, não vamos dizer que tudo tem a ver com o Brexit.

² Por mais que eu tente, não consigo encontrar a fonte original desse esquema de classificação.

³ Recomendo o livro *Understanding Design Thinking, Lean, and Agile* (O'Reilly), de Jonny, para ver mais de seus insights.

⁴ Era mais uma discussão passiva-agressiva, muitas vezes sem a parte “passiva”.

⁵ Adam Shostack, *Threat Modeling: Designing for Security* (Indianapolis: Wiley, 2014).

⁶ Troy Hunt, “Passwords Evolved: Authentication Guidance for the Modern Era” (As senhas evoluíram: orientações para autenticação na era moderna), 26 de julho de 2017, <https://oreil.ly/T7PYM>.

⁷ Neil McAllister, “Code Spaces Goes Titsup FOREVER After Attacker NUKES Its Amazon-Hosted Data” (Code Spaces fora de combate PARA SEMPRE depois de invasor ter DESTRUÍDO seus dados hospedados na Amazon), *The Register*, 18 de junho de 2014, <https://oreil.ly/mw7PC>.

⁸ Algumas pessoas se preocupam com o fato de os secrets serem armazenados em formato texto simples. Isso ser ou não um problema para você dependerá principalmente de seu modelo de ameaças. Para que os secrets possam ser lidos, um invasor precisaria ter acesso direto aos sistemas básicos que executam seu cluster e, se for esse o caso, não há dúvidas de que seu cluster já estaria irremediavelmente comprometido.

⁹ Eu certamente inventei esse termo, mas também acho que é muito provável que não tenha sido o único.

¹⁰ Assim como Niels Bohr argumentou que o gato de Schrödinger estava igualmente vivo e morto até você realmente abrir a caixa para verificar.

¹¹ N.T.: Nome de um filme de suspense que, no Brasil, recebeu o título de *Quando um Estranho Chama*.

¹² Jan Schaumann (@jschauma), Twitter, 5 de novembro de 2020, 16h22min, <https://oreil.ly/QaCm2>.

- 13 O “S” de “HTTPS” costumava se relacionar com o SSL (Secure Socket Layer, ou Camada de Socket Seguro) mais antigo, que foi substituído pelo TLS por uma série de motivos. De forma confusa, o termo *SSL* ainda permanece em uso, mesmo quando o TLS é usado. A biblioteca OpenSSL, por exemplo, é amplamente usada para implementar o TLS, e quando um certificado SSL é gerado, na verdade será um certificado para TLS. Nós não facilitamos a vida para nós mesmos, não é verdade?
- 14 Não criptografamos senhas em repouso, pois a criptografia implica que qualquer um com a chave correta possa ler de volta a senha.
- 15 N.T.: Em inglês, o termo *principal* refere-se à pessoa mais importante, que exerce autoridade, em um grupo.
- 16 No passado, falaríamos da autenticação de dois fatores (2FA). A MFA representa o mesmo conceito, porém introduz a ideia de que, hoje em dia, geralmente permitimos que nossos usuários forneçam um fator adicional a partir de diversos dispositivos, por exemplo, um token de segurança, um aplicativo móvel de autenticação, ou talvez o uso de biometria. Podemos considerar a 2FA como um subconjunto da MFA.
- 17 Não posso ser responsabilizado por isso!
- 18 O site do JWT (<https://jwt.io>) tem uma visão geral excelente de quais bibliotecas aceitam quais alegações públicas – é um ótimo recurso geral para tudo que se relaciona ao JWT.
- 19 Laura Bell et al., *Agile Application Security* (Sebastopol: O’Reilly, 2017).

Resiliência

À medida que o software se torna uma parte cada vez mais essencial da vida de nossos usuários, precisamos continuamente melhorar a qualidade dos serviços que oferecemos. Um software que falha pode causar um impacto significativo na vida das pessoas, mesmo que o software não se enquadre na categoria de “segurança máxima”, como os sistemas de controle de aeronaves. Durante a pandemia de COVID-19, que estava em andamento quando escrevi este livro, serviços como os de compras de supermercado online passaram de serviços convenientes para serem uma necessidade para muitas pessoas que não podiam sair de suas casas.

Em um contexto como esse, muitas vezes recebemos a incumbência de criar softwares que sejam cada vez mais *confiáveis*. As expectativas de nossos usuários mudaram quanto àquilo que os softwares podem fazer e quando devem estar disponíveis. Os dias em que havia suporte aos softwares somente nos horários comerciais estão cada vez mais raros, e há cada vez menos tolerância a downtime por causa de alguma manutenção.

Conforme discutimos no início deste livro, há uma série de motivos pelos quais as arquiteturas de microsserviços vêm sendo escolhidas pelas empresas no mundo todo. Contudo, para muitos, a perspectiva de melhorar a *resiliência* de suas ofertas de serviços é citada como um motivo principal.

Antes de passarmos para os detalhes de como uma arquitetura de microsserviços pode oferecer resiliência, é importante dar um passo para trás e considerar o que é, exatamente, a resiliência. O fato é que, quando se trata de aumentar a resiliência de nosso software, adotar uma arquitetura de microsserviços é somente uma parte do quebra-cabeça.

O que é resiliência?

Empregamos o termo *resiliência* em vários contextos diferentes, e de diversas maneiras. Isso pode gerar confusão a respeito do significado do termo, além de nos levar a pensar de modo muito limitado sobre a área. Fora dos limites da TI, há uma área mais ampla de engenharia de resiliência, que encara o conceito de resiliência conforme aplicado a uma série de sistemas – de combate a incêndios ao controle de tráfego aéreo, passando por sistemas biológicos e administração de teatros. Com base nessa área, David D. Woods procurou classificar os diferentes aspectos da resiliência a fim de nos ajudar a pensar de modo mais amplo o que, exatamente, significa a resiliência.¹ Eis os quatro conceitos:

Robustez

A capacidade de absorver perturbações esperadas.

Recuperação

A capacidade de se recuperar após um evento traumático.

Extensibilidade com elegância

Quão bem lidamos com uma situação inesperada.

Adaptabilidade sustentável

A capacidade de se adaptar continuamente a ambientes, stakeholders e demandas que mudam.

Vamos ver esses conceitos um a um e analisar como essas ideias podem (ou não) ser traduzidas para o nosso mundo de construção de arquiteturas de microsserviços.

Robustez

A robustez é o conceito segundo o qual incluímos métodos em nossos softwares e processos a fim de acomodar problemas inesperados. Temos um conhecimento prévio dos tipos de perturbações que podemos enfrentar e implementamos medidas para combatê-las, de modo que, quando esses problemas surgirem, nosso sistema possa lidar com eles. No contexto de nossa arquitetura de microsserviços, há diversas perturbações que

poderíamos esperar: um host pode falhar, um timeout pode ocorrer em uma conexão de rede, um microsserviço pode estar indisponível. É possível melhorar a robustez de nossa arquitetura de várias maneiras para lidar com essas perturbações, por exemplo, iniciando um host substituto de modo automático, fazendo novas tentativas ou lidando com a falha de um dado microsserviço com elegância.

A robustez, porém, vai além do software. Ela pode se aplicar às pessoas. Se houver uma única pessoa de plantão para cuidar de seu software, o que acontecerá se essa pessoa ficar doente ou se ela não estiver acessível quando um incidente ocorrer? Essa é uma situação razoavelmente simples para ser considerada, e a solução poderia ser ter uma pessoa de backup no plantão.

A robustez, por definição, exige um conhecimento prévio – adotamos medidas para lidar com perturbações conhecidas. Esse conhecimento pode se basear em previsões: poderíamos tirar proveito do nosso entendimento sobre o sistema computacional que estamos criando, sobre os serviços auxiliares e sobre o quadro de funcionários para considerar o que poderia ocorrer de errado. No entanto, a robustez também pode vir de experiências anteriores: podemos melhorar a robustez de nosso sistema após algo inesperado ter acontecido. Talvez jamais tivéssemos considerado o fato de que nosso sistema de arquivos global pudesse estar indisponível, ou, quem sabe, tenhamos subestimado o impacto de os representantes de nosso serviço de atendimento ao cliente estarem indisponíveis fora do horário comercial.

Um dos desafios para melhorar a robustez de nosso sistema é que, à medida que aumentamos a robustez de nossa aplicação, introduzimos mais complexidade no sistema, o que pode ser a causa de novos problemas. Vamos supor que você esteja passando sua arquitetura de microsserviços para o Kubernetes porque quer que ele cuide do gerenciamento de estado desejado (desired state management) das cargas de trabalho de seus microsserviços. Como resultado, você pode ter melhorado alguns aspectos relacionados à robustez de sua aplicação, mas terá também introduzido novos pontos de complicação em potencial. Desse modo, qualquer

tentativa de aumentar a robustez de uma aplicação deve ser considerada não só por meio de uma análise simples de custo/benefício, mas também avaliando se você estará satisfeito ou não em ter como resultado um sistema mais complexo.

A robustez é uma área na qual os microsserviços oferecem uma série de opções, e boa parte do que se segue neste capítulo terá como foco o que você pode fazer em seu software para melhorar a robustez do sistema. Lembre-se, porém, de que essa é somente uma das facetas da resiliência como um todo, e há vários outros aspectos da robustez que não estão relacionados ao software, mas que podem exigir uma consideração.

Recuperação

O modo como nos recuperamos – retornamos – de um distúrbio é uma parte essencial na criação de um sistema resiliente. Com muita frequência, vejo pessoas dedicando tempo e energia na tentativa de eliminar a possibilidade de uma indisponibilidade do sistema, somente para estarem totalmente despreparadas quando uma indisponibilidade realmente ocorrer. Sem dúvida, você deve se esforçar ao máximo para se proteger dos acontecimentos ruins que você acha que poderiam acontecer – melhorando a *robustez* de seu sistema –, mas saiba também que, à medida que seu sistema aumentar em escala e em complexidade, eliminar todos os problemas em potencial se torna insustentável.

Podemos melhorar nossa capacidade de recuperação de um incidente tomando certas medidas com antecedência. Por exemplo, ter backups de prontidão pode possibilitar uma melhor recuperação logo após uma perda de dados (supondo que nossos backups estejam testados, é claro!). A melhoria em nossa capacidade de recuperação também pode incluir a existência de um manual que possa ser consultado no caso de uma indisponibilidade do sistema: as pessoas sabem quais são suas funções quando uma indisponibilidade do sistema ocorrer? Quem será a pessoa designada para lidar com a situação? Com que rapidez devemos deixar que nossos usuários saibam o que está acontecendo? Como nos comunicaremos com nossos usuários? Tentar pensar com clareza sobre o

modo de lidar com uma indisponibilidade do sistema enquanto ela estiver em andamento será complicado por causa do estresse e do caos inerentes à situação. Ter um plano de ação definido com antecedência para esse tipo de problema pode ajudar no processo de recuperação.

Extensibilidade com elegância

Com a recuperação e a robustez, lidamos basicamente com o esperado. Implementamos métodos para lidar com problemas que podemos prever. Porém, o que acontecerá se formos pegos de surpresa? Se não estivermos preparados para uma surpresa – para o fato de a nossa visão esperada do mundo estar errada – acabaremos com um sistema frágil. À medida que nos aproximarmos dos limites com os quais esperamos que nosso sistema saiba lidar, a situação começará a se complicar – não seremos capazes de ter um desempenho adequado.

Organizações mais horizontais – em que a responsabilidade está distribuída pela empresa, em vez de estar centralizada – geralmente estarão mais bem preparadas para lidar com a surpresa. Quando o inesperado ocorrer, se as pessoas tiverem restrições quanto àquilo que devem fazer, ou se tiverem de obedecer a um conjunto estrito de regras, sua capacidade de lidar com a surpresa estará severamente limitada.

Muitas vezes, na tentativa de otimizar nosso sistema, poderemos aumentar a sua fragilidade, como um efeito colateral indesejado. Considere a automação como exemplo. A automação é incrível – ela nos permite fazer mais com as pessoas que temos, mas também nos permite *reduzir* a quantidade de pessoas que temos, pois mais tarefas poderão ser feitas com a automação. Contudo, essa redução no quadro de funcionários pode ser preocupante. A automação não é capaz de lidar com surpresas – nossa capacidade de estender nosso sistema de modo elegante para lidar com surpresas é uma consequência de termos pessoas à disposição com as habilidades, a experiência e a responsabilidade apropriadas para lidar com essas situações quando elas surgirem.

Adaptabilidade sustentável

Ter uma adaptabilidade sustentável exige que não sejamos complacentes. Segundo David Woods: “Não importa quão bom tenhamos sido antes, não importa quão bem-sucedidos fomos, o futuro pode ser diferente, e pode ser que não estejamos bem adaptados. Talvez sejamos insuficientes e frágeis diante desse novo futuro”.² O fato de ainda não termos sofrido uma indisponibilidade catastrófica de sistema não significa que ela não possa ocorrer. Precisamos nos desafiar para garantir que estejamos constantemente adaptando o que fazemos como empresa a fim de garantir que tenhamos resiliência no futuro. Se feito de forma correta, um conceito como engenharia de caos (chaos engineering) – que exploraremos rapidamente, mais adiante neste capítulo – pode ser uma ferramenta útil para ajudar a promover uma adaptabilidade sustentável.

Uma adaptabilidade sustentável em geral exige uma visão mais holística do sistema. De modo paradoxal, é nesse caso que uma tendência a ter equipes menores e mais autônomas, com mais responsabilidades locais e específicas, pode acabar nos levando a uma perda da visão do panorama mais amplo. Conforme exploraremos no Capítulo 15, deve haver um equilíbrio entre otimizações globais e locais quando se trata da dinâmica organizacional, e esse equilíbrio não é estático. Nesse capítulo, veremos o papel desempenhado pelas equipes alinhadas a fluxos (stream-aligned teams), com foco específico, responsáveis pelos microsserviços que oferecem funcionalidades voltadas aos usuários, e que têm níveis cada vez maiores de responsabilidade para fazer isso acontecer. Veremos também o papel das equipes de capacitação (enabling teams), que auxiliam essas equipes alinhadas a fluxo em seu trabalho, e como as equipes de capacitação podem desempenhar uma função importante para que haja uma adaptabilidade sustentável no nível organizacional.

Criar uma cultura que priorize um ambiente no qual as pessoas possam compartilhar informações livremente, sem medo de retaliações, é essencial para incentivar o aprendizado na ocorrência de um incidente. Ter a oportunidade para realmente analisar essas surpresas e extrair os aprendizados mais importantes exige tempo, energia e pessoas – tudo isso reduzirá os recursos disponíveis para o lançamento de funcionalidades no curto prazo. Decidir pela adoção da adaptabilidade sustentável

parcialmente diz respeito a encontrar um ponto de equilíbrio entre a entrega no curto prazo e a adaptabilidade no longo prazo.

Trabalhar em direção a uma adaptabilidade sustentável significa que você está tentando descobrir o que você não sabe. Isso exige um investimento contínuo, em vez de serem atividades transacionais, que ocorrem uma só vez – o termo *sustentável*, nesse caso, é importante. Trata-se de fazer da adaptabilidade sustentável uma parte fundamental da estratégia e da cultura de sua empresa.

E a arquitetura de microsserviços

Conforme já discutimos, podemos notar que uma arquitetura de microsserviços pode nos ajudar a alcançar a propriedade da robustez, mas ela não será suficiente se quisermos ter *resiliência*.

Falando de modo mais amplo, a capacidade de oferecer resiliência é uma propriedade não do software propriamente dito, mas das pessoas que criam e executam o sistema. Dado o foco deste livro, boa parte do que será apresentado a seguir neste capítulo se concentrará principalmente naquilo que uma arquitetura de microsserviços pode oferecer no que diz respeito à resiliência – e isso está quase totalmente limitado a melhorar a *robustez* das aplicações.

As falhas estão em todos os lugares

Sabemos que falhas podem acontecer. Discos rígidos podem falhar. Nosso software pode parar de funcionar. E, como qualquer um que já tenha lido sobre as falácias da computação distribuída (<https://oreil.ly/aYIjx>) poderia dizer, a rede não é confiável. Podemos nos esforçar ao máximo para tentar limitar as causas das falhas, mas, em certa escala, elas serão inevitáveis. Os discos rígidos, por exemplo, são mais confiáveis hoje em dia do que jamais foram no passado, mas, ocasionalmente, podem falhar. Quanto mais discos rígidos você tiver, maior será a probabilidade de falha em uma unidade específica em um dia qualquer; em larga escala, uma falha passa a ser uma certeza estatística.

Mesmo para aqueles de nós que não pensamos em uma escala radical, se

podermos aceitar a possibilidade de falha, estaremos em uma situação muito melhor. Por exemplo, se pudermos lidar com a falha de um microsserviço de forma elegante, segue-se daí que também poderemos fazer upgrades de um serviço em funcionamento, pois será muito mais fácil lidar com uma indisponibilidade planejada do que com uma não planejada.

Também podemos gastar um pouco menos de tempo tentando impedir o inevitável, e um pouco mais para lidar com a situação de forma elegante. Fico surpreso com o número de empresas que criam processos e controles na tentativa de impedir uma falha, mas pensam muito pouco, ou não pensam, antes de tudo, em facilitar uma recuperação no caso de falha. Saber o que pode provavelmente falhar é essencial para aumentar a *robustez* do nosso sistema.

Partir do pressuposto de que tudo pode e vai falhar nos leva a pensar de modo diferente na resolução dos problemas. Você se lembra da história sobre os servidores do Google que vimos no Capítulo 10? Os sistemas do Google foram criados de modo que, caso uma máquina falhe, isso não levaria a uma interrupção do serviço – melhorando a *robustez* do sistema como um todo. O Google vai além na tentativa de melhorar a robustez de seus servidores de outras formas – a empresa vem discutindo o fato de cada servidor conter o seu próprio suprimento local de energia para garantir que possa continuar funcionando caso o data center tenha alguma indisponibilidade de serviço.³ Conforme você deve se lembrar com base no que vimos no Capítulo 10, os discos rígidos nesses servidores ficavam presos com velcro em vez de estarem parafusados, para facilitar a sua substituição – ajudando o Google a deixar a máquina pronta para executar rapidamente no caso de uma falha no drive, o que, por sua vez, ajuda esse componente do sistema a se *recuperar* com mais eficácia.

Então, deixe que eu repita: em larga escala, mesmo que você compre o melhor dos kits, o mais caro dos hardwares, não será possível evitar o fato de que algo pode e vai falhar. Assim, é preciso partir do pressuposto de que falhas podem ocorrer. Se você incluir esse raciocínio em tudo que fizer e se planejar para as falhas, poderá fazer avaliações de custo-benefício bem

fundamentadas. Se souber que seu sistema é capaz de lidar com o fato de que um servidor pode e vai falhar, talvez haja retornos cada vez menores por gastar mais dinheiro em máquinas individuais. Em vez disso, ter um número maior de máquinas mais baratas (talvez usando componentes mais baratos e um pouco de velcro!) como fez o Google poderia fazer muito mais sentido.

Quando é demais?

Mencionamos o tópico acerca dos requisitos multifuncionais no Capítulo 9. Conhecer os requisitos multifuncionais diz respeito totalmente a considerar aspectos como durabilidade dos dados, disponibilidade dos serviços, throughput e latência aceitável das operações. Muitas das técnicas discutidas neste capítulo falam de abordagens para implementar esses requisitos, mas somente você saberá exatamente quais são esses requisitos em questão. Portanto tenha seus próprios requisitos em mente durante a leitura.

Ter um sistema com escalabilidade automática, capaz de reagir a um aumento de carga ou à falha de nós individuais pode ser incrível, porém seria um exagero para um sistema de relatórios que precisasse executar somente duas vezes por mês, para o qual estar inativo por um ou dois dias não seria um problema tão grave. De modo semelhante, descobrir como fazer implantações sem downtime para evitar interrupção de serviço pode fazer sentido para o seu sistema de comércio eletrônico online, mas para sua base de conhecimento corporativo na intranet, provavelmente seria um passo um pouco exagerado.

O nível de falhas que você pode tolerar ou a rapidez que seu sistema deve ter são determinados pelos usuários desse sistema. Essas informações, por sua vez, ajudam a compreender quais técnicas farão mais sentido para você. Apesar disso, seus usuários nem sempre serão capazes de explicar quais são, exatamente, os seus requisitos. Assim, será necessário fazer perguntas para ajudar a extrair as informações corretas e ajudá-los a entender os custos relativos de se oferecer diferentes níveis de serviço.

Conforme mencionei antes, esses requisitos multifuncionais podem variar

de serviço para serviço, mas eu sugeriria definir alguns requisitos multifuncionais genéricos, e então substituí-los para casos de uso específicos. Quando se trata de considerar se, e como, escalar seu sistema para lidar melhor com a carga ou com falhas, comece tentando entender os seguintes requisitos:

Tempo de resposta/latência

Quanto tempo as diversas operações devem demorar? Pode ser conveniente fazer essas medições com diferentes quantidades de usuários a fim de entender como o aumento da carga causará impacto no tempo de resposta. Dada a natureza das redes, sempre haverá exceções, portanto definir metas para um dado percentil das respostas monitoradas pode ser conveniente. A meta também deve incluir o número de conexões/usuários concorrentes que você espera que seu software irá tratar. Assim, poderíamos dizer algo como: “Esperamos que o site tenha um tempo de resposta de dois segundos no percentil 90 ao lidar com 200 conexões concorrentes por segundo”.

Disponibilidade

Podemos esperar que um serviço fique inativo? Esse é um serviço considerado 24/7? Algumas pessoas gostam de ter períodos aceitáveis de downtime quando avaliam a disponibilidade, mas quão útil será isso para alguém que chame o seu serviço? Eu deveria poder contar com o fato de que seu serviço vai responder ou não. Medir períodos de downtime será realmente mais útil do ponto de vista de informações históricas.

Durabilidade dos dados

Qual é o nível aceitável de perda de dados? Por quanto tempo os dados devem ser mantidos? É extremamente provável que isso mude de acordo com cada caso. Por exemplo, você poderia optar por manter os logs das sessões de usuários por um ano ou menos a fim de economizar espaço, porém os registros de suas transações financeiras talvez precisem ser mantidos por vários anos.

Partir dessas ideias e combiná-las com os SLOs (Service-Level Objectives, ou Objetivos de Nível de Serviço), que discutimos no Capítulo 10, pode ser uma boa forma de incluir esses requisitos como uma parte essencial de seu

processo de entrega de software.

Degradação das funcionalidades

Uma parte essencial da construção de um sistema resiliente, particularmente quando suas funcionalidades estão espalhadas por vários microsserviços diferentes que podem estar ativos ou não, é a capacidade de fazer as funcionalidades se degradarem com segurança. Vamos pensar em uma página web padrão em nosso site de comércio eletrônico. Para compor as várias partes desse site, podemos precisar de vários microsserviços desempenhando uma função. Um microsserviço poderia exibir os detalhes do item à venda. Outro poderia mostrar o preço e o nível de estoque. E provavelmente mostraremos o conteúdo do carrinho de compras também, o que poderia ser feito por outro microsserviço distinto. Se um desses serviços estiver inativo, e isso resultar na indisponibilidade da página web como um todo, certamente teríamos criado um sistema que é menos resiliente do que um que exigisse que apenas um serviço estivesse disponível.

O que precisamos fazer é entender o impacto de cada indisponibilidade de serviço e descobrir como fazer as funcionalidades se degradarem de modo apropriado. Do ponto de vista dos negócios, queremos que nosso fluxo de recebimento de pedidos seja tão robusto quanto possível, e poderíamos ficar satisfeitos em aceitar certo nível de degradação das funcionalidades para garantir que elas continuem funcionando. Se os níveis de estoque estiverem indisponíveis, poderíamos tomar a decisão de prosseguir com a venda e descobrir os detalhes mais tarde. Se o microsserviço do carrinho de compras estiver indisponível, provavelmente teremos um problema sério, mas ainda poderíamos exibir a página web com o produto. Talvez simplesmente ocultássemos o carrinho de compras ou o substituíssemos por um ícone no qual se leria “Voltaremos em breve!”.

Com uma aplicação monolítica com um só processo, não temos muitas decisões para tomar. A saúde do sistema, até certo ponto, é binária nesse contexto – o processo está ativo ou inativo. Porém, em uma arquitetura de microsserviços, é preciso considerar uma situação com muito mais

nuances. A decisão correta em qualquer situação em geral não será uma decisão técnica. Podemos saber o que é tecnicamente possível fazer se o carrinho de compras estiver inativo, mas, a menos que tenhamos conhecimento do contexto de negócios, não saberemos qual é a atitude que devemos tomar. Por exemplo, talvez fechemos totalmente o site, permitimos que as pessoas naveguem pelo catálogo de produtos ou substituimos a parte da UI que contém o controle do carrinho de compras por um número de telefone para fazer um pedido. No entanto, para cada interface voltada ao cliente que utilize vários microsserviços, ou para cada microsserviço que dependa de vários colaboradores downstream, será necessário perguntar a si mesmo: “O que acontecerá se isso estiver inativo?” – e saber o que deve ser feito.

Ao pensar na importância de cada uma de nossas funcionalidades no que diz respeito aos requisitos multifuncionais, estaremos em uma posição muito melhor para saber o que pode ser feito. Vamos agora considerar algumas tarefas que podemos fazer do ponto de vista técnico para garantir que, se houver uma falha, seremos capazes de lidar com ela de forma elegante.

Padrões de estabilidade

Há alguns padrões que podemos usar para garantir que, se algo de errado acontecer, isso não causará terríveis efeitos em cascata. É muito importante conhecer essas ideias, e você deve considerar fortemente o uso delas em seu sistema para garantir que um componente ruim não faça o mundo todo ruir ao seu redor. Em breve, veremos algumas medidas de segurança essenciais que devem ser consideradas; antes disso, porém, gostaria de apresentar uma pequena história para ilustrar o tipo de problema que pode ocorrer.

Muitos anos atrás, eu era líder técnico de um projeto na AdvertCorp. A AdvertCorp (o nome da empresa e os detalhes foram alterados para proteger os inocentes!) oferecia anúncios de classificados online por meio de um site muito conhecido. O site em questão lidava com volumes razoavelmente altos de anúncios e gerava bons lucros para a empresa. O

projeto no qual eu estava trabalhando foi incumbido de consolidar uma série de serviços existentes, usados para disponibilizar funcionalidades similares a diferentes tipos de anúncios. As funcionalidades para os diferentes tipos de anúncios estavam sendo lentamente migradas para o novo sistema que estávamos criando, com vários tipos diferentes de anúncios ainda atendidos por serviços mais antigos. Para deixar essa transição transparente para o cliente final, interceptávamos todas as chamadas para os diferentes tipos de anúncios em nosso novo sistema, desviando-as para os sistemas antigos quando necessário, conforme representado na Figura 12.1. Na verdade, esse é um exemplo do *padrão strangler fig*, discutido rapidamente na seção “Padrões convenientes para decomposição”, na página 107.

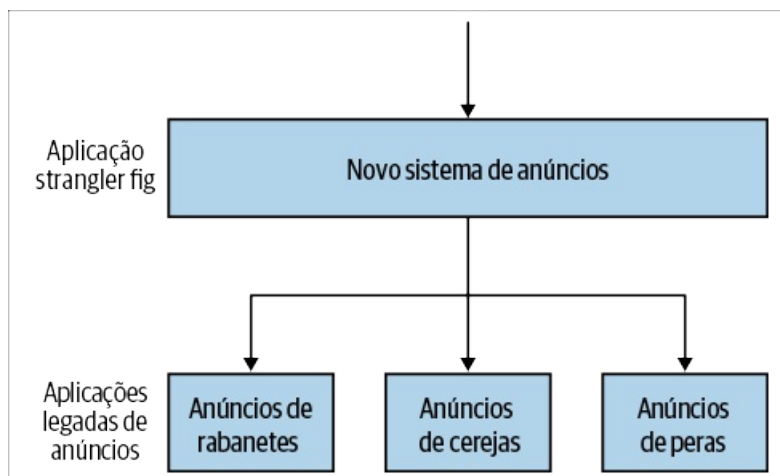


Figura 12.1 – Um padrão *strangler fig* usado para direcionar chamadas para sistemas legados mais antigos.

Havíamos acabado de passar o produto mais lucrativo, que lidava com o maior volume de anúncios, para o novo sistema, mas boa parte dos demais anúncios ainda estava sendo tratada por diversas aplicações mais antigas. Havia várias aplicações pouco lucrativas, com um número baixo de buscas – muitas dessas aplicações mais antigas recebiam um pequeno volume de tráfego e tinham um baixo faturamento. O novo sistema estava ativo há um tempo e estava se comportando muito bem, lidando com uma carga que não era insignificante. Naquela época, devíamos estar lidando com aproximadamente 6.000 a 7.000 requisições por segundo no horário de pico e, embora a maior parte delas fosse comumente armazenada em cache

por proxies reversos que ficavam na frente de nossos servidores de aplicações, as buscas pelos produtos (o aspecto mais importante de nosso site), em sua maior parte, não ficavam em cache, e exigiam um acesso completo de ida e volta ao servidor.

Certa manhã, pouco antes de atingirmos nosso horário de pico diário no almoço, o sistema passou a ficar lento, e então começou a falhar. Tínhamos certo nível de monitoração em nossa nova aplicação principal, suficiente para nos informar que cada um dos nós de nossa aplicação estava atingindo um pico de 100% de CPU, muito acima dos níveis usuais, até mesmo no horário de pico. Em pouco tempo, o site todo ficou inativo.

Conseguimos descobrir o culpado e trazer o site de volta à vida. O fato é que havia sido um dos sistemas de anúncios downstream que, para esse estudo de caso anônimo, diremos que era o responsável pelos anúncios relacionados aos rabanetes. O serviço de anúncios de rabanetes, um dos serviços mais antigos e com menos manutenção, havia começado a responder de modo muito lento. Responder de forma lenta é um dos piores modos de falha que podemos ter. Se um sistema simplesmente não está presente, descobrimos rapidamente. Se estiver apenas *lento*, acabaremos esperando um tempo antes de desistir – esse processo de espera pode deixar todo o sistema lento, provocar contenção de recursos (resource contention) e, como aconteceu em nosso caso, resultar em uma falha em cascata. No entanto, qualquer que tenha sido a causa da falha, havíamos criado um sistema que era vulnerável a um problema em um serviço downstream, capaz de provocar uma falha em cascata que atingiu o sistema como um todo. Um serviço downstream, sobre o qual tínhamos pouco controle, foi capaz de deixar nosso sistema como um todo inativo.

Enquanto uma equipe se encarregava dos problemas no sistema de rabanetes, nós começamos a procurar o que havia de errado em nossa aplicação. Encontramos alguns problemas, que estão representados na Figura 12.2. Estávamos usando um pool de conexões HTTP para lidar com nossas conexões downstream. As threads no pool em questão tinham timeouts configurados para o tempo que poderiam esperar quando fizessem a chamada HTTP downstream, o que era bom. O problema era

que demorava um tempo para que houvesse um timeout nos workers por causa do serviço downstream lento. Enquanto esperavam, mais requisições chegavam ao pool, solicitando threads workers. Sem workers disponíveis, essas requisições ficavam travadas. O que aconteceu é que a biblioteca de pool de conexões que estávamos usando tinha um timeout para workers em espera, mas ele estava *desativado por default*! Isso levou a um acúmulo enorme de threads bloqueadas. Nossa aplicação em geral tinha 40 conexões concorrentes em qualquer dado instante. No intervalo de cinco minutos, essa situação fez com que um pico de aproximadamente 800 conexões fosse atingido, derrubando o sistema.

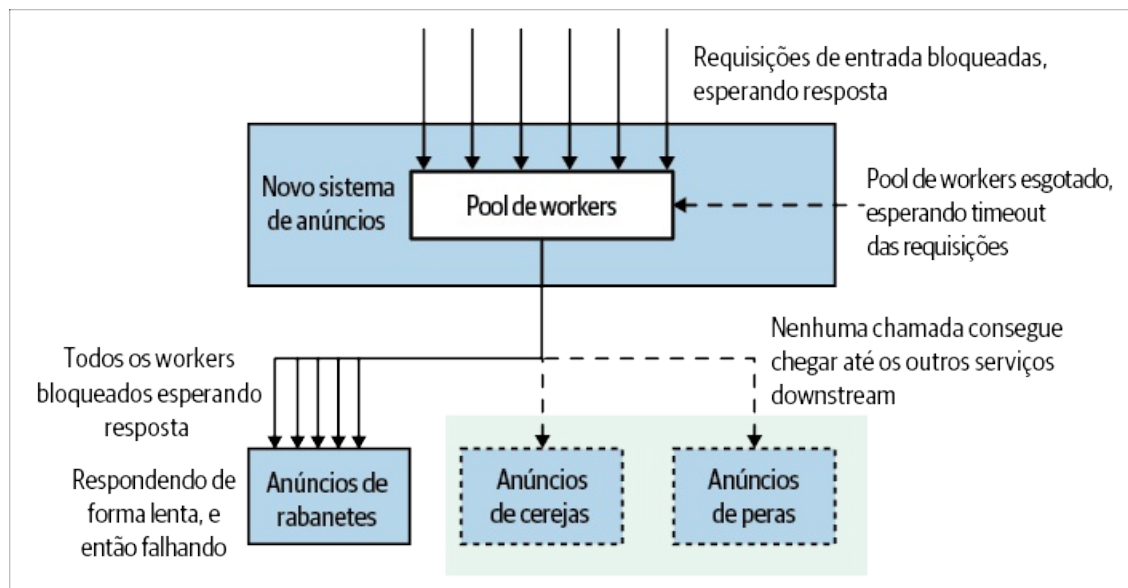


Figura 12.2 – Uma representação dos problemas que causaram a indisponibilidade do sistema.

O pior era que o serviço downstream com o qual estávamos conversando representava uma funcionalidade que menos de 5% de nossa base de clientes utilizava, e gerava um lucro menor ainda. Ao analisar com mais detalhes, descobrimos, do modo difícil, que é *muito* mais complicado lidar com sistemas que simplesmente se tornam mais lentos do que com sistemas que falham rapidamente. Em um sistema distribuído, a latência mata.

Mesmo que tivéssemos os timeouts definidos corretamente no pool, ainda estávamos compartilhando um único pool de conexões HTTP para todas

as requisições de saída. Isso implicava que um único serviço downstream lento poderia sozinho fazer com que o número de workers disponíveis se esgotasse, ainda que tudo o mais estivesse saudável. Por fim, em virtude dos timeouts e erros frequentes, estava claro que o serviço downstream em questão não estava saudável; apesar disso, continuávamos enviando tráfego em sua direção. Em nossa situação, isso significava que estávamos piorando ainda mais uma situação que já era ruim, pois o serviço downstream não tinha chances de se recuperar. Acabamos implementando três correções para evitar que isso ocorresse novamente: definimos nossos *time-outs* corretamente, implementamos *bulkheads* para separar os diferentes pools de conexão e implementamos um *circuit breaker* (interruptor de circuito, ou disjuntor) para, antes de tudo, evitar o envio de chamadas para um sistema que não estivesse saudável.

Timeouts

É fácil se esquecer dos timeouts, mas, em um sistema distribuído, é importante que sejam definidos corretamente. Quanto tempo posso esperar antes de desistir de uma chamada para um serviço downstream? Se esperarmos demais para decidir que a chamada falhou, o sistema todo poderá ficar lento. Se o timeout ocorrer rápido demais, consideraremos que uma chamada que poderia ter funcionado falhou. Não tenha nenhum timeout e um serviço downstream que ficar inativo poderá deixar todo o seu sistema travado.

No caso da AdvertCorp, tínhamos dois problemas relacionados a timeout. Em primeiro lugar, não tínhamos um timeout no pool de requisições HTTP, o que implicava que, quando pedíamos um worker para fazer uma requisição HTTP downstream, a thread de requisição ficava bloqueada por tempo indeterminado, até que um worker ficasse disponível. Em segundo, quando finalmente tínhamos um worker HTTP disponível para fazer uma requisição ao sistema de anúncios de rabanetes, esperávamos tempo demais para desistir da chamada. Assim, como mostra a Figura 12.3, tivemos de adicionar um novo timeout e alterar o timeout existente.

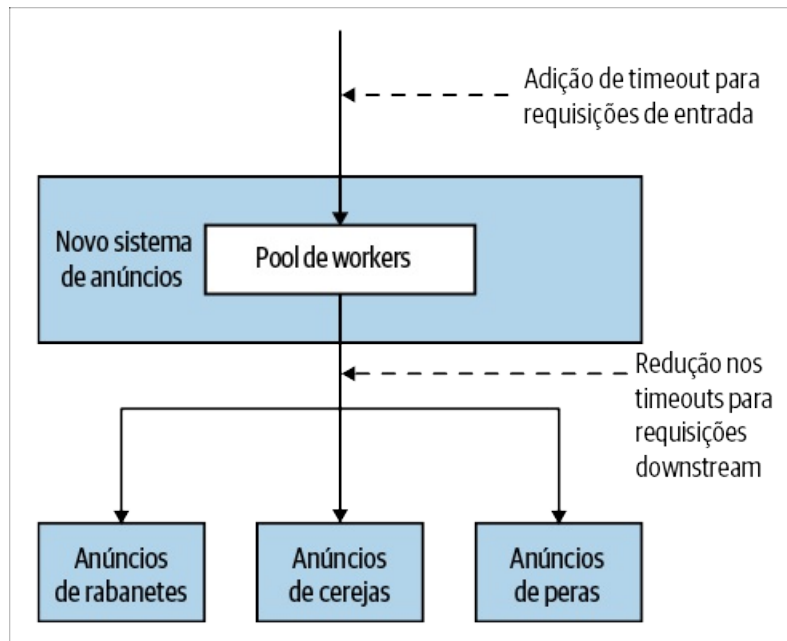


Figura 12.3 – Mudando os timeouts no sistema da AdvertCorp.

Os timeouts para as requisições HTTP downstream haviam sido definidos para 30 segundos – portanto, esperávamos 30 segundos por uma resposta do sistema de rabanetes antes de desistir. O problema é que, no contexto mais amplo no qual essa chamada era feita, esperar todo esse tempo não fazia sentido. Os anúncios relacionados aos rabanetes eram requisitados como resultado de um acesso de nossos usuários ao site usando um navegador. Mesmo naquela época, quando isso acontecia, ninguém esperava 30 segundos para uma página ser carregada. Pense no que acontecerá se uma página web não for carregada depois de 5, 10 ou, talvez, 15 segundos. O que você faria? Você atualizaria a página. Assim, estávamos esperando 30 segundos para que o sistema de anúncios de rabanetes respondesse, mas, muito antes disso, a requisição original já não era mais válida, pois o usuário teria atualizado a página, gerando uma nova requisição de entrada. Isso, por sua vez, fazia com que outra requisição chegasse ao sistema de anúncios, e assim sucessivamente.

Ao observar o comportamento usual do sistema de anúncios de rabanetes, podíamos notar que, em geral, esperávamos uma resposta em muito menos de um segundo, portanto esperar 30 segundos era um exagero. Além do mais, tínhamos como meta apresentar uma página ao usuário no intervalo de 4 a 6 segundos. Com base nisso, deixamos o timeout muito mais

agressivo, configurando-o com um segundo. Também criamos um timeout de um segundo para esperar que um worker HTTP estivesse disponível. Isso significava que, no pior caso, poderíamos esperar aproximadamente dois segundos pelas informações do sistema de rabanetes.



Os timeouts são extremamente úteis. Insira timeouts em todas as chamadas externas ao processo, e escolha um timeout default para tudo. Registre no log a ocorrência dos timeouts, observe o que acontece e faça as devidas alterações nesses timeouts. Observe os tempos de resposta saudáveis, considerados “normais”, para seus serviços downstream, e utilize essas informações para orientar sua definição dos valores dos timeouts.

Definir um timeout para uma única chamada de serviço talvez não seja suficiente. O que acontecerá se esse timeout ocorrer como parte de um conjunto maior de operações do qual você possa querer desistir até mesmo antes de o timeout ocorrer? No caso da AdvertCorp, por exemplo, não faz sentido esperar pelos preços mais recentes dos rabanetes se houver uma boa chance de o usuário já ter desistido de saber. Em uma situação como essa, faz sentido ter um timeout para a operação geral e desistir caso esse timeout tenha ocorrido. Para que isso funcione, o tempo que resta para a operação deverá ser passado adiante. Por exemplo, se a operação geral de apresentar uma página tivesse de ser concluída em 1.000 ms e, quando fizéssemos a chamada para o serviço de anúncio de rabanetes downstream, 300 ms já tivessem se passado, teríamos então de garantir que não esperaríamos mais do que 700 ms para que a parte restante das chamadas fosse concluída.



Não pense no timeout de uma única chamada de serviço; pense também em um timeout para a operação como um todo e cancele a operação se esse timeout geral se esgotar.

Novas tentativas

Alguns problemas com chamadas downstream são temporários. Pacotes podem ser enviados de forma indevida, ou os gateways podem ter um pico anormal de carga, causando um timeout. Muitas vezes, fazer novas

tentativas para uma chamada faz bastante sentido. Retornando ao assunto que acabamos de discutir, com que frequência você atualizou uma página web que não foi carregada, somente para descobrir que a segunda tentativa funcionou bem? Essa é uma ação em que uma nova tentativa foi feita.

Seria conveniente considerar os tipos de falhas em chamadas downstream que justifiquem novas tentativas. Se um protocolo como o HTTP estiver em uso, por exemplo, algumas informações úteis podem ser recebidas nos códigos de resposta, as quais poderão ajudar a determinar se uma nova tentativa se justifica. Se um `404 Not Found` (Não encontrado) for recebido como resposta, é improvável que uma nova tentativa seja uma boa ideia. Por outro lado, um `503 Service Unavailable` (Serviço indisponível) ou um `504 Gateway Time-out` (Timeout do gateway) poderiam ser considerados erros temporários e justificariam uma nova tentativa.

É provável que você vá precisar esperar um pouco antes de tentar novamente. Se o timeout ou o erro inicial forem causados pelo fato de o microsserviço downstream estar com muita carga, bombardeá-lo com requisições adicionais certamente seria uma péssima ideia.

Se você pretende fazer uma nova tentativa, é preciso levar isso em consideração ao pensar no limiar de seu timeout. Se o limiar do timeout para uma chamada downstream for definido com 500 ms, mas até três novas tentativas forem permitidas, com um segundo entre cada tentativa, você poderá acabar esperando até 3,5 segundos antes de desistir. Conforme já mencionei, ter uma previsão para o tempo que uma operação pode demorar pode ser uma boa ideia – talvez você decida que não fará a terceira tentativa (ou nem mesmo a segunda) caso já tenha excedido o limite do timeout geral. Por outro lado, se isso acontecer como parte de uma operação que não está voltada diretamente ao usuário, esperar mais para que algo seja feito poderia ser totalmente aceitável.

Bulkheads

No livro *Release It!*,⁴ Michael Nygard apresenta o conceito de *bulkhead* (antepara) como uma forma de se isolar de uma falha. Na navegação, um *bulkhead*, isto é, uma antepara, é a parte do navio que pode ser isolada

para proteger a parte restante da embarcação. Assim, se o navio apresentar um vazamento, podemos fechar as portas das anteparas. Você perderá parte do navio, mas a parte restante permanecerá intacta.

No que concerne a uma arquitetura de software, há vários tipos diferentes de bulkheads que podemos considerar. Voltando à minha própria experiência com a AdvertCorp, na verdade nós perdemos a oportunidade de implementar um bulkhead no caso das chamadas downstream. Deveríamos ter usado diferentes pools de conexão para cada conexão downstream. Dessa forma, se um pool de conexões se esgotasse, as outras conexões não seriam afetadas, como podemos ver na Figura 12.4.

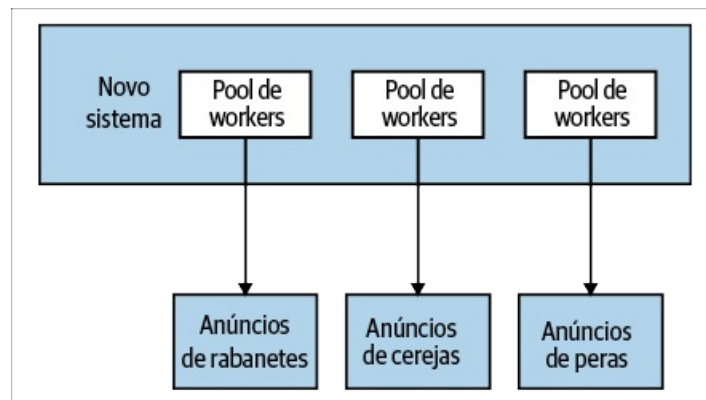


Figura 12.4 – Usando um pool de conexões para cada serviço downstream para uma solução com bulkheads.

A separação de responsabilidades também pode ser uma forma de implementar bulkheads. Ao separar as funcionalidades em microserviços distintos, reduzimos as chances de uma falha em uma área afetar outra.

Observe todos os aspectos de seu sistema que podem dar errado, tanto dentro de seus microserviços como entre eles. Você tem bulkheads implementados? Eu sugeriria que, no mínimo, você comece com pools de conexão separados para cada conexão downstream. No entanto, talvez você queira ir além e considerar também o uso de circuit breakers (interruptores de circuito), que veremos em breve.

Em vários aspectos, os bulkheads são o padrão mais importante que vimos até agora. Timeouts e circuit breakers ajudam a liberar recursos quando esses começam a ficar limitados, mas os bulkheads, antes de tudo, podem

garantir que eles não fiquem limitados. Os bulkheads também permitem rejeitar requisições em determinadas condições a fim de garantir que os recursos não fiquem mais saturados ainda; isso é conhecido como *load shedding* (limitação de carga). Às vezes, rejeitar uma requisição é a melhor maneira de impedir que um sistema importante fique sobrecarregado e se torne um gargalo para vários serviços upstream.

Circuit breakers

Em sua própria casa, há circuit breakers (interruptores de circuito, ou disjuntores) para proteger seus equipamentos eletrônicos contra picos de energia. Se houver um pico, o disjuntor é acionado, protegendo seus eletrodomésticos valiosos. Você também pode desligar manualmente um disjuntor para cortar a energia em parte de sua residência, permitindo trabalhar com segurança no sistema elétrico. Em outro padrão do livro *Release It!*, Nygard mostra como a mesma ideia pode fazer maravilhas como um método de proteção em nosso software.

Podemos pensar em nossos circuit breakers como um método automático para fechar um bulkhead, com o intuito não só de proteger o consumidor contra o problema downstream, mas também para, possivelmente, proteger o serviço downstream de receber mais chamadas que possam causar um impacto negativo. Considerando os perigos de uma falha em cascata, eu recomendaria circuit breakers obrigatórios para todas as chamadas síncronas downstream. No entanto, você não precisa escrever seus próprios circuit breakers – nos anos que se passaram desde a primeira edição deste livro, muitas implementações de circuit breakers se tornaram amplamente disponíveis.

Voltando à AdvertCorp, considere o problema que tínhamos com o sistema de rabanetes respondendo muito lentamente, até devolver um erro em algum momento. Ainda que nossos timeouts estivessem definidos de forma correta, esperaríamos um bom tempo até receber o erro. E então tentaríamos novamente da próxima vez que uma requisição chegasse, e esperaríamos. Já era ruim que o serviço downstream não estivesse funcionando bem e, além disso, ele também estava deixando a parte

restante do sistema lenta.

Com um circuit breaker, após determinado número de requisições ao recurso downstream ter falhado (em virtude de um erro ou de um timeout), o circuit breaker seria acionado. Todas as requisições subsequentes que passarem por esse circuit breaker falharão rapidamente enquanto ele estiver no estado acionado (aberto),⁵ como podemos ver na Figura 12.5. Após determinado tempo, o cliente enviará algumas requisições para ver se o serviço downstream se recuperou, e se receber um número suficiente de respostas saudáveis, o circuit breaker será reiniciado.

O modo de implementar um circuit breaker dependerá do que significa uma requisição “com falha”, mas, quando eu o implemento para conexões HTTP, considero que uma falha quer dizer que houve um timeout ou que um subconjunto dos códigos de resposta HTTP 5XX foi recebido. Dessa forma, quando ocorre um timeout em um recurso downstream ou um erro é devolvido, depois que um certo limiar é alcançado, paramos automaticamente de enviar tráfego e começamos a falhar rapidamente. E podemos reiniciar automaticamente quando o serviço estiver saudável novamente.

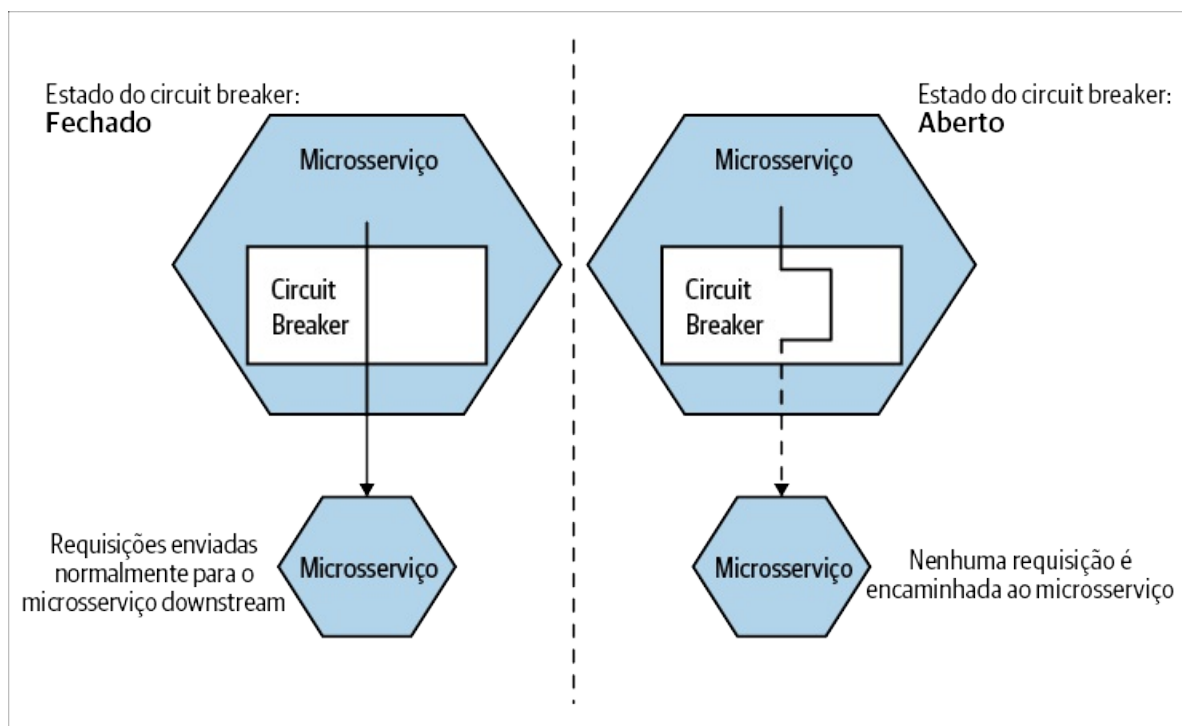


Figura 12.5 – Visão geral dos circuit breakers.

Ter a configuração correta pode ser um pouco complicado. Não queremos que o circuito seja acionado de modo rápido demais, nem queremos demorar muito. De modo semelhante, queremos realmente garantir que o serviço downstream esteja saudável novamente para voltar a enviar tráfego. Como ocorre com os timeouts, eu escolheria alguns valores default razoáveis e os utilizaria em todos os lugares, e então os alteraria para casos específicos.

Quando o circuit breaker é acionado, temos algumas opções. Uma delas é enfileirar as requisições e fazer novas tentativas mais tarde. Para alguns casos de uso, essa solução poderia ser apropriada, sobretudo se você estiver fazendo algum trabalho como parte de uma tarefa assíncrona. Contudo, se essa chamada for feita como parte de uma cadeia de chamadas síncronas, provavelmente será melhor falhar com mais rapidez. Isso poderia significar propagar um erro para cima na cadeia de chamadas, ou uma degradação mais sutil da funcionalidade.

No caso da AdvertCorp, encapsulamos as chamadas downstream aos sistemas legados com circuit breakers, como mostra a Figura 12.6. Quando esses circuit breakers eram acionados, atualizávamos o site por meio de código, mostrando que, no momento, não era possível exibir anúncios, por exemplo, de rabanetes. Mantínhamos a parte restante do site em funcionamento e informávamos claramente aos clientes que havia um problema restrito a uma parte de nosso produto, tudo de modo totalmente automatizado.

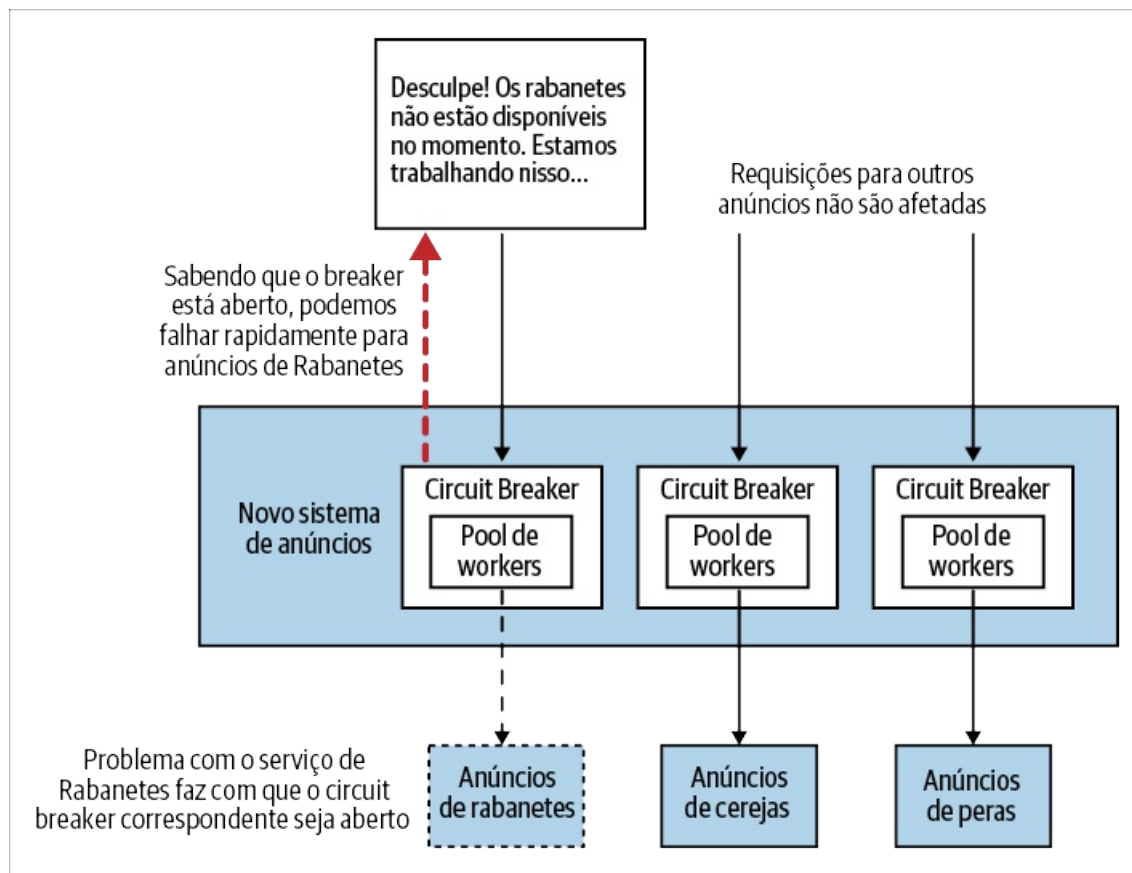


Figura 12.6 – Acrescentando circuit breakers na AdvertCorp.

Pudemos definir o escopo de nossos circuit breakers de modo que tivéssemos um para cada sistema downstream legado – isso se alinhava muito bem à nossa decisão de ter diferentes pools de workers para requisições para cada serviço downstream.

Se tivermos essa solução implementada (como ocorre com os disjuntores em nossa residência), poderíamos acioná-la manualmente para deixar nosso trabalho mais seguro. Por exemplo, se quiséssemos desativar um microserviço como parte de uma manutenção de rotina, poderíamos abrir manualmente todos os circuit breakers dos consumidores upstream de modo que falhassem rapidamente enquanto o microserviço estivesse offline. Assim que ele retornasse, poderíamos fechar os circuit breakers, e tudo voltaria ao normal novamente. Criar scripts para o processo de abrir e fechar manualmente um circuit breaker como parte de um processo automatizado de implantação poderia ser um próximo passo razoável.

Os circuit breakers ajudam nossa aplicação a falhar rapidamente – e falhar

rápido é sempre melhor do que demorar para falhar. Os circuit breakers nos permitem falhar antes de desperdiçar um tempo (e recursos) precioso à espera de um microsserviço downstream não saudável responder. Em vez de esperar para falhar até tentarmos usar o microsserviço downstream, poderíamos verificar antes o status de nossos circuit breakers. Se um microsserviço do qual dependemos como parte de uma operação estiver indisponível no momento, poderemos cancelar a operação antes mesmo que ela seja iniciada.

Isolamento

Quanto mais um microsserviço depender da disponibilidade de outro serviço, mais a saúde de um terá impacto na capacidade do outro de fazer o seu trabalho. Se pudermos usar uma tecnologia que permita que um servidor downstream esteja offline – por exemplo, com o uso de um middleware ou de algum outro tipo de sistema de bufferização de chamadas – é menos provável que os microsserviços upstream sejam afetados por indisponibilidade dos microsserviços downstream, sejam elas planejadas ou não.

Há outra vantagem em aumentar o isolamento entre os serviços. Se os serviços estiverem isolados entre si, muito menos trabalho de coordenação será necessário entre os responsáveis pelos serviços. Quanto menos coordenação for necessária entre as equipes, mais autonomia elas terão, pois essas equipes poderão operar e fazer os serviços evoluírem com mais liberdade.

O isolamento também se aplica quando passamos do plano lógico para o físico. Considere dois microsserviços que pareçam estar totalmente isolados um do outro. Eles não se comunicam entre si de forma alguma. Um problema em um deles não deveria causar impacto no outro, certo? Mas e se os dois microsserviços estiverem executando no mesmo host, e um dos microsserviços começar a consumir toda a CPU, fazendo com que esse host tenha problemas?

Considere outro exemplo. Dois microsserviços têm, cada um, o seu próprio banco de dados isolado do ponto de vista lógico. No entanto, os

dois bancos de dados estão implantados na mesma infraestrutura. Uma falha nessa infraestrutura de banco de dados causaria impacto nos dois microsserviços.

Ao considerar o modo como queremos implantar nossos microsserviços, também devemos nos esforçar para garantir que haja certo grau de isolamento nas falhas a fim de evitar problemas como esses. Por exemplo, garantir que os microsserviços executem em hosts independentes, com seu próprio sistema operacional e recursos computacionais isolados, é uma atitude sensata – é isso que conseguimos quando executamos instâncias de microsserviços em sua própria máquina virtual ou contêiner. Esse tipo de isolamento, porém, pode ter um custo.

Podemos isolar nossos microsserviços uns dos outros com mais eficácia se os executarmos em diferentes máquinas. Isso significa que precisaremos de mais infraestrutura, além de ferramentas para gerenciar essa infraestrutura. Essa solução implica um custo direto, além da possibilidade de aumentar a complexidade de nosso sistema, expondo novos pontos de falha. Cada microsserviço poderia ter sua própria infraestrutura totalmente dedicada de banco de dados, mas então teríamos mais infraestrutura para gerenciar. Poderíamos usar um middleware para proporcionar um desacoplamento temporário entre dois microsserviços, mas agora teremos um broker com o qual devemos nos preocupar.

O isolamento, como tantas outras técnicas que vimos, pode ajudar a aumentar a robustez de nossas aplicações, mas é raro que faça isso gratuitamente. Decidir quais são os custos-benefícios aceitáveis entre ter isolamento *versus* o custo e o aumento da complexidade, como ocorre em vários outros casos, é essencial.

Redundância

Ter mais de um item de algo pode ser uma ótima maneira de aumentar a robustez de um componente. Ter mais de uma pessoa que saiba como o banco de dados de produção funciona parece ser sensato caso alguém deixe a empresa ou esteja de licença. Ter mais de uma instância de microsserviço faz sentido, pois permite tolerar a falha de uma dessas

instâncias, ao mesmo tempo que continuamos tendo a chance de disponibilizar a funcionalidade necessária.

Descobrir o nível de redundância de que precisamos, e em quais partes, dependerá do quão bem conhecemos os possíveis modos de falha de cada componente, do impacto causado pela indisponibilidade da funcionalidade e do custo de incluir a redundância.

Na AWS, por exemplo, não temos um SLA para o uptime de uma única instância EC2 (máquina virtual). Temos de trabalhar com o pressuposto de que ela pode e vai falhar. Assim, faz sentido ter mais de uma. Contudo, indo além, as instâncias EC2 são implantadas em zonas de disponibilidade (data centers virtuais), e tampouco temos garantias quanto à disponibilidade de uma única zona, o que significa que vamos querer que essa segunda instância esteja em uma zona de disponibilidade *diferente* para diluir o risco.

Ter mais cópias de algo pode ajudar quando se trata de implementar redundância, mas também pode ser vantajoso para escalar nossas aplicações a fim de lidar com um aumento de carga. No próximo capítulo, apresentaremos exemplos de como os sistemas são escalados, e veremos como escalar com vistas à redundância ou a um aumento de carga podem ser diferentes.

Middleware

Na seção “Brokers de mensagens”, na página 167, vimos o papel de um middleware na forma de brokers de mensagens para ajudar na implementação de interações tanto de requisição-resposta como baseadas em eventos. Uma das propriedades interessantes da maioria dos brokers de mensagens é sua capacidade de oferecer uma garantia de entrega. Enviamos uma mensagem para um componente downstream, e o broker garante que vai entregá-la, com algumas ressalvas que exploramos antes. Internamente, para oferecer essa garantia, o software do broker de mensagens terá de implementar recursos como novas tentativas e timeouts para você – os mesmos tipos de operações que você teria de fazer por conta própria serão executados, porém serão feitos por um software escrito por

especialistas, com foco nesse tipo de tarefa. Ter pessoas inteligentes para fazer o trabalho para você geralmente é uma boa ideia.

No entanto, no caso do nosso exemplo específico com a AdvertCorp, usar um middleware para gerenciar a comunicação de requisição-resposta com o sistema de rabanetes downstream talvez não tivesse ajudado muito. Ainda não iríamos obter respostas para os nossos clientes. A única possível vantagem seria diminuir a contenção de recursos em nosso próprio sistema, mas isso apenas mudaria para um número cada vez maior de requisições pendentes sendo mantidas no broker. Pior ainda, muitas dessas requisições solicitando os preços mais recentes dos rabanetes poderiam estar relacionadas a requisições de usuários que não fossem mais válidas.

Uma alternativa seria inverter a interação e utilizar o middleware para fazer o sistema de rabanetes efetuar um broadcast dos últimos anúncios de rabanetes, e poderíamos então consumi-los. Entretanto, se o sistema de rabanetes downstream tivesse um problema, continuaríamos sendo incapazes de ajudar o cliente que estivesse buscando os melhores preços para os rabanetes.

Assim, usar um middleware como os brokers de mensagens para ajudar a atenuar algumas preocupações com a robustez poderia ser conveniente, mas não em todas as situações.

Idempotência

Nas operações *idempotentes*, o resultado não muda após a primeira aplicação, mesmo que a operação seja subsequentemente aplicada várias vezes. Se as operações forem idempotentes, podemos repetir a chamada várias vezes, sem que haja um impacto inesperado. Isso é muito conveniente quando queremos repetir mensagens para as quais não temos certeza de que foram processadas, e é uma forma comum de recuperação em caso de erros.

Vamos considerar uma chamada simples para somar alguns pontos, como consequência de um de nossos clientes ter feito um pedido. Poderíamos fazer uma chamada com o tipo de payload exibido no Exemplo 12.1.

Exemplo 12.1 – Somando pontos a uma conta


```
<credit>
  <amount>100</amount>
  <forAccount>1234</account>
</credit>
```

Se essa chamada for recebida várias vezes, iríamos acrescentar 100 pontos várias vezes. Da forma como está, portanto, essa chamada não é idempotente. Com um pouco mais de informações, porém, permitimos que o gerenciador de pontos deixe essa chamada idempotente, como mostra o Exemplo 12.2.

Exemplo 12.2 – Acrescentando mais informações no crédito de pontos para deixá-lo idempotente

```
<credit>
  <amount>100</amount>
  <forAccount>1234</account>
  <reason>
    <forPurchase>4567</forPurchase>
  </reason>
</credit>
```

Sabemos que esse crédito está relacionado a um pedido 4567 específico. Supondo que pudéssemos receber somente um crédito para um dado pedido, poderíamos aplicar esse crédito novamente sem que houvesse um aumento no número total de pontos.

Esse método também funciona bem com uma colaboração baseada em eventos, e pode ser particularmente conveniente caso várias instâncias do mesmo tipo de serviço se registrem para receber os eventos. Mesmo que armazenássemos os eventos que foram processados, em algumas formas de entrega de mensagens assíncronas, pode haver pequenos intervalos de tempo nos quais dois workers poderão ver a mesma mensagem. Ao processar os eventos de maneira idempotente, garantimos que não haverá nenhum problema.

Algumas pessoas passam a gostar tanto desse conceito que acham que chamadas subsequentes com os mesmos parâmetros não poderão causar *nenhum* impacto, o que nos deixa em uma posição interessante. Ainda gostaríamos de registrar no log o fato de que uma chamada foi recebida, por exemplo. Queremos registrar o tempo de resposta da chamada e

coletar esses dados para monitoração. O ponto principal, nesse caso, é que é a operação de negócios subjacente que consideramos como idempotente, e não o estado total do sistema.

Alguns dos verbos HTTP, como GET e PUT, são definidos na especificação do HTTP como idempotentes, mas, para que isso aconteça, eles contam com o fato de seu serviço lidar com essas chamadas de maneira idempotente. Se você começar a deixar esses verbos não idempotentes, mas quem fizer as chamadas achar que pode executá-los repetidamente de forma segura, você poderá se meter em uma bela confusão. Lembre-se de que, somente por estar usando o HTTP como protocolo subjacente, não significa que você terá tudo de graça!

Diluindo o seu risco

Uma forma de escalar com vistas à resiliência é garantir que não colocará todos os ovos em uma só cesta. Um exemplo simplista desse caso é garantir que não teremos vários serviços em um único host, situação em que uma falha poderia causar impacto em vários serviços. No entanto, vamos considerar o que quer dizer um *host*. Na maioria das situações atualmente, um “host” é, na verdade, um conceito virtual. Então, e se eu tivesse todos os meus serviços em hosts diferentes, mas todos esses hosts na verdade fossem virtuais, executando no mesmo equipamento físico? Se esse equipamento ficar inativo, eu poderia perder vários serviços. Algumas plataformas de virtualização permitem garantir que seus hosts sejam distribuídos em vários equipamentos físicos distintos a fim de reduzir as chances de esse problema ocorrer.

Em plataformas de virtualização internas, uma prática comum é ter a partição raiz da máquina virtual mapeada para uma única SAN (Storage Area Network, ou Rede Local de Armazenamento). Se essa SAN ficar inativa, ela poderá derrubar todas as VMs conectadas. As SANs são grandes, caras e projetadas para não falhar. Apesar disso, já vi SANs grandes e caras apresentarem falhas pelo menos duas vezes nos últimos dez anos, e em cada caso, os resultados foram um tanto quanto graves.

Outra forma comum de separação para reduzir as falhas é garantir que nem

todos os seus serviços executem em um único rack no data center, ou que seus serviços estejam distribuídos em mais de um data center. Se você utiliza um provedor de serviços subjacente, é importante saber se um SLA é oferecido, e se planejar de acordo com isso. Se for necessário garantir que seus serviços não estejam inativos por mais de quatro horas por trimestre, mas seu provedor de hospedagem apenas puder garantir um downtime máximo de oito horas por trimestre, será preciso alterar o SLA, ou então procurar uma solução alternativa.

A AWS, por exemplo, é dividida em regiões, que podem ser imaginadas como nuvens distintas. Cada região, por sua vez, é dividida em duas ou mais zonas de disponibilidade, conforme discutimos antes. Essas zonas de disponibilidade são o equivalente da AWS a um data center. É essencial ter os serviços distribuídos em várias zonas de disponibilidade, pois a AWS não oferece nenhuma garantia sobre a disponibilidade de um único nó, ou nem mesmo de uma zona de disponibilidade inteira. Para seu serviço de computação, ela oferece somente um uptime de 99,95% em um dado período mensal na região como um todo, portanto você deve distribuir suas cargas de trabalho em várias zonas de disponibilidade dentro de uma única região. Para algumas pessoas, isso não basta, e elas executam seus serviços em várias regiões também.

Devemos observar, é claro, que, como os provedores oferecem uma “garantia” de SLA, eles tendem a limitar sua responsabilidade! Se o fato de eles não cumprirem suas metas resultar para você na perda de clientes e de um grande valor monetário, você poderá se ver vasculhando os contratos a fim de verificar se conseguirá algo de volta deles. Portanto, sugiro fortemente que você saiba qual é o impacto da falha de um fornecedor em cumprir suas obrigações e determine se será necessário ter um plano B (ou C) na manga. Mais de um cliente com o qual já trabalhei tinha uma plataforma de hospedagem para recuperação de desastres junto a um fornecedor diferente, por exemplo, com o intuito de garantir que não estivessem vulneráveis demais aos erros de uma única empresa.

Teorema CAP

Queremos ter tudo, mas, infelizmente, sabemos que não é possível. E, quando se trata de sistemas distribuídos como aqueles que construímos usando arquiteturas de microsserviços, temos até mesmo uma prova matemática que diz que não podemos. Você já deve ter ouvido falar do teorema CAP, particularmente em discussões sobre os méritos dos vários tipos diferentes de bancos de dados. Em sua essência, o teorema diz que, em um sistema distribuído, temos três itens que podem ser ponderados entre si: *consistência* (consistency), *disponibilidade* (availability) e *tolerância à partição* (partition tolerance). Especificamente, o teorema nos diz que conseguimos preservar dois em um modo de falha.

A consistência é a característica do sistema segundo a qual teremos a mesma resposta se acessarmos vários nós. A disponibilidade significa que toda requisição receberá uma resposta. A tolerância à partição é a capacidade do sistema de lidar com o fato de que a comunicação entre suas partes às vezes não será possível.

Desde que Eric Brewer publicou sua conjectura original, a ideia ganhou uma prova matemática. Não vou detalhar a matemática envolvida na prova em questão, não só porque este não é o tipo de livro para isso, mas também porque posso garantir que eu o faria da forma incorreta. Em vez disso, vamos utilizar alguns exemplos que foram pensados para nos ajudar a entender que, em sua base, o teorema CAP é a sintetização de um conjunto muito lógico de raciocínios.

Vamos supor que nosso microsserviço *Estoque* seja implantado em dois data centers distintos, como mostra a Figura 12.7. Auxiliando a instância de nosso serviço em cada data center há um banco de dados, e esses dois bancos de dados conversam para tentar sincronizar os dados entre eles. Leituras e escritas são feitas por meio do nó local do banco de dados, e a replicação é usada para sincronizar os dados entre os nós.

Vamos agora pensar no que acontecerá se algo falhar. Imagine que algo tão simples quanto o link de rede entre os dois data centers pare de funcionar. A sincronização nesse momento falhará. Escritas feitas no banco de dados principal no DC1 não serão propagadas para o DC2, e vice-versa. A maioria dos bancos de dados que aceitam essas configurações também

aceita algum tipo de técnica de fila para garantir que possam se recuperar desse problema depois, mas o que acontecerá nesse ínterim?

Sacrificando a consistência

Vamos supor que não desativaremos totalmente o microserviço *Estoque*. Se fizermos uma mudança nos dados do DC1 agora, o banco de dados em DC2 não verá. Isso significa que qualquer requisição feita ao nosso nó de estoque em DC2 verá dados possivelmente desatualizados. Em outras palavras, nosso sistema continua *disponível* nos dois nós, isto é, ambos são capazes de atender às requisições, e mantivemos o sistema em execução apesar da *partição*, mas perdemos a *consistência*; não conseguimos preservar todos os três traços. Em geral, esse sistema é chamado de *AP*, por causa de sua disponibilidade (*availability*) e da tolerância à partição (*partition tolerance*).

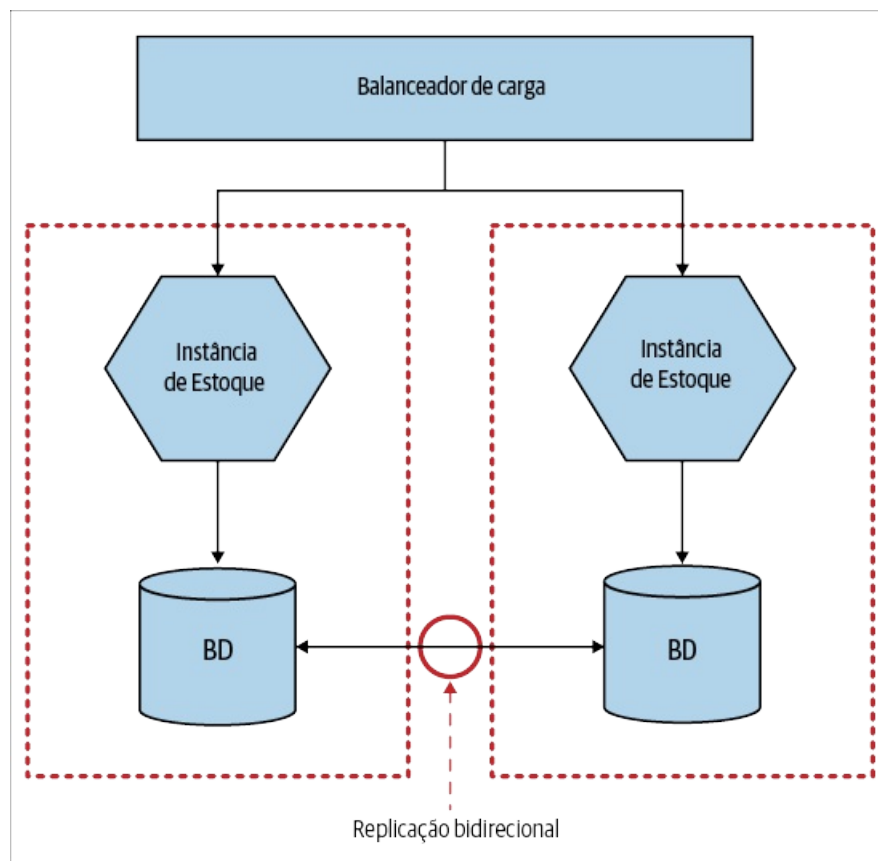


Figura 12.7 – Utilizando uma replicação multimaster para compartilhar dados entre dois nós do banco de dados.

Durante essa partição, se continuarmos aceitando escritas, devemos aceitar o fato de que, em algum momento no futuro, elas terão de ser ressincronizadas. Quanto mais tempo a partição durar, mais difícil poderá ser essa ressincronização.

A verdade é que, ainda que não tenhamos uma falha de rede entre os nós de nossos bancos de dados, a replicação de dados não será instantânea. Conforme mencionamos antes, os sistemas que estiverem satisfeitos em sacrificar a consistência para preservar a tolerância à partição e a disponibilidade são chamados de *eventualmente consistentes* (eventually consistent), ou seja, esperamos que, em algum momento no futuro, todos os nós vejam os dados atualizados, mas isso não acontecerá ao mesmo tempo, portanto precisamos conviver com a possibilidade de os usuários verem dados antigos.

Sacrificando a disponibilidade

O que acontecerá se precisarmos manter a consistência e quisermos descartar algum outro aspecto? Bem, para manter a consistência, cada nó do banco de dados deve saber se a cópia dos dados que tem é igual à do outro nó do banco de dados. Durante a partição, se os nós dos bancos de dados não puderem conversar, eles não poderão se coordenar para garantir que haja consistência. Seremos incapazes de garantir a consistência, portanto nossa única opção será nos recusarmos a responder à requisição. Em outras palavras, sacrificamos a disponibilidade. Nosso sistema será consistente (consistent) e tolerante à partição (partition tolerant), isto é, será um sistema CP. Nesse modo, nosso serviço terá de descobrir como fazer com que haja uma degradação das funcionalidades, até que a partição seja resolvida e os nós dos bancos de dados possam ser ressincronizados.

Manter a consistência entre vários nós é realmente difícil. Há poucas tarefas (talvez nenhuma) que sejam mais difíceis em sistemas distribuídos. Pense nela por um instante. Suponha que eu queira ler um registro do nó do banco de dados local. Como saberei que ele está atualizado? Eu teria de perguntar ao outro nó. No entanto, eu também teria de pedir a esse nó do banco de dados que não permita que ele seja atualizado até a leitura ser

concluída; em outras palavras, preciso iniciar uma leitura transacional entre vários nós do banco de dados a fim de garantir que haja consistência. Em geral, porém, as pessoas não fazem leituras transacionais, não é mesmo? Porque as leituras transacionais são lentas. Elas exigem locks (travas). Uma leitura poderia deixar um sistema todo bloqueado.

Conforme já discutimos, os sistemas distribuídos precisam estar preparados para falhas. Considere nossa leitura transacional envolvendo um conjunto de nós consistentes. Eu peço a um nó remoto que faça um lock em um dado registro quando a leitura for iniciada. Concluo a leitura e peço ao nó remoto que libere seu lock, mas agora não consigo conversar com ele. O que acontecerá então? É muito difícil implementar os locks corretamente, mesmo em um sistema com um só processo, e eles são significativamente mais difíceis de serem bem implementados em um sistema distribuído.

Você se lembra de quando falamos de transações distribuídas no Capítulo 6? O principal motivo de eles serem tão desafiadores se deve a esse problema de garantir que haja consistência entre vários nós.

Ter consistência entre vários nós da forma correta é tão difícil que eu *fortemente insisto* que, se precisar disso, não tente inventar uma solução por conta própria. Em vez disso, escolha um banco de dados ou um serviço de lock que ofereçam essas características. O Consul, por exemplo, que foi discutido na seção “Registros dinâmicos de serviços”, na página 196, implementa um repositório de chave-valor altamente consistente, projetado para compartilhar configurações entre vários nós. Junto com “Amigos não deixam que amigos escrevam sua própria solução de criptografia”, devemos ter “Amigos não deixam que amigos escrevam seu próprio repositório de dados consistente e distribuído”. Se você acha que precisa escrever sua própria solução de armazenamento de dados CP, leia todos os artigos sobre o assunto antes, em seguida faça um doutorado, e então espere gastar alguns anos errando. Enquanto isso, eu utilizaria alguma solução pronta que fizesse isso para mim ou, mais provavelmente, me *esforçaria ao máximo* para criar sistemas AP eventualmente consistentes.

Sacrificando a tolerância à partição?

Temos de escolher dois, certo? Então, temos nosso sistema AP eventualmente consistente. Temos nosso sistema CP consistente, mas que é difícil de construir e escalar. Por que não um sistema CA? Bem, como podemos sacrificar a tolerância à partição? Se nosso sistema não tiver tolerância à partição, ele não poderá executar em uma rede. Em outras palavras, ele precisa ser um sistema com um único processo, funcionando localmente. Sistemas CA não existem em sistemas distribuídos.

AP ou CP?

Qual é a opção correta, AP ou CP? Bem, a verdade é que isso *depende*. Por sermos as pessoas que estão criando o sistema, sabemos que há aspectos que devem ser ponderados. Sabemos que os sistemas AP escalam com mais facilidade e são mais fáceis de criar, e sabemos que um sistema CP exigirá mais trabalho em virtude dos desafios de termos uma consistência distribuída. No entanto, talvez não saibamos o impacto dessas avaliações de custo-benefício nos negócios. Em nosso sistema de estoque, se um registro estiver desatualizado em cinco minutos, estaria tudo bem? Se a resposta for sim, um sistema AP poderia ser a solução. Mas e se considerássemos o saldo de um cliente em um banco? Essa informação poderia estar desatualizada? Sem saber o contexto no qual a operação será usada, não é possível saber qual será a decisão correta. Conhecer o teorema CAP ajuda a entender que essas avaliações de custo-benefício devem ser feitas e quais perguntas devem ser formuladas.

Não é uma questão de tudo ou nada

Nosso sistema como um todo não precisa ser exclusivamente AP ou CP. Nosso catálogo na MusicCorp poderia ser AP, pois não nos preocuparíamos muito com um registro desatualizado. No entanto, poderíamos decidir que nosso serviço de estoque precisa ser CP, pois não queremos vender para um cliente um item que não temos, e então ter de nos desculpar depois.

Até mesmo serviços individuais não precisam ser exclusivamente CP ou

AP.

Vamos pensar em nosso microserviço Saldo de Pontos, no qual armazenamos registros de quantos pontos de fidelidade nossos clientes têm acumulado. Poderíamos decidir que não nos importaremos se o saldo exibido a um cliente está desatualizado, mas, quando se tratar da atualização de um saldo, precisamos que esse valor esteja consistente a fim de garantir que os clientes não utilizem mais pontos do que têm à disposição. Esse microserviço é CP, AP ou ambos? Na verdade, o que fizemos foi levar as ponderações relacionadas ao teorema CAP até o nível das funcionalidades de um microserviço em particular.

Outra complexidade é que nem a consistência nem a disponibilidade são uma questão de tudo ou nada. Muitos sistemas nos permitem fazer ponderações com muito mais nuances. Por exemplo, com o Cassandra, posso fazer diferentes avaliações de custo-benefício para chamadas individuais. Assim, se eu precisar de uma consistência rígida, posso efetuar uma leitura que cause um bloqueio até que todas as réplicas tenham respondido, confirmando que o valor está consistente, ou até que um quórum específico de réplicas, ou até mesmo um único nó, tenha respondido. Obviamente, se a operação ficar bloqueada até que todas as réplicas respondam e uma delas estiver indisponível, a operação permanecerá bloqueada por um bom tempo. Por outro lado, se quiser que minha leitura seja respondida do modo mais rápido possível, eu poderia esperar a resposta de apenas um nó – nesse caso, há a possibilidade de haver inconsistência nessa visão dos meus dados.

Com frequência, você verá postagens a respeito de pessoas que “venceram” o teorema CAP. Elas não venceram. O que elas fizeram foi criar um sistema em que algumas funcionalidades são CP, enquanto outras são AP. A prova matemática por trás do teorema CAP continua válida.

E o mundo real

Boa parte do que discutimos se refere ao mundo da eletrônica – bits e bytes armazenados em memória. Falamos da consistência de forma quase infantil; imaginamos que, no escopo do sistema que criamos, podemos

parar o mundo e fazer com que tudo tenha sentido. Apesar disso, quase tudo que construímos é apenas um reflexo do mundo real, e não é possível controlá-lo, não é mesmo?

Vamos rever o nosso sistema de estoque. Ele se mapeia a itens físicos do mundo real. Armazenamos um contador em nosso sistema que representa a quantidade de álbuns presente nos depósitos da MusicCorp. No início do dia, tínhamos 100 cópias de *Give Blood* da banda Brakes. Vendemos uma. Agora temos 99 cópias. Fácil, não é mesmo? Mas o que acontecerá se, quando o pedido estiver sendo enviado, alguém derrubar uma cópia do álbum no chão, uma pessoa pisar e quebrá-la? E agora? Nossos sistemas dizem que há 99 cópias na prateleira, mas, na verdade, há somente 98.

E se fizermos com que o nosso sistema de estoque seja AP, e ocasionalmente tivéssemos de entrar em contato com um usuário mais tarde e lhe dizer que um de seus itens na verdade não está disponível em estoque? Isso seria a pior coisa do mundo? Certamente seria muito mais fácil criar e escalar o sistema, e garantir que estivesse correto.

Temos de reconhecer que, não importa quão consistentes possam ser os nossos sistemas propriamente ditos, eles não têm como saber tudo o que acontece, sobretudo quando mantemos registros do mundo real. Esse é um dos principais motivos pelos quais os sistemas AP acabam sendo a opção correta em muitas situações. Afora a complexidade de criar sistemas CP, eles não são capazes de corrigir todos os nossos problemas, de qualquer maneira.

Antifragilidade

Na primeira edição, falei do conceito de antifrágil, conforme popularizado por Nassim Taleb. Esse conceito descreve como os sistemas na verdade se beneficiam de falhas e desordem, e recebeu destaque como inspiração para o modo de operação de algumas partes da Netflix, particularmente com relação a conceitos como engenharia de caos (chaos engineering). Ao observar de modo mais amplo o conceito de resiliência, porém, percebemos que a antifragilidade é apenas um subconjunto do conceito de resiliência. Quando consideramos os conceitos de extensibilidade com elegância e

adaptabilidade sustentável, que apresentamos antes, isso se torna claro. Acho que, quando a antifragilidade se tornou, por um breve momento, um conceito da moda em TI, isso ocorria em um contexto no qual pensávamos de modo muito restrito na resiliência – pensávamos somente na robustez, e talvez na recuperação, mas ignorávamos a parte restante. Com a área de engenharia de resiliência atualmente ganhando mais impulso e reconhecimento, parece ser apropriado ir além do termo *antifrágil*, ao mesmo tempo que continuamos dando destaque a algumas das ideias na base desse conceito, as quais realmente fazem parte da resiliência como um todo.

Engenharia de caos

Desde a primeira edição deste livro, outra técnica que passou a ganhar mais atenção é a *engenharia de caos* (chaos engineering). Ela recebeu seu nome com base nas práticas da Netflix e pode ser uma abordagem conveniente para ajudar a aumentar a sua resiliência – tanto para garantir que seus sistemas sejam tão *robustos* quanto você imagina que são, ou como parte de uma abordagem em direção a um sistema com uma *adaptabilidade sustentável*.

Inspirado originalmente no trabalho que a Netflix fazia internamente, há uma certa dificuldade com o termo *engenharia de caos* por causa das explicações confusas acerca de seu significado. Para muitos, significa “executar uma ferramenta em meu software e ver o que ela diz”, e não ajuda muito o fato de que muitos dos mais empenhados proponentes da engenharia de caos geralmente são, eles próprios, vendedores de ferramentas para execução da... bem, engenharia de caos, em seu software.

Ter uma definição clara do que significa a engenharia de caos para os que a colocam em prática é difícil. A melhor definição (ao menos para mim) com a qual já deparei é esta:

A Engenharia de Caos é a disciplina de fazer experimentos em um sistema a fim de aumentar a confiança na capacidade do sistema de suportar condições turbulentas em produção.

– Principles of Chaos Engineering (Princípios da Engenharia de Caos,

<https://principlesofchaos.org>)

A palavra *sistema* nesse caso tem muitas funções. Alguns a verão de forma restrita, como os componentes de software e de hardware. Porém, no contexto da engenharia de resiliência, é importante ver o *sistema* como a totalidade de pessoas, processos, cultura e, sim, do software e da infraestrutura envolvidos na criação de nosso produto. Isso significa que devemos encarar a engenharia de caos de modo mais amplo do que apenas “vamos desligar algumas máquinas e ver o que acontece”.

Game Days

Muito antes que a engenharia de caos tivesse seu nome, as pessoas organizavam exercícios de Game Day para testar o nível de preparação das pessoas para determinados eventos. Planejados com antecedência – porém idealmente iniciados de surpresa (para os participantes) – os exercícios oferecem uma chance de testar as pessoas e os processos diante de uma situação realista, porém fictícia. Durante minha estada no Google, essa era uma ocorrência razoavelmente comum em diversos sistemas, e acho que, sem dúvida, muitas empresas poderiam se beneficiar se tivessem esses tipos de exercícios regularmente. O Google vai além dos testes simples para simular falhas em servidores, e como parte de seus exercícios DiRT (Disaster Recovery Test, ou Teste de Recuperação de Desastres), a empresa já simulou desastres de larga escala, por exemplo, terremotos.⁶

Os Game Days podem ser usados para sondar causas suspeitas de fragilidade no sistema. Em seu livro *Learning Chaos Engineering*⁷, Russ Miles apresenta um exemplo de um exercício de Game Day do qual ele foi o facilitador, que havia sido projetado para analisar, em parte, a dependência exagerada em relação a um único funcionário chamado Bob. No Game Day, Bob foi isolado em uma sala e não podia ajudar a equipe durante a indisponibilidade de serviço simulada. No entanto, Bob ficou observando tudo e acabou tendo de interferir quando a equipe, em sua tentativa de corrigir problemas no sistema “simulado”, acabou fazendo login no ambiente de produção por engano e estava no processo de destruir dados de produção. Só podemos acreditar que muitas lições foram aprendidas logo após esse exercício.

Experimentos no ambiente de produção

A escala com a qual a Netflix opera é bem conhecida, assim como o fato de a Netflix se basear totalmente na infraestrutura da AWS. Esses dois fatores implicam que ela precisa aceitar que haja falhas. A Netflix percebeu que se planejar para falhas e realmente *saber* que seu software lidará com essa falha quando ela ocorrer são dois fatos distintos. Para isso, a Netflix *incita* falhas com o intuito de garantir que seus sistemas sejam tolerantes a elas, executando ferramentas em seus sistemas.

A mais famosa dessas ferramentas é o Chaos Monkey que, em certas horas do dia, desliga aleatoriamente algumas máquinas no ambiente de produção. Saber que isso pode e vai acontecer em produção significa que os desenvolvedores que criam os sistemas devem estar realmente preparados para essa ocorrência. O Chaos Monkey é apenas uma parte do Simian Army (Exército Símio) da Netflix, composto de bots para falhas. O Chaos Gorilla é usado para derrubar uma zona de disponibilidade inteira (o equivalente a um data center na AWS), enquanto o Latency Monkey simula uma conectividade de rede lenta entre as máquinas. Para muitos, o teste definitivo para saber se o sistema está realmente robusto poderia ser lançar o seu próprio Simian Army na infraestrutura de produção.

Partindo da robustez e indo além

Se aplicada em sua forma mais restrita, a engenharia de caos poderia ser uma atividade útil para melhorar a robustez de nossa aplicação. Lembre-se de que a robustez no contexto da engenharia de resiliência significa até que ponto nosso sistema é capaz de lidar com problemas esperados. A Netflix sabia que não podia contar com o fato de uma dada máquina virtual estar sempre disponível em seu ambiente de produção, portanto ela criou o Chaos Monkey para garantir que seu sistema pudesse sobreviver a esse problema *esperado*.

Se você, porém, faz uso de ferramentas de engenharia de caos como parte de uma abordagem de questionamento contínuo da resiliência de seu sistema, elas poderão ter uma aplicabilidade muito maior. Usar ferramentas dessa área para ajudar a responder perguntas do tipo “e se”

que você possa fazer, questionando constantemente o seu entendimento, pode ter um impacto muito maior. O Chaos Toolkit (<https://chaostoolkit.org>) é um projeto open source para ajudar a conduzir experimentos em seu sistema, e tem se provado bastante popular. A Reliably (<https://reliably.com>), empresa fundada pelos criadores do Chaos Toolkit, oferece uma grande variedade de ferramentas para ajudar com a engenharia de caos em geral, embora o fornecedor mais conhecido nessa área talvez seja a Gremlin (<https://www.gremlin.com>).

Lembre-se apenas de que executar uma ferramenta de engenharia de caos não torna você resiliente.

Acusações

Quando houver algo errado, há várias maneiras de lidar com essa situação. Obviamente, logo após um incidente, nosso foco estará em fazer com que tudo volte a funcionar novamente, o que é razoável. Depois disso, com muita frequência, vêm as recriminações. Como norma, há uma postura de procurar algo ou alguém a quem acusar. O conceito de “análise da causa-raiz” (root cause analysis) implica que *há* uma causa-raiz. É surpreendente a frequência com que queremos que essa causa-raiz seja uma pessoa.

Alguns anos atrás, na época em que eu trabalhava na Austrália, houve uma falha na Telstra, a principal empresa de telecomunicações (e que detinha o monopólio anteriormente), a qual causou impacto tanto nos serviços de voz como nos de telefonia. Esse incidente foi particularmente problemático por causa do escopo e da duração da indisponibilidade dos serviços. A Austrália tem muitas comunidades rurais bastante isoladas, e falhas como essas tendem a ser particularmente graves. Quase imediatamente após a falha, a COO da Telstra divulgou um comunicado que deixou claro que a empresa sabia exatamente o que havia causado o problema:⁸

“Desativamos aquele nó, mas, infelizmente o indivíduo que estava administrando esse problema não seguiu o procedimento correto e reconectou os clientes ao nó com mau funcionamento, em vez de transferi-los aos outros nove nós redundantes para os quais ele deveria ter transferido as pessoas” – disse Ms. McKenzie aos repórteres na tarde

de terça-feira.

“Pedimos desculpas a toda a nossa base de clientes. Foi um erro humano constrangedor.”

Em primeiro lugar, observe que essa declaração foi feita algumas horas depois da falha. Apesar disso, a Telstra já havia vasculhado o que deveria ser um sistema extremamente complexo para saber exatamente quem deveria ser o culpado – uma pessoa. Mas se é verdade que uma única pessoa cometendo um erro possa realmente derrubar uma empresa de telecomunicação inteira, diríamos que isso diz mais a respeito dessa operadora do que do indivíduo. Além do mais, a Telstra claramente mostrou aos seus funcionários naquela ocasião que estava muito satisfeita em apontar dedos para os culpados.²

O problema de acusar as pessoas logo após incidentes como esses é que algo que começa como uma tentativa de jogar a responsabilidade para outros no curto prazo acaba criando uma cultura de medo, na qual as pessoas não se sentirão dispostas a se manifestar quando houver algo errado. Como resultado, perdemos a oportunidade de aprender com os erros e ficamos suscetíveis à ocorrência dos mesmos problemas novamente. Criar uma empresa na qual as pessoas se sintam seguras para admitir quando cometem erros é essencial para criar uma cultura de aprendizado, o que, por sua vez, contribui bastante para criar uma empresa capaz de desenvolver softwares mais robustos, afora as vantagens mais evidentes de criar um local que seja mais satisfatório para trabalhar.

Voltando à Telstra, com a causa do problema claramente identificada na investigação detalhada conduzida poucas horas após a falha de nível nacional, certamente não esperaríamos que houvesse outras falhas na sequência, não é mesmo? Infelizmente a Telstra sofreu uma série de falhas subsequentes. Mais erros humanos? Talvez a Telstra pensasse assim – logo após a série de incidentes, a COO pediu demissão.

Para uma abordagem mais detalhada acerca de como criar uma empresa na qual podemos tirar o melhor proveito possível dos erros, além de proporcionar um ambiente mais agradável aos funcionários, o artigo “Blameless Post-Mortems and a Just Culture” (Post-mortems sem

acusações e uma cultura justa), de John Allspaw, é um ótimo ponto de partida.¹⁰

Em última instância, conforme já enfatizei inúmeras vezes neste capítulo, a *resiliência* exige uma mente questionadora – uma tendência a analisar constantemente os pontos fracos de nosso sistema. Isso exige uma cultura de aprendizado e, em geral, os melhores aprendizados podem vir logo após um incidente. Desse modo, é essencial garantir que nos esforcemos ao máximo para criar um ambiente no qual, quando o pior acontecer, possamos coletar o maior volume possível de informações logo após o incidente a fim de reduzir as chances de o problema ocorrer novamente.

Resumo

À medida que nosso software se torna mais essencial às vidas de nossos usuários, a busca por melhorar a resiliência do software que criamos se torna mais acentuada. Conforme vimos neste capítulo, porém, não conseguiremos ter resiliência *somente* pensando em nosso software e na infraestrutura; também temos de pensar nas pessoas, nos processos e nas organizações. Neste capítulo, vimos os quatro conceitos básicos da resiliência, conforme descritos por David Woods:

Robustez

A capacidade de absorver perturbações esperadas.

Recuperação

A capacidade de se recuperar após um evento traumático.

Extensibilidade com elegância

Quão bem lidamos com uma situação inesperada.

Adaptabilidade sustentável

A capacidade de se adaptar continuamente a ambientes, stakeholders e demandas que mudam.

Olhando mais especificamente para os microsserviços, vemos que eles nos oferecem uma série de maneiras para melhorar a *robustez* de nossos sistemas. No entanto, esse aumento de robustez não vem gratuitamente –

ainda será preciso decidir as opções a serem usadas. Os principais padrões de estabilidade como circuit breakers (interruptores de circuito), timeouts, redundância, isolamento, idempotência e outros padrões desse tipo são todas ferramentas que estão à nossa disposição, porém é necessário decidir quando e onde elas serão usadas. Contudo, para além desses conceitos restritos, também devemos constantemente nos manter atentos àquilo que não sabemos.

Também devemos determinar o nível de resiliência que queremos – e isso será quase sempre definido pelos usuários e pelos responsáveis pelos negócios em seu sistema. Como profissional da área técnica, você pode ser responsável pelo modo como o trabalho é feito, mas saber qual é o nível de resiliência necessário exigirá uma comunicação eficaz, estreita e frequente com os usuários e os donos de produto (product owners).

Vamos voltar à citação de David Woods apresentada antes neste capítulo, que apresentamos quando discutimos a adaptabilidade sustentável:

“Não importa quão bom tenhamos sido antes, não importa quão bem-sucedidos fomos, o futuro pode ser diferente, e pode ser que não estejamos bem adaptados. Talvez sejamos insuficientes e frágeis diante desse novo futuro.”

Fazer as mesmas perguntas repetidamente não ajudará a entender se você está preparado para o futuro incerto. Você não sabe o que não sabe – adotar uma abordagem na qual aprendemos e questionamos constantemente é essencial para adquirir resiliência.

Um tipo de padrão de estabilidade que vimos, a redundância, pode ser muito eficaz. No próximo capítulo, daremos um prosseguimento natural a essa ideia; veremos diferentes maneiras de escalar nossos microserviços, o que, além de nos ajudar a lidar com mais carga, também pode ser um modo eficaz de ajudar na implementação da redundância em nossos sistemas e, desse modo, melhorar também a sua robustez.

¹ David D. Woods, “Four Concepts for Resilience and the Implications for the Future of Resilience Engineering” (Quatro conceitos sobre resiliência e as implicações para o futuro da engenharia de resiliência), *Reliability Engineering & System Safety* 141 (setembro de 2015): 5–9, doi.org/10.1016/j.ress.2015.03.018.

- 2 ”Jason Bloomberg, “Innovation: The Flip Side of Resilience” (Inovação: o outro lado da resiliência), *Forbes*, 23 de setembro de 2014, <https://oreil.ly/avSmU>.
- 3 Consulte “Google Uncloaks Once-Secret Server” (Google revela servidor anteriormente secreto, <https://oreil.ly/k7b3h>), de Stephen Shankland, para mais informação sobre esse assunto, incluindo uma visão geral interessante sobre o porquê de o Google achar que essa abordagem pode ser melhor que os sistemas UPS tradicionais.
- 4 Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2ª edição, (Raleigh: Pragmatic Bookshelf, 2018).
- 5 A terminologia de um breaker “aberto”, significando que as requisições não podem fluir, pode ser confusa, mas é proveniente dos circuitos elétricos. Quando o breaker está “aberto”, o circuito é interrompido e a corrente não consegue fluir. Fechar o breaker permite que o circuito se complete e a corrente flua novamente.
- 6 Kripa Krishnan, “Weathering the Unexpected” (Enfrentando o inesperado), *acmqueue* 10, nº 9 (2012), <https://oreil.ly/BCSQ7>.
- 7 Russ Miles, *Learning Chaos Engineering* (Sebastopol: O’Reilly, 2019).
- 8 Kate Aubusson e Tim Biggs, “Major Telstra Mobile Outage Hits Nationwide, with Calls and Data Affected” (Falha grave em serviços móveis da Telstra atinge toda a nação, com chamadas e dados afetados), *Sydney Morning Herald*, 9 de fevereiro de 2016, <https://oreil.ly/4cBcy>.
- 9 Escrevi mais a respeito do incidente da Telstra naquela época – “Telstra, Human Error and Blame Culture” (Telstra, erro humano e a cultura de acusações), <https://oreil.ly/OXgUQ>. Quando escrevi essa postagem de blog, não percebi que a empresa na qual eu trabalhava tinha a Telstra como cliente; meu empregador na ocasião, na verdade, lidou muito bem com a situação, apesar de o caso ter resultado em algumas horas de desconforto quando parte dos meus comentários foi notada pela imprensa nacional.
- 10 John Allspaw, “Blameless Post-Mortems and a Just Culture” (Post-mortems sem acusações e uma cultura justa) Code as Craft (blog), Etsy, 22 de maio de 2012, <https://oreil.ly/7LzmL>.

Escalabilidade

“Você vai precisar de um barco maior.”

– Delegado Brody, *Tubarão*

Quando escalamos nossos sistemas, fazemos isso por um ou dois motivos. Em primeiro lugar, escalar um sistema nos permite melhorar o seu desempenho, talvez possibilitando lidar com mais carga ou melhorando a latência. Em segundo, podemos escalar nosso sistema para aumentar a sua robustez. Neste capítulo, apresentaremos um modelo que descreve as diferentes formas de escalar, e então veremos em detalhes como cada opção pode ser implementada em uma arquitetura de microsserviços. No final deste capítulo, você terá um conjunto de técnicas para lidar com problemas de escalabilidade que possam surgir em seu caminho.

Para começar, porém, vamos ver as diferentes opções que você pode aplicar para escalar o seu sistema.

Quatro eixos da escalabilidade

Não há apenas uma maneira correta de escalar um sistema, pois a técnica usada dependerá do tipo de limitação que você possa ter. Temos vários modos diferentes de escalar à nossa disposição para ajudar no que diz respeito ao desempenho, à robustez, ou talvez ambos. Um modelo que uso com frequência para descrever as diferentes formas de escalar é o Scale Cube (Cubo da Escalabilidade) extraído do livro *The Art of Scalability*,¹ que divide as opções para escalar em três categorias, as quais, no contexto dos sistemas computacionais, incluem decomposição funcional, duplicação horizontal e partição de dados. A importância desse modelo está no fato de ele ajudar a entender que é possível escalar um sistema ao longo de um, dois ou até dos três eixos, dependendo de suas necessidades.

No entanto, particularmente em um mundo de infraestrutura virtualizada, sempre achei que um quarto eixo de escalabilidade vertical faz falta nesse modelo, embora, com isso, o modelo teria a propriedade, de certa forma infeliz, de não mais ser um cubo. Apesar disso, acho que o modelo representa um conjunto conveniente de métodos para determinarmos a melhor maneira de escalar nossas arquiteturas de microsserviços. Antes de analisar essas formas de escalar em detalhes, junto com os prós e contras associados, um breve resumo se faz necessário:

Escalabilidade vertical

Em poucas palavras, significa adquirir uma máquina mais potente.

Duplicação horizontal

Ter vários itens capazes de fazer o mesmo trabalho.

Partição de dados

Dividir o trabalho com base em algum atributo dos dados, por exemplo, o grupo ao qual pertence o cliente.

Decomposição funcional

Separação do trabalho com base no tipo, por exemplo, uma decomposição em microsserviços.

Entender qual combinação dessas técnicas para escalar é a mais apropriada basicamente se reduzirá à natureza dos problemas de escalabilidade que você estiver enfrentando. Para explorar esse assunto com mais detalhes, assim como ver exemplos de como esses conceitos poderiam ser implementados na MusicCorp, analisaremos também sua adequação a uma empresa real, a FoodCo.² A FoodCo oferece serviços de entrega de comida diretamente aos clientes em vários países do mundo.

Escalabilidade vertical

Algumas operações simplesmente podem se beneficiar se houver mais potência. Ter uma máquina maior, com uma CPU mais rápida e E/S mais eficaz geralmente melhora a latência e o throughput, permitindo processar mais tarefas em menos tempo. Desse modo, se sua aplicação não está rápida o suficiente para você, ou não é capaz de lidar com as requisições

necessárias, por que não adquirir uma máquina mais potente?

No caso da FoodCo, um dos desafios que ela enfrenta é um aumento na contenção de escritas em seu banco de dados principal. Em geral, a escalabilidade vertical é uma opção preferida para escalar rapidamente as escritas em um banco de dados relacional e, de fato, a FoodCo já fez um upgrade da infraestrutura do banco de dados várias vezes. O problema é que a FoodCo já levou essa solução ao limite máximo com o qual se sente satisfeita. Escalar verticalmente funcionou por muitos anos, mas considerando as projeções de crescimento da empresa, mesmo que a FoodCo pudesse adquirir uma máquina mais potente, essa opção provavelmente não resolveria o problema no longo prazo.

Historicamente, quando a escalabilidade vertical exigia a compra de hardware, essa técnica era mais complicada. O prazo necessário para comprar o hardware implicava que essa não era uma opção a ser abordada de forma leviana, e se por acaso uma máquina mais potente não solucionasse os problemas, você provavelmente teria gastado muito dinheiro sem que fosse necessário. Além do mais, era comum superestimar a máquina necessária por causa das dificuldades de obter aprovações para os orçamentos, esperar a máquina chegar, e assim por diante, o que, por sua vez, levava a uma subutilização significativa de recursos nos data centers.

No entanto, a mudança para a virtualização e o surgimento da nuvem pública têm ajudado bastante nessa forma de escalar os sistemas.

Implementação

A implementação variará conforme a infraestrutura na qual você está executando. Se estiver executando em sua própria infraestrutura virtualizada, você poderá simplesmente redimensionar a VM de modo a usar mais do hardware subjacente – é uma tarefa que deve ser rápida e razoavelmente livre de riscos para implementar. Se a VM for tão grande quanto o hardware subjacente é capaz de tratar, essa opção obviamente não faria sentido – você teria de comprar mais hardware. De modo semelhante, se estiver executando seu sistema em seus próprios servidores

bare metal (físicos) e não houver hardware sobressalente que seja mais potente do que aquele no qual você está executando no momento, mais uma vez, será necessário considerar a compra de mais máquinas.

Em geral, se eu chegar ao ponto em que tivesse de comprar uma nova infraestrutura para tentar escalar o sistema verticalmente, por causa do custo (e do tempo) adicional para que isso cause algum impacto, poderia muito bem deixar de lado essa forma de escalar por ora, e considerar a duplicação horizontal, que veremos a seguir.

No entanto, o surgimento da nuvem pública também nos permite alugar facilmente máquinas totalmente gerenciadas, por hora (e, em alguns casos, até mesmo por menos tempo), junto aos fornecedores de nuvem pública. Além do mais, os principais provedores de nuvem oferecem uma variedade maior de máquinas para diferentes tipos de problemas. Sua carga de trabalho exige mais memória? Regale-se com uma instância AWS `u-24tb1.metal`, que oferece 24 TB de memória (sim, você leu corretamente). Todavia, as cargas de trabalho que podem realmente precisar dessa quantidade de memória parecem ser muito raras, mas você tem essa opção. Há também máquinas personalizadas para uso intensivo de E/S, CPU ou GPU. Se sua solução atual já faz uso da nuvem pública, essa é uma forma bastante trivial para experimentar escalar o seu sistema, e se você está em busca de uma vitória rápida, é uma opção muito fácil.

Principais vantagens

Em uma infraestrutura virtualizada, particularmente em um provedor de nuvem pública, implementar essa forma de escalar será uma tarefa *rápida*. Muito do trabalho relacionado a escalar aplicações se reduz à experimentação – ter uma ideia sobre algo que possa melhorar seu sistema, fazer a mudança e avaliar o impacto. Sempre vale a pena executar primeiro as atividades que são rápidas e razoavelmente livres de risco. E a escalabilidade vertical se enquadra nesse caso.

Também vale a pena observar que a escalabilidade vertical pode fazer com que outras formas de escalar sejam mais fáceis. Como um exemplo concreto, passar sua infraestrutura de banco de dados para uma máquina

maior poderia permitir que ela hospedasse os bancos de dados dos microsserviços recém-criados, isolados do ponto de vista lógico, como parte de uma decomposição funcional.

É improvável que seu código ou o banco de dados precisem de qualquer alteração para que façam uso da infraestrutura subjacente maior, supondo que o sistema operacional e os chipsets continuem os mesmos. Ainda que mudanças sejam necessárias em sua aplicação para que ela disponha da mudança no hardware, elas poderiam estar limitadas a aspectos como aumentar a quantidade de memória disponível ao seu runtime por meio de flags de execução.

Limitações

Quando escalamos as máquinas nas quais executamos nossos sistemas, nossas CPUs muitas vezes não ficam mais rápidas; apenas teremos mais cores (núcleos de CPU). Essa foi uma mudança nos últimos 5 a 10 anos. Era comum que cada nova geração de hardware proporcionasse grandes melhorias na velocidade do clock da CPU, o que significava que nossos programas tinham grandes saltos no desempenho. Contudo, as melhorias na velocidade do clock diminuíram drasticamente e, em vez disso, temos um número cada vez maior de cores de CPU para utilizar. O problema é que, em geral, nosso software não é escrito para tirar proveito de um hardware multicore. Isso pode significar que a mudança de sua aplicação de um sistema de 4 para 8 cores poderia resultar em poucas melhorias, ou em nenhuma, mesmo que seu sistema atual seja limitado por CPU (CPU-bound). Alterar o código para tirar proveito de um hardware multicore pode ser um empreendimento significativo, e exigir uma mudança completa nos padrões (idioms) de programação.

Ter uma máquina mais potente também provavelmente não contribuirá muito para melhorar a robustez. Um servidor maior, mais novo, pode oferecer mais confiabilidade, mas, em última instância, se essa máquina ficar inativa, então ela estará inativa. De modo diferente das outras formas de escalar que veremos, é improvável que a escalabilidade vertical tenha muita influência para melhorar a robustez de seu sistema.

Por fim, à medida que as máquinas ficam mais potentes, mais caras elas serão – porém nem sempre de uma maneira proporcional ao aumento de recursos disponíveis a você. Às vezes, ter mais máquinas menos potentes pode ter um custo melhor do que ter menos máquinas mais potentes.

Duplicação horizontal

Na duplicação horizontal, duplicamos parte do sistema para que lide com mais cargas de trabalho. O método exato varia – veremos as implementações em breve – mas, basicamente, a duplicação horizontal exige que tenhamos uma maneira de distribuir o trabalho entre essas duplicatas.

Assim como na escalabilidade vertical, essa forma de escalar está no lado mais simples do espectro, e é uma das opções que eu tentaria antes. Se seu sistema monolítico não é capaz de lidar com a carga, inicie várias cópias dele e veja se isso ajuda!

Implementações

Provavelmente, a forma mais clara de duplicação horizontal que vem à mente é fazer uso de um balanceador de carga (load balancer) para distribuir as requisições entre várias cópias de sua funcionalidade, como vemos na Figura 13.1, em que balanceamos a carga entre várias instâncias do microsserviço catálogo da MusicCorp. Os recursos dos balanceadores de carga diferem, mas espera-se que todos tenham algum método para distribuir a carga entre os nós, além de detectar quando um nó está indisponível e removê-lo do pool do balanceador de carga. Do ponto de vista do consumidor, o balanceador de carga é uma questão de implementação totalmente transparente – podemos encará-lo como parte da fronteira lógica do microsserviço quanto a esse aspecto. Historicamente, os balanceadores de carga eram pensados principalmente como um hardware dedicado, mas isso deixou de ser comum há um bom tempo – em vez disso, o balanceamento de carga é feito cada vez mais por software, geralmente executando do lado do cliente.

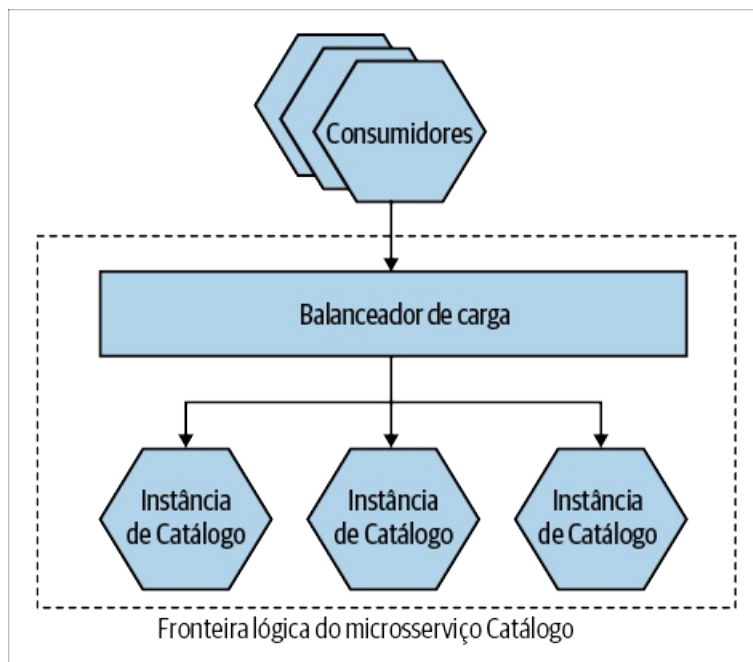


Figura 13.1 – O microserviço Catálogo implantado com várias instâncias, com um balanceador de carga para distribuir as requisições.

Outro exemplo de duplicação horizontal poderia ser o padrão de consumidores concorrentes (*competing consumer pattern*), conforme detalhado no livro *Enterprise Integration Patterns*.³ Na Figura 13.2, vemos o upload de novas músicas na MusicCorp. Essas novas músicas precisam ser transcodificadas em diferentes arquivos para serem usadas como parte do novo produto de streaming da MusicCorp. Temos uma fila comum de tarefas na qual esses jobs são inseridos, e um conjunto de instâncias de Transcodificador de Música consumindo a fila – as diferentes instâncias competem pelos jobs. Para aumentar o throughput do sistema, podemos aumentar o número de instâncias de Transcodificador de Música.

No caso da FoodCo, uma forma de duplicação horizontal foi usada para reduzir a carga de leitura no banco de dados principal, por meio do uso de réplicas de leitura, conforme vemos na Figura 13.3. Isso reduziu a carga de leitura no nó do banco de dados principal, disponibilizando recursos para lidar com as escritas, e funcionou de modo bastante eficaz, pois boa parte da carga de trabalho no sistema principal exigia muitas leituras. Essas leituras podiam ser facilmente redirecionadas para essas réplicas de leitura, e é comum usar um balanceador de carga com várias réplicas de leitura.

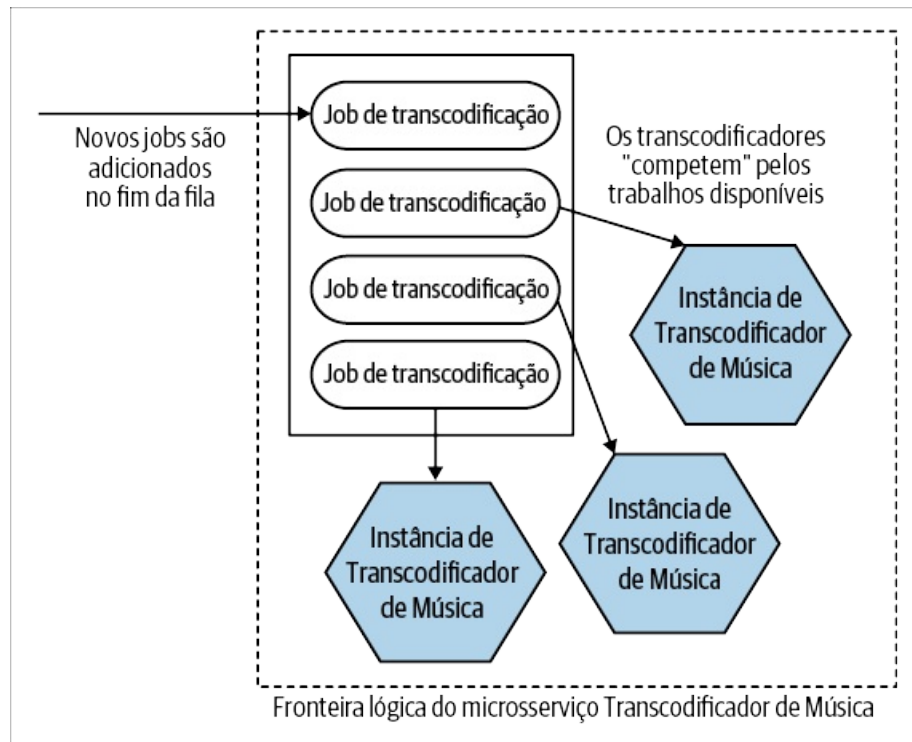


Figura 13.2 – Transcodificação para streaming sendo escalado com o padrão de consumidores concorrentes.

O roteamento para o banco de dados principal ou para uma réplica de leitura é tratado internamente no microsserviço. Para os consumidores desse microsserviço, o fato de uma requisição acabar sendo enviada para o banco de dados principal ou para uma réplica de leitura é transparente.

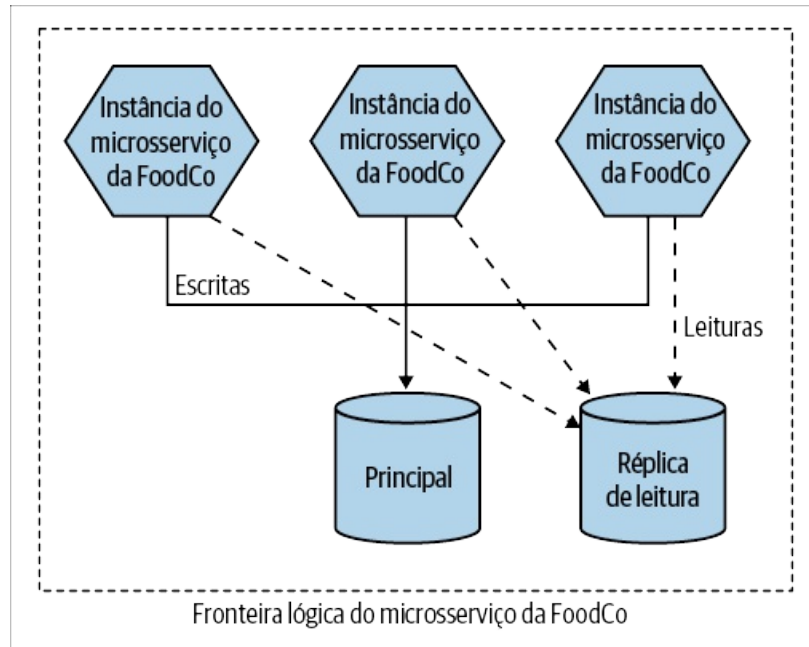


Figura 13.3 – FoodCo fazendo uso de réplicas de leitura para escalar o tráfego de leitura.

Principais vantagens

A duplicação horizontal é relativamente simples. É raro que a aplicação precise ser atualizada, pois o trabalho de distribuir a carga geralmente pode ser feito em outros lugares – por exemplo, por meio de uma fila executando em um broker de mensagens, ou talvez em um balanceador de carga. Se a escalabilidade vertical não estiver disponível para mim, essa forma de escalar em geral seria a próxima opção que eu consideraria.

Supondo que o trabalho possa ser facilmente distribuído entre as duplicatas, essa é uma forma elegante de distribuir a carga e reduzir a contenção em recursos computacionais básicos.

Limitações

Como ocorre praticamente com todas as opções que veremos para escalar um sistema, a duplicação horizontal exige mais infraestrutura, o que, é claro, pode custar mais. Ela pode ser também um instrumento um pouco grosseiro – poderíamos executar várias cópias completas de nossa aplicação monolítica, por exemplo, ainda que somente parte desse sistema estivesse realmente sofrendo de problemas relacionados à escalabilidade.

Boa parte do trabalho nesse caso está na implementação de seus métodos para distribuição de carga. Eles podem variar do simples, como um balanceamento de carga HTTP, ao mais complexo, como o uso de um broker de mensagens ou a configuração de réplicas de leitura do banco de dados. Você contará com esse método de distribuição de carga para fazer o seu trabalho – entender como ele funciona e saber quais são as limitações de sua escolha específica será essencial.

Alguns sistemas podem exigir requisitos adicionais quanto ao método de distribuição de carga. Por exemplo, podem exigir que qualquer requisição associada à mesma sessão de usuário seja direcionada para a mesma réplica. Esse problema pode ser resolvido com o uso de um balanceador de carga que exija um balanceamento com sessões persistentes (sticky session), mas, por sua vez, isso pode limitar os métodos de distribuição de carga que você poderia considerar. Vale a pena notar que sistemas como esses, que exigem balanceamento de carga com sessões persistentes, são suscetíveis a outros problemas e, em geral, eu evitaria criar sistemas que tenham esse requisito.

Partição de dados

Depois de termos começado com as formas mais simples de escalar, entraremos agora em um território mais complexo. A partição de dados exige distribuir a carga com base em algum aspecto dos dados – talvez com base no usuário, por exemplo.

Implementação

A partição de dados funciona tomando-se uma chave associada à carga de trabalho e aplicando a ela uma função; o resultado será a *partição* (às vezes chamada de shard) à qual distribuiremos o trabalho. Na Figura 13.4, temos duas partições, e nossa função é bem simples – enviamos a requisição para um banco de dados se a primeira letra do sobrenome variar de A a M, ou para um banco de dados diferente se variar de N a Z. Contudo, esse, na verdade, é um exemplo ruim de um algoritmo de partição (veremos o porquê em breve), mas espero que seja simples o bastante para demonstrar a ideia.

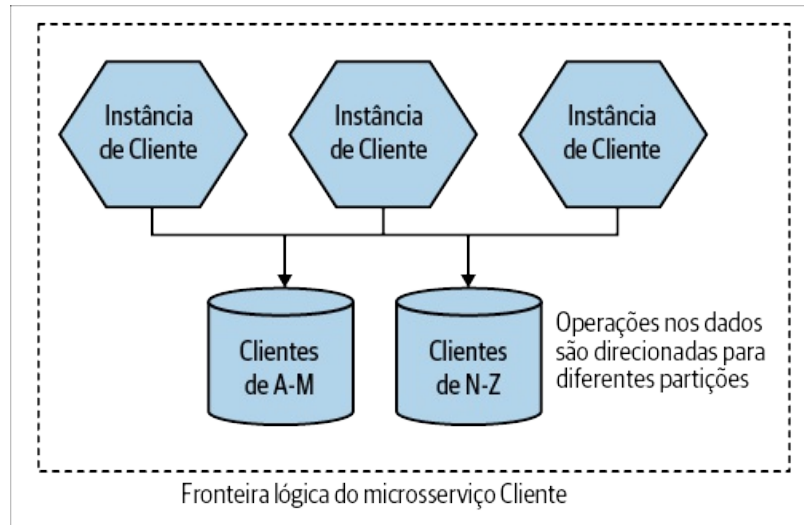


Figura 13.4 – Dados dos clientes são particionados em dois bancos de dados distintos.

Nesse exemplo, fizemos a partição no nível do banco de dados. Uma requisição ao microserviço cliente pode chegar a qualquer instância do microserviço. No entanto, quando executamos uma operação que exija o banco de dados (leituras ou escritas), essa requisição será direcionada para o nó apropriado do banco de dados, com base no nome do cliente. No caso de um banco de dados relacional, o esquema dos dois nós seria idêntico, porém o conteúdo de cada banco de dados se aplicaria somente a um subconjunto dos clientes.

Uma partição no nível do banco de dados em geral faz sentido se a tecnologia de banco de dados em uso aceitar esse conceito de modo nativo, pois você poderá então passar esse problema para uma implementação existente. Entretanto, poderíamos fazer a partição no nível das instâncias do microserviço, conforme vemos na Figura 13.5. Nesse caso, temos de descobrir, com base na requisição de entrada, para qual partição a requisição deve ser mapeada – em nosso exemplo, isso é feito com algum tipo de proxy. No caso de nosso modelo de partição baseado no cliente, se o nome do cliente estiver nos cabeçalhos da requisição, isso bastaria. Essa abordagem faz sentido se quisermos instâncias de microserviço dedicadas para cada partição, o que seria conveniente se houver caching em memória. Também significa que será possível escalar cada partição tanto no nível do banco de dados como no nível das instâncias do microserviço.

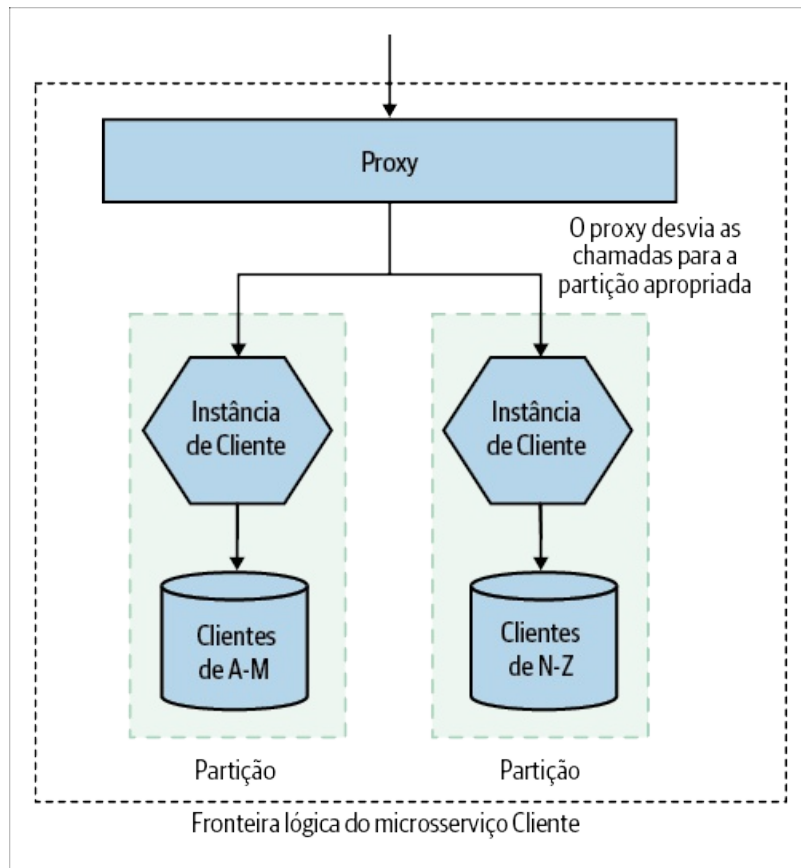


Figura 13.5 – As requisições são direcionadas à instância apropriada do microserviço.

Como no exemplo das réplicas de leitura, queremos que o sistema seja escalado de modo que os consumidores do microserviço não tenham ciência desse detalhe de implementação. Quando um consumidor faz uma requisição ao microserviço cliente na Figura 13.5, queremos que sua requisição seja dinamicamente encaminhada para a partição correta. O fato de termos implementado uma partição de dados deve ser tratado como um detalhe de implementação interno ao microserviço em questão – isso nos dá a liberdade de mudar o esquema de partição ou, quem sabe, substituir totalmente o método de partição.

Outro exemplo comum de partição de dados é fazê-la com base na localização geográfica. Poderíamos ter uma partição por país, ou uma por região.

Na FoodCo, uma opção para lidar com a contenção em seu banco de dados principal seria fazer a partição dos dados com base no país. Assim,

os clientes em Gana acessariam um banco de dados, enquanto os clientes em Jersey acessariam outro. Esse modelo não faria sentido para a FoodCo por uma série de fatores. O principal problema é que a FoodCo tem planos para continuar sua expansão geográfica, e espera aumentar sua eficiência atendendo a várias localidades geográficas com o mesmo sistema. A ideia de ter de iniciar continuamente novas partições para cada país aumentaria drasticamente o custo de entrar em novos países.

Com muita frequência, a partição será feita pelo subsistema do qual você depende. Por exemplo, o Cassandra utiliza partições para distribuir tanto as leituras como as escritas entre os nós em um dado “ring” (anel), enquanto o Kafka aceita uma distribuição de mensagens entre tópicos particionados.

Principais vantagens

A partição de dados escala realmente muito bem para cargas de trabalho transacionais. Se seu sistema é limitado por escritas, por exemplo, a partição de dados pode proporcionar grandes melhorias.

A criação de várias partições também pode fazer com que seja mais fácil reduzir o impacto e o escopo das atividades de manutenção. O rollout de atualizações pode ser feito individualmente por partição e as operações que, de outro modo, exigiriam downtime podem ter o impacto reduzido, pois afetarão apenas uma única partição. Por exemplo, se a partição for feita com base em regiões geográficas, as operações que poderiam resultar em uma interrupção de serviço poderiam ser feitas em um horário que causasse menos impacto, talvez nas primeiras horas da manhã. Uma partição geográfica também pode ser muito conveniente se for necessário garantir que os dados não saiam de determinadas jurisdições – garantindo que dados associados aos cidadãos da União Europeia fiquem armazenados na União Europeia, por exemplo.

A partição de dados pode funcionar bem com a duplicação horizontal – cada partição poderia ser composta de vários nós capazes de lidar com o seu trabalho.

Limitações

Vale a pena destacar que a partição de dados tem uma utilidade limitada no que diz respeito à melhoria da robustez do sistema. Se uma partição falhar, essa parcela de suas requisições falhará. Por exemplo, se sua carga estiver uniformemente distribuída entre quatro partição e uma delas falhar, então 25% de suas requisições acabará falhando. Não é tão ruim quanto uma falha total, mas ainda é muito ruim. É por isso que, conforme mencionado antes, é comum combinar uma partição de dados com uma técnica como a de duplicação horizontal para melhorar a robustez de uma dada partição.

Definir uma chave de partição correta pode ser difícil. Na Figura 13.5, usamos um esquema de partição razoavelmente simples, no qual particionamos a carga de trabalho com base no sobrenome do cliente. Os clientes cujos sobrenomes começam com as letras de A a M são direcionados para a partição 1, enquanto os clientes cujos sobrenomes começam com as letras de N a Z são direcionados para a partição 2. Conforme ressaltai quando apresentei esse exemplo, essa não é uma boa estratégia de partição. Na partição de dados, queremos que haja uma distribuição uniforme da carga. No entanto, não podemos esperar que haja uma distribuição uniforme com o esquema que apresentei. Na China, por exemplo, historicamente, há poucos sobrenomes, e até mesmo hoje em dia, estima-se que há menos de 4 mil. Os 100 sobrenomes mais comuns, que correspondem a mais de 80% da população, tendem fortemente a ser sobrenomes que começam com as letras de N a Z em mandarim. Esse é um exemplo de um esquema para escalar um sistema que provavelmente não resultará em uma distribuição uniforme da carga; se considerarmos diferentes países e culturas, os resultados poderiam variar bastante.

Uma alternativa mais sensata seria fazer a partição com base em um ID único atribuído a cada cliente quando fizessem seu cadastro. É muito mais provável que isso nos dará uma distribuição uniforme da carga, além de dar conta da situação em que alguém alterasse o seu sobrenome.

Acrescentar novas partições em um esquema existente em geral pode ser feito sem que haja muitos problemas. Por exemplo, acrescentar um novo nó em um ring Cassandra não exige nenhum novo balanceamento manual

dos dados; o Cassandra está preparado para distribuir dinamicamente os dados entre os nós. O Kafka também facilita razoavelmente o acréscimo de novas partições posteriormente, embora as mensagens que já estejam em uma partição não sejam movidas – no entanto, os produtores e consumidores podem ser dinamicamente notificados.

A situação se torna mais complicada quando percebemos que nosso esquema de partição simplesmente não atende aos requisitos, como no caso do esquema baseado em sobrenomes que apresentamos antes. Nessa situação, você poderá ter um caminho difícil à frente. Lembro-me de conversar com um cliente muitos anos atrás, que acabou tendo de deixar seu sistema principal offline no ambiente de produção por três dias para alterar o esquema de partição de seu banco de dados principal.

Também pode haver um problema com as consultas. Consultar um único registro é fácil, pois posso simplesmente aplicar a função de hashing para encontrar a instância em que os dados devem estar, e então obtê-los do shard correto. Mas e as consultas cujos dados estejam distribuídos em vários nós – por exemplo, encontrar todos os clientes que tenham mais de 18 anos? Se quiser consultar todos os shards, será necessário consultar cada shard individualmente e fazer uma junção (join) em memória, ou então ter um repositório alternativo de leitura no qual os dois conjuntos de dados estejam disponíveis. Em geral, executar consultas em vários shards é uma tarefa tratada por um método assíncrono, utilizando resultados em cache. O Mongo utiliza jobs map/reduce, por exemplo, para executar essas consultas.

Como você deve ter inferido com base nessa apresentação geral rápida, a situação começa a ficar muito complicada quando queremos escalar bancos de dados para escritas, e é nesse caso que as funcionalidades dos diversos bancos de dados começam a se diferenciar. Com frequência, vejo pessoas mudarem a tecnologia do banco de dados quando começam a atingir limites no que diz respeito à facilidade com que conseguem escalar seu volume atual de escritas. Se isso acontecer com você, comprar uma máquina mais potente muitas vezes será o modo mais rápido de resolver o problema, mas, em segundo plano, você poderia analisar outros tipos de

bancos de dados que possam lidar melhor com os seus requisitos. Com a variedade de tipos de bancos de dados disponíveis, selecionar um novo banco de dados pode ser uma tarefa assustadora; como ponto de partida, porém, recomendo o livro *NoSQL Distilled*,⁴ agradavelmente conciso, que apresenta uma visão geral dos diferentes estilos de bancos de dados NoSQL à sua disposição – de bancos de dados altamente relacionais como os bancos de dados orientados a grafos, a bancos de dados baseados em documentos, colunas e chave-valor.

Basicamente, uma partição de dados implica mais trabalho, sobretudo porque provavelmente exigirá muitas alterações nos dados de seu sistema atual. Contudo, é provável que haja pouco impacto no código da aplicação.

Decomposição funcional

Na decomposição funcional, extraímos uma funcionalidade e permitimos que ela seja escalada de modo independente. Extrair uma funcionalidade de um sistema existente e criar um microserviço é quase sempre o exemplo canônico da decomposição funcional. Na Figura 13.6, vemos um exemplo da MusicCorp na qual a funcionalidade de pedidos é extraída do sistema principal para permitir que essa funcionalidade seja escalada de modo independente das outras partes do sistema.

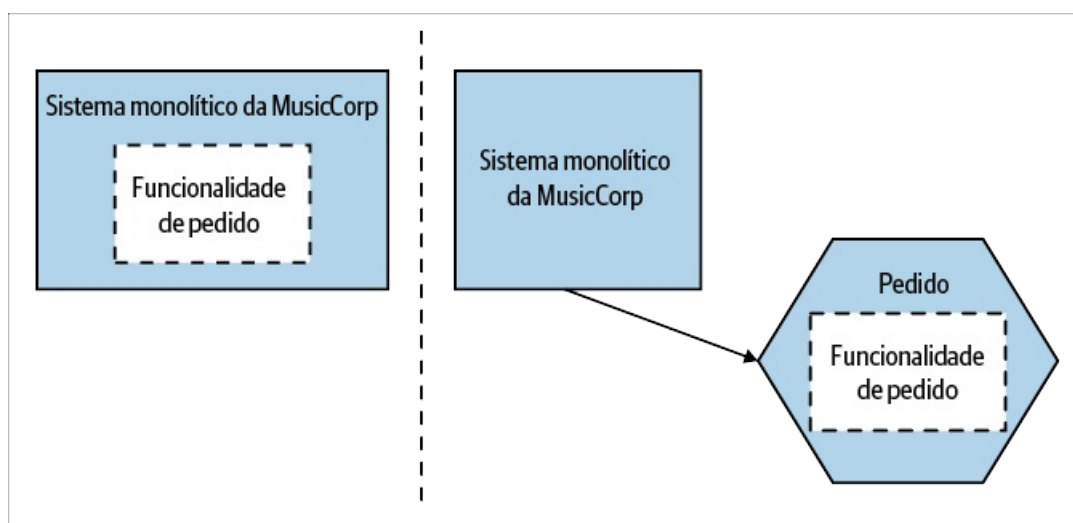


Figura 13.6 – O microserviço Pedido é extraído do sistema atual da MusicCorp.

No caso da FoodCo, essa foi a direção que a empresa tomou. Ela havia chegado ao limite da escalabilidade vertical, havia usado a duplicação horizontal até o máximo que podia, descontando-se a partição de dados, o que restou foi começar a se mover em direção à decomposição funcional. Dados essenciais e cargas de trabalho estão sendo removidos do sistema e do banco de dados básicos para que essa alteração ocorra. Algumas tarefas que constituiriam vitórias rápidas foram identificadas, incluindo a remoção de dados associados a entregas e cardápios do banco de dados principal, passando-os para microsserviços dedicados. Essa solução tem a vantagem adicional de criar oportunidades para que a crescente equipe de entrega de software da FoodCo comece a se organizar com base na responsabilidade sobre esses novos microsserviços.

Implementação

Não vou me demorar muito nesse método de escalar, pois já discutimos bastante o básico sobre os microsserviços neste livro. Para uma discussão mais detalhada sobre como fazer uma mudança como essa acontecer, consulte o Capítulo 3.

Principais vantagens

O fato de termos feito a separação dos diferentes tipos de cargas de trabalho significa que agora será possível dimensionar corretamente a infraestrutura subjacente necessária ao nosso sistema. Uma funcionalidade decomposta que seja usada apenas ocasionalmente poderia ser desativada quando não fosse necessária. Uma funcionalidade que tenha requisitos de carga modestos poderia ser implantada em máquinas menos potentes. Por outro lado, uma funcionalidade que estivesse atualmente limitada poderia ter mais hardware à sua disposição, talvez combinando a decomposição funcional com um dos outros eixos da escalabilidade – por exemplo, executando várias cópias do nosso microsserviço.

Essa capacidade de dimensionar corretamente a infraestrutura necessária para executar as cargas de trabalho nos dá mais flexibilidade para otimizar o custo da infraestrutura de que precisamos para executar o sistema. Esse é um motivo importante pelo qual grandes provedores de SaaS fazem uso tão

intensivo de microsserviços, pois ser capaz de encontrar o equilíbrio correto para os custos da infraestrutura pode ajudar no aumento dos lucros.

Por si só, a decomposição funcional não deixará o nosso sistema mais robusto, mas, ao menos, criará a oportunidade para que criemos um sistema capaz de tolerar uma falha parcial das funcionalidades – assunto que exploramos com mais detalhes no Capítulo 12.

Supondo que você tenha optado pelo caminho dos microsserviços para uma decomposição funcional, haverá mais oportunidades de empregar diferentes tecnologias que permitam escalar os microsserviços decompostos. Por exemplo, seria possível passar a funcionalidade para uma linguagem de programação e um runtime que sejam mais eficientes para o tipo de trabalho que fazem, ou talvez migrar os dados para um banco de dados mais apropriado ao seu tráfego de leituras e escritas.

Embora, neste capítulo, nosso foco esteja principalmente na escalabilidade no contexto de nosso sistema de software, a decomposição funcional também facilita escalar a empresa, um assunto que será retomado no Capítulo 15.

Limitações

Conforme exploramos em detalhes no Capítulo 3, separar as funcionalidades pode ser uma atividade complexa, e é improvável que traga benefícios no curto prazo. De todas as formas de escalar que analisamos, essa provavelmente é aquela que causará mais impacto no código de sua aplicação – tanto no frontend como no backend. Ela também pode exigir um volume significativo de trabalho na camada de dados caso você opte por migrar para microsserviços.

O número de microsserviços executados acabará aumentando, o que aumentará a complexidade geral do sistema – isso pode resultar em mais componentes a serem mantidos, os quais devemos tornar robustos, com a possibilidade de serem escalados. Em geral, quando se trata de escalar um sistema, procuro esgotar todas as demais possibilidades antes de considerar a decomposição funcional. Meu ponto de vista pode mudar se a migração

para os microsserviços puder proporcionar uma série de outras vantagens que a empresa esteja buscando. No caso da FoodCo, por exemplo, sua motivação para aumentar a equipe de desenvolvimento tanto para oferecer suporte a mais países como para disponibilizar mais funcionalidades é essencial, portanto uma migração para microsserviços dá à empresa uma chance de solucionar não só alguns dos problemas de escalabilidade de seus sistemas, mas também dos problemas de escalabilidade da empresa.

Combinando modelos

Uma das principais motivações para o Scale Cube (Cubo de escalabilidade) original era fazer com que parássemos de pensar de modo restrito em um só tipo de forma de escalar, e nos ajudar a entender que, muitas vezes, faz sentido escalar nossa aplicação ao longo de vários eixos, dependendo de nossas necessidades. Vamos voltar ao exemplo apresentado na Figura 13.6. Extraímos nossa funcionalidade `Pedido`, de modo que ela agora pode executar em sua própria infraestrutura. Como próximo passo, poderíamos escalar o microsserviço `Pedido` isoladamente, tendo várias cópias dele, conforme vemos na Figura 13.7.

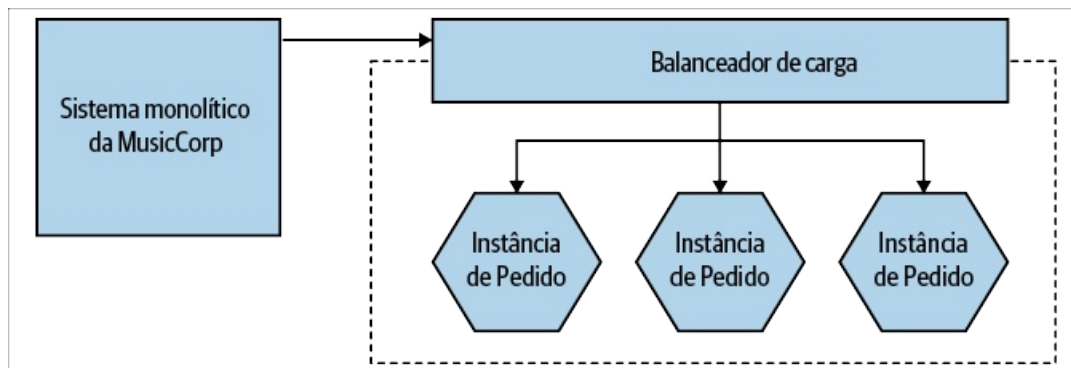


Figura 13.7 – O microsserviço `Pedido` extraído agora está duplicado como uma forma de escalar.

Em seguida, poderíamos decidir executar diferentes shards de nosso microsserviço `Pedido` para diferentes localidades geográficas, como mostra a Figura 13.8. A duplicação horizontal se aplica dentro de cada fronteira geográfica.

Vale a pena observar que, ao escalar em um eixo, pode ser mais fácil

escalar nos demais. Por exemplo, a decomposição funcional de `Pedido` nos permite iniciar várias duplicatas do microserviço `Pedido`, além de fazer uma partição na carga do processamento de pedidos. Sem a decomposição funcional inicial, estaríamos limitados a aplicar essas técnicas no sistema monolítico como um todo.

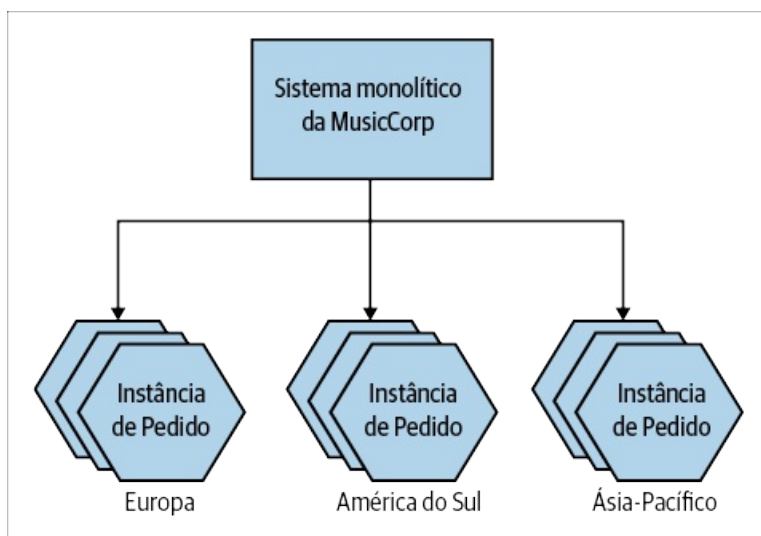


Figura 13.8 – O microserviço `Pedido` da MusicCorp agora está particionado em diferentes regiões geográficas, com duplicação em cada grupo.

O objetivo ao escalar um sistema não é necessariamente escalar em todos os eixos, mas devemos estar cientes de que temos esses diferentes métodos à disposição. Considerando que temos essas opções, é importante conhecer os prós e contras de cada método a fim de determinar qual deles faz mais sentido.

Comece em pequena escala

No livro *The Art of Computer Programming* (Addison-Wesley), Donald Knuth faz uma famosa afirmação:

O verdadeiro problema é que os programadores têm passado tempo demais se preocupando com a eficiência nos lugares errados e nas horas erradas; uma otimização prematura é a raiz de todos os males (ou pelo menos da maioria) na programação.

Otimizar nosso sistema para resolver problemas que não temos é uma

ótima maneira de desperdiçar um tempo que poderia ser mais bem gasto com outras atividades, além de garantir que teremos um sistema que será desnecessariamente mais complexo. Qualquer forma de otimização deve ser motivada por uma necessidade real. Conforme discutimos na seção “Robustez”, na página [456](#), adicionar mais complexidade em nosso sistema também pode introduzir novas causas de fragilidade. Ao escalar uma parte de nossa aplicação, criamos um ponto fraco em outro lugar. Nosso microsserviço `Pedido` pode estar executando agora em sua própria infraestrutura, nos ajudando a lidar melhor com a carga do sistema, porém teremos então outro microsserviço que devemos garantir que esteja disponível se quisermos que nosso sistema funcione, e mais infraestrutura que precisamos tornar robusta e que deve ser gerenciada.

Mesmo que você ache que tenha identificado um gargalo, um processo de experimentação é essencial para garantir que você tem razão e que um trabalho adicional se justifica. Acho impressionante o número de pessoas que se descreveriam como cientistas da computação sem hesitar, mas que parecem não ter nem mesmo uma compreensão básica do método científico.⁵ Se você identificou algo que acha que é um problema, procure determinar um pequeno volume de trabalho que possa ser feito para confirmar se a solução que você propõe irá funcionar. No contexto de escalar sistemas para lidar com carga, ter uma suíte de testes de carga automatizados, por exemplo, pode ser extremamente conveniente. Execute os testes para ter uma base de referência e reproduzir o gargalo que está ocorrendo, faça uma alteração e observe a diferença. Não é uma ciência rigorosa, mas é, ainda que em uma escala bem pequena, uma tentativa de adotar uma abordagem ao menos vagamente científica.

CQRS e event sourcing

O padrão CQRS (Command Query Responsibility Segregation, ou Segregação de Responsabilidade por Comando e Consulta) refere-se a um modelo alternativo para armazenar e consultar informações. Em vez de termos um único modelo tanto para manipular como para consultar dados, como é comum, as responsabilidades pelas leituras e pelas escritas são tratadas por modelos distintos. Esses modelos de leitura e escrita separados, implementados no código, poderiam ser implantados

como unidades distintas, o que nos dá a capacidade de escalar as leituras e as escritas de modo independente. O CQRS, com frequência, embora nem sempre, é usado em conjunto com o *event sourcing* (histórico de eventos), no qual, em vez de armazenar o estado atual de uma entidade como um único registro, projetamos o estado dessa entidade analisando o histórico de eventos relacionados a ela.

Sem dúvida, o CQRS faz algo muito semelhante em nossa camada de aplicação àquilo que as réplicas de leitura podem fazer na camada de dados, embora, por causa das inúmeras maneiras diferentes pelas quais o CQRS pode ser implementado, isso seja uma simplificação.

Pessoalmente, porém, embora eu reconheça a importância do padrão CQRS em algumas situações, acho que é um padrão complexo para ser bem executado. Já conversei com pessoas muito inteligentes que tiveram problemas que não foram insignificantes para fazer o CQRS funcionar. Portanto, se você está considerando o CQRS uma forma de ajudar a escalar sua aplicação, encare-o como uma das formas mais difíceis de escalar que você teria de implementar, ou então tente algumas das opções mais fáceis antes. Por exemplo, se você está simplesmente limitado por causa das leituras, uma réplica de leitura pode ser uma abordagem significativamente menos arriscada e mais rápida para começar. Minhas preocupações quanto à complexidade da implementação se estendem ao *event sourcing* – há algumas situações em que ele é bastante apropriado, mas essa solução traz consigo muitas dores de cabeça que terão de ser resolvidas. Os dois padrões exigem uma grande mudança na mentalidade dos desenvolvedores, o que sempre deixa tudo mais desafiador. Se você decidir usar um desses padrões, certifique-se de que esse aumento na carga cognitivo de seus desenvolvedores é algo que compensa ter.

Uma última observação a respeito do CQRS e do *event sourcing*: do ponto de vista de uma arquitetura de microsserviços, a decisão de usar ou não essas técnicas é um detalhe de implementação interno a um microsserviço. Se você decidiu implementar um microsserviço separando as responsabilidades para as leituras e as escritas em diferentes processos e modelos, por exemplo, isso deve ser invisível aos consumidores do microsserviço. Se as requisições de entrada tiverem de

ser redirecionadas ao modelo apropriado com base na requisição feita, faça com que essa seja uma responsabilidade do microserviço que implementa o CQRS. Manter esses detalhes de implementação ocultos dos consumidores fará com que você tenha bastante flexibilidade caso queira mudar de ideia depois, ou mudar o modo de usar esses padrões.

Caching

O caching é uma otimização de desempenho comum, na qual o resultado anterior de uma operação é armazenado de modo que requisições subsequentes possam utilizar esse valor armazenado, em vez de gastar tempo e recursos recalculando o valor.

Como exemplo, considere um microserviço *Recomendações* que precisa verificar os níveis de estoque antes de recomendar um item – não faz nenhum sentido recomendar algo que não temos em estoque! No entanto, decidimos manter uma cópia local dos níveis de estoque em *Recomendações* (uma forma de caching do lado do cliente) para melhorar a latência de nossas operações – evitamos a necessidade de verificar os níveis de estoque sempre que precisarmos recomendar algo. A fonte da verdade para os níveis de estoque é o microserviço *Estoque*, que é considerado a *origem* do cache do cliente no microserviço *Recomendações*. Quando *Recomendações* precisa consultar um nível de estoque, ele pode consultar antes o seu cache local. Se a entrada que esse microserviço precisa for encontrada, teremos um *cache hit* (acerto no cache). Se o dado não for encontrado, temos um *cache miss* (ausência no cache), que resulta na necessidade de buscar a informação junto ao microserviço downstream *Estoque*. Como os dados podem evidentemente mudar na origem, precisamos de alguma forma de *invalidar* as entradas no cache de *Recomendações* para que saibamos que os dados no cache local estão tão desatualizados a ponto de não poderem mais ser usados.

Os caches podem armazenar os resultados de consultas simples, como nesse exemplo, mas, na verdade, podem armazenar qualquer dado, por exemplo, o resultado de um cálculo complexo. Podemos usar um cache para ajudar a melhorar o desempenho de nosso sistema como parte do

processo de reduzir a latência, escalar nossa aplicação e, em alguns casos, até mesmo melhorar a robustez de nosso sistema. Somado ao fato de que há diversos métodos de invalidação que podemos usar, e vários lugares em que é possível utilizar um cache, isso significa que temos vários aspectos a serem discutidos quando se trata de caching em uma arquitetura de microsserviços. Vamos começar falando dos tipos de problemas com os quais os caches podem ajudar.

Para desempenho

Com os microsserviços, em geral nos preocupamos com o impacto negativo da latência da rede, e com o custo de precisar interagir com vários microsserviços para obter alguns dados. Buscar dados de um cache pode ajudar bastante nesse caso, pois evita a necessidade de fazer chamadas de rede, o que também tem como consequência reduzir a carga nos microsserviços downstream. Além de evitar hops de rede, reduzimos a necessidade de criar dados a cada requisição. Considere uma situação na qual pedimos a lista dos itens mais populares de acordo com o gênero musical. Essa operação poderia envolver uma query custosa de junção no nível do banco de dados. Poderíamos fazer cache do resultado dessa query, o que significa que teremos de gerar novamente o resultado somente quando os dados em cache forem invalidados.

Para escalabilidade

Se for possível direcionar as leituras para os caches, podemos evitar uma contenção em partes de nosso sistema, permitindo que ele escale melhor. Um exemplo disso, o qual já foi discutido neste capítulo, é usar réplicas do banco de dados para leitura. O tráfego de leitura será atendido pelas réplicas de leitura, reduzindo a carga no nó do banco de dados principal e permitindo que as leituras possam ser escaladas com eficácia. As leituras em uma réplica serão feitas em dados que poderão estar desatualizados. A réplica de leitura em algum momento será atualizada por meio da replicação do nó principal para a réplica – essa forma de invalidação de cache é tratada automaticamente pela tecnologia do banco de dados.

De modo mais amplo, o caching com vistas à escalabilidade será

conveniente em qualquer situação na qual a origem seja um ponto de contenção. Colocar caches entre os clientes e a origem pode reduzir a carga na origem, permitindo escalar com mais facilidade.

Para robustez

Se você tem um conjunto completo de dados disponíveis em um cache local, será possível operar ainda que a origem esteja indisponível – por sua vez, isso poderia melhorar a robustez de seu sistema. Há alguns pontos a serem observados a respeito do caching com vistas à robustez. O ponto principal é que provavelmente será preciso configurar seu método de invalidação de cache de modo a não invalidar os dados desatualizados de forma automática, e manter os dados no cache até que possam ser atualizados. Caso contrário, à medida que forem invalidados, os dados serão removidos do cache, resultando em um cache miss (ausência no cache) e uma falha para obter qualquer dado, pois a origem estará indisponível. Isso significa que você precisaria estar preparado para ler dados que possam estar bastante desatualizados se a origem estiver offline. Em algumas situações, esse talvez não seja um problema, enquanto, em outras, seria extremamente complicado.

Basicamente, usar um cache local para ter robustez em uma situação na qual a origem está indisponível significa que você favorecerá a disponibilidade em relação à consistência.

Uma técnica que já vi sendo usada no *the Guardian*, e depois em outro lugar, consistia em percorrer o site “ao vivo” periodicamente a fim de gerar uma versão estática do site que pudesse ser usada no caso de haver uma indisponibilidade do sistema. Embora essa versão não fosse tão atual quanto o conteúdo em cache fornecido pelo sistema ativo, em caso de emergência, ela podia garantir que uma versão do site fosse exibida.

Onde usar um cache

Conforme já mencionamos diversas vezes, os microsserviços oferecem opções. E esse é, sem dúvida, o caso com o caching. Temos vários lugares diferentes em que poderíamos usar um cache. Os diferentes lugares para

um cache, que serão descritos a seguir, têm diferentes relações de custo-benefício, e o tipo de otimização que você estiver tentando fazer provavelmente apontará para o local em que fará mais sentido usar o cache em seu caso.

Para explorar nossas opções de caching, vamos rever uma situação que vimos antes na seção “Preocupações na decomposição de dados”, na página [109](#), quando extraímos informações sobre vendas na MusicCorp. Na Figura 13.9, o microserviço `Vendas` mantém um registro dos itens que foram vendidos. Ele armazena somente o ID do item vendido e a data da venda. Ocasionalmente queremos pedir ao microserviço `Vendas` uma lista dos dez itens mais vendidos nos últimos sete dias.

O problema é que o microserviço `Vendas` não conhece os nomes dos discos, somente os IDs. Não adiantaria muito dizer que “O campeão de vendas desta semana tinha o ID 366548, e vendemos 35.345 cópias!”. Também queremos saber o nome do CD cujo ID é 366548. O microserviço `catálogo` armazena essa informação. Isso significa, conforme mostra a Figura 13.9, que, ao responder à requisição dos dez itens mais vendidos, o microserviço `Vendas` precisa pedir os nomes associados aos IDs dos dez mais vendidos. Vamos ver como o caching poderia nos ajudar, e quais tipos de caches poderiam ser usados.

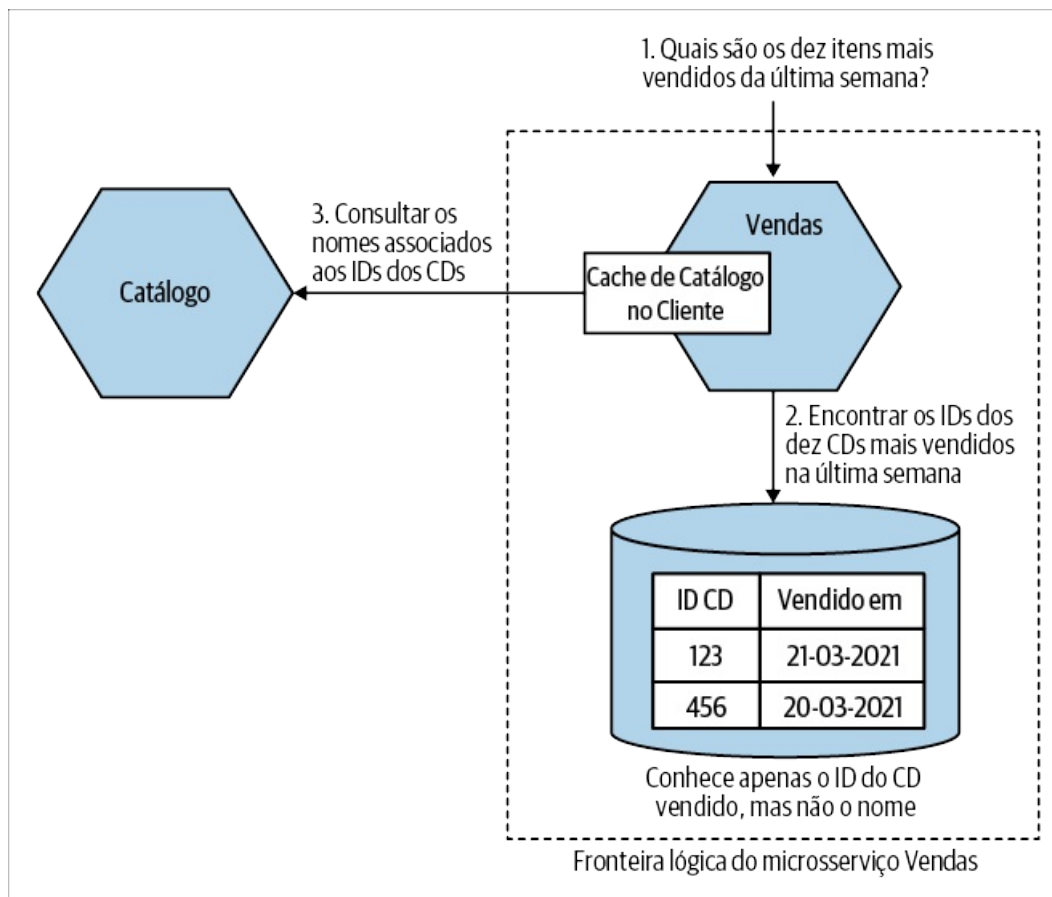


Figura 13.9 – Uma visão geral de como a MusicCorp determina os itens mais vendidos.

Lado do cliente

Com um caching do lado do cliente, os dados são mantidos em um cache fora do escopo da origem. Em nosso exemplo, isso poderia ser feito simplesmente mantendo uma tabela hash em memória, com um mapeamento entre o ID e o nome do álbum dentro do processo **Vendas** em execução, como mostra a Figura 13.10. Isso significa que para gerar a lista dos dez itens mais vendidos, qualquer interação com **catálogo** estará fora do escopo, supondo que tenhamos um cache hit para cada consulta que for preciso fazer. É importante observar que poderíamos decidir que o cache no cliente armazenará somente algumas informações obtidas do microserviço. Por exemplo, poderíamos obter várias informações ao pedir dados de um CD, mas, se tudo que nos interessa for o nome do álbum, é somente esse dado que teremos de armazenar em nosso cache local.

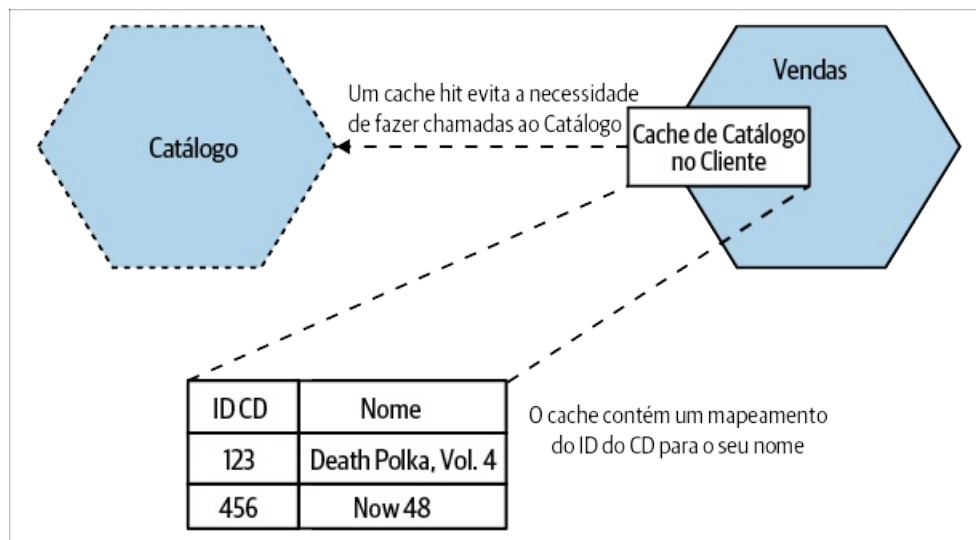


Figura 13.10 – Vendas mantém uma cópia local dos dados de Catálogo.

Em geral, caches do lado do cliente tendem a ser muito eficazes, pois evitam a chamada de rede para o microserviço downstream. Isso os torna apropriados não só para melhorar a latência, mas também para a robustez.

Um caching do lado do cliente, porém, tem algumas desvantagens. Em primeiro lugar, há a tendência de haver mais restrições quanto às opções para os métodos de invalidação – algo que exploraremos em breve. Em segundo, quando houver um volume grande de caching do lado do cliente, pode haver certo grau de inconsistência entre os clientes. Considere uma situação em que os microserviços *Vendas*, *Recomendações* e *Promoções* tenham, cada um, um cache do lado do cliente para os dados de *catálogo*. Quando os dados de *catálogo* mudarem, qualquer método de invalidação que provavelmente utilizarmos será incapaz de garantir que os dados sejam atualizados exatamente no mesmo instante em cada um desses três clientes. Isso significa que poderíamos ter uma visão diferente dos dados em cache em cada um desses clientes ao mesmo tempo. Quanto mais clientes houver, mais problemática provavelmente será essa situação. Técnicas como invalidação baseada em notificação, que veremos em breve, podem ajudar a atenuar o problema, mas não irão eliminá-lo.

Outra forma de atenuar esse problema é ter um cache compartilhado do lado do cliente, talvez fazendo uso de uma ferramenta dedicada de caching como o Redis ou o memcached, conforme mostra a Figura 13.11. Nesse

caso, evitamos o problema da inconsistência entre os diferentes clientes. Essa solução também pode ser mais eficaz no que concerne ao uso de recursos, pois reduzimos o número de cópias dos dados que teremos de gerenciar (os caches geralmente acabam sendo em memória, e a memória muitas vezes é uma das principais limitações na infraestrutura). O outro lado da moeda é que nossos clientes agora terão de fazer um acesso de ida e volta ao cache compartilhado.

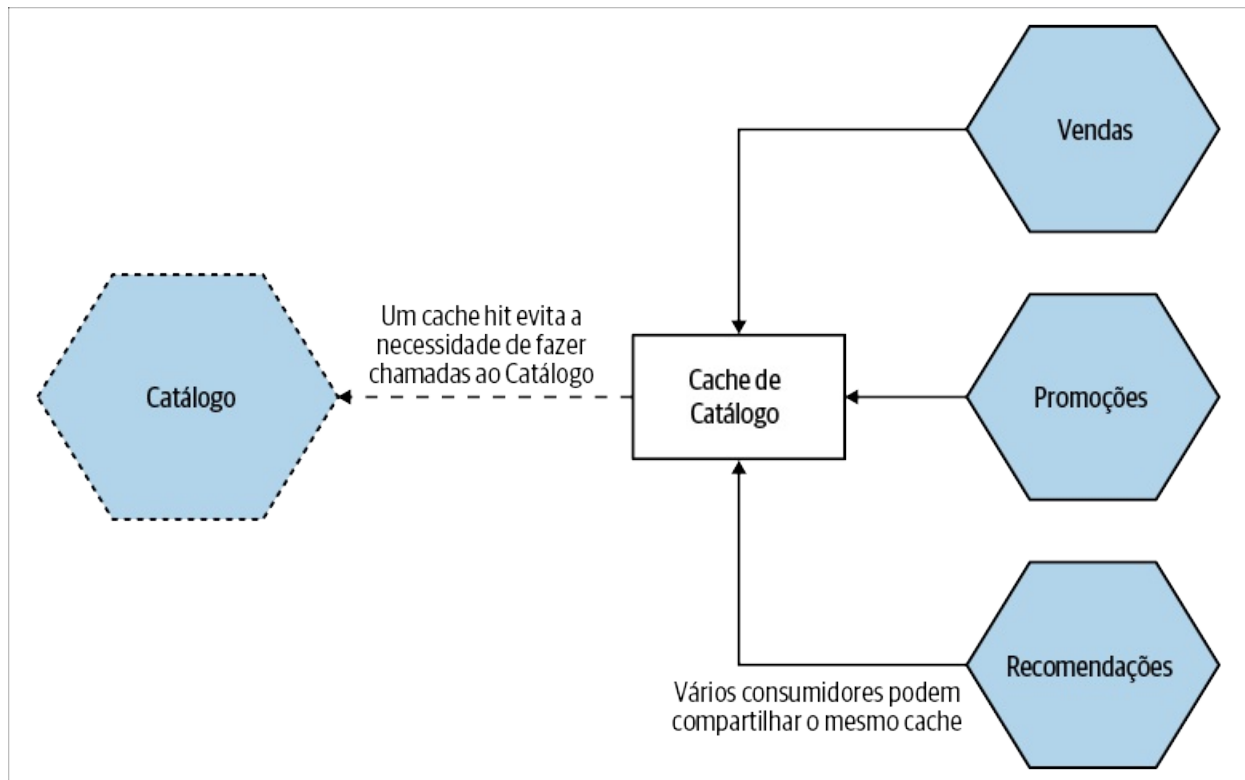


Figura 13.11 – Vários consumidores de catálogo utilizando um único cache compartilhado.

Outro aspecto a ser considerado nesse caso é saber quem é o responsável por esse cache compartilhado. Dependendo de quem for e como isso é implementado, um cache compartilhado como esse pode tornar indistintas as linhas que separam um caching do lado do cliente e um caching do lado do servidor, assunto que exploraremos a seguir.

Lado do servidor

Na Figura 13.12, vemos nosso exemplo dos dez itens mais vendidos fazendo uso de caching do lado do servidor. Nesse caso, o próprio

microserviço `catálogo` mantém um cache para os seus consumidores. Quando o microserviço `vendas` faz sua requisição pedindo os nomes dos CDs, essas informações são disponibilizadas de modo transparente por meio de um cache.

Nesse cenário, o microserviço `catálogo` tem total responsabilidade pelo gerenciamento do cache. Em virtude da natureza da implementação desses caches em geral – por exemplo, com uma estrutura de dados em memória, ou com um nó de caching local dedicado – será mais fácil implementar métodos mais sofisticados de invalidação de cache. Caches write-through, por exemplo (que veremos em breve), seriam muito mais fáceis de implementar nessa situação. Ter um cache do lado do servidor também facilita evitar o problema de diferentes consumidores verem diferentes valores em cache, que pode ocorrer com um caching do lado do cliente.

Vale a pena observar que, embora do ponto de vista do consumidor esse caching seja invisível (é um detalhe de implementação interno), isso não significa que teríamos de implementar essa solução de caching com um código na instância do microserviço. Poderíamos, por exemplo, manter um proxy reverso dentro da fronteira lógica de nosso microserviço, usar um nó Redis oculto ou desviar as consultas de leitura para as réplicas de leitura de um banco de dados.

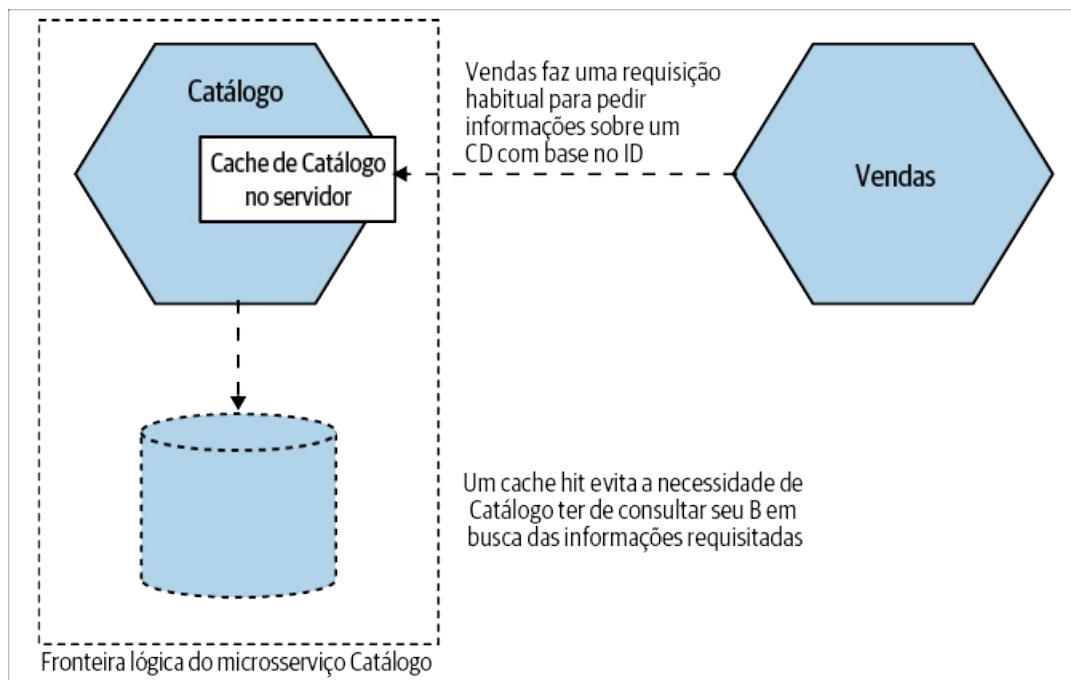


Figura 13.12 – Catálogo implementa um caching internamente, deixando-o invisível aos consumidores.

O principal problema com essa forma de caching é que ela tem um escopo reduzido para otimização com vistas à latência, pois um acesso de ida e volta dos consumidores até o microserviço continuará sendo necessário. Ao fazer um caching no perímetro de um microserviço, ou próximo a ele, o cache é capaz de garantir que não será necessário executar operações custosas adicionais (como queries em um banco de dados), mas a chamada terá de ser feita. Isso também reduz a eficácia dessa forma de caching para qualquer melhoria de robustez.

Pode parecer que essa forma de caching seja menos útil, mas é muito importante melhorar o desempenho para todos os consumidores de um microserviço de modo transparente, somente tomando a decisão de implementar um caching internamente. Um microserviço que seja amplamente usado em toda a empresa pode se beneficiar bastante com a implementação de alguma forma de caching interno, ajudando, talvez, a melhorar os tempos de resposta para diversos consumidores, ao mesmo tempo que permite que o microserviço escale com mais eficácia.

No caso de nosso cenário com os dez itens mais vendidos, teríamos de considerar se essa forma de caching poderia ou não ajudar. Nossa decisão

dependeria basicamente de qual seria nossa principal preocupação. Se a questão for a latência da operação de ponta a ponta, quanto tempo economizaríamos com um cache do lado do servidor? Um caching do lado do cliente provavelmente nos traria mais vantagens do ponto de vista do desempenho.

Cache de requisições

Com um cache de requisições, armazenamos uma resposta da requisição original em cache. Assim, na Figura 13.13 por exemplo, armazenamos as entradas dos dez itens mais vendidos. As requisições subsequentes solicitando os dez itens mais vendidos resultarão na devolução do resultado em cache. Nenhuma consulta nos dados de `Vendas` será necessária, nem acessos de ida e volta ao `Catálogo` – esse é, de longe, o cache mais eficaz no que diz respeito à otimização para velocidade.

As vantagens, nesse caso, são evidentes. Para começar, é uma solução extremamente eficiente. No entanto, precisamos reconhecer que essa forma de caching é extremamente específica. Armazenamos em cache somente o resultado dessa requisição específica. Isso significa que outras operações que cheguem até `Vendas` ou `Catálogo` não terão um acerto no cache e, desse modo, não se beneficiarão de maneira nenhuma com essa forma de otimização.

Invalidação

Há somente duas tarefas difíceis em ciência da computação: invalidar caches e dar nome às coisas.

– *Phil Karlton*

A invalidação é o processo pelo qual retiramos dados de nosso cache. É uma ideia conceitualmente simples, porém complexa quanto à execução e, senão por outros motivos, somente pelo fato de haver inúmeras opções para implementá-la, além de haver vários aspectos de custo-benefício para considerar no que diz respeito a utilizar dados que possam estar desatualizados. Basicamente, porém, a invalidação se reduz a decidir em quais situações um dado deve ser removido do cache. Às vezes, isso

acontece porque somos informados de que uma nova versão de um dado está disponível; em outras ocasiões, talvez tenhamos de supor que nossa cópia em cache está desatualizada, e tivéssemos de buscar uma nova cópia na origem.

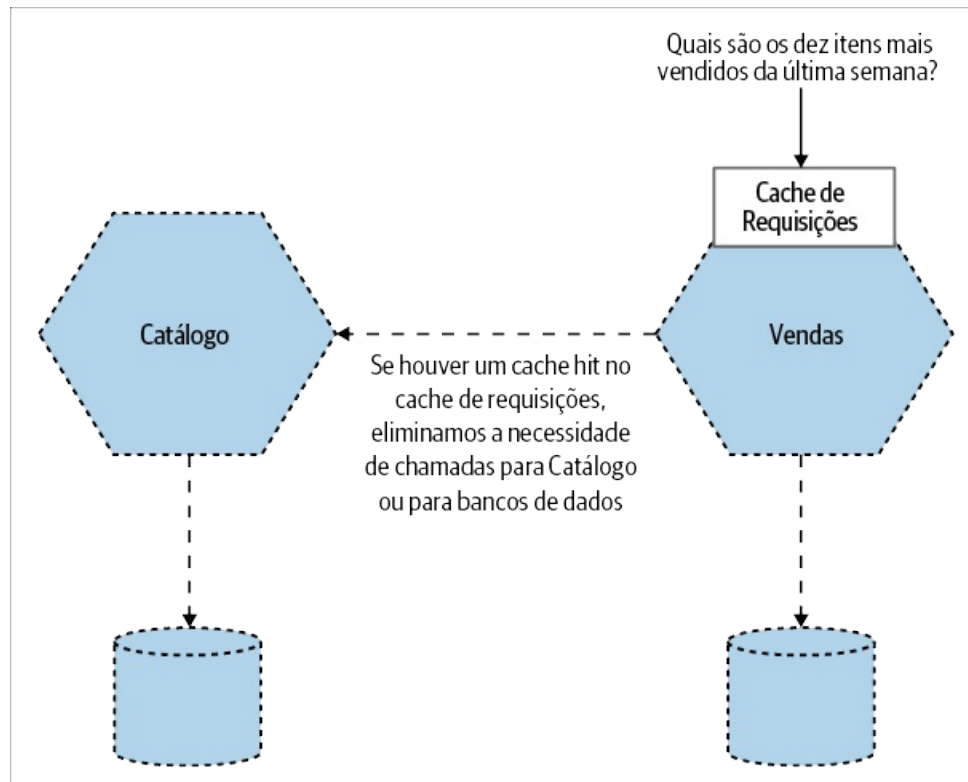


Figura 13.13 – Armazenando em cache os resultados da requisição para os dez itens mais vendidos.

Considerando as opções para invalidação, acho que é uma boa ideia analisar algumas que podem ser usadas em uma arquitetura de microserviços. Não considere, porém, que esta seja uma apresentação completa de todas as opções!

Tempo de vida (TTL)

Esse é um dos métodos mais simples para invalidação de cache. Supomos que cada entrada no cache seja válida somente por um determinado período. Passado esse tempo, o dado será invalidado e buscamos uma nova cópia. Podemos especificar o prazo de validade utilizando uma duração TTL (Time To Live, ou Tempo de Vida) simples – assim, um TTL de cinco minutos significa que nosso cache disponibilizará o dado em cache por até

cinco minutos, após o qual a entrada em cache será considerada inválida e uma nova cópia será necessária. Variações desse tema podem incluir o uso de um timestamp que se esgotará, o que, em algumas situações, pode ser mais eficaz, sobretudo se você estiver lendo em vários níveis de cache.

O HTTP aceita um TTL (por meio do cabeçalho `Cache-Control`), mas também oferece a possibilidade de definir um timestamp como prazo de validade por meio do cabeçalho `Expires` nas respostas, o que pode ser extremamente conveniente. Isso significa que a própria origem é capaz de dizer aos clientes downstream por quanto tempo eles devem supor que os dados estão atualizados. De volta ao nosso microsserviço `Estoque`, poderíamos imaginar uma situação em que esse microsserviço forneça um TTL menor para os níveis de estoque de itens que são rapidamente vendidos ou para itens que estejam quase sem estoque. Para itens que não são muito vendidos, o TTL fornecido poderia ser maior. Essa solução representa um uso, de certa forma, mais sofisticado dos controles de cache do HTTP, e ajustá-los de acordo com cada resposta dessa forma é algo que eu faria somente quando estivesse ajustando a eficácia de um cache. Um TTL simples, usado para todos os casos, para um dado tipo de recurso qualquer será um ponto de partida razoável.

Mesmo que você não utilize HTTP, a ideia de a origem dar pistas ao cliente acerca de como (e se) os dados devem ficar em cache é um conceito muito eficaz. Isso significa que você não precisará adivinhar essa informação do lado do cliente; será possível tomar uma decisão bem fundamentada acerca de como lidar com um dado.

O HTTP possui recursos mais sofisticados de caching do que esse, e veremos os GETs condicionais como exemplo disso em breve.

Um dos desafios da invalidação baseada em TTL é que, embora seja simples de implementar, é um instrumento bastante grosseiro. Se solicitarmos uma nova cópia de um dado que tenha um TTL de cinco minutos, e um segundo depois o dado mudar na origem, nosso cache trabalhará com um dado desatualizado pelos restantes 4 minutos e 59 segundos. Assim, deve haver uma ponderação entre a simplicidade da implementação e o nível de tolerância que temos para operar com um dado

desatualizado.

GETs condicionais

Vale a pena mencionar a capacidade de gerar requisições com GETs condicionais no HTTP, pois muitas vezes esquecemos que esse recurso existe. Conforme já mencionamos rapidamente, o HTTP permite especificar os cabeçalhos `Cache-Control` e `Expires` nas respostas a fim de permitir um caching mais inteligente do lado do cliente. No entanto, se estivermos trabalhando diretamente com o HTTP, teremos outra opção em nosso arsenal de recursos HTTP: as tags de entidade, ou ETags. Uma ETag é usada para determinar se o valor de um recurso mudou. Se um registro de cliente for atualizado, o URI do recurso será o mesmo, porém o valor será diferente, portanto eu esperaria que a ETag mudasse. Isso se tornará importante se usarmos aquilo que chamamos de *GET condicional*. Ao fazer uma requisição GET, podemos especificar cabeçalhos adicionais, dizendo ao serviço que nos envie o recurso somente se determinados critérios forem atendidos.

Por exemplo, vamos supor que buscamos um registro de cliente e a ETag devolvida seja `o5t6fkd2sa`. Mais tarde, talvez porque uma diretiva `Cache-Control` tenha nos informado que o recurso deve ser considerado desatualizado, queremos garantir que obteremos a versão mais recente. Ao enviar a requisição GET subsequente, podemos passar um `If-None-Match: o5t6fkd2sa`. Isso informará ao servidor que queremos o recurso no URI especificado, a menos que ele já corresponda a esse valor de ETag. Se já tivermos a versão atualizada, o serviço enviará uma resposta `304 Not Modified` (Não modificado) dizendo que já temos a versão mais recente. Se houver uma versão mais nova disponível, receberemos um `200 OK` com o recurso alterado e uma nova ETag para o recurso.

É claro que, com um GET condicional, ainda fazemos a requisição do cliente para o servidor. Se você usa caching para reduzir os acessos de ida e volta na rede, essa solução talvez não ajude muito. Sua utilidade está em evitar o custo de gerar recursos novamente, sem que seja necessário. Com uma invalidação baseada em TTL, o cliente pedirá uma nova cópia do recurso, mesmo que esse recurso não tenha mudado – o microsserviço que

receber essa requisição terá de gerar esse recurso novamente, ainda que ele acabe sendo exatamente igual àquele que o cliente já tem. Se o custo de criar a resposta for alto, talvez exigindo um conjunto custoso de queries no banco de dados, as requisições com GETs condicionais poderão ser um método eficiente.

Invalidação baseada em notificação

Com uma invalidação baseada em notificação, usamos eventos para ajudar os componentes interessados nesses eventos a saber se suas entradas no cache local precisam ser invalidadas. Em minha opinião, esse é o método mais elegante de invalidação, embora seja necessário considerar a sua relativa complexidade em relação a uma invalidação baseada em TTL.

Na Figura 13.14, nosso microserviço *Recomendações* mantém um cache do lado do cliente. As entradas nesse cache são invalidadas quando o microserviço *Estoque* gera um evento *Mudança no Estoque*, permitindo que *Recomendações* (ou qualquer outro microserviço que tenha se registrado para receber esse evento) saiba que o nível do estoque aumentou ou diminuiu para um dado item.

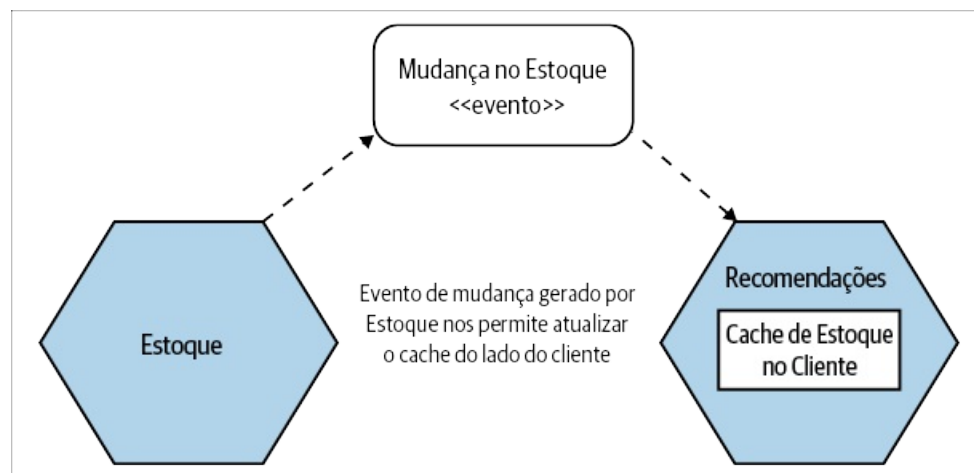


Figura 13.14 – Estoque gera eventos Mudança no Estoque, que Recomendações pode usar para atualizar seu cache local.

A principal vantagem desse método é que ele reduz o possível intervalo de tempo no qual o cache pode fornecer dados desatualizados. O intervalo no qual um cache poderia agora fornecer dados desatualizados é limitado ao tempo necessário para que a notificação seja enviada e processada.

Dependendo do método usado para enviar a notificação, isso poderia ser bem rápido.

A desvantagem, nesse caso, é a complexidade da implementação. Precisamos que a origem seja capaz de gerar notificações, e precisamos que as partes interessadas sejam capazes de responder a essas notificações. Essa é uma ocasião natural para fazer uso de algo como um broker de mensagens, pois esse modelo é bastante apropriado para as típicas interações no estilo pub/sub que muitos brokers oferecem. As garantias adicionais que o broker poderia nos proporcionar também poderão ser convenientes. Apesar disso, conforme já discutimos na seção “Brokers de mensagens”, na página 167, há um overhead para gerenciar um middleware de troca de mensagens, e poderia ser um exagero se o broker fosse usado somente para esse propósito. Se você estiver usando brokers para outras formas de comunicação entre microsserviços, porém, faria sentido fazer uso da tecnologia que já está à sua disposição.

Um problema do qual devemos estar ciente ao usar uma invalidação baseada em notificações é que você talvez queira saber se o sistema de notificações está realmente funcionando ou não. Considere uma situação em que não recebemos nenhum evento *Mudança no Estoque* do microsserviço *Estoque* há um tempo. Isso significa que não vendemos nenhum item ou que houve uma reposição no estoque nesse período? Talvez. Também poderia significar que nosso sistema de notificações está inativo, e que não estamos mais recebendo as atualizações. Se essa for uma preocupação, poderíamos enviar um evento de heartbeat (batimento cardíaco) pelo mesmo sistema de notificações – *Estoque*, em nosso caso – para permitir que os microsserviços que se registraram para receber as notificações saibam que elas continuam chegando, mas que nada realmente mudou. Se um evento de heartbeat não for recebido, o cliente poderá supor que há um problema e fazer o que for mais apropriado – talvez informar os usuários de que estão vendo um dado desatualizado ou, quem sabe, simplesmente desativar a funcionalidade.

Também precisamos considerar o que a notificação deve conter. Se a notificação simplesmente informar que “houve mudança nesse item”, sem

dizer qual foi a mudança, ao receber a notificação, um consumidor teria de acessar a origem e buscar os novos dados. Por outro lado, se a notificação contiver o estado atual dos dados, os consumidores poderão carregá-los diretamente para o seu cache local. Ter uma notificação que contenha mais dados pode causar problemas relacionados ao tamanho, além de haver o risco em potencial de expor dados sigilosos de modo muito amplo. Exploramos antes essa relação de custo-benefício com mais detalhes, quando discutimos a comunicação orientada a eventos na seção “O que há em um evento?”, na página [141](#).

Write-through

Com um cache write-through, o cache é atualizado ao mesmo tempo que o estado na origem. “Ao mesmo tempo” é o ponto em que os caches write-through se tornam complicados, é claro. Implementar uma solução write-through em um cache do lado do servidor é, de certo modo, simples, pois poderíamos atualizar um banco de dados e um cache em memória na mesma transação, sem muita dificuldade. Se o cache estiver em outro lugar, será mais difícil pensar no que significa “ao mesmo tempo” no que diz respeito à atualização dessas entradas.

Por causa dessa dificuldade, geralmente veríamos um caching write-through ser usado em uma arquitetura de microsserviços do lado do servidor. As vantagens são muito claras – o intervalo durante o qual um cliente poderia ver dados desatualizados seria praticamente eliminado. Devemos ponderar isso em relação ao fato de que caches do lado do servidor, de modo geral, podem ser menos úteis, limitando as circunstâncias em que um cache write-through seria eficaz com os microsserviços.

Write-behind

Em um cache write-behind, o cache em questão é atualizado *antes*, e então a origem é atualizada. Conceitualmente, podemos pensar no cache como um buffer. Escrever no cache é mais rápido que atualizar a origem. Assim, escrevemos o resultado no cache, permitindo leituras subsequentes mais rápidas, e acreditamos que a origem será atualizada depois.

A principal preocupação relacionada aos caches write-behind será a possibilidade de haver perda de dados. Se o cache em questão não for durável, poderíamos perder os dados antes que fossem escritos na origem. Além do mais, estamos agora em uma situação interessante – o que é a *origem* nesse contexto? Esperaríamos que a origem fosse o microserviço do qual esses dados são provenientes – mas, se atualizarmos o cache antes, ele será realmente a origem? Qual é a nossa fonte da verdade? Ao fazer uso de caching, é importante diferenciar quais são os dados em cache (com a possibilidade de estarem desatualizados) e quais são os dados que realmente podem ser considerados atualizados. Os caches write-behind no contexto dos microserviços deixam isso muito menos claro.

Embora os caches write-behind muitas vezes sejam usados para otimizações internas aos processos, eu os vejo muito raramente em arquiteturas de microserviços, parcialmente porque há outras formas mais simples de caching que seriam suficientes, mas principalmente por causa da complexidade de lidar com a perda de dados em cache, e que não foram escritos.

Regra de ouro do caching

Tome cuidado com o uso de caching em muitos lugares! Quanto mais caches houver entre você e a fonte de dados atualizados, mais desatualizados poderão estar os dados, e mais difícil será determinar se os dados que um cliente vê estão atualizados. Também poderá ser mais difícil saber em quais lugares os dados precisam ser invalidados. A relação de custo-benefício associada ao caching – encontrar um equilíbrio entre os dados estarem atualizados e a otimização de seu sistema com vistas à carga ou latência – é delicada; se você não puder determinar facilmente até que ponto os dados estão (ou não) atualizados, essa pode ser uma tarefa difícil.

Considere uma situação em que o microserviço *Estoque* faz caching dos níveis de estoque. As requisições ao microserviço *Estoque* solicitando os níveis de estoque podem ser atendidas com base nesse cache do lado do servidor, agilizando, desse modo, a requisição. Vamos agora supor também que definimos um TTL de um minuto para esse cache interno, o que

significa que o nosso cache do lado do servidor poderia estar até um minuto atrasado em relação ao nível de estoque em questão. O fato, porém, é que estamos também fazendo caching do lado do cliente em *Recomendações*, usando também um TTL de um minuto. Quando uma entrada no cache do lado do cliente expira, fazemos uma requisição de *Recomendações* para *Estoque* a fim de obter um nível de estoque atualizado; contudo, sem que seja do nosso conhecimento, nossa requisição chega ao cache do lado do servidor, o qual, nesse momento, também poderia estar desatualizado em até um minuto. Assim, poderíamos acabar armazenando um registro em nosso cache do lado do cliente que já esteja um minuto desatualizado desde o princípio. Isso significa que os níveis de estoque que *Recomendações* está usando poderiam estar até *dois* minutos desatualizados, ainda que, do ponto de vista de *Recomendações*, achemos que poderiam estar desatualizados em no máximo um minuto.

Há diversas maneiras de evitar problemas como esse. Usar um prazo de validade baseado em timestamp, para começar, seria melhor que usar TTLs, mas é também um exemplo do que acontece quando o caching é efetivamente aninhado. Se você armazenar o resultado de uma operação em cache, o qual, por sua vez, se baseia em entradas que estão em cache, como você saberá até que ponto o resultado está atualizado?

Voltando à famosa citação anterior de Knuth, uma otimização prematura pode causar problemas. O caching acrescenta complexidade, e queremos o acréscimo do mínimo possível de complexidade. O número ideal de lugares para usar um cache é zero. Qualquer outra opção deve resultar de uma otimização que você *precisa* fazer – porém, esteja ciente da complexidade que isso traz.



Trate o caching basicamente como uma otimização com vistas ao desempenho. Utilize um cache no mínimo possível de lugares para que seja mais fácil saber quão atualizados estão os seus dados.

Dados atualizados versus otimização

Voltando ao nosso exemplo da invalidação com base em TTL, expliquei antes que, se solicitarmos uma nova cópia de um dado que tenha um TTL

de cinco minutos, e um segundo depois o dado mudar na origem, nosso cache irá trabalhar com um dado desatualizado pelos restantes 4 minutos e 59 segundos. Se isso for inaceitável, uma solução seria reduzir o TTL, diminuindo, assim, o intervalo no qual poderíamos atuar com dados desatualizados. Desse modo, poderíamos reduzir o TTL para um minuto. Isso significa que o período em que teremos dados desatualizados será reduzido para um quinto do que tínhamos antes, porém faremos cinco vezes mais chamadas à origem, portanto temos de considerar a latência associada e o impacto na carga.

Encontrar um equilíbrio entre essas forças se reduzirá a entender os requisitos do usuário final e do sistema mais amplo. Obviamente, os usuários sempre querem operar com os dados mais atualizados possíveis, mas não se isso significa que o sistema vai falhar por causa da carga. De modo semelhante, às vezes, a atitude mais segura será desativar as funcionalidades caso um cache falhe a fim de evitar que uma sobrecarga na origem cause problemas mais graves. Quanto se trata de fazer ajustes detalhados acerca dos dados a serem colocados em cache, onde usar os caches e como usá-los, em geral você se verá tendo de encontrar um equilíbrio entre vários eixos. Esse é apenas outro motivo para tentar manter o máximo de simplicidade possível – quanto menos caches houver, mais fácil será entender o sistema.

Envenenamento de cache: uma lição

Com o caching, muitas vezes achamos que, se cometermos um erro, o pior que poderia acontecer é fornecer dados desatualizados por um tempo. Mas o que acontecerá se você acabar fornecendo dados desatualizados indefinidamente? No Capítulo 12, apresentei a AdvertCorp, empresa na qual trabalhei para ajudar na migração de uma série de aplicações legadas para uma nova plataforma, fazendo uso do padrão strangler fig. Essa tarefa envolvia interceptar chamadas para várias aplicações legadas, e se essas aplicações tivessem sido movidas para a nova plataforma, as chamadas eram desviadas. Nossa nova aplicação funcionava efetivamente como um proxy. O tráfego para as aplicações legadas mais antigas, que ainda não haviam sido migradas, era encaminhado por meio de nossa nova aplicação

para as aplicações legadas downstream. Para as chamadas às aplicações legadas, fizemos algumas tarefas para deixar o sistema mais organizado; por exemplo, garantimos que os resultados da aplicação legada tivessem os devidos cabeçalhos de cache HTTP aplicados.

Certo dia, logo após um lançamento de rotina, algo estranho começou a acontecer. Um bug havia sido introduzido, e, como consequência, um pequeno subconjunto de páginas caía em uma condição default em uma lógica de nosso código de inserção de cabeçalho para cache e, como resultado, nenhum header era alterado. Infelizmente, essa aplicação downstream também havia sido alterada um tempo atrás e havia incluído um cabeçalho HTTP `Expires: Never`. Até então, essa mudança não havia tido nenhum efeito, pois estávamos sobrescrevendo esse cabeçalho. Agora não estávamos mais.

Nossa aplicação fazia uso intensivo do Squid para cache de tráfego HTTP, e percebemos rapidamente o problema, pois estávamos vendo mais requisições passando direto pelo próprio Squid e chegando até os nossos servidores de aplicação. Corrigimos o código do cabeçalho de cache e fizemos um novo lançamento de versão, além de termos limpado manualmente a parte relevante do cache do Squid. No entanto, isso não bastou.

Conforme acabamos de discutir, podemos ter cache em vários lugares – às vezes, porém, ter muitos caches dificulta a vida, em vez de facilitá-la. Para fornecer conteúdo aos usuários de uma aplicação web voltada ao público, vários caches podem existir entre você e seu cliente. Você não só poderia ter diante de seu site algo como uma rede de fornecimento de conteúdo (content delivery network), como também alguns ISPs fazem uso de caching. Você é capaz de controlar esses caches? E mesmo que pudesse, há um cache sobre o qual você teria pouco controle: o cache no navegador de um usuário.

Aquelas páginas com `Expires: Never` permaneceram nos caches de muitos de nossos usuários, e jamais seriam invalidados até que o cache enchesse ou o usuário os limpasse manualmente. É claro que não podíamos fazer nenhum desses fatos acontecerem; nossa única opção foi alterar os URLs

dessas páginas para que fossem buscadas novamente.

O caching pode ser realmente muito eficaz, mas precisamos entender o caminho completo dos dados em cache, da origem até o destino, para avaliar suas complexidades e saber o que pode dar errado.

Escalabilidade automática

Se tiver a sorte de ter um provisionamento de hosts virtuais totalmente automatizado e puder automatizar por completo a implantação das instâncias de seus microsserviços, você terá os blocos de construção que permitem escalar seus microsserviços automaticamente.

Por exemplo, a ação de escalar poderia ser disparada por tendências bem conhecidas. Você poderia saber que a carga de pico de seu sistema ocorre entre 09:00 e 17:00 horas, portanto adicionará outras instâncias às 8:45, desativando-as às 17:15 horas. Se estiver usando algo como a AWS (que tem um suporte muito bom para escalabilidade automática), desativar instâncias que não sejam mais necessárias ajudará você a economizar. Será preciso ter dados para saber como a sua carga muda com o tempo, de dia para dia e de semana para semana. Alguns negócios também têm ciclos sazonais evidentes, portanto talvez seja necessário ter dados com um histórico bem longo a fim de tomar as decisões apropriadas.

Por outro lado, você poderia ter uma atitude reativa, incluindo instâncias adicionais quando perceber um aumento na carga ou se houver uma falha de instância, removendo essas instâncias quando não precisar mais delas. Saber a rapidez com que é possível escalar assim que identificar uma alta na tendência é essencial. Se você souber que será informado com apenas alguns minutos de antecedência a respeito de um aumento na carga, porém escalar o sistema é uma operação que demorará pelo menos dez minutos, você sabe que precisará ter uma capacidade sobressalente para cobrir essa lacuna. Ter uma boa suíte de testes de carga é quase sempre fundamental nesse caso. Elas podem ser usadas para testar suas regras para escalar automaticamente o sistema. Se não houver testes que possam reproduzir as diferentes cargas que irão disparar a operação de escalar, você descobrirá se suas regras estão incorretas somente no ambiente de produção. E as

consequências de uma falha não serão agradáveis!

Um site de notícias é um ótimo exemplo de um tipo de negócios em que você poderia querer uma combinação de escalabilidade prevista e reativa. No último site de notícias em que trabalhei, víamos claramente as tendências diárias, subindo da manhã até o horário do almoço, e então começando a declinar. Esse padrão era repetido diariamente, com um tráfego geralmente mais baixo nos fins de semana. Isso nos mostrava uma tendência razoavelmente clara, capaz de direcionar uma ação proativa para redimensionar os recursos, para cima ou para baixo. Por outro lado, uma notícia extraordinária poderia causar um pico inesperado, exigindo mais capacidade e, com frequência, a curto prazo.

Na verdade, vejo a escalabilidade automática ser muito mais usada para lidar com falha de instâncias do que para reagir a condições de carga. A AWS permite especificar regras como “Deve haver pelo menos cinco instâncias nesse grupo”, de modo que, se uma instância falhar, uma nova será automaticamente iniciada. Já vi essa abordagem levar a um jogo divertido de “acerte a toupeira”⁶, quando alguém se esquece de desativar a regra e então tenta desativar as instâncias para manutenção, somente para vê-las sendo reiniciadas!

Escalar de forma tanto reativa ou prevista é muito conveniente, e pode ajudar a reduzir os custos caso você esteja usando uma plataforma que permita pagar somente pelos recursos computacionais utilizados. No entanto, isso também exige uma observação atenta dos dados disponíveis a você. Eu sugeriria fazer uso da escalabilidade automática para condições de falha antes, enquanto uma coleta de dados é feita. Assim que quiser começar a usar a escalabilidade automática para carga, certifique-se de ser muito cauteloso para não reduzir os recursos de modo excessivamente prematuro. Na maioria das situações, ter mais capacidade de processamento que o necessário à disposição será muito melhor do que não ter capacidade suficiente!

Começando de novo

A arquitetura com a qual você começou pode não ser a arquitetura que

continuará em uso quando seu sistema tiver de lidar com volumes muito diferentes de carga. Conforme já vimos, há algumas formas de escalar que podem ter um impacto extremamente limitado na arquitetura de seu sistema – a escalabilidade vertical e a duplicação horizontal, por exemplo. Em determinados momentos, porém, será preciso fazer algo bastante radical e modificar a arquitetura de seu sistema para se adequar ao seu próximo patamar de crescimento.

Lembre-se da história da Gilt, que mencionamos na seção “Execução isolada”, na página [273](#). Uma simples aplicação Rails monolítica funcionou bem para a Gilt por dois anos. Seus negócios se tornaram cada vez mais bem-sucedidos, implicando mais clientes e mais carga. Em um determinado momento de auge da empresa, ela precisou refazer o design da aplicação para lidar com a carga que estava vendo.

Uma remodelagem pode significar a separação de um sistema monolítico existente, como fez a Gilt. Ou poderia significar escolher novos bancos de dados capazes de lidar melhor com a carga. Também poderia implicar a adoção de novas técnicas, por exemplo, passar de sistemas síncronos com requisição-resposta para sistemas baseados em eventos, adotar novas plataformas de implantação, mudar conjuntos completos de tecnologias, e qualquer mudanças desse tipo.

Há o perigo de as pessoas verem a necessidade de remodelar o sistema quando determinados limites para escalar são atingidos como um motivo para construir o sistema com vistas a uma escala gigantesca desde o princípio. Isso pode ser desastroso. No início de um novo projeto, em geral não sabemos exatamente o que queremos construir, nem sabemos se teremos sucesso. Precisamos ser capazes de fazer experimentos rapidamente e saber quais são as funcionalidades que devemos implementar. Se tentássemos criar o sistema já visando a uma escala gigantesca, acabaríamos com um volume enorme de trabalho prévio para nos prepararmos para uma carga que talvez jamais tivéssemos, ao mesmo tempo que desperdiçaríamos esforços que poderiam ser investidos em atividades mais importante, como saber se alguém realmente vai querer usar o nosso produto. Eric Ries conta a história de gastar seis meses

criando um produto que ninguém jamais baixou. Refletindo sobre o assunto, diz que poderia ter colocado um link em uma página web que devolvesse 404 quando as pessoas clicassem aí para ver se haveria alguma demanda, podia ter passado seis meses na praia, e teria aprendido o mesmo!

A necessidade de modificar nossos sistemas para lidar com escala não é um sinal de fracasso. É um sinal de sucesso.

Resumo

Como podemos ver, qualquer que seja a forma de escalar que você esteja procurando, os microsserviços oferecem muitas opções diferentes quanto ao modo de abordar o problema.

Os eixos de escalabilidade podem ser um modelo conveniente a ser usado ao considerarmos as formas de escalar que temos à disposição:

Escalabilidade vertical

Em poucas palavras, significa adquirir uma máquina mais potente.

Duplicação horizontal

Ter vários itens capazes de fazer o mesmo trabalho.

Partição de dados

Dividir o trabalho com base em algum atributo dos dados, por exemplo, o grupo ao qual pertence o cliente.

Decomposição funcional

Separação do trabalho com base no tipo, por exemplo, uma decomposição em microsserviços.

Essencial para boa parte desses casos é saber o que você quer – técnicas que são eficazes para escalar com vistas à latência podem não ser tão eficazes para ajudar a escalar com vistas a volume.

Contudo, espero ter mostrado também que muitas das formas de escalar que discutimos resultam em um aumento de complexidade no sistema. Portanto, ter uma meta clara acerca do que você está tentando alterar e evitar os perigos de uma otimização prematura é importante.

A seguir, passaremos da discussão sobre o que acontece internamente em nossos sistemas para as suas partes mais visíveis – a interface de usuário.

- 1 Martin L. Abbott e Michael T. Fisher, *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise*, 2ª edição (Nova York: Addison-Wesley, 2015).
- 2 Como já fiz antes, preservei o anonimato da empresa – FoodCo não é o seu nome verdadeiro!
- 3 Gregor Hohpe e Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).
- 4 Pramod J. Sadalage e Martin Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence* (Upper Saddle River, NJ: Addison-Wesley 2012). [N.T.: Edição publicada no Brasil: *NoSQL Essencial, um guia conciso para o mundo emergente da persistência poliglota* (Novatec, 2013).]
- 5 Sem falar das pessoas que começam a falar de hipóteses, e então passam a selecionar informações específicas para confirmar suas crenças previamente estabelecidas.
- 6 N.T.: “Acerte a toupeira”, ou *whack-a-mole* em inglês, é um típico jogo de arcade em que, para pontuar, o jogador deve acertar as toupeiras que saem aleatoriamente de suas tocas com uma espécie de bastão ou martelo.

PARTE III

Pessoas

Interfaces de usuário

Até agora, ainda não discutimos realmente o mundo das interfaces de usuário. Alguns de vocês talvez estejam disponibilizando apenas uma API fria, séria e austera para seus clientes, mas muitos de nós nos vemos querendo criar belas interfaces de usuário funcionais, que encantarão nossos clientes. Afinal de contas, a interface de usuário é onde todos esses microserviços serão reunidos em algo que faça sentido aos nossos consumidores.

Quando comecei na área de computação, falávamos basicamente de clientes grandes e pesados que executavam em nossos desktops. Passei muitas horas no Motif, e depois no Swing, tentando criar o melhor software possível para ser usado. Com frequência, esses sistemas serviam apenas para a criação e manipulação de arquivos locais, porém muitos deles tinham um componente do lado do servidor. Meu primeiro trabalho na Thoughtworks envolvia criar um sistema eletrônico para os pontos de venda com base no Swing; essa era apenas uma das várias partes constituintes do sistema, a maioria das quais estava no servidor.

Então surgiu a web. Começamos a pensar em nossas UIs como mais “magras”, com mais lógica do lado do servidor. No princípio, nossos programas do lado do servidor renderizavam a página toda e a enviavam ao navegador do cliente, que fazia muito pouco. Quaisquer interações eram tratadas do lado do servidor por meio de GETs e POSTs disparados por cliques do usuário em links ou pelo preenchimento de formulários. Com o tempo, o JavaScript se tornou uma opção mais popular para agregar um comportamento dinâmico à UI no navegador, e algumas aplicações, sem dúvida, podiam ser tão “gordas” quanto os antigos clientes desktop. Na sequência, tivemos a ascensão dos aplicativos móveis e, atualmente, temos

um panorama diversificado para disponibilizar interfaces gráficas aos nossos usuários – diferentes plataformas, e diferentes tecnologias para essas plataformas. Essa variedade de tecnologias nos dá uma série de opções para criar interfaces de usuário eficazes, com apoio dos microsserviços. Exploraremos tudo isso com mais detalhes neste capítulo.

Em direção ao digital

Nos últimos anos, as empresas começaram a se afastar da ideia de pensar que o mundo web e o mundo móvel deveriam ser tratados de modo distinto, passando a pensar mais holisticamente no mundo digital. Qual é a melhor maneira de os nossos clientes usarem os serviços que oferecemos? E o que isso implica para a arquitetura do nosso sistema? A compreensão de que não é possível prever exatamente como um cliente pode acabar interagindo com nossos produtos tem levado à adoção de APIs mais específicas, como aquelas disponibilizadas pelos microsserviços. Ao combinar as funcionalidades expostas pelos nossos microsserviços de diversas maneiras, podemos criar diferentes experiências para nossos clientes em sua aplicação desktop, em um dispositivo móvel ou um dispositivo wearable (vestível), e até mesmo fisicamente, caso eles visitem nossa loja física.

Portanto, pense nas interfaces de usuário como os lugares em que entrelaçamos os diversos fios das funcionalidades que queremos oferecer aos nossos usuários. Com isso em mente, como podemos reunir todos esses fios? Precisamos encarar esse problema de dois lados: o quem e o como. Em primeiro lugar, vamos considerar os aspectos organizacionais – quem é responsável por o quê, quando se trata de entregar interfaces de usuário? Em segundo, veremos um conjunto de padrões que pode ser usado para implementar essas interfaces.

Modelos de responsabilidade

Conforme discutimos lá atrás no Capítulo 1, a tradicional arquitetura em camadas pode causar problemas para uma entrega de software eficaz. Na

Figura 14.1, vemos um exemplo no qual a responsabilidade pela camada de interface de usuário é de uma única equipe de frontend, enquanto o trabalho nos serviços do backend é feito por outra equipe. Nesse exemplo, acrescentar um simples controle envolve um trabalho feito por três equipes diferentes. Esses tipos de estruturas organizacionais em camadas podem causar impactos significativos em nossa velocidade de entrega por causa da necessidade constante de coordenação para fazer as mudanças, com o trabalho sendo passado de uma equipe para outra.

O modelo de que mais gosto, e que acho que está mais alinhado com a meta de ter implantações independentes, é ter a UI separada e gerenciada por uma equipe que também gerencie os componentes do lado do servidor, conforme vemos na Figura 14.2. Nesse caso, uma única equipe acaba sendo responsável por todas as mudanças necessárias para o acréscimo do novo controle.

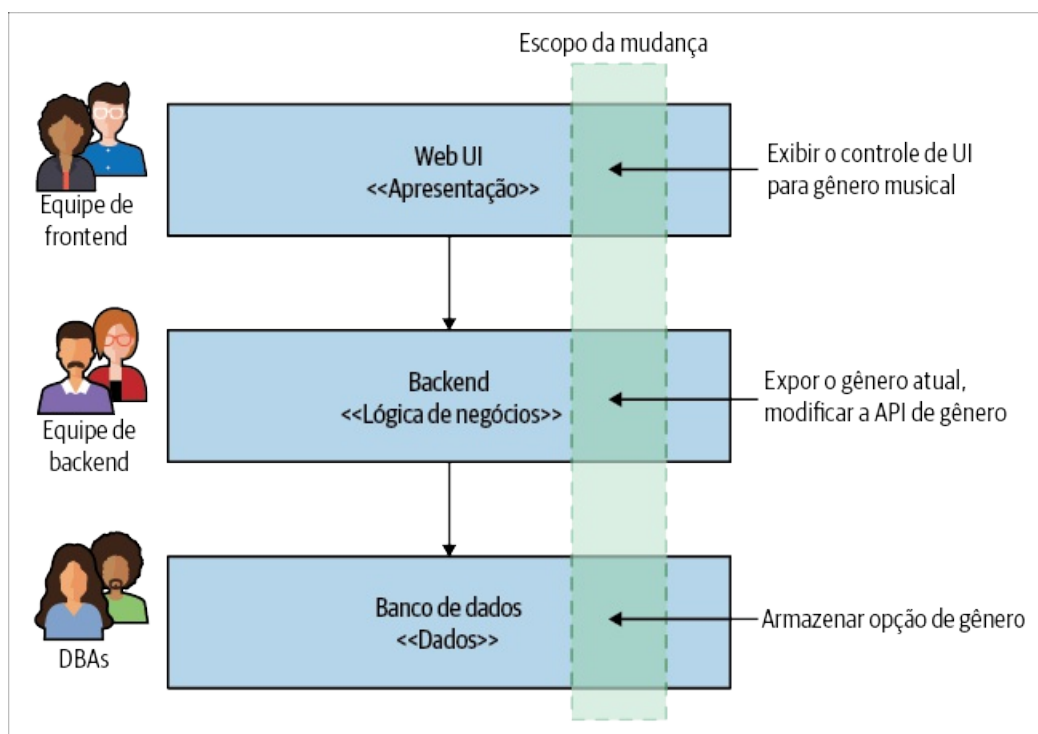


Figura 14.1 – Fazer uma alteração envolvendo todas as três camadas é mais complicado.

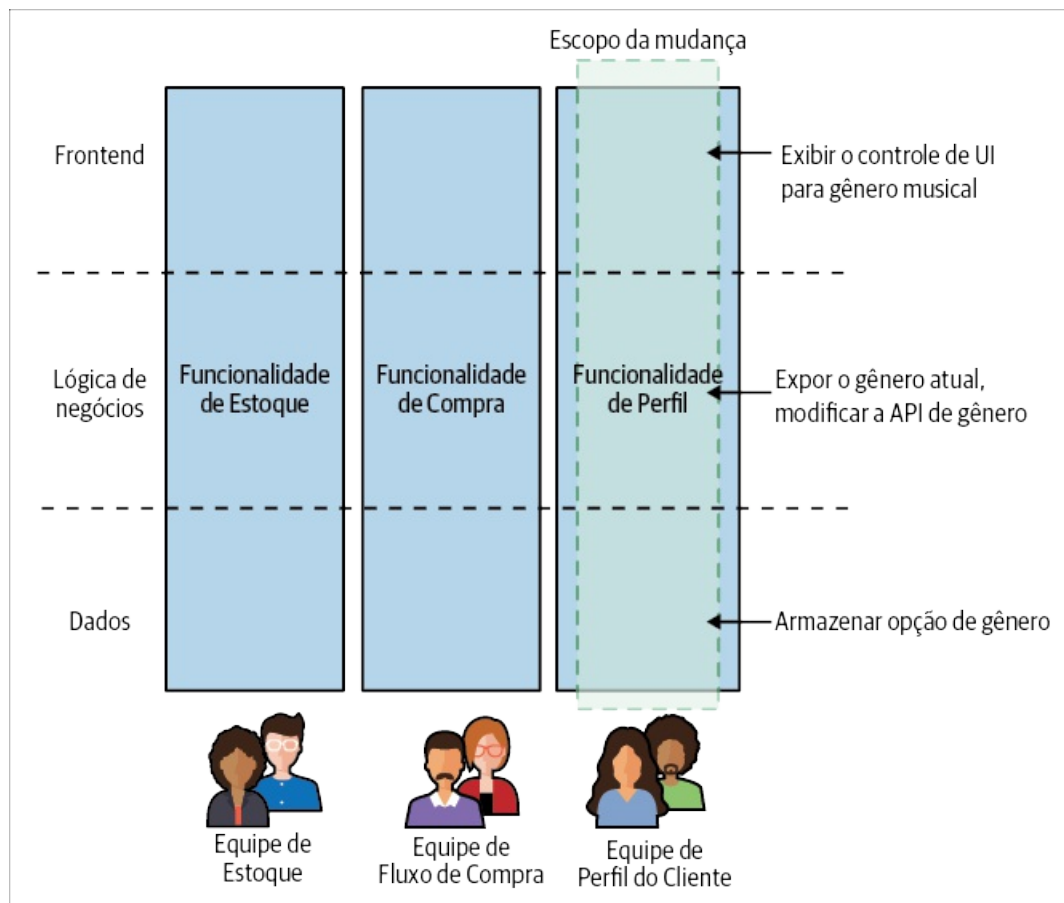


Figura 14.2 – A UI é dividida e fica sob a responsabilidade de uma equipe que também gerencia a funcionalidade que dá suporte à UI do lado do servidor.

Equipes com responsabilidade total por uma funcionalidade são capazes de fazer alterações com mais rapidez. Ter responsabilidade total incentiva cada equipe a ter um ponto de contato direto com um usuário final do software. Com equipes de backend, é fácil perder a noção de quem é o usuário final.

Apesar das desvantagens, (infelizmente) ainda vejo a equipe dedicada de frontend como o padrão organizacional mais comum entre as empresas que fazem uso de microserviços. Por que isso acontece?

Razões para haver equipes dedicadas de frontend

O desejo de ter equipes dedicadas de frontend parece se resumir a três fatores essenciais: escassez de especialistas, desejo de consistência e

desafios técnicos.

Em primeiro lugar, entregar uma interface de usuário exige certo grau de habilidades específicas. Temos os aspectos relacionados à interação e ao design gráfico, além do know-how técnico necessário para oferecer uma ótima experiência na web ou em uma aplicação nativa. Especialistas com essas aptidões podem ser difíceis de encontrar, e como essas pessoas são um recurso raro, a tentação é mantê-las juntas para garantir que elas mantenham o foco somente em sua especialidade.

O segundo motivo para ter uma equipe de frontend separada é a consistência. Se houver uma única equipe responsável por entregar a interface de usuário voltada aos clientes, será possível garantir que a UI tenha uma aparência consistente. Um conjunto consistente de controles para resolver problemas semelhantes será usado, de modo que a interface de usuário passará a impressão de ser uma entidade única e coesa.

Por fim, pode ser desafiador trabalhar com algumas tecnologias para interface de usuário em um sistema não monolítico. Nesse caso, estou pensando especificamente nas SPAs (Single-Page Applications, ou Aplicações de Página Única), que, historicamente pelo menos, não têm sido fáceis de decompor. Tradicionalmente, uma interface de usuário web seria composta de várias páginas web, e navegaríamos de uma página para outra. Com as SPAs, a aplicação como um todo é disponibilizada em uma única página web. Frameworks como Angular, React e Vue teoricamente permitem a criação de interfaces de usuário mais sofisticadas que os sites “à moda antiga”. Veremos um conjunto de padrões que podem nos dar diferentes opções para decompor uma interface de usuário mais adiante neste capítulo; no que concerne às SPAs, mostrarei como o conceito de microfrontend pode permitir o uso de frameworks SPA, ao mesmo tempo que evita a necessidade de termos uma interface de usuário monolítica.

Em direção a equipes alinhadas a fluxos

Acho que ter uma equipe de frontend dedicada em geral é um erro se você está tentando otimizar com vistas a um bom throughput – isso cria novos pontos em que o trabalho precisará ser passado para outras equipes em sua

empresa, causando lentidão. O ideal é que nossas equipes estejam alinhadas com base em fatias que representem funcionalidades de ponta a ponta, permitindo que cada equipe entregue novas funcionalidades aos seus clientes, ao mesmo tempo que o volume de coordenação necessário é reduzido. Meu modelo preferido é aquele em que uma equipe é responsável pela entrega da funcionalidade de ponta a ponta em uma parte específica do domínio. Essa ideia está de acordo com o que Matthew Skelton e Manuel Pais descrevem como *equipes alinhadas a fluxo* (stream-aligned teams) em seu livro *Team Topologies*.¹ Segundo eles:

Uma equipe alinhada a fluxo (stream-aligned team) é uma equipe alinhada a um único fluxo de trabalho importante...[A] equipe tem autonomia para desenvolver e agregar valor ao cliente ou usuário do modo mais rápido, seguro e independente possível, sem que seja preciso passar o trabalho a outras equipes para que façam partes do trabalho.²

Em certo sentido, estamos falando de *equipes full stack* (em vez de desenvolvedores full stack).³ Uma equipe com responsabilidade fim a fim pela entrega de funcionalidades voltadas aos usuários também terá uma conexão mais clara e direta com o usuário final. Com muita frequência, tenho visto equipes de “backend” com uma ideia muito vaga acerca do que o software faz ou do que os usuários precisam, podendo causar todo tipo de mal-entendidos quando se trata da implementação de uma nova funcionalidade. Por outro lado, as equipes fim a fim acharão muito mais fácil estabelecer uma conexão direta com as pessoas que utilizam o software que elas criam – essas equipes poderão se concentrar mais em garantir que as pessoas a quem elas servem obtenham aquilo de que elas precisam.

Como um exemplo concreto, passei um tempo trabalhando com a FinanceCo, uma empresa Fintech bem-sucedida e em ascensão na Europa. Na FinanceCo, praticamente todas as equipes trabalham com um software que causa impactos diretos na experiência dos usuários, e têm KPIs (Key Performance Indicators, ou Indicadores Básicos de Desempenho) voltados aos clientes – o sucesso de uma dada equipe é menos determinado pela

quantidade de funcionalidades lançadas, e mais por ajudar a melhorar a experiência das pessoas que utilizam o software. Isso deixa muito claro o modo como uma alteração causa impacto nos clientes. Essa abordagem somente é possível porque a maioria das equipes tem responsabilidades diretamente voltadas aos clientes no que diz respeito ao software que entregam. Quando estamos mais afastados do usuário final, será mais difícil saber se nossas contribuições são bem-sucedidas, e podemos acabar mantendo o foco em objetivos que estão muito distantes dos aspectos que interessam às pessoas que utilizam o nosso software.

Vamos rever os motivos pelos quais existem equipes dedicadas para o frontend – especialistas, consistência e desafios técnicos – e ver como essas questões podem ser resolvidas.

Compartilhando especialistas

Bons desenvolvedores podem ser difíceis de achar, e encontrá-los se torna mais complicado ainda quando precisamos de desenvolvedores com certa especialização. Na área de interfaces de usuário, por exemplo, se você disponibiliza interfaces móveis nativas e interfaces web, poderia se ver precisando de pessoas com experiência em iOS e Android, assim como com experiência em desenvolvimento web moderno. Isso, afóra o fato de talvez você querer designers de interação dedicados, designers gráficos, especialistas em acessibilidade e profissionais semelhantes. Pessoas com as devidas aptidões nessas áreas mais específicas podem ser escassas e ter sempre mais trabalho do que tempo disponível para fazê-lo.

Conforme já mencionamos, de acordo com a abordagem tradicional para as estruturas organizacionais, todas as pessoas que tenham o mesmo conjunto de aptidões seriam colocadas na mesma equipe, permitindo controlar rigorosamente o seu trabalho. No entanto, como já discutimos também, isso resulta em uma organização com silos.

Colocar pessoas com aptidões específicas em uma própria equipe dedicada também privará você de dar uma oportunidade para que outros desenvolvedores adquiram essas aptidões necessárias. Não é preciso que todos os desenvolvedores se tornem especialistas em iOS, por exemplo,

mas poderia ser conveniente que alguns adquirissem aptidões suficientes nessa área para ajudar nas tarefas simples, deixando seus especialistas livres para enfrentarem as tarefas realmente difíceis. Podemos facilitar também o compartilhamento de habilidades com a criação de comunidades de prática – poderíamos considerar ter uma comunidade de UI que abrangesse todas as equipes, permitindo que as pessoas compartilhassem ideias e desafios com seus colegas.

Eu me lembro da época em que todas as mudanças em bancos de dados tinham de ser feitas por um grupo centralizado de DBAs (Database Administrators, ou Administradores de Bancos de Dados). Como consequência, os desenvolvedores tinham pouco conhecimento de como os bancos de dados funcionavam e, com frequência, criavam um software que fazia um uso precário do banco de dados. Além do mais, muito do trabalho que se solicitava que os DBAs experientes fizessem eram alterações triviais. À medida que o trabalho com os bancos de dados passou para as equipes de entrega, os desenvolvedores em geral puderam conhecer melhor os bancos de dados e começaram a fazer as tarefas triviais por conta própria, deixando os valiosos DBAs livres para que pudessem se concentrar mais nas questões mais complexas dos bancos de dados, fazendo melhor uso de suas aptidões e experiências específicas. Uma mudança semelhante ocorreu na área de operações e testes, com uma parcela maior desse trabalho sendo transferida para dentro das equipes.

Assim, pelo contrário, tirar os especialistas das equipes dedicadas não inibirá a sua capacidade de fazer seu trabalho; na verdade, é provável que eles tenham mais espaço para manter o foco nos problemas difíceis, que realmente exijam a sua atenção.

O truque é encontrar um modo mais eficaz de disponibilizar seus especialistas. O ideal é que eles estivessem incluídos nas equipes. Às vezes, porém, pode não haver trabalho suficiente para justificar sua presença em tempo integral em uma dada equipe e, nesse caso, eles poderiam dividir seu tempo entre várias equipes. Outro modelo consiste em ter uma equipe dedicada com essas habilidades, cujo trabalho explícito seja *capacitar* (enable) outras equipes. No livro *Team Topologies*, Skelton e Pais

descrevem essas equipes como *equipes de capacitação* (enabling teams). Sua tarefa é sair por aí e ajudar outras equipes com foco na entrega de novas funcionalidades a fazerem o seu trabalho. Podemos pensar nessas equipes como se fossem um pouco parecidas com uma consultoria interna – elas podem passar um tempo definido com uma equipe alinhada a fluxo (stream-aligned team), ajudando-a a ser mais autossuficiente em uma área específica, ou dedicar certo tempo para ajudar no rollout de um trabalho particularmente difícil.

Assim, independentemente de seus especialistas estarem incluídos em tempo integral em uma dada equipe ou trabalharem para capacitar outras a fazerem o mesmo trabalho, será possível acabar com os silos organizacionais e, ao mesmo tempo, ajudar seus colegas a desenvolverem novas aptidões.

Garantindo a consistência

O segundo problema geralmente citado como motivo para ter equipes dedicadas de frontend é a consistência. Ao ter uma única equipe responsável por uma interface de usuário, garantimos que a UI tenha uma aparência consistente. Isso pode se estender e incluir aspectos simples como usar as mesmas fontes e cores, até resolver os mesmos problemas de interface da mesma maneira – utilizando um design consistente e uma linguagem que ajude os usuários a interagirem com o sistema. Essa consistência não só ajuda a transmitir certo grau de zelo quanto ao próprio produto, como também garante que os usuários tenham mais facilidade para usar uma nova funcionalidade quando ela for lançada.

Contudo, há meios de ajudar a garantir que haja certo grau de consistência entre as equipes. Se você utiliza o modelo de equipes de capacitação, com especialistas passando um tempo com várias equipes, eles podem ajudar a garantir que o trabalho feito por cada equipe seja consistente. A criação de recursos compartilhados, por exemplo, um guia de estilo CSS usado na prática, ou componentes de UI compartilhados, também pode ajudar.

Como um exemplo concreto de uma equipe de capacitação usada para ajudar na consistência, a equipe Origami do *Financial Times* cria

componentes web em colaboração com a equipe de design, que inclui a identidade da marca – garantindo que haja uma aparência consistente entre todas as equipes alinhadas a fluxo. Esse tipo de equipe de capacitação oferece duas formas de assistência – em primeiro lugar, ela compartilha sua expertise na entrega de componentes prontos e, em segundo, ajuda a garantir que as UIs apresentem uma experiência de usuário consistente.

Vale a pena notar, porém, que o desejo de ter consistência não deve ser considerado universalmente correto. Algumas empresas tomam decisões aparentemente conscientes de não exigir consistência em suas interfaces de usuário porque acham que é preferível permitir que as equipes tenham mais autonomia. A Amazon é uma empresa desse tipo. Versões mais antigas de seu site principal de compras tinham graus elevados de inconsistência, com widgets usando estilos de controle bastante diferentes.

Isso se torna evidente em um nível mais acentuado se observarmos o painel de controle web da Amazon Web Services (AWS). Diferentes produtos na AWS têm modelos de interação extremamente distintos, deixando a interface de usuário um tanto quanto confusa. Isso, porém, parece ser uma extensão lógica do desejo da Amazon de reduzir o trabalho de coordenação interna entre as equipes.

A maior autonomia das equipes de produto na AWS parece se manifestar também de outras maneiras, e não apenas por meio de uma experiência de usuário muitas vezes heterogênea. Com frequência, há várias maneiras distintas de fazer a mesma tarefa (por exemplo, executando uma carga de trabalho em contêiner), com diferentes equipes de produto na AWS muitas vezes se sobrepondo entre si com soluções similares, porém incompatíveis. Você poderia criticar o resultado, mas a AWS tem mostrado que, ao ter essas equipes orientadas a produto extremamente autônomas, ela criou uma empresa que mantém claramente a liderança no mercado. A rapidez na entrega é mais importante que a consistência na experiência do usuário, pelo menos no que concerne à AWS.

Resolvendo os desafios técnicos

Passamos por algumas evoluções interessantes no desenvolvimento das

interfaces de usuário – das interfaces de usuário textuais em um terminal com tela verde, até as ricas aplicações desktop, a web e as experiências com aplicativos móveis nativos atualmente. Em vários aspectos, passamos pelo ciclo completo e fomos um pouco além – nossas aplicações clientes atualmente são criadas com tanta complexidade e sofisticação que, sem dúvida, rivalizam com a complexidade das ricas aplicações desktop que eram o cerne do desenvolvimento das interfaces de usuário até a primeira década do século vinte e um.

Até certo ponto, quanto mais as coisas mudam, mais elas permanecem iguais. Com frequência, ainda trabalhamos com os mesmos controles de UI de vinte anos atrás – botões, caixas de seleção, formulários, caixas de combinação (combo boxes), e itens semelhantes. Adicionamos alguns novos componentes nessa área, mas muito menos do que você poderia imaginar. O que mudou, antes de tudo, foi a tecnologia usada para criar essas interfaces gráficas de usuário.

Algumas tecnologias mais recentes nessa área, especificamente as aplicações single-page, nos causam problemas quando se trata de decompor uma interface de usuário. Além disso, a maior diversidade de dispositivos nos quais esperamos que a mesma interface de usuário seja disponibilizada causa outros problemas que precisam ser resolvidos.

Basicamente, nossos usuários querem se envolver com o nosso software do modo mais natural possível. Seja por meio de um navegador em um desktop ou com um aplicativo móvel web ou nativo, o resultado é o mesmo – os usuários interagem com nosso software por meio de uma única tela de vidro. Eles não deveriam se importar se a interface de usuário foi criada de forma modular ou monolítica. Portanto, precisamos buscar maneiras de separar nossa funcionalidade de interface de usuário e juntá-la novamente, ao mesmo tempo que resolvemos os problemas causados por aplicações single-page, dispositivos móveis e outros. Esses problemas nos manterão ocupados na parte restante deste capítulo.

Padrão: frontend monolítico

O padrão *frontend monolítico* (monolithic frontend) descreve uma

arquitetura na qual todo o estado e o comportamento da UI são definidos na própria UI, com chamadas feitas aos microsserviços de apoio para obter os dados necessários ou executar as operações exigidas. A Figura 14.3 mostra um exemplo desse padrão. Nossa tela quer exibir informações sobre um álbum e sua lista de músicas, portanto a UI faz uma requisição para obter esses dados junto ao microsserviço `Álbum`. Também exibimos informações sobre as ofertas especiais mais recentes, requisitando informações do microsserviço `Promoções`. Nesse exemplo, nossos microsserviços devolvem dados JSON, que a UI utiliza para atualizar as informações exibidas.

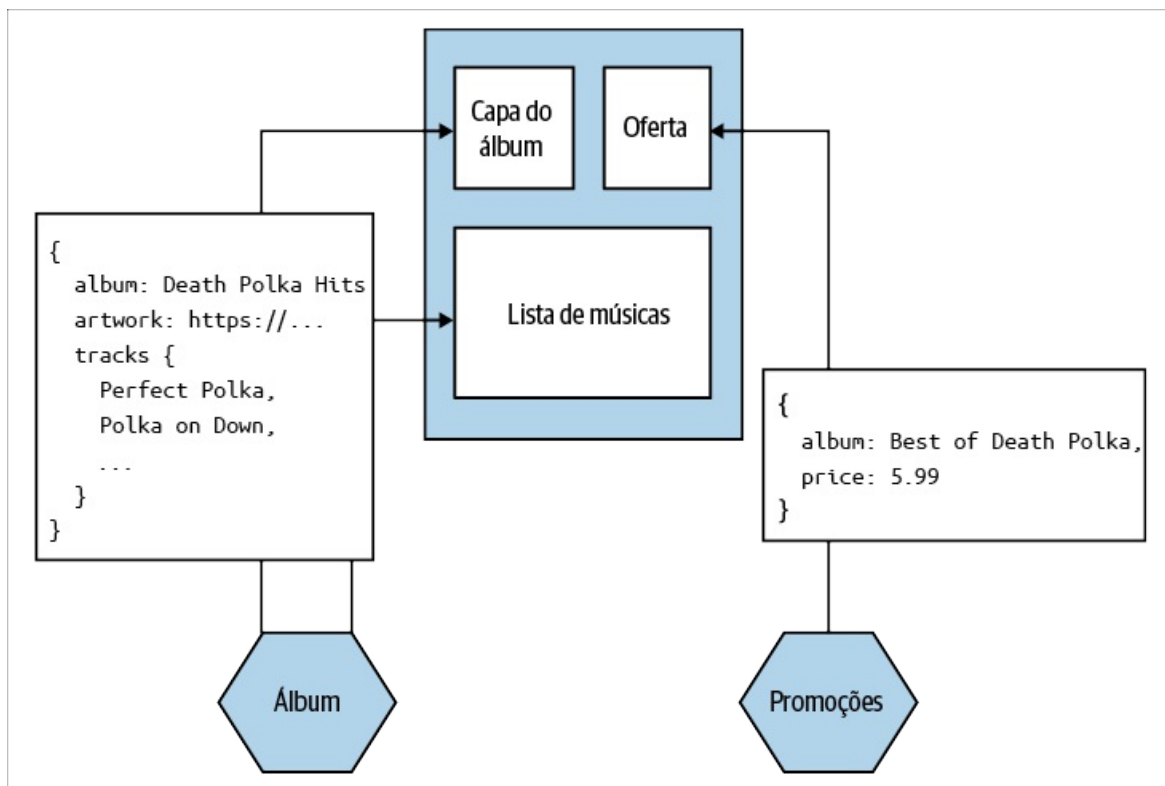


Figura 14.3 – Nossa tela de detalhes de álbum obtém informações dos microsserviços downstream para exibir a UI.

Esse modelo é o mais comum para pessoas que criam aplicações monolíticas single-page, geralmente com uma equipe dedicada de frontend. Os requisitos de nossos microsserviços são bem simples – eles só precisam compartilhar informações em um formato que possa ser facilmente interpretado pela UI. No caso de uma UI web, isso significa que

nossos microsserviços provavelmente terão de disponibilizar os dados em um formato textual, com o JSON sendo a opção mais provável. A UI precisaria então criar os vários componentes que constituem a interface, tratando da sincronização do estado e tarefas desse tipo com o backend. Usar um protocolo binário para a comunicação de serviço para serviço seria mais difícil para clientes web, mas poderia não ser um problema para dispositivos móveis nativos ou aplicações desktop “pesadas”.

Quando usar

Há algumas desvantagens nessa abordagem. Em primeiro lugar, por causa de sua natureza como entidade monolítica, ela pode incentivar (ou ser causada por) uma equipe dedicada de frontend. Ter várias equipes compartilhando a responsabilidade por esse frontend monolítico pode ser complicado por causa das diversas razões para haver contenção. Em segundo lugar, nossa capacidade de personalizar as respostas para os diferentes tipos de dispositivos é baixa. Se uma tecnologia web estiver em uso, podemos alterar o layout de uma tela a fim de nos adaptarmos às limitações dos diferentes dispositivos, mas isso não se estende necessariamente a mudar as chamadas feitas aos microsserviços de apoio. Meu cliente móvel talvez seja capaz de exibir somente dez campos de um pedido, mas se o microsserviço envia todos os cem campos do pedido, acabaremos obtendo dados desnecessários. Uma solução para essa abordagem consiste na interface de usuário especificar quais campos obterá ao fazer uma requisição, mas isso pressupõe que cada microsserviço de apoio aceitará essa forma de interação. Na seção “GraphQL”, na página [568](#), veremos como o uso tanto do padrão backend para frontend como do GraphQL pode ajudar nesse caso.

Na verdade, esse padrão funciona melhor quando *queremos* que toda a implementação e o comportamento da UI estejam em uma única unidade de implantação. Com uma única equipe desenvolvendo tanto o frontend como todos os microsserviços de apoio, isso talvez não seja um problema. Pessoalmente, se você tiver mais de uma equipe trabalhando em seu software, acho que deveria lutar contra esse impulso, pois pode acabar ficando com uma arquitetura em camadas, com os silos organizacionais

associados. Contudo, se não puder evitar uma arquitetura em camadas e a estrutura organizacional correspondente, esse provavelmente será o padrão que você acabará usando.

Padrão: microfrontends

A abordagem de *microfrontend* é um padrão organizacional segundo o qual é possível trabalhar em diferentes partes de um frontend e implantá-las de modo independente. Para trazer uma citação de um artigo altamente recomendado de Cam Jackson sobre o assunto,⁴ podemos definir *microfrontends* da seguinte maneira: “É um estilo de arquitetura no qual aplicações de frontend que podem ser lançadas de modo independente são compostas em um todo maior”.

É um padrão essencial para equipes alinhadas a fluxo que querem ser responsáveis pela entrega tanto dos microsserviços no backend como pela UI de apoio. Enquanto os microsserviços proporcionam implantações independentes para as funcionalidades no backend, os microfrontends permitem implantações independentes no frontend.

O conceito de microfrontend tem ganhado popularidade em virtude dos desafios criados pelas UIs web monolíticas, com uso intensivo de JavaScript, conforme típicos de aplicações single-page. Com um microfrontend, diferentes equipes podem trabalhar e fazer alterações em diferentes partes do frontend. Voltando à Figura 14.2, a equipe de estoque, a equipe de fluxo de compras e a equipe de perfil do cliente podem, cada uma, alterar as funcionalidades do frontend associadas ao seu fluxo de trabalho, independentemente das outras equipes.

Implementação

Para frontends web, podemos considerar duas técnicas básicas de decomposição que podem auxiliar na implementação do padrão microfrontend. Uma decomposição baseada em widgets envolve combinar diferentes partes de um frontend em uma única tela. Uma decomposição baseada em páginas, por outro lado, tem o frontend dividido em páginas web independentes. As duas abordagens merecem uma melhor exploração,

e faremos isso em breve.

Quando usar

O padrão microfrontend é essencial se você quiser adotar equipes fim a fim, alinhadas a fluxos, na tentativa de se afastar de uma arquitetura em camadas. Também consigo imaginá-lo sendo útil em uma situação na qual queremos manter uma estrutura em camadas, mas a funcionalidade do frontend agora é tão grande que várias equipes dedicadas de frontend são necessárias.

Há um problema essencial com essa abordagem, o qual não tenho certeza de que possa ser resolvido. Às vezes, as funcionalidades oferecidas por um microsserviço não se enquadram muito bem em um widget ou uma página. É claro que eu gostaria de exibir as recomendações em uma caixa em uma página de nosso site, mas e se eu quisesse incluir recomendações dinâmicas em outro lugar? Quando fizer uma pesquisa, gostaria que a digitação já disparasse automaticamente novas recomendações, por exemplo. Quanto mais abrangente for uma forma de interação, menor serão as chances de esse modelo ser apropriado, e maior será a probabilidade de recorrermos a simples chamadas de API.

Sistemas autocontidos

Um scs (Self-Contained System, ou Sistema Autocontido) é um estilo de arquitetura que, sem dúvida, surgiu pela falta de foco nos problemas de UI durante os primeiros anos dos microsserviços. Um SCS poderia ser composto de várias partes (possivelmente, de microsserviços) que, quando reunidas, compõem um único SCS.

Conforme a definição, um sistema autocontido deve obedecer a alguns critérios específicos que podem se sobrepor a alguns dos mesmos aspectos que tentamos alcançar com os microsserviços. Mais informações sobre sistemas autocontidos podem ser encontradas no site do SCS (<https://scs-architecture.org>), que é muito claro, mas eis alguns dos seus destaques:

- Cada SCS é uma aplicação web autônoma, sem UI compartilhada.
- Cada SCS pertence a uma única equipe.

- Uma comunicação assíncrona deve ser usada sempre que for possível.
- Nenhum código de negócios pode ser compartilhado entre SCSs.

A abordagem de SCS não se popularizou no mesmo nível que os microsserviços, e não é um conceito que vejo com muita frequência, apesar de eu concordar com muitos dos princípios que ele define. Particularmente, gosto de ressaltar que um sistema autocontido deve pertencer a uma só equipe. Eu me pergunto se essa falta de uso mais amplo explica por que alguns aspectos da abordagem SCS parecem ser muito restritivos e prescritivos. Por exemplo, a insistência de que cada SCS deva ser uma “aplicação web autônoma” implica que vários tipos de interfaces de usuário não poderiam ser considerados um SCS. Isso significa que a aplicação iOS nativa que criei, a qual utiliza gRPC, pode ser parte de um SCS ou não?

A abordagem com SCS, então, está em conflito com os microsserviços? Não exatamente. Já trabalhei com muitos microsserviços que, quando considerados de forma isolada, se enquadrariam, por si mesmos, na definição de um SCS. Há algumas ideias interessantes na abordagem de SCS com as quais concordo, e muitas delas já foram discutidas neste livro. Eu apenas acho que a abordagem é excessivamente prescritiva, a ponto de alguém que esteja interessado em SCSs poder achar que adotar a abordagem seria extremamente desafiador, pois poderia exigir mudanças completas em vários aspectos de sua entrega de software.

Eu me preocupo com o fato de que manifestos como o do conceito de SCS possam nos levar a um caminho em que o foco esteja excessivamente nas atividades, em vez de estarem nos princípios e resultados. Você poderia obedecer a todas as características de um SCS e, ainda assim, fracassar. Refletindo, acho que a abordagem de SCS é uma abordagem com foco em tecnologia para promover um conceito organizacional. Desse modo, prefiro manter o foco na importância das equipes alinhadas a fluxo com menos trabalho de coordenação, e deixar que a tecnologia e a arquitetura fluam a partir daí.

Padrão: decomposição baseada em páginas

Em uma decomposição baseada em páginas (page-based decomposition), nossa UI é decomposta em várias páginas web. Diferentes conjuntos de páginas podem ser atendidas por diferentes microsserviços. Na Figura 14.4, vemos um exemplo desse padrão na MusicCorp. Requisições para páginas em `/albums/` são encaminhadas diretamente ao microsserviço `Álbuns`, que cuida de servir essas páginas, e fazemos algo parecido com `/artists/`. Uma navegação em comum é usada para ajudar a manter a unidade dessas páginas. Por sua vez, esses microsserviços podem buscar as informações necessárias para construir essas páginas – por exemplo, buscar os níveis de estoque no microsserviço `Estoque` para exibir os itens em estoque na UI.

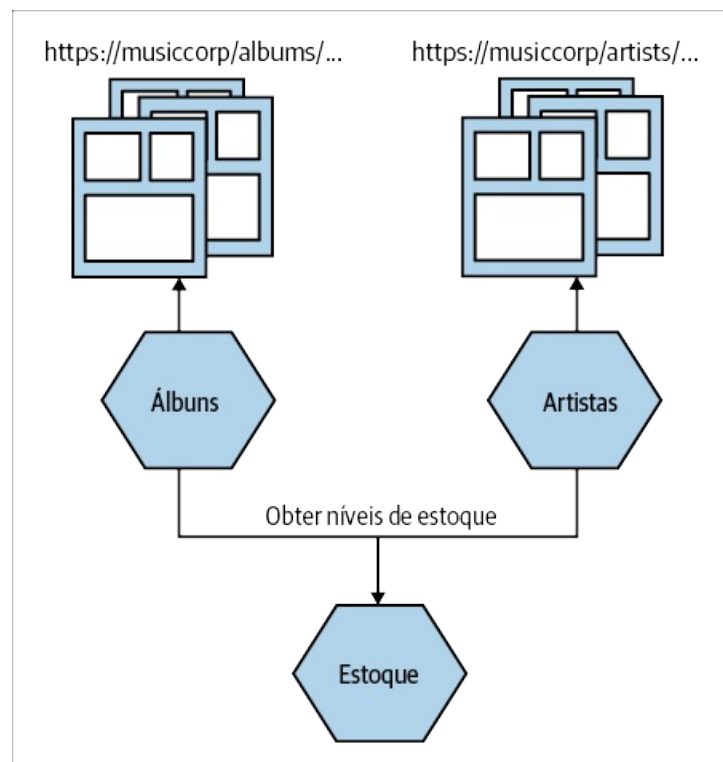


Figura 14.4 – A interface de usuário é composta de várias páginas, com diferentes grupos de páginas atendidas por diferentes microsserviços.

Nesse modelo, uma equipe responsável pelo microsserviço `Álbuns` seria capaz de renderizar uma UI completa, de ponta a ponta, facilitando à equipe entender como suas alterações impactariam o usuário.

A web

Antes das aplicações single-page, tínhamos a web. Nossa interação com a web era baseada no acesso a URLs e nos cliques em links que faziam

com que novas páginas fossem carregadas em nosso navegador. Nossos navegadores foram criados para permitir a navegação por essas páginas, com marcadores (bookmarks) para marcar as páginas de interesse, e controles para a frente e para trás para visitar novamente as páginas que já haviam sido acessadas antes. Todos vocês poderiam estar revirando os olhos e pensando, “É claro que eu sei como a web funciona!”; no entanto, esse é um estilo de interface de usuário que parece estar caindo em desuso. Sua simplicidade é algo da qual sinto falta quando vejo nossas implementações atuais de interface de usuário para web – perdemos muito por supor automaticamente que uma UI web significa aplicações single-page.

No que diz respeito a lidar com diferentes tipos de clientes, não há nada que impeça a página de adaptar o que ela exibe de acordo com a natureza do dispositivo que estiver requisitando a página. Os conceitos de melhoria progressiva (ou degradação elegante) devem ser bem compreendidos a essa altura.

A simplicidade da decomposição baseada em páginas, do ponto de vista técnico de uma implementação, é muito atraente nesse caso. Não precisamos de nenhum JavaScript sofisticado executando no navegador, nem precisamos utilizar iFrames problemáticos. O usuário clica em um link, e uma nova página é requisitada.

Quando usar

Conveniente tanto para um frontend monolítico como para uma abordagem de microfrontend, a decomposição baseada em páginas seria minha opção default para uma decomposição da interface de usuário caso minha interface fosse um site. A página web como uma unidade de decomposição é um conceito tão básico da web como um todo que passa a ser uma técnica simples e clara para separar uma interface de usuário grande para web.

Acho que o problema é que, na pressa de usar a tecnologia de aplicações single-page, essas interfaces de usuário estão se tornando cada vez mais raras, a ponto de as experiências de usuário que, em minha opinião, seriam mais apropriadas para uma implementação de site, acabarem sendo

forçadas a serem uma aplicação single-page.⁵ É claro que podemos combinar uma decomposição baseada em páginas com algum dos outros padrões que discutimos. Poderíamos ter uma página que contenha widgets, por exemplo – algo que veremos a seguir.

Padrão: decomposição baseada em widgets

Na decomposição baseada em widgets (widget-based decomposition), uma tela em uma interface gráfica contém widgets que podem ser alterados de modo independente. Na Figura 14.5, vemos um exemplo do frontend da MusicCorp, com dois widgets oferecendo as funcionalidades de UI para o carrinho de compras e as recomendações.

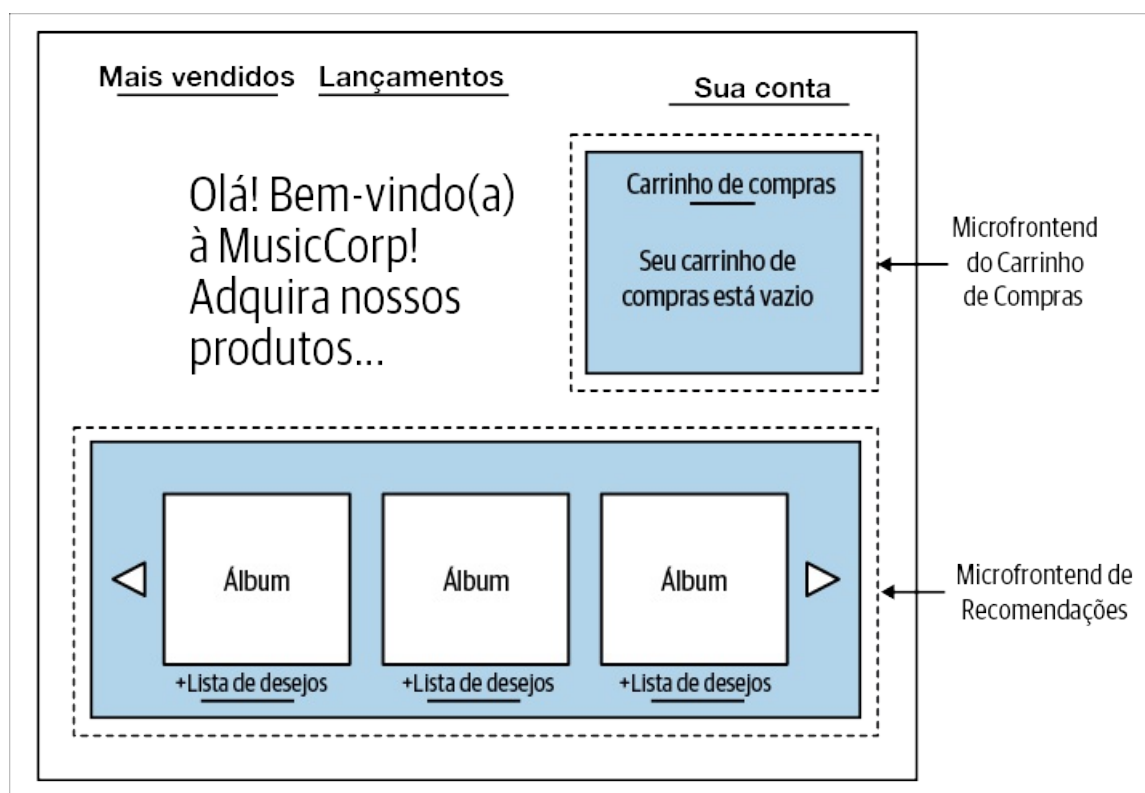


Figura 14.5 – Widgets para carrinho de compras e recomendações usados na MusicCorp.

O widget de recomendações da MusicCorp obtém um carrossel (carousel) de recomendações pelo qual podemos circular e que podemos filtrar. Como vemos na Figura 14.6, quando o usuário interage com o widget de recomendações – acessando o próximo conjunto de recomendações

ciclicamente, por exemplo, ou acrescentando itens à sua lista de desejos – essa ação pode resultar em chamadas feitas aos microserviços de apoio que, nesse caso, poderiam ser aos microserviços *Recomendações* e *Lista de Desejos*. Essa solução estaria bem alinhada com a existência de uma equipe que fosse responsável tanto pelos microserviços de apoio como pelo próprio componente.

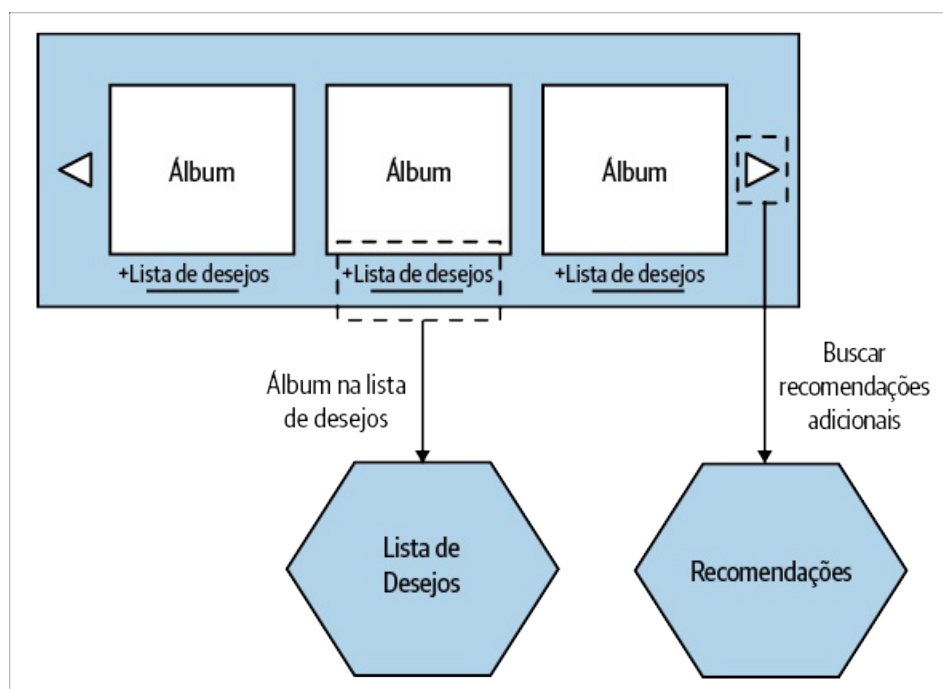


Figura 14.6 – Interações entre o microfrontend de recomendações e os microserviços de apoio.

Falando de modo geral, precisaremos de uma aplicação “contêiner” que defina aspectos como a navegação básica para a interface e quais widgets precisam ser incluídos. Se estivessemos pensando com base em equipes orientadas a fluxos, poderíamos imaginar uma única equipe disponibilizando o widget de recomendações, além de ser responsável por um microserviço de apoio, *Recomendações*.

Esse padrão é muito visto no mundo real. A interface de usuário do Spotify, por exemplo, faz uso intensivo desse padrão. Um widget pode conter uma playlist, outro poderia conter informações sobre um artista, enquanto um terceiro poderia conter informações sobre os artistas, enquanto outro, os usuários do Spotify que você segue. Esses widgets são

combinados de diferentes maneiras, em diferentes situações.

Será necessário ter ainda algum tipo de camada de montagem para reunir essas partes. Contudo, poderia ser algo simples, por exemplo com o uso de templates do lado do servidor ou do lado do cliente.

Implementação

O modo de combinar um widget com sua UI dependerá, em boa medida, do modo como a UI é criada. Em um site simples, incluir widgets como fragmentos HTML usando templating do lado do cliente ou do lado do servidor pode ser muito simples, embora possa haver problemas caso os widgets tenham um comportamento mais complexo. Como exemplo, se nosso widget de recomendações contiver muitas funcionalidades JavaScript, como iríamos garantir que elas não estariam em conflito com um comportamento presente nas outras partes da página web? O ideal é que o widget como um todo possa ser empacotado de um modo que não cause falhas em outros aspectos da UI.

O problema de disponibilizar funcionalidades autocontidas em uma UI, sem causar falhas em outras funcionalidades, tem sido historicamente complicado em aplicações single-page em particular, parcialmente porque o conceito de modularização não parece ter sido uma preocupação principal no modo como os frameworks de suporte a SPA foram criados. Vale a pena explorar esses problemas com mais detalhes.

Dependências

Embora o iFrames tenha sido uma técnica intensamente empregada no passado, tendemos a evitar seu uso para combinar diferentes widgets em uma única página web. O iFrames apresenta uma série de desafios quanto ao tamanho, além de dificultar a comunicação entre as diferentes partes do frontend. Como alternativa, os widgets geralmente são combinados na UI com o uso de um templating do lado do servidor, ou então são dinamicamente inseridos no navegador do lado do cliente. Nos dois casos, o desafio está no fato de o widget executar na mesma página de navegador que outras partes do frontend, o que significa que você terá de ser cuidadoso para que os diferentes widgets não entrem em conflito entre si.

Por exemplo, nosso widget de recomendações poderia fazer uso do React v16, enquanto o widget de carrinho de compras ainda usaria o React v15. Isso pode ser uma bênção, é claro, pois nos ajudaria a testar uma tecnologia diferente (poderíamos usar diferentes frameworks de SPA para widgets distintos), mas também pode ajudar quando se trata de atualizar as versões dos frameworks em uso. Conversei com algumas equipes que tiveram problemas para mudar de versões de Angular ou de React, principalmente por causa das diferenças de convenções usadas nas versões mais recentes dos frameworks. Fazer um upgrade de toda a UI monolítica pode ser uma tarefa assustadora, mas se puder fazê-lo de forma gradual, atualizando partes de seu frontend, uma de cada vez, será possível dividir o trabalho, além de reduzir os riscos de o upgrade introduzir novos problemas.

A desvantagem é que você pode acabar com muita duplicação entre as dependências, o que, por sua vez, pode levar a um aumento exagerado no tamanho da página a ser carregada. Eu poderia acabar incluindo, por exemplo, várias versões diferentes do framework React e suas dependências transitivas associadas. Não é de se surpreender que muitos sites atualmente tenham um tamanho de página a ser carregado muitas vezes maior do que o tamanho de alguns sistemas operacionais. Em uma rápida pesquisa não científica, verifiquei o tamanho da página carregada no site da CNN atualmente – quando este livro foi escrito –, e foi de 7.9 MB, que é muito maior que os 5 MB do Alpine Linux, por exemplo. O tamanho de 7.9 MB, na verdade, está na faixa menor de alguns dos tamanhos de página que vejo serem carregadas.

Comunicação entre widgets na mesma página

Embora nossos widgets possam ser criados e implantados de modo independente, ainda queremos que eles sejam capazes de interagir uns com os outros. Como um exemplo na MusicCorp, quando um usuário seleciona um dos álbuns na lista de campeões de venda, queremos que as outras partes da UI se atualizem com base na seleção, como mostra a Figura 14.7.

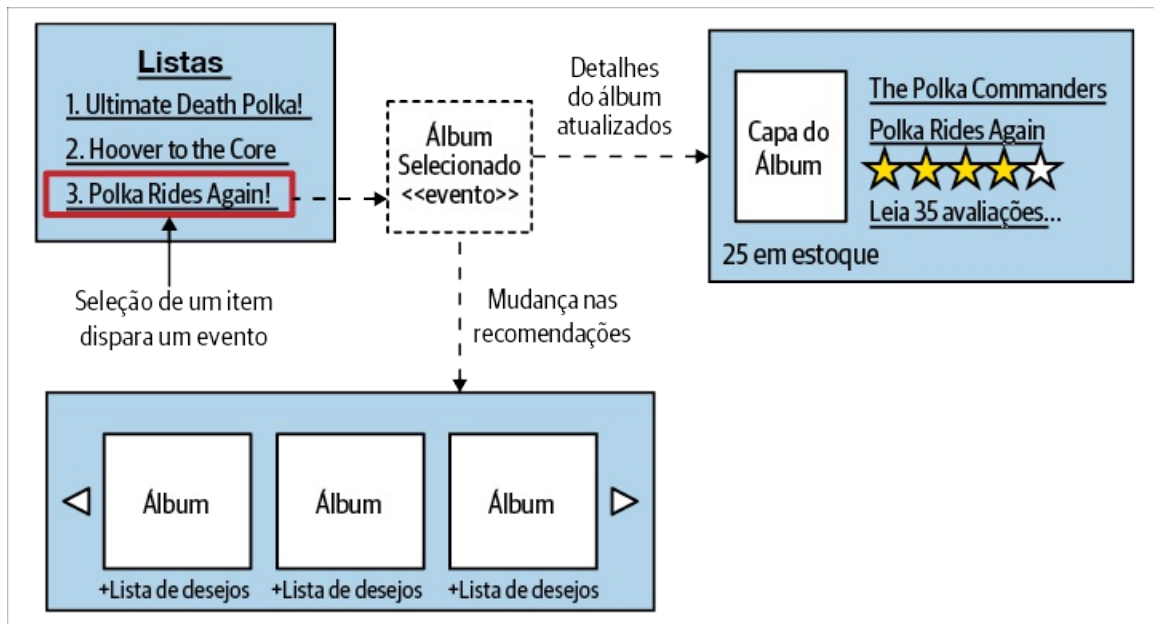


Figura 14.7 – O widget de listas é capaz de gerar eventos que interessam a outras partes da UI.

O modo como implementaríamos isso seria fazendo com que o widget de listas gerasse um evento personalizado. Os navegadores já aceitam uma série de eventos padrões que podemos usar para disparar certos comportamentos. Esses eventos nos permitem reagir a botões pressionados, rolagens do mouse e eventos semelhantes, e você provavelmente já deve ter feito uso intensivo desses tratamentos de eventos caso tenha passado algum tempo criando frontends JavaScript. Criar seus próprios eventos personalizados é um passo simples.

Assim, em nosso caso, quando um item é selecionado na lista, o widget gera um evento `Álbum Selecionado` personalizado. Tanto o widget de recomendações como o de detalhes de álbum se inscrevem para receber o evento e reagem de acordo com ele, com as recomendações atualizadas de acordo com a seleção, e os detalhes do álbum sendo carregados. Essa interação, é claro, já deve ser familiar para nós, pois imita a interação orientada a eventos entre os microserviços, discutida na seção “Padrão: comunicação orientada a eventos”, na página 137. A única diferença real é que essas interações com evento ocorrem dentro do navegador.

Componentes web

À primeira vista, o Web Component Standard (Padrão de

Componentes Web) deveria ser uma maneira óbvia de implementar esses widgets. O Web Component Standard descreve como podemos criar um componente de UI capaz de manter seus aspectos HTML, CSS e JavaScript em uma sandbox. Infelizmente, o padrão de componentes web parece ter demorado muito tempo para se consolidar, e mais tempo ainda para ser devidamente aceito pelos navegadores. Boa parte do trabalho inicial nesse padrão aparentemente ficou parado, o que, obviamente, causou impactos à sua adoção. Ainda não vi nenhuma empresa que utilize componentes web servidos a partir de microsserviços, por exemplo.

Considerando que o Web Component Standard atualmente tem uma aceitação razoavelmente boa, é possível que ele venha a se tornar uma forma comum de implementar widgets em sandboxes ou microfrontends maiores no futuro, mas, depois de anos à espera de isso acontecer, não estou mais tão ansioso.

Quando usar

Esse padrão facilita que várias equipes alinhadas a fluxos contribuam com a mesma UI. Permite que haja mais flexibilidade do que em uma decomposição baseada em páginas, pois os widgets lançados por diferentes equipes podem coexistir todos na mesma UI, ao mesmo tempo. Também cria a oportunidade para as equipes de capacitação fornecerem widgets reutilizáveis, que possam ser usados pelas equipes alinhadas a fluxos – apresentei antes um exemplo desse caso, quando mencionei o papel da equipe Origami no *Financial Times*.

O padrão de decomposição com widgets será extremamente conveniente se você estiver criando uma interface de usuário rica para web, e eu sugeriria fortemente o uso de widgets em qualquer situação na qual um framework SPA esteja em uso e você queira separar as responsabilidades pelo frontend, passando para uma abordagem de microfrontend. As técnicas e as tecnologias auxiliares associadas a esse conceito melhoraram significativamente nos últimos anos, a ponto de, ao criar uma interface web SPA, separar a UI em microfrontends ser minha abordagem padrão.

Minhas principais preocupações relacionadas à decomposição em widgets

no contexto das SPAs dizem respeito ao trabalho necessário para preparar o empacotamento separado dos componentes, e os problemas com o tamanho dos payloads. O primeiro problema provavelmente implica um custo que incorrerá apenas uma vez, envolvendo apenas determinar o estilo de empacotamento que for mais apropriado para o seu conjunto atual de ferramentas. O segundo problema é mais complicado. Uma pequena alteração nas dependências de um widget poderia resultar na inclusão de um conjunto completo de novas dependências na aplicação, aumentando drasticamente o tamanho da página. Se você está criando uma interface de usuário na qual o tamanho da página é uma preocupação, eu sugeriria inserir algumas verificações automáticas para avisar caso esse tamanho ultrapasse um determinado limite aceitável.

Por outro lado, se os widgets forem de natureza mais simples e, em sua maior parte, forem componentes estáticos, ser capaz de incluí-los utilizando algo simples como um templating do lado do cliente ou do lado do servidor, por comparação, será uma tarefa muito simples.

Limitações

Antes de prosseguir para a discussão do nosso próximo padrão, gostaria de abordar o tópico sobre limitações. Cada vez mais, os usuários de nosso software interagem com ele a partir de diversos dispositivos diferentes. Cada um desses dispositivos impõe limitações distintas às quais nosso software precisa se adaptar. Em uma aplicação web desktop, por exemplo, consideramos limitações como o navegador usado pelos visitantes ou a resolução de suas telas. Pessoas com deficiência visual talvez façam uso de nosso software por meio de leitores de tela, e as pessoas com mobilidade restrita provavelmente utilizariam entradas semelhantes às de teclado para navegar pela tela.

Assim, embora nossos serviços essenciais – nossos produtos essenciais – possam ser os mesmos, precisamos de uma forma de adaptá-los às diferentes limitações existentes em cada tipo de interface e para as diferentes necessidades de nossos usuários. Isso pode ser motivado por um ponto de vista puramente financeiro, se você quiser – mais clientes

satisfeitos implica mais dinheiro. Contudo, há também uma consideração ética e humana que se sobressai: quando ignoramos clientes que têm necessidades específicas, nós lhes negamos a chance de utilizarem nossos serviços. Em alguns contextos, impossibilitar que as pessoas naveguem por uma UI por causa de decisões de design resultou em ações legais e multas – por exemplo, o Reino Unido, além de vários outros países, com razão, têm uma legislação definida para garantir o acesso a sites por pessoas com deficiências.⁶

Os dispositivos móveis trouxeram todo um conjunto de novas restrições. O modo como nossos aplicativos móveis se comunicam com um servidor pode causar impacto. Não é somente uma questão de preocupação com largura de banda, em que as limitações das redes móveis podem desempenhar um papel relevante. Diferentes tipos de interações podem consumir a bateria, deixando alguns clientes irritados.

A natureza das interações muda de acordo com o dispositivo também. Não consigo clicar facilmente com o botão da direita em um tablet. Em um dispositivo móvel, posso querer o design de minha interface para ser usado basicamente com uma só mão, com a maior parte das operações sendo controladas pelo polegar. Em outros casos, eu poderia permitir que as pessoas interagissem com os serviços por meio de SMS nos lugares em que é mais difícil ter largura de banda – o uso de SMS como uma interface é intenso no hemisfério sul, por exemplo.

Uma discussão mais ampla sobre a acessibilidade das interfaces de usuário está fora do escopo deste livro, mas, ao menos, podemos explorar os desafios específicos que resultam de ter diferentes tipos de clientes, por exemplo, os dispositivos móveis. Uma solução comum para lidar com as diferentes necessidades dos dispositivos de clientes é executar algum tipo de filtragem e agregação de chamadas do lado do cliente. Os dados que não são necessários podem ser removidos e não precisam ser enviados ao dispositivo; várias chamadas podem ser combinadas em uma única chamada.

A seguir, veremos dois padrões que podem ser convenientes nessa área – o padrão gateway de agregação central e o padrão backend para frontend.

Também veremos como o GraphQL é usado para ajudar a personalizar as respostas para diferentes tipos de interfaces.

Padrão: gateway de agregação central

Um gateway de agregação central fica entre as interfaces de usuário externas e os microsserviços downstream, e faz filtragem e agregação de chamadas para todas as interfaces de usuário. Sem a agregação, uma interface de usuário teria de fazer várias chamadas para buscar as informações de que precisa, muitas vezes descartando dados que foram obtidos, mas que não são necessários.

Na Figura 14.8, vemos uma situação desse tipo. Queremos exibir uma tela com informações sobre os pedidos recentes de um cliente. A tela precisa exibir algumas informações gerais sobre um cliente, e então listar alguns de seus pedidos em ordem cronológica, junto com informações resumidas, mostrando a data e o status de cada pedido, assim como o preço.

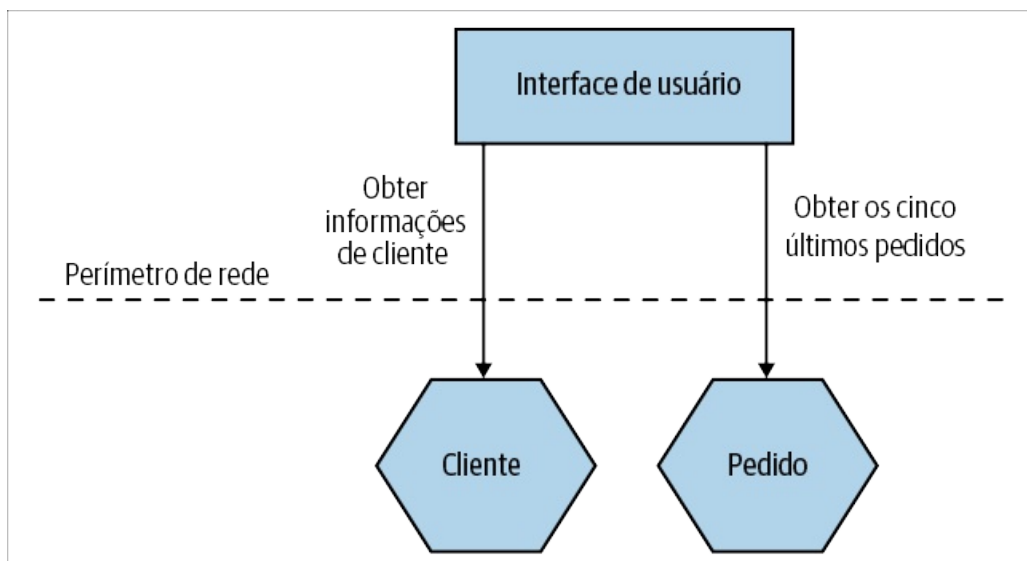


Figura 14.8 – Fazendo várias chamadas para buscar informações para uma única tela.

Fazemos uma chamada direta ao microsserviço `cliente`, obtendo informações completas sobre o cliente, apesar de precisarmos somente de alguns campos. Em seguida, buscamos os detalhes dos pedidos junto ao microsserviço `pedido`. Poderíamos melhorar a situação, de certa forma,

talvez alterando o microserviço cliente ou Pedido de modo que devolvessem dados mais apropriados aos nossos requisitos nesse caso específico, mas isso ainda exigiria que duas chamadas fossem feitas.

Com um gateway de agregação, podemos fazer uma única chamada da interface de usuário para o gateway. O gateway de agregação fará então todas as chamadas necessárias, combinará os resultados em uma única resposta e descartará qualquer dado que não seja necessário à interface de usuário (Figura 14.9).

Um gateway como esse também poderia ajudar a reunir as chamadas em batch (lote). Por exemplo, em vez de ter de consultar dez IDs de pedidos em chamadas separadas, poderíamos enviar uma requisição em batch ao gateway de agregação e ele cuidaria do resto.

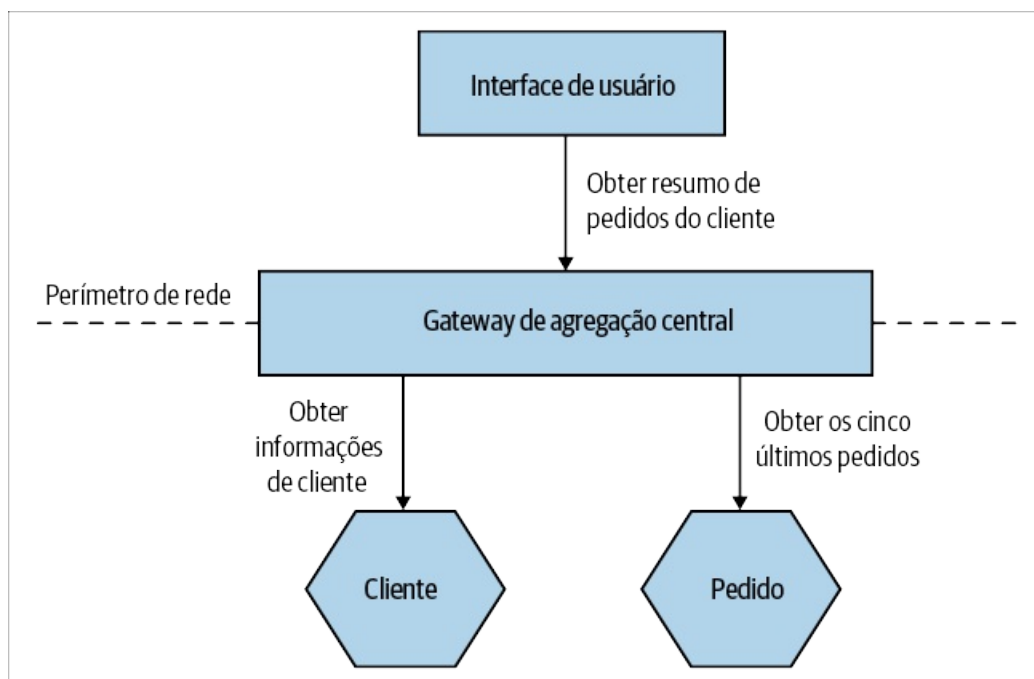


Figura 14.9 – O gateway central do lado do servidor cuida da filtragem e da agregação de chamadas aos microserviços downstream.

Basicamente, ter algum tipo de gateway de agregação pode reduzir o número de chamadas que o cliente externo precisa fazer, reduzindo a quantidade de dados a ser enviada de volta. Isso pode resultar em vantagens significativas quanto à redução no uso de banda e à melhoria na latência da aplicação.

Responsabilidade

À medida que mais interfaces de usuário fazem uso do gateway central, e mais microsserviços precisam de agregação de chamadas e de uma lógica de filtragem para essas interfaces, o gateway se tornará uma possível causa de contenção. Quem é o responsável pelo gateway? São as pessoas que criam as interfaces de usuário ou as pessoas responsáveis pelos microsserviços? Muitas vezes, acho que o gateway de agregação central faz tantas tarefas que acaba ficando sob a responsabilidade de uma equipe dedicada. Olá, arquitetura em camadas com silos!

Basicamente, a natureza da agregação de chamadas e da filtragem é, em boa medida, determinada pelos requisitos das interfaces de usuário externas. Assim, faria sentido que o gateway naturalmente ficasse sob a responsabilidade da(s) equipe(s) que cria(m) a UI. Infelizmente, sobretudo em uma empresa na qual há uma equipe dedicada de frontend, essa equipe pode não ter as habilidades necessárias para criar um componente de backend tão essencial.

Independentemente de quem acabará com a responsabilidade pelo gateway central, ele tem o potencial de se tornar um gargalo para a entrega de software. Se várias equipes precisarem fazer alterações no gateway, o desenvolvimento aí exigirá uma coordenação entre todas essas equipes, deixando o trabalho mais lento. Se uma só equipe for responsável pelo gateway, essa equipe poderia se transformar em um gargalo na entrega do software. Em breve, veremos como o padrão backend para frontend pode ajudar a solucionar esses problemas.

Diferentes tipos de interfaces de usuário

Se os desafios relacionados à responsabilidade puderem ser administrados, um gateway de agregação central ainda pode funcionar bem, até considerarmos o problema de haver dispositivos distintos, com suas diferentes necessidades. Conforme já discutimos, as possibilidades com um dispositivo móvel são muito diferentes. Há menos espaço na tela, o que significa que poderemos exibir menos dados. Iniciar muitas conexões aos recursos do lado do servidor pode acabar com a carga da bateria e

consumir planos em que há limitação de dados. Além disso, a natureza das interações que queremos oferecer em um dispositivo móvel podem ser drasticamente diferentes. Pense em uma loja física de varejo típica. Em uma aplicação desktop, eu poderia permitir que você olhasse os itens à venda e fizesse seu pedido online, ou os reservasse em uma loja. No dispositivo móvel, porém, eu poderia permitir que você escaneasse códigos de barras para fazer comparações de preços, ou poderia fazer ofertas dependentes de contexto enquanto você estivesse na loja. À medida que criamos mais aplicativos móveis, começamos a perceber que as pessoas os usam de modo muito diferente e, desse modo, as funcionalidades que precisamos expor também serão diferentes.

Assim, na prática, queremos que nossos dispositivos móveis façam chamadas diferentes, em menor quantidade, e exibam dados distintos (provavelmente menos dados), em comparação com suas contrapartidas no desktop. Isso significa que precisamos acrescentar outras funcionalidades em nossa API no backend a fim de aceitar diferentes tipos de interfaces de usuário. Na Figura 14.10, vemos a interface web e a interface móvel da MusicCorp, ambas utilizando o mesmo gateway para a tela de resumo do cliente, mas cada cliente quer um conjunto diferente de informações. A interface web deseja mais informações sobre o cliente, além de apresentar um breve resumo dos itens em cada pedido. Isso nos leva a implementar duas chamadas diferentes de agregação e filtragem em nosso gateway no backend.

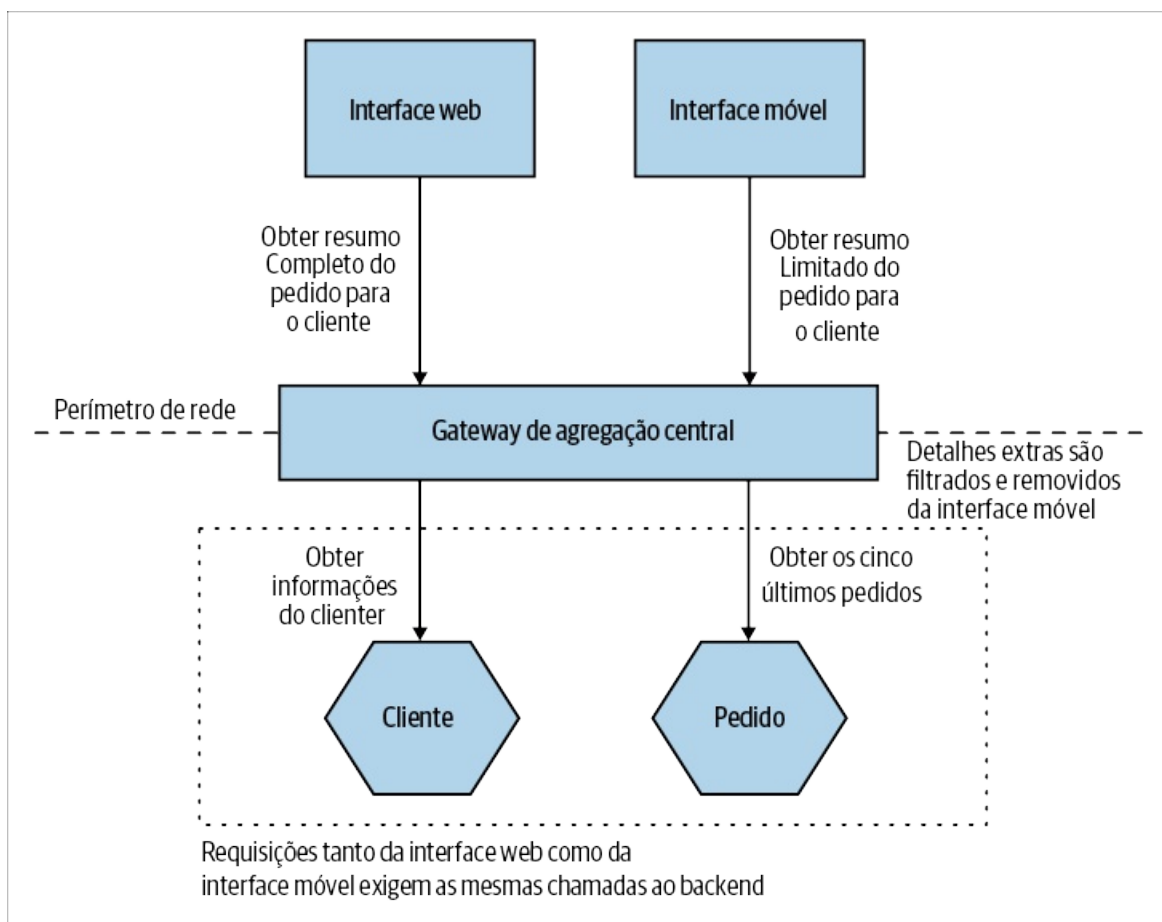


Figura 14.10 – Aceitando diferentes chamadas de agregação para dispositivos distintos.

Essa solução pode resultar em muito código no gateway, sobretudo se considerarmos diferentes aplicativos móveis nativos, sites voltados aos clientes, interfaces de administração interna e itens semelhantes. Também temos, é claro, o problema de que, embora essas diferentes UIs possam pertencer a equipes distintas, o gateway é uma única unidade – teremos os mesmos velhos problemas de ter diversas equipes tendo de trabalhar na mesma unidade de implantação. Nosso backend único de agregação pode acabar se tornando um gargalo, pois haverá muitas tentativas de alterações no mesmo artefato de implantação.

Preocupações variadas

Há uma série de preocupações que precisam ser consideradas do lado do servidor quando se trata de lidar com chamadas de API. Além da agregação

de chamadas e filtragem, podemos pensar em preocupações mais genéricas como gerenciamento de chaves de API, autenticação de usuários ou roteamento de chamadas. Com frequência, essas preocupações genéricas podem ser tratadas por produtos que sejam gateways de API, disponíveis em vários tamanhos e a preços muito distintos (alguns são extremamente caros!). Dependendo do nível de sofisticação que você precisar, pode fazer muito sentido comprar um produto (ou adquirir uma licença para um serviço) para lidar com algumas dessas preocupações. Você realmente quer gerenciar a concessão de chaves, monitoração, limitação de taxa e outras questões sobre API por conta própria? Não deixe de pesquisar os produtos disponíveis nessa área para resolver esses problemas genéricos, mas seja cauteloso quanto a tentar usar esses produtos para fazer agregação de chamadas e filtragem, ainda que aleguem que possam fazê-los.

Ao personalizar um produto criado por outros, você geralmente terá de trabalhar no mundo deles. Seu conjunto de ferramentas será restrito porque talvez você não possa utilizar a sua linguagem de programação e as suas práticas de desenvolvimento. Em vez de escrever um código Java, você irá configurar regras de roteamento em alguma DSL incomum, específica do produto (provavelmente utilizando JSON). Pode ser uma experiência frustrante, e você incluirá parte da inteligência de seu sistema em um produto de terceiros. Você terá menos chances de modificar esse comportamento depois. É comum perceber que um padrão de agregação de chamadas na verdade se relaciona a alguma funcionalidade do domínio que poderia justificar um microserviço próprio (algo que exploraremos melhor em breve, quando discutirmos os BFFs). Se esse comportamento estiver em uma configuração que seja específica de um fornecedor, alterar essa funcionalidade poderá ser mais complicado, pois teríamos provavelmente de recriá-la.

A situação pode piorar mais ainda se o gateway de agregação se tornar muito complexo, a ponto de exigir uma equipe dedicada que seja responsável por ele e o administre. No pior caso, adotar uma distribuição de responsabilidades mais horizontal entre as equipes pode levar a uma situação em que, para fazer o rollout de novas funcionalidades, será necessário que a equipe de frontend faça alterações, a equipe de gateway de

agregação faça alterações e a(s) equipe(s) responsável(is) pelo microsserviço também faça(m) suas alterações. Repentinamente, tudo começa a ficar muito mais lento.

Portanto, se quiser fazer uso de um gateway de API dedicado, vá em frente – mas considere fortemente ter sua lógica de filtragem e de agregação em outro lugar.

Quando usar

Para uma solução sob a responsabilidade de uma única equipe, na qual ela desenvolve a interface de usuário e os microsserviços no backend, eu não veria problemas em ter um único gateway de agregação central. Apesar disso, me parece que essa equipe teria MUITO trabalho – em situações com essas, em geral vejo um grau elevado de consistência entre as interfaces de usuário, e isso muitas vezes elimina a necessidade de haver esses pontos de agregação, para começar.

Se você realmente decidir adotar um único gateway de agregação central, tome cuidado para limitar as funcionalidades que incluirá aí. Eu seria extremamente cauteloso para colocar essa funcionalidade em um produto que seja um gateway de API mais genérico, por exemplo, pelos motivos apresentados antes.

No entanto, o conceito de fazer algum tipo de filtragem e agregação de chamadas no backend pode ser muito importante para otimizar a experiência dos usuários de nossas interfaces. O problema é que, em uma empresa com várias equipes que entregam software, um gateway central pode levar à exigência de muita coordenação entre essas equipes.

Assim, se ainda quisermos fazer agregação e filtragem no backend, mas quisermos eliminar os problemas associados ao modelo de responsabilidade de um gateway central, o que poderíamos fazer? É nesse caso que o padrão *backend para frontend* entra em ação.

Padrão: backend para frontend (BFF)

A principal distinção entre um BFF e um gateway de agregação central é

que um BFF tem um propósito único por natureza – é desenvolvido para uma interface de usuário específica. Esse padrão tem se provado ser muito bem-sucedido para ajudar a lidar com os diferentes problemas relacionados às interfaces de usuário, e tenho visto esse padrão funcionar bem em várias empresas, incluindo a SoundCloud^z e a REA. Como vemos na Figura 14.11, retomando o exemplo com a MusicCorp, as interfaces web e móvel para compras agora têm seus próprios backends de agregação.

Em virtude de sua natureza específica, o BFF elimina alguns dos problemas relacionados ao gateway de agregação central. Como não tentamos ser tudo para todos, um BFF evita se tornar um gargalo para o desenvolvimento, com várias equipes tentando dividir a responsabilidade por ele. Também teremos menos preocupações quanto ao acoplamento com a interface de usuário, pois ele será muito mais aceitável. Um dado BFF serve para uma interface de usuário específica – supondo que a mesma equipe seja responsável por ambos, o acoplamento inerente será muito mais fácil de administrar. Com frequência, descrevo o uso de um BFF com uma interface de usuário como se a UI estivesse separada em duas partes. Uma parte fica no dispositivo do cliente (a interface web ou o aplicativo móvel nativo), enquanto a segunda parte, isto é, o BFF, fica do lado do servidor.

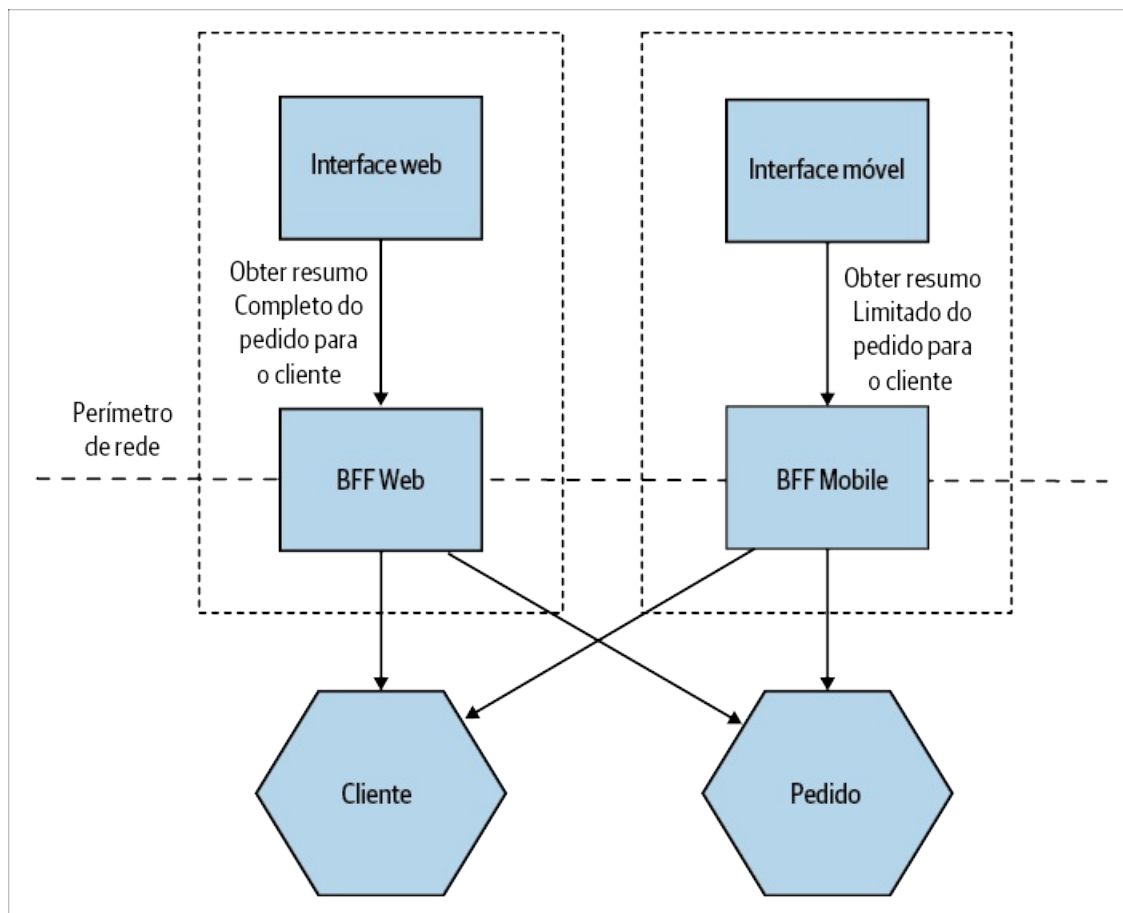


Figura 14.11 – Cada interface de usuário tem seu próprio BFF.

O BFF tem um alto acoplamento com uma experiência de usuário específica e, em geral, será mantido pela mesma equipe responsável pela interface do usuário, facilitando, desse modo, definir e adaptar a API conforme exigido pela UI, além de simplificar o processo de alinhar um lançamento dos componentes tanto do lado do cliente como do lado do servidor.

Quantos BFFs?

Quando se trata de oferecer a mesma experiência de usuário (ou uma experiência similar) em diferentes plataformas, tenho visto duas abordagens distintas. O modelo que eu prefiro (e que vejo com mais frequência) é ter estritamente um único BFF para cada tipo diferente de cliente – esse é um modelo que vi ser usado na REA, conforme mostrado na Figura 14.12. As aplicações Android e iOS, embora incluam

funcionalidades semelhantes, têm, cada uma, o seu próprio BFF.

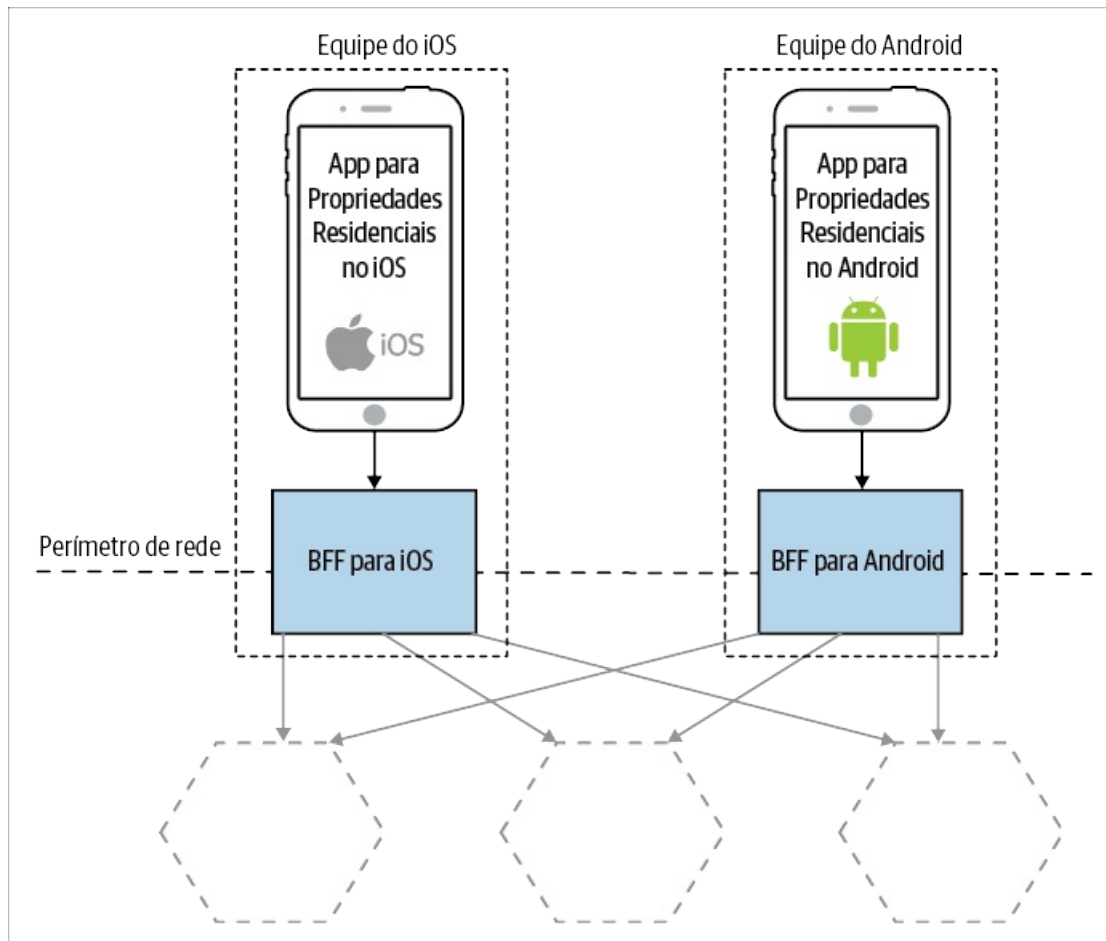


Figura 14.12 – As aplicações iOS e Android da REA têm diferentes BFFs.

Uma variação consiste em procurar oportunidades para usar o mesmo BFF para mais de um tipo de cliente, embora seja para o mesmo tipo de interface de usuário. O aplicativo Listener da SoundCloud permite que as pessoas ouçam conteúdos em seus dispositivos Android ou iOS. A SoundCloud utiliza um único BFF tanto para o Android como para o iOS, como mostra a Figura 14.13.

Minha principal preocupação nesse segundo modelo é que, quanto mais tipos de clientes utilizarem um único BFF, maior será a tentação de deixar o BFF inflado por lidar com vários problemas. Porém, a questão principal a ser compreendida nesse caso é que, ainda que os clientes compartilhem um BFF, ele será usado para o mesmo tipo de interface de usuário – assim, embora os aplicativos nativos Listener da SoundCloud para iOS e para

Android utilizem o mesmo BFF, outros aplicativos nativos utilizariam diferentes BFFs. Fico mais despreocupado com o uso desse modelo se a mesma equipe for responsável pelos aplicativos tanto para Android como para iOS e pelo BFF também – se esses aplicativos forem mantidos por equipes diferentes, eu ficaria mais inclinado a recomendar o modelo mais rígido. Desse modo, podemos ver a estrutura organizacional como um dos principais fatores determinantes para decidir qual modelo fará mais sentido (a lei de Conway vence mais uma vez).

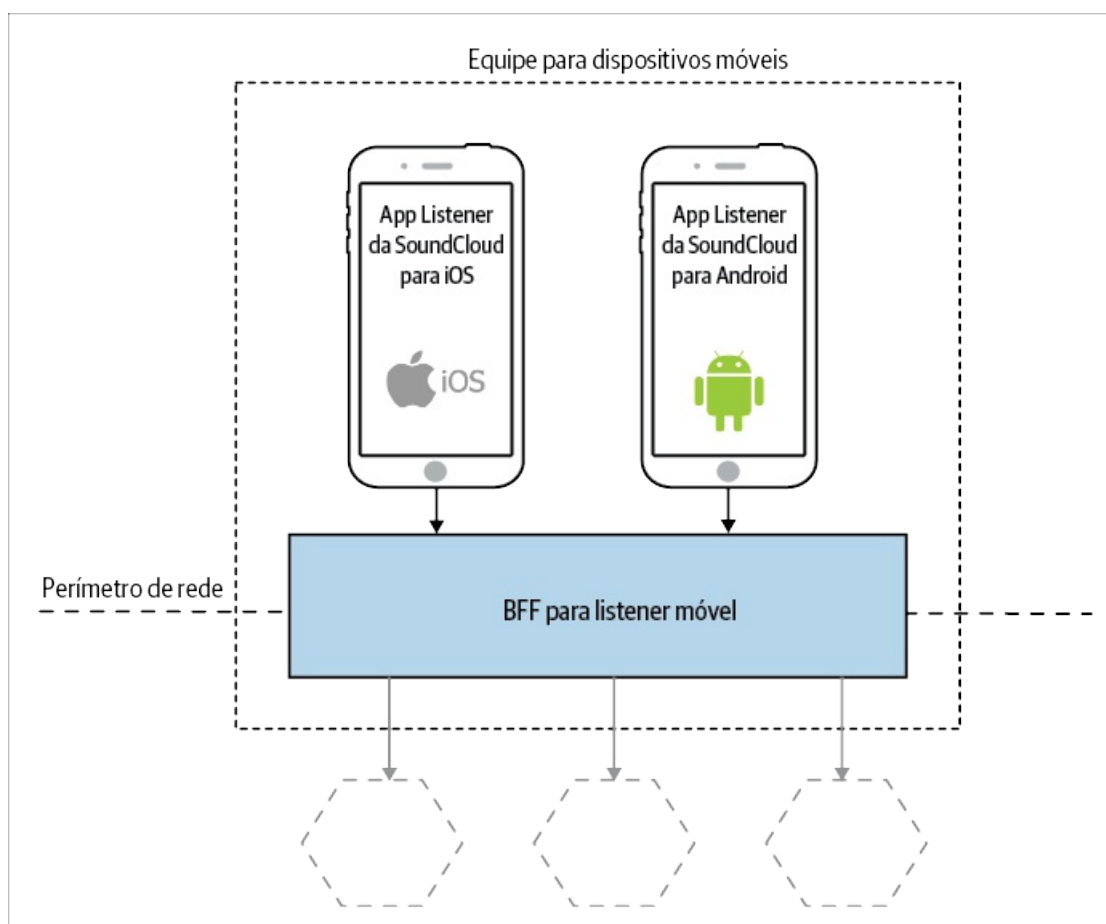


Figura 14.13 – A SoundCloud compartilha um BFF entre os aplicativos iOS e Android.

Stewart Gleadow da REA sugeriu a seguinte diretriz: “uma experiência, um BFF”. Assim, se as experiências para iOS e para Android forem muito parecidas, será mais fácil justificar ter um único BFF.⁸ Contudo, se elas divergirem bastante, ter BFFs distintos fará mais sentido. No caso da REA, embora houvesse pontos em comum entre as duas experiências, equipes

distintas eram responsáveis por elas e faziam o rollout de funcionalidades semelhantes de maneiras diferentes. Às vezes, a mesma funcionalidade poderia ser implantada de modo distinto em um dispositivo móvel diferente – o que poderia ser considerada uma experiência nativa em um aplicativo Android poderia exigir retrabalhos para parecer nativa no iOS.

Outra lição aprendida com a história da REA (e sobre a qual já falamos várias vezes) é que o software geralmente funciona melhor quando está alinhado com base nas fronteiras das equipes, e os BFFs não são uma exceção. A SoundCloud, por ter uma única equipe para dispositivos móveis, faz com que ter um único BFF pareça ser sensato à primeira vista, assim como o fato de a REA ter dois BFFs diferentes para as duas equipes distintas. Vale a pena notar que os engenheiros da SoundCloud com quem conversei sugeriram que ter um BFF para os aplicativos listener para Android e para iOS era algo que eles poderiam reconsiderar – havia uma única equipe para dispositivos móveis, mas, na verdade, ela era composta de uma mistura de especialistas em Android e em iOS, e eles se viam, na maior parte do tempo, trabalhando em um ou em outro aplicativo, implicando que, na prática, eram efetivamente duas equipes.

Muitas vezes, a tendência para ter menos BFFs se deve a um desejo de reutilizar funcionalidades do lado do servidor a fim de evitar muita duplicação, mas há outras formas de lidar com esse problema, as quais discutiremos a seguir.

Reutilização de código e BFFs

Uma das preocupações em ter um único BFF por interface de usuário é que podemos acabar com muita duplicação entre os BFFs. Por exemplo, eles podem acabar fazendo os mesmos tipos de agregações, ter o mesmo código ou um código semelhante para fazer interface com os serviços downstream, e assim por diante. Se você está tentando extrair funcionalidades comuns, um dos desafios geralmente será encontrá-las. Essa duplicação pode ocorrer nos próprios BFFs, mas também pode acabar acontecendo nos diferentes clientes. Por causa do fato de esses clientes utilizarem conjuntos muito distintos de tecnologias, identificar a ocorrência dessas duplicações

pode ser difícil. Em empresas que tendem a ter um conjunto de tecnologias em comum para os componentes do lado do servidor, talvez seja mais fácil identificar e fatorar os vários BFFs com duplicação.

Algumas pessoas reagem a isso querendo combinar de volta os BFFs, e acabam com um gateway de agregação de propósito geral. Minha preocupação em regredir para um único gateway de agregação é o fato de podermos acabar perdendo mais do que ganhamos, sobretudo porque pode haver outras formas de lidar com essa duplicação.

Como eu já havia dito antes, fico razoavelmente despreocupado com um código duplicado entre microsserviços. Isso equivale a dizer que, embora dentro das fronteiras de um único microsserviço eu vá geralmente fazer tudo que puder para refatorar e eliminar uma duplicação criando abstrações apropriadas, não reajo da mesma maneira diante de uma duplicação entre microsserviços. Em boa medida, isso ocorre porque, na maioria das vezes, eu me preocupo mais com a possibilidade de a extração de um código compartilhado resultar em um alto acoplamento entre os serviços (um assunto que exploramos na seção “DRY e os perigos da reutilização de código em um mundo de microsserviços”, na página [190](#)). Apesar disso, certamente há casos em que eliminar a duplicação se justifica.

Quando surgir a necessidade de realmente extrair um código comum a fim de permitir sua reutilização entre os BFFs, há duas opções óbvias. A primeira, geralmente menos custosa, porém mais frágil, é extrair alguma espécie de biblioteca compartilhada. Essa solução pode ser problemática porque as bibliotecas compartilhadas são uma causa significativa de acoplamento, sobretudo quando usadas para gerar bibliotecas de clientes para chamadas de serviços downstream. Apesar disso, há situações em que isso parece ser a decisão correta – particularmente quando o código sendo abstraído é uma preocupação puramente interna ao serviço.

A outra opção consiste em extrair a funcionalidade compartilhada em um novo microsserviço. Isso pode funcionar bem se a funcionalidade sendo extraída representa uma funcionalidade do domínio de negócios. Uma variação dessa abordagem poderia ser levar as responsabilidades pela

agregação a outros microsserviços downstream. Vamos considerar uma situação na qual queremos exibir os itens que estão na lista de desejos de um cliente, junto com informações sobre esses itens estarem ou não em estoque e o preço atual, conforme mostra a Tabela 14.1.

Tabela 14.1 – Exibindo a lista de desejos de um cliente da MusicCorp

The Brakes, Give Blood	Em estoque!	US\$ 5,99
Blue Juice, Retrospectable	Fora de estoque	US\$ 7,50
Hot Chip, Why Make Sense?	Vendendo rapidamente! (restam 2)	US\$ 9,99

O microsserviço `cliente` armazena informações sobre a lista de desejos e o ID de cada item. O microsserviço `catálogo` armazena o nome e o preço de cada item, enquanto os níveis de estoque são armazenados em nosso microsserviço `Estoque`. Para exibir esse mesmo controle tanto no aplicativo iOS como no aplicativo Android, cada BFF teria de fazer as mesmas três chamadas aos microsserviços de apoio, como mostra a Figura 14.14.

Uma forma de reduzir a duplicação na funcionalidade nesse caso seria extrair esse comportamento comum e passá-lo para um novo microsserviço. Na Figura 14.15, vemos nosso novo microsserviço dedicado `Lista de Desejos`, do qual tanto o aplicativo Android como o aplicativo iOS podem fazer uso.

Devo dizer que o mesmo código ser usado em dois lugares não me faria automaticamente querer extraí-lo em um novo serviço dessa forma, mas eu certamente consideraria isso se o custo da tarefa de criar outro serviço fosse suficientemente baixo, ou se eu estivesse usando o código em mais que apenas dois lugares – nessa situação específica, se estivessemos também exibindo as listas de desejo em nossa interface web, por exemplo, um microsserviço dedicado pareceria ainda mais atraente. Acho que o velho ditado de criar uma abstração quando você estiver prestes a implementar algo pela terceira vez ainda me parece ser uma boa regra geral, mesmo no nível dos serviços.

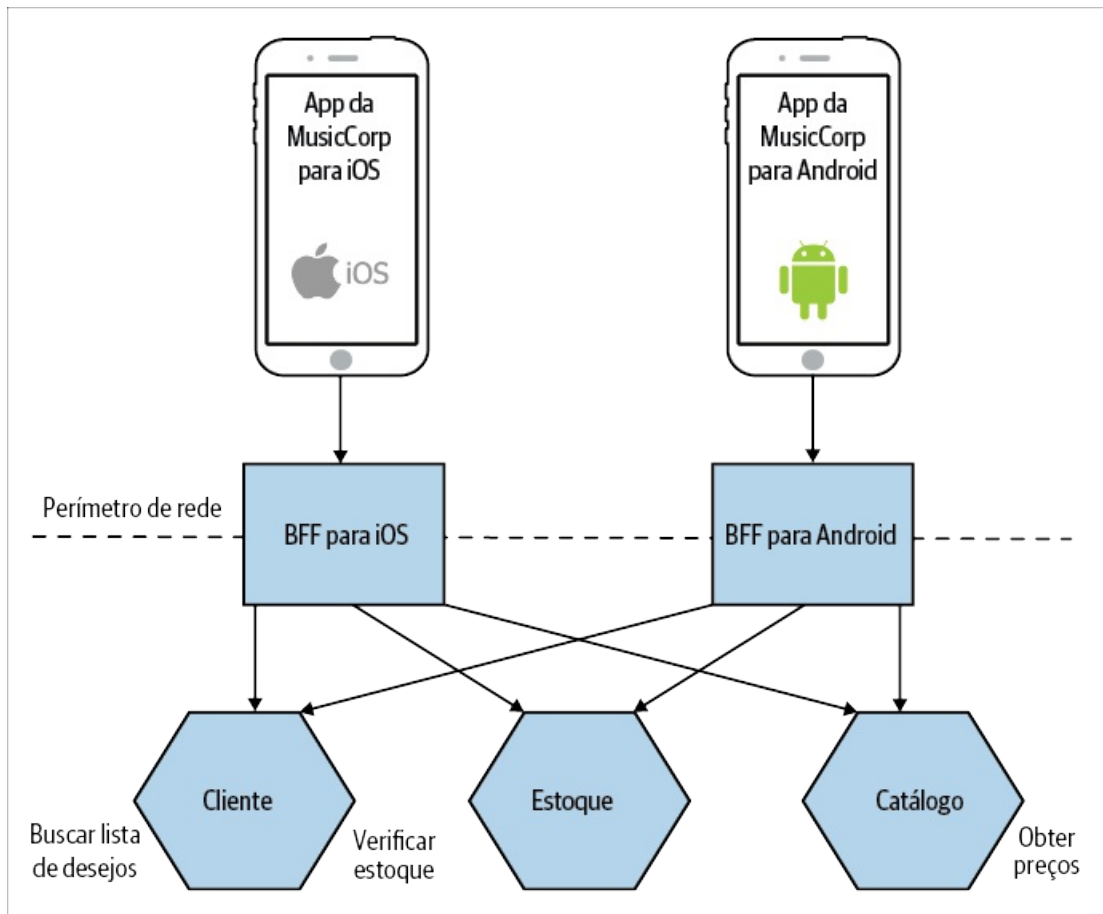


Figura 14.14 – Os dois BFFs executam as mesmas operações para exibir uma lista de desejos.

BFFs para web desktop e além

Podemos pensar no uso dos BFFs somente para resolver as limitações dos dispositivos móveis. A experiência web para desktop geralmente ocorre em dispositivos mais potentes, com melhor conectividade, nos quais o custo de fazer várias chamadas downstream é administrável. Isso permite que sua aplicação web faça várias chamadas diretamente para os serviços downstream, sem a necessidade de um BFF.

Contudo, já vi situações em que o uso de um BFF para a web também pode ser muito útil. Ao gerar uma grande parcela da UI web do lado do servidor (por exemplo, utilizando templating do lado do servidor), um BFF será o local óbvio em que isso poderá ser feito. Essa abordagem também pode simplificar de certo modo o caching, pois podemos colocar um proxy

reverso na frente do BFF, permitindo fazer cache dos resultados das chamadas agregadas.

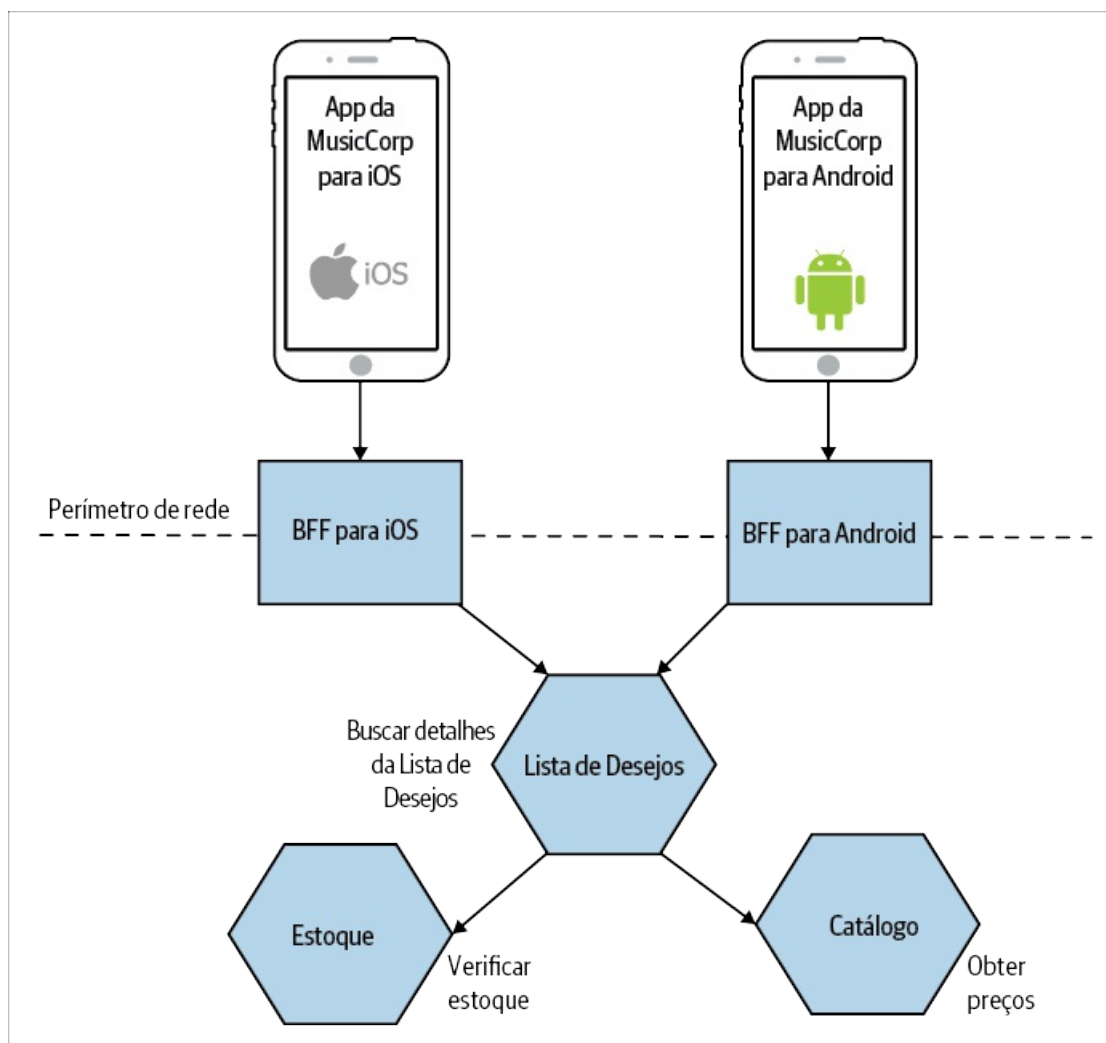


Figura 14.15 – Uma funcionalidade comum é extraída para um microserviço Lista de Desejos, permitindo que seja reutilizado entre os BFFs.

Já vi pelo menos uma empresa que utiliza BFFs para outros componentes externos que precisam fazer chamadas. Voltando ao meu exemplo de sempre com a MusicCorp, eu poderia expor um BFF para permitir que terceiros extraíssem informações sobre pagamento de royalties, ou para permitir streaming para uma série de dispositivos set-top box, conforme vemos na Figura 14.16. Esses não são mais BFFs de verdade, pois os componentes externos não apresentam uma “interface de usuário”, mas é um exemplo do mesmo padrão usado em um contexto diferente, portanto

achei que valeria a pena compartilhar.

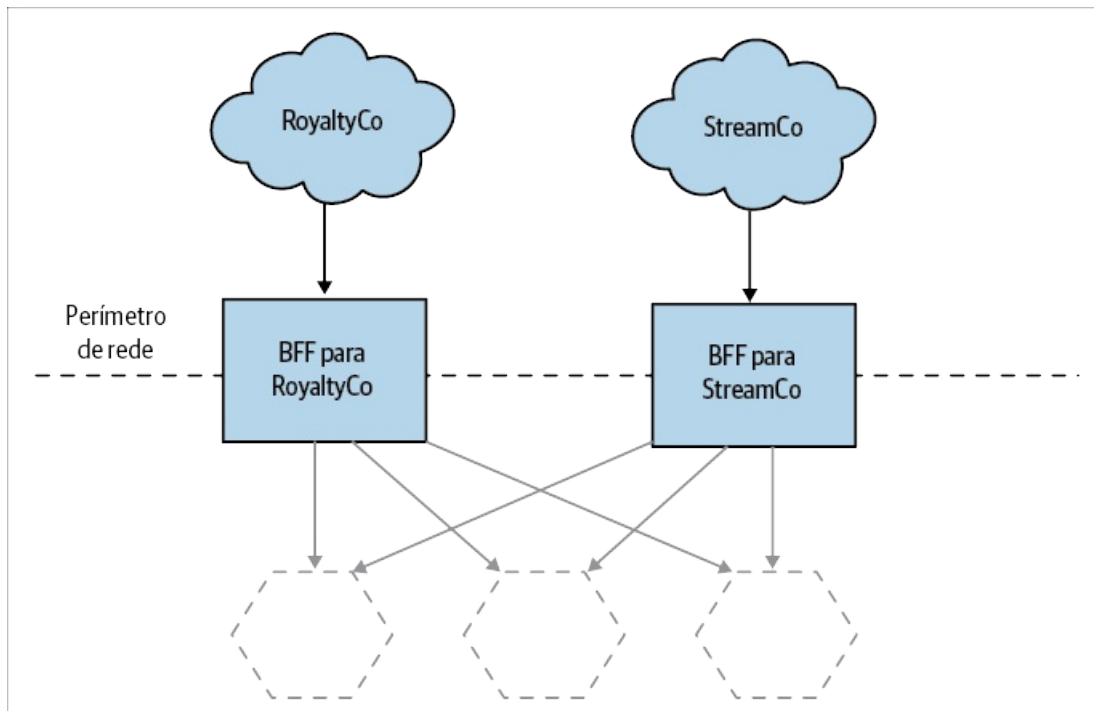


Figura 14.16 – Usando BFFs para gerenciar APIs externas.

Essa abordagem pode ser particularmente eficaz, pois os terceiros muitas vezes têm uma capacidade limitada, ou não têm capacidade (nem desejo), de usar ou modificar as chamadas de API que fazem. Com um backend de API centralizado, talvez você tenha de manter versões antigas da API presentes, somente para satisfazer um pequeno subconjunto dos terceiros que sejam incapazes de fazer uma alteração; com um BFF, esse problema será substancialmente reduzido. Ele também limita o impacto de haver alterações que causem incompatibilidade. Poderíamos alterar a API para o Facebook de um modo que houvesse quebra de compatibilidade com outras partes, mas, por utilizarem um BFF diferente, elas não seriam afetadas por essa mudança.

Quando usar

Para uma aplicação que disponibilize apenas uma UI web, suspeito que um BFF fará sentido somente se, e quando, houver um volume significativo de agregações necessárias do lado do servidor. Caso contrário, acho que algumas das outras técnicas de composição de UI que já discutimos

poderão funcionar igualmente bem, sem exigir um componente adicional do lado do servidor.

Porém, no instante em que você tiver de oferecer uma funcionalidade específica para uma UI móvel ou para um terceiro, eu recomendaria fortemente que você considerasse o uso de BFFs para cada cliente desde o princípio. Eu poderia reconsiderar se o custo de implantar serviços adicionais for alto, mas a separação de responsabilidades que um BFF pode proporcionar faz com que ele seja uma proposta muito atraente na maioria dos casos. Eu estaria mais inclinado ainda a usar um BFF se houver uma separação significativa entre as pessoas que criam a UI e os serviços downstream, por motivos que já apresentei.

Chegamos então à questão acerca de como implementar o BFF – vamos analisar o GraphQL e o papel que ele desempenha.

GraphQL

O GraphQL é uma linguagem de consulta que permite que os clientes executem queries para acessar ou modificar dados. Assim como o SQL, o GraphQL permite que essas queries sejam alteradas dinamicamente, possibilitando que o cliente defina exatamente as informações que quer obter. Com uma chamada REST sobre HTTP padrão, por exemplo, ao gerar uma requisição GET para um recurso `Pedido`, obteríamos todos os campos desse pedido. Mas, e se, nessa situação em particular, quiséssemos somente o valor total do pedido? Poderíamos simplesmente ignorar os outros campos, é claro, ou então disponibilizar um recurso alternativo (um `Resumo de Pedido`, talvez) que contivesse somente as informações de que precisássemos. Com o GraphQL, podemos gerar uma requisição pedindo somente os campos necessários, conforme vemos no Exemplo 14.1.

Exemplo 14.1 – Exemplo de uma query GraphQL usada para buscar informações de um pedido

```
{  
  order(id: 123) {  
    date  
    total  
  }  
}
```

```

    status
    delivery {
      company
      driver
      duedate
    }
  }
}

```

Nessa query, solicitamos o pedido 123, e pedimos o preço total e o status do pedido. Fomos além, e pedimos informações sobre a entrega desse pedido, de modo que possamos obter informações sobre o nome do motorista que entregará o nosso pacote, a empresa na qual trabalha e quando se espera que o pacote chegará. Com uma API REST usual, a menos que as informações de entrega estivessem contidas no recurso `Pedido`, seria necessário fazer uma chamada adicional para buscar esses dados. Assim, o GraphQL não só nos ajuda a pedir exatamente os campos que queremos, como também reduz os acessos de ida e volta. Uma query como essa exige que façamos a definição dos diversos tipos de dados que acessamos – definir explicitamente os tipos é uma parte essencial do GraphQL.

Para implementar o GraphQL, precisamos de um *resolver* para lidar com as queries. Um resolver do GraphQL fica do lado do servidor e mapeia as queries GraphQL às chamadas que irão buscar as informações. Assim, no caso de uma arquitetura de microsserviços, precisaríamos de um resolver que fosse capaz de mapear a requisição do pedido cujo `id` é 123 para uma chamada equivalente a um microsserviço.

Dessa maneira, podemos usar o GraphQL para implementar um gateway de agregação, ou até mesmo um BFF. A vantagem do GraphQL é que podemos facilmente alterar a agregação e a filtragem desejadas, simplesmente modificando a query do cliente; nenhuma mudança será necessária do lado do servidor no GraphQL, desde que os tipos do GraphQL aceitem a query que queremos fazer. Se não quiséssemos mais ver o nome do motorista na query de exemplo, poderíamos apenas omiti-lo na própria query, e essa informação não seria mais enviada. Por outro lado, se quiséssemos ver o número de pontos que foram concedidos por esse pedido, supondo que essa informação estivesse disponível no tipo referente

ao pedido, poderíamos simplesmente adicioná-la à query, e esse dado seria devolvido. Essa é uma vantagem significativa em relação às implementações de BFF, que exigem que mudanças na lógica de agregação sejam também aplicadas ao próprio BFF.

A flexibilidade que o GraphQL oferece para que o dispositivo cliente altere dinamicamente as queries feitas, sem que haja mudanças do lado do servidor, implica que há menos chances de seu servidor GraphQL se tornar um recurso compartilhado, para o qual haveria contenção, como foi discutido no caso do gateway de agregação de propósito geral. Apesar disso, alterações do lado do servidor ainda serão necessárias se for preciso expor novos tipos ou acrescentar campos aos tipos existentes. Desse modo, você ainda pode querer ter vários servidores GraphQL no backend alinhados de acordo com as fronteiras das equipes – portanto o GraphQL passa a ser uma forma de implementar um BFF.

Eu tenho preocupações associadas ao GraphQL, as quais apresentei em detalhes no Capítulo 5. Apesar disso, essa é uma solução interessante, que permite ter queries dinâmicas, apropriadas às necessidades dos diferentes tipos de interfaces de usuário.

Uma abordagem híbrida

Muitas das opções mencionadas antes não precisam servir para todos os casos. Eu poderia ver uma empresa adotando a abordagem de decomposição baseada em widgets para criar um site, mas usar uma abordagem de backend para frontend para seu aplicativo móvel. O ponto principal é que devemos manter a coesão das funcionalidades subjacentes que oferecemos aos nossos usuários. Precisamos garantir que a lógica associada a pedidos de produtos musicais ou à alteração de detalhes dos clientes estejam nos serviços que tratem essas operações, em vez de estarem espalhadas por todo o nosso sistema. Evitar a armadilha de colocar comportamentos demais em qualquer camada intermediária é uma decisão de custo-benefício complicada.

Resumo

Conforme espero ter mostrado, a decomposição de funcionalidades não precisa parar do lado do servidor, e ter equipes dedicadas de frontend não é inevitável. Apresentei várias maneiras diferentes de criar uma interface de usuário que possa fazer uso de microsserviços de apoio, ao mesmo tempo que permitem fazer entregas de software fim a fim, com bastante foco.

Em nosso próximo capítulo, passaremos do aspecto técnico para o lado das pessoas, explorando com mais detalhes a inter-relação entre os microsserviços e as estruturas organizacionais.

¹ Matthew Skelton e Manuel Pais, *Team Topologies* (Portland, OR: IT Revolution, 2019).

² N.T.: Esta e as demais citações deste capítulo foram traduzidas livremente, com base no texto original em inglês.

³ Como diz Charity Majors, “Você não é um desenvolvedor full-stack, a menos que crie os chips”.

⁴ Cam Jackson, “Micro Frontends”, martinowler.com, 19 de junho de 2019, <https://oreil.ly/U3K40>.

⁵ Estou olhando para você, *Sydney Morning Herald*!

⁶ Espero que nem seja preciso dizer que não sou advogado, mas, caso você queira consultar a legislação que trata desse assunto no Reino Unido, é o Equality Act (Lei da Igualdade) 2010, especificamente a seção 20. O W3C também tem uma boa visão geral das diretrizes para acessibilidade (<https://www.w3.org/TR/WCAG>).

⁷ Veja o artigo “BFF @ SoundCloud” (<https://oreil.ly/DdnzN>), de Lukasz Plotnicki, que apresenta uma ótima visão geral do uso do padrão BFF pela SoundCloud.

⁸ Stewart, por sua vez, atribuiu essa recomendação a Phil Calçado e Mustafa Sezgin.

Estruturas organizacionais

Embora boa parte do livro até agora tenha tido como foco os desafios técnicos para ir em direção a uma arquitetura com mais componentes menores e especializados, também vimos a inter-relação entre nossa arquitetura de microsserviços e a organização de nossas equipes. Na seção “Em direção a equipes alinhadas a fluxos”, na página [536](#), vimos o conceito de equipes alinhadas a fluxos (stream-aligned teams), com responsabilidade pela entrega de ponta a ponta das funcionalidades voltadas aos usuários, e como os microsserviços podem ajudar a fazer dessas estruturas de equipes uma realidade.

Precisamos agora dar corpo a essas ideias e analisar as demais considerações do ponto de vista organizacional. Conforme veremos, se quiser tirar o máximo de proveito dos microsserviços, ignore o organograma de sua empresa por sua conta e risco!

Empresas com baixo acoplamento

Neste livro, defendi uma arquitetura com baixo acoplamento e argumentei que é mais provável que uma aderência a equipes alinhadas a fluxos, mais autônomas, com baixo acoplamento, trará os melhores resultados. Uma mudança para uma arquitetura de microsserviços sem uma mudança na estrutura organizacional reduzirá a utilidade dos microsserviços – você poderá acabar pagando pelo custo (considerável) da mudança na arquitetura, sem obter o retorno sobre o seu investimento. Escrevi, de modo geral, sobre a necessidade de diminuir o trabalho de coordenação entre as equipes para ajudar a agilizar a entrega, o que, por sua vez, permite que as equipes tomem mais decisões por conta própria. Essas são ideias que exploraremos com mais detalhes neste capítulo, e apresentaremos de

forma concreta algumas dessas mudanças organizacionais e comportamentais necessárias; antes disso, porém, acho que é importante apresentar a minha visão de como é uma empresa com baixo acoplamento. Em seu livro *Accelerate*,¹ Nicole Forsgren, Jez Humble e Gene Kim analisaram as características de equipes autônomas, com baixo acoplamento, para entender melhor quais são os comportamentos mais importantes para alcançar o desempenho ideal. De acordo com os autores, o essencial é que as equipes possam:

- Fazer alterações de larga escala no design de seus sistemas sem a permissão de alguém externo à equipe.
- Fazer alterações de larga escala no design de seus sistemas sem depender de outras equipes fazerem alterações em seus sistemas nem gerar um volume significativo de trabalho para outras equipes.
- Concluir seu trabalho sem a necessidade de comunicação e coordenação com pessoas externas à sua equipe.
- Implantar e lançar seu produto ou serviço por demanda, independentemente de outros serviços dos quais ele dependa.
- Fazer a maior parte de seus testes por demanda, sem exigir um ambiente de testes integrado.
- Fazer implantações durante o horário comercial com um downtime desprezível.

A equipe alinhada a fluxo (stream-aligned team) – um conceito que vimos inicialmente no Capítulo 1 – se alinha a essa visão de uma empresa com baixo acoplamento. Se você está tentando passar para uma estrutura de equipes alinhadas a fluxos, essas características compõem uma lista fantástica de verificação para garantir que você está na direção correta.

Algumas dessas características parecem ser mais técnicas por natureza – por exemplo, ser capaz de fazer implantações durante o horário comercial poderia ser viabilizado por meio de uma arquitetura que aceite implantações com downtime zero. No entanto, tudo isso, na verdade, exige uma mudança comportamental. Para permitir que as equipes tenham um senso de responsabilidade mais completo pelos seus sistemas, será

necessário afastar-se de um controle centralizado, inclusive no modo como as decisões sobre a arquitetura são tomadas (algo que exploraremos no Capítulo 16). Basicamente, ter estruturas organizacionais com baixo acoplamento exige que a capacidade e a responsabilidade sejam *descentralizadas*.

A maior parte deste capítulo discutirá como fazer tudo isso funcionar, analisando os tamanhos das equipes, os tipos de modelos de responsabilidade, o papel da plataforma e outros aspectos. Há uma série de mudanças que podemos considerar para colocar a empresa na direção certa.

Antes de tudo isso, porém, vamos explorar um pouco mais a inter-relação entre a estrutura organizacional e a arquitetura.

Lei de Conway

Nosso mercado é jovem e parece estar constantemente se reinventando. Apesar disso, algumas “leis” básicas têm resistido ao teste do tempo. A lei de Moore, por exemplo, afirma que a densidade dos transistores nos circuitos integrados dobra a cada dois anos, e tem se provado ser surpreendentemente precisa (embora essa tendência venha diminuindo). Uma lei que acho ser quase universalmente verdadeira, e que é muito mais útil em meu trabalho cotidiano, é a lei de Conway.

O artigo “How Do Committees Invent?” (Como os comitês inventam?), de Melvin Conway, publicado na revista *Datamation* em abril de 1968, observou que:

Qualquer empresa que projeta um sistema (definido de modo mais amplo, indo além dos sistemas de informação, nesse caso) produzirá inevitavelmente um design cuja estrutura é uma cópia da estrutura de comunicação da empresa.²

Essa afirmação é citada com frequência, de diversas formas, como a lei de Conway. Eric S. Raymond resumiu esse fenômeno no *The New Hacker's Dictionary* (MIT Press), afirmando que: “Se houver quatro grupos trabalhando em um compilador, você terá um compilador de quatro

passos”.

A lei de Conway nos mostra que uma empresa com baixo acoplamento resultará em uma arquitetura com baixo acoplamento (e vice-versa), reforçando a ideia de que esperar desfrutar os benefícios de uma arquitetura de microsserviços com baixo acoplamento será problemático sem também considerar a estrutura organizacional que cria o software.

Evidências

Diz a história que, quando Melvin Conway submeteu seu artigo sobre esse assunto à *Harvard Business Review*, a revista o rejeitou, argumentando que ele não havia provado sua tese. Já vi sua teoria ser confirmada em tantas situações diferentes que a aceitei como verdadeira. No entanto, você não precisa acreditar somente no que eu digo: desde a submissão original do artigo de Conway, muito trabalho foi feito nessa área. Uma série de estudos foram conduzidos para explorar a inter-relação entre a estrutura organizacional e os sistemas criados por ela.

Empresas com baixo e alto acoplamento

Em “Exploring the Duality Between Product and Organizational Architectures” (Explorando a dualidade entre a arquitetura do produto e a arquitetura organizacional)³, os autores analisaram uma série de diferentes sistemas de software, classificando-os, grosso modo, como criados por “empresas com baixo acoplamento” ou “empresas com alto acoplamento”. Para as empresas com alto acoplamento, pense em firmas de produtos comerciais que geralmente se apresentam com visões e objetivos fortemente alinhados, enquanto as empresas com baixo acoplamento são bem representadas por comunidades open source distribuídas.

Em seu estudo, no qual combinaram pares de produtos semelhantes de cada tipo de empresa, os autores constataram que as empresas com mais baixo acoplamento criavam sistemas mais modulares, com menos acoplamento, enquanto os softwares das empresas com mais alto acoplamento eram menos modularizados.

Windows Vista

A Microsoft conduziu um estudo empírico⁴ no qual analisou como a sua própria estrutura organizacional causou impacto na qualidade de um produto específico de software, o Windows Vista. Mais exatamente, os pesquisadores analisaram vários fatores para determinar até que ponto um componente do sistema seria suscetível a erros.⁵ Após analisar várias métricas, incluindo métricas comumente usadas para avaliar a qualidade de um software, como complexidade do código, eles constataram que as métricas associadas às estruturas organizacionais (por exemplo, o número de engenheiros que haviam trabalhado em uma parte do código) provaram ser as medidas mais estatisticamente relevantes.

Então, nesse caso, temos outro exemplo da estrutura de uma empresa causando impacto na natureza do sistema criado por ela.

Netflix e Amazon

Provavelmente, as duas principais vitrines para a ideia de que estrutura organizacional e arquitetura devem estar alinhadas são a Amazon e a Netflix. Muito cedo, a Amazon começou a entender as vantagens de as equipes serem responsáveis pelo ciclo de vida completo dos sistemas que gerenciavam. Ela queria equipes que fossem responsáveis e operassem os sistemas dos quais cuidavam, gerenciando o ciclo de vida completo. A Amazon, porém, também sabia que equipes pequenas podiam trabalhar com mais rapidez que as equipes grandes. Isso resultou em suas famosas *equipes duas pizzas* (two-pizza teams), segundo as quais nenhuma equipe poderia ser tão grande a ponto de não poder ser alimentada com duas pizzas. Essa, é claro, não é uma métrica muito útil – jamais descobrimos se estávamos comendo a pizza no almoço ou no jantar (ou no café da manhã!), nem qual era o tamanho das pizzas – mas, novamente, a ideia geral é que o tamanho ideal de uma equipe seria de 8 a 10 pessoas, e essa equipe deve estar voltada para os clientes. Essa tendência de ter equipes menores responsáveis pelo ciclo de vida completo dos serviços é um dos motivos mais importantes pelos quais a Amazon desenvolveu a Amazon Web Services. Ela precisava criar as ferramentas que permitissem que suas equipes fossem autossuficientes.

A Netflix aprendeu com esse exemplo e garantiu que, desde o princípio, ela

fosse estruturada com base em equipes pequenas e independentes, de modo que os serviços que ela criasse também fossem independentes entre si. Isso garantiu que a arquitetura do sistema fosse otimizada com vistas à agilidade para mudanças. Com efeito, a Netflix projetou a estrutura organizacional de acordo com a arquitetura de sistemas que ela queria. Também já ouvi falar que essa ideia se estendeu ao planejamento dos locais em que estariam as equipes da Netflix – equipes cujos serviços conversassem uns com os outros estariam em locais próximos. A ideia é que você iria querer ter uma comunicação mais frequente com as equipes que usassem seus serviços, ou com as equipes cujos serviços você utilizaria.

Tamanho das equipes

Pergunte a qualquer desenvolvedor qual é o tamanho que uma equipe deveria ter, e, embora você vá obter respostas variadas, haverá um consenso de que, até certo ponto, uma equipe menor seria melhor. Se você pressionar para que digam um número para o tamanho “ideal” de uma equipe, de modo geral, você obterá respostas que variam de 5 a 10 pessoas.

Fiz algumas pesquisas a respeito do melhor tamanho de equipe para o desenvolvimento de software. Encontrei vários estudos, mas muitos apresentam tantas falhas que é difícil tirar conclusões para o amplo mundo do desenvolvimento de software. O melhor estudo que encontrei, “Empirical Findings on Team Size and Productivity in Software Development” (Descobertas empíricas sobre tamanho de equipe e produtividade no desenvolvimento de software)⁶, ao menos se baseou em um *corpus* grande de dados, embora não fosse necessariamente representativo da área de desenvolvimento de software como um todo. As descobertas dos pesquisadores mostraram que, “conforme esperado com base na literatura, a produtividade é pior para os projetos com (um tamanho médio de equipe de) nove ou mais pessoas”. Essa pesquisa pelo menos parece sustentar minhas próprias observações.

Gostamos de trabalhar em equipes pequenas, e não é difícil ver o porquê. Em uma equipe pequena, em que todos têm como foco os mesmos resultados, é mais fácil que as pessoas permaneçam alinhadas e será mais

simples coordenar o trabalho. Na verdade, tenho como hipótese (não testada) que estar geograficamente distante ou haver grandes diferenças de fuso horário entre os membros da equipe trará desafios que limitarão a equipe com tamanho ideal, mas essa ideia provavelmente seria mais bem explorada por outra pessoa que não eu.

Assim, equipes pequenas são boas, equipes grandes são ruins. Isso parece bem direto. Mas, se puder fazer todo o trabalho que você tiver de fazer com uma única equipe, ótimo! Seu mundo será simples e você provavelmente poderia ignorar boa parte do que resta ser apresentado neste capítulo. No entanto, e se houver mais trabalho do que tempo para fazê-lo? Uma resposta óbvia seria acrescentar mais pessoas. Mas, como sabemos, acrescentar pessoas não necessariamente contribuiria para que mais trabalho fosse feito.

Entendendo a lei de Conway

Evidências empíricas ou baseadas em observações pessoais sugerem que nossa estrutura organizacional tem forte influência sobre a natureza (e a qualidade) dos sistemas que criamos. Também sabemos que queremos equipes menores. Então, como esse entendimento poderia nos ajudar? Bem, basicamente, se quisermos uma arquitetura com baixo acoplamento para permitir que alterações sejam feitas com mais facilidade, devemos ter uma empresa com baixo acoplamento também. Falando de outro modo, o motivo pelo qual geralmente queremos uma empresa com mais baixo acoplamento é que queremos que diferentes partes dela sejam capazes de decidir e agir com mais rapidez e eficiência, e uma arquitetura de sistemas com baixo acoplamento ajuda bastante nessa tarefa.

No livro *Accelerate*, os autores identificaram uma correlação significativa entre as empresas que tinham uma arquitetura com baixo acoplamento e sua capacidade de fazer um uso mais eficaz das equipes de entrega:

Se conseguirmos ter uma arquitetura bem encapsulada, com baixo acoplamento, com uma estrutura organizacional correspondente, dois fatos importantes ocorrerão. O primeiro é que poderemos ter um desempenho melhor nas entregas, aumentando tanto o ritmo quanto a

estabilidade, ao mesmo tempo que reduziremos o estresse e as dificuldades na implantação. O segundo fato é que poderemos aumentar substancialmente o tamanho de nossa estrutura de engenharia e aumentar a produtividade linearmente – ou mais do que linearmente – à medida que fizermos isso.

Do ponto de vista organizacional, a mudança que já vem acontecendo há um tempo, sobretudo nas empresas que atuam em larga escala, é um afastamento dos modelos de comando e controle centralizados. Com uma tomada de decisões centralizada, a velocidade com a qual nossa empresa é capaz de reagir será significativamente reduzida. Isso piora à medida que a empresa cresce – quanto maior ela se tornar, menor será a eficiência na tomada de decisões e na velocidade de ação, em virtude de sua natureza centralizada.

As empresas cada vez mais têm reconhecido que, se quiserem escalar, mas quiserem continuar sendo rápidas, será necessário distribuir a responsabilidade de modo mais eficaz, separando a tomada de decisões centralizada e levando as decisões para partes da empresa que possam atuar com uma autonomia cada vez maior.

O truque, então, é criar empresas grandes com base em equipes menores e autônomas.

Equipes pequenas, empresa grande

Adicionar força de trabalho a um projeto de software atrasado o deixará mais atrasado ainda.

– *Fred Brooks (Lei de Brooks)*

Em seu famoso ensaio “The Mythical Man-Month”², o autor Fred Brooks tenta explicar por que usar “um homem-mês” como uma técnica de estimativa é problemático, pois nos faz cair na armadilha de pensar que podemos adicionar mais pessoas para que o problema seja resolvido com mais rapidez. Eis o que diz a teoria: se um trabalho exigirá seis meses de um desenvolvedor, então, se adicionarmos um segundo desenvolvedor, serão necessários apenas três meses. Se acrescentarmos cinco

desenvolvedores, agora teríamos seis no total, e o trabalho deverá ser feito em apenas um mês! É claro que o software não funciona assim.

Para que você seja capaz de acrescentar mais pessoas (ou equipes) para resolver um problema com mais rapidez, é necessário que o trabalho seja dividido em tarefas que possam, até certo ponto, ser conduzidas em paralelo. Se um desenvolvedor faz um trabalho pelo qual outro desenvolvedor precisa esperar, os trabalhos não poderão ser feitos em paralelo, mas deverão ser feitos sequencialmente. Mesmo que os trabalhos possam ser feitos em paralelo, geralmente há a necessidade de coordenar as pessoas que estão executando os diferentes fluxos de trabalho, resultando em um overhead adicional. Quanto mais entrelaçados estiverem os trabalhos, menos eficácia terá o acréscimo de mais pessoas.

Se não for possível separar o trabalho em subtarefas nas quais se possa trabalhar de modo independente, você não poderá simplesmente acrescentar pessoas para resolver o problema. Pior ainda, fazer isso provavelmente causará mais atrasos – adicionar novas pessoas ou criar outras equipes têm um custo. É preciso tempo para ajudar essas pessoas a se tornarem totalmente produtivas e, com frequência, os desenvolvedores que têm muito trabalho a fazer são os mesmos que terão de dedicar seu tempo ajudando as pessoas a estarem prontas para produzir.

O maior custo para trabalhar com eficiência em larga escala na entrega de software é a necessidade de coordenação. Quanto mais coordenação for necessária entre as equipes que trabalham em diferentes tarefas, mais lento você será. A Amazon, como empresa, reconheceu esse problema, e ela se estrutura de um modo a reduzir a necessidade de coordenação entre suas pequenas equipes duas pizzas. Com efeito, há uma tendência consciente para limitar o volume de coordenação entre as equipes exatamente por esse motivo, e restringir esse trabalho de coordenação, quando possível, às áreas em que seja absolutamente necessário, isto é, entre equipes que compartilham uma fronteira entre microsserviços. De acordo com o livro *Think Like Amazon*⁸, escrito por um ex-executivo da Amazon, John Rossman:

A Equipe Duas Pizzas é autônoma. A interação com outras equipes é

limitada, e quando ocorre, é bem documentada, e as interfaces são claramente definidas. Ela é dona e é responsável por todos os aspectos de seus sistemas. Um dos principais objetivos é reduzir o overhead de comunicações nas organizações, incluindo a quantidade de reuniões, pontos de coordenação, planejamento, testes ou lançamentos. Equipes mais independentes são mais rápidas.

Descobrir como uma equipe se enquadra na estrutura organizacional mais ampla é essencial. O livro *Team Topologies* define o conceito de uma *API de equipe* que, de modo geral, define como essa equipe interage com as outras partes da organização, não só no que concerne às interfaces de microsserviços, mas também quanto às práticas de trabalho:²

A API de equipe deve considerar explicitamente a usabilidade pelas outras equipes. As outras equipes acharão que interagir conosco é fácil e descomplicado, ou difícil e confuso? Com que nível de facilidade uma nova equipe entenderá o nosso código e as nossas práticas de trabalho? Como respondemos a pull requests e sugestões de outras equipes? O backlog de nossa equipe e os planos gerais para o produto são facilmente visíveis e compreensíveis às outras equipes?

Sobre a autonomia

Não importa o mercado no qual você atue, o mais importante são as pessoas e fazer com que elas façam o que é certo, proporcionando-lhes a confiança, a motivação, a liberdade e o desejo de atingirem o seu verdadeiro potencial.

– *John Timpson*

Ter várias equipes pequenas por si só não ajudará se essas equipes se transformarem em silos que continuem sendo dependentes de outras equipes para que seu trabalho seja feito. Precisamos garantir que cada uma dessas equipes pequenas tenha autonomia para fazer o trabalho pelo qual elas são responsáveis. Isso significa que devemos dar mais capacidade às equipes para tomar decisões, além das ferramentas para garantir que possam fazer o máximo possível sem precisar constantemente coordenar seu trabalho com outras equipes. Portanto permitir que haja autonomia é

fundamental.

Muitas empresas já mostraram as vantagens de criar equipes autônomas. Manter grupos organizacionais pequenos, permitindo que construam relacionamentos fortes e trabalhem juntos com eficácia, sem introduzir muita burocracia, tem ajudado muitas empresas a crescer e a escalar com mais eficiência em comparação com algumas empresas similares. A W. L. Gore and Associates¹⁰ fez muito sucesso garantindo que nenhuma de suas unidades de negócios jamais tivesse mais do que 150 pessoas, a fim de se certificar de que todos conhecessem uns aos outros. Para que essas unidades de negócio pequenas funcionem, elas precisam ter capacidade e responsabilidade para trabalhar como unidades autônomas.

Muitas dessas empresas parecem ter se baseado no trabalho feito pelo antropólogo Robin Dunbar, que analisou a capacidade humana de formar grupos sociais. Segundo sua teoria, nossa capacidade cognitiva estabelece limites para a eficácia com que podemos manter diferentes formas de relacionamentos sociais. Ele estimou em 150 pessoas como o tamanho máximo que um grupo poderia atingir até que fosse necessário separá-lo; caso contrário, ele entraria em colapso sob o seu próprio peso.

A Timpsons – uma empresa de varejo britânica extremamente bem-sucedida – atingiu uma escala gigantesca ao dar autonomia à sua força de trabalho, reduzindo a necessidade de funções centralizadas e permitindo que as lojas locais tomassem decisões por conta própria, por exemplo, acerca do valor de reembolso aos consumidores insatisfeitos. O presidente atual da empresa, John Timpson, ficou famoso por acabar com todas as regras internas e substituí-las por apenas duas:

- Observe o produto.
- Coloque o dinheiro na caixa registradora.

A autonomia também funciona em escalas menores, e as empresas mais modernas com as quais trabalho estão procurando criar equipes mais autônomas em suas organizações, muitas vezes tentando copiar modelos de outras empresas, como o modelo de equipe duas pizzas da Amazon ou o “modelo do Spotify”, que popularizou o conceito de quilds e chapters.¹¹ Eu devo, é claro, fazer uma advertência nesse caso – com toda certeza, você

deve aprender com o que as outras empresas fazem, mas saiba que copiar o que os outros fazem e esperar os mesmos resultados, sem realmente entender *por que* a outra empresa faz o que faz, pode não produzir o resultado que você espera.

Se feito da forma correta, dar autonomia às equipes pode deixar as pessoas mais capacitadas, pode ajudá-las a assumir mais responsabilidades e a crescer, e agilizará o trabalho. Quando as equipes se tornam responsáveis pelos microsserviços e têm total controle sobre eles, essas equipes terão mais autonomia na estrutura organizacional mais ampla.

O conceito de autonomia começa a mudar nossa compreensão acerca da responsabilidade em uma arquitetura de microsserviços. Vamos explorar esse assunto com mais detalhes.

Responsabilidade forte versus coletiva

Na seção “Definição de responsabilidades”, na página [256](#), discutimos os diferentes tipos de responsabilidade e exploramos as implicações desses estilos de responsabilidade no contexto de fazer alterações no código. Para uma breve recapitulação, as duas formas principais de responsabilidade pelo código que descrevemos são:

Responsabilidade forte

Um microsserviço pertence a uma equipe, e essa equipe decide as alterações que serão feitas nesse microsserviço. Se uma equipe externa quiser fazer alterações, ela precisará pedir à equipe responsável que as faça por ela, ou poderá ser solicitada a enviar um pull request – caberia, então, totalmente à equipe responsável decidir as circunstâncias nas quais o modelo de pull request seria aceito. Uma equipe pode ser responsável por mais de um microsserviço.

Responsabilidade coletiva

Qualquer equipe pode alterar qualquer microsserviço. Um trabalho cuidadoso de coordenação é necessário para garantir que as equipes não atrapalhem umas às outras.

Vamos explorar melhor as implicações desses modelos de responsabilidade

e como eles podem ajudar (ou dificultar) para seguirmos em direção a um aumento na autonomia das equipes.

Responsabilidade forte

Com a responsabilidade forte, a equipe responsável pelo microsserviço toma as decisões. No nível mais básico, ela tem controle total sobre as alterações de código a serem feitas. Indo um pouco além, a equipe pode decidir a respeito de padrões para escrita de código, idioms (padrões) de programação, quando implantar o software, qual tecnologia será usada para criar o microsserviço, a plataforma de implantação e outros aspectos. Ao ter mais responsabilidade sobre as mudanças que ocorrem no software, as equipes com responsabilidade forte terão um grau mais elevado de autonomia, com todas as vantagens que isso implica.

Uma responsabilidade forte, em última instância, diz respeito a otimizar com vistas à autonomia dessa equipe. Voltando ao modo Amazon de trabalhar, de acordo com o livro *Think Like Amazon*:

Quando se trata das famosas Equipes Duas Pizzas da Amazon, a maioria das pessoas não compreende o ponto principal. Não se trata do tamanho da equipe. Trata-se da autonomia, da responsabilidade e da mentalidade empreendedora da equipe. A Equipe Duas Pizzas diz respeito a dar condições a uma equipe pequena para atuar de modo independente e ágil em uma empresa.

Os modelos de responsabilidade forte podem permitir mais variações locais. Poderíamos ficar despreocupados, por exemplo, com o fato de uma equipe decidir criar seu microsserviço com um estilo funcional de Java, pois essa seria uma decisão que causaria impacto somente nessa equipe. É claro que essa variação deve ser, de certa forma, ponderada, pois algumas decisões exigem certo grau de consistência. Por exemplo, se todos os demais fazem uso de APIs REST sobre HTTP para os endpoints de seus microsserviços, mas você decide usar gRPC, isso poderia causar problemas para outras pessoas que quisessem fazer uso de seu microsserviço. Por outro lado, se esse endpoint gRPC for usado apenas internamente em sua equipe, isso talvez não seja problemático. Assim, ao tomar decisões locais,

mas que causem impacto em outras equipes, um trabalho de coordenação ainda poderá ser necessário. Descobrir quando e como envolver outras partes da empresa é algo que exploraremos em breve, quando analisarmos o modo de balancear otimizações locais *versus* globais.

Basicamente, quanto mais forte for o modelo de responsabilidade adotado por uma equipe, menos coordenação será necessária e, desse modo, mais produtiva poderá ser a equipe.

Até onde a responsabilidade forte pode chegar?

Até agora, falamos basicamente de aspectos como fazer alterações de código e escolher a tecnologia. Porém, o conceito de responsabilidade pode ir muito além. Algumas empresas adotam um modelo que eu descrevo como *responsabilidade total pelo ciclo de vida*. Na responsabilidade total pelo ciclo de vida, uma única equipe cria o design, faz as alterações, implanta o microsserviço, gerencia-o no ambiente de produção e, em última análise, aposenta o microsserviço quando ele não for mais necessário.

Esse modelo de responsabilidade total pelo ciclo de vida aumenta mais ainda a autonomia da equipe, pois reduz a necessidade de uma coordenação externa. Tickets não serão abertos junto às equipes de operações para que implantações sejam feitas, não será preciso ter autorizações externas para fazer mudanças e a equipe decide quais alterações serão feitas e quando serão lançadas.

Para muitos, um modelo como esse não seria realista, pois já haveria uma série de procedimentos definidos quanto ao modo como o trabalho deve ser feito. Talvez você também não tenha as habilidades apropriadas na equipe para assumir uma responsabilidade total, ou isso exigiria novas ferramentas (por exemplo, métodos de implantação para serem usados por conta própria). Vale a pena observar, é claro, que você não terá um modelo de responsabilidade total pelo ciclo de vida da noite para o dia, ainda que o considere uma meta desejada. Não fique surpreso se essa mudança levar anos para ser totalmente concretizada, sobretudo em uma empresa maior. Muitos aspectos da responsabilidade total pelo ciclo de vida podem exigir

mudanças culturais e ajustes significativos quanto às expectativas que você tem para uma parte de seu pessoal, por exemplo, a necessidade de oferecer suporte ao software fora do horário comercial. No entanto, à medida que sua equipe puder assumir mais responsabilidades pelos diversos aspectos de seu microsserviço, ela terá mais autonomia.

Não estou querendo sugerir que esse modelo seja, de alguma forma, essencial para usar microsserviços – acredito firmemente que uma responsabilidade forte em empresas com várias equipes é o modelo mais sensato para tirar o máximo de proveito dos microsserviços. Um modelo de responsabilidade forte para alterações no código é um bom ponto de partida – você pode trabalhar para seguir em direção a uma responsabilidade total pelo ciclo de vida com o passar do tempo.

Responsabilidade coletiva

Em um modelo com responsabilidade coletiva, um microsserviço pode ser alterado por qualquer pessoa de diversas equipes. Uma das principais vantagens da responsabilidade coletiva é que será possível deslocar as pessoas para onde elas forem necessárias. Isso pode ser conveniente caso seu gargalo no que diz respeito à entrega de software seja causado pela falta de pessoas. Por exemplo, várias alterações exigem que algumas atualizações sejam feitas no microsserviço `Pagamento` para permitir uma cobrança mensal automática – você poderia simplesmente designar pessoas extras para implementar essa alteração. É claro que acrescentar pessoas para resolver um problema nem sempre garantirá mais rapidez, mas, com o modelo de responsabilidade coletiva, certamente você terá mais flexibilidade quanto a esse aspecto.

Com equipes – e pessoas – se deslocando com mais frequência de um microsserviço para outro, precisamos ter um grau mais elevado de consistência acerca de como o trabalho é feito. Não podemos nos dar ao luxo de ter muita variedade de tecnologias ou diferentes tipos de modelos de implantação se esperamos que um desenvolvedor vá trabalhar com um microsserviço diferente a cada semana. Para ter algum nível de eficácia com um modelo de responsabilidade coletiva usando microsserviços, você

acabará tendo de garantir que trabalhar em um microserviço será muito semelhante a trabalhar com qualquer outro microserviço.

De forma inerente, isso pode acabar com uma das principais vantagens dos microserviços. Voltando a uma citação que usamos no início do livro, James Lewis disse que “os microserviços comprem opções para você”. Com um modelo de responsabilidade mais coletiva, é provável que você precisará *reduzir* as opções a fim de introduzir mais consistência entre o que as equipes fazem e como os microserviços são implementados.

A responsabilidade coletiva exige mais coordenação entre os indivíduos e entre as equipes em que estão esses indivíduos. Esse nível maior de coordenação resulta em mais acoplamento no nível organizacional. Voltando ao artigo de MacCormack et al. mencionado no início deste capítulo, temos a seguinte observação:

Em empresas com alto acoplamento... ainda que não seja uma escolha gerencial explícita, o design naturalmente se tornará mais acoplado.

Mais coordenação pode levar a um mais alto acoplamento organizacional, o que, por sua vez, resultará em designs de sistema mais acoplados. Os microserviços funcionam melhor quando podem adotar o conceito de implantações independentes de modo mais completo – e uma arquitetura com alto acoplamento será o oposto do que queremos.

Se houver poucos desenvolvedores, e talvez uma só equipe, um modelo de responsabilidade coletiva poderá ser totalmente apropriado. Contudo, à medida que o número de desenvolvedores aumentar, uma coordenação mais minuciosa, necessária para fazer o modelo de responsabilidade coletiva funcionar, acabará se transformando em um fator negativo relevante para desfrutar as vantagens da adoção das arquiteturas de microserviços.

No nível de uma equipe versus no nível organizacional

Os conceitos de responsabilidade forte e coletiva podem ser aplicados a diferentes níveis de uma empresa. Dentro de uma equipe, queremos que as pessoas estejam na mesma sintonia, sejam capazes de colaborar entre si com eficácia, e, como resultado, queremos garantir que haja um grau

elevado de responsabilidade coletiva. Como exemplo, isso se manifestaria no fato de todos os membros de uma equipe serem capazes de fazer alterações diretamente na base de código. Uma equipe com múltiplas habilidades, com foco na entrega de softwares voltados aos clientes de ponta a ponta, terá de ser muito eficaz para trabalhar com um modelo de responsabilidade coletiva. No nível organizacional, se quiser que as equipes tenham bastante autonomia, será importante que elas também tenham um modelo de responsabilidade forte.

Achando um equilíbrio entre os modelos

Em última análise, quanto mais tendermos a um modelo de responsabilidade coletiva, mais importante será ter consistência no modo de trabalhar. Quanto mais sua empresa tender à responsabilidade forte, mais otimizações locais poderão ser permitidas, conforme vemos na Figura 15.1. Esse equilíbrio não precisa ser imutável – é provável que você o mudará em diferentes ocasiões, com base em diferentes fatores. Poderíamos, por exemplo, dar liberdade total às equipes para escolher uma linguagem de programação, mas ainda exigir que elas façam as implantações na mesma plataforma de nuvem.

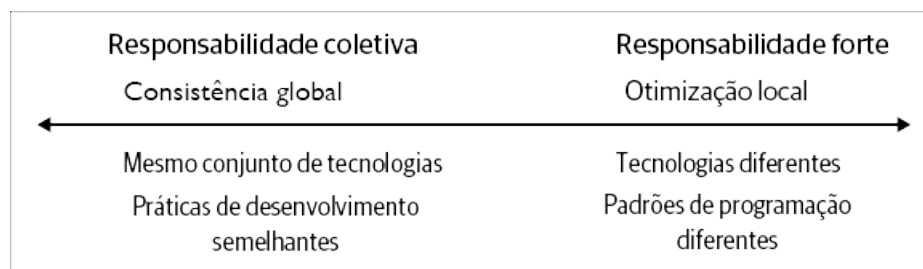


Figura 15.1 – Equilíbrio entre consistência global e otimizações locais.

Basicamente, porém, em um modelo de responsabilidade coletiva, quase sempre você será forçado a ir em direção à extremidade esquerda desse espectro – isto é, em direção a uma necessidade de haver mais consistência global. Com base em minha experiência, posso dizer que as empresas que mais tiram proveito dos microsserviços estão constantemente tentando encontrar maneiras de fazer a balança pender mais para a direita. Apesar disso, nas melhores empresas, isso não está gravado a ferro e fogo, mas é algo que continua sendo constantemente avaliado.

A verdade é que você não poderá fazer nenhuma ponderação quanto a essas questões, a menos que esteja ciente, até certo ponto, do que acontece na empresa como um todo. Ainda que transfira uma grande parcela da responsabilidade para as próprias equipes, pode ser que seja importante haver um cargo na estrutura organizacional de entrega de softwares que faça essas ponderações.

Equipes de capacitação

Vimos as equipes de capacitação (enabling teams) pela última vez na seção “Compartilhando especialistas”, na página 537, no contexto das interfaces de usuário, mas elas têm uma aplicação mais ampla do que aquela. Conforme descrito no livro *Team Topologies*, essas são as equipes que trabalham para oferecer suporte às nossas equipes alinhadas a fluxos (stream-aligned teams). Sempre que nossas equipes alinhadas a fluxos, responsáveis pelos microsserviços de ponta a ponta, estiverem concentradas na entrega de funcionalidades voltadas aos usuários, elas precisarão da ajuda de outros para fazerem o seu trabalho. Quando discutimos a interface de usuário, falamos da ideia de ter uma equipe de capacitação que pudesse auxiliar outras equipes na criação de experiências de usuário consistentes e eficazes. Como mostra a Figura 15.2, podemos vislumbrar isso na forma de equipes de capacitação trabalhando para oferecer suporte a várias equipes alinhadas a fluxos em algum aspecto que seja comum a várias delas.

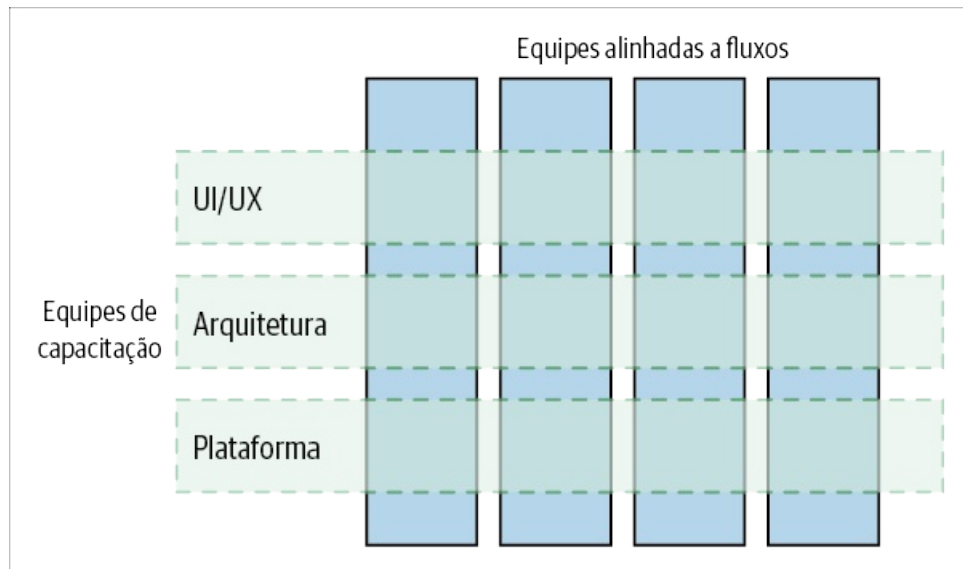


Figura 15.2 – Equipes de capacitação oferecem suporte às várias equipes alinhadas a fluxos.

No entanto, as equipes de capacitação podem ter diferentes formas e tamanhos.

Considere a Figura 15.3. Cada equipe decidiu escolher uma linguagem de programação diferente. Quando observadas isoladamente, cada uma dessas decisões parece fazer sentido – cada equipe escolheu a linguagem de programação com a qual se sente mais satisfeita. Mas e quanto à empresa como um todo? Você quer oferecer suporte a várias linguagens de programação diferentes em sua empresa? Como isso complica a rotatividade entre as equipes, e como afetará as contratações, se for o caso?

Você poderia decidir que, na verdade, não quer ter esse nível de otimizações locais – contudo, é preciso estar ciente dessas diferentes escolhas sendo feitas, e ser capaz de discutir essas mudanças caso quera ter certo nível de controle. É nesse caso que geralmente consigo ver um pequeno grupo auxiliar trabalhando com as equipes para ajudar a conectar as pessoas a fim de que essas discussões possam acontecer de forma apropriada.

São esses grupos de apoio para todas as equipes que consigo ver como o local mais óbvio para os arquitetos estarem, pelo menos em parte de seu tempo. Um arquiteto à moda antiga diria às pessoas o que elas deveriam fazer. Na empresa nova, moderna e descentralizada, os arquitetos estudam

o panorama, identificam tendências, ajudam a conectar pessoas e atuam para promover um espaço de discussão para ajudar outras equipes a fazerem o seu trabalho. Eles não são uma unidade de controle nesse mundo; eles têm uma função para capacitação (muitas vezes, com um novo nome – já vi termos como *principal engineer* (engenheiro principal) usado para pessoas que desempenham o papel daquilo que eu consideraria um arquiteto). Exploraremos melhor o papel dos arquitetos no Capítulo 16.

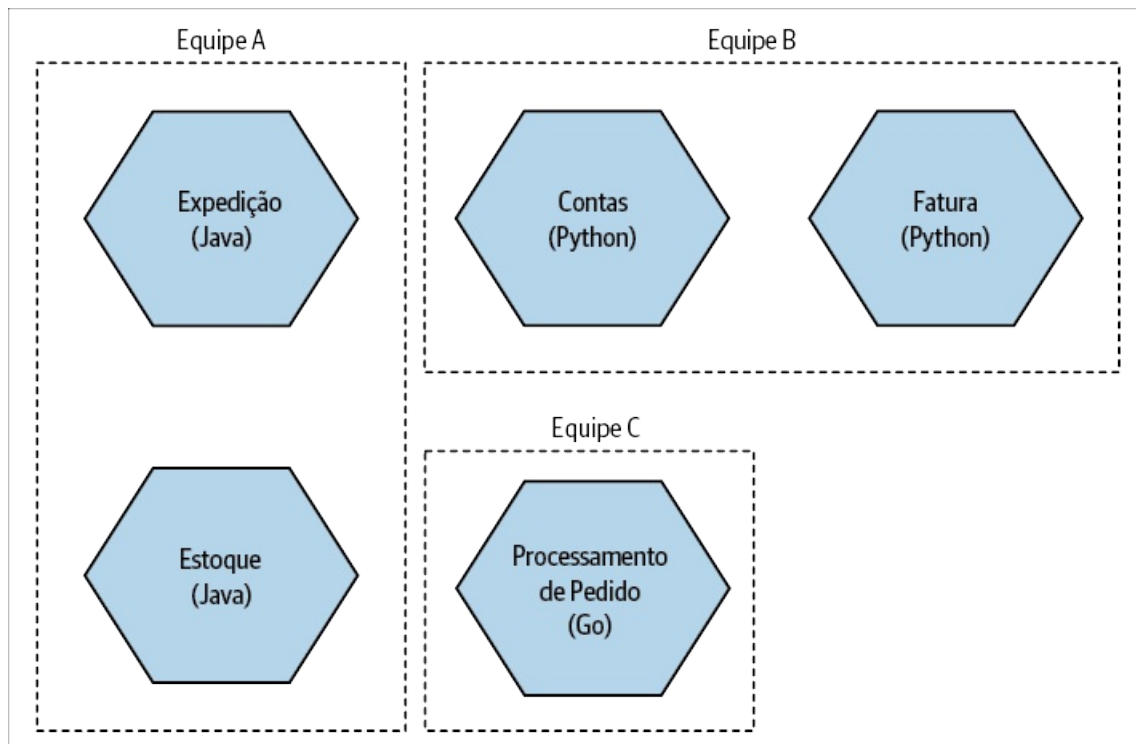


Figura 15.3 – Cada equipe escolheu uma linguagem de programação diferente.

Uma equipe de capacitação também pode ajudar a identificar problemas que seriam corrigidos de modo mais apropriado fora das equipes. Considere um cenário em que cada equipe estivesse com dificuldades para iniciar bancos de dados contendo dados de teste. Cada equipe contornou esse problema de uma maneira diferente, porém o problema jamais foi suficientemente importante para que uma equipe o corrigisse de modo apropriado. Contudo, observamos as várias equipes e constatamos que muitas delas se beneficiariam com uma solução apropriada para o

problema e, de repente, torna-se óbvio que ele precisa ser resolvido.

Comunidades de prática

Uma *comunidade de prática* (Community of Practice, ou CoP) é um grupo abrangente que promove o compartilhamento e o aprendizado entre pares. Quando organizadas da forma correta, as comunidades de prática são uma maneira incrível de criar uma organização na qual as pessoas possam aprender e crescer continuamente. Em seu excelente livro sobre o assunto, *Building Successful Communities of Practice*¹², Emily Webber escreve o seguinte:

As comunidades de prática criam o ambiente apropriado para o aprendizado social, o aprendizado experimental e um currículo completo, levando a uma aprendizagem rápida pelos membros... Elas podem incentivar uma cultura de aprendizado na qual as pessoas procuram melhores maneiras de trabalhar, em vez de apenas utilizar os modelos existentes.

Acredito, porém, que, em alguns casos, o mesmo grupo poderia ser igualmente uma CoP e uma equipe de capacitação, mas, com base em minha própria experiência, posso dizer que isso é raro. Contudo, há, sem dúvida, uma sobreposição. Tanto as equipes de capacitação como as comunidades de prática oferecem insights acerca do que acontece em diferentes equipes de sua empresa. Esses insights podem ajudar a saber se é necessário avaliar novamente o equilíbrio entre as otimizações globais e as otimizações locais, ou ajudam a identificar se um trabalho mais centralizado se faz necessário, porém a diferenciação nesse caso está nas responsabilidades e capacidades do grupo.

Membros de uma equipe de capacitação em geral trabalham como parte da equipe em tempo integral, ou têm uma parcela significativa de seu tempo reservada para esse propósito. Desse modo, eles têm mais espaço de manobra para colocar as mudanças em ação – para realmente trabalhar com outras equipes e ajudá-las. As comunidades de prática se preocupam mais em promover o aprendizado – os indivíduos do grupo geralmente participam de um fórum ao qual dedicam algumas horas da semana no

máximo, e a participação em um grupo como esse geralmente é fluida.

As CoPs e as equipes de capacitação evidentemente podem trabalhar juntas com muita eficácia. Em geral, uma CoP é capaz de oferecer insights valiosos que podem ajudar uma equipe de capacitação a entender melhor o que é necessário. Considere uma CoP de Kubernetes que compartilhe sua experiência sobre as dificuldades que tiveram para trabalhar no cluster de desenvolvimento de sua empresa com a equipe de plataforma que gerencia o cluster. No que concerne às equipes de plataforma, esse é um assunto que vale a pena ver com mais detalhes.

A plataforma

Para voltar às nossas equipes alinhadas a fluxos, com baixo acoplamento, esperamos que elas façam seus próprios testes em ambientes isolados, gerenciem as implantações de um modo que possam ser feitas durante o dia e façam mudanças em sua arquitetura de sistema quando for necessário. Tudo isso parece levar cada vez mais responsabilidade – e trabalho – para essas equipes. As equipes de capacitação, como um conceito geral, podem ajudar nesse caso, mas, em última instância, as equipes alinhadas a fluxos precisam de um conjunto de ferramentas que permita que elas façam o seu trabalho por conta própria – é a plataforma.

Com efeito, sem uma plataforma, pode ser difícil mudar a estrutura organizacional. Em seu artigo “Convergence to Kubernetes” (Convergência para o Kubernetes)¹³, Paul Ingles, CTO da RVU, apresenta sua experiência com o site de comparação de preços Uswitch, na qual os serviços de baixo nível da AWS deixaram de ser usados, e uma plataforma com um nível mais alto de abstração, baseada no Kubernetes, passou a ser utilizada. A ideia é que essa plataforma permitiu que as equipes alinhadas a fluxo da RVU mantivessem o foco na entrega de novas funcionalidades e passassem menos tempo gerenciando a infraestrutura. Segundo Paul:

Não mudamos nossa estrutura organizacional porque queríamos usar o Kubernetes; usamos o Kubernetes porque queríamos mudar a nossa estrutura organizacional.

Uma plataforma capaz de implementar funcionalidades comuns, por

exemplo, a capacidade de lidar com o gerenciamento de estado desejado (desired state management) para os microsserviços, agregação de logs, e autorização e autenticação entre microsserviços, pode proporcionar melhorias incríveis de produtividade e permitir que as equipes assumam mais responsabilidades, sem que tenham também de aumentar drasticamente o volume de trabalho que fazem. Com efeito, uma plataforma deve oferecer mais recursos para que as equipes possam manter o foco na entrega de funcionalidades.

Equipe de plataforma

Uma plataforma exige que haja alguém para operá-la e administrá-la. Esses conjuntos de tecnologia podem ser muito complexos, a ponto de justificar alguma expertise específica. Minha preocupação, porém, é que, ocasionalmente, é muito fácil que essas equipes de plataforma percam a visão do porquê de elas existirem.

Uma equipe de plataforma tem usuários, do mesmo modo que qualquer outra equipe. Os usuários da equipe de plataforma são os outros desenvolvedores – seu trabalho, se você estiver em uma equipe de plataforma, é fazer com que a vida deles seja mais fácil (esse, é claro, é o trabalho de qualquer equipe de capacitação). Isso significa que a plataforma que você criar precisa atender às necessidades da equipe que a utiliza. Também significa que é preciso trabalhar com as equipes que utilizam a sua plataforma não só para ajudá-las a utilizar bem a plataforma, mas também para ouvir seu feedback e suas exigências a fim de melhorar a plataforma que você disponibiliza.

No passado, eu preferiria chamar uma equipe como essa de algo como “delivery services” (serviços para entrega) ou “delivery support” (suporte para entrega) para explicitar melhor seus objetivos. Na verdade, o trabalho de uma equipe de plataforma *não é* criar uma plataforma; é facilitar o desenvolvimento e o lançamento de funcionalidades. Criar uma plataforma é somente uma das maneiras pelas quais os membros de uma equipe de plataforma podem fazer isso. Eu me preocupo com o fato de que, ao chamá-los de equipe de plataforma, eles vejam todos os problemas como algo que possa e deva ser resolvido pela plataforma, em vez de pensar de

um modo mais amplo em outras formas de facilitar a vida dos desenvolvedores.

Como qualquer boa equipe de capacitação, uma equipe de plataforma precisa atuar quase como uma consultoria interna, até certo ponto. Se você estiver em uma equipe de plataforma, será preciso sair por aí e descobrir quais são os problemas que as pessoas estão enfrentando e trabalhar com elas para ajudá-las a resolver esses problemas. No entanto, como você acabará construindo a plataforma também, será necessário que haja uma boa dose de trabalho de desenvolvimento de produto nesse caso também. Com efeito, adotar uma abordagem de desenvolvimento de produto ao modo de construir a sua plataforma é uma ótima ideia, e poderia ser uma ótima oportunidade para ajudar no desenvolvimento de novos donos de produtos (product owners).

Estrada pavimentada

Um conceito que tem se tornado popular no desenvolvimento de software é o da “estrada pavimentada” (paved road). A ideia é que você comunique claramente como quer que o trabalho seja feito, e então ofereça os meios pelos quais isso possa ser feito facilmente. Por exemplo, talvez você queira garantir que todos os microsserviços se comuniquem por meio de TLS mútuo. Você poderia, então, oferecer suporte para esse recurso disponibilizando um framework comum ou uma plataforma de implantação que disponibilizasse automaticamente o TLS mútuo para os microsserviços que executassem aí. Uma plataforma pode ser uma ótima maneira de oferecer essa estrada pavimentada.

O conceito principal na ideia de estrada pavimentada é que o uso dessa estrada não é obrigatório – ela apenas oferece um modo mais fácil de chegar ao seu destino. Assim, se uma equipe quiser garantir que seus microsserviços se comuniquem por meio de TLS mútuo sem utilizar o framework comum, ela teria de encontrar outra maneira de realizar a tarefa, mas isso seria permitido. A analogia, nesse caso, é que, embora nosso desejo seja de que todos cheguem ao mesmo destino, as pessoas são livres para encontrar seus próprios caminhos – a expectativa é que a estrada pavimentada seja o modo mais fácil de chegar até onde você

precisa.

O conceito de estrada pavimentada tem como objetivo facilitar os casos comuns, ao mesmo tempo que deixa espaço para as exceções quando essas se justificarem.

Se pensarmos na plataforma como uma estrada pavimentada, encarando-a como opcional, incentivamos a equipe de plataforma a deixar a plataforma fácil de usar. Eis o que diz Paul Ingles novamente, a respeito de avaliar a eficácia da equipe de plataforma:¹⁴

Havíamos definido [OKRs (Objectives and Key Results, ou Objetivos e Resultados Essenciais)] com base no número de equipes que queríamos que adotassem a plataforma, no número de aplicações que usavam o serviço de escalabilidade automática (autoscaling) da plataforma, na proporção das aplicações que haviam passado para o serviço dinâmico de credenciais da plataforma, e assim por diante. Monitorávamos algumas dessas informações por longos períodos, enquanto outras eram úteis para orientar o progresso durante um trimestre, e então as descartávamos em favor de outros dados.

Jamais obrigamos o uso da plataforma, portanto definir resultados essenciais com base no número de equipes que a adotavam nos forçava a manter o foco na resolução dos problemas que incentivaria a adoção da plataforma. Também observávamos as medidas naturais de progresso: a proporção do tráfego atendido pela plataforma e a proporção da receita gerada por meio dos serviços da plataforma são, ambos, bons exemplos disso.

Se colocarmos obstáculos que pareçam ser arbitrários ou ser apenas um capricho no caminho das pessoas, elas encontrarão maneiras de contorná-los para que um trabalho seja feito. Portanto, em geral, acho muito mais eficaz explicar por que as tarefas deveriam ser feitas de uma determinada maneira, e então facilitar para que sejam feitas dessa forma, em vez de tentar tornar impossíveis as soluções das quais você não gosta.

A mudança para equipes alinhadas a fluxo, mais autônomas, não elimina a necessidade de ter uma visão técnica clara ou de ser claro a respeito de determinadas tarefas que todas as equipes devam fazer. Se houver

restrições concretas (a necessidade de ser independente do provedor de nuvem, por exemplo) ou requisitos específicos que todas as equipes devam obedecer (todas as PII em repouso devem ser criptografadas usando algoritmos específicos), isso ainda precisa ser claramente comunicado, e os motivos devem estar claros. A plataforma poderá então desempenhar uma função para facilitar essas tarefas. Ao usar a plataforma, você estará na estrada pavimentada – muitas tarefas acabarão sendo feitas corretamente, sem precisar dispendar muitos esforços.

Por outro lado, vejo algumas empresas que tentam governar por meio da plataforma. Em vez de explicar claramente o que precisa ser feito e o porquê, elas simplesmente dizem: “você deve usar esta plataforma”. O problema com isso é que, se a plataforma não for fácil de usar ou não for apropriada para um caso de uso em particular, as pessoas encontrarão maneiras de ignorá-la. Se as equipes trabalharem fora da plataforma, elas não terão uma noção clara das restrições que são importantes para a empresa, e se verão empregando as soluções “incorretas” sem perceber.

Microsserviços compartilhados

Conforme já discutimos, sou um forte defensor dos modelos de responsabilidade forte para os microsserviços. Em geral, um microsserviço deve pertencer a uma só equipe. Apesar disso, ainda percebo que é comum que os microsserviços pertençam a várias equipes. Por que isso acontece? E o que podemos (ou devemos) fazer a esse respeito? É importante entender os motivos que levam as pessoas a terem microsserviços compartilhados por várias equipes, sobretudo porque é possível encontrar alguns modelos alternativos atraentes, capazes de resolver as preocupações básicas das pessoas.

Difícil demais para separar

Obviamente, um dos motivos pelos quais poderíamos ter um microsserviço sob a responsabilidade de mais de uma equipe é que o custo de decompor o microsserviço em partes que possam pertencer a diferentes equipes é alto demais, ou porque a sua empresa talvez não veja os motivos para isso. Essa

é uma ocorrência comum em sistemas monolíticos grandes. Se esse é o principal desafio que você enfrenta, espero que alguns dos conselhos apresentados no Capítulo 3 sejam úteis. Poderíamos também considerar uma fusão das equipes para ter um melhor alinhamento com a arquitetura em questão.

A FinanceCo, uma empresa FinTech que conhecemos antes na seção “Em direção a equipes alinhadas a fluxos”, na página 536, em sua maior parte, opera com um modelo de responsabilidade forte, com um grau elevado de autonomia nas equipes. No entanto, ela ainda tem um sistema monolítico que está sendo dividido aos poucos. Essa aplicação monolítica, para todos os fins e propósitos, era compartilhada por várias equipes, e o custo alto de se trabalhar com essa base de código compartilhada era evidente.

Alterações abrangentes

Muito do que discutimos até agora neste livro quanto à inter-relação entre a estrutura organizacional e a arquitetura tem como propósito reduzir a necessidade de coordenação entre as equipes. Boa parte disso diz respeito a tentar reduzir ao máximo as alterações que envolvam várias equipes. Temos, porém, de reconhecer que algumas alterações abrangentes podem ser inevitáveis.

A FinanceCo se viu diante de um problema desse tipo. Originalmente, uma conta estava associada a um único usuário. À medida que a empresa cresceu e passou a ter mais usuários que eram pessoas jurídicas (ela estava mais focada em pessoas físicas anteriormente), isso se tornou uma limitação. A empresa queria passar para um modelo no qual uma única conta da FinanceCo pudesse acomodar vários usuários. Essa era uma mudança fundamental, pois, até aquele momento, o pressuposto em todo o sistema havia sido: uma conta = um usuário.

Uma única equipe foi composta para fazer essa mudança acontecer. O problema era que uma grande parte do trabalho envolvia fazer alterações em microsserviços que já pertenciam a outras equipes. Isso significava que o trabalho da equipe, parcialmente, dizia respeito a fazer alterações e submeter pull requests, ou pedir às outras equipes que fizessem as

alterações acontecerem. Fazer a coordenação dessas mudanças era muito complicado, pois um número significativo de microsserviços precisava ser modificado a fim de aceitar a nova funcionalidade.

Ao reestruturar nossas equipes e a arquitetura para eliminar um conjunto de mudanças abrangentes, poderíamos, com efeito, nos expor a um conjunto diferente de alterações abrangentes que poderiam causar um impacto maior. Foi o que aconteceu na FinanceCo – o tipo de reorganização necessário para reduzir o custo da funcionalidade de múltiplos usuários iria aumentar o custo de fazer outras alterações mais comuns. A FinanceCo entendeu que essa mudança em particular seria muito complicada, mas que era um tipo de alteração tão excepcional que essa complicação seria aceitável.

Gargalos na entrega

Um motivo principal para as pessoas adotarem uma responsabilidade coletiva, com microsserviços compartilhados entre as equipes, é evitar gargalos na entrega. E se houver um grande backlog de alterações que precisam ser feitas em um único serviço? Vamos voltar à MusicCorp e supor que estamos fazendo o rollout de uma funcionalidade que permita que um cliente veja o gênero musical em nossos produtos, assim como acrescentando um tipo totalmente novo de produto: ringtones (toques) musicais virtuais para telefones celulares. A equipe do site precisa fazer uma alteração para apresentar a informação sobre gênero musical, enquanto a equipe do aplicativo móvel trabalha para permitir que os usuários naveguem, ouçam previamente e comprem os ringtones. As duas mudanças devem ser feitas no microsserviço *catálogo*, mas, infelizmente, metade da equipe está ocupada diagnosticando uma falha no ambiente de produção, com a outra metade ausente por causa de uma intoxicação alimentar, depois que a equipe saiu para comer em um food truck que funcionava em um beco.

Temos algumas opções que poderíamos considerar a fim de evitar a necessidade de as equipes do site e do aplicativo móvel compartilharem o microsserviço *catálogo*. A primeira é simplesmente esperar. As equipes do

site e do aplicativo móvel passam para outras tarefas. Dependendo da importância da funcionalidade ou de quanto tempo demorará, isso talvez não seja um problema – ou poderia ser um problema sério.

Como alternativa, poderíamos acrescentar pessoas na equipe de catálogo para ajudá-la a fazer mais rapidamente o seu trabalho. Quanto mais padronizado for o conjunto de tecnologias e os padrões (idioms) de programação em uso em seu sistema, mais fácil será para que outras pessoas façam alterações em seus serviços. O outro lado da moeda, é claro, conforme já discutimos, é que a padronização tende a reduzir a capacidade da equipe de adotar a solução apropriada para o trabalho, e pode levar a diferentes tipos de ineficiências.

Outra opção que evitaria a necessidade de um microsserviço catálogo compartilhado poderia ser dividir o catálogo em um catálogo musical genérico e um catálogo de ringtones. Se a mudança sendo feita para aceitar ringtones for relativamente pequena, e as chances de haver um intenso desenvolvimento nessa área no futuro também forem muito baixas, essa poderia ser uma decisão prematura. Por outro lado, se houver dez semanas de funcionalidades relacionadas a ringtones à espera, decompor o serviço poderia fazer sentido, com a equipe do aplicativo móvel assumindo a responsabilidade.

Há, entretanto, outros modelos diferentes que poderíamos considerar. Em breve, veremos o que pode ser feito para deixar o microsserviço compartilhado mais “plugável”, permitindo que outras equipes contribuam com código por meio de bibliotecas ou o estendam a partir de um framework comum. Antes, porém, devemos explorar o potencial de trazer algumas ideias do mundo do desenvolvimento open source para a nossa empresa.

Open source interno

Muitas empresas têm decidido implementar alguma forma de open source interno, tanto para ajudar a gerenciar o problema de bases de código compartilhadas, como para facilitar que pessoas externas a uma equipe contribuam com alterações em um microsserviço do qual elas possam fazer

uso.

No open source usual, um pequeno grupo de pessoas é considerada core committers (committers principais). Elas são as guardiãs do código. Se você quiser que uma mudança seja feita em um projeto open source, terá de solicitar que um dos committers faça a alteração para você, ou você mesmo fará a alteração e enviará um pull request. Os core committers continuam responsáveis pela base de código; eles são os seus donos.

Em uma empresa, esse padrão também pode funcionar bem. Talvez as pessoas que trabalhavam originalmente no serviço já não estejam mais juntas em uma equipe; talvez elas estejam agora espalhadas pela empresa. Se elas ainda tiverem direitos de commit, você poderá encontrá-las e solicitar a sua ajuda, talvez trabalhando junto com elas; ou, se você tiver as ferramentas apropriadas, poderá enviar um pull request.

Papel dos core committers

Queremos que nossos serviços sejam razoáveis. Queremos que o código tenha uma qualidade decente e que o microsserviço em questão exiba algum tipo de consistência quanto ao modo como é criado. Também queremos garantir que as mudanças feitas agora não deixem as alterações planejadas para o futuro muito mais difíceis do que deveriam ser. Isso significa que também precisamos adotar internamente os mesmos padrões empregados no open source usual, o que significa separar nossos committers confiáveis (a equipe principal) dos committers não confiáveis (pessoas fora da equipe, que submetem alterações).

A equipe principal precisa ter alguma maneira de avaliar e aprovar as mudanças. Precisa garantir que as mudanças sejam consistentes do ponto de vista idiomático – isto é, que obedeçam às diretrizes gerais de escrita de código usadas em outras partes da base de código. As pessoas que fazem a avaliação, portanto, terão de dedicar tempo trabalhando com quem faz as submissões para alterações a fim de garantir que elas tenham uma qualidade aceitável.

Bons guardiões investem bastante esforço nessa tarefa, comunicando-se claramente com quem faz as submissões e incentivando um bom

comportamento. Guardiões ruins podem usar isso como uma desculpa para exercer poder sobre outros, ou para travar guerras religiosas sobre decisões técnicas arbitrárias. Tendo visto os dois tipos de comportamento, posso dizer que um fato é evidente: qualquer das opções exige tempo. Ao considerar permitir que committers não confiáveis submetam alterações à sua base de código, é necessário decidir se vale a pena ter o overhead de ser um guardião: a equipe principal poderia estar fazendo algo melhor com o tempo que gasta avaliando as alterações?

Maturidade

Quanto menos estável ou maduro for um serviço, mais difícil será permitir que pessoas externas à equipe principal submetam correções. Antes que a espinha dorsal de um serviço esteja definida, a equipe talvez não saiba o que é “bom” e, desse modo, pode ter dificuldade para saber como seria uma boa requisição. Durante essa fase, o próprio serviço passa por muitas mudanças.

A maioria dos projetos open source não aceitará submissões de um grupo mais amplo de committers não confiáveis, até que a base da primeira versão esteja pronta. Adotar um modelo semelhante em sua própria empresa faz sentido. Se um serviço estiver bastante maduro e raramente for alterado – por exemplo, nosso serviço de carrinho de compras – talvez seja hora de abri-lo para aceitar outras contribuições.

Ferramentas

Para uma melhor aceitação de um modelo de open source interno, será necessário ter algumas ferramentas à disposição. O uso de uma ferramenta distribuída para controle de versões, que permita que as pessoas submetam pull requests (ou algo semelhante), é importante. Dependendo do tamanho da empresa, talvez você precise também de ferramentas que permitam fazer uma discussão sobre as requisições de alterações e acompanhar a sua evolução; isso pode ou não implicar um sistema completo de revisão de código, porém a capacidade de fazer comentários inline nas alterações será muito conveniente. Por fim, será preciso facilitar bastante que um committer construa e implante seu software e o disponibilize para outras

pessoas. Em geral, isso envolve ter pipelines bem definidos para construção e implantação, além de repositórios centralizados para os artefatos. Quanto mais padronizado for o seu conjunto de tecnologias, mais fácil será para as pessoas de outras equipes editarem e contribuírem com alterações para um microsserviço.

Microsserviços modulares e plugáveis

Na FinanceCo, vi um desafio interessante associado a um microsserviço em particular, que estava se tornando um gargalo para muitas equipes. Para cada país, a FinanceCo tinha equipes dedicadas, com foco em funcionalidades específicas do país em questão. Essa opção fazia muito sentido, pois cada país tinha requisitos e desafios específicos. No entanto, isso gerava problemas para esse serviço central, pois era preciso atualizar aí as funcionalidades específicas para cada país. A equipe responsável pelo microsserviço central estava sobrecarregada com pull requests enviados a ela. Essa equipe estava fazendo um ótimo trabalho para processar rapidamente esses pull requests e, com efeito, mantinha o foco nessa tarefa como uma parte essencial de suas responsabilidades; todavia, estruturalmente, a situação não era realmente sustentável.

Esse é um exemplo no qual uma equipe com um número elevado de pull requests pode ser um sinal de vários problemas em potencial. Os pull requests de outras equipes estão sendo levados a sério? Ou esses pull requests são um sinal de que a responsabilidade pelo microsserviço deveria possivelmente ser passada para outros?



Se uma equipe recebe vários pull requests, esse poderia ser um sinal de que há realmente um microsserviço compartilhado por várias equipes.

Mudança de responsabilidade

Às vezes, a decisão certa a ser tomada é mudar o responsável por um microsserviço. Considere um exemplo na MusicCorp. A equipe de Engajamento de Clientes precisa enviar vários pull requests relacionados ao microsserviço *Recomendações* à equipe de Marketing e Promoções. Isso ocorre

porque várias mudanças estão ocorrendo, relacionadas ao modo como as informações dos clientes são gerenciadas, e porque precisamos também que essas recomendações sejam apresentadas de diferentes maneiras.

Nessa situação, talvez faça sentido que a equipe de Engajamento de Clientes simplesmente assuma a responsabilidade pelo microserviço *Recomendações*. No exemplo da FinanceCo, porém, essa opção não existia. O problema era que os pull requests eram originados por *várias* equipes distintas. Então, o que mais poderia ser feito?

Executar diversas variantes

Uma opção que exploramos foi que cada equipe associada a um país executasse a sua própria variante do microserviço compartilhado. Assim, a equipe dos Estados Unidos executaria sua versão, a equipe de Singapura executaria a sua própria variante, e assim sucessivamente.

É claro que o problema com essa abordagem está na duplicação de código. O microserviço compartilhado implementava um conjunto de comportamentos padrões e regras comuns, mas também era desejado que algumas das funcionalidades fossem diferentes para cada país. Não queríamos duplicar funcionalidades comuns. A ideia era que a equipe que até então gerenciava o microserviço compartilhado disponibilizasse um framework, que fosse composto do microserviço existente contendo somente as funcionalidades comuns. As equipes específicas de cada país podiam iniciar sua própria instância do microserviço esqueleto, plugando as próprias funcionalidades personalizadas, como mostra a Figura 15.4.

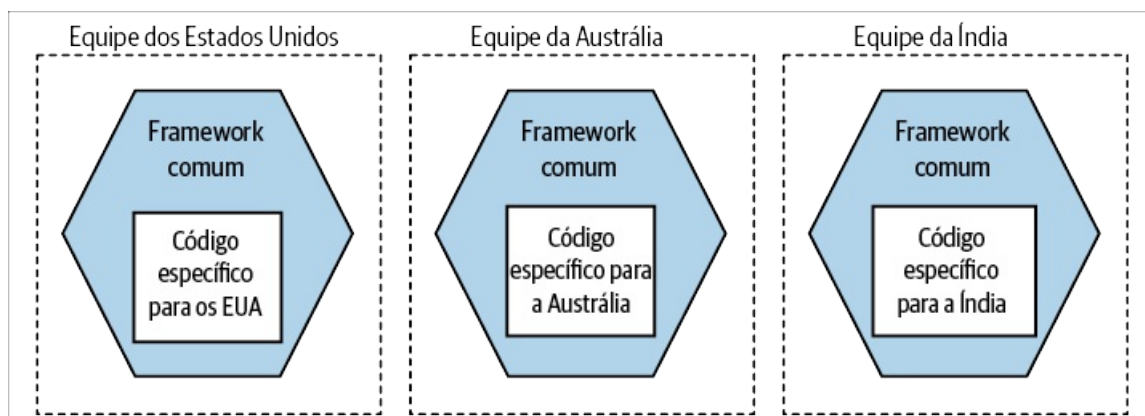


Figura 15.4 – Um framework comum poderia permitir que diversas variantes

do mesmo microsserviço fossem executadas por diferentes equipes.

O ponto importante a ser observado nesse caso é que, embora possamos compartilhar, nesse exemplo, as funcionalidades comuns entre as variantes do microsserviço específicas para cada país, essas funcionalidades comuns não podem ser atualizadas em todas as variantes do microsserviço ao mesmo tempo, sem exigir um lançamento sincronizado em larga escala. A equipe principal, que gerencia o framework, poderia disponibilizar uma nova versão, mas caberia a cada equipe incluir a versão mais recente do código comum e reimplantá-la. Nessa situação em particular, essa restrição não era um problema para a FinanceCo.

Vale a pena ressaltar que essa situação específica era bastante rara, e foi algo com o qual eu havia deparado apenas uma ou duas vezes antes. Meu foco inicial estava em encontrar maneiras de dividir as responsabilidades por esse microsserviço central compartilhado, ou então fazer uma nova atribuição das responsabilidades. Minha preocupação era o fato de que criar frameworks internos pudesse ser uma atividade complicada. É fácil demais que o framework se torne excessivamente inflado, ou que limite o desenvolvimento das equipes que o utilizam. Esse é o tipo de problema que não se manifesta logo no primeiro dia. Ao criar um framework interno, tudo começa com a melhor das intenções. No caso da FinanceCo, embora achasse que fosse o caminho certo, eu faria uma advertência quanto à adoção imediata dessa abordagem, a menos que você tenha esgotada suas outras opções.

Contribuições externas por meio de bibliotecas

Uma variação dessa abordagem seria fazer com que cada equipe específica de um país contribuísse com uma biblioteca contendo as funcionalidades específicas para o país em questão, e então fazer com que essas bibliotecas fossem empacotadas em um único microsserviço compartilhado, como mostra a Figura 15.5.

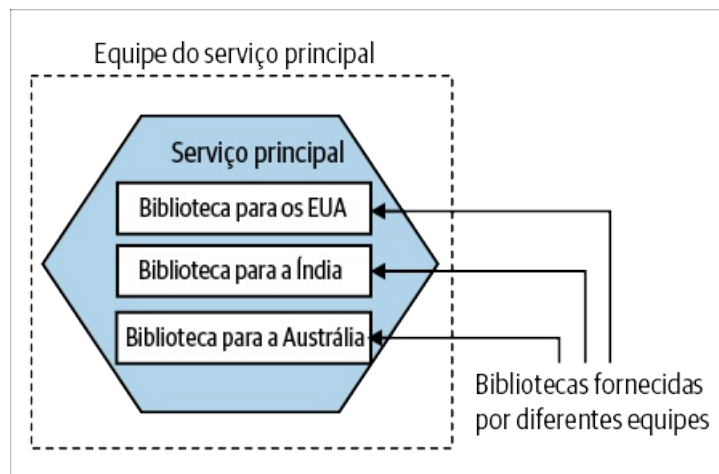


Figura 15.5 – As equipes contribuem com bibliotecas contendo seus comportamentos personalizados, a serem incluídas em um microserviço central.

A ideia, nesse caso, é que, se a equipe dos Estados Unidos precisar implementar uma lógica específica para esse país, ela faria uma alteração em uma biblioteca que seria então incluída como parte da construção do microserviço central.

Essa abordagem reduz a necessidade de executar microserviços adicionais. Não será necessário executar um serviço por país – podemos executar um único microserviço central, que cuidará das funcionalidades personalizadas de *cada* país. O desafio, nesse caso, é que as equipes de cada país não são responsáveis por decidir quando suas funcionalidades personalizadas serão lançadas. Elas podem fazer a alteração e solicitar que essa mudança seja implantada, mas a equipe central teria de agendar essa implantação.

Além do mais, é possível que um bug em uma dessas bibliotecas específicas de cada país pudesse causar um problema em produção, o qual a equipe central seria então a responsável por resolver. Como resultado, a resolução de problemas no ambiente de produção poderia ficar mais complexa.

Apesar disso, talvez valesse a pena considerar essa opção caso ela ajude você a se afastar de um microserviço central com um modelo de responsabilidade coletiva, sobretudo quando não for possível justificar a execução de diversas variantes do mesmo microserviço.

Revisões de alterações

Ao adotar uma abordagem de open source interno, o conceito de revisão é um princípio fundamental – a alteração deve ser revisada antes de ser aceita. Contudo, mesmo trabalhando em uma equipe com uma base de código na qual você tenha permissões de commits diretos, será importante que as alterações sejam revisadas.

Sou grande fã de ter minhas alterações revisadas. Sempre achei que meu código se beneficia com um segundo par de olhos. De longe, minha forma preferida de revisão é a revisão imediata que ocorre como parte da programação aos pares (pair programming). Você e outro desenvolvedor escrevem o código juntos e discutem as alterações entre si. Elas são revisadas antes do check in.

Você não precisa acreditar em minhas palavras. Voltando ao livro *Accelerate*, que já mencionei diversas vezes até agora:

Constatamos que não havia uma correlação entre aprovações somente para as alterações de alto risco e o desempenho na entrega de softwares. As equipes que informaram que não tinham um processo de aprovação ou faziam uso de revisão por pares (peer review) tiveram um desempenho melhor na entrega de software. Por fim, as equipes que exigiam aprovações de alguém externo tiveram um desempenho pior.

Nesse caso, vemos uma distinção feita entre revisão por pares e revisão de alterações externa. Uma revisão de alteração por pares é feita por alguém que muito provavelmente estará na mesma equipe que você, e que trabalha na mesma base de código que você. Essas pessoas, obviamente, estarão mais bem posicionadas para avaliar o que seria uma boa alteração, além de provavelmente fazerem a revisão com mais rapidez (mais sobre esse assunto em breve). Uma revisão externa, porém, é sempre mais complicada. Como o indivíduo não pertence à sua equipe, essa pessoa provavelmente avaliará a alteração com base em uma lista de critérios que pode ou não fazer sentido, e por estar em uma equipe diferente, ela poderá demorar para dar atenção às suas alterações. Conforme observam os autores de *Accelerate*:

Quais são as chances de um indivíduo externo, não intimamente

familiarizado com o funcionamento interno de um sistema, poder revisar alterações em dezenas de milhares de linhas de código, feitas por centenas de engenheiros em potencial, e determinar com exatidão o impacto em um sistema de produção complexo?

Assim, de modo geral, devemos fazer uso de revisões de alterações por pares e evitar a necessidade de revisões de código externas.

Revisões de código síncronas versus assíncronas

Na programação por pares, a revisão de código ocorre inline, quando o código é escrito. Na verdade, é mais do que isso. Quando programamos aos pares, temos o driver (a pessoa que está no teclado) e o navegador (que atua como um segundo par de olhos). Ambos os participantes mantêm um diálogo constante sobre as mudanças que fazem – o ato da revisão e o ato de fazer as alterações ocorrem simultaneamente. A revisão se transforma em um aspecto implícito e contínuo do relacionamento de pares. Isso significa que, à medida que os problemas são identificados, eles são imediatamente corrigidos.

Se você não trabalha com um par, o ideal é que a revisão ocorra bem rápido, logo após a escrita do código. Você iria querer então que a revisão em questão fosse o mais síncrono possível. Você quer ser capaz de discutir qualquer problema que houver diretamente com o revisor, chegar a um acordo acerca de como proceder, fazer as alterações e seguir em frente.

Quanto mais rápido você receber um feedback sobre uma alteração de código, mais rápido poderá olhar o feedback, avaliá-lo, pedir esclarecimentos, discutir melhor o problema se for necessário e, em última instância, fazer quaisquer mudanças necessárias. Quanto mais demorar entre a submissão da mudança de código para revisão e a revisão propriamente dita ocorrer, mais demorado e difícil será o trabalho.

Se você submeter uma mudança de código para revisão e não obtiver feedback sobre essa alteração até que vários dias tenham se passado, é provável que você tenha prosseguido para outras tarefas. Para processar o feedback, será preciso uma mudança de contexto e um novo envolvimento com o trabalho que você havia feito antes. Você poderia concordar com as

mudanças do revisor (se forem necessárias), caso em que poderia implementá-las e submeter novamente a alteração para aprovação. No pior caso, poderia ser necessário haver novas discussões sobre os pontos que foram levantados. Esse processo assíncrono de ida e volta entre quem submete a alteração e o revisor pode acrescentar *dias* no processo de fazer as alterações.



Faça as revisões de código imediatamente!

Se quiser fazer revisões de código, mas você não faz uso da programação aos pares, faça as revisões o mais rápido possível após a mudança ser submetida, e avalie o feedback do modo mais síncrono possível – o ideal é que seja pessoalmente com o revisor.

Programação em conjunto

A programação em conjunto (ensemble programming) – também conhecida como programação mob (mob programming)¹⁵ – às vezes é discutida como uma forma de fazer revisões de código inline. Na programação em conjunto, um grupo maior de pessoas (talvez a equipe toda) trabalha junto em uma alteração. Trata-se principalmente de trabalhar coletivamente em um problema, recebendo contribuições de vários indivíduos.

Entre as equipes com quem conversei e que fazem uso da programação em conjunto, a maioria a utiliza apenas ocasionalmente, para problemas específicos e intrincados ou alterações importantes, mas muitos trabalhos de desenvolvimento também ocorrem externamente ao exercício em conjunto. Assim, embora seja provável que o exercício de programação em conjunto possibilite revisões suficientes para as mudanças feitas durante o próprio exercício, e de modo bastante síncrono, ainda seria necessário ter uma maneira de garantir que houvesse uma revisão das alterações feitas fora do exercício.

Algumas pessoas argumentariam que bastaria fazer uma revisão das alterações de alto risco, e, desse modo, fazer revisões somente como parte da programação em conjunto seria suficiente. Vale a pena observar que os autores do livro *Accelerate* surpreendentemente não encontraram *nenhuma*

correlação entre o desempenho na entrega de software e a revisão apenas das alterações de alto risco, em comparação com uma correlação positiva quando todas as alterações são revisadas por pares. Desse modo, se você realmente quiser fazer uso da programação em conjunto, vá em frente! No entanto, considere a revisão de outras alterações feitas fora da programação em conjunto também.

Pessoalmente, tenho sérias reservas quanto a alguns aspectos da programação em conjunto. Você perceberá que, com efeito, sua equipe é composta de personalidades distintas, e o desequilíbrio de poder na programação em conjunto poderia prejudicar o objetivo de resolver o problema de forma coletiva. Nem todos se sentem confortáveis em trabalhar em um grupo, e a programação em conjunto definitivamente é isso. Algumas pessoas florescerão em um ambiente como esse, enquanto outras se sentirão totalmente incapazes de contribuir. Quando levantei essa questão com alguns proponentes da programação em conjunto, recebi diversas respostas, e muitas delas se reduziam à crença de que, se você criar o ambiente apropriado para uma programação em conjunto, qualquer um será capaz de “sair de sua concha e contribuir”. Vamos apenas dizer que, depois dessas discussões em particular, revirei tanto os meus olhos que quase fiquei cego. Para ser justo, essas mesmas preocupações podem ser levantadas a respeito da programação aos pares também!

Embora eu não tenha nenhuma dúvida de que muitos dos proponentes da programação em conjunto não seriam tão alienados ou ignorantes a esse ponto, é importante lembrar que criar um ambiente de trabalho inclusivo, em parte, diz respeito a saber como criar um ambiente no qual todos os membros da equipe possam dar total contribuição, de um modo que seja seguro e confortável para eles. E não se deixe enganar acreditando que só porque você conseguiu pôr todo mundo em uma sala, todos estão realmente contribuindo. Se quiser algumas dicas concretas sobre programação em conjunto, eu sugeriria a leitura do livro conciso *Ensemble Programming Guidebook*, de Maaret Pyhäjärvi, publicado de forma independente.¹⁶

Serviço órfão

O que dizer dos serviços que não estão mais em manutenção ativa? À medida que passamos para arquiteturas com partes menores e especializadas, os próprios microsserviços se tornam menores. Uma das vantagens de ter microsserviços menores, conforme já discutimos, é o fato de eles serem mais simples. Microsserviços mais simples, com menos funcionalidades, podem não precisar de alterações por um tempo. Considere o humilde microsserviço `Carrinho de Compras`, que oferece algumas funcionalidades razoavelmente modestas: adicionar no carrinho, remover do carrinho, e assim por diante. É bastante concebível que esse microsserviço não precise de alterações por meses após ter sido inicialmente escrito, ainda que um desenvolvimento ativo continue em andamento. O que acontece nesse caso? Quem é o responsável por esse microsserviço?

Se as estruturas de suas equipes estiverem alinhadas aos contextos delimitados de sua empresa, até mesmo os serviços que não são alterados com frequência ainda terão um responsável na prática. Pense em uma equipe alinhada com o contexto de vendas para clientes na web. Ela poderia lidar com a interface de usuário web e com os microsserviços `Carrinho de Compras` e `Recomendações`. Mesmo que o serviço de carrinho de compras não tenha mudado em meses, seria natural que as alterações ficassem a cargo dessa equipe caso fossem necessárias. Uma das vantagens dos microsserviços, é claro, é que, se a equipe precisar modificar o microsserviço a fim de acrescentar uma nova funcionalidade, mas não ficar satisfeita com ela, reescrevê-la não deverá ser uma tarefa muito demorada.

Apesar disso, se você adotou uma abordagem verdadeiramente poliglota e faz uso de vários conjuntos de tecnologias, os desafios de fazer alterações em um serviço órfão poderiam ser agravados caso sua equipe já não conheça mais o conjunto de tecnologias.

Estudo de caso: realestate.com.au

Na primeira edição deste livro, passei um tempo conversando com a

realestate.com.au (REA) sobre o seu uso de microsserviços, e muito do que aprendi ajudou bastante para apresentar exemplos de microsserviços em ação no mundo real. Também achei a interação entre a estrutura organizacional e a arquitetura na REA particularmente fascinante. Essa visão geral de sua estrutura organizacional tem como base as nossas discussões feitas em 2014.

Tenho certeza de que a REA está bem diferente hoje em dia. Essa visão geral representa uma imagem instantânea obtida em um ponto no tempo. Não estou sugerindo que essa seja a melhor maneira de estruturar uma empresa – apenas que era o que funcionava melhor para a REA naquela época. Aprender com outras empresas é uma atitude sensata; copiar o que elas fazem sem entender por que elas fazem é tolice.

Assim como acontece nos dias de hoje, os negócios principais da REA no mercado imobiliário abrangiam diferentes aspectos. Em 2014, a REA estava dividida em LOBs (Lines Of Business, ou Linhas de Negócios) independentes. Por exemplo, uma linha de negócios lidava com propriedades residenciais na Austrália, outra com propriedades comerciais, e outra linha cuidava de um dos negócios da REA no estrangeiro. Essas linhas de negócios tinham equipes de TI para entregas (ou “squads”, isto é, esquadrões) associadas a elas; o único detalhe é que algumas tinham um único squad, enquanto a linha principal possuía quatro. Assim, para as propriedades residenciais, havia várias equipes envolvidas na criação do site e na apresentação dos serviços que permitiam que as pessoas navegassem pelas propriedades. Havia uma rotatividade ocasional de pessoas entre essas equipes, mas elas tendiam a permanecer na mesma linha de negócios específica por longos períodos, garantindo que os membros da equipe pudessem adquirir um conhecimento sólido sobre aquela parte do domínio. Isso, por sua vez, ajudava na comunicação entre os vários stakeholders de negócios e a equipe que entregava as funcionalidades para eles.

Era esperado que cada squad em uma linha de negócios fosse responsável pelo ciclo de vida completo de cada serviço que criava, incluindo construir, testar e lançar, dar suporte, e até mesmo aposentar o serviço. A equipe

principal de serviços para entrega tinha como tarefa oferecer conselhos, orientações e ferramentas aos squads nas LOBs, ajudando-os a fazer suas entregas com mais eficácia. Utilizando nossa mais nova terminologia, a equipe principal de serviços para entrega estava desempenhando o papel de equipe de capacitação (enabling team). Uma sólida cultura de automação era essencial, e a REA fazia uso intensivo da AWS como uma parte importante para possibilitar que as equipes se tornassem mais autônomas. A Figura 15.6 mostra como tudo isso funcionava.

Não era apenas a organização para entregas que estava alinhada ao modo de operação dos negócios. Esse modelo se estendia também à arquitetura. Um exemplo disso eram os métodos de integração. Em uma LOB, todos os serviços eram livres para conversar uns com os outros da maneira que considerassem mais apropriada, conforme decidido pelos squads que atuavam como seus guardiões. No entanto, era obrigatório que a comunicação entre as LOBs fosse assíncrona em lote (batch), uma das poucas regras rígidas da pequena equipe de arquitetura. Essa comunicação em um nível mais alto correspondia àquela que havia entre as diferentes partes dos negócios também. Ao insistir que ela fosse feita em lote, cada LOB tinha bastante liberdade para atuar e se gerenciar. Ela podia se dar ao luxo de desativar seus serviços sempre que quisesse, sabendo que, desde que pudesse obedecer ao requisito de integração em lote com as outras partes dos negócios e satisfazer os seus próprios stakeholders de negócios, ninguém se importaria.

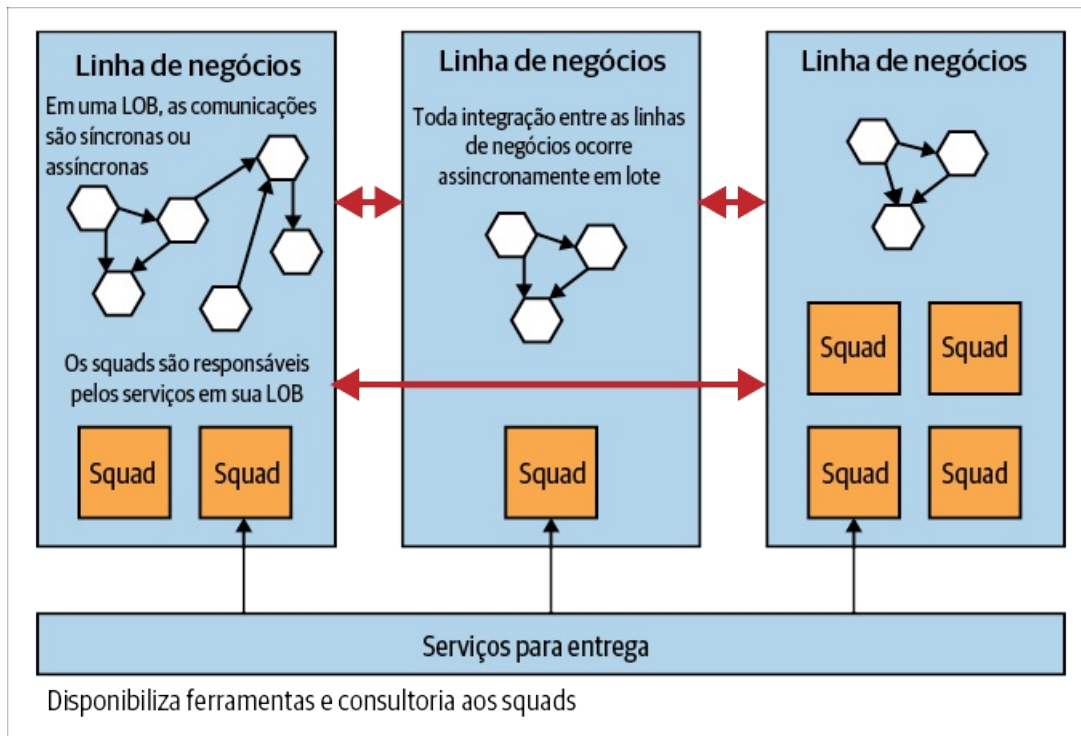


Figura 15.6 – Uma visão geral da estrutura organizacional e da estrutura de equipes da realestate.com.au, e seu alinhamento com a arquitetura.

Essa estrutura permitia que houvesse uma autonomia significativa não só entre as equipes, mas também entre as diferentes partes da empresa, e a capacidade de lançar mudanças ajudou a empresa a conquistar um sucesso relevante no mercado local. Essa estrutura mais autônoma também contribuiu para o crescimento da empresa, que passou de um punhado de serviços em 2010 para centenas por volta de 2014, aumentando sua capacidade de fazer mudanças com mais rapidez.

As empresas que são capazes de se adaptar de modo a mudar não só a arquitetura de seus sistemas, mas também a sua estrutura organizacional, podem colher muitos benefícios no que concerne a ter equipes com mais autonomia e alcançar o mercado com novos produtos e funcionalidades com mais rapidez. A REA é apenas uma entre diversas empresas que perceberam que uma arquitetura de sistemas não existe no vácuo.

Distribuição geográfica

Equipes que estão no mesmo lugar acharão a comunicação síncrona muito

simples, sobretudo porque, em geral, as pessoas estarão no mesmo lugar ao mesmo tempo. Se sua equipe estiver distribuída, uma comunicação síncrona poderá ser mais difícil, mas ainda será viável se os membros da equipe estiverem no mesmo fuso horário ou em fusos próximos. Ao se comunicar com pessoas em diversos fusos horários distintos, o custo da coordenação poderá aumentar drasticamente. Em um de meus cargos anteriores, trabalhei como arquiteto, ajudando a dar suporte para equipes que estavam na Índia, no Reino Unido, no Brasil e nos Estados Unidos, enquanto eu mesmo estava na Austrália. Marcar uma reunião comigo e com os líderes das diversas equipes era extremamente difícil. Isso significava que essas reuniões ocorriam raramente (em geral, uma vez por mês), e ainda tínhamos de garantir que discutiríamos somente as questões mais importantes nessas reuniões, pois, com frequência, mais da metade dos participantes estaria trabalhando fora de seus horários comerciais.

Afora essas reuniões, tínhamos uma comunicação assíncrona, em sua maior parte por email, para discutir outras questões que não fossem tão urgentes. No entanto, pelo fato de eu estar na Austrália, a demora nessa forma de comunicação era significativa. Eu acordava em uma segunda-feira de manhã para um início de semana bem tranquilo, pois a maior parte do mundo ainda não tinha acordado – isso me dava tempo para processar os emails que eu havia recebido das equipes do Reino Unido, do Brasil e dos Estados Unidos, enviados por elas na sexta-feira à tarde, e que, para mim, já era sábado de manhã.

Eu me lembro do projeto de um cliente no qual trabalhei, em que a responsabilidade por um único microsserviço era compartilhada entre duas localidades geográficas distintas. Com o tempo, cada localidade começou a se especializar no trabalho que fazia. Isso permitiu que cada uma assumisse a responsabilidade por uma parte da base de código, para a qual o custo de fazer uma alteração seria menor. As equipes tinham então uma comunicação de nível mais genérico acerca de como as duas partes se inter-relacionavam; efetivamente, as vias de comunicação que se tornaram possíveis na estrutura organizacional correspondiam à API de nível mais alto que servia de fronteira entre as duas metades da base de código.

Então, em que situação isso nos deixa quando consideramos a evolução no design de nosso serviço? Bem, eu sugeriria que fronteiras geográficas entre pessoas envolvidas no desenvolvimento deveriam ser uma consideração relevante quando procuramos definir as fronteiras tanto das equipes quanto do software. É muito mais fácil criar uma única equipe quando seus membros estão no mesmo lugar. Se não for possível que estejam no mesmo lugar e você pretende criar uma equipe distribuída, garantir que os membros da equipe estejam no mesmo fuso horário ou em fusos muito próximos ajudará na comunicação interna dessa equipe, pois reduzirá a necessidade de uma comunicação assíncrona.

Talvez sua empresa decida que quer aumentar o número de pessoas que trabalha em seu projeto, abrindo um escritório em outro país. A essa altura, você deve pensar realmente nas partes de seu sistema que poderão ser transferidas para lá. Talvez isso seja o aspecto que orientará suas decisões sobre as funcionalidades a serem separadas na sequência.

A essa altura, também vale a pena observar que, ao menos com base nas observações dos autores do relatório “Exploring the Duality Between Product and Organizational Architectures” (Explorando a dualidade entre a arquitetura de um produto e a arquitetura organizacional) que mencionei antes, se a empresa que está criando o sistema tiver menos acoplamento (por exemplo, é composta de equipes geograficamente distribuídas), os sistemas criados tenderão a ser mais modulares e, desse modo, esperamos que sejam menos acoplados. A tendência de uma única equipe responsável por vários serviços fazer uma integração mais intensa será muito difícil de ser mantida em uma empresa mais distribuída.

Lei de Conway ao inverso

Até agora falamos de como a estrutura organizacional exerce impacto no design dos sistemas. Mas e o contrário? Ou seja, o design de um sistema pode mudar a organização da empresa? Embora eu não tenha conseguido encontrar evidências de mesma qualidade para sustentar a ideia de que a lei de Conway funciona ao inverso, já vi essa situação por experiência própria.

Provavelmente, o melhor exemplo é um cliente com quem trabalhei há

muitos anos. Lá atrás, na época em que a web ainda estava nascendo e a internet era vista como algo que chegava em sua porta em um disquete da AOL, essa empresa era uma grande firma de impressão gráfica, com um site pequeno e modesto. Ela tinha um site, pois era o que se devia fazer, mas no grande esquema das coisas, o site tinha pouca importância para o modo de operação da empresa. Quando o sistema original foi criado, uma decisão técnica razoavelmente arbitrária havia sido tomada acerca de como o sistema funcionaria.

O conteúdo para esse sistema era gerado de várias maneiras, mas a maior parte era proveniente de terceiros que colocavam anúncios a serem visualizados pelo público em geral. Havia um sistema de entrada que permitia que conteúdos fossem criados por terceiros pagantes, um sistema central que recebia esses dados, enriquecendo-os de várias maneiras, e um sistema de saída que criava o site final que o público em geral podia acessar.

O fato de as decisões originais sobre o design terem sido corretas à época é uma discussão para os historiadores, mas, depois de muitos anos, a empresa havia mudado consideravelmente, e eu e muitos dos meus colegas estávamos começando a pensar se o design do sistema era adequado ao estado atual da empresa. Os negócios na área de impressão física haviam diminuído significativamente, e o faturamento e, portanto, as operações da empresa, eram agora dominados pela sua presença online.

O que víamos na época era uma empresa rigidamente alinhada a esse sistema de três partes. Três canais ou divisões no lado de TI dos negócios estavam alinhados a cada uma das partes – entrada, parte principal e saída – dos negócios. Em cada um desses canais, havia equipes de entrega separadas. O que eu não havia percebido na época é que essas estruturas organizacionais não eram anteriores ao design do sistema, mas, na verdade, haviam crescido em torno dele. À medida que a parte de impressão gráfica dos negócios diminuía e o lado digital dos negócios aumentava, o design do sistema inadvertidamente determinou o caminho para o modo como a empresa cresceu.

No final, percebemos que, quaisquer que fossem as deficiências do design

do sistema, teríamos de fazer mudanças na estrutura organizacional para fazer uma transição. A empresa atualmente está bastante mudada, mas isso ocorreu em um período de muitos anos.

Pessoas

Não importa o que pareça à primeira vista, é sempre um problema de pessoas.

– *Gerry Weinberg, Segunda Lei da Consultoria*

Temos de aceitar que, em um ambiente de microsserviço, é mais difícil para um desenvolvedor pensar na escrita de código em seu próprio mundo pequeno. Os desenvolvedores precisam estar mais cientes das implicações de certos aspectos como chamadas que cruzam fronteiras de rede, ou das implicações de uma falha. Também falamos da capacidade dos microsserviços de facilitar os experimentos com novas tecnologias, de bancos de dados a outras linguagens. Contudo, se você está migrando de um mundo no qual havia um sistema monolítico, no qual a maioria de seus desenvolvedores precisavam usar somente uma linguagem e permaneciam totalmente despreocupados com as questões operacionais, jogá-los no mundo dos microsserviços poderia ser uma forma rude de despertá-los.

De modo semelhante, dar mais poder às equipes de desenvolvimento para aumentar sua autonomia pode ser complicado. As pessoas que, no passado, jogavam tarefas por cima do muro para outras pessoas, estão acostumadas a ter outras pessoas a quem culpar, e podem não se sentir à vontade por serem totalmente responsabilizadas pelo seu trabalho. Você poderia até deparar com barreiras contratuais para fazer com que seus desenvolvedores portassem paggers para os sistemas aos quais darão suporte! Essas mudanças podem ser feitas gradualmente e, no começo, fará sentido procurar dar mais responsabilidades às pessoas que estejam mais dispostas e sejam capazes de fazer a mudança.

Embora este livro, em sua maior parte, seja a respeito de tecnologias, as pessoas não são apenas uma consideração secundária; elas são as pessoas que construíram o que você tem agora, e construirão o que virá em

seguida. Definir uma visão acerca de como o trabalho deve ser feito sem considerar como o seu quadro atual de funcionários se sentirá a respeito disso e sem considerar as habilidades que eles têm provavelmente levará a resultados negativos.

Cada empresa tem o próprio conjunto de dinâmicas relacionadas a esse assunto. Entenda o apetite de seus funcionários por mudanças. Não os pressione prematuramente! Talvez você possa ter uma equipe separada para lidar com a linha de frente no suporte ou com as implantações por um curto período, dando tempo para que seus desenvolvedores se ajustem às novas práticas. Contudo, talvez você tenha de aceitar que precisará de diferentes tipos de pessoas em sua empresa para fazer com que tudo isso funcione. É provável que seja necessário mudar o modo de contratar as pessoas – com efeito, poderia ser mais fácil mostrar o que é possível fazer trazendo novas pessoas de fora, que já tenham experiência em trabalhar da forma como você deseja. Se contratar as pessoas certas, isso poderá ser muito útil para mostrar aos outros o que é possível.

Independentemente da sua abordagem, saiba que você precisa ser claro ao explicar as responsabilidades de seus funcionários em um mundo de microsserviços, e os motivos pelos quais essas responsabilidades são importantes. Isso pode ajudar você a perceber quais são as lacunas que existem quanto às habilidades e pensar em como preenchê-las. Para muitos, essa será uma jornada bastante assustadora. Apenas se lembre de que, sem pessoas comprometidas, qualquer mudança que você queira fazer poderia estar condenada desde o princípio.

Resumo

A lei de Conway enfatiza os perigos de tentar impor um design de sistema que não corresponda à estrutura organizacional. Isso nos aponta, pelo menos no caso dos microsserviços, para um modelo no qual uma responsabilidade forte pelos microsserviços seja a norma. Compartilhar microsserviços ou tentar colocar uma responsabilidade coletiva em prática em larga escala muitas vezes pode resultar em anular as vantagens dos microsserviços.

Se a estrutura organizacional e a arquitetura não estiverem alinhadas, haverá pontos de tensão, conforme mostrado neste capítulo. Ao reconhecer a ligação entre ambas, garantimos que o sistema que estamos tentando criar faça sentido para a estrutura organizacional à qual estamos criando esse sistema.

Se quiser explorar melhor esse assunto, além do livro *Team Topologies* já mencionado, posso recomendar a palestra “Scale, Microservices and Flow” (Escala, microsserviços e fluxo), de James Lewis,¹⁷ da qual obtive muitos insights que ajudaram a moldar este capítulo. Vale muito a pena assistir a ela se você estiver interessado em explorar com mais detalhes algumas das ideias que foram discutidas neste capítulo.

Em nosso próximo capítulo, exploraremos melhor um assunto que já mencionamos antes – o papel do arquiteto.

¹ Nicole Forsgren, Jez Humble e Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

² N.T.: Esta e as demais citações deste capítulo foram traduzidas livremente, com base no texto original em inglês.

³ Alan MacCormack, Carliss Baldwin e John Rusnak, “Exploring the Duality Between Product and Organizational Architectures: A Test of the *Mirroring* Hypothesis” (Explorando a dualidade entre a arquitetura do produto e a arquitetura organizacional: um teste da hipótese de *espelhamento*), *Research Policy* 41, nº 8 (outubro de 2012): 1309–24.

⁴ Nachiappan Nagappan, Brendan Murphy e Victor Basili, “The Influence of Organizational Structure on Software Quality: An Empirical Case Study” (A influência da estrutura organizacional na qualidade do software: um caso de estudo empírico), *ICSE '08: Proceedings of the 30th International Conference on Software Engineering* (Nova York: ACM, 2008).

⁵ E todos nós sabemos que o Windows Vista era bastante suscetível a erros!

⁶ Daniel Rodriguez et al., “Empirical Findings on Team Size and Productivity in Software Development” (Descobertas empíricas sobre tamanho de equipe e produtividade no desenvolvimento de software), *Journal of Systems and Software* 85, nº 3 (2012), doi.org/10.1016/j.jss.2011.09.009.

⁷ Frederick P. Brooks Jr, *The Mythical Man-Month: Essays on Software Engineering*, Edição de aniversário (Boston: Addison-Wesley, 1995) [N.T.: Edição publicada no Brasil: *O Mítico Homem-Mês: ensaios sobre engenharia de software* (Alta Books, 2018)].

⁸ John Rossman, *Think Like Amazon: 50 1/2 Ideas to Become a Digital Leader* (Nova York: McGraw-Hill, 2019).

⁹ Matthew Skelton e Manuel Pais, *Team Topologies* (Portland, OR: IT Revolution, 2019).

¹⁰ Famosa pelo desenvolvimento do material à prova d'água Gore-Tex.

- 11 Que, como todos sabem, nem mesmo o Spotify usa atualmente.
- 12 Emily Webber, *Building Successful Communities of Practice* (San Francisco: Blurb, 2016).
- 13 Paul Ingles, “Convergence to Kubernetes” (Convergência para o Kubernetes), Medium, 18 de junho de 2018, <https://oreil.ly/Ho7kY>.
- 14 “Organizational Evolution for Accelerating Delivery of Comparison Services at Uswitch” (Evolução organizacional para acelerar a entrega de serviços de comparação no Uswitch), Team Topologies, 24 de junho de 2020, <https://oreil.ly/zoyvv>.
- 15 Embora o termo *programação mob* seja mais difundido, não gosto da imagem que ele suscita e prefiro o termo *em conjunto* (ensemble) em seu lugar, pois deixa claro que temos um conjunto de pessoas trabalhando juntas, em vez de um bando de pessoas lançando coquetéis molotov e quebrando janelas. (N.T.: o termo *mob* em inglês significa multidão, populacho). Não estou 100% certo de quem renomeou o termo, mas acho que foi Maaret Pyhäjärvi.
- 16 Maaret Pyhäjärvi, *Ensemble Programming Guidebook* (publicação independente, 2015–2020), <https://ensembleprogramming.xyz>.
- 17 James Lewis, “Scale, Microservices and Flow” (Escala, microserviços e fluxo), YOW! Conferences, 10 de fevereiro de 2020, vídeo no YouTube, 51:03, <https://oreil.ly/ON81J>.

O arquiteto em evolução

Conforme vimos até agora, os microsserviços nos dão muitas opções e, na mesma medida, exigem que muitas decisões sejam tomadas. Por exemplo, quantas tecnologias diferentes devemos usar, devemos deixar que diferentes equipes utilizem diferentes padrões (idioms) de programação, e devemos separar ou combinar um microsserviço? Como fazemos para tomar essas decisões? Com o ritmo mais acelerado de mudanças e o ambiente mais fluido que essas arquiteturas possibilitam, o papel do arquiteto também precisa mudar. Neste capítulo, apresentarei uma opinião um tanto quanto forte acerca do papel de um arquiteto, e espero lançar um ataque final à sua torre de marfim.

O que há em um nome?

Você fica usando essa palavra. Eu não acho que ela signifique o que você acha que significa.

– Inigo Montoya, de *The Princess Bride* (A Princesa Prometida)

Os arquitetos têm um trabalho importante. Eles são responsáveis por garantir que o sistema tenha uma visão técnica conjunta, uma visão que ajude a entregar o software de que os clientes precisam. Em alguns lugares, os arquitetos precisam trabalhar com uma só equipe, caso em que o papel do arquiteto e o do líder técnico muitas vezes são os mesmos. Em outros, eles podem definir a visão para todo um programa de trabalho, efetuando a coordenação com várias equipes pelo mundo, ou até mesmo na empresa como um todo. Qualquer que seja o nível em que os arquitetos atuem, sua função é difícil de ser definida com exatidão, e, apesar de ser a progressão natural da carreira dos desenvolvedores em empresas de grande porte, é também uma função que recebe mais críticas do que praticamente

qualquer outra em nossa área. Mais do que quaisquer outras funções, os arquitetos podem causar um impacto direto na qualidade dos sistemas criados, nas condições de trabalho de seus colegas e na capacidade da empresa de responder a mudanças; apesar disso, sua função parece ser muito mal compreendida. Por quê?

Nosso mercado é jovem. Às vezes, parece que esquecemos que estamos criando programas que executam naquilo que reconhecemos como computadores há apenas cerca de 75 anos. Nossa profissão não pode ser colocada em uma bela caixinha que a sociedade como um todo entenda. Não somos eletricitas, encanadores, médicos ou engenheiros. Quantas vezes você já esteve em uma festa e disse para alguém o que você faz, somente para ver que a conversava deixava de fluir? O mundo como um todo tem dificuldade para entender o que é o desenvolvimento de software – conforme já mostrei várias vezes neste livro, muitas vezes nem nós mesmos sabemos.

Assim, fazemos empréstimos de outras profissões. Nós nos denominamos “engenheiros” ou “arquitetos” de software. No entanto, não somos arquitetos nem engenheiros do modo como a sociedade entende essas profissões. Os arquitetos e engenheiros têm um rigor e uma disciplina com os quais poderíamos apenas sonhar, e sua importância para a sociedade é bem compreendida. Eu me lembro de conversar com um amigo meu um dia antes de ele ter se tornado um arquiteto certificado. “Amanhã” – ele disse –, “se eu der um conselho a você lá no bar acerca de como construir algo e estiver errado, serei responsabilizado por isso. Eu poderia ser processado, pois, aos olhos da lei, sou agora um arquiteto certificado, e devo ser responsabilizado caso faça algo errado.” A importância desses trabalhos para a sociedade implica que há qualificações necessárias que as pessoas devem ter. No Reino Unido, por exemplo, um mínimo de sete anos de estudos é necessário antes que você possa ser chamado de arquiteto. No entanto, esses trabalhos também se baseiam em um corpo de conhecimentos que remonta a milhares de anos. E os arquitetos de software? Nem tanto. E é por isso que, em parte, não dou valor a várias formas de certificação de TI, pois sabemos muito pouco sobre o que é “bom”.

Não digo isso para menosprezar o termo *engenharia de software*,¹ criado anteriormente nos anos 1960 por Margaret Hamilton, mas o termo representava uma aspiração à época, tanto quanto o seria para a realidade atual. O termo surgiu como um chamado para melhorar a qualidade do software criado, e em reconhecimento ao fato de que os projetos de software muitas vezes falhavam, mas, apesar disso, estavam sendo cada vez mais usados em áreas com propósitos vitais e de segurança máxima. Muito trabalho foi feito para melhorar a situação desde então, mas minha própria avaliação após vinte anos no mercado é que ainda temos muito que aprender sobre como fazer um bom trabalho (ou, pelo menos, um trabalho melhor).

Uma parte de nós deseja ter reconhecimento, portanto emprestamos nomes de outras profissões que já têm o reconhecimento que desejamos. No entanto, isso pode se provar problemático se emprestarmos as *práticas de trabalho* dessas profissões, sem entender a mentalidade subjacente nem levar em consideração o modo como o desenvolvimento de software difere, digamos, da engenharia civil. Nada disso deveria ser usado como argumento para dizer que não deveríamos ter mais rigor em nosso trabalho como meta – mas somente para dizer que não podemos simplesmente emprestar ideias de outras áreas e supor que funcionarão para nós. Nosso mercado é muito jovem, e o problema é que, como um mercado, temos poucos fatos absolutos com os quais concordamos.

Talvez o termo *arquiteto*, ou pelo menos a compreensão comum acerca do que os arquitetos fazem, tenha causado os maiores danos: a ideia de alguém que cria planos detalhados para que outros interpretem e que espera que esses planos sejam obedecidos; o equilíbrio da parte artista com a parte engenheiro, supervisionando a criação daquilo que muitas vezes é uma visão singular, com todos os demais pontos de vista sendo subservientes, exceto pela ocasional objeção do engenheiro de estruturas com relação às leis da física. Em nosso mercado, essa visão do arquiteto leva a algumas práticas ruins, com os arquitetos criando diagrama após diagrama, página após página de documentação, com uma visão que dará base à construção do sistema perfeito, ao mesmo tempo que deixa de levar em consideração o futuro basicamente desconhecido, desprovido

totalmente de qualquer conhecimento da dificuldade de implementar seus planos, ou se eles realmente funcionarão ou não, e muito menos se poderão mudar à medida que aprendermos mais.

No entanto, os arquitetos do ambiente de construção civil atuam em uma esfera diferente daquela dos arquitetos de software. Suas restrições são diferentes, o produto resultante é diferente. O custo de uma mudança é muito mais alto na construção civil em comparação com o custo no desenvolvimento de software. Não podemos voltar atrás depois que o concreto é despejado, mas podemos alterar um código; até mesmo a infraestrutura na qual executamos nosso código é muito mais maleável do que era no passado, graças à virtualização. Os prédios são razoavelmente fixos depois que são construídos – podem ser modificados, expandidos ou derrubados, mas os custos associados serão muito elevados. No entanto, esperamos que nosso software mude continuamente para atender às nossas necessidades.

Assim, se a arquitetura de software é diferente da arquitetura no ambiente de construção civil, talvez devêssemos ser um pouco mais claros quanto àquilo que a arquitetura de software realmente é.

O que é arquitetura de software?

Uma das definições mais famosas de arquitetura de software é proveniente de um email de Ralph Johnson: “A arquitetura diz respeito às coisas importantes. O que quer que sejam elas”.² Então, isso significa que tudo o que é importante é feito pelo arquiteto? Significa que todos os demais trabalhos feitos *não* são importantes? O problema com essa afirmação frequentemente citada é que, muitas vezes, ela é usada isoladamente, sem qualquer compreensão da resposta mais ampla em que Ralph a usou. Em primeiro lugar, está claro que ele estava falando do ponto de vista de um desenvolvedor de software. Ele prossegue dizendo:

Então, uma melhor definição seria: “Na maioria dos projetos de software de sucesso, os desenvolvedores mais experientes que trabalham nesse projeto têm um entendimento compartilhado acerca do design do sistema. Esse entendimento compartilhado é chamado de

‘arquitetura’. Ele inclui como o sistema é dividido em componentes e como os componentes interagem por meio de interfaces. Esses componentes geralmente são compostos de componentes menores, mas a arquitetura inclui somente os componentes e as interfaces que são conhecidas por todos os desenvolvedores.”

Essa seria uma melhor definição, pois deixa claro que a arquitetura é uma construção social (bem, o software também é, mas a arquitetura é mais ainda) porque ela não depende somente do software, mas da parte do software que todo o grupo concorda que é importante.³

Nesse caso, vemos Ralph usando o termo *componentes* em seu sentido mais genérico. No contexto deste livro, podemos pensar nos componentes como os nossos microsserviços, e talvez nos módulos dentro desses microsserviços.

A arquitetura de software é o formato do sistema. A arquitetura acontece, por design ou por acidente. Tomamos uma série de decisões *ad hoc* e acabamos com certos resultados – sem pensar em termos de arquitetura, acabamos, apesar disso, com uma arquitetura. Às vezes, a arquitetura pode ser o que acontece enquanto estamos ocupados fazendo outros planos.

Um arquiteto dedicado é alguém que deve ver e entender esse sistema como um todo e conhece as forças que atuam nesse sistema. Os arquitetos devem garantir que haja uma visão para a arquitetura que seja adequada ao seu propósito e seja claramente compreendida – uma visão da arquitetura que satisfaça as necessidades do sistema e de seus usuários, assim como das pessoas que trabalham no sistema em questão. Olhar para um só aspecto – por exemplo, para o lógico, mas não para o físico, para a forma, mas não para a experiência do desenvolvedor – limitará a eficácia de um arquiteto. Se você aceitar que a arquitetura diz respeito a entender o sistema, limitar o escopo daquilo com o qual você se preocupa restringirá a sua capacidade de raciocinar e de fazer mudanças.

A arquitetura pode ser invisível às pessoas que convivem com ela. Ela pode ser tão sutil que parece não estar realmente presente. Às vezes, pode ser algo que orienta e ajuda a alcançar o resultado correto. Pode ser sufocante e opressiva. Pode satisfazer sem que você sequer perceba a sua existência,

ou pode abater o seu espírito sem que houvesse qualquer intenção maldosa. Então, independentemente de a arquitetura ser algo que “diz respeito às coisas importantes”, certamente ela é *importante*.

Outra citação expressiva, mencionada com frequência para definir a arquitetura de software, é proveniente do mesmo artigo em que Martin compartilha a visão de Ralph: “Assim, você pode acabar definindo a arquitetura como *aquilo que as pessoas percebem como difíceis de mudar*.” A ideia de Martin de que a arquitetura se refere àquilo que é difícil de mudar faz sentido em certo nível, e nos leva de volta ao conceito de arquitetura no ambiente da construção civil. Quando algo é difícil de mudar, é preciso pensar com um pouco mais de antecedência para realmente nos certificarmos de que estamos indo na direção correta. No entanto, há um problema em tomar uma definição simples para uma ideia complexa e empregá-la como uma definição usada no trabalho – se essa declaração for tudo o que você pensa sobre arquitetura de software, muitos aspectos seriam negligenciados. Sim, uma boa parte da arquitetura de software diz respeito a pensar em aspectos que serão difíceis de mudar, mas também tem a ver com criar espaço para permitir mudanças no design.

Tornando as mudanças possíveis

Voltando ao mundo dos prédios em vez dos sistemas de software: o arquiteto Mies van der Rohe sem dúvida fez mais que qualquer outro arquiteto como pioneiro naquilo que atualmente pensamos como um arranha-céu moderno – seu famoso Seagram Building passou a ser um modelo para muito do que veio em seguida. O Seagram Building difere de muitos prédios que vieram antes dele. As paredes externas do prédio não são estruturais – elas envolvem uma moldura externa de aço. Os principais serviços do prédio – elevadores, escadas, ar-condicionado, água e esgoto e o sistema elétrico – passam por um núcleo central de concreto. Observe um arranha-céu moderno sendo construído atualmente, e você verá que é esse núcleo central de concreto que é construído inicialmente, com um guindaste gigantesco muitas vezes lá no topo. Nenhum andar do Seagram Building tem paredes internas estruturais – isso significa que você tem total

flexibilidade no que concerne ao modo como o espaço é usado. Pode reconfigurar a área conforme achar adequado, levando a fiação elétrica e o ar-condicionado para diferentes partes de cada andar pelos tetos suspensos e por dutos no próprio piso.

É interessante observar que o Seagram Building foi desenvolvido com um processo segundo o qual o projeto do prédio evoluiu enquanto a construção estava em andamento. Onde foi mesmo que vimos essa ideia antes?

Com esse projeto, a ideia era entregar o que Mies van der Rohe chamava de “espaço universal” – um volume grande e aberto que pudesse ser reconfigurado para atender a diferentes necessidades. O uso dos prédios muda, portanto a ideia era entregar um espaço que fosse o mais flexível possível no que diz respeito ao modo como poderia ser utilizado. Dessa forma, Mies van der Rohe não só teve de manter o foco nos aspectos estéticos fundamentais do prédio, encontrando espaço para os serviços essenciais que seriam difíceis, se não impossíveis, de serem modificados depois, mas tinha de garantir também que o prédio pudesse ser utilizado de diferentes maneiras daquelas vislumbradas originalmente. Em breve, veremos como fazer com que seja possível efetuar mudanças em uma arquitetura de microserviços.

Uma visão evolucionária para o arquiteto

Nossos requisitos como arquitetos de software mudam com mais rapidez do que para as pessoas que projetam e constroem prédios – assim como as ferramentas e técnicas à nossa disposição. Aquilo que criamos não permanece fixo no tempo. Assim que é lançado no ambiente de produção, nosso software continuará a evoluir, pois o modo como ele é usado mudará. Para a maior parte daquilo que criamos, temos de aceitar que, assim que o software chega às mãos de nossos usuários, teremos de reagir e nos adaptar, em vez de esperar que tenhamos um artefato que jamais vai mudar. Desse modo, os arquitetos de software precisam parar de pensar em criar o produto perfeito; em vez disso, devem manter o foco em ajudar a criar uma *moldura* na qual os sistemas apropriados possam surgir e

continuar a crescer à medida que aprendermos mais.

Embora eu tenha passado boa parte do capítulo desencorajando comparações com outras profissões, há uma analogia da qual eu gosto quando se trata do papel do arquiteto de TI, e que acho que engloba bem esse aspecto do papel. Erik Doernenburg da Thoughtworks inicialmente me apresentou a ideia de que devemos pensar no papel do arquiteto mais como o de alguém que planeja uma cidade do que como o de um arquiteto da área de construção civil. O papel de alguém que planeja uma cidade deve ser familiar para qualquer um de vocês que já tenha jogado SimCity ou Cities: Skylines. Sua função é olhar para diversas fontes de informações, e então tentar otimizar o layout de uma cidade para melhor atender às necessidades dos cidadãos atuais, ao mesmo tempo que leva em conta o seu uso no futuro. O modo como eles influenciam a evolução das cidades, porém, é interessante. Eles não dizem “Construa esse prédio específico ali”; em vez disso, eles definem zonas que permitem uma tomada de decisão local com base em determinadas restrições. Assim, como em SimCity, você poderia designar parte de sua cidade como uma zona industrial e outra parte como uma zona residencial. Caberá então a outras pessoas decidir quais prédios serão criados, porém há restrições: se quiser construir uma fábrica, ela deverá estar em uma zona industrial. Em vez de se preocupar demais com o que acontece em uma zona, a pessoa que planeja uma cidade gastará muito mais tempo definindo como as pessoas e os serviços públicos transitam de uma zona para outra.

Mais de uma pessoa já comparou uma cidade a uma criatura viva. A cidade muda com o tempo. Ela muda e evolui à medida que seus ocupantes a utilizam de diferentes maneiras, ou à medida que forças externas a moldam. A pessoa que planeja uma cidade se esforça ao máximo para prever essas mudanças, mas aceita que tentar exercer um controle direto sobre todos os aspectos do que acontece é inútil. Assim, nossos arquitetos, como aqueles que planejam uma cidade, precisam definir a direção geral, e devem se envolver em detalhes de implementação altamente específicos somente em casos limitados. Eles precisam garantir que o sistema seja adequado ao seu propósito agora, mas que também seja uma plataforma para o futuro.

A comparação com o software deve ser evidente. À medida que nossos usuários utilizam o nosso software, precisamos reagir e mudar. Não podemos prever tudo que acontecerá; portanto, em vez de nos planejarmos para toda e qualquer eventualidade, devemos nos planejar para permitir que haja mudanças, evitando o impulso de especificar tudo até os últimos detalhes. Nossa cidade – o sistema – deve ser um lugar bom e feliz para todos que a utilizam.

Definindo as fronteiras do sistema

Para continuar por mais um tempo com a metáfora do arquiteto como alguém que planeja uma cidade, o que seriam as nossas zonas? São as fronteiras de nossos microsserviços, ou talvez os grupos mais genéricos de microsserviços. Como arquitetos, devemos nos preocupar muito menos com o que acontece *dentro* de uma zona, e mais com o que acontece *entre* as zonas. Isso significa que precisamos investir tempo pensando em como nossos microsserviços conversam uns com os outros e garantir que poderemos monitorar devidamente a saúde geral de nosso sistema. Da área de arquitetura, é assim que criamos o nosso próprio *espaço universal* – ao definir algumas fronteiras específicas, enfatizamos aos nossos colegas que constroem o sistema quais são as áreas em que as mudanças podem ser feitas mais livremente, sem danificar algum aspecto fundamental da nossa arquitetura.

Para um exemplo bem simples, na Figura 16.1, vemos o microsserviço *Recomendações* acessando informações dos microsserviços *Promoções* e *Vendas*. Conforme já discutimos várias vezes, somos livres para alterar as funcionalidades ocultas nesses três microsserviços, sem nos preocupar em causar falhas no sistema como um todo – posso modificar o que eu quiser em *Vendas* ou *Promoções*, desde que eu continue preservando as expectativas que *Recomendações* tem sobre como irá interagir com esses microsserviços downstream.

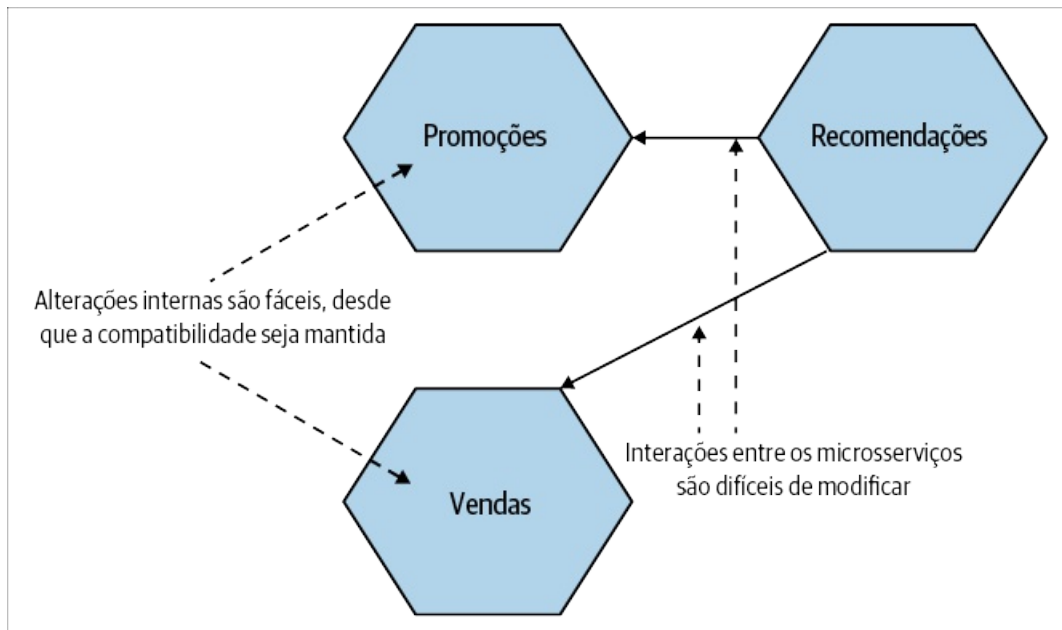


Figura 16.1 – Mudanças dentro das fronteiras de um microserviço são fáceis de fazer, desde que as interações entre os microserviços não mudem.

Podemos criar espaço para mudanças em níveis com um escopo mais amplo também. Na Figura 16.2, vemos que os microserviços da Figura 16.1 estão em uma zona de marketing que se mapeia às responsabilidades de uma equipe específica. Definimos um comportamento esperado no que concerne ao modo como a funcionalidade de marketing interage com o sistema mais amplo. Dentro da zona de marketing, podemos fazer qualquer alteração que quisermos, desde que a compatibilidade com o sistema mais amplo seja mantida. Voltando à ideia de entender aquilo que é difícil de mudar, as estruturas organizacionais geralmente se enquadram nessa categoria e, desse modo, as estruturas de equipe existentes podem ajudar a definir essas zonas para você. Coordenar mudanças em vários microserviços sob a responsabilidade da mesma equipe será mais fácil do que modificar as interações expostas a outras equipes.

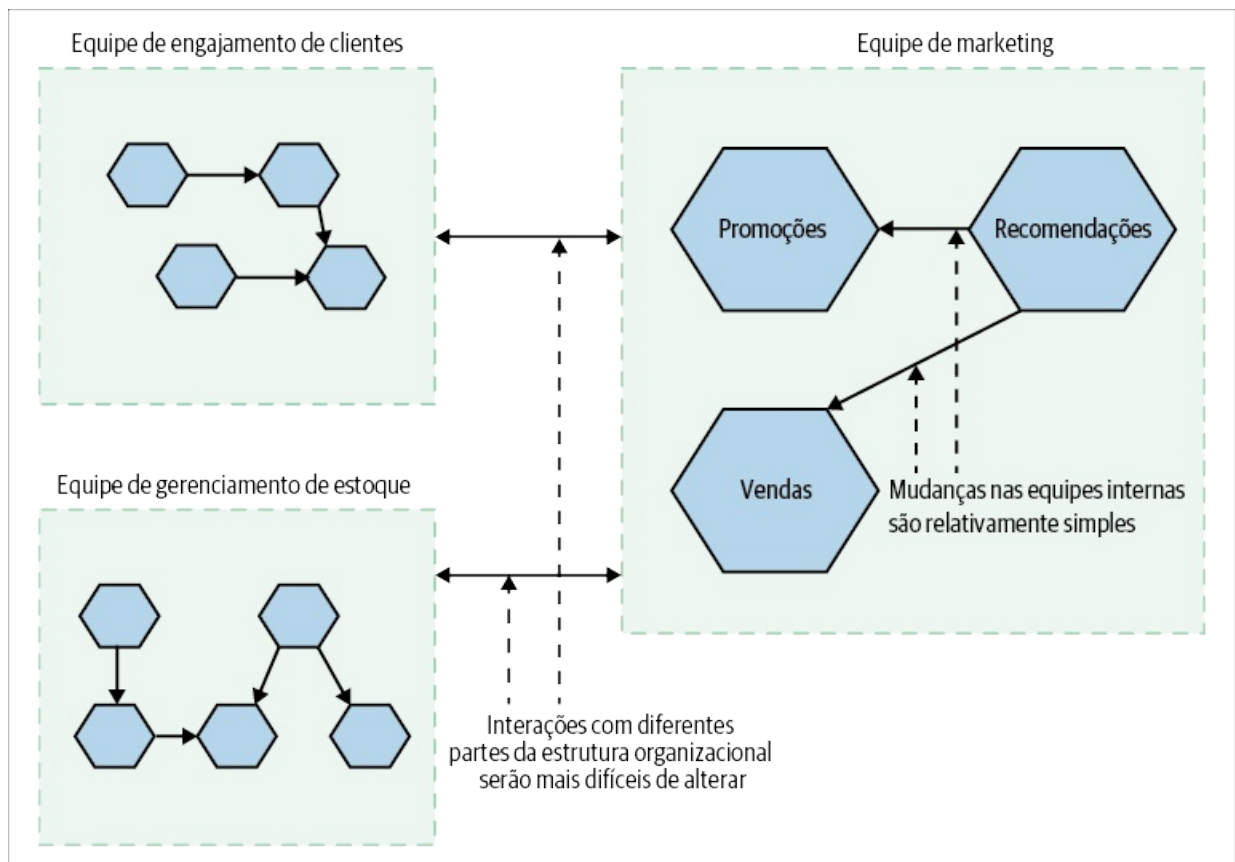


Figura 16.2 – Mudanças dentro de uma zona são mais fáceis de fazer do que mudanças entre zonas.

Isso se alinha muito bem com o conceito de uma API de equipe, que discutimos na seção “Equipes pequenas, empresa grande”, na página [577](#). Um arquiteto pode facilitar a criação de uma API de equipe, garantindo que os microsserviços da equipe e as práticas de trabalho estejam adequados à estrutura organizacional mais ampla.

Ao definir espaços nos quais essas mudanças possam ser feitas sem comprometer o sistema como um todo, facilitamos a vida dos desenvolvedores, além de manter nossa atenção nas partes do sistema que são mais difíceis de mudar. Você se lembra do conceito de ocultação de informações que exploramos no Capítulo 2? Conforme exploramos naquela ocasião, ocultar informações dentro das fronteiras de um microsserviço facilita muito a criação de uma interface estável para os consumidores. Quando fizermos alterações no microsserviço, será mais fácil garantir que não haverá incompatibilidade com os consumidores

externos. Nesse caso, podemos definir uma arquitetura que permita ocultar informações no nível de equipes, em vez de apenas no nível dos microsserviços. Isso nos dará outro nível para ocultar informações, além de criar um espaço seguro e mais abrangente no qual uma equipe possa fazer alterações locais sem causar falhas no sistema mais amplo.

Dentro de cada microsserviço ou de uma zona maior, podemos considerar que não será um problema se a equipe responsável por essa zona escolher um conjunto diferente de tecnologias ou um banco de dados distinto. Outras preocupações, é claro, poderão surgir nesse caso. Sua tendência em deixar que as equipes escolham a ferramenta correta para o trabalho pode ser influenciada pelo fato de que será mais difícil contratar pessoas ou fazê-las circular pelas equipes se houver dez conjuntos diferentes de tecnologias com as quais trabalhar. De modo semelhante, se cada equipe escolher um banco de dados totalmente diferente, você poderá se ver sem experiência suficiente para executar qualquer um deles em larga escala. A Netflix, por exemplo, padronizou o Cassandra como sua tecnologia de banco de dados na maior parte dos casos. Embora possa não ser a melhor opção para todas as situações, a Netflix acha que as vantagens obtidas com a criação de ferramentas e a aquisição de expertise para trabalhar com o Cassandra são mais importantes em comparação a ter de oferecer suporte e operar com várias plataformas diferentes em larga escala, as quais pudessem ser mais apropriadas para determinadas tarefas. A Netflix é um exemplo radical, na qual a escala provavelmente é o fator geral mais significativo, mas você entendeu a ideia.

Contudo, é entre os microsserviços que a situação pode se complicar. Se um microsserviço decide expor REST sobre HTTP, outro faz uso de gRPC e um terceiro utiliza Java RMI, a integração poderá se tornar um pesadelo, pois os microsserviços consumidores terão de entender e aceitar vários estilos de interação. É por isso que eu tento me ater à diretriz segundo a qual devemos “estar preocupados com o que acontece entre as máquinas, e ser liberal quanto ao que acontece dentro delas”.

Portanto uma arquitetura bem-sucedida, acima de tudo, diz respeito a permitir que mudanças sejam feitas para atender às necessidades de nossos

usuários. No entanto, um ponto que, com frequência, as pessoas esquecem é que o nosso sistema não acomoda apenas os usuários, mas também deve acomodar as próprias pessoas que criam o software. Uma arquitetura bem-sucedida também ajuda a criar um ambiente satisfatório para fazermos o nosso trabalho.

Uma construção social

Nenhum plano sobrevive ao contato com o inimigo.

– *Helmuth von Moltke (com uma boa paráfrase)*

Então, você pensou na visão, nas restrições e no que precisa realizar. Acha que sabe o que será difícil de mudar e as áreas em que precisa deixar as modificações possíveis. E agora? Bem, a arquitetura é o que acontece, e não o que você acha que deveria acontecer – essa é a diferença entre visão e realidade. Os arquitetos da área de construção civil precisam trabalhar com as pessoas que constroem o prédio para ajudá-las a entender a visão, mas também para modificar o plano quando a realidade desafiar essa visão. Pode ser que o que você acha que é possível essencialmente não seja. Se um arquiteto não estiver, até certo ponto, envolvido com as pessoas que criam o sistema, ele será incapaz de comunicar a visão às pessoas que fazem o trabalho, nem saberá se essa visão deixou de ser apropriada para o seu propósito. A equipe de construção pode encontrar problemas no terreno que não haviam sido previstos, ou talvez uma falta de suprimentos possa exigir uma reavaliação no projeto.

A arquitetura é o que acontece, e não o que foi planejado. Se, como arquiteto, você se afastar do processo de fazer essa visão acontecer, você não é um arquiteto – é um sonhador. A arquitetura que surgirá poderá ou não manter alguma relação com o que você deseja. Ela acontecerá com ou sem você. Implementar uma arquitetura exige o trabalho de muitas pessoas e uma série de decisões, grandes e pequenas.

De acordo com Grady Booch:⁴

No início, a arquitetura de um sistema com muito software é uma declaração da visão. No final, a arquitetura de todo sistema desse tipo

será um reflexo de bilhões e bilhões de decisões de design, pequenas e grandes, intencionais e acidentais, feitas ao longo do caminho.

Isso significa que, ainda que você tenha um indivíduo dedicado que, em última instância, seja responsável pela arquitetura, há muitas pessoas responsáveis por colocar essa visão em prática. Implementar uma arquitetura de sucesso será um esforço de equipe. Voltando à citação anterior de Ralph Johnson, “a arquitetura é uma construção social”. Um ótimo exemplo disso vem da Comcast, que compartilhou suas experiências acerca de como ela descentralizou a tomada de decisões com o uso de uma guilda de arquitetura (architecture guild).⁵ Considerando a sua escala, a Comcast decidiu tirar proveito das experiências dos grupos que direcionam o mercado, nas quais uma tomada de decisão coletiva é essencial:

Na Comcast, percebemos que esse problema parecia ser muito semelhante ao modo como as instituições de padrões abertos funcionam: fazer com que vários grupos autônomos cheguem a um consenso sobre abordagens técnicas. Criamos uma Guilda de Arquitetura (Architecture Guild) interna, modelada explicitamente com base em uma instituição muito bem-sucedida de padrões, a Internet Engineering Task Force (IETF), que define vários protocolos importantes da Internet.

– *Jon Moore, arquiteto de software líder na Comcast Cable*

A abordagem da Comcast tem certo nível de formalidade que algumas empresas podem achar onerosas, mas parece funcionar bem para essa empresa, considerando o seu tamanho e sua distribuição.

Habitabilidade

Mais um conceito que vem da área de construção civil e tem ressonância na área de desenvolvimento de software é o conceito de *habitabilidade*. Ouvi esse termo pela primeira vez com Frank Buschmann – ele explicou que os arquitetos são responsáveis por garantir que o ambiente que criam seja agradável para trabalhar. Se a arquitetura é a moldura do sistema, que descreve como os itens difíceis de mudar se encaixam, há também ocasiões em que será preciso definir restrições. Contudo, faça isso da forma errada,

e trabalhar no sistema poderá ser complicado, deixando-o suscetível a erros.

Conforme explica Richard Gabriel, autor do livro *Patterns of Software*⁶:

A habitabilidade é a característica do código-fonte que permite que os programadores retornem ao código em algum momento no futuro para entender a sua construção e as intenções, e possam alterá-lo com conforto e confiança.

Um ecossistema moderno de desenvolvimento de software, no entanto, é constituído por mais que apenas o código – ele vai além e inclui as tecnologias que usamos e as práticas de trabalho que adotamos. Com muita frequência, vejo desenvolvedores maldizendo a tecnologia que lhes foi imposta – muitas vezes, é uma tecnologia escolhida por pessoas que jamais tiveram de fazer uso dela. Quanto mais colaborativo for o processo de evolução de sua arquitetura e da seleção das ferramentas e da tecnologia usadas, mais fácil será garantir que o resultado seja um ambiente habitável, no qual as pessoas que criam o sistema se sentirão felizes e produtivas em seu trabalho.

Se pretendemos garantir que os sistemas que criamos sejam habitáveis para os nossos desenvolvedores, então nossos arquitetos e as outras pessoas responsáveis pela tomada de decisões devem entender o impacto de suas decisões. No mínimo, isso significa passar um tempo com a equipe, e o ideal é dedicar um tempo realmente programando com a equipe. Para aqueles que colocam a programação aos pares (pair programming) em prática, será uma simples questão de um arquiteto se juntar a uma equipe por um curto período como um membro de um par. Participar de exercícios de programação em conjunto (ensemble programming) também poderia trazer vantagens significativas, embora um arquiteto que faça parte de uma atividade de grupo como essa precisa estar ciente de como a sua presença poderia alterar a dinâmica do trabalho conjunto.

O ideal é que você deva trabalhar em atividades habituais para realmente entender o que é um trabalho “habitual”. Não me canso de enfatizar a importância de o arquiteto realmente passar um tempo com as equipes que criam o sistema! Isso será significativamente mais eficaz do que fazer uma

ligação telefônica ou simplesmente olhar o código. Quanto à frequência com que você deveria fazer isso, dependerá bastante do tamanho da(s) equipe(s) com a(s) qual(is) você trabalha. O essencial, porém, é que seja uma atividade de rotina. Se você trabalha com quatro equipes, por exemplo, talvez possa garantir que passará metade de um dia com cada equipe a cada quatro semanas, trabalhando com elas em suas tarefas de entrega de software a fim de garantir que aumentará o seu conhecimento e melhorará a comunicação com as equipes com as quais você trabalha.

Uma abordagem com princípios

Regras são para a obediência dos tolos, enquanto a orientação é para os sábios.

– Atribuído, de modo geral, a Douglas Bader

Tomar decisões no design de um sistema diz respeito totalmente a fazer ponderações, e as arquiteturas de microsserviços nos dão muitas oportunidades para fazê-las! Ao escolher um banco de dados, escolhemos uma plataforma com a qual temos menos experiência, mas que nos permite escalar melhor? Seria um problema para nós se tivéssemos dois conjuntos diferentes de tecnologias em nosso sistema?

E três? Algumas decisões podem ser tomadas prontamente com as informações que tivermos disponíveis, e essas são as mais fáceis de fazer. Mas, e as decisões que teriam de ser feitas com informações incompletas?

Um enquadramento pode ajudar nesse caso, e uma ótima maneira de ajudar a enquadrar nossa tomada de decisões é definir um conjunto de princípios e práticas que a orientem, com base em objetivos que estamos tentando alcançar. Vamos analisar cada um dos aspectos do enquadramento, um de cada vez.

Objetivos estratégicos

O papel do arquiteto já é suficientemente assustador, portanto, felizmente, não precisamos também definir os objetivos estratégicos! Os objetivos estratégicos devem estar relacionados à direção à qual sua empresa está caminhando e como ela se vê para deixar seus clientes satisfeitos. São

objetivos de nível geral, e podem nem sequer incluir a tecnologia. Poderiam ser definidos no nível da empresa ou de uma divisão. Exemplos seriam algo como “Expandir-se para o sudeste da Ásia para abrir novos mercados” ou “Vamos deixar que o cliente consiga fazer o máximo possível por conta própria”. O essencial é que esses objetivos definam a direção a ser seguida pela empresa, portanto é preciso garantir que a tecnologia esteja alinhada com isso.

Se você é a pessoa que define a visão técnica da empresa, talvez seja necessário passar mais tempo com os setores não técnicos de sua empresa (ou com “os negócios”, como essa parte geralmente é chamada). Qual é a visão que orienta os negócios? E como ela pode mudar?

Princípios

Os princípios são regras que foram definidas para alinhar o que fazemos com algum objetivo maior e, às vezes, eles podem mudar. Por exemplo, se um de seus objetivos estratégicos como empresa for diminuir o tempo para que novas funcionalidades cheguem ao mercado (time to market), você poderia definir um princípio que estabeleça que as equipes de entrega de software terão controle total de todo o ciclo de vida de seus softwares para fazerem lançamentos sempre que estiverem prontos, independentemente de qualquer outra equipe. Se outro objetivo da empresa for ser mais agressiva para aumentar a sua oferta de produtos em outros países, você poderia decidir implementar um princípio que estabeleça que o sistema como um todo deva ser portátil a fim de permitir que seja implantado localmente, respeitando a soberania dos dados.

É provável que você não vai querer ter muitos princípios. Menos de dez é um bom número – poucos a ponto de as pessoas se lembrarem, ou para caberem em um pequeno pôster. Quanto mais princípios você tiver, maior será a chance de eles se sobreporem ou se contradizerem.

Os Twelve Factors (Doze Fatores, <https://www.12factor.net>) da Heroku são um conjunto de princípios de design estruturados em torno do objetivo de ajudar você a criar aplicações que funcionem bem na plataforma Heroku. Esses princípios também podem fazer sentido em outros contextos.

Alguns, na verdade, são restrições sobre comportamentos que sua aplicação deve exibir para trabalhar com o Heroku. Uma restrição é algo que é muito difícil (ou praticamente impossível) de mudar, enquanto os princípios são algo que decidimos escolher. Você pode decidir denominar explicitamente os itens que são princípios *versus* aqueles que são restrições para ajudar a enfatizar aqueles que realmente não podem ser mudados. Pessoalmente, acho que pode ser bom manter tudo na mesma lista para incentivar que as restrições sejam ocasionalmente desafiadas e ver se elas realmente são imutáveis!

Práticas

Nossas práticas são o modo como garantimos que os princípios serão obedecidos. Elas são um conjunto de diretrizes detalhadas e práticas para executar as tarefas. Em geral, serão específicas para cada tecnologia e deverão ser suficientemente detalhadas para que qualquer desenvolvedor possa entendê-las. As práticas poderiam incluir diretrizes para escrita de código, o fato de que todos os dados de log precisam ser capturados de forma centralizada ou que o HTTP/REST é o estilo padrão de integração. Em virtude de sua natureza técnica, as práticas geralmente mudarão com mais frequência que os princípios.

Como ocorre com os princípios, às vezes as práticas refletem limitações em sua empresa. Por exemplo, se você decidiu escolher o Azure como sua plataforma de nuvem, isso deverá estar refletido em suas práticas.

As práticas devem sustentar seus princípios. Um princípio que estabeleça que as equipes de entrega controlam o ciclo de vida completo de seus sistemas pode implicar que haja uma prática que determine que todos os microsserviços devam ser implantados em contas isoladas na AWS, possibilitando um gerenciamento próprio dos recursos e um isolamento em relação às outras equipes.

Combinado princípios e práticas

Os princípios de uma pessoa são as práticas de outra. Você poderia decidir chamar o uso de HTTP/REST como um princípio em vez de prática, por

exemplo. E tudo bem. O ponto principal é que é importante ter ideias gerais que orientem como o sistema evolui, e ter detalhes suficientes para que as pessoas saibam como implementar essas ideias. Em um grupo pequeno, talvez com uma só equipe, não haveria problemas em combinar princípios e práticas. No entanto, em empresas maiores, nas quais a tecnologia e as práticas de trabalho podem ser diferentes de um lugar para outro, talvez você queira um conjunto distinto de práticas para cada lugar, desde que todas possam ser mapeadas para um conjunto comum de princípios. Uma equipe .NET, por exemplo, poderia ter um conjunto de práticas, enquanto uma equipe Java teria outro conjunto. Os princípios, no entanto, seriam iguais para ambos.

Um exemplo do mundo real

Um colega meu de longa data, Evan Bottcher, desenvolveu o diagrama mostrado na Figura 16.3 durante o tempo em que trabalhou com um de seus clientes. A figura mostra a inter-relação entre objetivos, princípios e práticas em um formato muito claro. Durante alguns anos, as práticas mais à direita mudarão com certa regularidade, enquanto os princípios permanecerão razoavelmente estáticos. Um diagrama como esse pode ser impresso em uma única folha de papel e compartilhado, e cada ideia é bem simples, de modo que qualquer desenvolvedor possa se lembrar. Evidentemente, há mais detalhes acerca de cada item apresentado, mas ser capaz de organizar essas ideias em um formato resumido é muito conveniente.

Faz sentido ter uma documentação que complemente alguns desses itens; melhor ainda será ter um código funcional que mostre como essas práticas podem ser implementadas. Na seção “A plataforma”, na página [589](#), vimos como a criação de um conjunto comum de ferramentas pode ajudar os desenvolvedores a fazerem o que é certo – o ideal é que a plataforma deva fazer com que seguir essas práticas seja o mais fácil possível e, à medida que as práticas mudarem, a plataforma deve acompanhar essas mudanças.

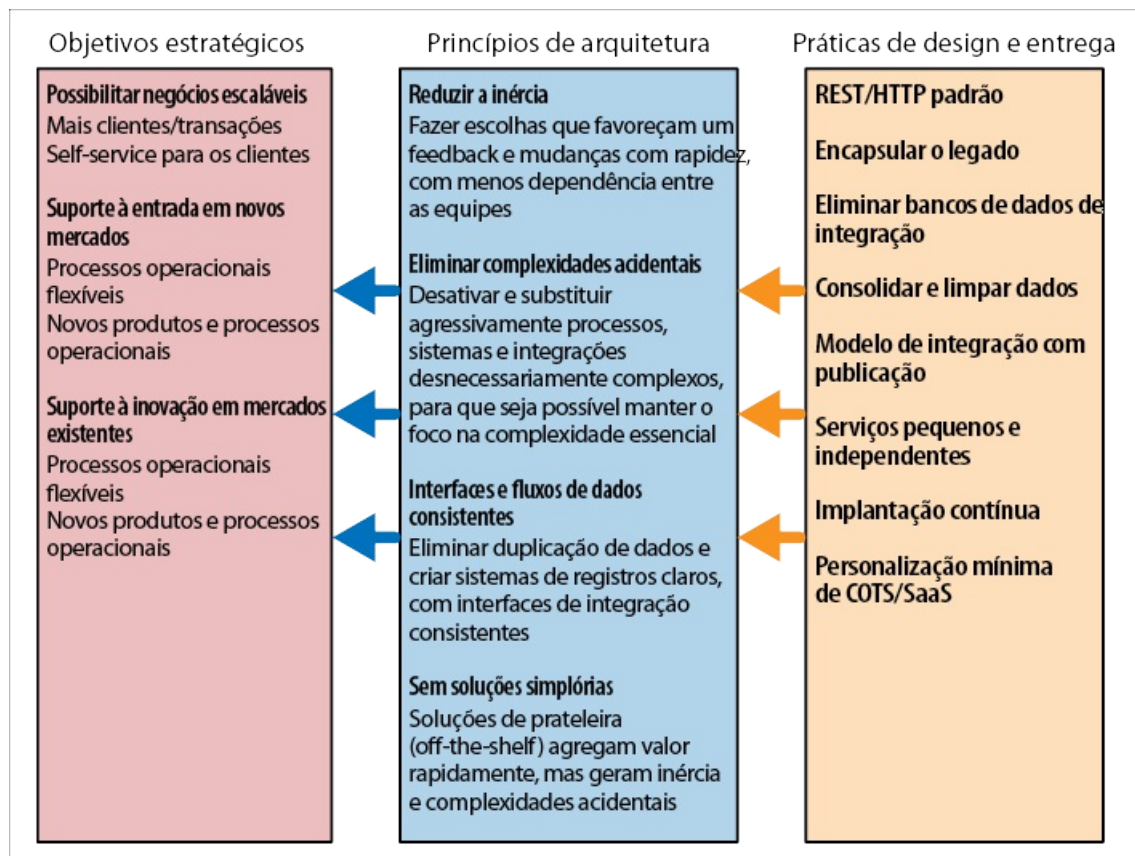


Figura 16.3 – Um exemplo real de princípios e práticas.

Direcionando uma arquitetura em evolução

Então, se nossa arquitetura não é estática, mas está sempre mudando e evoluindo, como garantimos que ela cresça e mude do modo que queremos, em vez de simplesmente sofrer mutações e se transformar em uma massa gigantesca e incontrolável de dor, sofrimento e recriminações? No livro *Building Evolutionary Architectures*⁷, os autores descrevem as funções de adequação (fitness functions) para ajudar a coletar informações sobre a relativa “adequação” da arquitetura, com o intuito de ajudar os arquitetos a decidir se precisam tomar alguma atitude. Segundo o livro:

A computação em evolução inclui uma série de métodos que permite que uma solução surja gradativamente por meio de pequenas mudanças em cada geração do software. Em cada geração da solução, o engenheiro avalia o estado atual: está mais próximo ou mais distante do objetivo? Por exemplo, ao usar um algoritmo genético para otimizar o design das asas, a função de adequação (fitness function) avalia a

resistência do vento, o peso, o fluxo de ar e outras características desejáveis para o bom design de uma asa. Os arquitetos definem uma função de adequação para explicar o que é melhor e ajudar a avaliar se o objetivo foi atingido. Em software, as funções de adequação verificam se os desenvolvedores preservam as características importantes da arquitetura.

A ideia de função de adequação (fitness function) é que ela seja usada para entender o estado atual de alguma propriedade importante, de modo que, se essa propriedade mudar e sair de determinados limites permitidos, a mudança terá de ser avaliada. Em geral, as funções de adequação são usadas para garantir que a arquitetura está sendo construída de modo a seguir os princípios e as restrições que foram estabelecidos.

Para emprestar um exemplo do livro *Building Evolutionary Architectures*, considere o requisito de que a resposta de um dado serviço deva ser recebida em 100 ms ou menos. Poderíamos implementar uma função de adequação para coletar dados de desempenho desse serviço, talvez em um ambiente de testes de desempenho ou de um sistema em execução no mundo real, a fim de garantir que esse comportamento do sistema atenda aos requisitos. O livro *Building Evolutionary Architectures* dá muito mais detalhes sobre o assunto, e eu definitivamente o recomendo, caso você queira explorar melhor esse conceito.

As funções de adequação para a arquitetura podem ter várias formas. O conceito básico, porém, é que você deve coletar dados do mundo real para entender se sua arquitetura continua “adequada” em relação a esses critérios. Esses dados podem estar relacionados ao desempenho do sistema, ao acoplamento de código, à duração dos ciclos ou a uma série de outros aspectos. As funções de adequação atuam como outra fonte de informações para ajudar um arquiteto a entender em que pontos o seu envolvimento poderia ser necessário. Entretanto, para mim, as funções de adequação funcionam melhor quando combinadas com uma colaboração estreita com as pessoas que criam o sistema. Essas funções devem ser um modo conveniente de ajudar você a entender se a arquitetura está indo para a direção correta, mas elas não eliminam a necessidade de conversar com as pessoas. Com efeito, eu sugeriria que definir as funções de adequação

corretas exigirá uma estreita colaboração com as pessoas.

Arquitetura em uma empresa alinhada a fluxos

No Capítulo 15, vimos como as empresas modernas que entregam software estão mudando para um modelo mais alinhado a fluxos, no qual equipes autônomas e independentes se concentram na entrega de funcionalidades de ponta a ponta, com suas prioridades determinadas pelos produtos. Falamos também de equipes transversais – as *equipes de capacitação* (enabling teams), que oferecem suporte às equipes alinhadas a fluxos. Em que lugar o arquiteto se enquadra nesse mundo? Bem, às vezes, o escopo de uma equipe alinhada a fluxos é suficientemente complexo a ponto de exigir um arquiteto dedicado (novamente, nesse caso, vemos com frequência uma linha indistinta entre os papéis do tradicional líder técnico e do arquiteto). Em muitos casos, porém, os arquitetos são requisitados para trabalhar com várias equipes.

Muitas das responsabilidades do arquiteto podem ser vistas como responsabilidades para facilitação – comunicar claramente a visão técnica, entender os desafios à medida que surgirem e ajudar a adaptar a visão técnica de acordo com isso. O arquiteto ajuda a conectar as pessoas, prestando atenção no panorama geral e ajudando as equipes a entenderem de que modo o que elas fazem se enquadra no todo mais amplo. Isso se alinha muito bem à ideia de um arquiteto como parte de uma equipe de capacitação (enabling team), conforme vemos na Figura 16.4. Uma equipe de capacitação como essa poderia ser composta de diversos tipos de pessoas – talvez pessoas dedicadas à equipe em tempo integral, e outras que entram para ajudar ocasionalmente.

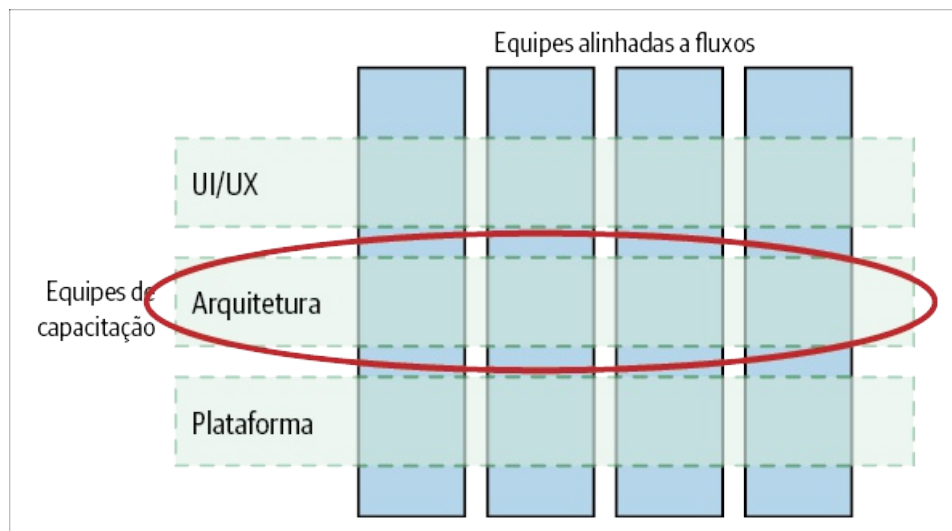


Figura 16.4 – Uma função de arquitetura em uma equipe de capacitação.

Um modelo de que gosto bastante é ter alguns arquitetos dedicados nessa equipe (talvez apenas uma ou duas pessoas, em muitos casos), mas ter essa equipe ampliada com o tempo, com especialistas em tecnologia de cada equipe de entrega – os líderes técnicos de cada equipe, no mínimo. O arquiteto é responsável por garantir que o grupo funciona. Com isso, o trabalho será distribuído e garantimos que haja um nível mais alto de comprometimento. Também garante que as informações fluam livremente das equipes para o grupo e, como resultado, a tomada de decisões será muito mais sensata e bem fundamentada.

Às vezes, o grupo pode tomar decisões das quais o arquiteto discorda. Nesse caso, o que o arquiteto deve fazer? Tendo estado nessa posição antes, posso dizer que essa é uma das situações mais desafiadoras a ser enfrentada. Em geral, minha abordagem é a de que devo acatar a decisão do grupo. Assumo que fiz o melhor para convencer as pessoas, mas, em última análise, não fui suficientemente convincente. Com frequência, o grupo é mais sábio que um indivíduo e, mais de uma vez, já me provaram que eu estava errado! E imagine como poderia ser desencorajador para um grupo se lhe dessem espaço para tomar uma decisão, e então, no final, ser ignorado. Em algumas ocasiões, porém, eu desacatei a decisão do grupo. Mas por quê, e quando? Como traçamos essas linhas?

Pense em uma criança sendo ensinada a andar de bicicleta. Você não pode

andar na bicicleta por ela. Você a vê se desequilibrar, mas, se interferir todas as vezes em que pareça que ela vai cair, a criança jamais aprenderá e, de qualquer modo, ela cairá com muito menos frequência do que você acha que cairá! No entanto, se você a vir prestes a se dirigir para o trânsito ou a cair em um lago nas proximidades, será necessário intervir. É claro que, com frequência, já me provaram que eu estava errado em situações desse tipo – deixei que a equipe fizesse algo que eu achava que estava errado, e o que eles fizeram funcionou! De modo semelhante, como arquiteto, você precisa ter um forte domínio acerca de quando, de modo figurado, sua equipe está se dirigindo a um lago. Você também deve estar ciente de que, mesmo que saiba que tem razão e desacate a decisão da equipe, isso poderá prejudicar a sua posição, além de fazer com que a equipe ache que ela não tem direito a uma opinião. Às vezes, o certo é acatar uma decisão com a qual você não concorda. Saber quando fazer isso e quando não fazer é difícil, mas, às vezes, é crucial.

A situação começa a ficar interessante, conforme discutiremos em breve, quando um arquiteto também precisa se envolver em atividades de governança. Isso pode causar certa confusão quanto ao papel de qualquer equipe de arquitetura abrangente. O que acontece quando uma equipe discorda da estratégia técnica? Seria um problema? Talvez seja uma exceção razoável, mas poderia também causar problemas mais básicos. Uma decisão de curto prazo, feita em nome da conveniência, poderia comprometer mudanças maiores que estivermos planejando fazer. Suponha que o grupo de arquitetura esteja tentando ajudar a mudar a empresa de modo que ela deixe de utilizar um banco de dados centralizado por causa do acoplamento e de problemas operacionais causados por ele, mas uma das equipes decide simplesmente inserir novos dados em um banco de dados compartilhado, pois está sendo pressionada a entregar o software rapidamente. O que acontecerá então?

Com base em minha experiência, posso dizer que tudo isso se resume a ter uma comunicação clara e eficaz, e um conhecimento das responsabilidades. Se eu vir um dono de produto (product owner) tomando decisões que eu achasse que prejudicariam algum tipo de atividade de caráter abrangente no qual eu estivesse trabalhando, eu iria até

essa pessoa e conversaria com ela. Talvez a resposta seja que a decisão de curto prazo seria a correta (e, sem dúvida, isso acabaria se transformando em uma espécie de débito técnico que iríamos assumir de forma consciente). Em outros casos, talvez o dono do produto pudesse mudar o seu planejamento a fim de ajudar a trabalhar com a estratégia geral. No pior dos casos, o problema talvez precise ser levado aos níveis superiores.

Na REA, a empresa imobiliária online da qual falei alguns capítulos antes, os donos de produto ocasionalmente tomavam decisões para priorizar o trabalho de um modo que fazia com que os débitos técnicos se acumulassem, levando a problemas no futuro. O problema era que os donos de produto eram basicamente responsabilizados quanto à sua capacidade de entregar funcionalidades e deixar os clientes satisfeitos, enquanto, de modo geral, os problemas relacionados aos débitos técnicos ficavam por conta dos líderes técnicos. Uma mudança foi feita para deixar os donos de produto responsáveis também pelos aspectos do software que eram técnicos por natureza – isso significava que eles tinham de ter um papel mais ativo para entender os aspectos mais técnicos do sistema (segurança ou desempenho, por exemplo), e trabalhar em colaboração mais estreita com os especialistas técnicos para priorizar o trabalho a ser feito. O ato de deixar os donos de produto que não são técnicos como mais responsáveis pela priorização das atividades técnicas não é trivial, mas, sem dúvida, vale a pena ser feito, com base em minha experiência.

Construindo uma equipe

Ser a principal pessoa responsável pela visão técnica de seu sistema e garantir que o trabalho está sendo feito de acordo com essa visão não é somente uma questão de garantir que as decisões tecnológicas corretas sejam tomadas. São as pessoas com quem você trabalha que farão o trabalho. É função de qualquer líder técnico ajudar essas pessoas a crescer – ajudá-las a fazer parte da criação dessa visão – e garantir que elas também possam ser participantes ativas na modelagem e implementação da visão.

Ajudar as pessoas ao seu redor no crescimento de suas carreiras pode ser

feito de várias maneiras, a maioria das quais estão fora do escopo deste livro. Há um aspecto, porém, no qual uma arquitetura de microsserviços é particularmente relevante. Em sistemas monolíticos maiores, há menos oportunidades para as pessoas se manifestarem e serem “donas” de algo. Com os microsserviços, por outro lado, há várias bases de código autônomas que terão seus próprios ciclos de vida independentes. Ajudar as pessoas a crescerem fazendo com que assumam a responsabilidade por microsserviços individuais antes de aceitarem outras responsabilidades pode ser uma ótima maneira de ajudá-las a atingir suas próprias metas na carreira, ao mesmo tempo que reduz o fardo de qualquer que seja a pessoa que estivesse com essa responsabilidade!

Acredito firmemente que um ótimo software é resultado de ótimas pessoas. Se você se preocupar somente com o lado tecnológico da equação, deixará de ver muito mais do que apenas a metade do quadro geral.

Padrão necessário

Quando estiver trabalhando com suas práticas e pensando nas decisões de custo-benefício que terá de fazer, um dos pontos mais importantes será achar um equilíbrio para o nível de variabilidade que será permitido em seu sistema. Um dos principais modos de identificar o que deve ser constante de microsserviço para microsserviço é definir como seria um bom microsserviço, com um bom comportamento. Como seria um microsserviço considerado um “bom cidadão” em seu sistema? Quais são as características que ele precisa ter para garantir que seu sistema seja administrável, e que um microsserviço ruim não cause falha no sistema como um todo? Assim como no caso das pessoas, o que seria um microsserviço considerado um “bom cidadão” em um contexto não reflete como ele seria em outro. Apesar disso, há algumas características comuns entre os microsserviços com bom comportamento, que acho que são muito importantes para serem observadas. Essa é uma das poucas áreas essenciais em que permitir divergências demais pode resultar em situações muito complicadas. Como diz Ben Christensen do Facebook, quando pensamos no quando geral, “é preciso que seja um sistema coeso, composto de

muitas partes pequenas, com ciclos de vida autônomos, mas com tudo funcionando em conjunto”. Portanto, temos de encontrar um equilíbrio para otimizar com vistas à autonomia dos microsserviços individuais, mas sem perder de vista o quadro geral. Definir atributos claros, que cada microsserviço deve ter, é uma forma de ser claro acerca desse ponto de equilíbrio. Vamos falar rapidamente de alguns desses atributos.

Monitoração

É essencial que sejamos capazes de obter visões coerentes, abrangendo vários serviços, da saúde de nosso sistema. Essa visão não deve ser específica apenas de um microsserviço, mas deve englobar o sistema todo. Conforme discutimos no Capítulo 10, conhecer a saúde de um microsserviço individual é conveniente, mas, em geral, somente quando você estiver tentando diagnosticar um problema maior ou entender uma tendência geral. Para facilitar essa tarefa ao máximo, eu sugeriria garantir que todos os microsserviços gerem métricas relacionadas à saúde e à monitoração em geral, da mesma maneira.

Você poderia optar por adotar um sistema de push, em que cada microsserviço envie esses dados para um local centralizado. Qualquer que seja a sua escolha, procure mantê-la padronizada. Faça com que a tecnologia interna às máquinas seja opaca, e não exija que seus sistemas de monitoração mudem para aceitá-la. O logging se enquadra na mesma categoria: precisamos que ele esteja em um só local.

Interfaces

Escolher poucas tecnologias definidas para interface ajuda na integração de novos consumidores. Ter um único padrão é bom. Dois também não é tão ruim. Ter vinte estilos diferentes de integração não é bom. Não se trata apenas de escolher a tecnologia e o protocolo. Se escolher HTTP/REST, por exemplo, você usará verbos ou nomes? Como lidará com a paginação dos recursos? Como lidará com o controle de versões dos endpoints?

Segurança na arquitetura

Não podemos nos dar ao luxo de permitir que um único microsserviço com um comportamento ruim estrague a festa de todos. Temos de garantir que nossos microsserviços se protejam devidamente contra chamadas downstream que não sejam saudáveis. Quanto mais microsserviços tivermos, que não lidem de modo apropriado com as possíveis falhas de chamadas downstream, mais frágeis serão os nossos sistemas. Isso pode significar, por exemplo, que você deve tornar obrigatórias algumas práticas relacionadas à comunicação entre serviços, por exemplo, exigir o uso de circuit breakers (um assunto que exploramos na seção “Padrões de estabilidade”, na página [464](#)).

Jogar conforme as regras é importante quando se trata de códigos de resposta também. Se seus circuit breakers dependem de códigos HTTP, e um microsserviço decide enviar códigos 2XX para erros ou confunde códigos 4XX com códigos 5XX, essas medidas de segurança irão por água abaixo. Preocupações semelhantes se aplicariam mesmo que você não estivesse usando HTTP; precisamos saber a diferença entre uma requisição que não apresentou problemas e foi processada corretamente, uma requisição ruim e que, portanto, impediu que o microsserviço fizesse algo com ela, e uma requisição que poderia estar correta, mas nada podemos dizer, pois o servidor estava inativo. Ter esse conhecimento é essencial para garantir que possamos falhar rapidamente e rastrear os problemas. Se nossos microsserviços desprezarem essas regras, acabaremos com um sistema mais vulnerável.

Governança e a estrada pavimentada

Parte daquilo com o qual os arquitetos precisam lidar é a governança. O que quero dizer com *governança*? O fato é que o framework COBIT (Control Objectives for Information Technologies, ou Objetivos de Controle para Tecnologias de Informação) tem uma definição muito boa:⁸

A governança garante que os objetivos corporativos sejam alcançados avaliando as necessidades, condições e opções dos stakeholders, definindo a direção por meio de priorização e tomada de decisões, e monitorando o desempenho, a conformidade e o progresso em relação

à direção e aos objetivos combinados.

Em poucas palavras, podemos considerar que a governança seja chegar a um acordo sobre como as tarefas devem ser feitas, garantir que as pessoas saibam como elas devem ser feitas e certificar-se de que sejam feitas dessa maneira. Em alguns ambientes, a governança acontece apenas informalmente, como parte das atividades usuais de desenvolvimento de software. Em outros, particularmente em empresas maiores, poderia ser uma função mais concreta.

A governança pode ser aplicada a vários aspectos da área de TI. Queremos manter o foco no aspecto da governança técnica – algo que acho que é o trabalho de um arquiteto. Se um dos trabalhos do arquiteto é garantir que haja uma visão técnica, então a governança diz respeito a garantir que o que estamos construindo está de acordo com essa visão, e que a visão evoluirá se for necessário.

Basicamente a governança deve ser uma atividade de grupo. Um grupo de governança que funcione de forma apropriada é capaz de trabalhar junto para dividir o trabalho e modelar a visão. Isso poderia ocorrer por meio de uma conversa informal no caso de uma equipe pequena, ou com uma reunião periódica mais estruturada, com uma adesão formal ao grupo, em um escopo mais amplo. É nesse grupo que eu acho que os princípios que abordamos antes devem ser discutidos e modificados conforme for necessário. Se houver a necessidade de um grupo formal, ele deve ser composto predominantemente de pessoas que executam o trabalho sujeito à governança. Esse grupo também deve ser responsável por monitorar e gerenciar os riscos técnicos.

Reunir-se e chegar a um consenso sobre como o trabalho deve ser feito é uma boa ideia. No entanto, gastar tempo para garantir que as pessoas estão seguindo essas diretrizes é menos divertido, assim como impor um fardo aos desenvolvedores para que implementem todos esses padrões que você espera que cada microsserviço tenha. Acredito firmemente em facilitar para que se faça o que é certo – e, conforme discutimos no Capítulo 15, o conceito de estrada pavimentada é realmente útil nesse caso. O arquiteto tem como função articular claramente a visão – para onde você está indo –

e fazer com que seja mais fácil chegar lá. Desse modo, ele deve se envolver para ajudar a moldar os requisitos de qualquer que seja a estrada pavimentada que você irá criar. Para muitos, a plataforma será o maior exemplo disso – o arquiteto acaba por se tornar um stakeholder importante para a equipe de plataforma.

Já vimos o papel da plataforma com certo nível de detalhes, portanto vamos ver outras técnicas que podemos usar para facilitar o máximo possível que as pessoas façam o que é certo.

Exemplares

Ter uma documentação por escrito é bom e conveniente. Percebo claramente a sua importância; afinal de contas, escrevi este livro. No entanto, os desenvolvedores também gostam de códigos – códigos que possam executar e explorar. Se você tem um conjunto de padrões ou de melhores práticas que gostaria de incentivar, ter exemplares que as pessoas possam usar como referência será conveniente. A ideia é que as pessoas não poderiam cometer muitos erros simplesmente se imitarem algumas das melhores partes de seu sistema.

O ideal é que os exemplares sejam microsserviços reais executando em seu sistema, que façam o que é certo, em vez de serem microsserviços isolados, implementados apenas para serem “exemplos perfeitos”. Ao garantir que seus exemplares sejam realmente usados, você garantirá que todos os princípios que tiver fazem sentido.

Template personalizado para os microsserviços

Não seria ótimo se você pudesse realmente facilitar para que todos os desenvolvedores seguissem a maioria das diretrizes definidas, com muito pouco trabalho? E se os desenvolvedores tivessem imediatamente a maior parte do código pronta para implementar os atributos básicos necessários a cada microsserviço?

Há muitos frameworks para diferentes linguagens de programação que procuram oferecer os blocos de construção para o template de seus próprios microsserviços. O Spring Boot (<https://oreil.ly/KYWe5>)

provavelmente é o exemplo mais bem-sucedido de um framework desse tipo para a JVM. O framework Spring Boot básico é razoavelmente leve, mas você pode decidir adicionar um conjunto de bibliotecas para disponibilizar recursos como verificar a saúde do microsserviço, servir HTTP ou expor métricas. Assim, de imediato, você terá um microsserviço “Hello World” simples, que poderá ser iniciado na linha de comando.

Desse modo, muitas pessoas utilizam esses frameworks e padronizam essa configuração em suas empresas. Por exemplo, ao iniciar um novo microsserviço, scripts poderiam ser criados para que as pessoas possam obter um template Spring Boot com as bibliotecas básicas usadas pela empresa já incluídas; o template já poderia incluir as bibliotecas que lidam com circuit breakers e ser configurado para lidar com autenticação JWT para chamadas de entrada. Geralmente, uma criação de template automática como essa criaria também um pipeline de construção correspondente.

Advertência

A seleção e configuração desses templates personalizados de microsserviços é uma tarefa comumente atribuída à equipe de plataforma. Ela pode, por exemplo, disponibilizar um template para cada linguagem aceita, garantindo que, se o template for usado, os microsserviços resultantes funcionarão bem na plataforma em questão. Isso, porém, pode causar problemas.

Já vi muitas equipes com o moral e a produtividade destruídos por terem um framework obrigatório imposto a elas. Em uma tentativa de melhorar a reutilização de código, cada vez mais trabalho é colocado em um framework centralizado, até ele se transformar em uma terrível monstruosidade. Se você decidir usar um template de microsserviço personalizado, pense com muito cuidado sobre a sua função. O ideal é que seu uso seja estritamente opcional, mas, se quiser ser mais impositivo quanto à sua adoção, é preciso entender que a facilidade de uso pelos desenvolvedores deve ser um fator determinante essencial. Permitir que os desenvolvedores que utilizam o template façam recomendações e até mesmo contribuam com mudanças no framework, talvez como parte de

um modelo de open source interno, pode ajudar bastante nesse caso.

Conforme discutimos na seção “DRY e os perigos da reutilização de código em um mundo de microsserviços”, na página [190](#), devemos estar cientes dos perigos do compartilhamento de códigos. Em nosso desejo de criar código reutilizáveis, podemos introduzir causas de acoplamento entre os microsserviços. Ao menos uma das empresas com as quais conversei está muito preocupada com isso, a ponto de copiar o código do template de seus microsserviços manualmente para cada microsserviço. Isso significa que um upgrade no template básico do microsserviço demorará mais para ser aplicado em todo o sistema, mas isso é menos preocupante para a empresa do que os perigos do acoplamento. Outras equipes com as quais conversei simplesmente tratam o template dos microsserviços como uma dependência binária compartilhada, embora precisem ser muito cautelosos para evitar que a tendência para o DRY (Don’t Repeat Yourself, ou Não se Repita) resulte em um sistema excessivamente acoplado!

Estrada pavimentada em escala

O uso de templates de microsserviços desenvolvidos internamente e de frameworks muitas vezes é visto em empresas que têm muitos microsserviços. A Netflix e a Monzo são duas empresas desse tipo. Cada uma delas decidiu padronizar seu conjunto de tecnologias até certo ponto (JVM no caso da Netflix, Go na Monzo), permitindo que agilizassem a criação de um novo microsserviço com um comportamento padrão esperado, por meio da utilização de um conjunto comum de ferramentas. Com um conjunto mais diversificado de tecnologias, ter um template padrão para microsserviços de acordo com as suas próprias necessidades será mais difícil.

Se você pretende adotar vários conjuntos díspares de tecnologia, seria necessário ter um template de microsserviço correspondente para cada um. No entanto, essa poderia ser uma maneira de restringir sutilmente as opções de linguagens em suas equipes. Se o template de microsserviço definido internamente aceitar somente a JVM, as pessoas poderão se sentir desestimuladas a escolher conjuntos alternativos de tecnologias caso elas

tenham muito mais trabalho a ser feito por conta própria. A Netflix, por exemplo, se preocupa particularmente com aspectos como tolerância a falhas, de modo a garantir que a indisponibilidade de uma parte de seu sistema não derrube todo o sistema. Para lidar com isso, muito trabalho foi feito para garantir que haja bibliotecas de clientes na JVM a fim de fornecer as ferramentas que as equipes precisam para manter seus microsserviços bem-comportados. Introduzir um novo conjunto de tecnologias significaria ter de reproduzir todo esse esforço. A principal preocupação da Netflix tem menos relação com os esforços duplicados, e mais com o fato de que é muito fácil cometer erros. O risco de um microsserviço implementar incorretamente uma tolerância a falhas é alto caso possa causar impacto em outras partes do sistema. A Netflix atenua esse problema utilizando “serviços sidecar”, os quais se comunicam localmente com uma JVM que utiliza as bibliotecas apropriadas.

Os service meshes nos oferecem outra possível maneira de transferir um comportamento comum para eles. Algumas funcionalidades que eram comumente vistas como uma responsabilidade interna do microsserviço agora podem ser passadas para um mesh de microsserviços. Isso garante mais consistência no comportamento entre os microsserviços escritos com diferentes linguagens de programação, além de reduzir as responsabilidades dos templates desses microsserviços.

Débito técnico

Muitas vezes, somos colocados em situações em que não podemos seguir estritamente a nossa visão técnica. Com frequência, precisamos decidir tomar alguns atalhos para que alguma funcionalidade urgente seja lançada. Essa é apenas mais uma decisão de custo-benefício diante da qual nos veremos. Nossa visão técnica existe por um motivo. Se nos desviarmos desse motivo, poderemos ter uma vantagem no curto prazo, mas haverá um custo no longo prazo. Um conceito que nos ajuda a entender essa relação de custo-benefício é o débito técnico. Quando acumulamos débitos técnicos, assim como uma dívida no mundo real, haverá um custo, e será algo que devemos pagar.

Às vezes, o débito técnico não é somente algo no qual incorremos por tomar atalhos. O que acontecerá se nossa visão para o sistema mudar, mas nem todas as partes do sistema estiverem de acordo com essa mudança? Nessa situação, também, gerariamos outras causas para débitos técnicos.

O trabalho do arquiteto é olhar para o panorama geral e entender esse equilíbrio. Ter certa visibilidade sobre o nível do débito e quando será preciso se envolver é importante. Dependendo da empresa, talvez você consiga oferecer orientações com tranquilidade, mas poderá fazer com que as próprias equipes decidam como controlar e pagar a dívida. Em outras empresas, talvez seja necessária uma melhor estrutura, talvez mantendo um registro dos débitos a ser revisado com regularidade.

Tratamento de exceções

Então, nossos princípios e práticas orientam o modo como nossos sistemas devem ser construídos. Mas o que acontecerá se nosso sistema se desviar deles? Às vezes, tomamos uma decisão que é simplesmente uma exceção à regra. Nesses casos, talvez valha a pena registrar essa decisão em algum lugar, para referência futura. Se houver *exceções* suficientes, em algum momento, pode ser que faça sentido modificar o princípio ou a prática em questão para que reflita um novo entendimento do mundo. Por exemplo, poderíamos ter uma prática que estabeleça que sempre usaremos o MySQL para armazenamento de dados. Entretanto, passamos a ver motivos convincentes para utilizar o Cassandra por ser um banco de dados altamente escalável; a essa altura, podemos alterar a nossa prática de modo que diga o seguinte: “Utilize o MySQL para a maioria dos requisitos de armazenamento, a menos que se espere um grande aumento no volume de dados; nesse caso, utilize o Cassandra”.

Vale a pena reforçar, porém, que cada empresa é diferente. Já trabalhei com algumas empresas em que as equipes de desenvolvimento tinham um grau elevado de confiança e autonomia, e os princípios eram genéricos (e a necessidade de um tratamento de exceção explícito será bastante reduzido, ou então será eliminado). Em empresas mais estruturadas, nas quais os desenvolvedores têm menos liberdade, manter o controle sobre as exceções

pode ser essencial para garantir que as regras estabelecidas reflitam de forma apropriada os desafios que as pessoas enfrentam. Apesar de tudo que foi dito, sou fã dos microsserviços como uma forma de otimizar com vistas à autonomia das equipes, dando-lhes o máximo possível de liberdade para solucionar os problemas que tiverem em mãos. Se você trabalha em uma empresa que imponha muitas restrições ao modo como os desenvolvedores podem fazer o seu trabalho, os microsserviços talvez não sejam uma opção apropriada para você.

Resumo

Para resumir este capítulo, eis o que vejo como as responsabilidades básicas de um arquiteto em evolução:

Visão

Garantir que haja uma visão técnica para o sistema, comunicada de forma clara, a qual ajudará o sistema a atender aos requisitos de seus clientes e da empresa.

Empatia

Compreender o impacto de suas decisões nos clientes e nos colegas.

Colaboração

Envolver-se com o máximo possível de pares e colegas para ajudar a definir, detalhar e pôr em prática a visão.

Adaptabilidade

Garantir que a visão técnica mude conforme exigido pelos seus clientes ou pela empresa.

Autonomia

Encontrar o equilíbrio correto entre padronização e possibilidade de autonomia para suas equipes.

Governança

Garantir que o sistema sendo implementado está adequado à visão técnica, e certificar-se de que será fácil para as pessoas fazerem o que é certo.

O arquiteto em evolução é aquele que entende que demonstrar essas responsabilidades básicas é um ato contínuo de ponderação. Há sempre forças que nos empurram para um lado ou para outro, e entender quando se deve lutar contra elas ou quando seguir com o fluxo muitas vezes é algo que saberemos fazer somente com a experiência. No entanto, a pior reação a todas essas forças que nos empurram em direção a mudanças é se tornar mais rígido ou inflexível em nosso pensamento.

Embora boa parte dos conselhos deste capítulo possam se aplicar a qualquer arquiteto de sistemas, os microsserviços exigem que tomemos muito mais decisões. Assim, ter mais capacidade de fazer todas essas ponderações é essencial. Se quiser explorar esse assunto com mais detalhes, posso recomendar o já citado livro *Building Evolutionary Architectures*, assim como o livro *The Software Architect Elevator*⁹, de Gregor Hohpe, que ajuda os arquitetos a entenderem como eles podem preencher a lacuna entre um pensamento estratégico de alto nível e uma entrega de software na prática.

Estamos quase no fim do livro e discutimos vários assuntos. No Posfácio, apresentarei uma síntese do que aprendemos.

¹ Por vários motivos, inclusive pelo fato de eu ter me formado em engenharia de software...

² Extraído de uma troca de emails na lista de discussão sobre programação extrema (extreme programming), que Martin Fowler então compartilhou em seu artigo “Who Needs an Architect?” (Quem precisa de um arquiteto?, <https://oreil.ly/6C0cI>).

³ N.T.: Esta e as demais citações deste capítulo foram traduzidas livremente, com base no texto original em inglês.

⁴ Grady Booch (@Grady_Booch), Twitter, 4 de setembro de 2020, 5h12min, <https://oreil.ly/ZgPRZ>.

⁵ Jon Moore, “Architecture with 800 of My Closest Friends: The Evolution of Comcast’s Architecture Guild” (Arquitetura com 800 dos meus melhores amigos: a evolução da guilda de arquitetura na Comcast), InfoQ, 14 de maio de 2019, <https://oreil.ly/aIvbi>.

⁶ Richard Gabriel, *Patterns of Software: Tales from the Software Community* (Nova York: Oxford University Press, 1996).

⁷ Neal Ford, Rebecca Parsons e Patrick Kua, *Building Evolutionary Architectures* (Sebastopol: O’Reilly, 2017).

⁸ COBIT 5: *A Business Framework for the Governance and Management of Enterprise IT* (COBIT 5: um framework de negócios para a governança e o gerenciamento de TI nas empresas), (Rolling Meadows, IL: ISACA, 2012).

⁹ Gregor Hohpe, *The Software Architect Elevator* (Sebastopol: O’Reilly, 2020).

Posfácio: reunindo tudo

Este livro abordou vários assuntos, e apresentei vários conselhos no caminho. Considerando a extensão de sua abrangência, achei razoável resumir alguns dos meus conselhos principais sobre as arquiteturas de microserviços. Para aqueles que leram o livro inteiro, será ótimo para recordar. Para os impacientes que foram direto para o final, saibam que há *muitos* detalhes por trás desses conselhos, e eu insistiria que vocês lessem os detalhes que estão na base dessas ideias, em vez de apenas as adotar cegamente.

Dito isso, meu objetivo é deixar este último capítulo tão conciso quanto possível, portanto vamos começar.

O que são microserviços?

Conforme apresentados no Capítulo 1, os *microserviços* são um tipo de arquitetura orientada a serviços com foco em implantações independentes. Ter *implantações independentes* significa que podemos fazer uma mudança em um microserviço, implantá-lo e lançar sua funcionalidade aos usuários finais sem a necessidade de alterar outros microserviços. Tirar o máximo de proveito de uma arquitetura de microserviços implica abraçar esse conceito. Geralmente, cada microserviço é implantado como um processo, com a comunicação com outros microserviços feita por meio de algum tipo de protocolo de rede. É comum implantar várias instâncias de um microserviço, talvez para poder escalar mais, ou ainda para aumentar a robustez pelo fato de haver redundância.

Para que as implantações independentes sejam possíveis, precisamos garantir que, quando alteramos um microserviço, não causaremos falhas nas interações com outros microserviços. Isso exige que nossas interfaces com outros microserviços sejam estáveis e que as alterações sejam feitas de modo a serem compatíveis com versões anteriores. A *ocultação de*

informações, que foi mais bem explicada no Capítulo 2, descreve uma abordagem segundo a qual ocultamos o máximo possível de informações (código, dados) atrás de uma interface. Devemos expor somente o mínimo nas interfaces de nossos serviços a fim de satisfazer nossos consumidores. Quanto menos expusermos, mais fácil será garantir que as mudanças feitas serão compatíveis com versões anteriores. A ocultação de informações também nos permite fazer mudanças nas tecnologias dentro das fronteiras de um microsserviço de modo que não haja impacto nos consumidores.

Um dos principais modos de implementar implantações independentes é *ocultando o banco dados*. Se um microsserviço tiver de armazenar estados em um banco de dados, isso deve estar totalmente oculto ao mundo externo. Bancos de dados internos não devem ser diretamente expostos aos consumidores externos, pois isso causará muito acoplamento entre ambos, o que prejudicará as implantações independentes. Em geral, evite situações em que vários microsserviços acessem o mesmo banco de dados.

Os microsserviços funcionam bem com o *DDD* (Domain-Driven Design, ou Design Orientado a Domínios). O DDD nos oferece conceitos que ajudam a identificar as fronteiras de nossos microsserviços, com a arquitetura resultante estando alinhada de acordo com o domínio de negócios. Isso será extremamente conveniente nas situações em que as empresas estejam criando equipes de TI mais centradas nos negócios. Com uma equipe mantendo o foco em uma única parte do domínio de negócios, ela poderá assumir a responsabilidade pelos microsserviços que correspondam a essa parte dos negócios.

Migrando para microsserviços

Os microsserviços trazem *muita* complexidade – complexidade suficiente a ponto de os motivos para usar os microsserviços deverem ser considerados com seriedade. Continuo convencido de que um sistema monolítico simples com um único processo é um ponto de partida totalmente sensato para um novo sistema. Com o tempo, porém, aprendemos mais e começamos a ver como nossa arquitetura de sistema atual não está mais adequada ao seu propósito. Nesse ponto, tentar fazer mudanças será

apropriado.

É importante entender *o que* você quer obter com uma arquitetura de microsserviços. Qual é o objetivo? Quais são os resultados positivos que você espera que uma mudança para microsserviços trará? O resultado almejado causará impactos diretos em *como* você vai decompor o seu sistema monolítico. Se estiver tentando mudar a arquitetura de seu sistema para lidar melhor com a escala, você acabará fazendo mudanças diferentes daquelas que faria caso seu fator determinante fosse melhorar a autonomia organizacional. Esse assunto foi mais bem discutido no Capítulo 3, e com mais detalhes ainda em meu livro *Migrando sistemas monolíticos para microsserviços* (Novatec, 2020).

Muitos dos problemas com os microsserviços se tornam evidentes somente depois que eles alcançam o ambiente de produção. Portanto, recomendo fortemente fazer uma decomposição gradual de um sistema monolítico existente, que evolua com o tempo, em vez de empreender uma reescrita “Big Bang”. Identifique um microsserviço que você queira criar, extraia a funcionalidade apropriada do sistema monolítico, implante o novo microsserviço no ambiente de produção e comece a usá-lo intensamente. Com base nisso, você verá se essa solução contribui para seguir em direção ao seu objetivo, ao mesmo tempo que aprenderá muito, facilitando a extração do próximo microsserviço – ou talvez ela sugira que os microsserviços, no fim das contas, não sejam o caminho certo para seguir adiante!

Estilos de comunicação

Resumimos as principais formas de comunicação entre microsserviços no Capítulo 4, e as apresentamos novamente na Figura A.1. Esse modelo não tem como propósito ser universal, mas a intenção é apenas apresentar uma visão geral dos diferentes tipos de comunicação mais comuns.

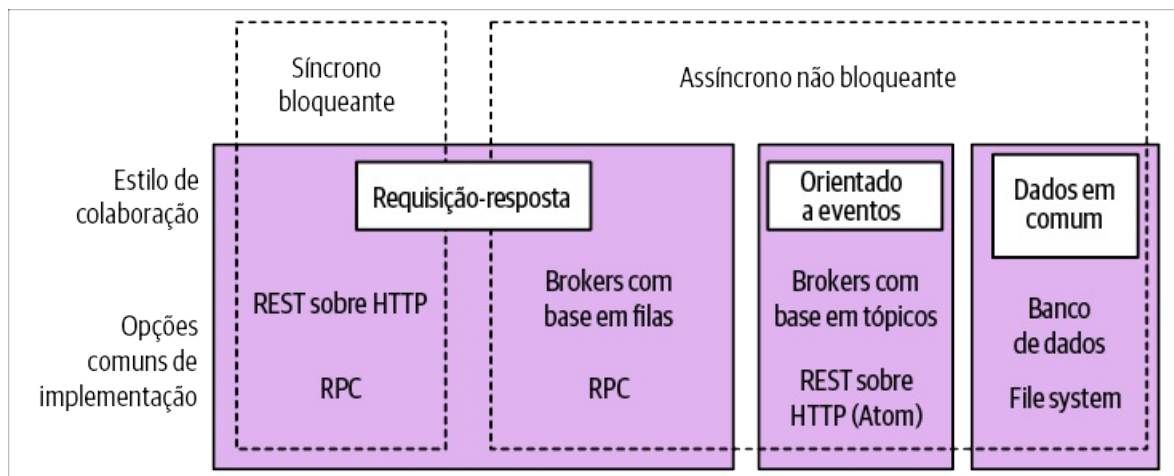


Figura A.1 – Diferentes estilos de comunicação entre microsserviços, com exemplos de tecnologias que implementam esses estilos.

Na comunicação por meio de *requisição-resposta*, um microsserviço envia uma requisição para um microsserviço downstream e espera uma resposta. Na comunicação síncrona com requisição-resposta, esperamos que a resposta seja devolvida para a instância do microsserviço que enviou a requisição. Na comunicação assíncrona com requisição-resposta, é possível que a resposta seja devolvida para uma instância diferente dos microsserviços upstream.

Na comunicação *orientada a eventos*, um microsserviço gera um evento, e outros microsserviços, caso estejam interessados nesse evento, podem reagir a ele. Os eventos são apenas declarações de fatos – informações que são compartilhadas a respeito de algo que aconteceu. Na comunicação orientada a eventos, um microsserviço não diz a outro microsserviço o que ele deve fazer, mas apenas compartilha os eventos. Cabe aos microsserviços downstream decidir o que fazer com essas informações. A comunicação orientada a eventos é, por definição, assíncrona por natureza.

Um microsserviço pode se comunicar usando mais de um protocolo. Por exemplo, na Figura A.2, vemos um microsserviço *Expedição* disponibilizando uma interface REST para uma interação por meio de requisição-resposta, mas que também gera eventos quando há mudanças.

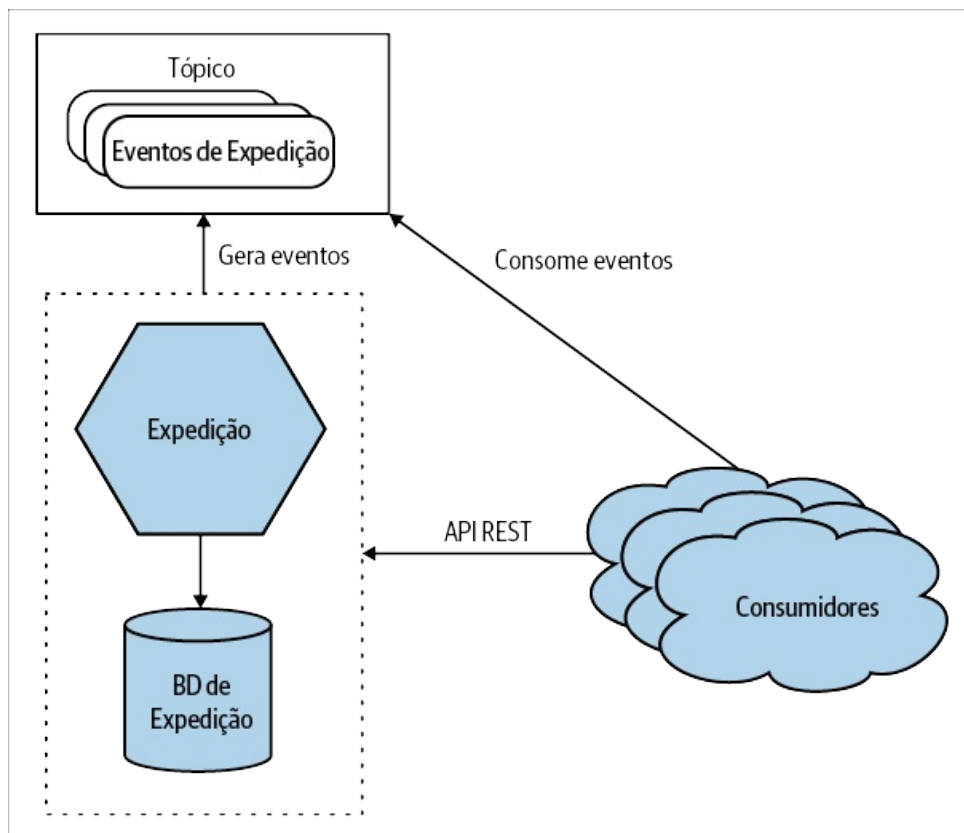


Figura A.2 – Um microserviço expõe sua funcionalidade por meio de uma API REST e um tópico.

A colaboração orientada a eventos pode facilitar a criação de arquiteturas com mais baixo acoplamento, mas pode exigir mais trabalho para entender como o sistema se comporta. Esse tipo de comunicação também exige geralmente o uso de uma tecnologia especializada, por exemplo, brokers de mensagens, o que pode complicar mais a situação. Se puder usar um broker de mensagens totalmente gerenciado, isso poderá ajudar a diminuir o custo desses tipos de sistemas.

Os modelos de interação por meio de requisição-resposta e orientado a eventos têm, cada um, o seu lugar; em geral, o modelo a ser usado dependerá de sua preferência pessoal. Alguns problemas simplesmente são mais apropriados a um modelo do que a outro, e é comum que uma arquitetura de microserviços tenha uma mistura dos estilos.

Fluxo de trabalho

Ao procurar fazer com que vários microsserviços colaborem para executar alguma operação mais abrangente, procure modelar explicitamente o processo usando *sagas* – um assunto que exploramos no Capítulo 6.

Em geral, as transações distribuídas devem ser evitadas em situações em que seja possível usar uma saga em seu lugar. As transações distribuídas agregam uma complexidade significativa aos sistemas, têm modos de falha complicados e, muitas vezes, não fazem o que você espera, mesmo quando funcionam. As sagas são, praticamente em todos os casos, mais apropriadas para a implementação de processos de negócios que envolvam vários microsserviços.

Há dois estilos diferentes de sagas a serem considerados: sagas *orquestradas* e sagas *coreografadas*. As sagas orquestradas utilizam um orquestrador centralizado para fazer a coordenação com outros microsserviços e garantir que a tarefa seja executada. Em geral, essa é uma abordagem simples e direta, porém o orquestrador central pode acabar fazendo tarefas demais se você não for cuidadoso, e poderá se transformar em uma causa de contenção se houver várias equipes trabalhando no mesmo processo de negócios. Nas sagas coreografadas, não há um coordenador central; em vez disso, a responsabilidade pelo processo de negócios será distribuída entre uma série de microsserviços em colaboração. Isso pode resultar em uma arquitetura mais complexa para implementar e exigirá mais trabalho para garantir que as tarefas corretas sejam executadas, mas, por outro lado, será muito menos suscetível a acoplamentos e funcionará bem se houver várias equipes.

Pessoalmente, gosto muito das sagas coreografadas, mas eu já as usei várias vezes e cometi muitos erros ao implementá-las. Meu conselho geral é que as sagas orquestradas funcionam bem quando uma única equipe é responsável por todo o processo, mas elas se tornam mais problemáticas se houver várias equipes. As sagas coreografadas podem justificar sua maior complexidade nas situações em que se espera que várias equipes colaborem com um processo.

Construção

Cada microsserviço deve ter seu próprio processo de construção (build) ou o seu próprio pipeline de CI. Quando faço uma alteração em um microsserviço, eu espero ser capaz de construí-lo de forma independente. Evite situações em que você tenha de construir todos os seus microsserviços juntos, pois isso fará com que seja muito mais difícil ter implantações independentes.

Por motivos que foram apresentados no Capítulo 7, não sou fã de monorepos. Se realmente quiser utilizá-los, por favor, entenda os problemas que eles causam em relação a ter linhas claras de responsabilidade e as possíveis complexidades nas construções de software. No entanto, certifique-se de que, independentemente de usar uma abordagem de monorepo ou de multirepo, cada microsserviço tenha seu próprio processo de construção de CI que possa ser disparado de modo independente de quaisquer outras construções.

Implantação

Os microsserviços geralmente são implantados como um processo. Esse processo pode ser implantado em uma máquina física, uma máquina virtual, um contêiner ou uma plataforma FaaS. O ideal é que os microsserviços estejam isolados uns dos outros o máximo possível no ambiente em que forem implantados. Não queremos ter uma situação na qual um microsserviço que consuma muitos recursos computacionais possa causar impactos em outro microsserviço. Em geral, isso significa que queremos ter cada microsserviço utilizando o seu próprio sistema operacional e seu conjunto de recursos computacionais isolados. Os contêineres são particularmente eficazes em proporcionar a cada instância de microsserviço o seu próprio conjunto isolado de recursos, fazendo com que sejam uma ótima opção para implantações de microsserviços.

O Kubernetes pode ser muito conveniente caso você pretenda executar cargas de trabalho em contêineres distribuídos em várias máquinas. Não é uma solução que eu recomendaria se houver poucos microsserviços, pois ela traz consigo suas próprias fontes de complexidade. Quando possível, utilize um cluster Kubernetes gerenciado, pois essa solução permite evitar

parte dessa complexidade.

O FaaS é um padrão interessante em ascensão para implantação de código. Em vez de ter de especificar quantas cópias de algo você quer, basta dar seu código à plataforma FaaS e dizer: “Quando isso acontecer, execute esse código”. Isso é *realmente* interessante do ponto de vista do desenvolvedor, e eu acho que uma abstração como essa provavelmente será o futuro para uma boa parte do desenvolvimento do lado do servidor. As implementações atuais, no entanto, ainda apresentam problemas. No que concerne aos microsserviços, implantar um microsserviço completo como uma única “função” em uma plataforma FaaS é um ponto de partida bastante apropriado.

Uma última observação: diferencie em sua mente os conceitos de implantação (deployment) e de lançamento (release). Somente porque você implantou algo no ambiente de produção não significa que isso tenha de ser lançado aos seus usuários. Ao separar esses conceitos, você criará oportunidades para fazer um rollout de seu software de várias maneiras – por exemplo, usando lançamentos canário (canary releases) ou execuções paralelas (parallel runs). Tudo isso e muito mais foi discutido em detalhes no Capítulo 8.

Testes

Faz bastante sentido ter uma suíte de testes funcionais automatizados para obter um feedback rápido sobre a qualidade de seu software antes que os usuários possam vê-lo e, certamente, isso é algo que você deve fazer. Os microsserviços oferecem muitas opções para os diferentes tipos de testes que podemos escrever, conforme exploramos no Capítulo 9.

Quando comparados com outros tipos de arquiteturas, porém, os testes fim a fim (end-to-end tests) podem ser particularmente problemáticos em arquiteturas de microsserviços. Eles podem acabar sendo mais custosos para escrever e manter em arquiteturas de microsserviços, em comparação com arquiteturas não distribuídas, mais simples; os próprios testes podem acabar apresentando muito mais falhas que não necessariamente apontarão para um problema em seu código. Os testes fim a fim que envolvam várias

equipes são particularmente desafiadores.

Com o tempo, procure reduzir sua dependência dos testes fim a fim; considere substituir parte do esforço investido nesses tipos de testes por contratos orientados a consumidores (consumer-driven contracts), verificação de compatibilidade de esquemas e testes em produção. Essas atividades podem ser muito mais eficazes que os testes fim a fim para identificar rapidamente os problemas em sistemas mais distribuídos.

Monitoração e observabilidade

No Capítulo 10, expliquei como a *monitoração* é uma atividade, isto é, algo que fazemos *em* um sistema, mas manter o foco na atividade em vez de se concentrar no resultado é problemático, e esse é um tema que esteve presente em todo o livro. Devemos manter o foco na *observabilidade* de nossos sistemas. A observabilidade é a extensão com que podemos entender o que o sistema está fazendo por meio da análise de saídas externas. Criar um sistema com boa observabilidade exige que essa ideia esteja incluída em nosso software, e devemos garantir que os tipos corretos de saídas externas estejam disponíveis.

Os sistemas distribuídos podem falhar de maneiras inusitadas, e os microsserviços não são diferentes. Não é possível prever todas as causas de falhas no sistema, portanto pode ser difícil saber com antecedência quais são as informações de que precisaremos para diagnosticar e corrigir os problemas. Usar ferramentas que possam ajudar a interrogar essas saídas externas de formas não previstas passa a ser cada vez mais importante. Sugiro que você dê uma olhada em ferramentas como o Lightstep e o Honeycomb, que foram criadas com essa ideia em mente.

Por fim, à medida que seu sistema escalar, será cada vez mais provável que sempre haverá um erro em algum lugar. No entanto, em um sistema de larga escala, uma máquina ter um problema não é necessariamente um motivo para que todos entrem imediatamente em ação, nem deveria resultar necessariamente em alguém ser rudemente despertado às três da manhã. O uso de técnicas de “testes em produção” como execuções paralelas e transações sintéticas pode ser muito mais eficaz para identificar

problemas que possam causar impacto nos usuários finais.

Segurança

Os microsserviços nos dão mais oportunidade para defender nossa aplicação em profundidade, o que, por sua vez, pode resultar em sistemas mais seguros. Por outro lado, em geral, os microsserviços apresentam uma maior área de superfície de ataque, podendo nos deixar mais expostos! Fazer essa ponderação é a razão pela qual é tão importante ter uma compreensão holística acerca da segurança – algo que discutimos no Capítulo 11.

Com mais informações fluindo pelas redes, será mais importante considerar a proteção dos dados em trânsito. O número maior de partes constituintes no sistema também significa que a automação será uma parte essencial na segurança dos microsserviços. Gerenciar patching, certificados e dados sigilosos (secrets) utilizando processos manuais, suscetíveis a erros, pode deixar seu sistema vulnerável a ataques. Portanto utilize ferramentas que permitam facilitar a automação.

Os JWTs podem ser usados para descentralizar a lógica de autorização de um modo que também evitará a necessidade de acessos adicionais de ida e volta. Isso pode ajudar a proteger seu sistema de problemas como o do confused deputy (agente enganado), ao mesmo tempo que garante que seu microsserviço possa executar de modo mais independente.

Por fim, um número cada vez maior de pessoas tem adotado uma mentalidade de *confiança zero* (zero trust). Na abordagem de confiança zero, operamos como se nosso sistema já estivesse comprometido, e precisamos construir nossos microsserviços de acordo com essa ideia. Pode parecer uma atitude paranoica, mas, cada vez mais, sou de opinião de que adotar esse princípio pode, na verdade, simplificar o modo de ver a segurança de nosso sistema.

Resiliência

No Capítulo 12, vimos a resiliência como um todo e apresentei os quatro

conceitos básicos que devem ser considerados quando pensamos na resiliência:

Robustez

A capacidade de absorver perturbações esperadas.

Recuperação

A capacidade de se recuperar após um evento traumático.

Extensibilidade com elegância

Quão bem lidamos com uma situação inesperada.

Adaptabilidade sustentável

A capacidade de se adaptar continuamente a ambientes, stakeholders e demandas que mudam.

Tomando como um todo, as arquiteturas de microsserviços podem ajudar em alguns desses aspectos (isto é, com a robustez e a recuperação), mas, conforme vemos com base nessa lista, elas, por si só, não fazem com que sejamos resilientes. Uma boa parte de ser resiliente diz respeito ao comportamento e à cultura das equipes e da empresa.

Basicamente, é necessário tomar atitudes explícitas que deixem sua aplicação mais robusta. A robustez não vem de graça – os microsserviços nos dão a opção de melhorar a resiliência de nossos sistemas, mas temos de fazer essa opção. Por exemplo, devemos entender que qualquer chamada que fizermos para outro microsserviço pode falhar, as máquinas podem falhar e coisas ruins podem acontecer com bons pacotes de rede. Padrões de estabilidade como bulkheads (anteparas), circuit breakers (interruptores de circuito) e timeouts devidamente configurados podem ajudar bastante.

Escalabilidade

Os microsserviços nos oferecem uma série de maneiras diferentes de escalar uma aplicação. No Capítulo 13, exploramos os quatro eixos da escalabilidade, apresentados a seguir:

Escalabilidade vertical

Em poucas palavras, significa adquirir uma máquina mais potente.

Duplicação horizontal

Ter vários itens capazes de fazer o mesmo trabalho.

Partição de dados

Dividir o trabalho com base em algum atributo dos dados, por exemplo, o grupo ao qual pertence o cliente.

Decomposição funcional

Separação do trabalho com base no tipo, por exemplo, uma decomposição em microsserviços.

Quando se trata de escalar um sistema, faça o que for mais fácil primeiro. A escalabilidade vertical e a duplicação horizontal são métodos rápidos e fáceis quando comparados com os outros dois eixos apresentados. Se funcionarem, ótimo! Caso contrário, você poderá tentar os outros métodos. Também é comum misturar diferentes formas de escalar – particionar seu tráfego com base nos clientes, por exemplo, e então fazer com que cada partição escale horizontalmente.

Interfaces de usuário

Com muita frequência, a interface de usuário é algo pensado *a posteriori* quando se trata da decomposição de um sistema – separamos nossos microsserviços, mas deixamos uma interface de usuário monolítica. Isso, por sua vez, leva aos problemas de ter equipes de frontend e de backend separadas. Em vez disso, queremos ter equipes alinhadas a fluxos (stream-aligned teams), com cada equipe responsável por todas as funcionalidades associadas a uma fatia fim a fim de uma funcionalidade voltada aos usuários. Para fazer essa mudança acontecer, e para que nos livremos de equipes de frontend e de backend atuando como silos, precisamos decompor nossas interfaces de usuário.

No Capítulo 14, mostrei como podemos usar *microfrontends* para ter interfaces de usuário decompostas utilizando frameworks de aplicações single-page como o React. As interfaces de usuário muitas vezes enfrentam problemas quanto ao número de chamadas que precisam fazer, ou porque precisam fazer agregação e filtragem de chamadas para atender aos

dispositivos móveis. O *padrão BFF* (Backend para Frontend) pode ajudar a disponibilizar agregação e filtragem do lado do servidor nessas situações, embora você possa deixar de lado o uso de BFFs caso consiga utilizar o GraphQL.

Estrutura organizacional

No Capítulo 15, vimos uma mudança na qual passamos de equipes alinhadas horizontalmente, atuando como silos, para uma estrutura de equipes mais organizada em torno de fatias contendo funcionalidades fim a fim. Essas equipes *alinhadas a fluxos* (stream-aligned teams), conforme os autores do livro *Team Topologies* as descrevem, têm o apoio das equipes de *capacitação* (enabling teams), conforme mostra a Figura A.3. As equipes de capacitação muitas vezes terão um foco mais abrangente, por exemplo, em segurança ou usabilidade, e dão apoio às equipes alinhadas a fluxos nesses aspectos.

Deixar essas equipes alinhadas a fluxos tão autônomas quanto possível significa que elas precisam ter ferramentas que possam usar por conta própria para evitar ter de pedir constantemente a outras equipes que façam o trabalho para elas. Como parte disso, uma plataforma pode ser extremamente conveniente. Contudo, é importante que vejamos uma plataforma como um tipo de *estrada pavimentada* – isto é, como algo que facilite fazer o que é certo, sem exigir que seu uso seja *obrigatório*. Deixar a plataforma opcional garante que fazer com que ela seja fácil de usar continue sendo o foco principal da equipe responsável por ela, ao mesmo tempo que permite que as equipes façam escolhas diferentes quando for justificado.

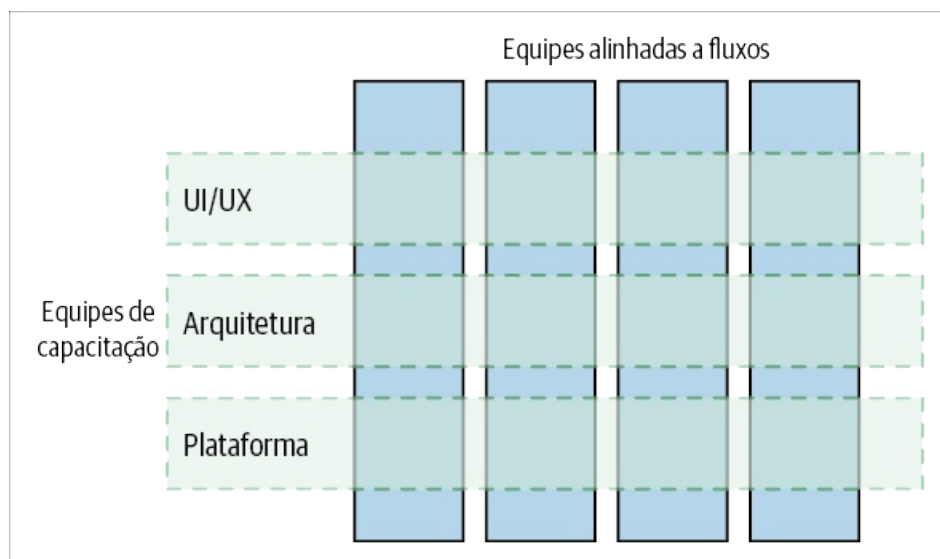


Figura A.3 – Equipes de capacitação oferecem suporte às várias equipes alinhadas a fluxos.

Arquitetura

É importante que não encaremos a arquitetura de nosso sistema como algo fixo e imutável. Em vez disso, devemos encará-la como algo que deva ser capaz de mudar continuamente à medida que as circunstâncias exigirem. Para que você tire o máximo de proveito das arquiteturas de microsserviços, passar para uma empresa na qual mais autonomia seja dada às equipes significa que a responsabilidade pela visão técnica deverá ser um processo mais colaborativo. O arquiteto que fica em sua torre de marfim será um obstáculo significativo para uma arquitetura de microsserviços, ou então se tornará uma irrelevância a ser ignorada.

A função de dar direção à arquitetura de um sistema pode estar totalmente distribuída entre as equipes e, em certo nível de escala, pode funcionar bem. No entanto, à medida que a empresa crescer, ter pessoas com tempo dedicado para observar o sistema como um todo será essencial. Você pode chamá-las de engenheiros líderes (principal engineers), donos de produto (product owners) técnicos ou arquitetos, não importa: o papel que elas devem desempenhar é o mesmo. Conforme mostrei no Capítulo 16, os arquitetos em uma empresa com microsserviços devem dar apoio às equipes, conectar pessoas, identificar padrões emergentes e passar tempo

suficiente com as equipes para ver como o quadro geral funciona realmente na prática.

Leituras complementares

Neste livro, fiz referência a vários artigos, apresentações e livros com os quais aprendi muito, e me certifiquei de listá-los na seção de Bibliografia. Desde a primeira edição, porém, vale a pena destacar os dois livros que causaram mais impacto em minhas ideias – e, como resultado, foram intensamente referenciados nesta nova edição – como “leituras obrigatórias”. O primeiro é *Accelerate*, de Nicole Forsgren, Jez Humble e Gene Kim. O segundo é *Team Topologies*, de Matthew Skelton e Manuel Pais. Esses dois, em minha opinião, são os livros mais úteis sobre desenvolvimento de software escritos nos últimos dez anos, e os considero leituras essenciais, esteja você trabalhando com microsserviços ou não.

Como complemento deste livro, meu próprio livro *Migrando sistemas monolíticos* explora com mais detalhes o modo de decompor arquiteturas de sistemas existentes.

Olhando para o futuro

No futuro, suspeito que a tecnologia que facilita a construção e a execução dos microsserviços continuará a ser aperfeiçoada, e estou particularmente ansioso para ver como será a segunda (e a terceira) geração dos produtos FaaS. Independentemente de o FaaS decolar, o Kubernetes se tornará mais popular, ainda que fique cada vez mais oculto em camadas de abstração mais adequadas aos desenvolvedores. O Kubernetes venceu, mas de um modo que acho que a maioria dos desenvolvedores de aplicações não deveria se preocupar. Continuo muito interessado em ver como o Wasm mudará o modo como pensamos nas implantações, e ainda suspeito que os unikernels poderão também ter o seu retorno.

Desde a primeira edição deste livro, os microsserviços realmente se consolidaram de um modo que me surpreendeu, mas que também me causa preocupação. Parece que muitas pessoas que estão adotando os

microserviços estão fazendo isso mais porque todos estão fazendo, em vez de ser pelo fato de os microserviços serem a opção correta para elas. Desse modo, acho que, sem sombra de dúvidas, ouviremos mais histórias de terror sobre implementações de microserviços que fracassaram, as quais irei digerir com prazer para ver o que podemos aprender com elas. Também acho que certamente haverá um contra-ataque mais amplo do mercado contra os microserviços em algum momento, quando os estudos de casos sobre desastres com microserviços alcançarem uma massa crítica. Aplicar um pensamento crítico para determinar qual é a abordagem que faça mais sentido em qualquer dada situação não é uma ideia muito atraente nem muito fácil de vender; não espero que isso vá mudar em um mundo no qual vender tecnologia é muito mais rentável do que vender ideias.

Não quero parecer pessimista! Como mercado, ainda somos muito jovens e ainda estamos tentando encontrar o nosso lugar no mundo. O volume de energia e a engenhosidade investidos no desenvolvimento de software continuam despertando o meu interesse, e mal posso esperar para ver o que a próxima década trará.

Palavras finais

As arquiteturas de microserviços nos dão mais opções, mas também mais decisões para tomar. Tomar decisões nesse mundo é uma atividade muito mais comum do que em sistemas monolíticos mais simples. Você não tomará todas essas decisões da forma correta – isso eu posso garantir a você. Então, sabendo que você cometerá alguns erros, quais são as suas opções? Bem, eu sugeriria encontrar maneiras de deixar que cada decisão tenha um escopo menor; dessa forma, se você cometer um erro, o impacto será apenas em uma pequena parte de seu sistema. Aprenda a adotar o conceito de arquitetura em evolução, na qual seu sistema possa ser dobrado, seja flexível e mude com o tempo à medida que você aprender mais. Não pense em reescritas Big Bang, mas em uma série de mudanças feitas em seu sistema com o tempo a fim de mantê-lo maleável.

A essa altura, espero ter compartilhado informações e experiências

suficientes para ajudar você a decidir se os microsserviços são uma opção correta para o seu caso. Se forem, espero que você pense neles como uma jornada, e não como um destino. Trabalhe gradualmente. Decomponha seu sistema por partes, aprendendo à medida que avançar. E acostume-se com isto: em muitos aspectos, ter disciplina para mudar continuamente e promover a evolução de nossos sistemas é uma lição muito mais importante do que qualquer outra que eu tenha compartilhado com você neste livro. Mudanças são inevitáveis. Aceite-as.

Bibliografia

- 2020 Data Breach Investigations Report (Relatório de investigações de violação de dados). Verizon, 2020. <https://oreil.ly/ps0Cx>.
- Abbott, Martin L. e Michael T. Fisher. *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise*. 2ª edição. Boston: Addison-Wesley, 2015.
- Allspaw, John. “Blameless Post-Mortems and a Just Culture” (Post-mortems sem acusações e uma cultura justa). *Code as Craft* (blog). Etsy, 22 de maio de 2012, <https://oreil.ly/P1BcX>.
- Bache, Emily. “End-to-End Automated Testing in a Microservices Architecture” (Testes fim a fim automatizados em uma arquitetura de microsserviços). NDC Conferences. 5 de julho de 2017. Vídeo no YouTube, 56:48, <https://oreil.ly/DbFdR>.
- Bell, Laura, Michael Brunton-Spall, Rich Smith e Jim Bird. *Agile Application Security*. Sebastopol: O’Reilly, 2017.
- Beyer, Betsy, Chris Jones, Jennifer Petoff e Niall Richard Murphy, eds. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O’Reilly, 2016¹.
- Beyer, Betsy, Niall Richard Murphy, David K. Rensin, Kent Kawahara e Stephen Thorne, eds. *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol: O’Reilly, 2018.
- Bird, Christian, Nachi Nagappan, Brendan Murphy, Harald Gall e Premkumar Devanbu. “Don’t Touch My Code! Examining the Effects of Ownership on Software Quality” (Não mexa no meu código! Analisando os efeitos da responsabilidade na qualidade do software). Em *ESEC/FSE ‘11: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, 4–14. Nova York: ACM, 2011. doi.org/10.1145/2025113.2025119.
- Brandolini, Alberto. *EventStorming*. Victoria, BC: Leanpub, em breve.
- Brooks, Frederick P., Jr. *The Mythical Man-Month: Essays on Software*

- Engineering*, Edição de aniversário. Boston: Addison-Wesley, 1995².
- Brown, Alanna, Nicole Forsgren, Jez Humble, Nigel Kersten e Gene Kim. *2016 State of DevOps Report* (Relatório sobre o estado de DevOps 2016). <https://oreil.ly/WJjhA>.
- Bryant, Daniel. “Apple Rebuilds Siri Backend Services Using Apache Mesos” (Apple recria serviços de backend do Siri usando o Apache Mesos). InfoQ, 3 de maio de 2015. <https://oreil.ly/NsjEQ>.
- Burns, Brendan, Brian Grant, David Oppenheimer, Eric Brewer e John Wilkes. “Borg, Omega, and Kubernetes”. *acmqueue* 14, n. 1 (2016). <https://oreil.ly/2TlYG>.
- Calçado, Phil. “Pattern: Using Pseudo-URIs with Microservices” (Padrão: usando pseudo-URIs com microsserviços). 22 de maio de 2017. <https://oreil.ly/uZuto>.
- Cockburn, Alistair. “Hexagonal Architecture” (Arquitetura hexagonal). alistair.cockburn.us, 4 de janeiro de 2005. <https://oreil.ly/0JeIm>.
- Cohn, Mike. *Succeeding with Agile*. Upper Saddle River, NJ: Addison-Wesley, 2009.
- Colyer, Adrian. “Information Distribution Aspects of Design Methodology” (Aspectos da distribuição de informações na metodologia de design). *The Morning Paper* (blog), 17 de outubro de 2016. <https://oreil.ly/qxj2m>.
- Crispin, Lisa e Janet Gregory. *Agile Testing: A Practical Guide for Testers and Agile Teams*. Upper Saddle River, NJ: Addison-Wesley, 2008.
- Evans, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley, 2004³.
- Ford, Neal, Rebecca Parsons e Patrick Kua. *Building Evolutionary Architectures*. Sebastopol: O’Reilly, 2017.
- Forsgren, Nicole, Dustin Smith, Jez Humble e Jessie Frazelle. *Accelerate: State of DevOps Report 2019* (Accelerate: Relatório sobre o estado de DevOps 2019). <https://oreil.ly/A3zGn>.
- Forsgren, Nicole, Jez Humble e Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations*. Portland, OR: IT Revolution, 2018.
- Fowler, Martin. “CodeOwnership” (Responsabilidade pelo código).

- martinfowler.com, 12 de maio de 2006. <https://oreil.ly/a42c7>.
- Fowler, Martin. “Eradicating Non-Determinism in Tests” (Erradicando a falta de determinismo nos testes). martinfowler.com, 14 de abril de 2011. <https://oreil.ly/sqPOD>.
- Fowler, Martin. “StranglerFigApplication” (Aplicação strangler fig). martinfowler.com, 29 de junho de 2004. <https://oreil.ly/foti0>.
- Freeman, Steve e Nat Pryce. *Growing Object-Oriented Software, Guided by Tests*. Upper Saddle River, NJ: Addison-Wesley, 2009.
- Friedrichsen, Uwe. “The Limits of the Saga Pattern” (Os limites do padrão saga). ufried.com (blog). 19 de fevereiro de 2021. <https://oreil.ly/X1BfK>.
- Garcia-Molina, Hector, Dieter Gawlick, Johannes Klein e Karl Kleissner. “Modeling Long-Running Activities as Nested Sagas” (Modelando atividades de longa duração como sagas aninhadas). *Data Engineering* 14, n. 1 (março de 1991): 14–18. <https://oreil.ly/RVp7A1>.
- Garcia-Molina, Hector e Kenneth Salem. “Sagas”. *ACM Sigmod Record* 16, n. 3 (1987): 249–59.
- Governor, James. “Towards Progressive Delivery” (Em direção à entrega progressiva). *James Governor’s MonkChips* (blog). RedMonk, 6 de agosto de 2018. <https://oreil.ly/OlkEY>.
- Heinemeier Hansson, David. “The Majestic Monolith” (O majestoso sistema monolítico). *Signal v. Noise*, 29 de fevereiro de 2016, <https://oreil.ly/fN5CR>.
- Hodgson, Pete. “Feature Toggles (aka Feature Flags)” (Toggles de funcionalidade [também conhecidos como flags de funcionalidade]). martinfowler.com, 9 de outubro de 2017, <https://oreil.ly/pSPrd>.
- Hohpe, Gregor. *The Software Architect Elevator: Redefining the Architect’s Role in the Digital Enterprise*. Sebastopol: O’Reilly, 2020.
- Hohpe, Gregor e Bobby Woolf, *Enterprise Integration Patterns*. Boston: Addison-Wesley, 2003.
- Humble, Jez e David Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison Wesley, 2010.⁴
- Hunt, Troy. “Passwords Evolved: Authentication Guidance for the Modern Era” (As senhas evoluíram: orientações para autenticação na era

moderna). troyhunt.com, 26 de julho de 2017. <https://oreil.ly/r4ava>.

Ingles, Paul. “Convergence to Kubernetes” (Convergência para o Kubernetes). Medium, 18 de junho de 2018, <https://oreil.ly/oB2FI>.

Ishmael, Johnathan. “Optimising Serverless for BBC Online” (Otimização de serverless para BBC online). *Technology and Creativity at the BBC* (blog), BBC, 26 de janeiro de 2021, <https://oreil.ly/mPp2L>.

Jackson, Cam. “Micro Frontends”. martinowler.com, 19 de junho de 2019. <https://oreil.ly/nYu15>.

Kingsbury, Kyle. “Jepsen: Elasticsearch”. Aphyr, 15 de junho de 2014. <https://oreil.ly/6l2sR>.

Kingsbury, Kyle. “Jepsen: Elasticsearch 1.5.0”. Aphyr, 27 de abril de 2015. <https://oreil.ly/jlu8p>.

Kleppmann, Martin. *Designing Data-Intensive Applications*. Sebastopol: O’Reilly, 2017.

Krishnan, Kripa. “Weathering the Unexpected” (Enfrentando o inesperado). *acmqueue* 10, n. 9 (2012). <https://oreil.ly/BN2Ek>.

Kubis, Robert. “Google Cloud Spanner: Global Consistency at Scale by Robert Kubis” (Google Cloud Spanner: consistência global em escala, por Robert Kubis). Devvxx. 7 de novembro de 2017. Vídeo no YouTube, 33:22. <https://oreil.ly/fwbMD>.

Lamport, Leslie. “Time, Clocks, and the Ordering of Events in a Distributed System” (Tempo, relógios e a ordem dos eventos em um sistema distribuído). *Communications of the ACM*. 21, n. 7 (julho de 1978): 558–65. <https://oreil.ly/Y07gU>.

Lewis, James. “Scale, Microservices and Flow” (Escala, microserviços e fluxo). YOW! Conferences. 10 de fevereiro de 2020. Vídeo no YouTube, 51:03. <https://oreil.ly/nzXqX>.

Losio, Renato. “Elastic Changes Licences for Elasticsearch and Kibana: AWS Forks Both” (Elastic muda licença do Elasticsearch e do Kibana: AWS cria forks para ambos). InfoQ, 25 de janeiro de 2021. <https://oreil.ly/PClFv>.

MacCormack, Alan, Carliss Y. Baldwin e John Rusnak. “Exploring the Duality Between Product and Organizational Architectures: A Test of the *Mirroring Hypothesis*” (Explorando a dualidade entre a arquitetura do

- produto e a arquitetura organizacional: um teste da hipótese de *espelhamento*). *Research Policy* 41, n. 8 (outubro de 2012): 1309–24.
- Majors, Charity. “Metrics: Not the Observability Droids You’re Looking For” (Métricas: não são os androides de observabilidade que você está procurando). *Honeycomb* (blog), 24 de outubro de 2017. <https://oreil.ly/RpZaZ>.
- Majors, Charity, Liz Fong-Jones e George Miranda. *Observability Engineering*. Sebastopol: O’Reilly, 2022.
- McAllister, Neil. “Code Spaces Goes Titsup FOREVER After Attacker NUKES Its Amazon-Hosted Data” (Code Spaces fora de combate PARA SEMPRE depois de invasor ter DESTRUÍDO seus dados hospedados na Amazon). *The Register*, 18 de junho de 2014. <https://oreil.ly/IUOD0>.
- Miles, Russ. *Learning Chaos Engineering*. Sebastopol: O’Reilly, 2019.
- Moore, Jon. “Architecture with 800 of My Closest Friends: The Evolution of Comcast’s Architecture Guild” (Arquitetura com 800 dos meus melhores amigos: a evolução da guilda de arquitetura na Comcast). *InfoQ*, 14 de maio de 2019. <https://oreil.ly/dVfhi>.
- Morris, Kief. *Infrastructure as Code*. 2ª edição. Sebastopol: O’Reilly, 2016.
- Nagappan, Nachiappan, Brendan Murphy e Victor Basili. “The Influence of Organizational Structure on Software Quality: An Empirical Case Study” (A influência da estrutura organizacional na qualidade do software: um caso de estudo empírico). *ICSE ‘08: Proceedings of the 30th International Conference on Software Engineering*. Nova York: ACM, 2008.
- Newman, Sam. *Monolith to Microservices*. Sebastopol: O’Reilly, 2019.⁵
- Noursalehi, Saeed. “Git Virtual File System Design History” (História do projeto Git Virtual File System). <https://t.co/mIQR4uzWKS?amp=1>.
- Nygard, Michael. *Release It!* 2ª edição. Raleigh: Pragmatic Bookshelf, 2018.
- Oberlehner, Markus. “Monorepos in the Wild” (Monorepos por aí). *Medium*, 12 de junho de 2017. <https://oreil.ly/Sk6am>.
- Padmanabhan, Senthil e Pranav Jha. “WebAssembly at eBay: A Real-World Use Case” (WebAssembly na eBay: um caso de uso real). *eBay*, 22 de maio de 2019. <https://oreil.ly/rlr7d>.
- Page-Jones, Meilir. *Practical Guide to Structured Systems Design*, 2ª edição.

- Nova York: Yourdon Press, 1980.⁶
- Palino, Todd, Neha Narkhede e Gwen Shapira. *Kafka: The Definitive Guide*. Sebastopol: O'Reilly, 2017.
- Parnas, David. "Information Distribution Aspects of Design Methodology" (Aspectos da distribuição de informações na metodologia de design). Em *Information Processing: Proceedings of the IFIP Congress*, 339–44. Vol. 1. Amsterdã: Holanda do Norte, 1972.
- Parnas, David. "On the Criteria to Be Used in Decomposing Systems into Modules" (Sobre os critérios a serem usados para decompor sistemas em módulos). Contribuição para uma revista acadêmica, Carnegie Mellon University, 1971. <https://oreil.ly/nWtQA>.
- Plotnicki, Lukasz. "BFF @ Soundcloud". ThoughtWorks, 9 de dezembro de 2015. <https://oreil.ly/ZyR0l>.
- Potvin, Rachel e Josh Levenberg. "Why Google Stores Billions of Lines of Code in a Single Repository" (Por que a Google armazena bilhões de linhas de código em um único repositório). *Communications of the ACM* 59, n. 7 (julho de 2016): 78–87. <https://oreil.ly/Eupyi>.
- Pyhäjärvi, Maaret. *Ensemble Programming Guidebook*. Publicação independente, 2015–2020. <https://ensembleprogramming.xyz>.
- Riggins, Jennifer. "The Rise of Progressive Delivery for Systems Resilience" (A ascensão da entrega progressiva para resiliência de sistemas). The New Stack, 1 de abril de 2019. <https://oreil.ly/merIs>.
- Rodriguez, Daniel, M. Ángel Sicilia, Elena García Barriocanal e Rachel Harrison. "Empirical Findings on Team Size and Productivity in Software Development" (Descobertas empíricas sobre tamanho de equipe e produtividade no desenvolvimento de software). *Journal of Systems and Software* 85, n. 3 (2012). doi.org/10.1016/j.jss.2011.09.009.
- Rossman, John. *Think Like Amazon: 50 1/2 Ideas to Become a Digital Leader*. Nova York: McGraw-Hill, 2019.
- Ruecker, Bernd. *Practical Process Automation*. Sebastopol: O'Reilly, 2021.
- Sadalage, Pramod e Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley, 2012.⁷
- Schneider, Jonny. *Understanding Design Thinking, Lean, and Agile*.

- Sebastopol: O'Reilly, 2017.
- Shankland, Stephen. "Google Uncloaks Once-Secret Server" (Google revela servidor anteriormente secreto). CNET, 11 de dezembro de 2009. <https://oreil.ly/hHKvE>.
- Shorrock, Steven. "Alarm Design: From Nuclear Power to WebOps" (Design de alarmes: da energia nuclear ao WebOps). *Humanistic Systems* (blog), 16 de outubro de 2015. <https://oreil.ly/AiJ5i>.
- Shostack, Adam. *Threat Modeling: Designing for Security*. Indianapolis: Wiley, 2014.
- Sigelman, Ben. "Three Pillars with Zero Answers – Towards a New Scorecard for Observability" (Três pilares sem nenhuma resposta – em direção a uma nova tabela de desempenho para observabilidade). Lightstep (postagem de blog), 5 de dezembro de 2018. <https://oreil.ly/qdtSS>.
- Skelton, Matthew e Manuel Pais. *Team Topologies*. Portland, OR: IT Revolution, 2019.
- Steen, Maarten van e Andrew Tanenbaum. *Distributed Systems*. 3ª edição. Scotts Valley, CA: CreateSpace Independent Publishing Platform, 2017.
- Stopford, Ben. *Designing Event-Driven Systems*. Sebastopol: O'Reilly, 2017.
- Valentino, Jason D. "Moving One of Capital One's Largest Customer-Facing Apps to AWS" (Passando uma das maiores aplicações voltada aos clientes da Capital One para a AWS). Medium/Capital One Tech, 24 de maio de 2017. <https://oreil.ly/IEIC3>.
- Vaughan, Diane. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. Chicago: University of Chicago Press, 1996.
- Vernon, Vaughn. *Domain-Driven Design Distilled*. Boston: Addison-Wesley, 2016.
- Vernon, Vaughn. *Implementing Domain-Driven Design*. Upper Saddle River, NJ: Addison-Wesley, 2013.
- Vocke, Ham. "The Practical Test Pyramid" (Pirâmide de testes na prática). martinowler.com, 26 de fevereiro de 2018. <https://oreil.ly/6rRoU>.
- Webber, Emily. *Building Successful Communities of Practice*. San Francisco:

Blurb, 2016.

Webber, Jim, Savas Parastatidis e Ian Robinson. *REST in Practice: Hypermedia and Systems Architecture*. Sebastopol: O'Reilly, 2010.

Woods, David D. “Four Concepts for Resilience and the Implications for the Future of Resilience Engineering” (Quatro conceitos sobre resiliência e as implicações para o futuro da engenharia de resiliência). *Reliability Engineering & System Safety* 141 (setembro de 2015): 5–9. doi.org/10.1016/j.ress.2015.03.018.

Yourdon, Edward e Larry L. Constantine. *Structured Design*. Nova York: Yourdon Press, 1976.⁸

Zimman, Adam. “Progressive Delivery, a History...Condensed” (Entrega progressiva, uma história...resumida). *Industry Insights* (blog). LaunchDarkly, 6 de agosto de 2018. <https://oreil.ly/4pVY7>.

- ¹ N.T.: Edição publicada no Brasil: *Engenharia de Confiabilidade do Google: como o Google administra seus sistemas de produção* (Novatec, 2016).
- ² N.T.: Edição publicada no Brasil: *O Mítico Homem-Mês: ensaios sobre engenharia de software* (Alta Books, 2018).
- ³ N.T.: Edição publicada no Brasil: *Domain-Driven Design: atacando as complexidades no coração do software* (Alta Books, 2016).
- ⁴ N.T.: Edição publicada no Brasil: *Entrega Contínua: como entregar software de forma rápida e confiável* (Bookman, 2014).
- ⁵ N.T.: Edição publicada no Brasil: *Migrando sistemas monolíticos para microserviços* (Novatec, 2020).
- ⁶ N.T.: Edição publicada no Brasil: *Projeto Estruturado de Sistemas* (McGraw-Hill, 1980).
- ⁷ N.T.: Edição publicada no Brasil: *NoSQL Essencial: um guia conciso para o mundo emergente da persistência poliglota* (Novatec, 2013).
- ⁸ N.T.: Edição publicada no Brasil: *Projeto Estruturado de Sistemas* (Editora Campus, 1992).

Glossário

acoplamento

A extensão com que as mudanças em uma parte do sistema exigem uma mudança em outra. Um baixo acoplamento geralmente é desejável.

acoplamento de domínio

Uma forma de acoplamento na qual um microserviço é “acoplado” ao protocolo de domínio exposto por outro microserviço.

acordo de nível de serviço (SLA)

Um acordo entre um usuário final e um provedor de serviços (por exemplo, cliente e fornecedor) que define a oferta de serviço minimamente aceitável e as penalidades que se aplicam caso o acordo não seja obedecido.

adaptabilidade sustentável

A capacidade de se adaptar continuamente a ambientes, stakeholders e demandas que mudam.

agregado

Um conjunto de objetos gerenciado como uma única entidade, geralmente se referindo a conceitos do mundo real. Um conceito do DDD.

Amazon Web Services (AWS)

A nuvem pública oferecida como produto pela Amazon.

aplicação single-page (SPA)

Um tipo de interface gráfica de usuário no qual a UI é disponibilizada em um único painel no navegador, sem a necessidade de navegação para outras páginas web.

arquitetura orientada a serviços (SOA)

Um tipo de arquitetura na qual o sistema é separado em serviços que podem executar em máquinas distintas. Os microserviços são um *tipo* de

SOA que priorizam a capacidade de ter implantações independentes.

autenticação

O processo pelo qual um *principal* prova que é quem alega ser. Poderia ser algo simples como uma pessoa fornecendo seu nome de usuário e a senha.

autorização

O processo que determina se um *principal* autorizado tem permissão para acessar uma dada funcionalidade.

Azure

A nuvem pública oferecida como produto pela Microsoft.

backend para frontend (BFF)

Um componente do lado do servidor que oferece agregação e filtragem para uma interface de usuário específica. É uma alternativa a um gateway de API de propósito geral.

biblioteca

Um conjunto de códigos empacotados de modo que possa ser reutilizado em vários programas.

branching por funcionalidade

Criar outro branch (ramificação) para cada funcionalidade na qual se trabalha, fazendo o merge desse branch de volta à linha principal (mainline) assim que a funcionalidade estiver concluída. É algo que eu não incentivo.

broker de mensagem

Software dedicado que gerencia a comunicação assíncrona entre processos, geralmente oferecendo recursos como garantia de entrega (para algumas definições da palavra *garantia*).

bulkhead (antepara)

Uma parte do sistema na qual uma falha pode ser isolada, de modo que a parte restante do sistema possa continuar funcionando, mesmo que uma falha ocorra.

circuit breaker (interruptor de circuito)

Um mecanismo que encapsula uma conexão com um serviço downstream e possibilita falhar rapidamente caso o serviço downstream esteja com problemas.

coesão

A extensão com a qual um código que muda junto permanece junto.

contêiner

Um pacote de código e as dependências que podem ser executadas de forma isolada em uma máquina. É conceitualmente semelhante às máquinas virtuais, embora os contêineres sejam muito mais leves.

contexto delimitado

Uma fronteira explícita em um domínio de negócios que provê funcionalidades para o sistema mais amplo, mas que também oculta suas complexidades. Em geral, mapeia-se com as fronteiras organizacionais. Um conceito do DDD.

controle para detecção

Um controle de segurança que ajudará a identificar se um ataque está em andamento/ocorreu.

controle para prevenção

Um controle de segurança cujo objetivo é impedir que um ataque aconteça.

controle para resposta

Um controle de segurança que ajuda a responder durante/após um ataque.

coreografia

Um estilo de saga no qual a responsabilidade por aquilo que deve ocorrer é distribuída entre vários microsserviços, em vez de ser gerenciada por uma única entidade.

desenvolvimento baseado em tronco

Um estilo de desenvolvimento no qual todas as alterações são feitas diretamente no tronco principal do sistema de controle de versões, incluindo as alterações que ainda não foram concluídas.

design orientado a domínio (DDD)

Um conceito segundo o qual o problema fundamental/domínio de negócios é explicitamente modelado no software.

Docker

Um conjunto de ferramentas para ajudar a criar e gerenciar contêineres.

duplicação horizontal

Escalar um sistema tendo várias cópias de algo.

entrega contínua (CD)

Uma abordagem de entrega na qual modelamos explicitamente o caminho até o ambiente de produção, tratamos todo check-in como um candidato a um lançamento e podemos facilmente avaliar a adequação de qualquer candidato a um lançamento para ser implantado em produção.

equipe de capacitação (enabling team)

Uma equipe que auxilia as equipes alinhadas a fluxos (stream-aligned teams) a fazerem seu trabalho. Em geral, uma equipe de capacitação tem um foco específico – por exemplo, usabilidade, arquitetura, segurança.

equipes alinhadas a fluxos (stream-aligned team)

Uma equipe com foco na entrega fim a fim de um fluxo de trabalho importante. É uma equipe de longa duração, que geralmente terá foco direto no cliente, abrangendo dados, código de backend e código de frontend.

escalabilidade vertical

Aumentar a escala do sistema adquirindo uma máquina mais potente.

evento

Algo que acontece no sistema e que pode ser do interesse de outras partes do sistema – por exemplo, “Pedido Feito” ou “Usuário Logado”.

extensibilidade com elegância

Quão bem lidamos com uma situação inesperada.

Função como Serviço (FaaS)

Um tipo de plataforma serverless (sem servidor) que chama códigos arbitrários com base em determinados tipos de triggers – por exemplo,

iniciar um código em resposta a uma chamada HTTP ou à recepção de uma mensagem.

gateway de API

Um componente que geralmente é colocado no perímetro de um sistema e encaminha chamadas de origens externas (por exemplo, de interfaces de usuário) para os microsserviços, entre outras tarefas.

governança

Chegar a um acordo sobre como as tarefas devem ser feitas e garantir que elas sejam feitas dessa maneira.

GraphQL

Um protocolo que permite que o cliente gere queries personalizadas que podem resultar em chamadas feitas a vários microsserviços downstream. Conveniente para auxiliar na agregação e filtragem de chamadas para clientes externos, sem exigir o uso de BFF ou de gateways de API.

idempotência

A propriedade de uma função segundo a qual, mesmo que ela seja chamada diversas vezes, o resultado será o mesmo. Conveniente para permitir que novas tentativas de operações nos microsserviços sejam feitas com segurança.

implantação contínua

Uma abordagem na qual qualquer build que passe por todos os passos automatizados é automaticamente implantado no ambiente de produção.

implantação sincronizada

A necessidade de implantar dois ou mais componentes ao mesmo tempo porque uma alteração assim o exige. É o oposto de implantações independentes. Em geral, evite.

implantações independentes

A capacidade de fazer uma alteração em um microsserviço e implantá-lo no ambiente de produção sem precisar modificar nem implantar mais nada.

indicador de nível de serviço (SLI)

Uma medida de como o seu sistema está se comportando – por exemplo, um tempo de resposta.

informações de identificação pessoal (PII)

Dados que, quando usados isoladamente ou em conjunto com outras informações, poderiam servir para identificar um indivíduo.

infraestrutura como código

Modelar a infraestrutura na forma de código, permitindo que seu gerenciamento seja automatizado e o código esteja sujeito ao sistema de controle de versões.

integração contínua (CI)

A integração regular (diária) de mudanças com a parte restante da base de código, junto com uma suíte de testes para validar se a integração funcionou.

JSON Web Token

Um padrão para criar uma estrutura de dados JSON que poderá ser opcionalmente criptografada. Em geral, é usado para transmitir informações sobre *principals* autenticados.

Kubernetes

Uma plataforma open source que gerencia cargas de trabalho em contêineres com várias máquinas subjacentes.

lei de Conway

A observação de que as estruturas de comunicação das empresas acabam determinando o design dos sistemas computacionais que essas empresas criam.

linguagem ubíqua

Definir e adotar uma linguagem comum, a ser usada no código e na descrição do domínio, para facilitar a comunicação. Um conceito do DDD.

máquina virtual (VM)

Uma emulação de uma máquina na qual, para todos os fins e propósitos, ela parece ser uma máquina física dedicada.

mensagem

Algo enviado a um ou mais microsserviços downstream por meio de um método de comunicação assíncrono, por exemplo, um broker. Pode conter diversos payloads, por exemplo, uma requisição, uma resposta ou um evento.

microsserviço

Um serviço que pode ser implantado de forma independente e que se comunica com outros microsserviços por meio de um ou mais protocolos de comunicação.

modelo de ameaças

O processo pelo qual compreendemos as ameaças que o sistema pode vir a enfrentar e priorizamos aquelas que devem ser resolvidas.

monorepo

Um único repositório que contém todo o código-fonte de todos os seus microsserviços.

multirepo

Uma abordagem segundo a qual cada microsserviço tem o próprio repositório de códigos-fontes.

objetivo de nível de serviço (SLO)

Um acordo para definir qual é a faixa aceitável de um dado SLI.

ocultação de informações

Uma abordagem segundo a qual, por padrão, todas as informações são ocultas dentro de uma fronteira e somente o mínimo é exposto para satisfazer aos consumidores externos.

orçamento para erros (erros budgets)

Relaciona-se ao nível aceitável no qual um SLO pode estar fora dos limites, em geral definido com base em um grau aceitável de downtime para um serviço.

orquestração

Um estilo de saga no qual uma unidade central (também conhecida como orquestrador) gerencia a operação de outros microsserviços para executar

um processo de negócios.

partição de dados

Escalar um sistema distribuindo a carga com base em algum aspecto dos dados. Por exemplo, separar a carga de acordo com o cliente ou o tipo de produto.

principal

Algo – geralmente uma pessoa, embora possa ser também um programa – que está pedindo para ser autenticado e autorizado para ter um acesso.

requisição

Enviada de um microsserviço para outro, pedindo que o microsserviço downstream faça algo.

requisito multifuncional (CFR)

Uma propriedade geral do sistema, por exemplo, a latência exigida para as operações, a segurança dos dados em repouso etc. Também conhecido como requisito não funcional (mas eu prefiro muito mais o termo multifuncional como descrição).

resposta

Transmitida de volta como resultado de uma requisição.

responsabilidade coletiva

Um estilo de responsabilidade segundo o qual qualquer desenvolvedor pode alterar qualquer parte do sistema.

responsabilidade forte

Um estilo de responsabilidade no qual partes do sistema ficam sob responsabilidade de equipes específicas, e mudanças em uma determinada parte do sistema poderão ser feitas somente pela equipe responsável por ela.

robustez

A capacidade de um sistema continuar funcionando mesmo quando algo ruim acontecer.

saga

Uma forma de modelar operações de longa duração de modo que os

recursos não precisem ficar travados por muito tempo. É preferível em relação a ter transações distribuídas ao implementar processos de negócios.

serverless (sem servidor)

Um termo abrangente para produtos de nuvem que, do ponto de vista do usuário, abstraem os computadores subjacentes, a ponto de o usuário não precisar mais se preocupar com eles. Exemplos desses produtos incluem AWS Lambda, AWS S3 e Azure Cosmos.

service mesh

Um tipo distribuído de middleware que oferece funcionalidades gerais, principalmente para chamadas síncronas ponto-a-ponto – por exemplo, TLS mútuo, descoberta de serviços ou circuit breakers (interruptores de circuito).

software personalizável de prateleira (COTS)

Software de terceiros extremamente personalizável pelo usuário final, além de ser geralmente executado em sua própria infraestrutura. Exemplos típicos incluem sistemas de gerenciamento de conteúdo e plataformas de gerenciamento de relacionamento com clientes.

widget

Um componente de uma interface gráfica de usuário.

Sobre o autor

Sam Newman é consultor independente, autor e palestrante. Em mais de vinte anos no mercado, trabalhou com diferentes conjuntos de tecnologias e diferentes domínios, com empresas no mundo todo. Seu foco principal está em ajudar as empresas a levar o software para o ambiente de produção com mais rapidez e segurança, ajudando-as a navegarem pelas complexidades dos microsserviços. É também o autor de *Migrando sistemas monolíticos para microsserviços* (Novatec).

Colofão

Os animais na capa de *Criando microsserviços*, segunda edição, são abelhas melíferas (do gênero *Apis*). Das mais de vinte mil espécies de abelhas, há apenas oito espécies que produzem mel. Essas abelhas que vivem em sociedade são excepcionais no modo como produzem e armazenam coletivamente o mel e constroem suas colmeias a partir da cera. Criar abelhas para obtenção do mel tem sido um desejo da humanidade há milhares de anos no mundo todo.

As colmeias contêm milhares de indivíduos e apresentam uma estrutura social extremamente organizada, composta de uma rainha, zangões e operárias. Cada colmeia tem uma única rainha; ela permanece fértil por três a cinco anos após o seu voo de acasalamento e põe até dois mil ovos por dia. Os zangões são machos que se acasalam com a rainha (e morrem no ato, por causa de seus órgãos sexuais farpados). As abelhas operárias são fêmeas estéreis que exercerão muitas funções durante a vida, por exemplo, cuidam das larvas, do trabalho de construção, da armazenagem de alimentos, atuam como guardas, cuidam dos mortos e saem em busca de alimento. Abelhas operárias carregadas de pólen retornam à colmeia e “dançam” de acordo com um conjunto de padrões para passar informações a respeito de alimentos nas proximidades.

Embora as rainhas sejam um pouco maiores, as abelhas melíferas têm uma aparência semelhante, com asas transparentes, seis patas e um corpo segmentado em cabeça, tórax e abdômen. Têm pequenos pelos felpudos em um padrão listrado de amarelo e preto. A dieta das abelhas adultas é composta exclusivamente de mel, que é produzido por um processo de digestão parcial e regurgitação do néctar das flores, rico em açúcares.

As abelhas são essenciais para a agricultura; à medida que coletam seu alimento, elas polinizam as plantações. Colmeias comerciais são transportadas pelos apicultores para os lugares em que é preciso polinizar as plantações. Na média, cada colmeia coleta cerca de trinta quilos de

pólen em um ano. Nos últimos anos, porém, transtornos que causam o colapso das colônias, provocados por diversas doenças e outros fatores de estresse, têm provocado um declínio alarmante nas espécies de abelhas melíferas.

As abelhas melíferas são vulneráveis aos mesmos pesticidas e aos parasitas e doenças introduzidos, que têm causado uma diminuição no número de abelhas silvestres e outros polinizadores; no entanto, as abelhas que produzem mel têm certo apoio e proteção dos seres humanos por causa de seu papel essencial na agricultura. Muitos dos animais das capas dos livros da O'Reilly estão ameaçados; todos são importantes para o mundo.

A imagem da capa é uma ilustração em cores feita por Karen Montgomery com base em uma gravura em preto e branco do livro *The Pictorial Museum of Animated Nature*.